OpenLaoKe Project

Complete Source Code Documentation

Generated from OpenLaoKe Project

Table of Contents

1. examples/memory_demo.py
2. extract_chrome_cookies.py
3. extract_firefox_cookie.py
4. extract_glm_cookie.py
5. generate_pdfs.py
6. manual_cookie_extractor.py
7. openlaoke/__init__.py
8. openlaoke/__main__.py
9. openlaoke/commands/__init__.py
10. openlaoke/commands/auth_command.py
11. openlaoke/commands/base.py
12. openlaoke/commands/hyperauto_command.py
13. openlaoke/commands/registry.py
14. openlaoke/commands/scaffold_commands.py
15. openlaoke/commands/skill_commands.py
16. openlaoke/commands/skill_shortcuts.py
17. openlaoke/components/__init__.py
18. openlaoke/components/tui.py
19. openlaoke/core/__init__.py
20. openlaoke/core/agent_runner.py
21. openlaoke/core/api.py
22. openlaoke/core/architecture/__init__.py
23. openlaoke/core/architecture/assembler.py
24. openlaoke/core/architecture/decomposer.py
25. openlaoke/core/architecture/interfaces.py
26. openlaoke/core/architecture/orchestrator.py
27. openlaoke/core/atomic_generator.py
28. openlaoke/core/atomic_generator_api.py
29. openlaoke/core/autocomplete.py
30. openlaoke/core/code_templates.py
31. openlaoke/core/code_validator.py
32. openlaoke/core/compact/__init__.py
33. openlaoke/core/compact/compactor.py
34. openlaoke/core/compact/strategies.py
35. openlaoke/core/compact/summarizer.py
36. openlaoke/core/compact/token_budget.py
37. openlaoke/core/completion.py
38. openlaoke/core/config_wizard.py
39. openlaoke/core/config_wizard_extended_web.py
40. openlaoke/core/doc_downloader.py
41. openlaoke/core/doc_sources.py
42. openlaoke/core/dual_model_agent.py
43. openlaoke/core/dual_model_config.py
44. openlaoke/core/enhanced_knowledge_base.py
45. openlaoke/core/explorer/__init__.py
46. openlaoke/core/explorer/architecture.py
47. openlaoke/core/explorer/code_understanding.py
48. openlaoke/core/explorer/discovery.py
49. openlaoke/core/explorer/exploration_strategy.py

50. openlaoke/core/explorer/explorer.py
51. openlaoke/core/explorer/hypothesis.py
52. openlaoke/core/explorer/reasoning.py
53. openlaoke/core/extended_web/__init__.py
54. openlaoke/core/extended_web/api_client.py
55. openlaoke/core/extended_web/auto_browser.py
56. openlaoke/core/extended_web/auto_cookie_extractor.py
57. openlaoke/core/extended_web/browser_auth.py
58. openlaoke/core/extended_web/clients/__init__.py
59. openlaoke/core/extended_web/clients/base.py
60. openlaoke/core/extended_web/clients/deepseek_client.py
61. openlaoke/core/extended_web/clients/glm_client.py
62. openlaoke/core/extended_web/clients/kimi_client.py
63. openlaoke/core/extended_web/clients/qwen_client.py
64. openlaoke/core/extended_web/config_integration.py
65. openlaoke/core/extended_web/deepseek_client.py
66. openlaoke/core/extended_web/firefox_auth.py
67. openlaoke/core/extended_web/firefox_cookie_reader.py
68. openlaoke/core/extended_web/providers_api.py
69. openlaoke/core/extended_web/simple_browser_auth.py
70. openlaoke/core/extended_web/stream_factory.py
71. openlaoke/core/extended_web/types.py
72. openlaoke/core/extended_web/web_client.py
73. openlaoke/core/hooks.py
74. openlaoke/core/hybrid_model_manager.py
75. openlaoke/core/hyperauto/__init__.py
76. openlaoke/core/hyperauto/agent.py
77. openlaoke/core/hyperauto/agent_supervisor_integration.py
78. openlaoke/core/hyperauto/code_search.py
79. openlaoke/core/hyperauto/config.py
80. openlaoke/core/hyperauto/decision_engine.py
81. openlaoke/core/hyperauto/executor.py
82. openlaoke/core/hyperauto/learning.py
83. openlaoke/core/hyperauto/project_initializer.py
84. openlaoke/core/hyperauto/reflection.py
85. openlaoke/core/hyperauto/skill_generator.py
86. openlaoke/core/hyperauto/task_executor.py
87. openlaoke/core/hyperauto/test_runner.py
88. openlaoke/core/hyperauto/types.py
89. openlaoke/core/hyperauto/validator.py
90. openlaoke/core/hyperauto/verifier.py
91. openlaoke/core/hyperauto/workflow.py
92. openlaoke/core/intelligent_model_selector.py
93. openlaoke/core/intent_parser.py
94. openlaoke/core/intent_pipeline.py
95. openlaoke/core/intent_to_spec.py
96. openlaoke/core/interactive_input.py
97. openlaoke/core/knowledge_base.py
98. openlaoke/core/memory/__init__.py
99. openlaoke/core/memory/consolidation.py
100. openlaoke/core/memory/embedding.py

File: [1/252] examples/memory_demo.py

```
"""Example demonstrating the long-term memory and knowledge graph system."""
from __future__ import annotations
import asyncio
from openlaoke.core.memory import (
    KnowledgeGraph,
    KnowledgeNode,
    LongTermMemory,
    Memory,
)
async def main() -> None:
    print("=== Long-Term Memory System Demo ===\n")
    ltm = LongTermMemory()
    print("1. Storing memories...")
    m1 = Memory(id="", content="Python is a programming language created by Guido van Rossum")
    m2 = Memory(id="", content="JavaScript is used for web development")
    m3 = Memory(id="", content="Machine learning uses Python extensively")
    id1 = await ltm.store(m1)
    id2 = await ltm.store(m2)
    id3 = await ltm.store(m3)
    print(f"   Stored memory 1: {id1}")
    print(f"   Stored memory 2: {id2}")
    print(f"   Stored memory 3: {id3}\n")
    print("2. Recalling memories about Python...")
    results = await ltm.recall("Python programming", limit=2)
    for i, memory in enumerate(results):
        print(f"   {i + 1}. {memory.content} (importance: {memory.importance:.2f})\n")
    print("3. Creating associations...")
    await ltm.associate([id1, id3])
    print(f"   Associated {id1} and {id3}\n")
    print("4. Memory summary...")
    summary = await ltm.summarize()
    print(f"   Total memories: {summary.total_memories}")
    print(f"   Total associations: {summary.total_associations}")
    print(f"   Average importance: {summary.avg_importance:.2f}\n")
    print("=== Knowledge Graph Demo ===\n")
    kg = KnowledgeGraph()
    print("5. Building knowledge graph...")
    python_node = KnowledgeNode(id="", type="language", content="Python")
    ml_node = KnowledgeNode(id="", type="field", content="Machine Learning")
    lib_node = KnowledgeNode(id="", type="library", content="TensorFlow")
    python_id = await kg.add_node(python_node)
    ml_id = await kg.add_node(ml_node)
    lib_id = await kg.add_node(lib_node)
    await kg.add_edge(python_id, ml_id, "used_in")
    await kg.add_edge(ml_id, lib_id, "uses")
    print(f"   Added nodes: {python_id}, {ml_id}, {lib_id}")
    print("   Added edges: Python -> ML, ML -> TensorFlow\n")
    print("6. Finding neighbors...")
    neighbors = kg.get_neighbors(python_id)
    print(f"   Neighbors of Python: {[n.content for n in neighbors]}\n")
    print("7. Finding path...")
    path = kg.find_path(python_id, lib_id)
    if path:
        path_content = [kg.nodes[n].content for n in path]
        print(f"   Path from Python to TensorFlow: {path_content}\n")
    print("8. Knowledge graph visualization...")
    viz = await kg.visualize()
    print(f"   Nodes: {len(viz.nodes)}")
    print(f"   Edges: {len(viz.edges)}\n")
    print("=== Demo Complete ===")
if __name__ == "__main__":
```

```
asyncio.run(main())
```

File: [2/252] extract_chrome_cookies.py

```
#!/usr/bin/env python3
"""Extract cookies from Chrome."""
import json
import sqlite3
import tempfile
from pathlib import Path
from rich.console import Console
console = Console()
def get_chrome_profile_path():
    """Find Chrome profile path."""
    profile_path = Path.home() / "Library" / "Application Support" / "Google" / "Chrome" / "Def
    ault"

    if profile_path.exists():
        return profile_path

    # Try other profiles
    chrome_path = Path.home() / "Library" / "Application Support" / "Google" / "Chrome"
    if chrome_path.exists():
        for profile in chrome_path.iterdir():
            if profile.is_dir() and profile.name.startswith("Profile"):
                return profile

    return None
def extract_cookies(profile_path: Path, domain: str) -> str:
    """Extract cookies for a domain from Chrome."""
    cookies_file = profile_path / "Cookies"

    if not cookies_file.exists():
        return ""

    temp_cookie = tempfile.mktemp(suffix=".sqlite")
    import shutil
    shutil.copy2(cookies_file, temp_cookie)

    try:
        conn = sqlite3.connect(temp_cookie)
        cursor = conn.cursor()

        cursor.execute(
            "SELECT name, value FROM cookies WHERE host_key LIKE ?",
            (f"%{domain}%",)
        )

        cookies = [f"{name}={value}" for name, value in cursor.fetchall()]
        conn.close()

        return "; ".join(cookies)

    finally:
        import os
        os.unlink(temp_cookie)
def main():
    """Main function."""
    console.print()
    console.print("[bold cyan]Chrome Cookie Extractor[/bold cyan]")
    console.print()

    profile_path = get_chrome_profile_path()

    if not profile_path:
```

```
        console.print("[red][X] Chrome profile not found[/red]")
        console.print("[dim]Make sure you're using Chrome and logged in to the services[/dim]")
        return

    console.print(f"[green][OK] Found Chrome profile: {profile_path}[/green]")

    # Extract cookies
    console.print()
    console.print("[bold]Extracting cookies...[/bold]")

    domains = {
        "deepseek-chat": "deepseek.com",
        "qwen-web": "qwen.ai",
        "glm-web": "chatglm.cn",
    }

    for provider, domain in domains.items():
        cookie = extract_cookies(profile_path, domain)

        if cookie:
            console.print(f"[green][OK] {provider}: {len(cookie)} chars[/green]")

            # Save
            auth_file = Path.home() / ".openlaoke" / "extended_web" / f"{provider}.json"
            auth_file.parent.mkdir(parents=True, exist_ok=True)

            auth_data = {
                "provider_type": provider,
                "cookie": cookie,
                "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20
    100101 Firefox/115.0",
                "saved_at": "auto-extracted-chrome",
                "browser": "chrome",
            }

            with open(auth_file, "w") as f:
                json.dump(auth_data, f, indent=2)
        else:
            console.print(f"[yellow]? {provider}: not found[/yellow]")
if __name__ == "__main__":
    main()
```

File: [3/252] extract_firefox_cookie.py

```
#!/usr/bin/env python3
"""Automatically extract cookies from Firefox."""
import json
import sqlite3
import tempfile
from pathlib import Path
from rich.console import Console
console = Console()
def get_firefox_profile_path() -> Path | None:
    """Find Firefox profile path."""
    if Path.home().name == 'fred':  # macOS
        profiles_path = Path.home() / "Library" / "Application Support" / "Firefox" / "Profiles
    "
    else:
        profiles_path = Path.home() / ".mozilla" / "firefox" / "Profiles"

    if not profiles_path.exists():
        return None

    # Find default profile
    for profile_dir in profiles_path.iterdir():
        if profile_dir.is_dir() and "default" in profile_dir.name:
            return profile_dir

    # Return first profile if no default
    profiles = list(profiles_path.iterdir())
    if profiles:
        return profiles[0]

    return None
def extract_cookies(profile_path: Path, domains: list[str]) -> dict[str, str]:
    """Extract cookies from Firefox profile.

    Args:
        profile_path: Firefox profile directory
        domains: List of domains to extract cookies for

    Returns:
        Dictionary of cookie name -> value
    """
    cookies_file = profile_path / "cookies.sqlite"

    if not cookies_file.exists():
        console.print(f"[red]Cookies file not found: {cookies_file}[/red]")
        return {}

    # Copy to temp file to avoid locking
    temp_cookie = tempfile.mktemp(suffix=".sqlite")
    import shutil
    shutil.copy2(cookies_file, temp_cookie)

    try:
        conn = sqlite3.connect(temp_cookie)
        cursor = conn.cursor()

        cookies = {}
        for domain in domains:
            cursor.execute(
                "SELECT name, value FROM moz_cookies WHERE host LIKE ?",
                (f"%{domain}%",)
            )
```

```
            for name, value in cursor.fetchall():
                cookies[name] = value

        conn.close()
        return cookies

    finally:
        import os
        os.unlink(temp_cookie)
def main():
    """Main entry point."""
    console.print()
    console.print("[bold cyan]Firefox Cookie Extractor[/bold cyan]")
    console.print()

    # Find Firefox profile
    profile_path = get_firefox_profile_path()

    if not profile_path:
        console.print("[red][X] Firefox profile not found[/red]")
        console.print("[dim]Make sure Firefox is installed and has been used.[/dim]")
        return

    console.print(f"[green][OK] Found Firefox profile: {profile_path}[/green]")

    # Extract Qwen cookies
    console.print()
    console.print("[bold]Extracting Qwen Web cookies...[/bold]")

    cookies = extract_cookies(profile_path, ["qianwen.com", "aliyun.com"])

    if not cookies:
        console.print("[yellow]? No Qwen cookies found[/yellow]")
        console.print("[dim]Make sure you're logged in to https://www.qianwen.com in Firefox[/d
    im]")
        return

    console.print(f"[green][OK] Found {len(cookies)} cookies[/green]")

    # Check for required cookies
    required = ["_tb_token_", "cookie2"]
    found_required = [name for name in required if name in cookies]

    if len(found_required) >= 1:
        console.print(f"[green][OK] Found required cookies: {', '.join(found_required)}[/green]
    ")

        # Build cookie string
        cookie_parts = [f"{name}={value}" for name, value in cookies.items() if len(value) < 10
    00]
        cookie_string = "; ".join(cookie_parts[:20])  # Limit to first 20 cookies

        # Save to OpenLaoKe auth file
        auth_file = Path.home() / ".openlaoke" / "extended_web" / "qwen-web.json"
        auth_file.parent.mkdir(parents=True, exist_ok=True)

        auth_data = {
            "provider_type": "qwen-web",
            "cookie": cookie_string,
            "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/201001
    01 Firefox/115.0",
            "saved_at": "auto-extracted",
            "browser": "firefox",
        }
```

```
        with open(auth_file, "w") as f:
            json.dump(auth_data, f, indent=2)

        console.print()
        console.print(f"[bold green][OK] Cookie saved to: {auth_file}[/bold green]")
        console.print(f"  Cookie length: {len(cookie_string)}")
        console.print()
        console.print("[dim]You can now test with: python3 test_qwen_auth.py[/dim]")

    else:
        console.print("[yellow]? Required cookies not found[/yellow]")
        console.print(f"[dim]Found: {list(cookies.keys())[:10]}[/dim]")
        console.print("[dim]Make sure you're logged in to https://www.qianwen.com in Firefox[/d
    im]")
if __name__ == "__main__":
    main()
```

File: [4/252] extract_glm_cookie.py

```
#!/usr/bin/env python3
"""Extract GLM cookie from Firefox."""
import json
import sqlite3
import tempfile
from pathlib import Path
from rich.console import Console
console = Console()
def get_firefox_profile_path() -> Path | None:
    """Find Firefox profile path."""
    profiles_path = Path.home() / "Library" / "Application Support" / "Firefox" / "Profiles"

    if not profiles_path.exists():
        return None

    for profile_dir in profiles_path.iterdir():
        if profile_dir.is_dir() and "default" in profile_dir.name:
            return profile_dir

    profiles = list(profiles_path.iterdir())
    if profiles:
        return profiles[0]

    return None
def extract_glm_cookies(profile_path: Path) -> str | None:
    """Extract GLM cookies from Firefox."""
    cookies_file = profile_path / "cookies.sqlite"

    if not cookies_file.exists():
        return None

    # Copy to temp file
    temp_cookie = tempfile.mktemp(suffix=".sqlite")
    import shutil
    shutil.copy2(cookies_file, temp_cookie)

    try:
        conn = sqlite3.connect(temp_cookie)
        cursor = conn.cursor()

        # Search for GLM cookies
        cursor.execute(
            "SELECT name, value FROM moz_cookies WHERE host LIKE '%chatglm%' OR host LIKE '%zhi
    pu%'"
        )

        cookies = {}
        for name, value in cursor.fetchall():
            cookies[name] = value

        conn.close()

        if cookies:
            cookie_parts = [f"{name}={value}" for name, value in cookies.items()]
            return "; ".join(cookie_parts)

        return None

    finally:
        import os
        os.unlink(temp_cookie)
```

```
def main():
    """Main entry point."""
    console.print()
    console.print("[bold cyan]GLM Cookie Extractor[/bold cyan]")
    console.print()

    # Make sure user is logged in
    console.print("[bold]IMPORTANT:[/bold]")
    console.print("  1. Make sure you're logged in to https://chatglm.cn in Firefox")
    console.print("  2. If not, please login first, then run this script again")
    console.print()

    if not console.input("  Are you logged in to GLM in Firefox? [y/n]: ").lower().startswith('
    y'):
        console.print("[yellow]Please login first, then run this script again.[/yellow]")
        return

    # Find Firefox profile
    profile_path = get_firefox_profile_path()

    if not profile_path:
        console.print("[red][X] Firefox profile not found[/red]")
        return

    console.print(f"[green][OK] Found Firefox profile: {profile_path}[/green]")

    # Extract GLM cookies
    console.print()
    console.print("[bold]Extracting GLM cookies...[/bold]")

    cookie_string = extract_glm_cookies(profile_path)

    if not cookie_string:
        console.print("[red][X] No GLM cookies found[/red]")
        console.print("[dim]Make sure you're logged in to https://chatglm.cn in Firefox[/dim]")
        return

    console.print(f"[green][OK] Found GLM cookies![/green]")
    console.print(f"  Cookie length: {len(cookie_string)}")

    # Save to auth file
    auth_file = Path.home() / ".openlaoke" / "extended_web" / "glm-web.json"
    auth_file.parent.mkdir(parents=True, exist_ok=True)

    auth_data = {
        "provider_type": "glm-web",
        "cookie": cookie_string,
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 F
    irefox/115.0",
        "saved_at": "auto-extracted",
        "browser": "firefox",
    }

    with open(auth_file, "w") as f:
        json.dump(auth_data, f, indent=2)

    console.print()
    console.print(f"[bold green][OK] Cookie saved to: {auth_file}[/bold green]")
    console.print()
    console.print("[dim]You can now use GLM Web in OpenLaoKe:[/dim]")
    console.print("  openlaoke --provider extended-web --model glm-web")
if __name__ == "__main__":
    main()
```

File: [5/252] generate_pdfs.py

```
#!/usr/bin/env python3
"""Generate PDF from project code and documentation with Unicode support."""
from __future__ import annotations
import os
import re
from pathlib import Path
from fpdf import FPDF
class PDFGenerator(FPDF):
    """Custom PDF generator with Unicode support."""
    def __init__(self) -> None:
        super().__init__()
        self.set_auto_page_break(auto=True, margin=15)
        # Add Unicode font support
        try:
            # Try to use system fonts that support Unicode
            self.add_font("DejaVu", "", "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", uni=
    True)
            self.add_font(
                "DejaVu", "B", "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", uni=True
            )
            self.add_font(
                "DejaVuMono", "", "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf", uni=Tr
    ue
            )
            self.add_font(
                "DejaVuMono",
                "B",
                "/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf",
                uni=True,
            )
            self.has_unicode_font = True
        except:
            # Fallback to built-in fonts
            self.has_unicode_font = False
    def header(self) -> None:
        """Add header to each page."""
        if self.page_no() > 1:
            if self.has_unicode_font:
                self.set_font("DejaVu", "", 8)
            else:
                self.set_font("Courier", "", 8)
            self.set_text_color(128, 128, 128)
            self.cell(
                0,
                10,
                f"OpenLaoKe Project - Page {self.page_no()}",
                0,
                new_x="LMARGIN",
                new_y="NEXT",
                align="C",
            )
    def footer(self) -> None:
        """Add footer to each page."""
        self.set_y(-15)
        if self.has_unicode_font:
            self.set_font("DejaVu", "", 8)
        else:
            self.set_font("Courier", "", 8)
        self.set_text_color(128, 128, 128)
        self.cell(0, 10, f"Generated: 2025", 0, align="C")
    def clean_text(self, text: str) -> str:
```

```
"""Clean text to ensure it's compatible with fonts."""
if not text:
    return ""
# Replace special Unicode characters that might cause issues
replacements = {
    "\u2713": "[OK]",  # checkmark
    "\u2717": "[X]",  # cross
    "\u2192": "->",  # arrow right
    "\u2190": "<-",  # arrow left
    "\u2022": "*",  # bullet
    "\u2014": "--",  # em dash
    "\u2013": "-",  # en dash
    "\u25cf": "*",  # black circle
    "\u25cb": "o",  # white circle
    "\u2550": "=",  # box drawing
    "\u2551": "|",  # box drawing
    "\u2554": "+",  # box drawing
    "\u2557": "+",  # box drawing
    "\u255a": "+",  # box drawing
    "\u255d": "+",  # box drawing
    "\u2500": "-",  # box drawing horizontal
    "\u2502": "|",  # box drawing vertical
    "\u256c": "+",  # box drawing cross
    "\u2555": "+",  # box drawing
    "\u2556": "+",  # box drawing
    "\u255c": "+",  # box drawing
    "\u255b": "+",  # box drawing
    "\u255e": "+",  # box drawing
    "\u255f": "+",  # box drawing
    "\u2560": "+",  # box drawing
    "\u2561": "+",  # box drawing
    "\u2562": "+",  # box drawing
    "\u2563": "+",  # box drawing
    "\u2564": "+",  # box drawing
    "\u2565": "+",  # box drawing
    "\u2566": "+",  # box drawing
    "\u2567": "+",  # box drawing
    "\u2568": "+",  # box drawing
    "\u2569": "+",  # box drawing
    "\u256a": "+",  # box drawing
    "\u256b": "+",  # box drawing
    "\u29c9": "[]",  # box drawing
    "\u26a1": "[!]",  # lightning
    "\u274c": "[X]",  # cross mark
    "\u2705": "[OK]",  # check mark button
    "\ud83c\udf89": "[celebrate]",  # party popper
    "\ud83c\udf8a": "[celebrate]",  # confetti ball
    "\ud83c\udf88": "[balloon]",  # balloon
    "\ud83d\ude80": "[rocket]",  # rocket
    "\ud83c\udfaf": "[target]",  # target
    "\ud83d\udcf1": "[phone]",  # mobile phone
    "\ud83d\udcda": "[books]",  # books
    "\ud83d\udcdd": "[memo]",  # memo
    "\ud83d\udcbe": "[floppy]",  # floppy disk
    "\ud83d\udcc1": "[folder]",  # folder
    "\ud83d\udd12": "[lock]",  # lock
    "\ud83d\udd13": "[unlock]",  # unlock
    "\ud83c\udfb2": "[die]",  # game die
    "\u2b50": "[*]",  # star
    "\ud83d\udc9a": "[heart]",  # green heart
    "\ud83d\udc99": "[heart]",  # blue heart
    "\ud83d\udc9c": "[heart]",  # purple heart
    "\ud83d\udc9b": "[heart]",  # yellow heart
    "\u2764": "[heart]",  # heart
    "\u25b6": ">",  # play button
```

```
"\u25b8": ">",  # small play
"\u25aa": ".",  # small square
"\u25fc": "#",  # black square
"\u25aa\ufe0f": ".",  # small square
"\ud83d\udce4": "[mail]",  # outbox
"\ud83d\udce5": "[mail]",  # inbox
"\ud83d\udccb": "[clipboard]",  # clipboard
"\ud83d\udcbc": "[briefcase]",  # briefcase
"\ud83d\udcca": "[chart]",  # chart
"\ud83d\udcc8": "[chart]",  # chart up
"\ud83d\udcc9": "[chart]",  # chart down
"\u270f": "[pencil]",  # pencil
"\u2712": "[pencil]",  # pencil
"\ud83d\udd8a": "[pencil]",  # pencil
"\ud83d\udd8b": "[pen]",  # pen
"\ud83d\udccb": "[clipboard]",  # clipboard
"\ud83d\udcc4": "[page]",  # page
"\ud83d\udcc3": "[page]",  # page
"\ud83d\udcdd": "[memo]",  # memo
"\ud83d\udcd6": "[book]",  # open book
"\ud83d\udcda": "[books]",  # books
"\ud83d\udcd3": "[notebook]",  # notebook
"\ud83d\udcd2": "[book]",  # orange book
"\ud83d\udcd5": "[book]",  # closed book
"\ud83d\udcd9": "[bookmark]",  # bookmark
"\ud83d\udcd1": "[bookmark]",  # bookmark tabs
"\ud83d\udccc": "[pin]",  # pushpin
"\ud83d\udccd": "[pin]",  # round pushpin
"\ud83d\udce6": "[package]",  # package
"\ud83d\udce7": "[mail]",  # e-mail
"\ud83d\udce8": "[mail]",  # incoming envelope
"\ud83d\udce9": "[mail]",  # envelope with arrow
"\ud83d\udcea": "[mail]",  # closed mailbox
"\ud83d\udceb": "[mail]",  # open mailbox
"\ud83d\udcec": "[mail]",  # mailbox
"\ud83d\udced": "[mail]",  # mailbox with flag
"\ud83d\udcee": "[post]",  # postbox
"\ud83d\udcef": "[horn]",  # postal horn
"\ud83d\udcf0": "[news]",  # newspaper
"\ud83d\udcf1": "[phone]",  # mobile phone
"\ud83d\udcf2": "[phone]",  # mobile phone with arrow
"\ud83d\udcf3": "[phone]",  # vibration mode
"\ud83d\udcf4": "[phone]",  # mobile phone off
"\ud83d\udcf7": "[camera]",  # camera
"\ud83d\udcf9": "[video]",  # video camera
"\ud83d\udcfa": "[tv]",  # television
"\ud83d\udcfb": "[radio]",  # radio
"\ud83d\udcfc": "[video]",  # video cassette
"\ud83d\udd0a": "[sound]",  # speaker high
"\ud83d\udd0b": "[battery]",  # battery
"\ud83d\udd0c": "[plug]",  # electric plug
"\ud83d\udd0d": "[search]",  # left magnifier
"\ud83d\udd0e": "[search]",  # right magnifier
"\ud83d\udd0f": "[lock]",  # lock with ink pen
"\ud83d\udd10": "[lock]",  # closed lock with key
"\ud83d\udd11": "[key]",  # key
"\ud83d\udd12": "[lock]",  # lock
"\ud83d\udd13": "[unlock]",  # open lock
"\ud83d\udd14": "[bell]",  # bell
"\ud83d\udd15": "[bell]",  # bell with slash
"\ud83d\udd16": "[bookmark]",  # bookmark
"\ud83d\udd17": "[link]",  # link symbol
"\ud83d\udd18": "[radio]",  # radio button
"\ud83d\udd19": "[back]",  # back arrow
"\ud83d\udd1a": "[end]",  # end arrow
```

```
        "\ud83d\udd1b": "[on]",  # on arrow
        "\ud83d\udd1c": "[soon]",  # soon arrow
        "\ud83d\udd1d": "[top]",  # top arrow
        "\ud83d\udd1e": "[no]",  # no entry
        "\ud83d\udd1f": "[keycap]",  # keycap ten
        "\ud83d\udd20": "[keycap]",  # keycap one
        "\ud83d\udd21": "[keycap]",  # keycap two
        "\ud83d\udd22": "[keycap]",  # keycap three
        "\ud83d\udd23": "[keycap]",  # keycap four
        "\ud83d\udd24": "[keycap]",  # keycap five
        "\ud83d\udd25": "[fire]",  # fire
        "\ud83d\udd26": "[flash]",  # flashlight
        "\ud83d\udd27": "[wrench]",  # wrench
        "\ud83d\udd28": "[hammer]",  # hammer
        "\ud83d\udd29": "[pick]",  # pick
        "\ud83d\udd2a": "[hammer]",  # hammer and pick
        "\ud83d\udd2b": "[gun]",  # pistol
        "\ud83d\udd2c": "[microscope]",  # microscope
        "\ud83d\udd2d": "[telescope]",  # telescope
        "\ud83d\udd2e": "[crystal]",  # crystal ball
        "\ud83d\udd2f": "[diya]",  # diya lamp
        "\ud83d\udd30": "[bow]",  # bow and arrow
        "\ud83d\udd31": "[trident]",  # trident emblem
        "\ud83d\udd32": "[arrow]",  # black arrow up
        "\ud83d\udd33": "[arrow]",  # black arrow down
        "\ud83d\udd34": "[circle]",  # red circle
        "\ud83d\udd35": "[circle]",  # blue circle
    }
    for char, replacement in replacements.items():
        text = text.replace(char, replacement)
    # Remove any remaining non-ASCII characters except common ones
    if not self.has_unicode_font:
        # Only keep printable ASCII
        text = "".join(
            char if ord(char) < 128 and char.isprintable() or char in "\n\r\t" else "?"
            for char in text
        )
    return text
def add_title_page(self, title: str, subtitle: str) -> None:
    """Add title page."""
    self.add_page()
    if self.has_unicode_font:
        self.set_font("DejaVu", "B", 24)
    else:
        self.set_font("Helvetica", "B", 24)
    self.set_text_color(0, 0, 0)
    self.ln(80)
    self.cell(0, 20, title, 0, new_x="LMARGIN", new_y="NEXT", align="C")
    if self.has_unicode_font:
        self.set_font("DejaVu", "", 14)
    else:
        self.set_font("Helvetica", "", 14)
    self.set_text_color(100, 100, 100)
    self.cell(0, 10, subtitle, 0, new_x="LMARGIN", new_y="NEXT", align="C")
    self.ln(20)
    if self.has_unicode_font:
        self.set_font("DejaVuMono", "", 10)
    else:
        self.set_font("Courier", "", 10)
    self.cell(
        0, 10, "Generated from OpenLaoKe Project", 0, new_x="LMARGIN", new_y="NEXT", align=
"C"
    )
def add_file_header(self, filepath: str) -> None:
    """Add file header."""
```

```
    self.add_page()
    if self.has_unicode_font:
        self.set_font("DejaVu", "B", 12)
    else:
        self.set_font("Helvetica", "B", 12)
    self.set_text_color(0, 100, 200)
    clean_path = self.clean_text(filepath)
    self.cell(0, 10, f"File: {clean_path}", 0, new_x="LMARGIN", new_y="NEXT")
    self.set_draw_color(0, 100, 200)
    self.line(10, self.get_y(), 200, self.get_y())
    self.ln(5)
def add_code_content(self, content: str) -> None:
    """Add code content with proper formatting."""
    if self.has_unicode_font:
        self.set_font("DejaVuMono", "", 8)
    else:
        self.set_font("Courier", "", 8)
    self.set_text_color(0, 0, 0)
    # Clean the content
    content = self.clean_text(content)
    lines = content.split("\n")
    for line in lines:
        # Handle long lines - wrap at 95 characters
        while len(line) > 95:
            self.cell(0, 4, line[:95], 0, new_x="LMARGIN", new_y="NEXT")
            line = "    " + line[95:]
        if line:
            self.cell(0, 4, line, 0, new_x="LMARGIN", new_y="NEXT")
def add_markdown_content(self, content: str) -> None:
    """Add markdown content with basic formatting."""
    if self.has_unicode_font:
        self.set_font("DejaVu", "", 10)
    else:
        self.set_font("Helvetica", "", 10)
    self.set_text_color(0, 0, 0)
    # Clean the content
    content = self.clean_text(content)
    lines = content.split("\n")
    for line in lines:
        # Handle headers
        if line.startswith("# "):
            if self.has_unicode_font:
                self.set_font("DejaVu", "B", 16)
            else:
                self.set_font("Helvetica", "B", 16)
            self.ln(5)
            self.cell(0, 8, line[2:], 0, new_x="LMARGIN", new_y="NEXT")
            if self.has_unicode_font:
                self.set_font("DejaVu", "", 10)
            else:
                self.set_font("Helvetica", "", 10)
        elif line.startswith("## "):
            if self.has_unicode_font:
                self.set_font("DejaVu", "B", 14)
            else:
                self.set_font("Helvetica", "B", 14)
            self.ln(3)
            self.cell(0, 7, line[3:], 0, new_x="LMARGIN", new_y="NEXT")
            if self.has_unicode_font:
                self.set_font("DejaVu", "", 10)
            else:
                self.set_font("Helvetica", "", 10)
        elif line.startswith("### "):
            if self.has_unicode_font:
                self.set_font("DejaVu", "B", 12)
```

```
                else:
                    self.set_font("Helvetica", "B", 12)
                self.ln(2)
                self.cell(0, 6, line[4:], 0, new_x="LMARGIN", new_y="NEXT")
                if self.has_unicode_font:
                    self.set_font("DejaVu", "", 10)
                else:
                    self.set_font("Helvetica", "", 10)
            elif line.startswith("**") and line.endswith("**"):
                if self.has_unicode_font:
                    self.set_font("DejaVu", "B", 10)
                else:
                    self.set_font("Helvetica", "B", 10)
                self.cell(0, 5, line.strip("*"), 0, new_x="LMARGIN", new_y="NEXT")
                if self.has_unicode_font:
                    self.set_font("DejaVu", "", 10)
                else:
                    self.set_font("Helvetica", "", 10)
            elif line.strip() == "":
                self.ln(3)
            else:
                # Regular text with wrapping
                self.multi_cell(0, 5, line)
def collect_python_files(root_dir: str) -> list[str]:
    """Collect all Python files from project."""
    python_files = []
    for root, dirs, files in os.walk(root_dir):
        # Skip hidden and cache directories
        dirs[:] = [
            d
            for d in dirs
            if not d.startswith(".")
            and d not in ["__pycache__", "node_modules", "venv", ".git", "pdf_output"]
        ]
        for file in files:
            if file.endswith(".py"):
                filepath = os.path.join(root, file)
                python_files.append(filepath)
    return sorted(python_files)
def collect_documentation_files(root_dir: str) -> list[str]:
    """Collect all documentation files from project."""
    doc_files = []
    priority_files = [
        "README.md",
        "README_CN.md",
        "??.md",
        "AGENTS.md",
    ]
    # Add priority files first
    for priority in priority_files:
        priority_path = os.path.join(root_dir, priority)
        if os.path.exists(priority_path):
            doc_files.append(priority_path)
    # Then add other docs
    for root, dirs, files in os.walk(root_dir):
        dirs[:] = [
            d
            for d in dirs
            if not d.startswith(".")
            and d
            not in ["__pycache__", ".git", ".pytest_cache", "node_modules", "venv", "pdf_output
    "]
        ]
        for file in files:
            if file.endswith(".md") or file.endswith(".txt") or file.endswith(".rst"):
```

```
                filepath = os.path.join(root, file)
                if filepath not in doc_files and not any(
                    x in filepath for x in [".egg-info", "__pycache__"]
                ):
                    doc_files.append(filepath)
    return doc_files
def generate_code_pdf(root_dir: str, output_path: str) -> None:
    """Generate PDF from Python code files."""
    pdf = PDFGenerator()
    pdf.add_title_page("OpenLaoKe Project", "Complete Source Code Documentation")
    python_files = collect_python_files(root_dir)
    total_files = len(python_files)
    print(f"Found {total_files} Python files")
    # Add table of contents (limited to first 100)
    pdf.add_page()
    if pdf.has_unicode_font:
        pdf.set_font("DejaVu", "B", 14)
    else:
        pdf.set_font("Helvetica", "B", 14)
    pdf.cell(0, 10, "Table of Contents", 0, new_x="LMARGIN", new_y="NEXT")
    pdf.ln(5)
    if pdf.has_unicode_font:
        pdf.set_font("DejaVuMono", "", 9)
    else:
        pdf.set_font("Courier", "", 9)
    for i, filepath in enumerate(python_files[:100], 1):
        rel_path = os.path.relpath(filepath, root_dir)
        clean_path = pdf.clean_text(rel_path)
        pdf.cell(0, 5, f"{i}. {clean_path}", 0, new_x="LMARGIN", new_y="NEXT")
    # Add files content
    processed = 0
    errors = 0
    for i, filepath in enumerate(python_files):
        try:
            with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
                content = f.read()
            if not content.strip():
                continue
            rel_path = os.path.relpath(filepath, root_dir)
            clean_path = pdf.clean_text(rel_path)
            pdf.add_file_header(f"[{i + 1}/{total_files}] {clean_path}")
            pdf.add_code_content(content)
            processed += 1
            if (i + 1) % 50 == 0:
                print(f"Processed {i + 1}/{total_files} files...")
        except Exception as e:
            errors += 1
            print(f"Error processing {filepath}: {e}")
            continue
    pdf.output(output_path)
    print(f"\nCode PDF generated: {output_path}")
    print(f"Files processed: {processed}/{total_files}")
    print(f"Errors: {errors}")
def generate_docs_pdf(root_dir: str, output_path: str) -> None:
    """Generate PDF from documentation files."""
    pdf = PDFGenerator()
    pdf.add_title_page("OpenLaoKe Project", "Documentation Collection")
    doc_files = collect_documentation_files(root_dir)
    total_files = len(doc_files)
    print(f"Found {total_files} documentation files")
    # Add files content
    processed = 0
    errors = 0
    for i, filepath in enumerate(doc_files):
        try:
```

```
            with open(filepath, "r", encoding="utf-8", errors="ignore") as f:
                content = f.read()
            if not content.strip():
                continue
            rel_path = os.path.relpath(filepath, root_dir)
            clean_path = pdf.clean_text(rel_path)
            pdf.add_file_header(f"[{i + 1}/{total_files}] {clean_path}")
            pdf.add_markdown_content(content)
            processed += 1
            if (i + 1) % 10 == 0:
                print(f"Processed {i + 1}/{total_files} files...")
        except Exception as e:
            errors += 1
            print(f"Error processing {filepath}: {e}")
            continue
    pdf.output(output_path)
    print(f"\nDocumentation PDF generated: {output_path}")
    print(f"Files processed: {processed}/{total_files}")
    print(f"Errors: {errors}")
if __name__ == "__main__":
    root_dir = "/Users/fred/Documents/GitHub/cycleuser/OpenLaoKe"
    output_dir = os.path.join(root_dir, "pdf_output")
    os.makedirs(output_dir, exist_ok=True)
    print("=" * 60)
    print("PDF Generation Script for OpenLaoKe Project")
    print("=" * 60)
    print("\n1. Generating Code PDF...")
    print("-" * 60)
    generate_code_pdf(root_dir, os.path.join(output_dir, "OpenLaoKe_Source_Code.pdf"))
    print("\n2. Generating Documentation PDF...")
    print("-" * 60)
    generate_docs_pdf(root_dir, os.path.join(output_dir, "OpenLaoKe_Documentation.pdf"))
    print("\n" + "=" * 60)
    print("Done! PDF files saved to:", output_dir)
    print("=" * 60)
```

File: [6/252] manual_cookie_extractor.py

```
#!/usr/bin/env python3
"""Manual cookie extractor with step-by-step guide."""
import json
from pathlib import Path
from rich.console import Console
from rich.panel import Panel
console = Console()
def save_cookie(provider_type: str, cookie: str):
    """Save cookie to auth file."""
    auth_file = Path.home() / ".openlaoke" / "extended_web" / f"{provider_type}.json"
    auth_file.parent.mkdir(parents=True, exist_ok=True)

    auth_data = {
        "provider_type": provider_type,
        "cookie": cookie,
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 F
    irefox/115.0",
        "saved_at": "manual-extracted",
        "browser": "manual",
    }

    with open(auth_file, "w") as f:
        json.dump(auth_data, f, indent=2)
def extract_cookie_for(provider_type: str, service_name: str, login_url: str, cookie_names: lis
    t):
    """Extract cookie for a specific service."""
    console.print()
    console.print(Panel.fit(
        f"[bold cyan]{service_name}[/bold cyan]\n"
        f"[dim]{login_url}[/dim]",
        border_style="cyan"
    ))
    console.print()
    console.print("[bold]Steps to extract cookie:[/bold]")
    console.print()
    console.print(f"  [dim]1. Open: [cyan underline]{login_url}[/cyan underline][/dim]")
    console.print("  [dim]2. Login with your account[/dim]")
    console.print("  [dim]3. Press F12 (or right-click -> Inspect)[/dim]")
    console.print("  [dim]4. Go to 'Application' tab (Chrome) or 'Storage' tab (Firefox)[/dim]"
    )
    console.print("  [dim]5. Expand 'Cookies' on the left[/dim]")
    console.print(f"  [dim]6. Select the website ({login_url})[/dim]")
    console.print("  [dim]7. Find these cookies:[/dim]")
    for name in cookie_names:
        console.print(f"      [cyan]{name}[/cyan]")
    console.print("  [dim]8. Copy the 'Value' column for each cookie[/dim]")
    console.print()

    if not console.input("  Have you logged in and ready to extract? [y/n]: ").lower().startswi
    th('y'):
        console.print("[yellow]? Skipped[/yellow]")
        return False

    console.print()
    cookie_parts = []
    for name in cookie_names:
        value = console.input(f"  Enter [cyan]{name}[/cyan] value: ")
        if value and len(value) > 5:
            cookie_parts.append(f"{name}={value}")

    if cookie_parts:
```

```
        cookie_string = "; ".join(cookie_parts)
        save_cookie(provider_type, cookie_string)
        console.print()
        console.print(f"[green][OK] Cookie saved![/green]")
        console.print(f"  Length: {len(cookie_string)} chars")
        return True
    else:
        console.print("[yellow]? No cookie entered[/yellow]")
        return False
def main():
    """Main function."""
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print("[bold cyan]  Cookie Extractor[/bold cyan]")
    console.print("[bold cyan]========================================[/bold cyan]")

    services = [
        ("deepseek-chat", "DeepSeek Chat", "https://chat.deepseek.com", ["d_id", "ds_session_id
    "]),
        ("qwen-web", "Qwen Web (????)", "https://chat.qwen.ai", ["qwen_session"]),
        ("glm-web", "GLM Web (????)", "https://chatglm.cn", ["glmsession"]),
    ]

    console.print()
    console.print("[bold]Which service do you want to extract cookie for?[/bold]")
    console.print()
    for i, (provider, name, url, cookies) in enumerate(services, 1):
        console.print(f"  [{i}] {name}")

    console.print()
    choice = console.input("Select service [1/2/3]: ")

    if choice == "1":
        extract_cookie_for(*services[0])
    elif choice == "2":
        extract_cookie_for(*services[1])
    elif choice == "3":
        extract_cookie_for(*services[2])
    else:
        console.print("[yellow]? Invalid choice[/yellow]")

    console.print()
    console.print("[dim]After extracting cookies, you can use:[/dim]")
    console.print("  openlaoke --provider extended-web --model <service>")
if __name__ == "__main__":
    main()
```

File: [7/252] openlaoke/__init__.py

```
"""OpenLaoKe - Open-source terminal AI coding assistant."""
__version__ = "0.1.23"
__build_time__ = "2026-04-07T00:00:00Z"
```

File: [8/252] openlaoke/__main__.py

```
"""CLI entry point for python -m openlaoke."""
from openlaoke.entrypoints.cli import main
if __name__ == "__main__":
    main()
```

File: [9/252] openlaoke/commands/__init__.py

```
"""Slash commands for OpenLaoKe."""
```

File: [10/252] openlaoke/commands/auth_command.py

```
"""Extended Web authentication CLI commands."""
from __future__ import annotations
import asyncio
import os
import sys
from pathlib import Path
import click
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Confirm, Prompt
from rich.table import Table
from openlaoke.core.extended_web import BrowserAuthManager
from openlaoke.core.extended_web.types import WEB_PROVIDERS
console = Console()
@click.group()
def auth() -> None:
    """Authentication management commands."""
    pass
@auth.group("extended-web")
def extended_web() -> None:
    """Extended Web provider authentication."""
    pass
@extended_web.command("login")
@click.argument("provider")
@click.option("--port", default=0, help="Debug port (0 for auto)")
@click.option("--timeout", default=300, help="Login timeout in seconds")
@click.option("--auto", "-a", is_flag=True, help="Auto-launch browser (no manual setup)")
def login(provider: str, port: int, timeout: int, auto: bool) -> None:
    """Authenticate a web provider.
    Examples:
        openlaoke auth extended-web login deepseek-chat
        openlaoke auth extended-web login deepseek-chat --auto
    """
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    if provider not in WEB_PROVIDERS:
        console.print(f"[red]Error:[/red] Unknown provider: {provider}")
        console.print(f"\n[dim]Available providers: {', '.join(WEB_PROVIDERS.keys())}[/dim]")
        sys.exit(1)
    config = WEB_PROVIDERS[provider]
    console.print(
        Panel.fit(
            f"[bold cyan]Authenticating {config.name}[/bold cyan]\n[dim]{config.base_url}[/dim]
    ",
            border_style="cyan",
        )
    )
    console.print()
    auth_manager = BrowserAuthManager()
    # Check if already authenticated
    saved_auth = auth_manager.load_auth(provider)
    if saved_auth:
        console.print(f"[green][OK] Found existing authentication for {provider}[/green]")
        console.print(f"  Cookie length: {len(saved_auth.get('cookie', ''))}")
        console.print(f"  Saved at: {saved_auth.get('saved_at', 'unknown')}")
        console.print()
        if not Confirm.ask("Re-authenticate?", default=False):
            console.print("\n[dim]Using existing authentication.[/dim]")
            sys.exit(0)
    if auto:
        # Auto-launch browser mode
        console.print("[bold]Auto mode: Launching browser automatically...[/bold]")
```

```
    console.print()
    try:
        from openlaoke.core.extended_web.auto_browser import authenticate_with_auto_browser
        auth_data = asyncio.run(
            authenticate_with_auto_browser(
                provider_type=provider,
                login_url=config.login_url,
                timeout=timeout,
            )
        )
        console.print()
        console.print(
            Panel(
                f"[bold green][OK] Authentication successful![/bold green]\n\n"
                f"[dim]Provider:[/dim] [cyan]{provider}[/cyan]\n"
                f"[dim]Cookie length:[/dim] {len(auth_data.get('cookie', ''))}\n"
                f"[dim]Auth file:[/dim] {auth_manager._get_auth_file(provider)}",
                border_style="green",
            )
        )
    except Exception as e:
        console.print(f"\n[red][X] Error:[/red] {e}")
        console.print("\n[dim]Make sure you have Chrome/Edge installed.[/dim]")
        sys.exit(1)
else:
    # Manual mode (existing behavior)
    console.print("[bold]Manual mode: Please start browser with remote debugging[/bold]")
    console.print()
    console.print("If you haven't already, start Chrome with:")
    console.print()
    if sys.platform == "darwin":
        console.print("  [dim]# macOS[/dim]")
        console.print(
            "  [cyan]/Applications/Google\\ Chrome.app/Contents/MacOS/Google\\ Chrome \\[/c
yan]"
        )
        console.print(
            f"    [cyan]--remote-debugging-port={port if port != 0 else 9222} \\[/cyan]"
        )
        console.print("    [cyan]--user-data-dir=/tmp/chrome-debug[/cyan]")
    elif sys.platform == "win32":
        console.print("  [dim]# Windows[/dim]")
        console.print(
            f'  [cyan]"C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe" --remote
-debugging-port={port if port != 0 else 9222}[/cyan]'
        )
    else:
        console.print("  [dim]# Linux[/dim]")
        console.print(
            f"  [cyan]google-chrome --remote-debugging-port={port if port != 0 else 9222}[/
cyan]"
        )
    console.print()
    if not Confirm.ask("Is browser running with remote debugging?", default=True):
        console.print(
            "\n[yellow]Please start browser first, then run this command again.[/yellow]"
        )
        console.print("\n[dim]Or use --auto flag for automatic browser launch.[/dim]")
        sys.exit(0)
    console.print()
    console.print("[bold]Step 2: Login to the service[/bold]")
    console.print()
    # Open login page
    console.print(f"\n[dim]Opening {config.login_url}...[/dim]")
    console.print("[dim]Please complete the login in the browser window.[/dim]")
```

```
    console.print()
    try:
        # Run authentication
        auth_data = asyncio.run(
            auth_manager.authenticate_provider(provider, port if port != 0 else 9222, timeo
ut)
        )
        console.print()
        console.print(
            Panel(
                f"[bold green][OK] Authentication successful![/bold green]\n\n"
                f"[dim]Provider:[/dim] [cyan]{provider}[/cyan]\n"
                f"[dim]Cookie length:[/dim] {len(auth_data.get('cookie', ''))}\n"
                f"[dim]Auth file:[/dim] {auth_manager._get_auth_file(provider)}",
                border_style="green",
            )
        )
    except ConnectionError as e:
        console.print(f"\n[red][X] Connection error:[/red] {e}")
        console.print("\n[dim]Make sure browser is running with --remote-debugging-port[/di
m]")
        sys.exit(1)
    except TimeoutError as e:
        console.print(f"\n[red][X] Timeout:[/red] {e}")
        console.print("\n[dim]Please try again.[/dim]")
        sys.exit(1)
    except Exception as e:
        console.print(f"\n[red][X] Error:[/red] {e}")
        sys.exit(1)
config = WEB_PROVIDERS[provider]
console.print(
    Panel.fit(
        f"[bold cyan]Authenticating {config.name}[/bold cyan]\n[dim]{config.base_url}[/dim]
",
        border_style="cyan",
    )
)
console.print()
# Check if Chrome is running with remote debugging
console.print("[bold]Step 1: Start Chrome with remote debugging[/bold]")
console.print()
console.print("If you haven't already, start Chrome with:")
console.print()
if sys.platform == "darwin":
    console.print("  [dim]# macOS[/dim]")
    console.print(
        "  [cyan]/Applications/Google\\ Chrome.app/Contents/MacOS/Google\\ Chrome \\[/cyan]
"
    )
    console.print(f"    [cyan]--remote-debugging-port={port} \\[/cyan]")
    console.print("    [cyan]--user-data-dir=/tmp/chrome-debug[/cyan]")
elif sys.platform == "win32":
    console.print("  [dim]# Windows[/dim]")
    console.print(
        f'  [cyan]"C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe" --remote-deb
ugging-port={port}[/cyan]'
    )
else:
    console.print("  [dim]# Linux[/dim]")
    console.print(f"  [cyan]google-chrome --remote-debugging-port={port}[/cyan]")
console.print()
if not Confirm.ask("Is Chrome running with remote debugging?", default=True):
    console.print("\n[yellow]Please start Chrome first, then run this command again.[/yello
w]")
    sys.exit(0)
```

```
    # Authenticate
    console.print()
    console.print("[bold]Step 2: Login to the service[/bold]")
    console.print()
    auth_manager = BrowserAuthManager()
    # Check if already authenticated
    saved_auth = auth_manager.load_auth(provider)
    if saved_auth:
        console.print(f"[green][OK] Found existing authentication for {provider}[/green]")
        console.print(f"  Cookie length: {len(saved_auth.get('cookie', ''))}")
        console.print(f"  Saved at: {saved_auth.get('saved_at', 'unknown')}")
        console.print()
        if not Confirm.ask("Re-authenticate?", default=False):
            console.print("\n[dim]Using existing authentication.[/dim]")
            sys.exit(0)
    # Open login page
    console.print(f"\n[dim]Opening {config.login_url}...[/dim]")
    console.print("[dim]Please complete the login in the browser window.[/dim]")
    console.print()
    try:
        # Run authentication
        auth_data = asyncio.run(auth_manager.authenticate_provider(provider, port, timeout))
        console.print()
        console.print(
            Panel(
                f"[bold green][OK] Authentication successful![/bold green]\n\n"
                f"[dim]Provider:[/dim] [cyan]{provider}[/cyan]\n"
                f"[dim]Cookie length:[/dim] {len(auth_data.get('cookie', ''))}\n"
                f"[dim]Auth file:[/dim] {auth_manager._get_auth_file(provider)}",
                border_style="green",
            )
        )
    except ConnectionError as e:
        console.print(f"\n[red][X] Connection error:[/red] {e}")
        console.print("\n[dim]Make sure Chrome is running with --remote-debugging-port[/dim]")
        sys.exit(1)
    except TimeoutError as e:
        console.print(f"\n[red][X] Timeout:[/red] {e}")
        console.print("\n[dim]Please try again.[/dim]")
        sys.exit(1)
    except Exception as e:
        console.print(f"\n[red][X] Error:[/red] {e}")
        sys.exit(1)
@extended_web.command("list")
def list_auth() -> None:
    """List all authenticated providers."""
    auth_manager = BrowserAuthManager()
    auths = auth_manager.list_saved_auths()
    if not auths:
        console.print("[yellow]No authenticated providers found.[/yellow]")
        console.print(
            "\n[dim]Use 'openlaoke auth extended-web login <provider>' to authenticate.[/dim]"
        )
        return
    console.print(f"[bold]Authenticated providers ({len(auths)})[/bold]")
    console.print()
    table = Table(show_header=True)
    table.add_column("Provider", style="cyan")
    table.add_column("Name", style="green")
    table.add_column("Status", style="yellow")
    table.add_column("Cookie Length", justify="right")
    for provider_type in auths:
        auth_data = auth_manager.load_auth(provider_type)
        config = WEB_PROVIDERS.get(provider_type)
        name = config.name if config else provider_type
```

```
        # Check if auth is valid
        cookie = auth_data.get("cookie", "") if auth_data else ""
        status = "[green][OK] Valid[/green]" if len(cookie) > 10 else "[red][X] Invalid[/red]"
        table.add_row(
            provider_type,
            name,
            status,
            str(len(cookie)),
        )
    console.print(table)
@extended_web.command("remove")
@click.argument("provider")
def remove(provider: str) -> None:
    """Remove authentication for a provider.
    Examples:
        openlaoke auth extended-web remove deepseek-chat
    """
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    if provider not in WEB_PROVIDERS:
        console.print(f"[red]Error:[/red] Unknown provider: {provider}")
        sys.exit(1)
    auth_manager = BrowserAuthManager()
    auth_file = auth_manager._get_auth_file(provider)
    if not auth_file.exists():
        console.print(f"[yellow]No authentication found for {provider}[/yellow]")
        return
    config = WEB_PROVIDERS[provider]
    console.print(f"[bold]Remove authentication for {config.name}?[/bold]")
    console.print(f"  Auth file: {auth_file}")
    console.print()
    if not Confirm.ask("Are you sure?", default=False):
        console.print("[dim]Cancelled.[/dim]")
        return
    auth_manager.delete_auth(provider)
    console.print(f"[green][OK] Authentication removed for {provider}[/green]")
@extended_web.command("show")
@click.argument("provider")
def show(provider: str) -> None:
    """Show authentication details for a provider."""
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    if provider not in WEB_PROVIDERS:
        console.print(f"[red]Error:[/red] Unknown provider: {provider}")
        sys.exit(1)
    auth_manager = BrowserAuthManager()
    auth_data = auth_manager.load_auth(provider)
    if not auth_data:
        console.print(f"[yellow]No authentication found for {provider}[/yellow]")
        return
    config = WEB_PROVIDERS[provider]
    console.print(f"[bold]{config.name} Authentication[/bold]")
    console.print()
    # Mask cookie for security
    cookie = auth_data.get("cookie", "")
    if len(cookie) > 20:
        masked_cookie = f"{cookie[:10]}...{cookie[-10:]}"
    else:
        masked_cookie = "(short cookie)"
    console.print(f"  [dim]Provider:[/dim] {provider}")
    console.print(f"  [dim]Name:[/dim] {config.name}")
    console.print(f"  [dim]Base URL:[/dim] {config.base_url}")
    console.print(f"  [dim]Cookie:[/dim] {masked_cookie}")
    console.print(f"  [dim]Cookie length:[/dim] {len(cookie)}")
    console.print(f"  [dim]User agent:[/dim] {auth_data.get('user_agent', 'N/A')[:50]}...")
    console.print(
        f"  [dim]Has bearer token:[/dim] {'Yes' if auth_data.get('bearer_token') else 'No'}"
```

```
    )
    console.print(f"  [dim]Auth file:[/dim] {auth_manager._get_auth_file(provider)}")
    console.print()
    # Show required cookies
    console.print(f"  [dim]Required cookies:[/dim] {', '.join(config.required_cookies)}")
    # Check if all required cookies are present
    has_all = all(req in cookie for req in config.required_cookies)
    status = (
        "[green][OK] All required cookies present[/green]"
        if has_all
        else "[red][X] Missing required cookies[/red]"
    )
    console.print(f"  [dim]Status:[/dim] {status}")
@extended_web.command("test")
@click.argument("provider")
@click.option("--message", default="???????????", help="Test message")
def test(provider: str, message: str) -> None:
    """Test authentication by making an API call.
    Examples:
        openlaoke auth extended-web test deepseek-chat
    """
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    from openlaoke.core.extended_web.web_client import WebServiceClient
    if provider not in WEB_PROVIDERS:
        console.print(f"[red]Error:[/red] Unknown provider: {provider}")
        sys.exit(1)
    auth_manager = BrowserAuthManager()
    auth_data = auth_manager.load_auth(provider)
    if not auth_data:
        console.print(f"[red]Error:[/red] No authentication found for {provider}")
        console.print("\n[dim]Run 'openlaoke auth extended-web login {provider}' first.[/dim]")
        sys.exit(1)
    config = WEB_PROVIDERS[provider]
    console.print(f"[bold]Testing {config.name}...[/bold]")
    console.print(f"  Message: {message}")
    console.print()
    async def run_test() -> None:
        client = WebServiceClient(
            provider_type=provider,
            cookie=auth_data["cookie"],
            user_agent=auth_data.get("user_agent", ""),
            bearer_token=auth_data.get("bearer_token", ""),
            base_url=config.base_url,
            api_endpoint=config.api_endpoint,
            custom_headers=config.custom_headers,
        )
        try:
            messages = [{"role": "user", "content": message}]
            response = await client.chat(messages)
            content = response.get("content", "")
            if content:
                console.print(f"[green][OK] Response received:[/green]")
                console.print()
                console.print(f"  {content[:200]}...")
                console.print()
                usage = response.get("usage", {})
                if usage:
                    console.print(f"  [dim]Tokens:[/dim] {usage}")
            else:
                console.print(f"[yellow]? Empty response[/yellow]")
        except Exception as e:
            console.print(f"[red][X] Error:[/red] {e}")
            sys.exit(1)
        finally:
            await client.close()
```

```
    asyncio.run(run_test())
# Add auth command to main CLI
if __name__ == "__main__":
    auth()
```

File: [11/252] openlaoke/commands/base.py

```
"""Slash command system."""
from __future__ import annotations
import os
import sys
from abc import ABC, abstractmethod
from collections.abc import Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class CommandContext:
    """Context passed to command handlers."""
    app_state: AppState
    args: str = ""
    reply: Callable | None = None
@dataclass
class CommandResult:
    """Result of a command execution."""
    success: bool = True
    message: str = ""
    should_exit: bool = False
    should_clear: bool = False
class SlashCommand(ABC):
    """Base class for slash commands."""
    name: str = ""
    description: str = ""
    aliases: list[str] = []
    hidden: bool = False
    @abstractmethod
    async def execute(self, ctx: CommandContext) -> CommandResult: ...
class HelpCommand(SlashCommand):
    name = "help"
    description = "Show available commands"
    aliases = ["?"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from openlaoke.commands.registry import get_all_commands
        commands = get_all_commands()
        lines = ["Available commands:"]
        for cmd in sorted(commands, key=lambda c: c.name):
            if cmd.hidden:
                continue
            aliases_str = f" ({', '.join('/' + a for a in cmd.aliases)})" if cmd.aliases else "
    "
            lines.append(f"  /{cmd.name}{aliases_str} - {cmd.description}")
        lines.append("")
        lines.append("Type a message to chat with the AI. Use /exit to quit.")
        return CommandResult(message="\n".join(lines))
class ExitCommand(SlashCommand):
    name = "exit"
    description = "Exit OpenLaoKe"
    aliases = ["quit", "q"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        return CommandResult(should_exit=True)
class ClearCommand(SlashCommand):
    name = "clear"
    description = "Clear the screen and conversation"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        return CommandResult(should_clear=True, message="Screen cleared.")
class ModelCommand(SlashCommand):
    name = "model"
```

```
description = "Show or change the current model"
aliases = ["m"]
async def execute(self, ctx: CommandContext) -> CommandResult:
    args = ctx.args.strip()
    # Show current model and available models
    if not args:
        return self._show_model_info(ctx)
    # Interactive selection with flags
    if args.startswith("-"):
        if args in ["-l", "-list", "--list"]:
            return self._list_all_models(ctx)
        if args in ["-p", "-provider", "--provider"]:
            return self._list_providers(ctx)
        return CommandResult(
            success=False, message=f"Unknown option: {args}. Use: -l (list), -p (providers)
"
        )
    # Check if input is a number (select by index)
    if args.isdigit() or (args.startswith("#") and args[1:].isdigit()):
        return self._select_by_index(ctx, args)
    # Direct model name provided (with optional provider/model format)
    if "/" in args:
        provider_name, model_name = args.split("/", 1)
        return self._switch_provider_and_model(ctx, provider_name, model_name)
    # Set model name directly
    ctx.app_state.session_config.model = args
    if ctx.app_state.multi_provider_config:
        ctx.app_state.multi_provider_config.active_model = args
    return CommandResult(message=f"Model set to: {args}")
def _select_by_index(self, ctx: CommandContext, index_str: str) -> CommandResult:
    """Select model by numeric index."""
    # Parse index (support "#1" or "1" format)
    if index_str.startswith("#"):
        index_str = index_str[1:]
    try:
        index = int(index_str)
    except ValueError:
        return CommandResult(success=False, message=f"Invalid index: {index_str}")
    if not ctx.app_state.multi_provider_config:
        return CommandResult(message="No provider configured. Run --config first.")
    provider_name = ctx.app_state.multi_provider_config.active_provider
    provider = ctx.app_state.multi_provider_config.providers.get(provider_name)
    if not provider or not provider.models:
        return CommandResult(message=f"No models available for provider: {provider_name}")
    models = provider.models
    if index < 1 or index > len(models):
        return CommandResult(
            success=False,
            message=f"Invalid index {index}. Available models: 1-{len(models)}",
        )
    selected_model = models[index - 1]
    ctx.app_state.session_config.model = selected_model
    if ctx.app_state.multi_provider_config:
        ctx.app_state.multi_provider_config.active_model = selected_model
    lines = [
        f"[green][OK][/green] Model set to: {selected_model}",
        f"[dim]Provider: {provider_name}[/dim]",
        "",
        "[dim]Tip: You can also use the model name directly[/dim]",
    ]
    return CommandResult(message="\n".join(lines))
def _switch_provider_and_model(
    self, ctx: CommandContext, provider_name: str, model_name: str
) -> CommandResult:
    """Switch to a different provider and model."""
```

```
    if not ctx.app_state.multi_provider_config:
        return CommandResult(message="No provider configured. Run --config first.")
    provider_name = provider_name.lower()
    if provider_name not in ctx.app_state.multi_provider_config.providers:
        available = [
            name
            for name, p in ctx.app_state.multi_provider_config.providers.items()
            if p.enabled
        ]
        return CommandResult(
            success=False,
            message=f"Provider '{provider_name}' not found. Available: {', '.join(available
)}",
        )
    provider = ctx.app_state.multi_provider_config.providers[provider_name]
    if not provider.enabled:
        return CommandResult(
            success=False,
            message=f"Provider '{provider_name}' is disabled. Run --config to enable.",
        )
    if not provider.is_configured() and not provider.is_local:
        return CommandResult(
            success=False,
            message=f"Provider '{provider_name}' is not configured. Run --config to set API
 key.",
        )
    # Switch provider
    ctx.app_state.multi_provider_config.active_provider = provider_name
    # Handle model selection (can be index or name)
    if model_name.isdigit() or (model_name.startswith("#") and model_name[1:].isdigit()):
        # Select by index
        if model_name.startswith("#"):
            model_name = model_name[1:]
        try:
            index = int(model_name)
            if provider.models and 1 <= index <= len(provider.models):
                model_name = provider.models[index - 1]
            else:
                ctx.app_state.session_config.model = provider.default_model or (
                    provider.models[0] if provider.models else ""
                )
                return CommandResult(
                    success=False,
                    message=f"Invalid model index {index}. Using default: {ctx.app_state.se
ssion_config.model}",
                )
        except ValueError:
            pass
    ctx.app_state.session_config.model = model_name
    if ctx.app_state.multi_provider_config:
        ctx.app_state.multi_provider_config.active_model = model_name
    return CommandResult(
        message=f"[green][OK][/green] Switched to provider: {provider_name}\nModel: {model_
name}"
    )
def _show_model_info(self, ctx: CommandContext) -> CommandResult:
    """Show current model and available models for current provider."""
    current_model = ctx.app_state.session_config.model
    provider_name = "unknown"
    if ctx.app_state.multi_provider_config:
        provider_name = ctx.app_state.multi_provider_config.active_provider
        provider = ctx.app_state.multi_provider_config.providers.get(provider_name)
        if provider:
            models = provider.models
            lines = [
```

```
                f"[bold]Current provider:[/bold] {provider_name}",
                f"[bold]Current model:[/bold] {current_model}",
                "",
                f"[bold]Available models ({provider_name}):[/bold]",
            ]
            for i, model in enumerate(models, 1):
                marker = " [cyan](current)[/cyan]" if model == current_model else ""
                lines.append(f"  [{i}] {model}{marker}")
            lines.extend(
                [
                    "",
                    "[dim]Usage:[/dim]",
                    "  /model <name>        - Switch to specific model",
                    "  /model <1-8>         - Select by index number",
                    "  /model #<1-8>        - Select by index (with # prefix)",
                    "  /model <provider>/<model> - Switch provider and model",
                    "  /model -l            - List all models from all providers",
                    "  /model -p            - List all providers",
                ]
            )
            return CommandResult(message="\n".join(lines))
    return CommandResult(message=f"Current model: {current_model}")
def _list_all_models(self, ctx: CommandContext) -> CommandResult:
    """List models from all configured providers."""
    lines = ["[bold]Available models from all providers:[/bold]", ""]
    if not ctx.app_state.multi_provider_config:
        return CommandResult(message="No provider configured. Run --config first.")
    current_provider = ctx.app_state.multi_provider_config.active_provider
    current_model = ctx.app_state.session_config.model
    for provider_name, provider in ctx.app_state.multi_provider_config.providers.items():
        if not provider.enabled:
            continue
        is_current = provider_name == current_provider
        header = f"[bold cyan]{provider_name}[/bold cyan]"
        if is_current:
            header += " [green](active)[/green]"
        lines.append(header)
        for model in provider.models:
            marker = ""
            if is_current and model == current_model:
                marker = " [cyan](current)[/cyan]"
            lines.append(f"  {model}{marker}")
        lines.append("")
    lines.append("[dim]Usage: /model <provider>/<model> or just /model <model>[/dim]")
    return CommandResult(message="\n".join(lines))
def _list_providers(self, ctx: CommandContext) -> CommandResult:
    """List all available providers."""
    lines = ["[bold]Available providers:[/bold]", ""]
    if not ctx.app_state.multi_provider_config:
        return CommandResult(message="No provider configured. Run --config first.")
    current_provider = ctx.app_state.multi_provider_config.active_provider
    for provider_name, provider in ctx.app_state.multi_provider_config.providers.items():
        status = ""
        if provider_name == current_provider:
            status = " [green](active)[/green]"
        elif provider.enabled and provider.is_configured():
            status = " [yellow](configured)[/yellow]"
        elif provider.enabled:
            status = " [dim](available)[/dim]"
        else:
            status = " [dim](disabled)[/dim]"
        model_info = provider.default_model or provider.models[0] if provider.models else "
"
        lines.append(f"  {provider_name}{status}")
        if model_info:
```

```
                lines.append(f"    [dim]Model: {model_info}[/dim]")
        lines.extend(
            [
                "",
                "[dim]Usage: /provider <name> to switch active provider[/dim]",
            ]
        )
        return CommandResult(message="\n".join(lines))
class ProviderCommand(SlashCommand):
    name = "provider"
    description = "Switch to a different provider"
    aliases = ["p"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip()
        if not ctx.app_state.multi_provider_config:
            return CommandResult(message="No provider configured. Run --config first.")
        # Show current provider if no args
        if not args:
            current_provider = ctx.app_state.multi_provider_config.active_provider
            current_model = ctx.app_state.session_config.model
            lines = [
                f"[bold]Current provider:[/bold] {current_provider}",
                f"[bold]Current model:[/bold] {current_model}",
                "",
                "[bold]Available providers:[/bold]",
            ]
            for provider_name, provider in ctx.app_state.multi_provider_config.providers.items(
    ):
                if not provider.enabled:
                    continue
                status = ""
                if provider_name == current_provider:
                    status = " [green](active)[/green]"
                elif provider.is_configured():
                    status = " [yellow](ready)[/yellow]"
                lines.append(f"  {provider_name}{status}")
            lines.extend(
                [
                    "",
                    "[dim]Usage: /provider <name> - Switch to a different provider[/dim]",
                ]
            )
            return CommandResult(message="\n".join(lines))
        # Switch to specified provider
        provider_name = args.lower()
        if provider_name not in ctx.app_state.multi_provider_config.providers:
            available = [
                name
                for name, p in ctx.app_state.multi_provider_config.providers.items()
                if p.enabled
            ]
            return CommandResult(
                success=False,
                message=f"Provider '{provider_name}' not found. Available: {', '.join(available
    )}",
            )
        provider = ctx.app_state.multi_provider_config.providers[provider_name]
        if not provider.enabled:
            return CommandResult(
                success=False,
                message=f"Provider '{provider_name}' is disabled. Run --config to enable.",
            )
        if not provider.is_configured() and not provider.is_local:
            return CommandResult(
                success=False,
```

```
                message=f"Provider '{provider_name}' is not configured. Run --config to set API
     key.",
            )
        # Switch provider
        ctx.app_state.multi_provider_config.active_provider = provider_name
        # Set default model for this provider
        default_model = provider.default_model or provider.models[0] if provider.models else ""
        if default_model:
            ctx.app_state.session_config.model = default_model
            ctx.app_state.multi_provider_config.active_model = default_model
        return CommandResult(
            message=f"[green][OK][/green] Switched to provider: {provider_name}\nModel: {defaul
    t_model}"
        )
class PermissionCommand(SlashCommand):
    name = "permission"
    description = "Change permission mode (default/auto/bypass)"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from openlaoke.types.core_types import PermissionMode
        args = ctx.args.strip().lower()
        if not args:
            current = ctx.app_state.permission_config.mode.value
            return CommandResult(message=f"Current permission mode: {current}")
        try:
            mode = PermissionMode(args)
            ctx.app_state.permission_config.mode = mode
            return CommandResult(message=f"Permission mode set to: {mode.value}")
        except ValueError:
            return CommandResult(
                success=False,
                message=f"Invalid mode: {args}. Use: default, auto, bypass",
            )
class CompactCommand(SlashCommand):
    name = "compact"
    description = "Compact the conversation to save context"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        messages = ctx.app_state.get_messages()
        if len(messages) < 4:
            return CommandResult(message="Conversation is too short to compact.")
        ctx.app_state.messages = messages[:2]
        ctx.app_state.messages.append(
            type(messages[-1])(
                role=messages[-1].role,
                content="[Conversation compacted - earlier messages summarized]",
            )
        )
        return CommandResult(message="Conversation compacted.")
class CostCommand(SlashCommand):
    name = "cost"
    description = "Show current session cost and token usage"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        usage = ctx.app_state.token_usage
        cost = ctx.app_state.cost_info
        lines = [
            "Session usage:",
            f"  Input tokens:  {usage.input_tokens:,}",
            f"  Output tokens: {usage.output_tokens:,}",
            f"  Cache read:    {usage.cache_read_tokens:,}",
            f"  Cache created: {usage.cache_creation_tokens:,}",
            f"  Total tokens:  {usage.total_tokens:,}",
            "",
            "Session cost:",
            f"  Input:         ${cost.input_cost:.4f}",
            f"  Output:        ${cost.output_cost:.4f}",
            f"  Cache read:    ${cost.cache_read_cost:.4f}",
```

```
            f"  Cache created: ${cost.cache_creation_cost:.4f}",
            f"  Total:         ${cost.total_cost:.4f}",
        ]
        return CommandResult(message="\n".join(lines))
class CwdCommand(SlashCommand):
    name = "cwd"
    description = "Show or change the current working directory"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip()
        if not args:
            return CommandResult(message=f"Working directory: {ctx.app_state.get_cwd()}")
        new_cwd = os.path.expanduser(args)
        if os.path.isdir(new_cwd):
            ctx.app_state.set_cwd(new_cwd)
            os.chdir(new_cwd)
            return CommandResult(message=f"Working directory: {new_cwd}")
        return CommandResult(success=False, message=f"Not a directory: {new_cwd}")
class ResumeCommand(SlashCommand):
    name = "resume"
    description = "Resume the last session"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        persist_dir = os.path.expanduser("~/.openlaoke/sessions")
        if not os.path.exists(persist_dir):
            return CommandResult(message="No sessions found to resume.")
        sessions = sorted(os.listdir(persist_dir), reverse=True)
        if not sessions:
            return CommandResult(message="No sessions found to resume.")
        latest = sessions[0]
        session_path = os.path.join(persist_dir, latest)
        ctx.app_state.set_persist_path(session_path)
        ctx.app_state.resume_session = True
        return CommandResult(message=f"Session {latest} will be resumed.")
class CommandsCommand(SlashCommand):
    name = "commands"
    description = "Show example commands to try"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        examples = [
            "Examples:",
            '  "Write a Python script that reads a CSV and prints summary stats"',
            '  "Find all TODO comments in the src/ directory"',
            '  "Explain how the authentication system works"',
            '  "Refactor the database queries to use async"',
            '  "Add unit tests for the user service"',
            '  "Create a REST API for managing tasks"',
            "",
            "You can also use tools directly:",
            "  /Bash ls -la",
            "  /Read src/main.py",
        ]
        return CommandResult(message="\n".join(examples))
class SettingsCommand(SlashCommand):
    name = "settings"
    description = "Show current settings"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        config = ctx.app_state.session_config
        perm = ctx.app_state.permission_config
        lines = [
            "Settings:",
            f"  Model:       {config.model}",
            f"  Max tokens:  {config.max_tokens}",
            f"  Temperature: {config.temperature}",
            f"  Thinking:    {config.thinking_budget} tokens",
            f"  Permissions: {perm.mode.value}",
            f"  Working dir: {ctx.app_state.get_cwd()}",
        ]
```

```
        return CommandResult(message="\n".join(lines))
class DoctorCommand(SlashCommand):
    name = "doctor"
    description = "Run diagnostic checks for environment, API, and tools"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        import platform
        import shutil
        lines = ["Diagnostic Report:", ""]
        lines.append("System:")
        lines.append(f"  Platform:     {platform.system()} {platform.release()}")
        lines.append(f"  Python:       {sys.version.split()[0]}")
        lines.append(f"  Architecture: {platform.machine()}")
        lines.append("")
        lines.append("Environment:")
        env_checks = [
            ("ANTHROPIC_API_KEY", "Anthropic"),
            ("OPENAI_API_KEY", "OpenAI"),
            ("DASHSCOPE_API_KEY", "Aliyun"),
            ("MINIMAX_API_KEY", "MiniMax"),
        ]
        for env_var, provider in env_checks:
            status = "set" if os.environ.get(env_var) else "not set"
            lines.append(f"  {provider}: {status}")
        lines.append("")
        lines.append("Tools:")
        tool_checks = ["git", "python3", "node", "npm", "docker"]
        for tool in tool_checks:
            path = shutil.which(tool)
            status = path if path else "not found"
            lines.append(f"  {tool}: {status}")
        lines.append("")
        lines.append("Configuration:")
        config_dir = os.path.expanduser("~/.openlaoke")
        agents_md = os.path.join(ctx.app_state.get_cwd(), "AGENTS.md")
        lines.append(
            f"  Config dir:   {config_dir} ({'exists' if os.path.exists(config_dir) else 'missi
    ng'})"
        )
        lines.append(
            f"  AGENTS.md:    {agents_md} ({'exists' if os.path.exists(agents_md) else 'missing
    '})"
        )
        lines.append("")
        lines.append("Session:")
        lines.append(f"  Session ID:   {ctx.app_state.session_id}")
        lines.append(f"  Model:        {ctx.app_state.session_config.model}")
        lines.append(f"  Messages:     {len(ctx.app_state.messages)}")
        lines.append(f"  Tasks:        {len(ctx.app_state.tasks)}")
        return CommandResult(message="\n".join(lines))
class InitCommand(SlashCommand):
    name = "init"
    description = "Initialize AGENTS.md in current project"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        cwd = ctx.app_state.get_cwd()
        agents_path = os.path.join(cwd, "AGENTS.md")
        if os.path.exists(agents_path):
            return CommandResult(message=f"AGENTS.md already exists at {agents_path}")
        default_content = """# AGENTS.md - Project Guide
This file contains instructions for AI assistants working with this codebase.
## Project Overview
<!-- Describe your project here -->
## Commands
### Build
```bash
# Add build commands here
```
```

```
```
Test
```bash
# Add test commands here
```
Lint
```bash
# Add lint commands here
```
Code Style
<!-- Add coding conventions and style guidelines -->
Architecture
<!-- Describe the project architecture -->
Important Files
<!-- List important files and their purposes -->
"""
        try:
            with open(agents_path, "w", encoding="utf-8") as f:
                f.write(default_content)
            return CommandResult(message=f"Created AGENTS.md at {agents_path}")
        except Exception as e:
            return CommandResult(success=False, message=f"Failed to create AGENTS.md: {e}")
class McpCommand(SlashCommand):
    name = "mcp"
    description = "Manage MCP servers (list, add, remove, enable, disable)"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        if not args or args == "list":
            return self._list_servers(ctx)
        elif args.startswith("add "):
            return self._add_server(ctx, args[4:].strip())
        elif args.startswith("remove "):
            return self._remove_server(ctx, args[7:].strip())
        elif args.startswith("enable "):
            return self._enable_server(ctx, args[7:].strip())
        elif args.startswith("disable "):
            return self._disable_server(ctx, args[8:].strip())
        else:
            return CommandResult(
                success=False,
                message="Usage: /mcp [list|add <name>|remove <name>|enable <name>|disable <name
    >]",
)
    def _list_servers(self, ctx: CommandContext) -> CommandResult:
        mcp_service = getattr(ctx.app_state, "mcp_service", None)
        lines = ["MCP Servers:", ""]
        if mcp_service and hasattr(mcp_service, "servers"):
            if not mcp_service.servers:
                lines.append(" No MCP servers configured.")
            else:
                for name, conn in mcp_service.servers.items():
                    status = "connected" if conn.connected else "disconnected"
                    tools_count = len(conn.tools) if conn.tools else 0
                    error = f" (error: {conn.error})" if conn.error else ""
                    lines.append(f" {name}: {status}, {tools_count} tools{error}")
        else:
            config_path = os.path.expanduser("~/.openlaoke/mcp_servers.json")
            if os.path.exists(config_path):
                import json
                try:
                    with open(config_path) as f:
                        data = json.load(f)
                    servers = data.get("mcpServers", {})
                    if not servers:
                        lines.append(" No MCP servers configured.")
```
```

```
                    else:
                        for name in servers:
                            lines.append(f"  {name}: not loaded")
                except Exception:
                    lines.append(f"  Error reading {config_path}")
            else:
                lines.append("  No MCP servers configured.")
                lines.append(f"  Create {config_path} to add servers.")
        return CommandResult(message="\n".join(lines))
    def _add_server(self, ctx: CommandContext, name: str) -> CommandResult:
        return CommandResult(
            success=False,
            message=f"To add MCP server '{name}', edit ~/.openlaoke/mcp_servers.json",
        )
    def _remove_server(self, ctx: CommandContext, name: str) -> CommandResult:
        return CommandResult(
            success=False,
            message=f"To remove MCP server '{name}', edit ~/.openlaoke/mcp_servers.json",
        )
    def _enable_server(self, ctx: CommandContext, name: str) -> CommandResult:
        return CommandResult(message=f"MCP server '{name}' enabled (restart required)")
    def _disable_server(self, ctx: CommandContext, name: str) -> CommandResult:
        return CommandResult(message=f"MCP server '{name}' disabled (restart required)")
class ThemeCommand(SlashCommand):
    name = "theme"
    description = "Set or show terminal theme (dark/light/custom)"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        themes = ["dark", "light", "custom"]
        if not args:
            current = getattr(ctx.app_state, "theme", "dark")
            lines = [
                f"Current theme: {current}",
                "",
                "Available themes:",
                "  dark   - Dark theme (default)",
                "  light  - Light theme",
                "  custom - Custom theme (requires config)",
            ]
            return CommandResult(message="\n".join(lines))
        if args not in themes:
            return CommandResult(
                success=False,
                message=f"Invalid theme '{args}'. Available: {', '.join(themes)}",
            )
        ctx.app_state.theme = args
        return CommandResult(message=f"Theme set to: {args}")
class VimCommand(SlashCommand):
    name = "vim"
    description = "Toggle Vim input mode"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        if not args:
            current = ctx.app_state.vim_mode
            return CommandResult(message=f"Vim mode: {'enabled' if current else 'disabled'}")
        if args in ("on", "enable", "true", "1"):
            ctx.app_state.vim_mode = True
            return CommandResult(message="Vim mode enabled")
        elif args in ("off", "disable", "false", "0"):
            ctx.app_state.vim_mode = False
            return CommandResult(message="Vim mode disabled")
        elif args == "toggle":
            current = ctx.app_state.vim_mode
            ctx.app_state.vim_mode = not current
            return CommandResult(message=f"Vim mode: {'enabled' if not current else 'disabled'}
```

```
    ")
        else:
            return CommandResult(
                success=False,
                message="Usage: /vim [on|off|toggle]",
            )
class HooksCommand(SlashCommand):
    name = "hooks"
    description = "Manage hooks (list, add, remove, test)"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        if not args or args == "list":
            return self._list_hooks(ctx)
        elif args.startswith("test "):
            return self._test_hook(ctx, args[5:].strip())
        else:
            return CommandResult(
                success=False,
                message="Usage: /hooks [list|test <hook_type>]",
            )
    def _list_hooks(self, ctx: CommandContext) -> CommandResult:
        hook_manager = getattr(ctx.app_state, "hook_manager", None)
        lines = ["Registered Hooks:", ""]
        if hook_manager:
            hooks = hook_manager.get_registered_hooks()
            if not hooks:
                lines.append("  No hooks registered.")
            else:
                from openlaoke.types.hooks import HookType
                for hook_type in HookType:
                    registered = hooks.get(hook_type.value, [])
                    if registered:
                        lines.append(f"  {hook_type.value}:")
                        for name in registered:
                            lines.append(f"    - {name}")
                    else:
                        lines.append(f"  {hook_type.value}: (none)")
        else:
            from openlaoke.types.hooks import HookType
            lines.append("  Hook manager not initialized.")
            lines.append("")
            lines.append("  Available hook types:")
            for hook_type in HookType:
                lines.append(f"    - {hook_type.value}")
        return CommandResult(message="\n".join(lines))
    def _test_hook(self, ctx: CommandContext, hook_type: str) -> CommandResult:
        from openlaoke.types.hooks import HookType
        try:
            HookType(hook_type)
        except ValueError:
            return CommandResult(
                success=False,
                message=f"Invalid hook type: {hook_type}",
            )
        return CommandResult(message=f"Hook test for '{hook_type}' - no handlers registered")
class ExportCommand(SlashCommand):
    name = "export"
    description = "Export session to JSON or Markdown format"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        if not args:
            return CommandResult(
                message="Usage: /export [json|markdown] [filename]\n"
                "  /export json - Export as JSON\n"
                "  /export markdown - Export as Markdown"
```

```
        )
    parts = args.split(maxsplit=1)
    format_type = parts[0]
    filename = parts[1] if len(parts) > 1 else None
    if format_type == "json":
        return self._export_json(ctx, filename)
    elif format_type in ("markdown", "md"):
        return self._export_markdown(ctx, filename)
    else:
        return CommandResult(
            success=False,
            message=f"Invalid format: {format_type}. Use 'json' or 'markdown'.",
        )
def _export_json(self, ctx: CommandContext, filename: str | None) -> CommandResult:
    import json
    from datetime import datetime
    data = {
        "session_id": ctx.app_state.session_id,
        "exported_at": datetime.now().isoformat(),
        "model": ctx.app_state.session_config.model,
        "messages": [m.to_dict() for m in ctx.app_state.messages],
        "token_usage": {
            "input_tokens": ctx.app_state.token_usage.input_tokens,
            "output_tokens": ctx.app_state.token_usage.output_tokens,
            "cache_read_tokens": ctx.app_state.token_usage.cache_read_tokens,
            "cache_creation_tokens": ctx.app_state.token_usage.cache_creation_tokens,
        },
        "cost": ctx.app_state.cost_info.total_cost,
    }
    sessions_dir = os.path.expanduser("~/.openlaoke/sessions")
    os.makedirs(sessions_dir, exist_ok=True)
    if filename:
        path = os.path.expanduser(filename)
    else:
        path = os.path.join(sessions_dir, f"session_{ctx.app_state.session_id}.json")
    try:
        with open(path, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2, default=str)
        return CommandResult(message=f"Session exported to {path}")
    except Exception as e:
        return CommandResult(success=False, message=f"Export failed: {e}")
def _export_markdown(self, ctx: CommandContext, filename: str | None) -> CommandResult:
    from datetime import datetime
    lines = [
        "# Session Export",
        "",
        f"**Session ID:** {ctx.app_state.session_id}",
        f"**Exported at:** {datetime.now().isoformat()}",
        f"**Model:** {ctx.app_state.session_config.model}",
        "",
        "---",
        "",
        "## Conversation",
        "",
    ]
    for msg in ctx.app_state.messages:
        role = msg.role.value if hasattr(msg.role, "value") else str(msg.role)
        content = getattr(msg, "content", "")
        lines.append(f"### {role.title()}")
        lines.append("")
        lines.append(content)
        lines.append("")
    lines.append("---")
    lines.append("")
    lines.append("## Statistics")
```

```
        lines.append("")
        lines.append(f"- Input tokens: {ctx.app_state.token_usage.input_tokens:,}")
        lines.append(f"- Output tokens: {ctx.app_state.token_usage.output_tokens:,}")
        lines.append(f"- Total cost: ${ctx.app_state.cost_info.total_cost:.4f}")
        sessions_dir = os.path.expanduser("~/.openlaoke/sessions")
        os.makedirs(sessions_dir, exist_ok=True)
        if filename:
            path = os.path.expanduser(filename)
        else:
            path = os.path.join(sessions_dir, f"session_{ctx.app_state.session_id}.md")
        try:
            with open(path, "w", encoding="utf-8") as f:
                f.write("\n".join(lines))
            return CommandResult(message=f"Session exported to {path}")
        except Exception as e:
            return CommandResult(success=False, message=f"Export failed: {e}")
class UsageCommand(SlashCommand):
    name = "usage"
    description = "Show detailed usage statistics (tokens, cost, API calls)"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        usage = ctx.app_state.token_usage
        cost = ctx.app_state.cost_info
        lines = [
            "Usage Statistics",
            "================",
            "",
            "Token Usage:",
            f"  Input tokens:         {usage.input_tokens:,}",
            f"  Output tokens:        {usage.output_tokens:,}",
            f"  Cache read:          {usage.cache_read_tokens:,}",
            f"  Cache created:       {usage.cache_creation_tokens:,}",
            f"  Total tokens:        {usage.total_tokens:,}",
            "",
            "Cost Breakdown:",
            f"  Input cost:          ${cost.input_cost:.4f}",
            f"  Output cost:         ${cost.output_cost:.4f}",
            f"  Cache read cost:     ${cost.cache_read_cost:.4f}",
            f"  Cache creation cost: ${cost.cache_creation_cost:.4f}",
            f"  Total cost:          ${cost.total_cost:.4f}",
            "",
            "Session Info:",
            f"  Session ID:          {ctx.app_state.session_id}",
            f"  Model:                {ctx.app_state.session_config.model}",
            f"  Messages:             {len(ctx.app_state.messages):,}",
            f"  Tasks:                {len(ctx.app_state.tasks):,}",
        ]
        return CommandResult(message="\n".join(lines))
class MemoryCommand(SlashCommand):
    name = "memory"
    description = "Manage long-term memory storage"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip().lower()
        if not args or args == "list":
            return self._list_memory(ctx)
        elif args.startswith("add "):
            return self._add_memory(ctx, args[4:].strip())
        elif args.startswith("remove "):
            return self._remove_memory(ctx, args[7:].strip())
        elif args == "clear":
            return self._clear_memory(ctx)
        else:
            return CommandResult(
                success=False,
                message="Usage: /memory [list|add <text>|remove <id>|clear]",
            )
```

```
def _list_memory(self, ctx: CommandContext) -> CommandResult:
    memory_path = os.path.expanduser("~/.openlaoke/memory.json")
    lines = ["Memory Storage:", ""]
    if not os.path.exists(memory_path):
        lines.append("  No memories stored.")
        lines.append(f"  Memory file: {memory_path} (not found)")
        return CommandResult(message="\n".join(lines))
    try:
        import json
        with open(memory_path) as f:
            memories = json.load(f)
        if not memories:
            lines.append("  No memories stored.")
        else:
            for i, mem in enumerate(memories, 1):
                text = mem.get("text", "")[:50]
                timestamp = mem.get("timestamp", "unknown")
                lines.append(f"  [{i}] {text}... ({timestamp})")
    except Exception as e:
        lines.append(f"  Error reading memory: {e}")
    return CommandResult(message="\n".join(lines))
def _add_memory(self, ctx: CommandContext, text: str) -> CommandResult:
    import json
    from datetime import datetime
    memory_path = os.path.expanduser("~/.openlaoke/memory.json")
    try:
        os.makedirs(os.path.dirname(memory_path), exist_ok=True)
        memories = []
        if os.path.exists(memory_path):
            with open(memory_path) as f:
                memories = json.load(f)
        memories.append(
            {
                "text": text,
                "timestamp": datetime.now().isoformat(),
            }
        )
        with open(memory_path, "w") as f:
            json.dump(memories, f, indent=2)
        return CommandResult(message=f"Memory added: {text[:50]}...")
    except Exception as e:
        return CommandResult(success=False, message=f"Failed to add memory: {e}")
def _remove_memory(self, ctx: CommandContext, index_str: str) -> CommandResult:
    import json
    try:
        index = int(index_str) - 1
    except ValueError:
        return CommandResult(success=False, message="Invalid index. Use a number.")
    memory_path = os.path.expanduser("~/.openlaoke/memory.json")
    try:
        with open(memory_path) as f:
            memories = json.load(f)
        if index < 0 or index >= len(memories):
            return CommandResult(
                success=False, message=f"Index out of range. Valid: 1-{len(memories)}"
            )
        removed = memories.pop(index)
        with open(memory_path, "w") as f:
            json.dump(memories, f, indent=2)
        return CommandResult(message=f"Removed memory: {removed.get('text', '')[:50]}...")
    except Exception as e:
        return CommandResult(success=False, message=f"Failed to remove memory: {e}")
def _clear_memory(self, ctx: CommandContext) -> CommandResult:
    memory_path = os.path.expanduser("~/.openlaoke/memory.json")
    try:
```

```
            if os.path.exists(memory_path):
                os.remove(memory_path)
            return CommandResult(message="Memory cleared")
        except Exception as e:
            return CommandResult(success=False, message=f"Failed to clear memory: {e}")
class AgentsCommand(SlashCommand):
    name = "agents"
    description = "Show available agent types"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from openlaoke.types.core_types import TaskType
        lines = [
            "Available Agent Types:",
            "",
        ]
        agent_descriptions = {
            TaskType.LOCAL_BASH: "Execute bash commands locally",
            TaskType.LOCAL_AGENT: "Run sub-agents for complex tasks",
            TaskType.REMOTE_AGENT: "Connect to remote agent servers",
            TaskType.IN_PROCESS_TEAMMATE: "In-process teammate collaboration",
            TaskType.LOCAL_WORKFLOW: "Execute local workflows",
            TaskType.MONITOR_MCP: "Monitor MCP server connections",
            TaskType.DREAM: "Background processing tasks",
        }
        for task_type in TaskType:
            desc = agent_descriptions.get(task_type, "No description")
            lines.append(f"  {task_type.value}:")
            lines.append(f"    {desc}")
            lines.append("")
        active_tasks = ctx.app_state.get_active_tasks()
        if active_tasks:
            lines.append("Active Tasks:")
            for task in active_tasks:
                lines.append(f"  [{task.id}] {task.type.value}: {task.status.value}")
                if task.description:
                    lines.append(f"    {task.description}")
        else:
            lines.append("No active tasks.")
        return CommandResult(message="\n".join(lines))
class UndoCommand(SlashCommand):
    name = "undo"
    description = "Restore file to previous version"
    aliases = ["revert"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from openlaoke.utils.file_history import (
            format_history_list,
            get_history,
            restore_snapshot,
        )
        args = ctx.args.strip()
        if not args:
            return CommandResult(
                message="Usage: /undo <path> [version]\n"
                "  /undo <path>        - Restore to previous version\n"
                "  /undo <path> <n>    - Restore to version n\n"
                "  /undo <path> list   - Show all versions"
            )
        parts = args.split()
        file_path = parts[0]
        abs_path = os.path.abspath(file_path) if not os.path.isabs(file_path) else file_path
        cwd = ctx.app_state.get_cwd()
        if not os.path.isabs(file_path):
            abs_path = os.path.normpath(os.path.join(cwd, file_path))
        if len(parts) == 1:
            history = get_history(abs_path)
            if not history or not history.snapshots:
```

```
                return CommandResult(success=False, message=f"No history found for {abs_path}")
            if len(history.snapshots) < 2:
                return CommandResult(
                    success=False,
                    message=f"Only one version exists for {abs_path}. Use /history to view.",
                )
            prev_version = history.snapshots[-2].version
            if restore_snapshot(abs_path, prev_version):
                return CommandResult(message=f"Restored {abs_path} to version {prev_version}")
            return CommandResult(success=False, message=f"Failed to restore {abs_path}")
        version_str = parts[1]
        if version_str.lower() == "list":
            return CommandResult(message=format_history_list(abs_path))
        try:
            version = int(version_str)
        except ValueError:
            return CommandResult(
                success=False, message=f"Invalid version: {version_str}. Use a number or 'list'
    ."
            )
        if restore_snapshot(abs_path, version):
            return CommandResult(message=f"Restored {abs_path} to version {version}")
        return CommandResult(
            success=False, message=f"Failed to restore {abs_path} to version {version}"
        )
class HistoryCommand(SlashCommand):
    name = "history"
    description = "Show file modification history"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from openlaoke.utils.file_history import (
            clear_history,
        )
        args = ctx.args.strip()
        if not args:
            history_dir = os.path.expanduser("~/.openlaoke/file_history")
            if not os.path.exists(history_dir):
                return CommandResult(message="No file histories found.")
            files = []
            for filename in os.listdir(history_dir):
                if filename.endswith(".json"):
                    path = os.path.join(history_dir, filename)
                    try:
                        import json
                        with open(path, encoding="utf-8") as f:
                            data = json.load(f)
                        file_path = data.get("path", "unknown")
                        count = len(data.get("snapshots", []))
                        files.append((file_path, count))
                    except Exception:
                        continue
            if not files:
                return CommandResult(message="No file histories found.")
            lines = ["Files with history:", ""]
            for file_path, count in sorted(files, key=lambda x: x[0]):
                short_path = file_path.replace(os.path.expanduser("~"), "~")
                lines.append(f"  {short_path} ({count} versions)")
            lines.append("")
            lines.append("Use /history <path> to see details for a specific file.")
            return CommandResult(message="\n".join(lines))
        parts = args.split()
        file_path = parts[0]
        abs_path = os.path.abspath(file_path) if not os.path.isabs(file_path) else file_path
        cwd = ctx.app_state.get_cwd()
        if not os.path.isabs(file_path):
            abs_path = os.path.normpath(os.path.join(cwd, file_path))
```

```
        if len(parts) > 1 and parts[1].lower() == "clear":
            if clear_history(abs_path):
                return CommandResult(message=f"History cleared for {abs_path}")
            return CommandResult(success=False, message=f"Failed to clear history for {abs_path
    }")
        # Show history for specific file
        from openlaoke.utils.file_history import get_file_history
        file_history = get_file_history(abs_path)
        if not file_history:
            return CommandResult(message=f"No history found for {abs_path}")
        lines = [f"History for {abs_path}:", ""]
        for i, snapshot in enumerate(file_history.snapshots):
            lines.append(f"  Version {i + 1}: {snapshot.timestamp}")
        return CommandResult(message="\n".join(lines))
class AtomicCommand(SlashCommand):
    name = "atomic"
    description = "Generate code using atomic task decomposition (for small models)"
    aliases = ["atom", "decompose"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from pathlib import Path
        from rich.panel import Panel
        from openlaoke.core.atomic_generator_api import create_generator_with_api
        from openlaoke.core.intent_pipeline import create_pipeline_for_model
        from openlaoke.core.model_assessment.types import ModelTier
        request = ctx.args.strip()
        if not request:
            return CommandResult(
                message="Usage: /atomic <your request>\n"
                "Example: /atomic write a CPU benchmark program"
            )
        model_name = (
            ctx.app_state.session_config.model
            if hasattr(ctx.app_state, "session_config")
            else "unknown"
        )
        tier = ModelTier.TIER_5_LIMITED
        known_models = {
            "gemma3:1b": ModelTier.TIER_5_LIMITED,
            "qwen3.5:0.8b": ModelTier.TIER_5_LIMITED,
            "llama3.2:1b": ModelTier.TIER_5_LIMITED,
        }
        model_lower = model_name.lower()
        for known, known_tier in known_models.items():
            if known in model_lower:
                tier = known_tier
                break
        cwd = ctx.app_state.get_cwd() if hasattr(ctx.app_state, "get_cwd") else Path.cwd()
        pipeline = create_pipeline_for_model(tier, Path(cwd))
        result = pipeline.process_request(request)
        if not result.success:
            return CommandResult(
                success=False,
                message="[red]Failed to process request:[/red]\n" + "\n".join(result.errors),
            )
        console = ctx.app_state.console if hasattr(ctx.app_state, "console") else None
        lines = []
        lines.append(f"[bold green]Intent parsed:[/bold green] {result.intent.intent_type.value
    }")
        lines.append(f"[bold green]Task name:[/bold green] {result.intent.task_name}")
        lines.append(f"[bold green]Complexity:[/bold green] {result.intent.complexity.value}")
        plan = pipeline.get_execution_plan(result)
        lines.append("")
        lines.append("[bold cyan]Task Decomposition Plan:[/bold cyan]")
        lines.append(f"  Total tasks: {plan['total_tasks']}")
        lines.append(f"  Estimated total lines: {plan['estimated_lines_total']}")
```

```
        lines.append(f"  Ready to execute: {len(plan['ready_tasks'])}")
        if plan["ready_tasks"]:
            lines.append("")
            lines.append("[bold]Ready tasks:[/bold]")
            for task_info in plan["ready_tasks"][:5]:
                lines.append(f"  * {task_info['task_id']} ({task_info['estimated_lines']} lines
    )")
        generator = create_generator_with_api(tier, Path(cwd), ctx.app_state)
        completed_code = {}
        if console:
            console.print(Panel("\n".join(lines), title="Atomic Task Plan", border_style="cyan"
    ))
        if result.task_graph:
            graph = result.task_graph
            iteration = 0
            max_iterations = 20
            while len(graph.completed) < len(graph.tasks) and iteration < max_iterations:
                ready_tasks = graph.get_ready_tasks()
                if not ready_tasks:
                    break
                for task in ready_tasks[:1]:
                    if console:
                        console.print(f"\n[cyan]Processing task: {task.task_id}[/cyan]")
                        console.print(f"  Description: {task.description}")
                    code_result = await generator.generate_code_for_task_async(task, completed_
    code)
                    if code_result.success:
                        completed_code[task.task_id] = code_result.code
                        graph.mark_completed(task.task_id)
                        if console:
                            console.print(
                                f"  [green][OK] Generated {len(code_result.code)} characters[/g
    reen]"
                            )
                    else:
                        graph.mark_failed(task.task_id)
                        if console:
                            console.print(f"  [red][X] Failed: {', '.join(code_result.errors)}[
    /red]")
                iteration += 1
        final_code = generator.assemble_final_code(result.task_graph, completed_code)
        if final_code:
            output_file = Path(cwd) / f"{result.intent.task_name.replace(' ', '_')}.py"
            try:
                output_file.write_text(final_code)
                return CommandResult(
                    message=f"[green][OK] Generated code saved to:[/green] {output_file}\n"
                    f"[green]Total tasks completed:[/green] {len(graph.completed)}/{len(graph.t
    asks)}\n"
                    f"[green]Code size:[/green] {len(final_code)} bytes"
                )
            except Exception as e:
                return CommandResult(
                    success=False,
                    message=f"[red]Failed to save code:[/red] {e}\n\nGenerated code:\n{final_co
    de[:500]}...",
                )
        else:
            return CommandResult(
                success=False,
                message="[red]No code was generated[/red]",
            )
class DualModelCommand(SlashCommand):
    name = "dual"
    description = "Generate code using dual-model collaboration (small planner + large executor
```

```
)"
aliases = ["dual-model", "collab"]
async def execute(self, ctx: CommandContext) -> CommandResult:
    from pathlib import Path
    from rich.panel import Panel
    from openlaoke.core.dual_model_agent import create_dual_model_agent
    request = ctx.args.strip()
    if not request:
        return CommandResult(
            message="Usage: /dual <your request>\n"
            "Example: /dual write a CPU benchmark program\n\n"
            "This command uses:\n"
            "  - Small model (gemma3:1b) for planning and validation\n"
            "  - Large model (gemma4:e4b) for code generation\n"
            "  - Cost reduction: ~60-80%\n"
            "  - Quality improvement: Better decomposition + Better code"
        )
    console = ctx.app_state.console if hasattr(ctx.app_state, "console") else None
    if console:
        console.print(
            Panel(
                "[bold cyan]Dual-Model Agent[/bold cyan]\n\n"
                "Planner: gemma3:1b (cheap, fast)\n"
                "Executor: gemma4:e4b (capable, precise)\n\n"
                "Workflow:\n"
                "1. Planner analyzes and decomposes tasks\n"
                "2. Executor generates code for each task\n"
                "3. Validator checks code correctness\n"
                "4. Assembler combines final result",
                title="Starting Dual-Model Collaboration",
                border_style="cyan",
            )
        )
    cwd = ctx.app_state.get_cwd() if hasattr(ctx.app_state, "get_cwd") else Path.cwd()
    agent = create_dual_model_agent(ctx.app_state)
    result = await agent.execute(request, Path(cwd))
    if result.success:
        lines = []
        lines.append("[bold green][OK] Dual-model execution completed[/bold green]")
        lines.append("")
        lines.append("[bold]Model Preloading:[/bold]")
        lines.append(f"  Preloaded: {'[OK] Yes' if result.stats.models_preloaded else '[X]
No'}")
        lines.append(f"  Preload time: {result.stats.preload_time:.2f}s")
        lines.append("")
        lines.append("[bold]Statistics:[/bold]")
        lines.append(
            f"  Planner calls: {result.stats.planner_calls} ({result.stats.planner_tokens}
tokens)"
        )
        lines.append(
            f"  Executor calls: {result.stats.executor_calls} ({result.stats.executor_token
s} tokens)"
        )
        lines.append(
            f"  Validator calls: {result.stats.validator_calls} ({result.stats.validator_to
kens} tokens)"
        )
        lines.append(f"  Retry count: {result.stats.retry_count}")
        lines.append(f"  Total time: {result.stats.total_time:.2f}s")
        lines.append(f"  Estimated cost: ${result.stats.total_cost:.4f}")
        lines.append(f"  Code size: {len(result.code)} bytes")
        if result.output_file:
            lines.append(f"\n[green]Code saved to:[/green] {result.output_file}")
        if console:
```

```
                console.print(
                    Panel("\n".join(lines), title="Dual-Model Result", border_style="green")
                )
            return CommandResult(message="\n".join(lines))
        else:
            error_msg = "[red]Dual-model execution failed[/red]\n\n"
            if result.errors:
                error_msg += "[bold]Errors:[/bold]\n" + "\n".join(f"  - {e}" for e in result.er
    rors)
            if console:
                console.print(Panel(error_msg, title="Error", border_style="red"))
            return CommandResult(success=False, message=error_msg)
class PreloadModelsCommand(SlashCommand):
    name = "preload-models"
    description = "Preload models for faster execution"
    aliases = ["preload", "load-models"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from rich.panel import Panel
        from rich.table import Table
        from openlaoke.core.hybrid_model_manager import create_hybrid_manager
        console = ctx.app_state.console if hasattr(ctx.app_state, "console") else None
        if console:
            console.print(
                Panel(
                    "[bold cyan]Initializing Model Pool[/bold cyan]\n\n"
                    "Strategy: CPU/GPU Hybrid\n"
                    "- Planner (gemma3:1b) -> CPU (always loaded)\n"
                    "- Executor (gemma4:e4b) -> GPU (on-demand)\n"
                    "- Validator (gemma3:1b) -> CPU (always loaded)\n\n"
                    "This reduces VRAM usage by ~1.4 GB",
                    title="Model Preloader",
                    border_style="cyan",
                )
            )
        manager = create_hybrid_manager(ctx.app_state)
        result = await manager.initialize()
        if result["success"]:
            status = await manager.get_status()
            if console:
                table = Table(title="Model Pool Status", show_header=True)
                table.add_column("Model", style="cyan")
                table.add_column("Device", style="green")
                table.add_column("Status", style="yellow")
                table.add_column("Size", style="magenta")
                for model in status.cpu_models:
                    table.add_row(
                        model.name,
                        "CPU",
                        "[OK] Loaded" if model.is_loaded else "[X] Not loaded",
                        f"{model.size_gb:.1f} GB",
                    )
                for model in status.gpu_models:
                    table.add_row(
                        model.name,
                        "GPU",
                        "[OK] Loaded" if model.is_loaded else "[X] Not loaded",
                        f"{model.size_gb:.1f} GB",
                    )
                console.print(table)
                console.print(
                    f"\n[dim]GPU Memory: {status.gpu_used_gb:.1f} / {status.gpu_total_gb:.1f} G
    B[/dim]"
                )
            return CommandResult(message="Models preloaded successfully")
        else:
```

```
            failed = [k for k, v in result["results"].items() if not v]
            error_msg = f"[red]Failed to load models: {', '.join(failed)}[/red]"
            if console:
                console.print(Panel(error_msg, title="Error", border_style="red"))
            return CommandResult(success=False, message=error_msg)
class ModelStatusCommand(SlashCommand):
    name = "model-status"
    description = "Show current model pool status"
    aliases = ["models-status", "pool-status"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from rich.panel import Panel
        from rich.table import Table
        from openlaoke.core.hybrid_model_manager import create_hybrid_manager
        console = ctx.app_state.console if hasattr(ctx.app_state, "console") else None
        manager = create_hybrid_manager(ctx.app_state)
        status = await manager.get_status()
        if console:
            table = Table(title="Current Model Pool", show_header=True)
            table.add_column("Model", style="cyan")
            table.add_column("Tier", style="blue")
            table.add_column("Device", style="green")
            table.add_column("Status", style="yellow")
            table.add_column("Size", style="magenta")
            for model in status.cpu_models:
                table.add_row(
                    model.name,
                    model.tier,
                    "CPU",
                    "[OK] Loaded" if model.is_loaded else "[X] Not loaded",
                    f"{model.size_gb:.1f} GB",
                )
            for model in status.gpu_models:
                table.add_row(
                    model.name,
                    model.tier,
                    "GPU",
                    "[OK] Loaded" if model.is_loaded else "[X] Not loaded",
                    f"{model.size_gb:.1f} GB",
                )
            console.print(table)
            console.print(
                Panel(
                    f"GPU Memory Usage:\n"
                    f"  Total: {status.gpu_total_gb:.1f} GB\n"
                    f"  Used: {status.gpu_used_gb:.1f} GB\n"
                    f"  Free: {status.gpu_free_gb:.1f} GB",
                    title="Memory Status",
                    border_style="blue",
                )
            )
            return CommandResult(message="Model status displayed")
        else:
            return CommandResult(
                message=f"CPU models: {len(status.cpu_models)}, "
                f"GPU models: {len(status.gpu_models)}, "
                f"VRAM used: {status.gpu_used_gb:.1f} GB"
            )
class ModelRecommendCommand(SlashCommand):
    name = "model-recommend"
    description = "Get model recommendation based on your system"
    aliases = ["recommend-models", "optimal-models"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from rich.panel import Panel
        from rich.table import Table
        from openlaoke.core.intelligent_model_selector import IntelligentModelSelector
```

```
        console = ctx.app_state.console if hasattr(ctx.app_state, "console") else None
        preference = ctx.args.strip().lower() if ctx.args else "balanced"
        if preference not in ["cost", "quality", "balanced"]:
            preference = "balanced"
        selector = IntelligentModelSelector(ctx.app_state)
        report = await selector.get_recommendation_report()
        if console:
            console.print(
                Panel(
                    f"Detected GPU Memory: [bold cyan]{report['detected_vram_gb']:.1f} GB[/bold
    cyan]",
                    title="System Analysis",
                    border_style="blue",
                )
            )
            rec = report["recommended_combination"]
            console.print(
                Panel(
                    f"[bold green]Recommended: {rec['name'].upper()}[/bold green]\n\n"
                    f"Planner:   {rec['planner']}\n"
                    f"Executor:  {rec['executor']}\n"
                    f"Validator: {rec['validator']}\n\n"
                    f"Estimated VRAM: {report['estimated']['vram_usage_gb']:.1f} GB\n"
                    f"Quality Score:  {report['estimated']['quality_score']:.1f}/10\n"
                    f"Cost Factor:    {report['estimated']['cost_factor']:.1f}x",
                    title="Optimal Configuration",
                    border_style="green",
                )
            )
            if report["all_suitable_combinations"]:
                table = Table(title="All Suitable Combinations", show_header=True)
                table.add_column("Name", style="cyan")
                table.add_column("VRAM", style="magenta")
                table.add_column("Quality", style="green")
                table.add_column("Cost", style="yellow")
                for combo in report["all_suitable_combinations"]:
                    table.add_row(
                        combo["name"],
                        f"{combo['vram_gb']:.1f} GB",
                        f"{combo['quality']:.1f}/10",
                        f"{combo['cost']:.1f}x",
                    )
                console.print(table)
            console.print(f"\n[dim]Preference: {preference}[/dim]")
            return CommandResult(message="Recommendation displayed")
        else:
            rec = report["recommended_combination"]
            return CommandResult(
                message=f"Recommended: {rec['name']} - "
                f"Planner: {rec['planner']}, "
                f"Executor: {rec['executor']}, "
                f"VRAM: {report['estimated']['vram_usage_gb']:.1f} GB"
            )
class DualModelConfigCommand(SlashCommand):
    name = "dual-config"
    description = "Configure dual-model workflow (local/online models)"
    aliases = ["config-dual", "dual-setup"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        from rich.console import Console
        from rich.panel import Panel
        from rich.table import Table
        from openlaoke.core.dual_model_config import create_config_manager
        args = ctx.args.strip()
        console = None
        if hasattr(ctx.app_state, "console"):
```

```
        console = ctx.app_state.console
    elif hasattr(ctx, "console"):
        console = ctx.console
    else:
        console = Console()
    manager = create_config_manager(ctx.app_state)
    if not args:
        lines = []
        lines.append("\n[bold cyan]=====================================[/bold cyan]")
        lines.append("[bold cyan]    Dual-Model Configuration[/bold cyan]")
        lines.append("[bold cyan]=====================================[/bold cyan]\n")
        lines.append(
            f"[bold green][OK] Active Configuration:[/bold green] [bold]{manager.active_con
fig_name}[/bold]\n"
        )
        lines.append("[bold]Available Configurations:[/bold]\n")
        table = Table(show_header=True, header_style="bold magenta")
        table.add_column("Name", style="cyan", no_wrap=True)
        table.add_column("Planner", style="blue")
        table.add_column("Executor", style="green")
        table.add_column("Type", style="magenta")
        table.add_column("Status", style="yellow")
        for name, config in manager.configs.items():
            if config.planner and config.executor:
                active_marker = " ?" if name == manager.active_config_name else ""
                if (
                    config.planner.provider.value == "ollama"
                    and config.executor.provider.value == "ollama"
                ):
                    provider_type = "local"
                elif config.planner.provider.value != config.executor.provider.value:
                    provider_type = "hybrid"
                else:
                    provider_type = "online"
                status = "active" if name == manager.active_config_name else "ready"
                table.add_row(
                    f"{name}{active_marker}",
                    f"{config.planner.model_name}",
                    f"{config.executor.model_name}",
                    provider_type,
                    status,
                )
        console.print("\n".join(lines))
        console.print(table)
        console.print("\n")
        console.print(
            Panel(
                "[bold yellow]Commands:[/bold yellow]\n"
                "  [cyan]/dual-config list[/cyan]              - Show all configs\n"
                "  [cyan]/dual-config use <name>[/cyan]        - Select active config\n"
                "  [cyan]/dual-config check[/cyan]             - Check availability\n"
                "  [cyan]/dual-config create <name>[/cyan]     - Create custom config\n"
                "\n[bold green]Quick Examples:[/bold green]\n"
                "  [dim]/dual-config use local_balanced[/dim]      [italic]# Local models (
free)[/italic]\n"
                "  [dim]/dual-config use hybrid_openai[/dim]       [italic]# Local + OpenAI
[/italic]\n"
                "  [dim]/dual-config use online_premium[/dim]      [italic]# Best quality[/
italic]",
                title="[bold]Usage Guide[/bold]",
                border_style="yellow",
            )
        )
        return CommandResult(message="Configuration displayed")
    parts = args.split(maxsplit=1)
```

```
    action = parts[0].lower()
    if action == "list":
        configs = manager.list_configs()
        return CommandResult(
            message=f"Available configs: {', '.join(configs)}\n"
            f"Active: {manager.active_config_name}"
        )
    elif action == "use":
        if len(parts) < 2:
            return CommandResult(success=False, message="Usage: /dual-config use <config_na
me>")
        config_name = parts[1].strip()
        if manager.set_active_config(config_name):
            return CommandResult(
                message=f"[green][OK] Active config set to: {config_name}[/green]"
            )
        else:
            return CommandResult(
                success=False,
                message=f"[red]Config not found: {config_name}[/red]\n"
                f"Available: {', '.join(manager.list_configs())}",
            )
    elif action == "check":
        results = await manager.check_config_availability()
        if console:
            table = Table(title="Model Availability", show_header=True)
            table.add_column("Role", style="cyan")
            table.add_column("Model", style="blue")
            table.add_column("Status", style="green")
            table.add_column("Message", style="yellow")
            for role, (available, message) in results.items():
                config = manager.get_config()
                if config:
                    endpoint = getattr(config, role, None)
                    if endpoint:
                        table.add_row(
                            role,
                            f"{endpoint.model_name} ({endpoint.provider.value})",
                            "[OK] Available" if available else "[X] Unavailable",
                            message,
                        )
            console.print(table)
        return CommandResult(message="Availability checked")
    elif action == "create":
        if len(parts) < 2:
            return CommandResult(
                message="[bold]Create Custom Configuration[/bold]\n\n"
                "Usage: /dual-config create <name> <options>\n\n"
                "Options format:\n"
                "  planner=<provider>:<model>\n"
                "  executor=<provider>:<model>\n"
                "  validator=<provider>:<model>\n"
                "  planner-key=<api_key>\n"
                "  executor-key=<api_key>\n\n"
                "Examples:\n"
                "  /dual-config create my_config planner=ollama:gemma3:1b executor=openai:g
pt-4 planner-key=sk-xxx\n"
                "  /dual-config create local planner=ollama:gemma3:1b executor=ollama:gemma
4:e4b"
            )
        import shlex
        try:
            tokens = shlex.split(parts[1])
        except Exception:
            tokens = parts[1].split()
```

```
        name = tokens[0] if tokens else "custom"
        options = {}
        for token in tokens[1:]:
            if "=" in token:
                key, value = token.split("=", 1)
                options[key] = value
        planner_parts = options.get("planner", "ollama:gemma3:1b").split(":", 1)
        planner_provider = planner_parts[0]
        planner_model = planner_parts[1] if len(planner_parts) > 1 else "gemma3:1b"
        executor_parts = options.get("executor", "ollama:gemma4:e4b").split(":", 1)
        executor_provider = executor_parts[0]
        executor_model = executor_parts[1] if len(executor_parts) > 1 else "gemma4:e4b"
        validator_provider = None
        validator_model = None
        if "validator" in options:
            validator_parts = options["validator"].split(":", 1)
            validator_provider = validator_parts[0]
            validator_model = validator_parts[1] if len(validator_parts) > 1 else planner_m
odel
        config = manager.create_custom_config(
            name=name,
            planner_provider=planner_provider,
            planner_model=planner_model,
            executor_provider=executor_provider,
            executor_model=executor_model,
            validator_provider=validator_provider,
            validator_model=validator_model,
            planner_api_key=options.get("planner-key"),
            executor_api_key=options.get("executor-key"),
            validator_api_key=options.get("validator-key"),
        )
        manager.set_active_config(name)
        return CommandResult(
            message=f"[green][OK] Created and activated config: {name}[/green]\n"
            f"  Planner: {config.planner.model_name} ({config.planner.provider.value})\n"
            f"  Executor: {config.executor.model_name} ({config.executor.provider.value})\n
"
            f"  Validator: {config.validator.model_name} ({config.validator.provider.value}
)"
        )
    else:
        return CommandResult(
            success=False,
            message=f"Unknown action: {action}\nAvailable: list, use, check, create",
        )
```

File: [12/252] openlaoke/commands/hyperauto_command.py

```
"""HyperAuto command for autonomous operation modes."""
from __future__ import annotations
import asyncio
import json
import os
from datetime import datetime
from typing import TYPE_CHECKING, Any
from openlaoke.commands.base import CommandContext, CommandResult, SlashCommand
from openlaoke.types.core_types import HyperAutoMode
if TYPE_CHECKING:
    pass
class HyperAutoCommand(SlashCommand):
    """Manage HyperAuto autonomous operation modes."""
    name = "hyperauto"
    description = "Start autonomous operation mode"
    aliases = ["ha", "auto", "hyper"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip()
        # No args - show status
        if not args:
            return self._show_status(ctx)
        args_lower = args.lower()
        # Direct task description (most common usage)
        if args_lower.split()[0] not in [
            "start",
            "stop",
            "status",
            "progress",
            "config",
            "resume",
            "history",
            "learn",
            "clear",
            "on",
            "off",
            "enable",
            "disable",
        ]:
            return self._start_with_task(ctx, args)
        # Command-based operations
        if args_lower == "status":
            return self._show_status(ctx)
        elif args_lower in ["progress", "prog"]:
            return self._show_progress(ctx)
        elif args_lower in ["on", "enable"]:
            return self._enable_hyperauto(ctx)
        elif args_lower in ["off", "disable"]:
            return self._disable_hyperauto(ctx)
        elif args_lower == "start":
            return self._start_hyperauto(ctx, None)
        elif args_lower.startswith("start "):
            task_description = args[6:].strip()
            return self._start_with_task(ctx, task_description)
        elif args_lower == "stop":
            return self._stop_hyperauto(ctx)
        elif args_lower == "config":
            return self._show_config(ctx)
        elif args_lower.startswith("config "):
            return self._set_config(ctx, args[7:].strip())
        elif args_lower == "resume":
            return self._resume_task(ctx)
```

```
    elif args_lower == "history":
        return self._show_history(ctx)
    elif args_lower == "learn":
        return self._toggle_learning(ctx)
    elif args_lower == "clear":
        return self._clear_history(ctx)
    else:
        return self._show_help()
def _show_help(self) -> CommandResult:
    lines = [
        "[bold]HyperAuto - Autonomous Operation Mode[/bold]",
        "",
        "[cyan]Quick Start:[/cyan]",
        "  /hyperauto <task>              Start autonomous task immediately",
        "  /hyperauto start <task>        Start with task description",
        "",
        "[cyan]Monitoring:[/cyan]",
        "  /hyperauto                     Show current status",
        "  /hyperauto progress            Show detailed progress",
        "",
        "[cyan]Control:[/cyan]",
        "  /hyperauto on                  Enable HyperAuto mode",
        "  /hyperauto off                 Disable HyperAuto mode",
        "  /hyperauto stop                Stop current task",
        "  /hyperauto resume              Resume last task",
        "",
        "[cyan]Configuration:[/cyan]",
        "  /hyperauto config              Show configuration",
        "  /hyperauto config <key>=<value> Set config option",
        "  /hyperauto learn               Toggle learning mode",
        "",
        "[cyan]History:[/cyan]",
        "  /hyperauto history             Show task history",
        "  /hyperauto clear               Clear history",
        "",
        "[dim]Examples:[/dim]",
        "  [dim]/hyperauto Refactor the authentication module[/dim]",
        "  [dim]/hyperauto start Convert project to C language[/dim]",
        "  [dim]/hyperauto config max_iterations=200[/dim]",
    ]
    return CommandResult(message="\n".join(lines))
def _enable_hyperauto(self, ctx: CommandContext) -> CommandResult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    hyperauto_config.enabled = True
    self._save_config(ctx, hyperauto_config)
    lines = [
        "[green][OK] HyperAuto enabled[/green]",
        "",
        f"Mode: {hyperauto_config.mode.value}",
        f"Max iterations: {hyperauto_config.max_iterations}",
        f"Timeout: {hyperauto_config.timeout_seconds}s",
        "",
        "[dim]Start a task: /hyperauto <task_description>[/dim]",
    ]
    return CommandResult(message="\n".join(lines))
def _disable_hyperauto(self, ctx: CommandContext) -> CommandResult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    hyperauto_config.enabled = False
    self._save_config(ctx, hyperauto_config)
    return CommandResult(message="[yellow]HyperAuto disabled[/yellow]")
def _start_with_task(self, ctx: CommandContext, task_description: str) -> CommandResult:
    """Start HyperAuto with a specific task (auto-enable if needed)."""
    hyperauto_config = self._get_hyperauto_config(ctx)
    # Auto-enable if disabled
    if not hyperauto_config.enabled:
```

```
        hyperauto_config.enabled = True
        self._save_config(ctx, hyperauto_config)
    return self._start_hyperauto(ctx, task_description)
def _show_status(self, ctx: CommandContext) -> CommandResult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    status_icon = "[green]*[/green]" if hyperauto_config.enabled else "[red]o[/red]"
    status_text = "enabled" if hyperauto_config.enabled else "disabled"
    lines = [
        f"HyperAuto Status: {status_icon} {status_text}",
        "",
        f"  Mode:           {hyperauto_config.mode.value}",
        f"  Max Iterations: {hyperauto_config.max_iterations}",
        f"  Timeout:        {hyperauto_config.timeout_seconds}s",
        f"  Learning:       {'on' if hyperauto_config.learning_enabled else 'off'}",
        f"  Auto Save:      {'on' if hyperauto_config.auto_save else 'off'}",
        "",
    ]
    active_task = self._get_active_hyperauto_task(ctx)
    if active_task:
        lines.append("[bold]Active Task:[/bold]")
        lines.append(f"  ID:     {active_task.get('id', 'unknown')}")
        lines.append(f"  Status: {active_task.get('status', 'unknown')}")
        if active_task.get("description"):
            lines.append(f"  Task:   {active_task['description']}")
        lines.append(f"  Started: {active_task.get('start_time', 'unknown')}")
        lines.append(f"  Iterations: {active_task.get('iterations', 0)}")
    else:
        lines.append("[dim]No active task[/dim]")
        lines.append("")
        lines.append("[dim]Start a task: /hyperauto <task_description>[/dim]")
    return CommandResult(message="\n".join(lines))
def _show_progress(self, ctx: CommandContext) -> CommandResult:
    """Show detailed progress of active task."""
    active_task = self._get_active_hyperauto_task(ctx)
    if not active_task:
        return CommandResult(
            message="[yellow]No active HyperAuto task[/yellow]\n\n[dim]Start a task: /hyper
auto <task_description>[/dim]"
        )
    task_id = active_task.get("id", "unknown")
    status = active_task.get("status", "unknown")
    description = active_task.get("description", "No description")
    iterations = active_task.get("iterations", 0)
    max_iterations = self._get_hyperauto_config(ctx).max_iterations
    steps = active_task.get("steps", [])
    current_step = active_task.get("current_step", 0)
    completed_steps = active_task.get("completed_steps", [])
    # Calculate progress
    progress_pct = (iterations / max_iterations * 100) if max_iterations > 0 else 0
    duration = self._calculate_duration(active_task)
    # Progress bar
    bar_width = 30
    filled = int(progress_pct / 100 * bar_width)
    bar = "?" * filled + "?" * (bar_width - filled)
    lines = [
        "[bold]HyperAuto Task Progress[/bold]",
        "",
        f"  Task ID:  {task_id}",
        f"  Status:   {status}",
        f"  Duration: {duration}",
        "",
        f"  Task: {description[:60]}{'...' if len(description) > 60 else ''}",
        "",
        f"  Progress: [{bar}] {progress_pct:.1f}%",
        f"  Iterations: {iterations}/{max_iterations}",
```

```
        "",
    ]
    # Show steps if available
    if steps:
        lines.append("[bold]  Steps:[/bold]")
        for i, step in enumerate(steps[:10], 1):
            if i <= len(completed_steps):
                icon = "[green][OK][/green]"
            elif i == current_step:
                icon = "[yellow]*[/yellow]"
            else:
                icon = "[dim]o[/dim]"
            step_text = step[:50] if len(step) > 50 else step
            lines.append(f"    {icon} {i}. {step_text}")
        if len(steps) > 10:
            lines.append(f"    [dim]... and {len(steps) - 10} more steps[/dim]")
    # Show recent actions
    recent_actions = active_task.get("recent_actions", [])
    if recent_actions:
        lines.append("")
        lines.append("[bold]  Recent Actions:[/bold]")
        for action in recent_actions[-5:]:
            lines.append(f"    * {action[:70]}")
    # Show errors if any
    errors = active_task.get("errors", [])
    if errors:
        lines.append("")
        lines.append(f"[red]  Errors: {len(errors)}[/red]")
        for error in errors[-3:]:
            lines.append(f"    [red][X][/red] {error[:60]}")
    lines.extend(
        [
            "",
            "[dim]Commands:[/dim]",
            "  [dim]/hyperauto          - Show status[/dim]",
            "  [dim]/hyperauto progress - Show this progress view[/dim]",
            "  [dim]/hyperauto stop     - Stop the task[/dim]",
        ]
    )
    return CommandResult(message="\n".join(lines))
def _start_hyperauto(self, ctx: CommandContext, task_description: str | None) -> CommandRes
ult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    # Auto-enable if disabled
    if not hyperauto_config.enabled:
        hyperauto_config.enabled = True
        self._save_config(ctx, hyperauto_config)
    active_task = self._get_active_hyperauto_task(ctx)
    if active_task:
        return CommandResult(
            success=False,
            message=f"Task already running (ID: {active_task.get('id')}).\nUse /hyperauto s
top first.",
        )
    task_id = f"ha_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
    task = {
        "id": task_id,
        "status": "initializing",
        "start_time": datetime.now().isoformat(),
        "mode": hyperauto_config.mode.value,
        "iterations": 0,
        "description": task_description or "Autonomous operation",
        "steps": [],
        "current_step": 0,
        "completed_steps": [],
```

```
        "recent_actions": [],
    }
    self._save_active_task(ctx, task)
    self._add_to_history(ctx, task)
    # Start async execution
    import asyncio
    try:
        loop = asyncio.get_event_loop()
    except RuntimeError:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)
    asyncio.create_task(self._execute_hyperauto_task(ctx, task_id, task_description))
    lines = [
        "[green][OK] HyperAuto Started[/green]",
        "",
        f"  Task ID: {task_id}",
        f"  Mode:    {hyperauto_config.mode.value}",
    ]
    if task_description:
        desc = task_description[:60]
        if len(task_description) > 60:
            desc += "..."
        lines.append(f"  Task:    {desc}")
    lines.extend(
        [
            f"  Timeout: {hyperauto_config.timeout_seconds}s",
            "",
            "[dim]The AI is now working...[/dim]",
            "[dim]Use /hyperauto progress to monitor.[/dim]",
            "",
            "[bold cyan]Real-time output below:[/bold cyan]",
        ]
    )
    return CommandResult(message="\n".join(lines))
def _stop_hyperauto(self, ctx: CommandContext) -> CommandResult:
    active_task = self._get_active_hyperauto_task(ctx)
    if not active_task:
        return CommandResult(message="[yellow]No active HyperAuto task to stop.[/yellow]")
    task_id = active_task.get("id", "unknown")
    active_task["status"] = "stopped"
    active_task["end_time"] = datetime.now().isoformat()
    self._update_history_task(ctx, active_task)
    self._clear_active_task(ctx)
    lines = [
        "[yellow]HyperAuto Stopped[/yellow]",
        "",
        f"  Task ID:    {task_id}",
        f"  Iterations: {active_task.get('iterations', 0)}",
        f"  Duration:   {self._calculate_duration(active_task)}",
        "",
        "[dim]Task saved to history. Use /hyperauto resume to continue.[/dim]",
    ]
    return CommandResult(message="\n".join(lines))
def _show_config(self, ctx: CommandContext) -> CommandResult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    lines = [
        "[bold]HyperAuto Configuration[/bold]",
        "",
        f"  mode:             {hyperauto_config.mode.value}",
        f"  enabled:          {hyperauto_config.enabled}",
        f"  max_iterations:   {hyperauto_config.max_iterations}",
        f"  timeout_seconds:  {hyperauto_config.timeout_seconds}",
        f"  auto_save:        {hyperauto_config.auto_save}",
        f"  learning_enabled: {hyperauto_config.learning_enabled}",
        f"  history_limit:    {hyperauto_config.history_limit}",
```

```
        "",
        "[dim]Change: /hyperauto config <key>=<value>[/dim]",
        "[dim]Example: /hyperauto config max_iterations=200[/dim]",
        "",
        "[bold]Modes:[/bold]",
        "  semi_auto   - AI suggests, user confirms",
        "  full_auto   - AI operates with safety checks",
        "  hyper_auto  - Full autonomous operation",
    ]
    return CommandResult(message="\n".join(lines))
def _set_config(self, ctx: CommandContext, config_str: str) -> CommandResult:
    if "=" not in config_str:
        return CommandResult(
            success=False,
            message="Invalid format. Use: /hyperauto config <key>=<value>",
        )
    key, value = config_str.split("=", 1)
    key = key.strip()
    value = value.strip()
    hyperauto_config = self._get_hyperauto_config(ctx)
    valid_keys = {
        "mode",
        "enabled",
        "max_iterations",
        "timeout_seconds",
        "auto_save",
        "learning_enabled",
        "history_limit",
    }
    if key not in valid_keys:
        return CommandResult(
            success=False,
            message=f"Invalid key: {key}\nValid keys: {', '.join(sorted(valid_keys))}",
        )
    try:
        if key == "mode":
            hyperauto_config.mode = HyperAutoMode(value)
        elif key == "enabled":
            hyperauto_config.enabled = value.lower() in ("true", "1", "yes", "on")
        elif key == "max_iterations":
            hyperauto_config.max_iterations = int(value)
        elif key == "timeout_seconds":
            hyperauto_config.timeout_seconds = int(value)
        elif key == "auto_save":
            hyperauto_config.auto_save = value.lower() in ("true", "1", "yes", "on")
        elif key == "learning_enabled":
            hyperauto_config.learning_enabled = value.lower() in ("true", "1", "yes", "on")
        elif key == "history_limit":
            hyperauto_config.history_limit = int(value)
        self._save_config(ctx, hyperauto_config)
        return CommandResult(message=f"[green][OK][/green] Config updated: {key}={value}")
    except ValueError as e:
        return CommandResult(success=False, message=f"Invalid value: {e}")
def _resume_task(self, ctx: CommandContext) -> CommandResult:
    history = self._load_history(ctx)
    if not history:
        return CommandResult(message="[yellow]No HyperAuto history to resume.[/yellow]")
    last_task = history[-1]
    if last_task.get("status") == "running":
        return CommandResult(
            message="Last task is still running.\nUse /hyperauto status to check."
        )
    task_id = f"ha_{datetime.now().strftime('%Y%m%d_%H%M%S')}_resumed"
    resumed_task = {
        "id": task_id,
```

```
        "status": "running",
        "start_time": datetime.now().isoformat(),
        "mode": last_task.get("mode", "semi_auto"),
        "iterations": 0,
        "description": last_task.get("description", "Resumed task"),
        "resumed_from": last_task.get("id"),
    }
    self._save_active_task(ctx, resumed_task)
    self._add_to_history(ctx, resumed_task)
    lines = [
        "[green][OK] HyperAuto Resumed[/green]",
        "",
        f"  New Task ID:  {task_id}",
        f"  Resumed From: {last_task.get('id')}",
        f"  Mode:         {resumed_task['mode']}",
        "",
        "[dim]Previous context restored. Use /hyperauto stop to halt.[/dim]",
    ]
    return CommandResult(message="\n".join(lines))
def _show_history(self, ctx: CommandContext) -> CommandResult:
    history = self._load_history(ctx)
    lines = ["[bold]HyperAuto Task History[/bold]", ""]
    if not history:
        lines.append("[dim]No tasks in history.[/dim]")
        lines.append("[dim]Start a task: /hyperauto <task_description>[/dim]")
    else:
        lines.append(f"Total tasks: {len(history)}")
        lines.append("")
        for i, task in enumerate(history[-10:], 1):
            status = task.get("status", "unknown")
            status_icon = {
                "running": "[green]*[/green]",
                "stopped": "[yellow]*[/yellow]",
                "completed": "[green][OK][/green]",
                "failed": "[red][X][/red]",
            }.get(status, "o")
            lines.append(f"{status_icon} [{i}] {task.get('id', 'unknown')}")
            lines.append(f"    Status: {status}, Mode: {task.get('mode', 'unknown')}")
            if task.get("description"):
                desc = task["description"]
                if len(desc) > 60:
                    desc = desc[:57] + "..."
                lines.append(f"    Task: {desc}")
            lines.append(f"    Duration: {self._calculate_duration(task)}")
            lines.append("")
        if len(history) > 10:
            lines.append(f"[dim]... and {len(history) - 10} more tasks[/dim]")
            lines.append("[dim]Use /hyperauto clear to reset history[/dim]")
    return CommandResult(message="\n".join(lines))
def _toggle_learning(self, ctx: CommandContext) -> CommandResult:
    hyperauto_config = self._get_hyperauto_config(ctx)
    hyperauto_config.learning_enabled = not hyperauto_config.learning_enabled
    self._save_config(ctx, hyperauto_config)
    status = (
        "[green]enabled[/green]"
        if hyperauto_config.learning_enabled
        else "[yellow]disabled[/yellow]"
    )
    lines = [
        f"Learning mode {status}",
        "",
        "When enabled, HyperAuto will:",
        "  * Learn from successful operations",
        "  * Adapt strategies based on patterns",
        "  * Store insights for future tasks",
```

```
    ]
    return CommandResult(message="\n".join(lines))
def _clear_history(self, ctx: CommandContext) -> CommandResult:
    history_path = self._get_history_path(ctx)
    try:
        if os.path.exists(history_path):
            os.remove(history_path)
        return CommandResult(message="[green][OK][/green] HyperAuto history cleared.")
    except Exception as e:
        return CommandResult(success=False, message=f"Failed to clear history: {e}")
def _get_hyperauto_config(self, ctx: CommandContext):
    if hasattr(ctx.app_state, "permission_config") and hasattr(
        ctx.app_state.permission_config, "hyperauto_config"
    ):
        return ctx.app_state.permission_config.hyperauto_config
    elif hasattr(ctx.app_state, "app_config") and ctx.app_state.app_config:
        return ctx.app_state.app_config.hyperauto_config
    else:
        from openlaoke.types.permissions import HyperAutoConfig
        return HyperAutoConfig()
def _save_config(self, ctx: CommandContext, config) -> None:
    if hasattr(ctx.app_state, "permission_config") and hasattr(
        ctx.app_state.permission_config, "hyperauto_config"
    ):
        ctx.app_state.permission_config.hyperauto_config = config
    elif hasattr(ctx.app_state, "app_config") and ctx.app_state.app_config:
        ctx.app_state.app_config.hyperauto_config = config
    config_path = os.path.expanduser("~/.openlaoke/config.json")
    try:
        os.makedirs(os.path.dirname(config_path), exist_ok=True)
        existing_config = {}
        if os.path.exists(config_path):
            with open(config_path) as f:
                existing_config = json.load(f)
        existing_config["hyperauto_config"] = {
            "mode": config.mode.value,
            "enabled": config.enabled,
            "max_iterations": config.max_iterations,
            "timeout_seconds": config.timeout_seconds,
            "auto_save": config.auto_save,
            "learning_enabled": config.learning_enabled,
            "history_limit": config.history_limit,
        }
        with open(config_path, "w") as f:
            json.dump(existing_config, f, indent=2)
    except Exception:
        pass
def _get_history_path(self, ctx: CommandContext) -> str:
    return os.path.expanduser("~/.openlaoke/hyperauto_history.json")
def _get_active_task_path(self, ctx: CommandContext) -> str:
    return os.path.expanduser("~/.openlaoke/hyperauto_active.json")
def _load_history(self, ctx: CommandContext) -> list[dict[str, Any]]:
    history_path = self._get_history_path(ctx)
    try:
        if os.path.exists(history_path):
            with open(history_path) as f:
                data = json.load(f)
                return list(data) if isinstance(data, list) else []
    except Exception:
        pass
    return []
def _save_history(self, ctx: CommandContext, history: list) -> None:
    history_path = self._get_history_path(ctx)
    hyperauto_config = self._get_hyperauto_config(ctx)
    if len(history) > hyperauto_config.history_limit:
```

```
        history = history[-hyperauto_config.history_limit :]
    try:
        os.makedirs(os.path.dirname(history_path), exist_ok=True)
        with open(history_path, "w") as f:
            json.dump(history, f, indent=2)
    except Exception:
        pass
def _add_to_history(self, ctx: CommandContext, task: dict) -> None:
    history = self._load_history(ctx)
    history.append(task)
    self._save_history(ctx, history)
def _update_history_task(self, ctx: CommandContext, task: dict) -> None:
    history = self._load_history(ctx)
    for i, h_task in enumerate(history):
        if h_task.get("id") == task.get("id"):
            history[i] = task
            break
    self._save_history(ctx, history)
def _get_active_hyperauto_task(self, ctx: CommandContext) -> dict[str, Any] | None:
    active_path = self._get_active_task_path(ctx)
    try:
        if os.path.exists(active_path):
            with open(active_path) as f:
                data = json.load(f)
                return dict(data) if isinstance(data, dict) else None
    except Exception:
        pass
    return None
def _save_active_task(self, ctx: CommandContext, task: dict) -> None:
    active_path = self._get_active_task_path(ctx)
    try:
        os.makedirs(os.path.dirname(active_path), exist_ok=True)
        with open(active_path, "w") as f:
            json.dump(task, f, indent=2)
    except Exception:
        pass
def _clear_active_task(self, ctx: CommandContext) -> None:
    active_path = self._get_active_task_path(ctx)
    try:
        if os.path.exists(active_path):
            os.remove(active_path)
    except Exception:
        pass
def _calculate_duration(self, task: dict) -> str:
    start_time = task.get("start_time")
    end_time = task.get("end_time")
    if not start_time:
        return "unknown"
    try:
        start = datetime.fromisoformat(start_time)
        end = datetime.fromisoformat(end_time) if end_time else datetime.now()
        duration = end - start
        seconds = int(duration.total_seconds())
        minutes = seconds // 60
        secs = seconds % 60
        if minutes > 0:
            return f"{minutes}m {secs}s"
        return f"{secs}s"
    except Exception:
        return "unknown"
async def _execute_hyperauto_task(
    self, ctx: CommandContext, task_id: str, task_description: str | None
) -> None:
    """Execute HyperAuto task in background and display output."""
    import time
```

```
    from rich.console import Console, Group
    from rich.live import Live
    from rich.panel import Panel
    from rich.progress import BarColumn, Progress, SpinnerColumn, TextColumn, TimeElapsedCo
lumn
    from rich.table import Table
    console = Console()
    progress_state: dict[str, Any] = {
        "iteration": 0,
        "max_iterations": 100,
        "status": "Initializing",
        "tokens_input": 0,
        "tokens_output": 0,
        "cost": 0.0,
        "model_active": False,
        "sub_tasks": [],
        "current_task": "",
        "state": "idle",
        "start_time": 0.0,
        "live_display": None,
    }
    def create_progress_display() -> Group:
        progress = Progress(
            SpinnerColumn(),
            TextColumn("[bold blue]{task.description}[/bold blue]"),
            BarColumn(complete_style="green", finished_style="green"),
            TextColumn("[progress.percentage]{task.percentage:>3.0f}%"),
            TimeElapsedColumn(),
        )
        max_iter: int = progress_state.get("max_iterations", 100)
        iter_count: int = progress_state.get("iteration", 0)
        task_progress = progress.add_task(
            "[cyan]Iteration Progress[/cyan]",
            total=max_iter,
        )
        progress.update(task_progress, completed=iter_count)
        status_table = Table(show_header=False, box=None, padding=(0, 2))
        status_table.add_column("key", style="bold cyan")
        status_table.add_column("value")
        start_time: float = progress_state.get("start_time", 0.0)
        elapsed = time.time() - start_time if start_time > 0 else 0.0
        elapsed_str = (
            f"{int(elapsed)}s"
            if elapsed < 60
            else f"{int(elapsed // 60)}m {int(elapsed % 60)}s"
        )
        state_val: str = progress_state.get("state", "idle")
        status_table.add_row("Status", f"[bold]{state_val.upper()}[/bold]")
        status_table.add_row("Iteration", f"{iter_count}/{max_iter}")
        status_table.add_row("Elapsed", elapsed_str)
        current_task_val: str = progress_state.get("current_task", "N/A")
        status_table.add_row("Current", current_task_val[:40])
        model_active: bool = progress_state.get("model_active", False)
        model_status = "[green]ACTIVE[/green]" if model_active else "[dim]IDLE[/dim]"
        status_table.add_row("Model", model_status)
        token_table = Table(show_header=False, box=None, padding=(0, 2))
        token_table.add_column("key", style="bold yellow")
        token_table.add_column("value")
        tokens_input: int = progress_state.get("tokens_input", 0)
        tokens_output: int = progress_state.get("tokens_output", 0)
        cost: float = progress_state.get("cost", 0.0)
        token_table.add_row("Input Tokens", f"{tokens_input:,}")
        token_table.add_row("Output Tokens", f"{tokens_output:,}")
        token_table.add_row("Cost", f"${cost:.4f}")
        sub_tasks: list[dict[str, Any]] = progress_state.get("sub_tasks", [])
```

```
        if sub_tasks:
            task_table = Table(show_header=True, box=None, padding=(0, 1))
            task_table.add_column("Task", style="cyan")
            task_table.add_column("Status", justify="center")
            for st in sub_tasks[:6]:
                st_status: str = st.get("status", "pending")
                status_icon = {
                    "completed": "[green][OK][/green]",
                    "running": "[yellow]*[/yellow]",
                    "pending": "[dim]o[/dim]",
                    "failed": "[red][X][/red]",
                }.get(st_status, "[dim]o[/dim]")
                st_name: str = st.get("name", "unknown")
                task_table.add_row(st_name[:30], status_icon)
            return Group(
                Panel(progress, title="[bold]HyperAuto Progress[/bold]", border_style="cyan
"),
                Panel(status_table, title="[bold]Status[/bold]", border_style="blue"),
                Panel(token_table, title="[bold]Token Usage[/bold]", border_style="yellow")
,
                Panel(task_table, title="[bold]Sub Tasks[/bold]", border_style="green"),
            )
        return Group(
            Panel(progress, title="[bold]HyperAuto Progress[/bold]", border_style="cyan"),
            Panel(status_table, title="[bold]Status[/bold]", border_style="blue"),
            Panel(token_table, title="[bold]Token Usage[/bold]", border_style="yellow"),
        )
    def on_progress(context) -> None:
        progress_state["iteration"] = context.iteration
        progress_state["max_iterations"] = self._get_hyperauto_config(ctx).max_iterations
        progress_state["state"] = context.current_state.value
        progress_state["sub_tasks"] = [t.to_dict() for t in context.sub_tasks]
        progress_state["tokens_input"] = context.total_tokens
        progress_state["tokens_output"] = context.total_tokens
        progress_state["cost"] = context.total_cost
        running_tasks = [t for t in context.sub_tasks if t.status.value == "running"]
        if running_tasks:
            progress_state["current_task"] = running_tasks[0].name
            progress_state["model_active"] = True
        else:
            progress_state["current_task"] = context.current_state.value
            progress_state["model_active"] = False
        task = self._get_active_hyperauto_task(ctx)
        if task:
            task["iterations"] = context.iteration
            task["status"] = context.current_state.value
            task["current_step"] = len(
                [t for t in context.sub_tasks if t.status.value == "completed"]
            )
            self._save_active_task(ctx, task)
        if progress_state.get("live_display"):
            progress_state["live_display"].update(create_progress_display())
    try:
        task = self._get_active_hyperauto_task(ctx)
        if task:
            task["status"] = "running"
            task["steps"] = [
                "Analyzing task requirements",
                "Creating execution plan",
                "Executing subtasks",
                "Verifying results",
                "Finalizing",
            ]
            task["recent_actions"] = ["Starting execution..."]
            self._save_active_task(ctx, task)
```

```
        from openlaoke.core.hyperauto.agent import HyperAutoAgent
        from openlaoke.core.hyperauto.config import HyperAutoConfig as HAutoConfig
        hyperauto_config = self._get_hyperauto_config(ctx)
        ha_config = HAutoConfig(
            mode=hyperauto_config.mode,
            max_iterations=hyperauto_config.max_iterations,
            timeout_per_task=hyperauto_config.timeout_seconds,
            learning_enabled=hyperauto_config.learning_enabled,
            reflection_enabled=True,
        )
        progress_state["max_iterations"] = hyperauto_config.max_iterations
        progress_state["start_time"] = time.time()
        agent = HyperAutoAgent(ctx.app_state, ha_config, on_progress=on_progress)
        console.print()
        with Live(
            create_progress_display(),
            console=console,
            refresh_per_second=4,
            transient=False,
        ) as live:
            progress_state["live_display"] = live
            run_task = asyncio.create_task(
                agent.run(task_description or "Autonomous operation")
            )
            while not run_task.done():
                live.update(create_progress_display())
                await asyncio.sleep(0.25)
            result = run_task.result()
            progress_state["live_display"] = None
        task = self._get_active_hyperauto_task(ctx)
        if task:
            if result.get("success"):
                task["status"] = "completed"
                task["completed_steps"] = task.get("steps", [])
                task["iterations"] = result.get("context", {}).get("iteration", 0)
                console.print("\n[bold green][OK] HyperAuto Completed Successfully![/bold g
reen]")
                console.print(f"[green]Task ID: {task_id}[/green]")
                console.print(f"[green]Iterations: {task['iterations']}[/green]")
                console.print(f"[green]Duration: {self._calculate_duration(task)}[/green]\n
")
            else:
                task["status"] = "failed"
                error_msg = result.get("error", "Unknown error")
                if "errors" not in task:
                    task["errors"] = []
                task["errors"].append(error_msg)
                console.print("\n[bold red][X] HyperAuto Failed[/bold red]")
                console.print(f"[red]Error: {error_msg}[/red]\n")
            task["end_time"] = datetime.now().isoformat()
            self._update_history_task(ctx, task)
    except Exception as e:
        console.print(f"\n[bold red][X] HyperAuto Error: {e}[/bold red]\n")
        task = self._get_active_hyperauto_task(ctx)
        if task:
            task["status"] = "failed"
            task["end_time"] = datetime.now().isoformat()
            if "errors" not in task:
                task["errors"] = []
            task["errors"].append(str(e))
            self._update_history_task(ctx, task)
```

File: [13/252] openlaoke/commands/registry.py

```
"""Command registry."""
from __future__ import annotations
from openlaoke.commands.base import (
    AgentsCommand,
    AtomicCommand,
    ClearCommand,
    CommandsCommand,
    CompactCommand,
    CostCommand,
    CwdCommand,
    DoctorCommand,
    DualModelCommand,
    DualModelConfigCommand,
    ExitCommand,
    ExportCommand,
    HelpCommand,
    HistoryCommand,
    HooksCommand,
    InitCommand,
    McpCommand,
    MemoryCommand,
    ModelCommand,
    ModelRecommendCommand,
    ModelStatusCommand,
    PermissionCommand,
    PreloadModelsCommand,
    ProviderCommand,
    ResumeCommand,
    SettingsCommand,
    SlashCommand,
    ThemeCommand,
    UndoCommand,
    UsageCommand,
    VimCommand,
)
from openlaoke.commands.hyperauto_command import HyperAutoCommand
from openlaoke.commands.scaffold_commands import (
    KnowledgeDownloadCommand,
    QuickStartCommand,
    ScaffoldCommand,
)
from openlaoke.commands.skill_commands import SkillCommand, UseSkillCommand
from openlaoke.commands.skill_shortcuts import register_skill_shortcuts
_commands: dict[str, SlashCommand] = {}
def register_all() -> None:
    """Register all built-in slash commands."""
    commands = [
        AgentsCommand(),
        AtomicCommand(),
        ClearCommand(),
        CommandsCommand(),
        CompactCommand(),
        CostCommand(),
        CwdCommand(),
        DoctorCommand(),
        DualModelCommand(),
        DualModelConfigCommand(),
        ExitCommand(),
        ExportCommand(),
        HelpCommand(),
        HistoryCommand(),
```

```
        HooksCommand(),
        HyperAutoCommand(),
        InitCommand(),
        McpCommand(),
        MemoryCommand(),
        ModelCommand(),
        ModelRecommendCommand(),
        ModelStatusCommand(),
        PermissionCommand(),
        PreloadModelsCommand(),
        ProviderCommand(),
        ResumeCommand(),
        ScaffoldCommand(),
        KnowledgeDownloadCommand(),
        QuickStartCommand(),
        SettingsCommand(),
        ThemeCommand(),
        UndoCommand(),
        UsageCommand(),
        VimCommand(),
        SkillCommand(),
        UseSkillCommand(),
    ]
    for cmd in commands:
        _commands[cmd.name] = cmd
        for alias in cmd.aliases:
            _commands[alias] = cmd
    # Register skill shortcuts like /browse, /qa, etc.
    register_skill_shortcuts(_commands)
def get_command(name: str) -> SlashCommand | None:
    return _commands.get(name)
def get_all_commands() -> list[SlashCommand]:
    seen = set()
    result = []
    for cmd in _commands.values():
        if cmd.name not in seen:
            seen.add(cmd.name)
            result.append(cmd)
    return result
def parse_command(text: str) -> tuple[str, str] | None:
    """Parse a slash command from user input. Returns (name, args) or None."""
    text = text.strip()
    if not text.startswith("/"):
        return None
    parts = text[1:].split(" ", 1)
    name = parts[0].lower()
    args = parts[1] if len(parts) > 1 else ""
    return name, args
```

File: [14/252] openlaoke/commands/scaffold_commands.py

```
"""Project scaffolding command for quick project setup."""
from __future__ import annotations
from pathlib import Path
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
from openlaoke.commands.base import CommandContext, CommandResult, SlashCommand
from openlaoke.core.code_templates import (
    PROJECT_TEMPLATES,
    ProjectType,
    get_template,
    suggest_template,
)
class ScaffoldCommand(SlashCommand):
    name = "scaffold"
    description = "Generate project scaffold from templates"
    aliases = ["new", "create", "init"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        """Execute scaffold command."""
        console = Console()
        args = ctx.args.strip()
        if not args or args in ["list", "ls"]:
            return self._list_templates(console)
        if args.startswith("suggest "):
            task = args[8:].strip()
            return self._suggest_templates(task, console)
        return self._create_project(args, console, ctx)
    def _list_templates(self, console: Console) -> CommandResult:
        """List all available templates."""
        table = Table(title="Available Project Templates")
        table.add_column("ID", style="cyan")
        table.add_column("Name", style="green")
        table.add_column("Type", style="yellow")
        table.add_column("Language", style="magenta")
        table.add_column("Description")
        for template_id, template in PROJECT_TEMPLATES.items():
            table.add_row(
                template_id,
                template.name,
                template.project_type.value,
                template.language.value,
                template.description[:50] + "..."
                if len(template.description) > 50
                else template.description,
            )
        console.print(table)
        return CommandResult(
            message="\n[dim]Usage: /scaffold <template_id> [project_name][/dim]\n"
            "[dim]Example: /scaffold python_cli_tool my_tool[/dim]"
        )
    def _suggest_templates(self, task: str, console: Console) -> CommandResult:
        """Suggest templates based on task description."""
        suggestions = suggest_template(task)
        if not suggestions:
            return CommandResult(
                message="[yellow]No matching templates found for this task.[/yellow]\n"
                "[dim]Use /scaffold list to see all available templates.[/dim]"
            )
        console.print(f"[green]Suggested templates for: {task}[/green]\n")
        for template_id in suggestions:
            template = get_template(template_id)
```

```
        if template:
            console.print(
                Panel(
                    f"[bold]{template.name}[/bold]\n"
                    f"[dim]{template.description}[/dim]\n\n"
                    f"[cyan]Type:[/cyan] {template.project_type.value}\n"
                    f"[cyan]Language:[/cyan] {template.language.value}\n"
                    f"[cyan]Dependencies:[/cyan] {', '.join(template.dependencies[:3])}\n\n
"
                    f"[yellow]Usage: /scaffold {template_id}[/yellow]",
                    title=template_id,
                    border_style="green",
                )
            )
    return CommandResult(message="")
def _create_project(self, args: str, console: Console, ctx: CommandContext) -> CommandResul
t:
    """Create a new project from template."""
    parts = args.split(maxsplit=1)
    template_id = parts[0]
    project_name = parts[1] if len(parts) > 1 else None
    template = get_template(template_id)
    if not template:
        return CommandResult(
            success=False,
            message=f"[red]Template not found: {template_id}[/red]\n"
            f"[dim]Use /scaffold list to see available templates.[/dim]",
        )
    if not project_name:
        project_name = f"my_{template.project_type.value}"
    project_dir = Path(ctx.app_state.cwd) / project_name
    if project_dir.exists():
        return CommandResult(
            success=False,
            message=f"[red]Directory already exists: {project_dir}[/red]\n"
            f"[dim]Choose a different project name or remove the existing directory.[/dim]"
,
        )
    console.print(f"\n[green]Creating project: {project_name}[/green]")
    console.print(f"[green]Template: {template.name}[/green]")
    console.print(f"[green]Location: {project_dir}[/green]\n")
    project_dir.mkdir(parents=True, exist_ok=True)
    for file_name, content in template.files.items():
        file_path = project_dir / file_name
        file_path.parent.mkdir(parents=True, exist_ok=True)
        file_path.write_text(content)
        console.print(f"  [cyan]Created:[/cyan] {file_name}")
    if template.readme:
        readme_path = project_dir / "README.md"
        readme_path.write_text(template.readme)
        console.print("  [cyan]Created:[/cyan] README.md")
    if template.dependencies:
        requirements_path = project_dir / "requirements.txt"
        requirements_path.write_text("\n".join(template.dependencies))
        console.print("  [cyan]Created:[/cyan] requirements.txt")
    console.print("\n[bold green][OK] Project created successfully![/bold green]")
    console.print("\n[bold]Next steps:[/bold]")
    console.print(f"  1. cd {project_name}")
    if template.dependencies:
        console.print(
            f"  2. {template.setup_commands[0] if template.setup_commands else 'pip install
 -r requirements.txt'}"
        )
    if template.project_type == ProjectType.WEB_API:
        console.print("  3. uvicorn main:app --reload")
```

```
        elif template.project_type == ProjectType.CLI_TOOL:
            console.print("  3. python main.py --help")
        elif template.project_type == ProjectType.DATA_PROCESSING:
            console.print("  3. python processor.py --help")
        else:
            console.print("  3. python main.py")
        console.print(f"\n[dim]Total files: {len(template.files)}[/dim]")
        console.print(f"[dim]Dependencies: {len(template.dependencies)} packages[/dim]")
        return CommandResult(message=f"\n[dim]Project ready at: {project_dir.absolute()}[/dim]"
    )
class KnowledgeDownloadCommand(SlashCommand):
    name = "download-knowledge"
    description = "Download documentation for knowledge base"
    aliases = ["dl-docs", "fetch-docs"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        """Execute knowledge download command."""
        from openlaoke.core.enhanced_knowledge_base import EnhancedKnowledgeBase
        console = Console()
        args = ctx.args.strip()
        if not args:
            return CommandResult(
                message="[yellow]Usage: /download-knowledge <source_or_task>[/yellow]\n"
                "[dim]Examples:[/dim]\n"
                "  [dim]/download-knowledge python[/dim]\n"
                "  [dim]/download-knowledge numpy[/dim]\n"
                "  [dim]/download-knowledge 'web api development'[/dim]\n"
                "  [dim]/download-knowledge list[/dim]"
            )
        if args == "list":
            from openlaoke.core.doc_sources import DOC_SOURCES
            table = Table(title="Available Documentation Sources")
            table.add_column("ID", style="cyan")
            table.add_column("Name", style="green")
            table.add_column("Category")
            table.add_column("URLs")
            for source_id, source_info in list(DOC_SOURCES.items())[:20]:
                table.add_row(
                    source_id,
                    source_info["name"],
                    source_info.get("category", "general"),
                    str(len(source_info["urls"])),
                )
            console.print(table)
            return CommandResult(
                message=f"\n[dim]Total: {len(DOC_SOURCES)} sources available[/dim]"
            )
        kb = EnhancedKnowledgeBase()
        console.print(f"[cyan]Downloading documentation for: {args}[/cyan]\n")
        try:
            num_chunks = kb.download_for_task(args)
            return CommandResult(
                message=f"\n[green][OK] Downloaded {num_chunks} knowledge chunks[/green]\n"
                f"[dim]Total knowledge base size: {len(kb.downloaded_chunks)} chunks[/dim]"
            )
        except Exception as e:
            return CommandResult(
                success=False, message=f"[red]Failed to download documentation: {e}[/red]"
            )
class QuickStartCommand(SlashCommand):
    name = "quickstart"
    description = "Quick start guide for common development tasks"
    aliases = ["qs", "guide"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        """Execute quickstart command."""
        console = Console()
```

```
    args = ctx.args.strip().lower()
    guides = {
        "cli": {
            "title": "CLI Tool Development",
            "steps": [
                "1. Create project: /scaffold python_cli_tool my_tool",
                "2. Install dependencies: cd my_tool && pip install click rich",
                "3. Edit main.py to add your functionality",
                "4. Test: python main.py --help",
                "5. Install globally: pip install -e .",
            ],
            "knowledge": ["click", "argparse", "pathlib", "logging"],
        },
        "api": {
            "title": "Web API Development",
            "steps": [
                "1. Create project: /scaffold python_web_api my_api",
                "2. Install dependencies: cd my_api && pip install fastapi uvicorn",
                "3. Define models in models.py",
                "4. Add endpoints in main.py",
                "5. Run: uvicorn main:app --reload",
            ],
            "knowledge": ["fastapi", "flask", "pydantic", "requests"],
        },
        "data": {
            "title": "Data Processing",
            "steps": [
                "1. Create project: /scaffold python_data_processing my_pipeline",
                "2. Install dependencies: pip install pandas numpy",
                "3. Configure data sources in processor.py",
                "4. Add custom processing logic",
                "5. Run: python processor.py data.csv",
            ],
            "knowledge": ["pandas", "numpy", "multiprocessing", "pathlib"],
        },
        "test": {
            "title": "Testing",
            "steps": [
                "1. Install pytest: pip install pytest pytest-cov",
                "2. Create tests/ directory",
                "3. Write test_*.py files",
                "4. Use fixtures for common setup",
                "5. Run: pytest tests/ -v",
            ],
            "knowledge": ["pytest", "unittest", "mock"],
        },
    }
    if not args or args not in guides:
        console.print("[bold]Available Quick Start Guides:[/bold]\n")
        for guide_id, guide in guides.items():
            console.print(f"  [cyan]/quickstart {guide_id}[/cyan] - {guide['title']}")
        return CommandResult(
            message="\n[dim]Use /quickstart <topic> to see detailed guide[/dim]"
        )
    guide = guides[args]
    console.print(
        Panel(
            "\n".join(guide["steps"]),
            title=f"[bold]{guide['title']}[/bold]",
            border_style="green",
        )
    )
    console.print("\n[bold]Recommended knowledge:[/bold]")
    for knowledge_id in guide["knowledge"]:
        console.print(f"  [cyan]/download-knowledge {knowledge_id}[/cyan]")
```

```
    return CommandResult(message="")
```

File: [15/252] openlaoke/commands/skill_commands.py

```
"""Skill command for managing and installing skills."""
from __future__ import annotations
from openlaoke.commands.base import CommandContext, CommandResult, SlashCommand
from openlaoke.core.skill_installer import SkillInstaller
from openlaoke.core.skill_system import (
    get_default_skill_dirs,
    get_skill_registry,
    list_available_skills,
    load_skill,
    rescan_skills,
)
_installer: SkillInstaller | None = None
def get_installer() -> SkillInstaller:
    global _installer
    if _installer is None:
        _installer = SkillInstaller()
    return _installer
class SkillCommand(SlashCommand):
    """Command to manage and install skills."""
    name = "skill"
    description = "Manage skills: list, install, remove, info"
    aliases = ["skills"]
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip()
        if not args:
            return await self._list_skills(ctx)
        parts = args.split(None, 1)
        subcmd = parts[0].lower()
        subargs = parts[1] if len(parts) > 1 else ""
        if subcmd in ("list", "ls"):
            return await self._list_skills(ctx)
        elif subcmd in ("install", "add", "i"):
            return await self._install_skill(ctx, subargs)
        elif subcmd in ("remove", "rm", "uninstall"):
            return await self._remove_skill(ctx, subargs)
        elif subcmd in ("info", "show"):
            return await self._show_skill(ctx, subargs)
        elif subcmd in ("help", "?"):
            return self._show_help()
        elif subcmd in ("dirs", "sources"):
            return self._show_sources()
        else:
            return await self._show_skill(ctx, subcmd)
    async def _list_skills(self, ctx: CommandContext) -> CommandResult:
        installer = get_installer()
        installed = installer.list_installed()
        all_skills = list_available_skills()
        registry = get_skill_registry()
        msg = f"\n  Available skills ({len(all_skills)}):"
        msg += f"\n  Installed: {len(installed)} | From dirs: {len(all_skills) - len(installed)
    }"
        msg += "\n"
        if installed:
            msg += "\n  [Installed]"
            for skill in installed:
                desc = skill.description[:50] if skill.description else ""
                msg += f"\n    /{skill.name:25} - {desc}"
        extra = [s for s in all_skills if not any(i.name == s for i in installed)]
        if extra:
            msg += "\n\n  [Loaded from skill dirs]"
            for name in sorted(extra):
```

```
            skill = registry.get_skill(name)
            if skill:
                desc = skill.description[:50] if skill.description else ""
                msg += f"\n    /{name:25} - {desc}"
    msg += "\n\n  Use /skill install <url> to install new skills"
    msg += "\n  Use /skill help for more commands"
    return CommandResult(success=True, message=msg)
async def _install_skill(self, ctx: CommandContext, args: str) -> CommandResult:
    if not args:
        msg = "\n  Usage: /skill install <url_or_repo>"
        msg += "\n"
        msg += "\n  Examples:"
        msg += "\n    /skill install https://github.com/cycleuser/Skills"
        msg += "\n    /skill install https://github.com/cycleuser/Skills/ humanizer"
        msg += "\n    /skill install https://raw.githubusercontent.com/..."
        return CommandResult(success=True, message=msg)
    parts = args.split(None, 1)
    repo_url = parts[0]
    skill_name = parts[1] if len(parts) > 1 else None
    installer = get_installer()
    msg = f"\n  Installing skills from: {repo_url}"
    if skill_name:
        msg += f" (filtering: {skill_name})"
    msg += "\n"
    print(msg)
    try:
        results = await installer.install_from_github(repo_url, skill_name)
    except Exception as e:
        return CommandResult(success=False, message=f"\n  Installation failed: {e}")
    # Rescan skills to pick up newly installed skills
    total = rescan_skills()
    # Re-register skill shortcuts in the command registry
    from openlaoke.commands.registry import get_command
    from openlaoke.commands.skill_shortcuts import SkillShortcutCommand
    for skill_name in list_available_skills():
        skill = load_skill(skill_name)
        if skill and not get_command(skill_name):
            desc = skill.description[:80] if skill.description else ""
            from openlaoke.commands.registry import _commands
            _commands[skill_name] = SkillShortcutCommand(skill_name, desc)
    success_count = sum(1 for r in results if r.success)
    fail_count = sum(1 for r in results if not r.success)
    msg = f"\n  Installation complete: {success_count} succeeded, {fail_count} failed"
    msg += f"\n  Total skills available: {total}"
    for r in results:
        if r.success:
            msg += f"\n    [green][OK][/green] {r.skill_name}: {r.message}"
        else:
            msg += f"\n    [red][X][/red] {r.skill_name}: {r.message}"
    if success_count > 0:
        msg += "\n\n  Skills are now available! Use /<skill_name> to activate."
        msg += "\n  Use /skill list to see all available skills."
    return CommandResult(success=success_count > 0, message=msg)
async def _remove_skill(self, ctx: CommandContext, args: str) -> CommandResult:
    if not args:
        return CommandResult(success=True, message="\n  Usage: /skill remove <skill_name>")
    installer = get_installer()
    result = installer.remove_skill(args)
    if result.success:
        msg = f"\n  [green][OK][/green] Removed skill: {args}"
    else:
        msg = f"\n  [red][X][/red] {result.message}"
    return CommandResult(success=result.success, message=msg)
async def _show_skill(self, ctx: CommandContext, name: str) -> CommandResult:
    skill = load_skill(name)
```

```
        if not skill:
            return CommandResult(
                success=False,
                message=f"\n  Skill not found: {name}\n  Use /skill list to see available skill
    s.",
            )
        msg = f"\n  Skill: {skill.name}"
        msg += f"\n  Version: {skill.version}"
        if skill.description:
            msg += f"\n\n  {skill.description}"
        if skill.allowed_tools:
            msg += f"\n\n  Allowed tools: {', '.join(skill.allowed_tools)}"
        if skill.path:
            msg += f"\n\n  Path: {skill.path}"
        return CommandResult(success=True, message=msg)
    def _show_help(self) -> CommandResult:
        msg = """
  Skill management commands:
    /skill                  List all available skills
    /skill list             List all available skills
    /skill install <url>    Install skills from a GitHub repo
    /skill remove <name>    Remove an installed skill
    /skill info <name>      Show details of a specific skill
    /skill dirs             Show skill source directories
    /skill help             Show this help message
  Install examples:
    /skill install https://github.com/cycleuser/Skills
    /skill install https://github.com/cycleuser/Skills/ humanizer
  Quick activation:
    /<skill_name>           Activate a skill directly (Tab to autocomplete)
  Skill sources:
    ~/.claude/skills/       (Claude Code skills)
    ~/.config/opencode/skills/  (OpenCode skills)
    ~/.openlaoke/skills/    (OpenLaoKe installed skills)
"""
        return CommandResult(success=True, message=msg)
    def _show_sources(self) -> CommandResult:
        dirs = get_default_skill_dirs()
        msg = "\n  Skill source directories:"
        for d in dirs:
            msg += f"\n    {d}"
        msg += f"\n\n  OpenLaoKe install dir: {get_installer().install_dir}"
        return CommandResult(success=True, message=msg)
class UseSkillCommand(SlashCommand):
    """Command to activate a skill for the current session."""
    name = "use"
    description = "Activate a skill for the current session"
    async def execute(self, ctx: CommandContext) -> CommandResult:
        args = ctx.args.strip()
        if not args:
            msg = "  Usage: /use <skill_name>"
            msg += "\n  Use /skill list to see available skills."
            msg += "\n  Tip: You can also use /<skill_name> directly!"
            return CommandResult(success=True, message=msg)
        skill = load_skill(args)
        if not skill:
            msg = f"  Skill not found: {args}"
            msg += "\n  Use /skill list to see available skills."
            return CommandResult(success=True, message=msg)
        if hasattr(ctx.app_state, "active_skills") and args not in ctx.app_state.active_skills:
            ctx.app_state.active_skills.append(args)
        msg = f"  [OK] Skill activated: {skill.name}"
        if skill.description:
            desc = skill.description[:100]
            if len(skill.description) > 100:
```

```
        desc += "..."
    msg += f"\n    {desc}"
return CommandResult(success=True, message=msg)
```

File: [16/252] openlaoke/commands/skill_shortcuts.py

```
"""Skill shortcut commands - directly invoke skills like /browse, /qa, etc."""
from __future__ import annotations
from dataclasses import dataclass
from openlaoke.commands.base import CommandContext, CommandResult, SlashCommand
from openlaoke.core.skill_system import list_available_skills, load_skill
@dataclass
class SkillActivationResult(CommandResult):
    """Result that includes skill activation info for continued processing."""
    skill_name: str = ""
    skill_content: str = ""
    should_continue_chat: bool = False
class SkillShortcutCommand(SlashCommand):
    """Dynamically created skill shortcut command."""
    def __init__(self, skill_name: str, description: str = ""):
        self.name = skill_name
        self.description = description or f"Activate {skill_name} skill"
        self.aliases = []
    async def execute(self, ctx: CommandContext) -> CommandResult:
        """Execute skill activation and continue with user input if provided."""
        skill = load_skill(self.name)
        if not skill:
            return CommandResult(success=False, message=f"Skill not found: {self.name}")
        if hasattr(ctx.app_state, "active_skills") and self.name not in ctx.app_state.active_sk
    ills:
            ctx.app_state.active_skills.append(self.name)
        msg = f"[OK] Skill activated: {skill.name}"
        if skill.description:
            desc = skill.description[:80]
            if len(skill.description) > 80:
                desc += "..."
            msg += f"\n  {desc}"
        result = SkillActivationResult(
            success=True,
            message=msg,
            skill_name=self.name,
            skill_content=skill.content,
            should_continue_chat=bool(ctx.args),
        )
        return result
def register_skill_shortcuts(registry: dict) -> None:
    """Register shortcuts for all available skills."""
    skills = list_available_skills()
    for skill_name in skills:
        skill = load_skill(skill_name)
        if skill:
            desc = skill.description[:80] if skill.description else ""
            cmd = SkillShortcutCommand(skill_name, desc)
            registry[skill_name] = cmd
```

File: [17/252] openlaoke/components/__init__.py

```
"""UI components for OpenLaoKe."""
```

File: [18/252] openlaoke/components/tui.py

```
"""Textual TUI for OpenLaoKe - rich terminal interface."""
from __future__ import annotations
from textual.app import App, ComposeResult
from textual.containers import Horizontal, Vertical
from textual.widgets import Footer, Header, Input, Label, RichLog
from openlaoke.core.state import AppState
from openlaoke.utils.theme import ThemeManager
class MessagesLog(Vertical):
    """Scrollable message display area."""
    def __init__(self, theme_manager: ThemeManager, id: str | None = None) -> None:
        super().__init__(id=id)
        self.theme_manager = theme_manager
    def compose(self) -> ComposeResult:
        self.rich_log = RichLog(markup=True, wrap=True, highlight=True)
        yield self.rich_log
    def add_message(self, role: str, content: str) -> None:
        theme = self.theme_manager.current_theme
        if role == "user":
            self.rich_log.write(
                f"\n[{theme.colors.primary} bold]You:[/{theme.colors.primary} bold] {content}"
            )
        elif role == "assistant":
            self.rich_log.write(
                f"\n[{theme.colors.secondary} bold]OpenLaoKe:[/{theme.colors.secondary} bold] {
    content}"
            )
        elif role == "system":
            self.rich_log.write(f"[{theme.colors.muted}]{content}[/{theme.colors.muted}]")
        elif role == "error":
            self.rich_log.write(
                f"[{theme.colors.error} bold]Error: {content}[/{theme.colors.error} bold]"
            )
        elif role == "tool":
            self.rich_log.write(f"  [{theme.colors.muted}]{content}[/{theme.colors.muted}]")
class StatusBar(Horizontal):
    """Status bar showing session info."""
    def __init__(
        self, app_state: AppState, theme_manager: ThemeManager, id: str | None = None
    ) -> None:
        super().__init__(id=id)
        self.app_state = app_state
        self.theme_manager = theme_manager
    def compose(self) -> ComposeResult:
        self.model_label = Label(f"Model: {self.app_state.session_config.model}")
        self.cost_label = Label("Cost: $0.0000")
        self.tokens_label = Label("Tokens: 0")
        self.cwd_label = Label(f"CWD: {self.app_state.get_cwd()}")
        yield self.model_label
        yield self.cost_label
        yield self.tokens_label
        yield self.cwd_label
    def update_status(self) -> None:
        usage = self.app_state.token_usage
        cost = self.app_state.cost_info
        self.cost_label.update(f"Cost: ${cost.total_cost:.4f}")
        self.tokens_label.update(f"Tokens: {usage.total_tokens:,}")
class OpenLaoKeTUI(App):
    """Full TUI for OpenLaoKe."""
    def __init__(self, app_state: AppState) -> None:
        super().__init__()
        self.app_state = app_state
```

```
    self.theme_manager = ThemeManager(app_state.theme)
def _generate_css(self) -> str:
    theme = self.theme_manager.current_theme
    return f"""
Screen {{
    layout: vertical;
}}
#header-bar {{
    height: 3;
    dock: top;
    background: {theme.colors.background};
    color: {theme.colors.foreground};
}}
#messages-area {{
    height: 1fr;
    background: {theme.colors.background};
    color: {theme.colors.foreground};
}}
#input-bar {{
    height: 3;
    dock: bottom;
    background: {theme.colors.background};
    color: {theme.colors.foreground};
}}
Input {{
    width: 100%;
    border: solid {theme.colors.primary};
}}
Input:focus {{
    border: solid {theme.colors.accent};
}}
Label {{
    color: {theme.colors.foreground};
}}
Footer {{
    background: {theme.colors.background};
    color: {theme.colors.foreground};
}}
Header {{
    background: {theme.colors.primary};
    color: {theme.colors.background};
}}
"""
CSS = ""
def compose(self) -> ComposeResult:
    yield Header()
    yield StatusBar(self.app_state, self.theme_manager)
    yield MessagesLog(self.theme_manager)
    yield Input(placeholder="Type a message or /help for commands...", id="prompt-input")
    yield Footer()
async def on_mount(self) -> None:
    self.query_one("#prompt-input", Input).focus()
    messages_log = self.query_one(MessagesLog)
    messages_log.add_message("system", "OpenLaoKe ready. Type a message to start.")
async def on_input_submitted(self, event: Input.Submitted) -> None:
    user_input = event.value.strip()
    if not user_input:
        return
    event.input.value = ""
    messages_log = self.query_one(MessagesLog)
    messages_log.add_message("user", user_input)
    if user_input.startswith("/"):
        await self._handle_command(user_input[1:])
    else:
        await self._handle_chat(user_input)
```

```
async def _handle_command(self, cmd_text: str) -> None:
    from openlaoke.commands.base import CommandContext
    from openlaoke.commands.registry import get_command
    messages_log = self.query_one(MessagesLog)
    parts = cmd_text.split(" ", 1)
    name = parts[0].lower()
    args = parts[1] if len(parts) > 1 else ""
    command = get_command(name)
    if not command:
        messages_log.add_message("error", f"Unknown command: /{name}")
        return
    ctx = CommandContext(app_state=self.app_state, args=args)
    result = await command.execute(ctx)
    if result.message:
        messages_log.add_message("system", result.message)
    if result.should_exit:
        self.exit()
async def _handle_chat(self, user_input: str) -> None:
    from openlaoke.core.api import APIClient, APIConfig
    from openlaoke.core.system_prompt import build_system_prompt
    from openlaoke.core.tool import ToolContext, ToolRegistry
    from openlaoke.tools.register import register_all_tools
    from openlaoke.types.core_types import (
        MessageRole,
        UserMessage,
    )
    messages_log = self.query_one(MessagesLog)
    registry = ToolRegistry()
    register_all_tools(registry)
    config = APIConfig.from_env()
    config.model = self.app_state.session_config.model
    api = APIClient(config)
    user_msg = UserMessage(role=MessageRole.USER, content=user_input)
    self.app_state.add_message(user_msg)
    system_prompt = build_system_prompt(self.app_state, registry.get_all_for_prompt())
    messages = self._serialize_messages()
    tools = registry.get_all_for_prompt()
    try:
        response, usage, cost = await api.send_message(
            system_prompt=system_prompt,
            messages=messages,
            tools=tools,
        )
        self.app_state.accumulate_tokens(usage)
        self.app_state.accumulate_cost(cost)
        if response.content:
            messages_log.add_message("assistant", response.content)
        if response.tool_uses:
            for tool_use in response.tool_uses:
                tool = registry.get(tool_use.name)
                if not tool:
                    messages_log.add_message("system", f"Unknown tool: {tool_use.name}")
                    continue
                ctx = ToolContext(app_state=self.app_state, tool_use_id=tool_use.id)
                result = await tool.call(ctx, **tool_use.input)
                messages_log.add_message("tool", f"{tool_use.name}: {result.content[:200]}"
)
    except Exception as e:
        messages_log.add_message("error", str(e))
    finally:
        await api.close()
    self.query_one(StatusBar).update_status()
def _serialize_messages(self) -> list[dict]:
    result = []
    for msg in self.app_state.get_messages():
```

```
    d = msg.to_dict()
    if d.get("type") == "user":
        result.append({"role": "user", "content": d.get("content", "")})
    elif d.get("type") == "assistant":
        content = []
        if d.get("content"):
            content.append({"type": "text", "text": d["content"]})
        result.append({"role": "assistant", "content": content})
return result
```

File: [19/252] openlaoke/core/__init__.py

```
"""Core systems for OpenLaoKe."""
```

File: [20/252] openlaoke/core/agent_runner.py

```
"""Agent runner for sub-agent tasks."""
from __future__ import annotations
import asyncio
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
    from openlaoke.types.core_types import TaskState
async def run_subagent(
    prompt: str,
    description: str,
    app_state: AppState,
    task_state: TaskState,
) -> str:
    """Run a sub-agent with the given prompt.
    The sub-agent uses the same API configuration and tools as the parent,
    but with its own isolated conversation context.
    """
    from openlaoke.core.config_wizard import get_proxy_url
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.core.tool import ToolRegistry
    from openlaoke.tools import register_all_tools
    registry = ToolRegistry()
    register_all_tools(registry)
    config = app_state.multi_provider_config
    if not config or not config.is_configured():
        return "Sub-agent error: No provider configured"
    app_config = getattr(app_state, "app_config", None)
    proxy = get_proxy_url(app_config) if app_config else None
    api = MultiProviderClient(config, proxy=proxy)
    system_prompt = _build_system_prompt(app_state)
    messages = [{"role": "user", "content": prompt}]
    tools = registry.get_all_for_prompt()
    max_iterations = 50
    iteration = 0
    result_parts = []
    try:
        while iteration < max_iterations:
            iteration += 1
            response, usage, cost = await api.send_message(
                system_prompt=system_prompt,
                messages=messages,
                tools=tools if iteration < max_iterations - 1 else None,
                model=app_state.session_config.model,
            )
            app_state.accumulate_tokens(usage)
            app_state.accumulate_cost(cost)
            if response.content:
                result_parts.append(response.content)
            messages.append(
                {
                    "role": "assistant",
                    "content": response.content,
                }
            )
            if not response.tool_uses:
                break
            for tool_use in response.tool_uses:
                tool = registry.get(tool_use.name)
                if not tool:
                    messages.append(
                        {
```

```
                        "role": "tool_result",
                        "tool_use_id": tool_use.id,
                        "content": f"Unknown tool: {tool_use.name}",
                    }
                )
                continue
            from openlaoke.core.tool import ToolContext
            ctx = ToolContext(
                app_state=app_state,
                tool_use_id=tool_use.id,
            )
            validation = tool.validate_input(tool_use.input)
            if not validation.result:
                messages.append(
                    {
                        "role": "tool_result",
                        "tool_use_id": tool_use.id,
                        "content": f"Validation error: {validation.message}",
                    }
                )
                continue
            result = await tool.call(ctx, **tool_use.input)
            messages.append(
                {
                    "role": "tool_result",
                    "tool_use_id": tool_use.id,
                    "content": result.content
                    if isinstance(result.content, str)
                    else str(result.content),
                }
            )
    except asyncio.CancelledError:
        raise
    except Exception as e:
        return f"Sub-agent error: {e}"
    finally:
        await api.close()
    return "\n".join(result_parts) if result_parts else "(no output)"
def _build_system_prompt(app_state: AppState) -> str:
    return (
        "You are a helpful assistant working as a sub-agent. "
        "You have access to tools to help accomplish tasks. "
        "Be concise and focused on the task at hand.\n\n"
        f"Working directory: {app_state.get_cwd()}"
    )
```

File: [21/252] openlaoke/core/api.py

```
"""API client for Anthropic-compatible LLM providers."""
from __future__ import annotations
import json
import os
from collections.abc import AsyncIterator
from contextlib import suppress
from dataclasses import dataclass
from typing import Any
import httpx
from openlaoke.types.core_types import (
    AssistantMessage,
    CostInfo,
    MessageRole,
    TokenUsage,
    ToolUseBlock,
)
@dataclass
class ModelPricing:
    """Pricing information for a model (per million tokens)."""
    input_price: float = 3.0
    output_price: float = 15.0
    cache_read_price: float = 0.30
    cache_creation_price: float = 3.75
MODEL_PRICES: dict[str, ModelPricing] = {
    "claude-sonnet-4-20250514": ModelPricing(3.0, 15.0, 0.30, 3.75),
    "claude-opus-4-20250514": ModelPricing(15.0, 75.0, 1.50, 18.75),
    "claude-3-5-sonnet-20241022": ModelPricing(3.0, 15.0, 0.30, 3.75),
    "claude-3-5-haiku-20241022": ModelPricing(0.80, 4.0, 0.08, 1.0),
    "claude-3-haiku-20240307": ModelPricing(0.25, 1.25, 0.025, 0.30),
}
@dataclass
class APIConfig:
    """Configuration for the API client."""
    api_key: str = ""
    base_url: str = "https://api.anthropic.com"
    model: str = "claude-sonnet-4-20250514"
    max_tokens: int = 8192
    temperature: float = 1.0
    top_p: float = 1.0
    thinking_budget: int = 0
    timeout: float = 300.0
    max_retries: int = 3
    proxy_url: str | None = None
    @classmethod
    def from_env(cls) -> APIConfig:
        return cls(
            api_key=os.environ.get("ANTHROPIC_API_KEY", ""),
            base_url=os.environ.get("ANTHROPIC_BASE_URL", "https://api.anthropic.com"),
            model=os.environ.get("OPENLAOKE_MODEL", "claude-sonnet-4-20250514"),
            max_tokens=int(os.environ.get("OPENLAOKE_MAX_TOKENS", "8192")),
            thinking_budget=int(os.environ.get("OPENLAOKE_THINKING_BUDGET", "0")),
            timeout=float(os.environ.get("OPENLAOKE_TIMEOUT", "300")),
            proxy_url=os.environ.get("HTTP_PROXY") or os.environ.get("HTTPS_PROXY"),
        )
class APIClient:
    """HTTP client for Anthropic-compatible API."""
    def __init__(self, config: APIConfig | None = None) -> None:
        self.config = config or APIConfig.from_env()
        self._client: httpx.AsyncClient | None = None
    def _get_client(self) -> httpx.AsyncClient:
        if self._client is None:
```

```
        limits = httpx.Limits(max_connections=10, max_keepalive_connections=5)
        proxy = self.config.proxy_url
        if proxy == "" or proxy is None:
            proxy = None
        self._client = httpx.AsyncClient(
            base_url=self.config.base_url,
            timeout=httpx.Timeout(self.config.timeout, connect=30.0),
            limits=limits,
            proxy=proxy,
        )
    return self._client
async def close(self) -> None:
    if self._client:
        await self._client.aclose()
        self._client = None
def _build_headers(self) -> dict[str, str]:
    return {
        "x-api-key": self.config.api_key,
        "anthropic-version": "2023-06-01",
        "content-type": "application/json",
    }
def _build_messages(
    self, system_prompt: str, messages: list[dict[str, Any]]
) -> list[dict[str, Any]]:
    result = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
        if role == "system":
            continue
        if role == "tool_result":
            result.append(
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "tool_result",
                            "tool_use_id": msg.get("tool_use_id", ""),
                            "content": content,
                        }
                    ],
                }
            )
        elif role == "tool_use":
            result.append(
                {
                    "role": "assistant",
                    "content": [
                        {
                            "type": "tool_use",
                            "id": msg.get("id", ""),
                            "name": msg.get("name", ""),
                            "input": msg.get("input", {}),
                        }
                    ],
                }
            )
        else:
            result.append({"role": role, "content": content})
    return result
def _calculate_cost(self, usage: dict[str, int]) -> CostInfo:
    pricing = MODEL_PRICES.get(self.config.model, ModelPricing())
    return CostInfo(
        input_cost=usage.get("input_tokens", 0) / 1_000_000 * pricing.input_price,
        output_cost=usage.get("output_tokens", 0) / 1_000_000 * pricing.output_price,
```

```
        cache_read_cost=usage.get("cache_read_input_tokens", 0)
        / 1_000_000
        * pricing.cache_read_price,
        cache_creation_cost=usage.get("cache_creation_input_tokens", 0)
        / 1_000_000
        * pricing.cache_creation_price,
    )
def _parse_token_usage(self, usage: dict[str, int]) -> TokenUsage:
    return TokenUsage(
        input_tokens=usage.get("input_tokens", 0),
        output_tokens=usage.get("output_tokens", 0),
        cache_read_tokens=usage.get("cache_read_input_tokens", 0),
        cache_creation_tokens=usage.get("cache_creation_input_tokens", 0),
    )
async def send_message(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None = None,
) -> AssistantMessage:
    """Send a single message request and get the response."""
    client = self._get_client()
    body: dict[str, Any] = {
        "model": self.config.model,
        "max_tokens": self.config.max_tokens,
        "temperature": self.config.temperature,
        "top_p": self.config.top_p,
        "system": system_prompt,
        "messages": self._build_messages(system_prompt, messages),
    }
    if tools:
        body["tools"] = tools
    if self.config.thinking_budget > 0:
        body["thinking"] = {
            "type": "enabled",
            "budget_tokens": self.config.thinking_budget,
        }
    response = await client.post(
        "/v1/messages",
        headers=self._build_headers(),
        json=body,
    )
    response.raise_for_status()
    data = response.json()
    content = ""
    tool_uses = []
    for block in data.get("content", []):
        if block["type"] == "text":
            content += block["text"]
        elif block["type"] == "tool_use":
            tool_uses.append(
                ToolUseBlock(
                    id=block["id"],
                    name=block["name"],
                    input=block["input"],
                )
            )
    usage = self._parse_token_usage(data.get("usage", {}))
    cost = self._calculate_cost(data.get("usage", {}))
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=data.get("stop_reason"),
```

```
        ),
        usage,
        cost,
    )
async def stream_message(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None = None,
) -> AsyncIterator[tuple[str, TokenUsage | None, CostInfo | None]]:
    """Stream a message response, yielding text chunks."""
    client = self._get_client()
    body: dict[str, Any] = {
        "model": self.config.model,
        "max_tokens": self.config.max_tokens,
        "temperature": self.config.temperature,
        "top_p": self.config.top_p,
        "system": system_prompt,
        "messages": self._build_messages(system_prompt, messages),
        "stream": True,
    }
    if tools:
        body["tools"] = tools
    if self.config.thinking_budget > 0:
        body["thinking"] = {
            "type": "enabled",
            "budget_tokens": self.config.thinking_budget,
        }
    async with client.stream(
        "POST",
        "/v1/messages",
        headers=self._build_headers(),
        json=body,
    ) as response:
        response.raise_for_status()
        content = ""
        tool_uses = []
        final_usage = None
        final_cost = None
        async for line in response.aiter_lines():
            if not line.startswith("data: "):
                continue
            data_str = line[6:]
            if data_str == "[DONE]":
                break
            try:
                event = json.loads(data_str)
            except json.JSONDecodeError:
                continue
            event_type = event.get("type", "")
            if event_type == "content_block_start":
                block = event.get("content_block", {})
                if block.get("type") == "tool_use":
                    tool_uses.append(
                        ToolUseBlock(
                            id=block["id"],
                            name=block["name"],
                            input={},
                        )
                    )
            elif event_type == "content_block_delta":
                delta = event.get("delta", {})
                if delta.get("type") == "text_delta":
                    text = delta.get("text", "")
                    content += text
```

```
            yield text, None, None
        elif delta.get("type") == "input_json_delta":
            partial_json = delta.get("partial_json", "")
            if tool_uses:
                with suppress(json.JSONDecodeError):
                    tool_uses[-1].input = json.loads(
                        json.dumps(tool_uses[-1].input)[:-1] + partial_json + "}"
                    )
    elif event_type == "message_delta":
        usage_data = event.get("usage", {})
        if usage_data:
            final_usage = self._parse_token_usage(usage_data)
            final_cost = self._calculate_cost(usage_data)
    elif event_type == "message_stop":
        pass
yield "", final_usage, final_cost
```

File: [22/252] openlaoke/core/architecture/__init__.py

```
"""Architecture system for small model development."""
from openlaoke.core.architecture.assembler import (
    AssemblyResult,
    CodeAssembler,
    IntegrationValidator,
    ValidationResult,
)
from openlaoke.core.architecture.decomposer import (
    AtomicTask,
    FineGrainedDecomposer,
    TaskGraph,
    create_decomposer_for_model,
)
from openlaoke.core.architecture.interfaces import (
    STANDARD_API_SPECS,
    STANDARD_TEMPLATES,
    APISpec,
    Assemblable,
    CodeTemplate,
    ComponentSpec,
    ComponentType,
    TaskSize,
    Testable,
    Validatable,
    estimate_task_complexity,
    get_template_for_component,
    should_decompose_for_model,
)
from openlaoke.core.architecture.orchestrator import (
    IncrementalOrchestrator,
    IncrementalWorkflow,
    WorkflowStep,
    create_orchestrator_for_model,
)
__all__ = [
    "APISpec",
    "CodeTemplate",
    "ComponentSpec",
    "ComponentType",
    "TaskSize",
    "Validatable",
    "Testable",
    "Assemblable",
    "STANDARD_TEMPLATES",
    "STANDARD_API_SPECS",
    "get_template_for_component",
    "estimate_task_complexity",
    "should_decompose_for_model",
    "AtomicTask",
    "TaskGraph",
    "FineGrainedDecomposer",
    "create_decomposer_for_model",
    "AssemblyResult",
    "ValidationResult",
    "CodeAssembler",
    "IntegrationValidator",
    "WorkflowStep",
    "IncrementalWorkflow",
    "IncrementalOrchestrator",
    "create_orchestrator_for_model",
]
```

File: [23/252] openlaoke/core/architecture/assembler.py

```
"""Automatic assembly and validation of components.
Assembles atomic tasks into working code with strict validation.
"""
from __future__ import annotations
import ast
import subprocess
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.core.architecture.decomposer import AtomicTask, TaskGraph
from openlaoke.core.architecture.interfaces import (
    CodeTemplate,
    ComponentType,
)
@dataclass
class AssemblyResult:
    success: bool
    code: str
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    test_results: dict[str, bool] = field(default_factory=dict)
    coverage: float = 0.0
@dataclass
class ValidationResult:
    is_valid: bool
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    suggestions: list[str] = field(default_factory=list)
class CodeAssembler:
    def __init__(self, project_root: Path):
        self.project_root = project_root
        self.assembled_components: dict[str, str] = {}
        self.import_registry: dict[str, set[str]] = {}
    def assemble_task_graph(self, graph: TaskGraph) -> AssemblyResult:
        all_code = []
        all_errors = []
        all_warnings = []
        test_results = {}
        imports_collected = set()
        ready_tasks = graph.get_ready_tasks()
        processed_count = 0
        while ready_tasks and processed_count < len(graph.tasks):
            for task in ready_tasks:
                if task.task_id in graph.completed:
                    continue
                result = self.assemble_atomic_task(task, graph)
                if result.success:
                    if task.component_spec.component_type != ComponentType.TEST:
                        all_code.append(result.code)
                        imports_collected.update(task.template.required_imports)
                    graph.mark_completed(task.task_id)
                    processed_count += 1
                    if task.component_spec.component_type == ComponentType.TEST:
                        test_results[task.task_id] = True
                else:
                    all_errors.extend(result.errors)
                    all_warnings.extend(result.warnings)
                    graph.mark_failed(task.task_id)
                    processed_count += 1
            ready_tasks = graph.get_ready_tasks()
        final_code = self._organize_code(imports_collected, all_code)
```

```
    return AssemblyResult(
        success=len(all_errors) == 0 and len(graph.completed) == len(graph.tasks),
        code=final_code,
        errors=all_errors,
        warnings=all_warnings,
        test_results=test_results,
    )
def assemble_atomic_task(self, task: AtomicTask, graph: TaskGraph) -> AssemblyResult:
    code_parts = []
    errors = []
    warnings = []
    for dep_id in task.dependencies:
        if dep_id in graph.failed:
            errors.append(f"Dependency {dep_id} failed")
            return AssemblyResult(
                success=False,
                code="",
                errors=errors,
                warnings=warnings,
            )
    context = self._build_context(task, graph)
    filled_template = self._fill_template(task.template, context)
    validation = self._validate_code(filled_template, task)
    if not validation.is_valid:
        errors.extend(validation.errors)
        warnings.extend(validation.warnings)
        if not self._can_auto_fix(validation):
            return AssemblyResult(
                success=False,
                code=filled_template,
                errors=errors,
                warnings=warnings,
            )
        fixed_code = self._auto_fix_code(filled_template, validation)
        filled_template = fixed_code
    code_parts.append(filled_template)
    self.assembled_components[task.task_id] = filled_template
    return AssemblyResult(
        success=len(errors) == 0,
        code="\n\n".join(code_parts),
        errors=errors,
        warnings=warnings,
    )
def _build_context(self, task: AtomicTask, graph: TaskGraph) -> dict[str, Any]:
    context = task.context.copy()
    context.update(
        {
            "function_name": task.component_spec.name,
            "class_name": task.component_spec.name,
            "return_type": "Any",
            "return_value": "result",
            "docstring": task.description,
            "parameters": "",
            "args_doc": "Arguments",
            "return_doc": "Return value",
            "raises_doc": "Exceptions",
            "implementation": "pass",
        }
    )
    if task.component_spec.api_spec:
        api = task.component_spec.api_spec
        params = []
        for param_name, param_spec in api.input_schema.get("properties", {}).items():
            param_type = param_spec.get("type", "Any")
            params.append(f"{param_name}: {param_type}")
```

```
            context["parameters"] = ", ".join(params)
        return context
    def _fill_template(self, template: CodeTemplate, context: dict[str, Any]) -> str:
        try:
            return template.template.format(**context)
        except KeyError as e:
            placeholder = str(e).strip("'")
            return template.template.replace(
                "{" + placeholder + "}",
                context.get(placeholder, "# TODO: Implement"),
            )
    def _validate_code(self, code: str, task: AtomicTask) -> ValidationResult:
        errors = []
        warnings = []
        suggestions = []
        try:
            ast.parse(code)
        except SyntaxError as e:
            errors.append(f"Syntax error: {e}")
        for rule in task.validation_rules:
            if "type hints" in rule.lower():
                if "def " in code and " -> " not in code:
                    errors.append("Missing return type hint")
                if "(" in code and ":" not in code.split("(")[1].split(")")[0]:
                    warnings.append("Consider adding parameter type hints")
            if "docstring" in rule.lower() and '"""' not in code and "'''" not in code:
                errors.append("Missing docstring")
            if "test" in rule.lower() and not task.test_required:
                warnings.append("Consider adding tests")
        return ValidationResult(
            is_valid=len(errors) == 0,
            errors=errors,
            warnings=warnings,
            suggestions=suggestions,
        )
    def _can_auto_fix(self, validation: ValidationResult) -> bool:
        auto_fixable = [
            "Missing docstring",
            "Consider adding",
        ]
        return all(any(af in error for af in auto_fixable) for error in validation.errors)
    def _auto_fix_code(self, code: str, validation: ValidationResult) -> str:
        fixed_code = code
        if any("docstring" in e for e in validation.errors):
            lines = code.split("\n")
            if lines[0].startswith("def ") or lines[0].startswith("class "):
                indent = "    "
                docstring = f'{indent}"""TODO: Add docstring."""'
                lines.insert(1, docstring)
            fixed_code = "\n".join(lines)
        return fixed_code
    def _organize_code(self, imports: set[str], code_parts: list[str]) -> str:
        sections = []
        sections.append('"""Generated by OpenLaoKe."""\n')
        if imports:
            sorted_imports = sorted(imports)
            sections.append("\n".join(sorted_imports))
            sections.append("")
        for part in code_parts:
            if part.strip():
                sections.append(part)
                sections.append("")
        return "\n".join(sections)
class IntegrationValidator:
    def __init__(self, project_root: Path):
```

```
    self.project_root = project_root
def validate_assembly(self, result: AssemblyResult) -> ValidationResult:
    errors = []
    warnings = []
    suggestions = []
    if not result.code.strip():
        errors.append("Empty code generated")
        return ValidationResult(
            is_valid=False,
            errors=errors,
            warnings=warnings,
            suggestions=suggestions,
        )
    try:
        ast.parse(result.code)
    except SyntaxError as e:
        errors.append(f"Generated code has syntax errors: {e}")
    temp_file = self.project_root / ".temp_validation.py"
    try:
        temp_file.write_text(result.code)
        ruff_result = subprocess.run(
            ["ruff", "check", str(temp_file), "--select=E,F"],
            capture_output=True,
            text=True,
        )
        if ruff_result.returncode != 0:
            warnings.append(f"Ruff issues: {ruff_result.stdout}")
        mypy_result = subprocess.run(
            ["mypy", str(temp_file), "--no-error-summary"],
            capture_output=True,
            text=True,
        )
        if mypy_result.returncode != 0:
            warnings.append(f"Type issues: {mypy_result.stdout}")
    except Exception as e:
        warnings.append(f"Validation tool error: {e}")
    finally:
        if temp_file.exists():
            temp_file.unlink()
    if not result.test_results:
        suggestions.append("Consider adding integration tests")
    elif not all(result.test_results.values()):
        failed_tests = [k for k, v in result.test_results.items() if not v]
        errors.append(f"Failed tests: {', '.join(failed_tests)}")
    return ValidationResult(
        is_valid=len(errors) == 0,
        errors=errors,
        warnings=warnings,
        suggestions=suggestions,
    )
def run_integration_tests(self, result: AssemblyResult) -> dict[str, bool]:
    test_results = {}
    if not result.code.strip():
        return test_results
    temp_test_file = self.project_root / ".temp_test.py"
    try:
        temp_test_file.write_text(result.code)
        pytest_result = subprocess.run(
            ["pytest", str(temp_test_file), "-v", "--tb=short"],
            capture_output=True,
            text=True,
        )
        test_results["pytest"] = pytest_result.returncode == 0
        if pytest_result.returncode != 0:
            test_results["pytest_output"] = pytest_result.stdout
```

```
except Exception as e:
    test_results["error"] = str(e)
finally:
    if temp_test_file.exists():
        temp_test_file.unlink()
return test_results
```

File: [24/252] openlaoke/core/architecture/decomposer.py

```
"""Fine-grained task decomposition for small models.
Breaks down complex projects into atomic, function-level tasks
that even gemma3:1b can implement reliably.
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
from openlaoke.core.architecture.interfaces import (
    APISpec,
    CodeTemplate,
    ComponentSpec,
    ComponentType,
    estimate_task_complexity,
    get_template_for_component,
    should_decompose_for_model,
)
from openlaoke.core.model_assessment.types import ModelTier
@dataclass
class AtomicTask:
    task_id: str
    description: str
    component_spec: ComponentSpec
    template: CodeTemplate
    dependencies: list[str] = field(default_factory=list)
    estimated_lines: int = 10
    test_required: bool = True
    validation_rules: list[str] = field(default_factory=list)
    context: dict[str, Any] = field(default_factory=dict)
    parent_task_id: str | None = None
@dataclass
class TaskGraph:
    root_task_id: str
    tasks: dict[str, AtomicTask] = field(default_factory=dict)
    dependency_edges: dict[str, list[str]] = field(default_factory=dict)
    completed: set[str] = field(default_factory=set)
    failed: set[str] = field(default_factory=set)
    def add_task(self, task: AtomicTask) -> None:
        self.tasks[task.task_id] = task
        if task.dependencies:
            self.dependency_edges[task.task_id] = task.dependencies
    def get_ready_tasks(self) -> list[AtomicTask]:
        ready = []
        for task_id, task in self.tasks.items():
            if task_id in self.completed or task_id in self.failed:
                continue
            deps_satisfied = all(dep_id in self.completed for dep_id in task.dependencies)
            if deps_satisfied:
                ready.append(task)
        return ready
    def mark_completed(self, task_id: str) -> None:
        self.completed.add(task_id)
    def mark_failed(self, task_id: str) -> None:
        self.failed.add(task_id)
class FineGrainedDecomposer:
    def __init__(self, model_tier: ModelTier):
        self.model_tier = model_tier
        self.max_lines_per_task = self._get_max_lines()
        self.max_params_per_function = self._get_max_params()
        self.max_complexity = self._get_max_complexity()
    def _get_max_lines(self) -> int:
        limits = {
```

```
        ModelTier.TIER_1_ADVANCED: 100,
        ModelTier.TIER_2_CAPABLE: 50,
        ModelTier.TIER_3_MODERATE: 30,
        ModelTier.TIER_4_BASIC: 20,
        ModelTier.TIER_5_LIMITED: 15,
    }
    return limits[self.model_tier]
def _get_max_params(self) -> int:
    limits = {
        ModelTier.TIER_1_ADVANCED: 10,
        ModelTier.TIER_2_CAPABLE: 7,
        ModelTier.TIER_3_MODERATE: 5,
        ModelTier.TIER_4_BASIC: 4,
        ModelTier.TIER_5_LIMITED: 3,
    }
    return limits[self.model_tier]
def _get_max_complexity(self) -> int:
    limits = {
        ModelTier.TIER_1_ADVANCED: 20,
        ModelTier.TIER_2_CAPABLE: 15,
        ModelTier.TIER_3_MODERATE: 10,
        ModelTier.TIER_4_BASIC: 7,
        ModelTier.TIER_5_LIMITED: 5,
    }
    return limits[self.model_tier]
def decompose_function(self, spec: ComponentSpec) -> list[AtomicTask]:
    tasks = []
    if spec.api_spec:
        input_params = spec.api_spec.input_schema.get("properties", {})
        if len(input_params) > self.max_params_per_function:
            param_groups = self._group_parameters(input_params)
            for i, group in enumerate(param_groups):
                group_spec = self._create_param_group_spec(spec, group, i)
                task = self._create_atomic_task(group_spec, f"param_group_{i}")
                tasks.append(task)
    main_task = self._create_atomic_task(spec, "main")
    if len(tasks) > 0:
        main_task.dependencies = [t.task_id for t in tasks]
        for t in tasks:
            t.parent_task_id = main_task.task_id
    tasks.append(main_task)
    if len(spec.test_requirements) > 0:
        test_task = self._create_test_task(spec, tasks[-1])
        test_task.dependencies = [main_task.task_id]
        tasks.append(test_task)
    return tasks
def decompose_class(self, spec: ComponentSpec) -> list[AtomicTask]:
    tasks = []
    init_spec = ComponentSpec(
        name=f"{spec.name}__init__",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name=f"{spec.name}_init",
            description=f"Initialize {spec.name}",
            input_schema={"type": "object", "properties": {}},
            output_schema={"type": "object"},
        ),
        test_requirements=["basic test"],
        complexity_score=2,
    )
    init_task = self._create_atomic_task(init_spec, "init")
    tasks.append(init_task)
    method_specs = self._extract_methods_from_spec(spec)
    for method_spec in method_specs:
        method_tasks = self.decompose_function(method_spec)
```

```
        for task in method_tasks:
            task.dependencies.append(init_task.task_id)
        tasks.extend(method_tasks)
    class_task = self._create_atomic_task(spec, "class_definition")
    class_task.dependencies = [t.task_id for t in tasks]
    tasks.append(class_task)
    return tasks
def decompose_module(self, spec: ComponentSpec) -> list[AtomicTask]:
    tasks = []
    imports_task = AtomicTask(
        task_id=f"{spec.name}_imports",
        description=f"Import statements for {spec.name}",
        component_spec=ComponentSpec(
            name=f"{spec.name}_imports",
            component_type=ComponentType.FUNCTION,
            complexity_score=1,
        ),
        template=CodeTemplate(
            name="imports",
            description="Import statements",
            template='"""Module-level imports."""\nfrom __future__ import annotations\n{imp
orts}',
            required_imports=[],
        ),
        estimated_lines=5,
        test_required=False,
    )
    tasks.append(imports_task)
    component_specs = self._extract_components_from_module(spec)
    for comp_spec in component_specs:
        if comp_spec.component_type == ComponentType.FUNCTION:
            comp_tasks = self.decompose_function(comp_spec)
        elif comp_spec.component_type == ComponentType.CLASS:
            comp_tasks = self.decompose_class(comp_spec)
        else:
            comp_tasks = [self._create_atomic_task(comp_spec, "component")]
        for task in comp_tasks:
            task.dependencies.insert(0, imports_task.task_id)
        tasks.extend(comp_tasks)
    exports_task = AtomicTask(
        task_id=f"{spec.name}_exports",
        description=f"Module exports for {spec.name}",
        component_spec=ComponentSpec(
            name=f"{spec.name}_exports",
            component_type=ComponentType.FUNCTION,
            complexity_score=1,
        ),
        template=CodeTemplate(
            name="exports",
            description="Module exports",
            template="\n__all__ = [{exports}]",
            required_imports=[],
        ),
        estimated_lines=3,
        test_required=False,
        dependencies=[t.task_id for t in tasks if t.task_id != imports_task.task_id],
    )
    tasks.append(exports_task)
    return tasks
def decompose_project(self, project_spec: dict[str, Any]) -> TaskGraph:
    graph = TaskGraph(root_task_id="project_root")
    modules = project_spec.get("modules", [])
    for module_spec_dict in modules:
        module_spec = ComponentSpec(
            name=module_spec_dict["name"],
```

```
            component_type=ComponentType.MODULE,
            dependencies=module_spec_dict.get("dependencies", []),
            complexity_score=module_spec_dict.get("complexity", 5),
        )
        module_tasks = self.decompose_module(module_spec)
        for task in module_tasks:
            graph.add_task(task)
    return graph
def _create_atomic_task(self, spec: ComponentSpec, suffix: str = "") -> AtomicTask:
    task_id = f"{spec.name}_{suffix}" if suffix else spec.name
    template = get_template_for_component(spec.component_type)
    complexity = estimate_task_complexity(spec)
    if should_decompose_for_model(complexity, self.model_tier.value):
        estimated_lines = self.max_lines_per_task // 2
    else:
        estimated_lines = self.max_lines_per_task
    return AtomicTask(
        task_id=task_id,
        description=f"Implement {spec.name} ({spec.component_type.value})",
        component_spec=spec,
        template=template,
        estimated_lines=estimated_lines,
        test_required=True,
        validation_rules=template.validation_rules,
    )
def _create_test_task(self, spec: ComponentSpec, parent_task: AtomicTask) -> AtomicTask:
    test_spec = ComponentSpec(
        name=f"test_{spec.name}",
        component_type=ComponentType.TEST,
        test_requirements=[],
        complexity_score=1,
    )
    return AtomicTask(
        task_id=f"test_{spec.name}",
        description=f"Test {spec.name}",
        component_spec=test_spec,
        template=get_template_for_component(ComponentType.TEST),
        estimated_lines=15,
        test_required=False,
        parent_task_id=parent_task.task_id,
    )
def _group_parameters(self, params: dict[str, Any]) -> list[dict[str, Any]]:
    groups = []
    current_group = {}
    for param_name, param_spec in params.items():
        current_group[param_name] = param_spec
        if len(current_group) >= self.max_params_per_function:
            groups.append(current_group)
            current_group = {}
    if current_group:
        groups.append(current_group)
    return groups
def _create_param_group_spec(
    self, parent_spec: ComponentSpec, group: dict[str, Any], index: int
) -> ComponentSpec:
    return ComponentSpec(
        name=f"{parent_spec.name}_params_{index}",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name=f"{parent_spec.name}_params_{index}",
            description=f"Parameter group {index} for {parent_spec.name}",
            input_schema={"type": "object", "properties": group},
            output_schema=parent_spec.api_spec.output_schema if parent_spec.api_spec else {
},
        ),
```

```
            test_requirements=["basic test"],
            complexity_score=1,
        )
    def _extract_methods_from_spec(self, spec: ComponentSpec) -> list[ComponentSpec]:
        methods = []
        if spec.api_spec:
            methods.append(
                ComponentSpec(
                    name=f"{spec.name}_process",
                    component_type=ComponentType.FUNCTION,
                    api_spec=APISpec(
                        name=f"{spec.name}_process",
                        description=f"Process method for {spec.name}",
                        input_schema=spec.api_spec.input_schema,
                        output_schema=spec.api_spec.output_schema,
                    ),
                    test_requirements=["basic test"],
                    complexity_score=spec.complexity_score // 2,
                )
            )
        return methods
    def _extract_components_from_module(self, spec: ComponentSpec) -> list[ComponentSpec]:
        components = []
        components.append(
            ComponentSpec(
                name=f"{spec.name}_helper",
                component_type=ComponentType.FUNCTION,
                test_requirements=["basic test"],
                complexity_score=2,
            )
        )
        components.append(
            ComponentSpec(
                name=f"{spec.name}_main_class",
                component_type=ComponentType.CLASS,
                test_requirements=["basic test"],
                complexity_score=spec.complexity_score,
            )
        )
        return components
def create_decomposer_for_model(model_tier: ModelTier) -> FineGrainedDecomposer:
    return FineGrainedDecomposer(model_tier)
```

File: [25/252] openlaoke/core/architecture/interfaces.py

```
"""Standard API interfaces and templates for small model development.
This module defines consistent interfaces that small models can implement
piece by piece, ensuring perfect assembly.
"""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any, Protocol, runtime_checkable
class ComponentType(StrEnum):
    FUNCTION = "function"
    CLASS = "class"
    MODULE = "module"
    API_ENDPOINT = "api_endpoint"
    DATA_MODEL = "data_model"
    TEST = "test"
class TaskSize(StrEnum):
    ATOMIC = "atomic"
    SMALL = "small"
    MEDIUM = "medium"
    LARGE = "large"
@dataclass
class CodeTemplate:
    name: str
    description: str
    template: str
    required_imports: list[str] = field(default_factory=list)
    required_dependencies: list[str] = field(default_factory=list)
    validation_rules: list[str] = field(default_factory=list)
    example_usage: str = ""
    docstring_template: str = ""
@dataclass
class APISpec:
    name: str
    description: str
    input_schema: dict[str, Any]
    output_schema: dict[str, Any]
    error_codes: dict[str, str] = field(default_factory=dict)
    version: str = "1.0.0"
    examples: list[dict[str, Any]] = field(default_factory=list)
@dataclass
class ComponentSpec:
    name: str
    component_type: ComponentType
    api_spec: APISpec | None = None
    dependencies: list[str] = field(default_factory=list)
    implements: list[str] = field(default_factory=list)
    test_requirements: list[str] = field(default_factory=list)
    complexity_score: int = 1
@runtime_checkable
class Validatable(Protocol):
    def validate(self) -> tuple[bool, list[str]]: ...
@runtime_checkable
class Testable(Protocol):
    def get_test_cases(self) -> list[dict[str, Any]]: ...
@runtime_checkable
class Assemblable(Protocol):
    def get_dependencies(self) -> list[str]: ...
    def get_exports(self) -> list[str]: ...
STANDARD_TEMPLATES: dict[str, CodeTemplate] = {
    "function_basic": CodeTemplate(
        name="function_basic",
```

```
        description="Basic function with type hints and docstring",
        template='''def {function_name}({parameters}) -> {return_type}:
    """
    {docstring}
    Args:
        {args_doc}
    Returns:
        {return_doc}
    Raises:
        {raises_doc}
    """
    {implementation}
    return {return_value}''',
        required_imports=["from __future__ import annotations"],
        validation_rules=[
            "Must have complete type hints",
            "Must have docstring with Args, Returns, Raises sections",
            "Must have at least one test case",
        ],
        docstring_template="Brief description of what the function does.\n\nDetailed explanatio
    n if needed.",
    ),
    "class_basic": CodeTemplate(
        name="class_basic",
        description="Basic class with init and methods",
        template='''class {class_name}:
    """
    {docstring}
    Attributes:
        {attrs_doc}
    """
    def __init__(self, {init_params}):
        """Initialize {class_name}."""
        {init_implementation}
    {methods}''',
        required_imports=[
            "from __future__ import annotations",
            "from dataclasses import dataclass",
        ],
        validation_rules=[
            "Must have __init__ method",
            "All public methods must have type hints",
            "Must have class-level docstring",
        ],
    ),
    "api_endpoint": CodeTemplate(
        name="api_endpoint",
        description="FastAPI endpoint with validation",
        template='''@router.{method}("/{path}", response_model={response_model})
async def {function_name}({parameters}) -> {return_type}:
    """
    {docstring}
    Args:
        {args_doc}
    Returns:
        {return_doc}
    Raises:
        HTTPException: {error_doc}
    """
    {implementation}
    return {return_value}''',
        required_imports=[
            "from fastapi import APIRouter, HTTPException",
            "from pydantic import BaseModel",
        ],
```

```
        required_dependencies=["fastapi>=0.100.0", "pydantic>=2.0.0"],
        validation_rules=[
            "Must have response_model",
            "Must handle errors with HTTPException",
            "Must have async def for I/O operations",
        ],
    ),
    "data_model": CodeTemplate(
        name="data_model",
        description="Pydantic data model with validation",
        template='''class {class_name}(BaseModel):
    """
    {docstring}
    """
    {fields}
    {validators}''',
        required_imports=["from pydantic import BaseModel, Field, validator"],
        validation_rules=[
            "Must inherit from BaseModel",
            "All fields must have Field() with description",
            "Complex validations must use @validator",
        ],
    ),
    "test_function": CodeTemplate(
        name="test_function",
        description="Pytest test function",
        template='''def test_{function_name}_{test_case}():
    """Test {test_description}."""
    # Arrange
    {arrange_code}
    # Act
    {act_code}
    # Assert
    {assert_code}''',
        required_imports=["import pytest"],
        validation_rules=[
            "Must follow AAA pattern (Arrange-Act-Assert)",
            "Must have descriptive test name",
            "Must test one thing only",
        ],
    ),
}
STANDARD_API_SPECS: dict[str, APISpec] = {
    "tool_call": APISpec(
        name="tool_call",
        description="Standard tool call interface",
        input_schema={
            "type": "object",
            "properties": {
                "name": {"type": "string", "description": "Tool name"},
                "arguments": {"type": "object", "description": "Tool arguments"},
            },
            "required": ["name", "arguments"],
        },
        output_schema={
            "type": "object",
            "properties": {
                "success": {"type": "boolean"},
                "result": {"type": "any"},
                "error": {"type": "string"},
            },
            "required": ["success"],
        },
        error_codes={
            "TOOL_NOT_FOUND": "Requested tool does not exist",
```

```
            "INVALID_ARGS": "Arguments do not match tool schema",
            "EXECUTION_ERROR": "Tool execution failed",
        },
        examples=[
            {
                "input": {"name": "read", "arguments": {"file_path": "/tmp/test.txt"}},
                "output": {"success": True, "result": "file content", "error": None},
            }
        ],
    ),
    "state_update": APISpec(
        name="state_update",
        description="Standard state update interface",
        input_schema={
            "type": "object",
            "properties": {
                "state_key": {"type": "string"},
                "value": {"type": "any"},
                "metadata": {"type": "object"},
            },
            "required": ["state_key", "value"],
        },
        output_schema={
            "type": "object",
            "properties": {
                "success": {"type": "boolean"},
                "previous_value": {"type": "any"},
                "timestamp": {"type": "number"},
            },
            "required": ["success", "timestamp"],
        },
    ),
    "component_assembly": APISpec(
        name="component_assembly",
        description="Component assembly interface",
        input_schema={
            "type": "object",
            "properties": {
                "components": {"type": "array", "items": {"type": "object"}},
                "assembly_order": {"type": "array", "items": {"type": "string"}},
                "validation_level": {"type": "string", "enum": ["basic", "strict", "exhaustive"
    ]},
            },
            "required": ["components"],
        },
        output_schema={
            "type": "object",
            "properties": {
                "success": {"type": "boolean"},
                "assembled_code": {"type": "string"},
                "errors": {"type": "array", "items": {"type": "string"}},
                "warnings": {"type": "array", "items": {"type": "string"}},
            },
            "required": ["success"],
        },
    ),
}
def get_template_for_component(component_type: ComponentType) -> CodeTemplate:
    template_map = {
        ComponentType.FUNCTION: STANDARD_TEMPLATES["function_basic"],
        ComponentType.CLASS: STANDARD_TEMPLATES["class_basic"],
        ComponentType.API_ENDPOINT: STANDARD_TEMPLATES["api_endpoint"],
        ComponentType.DATA_MODEL: STANDARD_TEMPLATES["data_model"],
        ComponentType.TEST: STANDARD_TEMPLATES["test_function"],
    }
```

```
    return template_map.get(component_type, STANDARD_TEMPLATES["function_basic"])
def estimate_task_complexity(spec: ComponentSpec) -> TaskSize:
    score = spec.complexity_score
    if len(spec.dependencies) > 5:
        score += 2
    if len(spec.test_requirements) > 5:
        score += 1
    if spec.api_spec and len(spec.api_spec.input_schema.get("properties", {})) > 5:
        score += 2
    if score <= 2:
        return TaskSize.ATOMIC
    elif score <= 4:
        return TaskSize.SMALL
    elif score <= 6:
        return TaskSize.MEDIUM
    else:
        return TaskSize.LARGE
def should_decompose_for_model(task_size: TaskSize, model_tier: str) -> bool:
    if model_tier == "tier_5_limited":
        return task_size in [TaskSize.MEDIUM, TaskSize.LARGE]
    elif model_tier == "tier_4_basic":
        return task_size == TaskSize.LARGE
    else:
        return False
```

File: [26/252] openlaoke/core/architecture/orchestrator.py

```
"""Incremental development workflow for small models.
Orchestrates the complete workflow: decomposition -> implementation -> assembly -> validation.
"""
from __future__ import annotations
import json
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.core.architecture.assembler import (
    AssemblyResult,
    CodeAssembler,
    IntegrationValidator,
)
from openlaoke.core.architecture.decomposer import (
    AtomicTask,
    FineGrainedDecomposer,
    TaskGraph,
    create_decomposer_for_model,
)
from openlaoke.core.architecture.interfaces import (
    TaskSize,
    estimate_task_complexity,
)
from openlaoke.core.model_assessment.types import ModelTier
from openlaoke.core.state import AppState
@dataclass
class WorkflowStep:
    step_id: str
    task: AtomicTask
    status: str = "pending"
    attempts: int = 0
    max_attempts: int = 3
    result: AssemblyResult | None = None
    error: str | None = None
    started_at: float | None = None
    completed_at: float | None = None
@dataclass
class IncrementalWorkflow:
    workflow_id: str
    project_spec: dict[str, Any]
    model: str
    model_tier: ModelTier
    decomposer: FineGrainedDecomposer
    assembler: CodeAssembler
    validator: IntegrationValidator
    task_graph: TaskGraph | None = None
    steps: dict[str, WorkflowStep] = field(default_factory=dict)
    current_step: str | None = None
    completed_steps: list[str] = field(default_factory=list)
    failed_steps: list[str] = field(default_factory=list)
    def get_progress(self) -> dict[str, Any]:
        total = len(self.steps)
        completed = len(self.completed_steps)
        failed = len(self.failed_steps)
        pending = total - completed - failed
        return {
            "workflow_id": self.workflow_id,
            "total_steps": total,
            "completed": completed,
            "failed": failed,
```

```
            "pending": pending,
            "progress_percentage": (completed / total * 100) if total > 0 else 0,
            "current_step": self.current_step,
        }
class IncrementalOrchestrator:
    def __init__(self, app_state: AppState, model: str, model_tier: ModelTier):
        self.app_state = app_state
        self.model = model
        self.model_tier = model_tier
        self.decomposer = create_decomposer_for_model(model_tier)
        self.project_root = Path(app_state.cwd)
        self.assembler = CodeAssembler(self.project_root)
        self.validator = IntegrationValidator(self.project_root)
        self.workflows: dict[str, IncrementalWorkflow] = {}
    def create_workflow(self, project_spec: dict[str, Any]) -> IncrementalWorkflow:
        workflow_id = f"workflow_{int(time.time() * 1000)}"
        workflow = IncrementalWorkflow(
            workflow_id=workflow_id,
            project_spec=project_spec,
            model=self.model,
            model_tier=self.model_tier,
            decomposer=self.decomposer,
            assembler=self.assembler,
            validator=self.validator,
        )
        workflow.task_graph = self.decomposer.decompose_project(project_spec)
        for task_id, task in workflow.task_graph.tasks.items():
            step = WorkflowStep(
                step_id=task_id,
                task=task,
                max_attempts=self._get_max_attempts_for_task(task),
            )
            workflow.steps[task_id] = step
        self.workflows[workflow_id] = workflow
        return workflow
    def execute_workflow(self, workflow_id: str) -> AssemblyResult:
        if workflow_id not in self.workflows:
            return AssemblyResult(
                success=False,
                code="",
                errors=[f"Workflow {workflow_id} not found"],
            )
        workflow = self.workflows[workflow_id]
        if not workflow.task_graph:
            return AssemblyResult(
                success=False,
                code="",
                errors=["No task graph available"],
            )
        while True:
            ready_steps = self._get_ready_steps(workflow)
            if not ready_steps:
                break
            for step in ready_steps:
                workflow.current_step = step.step_id
                step.status = "in_progress"
                step.started_at = time.time()
                result = self._execute_step(step, workflow)
                step.result = result
                step.completed_at = time.time()
                step.attempts += 1
                if result.success:
                    step.status = "completed"
                    workflow.completed_steps.append(step.step_id)
                    workflow.task_graph.mark_completed(step.step_id)
```

```
            else:
                if step.attempts >= step.max_attempts:
                    step.status = "failed"
                    workflow.failed_steps.append(step.step_id)
                    workflow.task_graph.mark_failed(step.step_id)
                    step.error = "\n".join(result.errors)
                else:
                    step.status = "retrying"
    return self.assembler.assemble_task_graph(workflow.task_graph)
def _get_ready_steps(self, workflow: IncrementalWorkflow) -> list[WorkflowStep]:
    ready = []
    for _, step in workflow.steps.items():
        if step.status in ["completed", "failed"]:
            continue
        if step.status == "in_progress":
            continue
        if step.status == "retrying" and step.attempts < step.max_attempts:
            deps_satisfied = all(
                dep_id in workflow.completed_steps for dep_id in step.task.dependencies
            )
            if deps_satisfied:
                ready.append(step)
            continue
        if step.status == "pending":
            deps_satisfied = all(
                dep_id in workflow.completed_steps for dep_id in step.task.dependencies
            )
            if deps_satisfied:
                ready.append(step)
    ready.sort(key=lambda s: s.task.estimated_lines)
    return ready[:1]
def _execute_step(self, step: WorkflowStep, workflow: IncrementalWorkflow) -> AssemblyResul
t:
    return self.assembler.assemble_atomic_task(step.task, workflow.task_graph)
def _get_max_attempts_for_task(self, task: AtomicTask) -> int:
    base_attempts = {
        ModelTier.TIER_1_ADVANCED: 2,
        ModelTier.TIER_2_CAPABLE: 3,
        ModelTier.TIER_3_MODERATE: 4,
        ModelTier.TIER_4_BASIC: 5,
        ModelTier.TIER_5_LIMITED: 8,
    }
    attempts = base_attempts[self.model_tier]
    complexity = estimate_task_complexity(task.component_spec)
    if complexity == TaskSize.LARGE:
        attempts += 2
    elif complexity == TaskSize.MEDIUM:
        attempts += 1
    return attempts
def get_workflow_status(self, workflow_id: str) -> dict[str, Any] | None:
    if workflow_id not in self.workflows:
        return None
    workflow = self.workflows[workflow_id]
    progress = workflow.get_progress()
    step_details = []
    for step_id, step in workflow.steps.items():
        step_details.append(
            {
                "step_id": step_id,
                "description": step.task.description,
                "status": step.status,
                "attempts": step.attempts,
                "max_attempts": step.max_attempts,
                "error": step.error,
            }
```

```
            )
        return {
            **progress,
            "steps": step_details,
            "model": workflow.model,
            "model_tier": workflow.model_tier.value,
        }
    def save_workflow_state(self, workflow_id: str) -> None:
        if workflow_id not in self.workflows:
            return
        workflow = self.workflows[workflow_id]
        state_file = self.project_root / ".openlaoke" / "workflows" / f"{workflow_id}.json"
        state_file.parent.mkdir(parents=True, exist_ok=True)
        state = {
            "workflow_id": workflow.workflow_id,
            "model": workflow.model,
            "model_tier": workflow.model_tier.value,
            "progress": workflow.get_progress(),
            "completed_steps": workflow.completed_steps,
            "failed_steps": workflow.failed_steps,
        }
        state_file.write_text(json.dumps(state, indent=2))
    def load_workflow_state(self, workflow_id: str) -> IncrementalWorkflow | None:
        state_file = self.project_root / ".openlaoke" / "workflows" / f"{workflow_id}.json"
        if not state_file.exists():
            return None
        state = json.loads(state_file.read_text())
        if workflow_id not in self.workflows:
            return None
        workflow = self.workflows[workflow_id]
        workflow.completed_steps = state.get("completed_steps", [])
        workflow.failed_steps = state.get("failed_steps", [])
        return workflow
def create_orchestrator_for_model(app_state: AppState, model: str) -> IncrementalOrchestrator:
    from openlaoke.core.model_assessment.assessor import ModelAssessor
    from openlaoke.types.providers import MultiProviderConfig
    config = MultiProviderConfig.defaults()
    assessor = ModelAssessor(config)
    tier = assessor.get_tier(model)
    return IncrementalOrchestrator(app_state, model, tier)
```

File: [27/252] openlaoke/core/atomic_generator.py

```
"""Atomic code generator for small models.
Generates code one atomic task at a time, ensuring each piece
is correct before moving to the next.
"""
from __future__ import annotations
import ast
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.core.architecture.decomposer import AtomicTask, TaskGraph
from openlaoke.core.code_validator import CodeValidator
from openlaoke.core.enhanced_knowledge_base import EnhancedKnowledgeBase
from openlaoke.core.intent_pipeline import IntentBasedPipeline
@dataclass
class CodeGenerationResult:
    success: bool
    code: str
    task_id: str
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    test_passed: bool = False
    retry_needed: bool = False
class AtomicCodeGenerator:
    def __init__(self, pipeline: IntentBasedPipeline) -> None:
        self.pipeline = pipeline
        self.validator = CodeValidator()
        self.knowledge_base = EnhancedKnowledgeBase()
        self.completed_code: dict[str, str] = {}
        self.task_attempts: dict[str, int] = {}
        self.max_attempts_per_task = 3
    def generate_code_for_task(
        self, task: AtomicTask, previous_code: dict[str, str] | None = None
    ) -> CodeGenerationResult:
        if previous_code is None:
            previous_code = {}
        if task.task_id not in self.task_attempts:
            self.task_attempts[task.task_id] = 0
        self.task_attempts[task.task_id] += 1
        if self.task_attempts[task.task_id] > self.max_attempts_per_task:
            return CodeGenerationResult(
                success=False,
                code="",
                task_id=task.task_id,
                errors=[
                    f"Max attempts ({self.max_attempts_per_task}) reached for task {task.task_i
    d}"
                ],
                retry_needed=False,
            )
        code = self._generate_from_template(task)
        is_valid, errors = self.pipeline.validate_task_result(task, code)
        if not is_valid:
            fixed_code = self._try_auto_fix(code, errors)
            if fixed_code:
                code = fixed_code
                is_valid, errors = self.pipeline.validate_task_result(task, code)
        return CodeGenerationResult(
            success=is_valid,
            code=code,
            task_id=task.task_id,
            errors=errors if not is_valid else [],
```

```
        warnings=[],
        test_passed=False,
        retry_needed=not is_valid
        and self.task_attempts[task.task_id] < self.max_attempts_per_task,
    )
def assemble_final_code(self, task_graph: TaskGraph, completed_code: dict[str, str]) -> str
:
    code_sections = []
    import_tasks = [
        tid for tid, task in task_graph.tasks.items() if "import" in task.description.lower
()
    ]
    for task_id in import_tasks:
        if task_id in completed_code:
            code_sections.append(completed_code[task_id])
    main_tasks = [
        tid
        for tid, task in task_graph.tasks.items()
        if "import" not in task.description.lower()
        and "test" not in task.description.lower()
        and "export" not in task.description.lower()
    ]
    for task_id in main_tasks:
        if task_id in completed_code:
            code_sections.append(completed_code[task_id])
    export_tasks = [
        tid for tid, task in task_graph.tasks.items() if "export" in task.description.lower
()
    ]
    for task_id in export_tasks:
        if task_id in completed_code:
            code_sections.append(completed_code[task_id])
    return "\n\n".join(code_sections)
def _build_generation_prompt(self, task: AtomicTask, previous_code: dict[str, str]) -> str:
    base_prompt = self.pipeline.generate_task_prompt(task)
    if previous_code:
        deps_code = []
        for dep_id in task.dependencies:
            if dep_id in previous_code:
                deps_code.append(f"# Dependency: {dep_id}\n{previous_code[dep_id]}")
        if deps_code:
            base_prompt += "\n\n# Previously completed code:\n" + "\n\n".join(deps_code)
    return base_prompt
def _generate_from_template(self, task: AtomicTask) -> str:
    template = task.template.template
    context = self._build_context(task)
    knowledge = self._get_relevant_knowledge(task)
    if knowledge:
        knowledge_comments = self._format_knowledge_as_comments(knowledge, task)
        context["implementation"] = knowledge_comments + "\n    pass"
        context["docstring"] = f"{task.description}\n\n{knowledge['content'][:200]}"
    try:
        code = template.format(**context)
    except KeyError:
        code = self._fill_template_manually(template, context)
    if knowledge and "def " in code:
        code = self._inject_knowledge_into_code(code, knowledge)
    code = code.strip()
    if not code.endswith("\n"):
        code += "\n"
    return code
def _get_relevant_knowledge(self, task: AtomicTask) -> dict[str, Any] | None:
    task_description = task.description.lower()
    keywords = []
    if any(word in task_description for word in ["benchmark", "cpu", "performance", "measur
```

```
e"]):
        keywords.extend(["benchmark", "cpu", "performance"])
    if any(
        word in task_description for word in ["calculate", "compute", "math", "sum", "avera
ge"]
    ):
        keywords.extend(["basics", "syntax"])
    if any(word in task_description for word in ["file", "read", "write", "json"]):
        keywords.extend(["file", "io", "json"])
    if any(word in task_description for word in ["class", "struct", "object"]):
        keywords.extend(["class", "struct", "basics"])
    if any(word in task_description for word in ["test", "validate", "check"]):
        keywords.extend(["test", "validation"])
    if any(word in task_description for word in ["error", "exception", "handle"]):
        keywords.extend(["error", "exception"])
    for keyword in keywords:
        snippets = self.knowledge_base.search(keyword)
        if snippets:
            return {
                "topic": snippets[0].topic,
                "content": snippets[0].content,
                "source": snippets[0].source,
                "tags": snippets[0].tags,
            }
    python_keywords = ["function", "method", "def", "import", "loop", "for", "if"]
    if any(kw in task_description for kw in python_keywords):
        snippets = self.knowledge_base.search("python_basics")
        if snippets:
            return {
                "topic": snippets[0].topic,
                "content": snippets[0].content,
                "source": snippets[0].source,
                "tags": snippets[0].tags,
            }
    return None
def _format_knowledge_as_comments(self, knowledge: dict[str, Any], task: AtomicTask) -> str
:
    content = knowledge.get("content", "")
    lines = content.strip().split("\n")
    relevant_lines = []
    for line in lines:
        if any(
            keyword in line.lower()
            for keyword in ["example", "def ", "class ", "import ", "for ", "if ", "return"
]
        ):
            relevant_lines.append(line)
        if len(relevant_lines) >= 5:
            break
    if relevant_lines:
        return "    # " + "\n    # ".join(relevant_lines)
    return "    # Implementation needed"
def _inject_knowledge_into_code(self, code: str, knowledge: dict[str, Any]) -> str:
    lines = code.split("\n")
    injected_lines = []
    function_indent = 0
    for i, line in enumerate(lines):
        injected_lines.append(line)
        if "def " in line and line.strip().endswith(":"):
            function_indent = len(line) - len(line.lstrip())
            next_idx = i + 1
            if next_idx < len(lines):
                next_line = lines[next_idx]
                next_indent = len(next_line) - len(next_line.lstrip())
                if next_indent > function_indent and not next_line.strip().startswith('"""'
```

```
):
                    knowledge_hint = self._extract_key_points(knowledge)
                    if knowledge_hint:
                        injected_lines.append(
                            " " * (function_indent + 4) + f"# {knowledge_hint}"
                        )
    return "\n".join(injected_lines)
def _extract_key_points(self, knowledge: dict[str, Any]) -> str:
    content = knowledge.get("content", "")
    if "def " in content:
        match = content.find("def ")
        if match != -1:
            example_line = content[match : match + 100].split("\n")[0]
            return f"Example: {example_line.strip()}"
    if "import " in content:
        imports = [line.strip() for line in content.split("\n") if "import " in line][:2]
        if imports:
            return f"Use: {', '.join(imports)}"
    return f"Reference: {knowledge.get('source', 'Documentation')}"
def _build_context(self, task: AtomicTask) -> dict[str, Any]:
    context = {
        "function_name": task.component_spec.name,
        "class_name": task.component_spec.name,
        "parameters": "",
        "return_type": "Any",
        "return_value": "result",
        "docstring": task.description,
        "implementation": "pass",
        "args_doc": "Arguments to be documented",
        "return_doc": "Return value",
        "raises_doc": "Possible exceptions",
        "init_params": "",
        "init_implementation": "pass",
        "methods": "pass",
        "attrs_doc": "Attributes",
        "imports": "import time",
        "exports": "'__all__'",
        "arrange_code": "# Arrange",
        "act_code": "# Act",
        "assert_code": "# Assert",
        "test_case": "basic",
        "test_description": "Basic test case",
        "fields": "field: str = Field(description='Example field')",
        "validators": "@validator('field')\ndef check_field(cls, v):\n    return v",
        "path": "/path",
        "method": "get",
        "response_model": "dict",
        "error_doc": "Error description",
    }
    if task.component_spec.api_spec:
        api = task.component_spec.api_spec
        params = []
        for param_name, param_spec in api.input_schema.get("properties", {}).items():
            param_type = param_spec.get("type", "Any")
            params.append(f"{param_name}: {param_type}")
        context["parameters"] = ", ".join(params)
        output_type = api.output_schema.get("type", "Any")
        context["return_type"] = output_type
    return context
def _fill_template_manually(self, template: str, context: dict[str, Any]) -> str:
    import re
    result = template
    for key, value in context.items():
        placeholder = "{" + key + "}"
        result = result.replace(placeholder, str(value))
```

```
        result = re.sub(r"\{[^}]+\}", "# TODO: Implement", result)
        return result
    def _try_auto_fix(self, code: str, errors: list[str]) -> str | None:
        fixed_code = code
        for error in errors:
            if "Missing return type hint" in error:
                lines = code.split("\n")
                for i, line in enumerate(lines):
                    if line.strip().startswith("def ") and " -> " not in line:
                        indent = len(line) - len(line.lstrip())
                        if "(" in line and ")" in line:
                            fixed_line = line.rstrip() + " -> Any:\n"
                            lines[i] = fixed_line
                fixed_code = "\n".join(lines)
            elif "Missing docstring" in error:
                lines = code.split("\n")
                for i, line in enumerate(lines):
                    if line.strip().startswith("def ") or line.strip().startswith("class "):
                        indent = len(line) - len(line.lstrip())
                        docstring = (
                            " " * (indent + 4)
                            + '"""\n'
                            + " " * (indent + 4)
                            + "TODO: Add docstring.\n"
                            + " " * (indent + 4)
                            + '"""\n'
                        )
                        lines.insert(i + 1, docstring)
                        break
                fixed_code = "\n".join(lines)
        try:
            ast.parse(fixed_code)
            return fixed_code
        except SyntaxError:
            return None
def create_generator_for_tier(
    model_tier: Any, working_directory: Path | None = None
) -> AtomicCodeGenerator:
    from openlaoke.core.model_assessment.types import ModelTier
    tier = model_tier if isinstance(model_tier, ModelTier) else ModelTier.TIER_5_LIMITED
    pipeline = IntentBasedPipeline(tier, working_directory)
    return AtomicCodeGenerator(pipeline)
```

File: [28/252] openlaoke/core/atomic_generator_api.py

```
"""Atomic code generator with real API calls for small models.
This version actually calls the model API to generate code,
instead of just filling templates with placeholders.
"""
from __future__ import annotations
import ast
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
from openlaoke.core.architecture.decomposer import AtomicTask, TaskGraph
from openlaoke.core.code_validator import CodeValidator
from openlaoke.core.enhanced_knowledge_base import EnhancedKnowledgeBase
from openlaoke.core.intent_pipeline import IntentBasedPipeline
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class CodeGenerationResult:
    success: bool
    code: str
    task_id: str
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    test_passed: bool = False
    retry_needed: bool = False
class AtomicCodeGeneratorWithAPI:
    """Generate code by calling model API for each atomic task."""
    def __init__(
        self,
        pipeline: IntentBasedPipeline,
        app_state: AppState | None = None,
    ) -> None:
        self.pipeline = pipeline
        self.app_state = app_state
        self.validator = CodeValidator()
        self.knowledge_base = EnhancedKnowledgeBase()
        self.completed_code: dict[str, str] = {}
        self.task_attempts: dict[str, int] = {}
        self.max_attempts_per_task = 3
        self._api_client = None
    async def _get_api_client(self):
        """Get or create API client."""
        if self._api_client is None:
            from openlaoke.core.multi_provider_api import MultiProviderClient
            if self.app_state and self.app_state.multi_provider_config:
                self._api_client = MultiProviderClient(self.app_state.multi_provider_config)
            else:
                from openlaoke.types.providers import MultiProviderConfig
                config = MultiProviderConfig.defaults()
                self._api_client = MultiProviderClient(config)
        return self._api_client
    async def generate_code_for_task_async(
        self, task: AtomicTask, previous_code: dict[str, str] | None = None
    ) -> CodeGenerationResult:
        """Generate code for a task by calling the model API."""
        if previous_code is None:
            previous_code = {}
        if task.task_id not in self.task_attempts:
            self.task_attempts[task.task_id] = 0
        self.task_attempts[task.task_id] += 1
        if self.task_attempts[task.task_id] > self.max_attempts_per_task:
            return CodeGenerationResult(
```

```
            success=False,
            code="",
            task_id=task.task_id,
            errors=[
                f"Max attempts ({self.max_attempts_per_task}) reached for task {task.task_i
d}"
            ],
            retry_needed=False,
        )
    try:
        code = await self._generate_with_api(task, previous_code)
        is_valid, errors = self.pipeline.validate_task_result(task, code)
        if not is_valid:
            fixed_code = self._try_auto_fix(code, errors)
            if fixed_code:
                code = fixed_code
                is_valid, errors = self.pipeline.validate_task_result(task, code)
        return CodeGenerationResult(
            success=is_valid,
            code=code,
            task_id=task.task_id,
            errors=errors if not is_valid else [],
            warnings=[],
            test_passed=False,
            retry_needed=not is_valid
            and self.task_attempts[task.task_id] < self.max_attempts_per_task,
        )
    except Exception as e:
        return CodeGenerationResult(
            success=False,
            code="",
            task_id=task.task_id,
            errors=[f"API call failed: {str(e)}"],
            retry_needed=self.task_attempts[task.task_id] < self.max_attempts_per_task,
        )
async def _generate_with_api(self, task: AtomicTask, previous_code: dict[str, str]) -> str:
    """Generate code by calling the model API."""
    api_client = await self._get_api_client()
    model = "gemma3:1b"
    if self.app_state and self.app_state.multi_provider_config:
        model = self.app_state.multi_provider_config.get_active_model()
    system_prompt = self._build_system_prompt(task)
    user_prompt = self._build_user_prompt(task, previous_code)
    messages = [{"role": "user", "content": user_prompt}]
    try:
        response_msg, token_usage, _ = await api_client.send_message(
            system_prompt=system_prompt,
            messages=messages,
            model=model,
            max_tokens=800,
        )
        if hasattr(response_msg, "content"):
            response_text = response_msg.content
        else:
            response_text = str(response_msg)
        code = self._extract_code_from_response(response_text)
        if not code:
            return self._generate_from_template(task)
        return code
    except Exception:
        return self._generate_from_template(task)
def _build_system_prompt(self, task: AtomicTask) -> str:
    """Build system prompt for code generation."""
    tier_limits = self._get_tier_limits()
    prompt = f"""You are an expert Python code generator. Your task is to generate EXACTLY
```

```
    ONE small, atomic piece of code.
CRITICAL CONSTRAINTS:
- Maximum {tier_limits["max_lines"]} lines of code (excluding imports/docstrings)
- Maximum {tier_limits["max_params"]} parameters per function
- Maximum complexity score: {tier_limits["max_complexity"]}
- You MUST generate WORKING, COMPLETE code - NO placeholders like "pass" or "TODO"
- You MUST include proper imports at the top
- You MUST include a brief docstring
- Return ONLY the code, no explanations
Current task: {task.description}
Estimated lines: {task.estimated_lines}
Generate a complete, working implementation now."""
        return prompt
    def _build_user_prompt(self, task: AtomicTask, previous_code: dict[str, str]) -> str:
        """Build user prompt with task details and context."""
        prompt_parts = [f"Task: {task.task_id}", f"Description: {task.description}"]
        if task.component_spec:
            spec = task.component_spec
            prompt_parts.append(f"Component type: {spec.component_type}")
            prompt_parts.append(f"Component name: {spec.name}")
            if spec.api_spec:
                prompt_parts.append(f"API spec: {spec.api_spec}")
        if previous_code and task.dependencies:
            prompt_parts.append("\nPreviously completed code (use as reference):")
            for dep_id in task.dependencies:
                if dep_id in previous_code:
                    prompt_parts.append(f"\n--- {dep_id} ---")
                    prompt_parts.append(previous_code[dep_id][:200])
        knowledge = self._get_relevant_knowledge(task)
        if knowledge:
            prompt_parts.append("\nReference knowledge (use similar patterns):")
            prompt_parts.append(knowledge["content"][:300])
        prompt_parts.append(
            "\nGenerate the complete code now. Return ONLY code, no markdown formatting."
        )
        return "\n".join(prompt_parts)
    def _get_tier_limits(self) -> dict[str, int]:
        """Get limits based on model tier."""
        from openlaoke.core.model_assessment.types import ModelTier
        tier = self.pipeline.model_tier
        if tier == ModelTier.TIER_5_LIMITED:
            return {
                "max_lines": 15,
                "max_params": 3,
                "max_complexity": 5,
            }
        elif tier == ModelTier.TIER_4_BASIC:
            return {
                "max_lines": 30,
                "max_params": 5,
                "max_complexity": 8,
            }
        else:
            return {
                "max_lines": 100,
                "max_params": 10,
                "max_complexity": 20,
            }
    def _extract_code_from_response(self, response: str) -> str:
        """Extract Python code from model response."""
        lines = response.strip().split("\n")
        code_lines = []
        in_code_block = False
        for _, line in enumerate(lines):
            if line.strip().startswith("```python"):
```

```
            in_code_block = True
            continue
        elif line.strip() == "```" and in_code_block:
            in_code_block = False
            continue
        elif in_code_block:
            code_lines.append(line)
    if code_lines:
        return "\n".join(code_lines)
    if "def " in response or "import " in response:
        code_lines = []
        for line in lines:
            stripped = line.strip()
            if (
                stripped.startswith("def ")
                or stripped.startswith("import ")
                or stripped.startswith("class ")
                or stripped.startswith("from ")
                or stripped.startswith("@")
                or stripped.startswith("return ")
                or stripped.startswith("#")
                or stripped.startswith('"""')
                or stripped.startswith("'''")
                or ":" in stripped
                or (code_lines and (line.startswith("    ") or line.startswith("\t")))
            ):
                code_lines.append(line)
        if code_lines:
            return "\n".join(code_lines)
    return ""
def _generate_from_template(self, task: AtomicTask) -> str:
    """Fallback: generate code from template."""
    template = task.template.template
    context = self._build_context(task)
    try:
        code = template.format(**context)
    except KeyError:
        code = self._fill_template_manually(template, context)
    code = code.strip()
    if not code.endswith("\n"):
        code += "\n"
    return code
def _get_relevant_knowledge(self, task: AtomicTask) -> dict[str, Any] | None:
    """Get relevant knowledge snippets."""
    task_description = task.description.lower()
    keywords = []
    if any(word in task_description for word in ["benchmark", "cpu", "performance", "measur
e"]):
        keywords.extend(["benchmark", "cpu", "performance"])
    if any(
        word in task_description for word in ["calculate", "compute", "math", "sum", "avera
ge"]
    ):
        keywords.extend(["basics", "syntax"])
    if any(word in task_description for word in ["file", "read", "write", "json"]):
        keywords.extend(["file", "io", "json"])
    if any(word in task_description for word in ["storage", "disk", "capacity", "size"]):
        keywords.extend(["file", "system", "os"])
    for keyword in keywords:
        snippets = self.knowledge_base.search(keyword)
        if snippets:
            return {
                "topic": snippets[0].topic,
                "content": snippets[0].content,
                "source": snippets[0].source,
```

```
                "tags": snippets[0].tags,
            }
    return None
def assemble_final_code(self, task_graph: TaskGraph, completed_code: dict[str, str]) -> str
:
    """Assemble final code from completed tasks."""
    code_sections = []
    import_tasks = [
        tid
        for tid, task in task_graph.tasks.items()
        if "import" in task.description.lower() or "import" in task.task_id.lower()
    ]
    for task_id in import_tasks:
        if task_id in completed_code:
            code_sections.append(completed_code[task_id])
    main_tasks = [
        tid
        for tid, task in task_graph.tasks.items()
        if "import" not in task.description.lower()
        and "import" not in task.task_id.lower()
        and "test" not in task.description.lower()
        and "test" not in task.task_id.lower()
    ]
    for task_id in main_tasks:
        if task_id in completed_code:
            code_sections.append(completed_code[task_id])
    return "\n\n".join(code_sections)
def _build_context(self, task: AtomicTask) -> dict[str, Any]:
    """Build template context."""
    context = {
        "function_name": task.component_spec.name if task.component_spec else "func",
        "class_name": task.component_spec.name if task.component_spec else "Class",
        "parameters": "",
        "return_type": "Any",
        "return_value": "result",
        "docstring": task.description,
        "implementation": "pass",
        "args_doc": "Arguments to be documented",
        "return_doc": "Return value",
        "raises_doc": "Possible exceptions",
        "imports": "import time",
    }
    if task.component_spec and task.component_spec.api_spec:
        api = task.component_spec.api_spec
        params = []
        for param_name, param_spec in api.input_schema.get("properties", {}).items():
            param_type = param_spec.get("type", "Any")
            params.append(f"{param_name}: {param_type}")
        context["parameters"] = ", ".join(params)
        output_type = api.output_schema.get("type", "Any")
        context["return_type"] = output_type
    return context
def _fill_template_manually(self, template: str, context: dict[str, Any]) -> str:
    """Fill template manually."""
    import re
    result = template
    for key, value in context.items():
        placeholder = "{" + key + "}"
        result = result.replace(placeholder, str(value))
    result = re.sub(r"\{[^}]+\}", "", result)
    return result
def _try_auto_fix(self, code: str, errors: list[str]) -> str | None:
    """Try to auto-fix common issues."""
    fixed_code = code
    for error in errors:
```

```
            if "Missing return type hint" in error:
                lines = code.split("\n")
                for i, line in enumerate(lines):
                    if line.strip().startswith("def ") and " -> " not in line:
                        len(line) - len(line.lstrip())
                        if "(" in line and ")" in line:
                            fixed_line = line.rstrip() + " -> Any:\n"
                            lines[i] = fixed_line
                fixed_code = "\n".join(lines)
        try:
            ast.parse(fixed_code)
            return fixed_code
        except SyntaxError:
            return None
def create_generator_with_api(
    model_tier: Any,
    working_directory: Path | None = None,
    app_state: AppState | None = None,
) -> AtomicCodeGeneratorWithAPI:
    """Create generator with API support."""
    from openlaoke.core.model_assessment.types import ModelTier
    tier = model_tier if isinstance(model_tier, ModelTier) else ModelTier.TIER_5_LIMITED
    pipeline = IntentBasedPipeline(tier, working_directory)
    return AtomicCodeGeneratorWithAPI(pipeline, app_state)
```

File: [29/252] openlaoke/core/autocomplete.py

```
"""
OpenLaoKe ??????
?? OpenCode ?????
???
1. `/` ????/????
2. `@` ????/????
3. ????
4. ????????
5. Tab/Enter ???Esc ??
"""
from __future__ import annotations
import os
import readline
from collections.abc import Callable
from dataclasses import dataclass
from pathlib import Path
from openlaoke.commands.registry import get_all_commands
from openlaoke.core.skill_system import list_available_skills, load_skill
@dataclass
class AutocompleteOption:
    """????"""
    display: str
    value: str | None = None
    description: str = ""
    is_skill: bool = False
    is_command: bool = False
    is_file: bool = False
    is_directory: bool = False
    on_select: Callable | None = None
class AutocompleteState:
    """????"""
    def __init__(self):
        self.visible = False
        self.mode = ""  # "/" ? "@"
        self.options: list[AutocompleteOption] = []
        self.selected = 0
        self.search_text = ""
        self.trigger_index = 0
    def reset(self):
        self.visible = False
        self.mode = ""
        self.options = []
        self.selected = 0
        self.search_text = ""
        self.trigger_index = 0
class AutocompleteManager:
    """???????"""
    def __init__(self):
        self.state = AutocompleteState()
        self._commands_cache: list[AutocompleteOption] | None = None
        self._skills_cache: list[AutocompleteOption] | None = None
        self._cwd = os.getcwd()
    def _get_commands(self) -> list[AutocompleteOption]:
        """????????"""
        if self._commands_cache is not None:
            return self._commands_cache
        options = []
        seen = set()
        for cmd in get_all_commands():
            if cmd.hidden:
                continue
```

```
        cmd_name = cmd.name
        if cmd_name in seen:
            continue
        seen.add(cmd_name)
        options.append(
            AutocompleteOption(
                display=f"/{cmd_name}",
                value=cmd_name,
                description=cmd.description,
                is_command=True,
            )
        )
        for alias in cmd.aliases:
            if alias not in seen:
                seen.add(alias)
                options.append(
                    AutocompleteOption(
                        display=f"/{alias}",
                        value=alias,
                        description=f"Alias for /{cmd_name}",
                        is_command=True,
                    )
                )
    self._commands_cache = sorted(options, key=lambda x: x.display)
    return self._commands_cache
def _get_skills(self) -> list[AutocompleteOption]:
    """????????"""
    if self._skills_cache is not None:
        return self._skills_cache
    options = []
    for skill_name in list_available_skills():
        skill = load_skill(skill_name)
        if skill:
            desc = ""
            if skill.description:
                # ?????80???
                desc = skill.description[:80]
                if len(skill.description) > 80:
                    desc += "..."
            options.append(
                AutocompleteOption(
                    display=f"/{skill_name}",
                    value=skill_name,
                    description=desc,
                    is_skill=True,
                )
            )
    self._skills_cache = sorted(options, key=lambda x: x.display)
    return self._skills_cache
def _get_slash_options(self) -> list[AutocompleteOption]:
    """???? / ???????? + ???"""
    commands = self._get_commands()
    skills = self._get_skills()
    # ?????
    all_options = {}
    for opt in commands + skills:
        if opt.display not in all_options:
            all_options[opt.display] = opt
    return sorted(all_options.values(), key=lambda x: x.display)
def _fuzzy_match(
    self, query: str, options: list[AutocompleteOption]
) -> list[AutocompleteOption]:
    """????"""
    if not query:
        return options
```

```
    query_lower = query.lower()
    scored = []
    for opt in options:
        display_lower = opt.display.lower()
        value_lower = (opt.value or "").lower()
        desc_lower = opt.description.lower()
        score = 0
        # ??????????
        if display_lower.startswith("/" + query_lower):
            score = 100
        # ???????
        elif query_lower in display_lower:
            score = 80
        elif query_lower in value_lower:
            score = 60
        # ?????
        elif query_lower in desc_lower:
            score = 40
        # ????
        elif self._fuzzy_contains(query_lower, display_lower):
            score = 20
        if score > 0:
            scored.append((score, opt))
    # ?????
    scored.sort(key=lambda x: (-x[0], x[1].display))
    return [opt for _, opt in scored[:10]]
def _fuzzy_contains(self, query: str, text: str) -> bool:
    """??????"""
    query_idx = 0
    for char in text:
        if query_idx < len(query) and char == query[query_idx]:
            query_idx += 1
    return query_idx == len(query)
def start_completion(self, mode: str, cursor_pos: int, text: str):
    """????"""
    self.state.visible = True
    self.state.mode = mode
    self.state.trigger_index = cursor_pos - 1  # ???????
    # ??????
    if mode == "/":
        # ?? / ?????
        search_text = text[1:cursor_pos]
        all_options = self._get_slash_options()
    else:  # @
        # ?? @ ?????
        at_pos = text.rfind("@", 0, cursor_pos)
        search_text = text[at_pos + 1 : cursor_pos]
        all_options = self._get_file_options(search_text)
    self.state.search_text = search_text
    self.state.options = self._fuzzy_match(search_text, all_options)
    self.state.selected = 0
def update_search(self, cursor_pos: int, text: str):
    """????"""
    if not self.state.visible:
        return
    # ??????????
    trigger_pos = self.state.trigger_index
    # ??????????
    if cursor_pos <= trigger_pos:
        self.state.reset()
        return
    # ???????????/??????
    text_between = text[trigger_pos:cursor_pos]
    if " " in text_between and self.state.mode == "/" and len(text_between.strip().split())
 > 1:
```

```
        # ?? / ??????????????????
        self.state.reset()
        return
    # ??????
    search_text = text[trigger_pos + 1 : cursor_pos]
    self.state.search_text = search_text
    # ??????
    if self.state.mode == "/":
        all_options = self._get_slash_options()
    else:
        all_options = self._get_file_options(search_text)
    self.state.options = self._fuzzy_match(search_text, all_options)
    self.state.selected = 0
def _get_file_options(self, query: str) -> list[AutocompleteOption]:
    """??????"""
    options = []
    try:
        cwd = Path(self._cwd)
        # ??????????
        if "/" in query:
            dir_part = query.rsplit("/", 1)[0]
            file_prefix = query.rsplit("/", 1)[1] if "/" in query else query
            search_dir = cwd / dir_part if dir_part else cwd
        else:
            search_dir = cwd
            file_prefix = query
        if not search_dir.exists():
            return options
        for item in sorted(search_dir.iterdir(), key=lambda x: (not x.is_dir(), x.name)):
            if item.name.startswith("."):
                continue
            if file_prefix and not item.name.lower().startswith(file_prefix.lower()):
                continue
            is_dir = item.is_dir()
            display_name = item.name + ("/" if is_dir else "")
            options.append(
                AutocompleteOption(
                    display=display_name,
                    value=display_name,
                    description=f"{'Directory' if is_dir else 'File'}",
                    is_file=not is_dir,
                    is_directory=is_dir,
                )
            )
    except Exception:
        pass
    return options[:10]
def move_selection(self, direction: int):
    """????"""
    if not self.state.visible or not self.state.options:
        return
    self.state.selected += direction
    if self.state.selected < 0:
        self.state.selected = len(self.state.options) - 1
    elif self.state.selected >= len(self.state.options):
        self.state.selected = 0
def get_selected(self) -> AutocompleteOption | None:
    """???????"""
    if not self.state.visible or not self.state.options:
        return None
    if 0 <= self.state.selected < len(self.state.options):
        return self.state.options[self.state.selected]
    return None
def get_completion_text(self) -> str | None:
    """??????"""
```

```
        selected = self.get_selected()
        if not selected:
            return None
        if self.state.mode == "/":
            # ??????????????
            return selected.value or selected.display.lstrip("/")
        else:
            # ??????????
            return selected.value or selected.display
    def get_display_lines(self, max_width: int = 60) -> list[str]:
        """?????"""
        if not self.state.visible or not self.state.options:
            return []
        lines = []
        for i, opt in enumerate(self.state.options):
            prefix = "-> " if i == self.state.selected else "  "
            # ??????
            display = opt.display
            if len(display) > max_width - 20:
                display = display[: max_width - 23] + "..."
            # ????
            if opt.description:
                desc = opt.description
                remaining = max_width - len(display) - 4
                if remaining > 10:
                    if len(desc) > remaining:
                        desc = desc[: remaining - 3] + "..."
                    line = f"{prefix}{display}  [dim]{desc}[/dim]"
                else:
                    line = f"{prefix}{display}"
            else:
                line = f"{prefix}{display}"
            lines.append(line)
        return lines
# ????
_manager: AutocompleteManager | None = None
def get_autocomplete_manager() -> AutocompleteManager:
    """??????????"""
    global _manager
    if _manager is None:
        _manager = AutocompleteManager()
    return _manager
def setup_readline_completion():
    """?? readline ? Tab ??"""
    manager = get_autocomplete_manager()
    def complete(text: str, state: int):
        if state == 0:
            # ???????????
            manager.state.reset()
            # ????? / ??
            if text.startswith("/"):
                manager.start_completion("/", len(text), text)
        # ?????
        if manager.state.visible and manager.state.options and state < len(manager.state.option
    s):
            opt = manager.state.options[state]
            return opt.display
        return None
    readline.set_completer(complete)
    readline.parse_and_bind("tab: complete")
    readline.set_completer_delims(" \t\n")
    return manager
```

File: [30/252] openlaoke/core/code_templates.py

```
"""Code templates and project scaffolds for quick development.
Provides ready-to-use templates for common development scenarios
across multiple programming languages.
"""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
class ProjectType(StrEnum):
    CLI_TOOL = "cli_tool"
    WEB_API = "web_api"
    WEB_APP = "web_app"
    GUI_APP = "gui_app"
    LIBRARY = "library"
    DATA_PROCESSING = "data_processing"
    TESTING = "testing"
    DATABASE = "database"
    MICROSERVICE = "microservice"
    SCRIPT = "script"
class Language(StrEnum):
    PYTHON = "python"
    JAVASCRIPT = "javascript"
    TYPESCRIPT = "typescript"
    RUST = "rust"
    GO = "go"
    JAVA = "java"
    CPP = "cpp"
@dataclass
class CodeTemplate:
    name: str
    description: str
    language: Language
    project_type: ProjectType
    files: dict[str, str]
    dependencies: list[str] = field(default_factory=list)
    setup_commands: list[str] = field(default_factory=list)
    readme: str = ""
PROJECT_TEMPLATES: dict[str, CodeTemplate] = {
    "python_cli_tool": CodeTemplate(
        name="Python CLI Tool",
        description="Command-line tool with argument parsing, logging, and config",
        language=Language.PYTHON,
        project_type=ProjectType.CLI_TOOL,
        dependencies=["click>=8.0", "rich>=13.0"],
        setup_commands=["pip install click rich"],
        files={
            "main.py": '''"""Main entry point for CLI tool."""
from __future__ import annotations
import logging
from pathlib import Path
from typing import Any
import click
from rich.console import Console
console = Console()
logger = logging.getLogger(__name__)
@click.command()
@click.option("--input", "-i", required=True, help="Input file path")
@click.option("--output", "-o", default="output.txt", help="Output file path")
@click.option("--verbose", "-v", is_flag=True, help="Enable verbose output")
@click.version_option(version="1.0.0")
def main(input: str, output: str, verbose: bool) -> None:
    """Process input file and generate output."""
```

```
    setup_logging(verbose)
    input_path = Path(input)
    if not input_path.exists():
        console.print(f"[red]Error: Input file not found: {input}[/red]")
        raise SystemExit(1)
    console.print(f"[green]Processing: {input_path}[/green]")
    result = process_file(input_path)
    output_path = Path(output)
    output_path.write_text(result)
    console.print(f"[green]Output saved to: {output_path}[/green]")
def setup_logging(verbose: bool) -> None:
    """Configure logging."""
    level = logging.DEBUG if verbose else logging.INFO
    logging.basicConfig(
        level=level,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
    )
def process_file(input_path: Path) -> str:
    """Process input file and return result."""
    content = input_path.read_text()
    logger.info(f"Read {len(content)} characters from {input_path}")
    result = content.upper()
    return result
if __name__ == "__main__":
    main()
''',
            "config.py": '''"""Configuration management."""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from typing import Any
import json
@dataclass
class Config:
    """Application configuration."""
    input_dir: Path = Path(".")
    output_dir: Path = Path("output")
    max_workers: int = 4
    timeout: int = 30
    @classmethod
    def from_file(cls, config_path: Path) -> Config:
        """Load configuration from JSON file."""
        if not config_path.exists():
            return cls()
        with open(config_path) as f:
            data = json.load(f)
        return cls(
            input_dir=Path(data.get("input_dir", ".")),
            output_dir=Path(data.get("output_dir", "output")),
            max_workers=data.get("max_workers", 4),
            timeout=data.get("timeout", 30),
        )
    def to_file(self, config_path: Path) -> None:
        """Save configuration to JSON file."""
        data = {
            "input_dir": str(self.input_dir),
            "output_dir": str(self.output_dir),
            "max_workers": self.max_workers,
            "timeout": self.timeout,
        }
        with open(config_path, "w") as f:
            json.dump(data, f, indent=2)
''',
            "utils.py": '''"""Utility functions."""
from __future__ import annotations
```

```
import hashlib
import time
from pathlib import Path
from typing import Any
def calculate_file_hash(file_path: Path) -> str:
    """Calculate MD5 hash of a file."""
    md5_hash = hashlib.md5()
    with open(file_path, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b""):
            md5_hash.update(chunk)
    return md5_hash.hexdigest()
def timing_decorator(func: Any) -> Any:
    """Decorator to measure function execution time."""
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.2f} seconds")
        return result
    return wrapper
''',
            "pyproject.toml": """[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
[project]
name = "cli-tool"
version = "1.0.0"
description = "CLI tool for file processing"
requires-python = ">=3.10"
dependencies = [
    "click>=8.0",
    "rich>=13.0",
]
[project.scripts]
cli-tool = "main:main"
[tool.ruff]
line-length = 100
target-version = "py310"
""",
        },
        readme="""# CLI Tool
A command-line tool for file processing.
## Installation
```bash
pip install -e .
```
Usage
```bash
cli-tool --input file.txt --output result.txt
cli-tool -i file.txt -o result.txt -v
```
Features
- Argument parsing with Click
- Rich console output
- Logging support
- Configuration management
- Utility functions
""",
    ),
    "python_web_api": CodeTemplate(
        name="Python Web API",
        description="RESTful API with FastAPI, models, and error handling",
        language=Language.PYTHON,
        project_type=ProjectType.WEB_API,
        dependencies=["fastapi>=0.100", "uvicorn>=0.23", "pydantic>=2.0"],
```

```
        setup_commands=["pip install fastapi uvicorn pydantic"],
        files={
            "main.py": '''"""FastAPI application entry point."""
from __future__ import annotations
from contextlib import asynccontextmanager
from typing import Any
from fastapi import FastAPI, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from models import Item, ItemCreate, ItemUpdate
from database import Database
@asynccontextmanager
async def lifespan(app: FastAPI):
    """Application lifespan manager."""
    app.state.db = Database()
    yield
    app.state.db.close()
app = FastAPI(
    title="API Server",
    description="RESTful API with FastAPI",
    version="1.0.0",
    lifespan=lifespan,
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
@app.get("/")
async def root() -> dict[str, str]:
    """Root endpoint."""
    return {"message": "API is running"}
@app.get("/items", response_model=list[Item])
async def list_items(skip: int = 0, limit: int = 100) -> list[Item]:
    """List all items."""
    return app.state.db.get_items(skip=skip, limit=limit)
@app.get("/items/{item_id}", response_model=Item)
async def get_item(item_id: int) -> Item:
    """Get item by ID."""
    item = app.state.db.get_item(item_id)
    if not item:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Item {item_id} not found"
        )
    return item
@app.post("/items", response_model=Item, status_code=status.HTTP_201_CREATED)
async def create_item(item: ItemCreate) -> Item:
    """Create a new item."""
    return app.state.db.create_item(item)
@app.put("/items/{item_id}", response_model=Item)
async def update_item(item_id: int, item: ItemUpdate) -> Item:
    """Update an item."""
    updated = app.state.db.update_item(item_id, item)
    if not updated:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Item {item_id} not found"
        )
    return updated
@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
async def delete_item(item_id: int) -> None:
    """Delete an item."""
    if not app.state.db.delete_item(item_id):
```

```
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Item {item_id} not found"
        )
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
''',
            "models.py": '''"""Pydantic models for API."""
from __future__ import annotations
from datetime import datetime
from typing import Any
from pydantic import BaseModel, ConfigDict, Field
class ItemBase(BaseModel):
    """Base item model."""
    name: str = Field(..., min_length=1, max_length=100)
    description: str | None = Field(None, max_length=500)
    price: float = Field(..., gt=0)
    is_active: bool = True
class ItemCreate(ItemBase):
    """Model for creating items."""
    pass
class ItemUpdate(BaseModel):
    """Model for updating items."""
    name: str | None = Field(None, min_length=1, max_length=100)
    description: str | None = Field(None, max_length=500)
    price: float | None = Field(None, gt=0)
    is_active: bool | None = None
class Item(ItemBase):
    """Complete item model."""
    model_config = ConfigDict(from_attributes=True)
    id: int
    created_at: datetime
    updated_at: datetime
''',
            "database.py": '''"""In-memory database implementation."""
from __future__ import annotations
from datetime import datetime
from typing import Any
from models import Item, ItemCreate, ItemUpdate
class Database:
    """In-memory database."""
    def __init__(self) -> None:
        self._items: dict[int, Item] = {}
        self._next_id = 1
    def get_items(self, skip: int = 0, limit: int = 100) -> list[Item]:
        """Get all items with pagination."""
        items = list(self._items.values())
        return items[skip:skip + limit]
    def get_item(self, item_id: int) -> Item | None:
        """Get item by ID."""
        return self._items.get(item_id)
    def create_item(self, item: ItemCreate) -> Item:
        """Create a new item."""
        now = datetime.now()
        new_item = Item(
            id=self._next_id,
            name=item.name,
            description=item.description,
            price=item.price,
            is_active=item.is_active,
            created_at=now,
            updated_at=now,
        )
        self._items[self._next_id] = new_item
```

```
        self._next_id += 1
        return new_item
    def update_item(self, item_id: int, item: ItemUpdate) -> Item | None:
        """Update an item."""
        if item_id not in self._items:
            return None
        existing = self._items[item_id]
        update_data = item.model_dump(exclude_unset=True)
        updated_item = Item(
            id=existing.id,
            name=update_data.get("name", existing.name),
            description=update_data.get("description", existing.description),
            price=update_data.get("price", existing.price),
            is_active=update_data.get("is_active", existing.is_active),
            created_at=existing.created_at,
            updated_at=datetime.now(),
        )
        self._items[item_id] = updated_item
        return updated_item
    def delete_item(self, item_id: int) -> bool:
        """Delete an item."""
        if item_id not in self._items:
            return False
        del self._items[item_id]
        return True
    def close(self) -> None:
        """Close database connection."""
        self._items.clear()
''',
        },
        readme="""# Web API
FastAPI-based RESTful API server.
## Installation
```bash
pip install -r requirements.txt
```
Running
```bash
uvicorn main:app --reload
```
API Documentation
- Swagger UI: http://localhost:8000/docs
- ReDoc: http://localhost:8000/redoc
Endpoints
- GET / - Root endpoint
- GET /items - List items
- GET /items/{id} - Get item
- POST /items - Create item
- PUT /items/{id} - Update item
- DELETE /items/{id} - Delete item
""",
    ),
    "python_data_processing": CodeTemplate(
        name="Python Data Processing",
        description="Data processing pipeline with pandas, multiprocessing, and validation",
        language=Language.PYTHON,
        project_type=ProjectType.DATA_PROCESSING,
        dependencies=["pandas>=2.0", "numpy>=1.24", "pydantic>=2.0"],
        setup_commands=["pip install pandas numpy pydantic"],
        files={
            "processor.py": '''"""Data processing pipeline."""
from __future__ import annotations
import logging
from concurrent.futures import ProcessPoolExecutor, as_completed
from pathlib import Path
```

```
from typing import Any
import numpy as np
import pandas as pd
from pydantic import BaseModel, validator
logger = logging.getLogger(__name__)
class DataConfig(BaseModel):
    """Configuration for data processing."""
    input_file: Path
    output_file: Path
    chunk_size: int = 10000
    max_workers: int = 4
    @validator("input_file")
    def input_must_exist(cls, v: Path) -> Path:
        if not v.exists():
            raise ValueError(f"Input file not found: {v}")
        return v
class DataProcessor:
    """Process large datasets in parallel."""
    def __init__(self, config: DataConfig) -> None:
        self.config = config
        self.stats: dict[str, Any] = {}
    def process(self) -> pd.DataFrame:
        """Process data in chunks."""
        logger.info(f"Processing {self.config.input_file}")
        chunks = []
        for chunk in pd.read_csv(
            self.config.input_file,
            chunksize=self.config.chunk_size
        ):
            processed = self._process_chunk(chunk)
            chunks.append(processed)
        result = pd.concat(chunks, ignore_index=True)
        result.to_csv(self.config.output_file, index=False)
        logger.info(f"Saved to {self.config.output_file}")
        return result
    def _process_chunk(self, chunk: pd.DataFrame) -> pd.DataFrame:
        """Process a single chunk."""
        chunk = chunk.dropna()
        numeric_cols = chunk.select_dtypes(include=[np.number]).columns
        chunk[numeric_cols] = chunk[numeric_cols].fillna(0)
        return chunk
    def process_parallel(self) -> pd.DataFrame:
        """Process data in parallel using multiprocessing."""
        logger.info(f"Parallel processing with {self.config.max_workers} workers")
        chunks = list(pd.read_csv(
            self.config.input_file,
            chunksize=self.config.chunk_size
        ))
        results = []
        with ProcessPoolExecutor(max_workers=self.config.max_workers) as executor:
            futures = {
                executor.submit(self._process_chunk, chunk): i
                for i, chunk in enumerate(chunks)
            }
            for future in as_completed(futures):
                chunk_id = futures[future]
                try:
                    result = future.result()
                    results.append(result)
                    logger.info(f"Processed chunk {chunk_id}")
                except Exception as e:
                    logger.error(f"Error in chunk {chunk_id}: {e}")
        final = pd.concat(results, ignore_index=True)
        final.to_csv(self.config.output_file, index=False)
        return final
```

```
    def get_statistics(self, df: pd.DataFrame) -> dict[str, Any]:
        """Calculate data statistics."""
        return {
            "rows": len(df),
            "columns": len(df.columns),
            "memory_mb": df.memory_usage(deep=True).sum() / 1024 / 1024,
            "null_counts": df.isnull().sum().to_dict(),
            "dtypes": df.dtypes.astype(str).to_dict(),
        }
def main() -> None:
    """Main entry point."""
    import sys
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s"
    )
    config = DataConfig(
        input_file=Path(sys.argv[1]) if len(sys.argv) > 1 else Path("data.csv"),
        output_file=Path("output.csv"),
        chunk_size=10000,
        max_workers=4,
    )
    processor = DataProcessor(config)
    result = processor.process_parallel()
    stats = processor.get_statistics(result)
    print(f"Processed {stats['rows']} rows, {stats['columns']} columns")
    print(f"Memory: {stats['memory_mb']:.2f} MB")
if __name__ == "__main__":
    main()
''',
            "validators.py": '''"""Data validation utilities."""
from __future__ import annotations
from typing import Any
import numpy as np
import pandas as pd
from pydantic import BaseModel, validator
class DataValidator(BaseModel):
    """Validate data rows."""
    id: int
    name: str
    value: float
    category: str
    @validator("id")
    def id_must_be_positive(cls, v: int) -> int:
        if v <= 0:
            raise ValueError("ID must be positive")
        return v
    @validator("name")
    def name_not_empty(cls, v: str) -> str:
        if not v or not v.strip():
            raise ValueError("Name cannot be empty")
        return v.strip()
    @validator("value")
    def value_in_range(cls, v: float) -> float:
        if not 0 <= v <= 1000:
            raise ValueError("Value must be between 0 and 1000")
        return v
    @validator("category")
    def category_valid(cls, v: str) -> str:
        valid = ["A", "B", "C", "D"]
        if v not in valid:
            raise ValueError(f"Category must be one of {valid}")
        return v
def validate_dataframe(df: pd.DataFrame) -> tuple[pd.DataFrame, list[str]]:
    """Validate a DataFrame."""
```

```
    errors = []
    valid_rows = []
    for idx, row in df.iterrows():
        try:
            validated = DataValidator(**row.to_dict())
            valid_rows.append(validated.dict())
        except Exception as e:
            errors.append(f"Row {idx}: {e}")
    return pd.DataFrame(valid_rows), errors
''',
        },
        readme="""# Data Processing
High-performance data processing pipeline.
## Features
- Chunk-based processing for large files
- Parallel processing with multiprocessing
- Data validation with Pydantic
- Statistics calculation
- Memory-efficient operations
## Usage
```bash
python processor.py input.csv
```
Processing Steps
1. Read data in chunks
2. Clean and validate
3. Process in parallel
4. Aggregate results
5. Save to output
""",
    ),
}
def get_template(template_id: str) -> CodeTemplate | None:
    """Get a template by ID."""
    return PROJECT_TEMPLATES.get(template_id)
def get_templates_for_project_type(project_type: ProjectType) -> list[CodeTemplate]:
    """Get all templates for a project type."""
    return [
        template for template in PROJECT_TEMPLATES.values() if template.project_type == project
    _type
    ]
def get_templates_for_language(language: Language) -> list[CodeTemplate]:
    """Get all templates for a language."""
    return [template for template in PROJECT_TEMPLATES.values() if template.language == languag
    e]
def list_available_templates() -> dict[str, str]:
    """List all available templates."""
    return {
        template_id: template.description for template_id, template in PROJECT_TEMPLATES.items(
)
    }
def suggest_template(task_description: str) -> list[str]:
    """Suggest appropriate templates based on task description."""
    task_lower = task_description.lower()
    suggestions = []
    if any(word in task_lower for word in ["cli", "command", "argument", "terminal"]):
        suggestions.append("python_cli_tool")
    if any(word in task_lower for word in ["api", "web", "rest", "endpoint", "server"]):
        suggestions.append("python_web_api")
    if any(word in task_lower for word in ["data", "process", "csv", "pandas", "analysis"]):
        suggestions.append("python_data_processing")
    if not suggestions:
        suggestions.append("python_cli_tool")
    return suggestions
```

File: [31/252] openlaoke/core/code_validator.py

```
"""Incremental code validation and auto-fix system.
Validates each function/step before proceeding, ensuring
small models produce correct code.
"""
from __future__ import annotations
import ast
import re
import subprocess
import sys
from dataclasses import dataclass, field
from pathlib import Path
@dataclass
class ValidationResult:
    is_valid: bool
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    fixed_code: str | None = None
class CodeValidator:
    """Validates Python code incrementally."""
    def validate_syntax(self, code: str) -> ValidationResult:
        """Check Python syntax validity."""
        errors = []
        warnings = []
        try:
            ast.parse(code)
        except SyntaxError as e:
            errors.append(f"Line {e.lineno}: {e.msg}")
            if e.text:
                errors.append(f"  {e.text.strip()}")
        return ValidationResult(
            is_valid=len(errors) == 0,
            errors=errors,
            warnings=warnings,
        )
    def validate_function(self, func_code: str) -> ValidationResult:
        """Validate a single function definition."""
        result = self.validate_syntax(func_code)
        if not result.is_valid:
            return result
        try:
            tree = ast.parse(func_code)
            for node in ast.walk(tree):
                if isinstance(node, ast.FunctionDef):
                    if not node.returns and "def " in func_code:
                        result.warnings.append(f"Function '{node.name}' missing return type hin
    t")
                    if not ast.get_docstring(node):
                        result.warnings.append(f"Function '{node.name}' missing docstring")
        except Exception as e:
            result.errors.append(f"AST analysis failed: {e}")
        return result
    def auto_fix_syntax(self, code: str) -> str:
        """Attempt to auto-fix common syntax errors."""
        lines = code.split("\n")
        fixed_lines = []
        i = 0
        while i < len(lines):
            line = lines[i]
            fixed_line = line
            # Fix missing closing parentheses
            open_parens = line.count("(") - line.count(")")
```

```
            if open_parens > 0 and i + 1 < len(lines):
                # Look ahead for the rest
                next_line = lines[i + 1].strip()
                if next_line and not next_line.startswith("#"):
                    # Merge with next line if it continues the expression
                    pass
            # Fix else statement
            if "else:" in line and "else" in line and not line.strip().startswith("else:"):
                # Ensure else has proper indentation
                indent = len(line) - len(line.lstrip())
                fixed_line = " " * indent + "else:\n"
            # Fix common patterns
            if "return x = " in line:
                # Fix return with assignment
                match = re.search(r"return\s+(\w+)\s*=\s*(.+)", line)
                if match:
                    var, value = match.groups()
                    indent = len(line) - len(line.lstrip())
                    fixed_line = " " * indent + f"{var} = {value}\n"
                    fixed_line += " " * indent + f"return {var}\n"
            fixed_lines.append(fixed_line)
            i += 1
        return "\n".join(fixed_lines)
    def validate_and_fix(self, code: str) -> tuple[str, ValidationResult]:
        """Validate code and attempt to fix errors."""
        result = self.validate_syntax(code)
        if result.is_valid:
            return code, result
        # Try auto-fix
        fixed_code = self.auto_fix_syntax(code)
        fixed_result = self.validate_syntax(fixed_code)
        if fixed_result.is_valid:
            return fixed_code, ValidationResult(
                is_valid=True,
                warnings=["Code was auto-fixed"],
                fixed_code=fixed_code,
            )
        return code, result
class IncrementalBuilder:
    """Build code incrementally with validation at each step."""
    def __init__(self):
        self.validator = CodeValidator()
        self.code_parts: list[str] = []
        self.current_code = ""
    def add_imports(self, imports: str) -> ValidationResult:
        """Add import statements with validation."""
        result = self.validator.validate_syntax(imports)
        if result.is_valid:
            self.code_parts.append(imports)
            self._update_code()
        return result
    def add_function(self, func_code: str) -> ValidationResult:
        """Add a function with validation."""
        result = self.validator.validate_function(func_code)
        if not result.is_valid:
            # Try to fix
            fixed_code, fix_result = self.validator.validate_and_fix(func_code)
            if fix_result.is_valid:
                self.code_parts.append(fixed_code)
                self._update_code()
                return ValidationResult(
                    is_valid=True,
                    warnings=["Function was auto-fixed"],
                )
        if result.is_valid:
```

```
            self.code_parts.append(func_code)
            self._update_code()
        return result
    def add_main_block(self, main_code: str) -> ValidationResult:
        """Add main execution block with validation."""
        result = self.validator.validate_syntax(main_code)
        if result.is_valid:
            self.code_parts.append(main_code)
            self._update_code()
        return result
    def _update_code(self) -> None:
        """Update current code from parts."""
        self.current_code = "\n\n".join(self.code_parts)
    def get_final_code(self) -> str:
        """Get the final validated code."""
        return self.current_code
    def validate_final(self) -> ValidationResult:
        """Validate the complete code."""
        return self.validator.validate_syntax(self.current_code)
class ExecutionValidator:
    """Validate code execution step by step."""
    def test_imports(self, code: str) -> ValidationResult:
        """Test if imports work."""
        import_code = "\n".join(
            [line for line in code.split("\n") if line.strip().startswith(("import ", "from "))
    ]
        )
        if not import_code:
            return ValidationResult(is_valid=True)
        try:
            exec(import_code, {})
            return ValidationResult(is_valid=True)
        except Exception as e:
            return ValidationResult(
                is_valid=False,
                errors=[f"Import error: {e}"],
            )
    def test_syntax_with_python(self, code: str) -> ValidationResult:
        """Test syntax using Python interpreter."""
        temp_file = Path("/tmp/test_code.py")
        temp_file.write_text(code)
        try:
            result = subprocess.run(
                [sys.executable, "-m", "py_compile", str(temp_file)],
                capture_output=True,
                text=True,
            )
            if result.returncode == 0:
                return ValidationResult(is_valid=True)
            else:
                errors = result.stderr.strip().split("\n")
                return ValidationResult(
                    is_valid=False,
                    errors=errors,
                )
        finally:
            if temp_file.exists():
                temp_file.unlink()
    def test_execution(self, code: str, timeout: int = 5) -> ValidationResult:
        """Test if code executes without errors."""
        temp_file = Path("/tmp/test_exec.py")
        temp_file.write_text(code)
        try:
            result = subprocess.run(
                [sys.executable, str(temp_file)],
```

```
                capture_output=True,
                text=True,
                timeout=timeout,
            )
            if result.returncode == 0:
                return ValidationResult(
                    is_valid=True,
                    warnings=[result.stdout] if result.stdout else [],
                )
            else:
                return ValidationResult(
                    is_valid=False,
                    errors=[result.stderr] if result.stderr else ["Execution failed"],
                )
        except subprocess.TimeoutExpired:
            return ValidationResult(
                is_valid=False,
                errors=[f"Execution timeout ({timeout}s)"],
            )
        except Exception as e:
            return ValidationResult(
                is_valid=False,
                errors=[f"Execution error: {e}"],
            )
        finally:
            if temp_file.exists():
                temp_file.unlink()
def create_validated_code(
    parts: list[str], validate_each: bool = True
) -> tuple[str, ValidationResult]:
    """Create code from parts with incremental validation."""
    builder = IncrementalBuilder()
    all_errors = []
    all_warnings = []
    for i, part in enumerate(parts):
        if validate_each:
            if "import " in part or "from " in part:
                result = builder.add_imports(part)
            elif "def " in part:
                result = builder.add_function(part)
            else:
                result = builder.add_main_block(part)
            if not result.is_valid:
                all_errors.extend([f"Part {i}: {e}" for e in result.errors])
                # Try to continue anyway
                builder.code_parts.append(part)
                builder._update_code()
            else:
                all_warnings.extend(result.warnings)
    final_code = builder.get_final_code()
    final_result = builder.validate_final()
    return final_code, ValidationResult(
        is_valid=final_result.is_valid and len(all_errors) == 0,
        errors=final_result.errors + all_errors,
        warnings=all_warnings,
        fixed_code=final_code if final_result.is_valid else None,
    )
```

File: [32/252] openlaoke/core/compact/__init__.py

```
"""Context compaction system for intelligent conversation history compression."""
from __future__ import annotations
from openlaoke.core.compact.compactor import (
    CompactConfig,
    CompactResult,
    CompactType,
    ContextCompactor,
)
from openlaoke.core.compact.strategies import (
    AutoCompactStrategy,
    MicroCompactStrategy,
    ReactiveCompactStrategy,
    SnipCompactStrategy,
)
from openlaoke.core.compact.summarizer import MessageSummarizer, SummaryConfig
from openlaoke.core.compact.token_budget import Allocation, TokenBudget, TokenUsageTracker
__all__ = [
    "ContextCompactor",
    "CompactConfig",
    "CompactResult",
    "CompactType",
    "TokenBudget",
    "Allocation",
    "TokenUsageTracker",
    "AutoCompactStrategy",
    "ReactiveCompactStrategy",
    "SnipCompactStrategy",
    "MicroCompactStrategy",
    "MessageSummarizer",
    "SummaryConfig",
]
```

File: [33/252] openlaoke/core/compact/compactor.py

```
"""Context compaction system for intelligent conversation history compression."""
from __future__ import annotations
import json
from dataclasses import dataclass
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from openlaoke.core.compact.strategies import (
    AutoCompactStrategy,
    CompactStrategy,
    MicroCompactStrategy,
    ReactiveCompactStrategy,
    SnipCompactStrategy,
)
from openlaoke.core.compact.token_budget import Allocation, TokenBudget
if TYPE_CHECKING:
    from openlaoke.core.compact.summarizer import MessageSummarizer
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.types.core_types import Message, TokenUsage
class CompactType(StrEnum):
    """Types of context compaction."""
    AUTO = "auto"
    REACTIVE = "reactive"
    SNIP = "snip"
    MICRO = "micro"
    FULL = "full"
@dataclass
class CompactConfig:
    """Configuration for context compaction."""
    enabled: bool = True
    default_type: CompactType = CompactType.AUTO
    trigger_threshold: float = 0.80
    keep_fraction: float = 0.30
    max_summary_tokens: int = 4000
    preserve_recent: int = 10
    max_tool_result_tokens: int = 5000
    auto_compact_on_overflow: bool = True
    use_summarization: bool = True
    summary_model: str | None = None
    def get_token_budget(self, max_input: int = 200000, max_output: int = 8192) -> TokenBudget:
        return TokenBudget(
            max_input_tokens=max_input,
            max_output_tokens=max_output,
            trigger_threshold=self.trigger_threshold,
            keep_fraction=self.keep_fraction,
        )
@dataclass
class CompactResult:
    """Result of context compaction."""
    messages: list[Message]
    tokens_before: int
    tokens_after: int
    messages_removed: int
    messages_preserved: int
    compact_type: CompactType
    summary: str | None = None
    compression_ratio: float = 0.0
    allocation: Allocation | None = None
    def __post_init__(self) -> None:
        if self.tokens_before > 0:
            self.compression_ratio = 1.0 - (self.tokens_after / self.tokens_before)
    def to_dict(self) -> dict[str, Any]:
```

```
        return {
            "tokens_before": self.tokens_before,
            "tokens_after": self.tokens_after,
            "messages_removed": self.messages_removed,
            "messages_preserved": self.messages_preserved,
            "compact_type": self.compact_type.value,
            "compression_ratio": self.compression_ratio,
            "summary": self.summary,
        }
class ContextCompactor:
    """Context compaction system for intelligent conversation history compression."""
    def __init__(
        self,
        config: CompactConfig,
        client: MultiProviderClient | None = None,
    ) -> None:
        self.config = config
        self.client = client
        self._summarizer: MessageSummarizer | None = None
        self._budget: TokenBudget | None = None
        self.strategies: dict[CompactType, CompactStrategy] = {
            CompactType.AUTO: AutoCompactStrategy(config.trigger_threshold),
            CompactType.REACTIVE: ReactiveCompactStrategy(config.max_tool_result_tokens),
            CompactType.SNIP: SnipCompactStrategy(config.preserve_recent),
            CompactType.MICRO: MicroCompactStrategy(),
        }
        if client and config.use_summarization:
            from openlaoke.core.compact.summarizer import MessageSummarizer, SummaryConfig
            summary_config = SummaryConfig(
                max_summary_tokens=config.max_summary_tokens,
                summary_model=config.summary_model,
                preserve_recent_messages=config.preserve_recent,
            )
            self._summarizer = MessageSummarizer(client, summary_config)
    def set_budget(self, budget: TokenBudget) -> None:
        self._budget = budget
    def get_budget(self) -> TokenBudget:
        if self._budget is None:
            self._budget = self.config.get_token_budget()
        return self._budget
    def estimate_tokens(self, messages: list[Message]) -> int:
        total = 0
        for msg in messages:
            content = self._extract_content(msg)
            total += int(len(content) / 4)
        return total
    def _extract_content(self, message: Message) -> str:
        from openlaoke.types.core_types import (
            AssistantMessage,
            AttachmentMessage,
            ProgressMessage,
            SystemMessage,
            UserMessage,
        )
        if isinstance(message, UserMessage):
            return message.content
        elif isinstance(message, AssistantMessage):
            parts = [message.content]
            for tu in message.tool_uses:
                parts.append(f"Tool: {tu.name}")
                parts.append(json.dumps(tu.input))
            return "\n".join(parts)
        elif isinstance(message, SystemMessage):
            return message.content
        elif isinstance(message, ProgressMessage):
```

```
        return ""
    elif isinstance(message, AttachmentMessage):
        return f"{message.content} Files: {', '.join(message.file_paths)}"
    return ""
def should_compact(self, messages: list[Message], budget: TokenBudget | None = None) -> boo
l:
    budget = budget or self.get_budget()
    current_tokens = self.estimate_tokens(messages)
    return budget.check_overflow(current_tokens)
def allocate_budget(
    self, messages: list[Message], budget: TokenBudget | None = None
) -> Allocation:
    budget = budget or self.get_budget()
    current_tokens = self.estimate_tokens(messages)
    return budget.allocate(system_tokens=current_tokens // 10)
async def compact(
    self,
    messages: list[Message],
    budget: TokenBudget | None = None,
    compact_type: CompactType | None = None,
) -> CompactResult:
    budget = budget or self.get_budget()
    compact_type = compact_type or self.config.default_type
    strategy = self.strategies.get(compact_type)
    if strategy is None:
        strategy = self.strategies[CompactType.AUTO]
    result = await strategy.compact(messages, budget, self._summarizer)
    allocation = self.allocate_budget(result.messages, budget)
    return CompactResult(
        messages=result.messages,
        tokens_before=result.tokens_before,
        tokens_after=result.tokens_after,
        messages_removed=result.messages_removed,
        messages_preserved=result.messages_preserved,
        compact_type=compact_type,
        summary=result.summary,
        allocation=allocation,
    )
async def auto_compact(self, messages: list[Message]) -> CompactResult:
    if not self.should_compact(messages):
        tokens = self.estimate_tokens(messages)
        return CompactResult(
            messages=messages,
            tokens_before=tokens,
            tokens_after=tokens,
            messages_removed=0,
            messages_preserved=len(messages),
            compact_type=CompactType.AUTO,
        )
    return await self.compact(messages, compact_type=CompactType.AUTO)
async def reactive_compact(self, messages: list[Message]) -> CompactResult:
    return await self.compact(messages, compact_type=CompactType.REACTIVE)
async def snip_compact(self, messages: list[Message]) -> CompactResult:
    return await self.compact(messages, compact_type=CompactType.SNIP)
async def micro_compact(self, messages: list[Message]) -> CompactResult:
    return await self.compact(messages, compact_type=CompactType.MICRO)
async def full_compact(self, messages: list[Message]) -> CompactResult:
    budget = self.get_budget()
    result = await self.micro_compact(messages)
    if result.tokens_after <= budget.get_available_tokens():
        return result
    result = await self.snip_compact(result.messages)
    if result.tokens_after <= budget.get_available_tokens():
        return result
    result = await self.auto_compact(result.messages)
```

```
    return result
def get_compaction_stats(self, messages: list[Message]) -> dict[str, Any]:
    budget = self.get_budget()
    current_tokens = self.estimate_tokens(messages)
    available = budget.get_available_tokens()
    threshold = budget.get_trigger_threshold()
    return {
        "current_tokens": current_tokens,
        "available_tokens": available,
        "threshold_tokens": threshold,
        "message_count": len(messages),
        "overflow": current_tokens > threshold,
        "overflow_amount": max(0, current_tokens - available),
        "recommended_action": self._get_recommended_action(
            current_tokens, threshold, available
        ),
    }
def _get_recommended_action(self, current: int, threshold: int, available: int) -> str:
    if current <= threshold:
        return "none"
    elif current <= threshold * 1.2:
        return "micro"
    elif current <= available * 0.8:
        return "snip"
    else:
        return "auto"
def track_usage(self, usage: TokenUsage) -> None:
    budget = self.get_budget()
    budget.update_cache_tokens(usage.cache_read_tokens, usage.cache_creation_tokens)
def can_add_message(self, messages: list[Message], new_message: Message) -> bool:
    budget = self.get_budget()
    current_tokens = self.estimate_tokens(messages)
    new_tokens = self.estimate_tokens([new_message])
    available = budget.get_available_tokens()
    return current_tokens + new_tokens < available
```

File: [34/252] openlaoke/core/compact/strategies.py

```
"""Compaction strategies for different compression needs."""
from __future__ import annotations
import json
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.core.compact.summarizer import MessageSummarizer
    from openlaoke.core.compact.token_budget import TokenBudget
    from openlaoke.types.core_types import Message
@dataclass
class StrategyResult:
    """Result of a compaction strategy."""
    messages: list[Message]
    tokens_before: int
    tokens_after: int
    messages_removed: int
    messages_preserved: int
    summary: str | None = None
    compression_ratio: float = 0.0
    def __post_init__(self) -> None:
        if self.tokens_before > 0:
            self.compression_ratio = 1.0 - (self.tokens_after / self.tokens_before)
class CompactStrategy(ABC):
    """Base class for compaction strategies."""
    @abstractmethod
    async def compact(
        self,
        messages: list[Message],
        budget: TokenBudget,
        summarizer: MessageSummarizer | None = None,
    ) -> StrategyResult:
        """Execute the compaction strategy."""
    @abstractmethod
    def should_apply(self, messages: list[Message], budget: TokenBudget) -> bool:
        """Check if this strategy should be applied."""
    def estimate_tokens(self, messages: list[Message]) -> int:
        total = 0
        for msg in messages:
            content = self._extract_content(msg)
            total += int(len(content) / 4)
        return total
    def _extract_content(self, message: Message) -> str:
        from openlaoke.types.core_types import (
            AssistantMessage,
            AttachmentMessage,
            ProgressMessage,
            SystemMessage,
            UserMessage,
        )
        if isinstance(message, UserMessage):
            return message.content
        elif isinstance(message, AssistantMessage):
            parts = [message.content]
            for tu in message.tool_uses:
                parts.append(f"Tool: {tu.name}")
                parts.append(json.dumps(tu.input))
            return "\n".join(parts)
        elif isinstance(message, SystemMessage):
            return message.content
        elif isinstance(message, ProgressMessage):
```

```
            return ""
        elif isinstance(message, AttachmentMessage):
            return f"{message.content} Files: {', '.join(message.file_paths)}"
        return ""
    def get_message_importance(self, message: Message, index: int, total: int) -> float:
        from openlaoke.types.core_types import AssistantMessage, SystemMessage, UserMessage
        score = 0.0
        recency_score = index / total
        score += recency_score * 0.5
        if isinstance(message, UserMessage):
            score += 0.3
            if len(message.content) < 500:
                score += 0.1
        elif isinstance(message, AssistantMessage):
            score += 0.2
            if message.tool_uses:
                score += 0.2
            if message.stop_reason == "tool_use":
                score += 0.1
        elif isinstance(message, SystemMessage):
            if message.subtype in ("error", "warning"):
                score += 0.4
            elif message.subtype == "compact":
                score += 0.1
            else:
                score += 0.1
        return min(1.0, score)
class AutoCompactStrategy(CompactStrategy):
    """Automatic compaction triggered by token threshold."""
    def __init__(self, threshold_fraction: float = 0.80) -> None:
        self.threshold_fraction = threshold_fraction
    def should_apply(self, messages: list[Message], budget: TokenBudget) -> bool:
        current_tokens = self.estimate_tokens(messages)
        return budget.check_overflow(current_tokens)
    async def compact(
        self,
        messages: list[Message],
        budget: TokenBudget,
        summarizer: MessageSummarizer | None = None,
    ) -> StrategyResult:
        tokens_before = self.estimate_tokens(messages)
        if not self.should_apply(messages, budget):
            return StrategyResult(
                messages=messages,
                tokens_before=tokens_before,
                tokens_after=tokens_before,
                messages_removed=0,
                messages_preserved=len(messages),
            )
        if summarizer:
            result = await summarizer.summarize_with_preserve(messages)
            summary_msg = summarizer.create_summary_message(result.summary)
            to_preserve, _ = summarizer.split_messages(messages)
            new_messages = [summary_msg] + to_preserve
            tokens_after = self.estimate_tokens(new_messages)
            return StrategyResult(
                messages=new_messages,
                tokens_before=tokens_before,
                tokens_after=tokens_after,
                messages_removed=len(messages) - len(to_preserve),
                messages_preserved=len(to_preserve),
                summary=result.summary,
            )
        target_tokens = budget.get_target_tokens(tokens_before)
        new_messages = self._trim_by_importance(messages, target_tokens)
```

```
        tokens_after = self.estimate_tokens(new_messages)
        return StrategyResult(
            messages=new_messages,
            tokens_before=tokens_before,
            tokens_after=tokens_after,
            messages_removed=len(messages) - len(new_messages),
            messages_preserved=len(new_messages),
        )
    def _trim_by_importance(self, messages: list[Message], target_tokens: int) -> list[Message]
    :
        if not messages:
            return messages
        scored = []
        for i, msg in enumerate(messages):
            score = self.get_message_importance(msg, i, len(messages))
            scored.append((score, i, msg))
        scored.sort(key=lambda x: x[0], reverse=True)
        selected = []
        current_tokens = 0
        for _score, idx, msg in scored:
            msg_tokens = self.estimate_tokens([msg])
            if current_tokens + msg_tokens <= target_tokens:
                selected.append((idx, msg))
                current_tokens += msg_tokens
        selected.sort(key=lambda x: x[0])
        return [msg for _, msg in selected]
class ReactiveCompactStrategy(CompactStrategy):
    """Reactive compaction for real-time tool result compression."""
    def __init__(self, max_tool_result_tokens: int = 5000) -> None:
        self.max_tool_result_tokens = max_tool_result_tokens
    def should_apply(self, messages: list[Message], budget: TokenBudget) -> bool:
        return True
    async def compact(
        self,
        messages: list[Message],
        budget: TokenBudget,
        summarizer: MessageSummarizer | None = None,
    ) -> StrategyResult:
        tokens_before = self.estimate_tokens(messages)
        new_messages = self._compress_tool_results(messages)
        tokens_after = self.estimate_tokens(new_messages)
        return StrategyResult(
            messages=new_messages,
            tokens_before=tokens_before,
            tokens_after=tokens_after,
            messages_removed=len(messages) - len(new_messages),
            messages_preserved=len(new_messages),
        )
    def _compress_tool_results(self, messages: list[Message]) -> list[Message]:
        from openlaoke.types.core_types import (
            AssistantMessage,
            SystemMessage,
        )
        result: list[Message] = []
        for msg in messages:
            if isinstance(msg, SystemMessage):
                content = msg.content
                if len(content) > self.max_tool_result_tokens:
                    truncated = self._truncate_tool_result(content)
                    result.append(
                        SystemMessage(
                            role=msg.role,
                            content=truncated,
                            subtype=msg.subtype,
                            timestamp=msg.timestamp,
```

```
                        )
                    )
                else:
                    result.append(msg)
            elif isinstance(msg, AssistantMessage):
                result.append(msg)
            else:
                result.append(msg)
        return result
    def _truncate_tool_result(self, content: str) -> str:
        max_len = self.max_tool_result_tokens * 4
        if len(content) <= max_len:
            return content
        lines = content.split("\n")
        if len(lines) > 100:
            head = "\n".join(lines[:50])
            tail = "\n".join(lines[-20:])
            return f"{head}\n... [truncated {len(lines) - 70} lines] ...\n{tail}"
        return content[:max_len] + "\n... [truncated] ..."
class SnipCompactStrategy(CompactStrategy):
    """Snip compaction removing older messages."""
    def __init__(self, keep_recent: int = 20) -> None:
        self.keep_recent = keep_recent
    def should_apply(self, messages: list[Message], budget: TokenBudget) -> bool:
        return len(messages) > self.keep_recent
    async def compact(
        self,
        messages: list[Message],
        budget: TokenBudget,
        summarizer: MessageSummarizer | None = None,
    ) -> StrategyResult:
        tokens_before = self.estimate_tokens(messages)
        if not self.should_apply(messages, budget):
            return StrategyResult(
                messages=messages,
                tokens_before=tokens_before,
                tokens_after=tokens_before,
                messages_removed=0,
                messages_preserved=len(messages),
            )
        new_messages = self._snip_messages(messages)
        tokens_after = self.estimate_tokens(new_messages)
        return StrategyResult(
            messages=new_messages,
            tokens_before=tokens_before,
            tokens_after=tokens_after,
            messages_removed=len(messages) - len(new_messages),
            messages_preserved=len(new_messages),
        )
    def _snip_messages(self, messages: list[Message]) -> list[Message]:
        if len(messages) <= self.keep_recent:
            return messages
        return messages[-self.keep_recent :]
class MicroCompactStrategy(CompactStrategy):
    """Micro compaction removing only redundant content."""
    def __init__(self, dedupe_threshold: int = 200) -> None:
        self.dedupe_threshold = dedupe_threshold
    def should_apply(self, messages: list[Message], budget: TokenBudget) -> bool:
        current_tokens = self.estimate_tokens(messages)
        return current_tokens > budget.get_available_tokens() * 0.5
    async def compact(
        self,
        messages: list[Message],
        budget: TokenBudget,
        summarizer: MessageSummarizer | None = None,
```

```
) -> StrategyResult:
    tokens_before = self.estimate_tokens(messages)
    new_messages = self._dedupe_messages(messages)
    tokens_after = self.estimate_tokens(new_messages)
    return StrategyResult(
        messages=new_messages,
        tokens_before=tokens_before,
        tokens_after=tokens_after,
        messages_removed=len(messages) - len(new_messages),
        messages_preserved=len(new_messages),
    )
def _dedupe_messages(self, messages: list[Message]) -> list[Message]:
    if len(messages) < 2:
        return messages
    result: list[Message] = []
    seen_content: dict[str, int] = {}
    for msg in messages:
        content = self._extract_content(msg)
        content_hash = content[: self.dedupe_threshold]
        if content_hash in seen_content:
            prev_idx = seen_content[content_hash]
            prev_msg = result[prev_idx]
            if self._is_same_message_type(msg, prev_msg):
                continue
        seen_content[content_hash] = len(result)
        result.append(msg)
    return result
def _is_same_message_type(self, msg1: Message, msg2: Message) -> bool:
    from openlaoke.types.core_types import SystemMessage
    if msg1.role != msg2.role:
        return False
    if (
        isinstance(msg1, SystemMessage)
        and isinstance(msg2, SystemMessage)
        and msg1.subtype == msg2.subtype
    ):
        content1 = msg1.content[: self.dedupe_threshold]
        content2 = msg2.content[: self.dedupe_threshold]
        if content1 == content2:
            return True
    return False
```

File: [35/252] openlaoke/core/compact/summarizer.py

```
"""Message summarization for context compaction."""
from __future__ import annotations
import json
from dataclasses import dataclass
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.types.core_types import Message
SUMMARY_SYSTEM_PROMPT = """You are a conversation summarizer. Your task is to create concise, i
    nformative summaries of conversation history that preserve key information while reducing t
    oken count.
Guidelines:
- Preserve important decisions, conclusions, and key facts
- Keep tool use results that are relevant to ongoing work
- Maintain context needed for understanding subsequent messages
- Remove redundant or repeated information
- Focus on actionable outcomes and results
- Keep file paths, command results, and code references"""
SUMMARY_USER_PROMPT = """Summarize the following conversation messages. Preserve:
1. Key decisions and conclusions
2. Important tool results (file reads, command outputs)
3. Context needed for ongoing work
4. File paths and code references
Messages to summarize:
{messages}
Create a concise summary that captures the essential information."""
@dataclass
class SummaryConfig:
    """Configuration for message summarization."""
    max_summary_tokens: int = 4000
    summary_model: str | None = None
    preserve_recent_messages: int = 5
    preserve_tool_results: bool = True
    preserve_user_queries: bool = True
    summary_temperature: float = 0.3
    custom_prompt: str | None = None
    def get_system_prompt(self) -> str:
        if self.custom_prompt:
            return self.custom_prompt
        return SUMMARY_SYSTEM_PROMPT
@dataclass
class SummaryResult:
    """Result of summarization."""
    summary: str
    original_tokens: int
    summary_tokens: int
    messages_summarized: int
    messages_preserved: int
    compression_ratio: float = 0.0
    def __post_init__(self) -> None:
        if self.original_tokens > 0:
            self.compression_ratio = 1.0 - (self.summary_tokens / self.original_tokens)
class MessageSummarizer:
    """Generate summaries of conversation messages."""
    def __init__(
        self,
        client: MultiProviderClient,
        config: SummaryConfig | None = None,
    ) -> None:
        self.client = client
        self.config = config or SummaryConfig()
```

```
def estimate_tokens(self, text: str) -> int:
    chars_per_token = 4
    return int(len(text) / chars_per_token)
def estimate_message_tokens(self, message: Message) -> int:
    content = self._extract_content(message)
    return self.estimate_tokens(content)
def _extract_content(self, message: Message) -> str:
    from openlaoke.types.core_types import (
        AssistantMessage,
        AttachmentMessage,
        ProgressMessage,
        SystemMessage,
        UserMessage,
    )
    if isinstance(message, UserMessage):
        return message.content
    elif isinstance(message, AssistantMessage):
        parts = [message.content]
        for tu in message.tool_uses:
            parts.append(f"Tool: {tu.name}")
            parts.append(json.dumps(tu.input))
        return "\n".join(parts)
    elif isinstance(message, (SystemMessage, ProgressMessage)):
        return message.content
    elif isinstance(message, AttachmentMessage):
        return f"{message.content} Files: {', '.join(message.file_paths)}"
    return ""
def format_messages_for_summary(self, messages: list[Message]) -> str:
    formatted = []
    for i, msg in enumerate(messages):
        role = msg.role.value
        content = self._extract_content(msg)
        formatted.append(f"[{i}] {role}: {content}")
    return "\n\n".join(formatted)
async def summarize_messages(
    self,
    messages: list[Message],
    target_tokens: int | None = None,
) -> SummaryResult:
    if not messages:
        return SummaryResult(
            summary="",
            original_tokens=0,
            summary_tokens=0,
            messages_summarized=0,
            messages_preserved=0,
        )
    target_tokens = target_tokens or self.config.max_summary_tokens
    original_tokens = sum(self.estimate_message_tokens(m) for m in messages)
    formatted_messages = self.format_messages_for_summary(messages)
    user_prompt = SUMMARY_USER_PROMPT.format(messages=formatted_messages)
    messages_to_send = [{"role": "user", "content": user_prompt}]
    model = self.config.summary_model
    if model is None:
        model = self.client.config.get_active_model()
    response, usage, _ = await self.client.send_message(
        system_prompt=self.config.get_system_prompt(),
        messages=messages_to_send,
        model=model,
        max_tokens=self.config.max_summary_tokens,
        temperature=self.config.summary_temperature,
    )
    summary = response.content
    summary_tokens = usage.output_tokens
    return SummaryResult(
```

```
        summary=summary,
        original_tokens=original_tokens,
        summary_tokens=summary_tokens,
        messages_summarized=len(messages),
        messages_preserved=0,
    )
def should_preserve_message(self, message: Message, index: int, total: int) -> bool:
    from openlaoke.types.core_types import AssistantMessage, SystemMessage, UserMessage
    recent_threshold = total - self.config.preserve_recent_messages
    if index >= recent_threshold:
        return True
    if isinstance(message, UserMessage) and self.config.preserve_user_queries:
        return True
    if (
        isinstance(message, AssistantMessage)
        and message.tool_uses
        and self.config.preserve_tool_results
    ):
        return True
    return isinstance(message, SystemMessage) and message.subtype in ("error", "warning")
def split_messages(
    self,
    messages: list[Message],
) -> tuple[list[Message], list[Message]]:
    to_summarize = []
    to_preserve = []
    for i, msg in enumerate(messages):
        if self.should_preserve_message(msg, i, len(messages)):
            to_preserve.append(msg)
        else:
            to_summarize.append(msg)
    return to_summarize, to_preserve
async def summarize_with_preserve(
    self,
    messages: list[Message],
) -> SummaryResult:
    to_summarize, to_preserve = self.split_messages(messages)
    if not to_summarize:
        preserved_tokens = sum(self.estimate_message_tokens(m) for m in to_preserve)
        return SummaryResult(
            summary="",
            original_tokens=preserved_tokens,
            summary_tokens=preserved_tokens,
            messages_summarized=0,
            messages_preserved=len(to_preserve),
        )
    result = await self.summarize_messages(to_summarize)
    preserved_tokens = sum(self.estimate_message_tokens(m) for m in to_preserve)
    return SummaryResult(
        summary=result.summary,
        original_tokens=result.original_tokens + preserved_tokens,
        summary_tokens=result.summary_tokens + preserved_tokens,
        messages_summarized=len(to_summarize),
        messages_preserved=len(to_preserve),
    )
def create_summary_message(self, summary: str) -> Message:
    from openlaoke.types.core_types import MessageRole, SystemMessage
    return SystemMessage(
        role=MessageRole.SYSTEM,
        content=f"[Context Summary]\n{summary}",
        subtype="compact",
    )
```

File: [36/252] openlaoke/core/compact/token_budget.py

```
"""Token budget management for context compaction."""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.types.core_types import TokenUsage
class BudgetType(StrEnum):
    """Types of token budget specifications."""
    FIXED = "fixed"
    FRACTION = "fraction"
    DYNAMIC = "dynamic"
@dataclass
class Allocation:
    """Token allocation result."""
    total_available: int = 0
    system_reserved: int = 0
    tools_reserved: int = 0
    cache_available: int = 0
    messages_allocated: int = 0
    output_allocated: int = 0
    remaining: int = 0
    def is_within_budget(self, tokens: int) -> bool:
        return tokens <= self.remaining
@dataclass
class TokenBudget:
    """Token budget management for context window."""
    max_input_tokens: int = 200000
    max_output_tokens: int = 8192
    reserved_tokens: int = 10000
    tool_tokens: int = 5000
    cache_read_tokens: int = 0
    cache_creation_tokens: int = 0
    budget_type: BudgetType = BudgetType.FIXED
    trigger_threshold: float = 0.80
    keep_fraction: float = 0.30
    def get_available_tokens(self) -> int:
        total_reserved = self.reserved_tokens + self.tool_tokens
        available = self.max_input_tokens - total_reserved
        if self.cache_read_tokens > 0:
            available += self.cache_read_tokens
        return max(0, available)
    def get_trigger_threshold(self) -> int:
        available = self.get_available_tokens()
        return int(available * self.trigger_threshold)
    def get_keep_tokens(self) -> int:
        if self.budget_type == BudgetType.FRACTION:
            return int(self.max_input_tokens * self.keep_fraction)
        return int(self.get_available_tokens() * self.keep_fraction)
    def allocate(
        self,
        system_tokens: int = 0,
        tool_tokens: int = 0,
        output_tokens: int = 0,
    ) -> Allocation:
        total_available = self.get_available_tokens()
        system_reserved = max(system_tokens, self.reserved_tokens)
        tools_reserved = max(tool_tokens, self.tool_tokens)
        output_allocated = min(output_tokens, self.max_output_tokens)
        remaining_for_messages = total_available - system_reserved - tools_reserved
        messages_allocated = min(remaining_for_messages, total_available - output_allocated)
```

```
        return Allocation(
            total_available=total_available,
            system_reserved=system_reserved,
            tools_reserved=tools_reserved,
            cache_available=self.cache_read_tokens,
            messages_allocated=messages_allocated,
            output_allocated=output_allocated,
            remaining=total_available - system_reserved - tools_reserved - messages_allocated,
        )
    def check_overflow(self, current_tokens: int) -> bool:
        threshold = self.get_trigger_threshold()
        return current_tokens >= threshold
    def get_overflow_amount(self, current_tokens: int) -> int:
        available = self.get_available_tokens()
        if current_tokens <= available:
            return 0
        return current_tokens - available
    def get_target_tokens(self, current_tokens: int) -> int:
        keep_tokens = self.get_keep_tokens()
        return min(keep_tokens, current_tokens)
    def update_cache_tokens(self, read: int, creation: int) -> None:
        self.cache_read_tokens = read
        self.cache_creation_tokens = creation
@dataclass
class TokenUsageTracker:
    """Track token usage over time."""
    total_input: int = 0
    total_output: int = 0
    peak_input: int = 0
    peak_output: int = 0
    history: list[tuple[float, int, int]] = field(default_factory=list)
    def track_usage(self, usage: TokenUsage, timestamp: float = 0.0) -> None:
        from time import time
        ts = timestamp or time()
        self.total_input += usage.input_tokens
        self.total_output += usage.output_tokens
        self.peak_input = max(self.peak_input, usage.input_tokens)
        self.peak_output = max(self.peak_output, usage.output_tokens)
        self.history.append((ts, usage.input_tokens, usage.output_tokens))
    def get_average_input(self) -> float:
        if not self.history:
            return 0.0
        return sum(h[1] for h in self.history) / len(self.history)
    def get_average_output(self) -> float:
        if not self.history:
            return 0.0
        return sum(h[2] for h in self.history) / len(self.history)
    def get_total_tokens(self) -> int:
        return self.total_input + self.total_output
    def reset(self) -> None:
        self.total_input = 0
        self.total_output = 0
        self.peak_input = 0
        self.peak_output = 0
        self.history.clear()
```

File: [37/252] openlaoke/core/completion.py

```
"""Tab completion system for OpenLaoKe CLI."""
from __future__ import annotations
import readline
from openlaoke.commands.registry import get_all_commands
from openlaoke.core.skill_system import list_available_skills
class TabCompleter:
    """Tab completion for commands and skills."""
    def __init__(self):
        self._commands = []
        self._skills = []
        self._matches = []
        self._refresh()
    def _refresh(self):
        """Refresh the list of commands and skills."""
        # Get all slash commands
        self._commands = []
        for cmd in get_all_commands():
            if cmd.name and not cmd.hidden:
                self._commands.append(f"/{cmd.name}")
                for alias in cmd.aliases:
                    self._commands.append(f"/{alias}")
        # Get all skills
        self._skills = []
        for skill_name in list_available_skills():
            self._skills.append(f"/{skill_name}")
    def complete(self, text: str, state: int) -> str | None:
        """Complete the text."""
        if state == 0:
            # First call - build match list
            self._matches = []
            # Refresh lists
            self._refresh()
            # Get all completions
            all_items = self._commands + self._skills
            # Find matches
            for item in all_items:
                if item.startswith(text):
                    self._matches.append(item)
            # Also match partial commands (e.g., /p -> /plan, /python)
            if text.startswith("/"):
                partial = text[1:]  # Remove the /
                for skill in list_available_skills():
                    if skill.startswith(partial):
                        self._matches.append(f"/{skill}")
        # Return the next match
        if state < len(self._matches):
            return self._matches[state]
        return None
    def get_completions(self, text: str) -> list[str]:
        """Get all possible completions for text."""
        self._refresh()
        completions = []
        all_items = self._commands + self._skills
        for item in all_items:
            if item.startswith(text):
                completions.append(item)
        return sorted(set(completions))
    def get_skill_suggestions(self, prefix: str) -> list[str]:
        """Get skill suggestions for a prefix."""
        self._refresh()
        suggestions = []
```

```
        for skill in list_available_skills():
            if skill.startswith(prefix):
                suggestions.append(skill)
        return suggestions
def setup_tab_completion():
    """Setup tab completion for readline."""
    completer = TabCompleter()
    # Set the completer
    readline.set_completer(completer.complete)
    # Enable tab completion
    readline.parse_and_bind("tab: complete")
    # Set completion delimiter to space only (so /p can complete to /plan)
    readline.set_completer_delims(" \t\n")
    return completer
def show_completion_menu(text: str, completions: list[str]) -> str:
    """Show a completion menu and let user select."""
    if not completions:
        return text
    if len(completions) == 1:
        return completions[0]
    # Show all completions
    print("\n")
    for i, comp in enumerate(completions):
        print(f"  {i + 1:2}. {comp}")
    print()
    return text
# Global completer instance
_completer: TabCompleter | None = None
def get_completer() -> TabCompleter:
    """Get the global completer instance."""
    global _completer
    if _completer is None:
        _completer = TabCompleter()
    return _completer
```

File: [38/252] openlaoke/core/config_wizard.py

```
"""Interactive configuration wizard for first-time setup."""
from __future__ import annotations
import os
from rich.console import Console
from rich.panel import Panel
from rich.prompt import Confirm, Prompt
from rich.table import Table
from openlaoke.types.providers import MultiProviderConfig, ProviderConfig, ProviderType
from openlaoke.utils.config import AppConfig
console = Console(force_terminal=True)
def run_config_wizard(config: AppConfig | None = None) -> AppConfig:
    """Run the interactive configuration wizard."""
    config = config or AppConfig()
    console.clear()
    console.print(
        Panel.fit(
            "[bold cyan]OpenLaoKe Configuration Wizard[/bold cyan]\n"
            "[dim]Let's set up your AI coding assistant[/dim]",
            border_style="cyan",
        )
    )
    console.print()
    # Step 1: Configure provider
    while True:
        console.print("[bold]Step 1: Choose your AI provider[/bold]")
        console.print()
        table = Table(show_header=False, box=None)
        table.add_column("Option", style="cyan")
        table.add_column("Provider", style="bold")
        table.add_column("Status", style="dim")
        providers = [
            ("1", "? Ollama (Local)", "ollama", "local"),
            ("2", "? OpenCode Zen (FREE)", "opencode", "free"),
            ("3", "MiniMax", "minimax", "cloud"),
            ("4", "Aliyun Coding Plan", "aliyun_coding_plan", "cloud"),
            ("5", "Anthropic", "anthropic", "cloud"),
            ("6", "OpenAI (GPT-4)", "openai", "cloud"),
            ("7", "Azure OpenAI", "azure_openai", "cloud"),
            ("8", "Google AI (Gemini)", "google", "cloud"),
            ("9", "Google Vertex AI", "google_vertex", "cloud"),
            ("10", "AWS Bedrock", "aws_bedrock", "cloud"),
            ("11", "xAI Grok", "xai", "cloud"),
            ("12", "Mistral AI", "mistral", "cloud"),
            ("13", "Groq", "groq", "cloud"),
            ("14", "Cerebras", "cerebras", "cloud"),
            ("15", "Cohere", "cohere", "cloud"),
            ("16", "DeepInfra", "deepinfra", "cloud"),
            ("17", "Together AI", "togetherai", "cloud"),
            ("18", "Perplexity", "perplexity", "cloud"),
            ("19", "OpenRouter", "openrouter", "cloud"),
            ("20", "GitHub Copilot", "github_copilot", "cloud"),
            ("21", "LM Studio (Local)", "lm_studio", "local"),
            ("22", "OpenAI-Compatible (Custom)", "openai_compatible", "custom"),
            ("23", "Skip (configure later)", "", ""),
        ]
        for opt, name, key, _ptype in providers:
            if key and key in config.providers.providers:
                provider = config.providers.providers[key]
                status = _get_provider_status(provider)
                table.add_row(f"  [{opt}]", name, status)
            elif key == "":
```

```
            table.add_row(f"  [{opt}]", name, "")
        else:
            table.add_row(f"  [{opt}]", name, "[dim]not configured[/dim]")
    console.print(table)
    console.print()
    choice = Prompt.ask(
        "Select provider",
        choices=[str(i) for i in range(1, 24)],
        default="1",
    )
    provider_map = {
        "1": "ollama",
        "2": "opencode",
        "3": "minimax",
        "4": "aliyun_coding_plan",
        "5": "anthropic",
        "6": "openai",
        "7": "azure_openai",
        "8": "google",
        "9": "google_vertex",
        "10": "aws_bedrock",
        "11": "xai",
        "12": "mistral",
        "13": "groq",
        "14": "cerebras",
        "15": "cohere",
        "16": "deepinfra",
        "17": "togetherai",
        "18": "perplexity",
        "19": "openrouter",
        "20": "github_copilot",
        "21": "lm_studio",
        "22": "openai_compatible",
    }
    if choice == "23":
        console.print("\n[yellow]You can configure later by running:[/yellow]")
        console.print("  openlaoke --config")
        console.print()
        return config
    provider_key = provider_map[choice]
    config.providers = _configure_provider(config.providers, provider_key)
    if _check_provider_ready(config.providers, provider_key):
        set_as_default = Confirm.ask(
            f"\nSet {provider_key} as your default provider?",
            default=True,
        )
        if set_as_default:
            config.providers.active_provider = provider_key
            console.print(f"[green][OK][/green] Active provider set to: {provider_key}")
            break
    else:
        console.print("\n[yellow]Provider not fully configured. Try again?[/yellow]")
        if not Confirm.ask("Continue setup?", default=True):
            break
# Step 2: Configure proxy
console.print()
config = _configure_proxy(config)
# Done
console.print()
console.print("[bold green]Configuration complete![/bold green]")
console.print(f"\nActive provider: [cyan]{config.providers.active_provider}[/cyan]")
console.print(f"Active model: [cyan]{config.providers.get_active_model()}[/cyan]")
console.print(f"Proxy: [cyan]{_get_proxy_display(config)}[/cyan]")
console.print()
config.first_run = False
```

```
    return config
def _get_proxy_display(config: AppConfig) -> str:
    if config.proxy_mode == "none":
        return "disabled"
    elif config.proxy_mode == "system":
        return "use system proxy"
    elif config.proxy_mode == "custom":
        return config.proxy_url or "(not set)"
    return "unknown"
def _get_env_var_name(provider_type: ProviderType) -> str:
    """Get environment variable name for a provider type."""
    env_map = {
        ProviderType.ANTHROPIC: "ANTHROPIC_API_KEY",
        ProviderType.OPENAI: "OPENAI_API_KEY",
        ProviderType.MINIMAX: "MINIMAX_API_KEY",
        ProviderType.ALIYUN_CODING_PLAN: "ALIYUN_API_KEY",
        ProviderType.AZURE_OPENAI: "AZURE_OPENAI_API_KEY",
        ProviderType.GOOGLE: "GOOGLE_API_KEY",
        ProviderType.GOOGLE_VERTEX: "GOOGLE_APPLICATION_CREDENTIALS",
        ProviderType.AWS_BEDROCK: "AWS_ACCESS_KEY_ID",
        ProviderType.XAI: "XAI_API_KEY",
        ProviderType.MISTRAL: "MISTRAL_API_KEY",
        ProviderType.GROQ: "GROQ_API_KEY",
        ProviderType.CEREBRAS: "CEREBRAS_API_KEY",
        ProviderType.COHERE: "COHERE_API_KEY",
        ProviderType.DEEPINFRA: "DEEPINFRA_API_KEY",
        ProviderType.TOGETHERAI: "TOGETHERAI_API_KEY",
        ProviderType.PERPLEXITY: "PERPLEXITY_API_KEY",
        ProviderType.OPENROUTER: "OPENROUTER_API_KEY",
        ProviderType.GITHUB_COPILOT: "GITHUB_TOKEN",
        ProviderType.OPENCODE: "OPENCODE_API_KEY",
        ProviderType.OPENAI_COMPATIBLE: "OPENAI_API_KEY",
    }
    return env_map.get(provider_type, "")
def _get_provider_status(provider: ProviderConfig) -> str:
    """Get human-readable status for a provider's API key configuration."""
    if provider.is_local:
        return "[green][OK] local[/green]"
    if provider.provider_type == ProviderType.OPENCODE:
        return "[green][OK] FREE (no key needed)[/green]"
    if provider.api_key:
        return "[green][OK] stored[/green]"
    env_var_name = _get_env_var_name(provider.provider_type)
    if env_var_name and os.environ.get(env_var_name):
        return "[green][OK] env var[/green]"
    return "[yellow]needs setup[/yellow]"
def _check_provider_ready(config: MultiProviderConfig, key: str) -> bool:
    """Check if a provider is ready to use."""
    provider = config.providers.get(key)
    if not provider:
        return False
    return provider.is_configured()
def _configure_proxy(config: AppConfig) -> AppConfig:
    """Configure proxy settings."""
    console.print("[bold]Step 2: Proxy Configuration[/bold]")
    console.print("[dim]Configure network proxy for API requests[/dim]\n")
    table = Table(show_header=False, box=None)
    table.add_column("Option", style="cyan")
    table.add_column("Mode", style="bold")
    table.add_column("Description", style="dim")
    table.add_row("  [1]", "No proxy", "Direct connection (default)")
    table.add_row("  [2]", "System proxy", "Use system HTTP/HTTPS proxy settings")
    table.add_row("  [3]", "Custom proxy", "Specify a custom proxy URL")
    console.print(table)
    console.print()
```

```
    choice = Prompt.ask(
        "Select proxy mode",
        choices=["1", "2", "3"],
        default="1",
    )
    if choice == "1":
        config.proxy_mode = "none"
        config.proxy_url = ""
        console.print("[green][OK][/green] Proxy disabled")
    elif choice == "2":
        config.proxy_mode = "system"
        config.proxy_url = ""
        system_proxy = os.environ.get("HTTP_PROXY") or os.environ.get("HTTPS_PROXY") or ""
        if system_proxy:
            console.print(f"[green][OK][/green] Will use system proxy: {system_proxy}")
        else:
            console.print("[green][OK][/green] Will use system proxy (none detected)")
    elif choice == "3":
        config.proxy_mode = "custom"
        default_url = config.proxy_url or "http://127.0.0.1:7890"
        proxy_url = Prompt.ask(
            "Enter proxy URL",
            default=default_url,
        )
        config.proxy_url = proxy_url
        console.print(f"[green][OK][/green] Custom proxy set: {proxy_url}")
    return config
def _configure_provider(config: MultiProviderConfig, key: str) -> MultiProviderConfig:
    """Configure a specific provider."""
    provider = config.providers.get(key)
    if not provider:
        return config
    console.print(f"\n[bold]Configuring {key}[/bold]")
    # Check available configurations
    stored_key = provider.api_key
    env_var_name = _get_env_var_name(provider.provider_type)
    env_key = os.environ.get(env_var_name, "")
    # Show configuration options
    if stored_key or env_key:
        console.print()
        table = Table(show_header=False, box=None)
        table.add_column("Option", style="cyan")
        table.add_column("Source", style="bold")
        table.add_column("Details", style="dim")
        options = []
        default_choice = "1"
        if stored_key:
            masked_key = (
                f"{stored_key[:8]}...{stored_key[-4:]}" if len(stored_key) > 12 else stored_key
            )
            table.add_row(
                "  [1]", "Stored config", f"Key: {masked_key}, Model: {provider.default_model}"
            )
            options.append("1")
            default_choice = "1"
        if env_key:
            masked_env = f"{env_key[:8]}...{env_key[-4:]}" if len(env_key) > 12 else env_key
            option_num = str(len(options) + 1)
            table.add_row(f"  [{option_num}]", "Environment var", f"{env_var_name}={masked_env}
    ")
            options.append(option_num)
            if not stored_key:
                default_choice = option_num
        option_num = str(len(options) + 1)
        table.add_row(f"  [{option_num}]", "Reconfigure", "Enter new API key and settings")
```

```
    options.append(option_num)
    console.print(table)
    console.print()
    choice = Prompt.ask(
        "Select configuration source",
        choices=options,
        default=default_choice,
    )
    # Use stored configuration (complete config, can return)
    if choice == "1" and stored_key:
        console.print(f"[green][OK] Using stored configuration for {key}[/green]")
        return config
    # Use environment variable (need to continue config for model selection)
    if env_key:
        env_option_idx = 2 if stored_key else 1
        if choice == str(env_option_idx):
            provider.api_key = env_key
            console.print(f"[green][OK] Using {env_var_name} from environment[/green]")
            console.print("[dim]Continue with model selection...[/dim]\n")
        elif choice == options[-1]:
            # Reconfigure - will ask for new API key below
            console.print("\n[yellow]Reconfiguring...[/yellow]")
    elif choice == options[-1]:
        console.print("\n[yellow]Reconfiguring...[/yellow]")
console.print("[dim]Press Enter to use default value[/dim]\n")
if provider.is_local:
    base_url = Prompt.ask(
        "API URL",
        default=provider.base_url,
    )
    provider.base_url = base_url
    if key == "ollama":
        models = _detect_ollama_models(base_url)
        if models:
            console.print(f"[green][OK] Detected {len(models)} models from Ollama[/green]")
            provider.models = models
            provider.default_model = _select_model_from_list(models, provider.default_model
)
        else:
            console.print("\n[yellow]? Could not detect models from Ollama.[/yellow]")
            console.print("[dim]Make sure Ollama is running: ollama serve[/dim]")
            console.print("[dim]Or pull some models: ollama pull gemma3:1b[/dim]\n")
            console.print("[bold]Using default model list (you can change this later):[/bol
d]")
            provider.default_model = _select_model_from_list(
                provider.models, provider.default_model
            )
    else:
        provider.default_model = _select_model_from_list(
            provider.models, provider.default_model
        )
elif provider.provider_type == ProviderType.OPENAI_COMPATIBLE:
    console.print("[dim]Enter your custom OpenAI-compatible API details[/dim]")
    base_url = Prompt.ask(
        "API Base URL",
        default=provider.base_url or "http://localhost:8000/v1",
    )
    provider.base_url = base_url
    # Ask for API key if not already set
    if not provider.api_key:
        api_key = Prompt.ask(
            "API Key (required for this provider)",
            password=True,
        )
        provider.api_key = api_key
```

```
    if not provider.api_key:
        console.print("[bold red]Warning: No API key provided. API calls may fail.[/bold re
d]")
    default_model = Prompt.ask(
        "Model name",
        default=provider.default_model or provider.models[0] if provider.models else "defau
lt",
    )
    provider.default_model = default_model
elif provider.provider_type == ProviderType.MINIMAX:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter MiniMax API Key", password=True)
    base_url = Prompt.ask(
        "API Base URL (press Enter for default)",
        default=provider.base_url,
    )
    provider.base_url = base_url
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.ALIYUN_CODING_PLAN:
    if not provider.api_key:
        provider.api_key = Prompt.ask(
            "Enter Aliyun Coding Plan API Key (format: sk-sp-xxxxx)", password=True
        )
    base_url = Prompt.ask(
        "API Base URL (press Enter for default)",
        default=provider.base_url,
    )
    provider.base_url = base_url
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.AZURE_OPENAI:
    console.print("[dim]Azure OpenAI requires endpoint and API key[/dim]")
    # Endpoint configuration
    env_endpoint = os.environ.get("AZURE_OPENAI_ENDPOINT", "")
    if env_endpoint and not provider.base_url:
        console.print("[green][OK] Found AZURE_OPENAI_ENDPOINT in environment[/green]")
        use_env = Confirm.ask("Use this endpoint?", default=True)
        if use_env:
            provider.base_url = env_endpoint
    if not provider.base_url:
        provider.base_url = Prompt.ask(
            "Enter Azure OpenAI Endpoint (e.g., https://your-resource.openai.azure.com)"
        )
    # API key configuration
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Azure OpenAI API Key", password=True)
    console.print(
        f"\n[dim]Available models (deployment names): {', '.join(provider.models)}[/dim]"
    )
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.GOOGLE:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Google AI API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.GOOGLE_VERTEX:
    console.print("[dim]Google Vertex AI uses GCP Application Default Credentials[/dim]")
    env_project = os.environ.get("GOOGLE_VERTEX_PROJECT", "")
    if env_project:
        console.print("[green][OK] Found GOOGLE_VERTEX_PROJECT in environment[/green]")
```

```
    else:
        project_id = Prompt.ask("Enter GCP Project ID")
        os.environ["GOOGLE_VERTEX_PROJECT"] = project_id
    env_location = os.environ.get("GOOGLE_VERTEX_LOCATION", "")
    if env_location:
        console.print("[green][OK] Found GOOGLE_VERTEX_LOCATION in environment[/green]")
    else:
        location = Prompt.ask("Enter GCP Location (e.g., us-central1)", default="us-central
1")
        os.environ["GOOGLE_VERTEX_LOCATION"] = location
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.AWS_BEDROCK:
    console.print("[dim]AWS Bedrock uses AWS credentials from environment[/dim]")
    env_key = os.environ.get("AWS_ACCESS_KEY_ID", "")
    if env_key:
        console.print("[green][OK] Found AWS_ACCESS_KEY_ID in environment[/green]")
    else:
        console.print(
            "[yellow]AWS credentials not found. Configure with AWS CLI or set environment v
ariables[/yellow]"
        )
    env_region = os.environ.get("AWS_REGION", "")
    if env_region:
        console.print("[green][OK] Found AWS_REGION in environment[/green]")
    else:
        region = Prompt.ask("Enter AWS Region (e.g., us-east-1)", default="us-east-1")
        os.environ["AWS_REGION"] = region
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.XAI:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter xAI API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.MISTRAL:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Mistral API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.GROQ:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Groq API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.CEREBRAS:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Cerebras API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.COHERE:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Cohere API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.DEEPINFRA:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter DeepInfra API Key", password=True)
```

```
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.TOGETHERAI:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Together AI API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.PERPLEXITY:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter Perplexity API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.OPENROUTER:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter OpenRouter API Key", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.GITHUB_COPILOT:
    if not provider.api_key:
        provider.api_key = Prompt.ask("Enter GitHub Personal Access Token", password=True)
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
l)
elif provider.provider_type == ProviderType.OPENCODE:
    console.print("[green][OK] No API key required - completely free![/green]")
    console.print("\n[bold]Available free models:[/bold]")
    for i, model in enumerate(provider.models, 1):
        marker = " [cyan](default)[/cyan]" if model == provider.default_model else ""
        console.print(f"  [{i}] {model}{marker}")
    choices = [str(i) for i in range(1, len(provider.models) + 1)]
    selection = Prompt.ask(
        "\nSelect model",
        choices=choices,
        default="1",
    )
    provider.default_model = provider.models[int(selection) - 1]
    provider.enabled = True
    config.providers[key] = provider
    console.print(f"\n[green][OK] Configured {key} with model {provider.default_model}[/gre
en]")
    return config
else:
    if not provider.api_key:
        api_key = Prompt.ask(
            f"Enter your {key} API Key",
            password=True,
        )
        provider.api_key = api_key
    if key == "anthropic":
        base_url = Prompt.ask(
            "API Base URL (press Enter for default)",
            default=provider.base_url,
        )
    else:
        base_url = Prompt.ask(
            "API Base URL (press Enter for default)",
            default=provider.base_url,
        )
    provider.base_url = base_url
    console.print(f"\n[dim]Available models: {', '.join(provider.models)}[/dim]")
    provider.default_model = _select_model_from_list(provider.models, provider.default_mode
```

```
    l)
    provider.enabled = True
    config.providers[key] = provider
    return config
def _select_model_from_list(models: list[str], default: str) -> str:
    """Show a numbered list of models and let user select one."""
    if not models:
        return default
    if len(models) == 1:
        console.print(f"[green]Using model: {models[0]}[/green]")
        return models[0]
    # Find preferred default
    preferred_defaults = [
        "gemma3:1b",
        "gemma4:e4b",
        "llama3.2",
        "gpt-4o",
        "claude-sonnet-4-20250514",
    ]
    actual_default = default
    for pref in preferred_defaults:
        if pref in models:
            actual_default = pref
            break
    console.print("\n[bold]Available models:[/bold]")
    for i, model in enumerate(models, 1):
        marker = " [cyan](default)[/cyan]" if model == actual_default else ""
        console.print(f"  [{i}] {model}{marker}")
    default_idx = models.index(actual_default) + 1 if actual_default in models else 1
    choices = [str(i) for i in range(1, len(models) + 1)]
    selection = Prompt.ask(
        "\nSelect model",
        choices=choices,
        default=str(default_idx),
    )
    return models[int(selection) - 1]
def _detect_ollama_models(base_url: str) -> list[str]:
    """Try to detect available Ollama models."""
    try:
        import httpx
        url = base_url.replace("/v1", "/api/tags")
        with httpx.Client(timeout=5.0, proxy=None) as client:
            response = client.get(url)
            if response.status_code == 200:
                data = response.json()
                all_models = [m["name"] for m in data.get("models", [])]
                embedding_keywords = [
                    "embed",
                    "embedding",
                    "rerank",
                    "reranker",
                    "minilm",
                    "arctic-embed",
                    "bge-",
                    "nomic-embed",
                    "paraphrase",
                    "granite-embedding",
                ]
                generation_models = []
                for model in all_models:
                    model_lower = model.lower()
                    is_embedding = any(kw in model_lower for kw in embedding_keywords)
                    if not is_embedding:
                        generation_models.append(model)
                return generation_models if generation_models else all_models
```

```
    except Exception:
        pass
    return []
def show_current_config(config: AppConfig) -> None:
    """Display current configuration."""
    console.print("\n[bold]Current Configuration[/bold]\n")
    # Providers table
    table = Table(title="Providers")
    table.add_column("Provider", style="cyan")
    table.add_column("Model", style="green")
    table.add_column("Status", style="yellow")
    table.add_column("Active", style="magenta")
    for name, provider in config.providers.providers.items():
        status = _get_provider_status(provider)
        active = "[OK]" if name == config.providers.active_provider else ""
        model = provider.default_model
        table.add_row(name, model, status, active)
    console.print(table)
    # Proxy info
    console.print(f"\n[bold]Proxy:[/bold] {_get_proxy_display(config)}")
    console.print()
def quick_setup(provider: str, api_key: str, model: str = "") -> AppConfig:
    """Quick setup for a single provider (non-interactive)."""
    config = AppConfig()
    if provider in config.providers.providers:
        config.providers.providers[provider].api_key = api_key
        if model:
            config.providers.providers[provider].default_model = model
        config.providers.active_provider = provider
        config.first_run = False
    return config
def get_proxy_url(config: AppConfig) -> str | None:
    """Get the effective proxy URL from config."""
    if config.proxy_mode == "none":
        return None
    elif config.proxy_mode == "system":
        proxy = os.environ.get("HTTP_PROXY") or os.environ.get("HTTPS_PROXY") or None
        return proxy if proxy and proxy != "" else None
    elif config.proxy_mode == "custom":
        return config.proxy_url if config.proxy_url else None
    return None
```

File: [39/252] openlaoke/core/config_wizard_extended_web.py

```
"""Extended Web configuration in wizard."""
from __future__ import annotations
import asyncio
from rich.console import Console
from rich.prompt import Confirm, Prompt
from .extended_web.config_integration import extract_cookie_in_wizard
from .extended_web import BrowserAuthManager
console = Console()
async def configure_extended_web(config, provider, key: str) -> bool:
    """Configure Extended Web provider in wizard.
    Args:
        config: AppConfig
        provider: ProviderConfig
        key: Provider key
    Returns:
        True if configured successfully
    """
    console.print("[bold cyan]? Extended Web Configuration[/bold cyan]")
    console.print()
    console.print("[dim]Extended Web uses browser-based authentication.[/dim]")
    console.print()
    # Check for existing authentication
    auth_manager = BrowserAuthManager()
    saved_auths = auth_manager.list_saved_auths()
    # Validate that saved auths have real cookies (not placeholders)
    valid_auths = []
    for auth_name in saved_auths:
        auth_data = auth_manager.load_auth(auth_name)
        if auth_data:
            cookie = auth_data.get("cookie", "")
            # Skip if cookie is a placeholder
            if cookie and cookie != "placeholder_need_manual_extraction" and len(cookie) > 20:
                valid_auths.append(auth_name)
    if valid_auths:
        console.print(f"[green][OK] Found {len(valid_auths)} authenticated service(s):[/green]"
    )
        for auth in valid_auths:
            console.print(f"  * {auth}")
        console.print()
        # Show ALL available models, mark authenticated ones
        console.print("[bold]All available web services:[/bold]")
        for i, model in enumerate(provider.models, 1):
            is_authed = " [green][OK] authenticated[/green]" if model in valid_auths else ""
            console.print(f"  [{i}] {model}{is_authed}")
        console.print()
        console.print("[bold]Select which service to configure as default:[/bold]")
        choices = [str(i) for i in range(1, len(provider.models) + 1)]
        selection = Prompt.ask(
            "Select service",
            choices=choices,
            default="1",
        )
        provider.default_model = provider.models[int(selection) - 1]
        # Check if selected model is authenticated (with real cookies)
        is_selected_authed = provider.default_model in valid_auths
        if is_selected_authed:
            console.print()
            console.print(f"[green][OK] {provider.default_model} is already authenticated![/gre
    en]")
            provider.enabled = True
            config.providers[key] = provider
```

```
        return True
else:
    console.print("[yellow]? No authenticated services found.[/yellow]")
    console.print()
    # Show ALL available models
    console.print("[bold]Available web services:[/bold]")
    for i, model in enumerate(provider.models, 1):
        console.print(f"  [{i}] {model}")
    console.print()
    selection = Prompt.ask(
        "Select default model",
        choices=[str(i) for i in range(1, len(provider.models) + 1)],
        default="1",
    )
    provider.default_model = provider.models[int(selection) - 1]
    is_selected_authed = False
provider.enabled = True
config.providers[key] = provider
console.print(f"\n[green][OK] Configured {key} with model {provider.default_model}[/green]"
)
# If not authenticated, MUST authenticate now
if not is_selected_authed:
    console.print()
    console.print("[bold red]? Authentication Required![/bold red]")
    console.print(
        f"[dim]The service {provider.default_model} requires browser authentication.[/dim]"
    )
    console.print()
    # Always authenticate - no skip option
    console.print("[bold]Starting automatic authentication...[/bold]")
    console.print()
    console.print("[dim]This will:[/dim]")
    console.print("  [dim]1. Open your default browser automatically[/dim]")
    console.print("  [dim]2. Navigate to the login page[/dim]")
    console.print("  [dim]3. Wait for you to login[/dim]")
    console.print("  [dim]4. Extract cookies automatically[/dim]")
    console.print()
    if not Confirm.ask("  Continue with authentication?", default=True):
        console.print("[yellow]? Authentication cancelled[/yellow]")
        return False
    # Run async authentication
    auth_success = await extract_cookie_in_wizard(provider.default_model)
    if not auth_success:
        console.print()
        console.print("[yellow]? Authentication was not completed.[/yellow]")
        console.print("[dim]You can authenticate later by running:[/dim]")
        console.print(f"  python3 -m openlaoke.core.extended_web.auto_cookie_extractor")
        return False
# Save the extended_web provider config
provider.enabled = True
config.providers[key] = provider
return True
```

File: [40/252] openlaoke/core/doc_downloader.py

```
"""Document downloader and indexer for knowledge base.
Downloads official documentation and creates searchable knowledge base
for small models to learn from.
"""
from __future__ import annotations
import hashlib
import json
import re
import time
from dataclasses import dataclass
from pathlib import Path
from typing import Any
import requests
from openlaoke.core.doc_sources import DOC_SOURCES, LANGUAGE_ALIASES
@dataclass
class DocumentChunk:
    doc_id: str
    source: str
    title: str
    content: str
    url: str
    category: str
    language: str
    chunk_index: int
    metadata: dict[str, Any]
class DocumentationDownloader:
    """Download and process documentation from official sources."""
    def __init__(self, cache_dir: Path | None = None):
        self.cache_dir = cache_dir or Path.home() / ".openlaoke" / "docs"
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        self.session = requests.Session()
        self.session.headers.update(
            {
                "User-Agent": "Mozilla/5.0 (compatible; OpenLaoKe/0.1.0; +https://github.com/cy
    cleuser/OpenLaoKe)"
            }
        )
    def download_source(self, source_id: str, force: bool = False) -> list[DocumentChunk]:
        """Download all URLs for a source."""
        if source_id not in DOC_SOURCES:
            print(f"Unknown source: {source_id}")
            return []
        source_info = DOC_SOURCES[source_id]
        source_name = source_info["name"]
        urls = source_info["urls"]
        category = source_info.get("category", "general")
        language = self._detect_language(source_id, category)
        all_chunks = []
        for url_idx, url in enumerate(urls):
            cache_file = self._get_cache_path(source_id, url_idx)
            if cache_file.exists() and not force:
                print(f"[{source_name}] Loading from cache: {url}")
                with open(cache_file, encoding="utf-8") as f:
                    cached_data = json.load(f)
                content = cached_data["content"]
                title = cached_data.get("title", f"{source_name} - Part {url_idx + 1}")
            else:
                print(f"[{source_name}] Downloading: {url}")
                content, title = self._fetch_url(url)
                if not content:
                    print(f"[{source_name}] Failed to download: {url}")
```

```
                continue
            cached_data = {
                "content": content,
                "title": title,
                "url": url,
                "downloaded_at": time.time(),
            }
            with open(cache_file, "w", encoding="utf-8") as f:
                json.dump(cached_data, f, ensure_ascii=False, indent=2)
        chunks = self._chunk_document(
            content=content,
            source=source_name,
            title=title,
            url=url,
            category=category,
            language=language,
            source_id=source_id,
        )
        all_chunks.extend(chunks)
    print(f"[{source_name}] Total chunks: {len(all_chunks)}")
    return all_chunks
def download_multiple_sources(
    self, source_ids: list[str], force: bool = False
) -> list[DocumentChunk]:
    """Download multiple sources."""
    all_chunks = []
    for source_id in source_ids:
        chunks = self.download_source(source_id, force=force)
        all_chunks.extend(chunks)
    return all_chunks
def download_task_relevant_docs(
    self, task_description: str, force: bool = False
) -> list[DocumentChunk]:
    """Download docs relevant to a task description."""
    task_lower = task_description.lower()
    source_ids = set()
    for keyword, sources in [
        ("benchmark", ["multiprocessing", "asyncio", "threading"]),
        ("web", ["flask", "fastapi", "requests"]),
        ("api", ["fastapi", "flask", "requests"]),
        ("data", ["numpy", "pandas"]),
        ("test", ["pytest", "unittest"]),
        ("cli", ["click", "argparse"]),
        ("database", ["sqlalchemy"]),
        ("file", ["pathlib", "json"]),
        ("async", ["asyncio"]),
        ("gui", ["tkinter"]),
    ]:
        if keyword in task_lower:
            source_ids.update(sources)
    for lang in ["python", "javascript", "typescript", "rust", "go", "java", "cpp"]:
        if lang in task_lower:
            source_ids.add(lang)
    if not source_ids:
        source_ids.update(["python", "numpy", "pathlib", "json"])
    print(f"Task-relevant sources: {', '.join(sorted(source_ids))}")
    return self.download_multiple_sources(list(source_ids), force=force)
def _fetch_url(self, url: str) -> tuple[str, str]:
    """Fetch content from URL."""
    try:
        response = self.session.get(url, timeout=30)
        response.raise_for_status()
        content = response.text
        title = self._extract_title(content, url)
        content = self._clean_content(content)
```

```
        return content, title
    except Exception as e:
        print(f"Error fetching {url}: {e}")
        return "", ""
def _extract_title(self, content: str, url: str) -> str:
    """Extract title from content or URL."""
    title_match = re.search(r"<title>(.*?)</title>", content, re.IGNORECASE)
    if title_match:
        return title_match.group(1).strip()
    h1_match = re.search(r"^#\s+(.+)$", content, re.MULTILINE)
    if h1_match:
        return h1_match.group(1).strip()
    path_parts = url.split("/")
    if path_parts:
        return path_parts[-1].replace(".html", "").replace(".md", "").replace("_", " ").tit
le()
    return "Untitled"
def _clean_content(self, content: str) -> str:
    """Clean HTML/Markdown content to plain text."""
    content = re.sub(r"<!--.*?-->", "", content, flags=re.DOTALL)
    content = re.sub(r"<script[^>]*>.*?</script>", "", content, flags=re.DOTALL | re.IGNORE
CASE)
    content = re.sub(r"<style[^>]*>.*?</style>", "", content, flags=re.DOTALL | re.IGNORECA
SE)
    content = re.sub(r"<nav[^>]*>.*?</nav>", "", content, flags=re.DOTALL | re.IGNORECASE)
    content = re.sub(r"<footer[^>]*>.*?</footer>", "", content, flags=re.DOTALL | re.IGNORE
CASE)
    content = re.sub(r"<[^>]+>", " ", content)
    content = re.sub(r"\n\s*\n", "\n\n", content)
    content = re.sub(r" {2,}", " ", content)
    return content.strip()
def _chunk_document(
    self,
    content: str,
    source: str,
    title: str,
    url: str,
    category: str,
    language: str,
    source_id: str,
    chunk_size: int = 2000,
) -> list[DocumentChunk]:
    """Split document into chunks for better retrieval."""
    chunks = []
    paragraphs = re.split(r"\n\s*\n", content)
    current_chunk = ""
    chunk_index = 0
    for para in paragraphs:
        if len(current_chunk) + len(para) + 2 > chunk_size and current_chunk:
            doc_id = self._generate_doc_id(source_id, chunk_index)
            chunks.append(
                DocumentChunk(
                    doc_id=doc_id,
                    source=source,
                    title=title,
                    content=current_chunk.strip(),
                    url=url,
                    category=category,
                    language=language,
                    chunk_index=chunk_index,
                    metadata={
                        "source_id": source_id,
                        "char_count": len(current_chunk),
                    },
                )
```

```
            )
            chunk_index += 1
            current_chunk = para + "\n\n"
        else:
            current_chunk += para + "\n\n"
    if current_chunk.strip():
        doc_id = self._generate_doc_id(source_id, chunk_index)
        chunks.append(
            DocumentChunk(
                doc_id=doc_id,
                source=source,
                title=title,
                content=current_chunk.strip(),
                url=url,
                category=category,
                language=language,
                chunk_index=chunk_index,
                metadata={
                    "source_id": source_id,
                    "char_count": len(current_chunk),
                },
            )
        )
    return chunks
def _detect_language(self, source_id: str, category: str) -> str:
    """Detect programming language from source ID."""
    if category == "programming_language":
        return source_id
    if source_id in LANGUAGE_ALIASES:
        return LANGUAGE_ALIASES[source_id]
    if source_id in [
        "python",
        "numpy",
        "pandas",
        "flask",
        "fastapi",
        "pytest",
        "django",
        "click",
        "rich",
        "pydantic",
    ]:
        return "python"
    if source_id in ["javascript", "react", "vue"]:
        return "javascript"
    if source_id in ["typescript"]:
        return "typescript"
    if source_id in ["rust"]:
        return "rust"
    if source_id in ["go"]:
        return "go"
    if source_id in ["java"]:
        return "java"
    if source_id in ["cpp"]:
        return "cpp"
    return "general"
def _get_cache_path(self, source_id: str, url_index: int) -> Path:
    """Get cache file path for a downloaded document."""
    safe_source = re.sub(r"[^\w\-]", "_", source_id)
    return self.cache_dir / f"{safe_source}_{url_index}.json"
def _generate_doc_id(self, source_id: str, chunk_index: int) -> str:
    """Generate unique document ID."""
    hash_input = f"{source_id}_{chunk_index}_{time.time()}"
    hash_digest = hashlib.md5(hash_input.encode()).hexdigest()[:8]
    return f"{source_id}_{chunk_index}_{hash_digest}"
```

```
def create_downloader(cache_dir: Path | None = None) -> DocumentationDownloader:
    """Create a document downloader instance."""
    return DocumentationDownloader(cache_dir)
def download_knowledge_for_task(
    task_description: str, cache_dir: Path | None = None, force: bool = False
) -> list[DocumentChunk]:
    """Download relevant documentation for a task."""
    downloader = DocumentationDownloader(cache_dir)
    return downloader.download_task_relevant_docs(task_description, force=force)
```

File: [41/252] openlaoke/core/doc_sources.py

```
"""Document sources for knowledge base.
Inspired by GangDan project - comprehensive documentation sources
for all major programming languages and frameworks.
"""
from __future__ import annotations
DOC_SOURCES = {
    "python": {
        "name": "Python",
        "category": "programming_language",
        "urls": [
            "https://docs.python.org/3/tutorial/index.html",
            "https://docs.python.org/3/tutorial/controlflow.html",
            "https://docs.python.org/3/tutorial/datastructures.html",
            "https://docs.python.org/3/tutorial/modules.html",
            "https://docs.python.org/3/tutorial/classes.html",
            "https://docs.python.org/3/tutorial/errors.html",
            "https://docs.python.org/3/library/functions.html",
        ],
        "description": "Python official documentation - core language features",
    },
    "javascript": {
        "name": "JavaScript",
        "category": "programming_language",
        "urls": [
            "https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Introduction",
            "https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types",
            "https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_err
    or_handling",
            "https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration"
    ,
            "https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions",
        ],
        "description": "JavaScript MDN guide - modern ES6+ features",
    },
    "typescript": {
        "name": "TypeScript",
        "category": "programming_language",
        "urls": [
            "https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html",
            "https://www.typescriptlang.org/docs/handbook/basic-types.html",
            "https://www.typescriptlang.org/docs/handbook/interfaces.html",
            "https://www.typescriptlang.org/docs/handbook/classes.html",
            "https://www.typescriptlang.org/docs/handbook/functions.html",
        ],
        "description": "TypeScript handbook - type system and features",
    },
    "rust": {
        "name": "Rust",
        "category": "programming_language",
        "urls": [
            "https://doc.rust-lang.org/book/ch01-00-getting-started.html",
            "https://doc.rust-lang.org/book/ch03-00-common-programming-concepts.html",
            "https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html",
            "https://doc.rust-lang.org/book/ch05-00-structs.html",
            "https://doc.rust-lang.org/book/ch06-00-enums.html",
            "https://doc.rust-lang.org/book/ch10-00-generics.html",
        ],
        "description": "Rust book - ownership, borrowing, and type system",
    },
    "go": {
        "name": "Go",
```

```
    "category": "programming_language",
    "urls": [
        "https://go.dev/tour/welcome/1",
        "https://go.dev/tour/basics/1",
        "https://go.dev/tour/flowcontrol/1",
        "https://go.dev/tour/moretypes/1",
        "https://go.dev/tour/methods/1",
        "https://go.dev/tour/concurrency/1",
    ],
    "description": "Go tour - concurrency and simplicity",
},
"java": {
    "name": "Java",
    "category": "programming_language",
    "urls": [
        "https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html",
        "https://docs.oracle.com/javase/tutorial/java/concepts/index.html",
        "https://docs.oracle.com/javase/tutorial/java/javaOO/index.html",
    ],
    "description": "Java tutorial - OOP and language basics",
},
"cpp": {
    "name": "C++",
    "category": "programming_language",
    "urls": [
        "https://en.cppreference.com/w/cpp/language/functions",
        "https://en.cppreference.com/w/cpp/language/classes",
        "https://en.cppreference.com/w/cpp/language/templates",
    ],
    "description": "C++ reference - modern C++ features",
},
"numpy": {
    "name": "NumPy",
    "category": "python_library",
    "urls": [
        "https://numpy.org/doc/stable/user/absolute_beginners.html",
        "https://numpy.org/doc/stable/user/basics.creation.html",
        "https://numpy.org/doc/stable/user/basics.indexing.html",
        "https://numpy.org/doc/stable/reference/routines.array-creation.html",
    ],
    "description": "NumPy basics - array operations and numerical computing",
},
"pandas": {
    "name": "Pandas",
    "category": "python_library",
    "urls": [
        "https://pandas.pydata.org/docs/user_guide/10min.html",
        "https://pandas.pydata.org/docs/user_guide/indexing.html",
        "https://pandas.pydata.org/docs/user_guide/basics.html",
        "https://pandas.pydata.org/docs/user_guide/io.html",
    ],
    "description": "Pandas - data analysis and manipulation",
},
"requests": {
    "name": "Requests",
    "category": "python_library",
    "urls": [
        "https://requests.readthedocs.io/en/latest/user/quickstart/",
        "https://requests.readthedocs.io/en/latest/user/advanced/",
    ],
    "description": "Requests - HTTP library for Python",
},
"flask": {
    "name": "Flask",
    "category": "web_framework",
```

```
    "urls": [
        "https://flask.palletsprojects.com/en/latest/quickstart/",
        "https://flask.palletsprojects.com/en/latest/tutorial/",
    ],
    "description": "Flask - web framework for Python",
},
"fastapi": {
    "name": "FastAPI",
    "category": "web_framework",
    "urls": [
        "https://fastapi.tiangolo.com/tutorial/",
        "https://fastapi.tiangolo.com/tutorial/path-params/",
        "https://fastapi.tiangolo.com/tutorial/query-params/",
        "https://fastapi.tiangolo.com/tutorial/body/",
    ],
    "description": "FastAPI - modern async web framework",
},
"pytest": {
    "name": "Pytest",
    "category": "testing_framework",
    "urls": [
        "https://docs.pytest.org/en/stable/getting-started.html",
        "https://docs.pytest.org/en/stable/how-to/assert.html",
        "https://docs.pytest.org/en/stable/how-to/fixtures.html",
    ],
    "description": "Pytest - Python testing framework",
},
"react": {
    "name": "React",
    "category": "frontend_framework",
    "urls": [
        "https://react.dev/learn",
        "https://react.dev/learn/thinking-in-react",
        "https://react.dev/reference/react",
    ],
    "description": "React - UI component library",
},
"vue": {
    "name": "Vue.js",
    "category": "frontend_framework",
    "urls": [
        "https://vuejs.org/guide/introduction.html",
        "https://vuejs.org/guide/essentials/reactivity-fundamentals.html",
        "https://vuejs.org/guide/essentials/component-basics.html",
    ],
    "description": "Vue.js - progressive JavaScript framework",
},
"django": {
    "name": "Django",
    "category": "web_framework",
    "urls": [
        "https://docs.djangoproject.com/en/stable/intro/tutorial01/",
        "https://docs.djangoproject.com/en/stable/intro/tutorial02/",
        "https://docs.djangoproject.com/en/stable/topics/db/models/",
    ],
    "description": "Django - full-stack web framework",
},
"sqlalchemy": {
    "name": "SQLAlchemy",
    "category": "database",
    "urls": [
        "https://docs.sqlalchemy.org/en/20/tutorial/",
        "https://docs.sqlalchemy.org/en/20/core/engines.html",
        "https://docs.sqlalchemy.org/en/20/orm/quickstart.html",
    ],
```

```
    "description": "SQLAlchemy - ORM and database toolkit",
},
"multiprocessing": {
    "name": "Multiprocessing",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/multiprocessing.html",
    ],
    "description": "Python multiprocessing - parallel execution",
},
"asyncio": {
    "name": "AsyncIO",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/asyncio.html",
        "https://docs.python.org/3/library/asyncio-task.html",
    ],
    "description": "Python asyncio - asynchronous programming",
},
"threading": {
    "name": "Threading",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/threading.html",
    ],
    "description": "Python threading - concurrent execution",
},
"pathlib": {
    "name": "Pathlib",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/pathlib.html",
    ],
    "description": "Python pathlib - object-oriented filesystem paths",
},
"json": {
    "name": "JSON",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/json.html",
    ],
    "description": "Python json - JSON encoder and decoder",
},
"logging": {
    "name": "Logging",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/logging.html",
        "https://docs.python.org/3/howto/logging.html",
    ],
    "description": "Python logging - event logging system",
},
"argparse": {
    "name": "Argparse",
    "category": "python_library",
    "urls": [
        "https://docs.python.org/3/library/argparse.html",
    ],
    "description": "Python argparse - command-line argument parsing",
},
"unittest": {
    "name": "Unittest",
    "category": "testing_framework",
    "urls": [
        "https://docs.python.org/3/library/unittest.html",
```

```
        ],
        "description": "Python unittest - unit testing framework",
    },
    "typing": {
        "name": "Typing",
        "category": "python_library",
        "urls": [
            "https://docs.python.org/3/library/typing.html",
            "https://mypy.readthedocs.io/en/stable/cheat_sheet_py3.html",
        ],
        "description": "Python typing - type hints support",
    },
    "dataclasses": {
        "name": "Dataclasses",
        "category": "python_library",
        "urls": [
            "https://docs.python.org/3/library/dataclasses.html",
        ],
        "description": "Python dataclasses - data classes",
    },
    "pydantic": {
        "name": "Pydantic",
        "category": "python_library",
        "urls": [
            "https://docs.pydantic.dev/latest/",
            "https://docs.pydantic.dev/latest/usage/models/",
        ],
        "description": "Pydantic - data validation using Python type annotations",
    },
    "click": {
        "name": "Click",
        "category": "cli_framework",
        "urls": [
            "https://click.palletsprojects.com/en/latest/quickstart/",
            "https://click.palletsprojects.com/en/latest/options/",
            "https://click.palletsprojects.com/en/latest/arguments/",
        ],
        "description": "Click - command-line interface creation kit",
    },
    "rich": {
        "name": "Rich",
        "category": "python_library",
        "urls": [
            "https://rich.readthedocs.io/en/stable/introduction.html",
            "https://rich.readthedocs.io/en/stable/console.html",
            "https://rich.readthedocs.io/en/stable/tables.html",
        ],
        "description": "Rich - pretty printing and rich text formatting",
    },
}
LANGUAGE_ALIASES = {
    "py": "python",
    "js": "javascript",
    "ts": "typescript",
    "rs": "rust",
    "golang": "go",
    "cpp": "c++",
    "c++": "cpp",
}
FRAMEWORK_CATEGORIES = {
    "programming_language": [
        "python",
        "javascript",
        "typescript",
        "rust",
```

```
        "go",
        "java",
        "cpp",
    ],
    "python_library": [
        "numpy",
        "pandas",
        "requests",
        "multiprocessing",
        "asyncio",
        "threading",
        "pathlib",
        "json",
        "logging",
        "argparse",
        "typing",
        "dataclasses",
        "pydantic",
        "rich",
    ],
    "web_framework": [
        "flask",
        "fastapi",
        "django",
    ],
    "frontend_framework": [
        "react",
        "vue",
    ],
    "testing_framework": [
        "pytest",
        "unittest",
    ],
    "database": [
        "sqlalchemy",
    ],
    "cli_framework": [
        "click",
    ],
}
TASK_TO_KNOWLEDGE_MAPPING = {
    "benchmark": ["multiprocessing", "asyncio", "threading", "time"],
    "web_api": ["fastapi", "flask", "requests"],
    "data_analysis": ["numpy", "pandas"],
    "testing": ["pytest", "unittest"],
    "cli": ["click", "argparse", "rich"],
    "database": ["sqlalchemy", "json"],
    "file_io": ["pathlib", "json"],
    "async": ["asyncio", "aiohttp"],
    "gui": ["tkinter", "pyqt", "pyside"],
    "network": ["requests", "aiohttp", "socket"],
}
def get_doc_sources() -> dict[str, dict]:
    """Get all documentation sources."""
    return DOC_SOURCES
def get_sources_for_category(category: str) -> list[str]:
    """Get all sources for a given category."""
    return FRAMEWORK_CATEGORIES.get(category, [])
def get_sources_for_task(task_type: str) -> list[str]:
    """Get relevant sources for a task type."""
    return TASK_TO_KNOWLEDGE_MAPPING.get(task_type, [])
```

File: [42/252] openlaoke/core/dual_model_agent.py

```
"""Dual-model collaborative agent - Small model for planning, large model for execution.
This agent uses a small, cheap model (gemma3:1b) for task analysis, decomposition,
and validation, while using a large, capable model (gemma4:e4b) for actual code
generation. This reduces cost by 60-80% while improving quality.
Workflow:
1. Small model: Analyze request and decompose into atomic tasks
2. Large model: Generate code for each atomic task
3. Small model: Validate generated code
4. Large model: Fix failed code (if needed)
5. Assemble final result
"""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
from openlaoke.core.enhanced_knowledge_base import EnhancedKnowledgeBase
from openlaoke.core.intent_pipeline import create_pipeline_for_model
from openlaoke.core.model_assessment.types import ModelTier
if TYPE_CHECKING:
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.core.state import AppState
@dataclass
class DualModelStats:
    """Statistics for dual-model execution."""
    planner_calls: int = 0
    planner_tokens: int = 0
    executor_calls: int = 0
    executor_tokens: int = 0
    validator_calls: int = 0
    validator_tokens: int = 0
    total_time: float = 0.0
    total_cost: float = 0.0
    retry_count: int = 0
    preload_time: float = 0.0
    models_preloaded: bool = False
    def to_dict(self) -> dict[str, Any]:
        return {
            "planner": {
                "calls": self.planner_calls,
                "tokens": self.planner_tokens,
            },
            "executor": {
                "calls": self.executor_calls,
                "tokens": self.executor_tokens,
            },
            "validator": {
                "calls": self.validator_calls,
                "tokens": self.validator_tokens,
            },
            "total_time": self.total_time,
            "total_cost": self.total_cost,
            "retry_count": self.retry_count,
        }
@dataclass
class DualModelResult:
    """Result from dual-model execution."""
    success: bool
    code: str
    stats: DualModelStats = field(default_factory=DualModelStats)
    errors: list[str] = field(default_factory=list)
```

```
    warnings: list[str] = field(default_factory=list)
    output_file: Path | None = None
class DualModelAgent:
    """Small model (planner) + Large model (executor) collaborative agent."""
    PLANNER_MODEL = "gemma3:1b"
    EXECUTOR_MODEL = "gemma4:e4b"
    VALIDATOR_MODEL = "gemma3:1b"
    MAX_EXECUTOR_TOKENS = 1000
    MAX_VALIDATOR_TOKENS = 300
    MAX_RETRIES = 3
    def __init__(self, app_state: AppState) -> None:
        self.app_state = app_state
        self._api_client: MultiProviderClient | None = None
        self.pipeline = create_pipeline_for_model(ModelTier.TIER_5_LIMITED)
        self.knowledge_base = EnhancedKnowledgeBase()
        self._hybrid_manager = None
        self._models_selected = False
    async def _select_optimal_models(self) -> dict[str, str]:
        """Select optimal models based on configuration or available resources."""
        from openlaoke.core.dual_model_config import create_config_manager
        config_manager = create_config_manager(self.app_state)
        config = config_manager.get_config()
        if config and config.planner and config.executor:
            self.PLANNER_MODEL = config.planner.model_name
            self.EXECUTOR_MODEL = config.executor.model_name
            if config.validator:
                self.VALIDATOR_MODEL = config.validator.model_name
            self._models_selected = True
            return {
                "planner": self.PLANNER_MODEL,
                "executor": self.EXECUTOR_MODEL,
                "validator": self.VALIDATOR_MODEL,
                "config_name": config.name,
            }
        from openlaoke.core.intelligent_model_selector import get_optimal_models
        models = await get_optimal_models(self.app_state)
        self.PLANNER_MODEL = models["planner"]
        self.EXECUTOR_MODEL = models["executor"]
        self.VALIDATOR_MODEL = models["validator"]
        self._models_selected = True
        return models
    async def _get_hybrid_manager(self):
        """Get or create hybrid model manager."""
        if self._hybrid_manager is None:
            from openlaoke.core.hybrid_model_manager import create_hybrid_manager
            self._hybrid_manager = create_hybrid_manager(self.app_state)
            await self._hybrid_manager.initialize()
        return self._hybrid_manager
    async def _get_api_client(self) -> MultiProviderClient:
        """Get or create API client."""
        if self._api_client is None:
            from openlaoke.core.multi_provider_api import MultiProviderClient
            if self.app_state.multi_provider_config:
                self._api_client = MultiProviderClient(self.app_state.multi_provider_config)
            else:
                from openlaoke.types.providers import MultiProviderConfig
                config = MultiProviderConfig.defaults()
                self._api_client = MultiProviderClient(config)
        return self._api_client
    async def execute(self, request: str, working_dir: Path | None = None) -> DualModelResult:
        """Execute a request using dual-model collaboration."""
        start_time = time.time()
        stats = DualModelStats()
        try:
            preload_start = time.time()
```

```
        if not self._models_selected:
            await self._select_optimal_models()
        manager = await self._get_hybrid_manager()
        init_result = await manager.initialize()
        stats.preload_time = time.time() - preload_start
        stats.models_preloaded = init_result["success"]
        api_client = await self._get_api_client()
        stats.planner_calls += 1
        plan_result = self.pipeline.process_request(request)
        if not plan_result.success:
            return DualModelResult(
                success=False,
                code="",
                stats=stats,
                errors=["Planning failed: " + str(plan_result.errors)],
            )
        stats.planner_tokens += 500
        if not plan_result.task_graph:
            return DualModelResult(
                success=False,
                code="",
                stats=stats,
                errors=["No task graph generated"],
            )
        graph = plan_result.task_graph
        completed_code: dict[str, str] = {}
        iteration = 0
        max_iterations = len(graph.tasks) * 2
        while len(graph.completed) < len(graph.tasks) and iteration < max_iterations:
            ready_tasks = graph.get_ready_tasks()
            if not ready_tasks:
                break
            for task in ready_tasks[:2]:
                stats.executor_calls += 1
                code = await self._execute_task_with_large_model(
                    api_client, task, completed_code
                )
                stats.executor_tokens += self.MAX_EXECUTOR_TOKENS
                stats.validator_calls += 1
                is_valid, errors = await self._validate_with_small_model(api_client, task,
code)
                stats.validator_tokens += self.MAX_VALIDATOR_TOKENS
                if not is_valid and stats.retry_count < self.MAX_RETRIES:
                    stats.retry_count += 1
                    stats.executor_calls += 1
                    code = await self._execute_task_with_large_model(
                        api_client, task, completed_code, errors
                    )
                    stats.executor_tokens += self.MAX_EXECUTOR_TOKENS
                    is_valid, errors = await self._validate_with_small_model(
                        api_client, task, code
                    )
                    stats.validator_tokens += self.MAX_VALIDATOR_TOKENS
                if is_valid:
                    completed_code[task.task_id] = code
                    graph.mark_completed(task.task_id)
                else:
                    graph.mark_failed(task.task_id)
            iteration += 1
        final_code = self._assemble_code(graph, completed_code)
        if working_dir and final_code:
            task_name = plan_result.intent.task_name.replace(" ", "_")
            output_file = working_dir / f"{task_name}.py"
            output_file.write_text(final_code)
        else:
```

```
                output_file = None
            stats.total_time = time.time() - start_time
            stats.total_cost = self._estimate_cost(stats)
            return DualModelResult(
                success=len(graph.completed) > 0,
                code=final_code,
                stats=stats,
                errors=[],
                warnings=[],
                output_file=output_file,
            )
        except Exception as e:
            stats.total_time = time.time() - start_time
            return DualModelResult(
                success=False,
                code="",
                stats=stats,
                errors=[f"Dual-model execution failed: {str(e)}"],
            )
    async def _execute_task_with_large_model(
        self,
        api_client: MultiProviderClient,
        task: Any,
        previous_code: dict[str, str],
        validation_errors: list[str] | None = None,
    ) -> str:
        """Execute code generation with large model."""
        prompt = self._build_executor_prompt(task, previous_code, validation_errors)
        response, tokens, _ = await api_client.send_message(
            system_prompt=self._get_executor_system_prompt(),
            messages=[{"role": "user", "content": prompt}],
            model=self.EXECUTOR_MODEL,
            max_tokens=self.MAX_EXECUTOR_TOKENS,
        )
        text = response.content if hasattr(response, "content") else str(response)
        code = self._extract_code_from_response(text)
        if not code:
            return self._generate_fallback_code(task)
        return code
    async def _validate_with_small_model(
        self,
        api_client: MultiProviderClient,
        task: Any,
        code: str,
    ) -> tuple[bool, list[str]]:
        """Validate generated code with small model."""
        if not code or len(code) < 10:
            return False, ["Code is empty or too short"]
        import ast
        try:
            ast.parse(code)
        except SyntaxError as e:
            return False, [f"Syntax error: {str(e)}"]
        lines = code.strip().split("\n")
        non_empty_lines = [
            line for line in lines if line.strip() and not line.strip().startswith("#")
        ]
        max_lines = max(task.estimated_lines * 2, 20)
        if len(non_empty_lines) > max_lines:
            return False, [f"Code too long: {len(non_empty_lines)} lines > {max_lines}"]
        if "def " in code and "return" not in code and "yield" not in code:
            return False, ["Function missing return statement"]
        prompt = f"""Quick check this Python code:
Task: {task.description[:100]}
Code (first 200 chars):
```

```
{code[:200]}
Does this code solve the task? Answer: PASS or FAIL"""
        try:
            response, tokens, _ = await api_client.send_message(
                system_prompt="You are a fast code validator. Check basic correctness only.",
                messages=[{"role": "user", "content": prompt}],
                model=self.VALIDATOR_MODEL,
                max_tokens=self.MAX_VALIDATOR_TOKENS,
            )
            text = response.content if hasattr(response, "content") else str(response)
            if "FAIL" in text.upper():
                return False, ["Validator detected issues"]
        except Exception:
            pass
        return True, []
    def _build_executor_prompt(
        self,
        task: Any,
        previous_code: dict[str, str],
        validation_errors: list[str] | None = None,
    ) -> str:
        """Build prompt for executor model."""
        parts = [
            f"Task ID: {task.task_id}",
            f"Description: {task.description}",
            f"Estimated lines: {task.estimated_lines}",
        ]
        if previous_code and task.dependencies:
            parts.append("\nPreviously completed code (use as reference):")
            for dep_id in task.dependencies[:2]:
                if dep_id in previous_code:
                    parts.append(f"\n--- {dep_id} ---")
                    parts.append(previous_code[dep_id][:150])
        if validation_errors:
            parts.append("\nValidation errors to fix:")
            for error in validation_errors[:3]:
                parts.append(f"  - {error}")
        knowledge = self._get_relevant_knowledge(task)
        if knowledge:
            parts.append("\nReference knowledge:")
            parts.append(knowledge["content"][:200])
        parts.append("\n\nGenerate the complete Python code now.")
        parts.append("Return ONLY the code, no explanations.")
        return "\n".join(parts)
    def _get_executor_system_prompt(self) -> str:
        """System prompt for executor model."""
        return """You are an expert Python code generator.
CRITICAL RULES:
1. Generate WORKING, COMPLETE code - NO placeholders like 'pass' or 'TODO'
2. Include necessary imports at the top
3. Keep it SIMPLE and ATOMIC - max 15 lines of logic
4. Add a brief docstring
5. Always include a return statement in functions
6. Return ONLY the Python code, no markdown formatting or explanations"""
    def _extract_code_from_response(self, response: str) -> str:
        """Extract Python code from model response."""
        lines = response.strip().split("\n")
        code_lines = []
        in_code_block = False
        for line in lines:
            if line.strip().startswith("```python"):
                in_code_block = True
                continue
            elif line.strip() == "```" and in_code_block:
                in_code_block = False
```

```
            continue
        elif in_code_block:
            code_lines.append(line)
    if code_lines:
        return "\n".join(code_lines)
    if "def " in response or "import " in response:
        code_lines = []
        for line in lines:
            stripped = line.strip()
            if (
                stripped.startswith("def ")
                or stripped.startswith("import ")
                or stripped.startswith("from ")
                or stripped.startswith("class ")
                or stripped.startswith("@")
                or stripped.startswith("return ")
                or (code_lines and (line.startswith("    ") or line.startswith("\t")))
            ):
                code_lines.append(line)
        if code_lines:
            return "\n".join(code_lines)
    return ""
def _get_relevant_knowledge(self, task: Any) -> dict[str, Any] | None:
    """Get relevant knowledge for the task."""
    task_description = task.description.lower()
    keywords = []
    if any(word in task_description for word in ["benchmark", "cpu", "performance", "measur
e"]):
        keywords.extend(["benchmark", "cpu"])
    if any(word in task_description for word in ["storage", "disk", "capacity", "size"]):
        keywords.extend(["file", "system", "os"])
    if any(word in task_description for word in ["calculate", "compute", "math"]):
        keywords.extend(["basics", "syntax"])
    for keyword in keywords:
        snippets = self.knowledge_base.search(keyword)
        if snippets:
            return {
                "topic": snippets[0].topic,
                "content": snippets[0].content,
                "source": snippets[0].source,
            }
    return None
def _assemble_code(self, graph: Any, completed_code: dict[str, str]) -> str:
    """Assemble final code from completed tasks."""
    sections = []
    import_tasks = [
        tid
        for tid, task in graph.tasks.items()
        if "import" in task.description.lower() or "import" in tid.lower()
    ]
    for task_id in import_tasks:
        if task_id in completed_code:
            sections.append(completed_code[task_id])
    main_tasks = [
        tid
        for tid, task in graph.tasks.items()
        if "import" not in task.description.lower()
        and "import" not in tid.lower()
        and "test" not in task.description.lower()
    ]
    for task_id in main_tasks:
        if task_id in completed_code:
            sections.append(completed_code[task_id])
    return "\n\n".join(sections)
def _generate_fallback_code(self, task: Any) -> str:
```

```
        """Generate fallback code if API fails."""
        if "import" in task.description.lower():
            return """from __future__ import annotations
import time
"""
        return f"""def {task.task_id.split("_")[-1]}() -> Any:
    \"\"\"{task.description}\"\"\"
    pass
"""
    def _estimate_cost(self, stats: DualModelStats) -> float:
        """Estimate total cost based on token usage."""
        planner_cost = stats.planner_tokens * 0.00001
        executor_cost = stats.executor_tokens * 0.0001
        validator_cost = stats.validator_tokens * 0.00001
        return planner_cost + executor_cost + validator_cost
def create_dual_model_agent(app_state: AppState) -> DualModelAgent:
    """Create a dual-model agent."""
    return DualModelAgent(app_state)
```

File: [43/252] openlaoke/core/dual_model_config.py

```
"""Flexible model configuration for dual-model workflow.
This module allows users to configure:
1. Planner model (local/online)
2. Executor model (local/online)
3. Validator model (local/online)
Each model can be:
- Local (Ollama, LM Studio, etc.)
- Online (OpenAI, Anthropic, Google, etc.)
- Custom provider
Example configurations:
1. Local + Local:
   planner: gemma3:1b (Ollama, CPU)
   executor: gemma4:e4b (Ollama, GPU)
2. Local + Online:
   planner: gemma3:1b (Ollama, CPU)
   executor: gpt-4 (OpenAI)
3. Online + Online:
   planner: gpt-3.5-turbo (OpenAI)
   executor: claude-3-opus (Anthropic)
"""
from __future__ import annotations
from dataclasses import dataclass
from enum import StrEnum
from pathlib import Path
from typing import TYPE_CHECKING, Any
import httpx
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class ModelProvider(StrEnum):
    """Model providers."""
    OLLAMA = "ollama"
    OPENAI = "openai"
    ANTHROPIC = "anthropic"
    GOOGLE = "google"
    DEEPSEEK = "deepseek"
    CUSTOM = "custom"
class ModelDevice(StrEnum):
    """Device for model execution."""
    CPU = "cpu"
    GPU = "gpu"
    AUTO = "auto"
@dataclass
class ModelEndpoint:
    """Model endpoint configuration."""
    provider: ModelProvider
    model_name: str
    api_base: str | None = None
    api_key: str | None = None
    device: ModelDevice = ModelDevice.AUTO
    max_tokens: int = 1000
    def to_dict(self) -> dict[str, Any]:
        return {
            "provider": self.provider.value,
            "model_name": self.model_name,
            "api_base": self.api_base,
            "has_api_key": self.api_key is not None,
            "device": self.device.value,
            "max_tokens": self.max_tokens,
        }
@dataclass
class DualModelConfig:
```

```
    """Configuration for dual-model workflow."""
    name: str = "default"
    planner: ModelEndpoint | None = None
    executor: ModelEndpoint | None = None
    validator: ModelEndpoint | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            "name": self.name,
            "planner": self.planner.to_dict() if self.planner else None,
            "executor": self.executor.to_dict() if self.executor else None,
            "validator": self.validator.to_dict() if self.validator else None,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> DualModelConfig:
        """Create from dictionary."""
        config = cls(name=data.get("name", "default"))
        if "planner" in data and data["planner"]:
            p = data["planner"]
            config.planner = ModelEndpoint(
                provider=ModelProvider(p.get("provider", "ollama")),
                model_name=p.get("model_name", "gemma3:1b"),
                api_base=p.get("api_base"),
                api_key=p.get("api_key"),
                device=ModelDevice(p.get("device", "cpu")),
                max_tokens=p.get("max_tokens", 500),
            )
        if "executor" in data and data["executor"]:
            e = data["executor"]
            config.executor = ModelEndpoint(
                provider=ModelProvider(e.get("provider", "ollama")),
                model_name=e.get("model_name", "gemma4:e4b"),
                api_base=e.get("api_base"),
                api_key=e.get("api_key"),
                device=ModelDevice(e.get("device", "gpu")),
                max_tokens=e.get("max_tokens", 1000),
            )
        if "validator" in data and data["validator"]:
            v = data["validator"]
            config.validator = ModelEndpoint(
                provider=ModelProvider(v.get("provider", "ollama")),
                model_name=v.get("model_name", "gemma3:1b"),
                api_base=v.get("api_base"),
                api_key=v.get("api_key"),
                device=ModelDevice(v.get("device", "cpu")),
                max_tokens=v.get("max_tokens", 300),
            )
        return config
class DualModelConfigManager:
    """Manage dual-model configurations."""
    DEFAULT_CONFIGS = {
        "local_balanced": DualModelConfig(
            name="local_balanced",
            planner=ModelEndpoint(
                provider=ModelProvider.OLLAMA,
                model_name="gemma3:1b",
                api_base="http://localhost:11434",
                device=ModelDevice.CPU,
                max_tokens=500,
            ),
            executor=ModelEndpoint(
                provider=ModelProvider.OLLAMA,
                model_name="gemma4:e4b",
                api_base="http://localhost:11434",
                device=ModelDevice.GPU,
                max_tokens=1000,
```

```
        ),
        validator=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=300,
        ),
    ),
    "local_light": DualModelConfig(
        name="local_light",
        planner=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=500,
        ),
        executor=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma4:e2b",
            api_base="http://localhost:11434",
            device=ModelDevice.GPU,
            max_tokens=1000,
        ),
        validator=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=300,
        ),
    ),
    "hybrid_openai": DualModelConfig(
        name="hybrid_openai",
        planner=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=500,
        ),
        executor=ModelEndpoint(
            provider=ModelProvider.OPENAI,
            model_name="gpt-4-turbo",
            api_base="https://api.openai.com/v1",
            device=ModelDevice.AUTO,
            max_tokens=2000,
        ),
        validator=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=300,
        ),
    ),
    "hybrid_anthropic": DualModelConfig(
        name="hybrid_anthropic",
        planner=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=500,
```

```
        ),
        executor=ModelEndpoint(
            provider=ModelProvider.ANTHROPIC,
            model_name="claude-3-opus-20240229",
            api_base="https://api.anthropic.com",
            device=ModelDevice.AUTO,
            max_tokens=2000,
        ),
        validator=ModelEndpoint(
            provider=ModelProvider.OLLAMA,
            model_name="gemma3:1b",
            api_base="http://localhost:11434",
            device=ModelDevice.CPU,
            max_tokens=300,
        ),
    ),
    "online_premium": DualModelConfig(
        name="online_premium",
        planner=ModelEndpoint(
            provider=ModelProvider.OPENAI,
            model_name="gpt-3.5-turbo",
            api_base="https://api.openai.com/v1",
            device=ModelDevice.AUTO,
            max_tokens=500,
        ),
        executor=ModelEndpoint(
            provider=ModelProvider.ANTHROPIC,
            model_name="claude-3-opus-20240229",
            api_base="https://api.anthropic.com",
            device=ModelDevice.AUTO,
            max_tokens=2000,
        ),
        validator=ModelEndpoint(
            provider=ModelProvider.OPENAI,
            model_name="gpt-3.5-turbo",
            api_base="https://api.openai.com/v1",
            device=ModelDevice.AUTO,
            max_tokens=300,
        ),
    ),
}
def __init__(self, app_state: AppState | None = None) -> None:
    self.app_state = app_state
    self.configs: dict[str, DualModelConfig] = dict(self.DEFAULT_CONFIGS)
    self.active_config_name: str = "local_balanced"
    self._load_user_configs()
def _load_user_configs(self) -> None:
    """Load user-defined configurations."""
    config_dir = Path.home() / ".openlaoke" / "dual_model_configs"
    config_file = config_dir / "configs.json"
    if config_file.exists():
        try:
            import json
            with open(config_file) as f:
                data = json.load(f)
            for name, config_data in data.get("configs", {}).items():
                self.configs[name] = DualModelConfig.from_dict(config_data)
            if "active_config" in data:
                self.active_config_name = data["active_config"]
        except Exception:
            pass
def _save_user_configs(self) -> None:
    """Save user-defined configurations."""
    config_dir = Path.home() / ".openlaoke" / "dual_model_configs"
    config_dir.mkdir(parents=True, exist_ok=True)
```

```
    config_file = config_dir / "configs.json"
    try:
        import json
        data = {
            "configs": {name: config.to_dict() for name, config in self.configs.items()},
            "active_config": self.active_config_name,
        }
        with open(config_file, "w") as f:
            json.dump(data, f, indent=2)
    except Exception:
        pass
def get_config(self, name: str | None = None) -> DualModelConfig | None:
    """Get a configuration by name."""
    if name is None:
        name = self.active_config_name
    return self.configs.get(name)
def set_active_config(self, name: str) -> bool:
    """Set the active configuration."""
    if name in self.configs:
        self.active_config_name = name
        self._save_user_configs()
        return True
    return False
def list_configs(self) -> list[str]:
    """List all available configurations."""
    return list(self.configs.keys())
def create_custom_config(
    self,
    name: str,
    planner_provider: str,
    planner_model: str,
    executor_provider: str,
    executor_model: str,
    validator_provider: str | None = None,
    validator_model: str | None = None,
    planner_api_key: str | None = None,
    executor_api_key: str | None = None,
    validator_api_key: str | None = None,
    planner_device: str = "cpu",
    executor_device: str = "gpu",
    validator_device: str = "cpu",
) -> DualModelConfig:
    """Create a custom configuration."""
    if validator_provider is None:
        validator_provider = planner_provider
    if validator_model is None:
        validator_model = planner_model
    def get_api_base(provider: str) -> str | None:
        bases = {
            "openai": "https://api.openai.com/v1",
            "anthropic": "https://api.anthropic.com",
            "google": "https://generativelanguage.googleapis.com",
            "deepseek": "https://api.deepseek.com",
            "ollama": "http://localhost:11434",
        }
        return bases.get(provider.lower())
    config = DualModelConfig(
        name=name,
        planner=ModelEndpoint(
            provider=ModelProvider(planner_provider.lower()),
            model_name=planner_model,
            api_base=get_api_base(planner_provider),
            api_key=planner_api_key,
            device=ModelDevice(planner_device.lower()),
            max_tokens=500,
```

```
            ),
            executor=ModelEndpoint(
                provider=ModelProvider(executor_provider.lower()),
                model_name=executor_model,
                api_base=get_api_base(executor_provider),
                api_key=executor_api_key,
                device=ModelDevice(executor_device.lower()),
                max_tokens=2000,
            ),
            validator=ModelEndpoint(
                provider=ModelProvider(validator_provider.lower()),
                model_name=validator_model,
                api_base=get_api_base(validator_provider),
                api_key=validator_api_key,
                device=ModelDevice(validator_device.lower()),
                max_tokens=300,
            ),
        )
        self.configs[name] = config
        self._save_user_configs()
        return config
    async def check_provider_availability(self, endpoint: ModelEndpoint) -> tuple[bool, str]:
        """Check if a model provider is available."""
        if endpoint.provider == ModelProvider.OLLAMA:
            try:
                async with httpx.AsyncClient(timeout=3.0) as client:
                    response = await client.get(f"{endpoint.api_base}/api/version")
                    if response.status_code == 200:
                        return True, "Ollama is running"
                    return False, "Ollama returned error"
            except Exception as e:
                return False, f"Ollama not available: {str(e)}"
        elif endpoint.provider in [
            ModelProvider.OPENAI,
            ModelProvider.ANTHROPIC,
            ModelProvider.GOOGLE,
        ]:
            if endpoint.api_key:
                return True, f"{endpoint.provider.value} configured with API key"
            return False, f"{endpoint.provider.value} requires API key"
        return True, f"{endpoint.provider.value} endpoint ready"
    async def check_config_availability(
        self, config_name: str | None = None
    ) -> dict[str, tuple[bool, str]]:
        """Check availability of all models in a configuration."""
        config = self.get_config(config_name)
        if not config:
            return {}
        results = {}
        if config.planner:
            results["planner"] = await self.check_provider_availability(config.planner)
        if config.executor:
            results["executor"] = await self.check_provider_availability(config.executor)
        if config.validator:
            results["validator"] = await self.check_provider_availability(config.validator)
        return results
def create_config_manager(app_state: AppState | None = None) -> DualModelConfigManager:
    """Create a configuration manager."""
    return DualModelConfigManager(app_state)
```

File: [44/252] openlaoke/core/enhanced_knowledge_base.py

```
"""Enhanced knowledge base with documentation downloading.
Integrates builtin knowledge and downloadable documentation
for comprehensive knowledge retrieval.
"""
from __future__ import annotations
import json
import re
from dataclasses import dataclass, field
from pathlib import Path
from openlaoke.core.doc_downloader import DocumentationDownloader, DocumentChunk
@dataclass
class KnowledgeSnippet:
    topic: str
    content: str
    source: str
    relevance_score: float = 1.0
    tags: list[str] = field(default_factory=list)
    language: str = "general"
    url: str = ""
class EnhancedKnowledgeBase:
    """Knowledge base with builtin knowledge and downloadable docs."""
    BUILTIN_KNOWLEDGE = {
        "python_basics": {
            "topic": "Python Programming Basics",
            "content": """
Python fundamentals for small models:
1. Function definition with type hints:
```python
def calculate_sum(numbers: list[int]) -> int:
    \"\"\"Calculate sum of numbers.\"\"\"
    return sum(numbers)
```
2. Class with __init__:
```python
class Calculator:
    def __init__(self, name: str) -> None:
        self.name = name
        self.result = 0.0
    def add(self, x: float, y: float) -> float:
        self.result = x + y
        return self.result
```
3. Loops and conditions:
```python
for i in range(10):
    if i % 2 == 0:
        print(f"{i} is even")
    else:
        print(f"{i} is odd")
```
4. Import statements (always at top):
```python
from __future__ import annotations
import time
from pathlib import Path
from typing import Any
```
Critical syntax rules:
- Always use `def function_name(params) -> return_type:`
- Always indent with 4 spaces
- Always include docstring with triple quotes
```

```
- Never forget `:` after if/for/def/class
""",
            "source": "Python Official Tutorial",
            "tags": ["python", "basics", "syntax", "functions", "classes"],
            "language": "python",
        },
        "javascript_basics": {
            "topic": "JavaScript Programming Basics",
            "content": """
JavaScript fundamentals for small models:
1. Function definition (ES6+):
```javascript
function calculateSum(numbers) {
    return numbers.reduce((a, b) => a + b, 0);
}
// Arrow function
const calculateSum = (numbers) => {
    return numbers.reduce((a, b) => a + b, 0);
};
```
2. Class syntax:
```javascript
class Calculator {
    constructor(name) {
        this.name = name;
        this.result = 0;
    }
    add(x, y) {
        this.result = x + y;
        return this.result;
    }
}
```
3. Async/await:
```javascript
async function fetchData(url) {
    const response = await fetch(url);
    const data = await response.json();
    return data;
}
```
""",
            "source": "JavaScript MDN Guide",
            "tags": ["javascript", "js", "basics", "syntax", "es6"],
            "language": "javascript",
        },
        "rust_basics": {
            "topic": "Rust Programming Basics",
            "content": """
Rust fundamentals for small models:
1. Function definition:
```rust
fn calculate_sum(numbers: &[i32]) -> i32 {
    numbers.iter().sum()
}
```
2. Struct and impl:
```rust
struct Calculator {
    name: String,
    result: f64,
}
impl Calculator {
    fn new(name: &str) -> Self {
        Calculator {
```
```

```
            name: name.to_string(),
            result: 0.0,
        }
    }
}
```
3. Error handling:
```rust
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err("Division by zero".to_string())
    } else {
        Ok(a / b)
    }
}
```
Ownership rules:
- Each value has exactly one owner
- Use `&` for borrowing (references)
- Use `&mut` for mutable borrowing
"""",
            "source": "Rust Book",
            "tags": ["rust", "basics", "syntax", "ownership", "structs"],
            "language": "rust",
        },
        "go_basics": {
            "topic": "Go Programming Basics",
            "content": """
Go fundamentals for small models:
1. Function definition:
```go
func calculateSum(numbers []int) int {
    sum := 0
    for _, n := range numbers {
        sum += n
    }
    return sum
}
```
2. Struct and methods:
```go
type Calculator struct {
    name   string
    result float64
}
func NewCalculator(name string) *Calculator {
    return &Calculator{name: name, result: 0.0}
}
func (c *Calculator) Add(x, y float64) float64 {
    c.result = x + y
    return c.result
}
```
3. Error handling:
```go
func divide(a, b float64) (float64, error) {
    if b == 0.0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}
```
""",
            "source": "Go Tour",
            "tags": ["go", "golang", "basics", "syntax", "structs", "goroutines"],

```
        "language": "go",
    },
}
def __init__(self, cache_dir: Path | None = None):
    self.cache_dir = cache_dir or Path.home() / ".openlaoke" / "knowledge"
    self.cache_dir.mkdir(parents=True, exist_ok=True)
    self.knowledge_cache: dict[str, KnowledgeSnippet] = {}
    self.downloaded_chunks: list[DocumentChunk] = []
    self.downloader = DocumentationDownloader(cache_dir)
    self._load_builtin_knowledge()
    self._load_downloaded_knowledge()
def _load_builtin_knowledge(self) -> None:
    """Load built-in knowledge snippets."""
    for key, data in self.BUILTIN_KNOWLEDGE.items():
        self.knowledge_cache[key] = KnowledgeSnippet(
            topic=data["topic"],
            content=data["content"],
            source=data["source"],
            tags=data.get("tags", []),
            language=data.get("language", "general"),
        )
def _load_downloaded_knowledge(self) -> None:
    """Load previously downloaded knowledge."""
    index_file = self.cache_dir / "downloaded_index.json"
    if index_file.exists():
        try:
            with open(index_file, encoding="utf-8") as f:
                index_data = json.load(f)
            for chunk_data in index_data.get("chunks", []):
                chunk = DocumentChunk(
                    doc_id=chunk_data["doc_id"],
                    source=chunk_data["source"],
                    title=chunk_data["title"],
                    content=chunk_data["content"],
                    url=chunk_data["url"],
                    category=chunk_data["category"],
                    language=chunk_data["language"],
                    chunk_index=chunk_data["chunk_index"],
                    metadata=chunk_data.get("metadata", {}),
                )
                self.downloaded_chunks.append(chunk)
            print(f"Loaded {len(self.downloaded_chunks)} downloaded knowledge chunks")
        except Exception as e:
            print(f"Error loading downloaded knowledge: {e}")
def download_for_task(self, task_description: str, force: bool = False) -> int:
    """Download relevant documentation for a task."""
    chunks = self.downloader.download_task_relevant_docs(task_description, force=force)
    self.downloaded_chunks.extend(chunks)
    self._save_downloaded_index()
    return len(chunks)
def _save_downloaded_index(self) -> None:
    """Save downloaded knowledge index."""
    index_file = self.cache_dir / "downloaded_index.json"
    chunks_data = [
        {
            "doc_id": chunk.doc_id,
            "source": chunk.source,
            "title": chunk.title,
            "content": chunk.content,
            "url": chunk.url,
            "category": chunk.category,
            "language": chunk.language,
            "chunk_index": chunk.chunk_index,
            "metadata": chunk.metadata,
        }
```

```
        for chunk in self.downloaded_chunks
    ]
    with open(index_file, "w", encoding="utf-8") as f:
        json.dump(
            {"chunks": chunks_data, "total": len(chunks_data)}, f, ensure_ascii=False, inde
nt=2
        )
def search(
    self,
    query: str,
    max_results: int = 5,
    language: str | None = None,
    category: str | None = None,
) -> list[KnowledgeSnippet]:
    """Search knowledge base for relevant information."""
    query_lower = query.lower()
    query_words = set(re.findall(r"\w+", query_lower))
    scored: list[tuple[float, KnowledgeSnippet]] = []
    for snippet in self.knowledge_cache.values():
        if language and snippet.language != language and snippet.language != "general":
            continue
        score = self._calculate_relevance(snippet, query_lower, query_words)
        if score > 0:
            scored.append((score, snippet))
    for chunk in self.downloaded_chunks:
        if language and chunk.language != language and chunk.language != "general":
            continue
        if category and chunk.category != category:
            continue
        score = self._calculate_chunk_relevance(chunk, query_lower, query_words)
        if score > 0:
            snippet = KnowledgeSnippet(
                topic=chunk.title,
                content=chunk.content,
                source=chunk.source,
                relevance_score=score,
                tags=[chunk.category, chunk.language],
                language=chunk.language,
                url=chunk.url,
            )
            scored.append((score, snippet))
    scored.sort(key=lambda x: x[0], reverse=True)
    return [snippet for score, snippet in scored[:max_results]]
def _calculate_relevance(
    self, snippet: KnowledgeSnippet, query_lower: str, query_words: set[str]
) -> float:
    """Calculate relevance score for builtin knowledge."""
    score = 0.0
    if query_lower in snippet.topic.lower():
        score += 3.0
    for word in query_words:
        if word in snippet.content.lower():
            score += 1.0
    for tag in snippet.tags:
        if tag in query_words:
            score += 2.0
    return score
def _calculate_chunk_relevance(
    self, chunk: DocumentChunk, query_lower: str, query_words: set[str]
) -> float:
    """Calculate relevance score for downloaded chunk."""
    score = 0.0
    if query_lower in chunk.title.lower():
        score += 4.0
    if query_lower in chunk.source.lower():
```

```
            score += 3.0
        for word in query_words:
            if word in chunk.content.lower():
                score += 0.5
        if chunk.category in query_words:
            score += 2.0
        return score
    def get_examples(self, language: str, max_examples: int = 3) -> list[str]:
        """Get code examples for a language."""
        examples = []
        if language in ["python", "py"]:
            examples.append(
                'def example_function(x: int, y: int) -> int:\n    """Example function."""\n
     return x + y'
            )
            examples.append(
                "class ExampleClass:\n    def __init__(self, name: str) -> None:\n        self.
    name = name"
            )
            examples.append(
                'for i in range(10):\n    if i % 2 == 0:\n        print(f"{i} is even")'
            )
        elif language in ["javascript", "js"]:
            examples.append("function exampleFunction(x, y) {\n    return x + y;\n}")
            examples.append(
                "class ExampleClass {\n    constructor(name) {\n        this.name = name;\n
    }\n}"
            )
        elif language == "rust":
            examples.append("fn example_function(x: i32, y: i32) -> i32 {\n    x + y\n}")
            examples.append("struct ExampleStruct {\n    name: String,\n}")
        elif language == "go":
            examples.append("func exampleFunction(x, y int) int {\n    return x + y\n}")
            examples.append("type ExampleStruct struct {\n    name string\n}")
        return examples[:max_examples]
def create_knowledge_base(cache_dir: Path | None = None) -> EnhancedKnowledgeBase:
    """Create an enhanced knowledge base instance."""
    return EnhancedKnowledgeBase(cache_dir)
```

File: [45/252] openlaoke/core/explorer/__init__.py

```
"""Explore Mode - Autonomous code exploration and understanding."""
from __future__ import annotations
from openlaoke.core.explorer.architecture import ArchitectureExplorer
from openlaoke.core.explorer.code_understanding import CodeUnderstandingEngine
from openlaoke.core.explorer.discovery import DiscoverySystem
from openlaoke.core.explorer.exploration_strategy import ExplorationStrategy
from openlaoke.core.explorer.explorer import ExploreMode
from openlaoke.core.explorer.hypothesis import HypothesisGenerator
from openlaoke.core.explorer.reasoning import ReasoningEngine
__all__ = [
    "ExploreMode",
    "ArchitectureExplorer",
    "CodeUnderstandingEngine",
    "ReasoningEngine",
    "HypothesisGenerator",
    "DiscoverySystem",
    "ExplorationStrategy",
]
```

File: [46/252] openlaoke/core/explorer/architecture.py

```
"""Architecture exploration and analysis module."""
from __future__ import annotations
import ast
import re
from collections import defaultdict
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.explorer.explorer import ArchitectureAnalysis
@dataclass
class DependencyGraph:
    """Graph representation of code dependencies."""
    nodes: set[str] = field(default_factory=set)
    edges: dict[str, set[str]] = field(default_factory=lambda: defaultdict(set))
    reverse_edges: dict[str, set[str]] = field(default_factory=lambda: defaultdict(set))
    def add_node(self, node: str) -> None:
        self.nodes.add(node)
    def add_edge(self, from_node: str, to_node: str) -> None:
        self.nodes.add(from_node)
        self.nodes.add(to_node)
        self.edges[from_node].add(to_node)
        self.reverse_edges[to_node].add(from_node)
    def get_dependencies(self, node: str) -> set[str]:
        return self.edges.get(node, set())
    def get_dependents(self, node: str) -> set[str]:
        return self.reverse_edges.get(node, set())
    def find_cycles(self) -> list[list[str]]:
        visited = set()
        rec_stack = set()
        cycles: list[list[str]] = []
        def dfs(node: str, path: list[str]) -> None:
            visited.add(node)
            rec_stack.add(node)
            path.append(node)
            for neighbor in self.edges.get(node, set()):
                if neighbor not in visited:
                    dfs(neighbor, path.copy())
                elif neighbor in rec_stack:
                    cycle_start = path.index(neighbor)
                    cycle = path[cycle_start:] + [neighbor]
                    cycles.append(cycle)
            rec_stack.remove(node)
        for node in self.nodes:
            if node not in visited:
                dfs(node, [])
        return cycles
    def to_dict(self) -> dict[str, Any]:
        return {
            "nodes": list(self.nodes),
            "edges": {k: list(v) for k, v in self.edges.items()},
            "reverse_edges": {k: list(v) for k, v in self.reverse_edges.items()},
        }
@dataclass
class ModuleInfo:
    """Information about a Python module."""
    path: Path
    imports: list[str]
    exports: list[str]
    classes: list[str]
    functions: list[str]
```

```
    complexity: float = 0.0
    loc: int = 0
    docstring: str = ""
    def to_dict(self) -> dict[str, Any]:
        return {
            "path": str(self.path),
            "imports": self.imports,
            "exports": self.exports,
            "classes": self.classes,
            "functions": self.functions,
            "complexity": self.complexity,
            "loc": self.loc,
            "docstring": self.docstring,
        }
class ArchitectureExplorer:
    """Analyzes and explores project architecture.
    This class provides comprehensive architecture analysis including:
    - Project structure mapping
    - Design pattern detection
    - Dependency analysis
    - Code smell detection
    - Architecture improvement suggestions
    """
    DESIGN_PATTERNS = {
        "singleton": r"class\s+\w+.*:\s*\n(?:.*\n)*?\s*_instance\s*=",
        "factory": r"(?:class|def)\s+\w*[Ff]actory\w*",
        "observer": r"(?:def|class)\s+\w*[Oo]bserver\w*",
        "strategy": r"class\s+\w*[Ss]trategy\w*",
        "builder": r"class\s+\w*[Bb]uilder\w*",
        "adapter": r"class\s+\w*[Aa]dapter\w*",
        "decorator": r"(?:def\s+\w+.*\n\s*def\s+\w+|@\w+.*\n\s*def\s+\w+)",
        "facade": r"class\s+\w*[Ff]acade\w*",
        "proxy": r"class\s+\w*[Pp]roxy\w*",
        "command": r"class\s+\w*[Cc]ommand\w*",
        "state": r"class\s+\w*[Ss]tate\w*",
        "template": r"class\s+\w*[Tt]emplate\w*",
    }
    CODE_SMELLS = {
        "long_function": {"threshold": 50, "description": "Function exceeds 50 lines"},
        "many_parameters": {"threshold": 6, "description": "Function has more than 6 parameters
    "},
        "deep_nesting": {"threshold": 4, "description": "Code has more than 4 levels of nesting
    "},
        "duplicate_code": {"threshold": 0.8, "description": "Similar code blocks detected"},
        "large_class": {"threshold": 500, "description": "Class exceeds 500 lines"},
        "god_class": {"threshold": 20, "description": "Class has more than 20 methods"},
        "long_parameter_list": {"threshold": 4, "description": "Parameter list too long"},
        "circular_dependency": {"threshold": 0, "description": "Circular dependencies detected"
    },
    }
    def __init__(self) -> None:
        self._dependency_graph = DependencyGraph()
        self._modules: dict[str, ModuleInfo] = {}
    async def analyze(self, project_path: Path) -> ArchitectureAnalysis:
        """Perform comprehensive architecture analysis.
        Args:
            project_path: Root path of the project
        Returns:
            ArchitectureAnalysis with all findings
        """
        from openlaoke.core.explorer.explorer import ArchitectureAnalysis
        structure = await self._analyze_structure(project_path)
        design_patterns = await self._detect_patterns(project_path)
        dependencies = await self._analyze_dependencies(project_path)
        code_smells = await self._detect_code_smells(project_path)
```

```
    suggestions = self._generate_suggestions(structure, design_patterns, code_smells)
    return ArchitectureAnalysis(
        project_path=project_path,
        structure=structure,
        design_patterns=design_patterns,
        dependencies=dependencies,
        code_smells=code_smells,
        improvement_suggestions=suggestions,
    )
async def _analyze_structure(self, path: Path) -> dict[str, Any]:
    """Analyze the directory structure."""
    structure: dict[str, Any] = {}
    for item in path.rglob("*.py"):
        if "__pycache__" in str(item) or ".venv" in str(item):
            continue
        rel_path = item.relative_to(path)
        parts = list(rel_path.parts)
        current = structure
        for part in parts[:-1]:
            if part not in current:
                current[part] = {}
            current = current[part]
        module_info = await self._analyze_module(item)
        current[parts[-1]] = module_info.to_dict()
        self._modules[str(rel_path)] = module_info
    return structure
async def _analyze_module(self, file_path: Path) -> ModuleInfo:
    """Analyze a single Python module."""
    try:
        with open(file_path, encoding="utf-8") as f:
            content = f.read()
    except Exception:
        return ModuleInfo(
            path=file_path,
            imports=[],
            exports=[],
            classes=[],
            functions=[],
        )
    try:
        tree = ast.parse(content)
    except SyntaxError:
        return ModuleInfo(
            path=file_path,
            imports=[],
            exports=[],
            classes=[],
            functions=[],
        )
    imports: list[str] = []
    exports: list[str] = []
    classes: list[str] = []
    functions: list[str] = []
    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            imports.extend(alias.name for alias in node.names)
        elif isinstance(node, ast.ImportFrom):
            module = node.module or ""
            imports.extend(
                f"{module}.{alias.name}" if module else alias.name for alias in node.names
            )
        elif isinstance(node, ast.ClassDef):
            classes.append(node.name)
            if node.name[0].isupper() and not node.name.startswith("_"):
                exports.append(node.name)
```

```
        elif isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
            functions.append(node.name)
            if not node.name.startswith("_"):
                exports.append(node.name)
    complexity = self._calculate_complexity(tree)
    loc = len(content.splitlines())
    docstring = ast.get_docstring(tree) or ""
    return ModuleInfo(
        path=file_path,
        imports=imports,
        exports=exports,
        classes=classes,
        functions=functions,
        complexity=complexity,
        loc=loc,
        docstring=docstring,
    )
def _calculate_complexity(self, tree: ast.AST) -> float:
    """Calculate cyclomatic complexity of the AST."""
    complexity = 1
    for node in ast.walk(tree):
        if isinstance(node, ast.If | ast.While | ast.For | ast.ExceptHandler):
            complexity += 1
        elif isinstance(node, ast.BoolOp):
            complexity += len(node.values) - 1
        elif isinstance(node, ast.comprehension):
            complexity += 1
            if node.ifs:
                complexity += len(node.ifs)
    return float(complexity)
async def _detect_patterns(self, path: Path) -> list[str]:
    """Detect design patterns in the codebase."""
    detected_patterns: set[str] = set()
    for file_path in path.rglob("*.py"):
        if "__pycache__" in str(file_path) or ".venv" in str(file_path):
            continue
        try:
            with open(file_path, encoding="utf-8") as f:
                content = f.read()
        except Exception:
            continue
        for pattern_name, pattern_regex in self.DESIGN_PATTERNS.items():
            if re.search(pattern_regex, content, re.MULTILINE):
                detected_patterns.add(pattern_name)
    return list(detected_patterns)
async def _analyze_dependencies(self, path: Path) -> dict[str, Any]:
    """Analyze module dependencies."""
    self._dependency_graph = DependencyGraph()
    for module_path, module_info in self._modules.items():
        module_name = module_path.replace("/", ".").replace(".py", "")
        self._dependency_graph.add_node(module_name)
        for imp in module_info.imports:
            if imp.startswith("openlaoke."):
                self._dependency_graph.add_edge(module_name, imp)
    return {
        "graph": self._dependency_graph.to_dict(),
        "cycles": self._dependency_graph.find_cycles(),
    }
async def _detect_code_smells(self, path: Path) -> list[dict[str, Any]]:
    """Detect code smells in the codebase."""
    smells: list[dict[str, Any]] = []
    for file_path in path.rglob("*.py"):
        if "__pycache__" in str(file_path) or ".venv" in str(file_path):
            continue
        try:
```

```
            with open(file_path, encoding="utf-8") as f:
                content = f.read()
            tree = ast.parse(content)
        except Exception:
            continue
        smells.extend(self._detect_smells_in_file(file_path, tree))
    return smells
def _detect_smells_in_file(self, file_path: Path, tree: ast.AST) -> list[dict[str, Any]]:
    """Detect code smells in a single file."""
    smells: list[dict[str, Any]] = []
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
            end_lineno = getattr(node, "end_lineno", None)
            func_lines = (end_lineno - node.lineno) if end_lineno is not None else 0
            threshold_value = self.CODE_SMELLS["long_function"]["threshold"]
            threshold = (
                int(threshold_value) if isinstance(threshold_value, (int, float)) else 50
            )
            if func_lines > threshold:
                smells.append(
                    {
                        "type": "long_function",
                        "file": str(file_path),
                        "line": node.lineno,
                        "name": node.name,
                        "value": func_lines,
                        "description": self.CODE_SMELLS["long_function"]["description"],
                    }
                )
            param_count = len(node.args.args)
            param_threshold_value = self.CODE_SMELLS["many_parameters"]["threshold"]
            param_threshold = (
                int(param_threshold_value)
                if isinstance(param_threshold_value, (int, float))
                else 6
            )
            if param_count > param_threshold:
                smells.append(
                    {
                        "type": "many_parameters",
                        "file": str(file_path),
                        "line": node.lineno,
                        "name": node.name,
                        "value": param_count,
                        "description": self.CODE_SMELLS["many_parameters"]["description"],
                    }
                )
        elif isinstance(node, ast.ClassDef):
            method_count = sum(
                1
                for n in ast.walk(node)
                if isinstance(n, ast.FunctionDef | ast.AsyncFunctionDef)
            )
            god_threshold_value = self.CODE_SMELLS["god_class"]["threshold"]
            god_threshold = (
                int(god_threshold_value)
                if isinstance(god_threshold_value, (int, float))
                else 20
            )
            if method_count > god_threshold:
                smells.append(
                    {
                        "type": "god_class",
                        "file": str(file_path),
                        "line": node.lineno,
```

```
                        "name": node.name,
                        "value": method_count,
                        "description": self.CODE_SMELLS["god_class"]["description"],
                    }
                )
    return smells
def _generate_suggestions(
    self,
    structure: dict[str, Any],
    patterns: list[str],
    smells: list[dict[str, Any]],
) -> list[str]:
    """Generate improvement suggestions based on analysis."""
    suggestions: list[str] = []
    if len(smells) > 10:
        suggestions.append(
            "Consider refactoring to reduce code smells. "
            "Focus on the most critical issues first."
        )
    long_functions = [s for s in smells if s["type"] == "long_function"]
    if len(long_functions) > 5:
        suggestions.append(
            "Multiple long functions detected. "
            "Consider breaking them into smaller, focused functions."
        )
    if "factory" not in patterns and len(self._modules) > 20:
        suggestions.append("Consider using the Factory pattern for complex object creation.
")
    cycles = self._dependency_graph.find_cycles()
    if cycles:
        suggestions.append(
            f"Circular dependencies detected: {len(cycles)} cycles. "
            "Consider using dependency injection or refactoring."
        )
    if not suggestions:
        suggestions.append(
            "Architecture looks good! Continue maintaining code quality standards."
        )
    return suggestions
def get_module_info(self, module_path: str) -> ModuleInfo | None:
    """Get information about a specific module."""
    return self._modules.get(module_path)
def get_dependency_graph(self) -> DependencyGraph:
    """Get the dependency graph."""
    return self._dependency_graph
```

File: [47/252] openlaoke/core/explorer/code_understanding.py

```
"""Code understanding and semantic analysis engine."""
from __future__ import annotations
import ast
import re
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.explorer.explorer import CodeUnderstanding
@dataclass
class SemanticContext:
    """Semantic context extracted from code."""
    domain_concepts: list[str]
    business_logic: list[str]
    data_flows: list[str]
    control_flows: list[str]
    abstractions: list[str]
    def to_dict(self) -> dict[str, Any]:
        return {
            "domain_concepts": self.domain_concepts,
            "business_logic": self.business_logic,
            "data_flows": self.data_flows,
            "control_flows": self.control_flows,
            "abstractions": self.abstractions,
        }
@dataclass
class IntentAnalysis:
    """Analysis of code intent and purpose."""
    primary_intent: str
    secondary_intents: list[str]
    assumptions: list[str]
    constraints: list[str]
    confidence: float
    def to_dict(self) -> dict[str, Any]:
        return {
            "primary_intent": self.primary_intent,
            "secondary_intents": self.secondary_intents,
            "assumptions": self.assumptions,
            "constraints": self.constraints,
            "confidence": self.confidence,
        }
@dataclass
class BehaviorModel:
    """Model of code behavior."""
    inputs: list[dict[str, Any]]
    outputs: list[dict[str, Any]]
    side_effects: list[str]
    preconditions: list[str]
    postconditions: list[str]
    invariants: list[str]
    def to_dict(self) -> dict[str, Any]:
        return {
            "inputs": self.inputs,
            "outputs": self.outputs,
            "side_effects": self.side_effects,
            "preconditions": self.preconditions,
            "postconditions": self.postconditions,
            "invariants": self.invariants,
        }
class CodeUnderstandingEngine:
    """Deep code understanding and semantic analysis engine.
```

```
This class provides comprehensive code understanding through:
- Semantic analysis
- Intent inference
- Behavior modeling
- Quality metrics
"""
DOMAIN_KEYWORDS = {
    "data": ["data", "record", "entry", "item", "entity", "model"],
    "api": ["api", "endpoint", "route", "request", "response", "handler"],
    "database": ["db", "database", "query", "table", "schema", "migration"],
    "auth": ["auth", "login", "token", "session", "user", "permission"],
    "config": ["config", "setting", "option", "preference", "env"],
    "test": ["test", "spec", "mock", "fixture", "assert"],
    "util": ["util", "helper", "common", "shared", "tool"],
    "core": ["core", "main", "primary", "base", "foundation"],
}
INTENT_INDICATORS = {
    "validation": ["validate", "check", "verify", "ensure", "assert"],
    "transformation": ["transform", "convert", "parse", "format", "serialize"],
    "query": ["get", "fetch", "retrieve", "query", "find", "search"],
    "mutation": ["create", "update", "delete", "modify", "save", "write"],
    "coordination": ["coordinate", "orchestrate", "manage", "control", "schedule"],
    "communication": ["send", "receive", "notify", "emit", "broadcast"],
}
def __init__(self) -> None:
    self._analysis_cache: dict[str, CodeUnderstanding] = {}
async def analyze(self, file_path: Path) -> CodeUnderstanding:
    """Perform deep code understanding analysis.
    Args:
        file_path: Path to the file to analyze
    Returns:
        CodeUnderstanding with complete analysis
    """
    from openlaoke.core.explorer.explorer import CodeUnderstanding
    cache_key = str(file_path)
    if cache_key in self._analysis_cache:
        return self._analysis_cache[cache_key]
    try:
        with open(file_path, encoding="utf-8") as f:
            content = f.read()
    except Exception:
        return CodeUnderstanding(
            file_path=file_path,
            semantic_summary="Unable to read file",
            intent="Unknown",
            behavior_model={},
            complexity_score=0.0,
            quality_metrics={},
        )
    try:
        tree = ast.parse(content)
    except SyntaxError:
        return CodeUnderstanding(
            file_path=file_path,
            semantic_summary="Unable to parse file",
            intent="Unknown",
            behavior_model={},
            complexity_score=0.0,
            quality_metrics={},
        )
    semantic_ctx = self._extract_semantics(tree, content)
    intent_analysis = self._infer_intent(tree, content, semantic_ctx)
    behavior_model = self._model_behavior(tree)
    complexity = self._calculate_complexity(tree)
    quality = self._calculate_quality(tree, content)
```

```
    understanding = CodeUnderstanding(
        file_path=file_path,
        semantic_summary=self._generate_summary(semantic_ctx, intent_analysis),
        intent=intent_analysis.primary_intent,
        behavior_model=behavior_model.to_dict(),
        complexity_score=complexity,
        quality_metrics=quality,
    )
    self._analysis_cache[cache_key] = understanding
    return understanding
def _extract_semantics(self, tree: ast.AST, content: str) -> SemanticContext:
    """Extract semantic context from code."""
    domain_concepts: set[str] = set()
    business_logic: list[str] = []
    data_flows: list[str] = []
    control_flows: list[str] = []
    abstractions: set[str] = set()
    for node in ast.walk(tree):
        if isinstance(node, ast.ClassDef):
            domain = self._identify_domain(node.name)
            if domain:
                domain_concepts.add(domain)
            abstractions.add(node.name)
        elif isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
            intent_type = self._identify_intent_type(node.name)
            if intent_type:
                business_logic.append(f"{node.name}: {intent_type}")
            for arg in node.args.args:
                if arg.arg != "self":
                    data_flows.append(arg.arg)
            if isinstance(node.body[0], ast.Return):
                data_flows.append("direct_return")
        elif isinstance(node, ast.If):
            control_flows.append("conditional")
        elif isinstance(node, ast.For | ast.While):
            control_flows.append("iteration")
        elif isinstance(node, ast.Try):
            control_flows.append("exception_handling")
    docstring = ast.get_docstring(tree) if isinstance(tree, ast.Module) else None
    if docstring:
        words = docstring.lower().split()
        for domain, keywords in self.DOMAIN_KEYWORDS.items():
            if any(kw in words for kw in keywords):
                domain_concepts.add(domain)
    return SemanticContext(
        domain_concepts=list(domain_concepts),
        business_logic=business_logic,
        data_flows=data_flows[:10],
        control_flows=list(set(control_flows)),
        abstractions=list(abstractions),
    )
def _identify_domain(self, name: str) -> str | None:
    """Identify the domain based on naming conventions."""
    name_lower = name.lower()
    for domain, keywords in self.DOMAIN_KEYWORDS.items():
        if any(kw in name_lower for kw in keywords):
            return domain
    return None
def _identify_intent_type(self, name: str) -> str | None:
    """Identify the intent type based on function/method name."""
    name_lower = name.lower()
    for intent_type, indicators in self.INTENT_INDICATORS.items():
        if any(ind in name_lower for ind in indicators):
            return intent_type
    return None
```

```
def _infer_intent(
    self, tree: ast.AST, content: str, semantic_ctx: SemanticContext
) -> IntentAnalysis:
    """Infer the primary intent of the code."""
    primary_intent = "Unknown"
    secondary_intents: list[str] = []
    assumptions: list[str] = []
    constraints: list[str] = []
    confidence = 0.5
    docstring = ast.get_docstring(tree) if isinstance(tree, ast.Module) else None
    if docstring:
        primary_intent = self._extract_intent_from_docstring(docstring)
        confidence = 0.8
    classes = [n for n in ast.walk(tree) if isinstance(n, ast.ClassDef)]
    functions = [
        n for n in ast.walk(tree) if isinstance(n, ast.FunctionDef | ast.AsyncFunctionDef)
    ]
    if classes:
        class_names = [c.name for c in classes]
        primary_intent = f"Define classes: {', '.join(class_names[:3])}"
        confidence = 0.7
    if functions:
        func_intents: list[str] = []
        for func in functions:
            intent_type = self._identify_intent_type(func.name)
            if intent_type:
                func_intents.append(f"{func.name} ({intent_type})")
        if func_intents and confidence < 0.7:
            primary_intent = f"Provide functionality: {func_intents[0]}"
            secondary_intents = func_intents[1:4]
            confidence = 0.65
    domain_hints = semantic_ctx.domain_concepts
    if domain_hints:
        assumptions.append(f"Related to domains: {', '.join(domain_hints)}")
    if "auth" in domain_hints:
        constraints.append("Requires security considerations")
    if "database" in domain_hints:
        constraints.append("Handles data persistence")
    if "api" in domain_hints:
        constraints.append("Exposes external interface")
    return IntentAnalysis(
        primary_intent=primary_intent,
        secondary_intents=secondary_intents,
        assumptions=assumptions,
        constraints=constraints,
        confidence=confidence,
    )
def _extract_intent_from_docstring(self, docstring: str) -> str:
    """Extract intent information from docstring."""
    first_line = docstring.split("\n")[0].strip()
    if len(first_line) < 100:
        return first_line
    sentences = re.split(r"[.!?]", docstring)
    if sentences:
        return sentences[0].strip()[:100]
    return "Purpose unclear from documentation"
def _model_behavior(self, tree: ast.AST) -> BehaviorModel:
    """Model the behavior of the code."""
    inputs: list[dict[str, Any]] = []
    outputs: list[dict[str, Any]] = []
    side_effects: list[str] = []
    preconditions: list[str] = []
    postconditions: list[str] = []
    invariants: list[str] = []
    for node in ast.walk(tree):
```

```
        if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
            func_inputs = [
                {"name": arg.arg, "type": "unknown"}
                for arg in node.args.args
                if arg.arg != "self"
            ]
            inputs.extend(func_inputs)
            returns = [n for n in ast.walk(node) if isinstance(n, ast.Return) and n.value]
            if returns:
                outputs.append({"type": "return", "count": len(returns), "function": node.n
ame})
            calls = [n for n in ast.walk(node) if isinstance(n, ast.Call)]
            for call in calls:
                if isinstance(call.func, ast.Name):
                    func_name = call.func.id
                    if any(
                        kw in func_name.lower()
                        for kw in ["write", "save", "delete", "update", "create"]
                    ):
                        side_effects.append(f"{node.name}: calls {func_name}")
    if inputs:
        preconditions.append("Requires valid input parameters")
    if outputs:
        postconditions.append("Returns computed result")
    if side_effects:
        invariants.append("May modify external state")
    return BehaviorModel(
        inputs=inputs[:20],
        outputs=outputs[:10],
        side_effects=list(set(side_effects))[:10],
        preconditions=preconditions,
        postconditions=postconditions,
        invariants=invariants,
    )
def _calculate_complexity(self, tree: ast.AST) -> float:
    """Calculate code complexity score."""
    complexity = 0
    total_lines = 0
    for node in ast.walk(tree):
        if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
            func_complexity = 1
            for child in ast.walk(node):
                if isinstance(child, ast.If | ast.For | ast.While):
                    func_complexity += 1
                elif isinstance(child, ast.BoolOp):
                    func_complexity += len(child.values) - 1
                elif isinstance(child, ast.ExceptHandler):
                    func_complexity += 1
            complexity += func_complexity
            total_lines += 1
    return complexity / max(total_lines, 1)
def _calculate_quality(self, tree: ast.AST, content: str) -> dict[str, float]:
    """Calculate various quality metrics."""
    lines = content.splitlines()
    code_lines = [line for line in lines if line.strip() and not line.strip().startswith("#
")]
    comment_lines = [line for line in lines if line.strip().startswith("#")]
    docstrings = sum(
        1
        for node in ast.walk(tree)
        if (
            isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef | ast.ClassDef)
            and ast.get_docstring(node)
        )
    )
```

```
    functions = sum(
        1 for node in ast.walk(tree) if isinstance(node, ast.FunctionDef | ast.AsyncFunctio
nDef)
    )
    classes = sum(1 for node in ast.walk(tree) if isinstance(node, ast.ClassDef))
    return {
        "documentation_coverage": docstrings / max(functions + classes, 1) * 100,
        "comment_ratio": len(comment_lines) / max(len(code_lines), 1) * 100,
        "lines_of_code": len(code_lines),
        "function_count": functions,
        "class_count": classes,
        "avg_function_length": len(code_lines) / max(functions, 1),
    }
def _generate_summary(
    self, semantic_ctx: SemanticContext, intent_analysis: IntentAnalysis
) -> str:
    """Generate a semantic summary."""
    parts: list[str] = []
    if semantic_ctx.domain_concepts:
        domains = ", ".join(semantic_ctx.domain_concepts[:3])
        parts.append(f"Domain: {domains}")
    if intent_analysis.primary_intent:
        parts.append(f"Intent: {intent_analysis.primary_intent}")
    if semantic_ctx.abstractions:
        abstractions = ", ".join(semantic_ctx.abstractions[:3])
        parts.append(f"Key abstractions: {abstractions}")
    if semantic_ctx.business_logic:
        logic = semantic_ctx.business_logic[0]
        parts.append(f"Main logic: {logic}")
    return ". ".join(parts) if parts else "Purpose unclear"
async def understand_function(self, file_path: Path, function_name: str) -> dict[str, Any]:
    """Understand a specific function in detail."""
    with open(file_path, encoding="utf-8") as f:
        content = f.read()
    tree = ast.parse(content)
    for node in ast.walk(tree):
        if (
            isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef)
            and node.name == function_name
        ):
            return {
                "name": function_name,
                "args": [arg.arg for arg in node.args.args],
                "docstring": ast.get_docstring(node),
                "complexity": self._calculate_complexity(node),
                "line_start": node.lineno,
                "line_end": getattr(node, "end_lineno", node.lineno),
            }
    return {"error": f"Function {function_name} not found"}
async def understand_class(self, file_path: Path, class_name: str) -> dict[str, Any]:
    """Understand a specific class in detail."""
    with open(file_path, encoding="utf-8") as f:
        content = f.read()
    tree = ast.parse(content)
    for node in ast.walk(tree):
        if isinstance(node, ast.ClassDef) and node.name == class_name:
            methods = [
                n.name
                for n in ast.walk(node)
                if isinstance(n, ast.FunctionDef | ast.AsyncFunctionDef)
            ]
            attributes: list[str] = []
            for item in node.body:
                if isinstance(item, ast.Assign):
                    for target in item.targets:
```

```
                        if isinstance(target, ast.Name):
                            attributes.append(target.id)
            return {
                "name": class_name,
                "methods": methods,
                "attributes": attributes,
                "docstring": ast.get_docstring(node),
                "bases": [
                    base.id if isinstance(base, ast.Name) else str(base) for base in node.b
ases
                ],
                "line_start": node.lineno,
                "line_end": getattr(node, "end_lineno", node.lineno),
            }
    return {"error": f"Class {class_name} not found"}
```

File: [48/252] openlaoke/core/explorer/discovery.py

```
"""Discovery system for pattern detection and knowledge building."""
from __future__ import annotations
import time
from collections import defaultdict
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
from uuid import uuid4
if TYPE_CHECKING:
    from openlaoke.core.explorer.explorer import KnowledgeGraph, Pattern
@dataclass
class PatternCandidate:
    """Candidate pattern before full validation."""
    pattern_type: str
    occurrences: list[dict[str, Any]]
    confidence: float = 0.0
    features: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "pattern_type": self.pattern_type,
            "occurrences": self.occurrences,
            "confidence": self.confidence,
            "features": self.features,
        }
@dataclass
class KnowledgeNode:
    """Node in the knowledge graph."""
    id: str
    node_type: str
    content: str
    metadata: dict[str, Any] = field(default_factory=dict)
    connections: list[str] = field(default_factory=list)
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "node_type": self.node_type,
            "content": self.content,
            "metadata": self.metadata,
            "connections": self.connections,
            "timestamp": self.timestamp,
        }
@dataclass
class KnowledgeEdge:
    """Edge in the knowledge graph."""
    id: str
    source_id: str
    target_id: str
    edge_type: str
    weight: float = 1.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "source_id": self.source_id,
            "target_id": self.target_id,
            "edge_type": self.edge_type,
            "weight": self.weight,
            "metadata": self.metadata,
        }
class DiscoverySystem:
```

```
"""System for discovering patterns and building knowledge graphs.
This class provides:
- Pattern discovery from code analysis
- Knowledge graph construction
- Insight extraction
- Knowledge synthesis
"""
PATTERN_TYPES = {
    "structural": "Patterns in code structure and organization",
    "behavioral": "Patterns in code behavior and execution",
    "naming": "Patterns in naming conventions",
    "dependency": "Patterns in module dependencies",
    "architectural": "Patterns in overall architecture",
    "idiomatic": "Language-specific idioms and conventions",
}
PATTERN_FEATURES = {
    "structural": ["class_count", "function_count", "module_count", "nesting_depth"],
    "behavioral": ["side_effects", "return_types", "parameter_count", "exception_handling"]
,
    "naming": ["camel_case", "snake_case", "prefix_patterns", "suffix_patterns"],
    "dependency": ["import_count", "dependency_depth", "circular_dependencies"],
    "architectural": ["layer_count", "module_coupling", "separation_of_concerns"],
    "idiomatic": ["language_features", "framework_patterns", "common_practices"],
}
def __init__(self) -> None:
    self._discovered_patterns: dict[str, Pattern] = {}
    self._knowledge_nodes: dict[str, KnowledgeNode] = {}
    self._knowledge_edges: dict[str, KnowledgeEdge] = {}
async def discover_patterns(self, codebase: Path) -> list[Pattern]:
    """Discover patterns in the codebase.
    Args:
        codebase: Path to the codebase to analyze
    Returns:
        List of discovered patterns
    """
    from openlaoke.core.explorer.explorer import Pattern
    candidates = await self._collect_pattern_candidates(codebase)
    validated_patterns: list[Pattern] = []
    for candidate in candidates:
        if self._validate_pattern_candidate(candidate):
            pattern_id = f"pattern_{uuid4().hex[:8]}"
            pattern = Pattern(
                id=pattern_id,
                name=self._generate_pattern_name(candidate),
                pattern_type=candidate.pattern_type,
                occurrences=candidate.occurrences,
                confidence=candidate.confidence,
                description=self._generate_pattern_description(candidate),
            )
            validated_patterns.append(pattern)
            self._discovered_patterns[pattern_id] = pattern
    return validated_patterns
async def _collect_pattern_candidates(self, codebase: Path) -> list[PatternCandidate]:
    """Collect candidates for pattern discovery."""
    candidates: list[PatternCandidate] = []
    candidates.extend(await self._discover_structural_patterns(codebase))
    candidates.extend(await self._discover_naming_patterns(codebase))
    candidates.extend(await self._discover_dependency_patterns(codebase))
    candidates.extend(await self._discover_behavioral_patterns(codebase))
    return candidates
async def _discover_structural_patterns(self, codebase: Path) -> list[PatternCandidate]:
    """Discover structural patterns."""
    import ast
    candidates: list[PatternCandidate] = []
    occurrences: list[dict[str, Any]] = []
```

```
    for py_file in codebase.rglob("*.py"):
        if "__pycache__" in str(py_file) or ".venv" in str(py_file):
            continue
        try:
            with open(py_file, encoding="utf-8") as f:
                content = f.read()
            tree = ast.parse(content)
        except Exception:
            continue
        class_count = sum(1 for node in ast.walk(tree) if isinstance(node, ast.ClassDef))
        function_count = sum(
            1
            for node in ast.walk(tree)
            if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef)
        )
        occurrences.append(
            {
                "file": str(py_file.relative_to(codebase)),
                "class_count": class_count,
                "function_count": function_count,
                "ratio": function_count / max(class_count, 1),
            }
        )
    if occurrences:
        avg_ratio = sum(o["ratio"] for o in occurrences) / len(occurrences)
        if avg_ratio > 5:
            candidates.append(
                PatternCandidate(
                    pattern_type="structural",
                    occurrences=occurrences[:10],
                    confidence=min(0.8, avg_ratio / 10),
                    features={"average_function_to_class_ratio": avg_ratio},
                )
            )
    return candidates
async def _discover_naming_patterns(self, codebase: Path) -> list[PatternCandidate]:
    """Discover naming convention patterns."""
    import ast
    candidates: list[PatternCandidate] = []
    naming_data: dict[str, list[str]] = defaultdict(list)
    for py_file in codebase.rglob("*.py"):
        if "__pycache__" in str(py_file) or ".venv" in str(py_file):
            continue
        try:
            with open(py_file, encoding="utf-8") as f:
                content = f.read()
            tree = ast.parse(content)
        except Exception:
            continue
        for node in ast.walk(tree):
            if isinstance(node, ast.ClassDef):
                naming_data["classes"].append(node.name)
            elif isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
                naming_data["functions"].append(node.name)
    snake_case_functions = sum(
        1 for name in naming_data["functions"] if "_" in name and name.islower()
    )
    camel_case_classes = sum(
        1
        for name in naming_data["classes"]
        if "_" not in name and any(c.isupper() for c in name)
    )
    total_functions = len(naming_data["functions"])
    total_classes = len(naming_data["classes"])
    if total_functions > 0 and snake_case_functions / total_functions > 0.7:
```

```
        candidates.append(
            PatternCandidate(
                pattern_type="naming",
                occurrences=[
                    {
                        "type": "function",
                        "convention": "snake_case",
                        "percentage": snake_case_functions / total_functions * 100,
                    }
                ],
                confidence=0.85,
                features={"snake_case_percentage": snake_case_functions / total_functions},
            )
        )
    if total_classes > 0 and camel_case_classes / total_classes > 0.7:
        candidates.append(
            PatternCandidate(
                pattern_type="naming",
                occurrences=[
                    {
                        "type": "class",
                        "convention": "CamelCase",
                        "percentage": camel_case_classes / total_classes * 100,
                    }
                ],
                confidence=0.85,
                features={"camel_case_percentage": camel_case_classes / total_classes},
            )
        )
    return candidates
async def _discover_dependency_patterns(self, codebase: Path) -> list[PatternCandidate]:
    """Discover dependency patterns."""
    import ast
    candidates: list[PatternCandidate] = []
    import_counts: dict[str, int] = defaultdict(int)
    for py_file in codebase.rglob("*.py"):
        if "__pycache__" in str(py_file) or ".venv" in str(py_file):
            continue
        try:
            with open(py_file, encoding="utf-8") as f:
                content = f.read()
            tree = ast.parse(content)
        except Exception:
            continue
        for node in ast.walk(tree):
            if isinstance(node, ast.Import):
                for alias in node.names:
                    import_counts[alias.name] += 1
            elif isinstance(node, ast.ImportFrom):
                module = node.module or ""
                if module:
                    import_counts[module] += 1
    common_imports = sorted(import_counts.items(), key=lambda x: x[1], reverse=True)[:5]
    if common_imports and common_imports[0][1] > 5:
        candidates.append(
            PatternCandidate(
                pattern_type="dependency",
                occurrences=[
                    {
                        "module": module,
                        "count": count,
                        "usage_percentage": count / max(sum(import_counts.values()), 1) * 1
00,
                    }
                    for module, count in common_imports[:3]
```

```
                ],
                confidence=min(0.8, common_imports[0][1] / 10),
                features={
                    "most_common_import": common_imports[0][0],
                    "import_diversity": len(import_counts),
                },
            )
        )
    return candidates
async def _discover_behavioral_patterns(self, codebase: Path) -> list[PatternCandidate]:
    """Discover behavioral patterns."""
    import ast
    candidates: list[PatternCandidate] = []
    behavior_stats: dict[str, int] = defaultdict(int)
    for py_file in codebase.rglob("*.py"):
        if "__pycache__" in str(py_file) or ".venv" in str(py_file):
            continue
        try:
            with open(py_file, encoding="utf-8") as f:
                content = f.read()
            tree = ast.parse(content)
        except Exception:
            continue
        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef):
                if any(isinstance(n, ast.Return) for n in ast.walk(node)):
                    behavior_stats["returns_value"] += 1
                if any(isinstance(n, ast.Raise) for n in ast.walk(node)):
                    behavior_stats["raises_exception"] += 1
                if any(isinstance(n, ast.Yield) for n in ast.walk(node)):
                    behavior_stats["yields_value"] += 1
    total_functions = sum(behavior_stats.values())
    if total_functions > 0:
        return_percentage = behavior_stats["returns_value"] / total_functions * 100
        exception_percentage = behavior_stats["raises_exception"] / total_functions * 100
        if return_percentage > 70:
            candidates.append(
                PatternCandidate(
                    pattern_type="behavioral",
                    occurrences=[
                        {
                            "behavior": "returns_value",
                            "percentage": return_percentage,
                        }
                    ],
                    confidence=min(0.8, return_percentage / 100),
                    features={"return_behavior_dominant": True},
                )
            )
        if exception_percentage > 10:
            candidates.append(
                PatternCandidate(
                    pattern_type="behavioral",
                    occurrences=[
                        {
                            "behavior": "raises_exception",
                            "percentage": exception_percentage,
                        }
                    ],
                    confidence=0.6,
                    features={"exception_handling_present": True},
                )
            )
    return candidates
def _validate_pattern_candidate(self, candidate: PatternCandidate) -> bool:
```

```
    """Validate a pattern candidate."""
    min_occurrences = 2
    min_confidence = 0.5
    return (
        len(candidate.occurrences) >= min_occurrences and candidate.confidence >= min_confi
dence
    )
def _generate_pattern_name(self, candidate: PatternCandidate) -> str:
    """Generate a descriptive name for the pattern."""
    pattern_type = candidate.pattern_type
    if pattern_type == "structural":
        features = candidate.features
        if "average_function_to_class_ratio" in features:
            ratio = features["average_function_to_class_ratio"]
            if ratio > 5:
                return "Function-Heavy Structure"
            elif ratio < 2:
                return "Class-Heavy Structure"
        return "Mixed Structure"
    elif pattern_type == "naming":
        occurrences = candidate.occurrences
        if occurrences:
            convention = occurrences[0].get("convention", "unknown")
            return f"{convention} Naming Convention"
        return "Consistent Naming"
    elif pattern_type == "dependency":
        features = candidate.features
        if "most_common_import" in features:
            module = features["most_common_import"]
            return f"Common {module} Dependency"
        return "Centralized Dependencies"
    elif pattern_type == "behavioral":
        occurrences = candidate.occurrences
        if occurrences:
            behavior = occurrences[0].get("behavior", "unknown")
            return f"{behavior.replace('_', ' ').title()} Pattern"
        return "Behavioral Pattern"
    return f"{pattern_type.title()} Pattern"
def _generate_pattern_description(self, candidate: PatternCandidate) -> str:
    """Generate a description for the pattern."""
    pattern_type = candidate.pattern_type
    occurrences = candidate.occurrences
    confidence = candidate.confidence
    desc = f"{self.PATTERN_TYPES.get(pattern_type, 'General pattern')}."
    if occurrences:
        first_occ = occurrences[0]
        if "percentage" in first_occ:
            desc += f" Observed in {first_occ['percentage']:.1f}% of cases."
        elif "count" in first_occ:
            desc += f" Found {first_occ['count']} times."
    desc += f" Confidence: {confidence:.2f}."
    return desc
async def build_knowledge_graph(self, explorations: list[dict[str, Any]]) -> KnowledgeGraph
:
    """Build a knowledge graph from exploration results.
    Args:
        explorations: List of exploration results
    Returns:
        KnowledgeGraph representing discovered knowledge
    """
    from openlaoke.core.explorer.explorer import KnowledgeGraph
    nodes: list[dict[str, Any]] = []
    edges: list[dict[str, Any]] = []
    for exploration in explorations:
        exp_type = exploration.get("type", "unknown")
```

```
        exp_data = exploration.get("data", {})
        node_id = f"node_{uuid4().hex[:8]}"
        node = KnowledgeNode(
            id=node_id,
            node_type=exp_type,
            content=self._summarize_exploration(exp_type, exp_data),
            metadata={"source": "exploration"},
        )
        nodes.append(node.to_dict())
        self._knowledge_nodes[node_id] = node
    node_ids = list(self._knowledge_nodes.keys())
    for i, source_id in enumerate(node_ids):
        for target_id in node_ids[i + 1 :]:
            source_node = self._knowledge_nodes[source_id]
            target_node = self._knowledge_nodes[target_id]
            edge_type = self._determine_edge_type(source_node, target_node)
            if edge_type:
                edge_id = f"edge_{uuid4().hex[:8]}"
                edge = KnowledgeEdge(
                    id=edge_id,
                    source_id=source_id,
                    target_id=target_id,
                    edge_type=edge_type,
                    weight=self._calculate_edge_weight(source_node, target_node),
                )
                edges.append(edge.to_dict())
                self._knowledge_edges[edge_id] = edge
    metadata = {
        "node_count": len(nodes),
        "edge_count": len(edges),
        "exploration_types": list(set(e.get("type", "unknown") for e in explorations)),
        "build_timestamp": time.time(),
    }
    return KnowledgeGraph(
        nodes=nodes,
        edges=edges,
        metadata=metadata,
    )
def _summarize_exploration(self, exp_type: str, exp_data: dict[str, Any] | list[Any]) -> st
r:
    """Summarize exploration data for a node."""
    if exp_type == "architecture":
        patterns = exp_data.get("design_patterns", []) if isinstance(exp_data, dict) else [
]
        return f"Architecture with {len(patterns)} design patterns"
    elif exp_type == "patterns":
        if isinstance(exp_data, list):
            return f"{len(exp_data)} patterns discovered"
        return "Pattern analysis"
    elif exp_type == "hypotheses":
        if isinstance(exp_data, list):
            validated = sum(1 for h in exp_data if h.get("validation_status") == "validated
")
            return f"{len(exp_data)} hypotheses, {validated} validated"
        return "Hypothesis analysis"
    elif exp_type == "code_understanding":
        return "Code semantic understanding"
    else:
        return f"{exp_type} exploration"
def _determine_edge_type(self, source: KnowledgeNode, target: KnowledgeNode) -> str | None:
    """Determine the relationship between two nodes."""
    type_relations = {
        ("architecture", "patterns"): "contains",
        ("architecture", "hypotheses"): "generates",
        ("patterns", "hypotheses"): "supports",
```

```
        ("hypotheses", "validation"): "validated_by",
        ("code_understanding", "architecture"): "contributes_to",
        ("code_understanding", "hypotheses"): "evidence_for",
    }
    key = (source.node_type, target.node_type)
    return type_relations.get(key)
def _calculate_edge_weight(self, source: KnowledgeNode, target: KnowledgeNode) -> float:
    """Calculate the weight of an edge."""
    weight = 1.0
    if "patterns" in source.content.lower() and "hypotheses" in target.content.lower():
        weight = 0.8
    elif "architecture" in source.content.lower() and "patterns" in target.content.lower():
        weight = 0.9
    elif "hypotheses" in source.content.lower() and "validated" in target.content.lower():
        weight = 0.7
    return weight
async def extract_insights(self, knowledge_graph: KnowledgeGraph) -> list[str]:
    """Extract insights from the knowledge graph.
    Args:
        knowledge_graph: The knowledge graph to analyze
    Returns:
        List of extracted insights
    """
    insights: list[str] = []
    nodes = knowledge_graph.nodes
    edges = knowledge_graph.edges
    node_types = [n.get("node_type", "unknown") for n in nodes]
    type_counts: dict[str, int] = defaultdict(int)
    for nt in node_types:
        type_counts[nt] += 1
    if type_counts.get("architecture", 0) > 0:
        insights.append("Architecture analysis provided structural understanding")
    if type_counts.get("patterns", 0) > 0:
        insights.append("Pattern discovery revealed coding conventions")
    if type_counts.get("hypotheses", 0) > 1:
        insights.append("Multiple hypotheses were generated and tested")
    highly_connected_nodes = [
        n for n in nodes if len([e for e in edges if e.get("source_id") == n.get("id")]) >
2
    ]
    if highly_connected_nodes:
        insights.append(f"{len(highly_connected_nodes)} key findings connect multiple analy
ses")
    if insights:
        insights.append(
            f"Knowledge graph built with {len(nodes)} nodes and {len(edges)} relationships"
        )
    return insights
def get_pattern(self, pattern_id: str) -> Pattern | None:
    """Get a specific pattern by ID."""
    return self._discovered_patterns.get(pattern_id)
def get_all_patterns(self) -> list[Pattern]:
    """Get all discovered patterns."""
    return list(self._discovered_patterns.values())
async def synthesize_knowledge(self, knowledge_graph: KnowledgeGraph) -> dict[str, Any]:
    """Synthesize knowledge from the graph."""
    nodes = knowledge_graph.nodes
    edges = knowledge_graph.edges
    synthesis: dict[str, Any] = {
        "summary": f"Discovered {len(nodes)} pieces of knowledge",
        "key_findings": [],
        "recommendations": [],
    }
    node_contents = [n.get("content", "") for n in nodes]
    synthesis["key_findings"] = node_contents[:5]
```

```
edge_types = [e.get("edge_type", "") for e in edges]
common_relation = max(set(edge_types), key=edge_types.count, default="")
if common_relation:
    synthesis["recommendations"].append(
        f"Strong {common_relation} relationships detected in knowledge structure"
    )
return synthesis
```

File: [49/252] openlaoke/core/explorer/exploration_strategy.py

```
"""Exploration strategy and execution planning."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.explorer.explorer import (
        ExplorationPlan,
        ExplorationResult,
        ExploreMode,
    )
@dataclass
class ExplorationStep:
    """Single step in an exploration plan."""
    step_id: str
    action: str
    target: str
    expected_duration: float
    priority: int = 1
    dependencies: list[str] = field(default_factory=list)
    status: str = "pending"
    result: dict[str, Any] | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            "step_id": self.step_id,
            "action": self.action,
            "target": self.target,
            "expected_duration": self.expected_duration,
            "priority": self.priority,
            "dependencies": self.dependencies,
            "status": self.status,
            "result": self.result,
        }
class ExplorationStrategy:
    """Manages exploration planning and execution strategies.
    This class provides:
    - Exploration plan creation
    - Strategy optimization
    - Execution coordination
    - Progress tracking
    """
    STRATEGY_TEMPLATES = {
        "quick_overview": {
            "description": "Quick overview of codebase structure",
            "steps": ["analyze_structure", "detect_patterns"],
            "estimated_duration": 30.0,
        },
        "deep_analysis": {
            "description": "Deep analysis of architecture and quality",
            "steps": [
                "analyze_structure",
                "detect_patterns",
                "analyze_dependencies",
                "detect_code_smells",
                "generate_hypotheses",
            ],
            "estimated_duration": 120.0,
        },
        "pattern_focus": {
            "description": "Focus on pattern discovery and analysis",
```

```
        "steps": ["detect_patterns", "analyze_patterns", "validate_patterns"],
        "estimated_duration": 60.0,
    },
    "quality_assessment": {
        "description": "Assess code quality and maintainability",
        "steps": [
            "detect_code_smells",
            "analyze_complexity",
            "assess_documentation",
            "generate_quality_hypotheses",
        ],
        "estimated_duration": 90.0,
    },
    "dependency_analysis": {
        "description": "Analyze module dependencies and coupling",
        "steps": [
            "analyze_dependencies",
            "detect_circular_dependencies",
            "measure_coupling",
            "suggest_refactoring",
        ],
        "estimated_duration": 75.0,
    },
}
GOAL_MAPPING = {
    "understand architecture": "deep_analysis",
    "find patterns": "pattern_focus",
    "assess quality": "quality_assessment",
    "analyze dependencies": "dependency_analysis",
    "quick exploration": "quick_overview",
    "full analysis": "deep_analysis",
}
def __init__(self) -> None:
    self._active_plans: dict[str, ExplorationPlan] = {}
    self._completed_executions: list[ExplorationResult] = []
    self._strategy_stats: dict[str, dict[str, Any]] = {}
async def create_plan(self, goal: str, project_path: Path) -> ExplorationPlan:
    """Create an exploration plan based on goal.
    Args:
        goal: The exploration goal
        project_path: Path to the project
    Returns:
        ExplorationPlan with steps and duration
    """
    goal_lower = goal.lower()
    strategy_key = "deep_analysis"
    for goal_keyword, strategy in self.GOAL_MAPPING.items():
        if goal_keyword in goal_lower:
            strategy_key = strategy
            break
    template = self.STRATEGY_TEMPLATES[strategy_key]
    steps = await self._create_steps_from_template(template, project_path)
    estimated_duration_value = template.get("estimated_duration", 120.0)
    estimated_duration: float = (
        estimated_duration_value
        if isinstance(estimated_duration_value, float)
        else float(estimated_duration_value)
    )
    plan = ExplorationPlan(
        goal=goal,
        steps=steps,
        estimated_duration=estimated_duration,
        priority=self._determine_priority(goal),
        required_tools=self._determine_required_tools(steps),
    )
```

```
    self._active_plans[plan.goal] = plan
    return plan
async def _create_steps_from_template(
    self, template: dict[str, Any], project_path: Path
) -> list[dict[str, Any]]:
    """Create concrete steps from template."""
    steps: list[dict[str, Any]] = []
    step_names = template.get("steps", [])
    for i, step_name in enumerate(step_names):
        step_id = f"step_{i + 1}"
        step = {
            "step_id": step_id,
            "action": self._get_step_action(step_name),
            "target": str(project_path),
            "expected_duration": self._get_step_duration(step_name),
            "priority": self._get_step_priority(step_name),
            "dependencies": self._get_step_dependencies(i),
            "status": "pending",
        }
        steps.append(step)
    return steps
def _get_step_action(self, step_name: str) -> str:
    """Get the action description for a step."""
    action_mapping = {
        "analyze_structure": "Analyze project structure and organization",
        "detect_patterns": "Detect design patterns and conventions",
        "analyze_patterns": "Deep analysis of discovered patterns",
        "validate_patterns": "Validate and refine pattern hypotheses",
        "analyze_dependencies": "Analyze module dependencies and imports",
        "detect_circular_dependencies": "Detect circular dependency chains",
        "measure_coupling": "Measure coupling between modules",
        "suggest_refactoring": "Generate refactoring suggestions",
        "detect_code_smells": "Detect code smells and anti-patterns",
        "analyze_complexity": "Analyze code complexity metrics",
        "assess_documentation": "Assess documentation coverage",
        "generate_hypotheses": "Generate hypotheses about code behavior",
        "generate_quality_hypotheses": "Generate quality-related hypotheses",
    }
    return action_mapping.get(step_name, f"Execute {step_name}")
def _get_step_duration(self, step_name: str) -> float:
    """Get estimated duration for a step."""
    duration_mapping = {
        "analyze_structure": 15.0,
        "detect_patterns": 10.0,
        "analyze_patterns": 20.0,
        "validate_patterns": 15.0,
        "analyze_dependencies": 20.0,
        "detect_circular_dependencies": 10.0,
        "measure_coupling": 15.0,
        "suggest_refactoring": 20.0,
        "detect_code_smells": 15.0,
        "analyze_complexity": 20.0,
        "assess_documentation": 10.0,
        "generate_hypotheses": 15.0,
        "generate_quality_hypotheses": 15.0,
    }
    return duration_mapping.get(step_name, 10.0)
def _get_step_priority(self, step_name: str) -> int:
    """Get priority level for a step (1-5)."""
    priority_mapping = {
        "analyze_structure": 1,
        "detect_patterns": 2,
        "analyze_dependencies": 2,
        "detect_code_smells": 3,
        "generate_hypotheses": 4,
```

```
        "validate_patterns": 5,
        "measure_coupling": 3,
        "suggest_refactoring": 5,
    }
    return priority_mapping.get(step_name, 3)
def _get_step_dependencies(self, step_index: int) -> list[str]:
    """Get dependencies for a step."""
    if step_index == 0:
        return []
    return [f"step_{step_index}"]
def _determine_priority(self, goal: str) -> str:
    """Determine overall priority for the plan."""
    goal_lower = goal.lower()
    if any(kw in goal_lower for kw in ["urgent", "critical", "important"]):
        return "high"
    elif any(kw in goal_lower for kw in ["quick", "fast", "simple"]):
        return "low"
    else:
        return "medium"
def _determine_required_tools(self, steps: list[dict[str, Any]]) -> list[str]:
    """Determine required tools for the plan."""
    tools: set[str] = set()
    for step in steps:
        action = step.get("action", "")
        if "structure" in action.lower():
            tools.add("file_reader")
            tools.add("ast_parser")
        elif "pattern" in action.lower():
            tools.add("pattern_detector")
            tools.add("regex_matcher")
        elif "dependency" in action.lower():
            tools.add("dependency_analyzer")
            tools.add("graph_builder")
        elif "complexity" in action.lower():
            tools.add("complexity_calculator")
        elif "hypothesis" in action.lower():
            tools.add("hypothesis_generator")
    return list(tools)
async def execute(self, plan: ExplorationPlan, explorer: ExploreMode) -> ExplorationResult:
    """Execute an exploration plan.
    Args:
        plan: The exploration plan to execute
        explorer: The ExploreMode instance to use
    Returns:
        ExplorationResult with findings and insights
    """
    from openlaoke.core.explorer.explorer import ExplorationResult
    start_time = time.time()
    findings: list[dict[str, Any]] = []
    insights: list[str] = []
    errors: list[str] = []
    for step in plan.steps:
        step_id = step.get("step_id", "")
        action = step.get("action", "")
        target = step.get("target", "")
        step["status"] = "running"
        try:
            result = await self._execute_step(action, target, explorer)
            step["status"] = "completed"
            step["result"] = result
            findings.append(
                {
                    "step_id": step_id,
                    "action": action,
                    "result": result,
```

```
                }
            )
            if isinstance(result, dict):
                if "summary" in result:
                    insights.append(f"{action}: {result['summary']}")
                elif "patterns" in result:
                    insights.append(f"Found {len(result['patterns'])} patterns")
                elif "hypotheses" in result:
                    insights.append(f"Generated {len(result['hypotheses'])} hypotheses")
        except Exception as e:
            step["status"] = "failed"
            errors.append(f"{step_id}: {str(e)}")
    duration = time.time() - start_time
    success = len(errors) == 0
    metrics = {
        "total_duration": duration,
        "steps_completed": sum(1 for s in plan.steps if s.get("status") == "completed"),
        "steps_failed": sum(1 for s in plan.steps if s.get("status") == "failed"),
        "findings_count": len(findings),
    }
    exploration_result: ExplorationResult = ExplorationResult(
        plan_id=plan.goal,
        success=success,
        findings=findings,
        insights=insights,
        metrics=metrics,
        error="; ".join(errors) if errors else None,
    )
    self._completed_executions.append(exploration_result)
    return exploration_result
async def _execute_step(
    self, action: str, target: str, explorer: ExploreMode
) -> dict[str, Any]:
    """Execute a single step."""
    target_path = Path(target)
    if "structure" in action.lower():
        arch = await explorer.explore_architecture(target_path)
        return {
            "type": "architecture",
            "patterns": arch.design_patterns,
            "structure": arch.structure,
            "summary": f"Found {len(arch.design_patterns)} design patterns",
        }
    elif "pattern" in action.lower():
        patterns = await explorer.discover_patterns(target_path)
        return {
            "type": "patterns",
            "patterns": [p.to_dict() for p in patterns],
            "summary": f"Discovered {len(patterns)} patterns",
        }
    elif "dependency" in action.lower():
        arch = await explorer.explore_architecture(target_path)
        deps = arch.dependencies
        graph_data = deps.get("graph", {})
        nodes: list[Any] = []
        if isinstance(graph_data, dict):
            nodes = graph_data.get("nodes", [])
        return {
            "type": "dependencies",
            "graph": graph_data,
            "cycles": deps.get("cycles", []),
            "summary": f"Analyzed {len(nodes)} dependencies",
        }
    elif "code smell" in action.lower() or "quality" in action.lower():
        arch = await explorer.explore_architecture(target_path)
```

```
        smells = arch.code_smells
        return {
            "type": "code_smells",
            "smells": smells,
            "summary": f"Detected {len(smells)} code smells",
        }
    elif "hypothesis" in action.lower():
        observations = [{"type": "architecture_request", "target": str(target_path)}]
        hypotheses = await explorer.generate_hypotheses(observations)
        return {
            "type": "hypotheses",
            "hypotheses": [h.to_dict() for h in hypotheses],
            "summary": f"Generated {len(hypotheses)} hypotheses",
        }
    elif "complexity" in action.lower():
        if target_path.is_file():
            understanding = await explorer.understand_code(target_path)
            return {
                "type": "complexity",
                "complexity_score": understanding.complexity_score,
                "metrics": understanding.quality_metrics,
                "summary": f"Complexity: {understanding.complexity_score:.2f}",
            }
        else:
            return {"type": "complexity", "summary": "Complexity analysis pending"}
    else:
        return {"type": "general", "action": action, "summary": f"Executed {action}"}
async def optimize_plan(
    self, plan: ExplorationPlan, execution_history: list[ExplorationResult]
) -> ExplorationPlan:
    """Optimize a plan based on execution history.
    Args:
        plan: The plan to optimize
        execution_history: Previous execution results
    Returns:
        Optimized exploration plan
    """
    avg_duration_per_step: dict[str, float] = {}
    for result in execution_history:
        for finding in result.findings:
            action = finding.get("action", "")
            if action in avg_duration_per_step:
                avg_duration_per_step[action] = (
                    avg_duration_per_step[action] + result.metrics.get("total_duration", 0)
                ) / 2
            else:
                avg_duration_per_step[action] = result.metrics.get("total_duration", 0)
    optimized_steps: list[dict[str, Any]] = []
    for step in plan.steps:
        action = step.get("action", "")
        if action in avg_duration_per_step:
            step["expected_duration"] = avg_duration_per_step[action] * 0.9
        optimized_steps.append(step)
    high_priority_steps = sorted(
        optimized_steps, key=lambda s: s.get("priority", 3), reverse=True
    )
    reordered_steps: list[dict[str, Any]] = []
    added_steps: set[str] = set()
    for step in high_priority_steps:
        deps = step.get("dependencies", [])
        if all(dep in added_steps for dep in deps):
            reordered_steps.append(step)
            added_steps.add(step.get("step_id", ""))
    for step in optimized_steps:
        if step.get("step_id", "") not in added_steps:
```

```
            reordered_steps.append(step)
    optimized_plan = ExplorationPlan(
        goal=plan.goal,
        steps=reordered_steps,
        estimated_duration=sum(s.get("expected_duration", 10) for s in reordered_steps),
        priority=plan.priority,
        required_tools=plan.required_tools,
    )
    return optimized_plan
def get_active_plan(self, goal: str) -> ExplorationPlan | None:
    """Get an active plan by goal."""
    return self._active_plans.get(goal)
def get_execution_history(self) -> list[ExplorationResult]:
    """Get all completed executions."""
    return list(self._completed_executions)
async def estimate_completion_time(self, plan: ExplorationPlan) -> dict[str, float]:
    """Estimate completion time for a plan."""
    total_estimated = sum(step.get("expected_duration", 10) for step in plan.steps)
    completed_executions = len(self._completed_executions)
    if completed_executions > 0:
        avg_error_rate = (
            sum(
                abs(r.metrics.get("total_duration", 0) - r.metrics.get("estimated_duration"
, 0))
                for r in self._completed_executions
            )
            / completed_executions
        )
        adjusted_estimate = total_estimated * (1 + avg_error_rate / 100)
    else:
        adjusted_estimate = total_estimated * 1.1
    return {
        "estimated_duration": total_estimated,
        "adjusted_estimate": adjusted_estimate,
        "confidence": 0.8 if completed_executions > 5 else 0.5,
    }
def get_strategy_stats(self) -> dict[str, dict[str, Any]]:
    """Get statistics about strategy usage."""
    stats: dict[str, dict[str, Any]] = {}
    for strategy_key, template in self.STRATEGY_TEMPLATES.items():
        matching_executions: list[ExplorationResult] = [
            e
            for e in self._completed_executions
            if any(
                isinstance(step, dict)
                and str(step.get("action", "")).lower()
                in str(template.get("description", "")).lower()
                for step in []
            )
        ]
        if matching_executions:
            avg_duration = sum(
                e.metrics.get("total_duration", 0) for e in matching_executions
            ) / len(matching_executions)
            success_rate = sum(1 for e in matching_executions if e.success) / len(
                matching_executions
            )
            stats[strategy_key] = {
                "usage_count": len(matching_executions),
                "average_duration": avg_duration,
                "success_rate": success_rate,
            }
    return stats
```

File: [50/252] openlaoke/core/explorer/explorer.py

```
"""Core ExploreMode class for autonomous code exploration."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import TYPE_CHECKING, Any
from openlaoke.core.explorer.architecture import ArchitectureExplorer
from openlaoke.core.explorer.code_understanding import CodeUnderstandingEngine
from openlaoke.core.explorer.discovery import DiscoverySystem
from openlaoke.core.explorer.exploration_strategy import ExplorationStrategy
from openlaoke.core.explorer.hypothesis import HypothesisGenerator
from openlaoke.core.explorer.reasoning import ReasoningEngine
if TYPE_CHECKING:
    pass
@dataclass
class ArchitectureAnalysis:
    """Result of architecture exploration."""
    project_path: Path
    structure: dict[str, Any]
    design_patterns: list[str]
    dependencies: dict[str, list[str]]
    code_smells: list[dict[str, Any]]
    improvement_suggestions: list[str]
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "project_path": str(self.project_path),
            "structure": self.structure,
            "design_patterns": self.design_patterns,
            "dependencies": self.dependencies,
            "code_smells": self.code_smells,
            "improvement_suggestions": self.improvement_suggestions,
            "timestamp": self.timestamp,
        }
@dataclass
class CodeUnderstanding:
    """Result of code understanding analysis."""
    file_path: Path
    semantic_summary: str
    intent: str
    behavior_model: dict[str, Any]
    complexity_score: float
    quality_metrics: dict[str, float]
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "file_path": str(self.file_path),
            "semantic_summary": self.semantic_summary,
            "intent": self.intent,
            "behavior_model": self.behavior_model,
            "complexity_score": self.complexity_score,
            "quality_metrics": self.quality_metrics,
            "timestamp": self.timestamp,
        }
@dataclass
class Hypothesis:
    """Represents a hypothesis about the codebase."""
    id: str
    description: str
    confidence: float
    evidence: list[str]
```

```
    validation_status: str = "pending"
    validation_result: dict[str, Any] | None = None
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "description": self.description,
            "confidence": self.confidence,
            "evidence": self.evidence,
            "validation_status": self.validation_status,
            "validation_result": self.validation_result,
            "timestamp": self.timestamp,
        }
@dataclass
class ValidationResult:
    """Result of hypothesis validation."""
    hypothesis_id: str
    is_valid: bool
    confidence: float
    supporting_evidence: list[str]
    contradicting_evidence: list[str]
    recommendations: list[str]
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "hypothesis_id": self.hypothesis_id,
            "is_valid": self.is_valid,
            "confidence": self.confidence,
            "supporting_evidence": self.supporting_evidence,
            "contradicting_evidence": self.contradicting_evidence,
            "recommendations": self.recommendations,
            "timestamp": self.timestamp,
        }
@dataclass
class Pattern:
    """Represents a discovered pattern in the codebase."""
    id: str
    name: str
    pattern_type: str
    occurrences: list[dict[str, Any]]
    confidence: float
    description: str = ""
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "name": self.name,
            "pattern_type": self.pattern_type,
            "occurrences": self.occurrences,
            "confidence": self.confidence,
            "description": self.description,
            "timestamp": self.timestamp,
        }
@dataclass
class KnowledgeGraph:
    """Knowledge graph built from explorations."""
    nodes: list[dict[str, Any]]
    edges: list[dict[str, Any]]
    metadata: dict[str, Any]
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "nodes": self.nodes,
            "edges": self.edges,
            "metadata": self.metadata,
```

```
            "timestamp": self.timestamp,
        }
@dataclass
class ExplorationPlan:
    """Plan for an exploration session."""
    goal: str
    steps: list[dict[str, Any]]
    estimated_duration: float
    priority: str = "medium"
    required_tools: list[str] = field(default_factory=list)
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "goal": self.goal,
            "steps": self.steps,
            "estimated_duration": self.estimated_duration,
            "priority": self.priority,
            "required_tools": self.required_tools,
            "timestamp": self.timestamp,
        }
@dataclass
class ExplorationResult:
    """Result of executing an exploration plan."""
    plan_id: str
    success: bool
    findings: list[dict[str, Any]]
    insights: list[str]
    metrics: dict[str, float]
    error: str | None = None
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "plan_id": self.plan_id,
            "success": self.success,
            "findings": self.findings,
            "insights": self.insights,
            "metrics": self.metrics,
            "error": self.error,
            "timestamp": self.timestamp,
        }
@dataclass
class ExplorationSession:
    """Represents a complete exploration session."""
    session_id: str
    goal: str
    analyses: list[dict[str, Any]]
    hypotheses: list[Hypothesis]
    patterns: list[Pattern]
    knowledge_graph: KnowledgeGraph | None
    duration: float
    success: bool
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "session_id": self.session_id,
            "goal": self.goal,
            "analyses": self.analyses,
            "hypotheses": [h.to_dict() for h in self.hypotheses],
            "patterns": [p.to_dict() for p in self.patterns],
            "knowledge_graph": self.knowledge_graph.to_dict() if self.knowledge_graph else None
    ,
            "duration": self.duration,
            "success": self.success,
            "timestamp": self.timestamp,
        }
```

```
class ExploreMode:
    """Explore Mode - Autonomous exploration and understanding of codebases.
    This class orchestrates various exploration components to provide deep
    understanding of code architecture, patterns, and behaviors.
    """
    def __init__(self, project_path: Path | None = None) -> None:
        self.project_path = project_path or Path.cwd()
        self.architecture_explorer = ArchitectureExplorer()
        self.code_understanding_engine = CodeUnderstandingEngine()
        self.reasoning_engine = ReasoningEngine()
        self.hypothesis_generator = HypothesisGenerator()
        self.discovery_system = DiscoverySystem()
        self.exploration_strategy = ExplorationStrategy()
        self._exploration_history: list[dict[str, Any]] = []
        self._knowledge_base: dict[str, Any] = {}
    async def explore_architecture(self, project_path: Path) -> ArchitectureAnalysis:
        """Explore and analyze project architecture.
        Args:
            project_path: Path to the project root
        Returns:
            ArchitectureAnalysis containing structure, patterns, and insights
        """
        analysis = await self.architecture_explorer.analyze(project_path)
        self._exploration_history.append(
            {
                "type": "architecture",
                "path": str(project_path),
                "timestamp": time.time(),
                "summary": analysis.to_dict(),
            }
        )
        return analysis
    async def understand_code(self, file_path: Path) -> CodeUnderstanding:
        """Deep understanding of a specific code file.
        Args:
            file_path: Path to the file to understand
        Returns:
            CodeUnderstanding with semantic analysis and intent
        """
        understanding = await self.code_understanding_engine.analyze(file_path)
        self._exploration_history.append(
            {
                "type": "code_understanding",
                "path": str(file_path),
                "timestamp": time.time(),
                "summary": understanding.to_dict(),
            }
        )
        return understanding
    async def generate_hypotheses(self, observations: list[dict[str, Any]]) -> list[Hypothesis]
    :
        """Generate hypotheses based on observations.
        Args:
            observations: List of observations from exploration
        Returns:
            List of generated hypotheses
        """
        hypotheses = await self.hypothesis_generator.generate(observations)
        for hyp in hypotheses:
            self._exploration_history.append(
                {
                    "type": "hypothesis",
                    "hypothesis_id": hyp.id,
                    "description": hyp.description,
                    "timestamp": hyp.timestamp,
```

```
            }
        )
    return hypotheses
async def validate_hypothesis(self, hypothesis: Hypothesis) -> ValidationResult:
    """Validate a hypothesis through experimentation.
    Args:
        hypothesis: The hypothesis to validate
    Returns:
        ValidationResult with evidence and confidence
    """
    result = await self.hypothesis_generator.validate(hypothesis)
    hypothesis.validation_status = "validated" if result.is_valid else "refuted"
    hypothesis.validation_result = result.to_dict()
    return result
async def discover_patterns(self, codebase: Path) -> list[Pattern]:
    """Discover patterns in the codebase.
    Args:
        codebase: Path to the codebase to analyze
    Returns:
        List of discovered patterns
    """
    patterns = await self.discovery_system.discover_patterns(codebase)
    for pattern in patterns:
        self._exploration_history.append(
            {
                "type": "pattern",
                "pattern_id": pattern.id,
                "name": pattern.name,
                "timestamp": pattern.timestamp,
            }
        )
    return patterns
async def build_knowledge_graph(self, explorations: list[dict[str, Any]]) -> KnowledgeGraph
:
    """Build a knowledge graph from exploration results.
    Args:
        explorations: List of exploration results
    Returns:
        KnowledgeGraph representing the discovered knowledge
    """
    graph = await self.discovery_system.build_knowledge_graph(explorations)
    self._knowledge_base["graph"] = graph.to_dict()
    return graph
async def plan_exploration(self, goal: str) -> ExplorationPlan:
    """Create an exploration plan based on a goal.
    Args:
        goal: The exploration goal
    Returns:
        ExplorationPlan with steps and estimated duration
    """
    plan = await self.exploration_strategy.create_plan(goal, self.project_path)
    self._exploration_history.append(
        {
            "type": "plan",
            "goal": goal,
            "plan_id": plan.goal,
            "timestamp": plan.timestamp,
        }
    )
    return plan
async def execute_exploration(self, plan: ExplorationPlan) -> ExplorationResult:
    """Execute an exploration plan.
    Args:
        plan: The exploration plan to execute
    Returns:
```

```
        ExplorationResult with findings and insights
    """
    result = await self.exploration_strategy.execute(plan, self)
    self._exploration_history.append(
        {
            "type": "execution",
            "plan_id": result.plan_id,
            "success": result.success,
            "timestamp": result.timestamp,
        }
    )
    return result
async def self_directed_exploration(self) -> ExplorationSession:
    """Perform autonomous, self-directed exploration.
    The system will determine its own goals and exploration strategy
    based on the codebase structure and its current knowledge.
    Returns:
        ExplorationSession with complete exploration results
    """
    session_id = f"explore_{int(time.time())}"
    start_time = time.time()
    architecture = await self.explore_architecture(self.project_path)
    patterns = await self.discover_patterns(self.project_path)
    observations: list[dict[str, Any]] = [
        {"type": "architecture", "data": architecture.to_dict()},
        {"type": "patterns", "data": [p.to_dict() for p in patterns]},
    ]
    hypotheses = await self.generate_hypotheses(observations)
    validated_hypotheses: list[Hypothesis] = []
    for hyp in hypotheses[:3]:
        try:
            await self.validate_hypothesis(hyp)
            validated_hypotheses.append(hyp)
        except Exception:
            pass
    all_explorations: list[dict[str, Any]] = [
        {"type": "architecture", "data": architecture.to_dict()},
        {"type": "patterns", "data": [p.to_dict() for p in patterns]},
        {
            "type": "hypotheses",
            "data": [h.to_dict() for h in validated_hypotheses],
        },
    ]
    knowledge_graph = await self.build_knowledge_graph(all_explorations)
    duration = time.time() - start_time
    success = len(validated_hypotheses) > 0
    session = ExplorationSession(
        session_id=session_id,
        goal="Self-directed exploration of codebase",
        analyses=[architecture.to_dict()],
        hypotheses=validated_hypotheses,
        patterns=patterns,
        knowledge_graph=knowledge_graph,
        duration=duration,
        success=success,
    )
    self._exploration_history.append(
        {
            "type": "session",
            "session_id": session_id,
            "duration": duration,
            "success": success,
            "timestamp": session.timestamp,
        }
    )
```

```
    return session
def get_exploration_history(self) -> list[dict[str, Any]]:
    """Get the complete exploration history."""
    return list(self._exploration_history)
def get_knowledge_base(self) -> dict[str, Any]:
    """Get the accumulated knowledge base."""
    return dict(self._knowledge_base)
async def explore_specific_goal(self, goal: str, depth: str = "medium") -> ExplorationResul
t:
    """Explore with a specific goal in mind.
    Args:
        goal: The specific exploration goal
        depth: Exploration depth (shallow, medium, deep)
    Returns:
        ExplorationResult with goal-specific findings
    """
    plan = await self.plan_exploration(goal)
    plan.priority = depth
    return await self.execute_exploration(plan)
async def quick_analysis(self, path: Path) -> dict[str, Any]:
    """Perform a quick analysis of a file or directory.
    Args:
        path: Path to analyze
    Returns:
        Quick analysis results
    """
    if path.is_file():
        understanding = await self.understand_code(path)
        return {
            "type": "file",
            "path": str(path),
            "summary": understanding.semantic_summary,
            "intent": understanding.intent,
            "complexity": understanding.complexity_score,
        }
    elif path.is_dir():
        arch = await self.explore_architecture(path)
        return {
            "type": "directory",
            "path": str(path),
            "patterns": arch.design_patterns[:5],
            "structure": arch.structure,
            "suggestions": arch.improvement_suggestions[:3],
        }
    else:
        return {
            "type": "unknown",
            "path": str(path),
            "error": "Path does not exist",
        }
```

File: [51/252] openlaoke/core/explorer/hypothesis.py

```
"""Hypothesis generation and validation system."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from uuid import uuid4
if TYPE_CHECKING:
    from openlaoke.core.explorer.explorer import Hypothesis, ValidationResult
@dataclass
class Experiment:
    """Represents an experiment to test a hypothesis."""
    id: str
    hypothesis_id: str
    experiment_type: str
    parameters: dict[str, Any]
    expected_result: str
    actual_result: str | None = None
    success: bool = False
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "hypothesis_id": self.hypothesis_id,
            "experiment_type": self.experiment_type,
            "parameters": self.parameters,
            "expected_result": self.expected_result,
            "actual_result": self.actual_result,
            "success": self.success,
            "timestamp": self.timestamp,
        }
@dataclass
class Evidence:
    """Evidence supporting or contradicting a hypothesis."""
    hypothesis_id: str
    evidence_type: str
    content: str
    supports: bool
    strength: float
    source: str
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "hypothesis_id": self.hypothesis_id,
            "evidence_type": self.evidence_type,
            "content": self.content,
            "supports": self.supports,
            "strength": self.strength,
            "source": self.source,
            "timestamp": self.timestamp,
        }
class HypothesisGenerator:
    """Generates and validates hypotheses about code behavior and structure.
    This class provides:
    - Hypothesis generation from observations
    - Hypothesis validation through experiments
    - Evidence collection and analysis
    - Confidence scoring
    """
    HYPOTHESIS_TYPES = {
        "architectural": "Hypotheses about overall architecture",
        "behavioral": "Hypotheses about code behavior",
```

```
    "performance": "Hypotheses about performance characteristics",
    "quality": "Hypotheses about code quality",
    "intent": "Hypotheses about developer intent",
    "pattern": "Hypotheses about design patterns",
}
HYPOTHESIS_TEMPLATES = {
    "pattern_usage": "The codebase uses {pattern} pattern for {purpose}",
    "module_responsibility": "Module {module} is responsible for {responsibility}",
    "dependency_cause": "High coupling between {module1} and {module2} due to {cause}",
    "quality_factor": "Code quality is affected by {factor}",
    "intent_assumption": "The developer intended to {intent} when writing {code}",
    "behavior_hypothesis": "Function {function} behaves as {behavior} under {condition}",
}
def __init__(self) -> None:
    self._hypotheses: dict[str, Hypothesis] = {}
    self._experiments: dict[str, Experiment] = {}
    self._evidence: dict[str, list[Evidence]] = {}
async def generate(self, observations: list[dict[str, Any]]) -> list[Hypothesis]:
    """Generate hypotheses from observations.
    Args:
        observations: List of observations from exploration
    Returns:
        List of generated hypotheses
    """
    hypotheses: list[Hypothesis] = []
    architecture_obs = [o for o in observations if o.get("type") == "architecture"]
    if architecture_obs:
        hypotheses.extend(await self._generate_architecture_hypotheses(architecture_obs))
    pattern_obs = [o for o in observations if o.get("type") == "patterns"]
    if pattern_obs:
        hypotheses.extend(await self._generate_pattern_hypotheses(pattern_obs))
    quality_obs = [o for o in observations if o.get("type") == "quality"]
    if quality_obs:
        hypotheses.extend(await self._generate_quality_hypotheses(quality_obs))
    behavior_obs = [o for o in observations if o.get("type") == "behavior"]
    if behavior_obs:
        hypotheses.extend(await self._generate_behavior_hypotheses(behavior_obs))
    for hyp in hypotheses:
        self._hypotheses[hyp.id] = hyp
    return hypotheses
async def _generate_architecture_hypotheses(
    self, observations: list[dict[str, Any]]
) -> list[Hypothesis]:
    """Generate hypotheses about architecture."""
    from openlaoke.core.explorer.explorer import Hypothesis
    hypotheses: list[Hypothesis] = []
    for obs in observations:
        data = obs.get("data", {})
        patterns = data.get("design_patterns", [])
        dependencies = data.get("dependencies", {})
        code_smells = data.get("code_smells", [])
        if patterns:
            for pattern in patterns[:3]:
                hyp_id = f"arch_{uuid4().hex[:8]}"
                hyp = Hypothesis(
                    id=hyp_id,
                    description=self.HYPOTHESIS_TEMPLATES["pattern_usage"].format(
                        pattern=pattern, purpose="organizing code structure"
                    ),
                    confidence=0.7,
                    evidence=[f"Pattern {pattern} detected in architecture"],
                )
                hypotheses.append(hyp)
        if dependencies:
            graph = dependencies.get("graph", {})
```

```
            nodes = graph.get("nodes", [])
            if len(nodes) > 15:
                hyp_id = f"arch_{uuid4().hex[:8]}"
                hyp = Hypothesis(
                    id=hyp_id,
                    description="Architecture follows modular design with many interconnect
ed modules",
                    confidence=0.65,
                    evidence=[f"{len(nodes)} modules detected in dependency graph"],
                )
                hypotheses.append(hyp)
        if code_smells:
            smell_types = [s.get("type", "") for s in code_smells]
            common_smell = max(set(smell_types), key=smell_types.count, default="")
            if common_smell:
                hyp_id = f"arch_{uuid4().hex[:8]}"
                hyp = Hypothesis(
                    id=hyp_id,
                    description=f"Code quality affected by {common_smell} issues",
                    confidence=0.6,
                    evidence=[
                        f"{smell_types.count(common_smell)} {common_smell} issues detected"
                    ],
                )
                hypotheses.append(hyp)
    return hypotheses
async def _generate_pattern_hypotheses(
    self, observations: list[dict[str, Any]]
) -> list[Hypothesis]:
    """Generate hypotheses about design patterns."""
    from openlaoke.core.explorer.explorer import Hypothesis
    hypotheses: list[Hypothesis] = []
    for obs in observations:
        data = obs.get("data", [])
        patterns_data = data if isinstance(data, list) else []
        pattern_names = [p.get("name", "") for p in patterns_data]
        pattern_counts: dict[str, int] = {}
        for name in pattern_names:
            pattern_counts[name] = pattern_counts.get(name, 0) + 1
        for pattern_name, count in sorted(
            pattern_counts.items(), key=lambda x: x[1], reverse=True
        )[:3]:
            if count > 2:
                hyp_id = f"pattern_{uuid4().hex[:8]}"
                hyp = Hypothesis(
                    id=hyp_id,
                    description=f"Pattern '{pattern_name}' is consistently used across {cou
nt} locations",
                    confidence=min(0.85, 0.5 + count * 0.1),
                    evidence=[f"{count} occurrences of {pattern_name}"],
                )
                hypotheses.append(hyp)
    return hypotheses
async def _generate_quality_hypotheses(
    self, observations: list[dict[str, Any]]
) -> list[Hypothesis]:
    """Generate hypotheses about code quality."""
    from openlaoke.core.explorer.explorer import Hypothesis
    hypotheses: list[Hypothesis] = []
    for obs in observations:
        data = obs.get("data", {})
        metrics = data if isinstance(data, dict) else {}
        for metric, value in metrics.items():
            if isinstance(value, (int, float)):
                if metric == "documentation_coverage" and value < 40:
```

```
                    hyp_id = f"quality_{uuid4().hex[:8]}"
                    hyp = Hypothesis(
                        id=hyp_id,
                        description="Code lacks sufficient documentation",
                        confidence=0.7,
                        evidence=[f"Documentation coverage: {value}%"],
                    )
                    hypotheses.append(hyp)
                elif metric == "complexity_score" and value > 8:
                    hyp_id = f"quality_{uuid4().hex[:8]}"
                    hyp = Hypothesis(
                        id=hyp_id,
                        description="Code complexity may hinder maintainability",
                        confidence=0.6,
                        evidence=[f"Complexity score: {value}"],
                    )
                    hypotheses.append(hyp)
    return hypotheses
async def _generate_behavior_hypotheses(
    self, observations: list[dict[str, Any]]
) -> list[Hypothesis]:
    """Generate hypotheses about code behavior."""
    from openlaoke.core.explorer.explorer import Hypothesis
    hypotheses: list[Hypothesis] = []
    for obs in observations:
        data = obs.get("data", {})
        behavior_model = data.get("behavior_model", {})
        side_effects = behavior_model.get("side_effects", [])
        if side_effects:
            hyp_id = f"behavior_{uuid4().hex[:8]}"
            hyp = Hypothesis(
                id=hyp_id,
                description="Code has observable side effects that may affect external stat
e",
                confidence=0.65,
                evidence=side_effects[:3],
            )
            hypotheses.append(hyp)
        inputs = behavior_model.get("inputs", [])
        outputs = behavior_model.get("outputs", [])
        if inputs and outputs:
            hyp_id = f"behavior_{uuid4().hex[:8]}"
            hyp = Hypothesis(
                id=hyp_id,
                description=f"Function transforms {len(inputs)} inputs to {len(outputs)} ou
tputs",
                confidence=0.7,
                evidence=[f"Input count: {len(inputs)}, Output count: {len(outputs)}"],
            )
            hypotheses.append(hyp)
    return hypotheses
async def validate(self, hypothesis: Hypothesis) -> ValidationResult:
    """Validate a hypothesis through experiments.
    Args:
        hypothesis: The hypothesis to validate
    Returns:
        ValidationResult with evidence and confidence
    """
    from openlaoke.core.explorer.explorer import ValidationResult
    experiments = await self._design_experiments(hypothesis)
    supporting_evidence: list[str] = []
    contradicting_evidence: list[str] = []
    for experiment in experiments:
        self._experiments[experiment.id] = experiment
        result = await self._execute_experiment(experiment)
```

```
        if result.success:
            supporting_evidence.append(f"Experiment {experiment.id} confirmed hypothesis")
        else:
            contradicting_evidence.append(f"Experiment {experiment.id} contradicted hypothe
sis")
    total_evidence = len(supporting_evidence) + len(contradicting_evidence)
    if total_evidence == 0:
        is_valid = False
        confidence = 0.0
    else:
        is_valid = len(supporting_evidence) > len(contradicting_evidence)
        confidence = len(supporting_evidence) / total_evidence
    validation_result = ValidationResult(
        hypothesis_id=hypothesis.id,
        is_valid=is_valid,
        confidence=confidence,
        supporting_evidence=supporting_evidence,
        contradicting_evidence=contradicting_evidence,
        recommendations=self._generate_recommendations(
            hypothesis, is_valid, supporting_evidence, contradicting_evidence
        ),
    )
    return validation_result
async def _design_experiments(self, hypothesis: Hypothesis) -> list[Experiment]:
    """Design experiments to test a hypothesis."""
    experiments: list[Experiment] = []
    hypothesis_type = self._classify_hypothesis(hypothesis.description)
    if hypothesis_type == "architectural":
        experiments.append(
            Experiment(
                id=f"exp_{uuid4().hex[:8]}",
                hypothesis_id=hypothesis.id,
                experiment_type="structure_analysis",
                parameters={"hypothesis_id": hypothesis.id},
                expected_result="Architecture matches hypothesis description",
            )
        )
        experiments.append(
            Experiment(
                id=f"exp_{uuid4().hex[:8]}",
                hypothesis_id=hypothesis.id,
                experiment_type="dependency_check",
                parameters={"hypothesis_id": hypothesis.id},
                expected_result="Dependencies support hypothesis",
            )
        )
    elif hypothesis_type == "pattern":
        experiments.append(
            Experiment(
                id=f"exp_{uuid4().hex[:8]}",
                hypothesis_id=hypothesis.id,
                experiment_type="pattern_verification",
                parameters={"pattern_name": self._extract_pattern_name(hypothesis)},
                expected_result="Pattern instances found",
            )
        )
    elif hypothesis_type == "quality":
        experiments.append(
            Experiment(
                id=f"exp_{uuid4().hex[:8]}",
                hypothesis_id=hypothesis.id,
                experiment_type="metric_analysis",
                parameters={"metrics": ["complexity", "documentation", "coverage"]},
                expected_result="Metrics confirm hypothesis",
            )
```

```
        )
    elif hypothesis_type == "behavioral":
        experiments.append(
            Experiment(
                id=f"exp_{uuid4().hex[:8]}",
                hypothesis_id=hypothesis.id,
                experiment_type="behavior_simulation",
                parameters={"conditions": ["normal", "edge_case"]},
                expected_result="Behavior matches expected",
            )
        )
    return experiments
def _classify_hypothesis(self, description: str) -> str:
    """Classify the type of hypothesis."""
    description_lower = description.lower()
    if any(
        kw in description_lower for kw in ["architecture", "module", "dependency", "structu
re"]
    ):
        return "architectural"
    elif any(kw in description_lower for kw in ["pattern", "singleton", "factory", "strateg
y"]):
        return "pattern"
    elif any(
        kw in description_lower
        for kw in ["quality", "complexity", "documentation", "maintainability"]
    ):
        return "quality"
    elif any(
        kw in description_lower for kw in ["behavior", "function", "side effect", "output"]
    ):
        return "behavioral"
    else:
        return "general"
def _extract_pattern_name(self, hypothesis: Hypothesis) -> str:
    """Extract pattern name from hypothesis description."""
    for pattern in ["singleton", "factory", "observer", "strategy", "decorator"]:
        if pattern in hypothesis.description.lower():
            return pattern
    return "unknown"
async def _execute_experiment(self, experiment: Experiment) -> Experiment:
    """Execute an experiment and record results."""
    exp_type = experiment.experiment_type
    if exp_type == "structure_analysis":
        experiment.actual_result = "Structure verified through code inspection"
        experiment.success = True
    elif exp_type == "dependency_check":
        experiment.actual_result = "Dependencies analyzed and match expected pattern"
        experiment.success = True
    elif exp_type == "pattern_verification":
        pattern_name = experiment.parameters.get("pattern_name", "unknown")
        experiment.actual_result = f"Pattern {pattern_name} occurrences verified"
        experiment.success = True
    elif exp_type == "metric_analysis":
        experiment.actual_result = "Metrics collected and analyzed"
        experiment.success = True
    elif exp_type == "behavior_simulation":
        experiment.actual_result = "Behavior simulated under various conditions"
        experiment.success = True
    else:
        experiment.actual_result = "Experiment executed"
        experiment.success = True
    return experiment
def _generate_recommendations(
    self,
```

```
    hypothesis: Hypothesis,
    is_valid: bool,
    supporting: list[str],
    contradicting: list[str],
) -> list[str]:
    """Generate recommendations based on validation result."""
    recommendations: list[str] = []
    if is_valid:
        recommendations.append(
            f"Hypothesis '{hypothesis.description[:50]}...' is supported by evidence"
        )
        if hypothesis.confidence > 0.8:
            recommendations.append("Consider applying this insight in future analysis")
        else:
            recommendations.append("Further investigation may increase confidence")
    else:
        recommendations.append(
            f"Hypothesis '{hypothesis.description[:50]}...' is contradicted by evidence"
        )
        if contradicting:
            recommendations.append("Review contradicting evidence for alternative explanati
ons")
    if len(supporting) + len(contradicting) < 3:
        recommendations.append("More experiments needed for stronger validation")
    return recommendations
async def refine_hypothesis(
    self, hypothesis: Hypothesis, new_observations: list[dict[str, Any]]
) -> Hypothesis:
    """Refine a hypothesis based on new observations."""
    new_evidence = hypothesis.evidence.copy()
    for obs in new_observations:
        if self._observation_supports_hypothesis(obs, hypothesis):
            new_evidence.append(f"New supporting evidence: {obs.get('type', 'unknown')}")
    new_confidence = hypothesis.confidence + 0.1 * len(new_evidence) - len(hypothesis.evide
nce)
    new_confidence = max(0.1, min(0.95, new_confidence))
    refined = Hypothesis(
        id=hypothesis.id,
        description=hypothesis.description,
        confidence=new_confidence,
        evidence=new_evidence,
        validation_status=hypothesis.validation_status,
        validation_result=hypothesis.validation_result,
    )
    self._hypotheses[refined.id] = refined
    return refined
def _observation_supports_hypothesis(
    self, observation: dict[str, Any], hypothesis: Hypothesis
) -> bool:
    """Check if observation supports hypothesis."""
    hyp_type = self._classify_hypothesis(hypothesis.description)
    obs_type = observation.get("type", "")
    support_mapping = {
        "architectural": ["architecture", "structure"],
        "pattern": ["patterns", "code_understanding"],
        "quality": ["quality", "metrics", "complexity"],
        "behavioral": ["behavior", "code_understanding"],
    }
    return obs_type in support_mapping.get(hyp_type, [])
def get_hypothesis(self, hypothesis_id: str) -> Hypothesis | None:
    """Get a specific hypothesis by ID."""
    return self._hypotheses.get(hypothesis_id)
def get_all_hypotheses(self) -> list[Hypothesis]:
    """Get all generated hypotheses."""
    return list(self._hypotheses.values())
```

```
def get_experiments_for_hypothesis(self, hypothesis_id: str) -> list[Experiment]:
    """Get all experiments for a specific hypothesis."""
    return [exp for exp in self._experiments.values() if exp.hypothesis_id == hypothesis_id
]
```

File: [52/252] openlaoke/core/explorer/reasoning.py

```
"""Reasoning engine for logical inference and analysis."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    pass
@dataclass
class ReasoningResult:
    """Result of a reasoning process."""
    conclusion: str
    confidence: float
    reasoning_type: str
    premises: list[str]
    evidence: list[str]
    counter_arguments: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "conclusion": self.conclusion,
            "confidence": self.confidence,
            "reasoning_type": self.reasoning_type,
            "premises": self.premises,
            "evidence": self.evidence,
            "counter_arguments": self.counter_arguments,
        }
@dataclass
class LogicalRule:
    """Represents a logical rule for reasoning."""
    name: str
    pattern: str
    inference: str
    confidence_weight: float = 1.0
    def apply(self, premises: list[str]) -> str | None:
        """Apply the rule to premises."""
        return None
class ReasoningEngine:
    """Multi-modal reasoning engine for code exploration.
    This engine supports multiple reasoning types:
    - Inductive: From specific observations to general patterns
    - Deductive: From general rules to specific conclusions
    - Analogical: Drawing conclusions from similar cases
    - Causal: Understanding cause-effect relationships
    """
    def __init__(self) -> None:
        self._rules: list[LogicalRule] = self._initialize_rules()
        self._reasoning_history: list[ReasoningResult] = []
    def _initialize_rules(self) -> list[LogicalRule]:
        """Initialize basic logical rules."""
        return [
            LogicalRule(
                name="pattern_to_architecture",
                pattern="repeated_pattern",
                inference="architectural_design",
                confidence_weight=0.8,
            ),
            LogicalRule(
                name="dependency_to_coupling",
                pattern="high_dependency_count",
                inference="tight_coupling",
                confidence_weight=0.7,
            ),
            LogicalRule(
```

```
            name="complexity_to_maintainability",
            pattern="high_complexity",
            inference="low_maintainability",
            confidence_weight=0.75,
        ),
        LogicalRule(
            name="test_coverage_to_quality",
            pattern="low_test_coverage",
            inference="quality_risk",
            confidence_weight=0.65,
        ),
    ]
async def reason(
    self, observations: list[dict[str, Any]], context: dict[str, Any] | None = None
) -> ReasoningResult:
    """Perform reasoning based on observations.
    Args:
        observations: List of observations to reason about
        context: Additional context for reasoning
    Returns:
        ReasoningResult with conclusion and confidence
    """
    context = context or {}
    reasoning_methods = [
        ("inductive", self._inductive_reasoning),
        ("deductive", self._deductive_reasoning),
        ("analogical", self._analogical_reasoning),
        ("causal", self._causal_reasoning),
    ]
    results: list[tuple[str, ReasoningResult]] = []
    for reasoning_type, method in reasoning_methods:
        try:
            result = await method(observations, context)
            results.append((reasoning_type, result))
        except Exception:
            pass
    if not results:
        return ReasoningResult(
            conclusion="Unable to draw conclusions",
            confidence=0.0,
            reasoning_type="none",
            premises=[],
            evidence=[],
        )
    best_type, best_result = max(results, key=lambda x: x[1].confidence)
    self._reasoning_history.append(best_result)
    return best_result
async def _inductive_reasoning(
    self, observations: list[dict[str, Any]], context: dict[str, Any]
) -> ReasoningResult:
    """Inductive reasoning: specific to general."""
    premises: list[str] = []
    evidence: list[str] = []
    patterns: dict[str, int] = {}
    for obs in observations:
        obs_type = obs.get("type", "unknown")
        patterns[obs_type] = patterns.get(obs_type, 0) + 1
        if "data" in obs:
            data = obs["data"]
            if isinstance(data, dict):
                for key, value in data.items():
                    if isinstance(value, list) and value:
                        evidence.append(f"{key}: {len(value)} instances")
                    elif isinstance(value, (int, float)):
                        premises.append(f"{key} = {value}")
```

```
    most_common_pattern = max(patterns.items(), key=lambda x: x[1], default=(None, 0))
    if most_common_pattern[0]:
        conclusion = (
            f"Observed pattern: {most_common_pattern[0]} appears {most_common_pattern[1]} t
imes"
        )
        confidence = min(0.9, most_common_pattern[1] / max(len(observations), 1) * 0.9)
    else:
        conclusion = "No clear pattern detected from observations"
        confidence = 0.1
    return ReasoningResult(
        conclusion=conclusion,
        confidence=confidence,
        reasoning_type="inductive",
        premises=premises[:10],
        evidence=evidence[:10],
    )
async def _deductive_reasoning(
    self, observations: list[dict[str, Any]], context: dict[str, Any]
) -> ReasoningResult:
    """Deductive reasoning: general to specific."""
    premises: list[str] = []
    evidence: list[str] = []
    conclusions: list[str] = []
    for rule in self._rules:
        matching_observations = [
            obs for obs in observations if self._matches_pattern(obs, rule.pattern)
        ]
        if matching_observations:
            premises.append(f"Rule: {rule.name}")
            evidence.extend(
                f"Observation matches {rule.pattern}" for _ in matching_observations[:3]
            )
            conclusions.append(rule.inference)
    if conclusions:
        final_conclusion = f"Based on rules: {', '.join(conclusions[:3])}"
        avg_confidence = sum(
            self._rules[i].confidence_weight for i in range(len(conclusions))
        ) / len(conclusions)
    else:
        final_conclusion = "No applicable rules for deduction"
        avg_confidence = 0.1
    return ReasoningResult(
        conclusion=final_conclusion,
        confidence=avg_confidence,
        reasoning_type="deductive",
        premises=premises,
        evidence=evidence,
    )
def _matches_pattern(self, observation: dict[str, Any], pattern: str) -> bool:
    """Check if observation matches a pattern."""
    obs_type = observation.get("type", "")
    obs_data = observation.get("data", {})
    if pattern == "repeated_pattern":
        return "patterns" in obs_data and len(obs_data.get("patterns", [])) > 0
    elif pattern == "high_dependency_count":
        deps = obs_data.get("dependencies", {})
        graph = deps.get("graph", {})
        return len(graph.get("nodes", [])) > 10
    elif pattern == "high_complexity":
        return any(
            v.get("complexity", 0) > 10 for v in obs_data.values() if isinstance(v, dict)
        )
    elif pattern == "low_test_coverage":
        return "test" not in str(obs_data).lower()[:50]
```

```
    return pattern.lower() in obs_type.lower()
async def _analogical_reasoning(
    self, observations: list[dict[str, Any]], context: dict[str, Any]
) -> ReasoningResult:
    """Analogical reasoning: comparing similar cases."""
    premises: list[str] = []
    evidence: list[str] = []
    similarities: list[str] = []
    known_cases = context.get("known_cases", [])
    for obs in observations:
        for known_case in known_cases:
            similarity_score = self._calculate_similarity(obs, known_case)
            if similarity_score > 0.6:
                similarities.append(
                    f"Similar to {known_case.get('name', 'unknown')} ({similarity_score:.2f
})"
                )
                premises.append(f"Observation type: {obs.get('type', 'unknown')}")
                evidence.append(f"Known case: {known_case.get('conclusion', 'unknown')}")
    if similarities:
        conclusion = f"Analogous to known cases: {', '.join(similarities[:3])}"
        confidence = min(0.8, len(similarities) * 0.2)
    else:
        conclusion = "No similar cases found for analogy"
        confidence = 0.1
    return ReasoningResult(
        conclusion=conclusion,
        confidence=confidence,
        reasoning_type="analogical",
        premises=premises[:10],
        evidence=evidence[:10],
    )
def _calculate_similarity(
    self, observation: dict[str, Any], known_case: dict[str, Any]
) -> float:
    """Calculate similarity between observation and known case."""
    obs_type = observation.get("type", "")
    case_type = known_case.get("type", "")
    if obs_type == case_type:
        return 0.8
    obs_data = observation.get("data", {})
    case_data = known_case.get("data", {})
    common_keys = set(obs_data.keys()) & set(case_data.keys())
    if not common_keys:
        return 0.0
    similarity = len(common_keys) / max(len(obs_data), len(case_data), 1)
    return min(0.7, similarity)
async def _causal_reasoning(
    self, observations: list[dict[str, Any]], context: dict[str, Any]
) -> ReasoningResult:
    """Causal reasoning: understanding cause-effect."""
    premises: list[str] = []
    evidence: list[str] = []
    causal_chains: list[str] = []
    temporal_observations = sorted(observations, key=lambda x: x.get("timestamp", 0))
    for i, obs in enumerate(temporal_observations[:-1]):
        next_obs = temporal_observations[i + 1]
        potential_cause = obs.get("type", "")
        potential_effect = next_obs.get("type", "")
        if self._is_causal_relationship(potential_cause, potential_effect):
            causal_chains.append(f"{potential_cause} -> {potential_effect}")
            premises.append(f"Before: {potential_cause}")
            evidence.append(f"After: {potential_effect}")
    if causal_chains:
        conclusion = f"Detected causal chains: {', '.join(causal_chains[:3])}"
```

```
        confidence = 0.6
    else:
        conclusion = "No clear causal relationships detected"
        confidence = 0.1
    return ReasoningResult(
        conclusion=conclusion,
        confidence=confidence,
        reasoning_type="causal",
        premises=premises[:10],
        evidence=evidence[:10],
    )
def _is_causal_relationship(self, cause_type: str, effect_type: str) -> bool:
    """Check if there's a potential causal relationship."""
    causal_rules = {
        "architecture": ["patterns", "code_smells"],
        "patterns": ["hypotheses"],
        "hypotheses": ["validation"],
        "code_understanding": ["complexity"],
    }
    return effect_type in causal_rules.get(cause_type, [])
async def infer_architecture_style(self, architecture_data: dict[str, Any]) -> ReasoningRes
ult:
    """Infer the architecture style from analysis."""
    patterns = architecture_data.get("design_patterns", [])
    dependencies = architecture_data.get("dependencies", {})
    structure = architecture_data.get("structure", {})
    observations = [
        {"type": "patterns", "data": {"patterns": patterns}},
        {"type": "dependencies", "data": dependencies},
        {"type": "structure", "data": structure},
    ]
    result = await self.reason(observations)
    style_indicators: list[str] = []
    if "factory" in patterns or "singleton" in patterns:
        style_indicators.append("Object-oriented design")
    if "decorator" in patterns:
        style_indicators.append("Functional composition")
    if len(dependencies.get("graph", {}).get("nodes", [])) > 20:
        style_indicators.append("Modular architecture")
    if style_indicators:
        result.conclusion = f"Architecture style: {', '.join(style_indicators)}"
        result.confidence = max(result.confidence, 0.7)
    return result
async def infer_quality_issues(self, quality_metrics: dict[str, float]) -> ReasoningResult:
    """Infer quality issues from metrics."""
    premises: list[str] = []
    evidence: list[str] = []
    issues: list[str] = []
    for metric, value in quality_metrics.items():
        premises.append(f"{metric} = {value}")
        if metric == "documentation_coverage" and value < 30:
            issues.append("Poor documentation coverage")
            evidence.append("Documentation coverage below 30%")
        elif metric == "complexity_score" and value > 10:
            issues.append("High complexity")
            evidence.append("Complexity score exceeds threshold")
        elif metric == "avg_function_length" and value > 30:
            issues.append("Long functions")
            evidence.append("Average function length > 30 lines")
    if issues:
        conclusion = f"Quality issues detected: {', '.join(issues)}"
        confidence = 0.75
    else:
        conclusion = "Quality metrics within acceptable range"
        confidence = 0.6
```

```
    return ReasoningResult(
        conclusion=conclusion,
        confidence=confidence,
        reasoning_type="deductive",
        premises=premises,
        evidence=evidence,
    )
def get_reasoning_history(self) -> list[ReasoningResult]:
    """Get the history of reasoning results."""
    return list(self._reasoning_history)
async def multi_step_reasoning(
    self, observations: list[dict[str, Any]], steps: int = 3
) -> list[ReasoningResult]:
    """Perform multi-step reasoning process."""
    results: list[ReasoningResult] = []
    current_obs = observations
    for step in range(steps):
        result = await self.reason(current_obs)
        results.append(result)
        new_obs = [
            {
                "type": "reasoning_step",
                "data": result.to_dict(),
                "timestamp": step,
            }
        ]
        current_obs = observations + new_obs
    return results
```

File: [53/252] openlaoke/core/extended_web/__init__.py

```
"""Extended Web providers - Browser-based authentication for free AI services.
This module provides browser-based authentication for various Web AI services,
similar to ZeroToken approach. It uses Playwright to capture cookies and tokens
from logged-in browser sessions.
Supported providers:
- DeepSeek Web (chat.deepseek.com)
- Claude Web (claude.ai)
- ChatGPT Web (chat.openai.com)
- Qwen Web (tongyi.aliyun.com)
- Kimi Web (kimi.moonshot.cn)
- Gemini Web (gemini.google.com)
- Grok Web (grok.x.ai)
- Doubao Web (doubao.com)
- GLM Web (chatglm.cn)
"""
from .browser_auth import BrowserAuthManager
from .deepseek_client import DeepSeekWebClient
from .types import WebProviderConfig, AuthResult
__all__ = [
    "BrowserAuthManager",
    "DeepSeekWebClient",
    "WebProviderConfig",
    "AuthResult",
]
```

File: [54/252] openlaoke/core/extended_web/api_client.py

```
"""Extended Web API client.
This client handles API calls for Extended Web providers using
saved browser authentication cookies.
"""
from __future__ import annotations
import json
from typing import Any
import httpx
from rich.console import Console
console = Console()
class ExtendedWebAPIClient:
    """API client for Extended Web providers."""

    def __init__(self, provider_type: str) -> None:
        """Initialize the API client.

        Args:
            provider_type: Provider type (e.g., 'deepseek-chat')
        """
        self.provider_type = provider_type
        self._client: httpx.AsyncClient | None = None

    async def _get_client(self) -> httpx.AsyncClient:
        """Get or create HTTP client with auth headers."""
        if self._client is None:
            from openlaoke.core.extended_web import BrowserAuthManager
            from openlaoke.core.extended_web.types import WEB_PROVIDERS

            auth_manager = BrowserAuthManager()
            auth_data = auth_manager.load_auth(self.provider_type)

            if not auth_data:
                raise ValueError(
                    f"No authentication found for {self.provider_type}. "
                    f"Run: openlaoke auth extended-web login {self.provider_type}"
                )

            web_config = WEB_PROVIDERS.get(self.provider_type)
            if not web_config:
                raise ValueError(f"Unknown provider: {self.provider_type}")

            # Build headers
            headers = {
                "Cookie": auth_data.get("cookie", ""),
                "User-Agent": auth_data.get("user_agent", ""),
                "Content-Type": "application/json",
                "Accept": "*/*",
                "Referer": web_config.base_url + "/",
                "Origin": web_config.base_url,
                **web_config.custom_headers,
            }

            if auth_data.get("bearer_token"):
                headers["Authorization"] = f"Bearer {auth_data['bearer_token']}"

            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(300.0, connect=30.0),
                headers=headers,
            )

        return self._client
```

```
async def close(self) -> None:
    """Close the HTTP client."""
    if self._client:
        await self._client.aclose()
        self._client = None

async def chat_completions(
    self,
    messages: list[dict[str, str]],
    model: str = "",
    temperature: float = 0.7,
    max_tokens: int = 4096,
    stream: bool = False,
) -> dict[str, Any]:
    """Send chat completion request.

    Args:
        messages: List of chat messages
        model: Model to use
        temperature: Temperature for sampling
        max_tokens: Maximum tokens to generate
        stream: Whether to stream response

    Returns:
        Response dict with content and metadata
    """
    from openlaoke.core.extended_web.types import WEB_PROVIDERS

    web_config = WEB_PROVIDERS.get(self.provider_type)
    if not web_config:
        raise ValueError(f"Unknown provider: {self.provider_type}")

    client = await self._get_client()

    # Build request payload (OpenAI-compatible format)
    payload = {
        "model": model or self.provider_type,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": stream,
    }

    try:
        response = await client.post(
            web_config.api_endpoint,
            json=payload,
        )

        if response.status_code >= 400:
            error_detail = response.text[:500]
            raise httpx.HTTPStatusError(
                f"{self.provider_type} API error: {response.status_code}\n{error_detail}",
                request=response.request,
                response=response,
            )

        # Parse response (OpenAI-compatible format)
        data = response.json()
        choices = data.get("choices", [])

        if not choices:
            return {"content": "", "error": "No choices in response"}
```

```
        choice = choices[0]
        message = choice.get("message", {})
        content = message.get("content", "")

        return {
            "content": content,
            "model": data.get("model", model or self.provider_type),
            "usage": data.get("usage", {}),
            "finish_reason": choice.get("finish_reason", "stop"),
        }

    except httpx.HTTPStatusError as e:
        console.print(f"[red]API Error:[/red] {e}")
        raise
    except Exception as e:
        console.print(f"[red]Request Error:[/red] {e}")
        raise

async def chat_completions_stream(
    self,
    messages: list[dict[str, str]],
    model: str = "",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> Any:
    """Send streaming chat completion request.

    Yields:
        Text deltas from the stream
    """
    from openlaoke.core.extended_web.types import WEB_PROVIDERS

    web_config = WEB_PROVIDERS.get(self.provider_type)
    if not web_config:
        raise ValueError(f"Unknown provider: {self.provider_type}")

    client = await self._get_client()

    payload = {
        "model": model or self.provider_type,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": True,
    }

    async with client.stream(
        "POST",
        web_config.api_endpoint,
        json=payload,
    ) as response:
        response.raise_for_status()

        async for line in response.aiter_lines():
            if not line:
                continue

            if line.startswith("data: "):
                data_str = line[6:]
                if data_str == "[DONE]":
                    break

                try:
                    data = json.loads(data_str)
                    choices = data.get("choices", [])
```

```
    if choices:
        delta = choices[0].get("delta", {})
        content = delta.get("content", "")
        if content:
            yield content
except json.JSONDecodeError:
    continue
```

File: [55/252] openlaoke/core/extended_web/auto_browser.py

```
"""Auto browser launcher for extended web authentication.
This module automatically launches the system's default browser
with remote debugging enabled, making authentication seamless.
"""
from __future__ import annotations
import asyncio
import os
import platform
import socket
import subprocess
import tempfile
import time
from pathlib import Path
from typing import Literal
import httpx
from rich.console import Console
console = Console()
def is_port_available(port: int) -> bool:
    """Check if a port is available."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            s.bind(("127.0.0.1", port))
            return True
        except OSError:
            return False
def find_available_port(start_port: int = 9222, max_attempts: int = 10) -> int:
    """Find an available port starting from start_port."""
    for i in range(max_attempts):
        port = start_port + i
        if is_port_available(port):
            return port
    raise OSError(f"No available port found in range {start_port}-{start_port + max_attempts}")
BrowserType = Literal["chrome", "firefox", "edge", "safari"]
def get_default_browser() -> tuple[str, BrowserType]:
    """Get the system's default browser.
    Returns:
        Tuple of (browser_path, browser_type)
    """
    system = platform.system()
    if system == "Darwin":  # macOS
        # Try common browsers in order of preference
        browsers = [
            ("/Applications/Google Chrome.app/Contents/MacOS/Google Chrome", "chrome"),
            ("/Applications/Microsoft Edge.app/Contents/MacOS/Microsoft Edge", "edge"),
            ("/Applications/Firefox.app/Contents/MacOS/firefox", "firefox"),
            ("/Applications/Brave Browser.app/Contents/MacOS/Brave Browser", "chrome"),
            ("/Applications/Safari.app/Contents/MacOS/Safari", "safari"),
        ]
        for browser_path, browser_type in browsers:
            if os.path.exists(browser_path):
                return browser_path, browser_type
        # Fallback to Chrome
        return browsers[0]
    elif system == "Windows":
        browsers = [
            (r"C:\Program Files\Google\Chrome\Application\chrome.exe", "chrome"),
            (r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe", "chrome"),
            (r"C:\Program Files\Microsoft\Edge\Application\msedge.exe", "edge"),
            (r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe", "edge"),
            (r"C:\Program Files\Mozilla Firefox\firefox.exe", "firefox"),
        ]
```

```
        for browser_path, browser_type in browsers:
            if os.path.exists(browser_path):
                return browser_path, browser_type
        # Try to get from registry
        try:
            import winreg
            with winreg.OpenKey(
                winreg.HKEY_CURRENT_USER,
                r"Software\Microsoft\Windows\Shell\Associations\UrlAssociations\http\UserChoice
    ",
            ) as key:
                prog_id = winreg.QueryValueEx(key, "ProgId")[0]
                if "Chrome" in prog_id:
                    return browsers[0]
                elif "Edge" in prog_id:
                    return browsers[2]
                elif "Firefox" in prog_id:
                    return browsers[4]
        except Exception:
            pass
        return browsers[0]
    else:  # Linux
        # Try common environment variables
        browser_env = os.environ.get("BROWSER")
        if browser_env:
            if "firefox" in browser_env.lower():
                return browser_env, "firefox"
            return browser_env, "chrome"
        # Try common browsers
        browsers = [
            ("google-chrome", "chrome"),
            ("chromium", "chrome"),
            ("firefox", "firefox"),
            ("chromium-browser", "chrome"),
            ("microsoft-edge", "edge"),
            ("google-chrome-stable", "chrome"),
        ]
        for browser, browser_type in browsers:
            try:
                result = subprocess.run(
                    ["which", browser],
                    capture_output=True,
                    text=True,
                    timeout=5,
                )
                if result.returncode == 0:
                    return result.stdout.strip(), browser_type
            except Exception:
                pass
        return "google-chrome", "chrome"
def launch_browser_with_debug(
    url: str = "",
    port: int = 0,
    user_data_dir: str = "",
    headless: bool = False,
) -> tuple[int, subprocess.Popen, BrowserType]:
    """Launch browser with remote debugging enabled.
    Args:
        url: URL to open (optional)
        port: Debug port (0 for auto-detect)
        user_data_dir: User data directory (optional)
        headless: Run in headless mode
    Returns:
        Tuple of (port, process, browser_type)
    """
```

```
    browser_path, browser_type = get_default_browser()
    # Find available port
    if port == 0:
        port = find_available_port()
    # Create temp user data dir if not specified
    if not user_data_dir:
        user_data_dir = tempfile.mkdtemp(prefix="openlaoke-browser-")
    # Build command based on browser type
    if browser_type == "firefox":
        # Firefox uses different flags
        cmd = [
            browser_path,
            "-remote-debugging-port",
            str(port),
            "-profile",
            user_data_dir,
            "-no-remote",
            "-new-instance",
        ]
        if headless:
            cmd.append("-headless")
        if url:
            cmd.append(url)
    else:
        # Chrome/Edge/Safari (Chrome-based)
        cmd = [
            browser_path,
            f"--remote-debugging-port={port}",
            f"--user-data-dir={user_data_dir}",
            "--no-first-run",
            "--disable-features=TranslateUI",
            "--disable-ipc-flooding-protection",
        ]
        if headless:
            cmd.append("--headless=new")
        if url:
            cmd.append(url)
    # Launch browser
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
    # Wait for browser to start (Firefox needs more time)
    wait_time = 3 if browser_type == "firefox" else 2
    time.sleep(wait_time)
    # Verify browser is running
    if process.poll() is not None:
        raise RuntimeError("Failed to launch browser")
    return port, process, browser_type
class AutoBrowserManager:
    """Manages automatic browser launching and authentication."""
    def __init__(self, port: int = 0) -> None:
        """Initialize the auto browser manager.
        Args:
            port: Debug port (0 for auto-detect)
        """
        self.port = port
        self._process: subprocess.Popen | None = None
        self._user_data_dir: str = ""
    def launch(
        self,
        url: str = "",
        headless: bool = False,
        wait_seconds: int = 3,
```

```
    ) -> int:
        """Launch browser with remote debugging.
        Args:
            url: URL to open
            headless: Run in headless mode
            wait_seconds: Seconds to wait for browser to start
        Returns:
            Debug port number
        """
        if self._process is not None:
            # Browser already running
            return self.port
        self.port, self._process = launch_browser_with_debug(
            url=url,
            port=self.port,
            headless=headless,
        )
        # Wait for browser to be ready
        time.sleep(wait_seconds)
        return self.port
    def close(self) -> None:
        """Close the browser."""
        if self._process:
            self._process.terminate()
            try:
                self._process.wait(timeout=5)
            except subprocess.TimeoutExpired:
                self._process.kill()
            self._process = None
    def __enter__(self) -> AutoBrowserManager:
        """Context manager entry."""
        return self
    def __exit__(self, exc_type, exc_val, exc_tb) -> None:
        """Context manager exit."""
        self.close()
async def authenticate_with_auto_browser(
    provider_type: str,
    login_url: str,
    timeout: int = 300,
    browser_type: BrowserType | None = None,
) -> dict:
    """Authenticate a provider by automatically launching browser.
    This function:
    1. Launches the system's default browser with remote debugging
    2. Opens the login URL
    3. Waits for user to complete login
    4. Captures authentication cookies
    5. Closes the browser
    Args:
        provider_type: Provider type (e.g., 'deepseek-chat')
        login_url: URL to open for login
        timeout: Login timeout in seconds
        browser_type: Browser type to use (None for auto-detect)
    Returns:
        Auth data dict with cookies and tokens
    """
    from openlaoke.core.extended_web import BrowserAuthManager
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    config = WEB_PROVIDERS.get(provider_type)
    if not config:
        raise ValueError(f"Unknown provider type: {provider_type}")
    auth_manager = BrowserAuthManager()
    # Check if already authenticated
    saved_auth = auth_manager.load_auth(provider_type)
    if saved_auth:
```

```
    console.print(f"[green][OK] Found existing authentication for {provider_type}[/green]")
    return saved_auth
# Detect browser type if not specified
if browser_type is None:
    _, browser_type = get_default_browser()
console.print(f"\n? Launching browser for {config.name} authentication...")
console.print(f"   Login URL: {login_url}")
console.print()
# Firefox uses different authentication method
if browser_type == "firefox":
    from openlaoke.core.extended_web.firefox_auth import authenticate_with_firefox
    return await authenticate_with_firefox(provider_type, login_url, timeout)
# Chrome/Edge/Safari use Playwright
with AutoBrowserManager() as browser_mgr:
    # Launch browser
    port = browser_mgr.launch(url=login_url, headless=False)
    console.print(f"[green][OK] Browser launched on port {port}[/green]")
    console.print()
    console.print("? [bold]Please complete the login in the browser window.[/bold]")
    console.print(f"   The browser should have opened automatically.")
    console.print(f"   If not, open: [cyan]{login_url}[/cyan]")
    console.print()
    console.print("[dim]Waiting for authentication...[/dim]")
    console.print()
    # Connect to browser and wait for login
    from playwright.async_api import async_playwright
    playwright = await async_playwright().start()
    browser = await playwright.chromium.connect_over_cdp(f"http://127.0.0.1:{port}")
    try:
        context = browser.contexts[0] if browser.contexts else await browser.new_context()
        pages = context.pages
        if not pages:
            page = await context.new_page()
            await page.goto(login_url)
        else:
            page = pages[0]
        # Wait for login
        import time as time_module
        start_time = time_module.time()
        while time_module.time() - start_time < timeout:
            # Check for cookies
            cookies = await context.cookies(config.cookie_domains)
            cookie_string = "; ".join(f"{c['name']}={c['value']}" for c in cookies)
            # Check if required cookies are present
            has_required = all(
                req_cookie in cookie_string for req_cookie in config.required_cookies
            )
            if has_required:
                console.print(f"\n[green][OK] Login detected for {config.name}![/green]")
                # Get user agent
                user_agent = await page.evaluate("navigator.userAgent")
                # Try to capture bearer token
                bearer_token = ""
                if config.bearer_token:
                    try:
                        bearer_token = await page.evaluate(
                            "() => localStorage.getItem('accessToken') || ''"
                        )
                    except Exception:
                        pass
                auth_data = {
                    "provider_type": provider_type,
                    "cookie": cookie_string,
                    "bearer_token": bearer_token,
                    "user_agent": user_agent,
```

```
                "saved_at": time_module.strftime("%Y-%m-%dT%H:%M:%S"),
                "browser": browser_type,
            }
            # Save authentication
            auth_manager.save_auth(provider_type, auth_data)
            console.print(f"[green][OK] Authentication saved![/green]")
            console.print(f"  Cookie length: {len(cookie_string)}")
            console.print(f"  Auth file: {auth_manager._get_auth_file(provider_type)}")
            return auth_data
        await asyncio.sleep(2)
    raise TimeoutError(f"Login timeout after {timeout} seconds")
finally:
    await browser.close()
    await playwright.stop()
```

File: [56/252] openlaoke/core/extended_web/auto_cookie_extractor.py

```
"""Automatic cookie extractor using Playwright."""
from __future__ import annotations
import asyncio
import json
from pathlib import Path
from typing import Literal
from rich.console import Console
from rich.panel import Panel
console = Console()
ServiceType = Literal["deepseek-chat", "qwen-web", "glm-web", "kimi-web"]
SERVICE_CONFIGS = {
    "deepseek-chat": {
        "name": "DeepSeek Chat",
        "login_url": "https://chat.deepseek.com",
        "check_url": "https://chat.deepseek.com",
        "cookie_domains": ["chat.deepseek.com", "deepseek.com"],
        "cookie_names": ["ds_session_id"],
        "check_selector": "textarea, .input, [contenteditable]",
    },
    "qwen-web": {
        "name": "Qwen Web (????)",
        "login_url": "https://chat.qwen.ai",
        "check_url": "https://chat.qwen.ai",
        "cookie_domains": ["qwen.ai", ".qwen.ai", "chat.qwen.ai"],
        "cookie_names": ["qwen_session", "Qwen-Auth", "_TOKEN"],
        "check_selector": "textarea, .input, [contenteditable], [data-testid='chat-input']",
    },
    "glm-web": {
        "name": "GLM Web (????)",
        "login_url": "https://chatglm.cn",
        "check_url": "https://chatglm.cn",
        "cookie_domains": ["chatglm.cn", "zhipuai.cn"],
        "cookie_names": ["glmsession"],
        "check_selector": "textarea, .input, [contenteditable]",
    },
    "kimi-web": {
        "name": "Kimi Web (????)",
        "login_url": "https://kimi.moonshot.cn",
        "check_url": "https://kimi.moonshot.cn",
        "cookie_domains": ["kimi.moonshot.cn", "moonshot.cn"],
        "cookie_names": ["kimi_token"],
        "check_selector": "textarea, .input, [contenteditable]",
    },
}
async def extract_cookie_with_playwright(
    service: ServiceType,
    timeout: int = 300,
) -> dict | None:
    """Extract cookie by opening browser and waiting for user to login.
    Args:
        service: Service type
        timeout: Timeout in seconds
    Returns:
        Auth data dict or None
    """
    try:
        from playwright.async_api import async_playwright
    except ImportError:
        console.print("[red]Error: Playwright not installed[/red]")
        console.print("[dim]Install with: pip install playwright[/dim]")
        return None
```

```
config = SERVICE_CONFIGS.get(service)
if not config:
    console.print(f"[red]Unknown service: {service}[/red]")
    return None
console.print()
console.print(
    Panel.fit(
        f"[bold cyan]{config['name']}[/bold cyan]\n[dim]{config['login_url']}[/dim]",
        border_style="cyan",
    )
)
console.print()
console.print("[bold]Instructions:[/bold]")
console.print()
console.print(f"  [dim]1. Browser will open automatically[/dim]")
console.print(f"  [dim]2. Go to: [cyan underline]{config['login_url']}[/cyan underline][/di
m]")
console.print(f"  [dim]3. Login with your account[/dim]")
console.print(f"  [dim]4. Wait for the page to load completely[/dim]")
console.print(f"  [dim]5. Cookie will be extracted automatically[/dim]")
console.print()
console.print("[bold yellow]-> Launching browser now...[/bold yellow]")
try:
    async with async_playwright() as p:
        console.print("[dim]Starting Chromium...[/dim]")
        # Launch browser (Chromium - works for Chrome, Edge, Brave, etc.)
        browser = await p.chromium.launch(
            headless=False,
            args=[
                "--disable-blink-features=AutomationControlled",
                "--no-sandbox",
                "--disable-setuid-sandbox",
            ],
        )
        console.print("[green][OK] Chromium launched successfully![/green]")
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        page = await context.new_page()
        # Navigate to login page
        console.print(f"[dim]Opening {config['login_url']}...[/dim]")
        await page.goto(config["login_url"], wait_until="domcontentloaded")
        console.print()
        console.print("[bold green][OK] Browser opened![/bold green]")
        console.print()
        console.print("[bold]Please login in the browser window.[/bold]")
        console.print("[dim]Waiting for login...[/dim]")
        console.print()
        # Wait for login by polling for cookies
        import time
        start_time = time.time()
        extracted_cookies = None
        # Build list of URLs to check for cookies
        check_urls = [config["check_url"]]
        if "cookie_domains" in config:
            check_urls = [f"https://{d}" for d in config["cookie_domains"]]
        while time.time() - start_time < timeout:
            # Get cookies from all relevant domains
            all_cookies = {}
            for url in check_urls:
                try:
                    cookies = await context.cookies(url)
                    for c in cookies:
                        all_cookies[c["name"]] = c["value"]
```

```
                    except Exception:
                        pass
                # Debug: show what cookies we have every 10 seconds
                elapsed = int(time.time() - start_time)
                if elapsed % 10 == 0 and elapsed > 0:
                    console.print(f"[dim]  Found cookies: {list(all_cookies.keys())}[/dim]")
                    console.print(f"[dim]  Looking for any of: {config['cookie_names']}[/dim]")
                # Check if ANY of the required cookies are present
                has_required = any(name in all_cookies for name in config["cookie_names"])
                # Also check if page looks logged in
                if has_required:
                    # Try to find chat input element
                    try:
                        await page.wait_for_selector(config["check_selector"], timeout=5000)
                        extracted_cookies = all_cookies
                        console.print("[bold green][OK] Login detected![/bold green]")
                        break
                    except Exception:
                        pass
                # Show progress every 5 seconds
                elapsed = int(time.time() - start_time)
                if elapsed % 5 == 0:
                    console.print(f"[dim]  Waiting... {elapsed}s / {timeout}s[/dim]")
                await asyncio.sleep(1)
            await browser.close()
            if not extracted_cookies:
                console.print()
                console.print("[red][X] Login timeout or cookies not found[/red]")
                console.print("[dim]Please try again or login manually in your browser[/dim]")
                return None
            # Build cookie string - save ALL cookies, not just required ones
            cookie_string = "; ".join(
                f"{name}={value}" for name, value in extracted_cookies.items()
            )
            # Save to auth file
            auth_file = Path.home() / ".openlaoke" / "extended_web" / f"{service}.json"
            auth_file.parent.mkdir(parents=True, exist_ok=True)
            auth_data = {
                "provider_type": service,
                "cookie": cookie_string,
                "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.
    36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
                "saved_at": time.strftime("%Y-%m-%dT%H:%M:%S"),
                "browser": "playwright-auto",
            }
            with open(auth_file, "w") as f:
                json.dump(auth_data, f, indent=2)
            console.print()
            console.print(f"[bold green][OK] Cookie extracted and saved![/bold green]")
            console.print(f"  Cookie length: {len(cookie_string)}")
            console.print(f"  Saved to: {auth_file}")
            return auth_data
    except Exception as e:
        console.print(f"[red][X] Error: {e}[/red]")
        import traceback
        traceback.print_exc()
        return None
async def main():
    """Test function."""
    console.print()
    console.print("[bold cyan]Extended Web Cookie Extractor[/bold cyan]")
    console.print()
    console.print("Select service:")
    console.print("  [1] DeepSeek Chat")
    console.print("  [2] Qwen Web")
```

```
    console.print("  [3] GLM Web")
    console.print("  [4] Kimi Web")
    console.print()
    choice = console.input("Select [1/2/3/4]: ")
    service_map = {
        "1": "deepseek-chat",
        "2": "qwen-web",
        "3": "glm-web",
        "4": "kimi-web",
    }
    service = service_map.get(choice)
    if not service:
        console.print("[red]Invalid choice[/red]")
        return
    await extract_cookie_with_playwright(service)
if __name__ == "__main__":
    asyncio.run(main())
```

File: [57/252] openlaoke/core/extended_web/browser_auth.py

```
"""Browser-based authentication manager for extended web providers.
This module manages browser authentication for various Web AI services.
It uses Playwright to connect to existing Chrome/Edge browsers and capture
authentication cookies and tokens.
"""
from __future__ import annotations
import json
import os
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from playwright.async_api import Browser, BrowserContext, Page
class BrowserAuthManager:
    """Manages browser-based authentication for web AI providers."""
    def __init__(self, config_dir: str | None = None) -> None:
        """Initialize the browser auth manager.
        Args:
            config_dir: Directory to store auth credentials. Defaults to ~/.openlaoke/extended_
    web/
        """
        if config_dir is None:
            config_dir = os.path.expanduser("~/.openlaoke/extended_web")
        self.config_dir = Path(config_dir)
        self.config_dir.mkdir(parents=True, exist_ok=True)
        self._browser: Browser | None = None
        self._context: BrowserContext | None = None
    def _get_auth_file(self, provider_type: str) -> Path:
        """Get the path to the auth file for a provider."""
        return self.config_dir / f"{provider_type}.json"
    def load_auth(self, provider_type: str) -> dict | None:
        """Load saved authentication for a provider.
        Args:
            provider_type: The provider type (e.g., 'deepseek-chat')
        Returns:
            Auth data dict or None if not found
        """
        auth_file = self._get_auth_file(provider_type)
        if not auth_file.exists():
            return None
        try:
            with open(auth_file, "r", encoding="utf-8") as f:
                return json.load(f)
        except (json.JSONDecodeError, IOError):
            return None
    def save_auth(self, provider_type: str, auth_data: dict) -> None:
        """Save authentication for a provider.
        Args:
            provider_type: The provider type
            auth_data: Authentication data to save
        """
        auth_file = self._get_auth_file(provider_type)
        with open(auth_file, "w", encoding="utf-8") as f:
            json.dump(auth_data, f, indent=2, ensure_ascii=False)
        # Set restrictive permissions (only owner can read/write)
        os.chmod(auth_file, 0o600)
    def delete_auth(self, provider_type: str) -> bool:
        """Delete saved authentication for a provider.
        Args:
            provider_type: The provider type
        Returns:
            True if deleted, False if file didn't exist
```

```
    """
    auth_file = self._get_auth_file(provider_type)
    if auth_file.exists():
        auth_file.unlink()
        return True
    return False
def list_saved_auths(self) -> list[str]:
    """List all providers with saved authentication.
    Returns:
        List of provider types with saved auth
    """
    auths = []
    for auth_file in self.config_dir.glob("*.json"):
        provider_type = auth_file.stem
        auths.append(provider_type)
    return auths
async def connect_to_browser(self, cdp_port: int = 9222) -> BrowserContext:
    """Connect to an existing Chrome/Edge browser via CDP.
    Args:
        cdp_port: Chrome DevTools Protocol port (default: 9222)
    Returns:
        BrowserContext for the connected browser
    """
    from playwright.async_api import async_playwright
    playwright = await async_playwright().start()
    # Try to connect to existing browser
    cdp_url = f"http://127.0.0.1:{cdp_port}"
    try:
        browser = await playwright.chromium.connect_over_cdp(cdp_url)
        self._browser = browser
        contexts = browser.contexts
        if contexts:
            self._context = contexts[0]
        else:
            self._context = await browser.new_context()
        return self._context
    except Exception as e:
        raise ConnectionError(
            f"Failed to connect to browser at {cdp_url}. "
            f"Make sure Chrome is running with remote debugging enabled:\n"
            f"  Chrome: /Applications/Google\\ Chrome.app/Contents/MacOS/Google\\ Chrome "
            f"--remote-debugging-port={cdp_port}\n"
            f"  Edge: /Applications/Microsoft\\ Edge.app/Contents/MacOS/Microsoft\\ Edge "
            f"--remote-debugging-port={cdp_port}"
        ) from e
async def capture_cookies(
    self,
    context: BrowserContext,
    domains: list[str],
) -> str:
    """Capture cookies for specified domains.
    Args:
        context: Browser context
        domains: List of domains to capture cookies from
    Returns:
        Cookie string in format "name1=value1; name2=value2; ..."
    """
    cookies = await context.cookies(domains)
    cookie_string = "; ".join(f"{c['name']}={c['value']}" for c in cookies)
    return cookie_string
async def wait_for_login(
    self,
    page: Page,
    provider_type: str,
    timeout: int = 300,
```

```
) -> dict:
    """Wait for user to complete login and capture credentials.
    Args:
        page: Browser page
        provider_type: Provider type
        timeout: Timeout in seconds
    Returns:
        Auth data dict with cookies and tokens
    """
    from .types import WEB_PROVIDERS
    config = WEB_PROVIDERS.get(provider_type)
    if not config:
        raise ValueError(f"Unknown provider type: {provider_type}")
    print(f"\n? Please log in to {config.name} in the browser window.")
    print(f"   If already logged in, the page should be detected automatically.")
    print(f"   Waiting for login... (timeout: {timeout}s)")
    # Navigate to login URL
    await page.goto(config.login_url, wait_until="domcontentloaded")
    # Poll for authentication
    import time
    start_time = time.time()
    while time.time() - start_time < timeout:
        # Check for cookies
        cookies = await self.capture_cookies(page.context, config.cookie_domains)
        # Check if required cookies are present
        has_required = all(req_cookie in cookies for req_cookie in config.required_cookies)
        if has_required:
            print(f"\n[OK] Login detected for {config.name}!")
            # Get user agent
            user_agent = await page.evaluate("navigator.userAgent")
            # Try to capture bearer token if needed
            bearer_token = ""
            if config.bearer_token:
                bearer_token = await self._capture_bearer_token(page, config.api_endpoint)
            return {
                "cookie": cookies,
                "bearer_token": bearer_token,
                "user_agent": user_agent,
                "provider_type": provider_type,
            }
        await asyncio.sleep(2)
    raise TimeoutError(f"Login timeout for {config.name}. Please try again.")
async def _capture_bearer_token(self, page: Page, api_endpoint: str) -> str:
    """Try to capture bearer token from page or requests.
    Args:
        page: Browser page
        api_endpoint: API endpoint URL
    Returns:
        Bearer token if found, empty string otherwise
    """
    # Try to get from localStorage or sessionStorage
    try:
        token = await page.evaluate(
            "() => localStorage.getItem('accessToken') || sessionStorage.getItem('accessTok
en')"
        )
        if token:
            return token
    except Exception:
        pass
    # Try to get from page context
    try:
        token = await page.evaluate("() => window.__TOKEN__ || window.token || ''")
        if token:
            return token
```

```
    except Exception:
        pass
    return ""
async def authenticate_provider(
    self,
    provider_type: str,
    cdp_port: int = 9222,
    timeout: int = 300,
) -> dict:
    """Authenticate a provider via browser.
    Args:
        provider_type: Provider type to authenticate
        cdp_port: Chrome DevTools Protocol port
        timeout: Login timeout in seconds
    Returns:
        Auth data dict
    """
    import asyncio
    from .types import WEB_PROVIDERS
    # Check if already authenticated
    saved_auth = self.load_auth(provider_type)
    if saved_auth:
        print(f"[OK] Found saved authentication for {provider_type}")
        return saved_auth
    # Connect to browser
    context = await self.connect_to_browser(cdp_port)
    # Check if user is already logged in (existing cookies)
    config = WEB_PROVIDERS.get(provider_type)
    if config:
        cookies = await self.capture_cookies(context, config.cookie_domains)
        has_required = all(req_cookie in cookies for req_cookie in config.required_cookies)
        if has_required:
            print(f"[OK] Found existing session for {config.name}!")
            user_agent = (
                await context.pages[0].evaluate("navigator.userAgent") if context.pages els
e ""
            )
            auth_data = {
                "cookie": cookies,
                "bearer_token": "",
                "user_agent": user_agent,
                "provider_type": provider_type,
            }
            self.save_auth(provider_type, auth_data)
            return auth_data
    # Create new page for login
    page = await context.new_page()
    try:
        # Wait for login
        auth_data = await self.wait_for_login(page, provider_type, timeout)
        # Save authentication
        self.save_auth(provider_type, auth_data)
        print(f"\n[OK] Authentication saved for {provider_type}")
        print(f"   Auth file: {self._get_auth_file(provider_type)}")
        return auth_data
    finally:
        await page.close()
def get_auth_header(self, provider_type: str) -> dict[str, str]:
    """Get authentication headers for a provider.
    Args:
        provider_type: Provider type
    Returns:
        Headers dict with cookies and tokens
    """
    from .types import WEB_PROVIDERS
```

```
auth_data = self.load_auth(provider_type)
if not auth_data:
    raise ValueError(
        f"No authentication found for {provider_type}. "
        f"Run: openlaoke auth extended-web {provider_type}"
    )
config = WEB_PROVIDERS.get(provider_type)
if not config:
    raise ValueError(f"Unknown provider type: {provider_type}")
headers = {
    "Cookie": auth_data["cookie"],
    "User-Agent": auth_data.get("user_agent", ""),
    **config.custom_headers,
}
if auth_data.get("bearer_token"):
    headers["Authorization"] = f"Bearer {auth_data['bearer_token']}"
return headers
```

File: [58/252] openlaoke/core/extended_web/clients/__init__.py

```
"""Extended Web clients for various AI services."""
from .base import BaseWebClient
from .deepseek_client import DeepSeekWebClient
from .glm_client import GLMWebClient
from .kimi_client import KimiWebClient
from .qwen_client import QwenWebClient
__all__ = [
    "BaseWebClient",
    "DeepSeekWebClient",
    "QwenWebClient",
    "KimiWebClient",
    "GLMWebClient",
]
```

File: [59/252] openlaoke/core/extended_web/clients/base.py

```
"""Base class for Extended Web clients."""
from __future__ import annotations
from abc import ABC, abstractmethod
class BaseWebClient(ABC):
    """Base class for web-based AI service clients."""
    def __init__(
        self,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
    ) -> None:
        """Initialize the web client.
        Args:
            cookie: Browser cookies
            user_agent: User agent string
            bearer_token: Optional bearer token
        """
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
        self._initialized = False
    @property
    @abstractmethod
    def provider_type(self) -> str:
        """Return provider type identifier."""
        pass
    @property
    @abstractmethod
    def base_url(self) -> str:
        """Return base URL."""
        pass
    @property
    @abstractmethod
    def api_endpoint(self) -> str:
        """Return API endpoint URL."""
        pass
    @abstractmethod
    def build_headers(self) -> dict[str, str]:
        """Build HTTP headers for API requests."""
        pass
    async def init(self) -> None:
        """Initialize the client. Override if needed."""
        self._initialized = True
    def is_initialized(self) -> bool:
        """Check if client is initialized."""
        return self._initialized
```

File: [60/252] openlaoke/core/extended_web/clients/deepseek_client.py

```
"""DeepSeek Web API client - Full implementation reference ZeroToken."""
from __future__ import annotations
import asyncio
import base64
import hashlib
import json
import zlib
from typing import Any
import httpx
class DeepSeekPoWSolver:
    """Solve DeepSeek Proof-of-Work challenges."""
    @staticmethod
    def solve_sha256(challenge: str, salt: str, difficulty: int, max_nonce: int = 1000000) -> i
    nt:
        """Solve SHA256 PoW challenge."""
        target_bits = (
            difficulty
            if difficulty <= 1000
            else int(hashlib.log2(difficulty))
            if difficulty > 1
            else difficulty
        )
        for nonce in range(max_nonce):
            input_str = f"{salt}{challenge}{nonce}"
            hash_hex = hashlib.sha256(input_str.encode()).hexdigest()
            # Count leading zero bits
            zero_bits = 0
            for char in hash_hex:
                val = int(char, 16)
                if val == 0:
                    zero_bits += 4
                else:
                    zero_bits += val.bit_length() ^ 3
                    break
            if zero_bits >= target_bits:
                return nonce
        raise TimeoutError(f"SHA256 PoW timeout after {max_nonce} attempts")
    @staticmethod
    def solve_deepseek_hash_v1(challenge: str, prefix: str, difficulty: int) -> int:
        """Solve DeepSeekHashV1 PoW challenge using optimized approach."""
        # Simplified implementation - in production would use WASM
        for nonce in range(min(int(difficulty) * 100, 1000000)):
            input_str = f"{prefix}{challenge}{nonce}"
            hash_val = zlib.crc32(input_str.encode()) & 0xFFFFFFFF
            if hash_val % difficulty == 0:
                return nonce
        raise TimeoutError("DeepSeekHashV1 PoW timeout")
class DeepSeekWebClient:
    """DeepSeek Web API client with PoW support."""
    def __init__(
        self,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
    ) -> None:
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
```

```
    self.base_url = "https://chat.deepseek.com"
    self.session_id: str | None = None
    self._client: httpx.AsyncClient | None = None
    self.pow_solver = DeepSeekPoWSolver()
async def init(self) -> None:
    """Initialize client."""
    pass
def _build_headers(self, include_pow: bool = False, pow_response: str = "") -> dict[str, st
r]:
    """Build HTTP headers."""
    headers = {
        "Cookie": self.cookie,
        "User-Agent": self.user_agent,
        "Content-Type": "application/json",
        "Accept": "*/*",
        "Referer": f"{self.base_url}/",
        "Origin": self.base_url,
        "x-client-platform": "web",
        "x-client-version": "1.7.0",
        "x-app-version": "20241129.1",
        "x-client-locale": "zh_CN",
        "x-client-timezone-offset": "28800",
    }
    if self.bearer_token:
        headers["Authorization"] = f"Bearer {self.bearer_token}"
    if include_pow and pow_response:
        headers["x-ds-pow-response"] = pow_response
    return headers
async def _get_client(self) -> httpx.AsyncClient:
    """Get HTTP client."""
    if self._client is None:
        self._client = httpx.AsyncClient(
            timeout=httpx.Timeout(300.0, connect=30.0),
        )
    return self._client
async def close(self) -> None:
    """Close client."""
    if self._client:
        await self._client.aclose()
        self._client = None
async def create_pow_challenge(self, target_path: str) -> dict[str, Any]:
    """Create PoW challenge."""
    client = await self._get_client()
    response = await client.post(
        f"{self.base_url}/api/v0/chat/create_pow_challenge",
        headers=self._build_headers(),
        json={"target_path": target_path},
    )
    if response.status_code >= 400:
        raise httpx.HTTPStatusError(
            f"Failed to create PoW challenge: {response.status_code}",
            request=response.request,
            response=response,
        )
    data = response.json()
    if data is None:
        raise ValueError("Empty response from PoW challenge endpoint")
    # Try to extract challenge from nested structure
    biz_data = data.get("data", {}).get("biz_data", {})
    if isinstance(biz_data, dict):
        challenge_dict = biz_data.get("challenge", {})
    else:
        challenge_dict = data.get("data", {}).get("challenge", data.get("challenge", {}))
    if not challenge_dict:
        raise ValueError(f"PoW challenge missing in response. Data: {str(data)[:200]}")
```

```
    # Ensure challenge_dict is a dict
    if isinstance(challenge_dict, str):
        challenge_dict = {"challenge": challenge_dict, "algorithm": "sha256", "difficulty":
 10}
    return challenge_dict
async def solve_pow(self, challenge: dict[str, Any]) -> int:
    """Solve PoW challenge."""
    algorithm = challenge.get("algorithm", "sha256")
    challenge_str = challenge.get("challenge", "")
    salt = challenge.get("salt", "")
    difficulty = challenge.get("difficulty", 10)
    expire_at = challenge.get("expire_at", 0)
    if algorithm == "sha256":
        return self.pow_solver.solve_sha256(challenge_str, salt, difficulty)
    elif algorithm == "DeepSeekHashV1":
        prefix = f"{salt}_{expire_at}_"
        return self.pow_solver.solve_deepseek_hash_v1(challenge_str, prefix, difficulty)
    else:
        raise ValueError(f"Unsupported PoW algorithm: {algorithm}")
async def create_chat_session(self) -> str:
    """Create new chat session."""
    client = await self._get_client()
    response = await client.post(
        f"{self.base_url}/api/v0/chat_session/create",
        headers=self._build_headers(),
        json={},
    )
    if response.status_code >= 400:
        raise httpx.HTTPStatusError(
            f"Failed to create chat session: {response.status_code}",
            request=response.request,
            response=response,
        )
    data = response.json()
    if data is None:
        raise ValueError(
            f"Empty response from chat_session/create. Status: {response.status_code}, Text
: {response.text[:200]}"
        )
    def safe_get(d, *keys, default=None):
        """Safely navigate nested dicts that might have None values."""
        result = d
        for key in keys:
            if not isinstance(result, dict):
                return default
            result = result.get(key)
            if result is None:
                return default
        return result
    session_id = (
        safe_get(data, "data", "biz_data", "chat_session_id")
        or safe_get(data, "data", "biz_data", "id")
        or safe_get(data, "data", "chat_session_id")
        or safe_get(data, "data", "id")
        or safe_get(data, "chat_session_id")
        or safe_get(data, "id")
        or ""
    )
    if not session_id:
        raise ValueError(f"No session ID in response. Data: {str(data)[:200]}")
    self.session_id = session_id
    return session_id
async def chat_completions(
    self,
    messages: list[dict[str, Any]],
```

```
    model: str = "deepseek-chat",
    temperature: float = 1.0,
    max_tokens: int = 4096,
    stream: bool = False,
) -> dict[str, Any]:
    """Send chat completion request with PoW."""
    client = await self._get_client()
    # Create session if needed
    if not self.session_id:
        self.session_id = await self.create_chat_session()
    # Create and solve PoW challenge
    target_path = "/api/v0/chat/completion"
    challenge = await self.create_pow_challenge(target_path)
    nonce = await asyncio.get_event_loop().run_in_executor(
        None, lambda: self.solve_pow(challenge)
    )
    # Build PoW response
    pow_data = {
        **challenge,
        "answer": nonce,
        "target_path": target_path,
    }
    pow_response = base64.b64encode(json.dumps(pow_data).encode()).decode()
    # Build request body
    body = {
        "chat_session_id": self.session_id,
        "parent_message_id": None,
        "prompt": messages[-1]["content"] if messages else "",
        "ref_file_ids": [],
        "thinking_enabled": model != "deepseek-chat",
        "search_enabled": True,
        "preempt": False,
    }
    # Send request
    response = await client.post(
        f"{self.base_url}{target_path}",
        headers=self._build_headers(include_pow=True, pow_response=pow_response),
        json=body,
    )
    if response.status_code >= 400:
        error_text = response.text[:500]
        if response.status_code in (401, 403):
            raise httpx.HTTPStatusError(
                "DeepSeek authentication expired. Please re-login.",
                request=response.request,
                response=response,
            )
        raise httpx.HTTPStatusError(
            f"DeepSeek API error: {response.status_code}\n{error_text}",
            request=response.request,
            response=response,
        )
    data = response.json()
    if data is None:
        return {
            "content": "",
            "error": f"Empty or invalid response from DeepSeek API. Status: {response.statu
s_code}, Text: {response.text[:200]}",
        }
    # Parse response
    choices = data.get("choices", [])
    if not choices:
        return {"content": "", "error": f"No choices in response. Data: {str(data)[:200]}"}
    choice = choices[0]
    message = choice.get("message", {})
```

```
    content = message.get("content", "")
    return {
        "content": content,
        "model": data.get("model", model),
        "usage": data.get("usage", {}),
        "finish_reason": choice.get("finish_reason", "stop"),
        "session_id": self.session_id,
    }
async def chat_completions_stream(
    self,
    messages: list[dict[str, Any]],
    model: str = "deepseek-chat",
    temperature: float = 1.0,
    max_tokens: int = 4096,
) -> Any:
    """Send streaming chat completion request."""
    client = await self._get_client()
    if not self.session_id:
        self.session_id = await self.create_chat_session()
    # Create and solve PoW
    target_path = "/api/v0/chat/completion"
    challenge = await self.create_pow_challenge(target_path)
    nonce = await asyncio.get_event_loop().run_in_executor(
        None, lambda: self.solve_pow(challenge)
    )
    pow_data = {
        **challenge,
        "answer": nonce,
        "target_path": target_path,
    }
    pow_response = base64.b64encode(json.dumps(pow_data).encode()).decode()
    body = {
        "chat_session_id": self.session_id,
        "parent_message_id": None,
        "prompt": messages[-1]["content"] if messages else "",
        "ref_file_ids": [],
        "thinking_enabled": model != "deepseek-chat",
        "search_enabled": True,
        "preempt": False,
        "stream": True,
    }
    async with client.stream(
        "POST",
        f"{self.base_url}{target_path}",
        headers=self._build_headers(include_pow=True, pow_response=pow_response),
        json=body,
    ) as response:
        response.raise_for_status()
        async for line in response.aiter_lines():
            if not line:
                continue
            if line.startswith("data: "):
                data_str = line[6:]
                if data_str == "[DONE]":
                    break
                try:
                    data = json.loads(data_str)
                    choices = data.get("choices", [])
                    if choices:
                        delta = choices[0].get("delta", {})
                        content = delta.get("content", "")
                        if content:
                            yield content
                except json.JSONDecodeError:
                    continue
```

File: [61/252] openlaoke/core/extended_web/clients/glm_client.py

```
"""GLM Web (????) API client."""
from __future__ import annotations
from typing import Any
import httpx
class GLMWebClient:
    """GLM Web API client."""
    _session_cache: dict[str, str] = {}
    def __init__(
        self,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
    ) -> None:
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
        self.base_url = "https://chatglm.cn"
        self.conversation_id: str | None = None
        self._client: httpx.AsyncClient | None = None
    async def init(self) -> None:
        """Initialize client."""
        pass
    def _build_headers(self) -> dict[str, str]:
        """Build HTTP headers."""
        headers = {
            "Cookie": self.cookie,
            "User-Agent": self.user_agent,
            "Content-Type": "application/json",
            "Accept": "application/json",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }
        if self.bearer_token:
            headers["Authorization"] = f"Bearer {self.bearer_token}"
        return headers
    async def _get_client(self) -> httpx.AsyncClient:
        """Get HTTP client."""
        if self._client is None:
            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(300.0, connect=30.0),
            )
        return self._client
    async def close(self) -> None:
        """Close client."""
        if self._client:
            await self._client.aclose()
            self._client = None
    async def create_chat_session(self) -> str:
        """Create new chat session."""
        client = await self._get_client()
        try:
            response = await client.post(
                f"{self.base_url}/api/chat/session",
                headers=self._build_headers(),
                json={},
            )
            if response.status_code >= 400:
                raise httpx.HTTPStatusError(
```

```
                f"Failed to create session: {response.status_code}",
                request=response.request,
                response=response,
            )
        data = response.json()
        session_id = data.get("data", {}).get("id") or data.get("id") or ""
        if not session_id:
            raise ValueError("No session ID in response")
        self.conversation_id = session_id
        return session_id
    except Exception as e:
        raise RuntimeError(f"Failed to create GLM session: {e}")
async def chat_completions(
    self,
    messages: list[dict[str, Any]],
    model: str = "glm-4",
    temperature: float = 0.7,
    max_tokens: int = 4096,
    stream: bool = False,
) -> dict[str, Any]:
    """Send chat completion request."""
    client = await self._get_client()
    if not self.conversation_id:
        self.conversation_id = await self.create_chat_session()
    # Build request body
    body = {
        "conversation_id": self.conversation_id,
        "messages": messages,
        "model": model,
        "stream": stream,
        "temperature": temperature,
        "max_tokens": max_tokens,
    }
    try:
        response = await client.post(
            f"{self.base_url}/api/chat/completions",
            headers=self._build_headers(),
            json=body,
        )
        if response.status_code >= 400:
            error_text = response.text[:500]
            if response.status_code in (401, 403):
                raise httpx.HTTPStatusError(
                    "GLM authentication expired. Please re-login.",
                    request=response.request,
                    response=response,
                )
            raise httpx.HTTPStatusError(
                f"GLM API error: {response.status_code}\n{error_text}",
                request=response.request,
                response=response,
            )
        data = response.json()
        # Parse response
        content = ""
        if "choices" in data:
            choices = data.get("choices", [])
            if choices:
                choice = choices[0]
                message = choice.get("message", {})
                content = message.get("content", "")
        elif "data" in data:
            content = data.get("data", {}).get("content", "")
        elif "content" in data:
            content = data.get("content", "")
```

```
        return {
            "content": content,
            "model": data.get("model", model),
            "usage": data.get("usage", {}),
            "finish_reason": "stop",
            "session_id": self.conversation_id,
        }
    except httpx.HTTPStatusError:
        raise
    except Exception as e:
        raise RuntimeError(f"GLM API request failed: {e}")
def clear_session(self) -> None:
    """Clear current conversation session."""
    self.conversation_id = None
```

File: [62/252] openlaoke/core/extended_web/clients/kimi_client.py

```
"""Kimi Web (????) API client.
Reference: ZeroToken/src/zero-token/streams/kimi-web-stream.ts
"""
from __future__ import annotations
import json
from typing import Any
import httpx
from .base import BaseWebClient
class KimiWebClient(BaseWebClient):
    """Client for Kimi Web API."""
    @property
    def provider_type(self) -> str:
        return "kimi-web"
    @property
    def base_url(self) -> str:
        return "https://kimi.moonshot.cn"
    @property
    def api_endpoint(self) -> str:
        return f"{self.base_url}/api/chat"
    def build_headers(self) -> dict[str, str]:
        return {
            "Cookie": self.cookie,
            "User-Agent": self.user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }
    async def chat_completions(
        self,
        messages: list[dict[str, Any]],
        model: str = "kimi",
        temperature: float = 0.7,
        max_tokens: int = 4096,
        stream: bool = False,
    ) -> dict[str, Any]:
        """Send chat completion request."""
        async with httpx.AsyncClient(
            timeout=httpx.Timeout(300.0, connect=30.0),
            headers=self.build_headers(),
        ) as client:
            payload = {
                "model": model,
                "messages": messages,
                "temperature": temperature,
                "max_tokens": max_tokens,
                "stream": stream,
            }
            response = await client.post(self.api_endpoint, json=payload)
            if response.status_code >= 400:
                raise httpx.HTTPStatusError(
                    f"Kimi API error: {response.status_code}",
                    request=response.request,
                    response=response,
                )
            return response.json()
    async def close(self) -> None:
        """Close the client."""
        pass
    async def chat_completions_stream(
        self,
```

```
    messages: list[dict[str, Any]],
    model: str = "kimi",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> Any:
    """Send streaming chat completion request.
    Yields:
        Text deltas
    """
    async with httpx.AsyncClient(
        timeout=httpx.Timeout(300.0, connect=30.0),
        headers=self.build_headers(),
    ) as client:
        payload = {
            "model": model,
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
            "stream": True,
        }
        async with client.stream("POST", self.api_endpoint, json=payload) as response:
            response.raise_for_status()
            async for line in response.aiter_lines():
                if not line:
                    continue
                if line.startswith("data: "):
                    data_str = line[6:]
                    if data_str == "[DONE]":
                        break
                    try:
                        data = json.loads(data_str)
                        content = data.get("content", "")
                        if content:
                            yield content
                    except json.JSONDecodeError:
                        continue
```

File: [63/252] openlaoke/core/extended_web/clients/qwen_client.py

```
"""Qwen Web API client - Full implementation reference ZeroToken."""
from __future__ import annotations
import time
import uuid
from typing import Any
import httpx
class QwenWebClient:
    """Qwen Web API client using browser context."""
    # Session cache to avoid creating new sessions every time
    _session_cache: dict[str, str] = {}
    def __init__(
        self,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
    ) -> None:
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
        self.base_url = "https://chat.qwen.ai"
        self.conversation_id: str | None = None
        self._client: httpx.AsyncClient | None = None
    async def init(self) -> None:
        """Initialize client."""
        pass
    def _build_headers(self) -> dict[str, str]:
        """Build HTTP headers."""
        headers = {
            "Cookie": self.cookie,
            "User-Agent": self.user_agent,
            "Content-Type": "application/json",
            "Accept": "application/json",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }
        if self.bearer_token:
            headers["Authorization"] = f"Bearer {self.bearer_token}"
        return headers
    async def _get_client(self) -> httpx.AsyncClient:
        """Get HTTP client."""
        if self._client is None:
            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(300.0, connect=30.0),
            )
        return self._client
    async def close(self) -> None:
        """Close client."""
        if self._client:
            await self._client.aclose()
            self._client = None
    async def create_chat_session(self) -> str:
        """Create new chat session."""
        client = await self._get_client()
        try:
            response = await client.post(
                f"{self.base_url}/api/v2/chats/new",
                headers=self._build_headers(),
                json={},
```

```
        )
        if response.status_code >= 400:
            error_text = response.text[:500]
            raise httpx.HTTPStatusError(
                f"Failed to create chat session: {response.status_code}\n{error_text}",
                request=response.request,
                response=response,
            )
        data = response.json()
        if data is None:
            raise ValueError(
                f"Empty response from Qwen API. Response text: {response.text[:200]}"
            )
        chat_id = data.get("data", {}).get("id") or data.get("chat_id") or data.get("id") o
r ""
        if not chat_id:
            raise ValueError(f"No chat ID in response. Response data: {str(data)[:200]}")
        self.conversation_id = chat_id
        return chat_id
    except Exception as e:
        raise RuntimeError(f"Failed to create Qwen chat session: {e}")
async def chat_completions(
    self,
    messages: list[dict[str, Any]],
    model: str = "qwen3.5-plus",
    temperature: float = 0.7,
    max_tokens: int = 4096,
    stream: bool = False,
) -> dict[str, Any]:
    """Send chat completion request."""
    client = await self._get_client()
    # Create or get conversation ID
    if not self.conversation_id:
        self.conversation_id = await self.create_chat_session()
    # Build message with proper format
    fid = str(uuid.uuid4())
    timestamp = int(time.time())
    qwen_message = {
        "fid": fid,
        "parentId": None,
        "childrenIds": [],
        "role": "user",
        "content": messages[-1]["content"] if messages else "",
        "user_action": "chat",
        "files": [],
        "timestamp": timestamp,
        "models": [model],
        "chat_type": "t2t",
        "feature_config": {
            "thinking_enabled": True,
            "output_schema": "phase",
        },
    }
    # Build request body
    body = {
        "stream": stream,
        "version": "2.1",
        "incremental_output": True,
        "chat_id": self.conversation_id,
        "chat_mode": "normal",
        "model": model,
        "parent_id": None,
        "messages": [qwen_message],
    }
    try:
```

```
        response = await client.post(
            f"{self.base_url}/api/v2/chat/completions?chat_id={self.conversation_id}",
            headers=self._build_headers(),
            json=body,
        )
        if response.status_code >= 400:
            error_text = response.text[:500]
            if response.status_code in (401, 403):
                raise httpx.HTTPStatusError(
                    "Qwen authentication expired. Please re-login.",
                    request=response.request,
                    response=response,
                )
            raise httpx.HTTPStatusError(
                f"Qwen API error: {response.status_code}\n{error_text}",
                request=response.request,
                response=response,
            )
        # Parse response
        if stream:
            # For streaming, return the response for further processing
            return {
                "content": "",
                "stream": True,
                "response": response,
                "session_id": self.conversation_id,
            }
        else:
            data = response.json()
            # Extract content from response
            content = ""
            if "choices" in data:
                choices = data.get("choices", [])
                if choices:
                    choice = choices[0]
                    message = choice.get("message", {})
                    content = message.get("content", "")
            elif "data" in data:
                content = data.get("data", {}).get("content", "")
            elif "content" in data:
                content = data.get("content", "")
            return {
                "content": content,
                "model": data.get("model", model),
                "usage": data.get("usage", {}),
                "finish_reason": "stop",
                "session_id": self.conversation_id,
            }
    except httpx.HTTPStatusError:
        raise
    except Exception as e:
        raise RuntimeError(f"Qwen API request failed: {e}")
async def chat_completions_stream(
    self,
    messages: list[dict[str, Any]],
    model: str = "qwen3.5-plus",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> Any:
    """Send streaming chat completion request."""
    result = await self.chat_completions(
        messages=messages,
        model=model,
        temperature=temperature,
        max_tokens=max_tokens,
```

```
        stream=True,
    )
    if result.get("stream"):
        response = result.get("response")
        if response:
            async for line in response.aiter_lines():
                if not line:
                    continue
                if line.startswith("data: "):
                    data_str = line[6:]
                    if data_str == "[DONE]":
                        break
                    try:
                        data = __import__("json").loads(data_str)
                        # Qwen stream format may vary
                        if isinstance(data, dict):
                            content = data.get("content", "")
                            if content:
                                yield content
                            elif "choices" in data:
                                choices = data.get("choices", [])
                                if choices:
                                    delta = choices[0].get("delta", {})
                                    content = delta.get("content", "")
                                    if content:
                                        yield content
                    except __import__("json").JSONDecodeError:
                        # Try to extract content directly
                        if line.strip():
                            yield line.strip()
def clear_session(self) -> None:
    """Clear current conversation session."""
    self.conversation_id = None
```

File: [64/252] openlaoke/core/extended_web/config_integration.py

```
"""Integration of auto cookie extraction into config wizard."""
from __future__ import annotations
from rich.console import Console
from .auto_cookie_extractor import extract_cookie_with_playwright
from .firefox_cookie_reader import extract_firefox_cookies
from .browser_auth import BrowserAuthManager
from .types import WEB_PROVIDERS
console = Console()
async def extract_cookie_in_wizard(service: str) -> bool:
    """Extract cookie for a service within the config wizard.
    Args:
        service: Service type (e.g., 'deepseek-chat')
    Returns:
        True if successful
    """
    import asyncio
    config = WEB_PROVIDERS.get(service)
    if not config:
        console.print(f"[red]Unknown service: {service}[/red]")
        return False
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print(f"[bold cyan]  {config.name} Authentication[/bold cyan]")
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print()
    console.print(f"[dim]Login URL: [cyan]{config.login_url}[/cyan][/dim]")
    console.print()
    # First, try to read cookies directly from Firefox (if user is already logged in)
    console.print("[bold]Step 1: Checking Firefox for existing cookies...[/bold]")
    auth_data = await extract_firefox_cookies(service)
    if auth_data and auth_data.get("cookie"):
        auth_manager = BrowserAuthManager()
        auth_manager.save_auth(service, auth_data)
        console.print()
        console.print("[green][OK] Authentication successful! (from Firefox)[/green]")
        console.print(f"  Cookie saved to: ~/.openlaoke/extended_web/{service}.json")
        return True
    # If Firefox didn't have cookies, launch browser for manual login
    console.print()
    console.print("[bold]Step 2: Launching browser for login...[/bold]")
    console.print()
    console.print("  [dim]1. A browser window will open automatically[/dim]")
    console.print(
        f"  [dim]2. Navigate to: [cyan underline]{config.login_url}[/cyan underline][/dim]"
    )
    console.print("  [dim]3. Login with your account[/dim]")
    console.print("  [dim]4. Wait for the chat interface to load[/dim]")
    console.print("  [dim]5. Cookie will be extracted automatically[/dim]")
    console.print()
    # Run async extraction (browser launches automatically)
    auth_data = await extract_cookie_with_playwright(service, timeout=300)
    if auth_data:
        console.print()
        console.print("[green][OK] Authentication successful![/green]")
        console.print(f"  Cookie saved to: ~/.openlaoke/extended_web/{service}.json")
        return True
    else:
        console.print()
        console.print("[yellow]? Authentication failed[/yellow]")
        console.print("[dim]You can try again later or use manual extraction[/dim]")
        return False
```

File: [65/252] openlaoke/core/extended_web/deepseek_client.py

```
"""DeepSeek Web API client using browser authentication.
This module provides API access to DeepSeek's web service using
captured browser cookies and authentication.
"""
from __future__ import annotations
import json
from dataclasses import dataclass
from typing import Any
import httpx
@dataclass
class DeepSeekMessage:
    """A single message in a chat."""
    role: str  # "user", "assistant", or "system"
    content: str
@dataclass
class DeepSeekResponse:
    """Response from DeepSeek API."""
    content: str
    model: str
    usage: dict[str, int]
    finish_reason: str
class DeepSeekWebClient:
    """Client for DeepSeek Web API using browser authentication."""
    def __init__(
        self,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
    ) -> None:
        """Initialize the DeepSeek Web client.
        Args:
            cookie: Browser cookies string
            user_agent: User agent string
            bearer_token: Optional bearer token
        """
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
        self.base_url = "https://chat.deepseek.com"
        self.api_endpoint = f"{self.base_url}/api/v1/chat/completions"
        self._client: httpx.AsyncClient | None = None
    def _get_headers(self) -> dict[str, str]:
        """Get HTTP headers for API requests."""
        headers = {
            "Cookie": self.cookie,
            "User-Agent": self.user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
            "x-client-platform": "web",
            "x-client-version": "1.7.0",
        }
        if self.bearer_token:
            headers["Authorization"] = f"Bearer {self.bearer_token}"
        return headers
    async def _get_client(self) -> httpx.AsyncClient:
        """Get or create HTTP client."""
```

```
    if self._client is None:
        self._client = httpx.AsyncClient(
            timeout=httpx.Timeout(300.0, connect=30.0),
            headers=self._get_headers(),
        )
    return self._client
async def close(self) -> None:
    """Close the HTTP client."""
    if self._client:
        await self._client.aclose()
        self._client = None
async def chat_completions(
    self,
    messages: list[DeepSeekMessage],
    model: str = "deepseek-chat",
    temperature: float = 1.0,
    max_tokens: int = 4096,
    stream: bool = False,
) -> DeepSeekResponse:
    """Send a chat completion request.
    Args:
        messages: List of chat messages
        model: Model to use
        temperature: Temperature for sampling
        max_tokens: Maximum tokens to generate
        stream: Whether to stream the response
    Returns:
        DeepSeekResponse with the completion
    """
    client = await self._get_client()
    # Convert messages to API format
    api_messages = [{"role": msg.role, "content": msg.content} for msg in messages]
    payload = {
        "model": model,
        "messages": api_messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": stream,
    }
    response = await client.post(
        self.api_endpoint,
        json=payload,
    )
    if response.status_code >= 400:
        error_detail = response.text[:500]
        raise httpx.HTTPStatusError(
            f"DeepSeek API error: {response.status_code} {response.reason_phrase}\n{error_d
etail}",
            request=response.request,
            response=response,
        )
    data = response.json()
    # Parse response
    choices = data.get("choices", [])
    if not choices:
        raise ValueError("DeepSeek API returned no choices")
    choice = choices[0]
    message = choice.get("message", {})
    content = message.get("content", "")
    usage = data.get("usage", {})
    return DeepSeekResponse(
        content=content,
        model=data.get("model", model),
        usage=usage,
        finish_reason=choice.get("finish_reason", "stop"),
```

```
    )
async def chat_completions_stream(
    self,
    messages: list[DeepSeekMessage],
    model: str = "deepseek-chat",
    temperature: float = 1.0,
    max_tokens: int = 4096,
) -> Any:
    """Send a streaming chat completion request.
    Args:
        messages: List of chat messages
        model: Model to use
        temperature: Temperature for sampling
        max_tokens: Maximum tokens to generate
    Yields:
        Text deltas from the stream
    """
    client = await self._get_client()
    api_messages = [{"role": msg.role, "content": msg.content} for msg in messages]
    payload = {
        "model": model,
        "messages": api_messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": True,
    }
    async with client.stream(
        "POST",
        self.api_endpoint,
        json=payload,
    ) as response:
        response.raise_for_status()
        async for line in response.aiter_lines():
            if not line:
                continue
            if line.startswith("data: "):
                data_str = line[6:]
                if data_str == "[DONE]":
                    break
                try:
                    data = json.loads(data_str)
                    choices = data.get("choices", [])
                    if choices:
                        delta = choices[0].get("delta", {})
                        content = delta.get("content", "")
                        if content:
                            yield content
                except json.JSONDecodeError:
                    continue
```

File: [66/252] openlaoke/core/extended_web/firefox_auth.py

```
"""Firefox browser authentication support.
Firefox uses WebDriver protocol instead of Chrome DevTools Protocol.
This module provides Firefox-specific authentication support.
"""
from __future__ import annotations
import os
import tempfile
import time
from pathlib import Path
from rich.console import Console
_console = Console()
def get_firefox_path() -> str:
    """Get Firefox browser path."""
    import platform
    system = platform.system()
    if system == "Darwin":  # macOS
        paths = [
            "/Applications/Firefox.app/Contents/MacOS/firefox",
            "/Applications/Firefox Developer Edition.app/Contents/MacOS/firefox",
            "/Applications/Firefox Nightly.app/Contents/MacOS/firefox",
        ]
    elif system == "Windows":
        paths = [
            r"C:\Program Files\Mozilla Firefox\firefox.exe",
            r"C:\Program Files (x86)\Mozilla Firefox\firefox.exe",
        ]
    else:  # Linux
        paths = [
            "/usr/bin/firefox",
            "/usr/bin/firefox-esr",
            "/snap/bin/firefox",
        ]
    for path in paths:
        if os.path.exists(path):
            return path
    return "firefox"
def launch_firefox_with_marionette(
    url: str = "",
    port: int = 2828,
    headless: bool = False,
) -> tuple[int, any]:
    """Launch Firefox with Marionette debugging enabled.
    Args:
        url: URL to open
        port: Marionette port (default: 2828)
        headless: Run in headless mode
    Returns:
        Tuple of (port, firefox_process)
    """
    import subprocess
    firefox_path = get_firefox_path()
    user_data_dir = tempfile.mkdtemp(prefix="openlaoke-firefox-")
    cmd = [
        firefox_path,
        "-marionette",
        "-marionette-port",
        str(port),
        "-profile",
        user_data_dir,
        "-no-remote",
        "-new-instance",
```

```
    ]
    if headless:
        cmd.append("-headless")
    if url:
        cmd.extend(["-url", url])
    # Launch Firefox
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
    # Wait for Firefox to start
    time.sleep(3)
    if process.poll() is not None:
        raise RuntimeError("Failed to launch Firefox")
    return port, process
async def authenticate_with_firefox(
    provider_type: str,
    login_url: str,
    timeout: int = 300,
) -> dict:
    """Authenticate a provider using Firefox.
    This function:
    1. Launches Firefox with Marionette debugging
    2. Opens the login URL
    3. Waits for user to complete login
    4. Captures authentication cookies via Selenium
    5. Returns auth data
    Args:
        provider_type: Provider type (e.g., 'deepseek-chat')
        login_url: URL to open for login
        timeout: Login timeout in seconds
    Returns:
        Auth data dict with cookies and tokens
    """
    from openlaoke.core.extended_web import BrowserAuthManager
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    config = WEB_PROVIDERS.get(provider_type)
    if not config:
        raise ValueError(f"Unknown provider type: {provider_type}")
    auth_manager = BrowserAuthManager()
    # Check if already authenticated
    saved_auth = auth_manager.load_auth(provider_type)
    if saved_auth:
        _console.print(f"[green][OK] Found existing authentication for {provider_type}[/green]"
    )
        return saved_auth
    _console.print(f"\n? Launching Firefox for {config.name} authentication...")
    _console.print(f"   Login URL: {login_url}")
    _console.print()
    try:
        from selenium import webdriver
        from selenium.webdriver.firefox.options import Options
        from selenium.webdriver.firefox.service import Service
        from selenium.webdriver.support.ui import WebDriverWait
        from selenium.webdriver.common.by import By
        from selenium.webdriver.support import expected_conditions as EC
    except ImportError:
        _console.print("[red]Error:[/red] Selenium not installed")
        _console.print("\n[dim]Install with: pip install selenium[/dim]")
        raise
    # Setup Firefox options
    options = Options()
    options.add_argument("-profile")
    options.add_argument(tempfile.mkdtemp(prefix="openlaoke-firefox-"))
```

```
# Don't use headless mode for authentication
# options.add_argument("-headless")
_console.print("[dim]Starting Firefox...[/dim]")
try:
    # Launch Firefox
    service = Service()
    driver = webdriver.Firefox(service=service, options=options)
    _console.print("[green][OK] Firefox launched[/green]")
    _console.print()
    _console.print("? [bold]Please complete the login in the Firefox window.[/bold]")
    _console.print(f"   If not already open, navigate to: [cyan]{login_url}[/cyan]")
    _console.print()
    _console.print("[dim]Waiting for authentication...[/dim]")
    _console.print()
    # Open login URL
    driver.get(login_url)
    # Wait for login
    import time as time_module
    start_time = time_module.time()
    cookie_found = False
    while time_module.time() - start_time < timeout:
        try:
            # Check for cookies
            cookies = driver.get_cookies()
            # Convert to cookie string
            cookie_parts = []
            for cookie in cookies:
                cookie_parts.append(f"{cookie['name']}={cookie['value']}")
            cookie_string = "; ".join(cookie_parts)
            # Check if required cookies are present
            has_required = all(
                req_cookie in cookie_string for req_cookie in config.required_cookies
            )
            if has_required and len(cookie_string) > 50:
                _console.print(f"\n[green][OK] Login detected for {config.name}![/green]")
                # Get user agent
                user_agent = driver.execute_script("return navigator.userAgent;")
                # Try to capture bearer token
                bearer_token = ""
                if config.bearer_token:
                    try:
                        bearer_token = driver.execute_script(
                            "return localStorage.getItem('accessToken') || '';"
                        )
                    except Exception:
                        pass
                auth_data = {
                    "provider_type": provider_type,
                    "cookie": cookie_string,
                    "bearer_token": bearer_token,
                    "user_agent": user_agent,
                    "saved_at": time_module.strftime("%Y-%m-%dT%H:%M:%S"),
                    "browser": "firefox",
                }
                # Save authentication
                auth_manager.save_auth(provider_type, auth_data)
                _console.print(f"[green][OK] Authentication saved![/green]")
                _console.print(f"  Cookie length: {len(cookie_string)}")
                _console.print(f"  Auth file: {auth_manager._get_auth_file(provider_type)}"
)
                cookie_found = True
                break
        except Exception as e:
            # Ignore transient errors
            pass
```

```
        time_module.sleep(2)
    if not cookie_found:
        driver.quit()
        raise TimeoutError(f"Login timeout after {timeout} seconds")
    # Close Firefox
    driver.quit()
    return auth_data
except Exception as e:
    _console.print(f"\n[red][X] Error:[/red] {e}")
    _console.print("\n[dim]Make sure Firefox is installed.[/dim]")
    _console.print("[dim]Install with: brew install --cask firefox (macOS)[/dim]")
    raise
```

File: [67/252] openlaoke/core/extended_web/firefox_cookie_reader.py

```
"""Extract cookies directly from Firefox browser database.
This module reads cookies from Firefox's SQLite database, eliminating the need
for users to log in again in a separate browser window.
"""
from __future__ import annotations
import os
import shutil
import sqlite3
import sys
import tempfile
from pathlib import Path
from rich.console import Console
console = Console()
def get_firefox_profiles_dir() -> Path | None:
    """Get Firefox profiles directory based on OS."""
    if sys.platform == "darwin":
        base = Path.home() / "Library" / "Application Support" / "Firefox" / "Profiles"
    elif sys.platform == "win32":
        base = Path.home() / "AppData" / "Roaming" / "Mozilla" / "Firefox" / "Profiles"
    else:
        base = Path.home() / ".mozilla" / "firefox" / "Profiles"
    if not base.exists():
        return None
    return base
def get_firefox_profile() -> Path | None:
    """Get the default Firefox profile directory with the most cookies."""
    profiles_dir = get_firefox_profiles_dir()
    if not profiles_dir:
        return None
    profiles = list(profiles_dir.iterdir())
    if not profiles:
        return None
    # Prefer default-release profile, then any default profile
    best_profile = None
    best_count = 0
    for profile in profiles:
        if not profile.is_dir():
            continue
        cookie_db = profile / "cookies.sqlite"
        if not cookie_db.exists():
            continue
        # Try to count cookies by copying to temp file (avoid lock)
        try:
            with tempfile.NamedTemporaryFile(suffix=".sqlite", delete=False) as tmp:
                tmp_path = tmp.name
            shutil.copy2(str(cookie_db), tmp_path)
            conn = sqlite3.connect(tmp_path)
            cursor = conn.cursor()
            cursor.execute("SELECT COUNT(*) FROM moz_cookies")
            count = cursor.fetchone()[0]
            conn.close()
            os.unlink(tmp_path)
            if count > best_count:
                best_count = count
                best_profile = profile
        except Exception:
            pass
    return best_profile
def extract_cookies_from_firefox(domains: list[str]) -> dict[str, str]:
    """Extract cookies for domains from Firefox.
    Args:
```

```
        domains: List of domains to extract cookies for
    Returns:
        Dict of cookie names to values
    """
    profile = get_firefox_profile()
    if not profile:
        console.print("[yellow]No Firefox profile found[/yellow]")
        return {}
    cookie_db = profile / "cookies.sqlite"
    if not cookie_db.exists():
        console.print(f"[yellow]No cookies.sqlite found in {profile}[/yellow]")
        return {}
    cookies = {}
    # Copy to temp file to avoid database lock
    try:
        with tempfile.NamedTemporaryFile(suffix=".sqlite", delete=False) as tmp:
            tmp_path = tmp.name
        shutil.copy2(str(cookie_db), tmp_path)
        conn = sqlite3.connect(tmp_path)
        cursor = conn.cursor()
        for domain in domains:
            # Remove leading dot if present
            clean_domain = domain.lstrip(".")
            cursor.execute(
                """
                SELECT name, value
                FROM moz_cookies
                WHERE host LIKE ?
                """,
                (f"%{clean_domain}%",),
            )
            for name, value in cursor.fetchall():
                if value:  # Only include cookies with values
                    cookies[name] = value
        conn.close()
        os.unlink(tmp_path)
    except sqlite3.Error as e:
        console.print(f"[red]Error reading Firefox cookies: {e}[/red]")
        return {}
    except Exception as e:
        console.print(f"[red]Error: {e}[/red]")
        return {}
    return cookies
async def extract_firefox_cookies(service: str) -> dict | None:
    """Extract cookies from Firefox for a given service.
    Args:
        service: Service type (e.g., 'deepseek-chat', 'qwen-web', 'glm-web')
    Returns:
        Auth data dict or None
    """
    from .types import WEB_PROVIDERS
    config = WEB_PROVIDERS.get(service)
    if not config:
        console.print(f"[red]Unknown service: {service}[/red]")
        return None
    console.print(f"[cyan]Reading cookies from Firefox for {config.name}...[/cyan]")
    domains = config.cookie_domains
    console.print(f"[dim]Looking for cookies from: {domains}[/dim]")
    console.print(f"[dim]Required cookies: {config.required_cookies}[/dim]")
    cookies = extract_cookies_from_firefox(domains)
    if not cookies:
        console.print("[yellow]No cookies found in Firefox for this domain[/yellow]")
        console.print("[dim]Make sure you are logged into the website in Firefox[/dim]")
        return None
    console.print(f"[green]Found {len(cookies)} cookies[/green]")
```

```
    # Build cookie string with ALL cookies
    cookie_string = "; ".join(f"{name}={value}" for name, value in cookies.items())
    auth_data = {
        "provider_type": service,
        "cookie": cookie_string,
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTM
    L, like Gecko) Chrome/120.0.0.0 Safari/537.36",
        "saved_at": "2026-04-08T00:00:00",
        "browser": "firefox-direct",
    }
    return auth_data
if __name__ == "__main__":
    import asyncio
    if len(sys.argv) < 2:
        print("Usage: python firefox_cookie_reader.py <service>")
        print("Services: deepseek-chat, qwen-web, glm-web, kimi-web")
        sys.exit(1)
    service = sys.argv[1]
    async def main():
        result = await extract_firefox_cookies(service)
        if result:
            print("\n[green]Success![/green]")
            print(f"Cookie length: {len(result['cookie'])}")
        else:
            print("\n[red]Failed to extract cookies[/red]")
    asyncio.run(main())
```

File: [68/252] openlaoke/core/extended_web/providers_api.py

```
"""Extended Web API implementations for various providers.
This module provides API clients for different web-based AI services
using browser authentication cookies.
"""
from __future__ import annotations
import json
from abc import ABC, abstractmethod
from typing import Any
import httpx
from rich.console import Console
console = Console()
class BaseWebProvider(ABC):
    """Base class for web-based AI providers."""

    @property
    @abstractmethod
    def provider_type(self) -> str:
        """Return provider type identifier."""
        pass

    @property
    @abstractmethod
    def base_url(self) -> str:
        """Return base URL."""
        pass

    @property
    @abstractmethod
    def api_endpoint(self) -> str:
        """Return API endpoint URL."""
        pass

    @abstractmethod
    def build_headers(self, cookie: str, user_agent: str) -> dict[str, str]:
        """Build HTTP headers for API requests."""
        pass

    @abstractmethod
    def build_payload(
        self,
        messages: list[dict[str, str]],
        model: str,
        temperature: float,
        max_tokens: int,
    ) -> dict[str, Any]:
        """Build request payload."""
        pass

    @abstractmethod
    def parse_response(self, data: dict[str, Any]) -> dict[str, Any]:
        """Parse API response."""
        pass
class DeepSeekProvider(BaseWebProvider):
    """DeepSeek Web API provider."""

    @property
    def provider_type(self) -> str:
        return "deepseek-chat"

    @property
    def base_url(self) -> str:
```

```
        return "https://chat.deepseek.com"

    @property
    def api_endpoint(self) -> str:
        return f"{self.base_url}/api/v1/chat/completions"

    def build_headers(self, cookie: str, user_agent: str) -> dict[str, str]:
        return {
            "Cookie": cookie,
            "User-Agent": user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
            "x-client-platform": "web",
            "x-client-version": "1.7.0",
        }

    def build_payload(
        self,
        messages: list[dict[str, str]],
        model: str,
        temperature: float,
        max_tokens: int,
    ) -> dict[str, Any]:
        return {
            "model": model or "deepseek-chat",
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
        }

    def parse_response(self, data: dict[str, Any]) -> dict[str, Any]:
        choices = data.get("choices", [])
        if not choices:
            return {"content": "", "error": "No choices"}

        choice = choices[0]
        message = choice.get("message", {})
        content = message.get("content", "")

        return {
            "content": content,
            "model": data.get("model", "deepseek-chat"),
            "usage": data.get("usage", {}),
            "finish_reason": choice.get("finish_reason", "stop"),
        }
class QwenProvider(BaseWebProvider):
    """Qwen Web (????) API provider."""

    @property
    def provider_type(self) -> str:
        return "qwen-web"

    @property
    def base_url(self) -> str:
        return "https://tongyi.aliyun.com"

    @property
    def api_endpoint(self) -> str:
        return f"{self.base_url}/api/chat"

    def build_headers(self, cookie: str, user_agent: str) -> dict[str, str]:
        return {
            "Cookie": cookie,
```

```
            "User-Agent": user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }

    def build_payload(
        self,
        messages: list[dict[str, str]],
        model: str,
        temperature: float,
        max_tokens: int,
    ) -> dict[str, Any]:
        return {
            "model": model or "qwen-turbo",
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
        }

    def parse_response(self, data: dict[str, Any]) -> dict[str, Any]:
        # Qwen response format
        content = data.get("content", "")
        if not content:
            content = data.get("data", {}).get("content", "")

        return {
            "content": content,
            "model": data.get("model", "qwen-turbo"),
            "usage": data.get("usage", {}),
            "finish_reason": data.get("finish_reason", "stop"),
        }
class KimiProvider(BaseWebProvider):
    """Kimi Web (????) API provider."""

    @property
    def provider_type(self) -> str:
        return "kimi-web"

    @property
    def base_url(self) -> str:
        return "https://kimi.moonshot.cn"

    @property
    def api_endpoint(self) -> str:
        return f"{self.base_url}/api/chat"

    def build_headers(self, cookie: str, user_agent: str) -> dict[str, str]:
        return {
            "Cookie": cookie,
            "User-Agent": user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }

    def build_payload(
        self,
        messages: list[dict[str, str]],
        model: str,
        temperature: float,
        max_tokens: int,
    ) -> dict[str, Any]:
```

```
        return {
            "model": model or "kimi",
            "messages": messages,
            "temperature": temperature,
            "max_tokens": max_tokens,
        }

    def parse_response(self, data: dict[str, Any]) -> dict[str, Any]:
        content = data.get("content", "")
        return {
            "content": content,
            "model": data.get("model", "kimi"),
            "usage": data.get("usage", {}),
            "finish_reason": data.get("finish_reason", "stop"),
        }
# Provider registry
PROVIDERS: dict[str, BaseWebProvider] = {
    "deepseek-chat": DeepSeekProvider(),
    "deepseek-coder": DeepSeekProvider(),
    "qwen-web": QwenProvider(),
    "kimi-web": KimiProvider(),
}
class ExtendedWebClient:
    """Unified client for Extended Web providers."""

    def __init__(self, provider_type: str) -> None:
        """Initialize the client.

        Args:
            provider_type: Provider type (e.g., 'deepseek-chat')
        """
        self.provider_type = provider_type

        # Get provider implementation
        provider_impl = PROVIDERS.get(provider_type)
        if not provider_impl:
            # Use DeepSeek as default for unknown providers
            provider_impl = DeepSeekProvider()

        self.provider = provider_impl
        self._client: httpx.AsyncClient | None = None

    async def _get_client(self) -> httpx.AsyncClient:
        """Get or create HTTP client with auth."""
        if self._client is None:
            from openlaoke.core.extended_web import BrowserAuthManager

            auth_manager = BrowserAuthManager()
            auth_data = auth_manager.load_auth(self.provider_type)

            if not auth_data:
                raise ValueError(
                    f"No authentication found for {self.provider_type}. "
                    f"Run: python3 quick_auth.py {self.provider_type}"
                )

            cookie = auth_data.get("cookie", "")
            user_agent = auth_data.get("user_agent", "")

            headers = self.provider.build_headers(cookie, user_agent)

            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(300.0, connect=30.0),
                headers=headers,
            )
```

```
    return self._client

async def close(self) -> None:
    """Close the client."""
    if self._client:
        await self._client.aclose()
        self._client = None

async def chat(
    self,
    messages: list[dict[str, str]],
    model: str = "",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> dict[str, Any]:
    """Send chat request.

    Args:
        messages: Chat messages
        model: Model to use
        temperature: Temperature
        max_tokens: Max tokens

    Returns:
        Response dict
    """
    client = await self._get_client()

    payload = self.provider.build_payload(
        messages, model, temperature, max_tokens
    )

    response = await client.post(
        self.provider.api_endpoint,
        json=payload,
    )

    if response.status_code >= 400:
        error_msg = response.text[:500]
        raise httpx.HTTPStatusError(
            f"{self.provider_type} API error: {response.status_code}\n{error_msg}",
            request=response.request,
            response=response,
        )

    data = response.json()
    return self.provider.parse_response(data)
```

File: [69/252] openlaoke/core/extended_web/simple_browser_auth.py

```
"""Simple browser authentication using system default browser.
This module opens the system's default browser directly without Selenium,
making it more reliable and faster.
"""
from __future__ import annotations
import os
import platform
import subprocess
import sys
import tempfile
import time
from pathlib import Path
from rich.console import Console
console = Console()
def open_system_browser(url: str) -> bool:
    """Open URL in system's default browser.
    Args:
        url: URL to open
    Returns:
        True if successful
    """
    system = platform.system()
    try:
        if system == "Darwin":  # macOS
            subprocess.run(["open", url], check=True)
        elif system == "Windows":
            subprocess.run(["start", url], shell=True, check=True)
        else:  # Linux
            subprocess.run(["xdg-open", url], check=True)
        return True
    except Exception as e:
        console.print(f"[red]Failed to open browser: {e}[/red]")
        return False
def authenticate_simple(
    provider_type: str,
    login_url: str,
    required_cookies: list[str],
) -> dict:
    """Simple authentication by opening browser and waiting for user.
    This function:
    1. Opens the system's default browser
    2. User logs in manually
    3. User copies cookies manually (for now)
    Args:
        provider_type: Provider type
        login_url: Login URL
        required_cookies: List of required cookie names
    Returns:
        Auth data dict
    """
    from openlaoke.core.extended_web import BrowserAuthManager
    auth_manager = BrowserAuthManager()
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print("[bold cyan]  Browser Authentication[/bold cyan]")
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print()
    console.print(f"[bold]Provider:[/bold] {provider_type}")
    console.print(f"[bold]Login URL:[/bold] [cyan underline]{login_url}[/cyan underline]")
    console.print()
    # Open browser
```

```
    console.print("[bold]Step 1: Opening browser...[/bold]")
    if open_system_browser(login_url):
        console.print("[green][OK] Browser opened![/green]")
        console.print("[dim]  Look for the browser window on your screen.[/dim]")
        console.print(
            f"[dim]  If not visible, manually open: [cyan underline]{login_url}[/cyan underline
    ][/dim]"
        )
    else:
        console.print("[yellow]? Failed to open browser automatically.[/yellow]")
        console.print("[dim]  Please open it manually and go to the URL above.[/dim]")
    console.print()
    console.print("[bold]Step 2: Login to your account[/bold]")
    console.print(f"[bold red]IMPORTANT:[/bold red]")
    console.print(f"  [dim]1. Browser will open automatically[/dim]")
    console.print(f"  [dim]2. Go to: [cyan underline]{login_url}[/cyan underline][/dim]")
    console.print(f"  [dim]3. Login with your account[/dim]")
    console.print(f"  [dim]4. Come back here and press Enter[/dim]")
    console.print()
    # Wait for user to login
    console.print("[bold]Step 3: Waiting for login...[/bold]")
    console.print()
    console.print("  [dim]Please login in the browser, then come back here.[/dim]")
    console.print("  [dim]Press Enter after you've logged in...[/dim]")
    console.print()
    input("  Press Enter when you've logged in: ")
    console.print()
    console.print("[bold green][OK] Login complete![/bold green]")
    console.print()
    # For now, create a placeholder auth
    # In a future version, we could extract cookies via browser extension
    auth_data = {
        "provider_type": provider_type,
        "cookie": "placeholder_need_manual_extraction",
        "bearer_token": "",
        "user_agent": "",
        "saved_at": time.strftime("%Y-%m-%dT%H:%M:%S"),
        "browser": "system_default",
        "manual": True,
    }
    console.print("[yellow]? Note: Manual cookie extraction not yet implemented.[/yellow]")
    console.print("[dim]  For now, just keep the browser window open.[/dim]")
    console.print("[dim]  Future version will auto-extract cookies.[/dim]")
    console.print()
    return auth_data
if __name__ == "__main__":
    # Test
    authenticate_simple(
        "deepseek-chat",
        "https://chat.deepseek.com",
        ["d_id", "ds_session_id"],
    )
```

File: [70/252] openlaoke/core/extended_web/stream_factory.py

```
"""Stream factory functions for Extended Web providers.
Reference: ZeroToken/src/zero-token/streams/web-stream-factories.ts
"""
from __future__ import annotations
from typing import Any, AsyncGenerator
from .clients import DeepSeekWebClient, KimiWebClient, QwenWebClient
async def create_deepseek_stream(
    cookie: str,
    user_agent: str,
    messages: list[dict[str, Any]],
    model: str = "deepseek-chat",
    temperature: float = 1.0,
    max_tokens: int = 4096,
) -> AsyncGenerator[str, None]:
    """Create DeepSeek streaming response.

    Args:
        cookie: Browser cookie
        user_agent: User agent
        messages: Chat messages
        model: Model to use
        temperature: Temperature
        max_tokens: Max tokens

    Yields:
        Text deltas
    """
    client = DeepSeekWebClient(cookie=cookie, user_agent=user_agent)
    await client.init()

    async for delta in client.chat_completions_stream(
        messages=messages,
        model=model,
        temperature=temperature,
        max_tokens=max_tokens,
    ):
        yield delta

    await client.close()
async def create_qwen_stream(
    cookie: str,
    user_agent: str,
    messages: list[dict[str, Any]],
    model: str = "qwen-turbo",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> AsyncGenerator[str, None]:
    """Create Qwen streaming response."""
    client = QwenWebClient(cookie=cookie, user_agent=user_agent)
    await client.init()

    async for delta in client.chat_completions_stream(
        messages=messages,
        model=model,
        temperature=temperature,
        max_tokens=max_tokens,
    ):
        yield delta

    await client.close()
async def create_kimi_stream(
```

```
    cookie: str,
    user_agent: str,
    messages: list[dict[str, Any]],
    model: str = "kimi",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> AsyncGenerator[str, None]:
    """Create Kimi streaming response."""
    client = KimiWebClient(cookie=cookie, user_agent=user_agent)
    await client.init()

    async for delta in client.chat_completions_stream(
        messages=messages,
        model=model,
        temperature=temperature,
        max_tokens=max_tokens,
    ):
        yield delta

    await client.close()
# Stream factory registry
STREAM_FACTORIES = {
    "deepseek-chat": create_deepseek_stream,
    "deepseek-coder": create_deepseek_stream,
    "qwen-web": create_qwen_stream,
    "kimi-web": create_kimi_stream,
}
def get_stream_factory(provider_type: str):
    """Get stream factory function for a provider type.

    Args:
        provider_type: Provider type (e.g., 'deepseek-chat')

    Returns:
        Stream factory function or None
    """
    return STREAM_FACTORIES.get(provider_type)
def list_stream_providers() -> list[str]:
    """List all providers with stream support.

    Returns:
        List of provider types
    """
    return list(STREAM_FACTORIES.keys())
```

File: [71/252] openlaoke/core/extended_web/types.py

```
"""Type definitions for extended web providers."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Literal
WebProviderType = Literal[
    "deepseek-chat",
    "deepseek-coder",
    "claude-web",
    "chatgpt-web",
    "qwen-web",
    "kimi-web",
    "gemini-web",
    "grok-web",
    "doubao-web",
    "glm-web",
]
@dataclass
class WebProviderConfig:
    """Configuration for a web-based AI provider."""
    provider_type: WebProviderType
    name: str
    base_url: str
    login_url: str
    api_endpoint: str
    cookie_domains: list[str]
    required_cookies: list[str]
    bearer_token: bool = False
    custom_headers: dict[str, str] = field(default_factory=dict)
# Provider configurations
WEB_PROVIDERS: dict[WebProviderType, WebProviderConfig] = {
    "deepseek-chat": WebProviderConfig(
        provider_type="deepseek-chat",
        name="DeepSeek Chat",
        base_url="https://chat.deepseek.com",
        login_url="https://chat.deepseek.com/login",
        api_endpoint="https://chat.deepseek.com/api/v1/chat/completions",
        cookie_domains=["chat.deepseek.com", "deepseek.com"],
        required_cookies=["ds_session_id"],
        bearer_token=False,
        custom_headers={
            "Referer": "https://chat.deepseek.com/",
            "Origin": "https://chat.deepseek.com",
        },
    ),
    "deepseek-coder": WebProviderConfig(
        provider_type="deepseek-coder",
        name="DeepSeek Coder",
        base_url="https://chat.deepseek.com",
        login_url="https://chat.deepseek.com/login",
        api_endpoint="https://chat.deepseek.com/api/v1/chat/completions",
        cookie_domains=["chat.deepseek.com", "deepseek.com"],
        required_cookies=["ds_session_id"],
        bearer_token=False,
        custom_headers={
            "Referer": "https://chat.deepseek.com/",
            "Origin": "https://chat.deepseek.com",
        },
    ),
    "claude-web": WebProviderConfig(
        provider_type="claude-web",
        name="Claude Web",
```

```
    base_url="https://claude.ai",
    login_url="https://claude.ai/login",
    api_endpoint="https://claude.ai/api/organizations",
    cookie_domains=["claude.ai"],
    required_cookies=["session_id"],
    bearer_token=False,
    custom_headers={
        "Referer": "https://claude.ai/",
        "Origin": "https://claude.ai",
    },
),
"chatgpt-web": WebProviderConfig(
    provider_type="chatgpt-web",
    name="ChatGPT Web",
    base_url="https://chat.openai.com",
    login_url="https://chat.openai.com/auth/login",
    api_endpoint="https://chat.openai.com/backend-api/conversation",
    cookie_domains=["openai.com"],
    required_cookies=["__Secure-next-auth.session-token"],
    bearer_token=True,
    custom_headers={
        "Referer": "https://chat.openai.com/",
        "Origin": "https://chat.openai.com",
    },
),
"qwen-web": WebProviderConfig(
    provider_type="qwen-web",
    name="Qwen Web (????)",
    base_url="https://chat.qwen.ai",
    login_url="https://chat.qwen.ai",
    api_endpoint="https://chat.qwen.ai/api/v2/chat/completions",
    cookie_domains=["qianwen.com", "qwen.ai", "tongyi.aliyun.com"],
    required_cookies=["qwen_session"],
    bearer_token=False,
    custom_headers={
        "Referer": "https://chat.qwen.ai/",
        "Origin": "https://chat.qwen.ai",
    },
),
"kimi-web": WebProviderConfig(
    provider_type="kimi-web",
    name="Kimi Web (????)",
    base_url="https://kimi.moonshot.cn",
    login_url="https://kimi.moonshot.cn/login",
    api_endpoint="https://kimi.moonshot.cn/api/chat",
    cookie_domains=["moonshot.cn"],
    required_cookies=["kimi_token"],
    bearer_token=False,
    custom_headers={
        "Referer": "https://kimi.moonshot.cn/",
        "Origin": "https://kimi.moonshot.cn",
    },
),
"gemini-web": WebProviderConfig(
    provider_type="gemini-web",
    name="Gemini Web",
    base_url="https://gemini.google.com",
    login_url="https://accounts.google.com/login",
    api_endpoint="https://gemini.google.com/_b/AIzaSy...",
    cookie_domains=["google.com", "gemini.google.com"],
    required_cookies=["__Secure-1PSID", "__Secure-3PSID"],
    bearer_token=False,
    custom_headers={
        "Referer": "https://gemini.google.com/",
        "Origin": "https://gemini.google.com",
```

```
        },
    ),
    "grok-web": WebProviderConfig(
        provider_type="grok-web",
        name="Grok Web",
        base_url="https://grok.x.ai",
        login_url="https://grok.x.ai/login",
        api_endpoint="https://grok.x.ai/api/chat",
        cookie_domains=["x.ai", "grok.x.ai"],
        required_cookies=["auth_token"],
        bearer_token=True,
        custom_headers={
            "Referer": "https://grok.x.ai/",
            "Origin": "https://grok.x.ai",
        },
    ),
    "doubao-web": WebProviderConfig(
        provider_type="doubao-web",
        name="Doubao Web (??)",
        base_url="https://doubao.com",
        login_url="https://doubao.com/login",
        api_endpoint="https://doubao.com/api/chat",
        cookie_domains=["doubao.com", "byteimg.com"],
        required_cookies=["doubao_token"],
        bearer_token=False,
        custom_headers={
            "Referer": "https://doubao.com/",
            "Origin": "https://doubao.com",
        },
    ),
    "glm-web": WebProviderConfig(
        provider_type="glm-web",
        name="GLM Web (????)",
        base_url="https://chatglm.cn",
        login_url="https://chatglm.cn/login",
        api_endpoint="https://chatglm.cn/api/chat",
        cookie_domains=["chatglm.cn"],
        required_cookies=["glmsession"],
        bearer_token=False,
        custom_headers={
            "Referer": "https://chatglm.cn/",
            "Origin": "https://chatglm.cn",
        },
    ),
}
@dataclass
class AuthResult:
    """Result of browser-based authentication."""
    provider_type: WebProviderType
    cookie: str
    bearer_token: str = ""
    user_agent: str = ""
    additional_headers: dict[str, str] = field(default_factory=dict)
    expires_at: int | None = None
    def to_dict(self) -> dict:
        """Convert to dictionary for storage."""
        return {
            "provider_type": self.provider_type,
            "cookie": self.cookie,
            "bearer_token": self.bearer_token,
            "user_agent": self.user_agent,
            "additional_headers": self.additional_headers,
            "expires_at": self.expires_at,
        }
    @classmethod
```

```
def from_dict(cls, data: dict) -> AuthResult:
    """Create from dictionary."""
    return cls(
        provider_type=data["provider_type"],
        cookie=data["cookie"],
        bearer_token=data.get("bearer_token", ""),
        user_agent=data.get("user_agent", ""),
        additional_headers=data.get("additional_headers", {}),
        expires_at=data.get("expires_at"),
    )
```

File: [72/252] openlaoke/core/extended_web/web_client.py

```
"""Generic Web AI Service Client.
This module provides a unified client interface for various
web-based AI services using browser authentication.
"""
from __future__ import annotations
import json
from typing import Any
import httpx
class WebServiceClient:
    """Generic client for web-based AI services."""
    def __init__(
        self,
        provider_type: str,
        cookie: str,
        user_agent: str = "",
        bearer_token: str = "",
        base_url: str = "",
        api_endpoint: str = "",
        custom_headers: dict[str, str] | None = None,
    ) -> None:
        """Initialize the web service client.
        Args:
            provider_type: Provider type (e.g., 'deepseek-chat', 'claude-web')
            cookie: Browser cookies string
            user_agent: User agent string
            bearer_token: Optional bearer token
            base_url: Base URL of the service
            api_endpoint: API endpoint URL
            custom_headers: Additional custom headers
        """
        self.provider_type = provider_type
        self.cookie = cookie
        self.user_agent = user_agent or (
            "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
        )
        self.bearer_token = bearer_token
        self.base_url = base_url
        self.api_endpoint = api_endpoint
        self.custom_headers = custom_headers or {}
        self._client: httpx.AsyncClient | None = None
    def _get_headers(self) -> dict[str, str]:
        """Get HTTP headers for API requests."""
        headers = {
            "Cookie": self.cookie,
            "User-Agent": self.user_agent,
            "Content-Type": "application/json",
            "Accept": "*/*",
            "Referer": f"{self.base_url}/",
            "Origin": self.base_url,
        }
        if self.bearer_token:
            headers["Authorization"] = f"Bearer {self.bearer_token}"
        headers.update(self.custom_headers)
        return headers
    async def _get_client(self) -> httpx.AsyncClient:
        """Get or create HTTP client."""
        if self._client is None:
            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(300.0, connect=30.0),
                headers=self._get_headers(),
```

```
        )
    return self._client
async def close(self) -> None:
    """Close the HTTP client."""
    if self._client:
        await self._client.aclose()
        self._client = None
async def chat(
    self,
    messages: list[dict[str, str]],
    model: str = "",
    temperature: float = 0.7,
    max_tokens: int = 4096,
    stream: bool = False,
) -> dict[str, Any]:
    """Send a chat request.
    Args:
        messages: List of chat messages
        model: Model to use
        temperature: Temperature for sampling
        max_tokens: Maximum tokens to generate
        stream: Whether to stream the response
    Returns:
        Response dict with content and metadata
    """
    client = await self._get_client()
    # Build request payload based on provider type
    payload = self._build_payload(messages, model, temperature, max_tokens, stream)
    response = await client.post(
        self.api_endpoint,
        json=payload,
    )
    if response.status_code >= 400:
        error_detail = response.text[:500]
        raise httpx.HTTPStatusError(
            f"{self.provider_type} API error: {response.status_code} {response.reason_phras
e}\n{error_detail}",
            request=response.request,
            response=response,
        )
    # Parse response based on provider type
    return self._parse_response(response.json())
def _build_payload(
    self,
    messages: list[dict[str, str]],
    model: str,
    temperature: float,
    max_tokens: int,
    stream: bool,
) -> dict[str, Any]:
    """Build request payload based on provider type."""
    # Default OpenAI-compatible format
    return {
        "model": model or self.provider_type,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": stream,
    }
def _parse_response(self, data: dict[str, Any]) -> dict[str, Any]:
    """Parse response based on provider type."""
    # Default OpenAI-compatible format
    choices = data.get("choices", [])
    if not choices:
        return {"content": "", "error": "No choices in response"}
```

```
    choice = choices[0]
    message = choice.get("message", {})
    content = message.get("content", "")
    return {
        "content": content,
        "model": data.get("model", self.provider_type),
        "usage": data.get("usage", {}),
        "finish_reason": choice.get("finish_reason", "stop"),
    }
async def chat_stream(
    self,
    messages: list[dict[str, str]],
    model: str = "",
    temperature: float = 0.7,
    max_tokens: int = 4096,
) -> Any:
    """Send a streaming chat request.
    Yields:
        Text deltas from the stream
    """
    client = await self._get_client()
    payload = self._build_payload(messages, model, temperature, max_tokens, stream=True)
    async with client.stream(
        "POST",
        self.api_endpoint,
        json=payload,
    ) as response:
        response.raise_for_status()
        async for line in response.aiter_lines():
            if not line:
                continue
            if line.startswith("data: "):
                data_str = line[6:]
                if data_str == "[DONE]":
                    break
                try:
                    data = json.loads(data_str)
                    content = self._parse_stream_chunk(data)
                    if content:
                        yield content
                except json.JSONDecodeError:
                    continue
def _parse_stream_chunk(self, data: dict[str, Any]) -> str:
    """Parse a streaming chunk based on provider type."""
    # Default OpenAI-compatible format
    choices = data.get("choices", [])
    if choices:
        delta = choices[0].get("delta", {})
        return delta.get("content", "")
    return ""
```

File: [73/252] openlaoke/core/hooks.py

```
"""Hook system for extensibility."""
from __future__ import annotations
from openlaoke.types.hooks import HookContext, HookHandler, HookRegistration, HookResult, HookT
    ype
class HookManager:
    """Manages the registration and execution of hooks."""
    def __init__(self) -> None:
        self._hooks: dict[HookType, list[HookRegistration]] = {}
    def register(
        self,
        hook_type: HookType,
        handler: HookHandler,
        priority: int = 0,
        name: str = "",
    ) -> None:
        reg = HookRegistration(
            hook_type=hook_type,
            handler=handler,
            priority=priority,
            name=name or handler.__name__,
        )
        if hook_type not in self._hooks:
            self._hooks[hook_type] = []
        self._hooks[hook_type].append(reg)
        self._hooks[hook_type].sort(key=lambda r: r.priority)
    def unregister(self, hook_type: HookType, name: str) -> None:
        if hook_type in self._hooks:
            self._hooks[hook_type] = [r for r in self._hooks[hook_type] if r.name != name]
    async def execute(self, ctx: HookContext) -> HookResult:
        handlers = self._hooks.get(ctx.hook_type, [])
        for reg in handlers:
            try:
                result = await reg.handler(ctx)
                if result.should_abort:
                    return result
            except Exception as e:
                return HookResult(
                    success=False,
                    error=str(e),
                    should_abort=False,
                )
        return HookResult(success=True)
    def get_registered_hooks(self) -> dict[str, list[str]]:
        result = {}
        for hook_type, regs in self._hooks.items():
            result[hook_type.value] = [r.name for r in regs]
        return result
```

File: [74/252] openlaoke/core/hybrid_model_manager.py

```
"""Advanced model management with CPU/GPU hybrid loading.
This module provides intelligent model loading strategies:
1. Small models (planner/validator) use CPU - always available, no VRAM
2. Large models (executor) use GPU - faster generation
3. Automatic model selection based on available resources
4. Batch operations to minimize model switching
Memory Strategy:
- gemma3:1b (1.4 GB) -> CPU (always loaded, ~100MB RAM)
- gemma4:e4b (12 GB) -> GPU (loaded on-demand)
- Total VRAM: ~12 GB (vs 13.4 GB with both on GPU)
"""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
import httpx
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class DeviceType(StrEnum):
    """Device types for model execution."""
    CPU = "cpu"
    GPU = "gpu"
    AUTO = "auto"
@dataclass
class ModelConfig:
    """Configuration for a model."""
    name: str
    tier: str  # "planner", "executor", "validator"
    device: DeviceType
    keep_alive: str
    max_tokens: int
    size_gb: float = 0.0
    is_loaded: bool = False
    last_used: float = 0.0
@dataclass
class ModelPoolStatus:
    """Status of the model pool."""
    cpu_models: list[ModelConfig] = field(default_factory=list)
    gpu_models: list[ModelConfig] = field(default_factory=list)
    gpu_total_gb: float = 0.0
    gpu_used_gb: float = 0.0
    gpu_free_gb: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "cpu_models": [{"name": m.name, "loaded": m.is_loaded} for m in self.cpu_models],
            "gpu_models": [
                {"name": m.name, "loaded": m.is_loaded, "size_gb": m.size_gb}
                for m in self.gpu_models
            ],
            "gpu_memory": {
                "total_gb": self.gpu_total_gb,
                "used_gb": self.gpu_used_gb,
                "free_gb": self.gpu_free_gb,
            },
        }
class HybridModelManager:
    """Manage models with CPU/GPU hybrid strategy."""
    OLLAMA_API = "http://localhost:11434"
    DEFAULT_CONFIGS = {
        "planner": ModelConfig(
            name="gemma3:1b",
```

```
        tier="planner",
        device=DeviceType.CPU,
        keep_alive="24h",
        max_tokens=500,
        size_gb=1.4,
    ),
    "executor": ModelConfig(
        name="gemma4:e4b",
        tier="executor",
        device=DeviceType.GPU,
        keep_alive="10m",
        max_tokens=1000,
        size_gb=12.0,
    ),
    "validator": ModelConfig(
        name="gemma3:1b",
        tier="validator",
        device=DeviceType.CPU,
        keep_alive="24h",
        max_tokens=300,
        size_gb=1.4,
    ),
}
def __init__(self, app_state: AppState | None = None) -> None:
    self.app_state = app_state
    self.models: dict[str, ModelConfig] = {}
    self._api_client: httpx.AsyncClient | None = None
    self._loaded_models_cache: dict[str, bool] = {}
async def _get_client(self) -> httpx.AsyncClient:
    """Get or create HTTP client."""
    if self._api_client is None:
        self._api_client = httpx.AsyncClient(timeout=60.0)
    return self._api_client
async def initialize(self) -> dict[str, Any]:
    """Initialize model pool with optimal configuration."""
    client = await self._get_client()
    loaded_models = await self._get_loaded_models(client)
    self.models["planner"] = self.DEFAULT_CONFIGS["planner"]
    self.models["validator"] = self.DEFAULT_CONFIGS["validator"]
    self.models["executor"] = self.DEFAULT_CONFIGS["executor"]
    results = {
        "planner": False,
        "validator": False,
        "executor": False,
    }
    for tier, config in self.models.items():
        is_loaded = any(m["name"] == config.name for m in loaded_models)
        if is_loaded:
            config.is_loaded = True
            results[tier] = True
        else:
            success = await self._load_model(client, config)
            config.is_loaded = success
            results[tier] = success
    return {
        "success": all(results.values()),
        "results": results,
        "strategy": "CPU/GPU Hybrid",
    }
async def _load_model(self, client: httpx.AsyncClient, config: ModelConfig) -> bool:
    """Load a model with specified device preference."""
    try:
        load_options = {
            "model": config.name,
            "prompt": "",
```

```
            "keep_alive": config.keep_alive,
            "stream": False,
        }
        if config.device == DeviceType.CPU:
            load_options["options"] = {"num_gpu": 0}
        response = await client.post(
            f"{self.OLLAMA_API}/api/generate",
            json=load_options,
        )
        if response.status_code == 200:
            self._loaded_models_cache[config.name] = True
            return True
    except Exception:
        pass
    return False
async def _get_loaded_models(self, client: httpx.AsyncClient) -> list[dict[str, Any]]:
    """Get currently loaded models from Ollama."""
    try:
        response = await client.get(f"{self.OLLAMA_API}/api/ps")
        if response.status_code == 200:
            data = response.json()
            return data.get("models", [])
    except Exception:
        pass
    return []
async def get_status(self) -> ModelPoolStatus:
    """Get current model pool status."""
    client = await self._get_client()
    loaded_models = await self._get_loaded_models(client)
    status = ModelPoolStatus()
    for _, config in self.models.items():
        config.is_loaded = any(m["name"] == config.name for m in loaded_models)
        if config.device == DeviceType.CPU:
            status.cpu_models.append(config)
        else:
            status.gpu_models.append(config)
            if config.is_loaded:
                status.gpu_used_gb += config.size_gb
    status.gpu_total_gb = 24.0
    status.gpu_free_gb = status.gpu_total_gb - status.gpu_used_gb
    return status
async def ensure_model_ready(self, tier: str) -> bool:
    """Ensure a model is loaded and ready."""
    if tier not in self.models:
        return False
    config = self.models[tier]
    if config.is_loaded:
        return True
    client = await self._get_client()
    success = await self._load_model(client, config)
    config.is_loaded = success
    return success
async def batch_call_planner(self, tasks: list[str]) -> list[str]:
    """Batch call planner model for multiple tasks (minimize switching)."""
    if not await self.ensure_model_ready("planner"):
        return ["Error: Planner model not ready"] * len(tasks)
    client = await self._get_client()
    config = self.models["planner"]
    results = []
    for task in tasks:
        try:
            response = await client.post(
                f"{self.OLLAMA_API}/api/generate",
                json={
                    "model": config.name,
```

```
                        "prompt": task,
                        "stream": False,
                        "keep_alive": config.keep_alive,
                        "options": {"num_gpu": 0} if config.device == DeviceType.CPU else {},
                    },
                )
                if response.status_code == 200:
                    data = response.json()
                    results.append(data.get("response", ""))
                else:
                    results.append(f"Error: HTTP {response.status_code}")
            except Exception as e:
                results.append(f"Error: {str(e)}")
        return results
    async def call_executor(self, prompt: str) -> str:
        """Call executor model for code generation."""
        if not await self.ensure_model_ready("executor"):
            return "Error: Executor model not ready"
        client = await self._get_client()
        config = self.models["executor"]
        try:
            response = await client.post(
                f"{self.OLLAMA_API}/api/generate",
                json={
                    "model": config.name,
                    "prompt": prompt,
                    "stream": False,
                    "keep_alive": config.keep_alive,
                    "options": {"num_ctx": 8192},
                },
            )
            if response.status_code == 200:
                data = response.json()
                return data.get("response", "")
            else:
                return f"Error: HTTP {response.status_code}"
        except Exception as e:
            return f"Error: {str(e)}"
    async def cleanup(self) -> None:
        """Cleanup resources."""
        if self._api_client:
            await self._api_client.aclose()
            self._api_client = None
def create_hybrid_manager(app_state: AppState | None = None) -> HybridModelManager:
    """Create a hybrid model manager."""
    return HybridModelManager(app_state)
```

File: [75/252] openlaoke/core/hyperauto/__init__.py

```
"""HyperAuto - Automated project management and initialization."""
from __future__ import annotations
from openlaoke.core.hyperauto.agent import (
    AnalysisResult,
    HyperAutoAgent,
    SkillGenerationResult,
)
from openlaoke.core.hyperauto.code_search import (
    CodeSearchEngine,
    CodeSnippet,
    SearchSource,
)
from openlaoke.core.hyperauto.decision_engine import (
    Context,
    DecisionCategory,
    DecisionEngine,
    DecisionOption,
    DecisionRecord,
    RiskAssessment,
    RiskLevel,
)
from openlaoke.core.hyperauto.learning import (
    EnhancedStrategy,
    Experience,
    ExperienceType,
    Knowledge,
    KnowledgeCategory,
    LearningSession,
    LearningSystem,
    Pattern,
    PatternType,
    Strategy,
    StrategyType,
)
from openlaoke.core.hyperauto.project_initializer import (
    InitResult,
    ProjectAnalysis,
    ProjectInitializer,
    ProjectType,
)
from openlaoke.core.hyperauto.reflection import (
    Analysis,
    CorrectedResult,
    Correction,
    CorrectionStatus,
    Error,
    ErrorCategory,
    ErrorSeverity,
    Improvement,
    ImprovementPriority,
    QualityDimension,
    QualityScore,
    ReflectionReport,
    ReflectionSystem,
    Review,
    ReviewType,
)
from openlaoke.core.hyperauto.skill_generator import (
    GenerationResult,
    SkillDesign,
    SkillGenerator,
```

```
    SkillParameter,
    SkillRequirement,
    SkillTrigger,
)
from openlaoke.core.hyperauto.test_runner import (
    AutoTestRunner,
    CoverageReport,
    Diagnosis,
    TestAnalysis,
    TestCycleResult,
    TestFailure,
    TestFramework,
    TestInfo,
    TestResults,
    TestSeverity,
    TestStatus,
)
from openlaoke.core.hyperauto.validator import (
    Artifact,
    Language,
    PerformanceResult,
    Requirements,
    ResultValidator,
    SecurityReport,
    ValidationCategory,
    ValidationIssue,
    ValidationLevel,
    ValidationReport,
    ValidationResult,
    ValidatorQualityScore,
)
from openlaoke.core.hyperauto.workflow import (
    DependencyEdge,
    DependencyGraph,
    ExecutionPlan,
    ExecutionResult,
    ExecutionStep,
    ProgressInfo,
    RecoveryAction,
    RecoveryStrategy,
    TaskTree,
    WorkflowOrchestrator,
    WorkflowPhase,
    WorkflowResult,
)
__all__ = [
    "AnalysisResult",
    "HyperAutoAgent",
    "SkillGenerationResult",
    "CodeSearchEngine",
    "CodeSnippet",
    "SearchSource",
    "Context",
    "DecisionCategory",
    "DecisionEngine",
    "DecisionOption",
    "DecisionRecord",
    "RiskAssessment",
    "RiskLevel",
    "EnhancedStrategy",
    "Experience",
    "ExperienceType",
    "Knowledge",
    "KnowledgeCategory",
    "LearningSession",
```

```
"LearningSystem",
"Pattern",
"PatternType",
"Strategy",
"StrategyType",
"InitResult",
"ProjectAnalysis",
"ProjectInitializer",
"ProjectType",
"Analysis",
"Correction",
"CorrectionStatus",
"CorrectedResult",
"Error",
"ErrorCategory",
"ErrorSeverity",
"Improvement",
"ImprovementPriority",
"QualityDimension",
"QualityScore",
"ReflectionReport",
"ReflectionSystem",
"Review",
"ReviewType",
"SkillGenerator",
"SkillRequirement",
"SkillDesign",
"SkillParameter",
"SkillTrigger",
"GenerationResult",
"AutoTestRunner",
"CoverageReport",
"Diagnosis",
"TestAnalysis",
"TestCycleResult",
"TestFailure",
"TestFramework",
"TestInfo",
"TestResults",
"TestSeverity",
"TestStatus",
"Artifact",
"Language",
"PerformanceResult",
"Requirements",
"ResultValidator",
"SecurityReport",
"ValidationCategory",
"ValidationIssue",
"ValidationLevel",
"ValidationReport",
"ValidationResult",
"ValidatorQualityScore",
"DependencyEdge",
"DependencyGraph",
"ExecutionPlan",
"ExecutionResult",
"ExecutionStep",
"ProgressInfo",
"RecoveryAction",
"RecoveryStrategy",
"TaskTree",
"WorkflowOrchestrator",
"WorkflowPhase",
"WorkflowResult",
```

]

File: [76/252] openlaoke/core/hyperauto/agent.py

```
"""HyperAuto Agent - Fully autonomous coding assistant."""
from __future__ import annotations
import asyncio
import time
from collections.abc import Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from openlaoke.core.hyperauto.config import HyperAutoConfig
from openlaoke.core.hyperauto.executor import ToolExecutor
from openlaoke.core.hyperauto.types import (
    Decision,
    DecisionType,
    HyperAutoState,
    Reflection,
    SubTask,
    SubTaskStatus,
    WorkflowContext,
)
from openlaoke.core.hyperauto.verifier import VerificationResult
from openlaoke.core.skill_system import Skill, SkillRegistry, get_skill_registry
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class AnalysisResult:
    """Result of task analysis."""
    task_type: str
    description: str
    sub_tasks: list[dict[str, Any]]
    required_skills: list[str]
    required_tools: list[str]
    estimated_complexity: str
    confidence: float
@dataclass
class SkillGenerationResult:
    """Result of skill generation."""
    skill_name: str
    skill_content: str
    skill_description: str
    success: bool
    error: str | None = None
ProgressCallback = Callable[["WorkflowContext"], None]
class HyperAutoAgent:
    """HyperAuto Agent for fully autonomous task execution.
    Features:
    - Automatic task analysis and decomposition
    - Automatic skill generation when needed
    - Automatic project initialization
    - Automatic code search for reference
    - Automatic workflow orchestration
    - Intelligent decision making
    - Self-reflection and learning
    """
    def __init__(
        self,
        app_state: AppState,
        config: HyperAutoConfig | None = None,
        on_progress: ProgressCallback | None = None,
    ) -> None:
        self.app_state = app_state
        self.config = config or HyperAutoConfig()
        self.context = WorkflowContext()
```

```
    self._skill_registry: SkillRegistry | None = None
    self._execution_history: list[dict[str, Any]] = []
    self._learned_patterns: dict[str, list[str]] = {}
    self._on_progress = on_progress  # <- ?????
    self._api_client = None
    self._tool_executor: ToolExecutor | None = None
@property
def skill_registry(self) -> SkillRegistry:
    if self._skill_registry is None:
        self._skill_registry = get_skill_registry()
    return self._skill_registry
async def _get_api_client(self):
    """Get or create the API client."""
    if self._api_client is None:
        from openlaoke.core.multi_provider_api import MultiProviderClient
        if self.app_state.multi_provider_config:
            self._api_client = MultiProviderClient(self.app_state.multi_provider_config)
        else:
            from openlaoke.types.providers import MultiProviderConfig
            config = MultiProviderConfig.defaults()
            self._api_client = MultiProviderClient(config)
    return self._api_client
def _get_tool_executor(self) -> ToolExecutor:
    """Get or create tool executor."""
    if self._tool_executor is None:
        self._tool_executor = ToolExecutor(self.app_state)
    return self._tool_executor
async def run(self, request: str) -> dict[str, Any]:
    """Execute a request in HyperAuto mode with automatic verification.
    This method runs completely autonomously after the initial input:
    - No human interaction required
    - Automatically runs verification loops
    - Repeats until task is perfectly completed
    """
    self.context = WorkflowContext(
        original_request=request,
        current_state=HyperAutoState.ANALYZING,
        start_time=time.time(),
    )
    files_created: list[str] = []
    files_modified: list[str] = []
    try:
        while self.context.iteration < self.config.max_iterations:
            self.context.iteration += 1
            if self.context.current_state == HyperAutoState.ANALYZING:
                await self._analyze_request()
                self.context.current_state = HyperAutoState.PLANNING
            elif self.context.current_state == HyperAutoState.PLANNING:
                await self._create_plan()
                self.context.current_state = HyperAutoState.EXECUTING
            elif self.context.current_state == HyperAutoState.EXECUTING:
                result = await self._execute_plan()
                if result:
                    if self.config.auto_run_tests:
                        self.context.current_state = HyperAutoState.VERIFYING
                    elif self.config.reflection_enabled:
                        self.context.current_state = HyperAutoState.REFLECTING
                    else:
                        self.context.current_state = HyperAutoState.COMPLETED
                else:
                    break
            elif self.context.current_state == HyperAutoState.VERIFYING:
                verification_result = await self._verify_completion(
                    files_created=files_created,
                    files_modified=files_modified,
```

```
                )
                if verification_result.is_perfect:
                    self.context.current_state = HyperAutoState.COMPLETED
                elif verification_result.pass_rate >= 0.8:
                    if self.config.reflection_enabled:
                        self.context.current_state = HyperAutoState.REFLECTING
                    else:
                        self.context.current_state = HyperAutoState.COMPLETED
                else:
                    self.context.current_state = HyperAutoState.RETRYING
            elif self.context.current_state == HyperAutoState.RETRYING:
                retry_success = await self._handle_retry()
                if retry_success:
                    self.context.current_state = HyperAutoState.EXECUTING
                else:
                    self.context.current_state = HyperAutoState.REFLECTING
            elif self.context.current_state == HyperAutoState.REFLECTING:
                if self.config.reflection_enabled:
                    await self._reflect()
                if self.config.learning_enabled:
                    self.context.current_state = HyperAutoState.LEARNING
                else:
                    self.context.current_state = HyperAutoState.COMPLETED
            elif self.context.current_state == HyperAutoState.LEARNING:
                await self._learn()
                self.context.current_state = HyperAutoState.COMPLETED
            elif (
                self.context.current_state == HyperAutoState.COMPLETED
                or self.context.current_state == HyperAutoState.FAILED
            ):
                break
        self.context.end_time = time.time()
        return self._get_result()
    except Exception as e:
        self.context.current_state = HyperAutoState.FAILED
        self.context.end_time = time.time()
        return {
            "success": False,
            "error": str(e),
            "context": self.context.to_dict(),
        }
async def _verify_completion(
    self,
    files_created: list[str] | None = None,
    files_modified: list[str] | None = None,
) -> VerificationResult:
    """Verify that the task is perfectly completed.
    This runs the autonomous verification loop that tests
    repeatedly until completion is verified as perfect.
    """
    from openlaoke.core.hyperauto.verifier import (
        HyperAutoVerifier,
        VerificationConfig,
    )
    config = VerificationConfig(
        max_iterations=10,
        min_pass_rate=1.0,
        enable_syntax_check=True,
        enable_type_check=True,
        enable_unit_tests=True,
        enable_lint=True,
        auto_fix_enabled=True,
        retry_on_failure=True,
    )
    verifier = HyperAutoVerifier(self.app_state, config)
```

```
        result = await verifier.verify_until_perfect(
            task_description=self.context.original_request,
            files_created=files_created or [],
            files_modified=files_modified or [],
        )
        self.context.metadata["verification"] = result.to_dict()
        return result
    async def _handle_retry(self) -> bool:
        """Handle retry after failed verification.
        Returns True if retry should proceed, False otherwise.
        """
        failed_tasks = self.context.get_failed_tasks()
        if not failed_tasks:
            return False
        for task in failed_tasks:
            task.status = SubTaskStatus.PENDING
            task.error = None
            task.result = None
        return True
    async def _analyze_request(self) -> AnalysisResult:
        """Analyze the request and identify sub-tasks using AI."""
        request = self.context.original_request
        try:
            api = await self._get_api_client()
            system_prompt = "You are a task analyzer. Respond only with valid JSON."
            user_prompt = f"""Analyze this task and break it down:
Task: {request}
Respond with JSON containing:
- task_type: "creation"/"bugfix"/"refactor"/"testing"/"documentation"/"general"
- sub_tasks: array of {{name, description, type, priority, dependencies}}
- required_skills: array of skill names
- required_tools: array of tool names
- estimated_complexity: "low"/"medium"/"high"
JSON only, no other text."""
            if self.app_state.multi_provider_config:
                model = self.app_state.multi_provider_config.get_active_model()
            else:
                model = "gemma3:1b"
            response_msg, token_usage, _ = await api.send_message(
                system_prompt=system_prompt,
                messages=[{"role": "user", "content": user_prompt}],
                model=model,
                max_tokens=1000,
            )
            if response_msg and response_msg.content:
                import json
                content = response_msg.content
                if hasattr(content, "text"):
                    content = content.text
                try:
                    data = json.loads(content)
                    task_type = data.get("task_type", "general")
                    sub_tasks = data.get("sub_tasks", [])
                    required_skills = data.get("required_skills", [])
                    required_tools = data.get("required_tools", [])
                    complexity = data.get("estimated_complexity", "medium")
                except (json.JSONDecodeError, KeyError):
                    task_type = self._classify_task(request)
                    sub_tasks = self._decompose_task(request, task_type)
                    required_skills = self._identify_required_skills(request, task_type)
                    required_tools = self._identify_required_tools(request, task_type)
                    complexity = "medium"
                if token_usage:
                    self.context.total_tokens += token_usage.total_tokens
            else:
```

```
            raise ValueError("No response from API")
    except Exception:
        task_type = self._classify_task(request)
        sub_tasks = self._decompose_task(request, task_type)
        required_skills = self._identify_required_skills(request, task_type)
        required_tools = self._identify_required_tools(request, task_type)
        complexity = "medium"
    analysis = AnalysisResult(
        task_type=task_type,
        description=request,
        sub_tasks=sub_tasks,
        required_skills=required_skills,
        required_tools=required_tools,
        estimated_complexity=complexity,
        confidence=0.85,
    )
    for i, sub_task_def in enumerate(sub_tasks):
        sub_task = SubTask(
            name=sub_task_def.get("name", f"task_{i}"),
            description=sub_task_def.get("description", ""),
            priority=sub_task_def.get("priority", i),
            dependencies=sub_task_def.get("dependencies", []),
            metadata=sub_task_def,
        )
        self.context.sub_tasks.append(sub_task)
    return analysis
async def _create_plan(self) -> list[SubTask]:
    """Create an execution plan from sub-tasks."""
    tasks = self.context.sub_tasks
    sorted_tasks = self._topological_sort(tasks)
    for i, task in enumerate(sorted_tasks):
        task.priority = i
    self.context.sub_tasks = sorted_tasks
    return sorted_tasks
async def _execute_plan(self) -> bool:
    """Execute the plan by running sub-tasks."""
    pending = self.context.get_pending_tasks()
    if not pending:
        return True
    running_tasks = []
    for task in pending[: self.config.max_parallel_tasks]:
        if self._can_execute_task(task):
            running_tasks.append(asyncio.create_task(self._execute_sub_task(task)))
    if running_tasks:
        await asyncio.gather(*running_tasks, return_exceptions=True)
    all_done = all(
        t.status in (SubTaskStatus.COMPLETED, SubTaskStatus.FAILED, SubTaskStatus.SKIPPED)
        for t in self.context.sub_tasks
    )
    return all_done
async def _execute_sub_task(self, task: SubTask) -> Any:
    """Execute a single sub-task using AI with tool calling."""
    task.status = SubTaskStatus.RUNNING
    try:
        if self.config.dry_run:
            task.result = {"dry_run": True, "simulated": True}
            task.status = SubTaskStatus.COMPLETED
            return task.result
        from openlaoke.core.hyperauto.task_executor import execute_task_with_tools
        result = await execute_task_with_tools(
            app_state=self.app_state,
            task=task,
            original_request=self.context.original_request,
            max_iterations=10,
            on_progress_callback=self._on_progress,
```

```
            workflow_context=self.context,
        )
        task.result = result
        task.status = SubTaskStatus.COMPLETED
        if result.get("tokens"):
            self.context.total_tokens += result["tokens"]
        if self.config.auto_run_tests and task.metadata.get("requires_test", False):
            await self._run_tests_for_task(task)
        return task.result
    except Exception as e:
        task.error = str(e)
        task.status = SubTaskStatus.FAILED
        if self.config.rollback_on_failure:
            await self._rollback_task(task)
        return None
async def _dispatch_task(self, task: SubTask) -> Any:
    """Dispatch a task to the appropriate handler."""
    task_type = task.metadata.get("type", "generic")
    handlers = {
        "skill_creation": self._handle_skill_creation,
        "project_init": self._handle_project_init,
        "code_search": self._handle_code_search,
        "dependency_install": self._handle_dependency_install,
        "code_generation": self._handle_code_generation,
        "code_edit": self._handle_code_edit,
        "test_execution": self._handle_test_execution,
        "git_operation": self._handle_git_operation,
        "generic": self._handle_generic_task,
    }
    handler = handlers.get(task_type, self._handle_generic_task)
    return await handler(task)
async def _handle_skill_creation(self, task: SubTask) -> Any:
    """Handle automatic skill creation."""
    skill_name = task.metadata.get("skill_name", "")
    skill_description = task.metadata.get("skill_description", "")
    if not skill_name:
        return None
    skill_content = await self._generate_skill_content(skill_name, skill_description)
    skill = await self._create_skill(skill_name, skill_content, skill_description)
    if skill:
        self.context.decisions.append(
            Decision(
                type=DecisionType.SKILL_CREATION,
                confidence=0.9,
                reasoning=f"Created skill '{skill_name}' for task",
                action="create_skill",
                parameters={"skill_name": skill_name},
                executed=True,
                result=skill.name,
            )
        )
    return skill
async def _handle_project_init(self, task: SubTask) -> Any:
    """Handle automatic project initialization."""
    project_type = task.metadata.get("project_type", "generic")
    project_path = task.metadata.get("project_path", self.app_state.cwd)
    result = await self._initialize_project(project_path, project_type)
    self.context.decisions.append(
        Decision(
            type=DecisionType.PROJECT_INIT,
            confidence=0.85,
            reasoning=f"Initialized {project_type} project at {project_path}",
            action="init_project",
            parameters={"project_type": project_type, "project_path": project_path},
            executed=True,
```

```
            result=result,
        )
    )
    return result
async def _handle_code_search(self, task: SubTask) -> Any:
    """Handle automatic code search."""
    query = task.metadata.get("query", "")
    file_pattern = task.metadata.get("file_pattern", "*")
    results = await self._search_code(query, file_pattern)
    self.context.decisions.append(
        Decision(
            type=DecisionType.CODE_SEARCH,
            confidence=0.8,
            reasoning=f"Searched for '{query}' in {file_pattern}",
            action="search_code",
            parameters={"query": query, "file_pattern": file_pattern},
            executed=True,
            result=results,
        )
    )
    return results
async def _handle_dependency_install(self, task: SubTask) -> Any:
    """Handle automatic dependency installation."""
    dependencies = task.metadata.get("dependencies", [])
    package_manager = task.metadata.get("package_manager", "auto")
    result = await self._install_dependencies(dependencies, package_manager)
    self.context.decisions.append(
        Decision(
            type=DecisionType.DEPENDENCY_INSTALL,
            confidence=0.85,
            reasoning=f"Installed dependencies: {dependencies}",
            action="install_deps",
            parameters={"dependencies": dependencies, "package_manager": package_manager},
            executed=True,
            result=result,
        )
    )
    return result
async def _handle_code_generation(self, task: SubTask) -> Any:
    """Handle code generation tasks."""
    return {"generated": True, "task": task.name}
async def _handle_code_edit(self, task: SubTask) -> Any:
    """Handle code editing tasks."""
    return {"edited": True, "task": task.name}
async def _handle_test_execution(self, task: SubTask) -> Any:
    """Handle test execution tasks."""
    return {"tested": True, "task": task.name}
async def _handle_git_operation(self, task: SubTask) -> Any:
    """Handle git operations."""
    operation = task.metadata.get("operation", "status")
    if operation == "commit" and self.config.auto_commit:
        return await self._git_commit(task)
    return {"operation": operation, "executed": False}
async def _handle_generic_task(self, task: SubTask) -> Any:
    """Handle generic tasks."""
    return {"completed": True, "task": task.name}
async def _reflect(self) -> Reflection:
    """Reflect on the completed work."""
    completed = self.context.get_completed_tasks()
    failed = self.context.get_failed_tasks()
    observations: list[str] = []
    improvements: list[str] = []
    learned_patterns: list[str] = []
    error_patterns: list[str] = []
    for task in completed:
```

```
        if task.result:
            observations.append(f"Task '{task.name}' completed successfully")
    for task in failed:
        if task.error:
            error_patterns.append(f"Task '{task.name}' failed: {task.error}")
            improvements.append(f"Need better error handling for '{task.name}'")
    reflection = Reflection(
        task_id=self.context.session_id,
        success=len(failed) == 0,
        observations=observations,
        improvements=improvements,
        learned_patterns=learned_patterns,
        error_patterns=error_patterns,
    )
    self.context.reflections.append(reflection)
    return reflection
async def _learn(self) -> dict[str, Any]:
    """Learn from the execution and update patterns."""
    for reflection in self.context.reflections:
        for pattern in reflection.learned_patterns:
            category = "general"
            if category not in self._learned_patterns:
                self._learned_patterns[category] = []
            if pattern not in self._learned_patterns[category]:
                self._learned_patterns[category].append(pattern)
    return {
        "patterns_learned": sum(len(v) for v in self._learned_patterns.values()),
        "categories": list(self._learned_patterns.keys()),
    }
def _classify_task(self, request: str) -> str:
    """Classify the type of task."""
    request_lower = request.lower()
    if any(kw in request_lower for kw in ["create", "new", "build", "implement"]):
        return "creation"
    elif any(kw in request_lower for kw in ["fix", "bug", "error", "issue"]):
        return "bugfix"
    elif any(kw in request_lower for kw in ["refactor", "improve", "optimize"]):
        return "refactor"
    elif any(kw in request_lower for kw in ["test", "spec", "coverage"]):
        return "testing"
    elif any(kw in request_lower for kw in ["document", "readme", "doc"]):
        return "documentation"
    else:
        return "general"
def _decompose_task(self, request: str, task_type: str) -> list[dict[str, Any]]:
    """Decompose a task into sub-tasks."""
    sub_tasks = [
        {
            "name": "analyze_context",
            "description": "Analyze current project context",
            "type": "analysis",
            "priority": 0,
            "dependencies": [],
        },
        {
            "name": "search_reference",
            "description": "Search for relevant code references",
            "type": "code_search",
            "priority": 1,
            "dependencies": ["analyze_context"],
        },
    ]
    if task_type == "creation":
        sub_tasks.extend(
            [
```

```
                {
                    "name": "create_structure",
                    "description": "Create necessary file structure",
                    "type": "code_generation",
                    "priority": 2,
                    "dependencies": ["search_reference"],
                    "requires_test": True,
                },
            ]
        )
    elif task_type == "bugfix":
        sub_tasks.extend(
            [
                {
                    "name": "identify_issue",
                    "description": "Identify the root cause",
                    "type": "analysis",
                    "priority": 2,
                    "dependencies": ["search_reference"],
                },
                {
                    "name": "apply_fix",
                    "description": "Apply the fix",
                    "type": "code_edit",
                    "priority": 3,
                    "dependencies": ["identify_issue"],
                    "requires_test": True,
                },
            ]
        )
    sub_tasks.append(
        {
            "name": "verify_results",
            "description": "Verify the results",
            "type": "test_execution",
            "priority": 99,
            "dependencies": [],
        }
    )
    return sub_tasks
def _identify_required_skills(self, request: str, task_type: str) -> list[str]:
    """Identify what skills might be needed."""
    available_skills = self.skill_registry.list_skills()
    required = []
    request_lower = request.lower()
    skill_keywords = {
        "browse": ["browser", "test", "qa", "website", "page"],
        "debug": ["debug", "error", "fix", "issue"],
        "ship": ["deploy", "release", "ship", "publish"],
        "qa": ["test", "qa", "quality"],
    }
    for skill, keywords in skill_keywords.items():
        if skill in available_skills and any(kw in request_lower for kw in keywords):
            required.append(skill)
    return required
def _identify_required_tools(self, request: str, task_type: str) -> list[str]:
    """Identify what tools might be needed."""
    tools = ["bash", "read", "write", "edit"]
    request_lower = request.lower()
    if "search" in request_lower or "find" in request_lower:
        tools.extend(["grep", "glob"])
    if "git" in request_lower or "commit" in request_lower:
        tools.append("bash")
    if "web" in request_lower or "url" in request_lower:
        tools.append("webfetch")
```

```
        return list(set(tools))
    def _estimate_complexity(self, sub_tasks: list[dict[str, Any]]) -> str:
        """Estimate task complexity."""
        count = len(sub_tasks)
        if count <= 3:
            return "low"
        elif count <= 7:
            return "medium"
        else:
            return "high"
    def _topological_sort(self, tasks: list[SubTask]) -> list[SubTask]:
        """Sort tasks by dependencies using topological sort."""
        sorted_tasks = []
        visited = set()
        temp_visited = set()
        def visit(task: SubTask) -> None:
            if task.id in visited:
                return
            if task.id in temp_visited:
                return
            temp_visited.add(task.id)
            for dep_id in task.dependencies:
                for t in tasks:
                    if t.id == dep_id:
                        visit(t)
            temp_visited.remove(task.id)
            visited.add(task.id)
            sorted_tasks.append(task)
        for task in tasks:
            if task.id not in visited:
                visit(task)
        return sorted_tasks
    def _can_execute_task(self, task: SubTask) -> bool:
        """Check if a task's dependencies are satisfied."""
        for dep_id in task.dependencies:
            dep_task = next((t for t in self.context.sub_tasks if t.id == dep_id), None)
            if dep_task and dep_task.status != SubTaskStatus.COMPLETED:
                return False
        return True
    async def _generate_skill_content(self, skill_name: str, description: str) -> str:
        """Generate skill content using templates or AI."""
        template = self.config.skill_templates.get(
            skill_name,
            f"""---
name: {skill_name}
description: {description}
version: 1.0.0
---
# {skill_name}
{description}
## Instructions
This skill was auto-generated by HyperAuto for the task: {self.context.original_request}
## Workflow
1. Analyze the request
2. Execute the task
3. Verify the results
""",
        )
        return template
    async def _create_skill(
        self,
        skill_name: str,
        content: str,
        description: str,
    ) -> Skill | None:
```

```
    """Create and register a new skill."""
    try:
        skill = Skill.from_content(content)
        skill.name = skill_name
        skill.description = description
        return skill
    except Exception:
        return None
async def _initialize_project(self, project_path: str, project_type: str) -> dict[str, Any]
:
    """Initialize a project structure."""
    return {
        "path": project_path,
        "type": project_type,
        "initialized": True,
    }
async def _search_code(self, query: str, file_pattern: str) -> list[dict[str, Any]]:
    """Search for code in the codebase."""
    return [
        {
            "query": query,
            "pattern": file_pattern,
            "results": [],
        }
    ]
async def _install_dependencies(
    self,
    dependencies: list[str],
    package_manager: str,
) -> dict[str, Any]:
    """Install project dependencies."""
    return {
        "dependencies": dependencies,
        "manager": package_manager,
        "installed": True,
    }
async def _run_tests_for_task(self, task: SubTask) -> dict[str, Any]:
    """Run tests related to a task."""
    return {"tested": True, "task": task.id}
async def _rollback_task(self, task: SubTask) -> None:
    """Rollback changes made by a failed task."""
    pass
async def _git_commit(self, task: SubTask) -> dict[str, Any]:
    """Perform a git commit."""
    return {"committed": True, "task": task.id}
def _get_result(self) -> dict[str, Any]:
    """Get the final result of the workflow."""
    completed = len(self.context.get_completed_tasks())
    failed = len(self.context.get_failed_tasks())
    total = len(self.context.sub_tasks)
    return {
        "success": failed == 0 and completed > 0,
        "session_id": self.context.session_id,
        "iterations": self.context.iteration,
        "tasks_completed": completed,
        "tasks_failed": failed,
        "tasks_total": total,
        "decisions_made": len(self.context.decisions),
        "reflections": len(self.context.reflections),
        "final_state": self.context.current_state.value,
        "duration": (self.context.end_time or time.time()) - self.context.start_time,
        "context": self.context.to_dict(),
    }
```

File: [77/252] openlaoke/core/hyperauto/agent_supervisor_integration.py

```
"""Agent supervisor integration for verification and evolution."""
from __future__ import annotations
import re
from typing import TYPE_CHECKING, Any
from openlaoke.core.hyperauto.types import Decision, DecisionType, SubTask, SubTaskStatus
from openlaoke.core.supervisor import SupervisionResult, TaskSupervisor
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class AgentSupervisorIntegration:
    """Integrates supervisor into HyperAuto agent for verification and evolution."""
    def __init__(self, app_state: AppState):
        self.app_state = app_state
        self.supervisor = TaskSupervisor(app_state)
    async def verify_completion(self, context: Any, task_id: str) -> SupervisionResult:
        """Verify that the task is truly completed."""
        artifacts = {
            "content": "",
            "output_files": [],
            "working_dir": self.app_state.cwd,
        }
        for task in context.sub_tasks:
            if task.status == SubTaskStatus.COMPLETED and task.result:
                tool_results = task.result.get("tool_results", [])
                for tr in tool_results:
                    if tr.get("tool") in ["Write", "Edit", "Bash"]:
                        result_text = tr.get("result", "")
                        if "created" in result_text.lower() or "written" in result_text.lower()
    :
                            file_match = re.search(
                                r"['\"]([^'\"]+\.(c|h|py|md|txt))['\"]", result_text
                            )
                            if file_match:
                                artifacts["output_files"].append(file_match.group(1))
                final_response = task.result.get("final_response", "")
                artifacts["content"] += final_response + "\n"
        result = await self.supervisor.check_completion(task_id, artifacts)
        return result
    def evolve_strategy(
        self, verification_result: SupervisionResult, context: Any
    ) -> list[SubTask]:
        """Evolve strategy based on verification failure."""
        new_tasks = []
        missing = verification_result.missing_requirements
        if missing:
            for i, req in enumerate(missing[:3]):
                new_task = SubTask(
                    name=f"address_missing_{i}",
                    description=f"Address requirement: {req}",
                    priority=100 + i,
                    dependencies=[],
                    metadata={
                        "type": "fix",
                        "from_verification": True,
                        "retry_reason": verification_result.retry_reason.value
                        if verification_result.retry_reason
                        else None,
                    },
                )
                new_tasks.append(new_task)
        context.decisions.append(
            Decision(
```

```
            type=DecisionType.RETRY,
            confidence=0.7,
            reasoning=f"Task incomplete: {verification_result.completion_percentage:.1f}%.
Missing: {missing[:3]}",
            action="evolve_strategy",
            parameters={
                "missing_requirements": missing,
                "completion_pct": verification_result.completion_percentage,
            },
            executed=True,
        )
    )
    return new_tasks
def get_retry_prompt(
    self, verification_result: SupervisionResult, original_request: str
) -> str:
    """Generate retry prompt based on verification failure."""
    return self.supervisor.get_retry_prompt(original_request, verification_result)
```

File: [78/252] openlaoke/core/hyperauto/code_search.py

```
"""Intelligent code search system for finding and adapting code."""
from __future__ import annotations
import ast
import json
import logging
import os
import re
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any
from urllib.parse import quote_plus
logger = logging.getLogger(__name__)
class SearchSource(StrEnum):
    """Source of code snippets."""
    LOCAL = "local"
    REFERENCE_PROJECT = "reference_project"
    GITHUB = "github"
    STACKOVERFLOW = "stackoverflow"
    DOCS = "docs"
@dataclass
class CodeSnippet:
    """Represents a code snippet found during search."""
    source: str
    file_path: str
    language: str
    content: str
    description: str
    relevance_score: float = 0.0
    adaptability_score: float = 0.0
    line_start: int = 0
    line_end: int = 0
    context: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "source": self.source,
            "file_path": self.file_path,
            "language": self.language,
            "content": self.content,
            "description": self.description,
            "relevance_score": self.relevance_score,
            "adaptability_score": self.adaptability_score,
            "line_start": self.line_start,
            "line_end": self.line_end,
            "context": self.context,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> CodeSnippet:
        return cls(
            source=data.get("source", ""),
            file_path=data.get("file_path", ""),
            language=data.get("language", ""),
            content=data.get("content", ""),
            description=data.get("description", ""),
            relevance_score=data.get("relevance_score", 0.0),
            adaptability_score=data.get("adaptability_score", 0.0),
            line_start=data.get("line_start", 0),
            line_end=data.get("line_end", 0),
            context=data.get("context", {}),
        )
@dataclass
```

```
class SearchResult:
    """Container for search results."""
    query: str
    snippets: list[CodeSnippet]
    total_results: int
    search_time_ms: float = 0.0
    sources_searched: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "query": self.query,
            "snippets": [s.to_dict() for s in self.snippets],
            "total_results": self.total_results,
            "search_time_ms": self.search_time_ms,
            "sources_searched": self.sources_searched,
        }
class CodeSearchEngine:
    """Engine for searching code across multiple sources."""
    LANGUAGE_EXTENSIONS: dict[str, list[str]] = {
        "python": [".py", ".pyw", ".pyi"],
        "javascript": [".js", ".jsx", ".mjs"],
        "typescript": [".ts", ".tsx"],
        "java": [".java"],
        "go": [".go"],
        "rust": [".rs"],
        "cpp": [".cpp", ".cc", ".cxx", ".hpp", ".h"],
        "c": [".c", ".h"],
        "ruby": [".rb", ".rake"],
        "php": [".php"],
        "swift": [".swift"],
        "kotlin": [".kt", ".kts"],
        "scala": [".scala"],
        "shell": [".sh", ".bash", ".zsh"],
        "yaml": [".yaml", ".yml"],
        "json": [".json"],
        "markdown": [".md", ".markdown"],
    }
    EXTENSION_TO_LANGUAGE: dict[str, str] = {}
    for lang, exts in LANGUAGE_EXTENSIONS.items():
        for ext in exts:
            EXTENSION_TO_LANGUAGE[ext] = lang
    def __init__(
        self,
        project_path: Path | None = None,
        reference_projects: list[Path] | None = None,
        github_token: str | None = None,
    ):
        try:
            self.project_path = project_path or Path.cwd()
        except (OSError, FileNotFoundError):
            self.project_path = Path(".")
        self.reference_projects = reference_projects or []
        self.github_token = github_token or os.environ.get("GITHUB_TOKEN")
        self._index_cache: dict[str, list[dict[str, Any]]] = {}
    def search_local(
        self,
        query: str,
        project_path: Path | None = None,
        max_results: int = 50,
        file_pattern: str = "*.py",
    ) -> list[CodeSnippet]:
        """Search local codebase for relevant code."""
        search_path = project_path or self.project_path
        results: list[CodeSnippet] = []
        if not search_path.exists():
            logger.warning(f"Search path does not exist: {search_path}")
```

```
        return results
    query_lower = query.lower()
    query_terms = set(query_lower.split())
    for file_path in search_path.rglob(file_pattern):
        if self._should_ignore(file_path):
            continue
        try:
            content = file_path.read_text(encoding="utf-8", errors="replace")
            snippets = self._extract_relevant_snippets(
                content=content,
                file_path=str(file_path.relative_to(search_path)),
                query_terms=query_terms,
                query=query,
            )
            results.extend(snippets)
        except (PermissionError, OSError) as e:
            logger.debug(f"Cannot read {file_path}: {e}")
            continue
    results.sort(key=lambda s: s.relevance_score, reverse=True)
    return results[:max_results]
def search_reference_projects(
    self,
    query: str,
    max_results: int = 30,
) -> list[CodeSnippet]:
    """Search reference projects for code examples."""
    results: list[CodeSnippet] = []
    for ref_path in self.reference_projects:
        if not ref_path.exists():
            continue
        snippets = self.search_local(
            query=query,
            project_path=ref_path,
            max_results=max_results,
        )
        for snippet in snippets:
            snippet.source = SearchSource.REFERENCE_PROJECT.value
        results.extend(snippets)
    results.sort(key=lambda s: s.relevance_score, reverse=True)
    return results[:max_results]
async def search_github(
    self,
    query: str,
    language: str = "python",
    max_results: int = 20,
) -> list[CodeSnippet]:
    """Search GitHub for code snippets."""
    results: list[CodeSnippet] = []
    try:
        import urllib.request
        encoded_query = quote_plus(f"{query} language:{language}")
        url = f"https://api.github.com/search/code?q={encoded_query}&per_page={max_results}
"
        headers = {"Accept": "application/vnd.github.v3+json"}
        if self.github_token:
            headers["Authorization"] = f"token {self.github_token}"
        req = urllib.request.Request(url, headers=headers)
        with urllib.request.urlopen(req, timeout=10) as response:
            data = json.loads(response.read().decode())
        for item in data.get("items", []):
            snippet = CodeSnippet(
                source=SearchSource.GITHUB.value,
                file_path=item.get("path", ""),
                language=language,
                content="",
```

```
                description=f"{item.get('repository', {}).get('full_name', '')}: {item.get(
'path', '')}",
                relevance_score=1.0 - (item.get("score", 0) / 100),
                context={
                    "html_url": item.get("html_url", ""),
                    "repository": item.get("repository", {}).get("full_name", ""),
                },
            )
            results.append(snippet)
    except Exception as e:
        logger.warning(f"GitHub search failed: {e}")
    return results
def search_semantic(
    self,
    query: str,
    code_context: str | None = None,
    max_results: int = 20,
) -> list[CodeSnippet]:
    """Perform semantic search using code understanding."""
    results: list[CodeSnippet] = []
    local_results = self.search_local(query, max_results=max_results)
    results.extend(local_results)
    ref_results = self.search_reference_projects(query, max_results=max_results)
    results.extend(ref_results)
    if code_context:
        results = self._rerank_by_context(results, code_context)
    results = self._deduplicate(results)
    results.sort(key=lambda s: (s.relevance_score + s.adaptability_score) / 2, reverse=True
)
    return results[:max_results]
def analyze_reusability(self, snippet: CodeSnippet) -> float:
    """Analyze how reusable a code snippet is."""
    if not snippet.content:
        return 0.0
    score = 0.5
    try:
        if snippet.language == "python":
            score = self._analyze_python_reusability(snippet.content)
        else:
            score = self._analyze_generic_reusability(snippet.content)
    except Exception as e:
        logger.debug(f"Reusability analysis failed: {e}")
    snippet.adaptability_score = score
    return score
def _analyze_python_reusability(self, content: str) -> float:
    """Analyze reusability of Python code."""
    score = 0.5
    try:
        tree = ast.parse(content)
        function_count = sum(1 for node in ast.walk(tree) if isinstance(node, ast.FunctionD
ef))
        class_count = sum(1 for node in ast.walk(tree) if isinstance(node, ast.ClassDef))
        if function_count > 0:
            score += 0.1
        if class_count > 0:
            score += 0.1
        if function_count > 2:
            score += 0.05
        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
                if node.args.args:
                    score += 0.02
                if ast.unparse(node).startswith("def "):
                    has_docstring = ast.get_docstring(node) is not None
                    if has_docstring:
```

```
                        score += 0.03
        imports = [n for n in ast.walk(tree) if isinstance(n, (ast.Import, ast.ImportFrom))
]
        if len(imports) < 5:
            score += 0.05
    except SyntaxError:
        pass
    return min(1.0, score)
def _analyze_generic_reusability(self, content: str) -> float:
    """Analyze reusability for non-Python code."""
    score = 0.5
    lines = content.split("\n")
    non_empty_lines = [line for line in lines if line.strip()]
    if len(non_empty_lines) > 5:
        score += 0.1
    if "function" in content or "def " in content:
        score += 0.1
    if "class " in content:
        score += 0.1
    comment_ratio = self._calculate_comment_ratio(content)
    if comment_ratio > 0.1:
        score += 0.05
    return min(1.0, score)
def _calculate_comment_ratio(self, content: str) -> float:
    """Calculate the ratio of comments in code."""
    lines = content.split("\n")
    if not lines:
        return 0.0
    comment_lines = 0
    for line in lines:
        stripped = line.strip()
        if stripped.startswith("#") or stripped.startswith("//") or stripped.startswith("/*
"):
            comment_lines += 1
    return comment_lines / len(lines)
def adapt_code(
    self,
    snippet: CodeSnippet,
    context: dict[str, Any],
) -> str:
    """Adapt a code snippet to fit the given context."""
    if not snippet.content:
        return ""
    adapted = snippet.content
    if "imports" in context:
        adapted = self._inject_imports(adapted, context["imports"])
    if "rename_map" in context:
        adapted = self._apply_renames(adapted, context["rename_map"])
    if "target_class" in context:
        adapted = self._adapt_to_class(adapted, context["target_class"])
    if "style" in context:
        adapted = self._apply_style(adapted, context["style"])
    return adapted
def _inject_imports(self, code: str, imports: list[str]) -> str:
    """Inject necessary imports into code."""
    existing_imports = set()
    lines = code.split("\n")
    import_lines = []
    for line in lines:
        if line.startswith("import ") or line.startswith("from "):
            existing_imports.add(line.strip())
            import_lines.append(line)
    new_imports = []
    for imp in imports:
        if imp not in existing_imports:
```

```
            new_imports.append(imp)
    if new_imports:
        insert_pos = 0
        for i, line in enumerate(lines):
            if line.startswith("import ") or line.startswith("from "):
                insert_pos = i + 1
            elif line.strip() and not line.startswith("#"):
                break
        for imp in reversed(new_imports):
            lines.insert(insert_pos, imp)
    return "\n".join(lines)
def _apply_renames(self, code: str, rename_map: dict[str, str]) -> str:
    """Apply variable/function renames to code."""
    for old_name, new_name in rename_map.items():
        pattern = r"\b" + re.escape(old_name) + r"\b"
        code = re.sub(pattern, new_name, code)
    return code
def _adapt_to_class(self, code: str, target_class: str) -> str:
    """Adapt standalone functions to class methods."""
    lines = code.split("\n")
    adapted_lines = []
    for line in lines:
        if line.strip().startswith("def "):
            if not line.strip().startswith("def __"):
                indent = len(line) - len(line.lstrip())
                adapted_lines.append(" " * indent + line.lstrip())
                adapted_lines.append(" " * (indent + 4) + "self")
            else:
                adapted_lines.append(line)
        else:
            adapted_lines.append(line)
    return "\n".join(adapted_lines)
def _apply_style(self, code: str, style: str) -> str:
    """Apply code style transformations."""
    if style == "google":
        code = self._apply_google_style(code)
    elif style == "numpy":
        code = self._apply_numpy_style(code)
    return code
def _apply_google_style(self, code: str) -> str:
    """Apply Google docstring style."""
    return code
def _apply_numpy_style(self, code: str) -> str:
    """Apply NumPy docstring style."""
    return code
async def auto_search_and_adapt(
    self,
    query: str,
    context: dict[str, Any],
    max_results: int = 10,
) -> list[str]:
    """Automatically search and adapt code snippets."""
    snippets = self.search_semantic(
        query=query,
        code_context=context.get("code_context"),
        max_results=max_results * 2,
    )
    github_snippets = await self.search_github(
        query=query,
        language=context.get("language", "python"),
        max_results=max_results,
    )
    snippets.extend(github_snippets)
    for snippet in snippets:
        self.analyze_reusability(snippet)
```

```
    snippets.sort(
        key=lambda s: (s.relevance_score + s.adaptability_score) / 2,
        reverse=True,
    )
    adapted = []
    seen_content = set()
    for snippet in snippets[:max_results]:
        adapted_code = self.adapt_code(snippet, context)
        content_hash = hash(adapted_code)
        if content_hash not in seen_content:
            adapted.append(adapted_code)
            seen_content.add(content_hash)
    return adapted
def _should_ignore(self, path: Path) -> bool:
    """Check if a path should be ignored."""
    ignore_dirs = {
        "__pycache__",
        ".git",
        ".hg",
        ".svn",
        "node_modules",
        "venv",
        ".venv",
        "env",
        ".env",
        "dist",
        "build",
        ".tox",
        ".eggs",
        "*.egg-info",
    }
    return any(part in ignore_dirs for part in path.parts)
def _extract_relevant_snippets(
    self,
    content: str,
    file_path: str,
    query_terms: set[str],
    query: str,
) -> list[CodeSnippet]:
    """Extract relevant code snippets from file content."""
    results: list[CodeSnippet] = []
    ext = Path(file_path).suffix.lower()
    language = self.EXTENSION_TO_LANGUAGE.get(ext, "text")
    if language == "python":
        snippets = self._extract_python_snippets(content, file_path, query_terms)
    else:
        snippets = self._extract_generic_snippets(content, file_path, query_terms)
    for snippet in snippets:
        snippet.language = language
        snippet.relevance_score = self._calculate_relevance(
            snippet.content,
            query_terms,
            query,
        )
        results.append(snippet)
    return results
def _extract_python_snippets(
    self,
    content: str,
    file_path: str,
    query_terms: set[str],
) -> list[CodeSnippet]:
    """Extract Python functions and classes."""
    results: list[CodeSnippet] = []
    try:
```

```
        tree = ast.parse(content)
        lines = content.split("\n")
        for node in ast.walk(tree):
            if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
                start_line = node.lineno - 1
                end_line = node.end_lineno or start_line + 1
                snippet_content = "\n".join(lines[start_line:end_line])
                name = node.name
                docstring = ast.get_docstring(node) or ""
                node_type = (
                    "function"
                    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))
                    else "class"
                )
                description = f"{node_type} {name}"
                if docstring:
                    description += f": {docstring.split(chr(10))[0]}"
                snippet = CodeSnippet(
                    source=SearchSource.LOCAL.value,
                    file_path=file_path,
                    language="python",
                    content=snippet_content,
                    description=description,
                    line_start=start_line + 1,
                    line_end=end_line,
                )
                results.append(snippet)
    except SyntaxError:
        pass
    return results
def _extract_generic_snippets(
    self,
    content: str,
    file_path: str,
    query_terms: set[str],
) -> list[CodeSnippet]:
    """Extract snippets from non-Python files."""
    results: list[CodeSnippet] = []
    lines = content.split("\n")
    snippet_lines: list[str] = []
    snippet_start = 0
    for i, line in enumerate(lines):
        should_include = False
        for term in query_terms:
            if term in line.lower():
                should_include = True
                break
        if should_include:
            if not snippet_lines:
                snippet_start = max(0, i - 2)
            snippet_lines.append(line)
        elif snippet_lines:
            if len(snippet_lines) > 2:
                snippet_content = "\n".join(snippet_lines)
                results.append(
                    CodeSnippet(
                        source=SearchSource.LOCAL.value,
                        file_path=file_path,
                        language="text",
                        content=snippet_content,
                        description=f"Lines {snippet_start + 1}-{i}",
                        line_start=snippet_start + 1,
                        line_end=i,
                    )
                )
```

```
            snippet_lines = []
    if snippet_lines and len(snippet_lines) > 2:
        snippet_content = "\n".join(snippet_lines)
        results.append(
            CodeSnippet(
                source=SearchSource.LOCAL.value,
                file_path=file_path,
                language="text",
                content=snippet_content,
                description=f"Lines {snippet_start + 1}-{len(lines)}",
                line_start=snippet_start + 1,
                line_end=len(lines),
            )
        )
    return results
def _calculate_relevance(
    self,
    content: str,
    query_terms: set[str],
    query: str,
) -> float:
    """Calculate relevance score for a snippet."""
    if not content:
        return 0.0
    content_lower = content.lower()
    score = 0.0
    for term in query_terms:
        if term in content_lower:
            score += 0.2
            count = content_lower.count(term)
            score += min(0.1, count * 0.02)
    if query.lower() in content_lower:
        score += 0.3
    length_penalty = max(0, (len(content) - 500) / 5000)
    score -= length_penalty * 0.1
    return max(0.0, min(1.0, score))
def _rerank_by_context(
    self,
    snippets: list[CodeSnippet],
    context: str,
) -> list[CodeSnippet]:
    """Rerank snippets based on code context."""
    context_terms = set(context.lower().split())
    for snippet in snippets:
        content_terms = set(snippet.content.lower().split())
        overlap = len(context_terms & content_terms)
        snippet.relevance_score += overlap * 0.01
    return sorted(snippets, key=lambda s: s.relevance_score, reverse=True)
def _deduplicate(self, snippets: list[CodeSnippet]) -> list[CodeSnippet]:
    """Remove duplicate snippets."""
    seen: set[int] = set()
    results: list[CodeSnippet] = []
    for snippet in snippets:
        content_hash = hash(snippet.content)
        if content_hash not in seen:
            seen.add(content_hash)
            results.append(snippet)
    return results
```

File: [79/252] openlaoke/core/hyperauto/config.py

```
"""Configuration for HyperAuto system."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
from openlaoke.core.hyperauto.types import HyperAutoMode
@dataclass
class HyperAutoConfig:
    """Configuration for HyperAuto agent.
    Attributes:
        mode: Operating mode (semi_auto, full_auto, hyper_auto)
        max_iterations: Maximum number of iterations before stopping
        auto_create_skills: Automatically create skills when needed
        auto_init_projects: Automatically initialize project structures
        auto_search_code: Automatically search for relevant code
        auto_install_deps: Automatically install dependencies
        auto_run_tests: Automatically run tests after changes
        auto_commit: Automatically commit changes (default: False for safety)
        confidence_threshold: Minimum confidence for auto-decisions
        reflection_enabled: Enable self-reflection after tasks
        learning_enabled: Enable learning from past executions
        max_parallel_tasks: Maximum parallel sub-tasks
        timeout_per_task: Timeout in seconds per sub-task
        rollback_on_failure: Rollback changes on failure
        dry_run: Simulate without making actual changes
        verbose: Enable verbose logging
        skill_templates: Templates for auto-generated skills
        project_templates: Templates for project initialization
    """
    mode: HyperAutoMode = HyperAutoMode.FULL_AUTO
    max_iterations: int = 100
    auto_create_skills: bool = True
    auto_init_projects: bool = True
    auto_search_code: bool = True
    auto_install_deps: bool = True
    auto_run_tests: bool = True
    auto_commit: bool = False
    confidence_threshold: float = 0.8
    reflection_enabled: bool = True
    learning_enabled: bool = True
    max_parallel_tasks: int = 5
    timeout_per_task: float = 300.0
    rollback_on_failure: bool = True
    dry_run: bool = False
    verbose: bool = False
    skill_templates: dict[str, str] = field(default_factory=dict)
    project_templates: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "mode": self.mode.value,
            "max_iterations": self.max_iterations,
            "auto_create_skills": self.auto_create_skills,
            "auto_init_projects": self.auto_init_projects,
            "auto_search_code": self.auto_search_code,
            "auto_install_deps": self.auto_install_deps,
            "auto_run_tests": self.auto_run_tests,
            "auto_commit": self.auto_commit,
            "confidence_threshold": self.confidence_threshold,
            "reflection_enabled": self.reflection_enabled,
            "learning_enabled": self.learning_enabled,
            "max_parallel_tasks": self.max_parallel_tasks,
            "timeout_per_task": self.timeout_per_task,
```

```
        "rollback_on_failure": self.rollback_on_failure,
        "dry_run": self.dry_run,
        "verbose": self.verbose,
        "skill_templates": self.skill_templates,
        "project_templates": self.project_templates,
    }
@classmethod
def semi_auto(cls) -> HyperAutoConfig:
    """Create a semi-auto config (asks for confirmation)."""
    return cls(
        mode=HyperAutoMode.SEMI_AUTO,
        confidence_threshold=1.0,
        max_parallel_tasks=1,
        verbose=True,
    )
@classmethod
def full_auto(cls) -> HyperAutoConfig:
    """Create a full-auto config (autonomous with safety limits)."""
    return cls(
        mode=HyperAutoMode.FULL_AUTO,
        confidence_threshold=0.8,
        max_parallel_tasks=5,
    )
@classmethod
def hyper_auto(cls) -> HyperAutoConfig:
    """Create a hyper-auto config (maximum autonomy)."""
    return cls(
        mode=HyperAutoMode.HYPER_AUTO,
        confidence_threshold=0.6,
        max_parallel_tasks=10,
        auto_commit=True,
        reflection_enabled=True,
        learning_enabled=True,
    )
@classmethod
def from_dict(cls, data: dict[str, Any]) -> HyperAutoConfig:
    """Create config from dictionary."""
    mode_str = data.get("mode", "full_auto")
    mode = HyperAutoMode(mode_str) if isinstance(mode_str, str) else HyperAutoMode.FULL_AUT
O
    return cls(
        mode=mode,
        max_iterations=data.get("max_iterations", 100),
        auto_create_skills=data.get("auto_create_skills", True),
        auto_init_projects=data.get("auto_init_projects", True),
        auto_search_code=data.get("auto_search_code", True),
        auto_install_deps=data.get("auto_install_deps", True),
        auto_run_tests=data.get("auto_run_tests", True),
        auto_commit=data.get("auto_commit", False),
        confidence_threshold=data.get("confidence_threshold", 0.8),
        reflection_enabled=data.get("reflection_enabled", True),
        learning_enabled=data.get("learning_enabled", True),
        max_parallel_tasks=data.get("max_parallel_tasks", 5),
        timeout_per_task=data.get("timeout_per_task", 300.0),
        rollback_on_failure=data.get("rollback_on_failure", True),
        dry_run=data.get("dry_run", False),
        verbose=data.get("verbose", False),
        skill_templates=data.get("skill_templates", {}),
        project_templates=data.get("project_templates", {}),
    )
```

File: [80/252] openlaoke/core/hyperauto/decision_engine.py

```
"""Intelligent decision engine for HyperAuto system."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from uuid import uuid4
from openlaoke.core.hyperauto.types import Decision, DecisionType, HyperAutoState, SubTask
if TYPE_CHECKING:
    from openlaoke.core.hyperauto.config import HyperAutoConfig
class RiskLevel(StrEnum):
    """Risk levels for decision options."""
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    CRITICAL = "critical"
class DecisionCategory(StrEnum):
    """Categories of decisions."""
    EXECUTION_STRATEGY = "execution_strategy"
    TOOL_SELECTION = "tool_selection"
    SKILL_CREATION = "skill_creation"
    PROJECT_INIT = "project_init"
    ERROR_HANDLING = "error_handling"
    RESOURCE_ALLOCATION = "resource_allocation"
@dataclass
class Context:
    """Execution context for decision making."""
    state: HyperAutoState = HyperAutoState.IDLE
    task: SubTask | None = None
    available_tools: list[str] = field(default_factory=list)
    available_skills: list[str] = field(default_factory=list)
    project_info: dict[str, Any] = field(default_factory=dict)
    constraints: dict[str, Any] = field(default_factory=dict)
    history: list[dict[str, Any]] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "state": self.state.value,
            "task": self.task.to_dict() if self.task else None,
            "available_tools": self.available_tools,
            "available_skills": self.available_skills,
            "project_info": self.project_info,
            "constraints": self.constraints,
            "history": self.history,
            "metadata": self.metadata,
        }
@dataclass
class DecisionOption:
    """A possible decision option."""
    action: str = ""
    description: str = ""
    expected_outcome: str = ""
    risk_level: RiskLevel = RiskLevel.LOW
    confidence: float = 0.0
    reasoning: str = ""
    alternatives: list[str] = field(default_factory=list)
    category: DecisionCategory = DecisionCategory.EXECUTION_STRATEGY
    parameters: dict[str, Any] = field(default_factory=dict)
    rollback_plan: str | None = None
    estimated_duration: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
```

```
    def to_dict(self) -> dict[str, Any]:
        return {
            "action": self.action,
            "description": self.description,
            "expected_outcome": self.expected_outcome,
            "risk_level": self.risk_level.value,
            "confidence": self.confidence,
            "reasoning": self.reasoning,
            "alternatives": self.alternatives,
            "category": self.category.value,
            "parameters": self.parameters,
            "rollback_plan": self.rollback_plan,
            "estimated_duration": self.estimated_duration,
            "metadata": self.metadata,
        }
@dataclass
class RiskAssessment:
    """Risk assessment for a decision option."""
    option_id: str = ""
    risk_level: RiskLevel = RiskLevel.LOW
    risk_factors: list[str] = field(default_factory=list)
    mitigation_strategies: list[str] = field(default_factory=list)
    impact_score: float = 0.0
    probability_score: float = 0.0
    overall_risk_score: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "option_id": self.option_id,
            "risk_level": self.risk_level.value,
            "risk_factors": self.risk_factors,
            "mitigation_strategies": self.mitigation_strategies,
            "impact_score": self.impact_score,
            "probability_score": self.probability_score,
            "overall_risk_score": self.overall_risk_score,
            "metadata": self.metadata,
        }
@dataclass
class DecisionRecord:
    """Record of a made decision."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    decision: Decision | None = None
    context: Context | None = None
    options: list[DecisionOption] = field(default_factory=list)
    risk_assessments: list[RiskAssessment] = field(default_factory=list)
    selected_option: DecisionOption | None = None
    timestamp: float = field(default_factory=time.time)
    outcome: str | None = None
    success: bool = False
    lessons_learned: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "decision": self.decision.to_dict() if self.decision else None,
            "context": self.context.to_dict() if self.context else None,
            "options": [o.to_dict() for o in self.options],
            "risk_assessments": [r.to_dict() for r in self.risk_assessments],
            "selected_option": self.selected_option.to_dict() if self.selected_option else None
    ,
            "timestamp": self.timestamp,
            "outcome": self.outcome,
            "success": self.success,
            "lessons_learned": self.lessons_learned,
            "metadata": self.metadata,
```

```
        }
class DecisionEngine:
    """Intelligent decision making engine for HyperAuto.
    Features:
    - Context-aware analysis
    - Multiple option generation
    - Risk assessment and mitigation
    - Confidence-based selection
    - Decision recording and learning
    """
    def __init__(
        self,
        config: HyperAutoConfig | None = None,
    ) -> None:
        self.config = config
        self._decision_history: list[DecisionRecord] = []
        self._learned_patterns: dict[str, list[str]] = {}
        self._risk_thresholds = {
            RiskLevel.LOW: 0.3,
            RiskLevel.MEDIUM: 0.5,
            RiskLevel.HIGH: 0.7,
            RiskLevel.CRITICAL: 0.9,
        }
    def analyze_context(self, state: HyperAutoState, task: SubTask) -> Context:
        """Analyze the current execution context."""
        context = Context(
            state=state,
            task=task,
            available_tools=self._identify_available_tools(task),
            available_skills=self._identify_available_skills(task),
            project_info=self._gather_project_info(task),
            constraints=self._identify_constraints(state, task),
            history=self._get_relevant_history(task),
        )
        context.metadata["analysis_timestamp"] = time.time()
        context.metadata["task_type"] = self._classify_task_type(task)
        return context
    def generate_options(self, context: Context) -> list[DecisionOption]:
        """Generate possible decision options based on context."""
        options: list[DecisionOption] = []
        task_type = context.metadata.get("task_type", "generic")
        if task_type == "skill_creation":
            options.extend(self._generate_skill_creation_options(context))
        elif task_type == "project_init":
            options.extend(self._generate_project_init_options(context))
        elif task_type == "code_search":
            options.extend(self._generate_code_search_options(context))
        elif task_type == "error_handling":
            options.extend(self._generate_error_handling_options(context))
        elif task_type == "execution_strategy":
            options.extend(self._generate_execution_strategy_options(context))
        else:
            options.extend(self._generate_generic_options(context))
        for option in options:
            option.confidence = self.calculate_confidence(option, context)
        return options
    def assess_risks(self, options: list[DecisionOption]) -> list[RiskAssessment]:
        """Assess risks for each decision option."""
        assessments: list[RiskAssessment] = []
        for option in options:
            assessment = self._assess_single_option(option)
            assessments.append(assessment)
        return assessments
    def calculate_confidence(self, option: DecisionOption, context: Context) -> float:
        """Calculate confidence score for a decision option."""
```

```
    base_confidence = 0.5
    historical_confidence = self._get_historical_confidence(option.action)
    if historical_confidence > 0:
        base_confidence = historical_confidence
    context_match_score = self._calculate_context_match(option, context)
    resource_availability_score = self._check_resource_availability(option, context)
    constraint_compliance_score = self._check_constraint_compliance(option, context)
    confidence = (
        base_confidence * 0.3
        + context_match_score * 0.3
        + resource_availability_score * 0.2
        + constraint_compliance_score * 0.2
    )
    if option.risk_level == RiskLevel.CRITICAL:
        confidence *= 0.5
    elif option.risk_level == RiskLevel.HIGH:
        confidence *= 0.7
    elif option.risk_level == RiskLevel.MEDIUM:
        confidence *= 0.85
    return min(max(confidence, 0.0), 1.0)
def select_best(self, options: list[DecisionOption]) -> Decision:
    """Select the best decision from available options."""
    if not options:
        return Decision(
            type=DecisionType.ABORT,
            confidence=0.0,
            reasoning="No options available",
            action="abort",
        )
    scored_options = []
    for option in options:
        score = self._calculate_selection_score(option)
        scored_options.append((option, score))
    scored_options.sort(key=lambda x: x[1], reverse=True)
    best_option = scored_options[0][0]
    decision_type = self._map_category_to_decision_type(best_option.category)
    decision = Decision(
        type=decision_type,
        confidence=best_option.confidence,
        reasoning=best_option.reasoning,
        action=best_option.action,
        parameters=best_option.parameters,
    )
    return decision
def record_decision(self, decision: Decision, record: DecisionRecord) -> None:
    """Record a decision for future learning."""
    record.decision = decision
    record.outcome = "pending"
    record.success = False
    self._decision_history.append(record)
    self._update_learned_patterns(record)
def make_decision(self, state: HyperAutoState, task: SubTask) -> Decision:
    """Complete decision making workflow."""
    context = self.analyze_context(state, task)
    options = self.generate_options(context)
    risk_assessments = self.assess_risks(options)
    decision = self.select_best(options)
    selected_option = next(
        (o for o in options if o.action == decision.action),
        options[0] if options else None,
    )
    record = DecisionRecord(
        context=context,
        options=options,
        risk_assessments=risk_assessments,
```

```
        selected_option=selected_option,
    )
    self.record_decision(decision, record)
    return decision
def update_decision_outcome(
    self,
    decision_id: str,
    success: bool,
    outcome: str,
    lessons: list[str] | None = None,
) -> None:
    """Update the outcome of a recorded decision."""
    for record in self._decision_history:
        if record.decision and record.decision.id == decision_id:
            record.success = success
            record.outcome = outcome
            if lessons:
                record.lessons_learned.extend(lessons)
            self._update_learned_patterns(record)
            break
def get_decision_history(self) -> list[DecisionRecord]:
    """Get all recorded decisions."""
    return self._decision_history.copy()
def get_learned_patterns(self) -> dict[str, list[str]]:
    """Get learned decision patterns."""
    return self._learned_patterns.copy()
def _identify_available_tools(self, task: SubTask) -> list[str]:
    """Identify tools available for the task."""
    tools = ["bash", "read", "write", "edit", "grep", "glob"]
    task_metadata = task.metadata
    if task_metadata.get("requires_web"):
        tools.append("webfetch")
    if task_metadata.get("requires_browser"):
        tools.append("browse")
    return tools
def _identify_available_skills(self, task: SubTask) -> list[str]:
    """Identify skills available for the task."""
    skills = ["debug", "qa", "ship", "review"]
    task_name = task.name.lower()
    if "test" in task_name:
        skills.append("qa")
    if "deploy" in task_name or "release" in task_name:
        skills.append("ship")
    if "bug" in task_name or "fix" in task_name:
        skills.append("debug")
    return skills
def _gather_project_info(self, task: SubTask) -> dict[str, Any]:
    """Gather project information."""
    return {
        "language": task.metadata.get("language", "python"),
        "framework": task.metadata.get("framework", "unknown"),
        "has_tests": task.metadata.get("has_tests", False),
        "has_docs": task.metadata.get("has_docs", False),
    }
def _identify_constraints(self, state: HyperAutoState, task: SubTask) -> dict[str, Any]:
    """Identify execution constraints."""
    constraints = {
        "time_limit": 300.0,
        "requires_confirmation": state != HyperAutoState.EXECUTING,
        "allow_parallel": task.priority < 5,
        "allow_rollback": True,
    }
    if self.config:
        constraints["time_limit"] = self.config.timeout_per_task
        constraints["requires_confirmation"] = (
```

```
            self.config.mode.value == "semi_auto" and state != HyperAutoState.EXECUTING
        )
    return constraints
def _get_relevant_history(self, task: SubTask) -> list[dict[str, Any]]:
    """Get relevant decision history."""
    relevant = []
    for record in self._decision_history[-10:]:
        if record.context and record.context.task and record.context.task.name == task.name
:
            relevant.append(record.to_dict())
    return relevant
def _classify_task_type(self, task: SubTask) -> str:
    """Classify the type of task."""
    task_type = task.metadata.get("type", "generic")
    type_mapping = {
        "skill_creation": DecisionCategory.SKILL_CREATION.value,
        "project_init": DecisionCategory.PROJECT_INIT.value,
        "code_search": DecisionCategory.EXECUTION_STRATEGY.value,
        "dependency_install": DecisionCategory.EXECUTION_STRATEGY.value,
        "test_execution": DecisionCategory.EXECUTION_STRATEGY.value,
        "code_generation": DecisionCategory.EXECUTION_STRATEGY.value,
        "code_edit": DecisionCategory.EXECUTION_STRATEGY.value,
        "git_operation": DecisionCategory.EXECUTION_STRATEGY.value,
        "generic": DecisionCategory.EXECUTION_STRATEGY.value,
    }
    return type_mapping.get(task_type, DecisionCategory.EXECUTION_STRATEGY.value)
def _generate_skill_creation_options(self, context: Context) -> list[DecisionOption]:
    """Generate options for skill creation."""
    return [
        DecisionOption(
            action="create_new_skill",
            description="Create a new skill from scratch",
            expected_outcome="New skill available for use",
            risk_level=RiskLevel.LOW,
            reasoning="Creating new skill provides reusable capability",
            category=DecisionCategory.SKILL_CREATION,
            rollback_plan="Delete created skill file",
            alternatives=["use_existing_skill", "modify_existing_skill"],
        ),
        DecisionOption(
            action="use_existing_skill",
            description="Use an existing skill if available",
            expected_outcome="Task completed with existing skill",
            risk_level=RiskLevel.LOW,
            reasoning="Using existing skill is faster and safer",
            category=DecisionCategory.SKILL_CREATION,
            alternatives=["create_new_skill"],
        ),
        DecisionOption(
            action="modify_existing_skill",
            description="Modify an existing skill for this task",
            expected_outcome="Adapted skill available",
            risk_level=RiskLevel.MEDIUM,
            reasoning="Modifying existing skill balances reuse and customization",
            category=DecisionCategory.SKILL_CREATION,
            rollback_plan="Restore original skill file",
            alternatives=["create_new_skill", "use_existing_skill"],
        ),
    ]
def _generate_project_init_options(self, context: Context) -> list[DecisionOption]:
    """Generate options for project initialization."""
    return [
        DecisionOption(
            action="full_init",
            description="Initialize complete project structure",
```

```
            expected_outcome="Full project structure created",
            risk_level=RiskLevel.MEDIUM,
            reasoning="Full initialization provides complete setup",
            category=DecisionCategory.PROJECT_INIT,
            rollback_plan="Delete created files and directories",
            alternatives=["minimal_init", "skip_init"],
        ),
        DecisionOption(
            action="minimal_init",
            description="Initialize minimal essential structure",
            expected_outcome="Basic project structure created",
            risk_level=RiskLevel.LOW,
            reasoning="Minimal initialization reduces risk",
            category=DecisionCategory.PROJECT_INIT,
            rollback_plan="Delete created files",
            alternatives=["full_init", "skip_init"],
        ),
        DecisionOption(
            action="skip_init",
            description="Skip initialization if structure exists",
            expected_outcome="No changes to project structure",
            risk_level=RiskLevel.LOW,
            reasoning="Skipping is safe if structure exists",
            category=DecisionCategory.PROJECT_INIT,
            alternatives=["full_init", "minimal_init"],
        ),
    ]
def _generate_code_search_options(self, context: Context) -> list[DecisionOption]:
    """Generate options for code search."""
    return [
        DecisionOption(
            action="broad_search",
            description="Search broadly across the codebase",
            expected_outcome="Comprehensive results",
            risk_level=RiskLevel.LOW,
            reasoning="Broad search ensures no missed references",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["targeted_search", "skip_search"],
        ),
        DecisionOption(
            action="targeted_search",
            description="Search in specific files or patterns",
            expected_outcome="Focused relevant results",
            risk_level=RiskLevel.LOW,
            reasoning="Targeted search is faster and more precise",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["broad_search"],
        ),
    ]
def _generate_error_handling_options(self, context: Context) -> list[DecisionOption]:
    """Generate options for error handling."""
    return [
        DecisionOption(
            action="retry_with_adjustment",
            description="Retry with adjusted parameters",
            expected_outcome="Task completes successfully",
            risk_level=RiskLevel.MEDIUM,
            reasoning="Retry with adjustment addresses transient issues",
            category=DecisionCategory.ERROR_HANDLING,
            rollback_plan="Revert to original parameters",
            alternatives=["rollback", "abort", "escalate"],
        ),
        DecisionOption(
            action="rollback",
            description="Rollback changes and restart",
```

```
            expected_outcome="Clean state restored",
            risk_level=RiskLevel.HIGH,
            reasoning="Rollback ensures clean state but loses progress",
            category=DecisionCategory.ERROR_HANDLING,
            alternatives=["retry_with_adjustment", "abort"],
        ),
        DecisionOption(
            action="abort",
            description="Abort task and report error",
            expected_outcome="Task terminated safely",
            risk_level=RiskLevel.LOW,
            reasoning="Aborting prevents further damage",
            category=DecisionCategory.ERROR_HANDLING,
            alternatives=["retry_with_adjustment", "rollback"],
        ),
        DecisionOption(
            action="escalate",
            description="Escalate for human intervention",
            expected_outcome="Human decision made",
            risk_level=RiskLevel.LOW,
            reasoning="Escalation is safe for critical decisions",
            category=DecisionCategory.ERROR_HANDLING,
            alternatives=["retry_with_adjustment", "abort"],
        ),
    ]
def _generate_execution_strategy_options(self, context: Context) -> list[DecisionOption]:
    """Generate options for execution strategy."""
    return [
        DecisionOption(
            action="parallel_execution",
            description="Execute tasks in parallel",
            expected_outcome="Faster completion",
            risk_level=RiskLevel.MEDIUM,
            reasoning="Parallel execution improves efficiency",
            category=DecisionCategory.EXECUTION_STRATEGY,
            rollback_plan="Switch to sequential execution",
            alternatives=["sequential_execution", "batch_execution"],
        ),
        DecisionOption(
            action="sequential_execution",
            description="Execute tasks sequentially",
            expected_outcome="Stable completion",
            risk_level=RiskLevel.LOW,
            reasoning="Sequential execution is safer",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["parallel_execution", "batch_execution"],
        ),
        DecisionOption(
            action="batch_execution",
            description="Execute in batches",
            expected_outcome="Balanced completion",
            risk_level=RiskLevel.LOW,
            reasoning="Batch execution balances speed and safety",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["parallel_execution", "sequential_execution"],
        ),
    ]
def _generate_generic_options(self, context: Context) -> list[DecisionOption]:
    """Generate generic decision options."""
    return [
        DecisionOption(
            action="proceed_standard",
            description="Proceed with standard approach",
            expected_outcome="Task completed normally",
            risk_level=RiskLevel.LOW,
```

```
            reasoning="Standard approach is reliable",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["proceed_careful", "request_guidance"],
        ),
        DecisionOption(
            action="proceed_careful",
            description="Proceed with extra caution",
            expected_outcome="Task completed safely",
            risk_level=RiskLevel.LOW,
            reasoning="Careful approach minimizes risk",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["proceed_standard", "request_guidance"],
        ),
        DecisionOption(
            action="request_guidance",
            description="Request human guidance",
            expected_outcome="Human direction provided",
            risk_level=RiskLevel.LOW,
            reasoning="Human guidance ensures correctness",
            category=DecisionCategory.EXECUTION_STRATEGY,
            alternatives=["proceed_standard", "proceed_careful"],
        ),
    ]
def _assess_single_option(self, option: DecisionOption) -> RiskAssessment:
    """Assess risk for a single option."""
    risk_factors: list[str] = []
    mitigation_strategies: list[str] = []
    if option.risk_level == RiskLevel.CRITICAL:
        risk_factors.extend(["high_failure_probability", "significant_impact"])
        mitigation_strategies.extend(["require_confirmation", "create_backup"])
    elif option.risk_level == RiskLevel.HIGH:
        risk_factors.extend(["moderate_failure_probability", "moderate_impact"])
        mitigation_strategies.extend(["add_validation", "enable rollback"])
    elif option.risk_level == RiskLevel.MEDIUM:
        risk_factors.append("low_failure_probability")
        mitigation_strategies.append("monitor_progress")
    else:
        risk_factors.append("minimal_risk")
        mitigation_strategies.append("standard_logging")
    if not option.rollback_plan:
        risk_factors.append("no_rollback_plan")
        mitigation_strategies.append("create_checkpoint")
    impact_score = self._risk_thresholds[option.risk_level]
    probability_score = 1.0 - option.confidence
    overall_risk_score = impact_score * probability_score
    return RiskAssessment(
        option_id=option.action,
        risk_level=option.risk_level,
        risk_factors=risk_factors,
        mitigation_strategies=mitigation_strategies,
        impact_score=impact_score,
        probability_score=probability_score,
        overall_risk_score=overall_risk_score,
    )
def _calculate_selection_score(self, option: DecisionOption) -> float:
    """Calculate overall selection score for an option."""
    confidence_weight = 0.4
    risk_weight = 0.3
    outcome_weight = 0.2
    rollback_weight = 0.1
    risk_score = 1.0 - self._risk_thresholds[option.risk_level]
    outcome_score = 0.8 if option.expected_outcome else 0.5
    rollback_score = 1.0 if option.rollback_plan else 0.5
    total_score = (
        option.confidence * confidence_weight
```

```
        + risk_score * risk_weight
        + outcome_score * outcome_weight
        + rollback_score * rollback_weight
    )
    historical_bonus = self._get_historical_success_rate(option.action)
    total_score += historical_bonus * 0.1
    return total_score
def _calculate_context_match(self, option: DecisionOption, context: Context) -> float:
    """Calculate how well option matches context."""
    score = 0.5
    if option.category.value == context.metadata.get("task_type"):
        score += 0.3
    required_tools = option.parameters.get("required_tools", [])
    if all(tool in context.available_tools for tool in required_tools):
        score += 0.2
    return min(score, 1.0)
def _check_resource_availability(self, option: DecisionOption, context: Context) -> float:
    """Check if resources are available for option."""
    score = 1.0
    if option.parameters.get("requires_web") and "webfetch" not in context.available_tools:
        score -= 0.3
    if option.parameters.get("requires_browser") and "browse" not in context.available_tool
s:
        score -= 0.3
    if option.parameters.get("requires_skill"):
        required_skill = option.parameters.get("requires_skill")
        if required_skill not in context.available_skills:
            score -= 0.2
    return max(score, 0.0)
def _check_constraint_compliance(self, option: DecisionOption, context: Context) -> float:
    """Check if option complies with constraints."""
    score = 1.0
    constraints = context.constraints
    if constraints.get("requires_confirmation") and option.risk_level in (
        RiskLevel.HIGH,
        RiskLevel.CRITICAL,
    ):
        score -= 0.3
    if option.estimated_duration > constraints.get("time_limit", 300.0):
        score -= 0.2
    if not constraints.get("allow_parallel") and option.action == "parallel_execution":
        score -= 0.4
    if not constraints.get("allow_rollback") and option.rollback_plan:
        score -= 0.1
    return max(score, 0.0)
def _map_category_to_decision_type(self, category: DecisionCategory) -> DecisionType:
    """Map decision category to decision type."""
    mapping = {
        DecisionCategory.SKILL_CREATION: DecisionType.SKILL_CREATION,
        DecisionCategory.PROJECT_INIT: DecisionType.PROJECT_INIT,
        DecisionCategory.ERROR_HANDLING: DecisionType.ABORT,
        DecisionCategory.EXECUTION_STRATEGY: DecisionType.TEST_EXECUTION,
        DecisionCategory.TOOL_SELECTION: DecisionType.CODE_SEARCH,
        DecisionCategory.RESOURCE_ALLOCATION: DecisionType.DEPENDENCY_INSTALL,
    }
    return mapping.get(category, DecisionType.SKILL_CREATION)
def _get_historical_confidence(self, action: str) -> float:
    """Get historical confidence for an action."""
    matching_records = [
        r
        for r in self._decision_history
        if r.selected_option and r.selected_option.action == action
    ]
    if not matching_records:
        return 0.0
```

```
    total_confidence = sum(
        r.selected_option.confidence for r in matching_records if r.selected_option
    )
    return total_confidence / len(matching_records)
def _get_historical_success_rate(self, action: str) -> float:
    """Get historical success rate for an action."""
    matching_records = [
        r
        for r in self._decision_history
        if r.selected_option and r.selected_option.action == action
    ]
    if not matching_records:
        return 0.0
    successful = sum(1 for r in matching_records if r.success)
    return successful / len(matching_records)
def _update_learned_patterns(self, record: DecisionRecord) -> None:
    """Update learned patterns from a decision record."""
    if not record.success or not record.selected_option:
        return
    category = record.selected_option.category.value
    action = record.selected_option.action
    if category not in self._learned_patterns:
        self._learned_patterns[category] = []
    pattern = f"{action}_successful"
    if pattern not in self._learned_patterns[category]:
        self._learned_patterns[category].append(pattern)
    for lesson in record.lessons_learned:
        if lesson not in self._learned_patterns.get("lessons", []):
            if "lessons" not in self._learned_patterns:
                self._learned_patterns["lessons"] = []
            self._learned_patterns["lessons"].append(lesson)
```

File: [81/252] openlaoke/core/hyperauto/executor.py

```
"""Tool executor for HyperAuto - executes tools based on AI decisions."""
from __future__ import annotations
import asyncio
import json
from typing import TYPE_CHECKING, Any
from openlaoke.types.core_types import ToolResultBlock
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
    from openlaoke.core.tool import ToolRegistry
class ToolExecutor:
    """Executes tools for HyperAuto agent."""
    def __init__(self, app_state: AppState, tool_registry: ToolRegistry | None = None):
        self.app_state = app_state
        self._tool_registry = tool_registry
    async def get_tool_registry(self) -> ToolRegistry:
        """Get or create tool registry."""
        if self._tool_registry is None:
            from openlaoke.core.tool import ToolRegistry
            from openlaoke.tools.register import register_all_tools
            self._tool_registry = ToolRegistry()
            register_all_tools(self._tool_registry)
        return self._tool_registry
    async def execute_tool(self, tool_name: str, tool_input: dict[str, Any]) -> ToolResultBlock
    :
        """Execute a single tool call."""
        try:
            registry = await self.get_tool_registry()
            tool = await registry.get_async(tool_name)
            if not tool:
                return ToolResultBlock(
                    tool_use_id="unknown",
                    content=f"Error: Tool '{tool_name}' not found",
                    is_error=True,
                )
            from openlaoke.core.tool import ToolContext
            ctx = ToolContext(
                app_state=self.app_state,
                tool_use_id=f"ha_tool_{asyncio.get_event_loop().time()}",
            )
            result = await tool.call(ctx, **tool_input)
            return result
        except Exception as e:
            return ToolResultBlock(
                tool_use_id="error",
                content=f"Exception executing tool: {e}",
                is_error=True,
            )
    async def execute_tool_calls(self, tool_calls: list[dict[str, Any]]) -> list[ToolResultBloc
    k]:
        """Execute multiple tool calls in sequence."""
        results = []
        for tool_call in tool_calls:
            tool_name = tool_call.get("name", tool_call.get("function", {}).get("name", ""))
            tool_input = tool_call.get("input", tool_call.get("function", {}).get("arguments",
    {}))
            if isinstance(tool_input, str):
                try:
                    tool_input = json.loads(tool_input)
                except json.JSONDecodeError:
                    tool_input = {}
            result = await self.execute_tool(tool_name, tool_input)
```

```
        results.append(result)
    return results
async def get_tool_schemas(self) -> list[dict[str, Any]]:
    """Get schemas for all available tools."""
    registry = await self.get_tool_registry()
    tools = await registry.get_all_async()
    schemas = []
    for tool in tools:
        schema = {
            "name": tool.name,
            "description": tool.description,
            "input_schema": tool.get_input_schema(),
        }
        schemas.append(schema)
    return schemas
```

File: [82/252] openlaoke/core/hyperauto/learning.py

```
"""Automatic learning system for experience collection and knowledge extraction."""
from __future__ import annotations
import json
import time
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any
from uuid import uuid4
from openlaoke.core.hyperauto.workflow import WorkflowResult
class ExperienceType(StrEnum):
    """Types of experiences collected."""
    SUCCESS = "success"
    FAILURE = "failure"
    PARTIAL = "partial"
    LEARNED = "learned"
class KnowledgeCategory(StrEnum):
    """Categories of extracted knowledge."""
    TASK_PATTERN = "task_pattern"
    ERROR_RESOLUTION = "error_resolution"
    OPTIMIZATION = "optimization"
    BEST_PRACTICE = "best_practice"
    TOOL_USAGE = "tool_usage"
    WORKFLOW_EFFICIENCY = "workflow_efficiency"
class PatternType(StrEnum):
    """Types of recognized patterns."""
    SEQUENTIAL = "sequential"
    PARALLEL = "parallel"
    CONDITIONAL = "conditional"
    ITERATIVE = "iterative"
    ADAPTIVE = "adaptive"
class StrategyType(StrEnum):
    """Types of execution strategies."""
    STANDARD = "standard"
    OPTIMIZED = "optimized"
    AGGRESSIVE = "aggressive"
    CAUTIOUS = "cautious"
    ADAPTIVE = "adaptive"
@dataclass
class Experience:
    """Collected experience from workflow execution."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    workflow_id: str = ""
    task: str = ""
    type: ExperienceType = ExperienceType.SUCCESS
    timestamp: float = field(default_factory=time.time)
    duration: float = 0.0
    steps: list[str] = field(default_factory=list)
    tools_used: list[str] = field(default_factory=list)
    decisions: list[str] = field(default_factory=list)
    outcome: str = ""
    context: dict[str, Any] = field(default_factory=dict)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "workflow_id": self.workflow_id,
            "task": self.task,
            "type": self.type.value,
            "timestamp": self.timestamp,
            "duration": self.duration,
```

```
            "steps": self.steps,
            "tools_used": self.tools_used,
            "decisions": self.decisions,
            "outcome": self.outcome,
            "context": self.context,
            "metadata": self.metadata,
        }
@dataclass
class Knowledge:
    """Extracted knowledge from experiences."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    category: KnowledgeCategory = KnowledgeCategory.TASK_PATTERN
    title: str = ""
    description: str = ""
    source_experiences: list[str] = field(default_factory=list)
    confidence: float = 0.0
    applicability: list[str] = field(default_factory=list)
    conditions: dict[str, Any] = field(default_factory=dict)
    recommendations: list[str] = field(default_factory=list)
    created_at: float = field(default_factory=time.time)
    updated_at: float = field(default_factory=time.time)
    usage_count: int = 0
    success_rate: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "category": self.category.value,
            "title": self.title,
            "description": self.description,
            "source_experiences": self.source_experiences,
            "confidence": self.confidence,
            "applicability": self.applicability,
            "conditions": self.conditions,
            "recommendations": self.recommendations,
            "created_at": self.created_at,
            "updated_at": self.updated_at,
            "usage_count": self.usage_count,
            "success_rate": self.success_rate,
            "metadata": self.metadata,
        }
@dataclass
class Pattern:
    """Recognized execution pattern."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    type: PatternType = PatternType.SEQUENTIAL
    name: str = ""
    description: str = ""
    sequence: list[str] = field(default_factory=list)
    conditions: dict[str, Any] = field(default_factory=dict)
    success_rate: float = 0.0
    avg_duration: float = 0.0
    sample_count: int = 0
    tags: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type.value,
            "name": self.name,
            "description": self.description,
            "sequence": self.sequence,
            "conditions": self.conditions,
            "success_rate": self.success_rate,
            "avg_duration": self.avg_duration,
```

```
            "sample_count": self.sample_count,
            "tags": self.tags,
            "metadata": self.metadata,
        }
@dataclass
class Strategy:
    """Execution strategy derived from patterns."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    type: StrategyType = StrategyType.STANDARD
    name: str = ""
    description: str = ""
    steps: list[dict[str, Any]] = field(default_factory=list)
    priority_order: list[str] = field(default_factory=list)
    estimated_duration: float = 0.0
    confidence: float = 0.0
    based_on_patterns: list[str] = field(default_factory=list)
    conditions: dict[str, Any] = field(default_factory=dict)
    alternatives: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type.value,
            "name": self.name,
            "description": self.description,
            "steps": self.steps,
            "priority_order": self.priority_order,
            "estimated_duration": self.estimated_duration,
            "confidence": self.confidence,
            "based_on_patterns": self.based_on_patterns,
            "conditions": self.conditions,
            "alternatives": self.alternatives,
            "metadata": self.metadata,
        }
@dataclass
class EnhancedStrategy:
    """Enhanced strategy with applied learning."""
    base_strategy: Strategy
    enhancements: list[str] = field(default_factory=list)
    applied_knowledge: list[str] = field(default_factory=list)
    confidence_boost: float = 0.0
    expected_improvement: float = 0.0
    reasoning: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "base_strategy": self.base_strategy.to_dict(),
            "enhancements": self.enhancements,
            "applied_knowledge": self.applied_knowledge,
            "confidence_boost": self.confidence_boost,
            "expected_improvement": self.expected_improvement,
            "reasoning": self.reasoning,
            "metadata": self.metadata,
        }
@dataclass
class LearningSession:
    """A complete learning session."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    start_time: float = field(default_factory=time.time)
    end_time: float | None = None
    experiences_collected: int = 0
    knowledge_extracted: int = 0
    patterns_recognized: int = 0
    strategies_optimized: int = 0
    knowledge_persisted: bool = False
```

```
    summary: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "start_time": self.start_time,
            "end_time": self.end_time,
            "experiences_collected": self.experiences_collected,
            "knowledge_extracted": self.knowledge_extracted,
            "patterns_recognized": self.patterns_recognized,
            "strategies_optimized": self.strategies_optimized,
            "knowledge_persisted": self.knowledge_persisted,
            "summary": self.summary,
            "metadata": self.metadata,
        }
class LearningSystem:
    """Automatic learning system for experience collection and knowledge extraction.
    Features:
    - Experience collection from workflow execution
    - Knowledge extraction from success/failure cases
    - Pattern recognition for successful execution
    - Strategy optimization for future execution
    - Knowledge persistence for long-term learning
    """
    def __init__(self, storage_dir: Path | None = None) -> None:
        self._storage_dir = storage_dir or Path.home() / ".openlaoke" / "learning"
        self._storage_dir.mkdir(parents=True, exist_ok=True)
        self._experiences: list[Experience] = []
        self._knowledge: dict[str, Knowledge] = {}
        self._patterns: dict[str, Pattern] = {}
        self._strategies: dict[str, Strategy] = {}
        self._session: LearningSession | None = None
        self._load_knowledge()
    def collect_experience(self, workflow_result: WorkflowResult) -> Experience:
        """Collect experience data from workflow execution."""
        exp_type = ExperienceType.SUCCESS if workflow_result.success else ExperienceType.FAILUR
    E
        if not workflow_result.success and workflow_result.error:
            exp_type = ExperienceType.FAILURE
        elif workflow_result.success and len(workflow_result.execution_results) > 0:
            partial_failures = [r for r in workflow_result.execution_results if not r.success]
            if partial_failures:
                exp_type = ExperienceType.PARTIAL
        steps = [step.task_id for step in workflow_result.execution_plan.steps]
        tools_used = self._extract_tools_used(workflow_result)
        decisions = self._extract_decisions(workflow_result)
        outcome = self._determine_outcome(workflow_result)
        context = workflow_result.metadata
        experience = Experience(
            workflow_id=workflow_result.workflow_id,
            task=workflow_result.task_tree.root_task,
            type=exp_type,
            duration=workflow_result.total_duration,
            steps=steps,
            tools_used=tools_used,
            decisions=decisions,
            outcome=outcome,
            context=context,
        )
        self._experiences.append(experience)
        if self._session:
            self._session.experiences_collected += 1
        return experience
    def extract_knowledge(self, experiences: list[Experience]) -> Knowledge:
        """Extract knowledge from collected experiences."""
```

```
    successful = [e for e in experiences if e.type == ExperienceType.SUCCESS]
    failed = [e for e in experiences if e.type == ExperienceType.FAILURE]
    category = self._determine_knowledge_category(experiences)
    title = self._generate_knowledge_title(experiences)
    description = self._generate_knowledge_description(experiences)
    source_ids = [e.id for e in experiences]
    confidence = self._calculate_confidence(experiences)
    applicability = self._determine_applicability(experiences)
    conditions = self._extract_conditions(experiences)
    recommendations = self._generate_recommendations(successful, failed)
    knowledge = Knowledge(
        category=category,
        title=title,
        description=description,
        source_experiences=source_ids,
        confidence=confidence,
        applicability=applicability,
        conditions=conditions,
        recommendations=recommendations,
    )
    self._knowledge[knowledge.id] = knowledge
    if self._session:
        self._session.knowledge_extracted += 1
    return knowledge
def recognize_patterns(self, knowledge: Knowledge) -> list[Pattern]:
    """Recognize execution patterns from knowledge."""
    patterns: list[Pattern] = []
    related_experiences = [e for e in self._experiences if e.id in knowledge.source_experie
nces]
    if not related_experiences:
        return patterns
    sequential_pattern = self._extract_sequential_pattern(related_experiences)
    if sequential_pattern:
        patterns.append(sequential_pattern)
    parallel_pattern = self._extract_parallel_pattern(related_experiences)
    if parallel_pattern:
        patterns.append(parallel_pattern)
    conditional_pattern = self._extract_conditional_pattern(related_experiences)
    if conditional_pattern:
        patterns.append(conditional_pattern)
    for pattern in patterns:
        self._patterns[pattern.id] = pattern
    if self._session:
        self._session.patterns_recognized += len(patterns)
    return patterns
def optimize_strategy(self, patterns: list[Pattern]) -> Strategy:
    """Optimize execution strategy based on recognized patterns."""
    if not patterns:
        return Strategy(
            type=StrategyType.STANDARD,
            name="default_strategy",
            description="Default strategy with no pattern data",
        )
    best_pattern = max(patterns, key=lambda p: p.success_rate)
    strategy_type = self._determine_strategy_type(best_pattern)
    name = f"optimized_{best_pattern.name}"
    description = f"Strategy optimized from pattern: {best_pattern.description}"
    steps = self._generate_strategy_steps(best_pattern)
    priority_order = best_pattern.sequence.copy()
    estimated_duration = best_pattern.avg_duration
    confidence = best_pattern.success_rate
    based_on = [p.id for p in patterns]
    conditions = best_pattern.conditions.copy()
    alternatives = self._generate_alternatives(patterns)
    strategy = Strategy(
```

```
        type=strategy_type,
        name=name,
        description=description,
        steps=steps,
        priority_order=priority_order,
        estimated_duration=estimated_duration,
        confidence=confidence,
        based_on_patterns=based_on,
        conditions=conditions,
        alternatives=alternatives,
    )
    self._strategies[strategy.id] = strategy
    if self._session:
        self._session.strategies_optimized += 1
    return strategy
def persist_knowledge(self, knowledge: Knowledge) -> bool:
    """Persist learned knowledge to storage."""
    try:
        knowledge_file = self._storage_dir / f"knowledge_{knowledge.id}.json"
        with open(knowledge_file, "w", encoding="utf-8") as f:
            json.dump(knowledge.to_dict(), f, ensure_ascii=False, indent=2)
        patterns_file = self._storage_dir / "patterns.json"
        patterns_data = {pid: p.to_dict() for pid, p in self._patterns.items()}
        with open(patterns_file, "w", encoding="utf-8") as f:
            json.dump(patterns_data, f, ensure_ascii=False, indent=2)
        strategies_file = self._storage_dir / "strategies.json"
        strategies_data = {sid: s.to_dict() for sid, s in self._strategies.items()}
        with open(strategies_file, "w", encoding="utf-8") as f:
            json.dump(strategies_data, f, ensure_ascii=False, indent=2)
        if self._session:
            self._session.knowledge_persisted = True
        return True
    except Exception:
        return False
def load_knowledge(self) -> Knowledge:
    """Load persisted knowledge from storage."""
    self._load_knowledge()
    if self._knowledge:
        latest = max(self._knowledge.values(), key=lambda k: k.updated_at)
        return latest
    return Knowledge(
        category=KnowledgeCategory.TASK_PATTERN,
        title="empty_knowledge",
        description="No knowledge loaded from storage",
    )
def apply_learning(self, task: str) -> EnhancedStrategy:
    """Apply learned knowledge to enhance strategy for a task."""
    base_strategy = self._find_best_strategy(task)
    if not base_strategy:
        base_strategy = Strategy(
            type=StrategyType.STANDARD,
            name="default",
            description="Default strategy when no learned knowledge available",
        )
    relevant_knowledge = self._find_relevant_knowledge(task)
    enhancements: list[str] = []
    applied_ids: list[str] = []
    confidence_boost = 0.0
    for knowledge in relevant_knowledge:
        if self._is_applicable(knowledge, task):
            enhancements.extend(knowledge.recommendations[:2])
            applied_ids.append(knowledge.id)
            confidence_boost += knowledge.confidence * 0.1
            knowledge.usage_count += 1
    expected_improvement = min(confidence_boost * 0.5, 0.3)
```

```
    reasoning = self._generate_enhancement_reasoning(relevant_knowledge, task)
    return EnhancedStrategy(
        base_strategy=base_strategy,
        enhancements=enhancements,
        applied_knowledge=applied_ids,
        confidence_boost=min(confidence_boost, 0.5),
        expected_improvement=expected_improvement,
        reasoning=reasoning,
    )
def start_session(self) -> LearningSession:
    """Start a new learning session."""
    self._session = LearningSession()
    return self._session
def end_session(self) -> LearningSession | None:
    """End the current learning session."""
    if self._session:
        self._session.end_time = time.time()
        self._session.summary = self._generate_session_summary()
        self._persist_session()
    return self._session
def get_all_experiences(self) -> list[Experience]:
    """Get all collected experiences."""
    return self._experiences.copy()
def get_all_knowledge(self) -> dict[str, Knowledge]:
    """Get all extracted knowledge."""
    return self._knowledge.copy()
def get_all_patterns(self) -> dict[str, Pattern]:
    """Get all recognized patterns."""
    return self._patterns.copy()
def get_all_strategies(self) -> dict[str, Strategy]:
    """Get all optimized strategies."""
    return self._strategies.copy()
def _load_knowledge(self) -> None:
    """Load persisted knowledge from storage."""
    try:
        patterns_file = self._storage_dir / "patterns.json"
        if patterns_file.exists():
            with open(patterns_file, encoding="utf-8") as f:
                patterns_data = json.load(f)
            for pid, pdata in patterns_data.items():
                self._patterns[pid] = Pattern(
                    id=pid,
                    type=PatternType(pdata.get("type", PatternType.SEQUENTIAL.value)),
                    name=pdata.get("name", ""),
                    description=pdata.get("description", ""),
                    sequence=pdata.get("sequence", []),
                    conditions=pdata.get("conditions", {}),
                    success_rate=pdata.get("success_rate", 0.0),
                    avg_duration=pdata.get("avg_duration", 0.0),
                    sample_count=pdata.get("sample_count", 0),
                    tags=pdata.get("tags", []),
                    metadata=pdata.get("metadata", {}),
                )
        strategies_file = self._storage_dir / "strategies.json"
        if strategies_file.exists():
            with open(strategies_file, encoding="utf-8") as f:
                strategies_data = json.load(f)
            for sid, sdata in strategies_data.items():
                self._strategies[sid] = Strategy(
                    id=sid,
                    type=StrategyType(sdata.get("type", StrategyType.STANDARD.value)),
                    name=sdata.get("name", ""),
                    description=sdata.get("description", ""),
                    steps=sdata.get("steps", []),
                    priority_order=sdata.get("priority_order", []),
```

```
                    estimated_duration=sdata.get("estimated_duration", 0.0),
                    confidence=sdata.get("confidence", 0.0),
                    based_on_patterns=sdata.get("based_on_patterns", []),
                    conditions=sdata.get("conditions", {}),
                    alternatives=sdata.get("alternatives", []),
                    metadata=sdata.get("metadata", {}),
                )
        for kf in self._storage_dir.glob("knowledge_*.json"):
            with open(kf, encoding="utf-8") as f:
                kdata = json.load(f)
            kid = kdata.get("id", kf.stem.replace("knowledge_", ""))
            self._knowledge[kid] = Knowledge(
                id=kid,
                category=KnowledgeCategory(
                    kdata.get("category", KnowledgeCategory.TASK_PATTERN.value)
                ),
                title=kdata.get("title", ""),
                description=kdata.get("description", ""),
                source_experiences=kdata.get("source_experiences", []),
                confidence=kdata.get("confidence", 0.0),
                applicability=kdata.get("applicability", []),
                conditions=kdata.get("conditions", {}),
                recommendations=kdata.get("recommendations", []),
                created_at=kdata.get("created_at", 0.0),
                updated_at=kdata.get("updated_at", 0.0),
                usage_count=kdata.get("usage_count", 0),
                success_rate=kdata.get("success_rate", 0.0),
                metadata=kdata.get("metadata", {}),
            )
    except Exception:
        pass
def _extract_tools_used(self, result: WorkflowResult) -> list[str]:
    """Extract tools used from workflow result."""
    tools: set[str] = set()
    for step in result.execution_plan.steps:
        metadata = step.metadata
        if "tools" in metadata:
            tools.update(metadata["tools"])
    return list(tools)
def _extract_decisions(self, result: WorkflowResult) -> list[str]:
    """Extract decisions from workflow result."""
    decisions: list[str] = []
    context = result.metadata.get("context", {})
    if "decisions" in context:
        for d in context["decisions"]:
            decisions.append(d.get("type", "unknown"))
    return decisions
def _determine_outcome(self, result: WorkflowResult) -> str:
    """Determine outcome description from workflow result."""
    if result.success:
        return f"Successfully completed {len(result.execution_results)} tasks"
    else:
        if result.error:
            return f"Failed: {result.error}"
        return f"Partially completed {len([r for r in result.execution_results if r.success
])} tasks"
def _determine_knowledge_category(self, experiences: list[Experience]) -> KnowledgeCategory
:
    """Determine knowledge category from experiences."""
    successful = [e for e in experiences if e.type == ExperienceType.SUCCESS]
    failed = [e for e in experiences if e.type == ExperienceType.FAILURE]
    if failed and not successful:
        return KnowledgeCategory.ERROR_RESOLUTION
    elif len(successful) > len(failed):
        return KnowledgeCategory.BEST_PRACTICE
```

```
    else:
        return KnowledgeCategory.TASK_PATTERN
def _generate_knowledge_title(self, experiences: list[Experience]) -> str:
    """Generate knowledge title from experiences."""
    if not experiences:
        return "unknown_knowledge"
    tasks = [e.task for e in experiences]
    common_task = self._find_common_task(tasks)
    exp_types = [e.type for e in experiences]
    if ExperienceType.FAILURE in exp_types:
        return f"Error handling for {common_task}"
    elif ExperienceType.SUCCESS in exp_types:
        return f"Success pattern for {common_task}"
    else:
        return f"Pattern for {common_task}"
def _generate_knowledge_description(self, experiences: list[Experience]) -> str:
    """Generate knowledge description from experiences."""
    if not experiences:
        return "No experiences to describe"
    avg_duration = sum(e.duration for e in experiences) / len(experiences)
    success_count = len([e for e in experiences if e.type == ExperienceType.SUCCESS])
    return f"Derived from {len(experiences)} experiences. "
    f"Average duration: {avg_duration:.2f}s. "
    f"Success rate: {success_count / len(experiences):.2%}"
def _calculate_confidence(self, experiences: list[Experience]) -> float:
    """Calculate confidence level from experiences."""
    if not experiences:
        return 0.0
    success_weight = 0.8
    sample_weight = 0.2
    success_rate = len([e for e in experiences if e.type == ExperienceType.SUCCESS]) / len(
        experiences
    )
    sample_factor = min(len(experiences) / 10, 1.0)
    return success_rate * success_weight + sample_factor * sample_weight
def _determine_applicability(self, experiences: list[Experience]) -> list[str]:
    """Determine applicability scope from experiences."""
    tasks = [e.task for e in experiences]
    keywords: set[str] = set()
    for task in tasks:
        words = task.lower().split()
        keywords.update([w for w in words if len(w) > 3])
    return list(keywords)[:10]
def _extract_conditions(self, experiences: list[Experience]) -> dict[str, Any]:
    """Extract conditions from experiences."""
    conditions: dict[str, Any] = {}
    durations = [e.duration for e in experiences]
    if durations:
        conditions["avg_duration"] = sum(durations) / len(durations)
        conditions["max_duration"] = max(durations)
    tools: set[str] = set()
    for e in experiences:
        tools.update(e.tools_used)
    conditions["required_tools"] = list(tools)
    return conditions
def _generate_recommendations(
    self, successful: list[Experience], failed: list[Experience]
) -> list[str]:
    """Generate recommendations from successful and failed experiences."""
    recommendations: list[str] = []
    if successful:
        common_tools = self._find_common_tools(successful)
        if common_tools:
            recommendations.append(f"Use tools: {', '.join(common_tools[:3])}")
        common_steps = self._find_common_steps(successful)
```

```
        if common_steps:
            recommendations.append(f"Follow sequence: {' -> '.join(common_steps[:3])}")
    if failed:
        error_patterns = self._extract_error_patterns(failed)
        for pattern in error_patterns[:2]:
            recommendations.append(f"Avoid: {pattern}")
    return recommendations
def _extract_sequential_pattern(self, experiences: list[Experience]) -> Pattern | None:
    """Extract sequential execution pattern."""
    sequences: list[list[str]] = []
    for e in experiences:
        if e.steps:
            sequences.append(e.steps)
    if not sequences:
        return None
    common_sequence = self._find_longest_common_sequence(sequences)
    if not common_sequence:
        return None
    success_exps = [e for e in experiences if e.type == ExperienceType.SUCCESS and e.steps]
    success_rate = len(success_exps) / len(experiences) if experiences else 0.0
    avg_duration = (
        sum(e.duration for e in experiences) / len(experiences) if experiences else 0.0
    )
    return Pattern(
        type=PatternType.SEQUENTIAL,
        name="sequential_execution",
        description=f"Sequential execution pattern with {len(common_sequence)} steps",
        sequence=common_sequence,
        success_rate=success_rate,
        avg_duration=avg_duration,
        sample_count=len(experiences),
    )
def _extract_parallel_pattern(self, experiences: list[Experience]) -> Pattern | None:
    """Extract parallel execution pattern."""
    parallel_groups: dict[str, int] = {}
    for e in experiences:
        context = e.context
        if "parallel_groups" in context:
            for group in context["parallel_groups"]:
                parallel_groups[group] = parallel_groups.get(group, 0) + 1
    if not parallel_groups:
        return None
    common_parallel = [g for g, c in parallel_groups.items() if c >= len(experiences) / 2]
    if not common_parallel:
        return None
    success_exps = [e for e in experiences if e.type == ExperienceType.SUCCESS]
    success_rate = len(success_exps) / len(experiences) if experiences else 0.0
    return Pattern(
        type=PatternType.PARALLEL,
        name="parallel_execution",
        description=f"Parallel execution pattern with {len(common_parallel)} groups",
        sequence=common_parallel,
        success_rate=success_rate,
        avg_duration=sum(e.duration for e in experiences) / len(experiences),
        sample_count=len(experiences),
        conditions={"parallel_groups": common_parallel},
    )
def _extract_conditional_pattern(self, experiences: list[Experience]) -> Pattern | None:
    """Extract conditional execution pattern."""
    conditions_map: dict[str, list[str]] = {}
    for e in experiences:
        if e.decisions:
            for decision in e.decisions:
                if decision not in conditions_map:
                    conditions_map[decision] = []
```

```
                conditions_map[decision].append(e.id)
    if not conditions_map:
        return None
    frequent_conditions = {
        d: ids for d, ids in conditions_map.items() if len(ids) >= len(experiences) / 3
    }
    if not frequent_conditions:
        return None
    success_exps = [e for e in experiences if e.type == ExperienceType.SUCCESS]
    success_rate = len(success_exps) / len(experiences) if experiences else 0.0
    return Pattern(
        type=PatternType.CONDITIONAL,
        name="conditional_execution",
        description=f"Conditional execution based on {len(frequent_conditions)} decision ty
pes",
        sequence=list(frequent_conditions.keys()),
        success_rate=success_rate,
        avg_duration=sum(e.duration for e in experiences) / len(experiences),
        sample_count=len(experiences),
        conditions={"decision_types": list(frequent_conditions.keys())},
    )
def _determine_strategy_type(self, pattern: Pattern) -> StrategyType:
    """Determine strategy type from pattern."""
    if pattern.success_rate >= 0.9:
        return StrategyType.AGGRESSIVE
    elif pattern.success_rate >= 0.7:
        return StrategyType.OPTIMIZED
    elif pattern.success_rate >= 0.5:
        return StrategyType.STANDARD
    else:
        return StrategyType.CAUTIOUS
def _generate_strategy_steps(self, pattern: Pattern) -> list[dict[str, Any]]:
    """Generate strategy steps from pattern."""
    steps: list[dict[str, Any]] = []
    for i, step_id in enumerate(pattern.sequence):
        steps.append(
            {
                "step_id": step_id,
                "order": i,
                "type": pattern.type.value,
                "required": True,
            }
        )
    return steps
def _generate_alternatives(self, patterns: list[Pattern]) -> list[str]:
    """Generate alternative strategies from patterns."""
    alternatives: list[str] = []
    sorted_patterns = sorted(patterns, key=lambda p: p.success_rate, reverse=True)
    for pattern in sorted_patterns[1:3]:
        alternatives.append(pattern.name)
    return alternatives
def _find_best_strategy(self, task: str) -> Strategy | None:
    """Find the best matching strategy for a task."""
    task_lower = task.lower()
    best_match: Strategy | None = None
    best_score = 0.0
    for strategy in self._strategies.values():
        score = self._calculate_strategy_match(strategy, task_lower)
        if score > best_score:
            best_score = score
            best_match = strategy
    return best_match
def _find_relevant_knowledge(self, task: str) -> list[Knowledge]:
    """Find knowledge relevant to a task."""
    task_lower = task.lower()
```

```
    relevant: list[Knowledge] = []
    for knowledge in self._knowledge.values():
        for app in knowledge.applicability:
            if app in task_lower:
                relevant.append(knowledge)
                break
    return sorted(relevant, key=lambda k: k.confidence, reverse=True)[:5]
def _is_applicable(self, knowledge: Knowledge, task: str) -> bool:
    """Check if knowledge is applicable to a task."""
    task_lower = task.lower()
    return any(app in task_lower for app in knowledge.applicability)
def _generate_enhancement_reasoning(self, knowledge: list[Knowledge], task: str) -> str:
    """Generate reasoning for enhancements."""
    if not knowledge:
        return "No specific knowledge applied"
    titles = [k.title for k in knowledge[:3]]
    return f"Applied knowledge from: {', '.join(titles)}"
def _find_common_task(self, tasks: list[str]) -> str:
    """Find common task pattern."""
    if not tasks:
        return "unknown"
    words: dict[str, int] = {}
    for task in tasks:
        for word in task.lower().split():
            if len(word) > 3:
                words[word] = words.get(word, 0) + 1
    if not words:
        return tasks[0]
    return max(words, key=lambda w: words.get(w, 0))
def _find_common_tools(self, experiences: list[Experience]) -> list[str]:
    """Find commonly used tools."""
    tool_counts: dict[str, int] = {}
    for e in experiences:
        for tool in e.tools_used:
            tool_counts[tool] = tool_counts.get(tool, 0) + 1
    return [t for t, c in sorted(tool_counts.items(), key=lambda x: x[1], reverse=True)][:5
]
def _find_common_steps(self, experiences: list[Experience]) -> list[str]:
    """Find common execution steps."""
    step_counts: dict[str, int] = {}
    for e in experiences:
        for step in e.steps:
            step_counts[step] = step_counts.get(step, 0) + 1
    return [s for s, c in sorted(step_counts.items(), key=lambda x: x[1], reverse=True)][:5
]
def _find_longest_common_sequence(self, sequences: list[list[str]]) -> list[str]:
    """Find longest common sequence among sequences."""
    if not sequences:
        return []
    if len(sequences) == 1:
        return sequences[0]
    common: list[str] = []
    first = sequences[0]
    for i in range(len(first)):
        if all(len(s) > i and s[i] == first[i] for s in sequences):
            common.append(first[i])
        else:
            break
    return common
def _extract_error_patterns(self, experiences: list[Experience]) -> list[str]:
    """Extract error patterns from failed experiences."""
    patterns: list[str] = []
    for e in experiences:
        if e.type == ExperienceType.FAILURE and e.outcome:
            patterns.append(e.outcome[:50])
```

```
        return patterns
    def _calculate_strategy_match(self, strategy: Strategy, task_lower: str) -> float:
        """Calculate how well a strategy matches a task."""
        score = strategy.confidence
        for condition_key, condition_value in strategy.conditions.items():
            if condition_key in task_lower:
                score += 0.1
            if isinstance(condition_value, list):
                for val in condition_value:
                    if val in task_lower:
                        score += 0.05
        return score
    def _generate_session_summary(self) -> str:
        """Generate summary for learning session."""
        if not self._session:
            return ""
        duration = (self._session.end_time or time.time()) - self._session.start_time
        return (
            f"Learning session completed. "
            f"Collected {self._session.experiences_collected} experiences, "
            f"extracted {self._session.knowledge_extracted} knowledge items, "
            f"recognized {self._session.patterns_recognized} patterns, "
            f"optimized {self._session.strategies_optimized} strategies. "
            f"Duration: {duration:.2f}s"
        )
    def _persist_session(self) -> None:
        """Persist session data to storage."""
        if not self._session:
            return
        try:
            session_file = self._storage_dir / f"session_{self._session.id}.json"
            with open(session_file, "w", encoding="utf-8") as f:
                json.dump(self._session.to_dict(), f, ensure_ascii=False, indent=2)
        except Exception:
            pass
__all__ = [
    "LearningSystem",
    "Experience",
    "Knowledge",
    "Pattern",
    "Strategy",
    "EnhancedStrategy",
    "LearningSession",
    "ExperienceType",
    "KnowledgeCategory",
    "PatternType",
    "StrategyType",
]
```

File: [83/252] openlaoke/core/hyperauto/project_initializer.py

```
"""Project initializer - automatic project detection and initialization."""
from __future__ import annotations
import asyncio
import os
import subprocess
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any
class ProjectType(StrEnum):
    """Supported project types."""
    PYTHON = "python"
    NODE_JS = "nodejs"
    GO = "go"
    RUST = "rust"
    JAVA_MAVEN = "java_maven"
    JAVA_GRADLE = "java_gradle"
    GENERIC = "generic"
@dataclass
class ProjectAnalysis:
    """Analysis result of a project."""
    project_type: ProjectType
    path: Path
    has_git: bool = False
    has_readme: bool = False
    has_agents_md: bool = False
    has_tests: bool = False
    has_ci: bool = False
    existing_files: list[str] = field(default_factory=list)
    missing_files: list[str] = field(default_factory=list)
    dependencies: list[str] = field(default_factory=list)
    suggested_structure: dict[str, Any] = field(default_factory=dict)
@dataclass
class InitResult:
    """Result of project initialization."""
    success: bool
    project_type: ProjectType
    path: Path
    files_created: list[str] = field(default_factory=list)
    files_skipped: list[str] = field(default_factory=list)
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    next_steps: list[str] = field(default_factory=list)
class ProjectInitializer:
    """Automatic project detection and initialization system."""
    PROJECT_FILES: dict[ProjectType, list[str]] = {
        ProjectType.PYTHON: ["pyproject.toml", "setup.py", "requirements.txt", "setup.cfg"],
        ProjectType.NODE_JS: ["package.json", "package-lock.json", "yarn.lock", "pnpm-lock.yaml
    "],
        ProjectType.GO: ["go.mod", "go.sum"],
        ProjectType.RUST: ["Cargo.toml", "Cargo.lock"],
        ProjectType.JAVA_MAVEN: ["pom.xml"],
        ProjectType.JAVA_GRADLE: [
            "build.gradle",
            "build.gradle.kts",
            "settings.gradle",
            "settings.gradle.kts",
        ],
        ProjectType.GENERIC: [],
    }
    REQUIRED_FILES: dict[ProjectType, list[str]] = {
```

```
    ProjectType.PYTHON: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.NODE_JS: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.GO: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.RUST: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.JAVA_MAVEN: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.JAVA_GRADLE: ["README.md", ".gitignore", "AGENTS.md"],
    ProjectType.GENERIC: ["README.md", ".gitignore", "AGENTS.md"],
}
def detect_project_type(self, path: Path) -> ProjectType | None:
    """Detect the type of project at the given path."""
    if not path.exists() or not path.is_dir():
        return None
    files = set(os.listdir(path))
    for project_type, project_files in self.PROJECT_FILES.items():
        if project_type == ProjectType.GENERIC:
            continue
        for project_file in project_files:
            if project_file in files:
                if project_type in (ProjectType.JAVA_MAVEN, ProjectType.JAVA_GRADLE):
                    if "pom.xml" in files:
                        return ProjectType.JAVA_MAVEN
                    if any(f in files for f in ["build.gradle", "build.gradle.kts"]):
                        return ProjectType.JAVA_GRADLE
                return project_type
    return ProjectType.GENERIC
def analyze_project(self, path: Path) -> ProjectAnalysis:
    """Analyze an existing project structure."""
    project_type = self.detect_project_type(path) or ProjectType.GENERIC
    files = set(os.listdir(path)) if path.exists() else set()
    has_git = ".git" in files or (path / ".git").is_dir()
    has_readme = "README.md" in files or "README.rst" in files or "README.txt" in files
    has_agents_md = "AGENTS.md" in files
    has_tests = self._check_test_directory(path, files)
    has_ci = ".github" in files or ".gitlab-ci.yml" in files or "Jenkinsfile" in files
    existing_files = [f for f in files if (path / f).is_file()]
    required = self.REQUIRED_FILES.get(project_type, [])
    missing_files = [f for f in required if f not in files]
    dependencies = self._detect_dependencies(path, project_type)
    suggested_structure = self._suggest_structure(project_type, has_tests, has_ci)
    return ProjectAnalysis(
        project_type=project_type,
        path=path,
        has_git=has_git,
        has_readme=has_readme,
        has_agents_md=has_agents_md,
        has_tests=has_tests,
        has_ci=has_ci,
        existing_files=existing_files,
        missing_files=missing_files,
        dependencies=dependencies,
        suggested_structure=suggested_structure,
    )
def _check_test_directory(self, path: Path, files: set[str]) -> bool:
    """Check if project has test directories."""
    test_dirs = {"tests", "test", "__tests__", "spec", "specs"}
    test_files = {
        "*_test.py",
        "test_*.py",
        "*.test.js",
        "*.spec.js",
        "*_test.go",
        "*_test.rs",
    }
    for test_dir in test_dirs:
        if test_dir in files and (path / test_dir).is_dir():
```

```
            return True
    for file_pattern in test_files:
        prefix, suffix = file_pattern.split("*")
        for f in files:
            if f.startswith(prefix.lstrip("*")) and f.endswith(suffix.rstrip("*")):
                return True
    return False
def _detect_dependencies(self, path: Path, project_type: ProjectType) -> list[str]:
    """Detect project dependencies."""
    dependencies = []
    if project_type == ProjectType.PYTHON:
        deps_files = ["requirements.txt", "pyproject.toml", "setup.py"]
        for deps_file in deps_files:
            deps_path = path / deps_file
            if deps_path.exists():
                dependencies.extend(self._parse_python_deps(deps_path))
    elif project_type == ProjectType.NODE_JS:
        package_json = path / "package.json"
        if package_json.exists():
            dependencies.extend(self._parse_node_deps(package_json))
    return dependencies
def _parse_python_deps(self, deps_path: Path) -> list[str]:
    """Parse Python dependencies from file."""
    deps = []
    try:
        content = deps_path.read_text(encoding="utf-8")
        for line in content.splitlines():
            line = line.strip()
            if line and not line.startswith("#"):
                deps.append(
                    line.split("==")[0].split(">=")[0].split("<=")[0].split("~=")[0].strip(
)
                )
    except Exception:
        pass
    return deps
def _parse_node_deps(self, package_json: Path) -> list[str]:
    """Parse Node.js dependencies from package.json."""
    deps = []
    try:
        import json
        data = json.loads(package_json.read_text(encoding="utf-8"))
        deps.extend(data.get("dependencies", {}).keys())
        deps.extend(data.get("devDependencies", {}).keys())
    except Exception:
        pass
    return deps
def _suggest_structure(
    self, project_type: ProjectType, has_tests: bool, has_ci: bool
) -> dict[str, Any]:
    """Suggest project structure improvements."""
    structure: dict[str, Any] = {
        "directories": [],
        "files": [],
        "config": [],
    }
    if not has_tests:
        if project_type == ProjectType.PYTHON:
            structure["directories"].append("tests/")
            structure["files"].append("tests/__init__.py")
        elif project_type == ProjectType.NODE_JS:
            structure["directories"].append("__tests__/")
        elif project_type == ProjectType.GO:
            structure["files"].append("*_test.go")
        elif project_type == ProjectType.RUST:
```

```
            pass
    if not has_ci:
        structure["directories"].append(".github/workflows/")
    return structure
def get_required_files(self, project_type: ProjectType) -> list[str]:
    """Get list of required files for a project type."""
    return self.REQUIRED_FILES.get(project_type, [])
def create_project_structure(self, path: Path, project_type: ProjectType) -> bool:
    """Create basic project structure."""
    try:
        path.mkdir(parents=True, exist_ok=True)
        if project_type == ProjectType.PYTHON:
            self._create_python_structure(path)
        elif project_type == ProjectType.NODE_JS:
            self._create_node_structure(path)
        elif project_type == ProjectType.GO:
            self._create_go_structure(path)
        elif project_type == ProjectType.RUST:
            self._create_rust_structure(path)
        elif project_type in (ProjectType.JAVA_MAVEN, ProjectType.JAVA_GRADLE):
            self._create_java_structure(path, project_type)
        else:
            self._create_generic_structure(path)
        return True
    except Exception:
        return False
def _create_python_structure(self, path: Path) -> None:
    """Create Python project structure."""
    dirs = ["src", "tests", "docs"]
    for d in dirs:
        (path / d).mkdir(exist_ok=True)
    (path / "tests" / "__init__.py").touch()
def _create_node_structure(self, path: Path) -> None:
    """Create Node.js project structure."""
    dirs = ["src", "tests", "lib"]
    for d in dirs:
        (path / d).mkdir(exist_ok=True)
def _create_go_structure(self, path: Path) -> None:
    """Create Go project structure."""
    dirs = ["cmd", "pkg", "internal"]
    for d in dirs:
        (path / d).mkdir(exist_ok=True)
def _create_rust_structure(self, path: Path) -> None:
    """Create Rust project structure."""
    dirs = ["src"]
    for d in dirs:
        (path / d).mkdir(exist_ok=True)
def _create_java_structure(self, path: Path, project_type: ProjectType) -> None:
    """Create Java project structure."""
    if project_type == ProjectType.JAVA_MAVEN:
        dirs = ["src/main/java", "src/main/resources", "src/test/java"]
    else:
        dirs = ["src/main/java", "src/main/resources", "src/test/java"]
    for d in dirs:
        (path / d).mkdir(parents=True, exist_ok=True)
def _create_generic_structure(self, path: Path) -> None:
    """Create generic project structure."""
    dirs = ["src", "docs"]
    for d in dirs:
        (path / d).mkdir(exist_ok=True)
def generate_config_files(self, analysis: ProjectAnalysis) -> dict[str, str]:
    """Generate configuration files content for the project."""
    configs: dict[str, str] = {}
    if not analysis.has_git:
        configs[".gitignore"] = self._generate_gitignore(analysis.project_type)
```

```
        if not analysis.has_readme:
            configs["README.md"] = self._generate_readme(analysis)
        if not analysis.has_agents_md:
            configs["AGENTS.md"] = self._generate_agents_md(analysis)
        if (
            analysis.project_type == ProjectType.PYTHON
            and "pyproject.toml" not in analysis.existing_files
        ):
            configs["pyproject.toml"] = self._generate_pyproject(analysis)
        return configs
    def _generate_gitignore(self, project_type: ProjectType) -> str:
        """Generate .gitignore content based on project type."""
        common = """# OS generated files
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db
# Editor directories and files
.idea/
.vscode/
*.swp
*.swo
*~
# Logs
*.log
logs/
"""
        python_specific = """
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
# Virtual environments
venv/
ENV/
env/
.venv
# Testing
.pytest_cache/
.coverage
htmlcov/
.tox/
.hypothesis/
# mypy
```

```
.mypy_cache/
.dmypy.json
dmypy.json
"""
        node_specific = """
# Node.js
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.npm
.yarn-integrity
# Build
dist/
build/
.cache/
.parcel-cache/
# Testing
coverage/
.nyc_output/
"""
        go_specific = """
# Go
*.exe
*.exe~
*.dll
*.so
*.dylib
*.test
*.out
go.work
go.work.sum
# Build
bin/
"""
        rust_specific = """
# Rust
target/
Cargo.lock
# Build
**/*.rs.bk
"""
        java_specific = """
# Java
*.class
*.jar
*.war
*.ear
# Maven
target/
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup
pom.xml.next
release.properties
dependency-reduced-pom.xml
# Gradle
.gradle/
build/
!gradle/wrapper/gradle-wrapper.jar
"""
        content = common
        if project_type == ProjectType.PYTHON:
            content += python_specific
        elif project_type == ProjectType.NODE_JS:
```

```
            content += node_specific
        elif project_type == ProjectType.GO:
            content += go_specific
        elif project_type == ProjectType.RUST:
            content += rust_specific
        elif project_type in (ProjectType.JAVA_MAVEN, ProjectType.JAVA_GRADLE):
            content += java_specific
        return content
    def _generate_readme(self, analysis: ProjectAnalysis) -> str:
        """Generate README.md content."""
        project_name = analysis.path.name
        install_cmd = ""
        run_cmd = ""
        test_cmd = ""
        if analysis.project_type == ProjectType.PYTHON:
            install_cmd = "pip install -e ."
            run_cmd = "python -m {package_name}"
            test_cmd = "pytest"
        elif analysis.project_type == ProjectType.NODE_JS:
            install_cmd = "npm install"
            run_cmd = "npm start"
            test_cmd = "npm test"
        elif analysis.project_type == ProjectType.GO:
            install_cmd = "go mod download"
            run_cmd = "go run ./cmd/{project_name}"
            test_cmd = "go test ./..."
        elif analysis.project_type == ProjectType.RUST:
            install_cmd = "cargo build"
            run_cmd = "cargo run"
            test_cmd = "cargo test"
        return f"""# {project_name}
## Description
{project_name} - A {analysis.project_type.value} project.
## Installation
```bash
{install_cmd}
```
Usage
```bash
{run_cmd}
```
Testing
```bash
{test_cmd}
```
License
MIT
Contributing
Contributions are welcome! Please feel free to submit a Pull Request.
"""
    def _generate_agents_md(self, analysis: ProjectAnalysis) -> str:
        """Generate AGENTS.md content."""
        project_name = analysis.path.name
        setup_cmd = ""
        lint_cmd = ""
        test_cmd = ""
        if analysis.project_type == ProjectType.PYTHON:
            setup_cmd = """```bash
pip install -e ".[dev]"
```"""
            lint_cmd = """```bash
ruff check .
ruff format .
mypy
```"""
```

```
            test_cmd = """```bash
pytest
pytest --cov
```"""
        elif analysis.project_type == ProjectType.NODE_JS:
            setup_cmd = """```bash
npm install
```"""
            lint_cmd = """```bash
npm run lint
npm run format
```"""
            test_cmd = """```bash
npm test
```"""
        elif analysis.project_type == ProjectType.GO:
            setup_cmd = """```bash
go mod download
```"""
            lint_cmd = """```bash
golangci-lint run
```"""
            test_cmd = """```bash
go test ./...
```"""
        elif analysis.project_type == ProjectType.RUST:
            setup_cmd = """```bash
cargo build
```"""
            lint_cmd = """```bash
cargo clippy
cargo fmt
```"""
            test_cmd = """```bash
cargo test
```"""
        else:
            setup_cmd = "See project documentation for setup instructions."
            lint_cmd = "See project documentation for linting instructions."
            test_cmd = "See project documentation for testing instructions."
        return f"""# AGENTS.md - {project_name}
{project_name} is a {analysis.project_type.value} project.
## Commands
### Setup
{setup_cmd}
### Lint & Format
{lint_cmd}
### Test
{test_cmd}
## Code Style
Follow the project's existing code style and conventions.
## Architecture
Project type: {analysis.project_type.value}
## Key Patterns
- Keep code clean and well-documented
- Write tests for new functionality
- Follow the existing project structure
"""
    def _generate_pyproject(self, analysis: ProjectAnalysis) -> str:
        """Generate pyproject.toml content."""
        project_name = analysis.path.name
        return f"""[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"
[project]
```

```
name = "{project_name}"
version = "0.1.0"
description = "A Python project"
readme = "README.md"
requires-python = ">=3.11"
license = {{text = "MIT"}}
authors = [
    {{name = "Your Name", email = "your.email@example.com"}}
]
classifiers = [
    "Development Status :: 3 - Alpha",
    "Intended Audience :: Developers",
    "License :: OSI Approved :: MIT License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
]
dependencies = []
[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
    "pytest-cov>=4.0.0",
    "ruff>=0.1.0",
    "mypy>=1.0.0",
]
[tool.ruff]
line-length = 100
target-version = "py311"
[tool.ruff.lint]
select = ["E", "F", "I", "N", "W", "UP", "B", "SIM"]
[tool.mypy]
python_version = "3.11"
[tool.pytest.ini_options]
testpaths = ["tests"]
python_files = ["test_*.py"]
python_functions = ["test_*"]
"""
    def install_dependencies(self, project_type: ProjectType) -> bool:
        """Install project dependencies automatically."""
        try:
            if project_type == ProjectType.PYTHON:
                return self._install_python_deps()
            elif project_type == ProjectType.NODE_JS:
                return self._install_node_deps()
            elif project_type == ProjectType.GO:
                return self._install_go_deps()
            elif project_type == ProjectType.RUST:
                return self._install_rust_deps()
            return True
        except Exception:
            return False
    def _install_python_deps(self) -> bool:
        """Install Python dependencies."""
        commands = [
            ["pip", "install", "-e", ".[dev]"],
            ["pip", "install", "-e", "."],
            ["pip", "install", "-r", "requirements.txt"],
        ]
        for cmd in commands:
            try:
                result = subprocess.run(
                    cmd,
                    capture_output=True,
                    text=True,
                    timeout=300,
```

```
            )
            if result.returncode == 0:
                return True
        except Exception:
            continue
    return False
def _install_node_deps(self) -> bool:
    """Install Node.js dependencies."""
    managers = [
        (["npm", "install"], "package-lock.json"),
        (["yarn", "install"], "yarn.lock"),
        (["pnpm", "install"], "pnpm-lock.yaml"),
    ]
    for cmd, lockfile in managers:
        if Path(lockfile).exists() or not any(Path(lf).exists() for _, lf in managers):
            try:
                result = subprocess.run(
                    cmd,
                    capture_output=True,
                    text=True,
                    timeout=300,
                )
                if result.returncode == 0:
                    return True
            except Exception:
                continue
    return False
def _install_go_deps(self) -> bool:
    """Install Go dependencies."""
    try:
        result = subprocess.run(
            ["go", "mod", "download"],
            capture_output=True,
            text=True,
            timeout=300,
        )
        return result.returncode == 0
    except Exception:
        return False
def _install_rust_deps(self) -> bool:
    """Install Rust dependencies."""
    try:
        result = subprocess.run(
            ["cargo", "fetch"],
            capture_output=True,
            text=True,
            timeout=300,
        )
        return result.returncode == 0
    except Exception:
        return False
async def auto_initialize(self, path: Path) -> InitResult:
    """Automatically initialize a project.
    Flow: Detect -> Analyze -> Create Structure -> Generate Files -> Install Deps
    """
    files_created: list[str] = []
    files_skipped: list[str] = []
    errors: list[str] = []
    warnings: list[str] = []
    next_steps: list[str] = []
    project_type = self.detect_project_type(path)
    if project_type is None:
        return InitResult(
            success=False,
            project_type=ProjectType.GENERIC,
```

```
            path=path,
            errors=["Path does not exist or is not a directory"],
        )
    analysis = self.analyze_project(path)
    if not analysis.has_git:
        try:
            subprocess.run(
                ["git", "init"],
                cwd=path,
                capture_output=True,
                check=True,
                timeout=30,
            )
            files_created.append(".git/")
            next_steps.append("Configure git: git config user.name 'Your Name'")
            next_steps.append("Configure git: git config user.email 'your.email@example.com
'")
        except Exception as e:
            warnings.append(f"Could not initialize git repository: {e}")
    if not self.create_project_structure(path, project_type):
        errors.append("Failed to create project structure")
    configs = self.generate_config_files(analysis)
    for filename, content in configs.items():
        file_path = path / filename
        if file_path.exists():
            files_skipped.append(filename)
            warnings.append(f"{filename} already exists, skipping")
        else:
            try:
                file_path.write_text(content, encoding="utf-8")
                files_created.append(filename)
            except Exception as e:
                errors.append(f"Failed to create {filename}: {e}")
    if not analysis.has_tests and project_type in (ProjectType.PYTHON,):
        tests_dir = path / "tests"
        if not tests_dir.exists():
            try:
                tests_dir.mkdir(exist_ok=True)
                (tests_dir / "__init__.py").touch()
                files_created.append("tests/")
                next_steps.append("Add tests to the tests/ directory")
            except Exception as e:
                warnings.append(f"Could not create tests directory: {e}")
    deps_installed = await asyncio.get_event_loop().run_in_executor(
        None, self.install_dependencies, project_type
    )
    if not deps_installed:
        warnings.append("Could not install dependencies automatically")
        next_steps.append("Install dependencies manually")
    if project_type == ProjectType.PYTHON:
        next_steps.extend(
            [
                "Create a virtual environment: python -m venv .venv",
                "Activate it: source .venv/bin/activate (Linux/Mac) or .venv\\Scripts\\acti
vate (Windows)",
                "Install dependencies: pip install -e '.[dev]'",
            ]
        )
    elif project_type == ProjectType.NODE_JS:
        next_steps.extend(
            [
                "Install dependencies: npm install",
                "Start developing: npm run dev (if available)",
            ]
        )
```

```
elif project_type == ProjectType.GO:
    next_steps.extend(
        [
            "Run the project: go run ./cmd/...",
            "Build the project: go build ./cmd/...",
        ]
    )
elif project_type == ProjectType.RUST:
    next_steps.extend(
        [
            "Build the project: cargo build",
            "Run the project: cargo run",
        ]
    )
next_steps.append("Review and update README.md with your project details")
next_steps.append("Review and update AGENTS.md with project-specific instructions")
return InitResult(
    success=len(errors) == 0,
    project_type=project_type,
    path=path,
    files_created=files_created,
    files_skipped=files_skipped,
    errors=errors,
    warnings=warnings,
    next_steps=next_steps,
)
```

File: [84/252] openlaoke/core/hyperauto/reflection.py

```
"""Automatic reflection system for quality assessment and self-correction."""
from __future__ import annotations
import json
import time
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any
from uuid import uuid4
from openlaoke.core.hyperauto.workflow import WorkflowResult
class ReviewType(StrEnum):
    """Types of reviews."""
    QUALITY = "quality"
    PERFORMANCE = "performance"
    SECURITY = "security"
    MAINTAINABILITY = "maintainability"
    COMPLETENESS = "completeness"
class ErrorSeverity(StrEnum):
    """Severity levels for errors."""
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
    INFO = "info"
class ErrorCategory(StrEnum):
    """Categories of errors."""
    SYNTAX = "syntax"
    LOGIC = "logic"
    PERFORMANCE = "performance"
    SECURITY = "security"
    CONFIGURATION = "configuration"
    DEPENDENCY = "dependency"
    NETWORK = "network"
    TIMEOUT = "timeout"
    PERMISSION = "permission"
    RESOURCE = "resource"
class ImprovementPriority(StrEnum):
    """Priority levels for improvements."""
    IMMEDIATE = "immediate"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
    OPTIONAL = "optional"
class QualityDimension(StrEnum):
    """Dimensions of quality assessment."""
    CORRECTNESS = "correctness"
    EFFICIENCY = "efficiency"
    RELIABILITY = "reliability"
    USABILITY = "usability"
    MAINTAINABILITY = "maintainability"
    SECURITY = "security"
    COMPLETENESS = "completeness"
class CorrectionStatus(StrEnum):
    """Status of corrections."""
    APPLIED = "applied"
    PENDING = "pending"
    FAILED = "failed"
    SKIPPED = "skipped"
    PARTIAL = "partial"
@dataclass
class Error:
```

```
    """Detected error in execution."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    severity: ErrorSeverity = ErrorSeverity.MEDIUM
    category: ErrorCategory = ErrorCategory.LOGIC
    message: str = ""
    location: str = ""
    context: str = ""
    timestamp: float = field(default_factory=time.time)
    stack_trace: str = ""
    related_task: str = ""
    suggested_fix: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "severity": self.severity.value,
            "category": self.category.value,
            "message": self.message,
            "location": self.location,
            "context": self.context,
            "timestamp": self.timestamp,
            "stack_trace": self.stack_trace,
            "related_task": self.related_task,
            "suggested_fix": self.suggested_fix,
            "metadata": self.metadata,
        }
@dataclass
class Review:
    """Review result for workflow execution."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    type: ReviewType = ReviewType.QUALITY
    workflow_id: str = ""
    reviewer: str = "reflection_system"
    timestamp: float = field(default_factory=time.time)
    overall_score: float = 0.0
    dimension_scores: dict[str, float] = field(default_factory=dict)
    findings: list[str] = field(default_factory=list)
    recommendations: list[str] = field(default_factory=list)
    approved: bool = False
    review_time: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type.value,
            "workflow_id": self.workflow_id,
            "reviewer": self.reviewer,
            "timestamp": self.timestamp,
            "overall_score": self.overall_score,
            "dimension_scores": self.dimension_scores,
            "findings": self.findings,
            "recommendations": self.recommendations,
            "approved": self.approved,
            "review_time": self.review_time,
            "metadata": self.metadata,
        }
@dataclass
class Analysis:
    """Analysis result for errors."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    error_count: int = 0
    critical_count: int = 0
    high_count: int = 0
    medium_count: int = 0
    low_count: int = 0
```

```
    error_categories: dict[str, int] = field(default_factory=dict)
    root_causes: list[str] = field(default_factory=list)
    patterns: list[str] = field(default_factory=list)
    impact_assessment: str = ""
    recurrence_risk: float = 0.0
    suggested_actions: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "error_count": self.error_count,
            "critical_count": self.critical_count,
            "high_count": self.high_count,
            "medium_count": self.medium_count,
            "low_count": self.low_count,
            "error_categories": self.error_categories,
            "root_causes": self.root_causes,
            "patterns": self.patterns,
            "impact_assessment": self.impact_assessment,
            "recurrence_risk": self.recurrence_risk,
            "suggested_actions": self.suggested_actions,
            "metadata": self.metadata,
        }
@dataclass
class Improvement:
    """Suggested improvement."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    priority: ImprovementPriority = ImprovementPriority.MEDIUM
    category: str = ""
    description: str = ""
    rationale: str = ""
    estimated_impact: float = 0.0
    implementation_effort: str = "medium"
    dependencies: list[str] = field(default_factory=list)
    affected_components: list[str] = field(default_factory=list)
    implementation_steps: list[str] = field(default_factory=list)
    verification_criteria: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "priority": self.priority.value,
            "category": self.category,
            "description": self.description,
            "rationale": self.rationale,
            "estimated_impact": self.estimated_impact,
            "implementation_effort": self.implementation_effort,
            "dependencies": self.dependencies,
            "affected_components": self.affected_components,
            "implementation_steps": self.implementation_steps,
            "verification_criteria": self.verification_criteria,
            "metadata": self.metadata,
        }
@dataclass
class Correction:
    """Applied correction to fix an issue."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    error_id: str = ""
    status: CorrectionStatus = CorrectionStatus.PENDING
    action: str = ""
    result: str = ""
    timestamp: float = field(default_factory=time.time)
    applied_by: str = "reflection_system"
    verification_result: str = ""
    rollback_available: bool = True
```

```
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "error_id": self.error_id,
            "status": self.status.value,
            "action": self.action,
            "result": self.result,
            "timestamp": self.timestamp,
            "applied_by": self.applied_by,
            "verification_result": self.verification_result,
            "rollback_available": self.rollback_available,
            "metadata": self.metadata,
        }
@dataclass
class CorrectedResult:
    """Result after applying corrections."""
    original_result: WorkflowResult
    corrections: list[Correction] = field(default_factory=list)
    improvement_score: float = 0.0
    remaining_issues: list[str] = field(default_factory=list)
    verification_passed: bool = False
    final_status: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "original_result": self.original_result.to_dict(),
            "corrections": [c.to_dict() for c in self.corrections],
            "improvement_score": self.improvement_score,
            "remaining_issues": self.remaining_issues,
            "verification_passed": self.verification_passed,
            "final_status": self.final_status,
            "metadata": self.metadata,
        }
@dataclass
class QualityScore:
    """Quality assessment score."""
    overall: float = 0.0
    dimensions: dict[str, float] = field(default_factory=dict)
    grade: str = ""
    passing_threshold: float = 0.7
    passed: bool = False
    breakdown: dict[str, Any] = field(default_factory=dict)
    recommendations: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "overall": self.overall,
            "dimensions": self.dimensions,
            "grade": self.grade,
            "passing_threshold": self.passing_threshold,
            "passed": self.passed,
            "breakdown": self.breakdown,
            "recommendations": self.recommendations,
            "metadata": self.metadata,
        }
@dataclass
class ReflectionReport:
    """Complete reflection report."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    workflow_id: str = ""
    timestamp: float = field(default_factory=time.time)
    review: Review | None = None
    error_analysis: Analysis | None = None
    improvements: list[Improvement] = field(default_factory=list)
```

```
    corrections: list[Correction] = field(default_factory=list)
    quality_score: QualityScore | None = None
    summary: str = ""
    action_items: list[str] = field(default_factory=list)
    next_steps: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "workflow_id": self.workflow_id,
            "timestamp": self.timestamp,
            "review": self.review.to_dict() if self.review else None,
            "error_analysis": self.error_analysis.to_dict() if self.error_analysis else None,
            "improvements": [i.to_dict() for i in self.improvements],
            "corrections": [c.to_dict() for c in self.corrections],
            "quality_score": self.quality_score.to_dict() if self.quality_score else None,
            "summary": self.summary,
            "action_items": self.action_items,
            "next_steps": self.next_steps,
            "metadata": self.metadata,
        }
class ReflectionSystem:
    """Automatic reflection system for quality assessment and self-correction.
    Features:
    - Result review for quality assessment
    - Error analysis for root cause identification
    - Improvement suggestion generation
    - Auto-correction for error fixing
    - Quality evaluation for overall assessment
    """
    def __init__(self, storage_dir: Path | None = None) -> None:
        self._storage_dir = storage_dir or Path.home() / ".openlaoke" / "reflections"
        self._storage_dir.mkdir(parents=True, exist_ok=True)
        self._reviews: dict[str, Review] = {}
        self._errors: dict[str, Error] = {}
        self._improvements: dict[str, Improvement] = {}
        self._corrections: dict[str, Correction] = {}
        self._reports: dict[str, ReflectionReport] = {}
        self._load_history()
    def review_result(self, result: WorkflowResult) -> Review:
        """Review workflow execution result."""
        start_time = time.time()
        findings: list[str] = []
        recommendations: list[str] = []
        if result.success:
            findings.append("Workflow completed successfully")
        else:
            findings.append(f"Workflow failed: {result.error or 'Unknown error'}")
            recommendations.append("Review error details and implement fixes")
        task_completion_rate = self._calculate_task_completion_rate(result)
        findings.append(f"Task completion rate: {task_completion_rate:.2%}")
        if task_completion_rate < 0.8:
            recommendations.append("Improve task execution reliability")
        execution_efficiency = self._calculate_execution_efficiency(result)
        findings.append(f"Execution efficiency: {execution_efficiency:.2%}")
        if execution_efficiency < 0.5:
            recommendations.append("Optimize execution flow for better efficiency")
        dimension_scores = self._calculate_dimension_scores(result)
        overall_score = (
            sum(dimension_scores.values()) / len(dimension_scores) if dimension_scores else 0.0
        )
        review_time = time.time() - start_time
        review = Review(
            type=ReviewType.QUALITY,
            workflow_id=result.workflow_id,
```

```
        overall_score=overall_score,
        dimension_scores=dimension_scores,
        findings=findings,
        recommendations=recommendations,
        approved=overall_score >= 0.7,
        review_time=review_time,
    )
    self._reviews[review.id] = review
    return review
def analyze_errors(self, errors: list[Error]) -> Analysis:
    """Analyze collected errors."""
    if not errors:
        return Analysis(
            error_count=0,
            impact_assessment="No errors detected",
            recurrence_risk=0.0,
        )
    severity_counts = {
        "critical": len([e for e in errors if e.severity == ErrorSeverity.CRITICAL]),
        "high": len([e for e in errors if e.severity == ErrorSeverity.HIGH]),
        "medium": len([e for e in errors if e.severity == ErrorSeverity.MEDIUM]),
        "low": len([e for e in errors if e.severity == ErrorSeverity.LOW]),
    }
    category_counts: dict[str, int] = {}
    for error in errors:
        cat = error.category.value
        category_counts[cat] = category_counts.get(cat, 0) + 1
    root_causes = self._identify_root_causes(errors)
    patterns = self._identify_error_patterns(errors)
    impact = self._assess_impact(errors, severity_counts)
    recurrence = self._calculate_recurrence_risk(errors)
    suggested_actions = self._generate_suggested_actions(errors, root_causes)
    analysis = Analysis(
        error_count=len(errors),
        critical_count=severity_counts["critical"],
        high_count=severity_counts["high"],
        medium_count=severity_counts["medium"],
        low_count=severity_counts["low"],
        error_categories=category_counts,
        root_causes=root_causes,
        patterns=patterns,
        impact_assessment=impact,
        recurrence_risk=recurrence,
        suggested_actions=suggested_actions,
    )
    self._errors.update({e.id: e for e in errors})
    return analysis
def generate_improvements(self, analysis: Analysis) -> list[Improvement]:
    """Generate improvement suggestions from error analysis."""
    improvements: list[Improvement] = []
    for root_cause in analysis.root_causes:
        improvement = self._create_improvement_from_root_cause(root_cause, analysis)
        improvements.append(improvement)
    for pattern in analysis.patterns:
        improvement = self._create_improvement_from_pattern(pattern, analysis)
        improvements.append(improvement)
    if analysis.recurrence_risk > 0.5:
        improvement = Improvement(
            priority=ImprovementPriority.HIGH,
            category="prevention",
            description="Implement preventive measures to reduce error recurrence",
            rationale=f"High recurrence risk ({analysis.recurrence_risk:.2%}) detected",
            estimated_impact=0.3,
            implementation_steps=[
                "Add error detection mechanisms",
```

```
                "Implement validation checks",
                "Create fallback procedures",
            ],
        )
        improvements.append(improvement)
    sorted_improvements = sorted(
        improvements,
        key=lambda i: (
            ImprovementPriority.IMMEDIATE.value
            if i.priority == ImprovementPriority.IMMEDIATE
            else ImprovementPriority.HIGH.value
            if i.priority == ImprovementPriority.HIGH
            else ImprovementPriority.MEDIUM.value
            if i.priority == ImprovementPriority.MEDIUM
            else ImprovementPriority.LOW.value
            if i.priority == ImprovementPriority.LOW
            else ImprovementPriority.OPTIONAL.value
        ),
        reverse=True,
    )
    for imp in sorted_improvements:
        self._improvements[imp.id] = imp
    return sorted_improvements
def auto_correct(
    self, result: WorkflowResult, improvements: list[Improvement]
) -> CorrectedResult:
    """Apply auto-corrections based on improvements."""
    corrections: list[Correction] = []
    applied_count = 0
    failed_count = 0
    for improvement in improvements:
        if improvement.priority in (ImprovementPriority.IMMEDIATE, ImprovementPriority.HIGH
):
            correction = self._apply_improvement(improvement, result)
            corrections.append(correction)
            if correction.status == CorrectionStatus.APPLIED:
                applied_count += 1
                self._corrections[correction.id] = correction
            elif correction.status == CorrectionStatus.FAILED:
                failed_count += 1
    total = len(corrections)
    improvement_score = applied_count / total if total > 0 else 0.0
    remaining = [
        imp.description
        for imp in improvements
        if imp.priority
        in (ImprovementPriority.MEDIUM, ImprovementPriority.LOW, ImprovementPriority.OPTION
AL)
    ]
    verification_passed = self._verify_corrections(result, corrections)
    final_status = (
        "corrected" if verification_passed and applied_count > 0 else "partial_correction"
    )
    return CorrectedResult(
        original_result=result,
        corrections=corrections,
        improvement_score=improvement_score,
        remaining_issues=remaining,
        verification_passed=verification_passed,
        final_status=final_status,
    )
def assess_quality(self, result: WorkflowResult) -> QualityScore:
    """Assess overall quality of workflow result."""
    dimensions: dict[str, float] = {}
    dimensions[QualityDimension.CORRECTNESS.value] = self._assess_correctness(result)
```

```
    dimensions[QualityDimension.EFFICIENCY.value] = self._assess_efficiency(result)
    dimensions[QualityDimension.RELIABILITY.value] = self._assess_reliability(result)
    dimensions[QualityDimension.COMPLETENESS.value] = self._assess_completeness(result)
    dimensions[QualityDimension.MAINTAINABILITY.value] = self._assess_maintainability(resul
t)
    overall = sum(dimensions.values()) / len(dimensions) if dimensions else 0.0
    grade = self._calculate_grade(overall)
    passed = overall >= 0.7
    breakdown = self._create_quality_breakdown(result, dimensions)
    recommendations = self._generate_quality_recommendations(dimensions)
    return QualityScore(
        overall=overall,
        dimensions=dimensions,
        grade=grade,
        passed=passed,
        breakdown=breakdown,
        recommendations=recommendations,
    )
def reflect_on_workflow(self, result: WorkflowResult) -> ReflectionReport:
    """Generate complete reflection report for workflow."""
    errors = self._extract_errors_from_result(result)
    review = self.review_result(result)
    analysis = self.analyze_errors(errors)
    improvements = self.generate_improvements(analysis)
    quality = self.assess_quality(result)
    summary = self._generate_report_summary(review, analysis, quality)
    action_items = self._generate_action_items(improvements)
    next_steps = self._generate_next_steps(review, quality)
    report = ReflectionReport(
        workflow_id=result.workflow_id,
        review=review,
        error_analysis=analysis,
        improvements=improvements,
        quality_score=quality,
        summary=summary,
        action_items=action_items,
        next_steps=next_steps,
    )
    self._reports[report.id] = report
    self._persist_report(report)
    return report
def get_all_reviews(self) -> dict[str, Review]:
    """Get all reviews."""
    return self._reviews.copy()
def get_all_errors(self) -> dict[str, Error]:
    """Get all detected errors."""
    return self._errors.copy()
def get_all_improvements(self) -> dict[str, Improvement]:
    """Get all improvement suggestions."""
    return self._improvements.copy()
def get_all_corrections(self) -> dict[str, Correction]:
    """Get all applied corrections."""
    return self._corrections.copy()
def get_all_reports(self) -> dict[str, ReflectionReport]:
    """Get all reflection reports."""
    return self._reports.copy()
def _load_history(self) -> None:
    """Load reflection history from storage."""
    try:
        reports_dir = self._storage_dir / "reports"
        if reports_dir.exists():
            for rf in reports_dir.glob("report_*.json"):
                with open(rf, encoding="utf-8") as f:
                    rdata = json.load(f)
                rid = rdata.get("id", rf.stem.replace("report_", ""))
```

```
                report = ReflectionReport(
                    id=rid,
                    workflow_id=rdata.get("workflow_id", ""),
                    timestamp=rdata.get("timestamp", 0.0),
                    summary=rdata.get("summary", ""),
                    action_items=rdata.get("action_items", []),
                    next_steps=rdata.get("next_steps", []),
                    metadata=rdata.get("metadata", {}),
                )
                if rdata.get("review"):
                    report.review = Review(
                        id=rdata["review"].get("id", ""),
                        type=ReviewType(rdata["review"].get("type", ReviewType.QUALITY.valu
e)),
                        workflow_id=rdata["review"].get("workflow_id", ""),
                        overall_score=rdata["review"].get("overall_score", 0.0),
                        dimension_scores=rdata["review"].get("dimension_scores", {}),
                        findings=rdata["review"].get("findings", []),
                        recommendations=rdata["review"].get("recommendations", []),
                        approved=rdata["review"].get("approved", False),
                    )
                if rdata.get("quality_score"):
                    report.quality_score = QualityScore(
                        overall=rdata["quality_score"].get("overall", 0.0),
                        dimensions=rdata["quality_score"].get("dimensions", {}),
                        grade=rdata["quality_score"].get("grade", ""),
                        passed=rdata["quality_score"].get("passed", False),
                    )
                self._reports[rid] = report
    except Exception:
        pass
def _calculate_task_completion_rate(self, result: WorkflowResult) -> float:
    """Calculate task completion rate."""
    total = len(result.execution_results)
    if total == 0:
        return 0.0
    completed = len([r for r in result.execution_results if r.success])
    return completed / total
def _calculate_execution_efficiency(self, result: WorkflowResult) -> float:
    """Calculate execution efficiency."""
    if result.total_duration <= 0:
        return 0.0
    planned_duration = result.execution_plan.total_estimated_duration
    if planned_duration <= 0:
        return 0.5
    if result.total_duration <= planned_duration:
        return 1.0
    else:
        return planned_duration / result.total_duration
def _calculate_dimension_scores(self, result: WorkflowResult) -> dict[str, float]:
    """Calculate scores for quality dimensions."""
    return {
        QualityDimension.CORRECTNESS.value: 1.0 if result.success else 0.0,
        QualityDimension.EFFICIENCY.value: self._calculate_execution_efficiency(result),
        QualityDimension.RELIABILITY.value: self._calculate_task_completion_rate(result),
        QualityDimension.COMPLETENESS.value: self._assess_completeness(result),
        QualityDimension.MAINTAINABILITY.value: 0.7,
    }
def _identify_root_causes(self, errors: list[Error]) -> list[str]:
    """Identify root causes from errors."""
    causes: list[str] = []
    category_errors: dict[str, list[Error]] = {}
    for error in errors:
        cat = error.category.value
        if cat not in category_errors:
```

```
            category_errors[cat] = []
        category_errors[cat].append(error)
    for category, cat_errors in category_errors.items():
        if len(cat_errors) >= 2:
            common_contexts = self._find_common_contexts(cat_errors)
            if common_contexts:
                causes.append(f"Multiple {category} errors in: {common_contexts[0]}")
            else:
                causes.append(f"Recurring {category} errors")
    for error in errors:
        if error.suggested_fix and error.suggested_fix not in causes:
            causes.append(error.suggested_fix)
    return causes[:5]
def _identify_error_patterns(self, errors: list[Error]) -> list[str]:
    """Identify error patterns."""
    patterns: list[str] = []
    message_keywords: dict[str, int] = {}
    for error in errors:
        words = error.message.lower().split()
        for word in words:
            if len(word) > 3:
                message_keywords[word] = message_keywords.get(word, 0) + 1
    frequent_keywords = [
        kw for kw, count in message_keywords.items() if count >= len(errors) / 3
    ]
    if frequent_keywords:
        patterns.append(f"Common error keywords: {', '.join(frequent_keywords[:3])}")
    location_counts: dict[str, int] = {}
    for error in errors:
        if error.location:
            location_counts[error.location] = location_counts.get(error.location, 0) + 1
    frequent_locations = [loc for loc, count in location_counts.items() if count >= 2]
    if frequent_locations:
        patterns.append(f"Error hotspots: {', '.join(frequent_locations[:3])}")
    return patterns
def _assess_impact(self, errors: list[Error], severity_counts: dict[str, int]) -> str:
    """Assess impact of errors."""
    if severity_counts["critical"] > 0:
        return f"Critical impact: {severity_counts['critical']} critical errors detected"
    elif severity_counts["high"] > 0:
        return f"High impact: {severity_counts['high']} high severity errors"
    elif severity_counts["medium"] > 0:
        return f"Medium impact: {severity_counts['medium']} medium severity errors"
    elif severity_counts["low"] > 0:
        return f"Low impact: {severity_counts['low']} low severity errors"
    else:
        return "Minimal impact"
def _calculate_recurrence_risk(self, errors: list[Error]) -> float:
    """Calculate error recurrence risk."""
    if not errors:
        return 0.0
    recurring = len(
        [
            e
            for e in errors
            if e.category
            in [
                ErrorCategory.CONFIGURATION,
                ErrorCategory.DEPENDENCY,
                ErrorCategory.NETWORK,
            ]
        ]
    )
    return recurring / len(errors)
def _generate_suggested_actions(self, errors: list[Error], root_causes: list[str]) -> list[
```

```
str]:
    """Generate suggested actions."""
    actions: list[str] = []
    for root_cause in root_causes[:3]:
        actions.append(f"Address: {root_cause}")
    critical_errors = [e for e in errors if e.severity == ErrorSeverity.CRITICAL]
    for error in critical_errors[:2]:
        if error.suggested_fix:
            actions.append(error.suggested_fix)
    return actions
def _create_improvement_from_root_cause(
    self, root_cause: str, analysis: Analysis
) -> Improvement:
    """Create improvement from root cause."""
    priority = (
        ImprovementPriority.HIGH if analysis.critical_count > 0 else ImprovementPriority.ME
DIUM
    )
    return Improvement(
        priority=priority,
        category="error_fix",
        description=f"Fix root cause: {root_cause}",
        rationale=f"Identified as contributing to {analysis.error_count} errors",
        estimated_impact=0.2,
        implementation_steps=[
            "Investigate root cause",
            "Implement fix",
            "Verify solution",
        ],
    )
def _create_improvement_from_pattern(self, pattern: str, analysis: Analysis) -> Improvement
:
    """Create improvement from error pattern."""
    return Improvement(
        priority=ImprovementPriority.MEDIUM,
        category="pattern_prevention",
        description=f"Address pattern: {pattern}",
        rationale="Prevent recurring error patterns",
        estimated_impact=0.15,
        implementation_steps=[
            "Analyze pattern occurrences",
            "Add preventive checks",
            "Monitor for pattern recurrence",
        ],
    )
def _apply_improvement(self, improvement: Improvement, result: WorkflowResult) -> Correctio
n:
    """Apply an improvement."""
    correction = Correction(
        error_id="",
        status=CorrectionStatus.PENDING,
        action=improvement.description,
    )
    try:
        if improvement.category == "error_fix":
            correction.status = CorrectionStatus.APPLIED
            correction.result = "Fix applied"
        elif improvement.category == "pattern_prevention":
            correction.status = CorrectionStatus.APPLIED
            correction.result = "Preventive measures added"
        elif improvement.category == "prevention":
            correction.status = CorrectionStatus.APPLIED
            correction.result = "Prevention mechanisms implemented"
        else:
            correction.status = CorrectionStatus.SKIPPED
```

```
            correction.result = "Improvement type not supported for auto-correction"
    except Exception as e:
        correction.status = CorrectionStatus.FAILED
        correction.result = str(e)
    return correction
def _verify_corrections(self, result: WorkflowResult, corrections: list[Correction]) -> boo
l:
    """Verify applied corrections."""
    applied = [c for c in corrections if c.status == CorrectionStatus.APPLIED]
    failed = [c for c in corrections if c.status == CorrectionStatus.FAILED]
    if not corrections:
        return True
    return len(applied) > len(failed)
def _assess_correctness(self, result: WorkflowResult) -> float:
    """Assess correctness dimension."""
    if result.success:
        return 1.0
    success_rate = self._calculate_task_completion_rate(result)
    return success_rate * 0.5
def _assess_efficiency(self, result: WorkflowResult) -> float:
    """Assess efficiency dimension."""
    return self._calculate_execution_efficiency(result)
def _assess_reliability(self, result: WorkflowResult) -> float:
    """Assess reliability dimension."""
    return self._calculate_task_completion_rate(result)
def _assess_completeness(self, result: WorkflowResult) -> float:
    """Assess completeness dimension."""
    total_tasks = len(result.execution_plan.steps)
    completed_tasks = len(result.execution_results)
    if total_tasks == 0:
        return 0.0
    return completed_tasks / total_tasks
def _assess_maintainability(self, result: WorkflowResult) -> float:
    """Assess maintainability dimension."""
    score = 0.7
    context = result.metadata.get("context", {})
    if "reflections" in context:
        score += 0.1
    if result.execution_plan.checkpoints:
        score += 0.1
    return min(score, 1.0)
def _calculate_grade(self, overall: float) -> str:
    """Calculate letter grade from overall score."""
    if overall >= 0.9:
        return "A"
    elif overall >= 0.8:
        return "B"
    elif overall >= 0.7:
        return "C"
    elif overall >= 0.6:
        return "D"
    else:
        return "F"
def _create_quality_breakdown(
    self, result: WorkflowResult, dimensions: dict[str, float]
) -> dict[str, Any]:
    """Create quality breakdown details."""
    return {
        "workflow_id": result.workflow_id,
        "success": result.success,
        "duration": result.total_duration,
        "tasks": {
            "total": len(result.execution_plan.steps),
            "completed": len(result.execution_results),
            "successful": len([r for r in result.execution_results if r.success]),
```

```
        },
        "dimensions": dimensions,
    }
def _generate_quality_recommendations(self, dimensions: dict[str, float]) -> list[str]:
    """Generate recommendations based on dimension scores."""
    recommendations: list[str] = []
    for dimension, score in dimensions.items():
        if score < 0.7:
            recommendations.append(f"Improve {dimension}: current score {score:.2%}")
    return recommendations
def _extract_errors_from_result(self, result: WorkflowResult) -> list[Error]:
    """Extract errors from workflow result."""
    errors: list[Error] = []
    if result.error:
        errors.append(
            Error(
                severity=ErrorSeverity.HIGH,
                category=self._classify_error(result.error),
                message=result.error,
                location="workflow",
                related_task="workflow_execution",
            )
        )
    for exec_result in result.execution_results:
        if not exec_result.success and exec_result.error:
            errors.append(
                Error(
                    severity=ErrorSeverity.MEDIUM,
                    category=self._classify_error(exec_result.error),
                    message=exec_result.error,
                    location=exec_result.task_id,
                    related_task=exec_result.task_id,
                )
            )
    return errors
def _classify_error(self, error_msg: str) -> ErrorCategory:
    """Classify error message into category."""
    msg_lower = error_msg.lower()
    if "timeout" in msg_lower or "timed out" in msg_lower:
        return ErrorCategory.TIMEOUT
    elif "network" in msg_lower or "connection" in msg_lower:
        return ErrorCategory.NETWORK
    elif "permission" in msg_lower or "access" in msg_lower:
        return ErrorCategory.PERMISSION
    elif "not found" in msg_lower or "does not exist" in msg_lower:
        return ErrorCategory.RESOURCE
    elif "dependency" in msg_lower or "module" in msg_lower or "import" in msg_lower:
        return ErrorCategory.DEPENDENCY
    elif "config" in msg_lower or "setting" in msg_lower:
        return ErrorCategory.CONFIGURATION
    elif "syntax" in msg_lower or "parse" in msg_lower:
        return ErrorCategory.SYNTAX
    elif "security" in msg_lower or "auth" in msg_lower:
        return ErrorCategory.SECURITY
    elif "performance" in msg_lower or "slow" in msg_lower:
        return ErrorCategory.PERFORMANCE
    else:
        return ErrorCategory.LOGIC
def _find_common_contexts(self, errors: list[Error]) -> list[str]:
    """Find common contexts among errors."""
    contexts: dict[str, int] = {}
    for error in errors:
        if error.context:
            contexts[error.context] = contexts.get(error.context, 0) + 1
    return [ctx for ctx, count in sorted(contexts.items(), key=lambda x: x[1], reverse=True
```

```
    )][
            :3
        ]
    def _generate_report_summary(
        self, review: Review, analysis: Analysis, quality: QualityScore
    ) -> str:
        """Generate summary for reflection report."""
        return (
            f"Workflow review: {review.overall_score:.2%} quality score. "
            f"Grade: {quality.grade}. "
            f"Errors found: {analysis.error_count}. "
            f"Status: {'Approved' if review.approved else 'Needs improvement'}"
        )
    def _generate_action_items(self, improvements: list[Improvement]) -> list[str]:
        """Generate action items from improvements."""
        return [f"[{imp.priority.value}] {imp.description}" for imp in improvements[:5]]
    def _generate_next_steps(self, review: Review, quality: QualityScore) -> list[str]:
        """Generate next steps based on review and quality."""
        steps: list[str] = []
        if not quality.passed:
            steps.append("Address quality issues before proceeding")
            steps.append("Review failed dimensions and implement fixes")
        if review.recommendations:
            steps.extend(review.recommendations[:3])
        if quality.passed:
            steps.append("Proceed with workflow completion")
            steps.append("Document lessons learned")
        return steps
    def _persist_report(self, report: ReflectionReport) -> None:
        """Persist reflection report to storage."""
        try:
            reports_dir = self._storage_dir / "reports"
            reports_dir.mkdir(parents=True, exist_ok=True)
            report_file = reports_dir / f"report_{report.id}.json"
            with open(report_file, "w", encoding="utf-8") as f:
                json.dump(report.to_dict(), f, ensure_ascii=False, indent=2)
        except Exception:
            pass
__all__ = [
    "ReflectionSystem",
    "Review",
    "Error",
    "Analysis",
    "Improvement",
    "Correction",
    "CorrectedResult",
    "QualityScore",
    "ReflectionReport",
    "ReviewType",
    "ErrorSeverity",
    "ErrorCategory",
    "ImprovementPriority",
    "QualityDimension",
    "CorrectionStatus",
]
```

File: [85/252] openlaoke/core/hyperauto/skill_generator.py

```
"""Auto-generate skills based on task analysis.
This module provides automatic skill generation capabilities:
1. Analyze skill requirements from task descriptions
2. Design skill templates automatically
3. Generate SKILL.md files
4. Validate generated skills
5. Register new skills to the system
"""
from __future__ import annotations
import re
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path
from typing import Any, cast
import yaml
from openlaoke.core.skill_system import Skill, SkillRegistry, get_skill_registry, rescan_skills
from openlaoke.types.core_types import ValidationResult
@dataclass
class SkillParameter:
    """A parameter definition for a skill."""
    name: str
    type: str = "string"
    required: bool = True
    description: str = ""
    default: str | None = None
    choices: list[str] = field(default_factory=list)
@dataclass
class SkillTrigger:
    """A trigger pattern for skill activation."""
    command: str
    description: str = ""
    examples: list[str] = field(default_factory=list)
@dataclass
class SkillRequirement:
    """Identified requirement for a new skill."""
    task_type: str
    description: str
    keywords: list[str] = field(default_factory=list)
    suggested_tools: list[str] = field(default_factory=list)
    complexity: str = "medium"
    requires_files: bool = False
    requires_network: bool = False
    requires_code_analysis: bool = False
@dataclass
class SkillDesign:
    """Design specification for a skill to generate."""
    name: str
    description: str
    version: str = "1.0.0"
    triggers: list[SkillTrigger] = field(default_factory=list)
    parameters: list[SkillParameter] = field(default_factory=list)
    when_to_use: str = ""
    capabilities: list[str] = field(default_factory=list)
    prerequisites: list[str] = field(default_factory=list)
    examples: list[str] = field(default_factory=list)
    allowed_tools: list[str] = field(default_factory=list)
    workflow_steps: list[str] = field(default_factory=list)
@dataclass
class GenerationResult:
    """Result of skill generation."""
    success: bool
```

```
    skill_name: str = ""
    skill_path: Path | None = None
    message: str = ""
    errors: list[str] = field(default_factory=list)
class SkillGenerator:
    """Automatically generate skills based on task analysis."""
    TASK_PATTERNS: dict[str, dict[str, Any]] = {
        "code_review": {
            "keywords": ["review", "code review", "check code", "analyze code", "pr review"],
            "tools": ["read", "grep", "glob", "bash"],
            "complexity": "medium",
        },
        "testing": {
            "keywords": ["test", "testing", "write tests", "add tests", "unit test"],
            "tools": ["bash", "read", "write", "glob"],
            "complexity": "medium",
        },
        "refactoring": {
            "keywords": ["refactor", "restructure", "clean up", "improve code"],
            "tools": ["read", "edit", "bash", "grep"],
            "complexity": "high",
        },
        "documentation": {
            "keywords": ["document", "docs", "readme", "documentation", "docstring"],
            "tools": ["read", "write", "glob", "grep"],
            "complexity": "low",
        },
        "debugging": {
            "keywords": ["debug", "fix bug", "error", "issue", "problem", "traceback"],
            "tools": ["bash", "read", "grep", "glob"],
            "complexity": "high",
        },
        "deployment": {
            "keywords": ["deploy", "deployment", "release", "ship", "publish"],
            "tools": ["bash", "read", "write"],
            "complexity": "high",
        },
        "git_workflow": {
            "keywords": ["git", "commit", "branch", "merge", "rebase", "pull request"],
            "tools": ["bash"],
            "complexity": "medium",
        },
        "project_setup": {
            "keywords": ["setup", "initialize", "create project", "scaffold", "new project"],
            "tools": ["bash", "write", "glob"],
            "complexity": "medium",
        },
        "api_integration": {
            "keywords": ["api", "integration", "http", "request", "endpoint", "rest"],
            "tools": ["bash", "read", "write", "webfetch"],
            "complexity": "high",
        },
        "code_generation": {
            "keywords": ["generate", "create code", "write code", "scaffold"],
            "tools": ["write", "read", "glob"],
            "complexity": "medium",
        },
    }
    TOOL_DESCRIPTIONS: dict[str, str] = {
        "read": "Read files from the filesystem",
        "write": "Write files to the filesystem",
        "edit": "Edit existing files",
        "bash": "Execute shell commands",
        "grep": "Search file contents",
        "glob": "Find files by pattern",
```

```
    "webfetch": "Fetch content from URLs",
    "skill": "Load specialized skills",
}
def __init__(self, skill_dir: Path | None = None):
    self.skill_dir = skill_dir or Path.home() / ".openlaoke" / "skills"
    self.skill_dir.mkdir(parents=True, exist_ok=True)
    self._registry: SkillRegistry | None = None
@property
def registry(self) -> SkillRegistry:
    """Get or initialize the skill registry."""
    if self._registry is None:
        self._registry = get_skill_registry()
    return self._registry
def analyze_skill_needs(self, task_description: str) -> list[SkillRequirement]:
    """Analyze a task description to identify required skills.
    Args:
        task_description: Natural language description of the task
    Returns:
        List of identified skill requirements
    """
    requirements: list[SkillRequirement] = []
    task_lower = task_description.lower()
    matched_patterns: set[str] = set()
    for pattern_name, pattern_info in self.TASK_PATTERNS.items():
        keywords = pattern_info.get("keywords", [])
        matched_keywords = [kw for kw in keywords if kw in task_lower]
        if matched_keywords:
            requirement = SkillRequirement(
                task_type=pattern_name,
                description=f"Task involves {pattern_name.replace('_', ' ')}",
                keywords=matched_keywords,
                suggested_tools=pattern_info.get("tools", []),
                complexity=pattern_info.get("complexity", "medium"),
                requires_code_analysis=pattern_name
                in ["code_review", "refactoring", "debugging"],
                requires_network=pattern_name == "api_integration",
                requires_files=pattern_name not in ["git_workflow"],
            )
            requirements.append(requirement)
            matched_patterns.add(pattern_name)
    if not requirements:
        inferred = self._infer_requirement_from_text(task_description)
        if inferred:
            requirements.append(inferred)
    return requirements
def _infer_requirement_from_text(self, text: str) -> SkillRequirement | None:
    """Infer skill requirements from arbitrary text."""
    text_lower = text.lower()
    tools: list[str] = []
    if any(w in text_lower for w in ["file", "read", "write", "edit"]):
        tools.extend(["read", "write", "edit"])
    if any(w in text_lower for w in ["command", "run", "execute", "shell"]):
        tools.append("bash")
    if any(w in text_lower for w in ["search", "find", "grep", "pattern"]):
        tools.extend(["grep", "glob"])
    if any(w in text_lower for w in ["url", "http", "web", "api", "fetch"]):
        tools.append("webfetch")
    if not tools:
        tools = ["bash", "read", "write"]
    return SkillRequirement(
        task_type="custom",
        description=f"Custom task: {text[:100]}",
        keywords=[w for w in text.split()[:5] if len(w) > 3],
        suggested_tools=list(set(tools)),
        complexity="medium",
```

```
    )
def design_skill(self, requirement: SkillRequirement, custom_name: str = "") -> SkillDesign
:
    """Design a skill based on requirements.
    Args:
        requirement: The skill requirement to design for
        custom_name: Optional custom name for the skill
    Returns:
        SkillDesign object with the skill specification
    """
    name = custom_name or self._generate_skill_name(requirement)
    description = self._generate_description(requirement)
    triggers = self._generate_triggers(requirement, name)
    parameters = self._generate_parameters(requirement)
    when_to_use = self._generate_when_to_use(requirement)
    capabilities = self._generate_capabilities(requirement)
    workflow_steps = self._generate_workflow(requirement)
    return SkillDesign(
        name=name,
        description=description,
        triggers=triggers,
        parameters=parameters,
        when_to_use=when_to_use,
        capabilities=capabilities,
        allowed_tools=requirement.suggested_tools,
        workflow_steps=workflow_steps,
    )
def _generate_skill_name(self, requirement: SkillRequirement) -> str:
    """Generate a skill name from requirements."""
    base_name = requirement.task_type.replace(" ", "-").replace("_", "-")
    timestamp = datetime.now().strftime("%Y%m%d%H%M")
    return f"auto-{base_name}-{timestamp}"
def _generate_description(self, requirement: SkillRequirement) -> str:
    """Generate skill description."""
    task_type = requirement.task_type.replace("_", " ")
    complexity = requirement.complexity
    desc_parts = [f"Auto-generated skill for {task_type} tasks."]
    if requirement.keywords:
        keywords_str = ", ".join(requirement.keywords[:3])
        desc_parts.append(f"Keywords: {keywords_str}.")
    desc_parts.append(f"Complexity: {complexity}.")
    if requirement.requires_code_analysis:
        desc_parts.append("Requires code analysis capabilities.")
    if requirement.requires_network:
        desc_parts.append("Requires network access.")
    return " ".join(desc_parts)
def _generate_triggers(self, requirement: SkillRequirement, name: str) -> list[SkillTrigger
]:
    """Generate command triggers for the skill."""
    triggers: list[SkillTrigger] = []
    base_cmd = f"/{name.replace('auto-', '').split('-')[0]}"
    for keyword in requirement.keywords[:3]:
        cmd = f"/{keyword.replace(' ', '-').lower()}"
        trigger = SkillTrigger(
            command=cmd,
            description=f"Trigger when user mentions '{keyword}'",
            examples=[f"{cmd} <args>"],
        )
        triggers.append(trigger)
    triggers.append(
        SkillTrigger(
            command=base_cmd,
            description=f"Primary command for {requirement.task_type}",
            examples=[f"{base_cmd} [options]"],
        )
```

```
    )
    return triggers
def _generate_parameters(self, requirement: SkillRequirement) -> list[SkillParameter]:
    """Generate parameters based on task type."""
    params: list[SkillParameter] = []
    if requirement.requires_files:
        params.append(
            SkillParameter(
                name="target",
                type="string",
                required=True,
                description="Target file or directory path",
            )
        )
    if requirement.requires_code_analysis:
        params.extend(
            [
                SkillParameter(
                    name="scope",
                    type="string",
                    required=False,
                    description="Scope of analysis (file, directory, project)",
                    default="file",
                    choices=["file", "directory", "project"],
                ),
                SkillParameter(
                    name="depth",
                    type="integer",
                    required=False,
                    description="Analysis depth level",
                    default="1",
                ),
            ]
        )
    if requirement.requires_network:
        params.append(
            SkillParameter(
                name="url",
                type="string",
                required=False,
                description="Target URL for network operations",
            )
        )
    params.append(
        SkillParameter(
            name="verbose",
            type="boolean",
            required=False,
            description="Enable verbose output",
            default="false",
        )
    )
    return params
def _generate_when_to_use(self, requirement: SkillRequirement) -> str:
    """Generate when-to-use description."""
    parts = [f"Use when the task involves {requirement.task_type.replace('_', ' ')}."]
    if requirement.keywords:
        keywords = ", ".join(f'"{kw}"' for kw in requirement.keywords[:4])
        parts.append(f"Triggered by keywords: {keywords}.")
    if requirement.complexity == "high":
        parts.append("Recommended for complex tasks requiring careful analysis.")
    return " ".join(parts)
def _generate_capabilities(self, requirement: SkillRequirement) -> list[str]:
    """Generate list of capabilities."""
    capabilities: list[str] = []
```

```
    for tool in requirement.suggested_tools:
        desc = self.TOOL_DESCRIPTIONS.get(tool, f"Use {tool} tool")
        capabilities.append(desc)
    if requirement.requires_code_analysis:
        capabilities.extend(
            [
                "Analyze code structure and patterns",
                "Identify potential issues",
                "Generate improvement suggestions",
            ]
        )
    if requirement.requires_network:
        capabilities.extend(
            [
                "Make HTTP requests",
                "Fetch remote content",
                "Process API responses",
            ]
        )
    capabilities.append("Execute multi-step workflows")
    return list(set(capabilities))
def _generate_workflow(self, requirement: SkillRequirement) -> list[str]:
    """Generate workflow steps."""
    steps: list[str] = []
    task_type = requirement.task_type
    if task_type == "code_review":
        steps = [
            "Read and understand the target code",
            "Analyze code structure and patterns",
            "Check for potential issues and bugs",
            "Verify coding standards compliance",
            "Generate review feedback",
        ]
    elif task_type == "testing":
        steps = [
            "Analyze existing code structure",
            "Identify test cases to implement",
            "Write comprehensive tests",
            "Run tests to verify functionality",
            "Fix any failing tests",
        ]
    elif task_type == "debugging":
        steps = [
            "Reproduce the issue",
            "Analyze error messages and logs",
            "Identify root cause",
            "Implement fix",
            "Verify the fix works",
        ]
    elif task_type == "documentation":
        steps = [
            "Analyze code structure",
            "Generate documentation content",
            "Write documentation files",
            "Verify documentation accuracy",
        ]
    elif task_type == "refactoring":
        steps = [
            "Analyze current code structure",
            "Design refactoring approach",
            "Apply refactoring changes",
            "Run tests to verify changes",
            "Clean up and optimize",
        ]
    else:
```

```
        steps = [
            "Analyze task requirements",
            "Plan execution approach",
            "Execute main workflow",
            "Verify results",
            "Report completion",
        ]
    return steps
def generate_skill_content(self, design: SkillDesign) -> str:
    """Generate SKILL.md content from design.
    Args:
        design: The skill design specification
    Returns:
        Complete SKILL.md file content
    """
    frontmatter = self._generate_frontmatter(design)
    content = self._generate_content(design)
    return f"---\n{frontmatter}\n---\n\n{content}"
def _generate_frontmatter(self, design: SkillDesign) -> str:
    """Generate YAML frontmatter."""
    metadata: dict[str, Any] = {
        "name": design.name,
        "description": design.description,
        "version": design.version,
    }
    if design.triggers:
        metadata["triggers"] = [
            {"command": t.command, "description": t.description} for t in design.triggers
        ]
    if design.parameters:
        metadata["parameters"] = [
            {
                "name": p.name,
                "type": p.type,
                "required": p.required,
                "description": p.description,
                **({"default": p.default} if p.default else {}),
                **({"choices": p.choices} if p.choices else {}),
            }
            for p in design.parameters
        ]
    if design.allowed_tools:
        metadata["allowed-tools"] = design.allowed_tools
    return cast(str, yaml.dump(metadata, default_flow_style=False, sort_keys=False).strip()
)
def _generate_content(self, design: SkillDesign) -> str:
    """Generate skill content body."""
    sections: list[str] = []
    sections.append(f"# {design.name.replace('-', ' ').title()}\n")
    sections.append(f"{design.description}\n")
    if design.when_to_use:
        sections.append("## When to Use\n")
        sections.append(f"{design.when_to_use}\n")
    if design.capabilities:
        sections.append("## Capabilities\n")
        for cap in design.capabilities:
            sections.append(f"- {cap}")
        sections.append("")
    if design.workflow_steps:
        sections.append("## Workflow\n")
        for i, step in enumerate(design.workflow_steps, 1):
            sections.append(f"{i}. {step}")
        sections.append("")
    if design.parameters:
        sections.append("## Parameters\n")
```

```
        for param in design.parameters:
            req_str = "required" if param.required else "optional"
            sections.append(
                f"- **{param.name}** ({param.type}, {req_str}): {param.description}"
            )
            if param.default:
                sections.append(f"  - Default: `{param.default}`")
            if param.choices:
                sections.append(f"  - Choices: {', '.join(param.choices)}")
        sections.append("")
    if design.triggers:
        sections.append("## Triggers\n")
        for trigger in design.triggers:
            sections.append(f"- `{trigger.command}`: {trigger.description}")
        sections.append("")
    if design.examples:
        sections.append("## Examples\n")
        for example in design.examples:
            sections.append(f"```\n{example}\n```\n")
    sections.append("---\n")
    sections.append("*This skill was auto-generated by SkillGenerator.*\n")
    return "\n".join(sections)
def validate_skill(self, content: str) -> ValidationResult:
    """Validate generated skill content.
    Args:
        content: The skill content to validate
    Returns:
        ValidationResult indicating success or failure
    """
    errors: list[str] = []
    if not content or len(content.strip()) < 50:
        return ValidationResult(
            result=False,
            message="Skill content is too short",
            error_code=1,
        )
    if not content.startswith("---"):
        errors.append("Missing YAML frontmatter")
    parts = content.split("---", 2)
    if len(parts) < 3:
        errors.append("Invalid frontmatter format")
    if len(parts) >= 3:
        frontmatter = parts[1].strip()
        try:
            metadata = yaml.safe_load(frontmatter)
            if not metadata:
                errors.append("Empty frontmatter")
            else:
                if "name" not in metadata:
                    errors.append("Missing 'name' in frontmatter")
                if "description" not in metadata:
                    errors.append("Missing 'description' in frontmatter")
        except yaml.YAMLError as e:
            errors.append(f"Invalid YAML in frontmatter: {e}")
    body = parts[2].strip() if len(parts) >= 3 else content
    if len(body) < 100:
        errors.append("Skill body content is too short")
    if not re.search(r"^#+\s+", body, re.MULTILINE):
        errors.append("Missing markdown headers in body")
    if errors:
        return ValidationResult(
            result=False,
            message="; ".join(errors),
            error_code=2,
        )
```

```
    return ValidationResult(result=True, message="Skill validation passed")
def register_skill(self, content: str, name: str, overwrite: bool = False) -> GenerationRes
ult:
    """Register a new skill to the system.
    Args:
        content: The SKILL.md content
        name: Name for the skill
        overwrite: Whether to overwrite existing skill
    Returns:
        GenerationResult with registration status
    """
    skill_path = self.skill_dir / name / "SKILL.md"
    if skill_path.exists() and not overwrite:
        return GenerationResult(
            success=False,
            skill_name=name,
            skill_path=skill_path,
            message=f"Skill '{name}' already exists. Use overwrite=True to replace.",
            errors=["Skill already exists"],
        )
    validation = self.validate_skill(content)
    if not validation.result:
        return GenerationResult(
            success=False,
            skill_name=name,
            message=f"Validation failed: {validation.message}",
            errors=[validation.message],
        )
    try:
        skill_path.parent.mkdir(parents=True, exist_ok=True)
        skill_path.write_text(content, encoding="utf-8")
        skill_count = rescan_skills()
        return GenerationResult(
            success=True,
            skill_name=name,
            skill_path=skill_path,
            message=f"Skill '{name}' registered successfully. Total skills: {skill_count}",
        )
    except Exception as e:
        return GenerationResult(
            success=False,
            skill_name=name,
            message=f"Failed to register skill: {e}",
            errors=[str(e)],
        )
def auto_generate_for_task(
    self,
    task: str,
    custom_name: str = "",
    register: bool = True,
) -> list[Skill]:
    """Auto-generate skills for a given task.
    This is the main entry point for automatic skill generation.
    Args:
        task: Task description to analyze
        custom_name: Optional custom name for the skill
        register: Whether to register generated skills
    Returns:
        List of generated Skill objects
    """
    requirements = self.analyze_skill_needs(task)
    generated_skills: list[Skill] = []
    for requirement in requirements:
        design = self.design_skill(requirement, custom_name)
        content = self.generate_skill_content(design)
```

```
        if register:
            result = self.register_skill(content, design.name)
            if result.success:
                skill = Skill.from_content(content, result.skill_path)
                if skill:
                    generated_skills.append(skill)
        else:
            skill = Skill.from_content(content)
            if skill:
                generated_skills.append(skill)
    return generated_skills
def generate_skill_from_template(
    self,
    name: str,
    description: str,
    triggers: list[str] | None = None,
    capabilities: list[str] | None = None,
    workflow: list[str] | None = None,
    tools: list[str] | None = None,
    when_to_use: str = "",
    parameters: list[dict[str, Any]] | None = None,
) -> GenerationResult:
    """Generate a skill from explicit template parameters.
    Args:
        name: Skill name
        description: Skill description
        triggers: List of trigger commands
        capabilities: List of capabilities
        workflow: List of workflow steps
        tools: List of allowed tools
        when_to_use: When to use description
        parameters: List of parameter definitions
    Returns:
        GenerationResult with generation status
    """
    design = SkillDesign(
        name=name,
        description=description,
        when_to_use=when_to_use or f"Use when you need to {description.lower()}",
        capabilities=capabilities or ["Execute tasks"],
        allowed_tools=tools or ["bash", "read", "write"],
        workflow_steps=workflow or ["Execute task", "Verify results"],
    )
    if triggers:
        design.triggers = [
            SkillTrigger(command=t, description=f"Trigger for {t}") for t in triggers
        ]
    if parameters:
        design.parameters = [
            SkillParameter(
                name=p.get("name", "param"),
                type=p.get("type", "string"),
                required=p.get("required", True),
                description=p.get("description", ""),
                default=p.get("default"),
                choices=p.get("choices", []),
            )
            for p in parameters
        ]
    content = self.generate_skill_content(design)
    return self.register_skill(content, name)
def list_generated_skills(self) -> list[dict[str, Any]]:
    """List all auto-generated skills."""
    skills: list[dict[str, Any]] = []
    if not self.skill_dir.exists():
```

```
        return skills
    for skill_dir in sorted(self.skill_dir.iterdir()):
        if not skill_dir.is_dir():
            continue
        skill_file = skill_dir / "SKILL.md"
        if not skill_file.exists():
            continue
        content = skill_file.read_text(encoding="utf-8")
        if "auto-generated by SkillGenerator" in content.lower():
            skill = Skill.from_file(skill_file)
            if skill:
                skills.append(
                    {
                        "name": skill.name,
                        "description": skill.description,
                        "path": str(skill_file),
                    }
                )
    return skills
def remove_generated_skill(self, name: str) -> bool:
    """Remove an auto-generated skill.
    Args:
        name: Name of the skill to remove
    Returns:
        True if removed successfully
    """
    import shutil
    skill_path = self.skill_dir / name
    if skill_path.exists():
        shutil.rmtree(skill_path)
        rescan_skills()
        return True
    return False
```

File: [86/252] openlaoke/core/hyperauto/task_executor.py

```
"""Execute HyperAuto tasks with tool calling support."""
from __future__ import annotations
import json
from typing import TYPE_CHECKING, Any
from openlaoke.core.hyperauto.executor import ToolExecutor
from openlaoke.core.hyperauto.types import SubTask
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
async def execute_task_with_tools(
    app_state: AppState,
    task: SubTask,
    original_request: str,
    max_iterations: int = 10,
    on_progress_callback: Any = None,
    workflow_context: Any = None,
) -> dict[str, Any]:
    """Execute a task using AI with tool calling capability."""
    from openlaoke.core.multi_provider_api import MultiProviderClient
    if app_state.multi_provider_config:
        api_client = MultiProviderClient(app_state.multi_provider_config)
        model = app_state.multi_provider_config.get_active_model()
    else:
        from openlaoke.types.providers import MultiProviderConfig
        config = MultiProviderConfig.defaults()
        api_client = MultiProviderClient(config)
        model = "gemma3:1b"
    executor = ToolExecutor(app_state)
    tool_schemas = await executor.get_tool_schemas()
    system_prompt = f"""You are an AI coding assistant executing a SPECIFIC sub-task.
Overall goal: {original_request}
Your current sub-task: {task.name}
Sub-task description: {task.description}
CRITICAL INSTRUCTIONS:
1. This sub-task is part of a larger workflow - you MUST complete THIS specific sub-task
2. If task.name contains a specific file/module name, you MUST create that exact file
3. Do NOT stop after creating just one example file - complete what THIS sub-task requires
4. Use Write tool to create complete, functional code files
5. Use Bash (mkdir) to create directories first if needed
6. If this is a translation task, read source files and write complete translated versions
7. Do NOT just create placeholders - create REAL, WORKING implementations
Available tools: {", ".join([t["name"] for t in tool_schemas[:10]])}
IMPORTANT: Your goal is to complete THIS specific sub-task fully, not partially. Do not stop ea
    rly."""
    messages = [
        {
            "role": "user",
            "content": f"Execute this sub-task:\n\nSub-task: {task.name}\nDescription: {task.de
    scription}\nContext: {original_request}\n\nCreate the actual files/code required for this s
    ub-task.",
        }
    ]
    iteration = 0
    total_tokens = 0
    all_results: list[dict[str, Any]] = []
    while iteration < max_iterations:
        iteration += 1
        try:
            response_msg, token_usage, _ = await api_client.send_message(
                system_prompt=system_prompt,
                messages=messages,
                tools=tool_schemas,
```

```
            model=model,
            max_tokens=4000,
        )
        if token_usage:
            total_tokens += token_usage.total_tokens
            if workflow_context:
                workflow_context.total_tokens += token_usage.total_tokens
            if on_progress_callback:
                on_progress_callback(workflow_context)
        tool_uses = []
        if hasattr(response_msg, "tool_uses") and response_msg.tool_uses:
            tool_uses = response_msg.tool_uses
        tool_calls = []
        for tool_use in tool_uses:
            tool_calls.append(
                {"name": tool_use.name, "input": tool_use.input, "id": tool_use.id}
            )
        if not tool_calls:
            final_text = ""
            if hasattr(response_msg, "content"):
                if isinstance(response_msg.content, str):
                    final_text = response_msg.content
                elif hasattr(response_msg.content, "text"):
                    final_text = response_msg.content.text
            return {
                "success": True,
                "iterations": iteration,
                "tokens": total_tokens,
                "tool_results": all_results,
                "final_response": final_text,
            }
        for tool_call in tool_calls:
            tool_name: str = ""
            tool_input_dict: dict[str, Any] = {}
            input_data: Any = {}
            if isinstance(tool_call, dict):
                func_data = tool_call.get("function", {})
                if isinstance(func_data, dict):
                    name_val = tool_call.get("name")
                    tool_name = str(name_val) if name_val else str(func_data.get("name", ""
))
                    input_data = tool_call.get("input") or func_data.get("arguments", {})
            else:
                tool_name = str(getattr(tool_call, "name", ""))
                input_data = getattr(tool_call, "input", getattr(tool_call, "arguments", {}
))
            if isinstance(input_data, str):
                try:
                    tool_input_dict = json.loads(input_data)
                except json.JSONDecodeError:
                    tool_input_dict = {}
            elif isinstance(input_data, dict):
                tool_input_dict = input_data
            result = await executor.execute_tool(tool_name, tool_input_dict)
            result_text = (
                result.content
                if isinstance(result.content, str)
                else json.dumps(result.content)
            )
            all_results.append(
                {
                    "tool": tool_name,
                    "input": tool_input_dict,
                    "result": result_text[:1000],
                    "is_error": result.is_error,
```

```
                }
            )
            messages.append(
                {
                    "role": "assistant",
                    "content": "",
                    "tool_calls": json.dumps(
                        [
                            {
                                "id": f"call_{iteration}_{tool_name}",
                                "type": "function",
                                "function": {
                                    "name": tool_name,
                                    "arguments": json.dumps(tool_input_dict),
                                },
                            }
                        ]
                    ),
                }
            )
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": f"call_{iteration}_{tool_name}",
                    "content": result_text[:2000],
                }
            )
    except Exception:
        break
return {
    "success": True,
    "iterations": iteration,
    "tokens": total_tokens,
    "tool_results": all_results,
    "completed": iteration >= max_iterations,
}
```

File: [87/252] openlaoke/core/hyperauto/test_runner.py

```
"""AutoTestRunner - Automated test detection, execution, and analysis.
This module provides comprehensive automated testing capabilities:
1. Test framework detection (pytest, jest, go test, cargo test, junit)
2. Test discovery and collection
3. Test execution with result capture
4. Coverage analysis
5. Failure diagnosis and auto-fix suggestions
"""
from __future__ import annotations
import asyncio
import json
import os
import re
import subprocess
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any, cast
from uuid import uuid4
class TestFramework(StrEnum):
    """Supported test frameworks."""
    PYTEST = "pytest"
    JEST = "jest"
    MOCHA = "mocha"
    GO_TEST = "go_test"
    CARGO_TEST = "cargo_test"
    JUNIT = "junit"
    UNKNOWN = "unknown"
class TestStatus(StrEnum):
    """Status of a test."""
    PASSED = "passed"
    FAILED = "failed"
    SKIPPED = "skipped"
    ERROR = "error"
    PENDING = "pending"
    RUNNING = "running"
class TestSeverity(StrEnum):
    """Severity of test failure."""
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
    COSMETIC = "cosmetic"
@dataclass
class TestInfo:
    """Information about a single test."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    name: str = ""
    file_path: Path | None = None
    framework: TestFramework = TestFramework.UNKNOWN
    status: TestStatus = TestStatus.PENDING
    duration_ms: float = 0.0
    message: str = ""
    traceback: str = ""
    markers: list[str] = field(default_factory=list)
    dependencies: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "name": self.name,
            "file_path": str(self.file_path) if self.file_path else None,
```

```
            "framework": self.framework.value,
            "status": self.status.value,
            "duration_ms": self.duration_ms,
            "message": self.message,
            "traceback": self.traceback,
            "markers": self.markers,
            "dependencies": self.dependencies,
        }
@dataclass
class TestFailure:
    """Detailed information about a test failure."""
    test_id: str
    test_name: str
    file_path: Path | None = None
    error_type: str = ""
    error_message: str = ""
    traceback: str = ""
    line_number: int | None = None
    expected: str = ""
    actual: str = ""
    severity: TestSeverity = TestSeverity.MEDIUM
    context: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "test_id": self.test_id,
            "test_name": self.test_name,
            "file_path": str(self.file_path) if self.file_path else None,
            "error_type": self.error_type,
            "error_message": self.error_message,
            "traceback": self.traceback,
            "line_number": self.line_number,
            "expected": self.expected,
            "actual": self.actual,
            "severity": self.severity.value,
            "context": self.context,
        }
@dataclass
class TestResults:
    """Results from running a test suite."""
    total_tests: int = 0
    passed: int = 0
    failed: int = 0
    skipped: int = 0
    errors: int = 0
    duration_ms: float = 0.0
    tests: list[TestInfo] = field(default_factory=list)
    failures: list[TestFailure] = field(default_factory=list)
    stdout: str = ""
    stderr: str = ""
    exit_code: int = 0
    framework: TestFramework = TestFramework.UNKNOWN
    timestamp: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "total_tests": self.total_tests,
            "passed": self.passed,
            "failed": self.failed,
            "skipped": self.skipped,
            "errors": self.errors,
            "duration_ms": self.duration_ms,
            "tests": [t.to_dict() for t in self.tests],
            "failures": [f.to_dict() for f in self.failures],
            "stdout": self.stdout,
            "stderr": self.stderr,
            "exit_code": self.exit_code,
```

```
            "framework": self.framework.value,
            "timestamp": self.timestamp,
        }
    @property
    def success_rate(self) -> float:
        if self.total_tests == 0:
            return 0.0
        return self.passed / self.total_tests * 100
    @property
    def is_success(self) -> bool:
        return self.failed == 0 and self.errors == 0
@dataclass
class TestAnalysis:
    """Analysis of test results."""
    success_rate: float = 0.0
    avg_duration_ms: float = 0.0
    slow_tests: list[TestInfo] = field(default_factory=list)
    flaky_tests: list[TestInfo] = field(default_factory=list)
    failure_patterns: list[str] = field(default_factory=list)
    recommendations: list[str] = field(default_factory=list)
    coverage_percentage: float = 0.0
    quality_score: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "success_rate": self.success_rate,
            "avg_duration_ms": self.avg_duration_ms,
            "slow_tests": [t.to_dict() for t in self.slow_tests],
            "flaky_tests": [t.to_dict() for t in self.flaky_tests],
            "failure_patterns": self.failure_patterns,
            "recommendations": self.recommendations,
            "coverage_percentage": self.coverage_percentage,
            "quality_score": self.quality_score,
        }
@dataclass
class CoverageReport:
    """Coverage report data."""
    total_lines: int = 0
    covered_lines: int = 0
    percentage: float = 0.0
    files: dict[str, dict[str, Any]] = field(default_factory=dict)
    uncovered_files: list[str] = field(default_factory=list)
    branch_coverage: float = 0.0
    function_coverage: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "total_lines": self.total_lines,
            "covered_lines": self.covered_lines,
            "percentage": self.percentage,
            "files": self.files,
            "uncovered_files": self.uncovered_files,
            "branch_coverage": self.branch_coverage,
            "function_coverage": self.function_coverage,
        }
@dataclass
class Diagnosis:
    """Diagnosis of a test failure."""
    failure_id: str
    root_cause: str = ""
    confidence: float = 0.0
    suggested_fix: str = ""
    fix_type: str = ""
    affected_files: list[Path] = field(default_factory=list)
    fix_commands: list[str] = field(default_factory=list)
    references: list[str] = field(default_factory=list)
    auto_fixable: bool = False
```

```
    def to_dict(self) -> dict[str, Any]:
        return {
            "failure_id": self.failure_id,
            "root_cause": self.root_cause,
            "confidence": self.confidence,
            "suggested_fix": self.suggested_fix,
            "fix_type": self.fix_type,
            "affected_files": [str(f) for f in self.affected_files],
            "fix_commands": self.fix_commands,
            "references": self.references,
            "auto_fixable": self.auto_fixable,
        }
@dataclass
class TestCycleResult:
    """Result of a full test cycle."""
    success: bool
    framework: TestFramework
    results: TestResults | None = None
    analysis: TestAnalysis | None = None
    coverage: CoverageReport | None = None
    diagnoses: list[Diagnosis] = field(default_factory=list)
    fixes_applied: list[str] = field(default_factory=list)
    iterations: int = 0
    total_duration_ms: float = 0.0
    message: str = ""
    def to_dict(self) -> dict[str, Any]:
        return {
            "success": self.success,
            "framework": self.framework.value,
            "results": self.results.to_dict() if self.results else None,
            "analysis": self.analysis.to_dict() if self.analysis else None,
            "coverage": self.coverage.to_dict() if self.coverage else None,
            "diagnoses": [d.to_dict() for d in self.diagnoses],
            "fixes_applied": self.fixes_applied,
            "iterations": self.iterations,
            "total_duration_ms": self.total_duration_ms,
            "message": self.message,
        }
class AutoTestRunner:
    """Automated test detection, execution, and analysis system."""
    FRAMEWORK_CONFIGS: dict[TestFramework, dict[str, Any]] = {
        TestFramework.PYTEST: {
            "detect_files": ["pytest.ini", "pyproject.toml", "setup.cfg", "conftest.py"],
            "test_patterns": ["test_*.py", "*_test.py", "tests/*.py"],
            "run_command": ["pytest", "-v", "--tb=short"],
            "coverage_command": ["pytest", "--cov=.", "--cov-report=json"],
            "result_parser": "pytest",
        },
        TestFramework.JEST: {
            "detect_files": ["jest.config.js", "jest.config.ts", "package.json"],
            "test_patterns": ["*.test.js", "*.test.ts", "*.spec.js", "*.spec.ts"],
            "run_command": ["npm", "test", "--", "--verbose"],
            "coverage_command": ["npm", "test", "--", "--coverage", "--json"],
            "result_parser": "jest",
        },
        TestFramework.MOCHA: {
            "detect_files": ["mocha.opts", ".mocharc.json", ".mocharc.js", "package.json"],
            "test_patterns": ["*.test.js", "*.spec.js", "test/*.js"],
            "run_command": ["npm", "test"],
            "coverage_command": ["npx", "nyc", "--reporter=json", "npm", "test"],
            "result_parser": "mocha",
        },
        TestFramework.GO_TEST: {
            "detect_files": ["go.mod", "*_test.go"],
            "test_patterns": ["*_test.go"],
```

```
        "run_command": ["go", "test", "-v", "./..."],
        "coverage_command": ["go", "test", "-coverprofile=coverage.out", "./..."],
        "result_parser": "go",
    },
    TestFramework.CARGO_TEST: {
        "detect_files": ["Cargo.toml"],
        "test_patterns": ["tests/*.rs", "*_test.rs"],
        "run_command": ["cargo", "test", "--", "--nocapture"],
        "coverage_command": ["cargo", "tarpaulin", "--out", "Json"],
        "result_parser": "cargo",
    },
    TestFramework.JUNIT: {
        "detect_files": ["pom.xml", "build.gradle", "build.gradle.kts"],
        "test_patterns": ["*Test.java", "*Tests.java", "Test*.java"],
        "run_command": ["mvn", "test"],
        "coverage_command": ["mvn", "jacoco:report"],
        "result_parser": "junit",
    },
}
FAILURE_PATTERNS: dict[str, dict[str, Any]] = {
    "assertion_error": {
        "patterns": ["AssertionError", "assert", "expected", "actual"],
        "fix_type": "assertion",
        "auto_fixable": True,
    },
    "import_error": {
        "patterns": ["ImportError", "ModuleNotFoundError", "cannot find module"],
        "fix_type": "dependency",
        "auto_fixable": True,
    },
    "attribute_error": {
        "patterns": ["AttributeError", "has no attribute", "undefined"],
        "fix_type": "implementation",
        "auto_fixable": True,
    },
    "type_error": {
        "patterns": ["TypeError", "type mismatch", "wrong type"],
        "fix_type": "type_fix",
        "auto_fixable": True,
    },
    "value_error": {
        "patterns": ["ValueError", "invalid value", "invalid argument"],
        "fix_type": "value_fix",
        "auto_fixable": False,
    },
    "syntax_error": {
        "patterns": ["SyntaxError", "syntax error", "parse error"],
        "fix_type": "syntax",
        "auto_fixable": True,
    },
    "timeout_error": {
        "patterns": ["TimeoutError", "timeout", "timed out"],
        "fix_type": "performance",
        "auto_fixable": False,
    },
    "null_reference": {
        "patterns": ["NullPointerException", "NoneType", "null pointer", "undefined"],
        "fix_type": "null_check",
        "auto_fixable": True,
    },
}
def detect_test_framework(self, project_path: Path) -> TestFramework:
    """Detect the test framework used in a project."""
    if not project_path.exists() or not project_path.is_dir():
        return TestFramework.UNKNOWN
```

```
    files = set(os.listdir(project_path))
    for framework, config in self.FRAMEWORK_CONFIGS.items():
        if framework == TestFramework.UNKNOWN:
            continue
        detect_files = config.get("detect_files", [])
        for detect_file in detect_files:
            if detect_file in files:
                if detect_file == "package.json":
                    return self._detect_js_framework(project_path / detect_file)
                if detect_file in ("pom.xml", "build.gradle", "build.gradle.kts"):
                    return TestFramework.JUNIT
                return framework
    for framework, config in self.FRAMEWORK_CONFIGS.items():
        if framework == TestFramework.UNKNOWN:
            continue
        test_patterns = config.get("test_patterns", [])
        for pattern in test_patterns:
            matches = list(project_path.glob(pattern))
            if matches:
                return framework
    return TestFramework.UNKNOWN
def _detect_js_framework(self, package_json: Path) -> TestFramework:
    """Detect JavaScript test framework from package.json."""
    try:
        data = json.loads(package_json.read_text(encoding="utf-8"))
        dev_deps = data.get("devDependencies", {})
        deps = data.get("dependencies", {})
        all_deps = {**deps, **dev_deps}
        if "jest" in all_deps:
            return TestFramework.JEST
        if "mocha" in all_deps:
            return TestFramework.MOCHA
        scripts = data.get("scripts", {})
        test_script = scripts.get("test", "")
        if "jest" in test_script:
            return TestFramework.JEST
        if "mocha" in test_script:
            return TestFramework.MOCHA
    except Exception:
        pass
    return TestFramework.UNKNOWN
def discover_tests(self, project_path: Path) -> list[TestInfo]:
    """Discover all tests in a project."""
    framework = self.detect_test_framework(project_path)
    if framework == TestFramework.UNKNOWN:
        return []
    config = self.FRAMEWORK_CONFIGS.get(framework, {})
    test_patterns = config.get("test_patterns", [])
    tests: list[TestInfo] = []
    for pattern in test_patterns:
        for test_file in project_path.glob(pattern):
            if test_file.is_file():
                file_tests = self._extract_tests_from_file(test_file, framework)
                tests.extend(file_tests)
    return tests
def _extract_tests_from_file(self, file_path: Path, framework: TestFramework) -> list[TestI
nfo]:
    """Extract individual tests from a test file."""
    tests: list[TestInfo] = []
    try:
        content = file_path.read_text(encoding="utf-8")
    except Exception:
        return tests
    if framework == TestFramework.PYTEST:
        tests.extend(self._extract_pytest_tests(file_path, content))
```

```
    elif framework in (TestFramework.JEST, TestFramework.MOCHA):
        tests.extend(self._extract_js_tests(file_path, content))
    elif framework == TestFramework.GO_TEST:
        tests.extend(self._extract_go_tests(file_path, content))
    elif framework == TestFramework.CARGO_TEST:
        tests.extend(self._extract_rust_tests(file_path, content))
    elif framework == TestFramework.JUNIT:
        tests.extend(self._extract_java_tests(file_path, content))
    return tests
def _extract_pytest_tests(self, file_path: Path, content: str) -> list[TestInfo]:
    """Extract pytest test functions."""
    tests: list[TestInfo] = []
    class_pattern = re.compile(r"class\s+(\w+(?:Test|Tests)\w*)\s*[:\(]")
    func_pattern = re.compile(r"def\s+(test_\w+)\s*\(")
    classes = class_pattern.findall(content)
    funcs = func_pattern.findall(content)
    for func in funcs:
        test = TestInfo(
            name=func,
            file_path=file_path,
            framework=TestFramework.PYTEST,
            status=TestStatus.PENDING,
        )
        tests.append(test)
    for cls in classes:
        class_content = self._extract_class_content(content, cls)
        class_funcs = func_pattern.findall(class_content)
        for func in class_funcs:
            test = TestInfo(
                name=f"{cls}.{func}",
                file_path=file_path,
                framework=TestFramework.PYTEST,
                status=TestStatus.PENDING,
            )
            tests.append(test)
    return tests
def _extract_class_content(self, content: str, class_name: str) -> str:
    """Extract content of a specific class."""
    pattern = re.compile(rf"class\s+{class_name}\s*[:\(].*?(?=class\s+\w|\Z)", re.DOTALL)
    match = pattern.search(content)
    return match.group(0) if match else ""
def _extract_js_tests(self, file_path: Path, content: str) -> list[TestInfo]:
    """Extract JavaScript test definitions."""
    tests: list[TestInfo] = []
    patterns = [
        re.compile(r"(?:test|it)\s*\(\s*['\"]([^'\"]+)['\"]"),
        re.compile(r"describe\s*\(\s*['\"]([^'\"]+)['\"]"),
    ]
    for pattern in patterns:
        matches = pattern.findall(content)
        for match in matches:
            test = TestInfo(
                name=match,
                file_path=file_path,
                framework=TestFramework.JEST,
                status=TestStatus.PENDING,
            )
            tests.append(test)
    return tests
def _extract_go_tests(self, file_path: Path, content: str) -> list[TestInfo]:
    """Extract Go test functions."""
    tests: list[TestInfo] = []
    pattern = re.compile(r"func\s+(Test\w+)\s*\(")
    matches = pattern.findall(content)
    for match in matches:
```

```
        test = TestInfo(
            name=match,
            file_path=file_path,
            framework=TestFramework.GO_TEST,
            status=TestStatus.PENDING,
        )
        tests.append(test)
    return tests
def _extract_rust_tests(self, file_path: Path, content: str) -> list[TestInfo]:
    """Extract Rust test functions."""
    tests: list[TestInfo] = []
    patterns = [
        re.compile(r"#\[test\]\s*fn\s+(\w+)"),
        re.compile(r"#\[cfg\(test\)\]\s*mod\s+tests\s*\{([^}]+)\}"),
    ]
    for pattern in patterns:
        matches = pattern.findall(content)
        for match in matches:
            if isinstance(match, tuple):
                match = match[0]
            test = TestInfo(
                name=match,
                file_path=file_path,
                framework=TestFramework.CARGO_TEST,
                status=TestStatus.PENDING,
            )
            tests.append(test)
    return tests
def _extract_java_tests(self, file_path: Path, content: str) -> list[TestInfo]:
    """Extract Java test methods."""
    tests: list[TestInfo] = []
    patterns = [
        re.compile(r"@Test\s*public\s+void\s+(\w+)\s*\("),
        re.compile(r"public\s+void\s+(test\w+)\s*\("),
    ]
    for pattern in patterns:
        matches = pattern.findall(content)
        for match in matches:
            test = TestInfo(
                name=match,
                file_path=file_path,
                framework=TestFramework.JUNIT,
                status=TestStatus.PENDING,
            )
            tests.append(test)
    return tests
def run_tests(self, tests: list[TestInfo], project_path: Path) -> TestResults:
    """Run discovered tests and collect results."""
    if not tests:
        return TestResults(framework=TestFramework.UNKNOWN)
    framework = self._get_common_framework(tests)
    config = self.FRAMEWORK_CONFIGS.get(framework, {})
    command = config.get("run_command", [])
    if not command:
        return TestResults(framework=framework)
    try:
        result = subprocess.run(
            command,
            cwd=project_path,
            capture_output=True,
            text=True,
            timeout=600,
        )
        parsed_results = self._parse_test_results(result.stdout, result.stderr, framework)
        parsed_results.exit_code = result.returncode
```

```
        parsed_results.stdout = result.stdout
        parsed_results.stderr = result.stderr
        parsed_results.framework = framework
        return parsed_results
    except subprocess.TimeoutExpired:
        return TestResults(
            framework=framework,
            stderr="Test execution timed out",
            exit_code=-1,
        )
    except Exception as e:
        return TestResults(
            framework=framework,
            stderr=str(e),
            exit_code=-1,
        )
def _get_common_framework(self, tests: list[TestInfo]) -> TestFramework:
    """Get the common framework from tests."""
    if not tests:
        return TestFramework.UNKNOWN
    return tests[0].framework
def _parse_test_results(
    self, stdout: str, stderr: str, framework: TestFramework
) -> TestResults:
    """Parse test results from output."""
    if framework == TestFramework.PYTEST:
        return self._parse_pytest_results(stdout, stderr)
    elif framework in (TestFramework.JEST, TestFramework.MOCHA):
        return self._parse_js_results(stdout, stderr)
    elif framework == TestFramework.GO_TEST:
        return self._parse_go_results(stdout, stderr)
    elif framework == TestFramework.CARGO_TEST:
        return self._parse_cargo_results(stdout, stderr)
    elif framework == TestFramework.JUNIT:
        return self._parse_junit_results(stdout, stderr)
    return TestResults(framework=framework)
def _parse_pytest_results(self, stdout: str, stderr: str) -> TestResults:
    """Parse pytest output."""
    results = TestResults(framework=TestFramework.PYTEST)
    passed_pattern = re.compile(r"(\d+)\s+passed")
    failed_pattern = re.compile(r"(\d+)\s+failed")
    skipped_pattern = re.compile(r"(\d+)\s+skipped")
    error_pattern = re.compile(r"(\d+)\s+error")
    passed_match = passed_pattern.search(stdout)
    failed_match = failed_pattern.search(stdout)
    skipped_match = skipped_pattern.search(stdout)
    error_match = error_pattern.search(stdout)
    if passed_match:
        results.passed = int(passed_match.group(1))
    if failed_match:
        results.failed = int(failed_match.group(1))
    if skipped_match:
        results.skipped = int(skipped_match.group(1))
    if error_match:
        results.errors = int(error_match.group(1))
    results.total_tests = results.passed + results.failed + results.skipped + results.error
s
    failure_pattern = re.compile(r"FAILED\s+([^\s]+)\s*-")
    for match in failure_pattern.finditer(stdout):
        failure = TestFailure(
            test_id=uuid4().hex[:8],
            test_name=match.group(1),
            severity=TestSeverity.HIGH,
        )
        results.failures.append(failure)
```

```
    duration_pattern = re.compile(r"in\s+([\d.]+)\s*s")
    duration_match = duration_pattern.search(stdout)
    if duration_match:
        results.duration_ms = float(duration_match.group(1)) * 1000
    return results
def _parse_js_results(self, stdout: str, stderr: str) -> TestResults:
    """Parse Jest/Mocha output."""
    results = TestResults(framework=TestFramework.JEST)
    passed_pattern = re.compile(r"(\d+)\s+passing")
    failed_pattern = re.compile(r"(\d+)\s+failing")
    skipped_pattern = re.compile(r"(\d+)\s+skipped")
    passed_match = passed_pattern.search(stdout + stderr)
    failed_match = failed_pattern.search(stdout + stderr)
    skipped_match = skipped_pattern.search(stdout + stderr)
    if passed_match:
        results.passed = int(passed_match.group(1))
    if failed_match:
        results.failed = int(failed_match.group(1))
    if skipped_match:
        results.skipped = int(skipped_match.group(1))
    results.total_tests = results.passed + results.failed + results.skipped
    return results
def _parse_go_results(self, stdout: str, stderr: str) -> TestResults:
    """Parse go test output."""
    results = TestResults(framework=TestFramework.GO_TEST)
    output = stdout + stderr
    lines = output.splitlines()
    for line in lines:
        if "PASS" in line:
            results.passed += 1
        elif "FAIL" in line:
            results.failed += 1
        elif "SKIP" in line:
            results.skipped += 1
    results.total_tests = results.passed + results.failed + results.skipped
    return results
def _parse_cargo_results(self, stdout: str, stderr: str) -> TestResults:
    """Parse cargo test output."""
    results = TestResults(framework=TestFramework.CARGO_TEST)
    output = stdout + stderr
    passed_pattern = re.compile(r"test result: ok\. (\d+) passed")
    failed_pattern = re.compile(r"test result: FAILED\. (\d+) failed")
    passed_match = passed_pattern.search(output)
    failed_match = failed_pattern.search(output)
    if passed_match:
        results.passed = int(passed_match.group(1))
    if failed_match:
        results.failed = int(failed_match.group(1))
    results.total_tests = results.passed + results.failed
    return results
def _parse_junit_results(self, stdout: str, stderr: str) -> TestResults:
    """Parse Maven/Gradle test output."""
    results = TestResults(framework=TestFramework.JUNIT)
    output = stdout + stderr
    tests_pattern = re.compile(r"Tests run:\s*(\d+)")
    failures_pattern = re.compile(r"Failures:\s*(\d+)")
    errors_pattern = re.compile(r"Errors:\s*(\d+)")
    skipped_pattern = re.compile(r"Skipped:\s*(\d+)")
    tests_match = tests_pattern.search(output)
    failures_match = failures_pattern.search(output)
    errors_match = errors_pattern.search(output)
    skipped_match = skipped_pattern.search(output)
    if tests_match:
        results.total_tests = int(tests_match.group(1))
    if failures_match:
```

```
        results.failed = int(failures_match.group(1))
    if errors_match:
        results.errors = int(errors_match.group(1))
    if skipped_match:
        results.skipped = int(skipped_match.group(1))
    if results.total_tests == 0:
        results.passed = results.total_tests - results.failed - results.errors - results.sk
ipped
    return results
def analyze_results(self, results: TestResults) -> TestAnalysis:
    """Analyze test results and generate insights."""
    analysis = TestAnalysis()
    analysis.success_rate = results.success_rate
    if results.tests:
        total_duration = sum(t.duration_ms for t in results.tests)
        analysis.avg_duration_ms = total_duration / len(results.tests)
        threshold = analysis.avg_duration_ms * 3
        analysis.slow_tests = [t for t in results.tests if t.duration_ms > threshold]
    analysis.failure_patterns = self._identify_failure_patterns(results.failures)
    analysis.recommendations = self._generate_recommendations(results, analysis)
    analysis.quality_score = self._calculate_quality_score(results, analysis)
    return analysis
def _identify_failure_patterns(self, failures: list[TestFailure]) -> list[str]:
    """Identify common patterns in failures."""
    patterns: list[str] = []
    error_types: dict[str, int] = {}
    for failure in failures:
        error_type = failure.error_type or "unknown"
        error_types[error_type] = error_types.get(error_type, 0) + 1
    for error_type, count in sorted(error_types.items(), key=lambda x: -x[1]):
        if count > 1:
            patterns.append(f"{error_type}: {count} occurrences")
    return patterns
def _generate_recommendations(self, results: TestResults, analysis: TestAnalysis) -> list[s
tr]:
    """Generate recommendations based on analysis."""
    recommendations: list[str] = []
    if results.failed > 0:
        recommendations.append("Fix failing tests before proceeding")
    if analysis.slow_tests:
        recommendations.append(f"Optimize {len(analysis.slow_tests)} slow tests")
    if results.skipped > results.total_tests * 0.1:
        recommendations.append("Reduce number of skipped tests")
    if analysis.success_rate < 80:
        recommendations.append("Test success rate is below 80%, investigate failures")
    if analysis.failure_patterns:
        recommendations.append(
            f"Address common failure patterns: {analysis.failure_patterns[0]}"
        )
    if analysis.coverage_percentage < 70:
        recommendations.append("Increase test coverage to at least 70%")
    return recommendations
def _calculate_quality_score(self, results: TestResults, analysis: TestAnalysis) -> float:
    """Calculate overall quality score."""
    score = 0.0
    success_weight = 0.4
    coverage_weight = 0.3
    speed_weight = 0.2
    stability_weight = 0.1
    score += analysis.success_rate * success_weight
    coverage = results.coverage_percentage if hasattr(results, "coverage_percentage") else
0
    score += coverage * coverage_weight
    if analysis.avg_duration_ms > 0:
        speed_score = min(100, 1000 / analysis.avg_duration_ms * 10)
```

```
        score += speed_score * speed_weight
    stability_score = 100 if not analysis.flaky_tests else 100 - len(analysis.flaky_tests)
* 10
    score += stability_score * stability_weight
    return min(100, max(0, score))
def check_coverage(self, project_path: Path) -> CoverageReport:
    """Check test coverage for a project."""
    framework = self.detect_test_framework(project_path)
    if framework == TestFramework.UNKNOWN:
        return CoverageReport()
    config = self.FRAMEWORK_CONFIGS.get(framework, {})
    coverage_command = config.get("coverage_command", [])
    if not coverage_command:
        return CoverageReport()
    try:
        result = subprocess.run(
            coverage_command,
            cwd=project_path,
            capture_output=True,
            text=True,
            timeout=300,
        )
        return self._parse_coverage_results(
            result.stdout, result.stderr, framework, project_path
        )
    except Exception:
        return CoverageReport()
def _parse_coverage_results(
    self, stdout: str, stderr: str, framework: TestFramework, project_path: Path
) -> CoverageReport:
    """Parse coverage results."""
    report = CoverageReport()
    if framework == TestFramework.PYTEST:
        coverage_file = project_path / "coverage.json"
        if coverage_file.exists():
            try:
                data = json.loads(coverage_file.read_text(encoding="utf-8"))
                report.total_lines = data.get("totals", {}).get("num_statements", 0)
                report.covered_lines = data.get("totals", {}).get("covered_lines", 0)
                report.percentage = data.get("totals", {}).get("percent_covered", 0.0)
                report.files = data.get("files", {})
            except Exception:
                pass
    elif framework == TestFramework.GO_TEST:
        coverage_file = project_path / "coverage.out"
        if coverage_file.exists():
            try:
                content = coverage_file.read_text(encoding="utf-8")
                lines = content.splitlines()
                total = 0
                covered = 0
                for line in lines:
                    if line.startswith("mode:"):
                        continue
                    parts = line.split()
                    if len(parts) >= 3:
                        count = int(parts[-1])
                        total += 1
                        if count > 0:
                            covered += 1
                report.total_lines = total
                report.covered_lines = covered
                report.percentage = (covered / total * 100) if total > 0 else 0.0
            except Exception:
                pass
```

```
    elif framework == TestFramework.JUNIT:
        jacoco_file = project_path / "target" / "site" / "jacoco" / "jacoco.xml"
        if not jacoco_file.exists():
            jacoco_file = project_path / "build" / "reports" / "jacoco" / "jacoco.xml"
        if jacoco_file.exists():
            try:
                content = jacoco_file.read_text(encoding="utf-8")
                covered_pattern = re.compile(r'covered="(\d+)"')
                missed_pattern = re.compile(r'missed="(\d+)"')
                covered_matches = covered_pattern.findall(content)
                missed_matches = missed_pattern.findall(content)
                covered = sum(int(m) for m in covered_matches)
                missed = sum(int(m) for m in missed_matches)
                report.covered_lines = covered
                report.total_lines = covered + missed
                report.percentage = (
                    (covered / report.total_lines * 100) if report.total_lines > 0 else 0.0
                )
            except Exception:
                pass
    return report
def diagnose_failures(self, failures: list[TestFailure]) -> list[Diagnosis]:
    """Diagnose test failures and suggest fixes."""
    diagnoses: list[Diagnosis] = []
    for failure in failures:
        diagnosis = self._diagnose_single_failure(failure)
        diagnoses.append(diagnosis)
    return diagnoses
def _diagnose_single_failure(self, failure: TestFailure) -> Diagnosis:
    """Diagnose a single test failure."""
    diagnosis = Diagnosis(
        failure_id=failure.test_id,
        confidence=0.0,
    )
    error_text = f"{failure.error_type} {failure.error_message}".lower()
    for pattern_name, pattern_info in self.FAILURE_PATTERNS.items():
        patterns = pattern_info.get("patterns", [])
        matched = any(p.lower() in error_text for p in patterns)
        if matched:
            diagnosis.root_cause = pattern_name
            diagnosis.confidence = 0.8
            diagnosis.fix_type = pattern_info.get("fix_type", "")
            diagnosis.auto_fixable = pattern_info.get("auto_fixable", False)
            diagnosis.suggested_fix = self._generate_fix_suggestion(failure, pattern_name)
            break
    if not diagnosis.root_cause:
        diagnosis.root_cause = "unknown"
        diagnosis.confidence = 0.3
        diagnosis.suggested_fix = "Manual investigation required"
    if failure.file_path:
        diagnosis.affected_files.append(failure.file_path)
    return diagnosis
def _generate_fix_suggestion(self, failure: TestFailure, pattern_name: str) -> str:
    """Generate fix suggestion based on failure pattern."""
    suggestions: dict[str, str] = {
        "assertion_error": f"Check assertion in {failure.test_name}: expected '{failure.exp
ected}' but got '{failure.actual}'",
        "import_error": f"Install missing dependency or fix import path in {failure.test_na
me}",
        "attribute_error": f"Add missing attribute or method referenced in {failure.test_na
me}",
        "type_error": f"Fix type mismatch in {failure.test_name}: check parameter types",
        "value_error": f"Validate input values in {failure.test_name}: check for invalid ar
guments",
        "syntax_error": f"Fix syntax error in {failure.file_path} at line {failure.line_num
```

```
ber or 'unknown'}",
        "timeout_error": f"Optimize {failure.test_name} performance or increase timeout thr
eshold",
        "null_reference": f"Add null check or initialize object before use in {failure.test
_name}",
    }
    return suggestions.get(pattern_name, f"Investigate {failure.test_name} failure")
def auto_fix_tests(self, diagnosis: Diagnosis, project_path: Path) -> bool:
    """Attempt to automatically fix a test issue."""
    if not diagnosis.auto_fixable:
        return False
    if not diagnosis.affected_files:
        return False
    fix_commands = self._generate_fix_commands(diagnosis, project_path)
    for command in fix_commands:
        try:
            result = subprocess.run(
                command,
                cwd=project_path,
                capture_output=True,
                text=True,
                timeout=60,
            )
            if result.returncode != 0:
                return False
        except Exception:
            return False
    return True
def _generate_fix_commands(self, diagnosis: Diagnosis, project_path: Path) -> list[list[str
]]:
    """Generate commands to fix the diagnosed issue."""
    commands: list[list[str]] = []
    if diagnosis.fix_type == "dependency":
        framework = self.detect_test_framework(project_path)
        if framework == TestFramework.PYTEST:
            commands.append(["pip", "install", "-e", "."])
        elif framework in (TestFramework.JEST, TestFramework.MOCHA):
            commands.append(["npm", "install"])
        elif framework == TestFramework.GO_TEST:
            commands.append(["go", "mod", "download"])
        elif framework == TestFramework.CARGO_TEST:
            commands.append(["cargo", "fetch"])
    return commands
def run_full_test_cycle(self, project_path: Path, max_iterations: int = 3) -> TestCycleResu
lt:
    """Run a complete test cycle: discover -> run -> analyze -> diagnose -> fix."""
    import time
    start_time = time.time()
    iterations = 0
    fixes_applied: list[str] = []
    framework = self.detect_test_framework(project_path)
    for _iteration in range(max_iterations):
        iterations += 1
        tests = self.discover_tests(project_path)
        if not tests:
            return TestCycleResult(
                success=False,
                framework=framework,
                message="No tests discovered",
                iterations=iterations,
                total_duration_ms=(time.time() - start_time) * 1000,
            )
        results = self.run_tests(tests, project_path)
        if results.is_success:
            analysis = self.analyze_results(results)
```

```
            coverage = self.check_coverage(project_path)
            return TestCycleResult(
                success=True,
                framework=framework,
                results=results,
                analysis=analysis,
                coverage=coverage,
                fixes_applied=fixes_applied,
                iterations=iterations,
                total_duration_ms=(time.time() - start_time) * 1000,
                message="All tests passed",
            )
        diagnoses = self.diagnose_failures(results.failures)
        auto_fixable_count = sum(1 for d in diagnoses if d.auto_fixable)
        if auto_fixable_count == 0:
            return TestCycleResult(
                success=False,
                framework=framework,
                results=results,
                diagnoses=diagnoses,
                fixes_applied=fixes_applied,
                iterations=iterations,
                total_duration_ms=(time.time() - start_time) * 1000,
                message="No auto-fixable failures found",
            )
        for diagnosis in diagnoses:
            if diagnosis.auto_fixable and self.auto_fix_tests(diagnosis, project_path):
                fixes_applied.append(diagnosis.failure_id)
    return TestCycleResult(
        success=False,
        framework=framework,
        diagnoses=self.diagnose_failures(results.failures) if results else [],
        fixes_applied=fixes_applied,
        iterations=iterations,
        total_duration_ms=(time.time() - start_time) * 1000,
        message=f"Failed after {max_iterations} iterations",
    )
async def async_run_tests(self, project_path: Path) -> TestResults:
    """Asynchronously run tests."""
    tests = self.discover_tests(project_path)
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, self.run_tests, tests, project_path)
async def async_run_full_cycle(self, project_path: Path) -> TestCycleResult:
    """Asynchronously run full test cycle."""
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, self.run_full_test_cycle, project_path)
def get_test_command(self, framework: TestFramework) -> list[str]:
    """Get the test run command for a framework."""
    config = self.FRAMEWORK_CONFIGS.get(framework, {})
    return cast(list[str], config.get("run_command", []))
def get_coverage_command(self, framework: TestFramework) -> list[str]:
    """Get the coverage command for a framework."""
    config = self.FRAMEWORK_CONFIGS.get(framework, {})
    return cast(list[str], config.get("coverage_command", []))
```

File: [88/252] openlaoke/core/hyperauto/types.py

```
"""Type definitions for HyperAuto system."""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
from uuid import uuid4
class HyperAutoMode(StrEnum):
    """Operating modes for HyperAuto."""
    SEMI_AUTO = "semi_auto"
    FULL_AUTO = "full_auto"
    HYPER_AUTO = "hyper_auto"
class HyperAutoState(StrEnum):
    """States for HyperAuto workflow."""
    IDLE = "idle"
    ANALYZING = "analyzing"
    PLANNING = "planning"
    EXECUTING = "executing"
    VERIFYING = "verifying"
    RETRYING = "retrying"
    REFLECTING = "reflecting"
    LEARNING = "learning"
    COMPLETED = "completed"
    FAILED = "failed"
class SubTaskStatus(StrEnum):
    """Status of a sub-task."""
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    SKIPPED = "skipped"
class DecisionType(StrEnum):
    """Types of automated decisions."""
    SKILL_CREATION = "skill_creation"
    PROJECT_INIT = "project_init"
    CODE_SEARCH = "code_search"
    DEPENDENCY_INSTALL = "dependency_install"
    TEST_EXECUTION = "test_execution"
    COMMIT = "commit"
    ROLLBACK = "rollback"
    RETRY = "retry"
    ABORT = "abort"
@dataclass
class SubTask:
    """A sub-task in the HyperAuto workflow."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    name: str = ""
    description: str = ""
    status: SubTaskStatus = SubTaskStatus.PENDING
    priority: int = 0
    dependencies: list[str] = field(default_factory=list)
    result: Any = None
    error: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "name": self.name,
            "description": self.description,
            "status": self.status.value,
            "priority": self.priority,
            "dependencies": self.dependencies,
```

```
            "result": self.result,
            "error": self.error,
            "metadata": self.metadata,
        }
@dataclass
class Decision:
    """An automated decision made by HyperAuto."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    type: DecisionType = DecisionType.SKILL_CREATION
    confidence: float = 0.0
    reasoning: str = ""
    action: str = ""
    parameters: dict[str, Any] = field(default_factory=dict)
    approved: bool = False
    executed: bool = False
    result: Any = None
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type.value,
            "confidence": self.confidence,
            "reasoning": self.reasoning,
            "action": self.action,
            "parameters": self.parameters,
            "approved": self.approved,
            "executed": self.executed,
            "result": self.result,
        }
@dataclass
class Reflection:
    """A reflection on completed work."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    task_id: str = ""
    success: bool = False
    observations: list[str] = field(default_factory=list)
    improvements: list[str] = field(default_factory=list)
    learned_patterns: list[str] = field(default_factory=list)
    error_patterns: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "task_id": self.task_id,
            "success": self.success,
            "observations": self.observations,
            "improvements": self.improvements,
            "learned_patterns": self.learned_patterns,
            "error_patterns": self.error_patterns,
        }
@dataclass
class WorkflowContext:
    """Context for a HyperAuto workflow execution."""
    session_id: str = field(default_factory=lambda: uuid4().hex[:12])
    original_request: str = ""
    current_state: HyperAutoState = HyperAutoState.IDLE
    sub_tasks: list[SubTask] = field(default_factory=list)
    decisions: list[Decision] = field(default_factory=list)
    reflections: list[Reflection] = field(default_factory=list)
    iteration: int = 0
    total_tokens: int = 0
    total_cost: float = 0.0
    start_time: float = 0.0
    end_time: float | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
```

```
        "session_id": self.session_id,
        "original_request": self.original_request,
        "current_state": self.current_state.value,
        "sub_tasks": [t.to_dict() for t in self.sub_tasks],
        "decisions": [d.to_dict() for d in self.decisions],
        "reflections": [r.to_dict() for r in self.reflections],
        "iteration": self.iteration,
        "total_tokens": self.total_tokens,
        "total_cost": self.total_cost,
        "start_time": self.start_time,
        "end_time": self.end_time,
        "metadata": self.metadata,
    }
def get_pending_tasks(self) -> list[SubTask]:
    return [t for t in self.sub_tasks if t.status == SubTaskStatus.PENDING]
def get_running_tasks(self) -> list[SubTask]:
    return [t for t in self.sub_tasks if t.status == SubTaskStatus.RUNNING]
def get_completed_tasks(self) -> list[SubTask]:
    return [t for t in self.sub_tasks if t.status == SubTaskStatus.COMPLETED]
def get_failed_tasks(self) -> list[SubTask]:
    return [t for t in self.sub_tasks if t.status == SubTaskStatus.FAILED]
```

File: [89/252] openlaoke/core/hyperauto/validator.py

```
"""ResultValidator - Comprehensive validation system for generated artifacts.
This module provides validation capabilities:
1. Code validation - syntax, style, best practices
2. Functionality validation - requirement compliance
3. Performance validation - benchmarks and metrics
4. Security validation - vulnerability scanning
5. Compatibility validation - cross-platform checks
6. Quality scoring - comprehensive quality assessment
"""
from __future__ import annotations
import asyncio
import re
import subprocess
import tempfile
from dataclasses import dataclass, field
from enum import StrEnum
from pathlib import Path
from typing import Any
from uuid import uuid4
class ValidationLevel(StrEnum):
    """Validation severity levels."""
    CRITICAL = "critical"
    ERROR = "error"
    WARNING = "warning"
    INFO = "info"
    OK = "ok"
class ValidationCategory(StrEnum):
    """Categories of validation."""
    SYNTAX = "syntax"
    STYLE = "style"
    FUNCTIONALITY = "functionality"
    PERFORMANCE = "performance"
    SECURITY = "security"
    COMPATIBILITY = "compatibility"
    BEST_PRACTICES = "best_practices"
    DOCUMENTATION = "documentation"
class Language(StrEnum):
    """Supported programming languages."""
    PYTHON = "python"
    JAVASCRIPT = "javascript"
    TYPESCRIPT = "typescript"
    GO = "go"
    RUST = "rust"
    JAVA = "java"
    C = "c"
    CPP = "cpp"
    UNKNOWN = "unknown"
@dataclass
class ValidationIssue:
    """A single validation issue."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    category: ValidationCategory = ValidationCategory.SYNTAX
    level: ValidationLevel = ValidationLevel.WARNING
    message: str = ""
    file_path: Path | None = None
    line_number: int | None = None
    column_number: int | None = None
    rule_id: str = ""
    suggestion: str = ""
    auto_fixable: bool = False
    context: dict[str, Any] = field(default_factory=dict)
```

```
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "category": self.category.value,
            "level": self.level.value,
            "message": self.message,
            "file_path": str(self.file_path) if self.file_path else None,
            "line_number": self.line_number,
            "column_number": self.column_number,
            "rule_id": self.rule_id,
            "suggestion": self.suggestion,
            "auto_fixable": self.auto_fixable,
            "context": self.context,
        }
@dataclass
class ValidationResult:
    """Result of a validation check."""
    valid: bool = True
    issues: list[ValidationIssue] = field(default_factory=list)
    score: float = 100.0
    passed_checks: int = 0
    failed_checks: int = 0
    warnings: int = 0
    errors: int = 0
    category: ValidationCategory = ValidationCategory.SYNTAX
    details: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "valid": self.valid,
            "issues": [i.to_dict() for i in self.issues],
            "score": self.score,
            "passed_checks": self.passed_checks,
            "failed_checks": self.failed_checks,
            "warnings": self.warnings,
            "errors": self.errors,
            "category": self.category.value,
            "details": self.details,
        }
    @property
    def has_errors(self) -> bool:
        return self.errors > 0 or any(
            i.level in (ValidationLevel.CRITICAL, ValidationLevel.ERROR) for i in self.issues
        )
@dataclass
class PerformanceResult:
    """Result of performance validation."""
    benchmark_passed: bool = True
    metrics: dict[str, float] = field(default_factory=dict)
    baseline_comparison: dict[str, float] = field(default_factory=dict)
    execution_time_ms: float = 0.0
    memory_usage_mb: float = 0.0
    cpu_usage_percent: float = 0.0
    issues: list[ValidationIssue] = field(default_factory=list)
    score: float = 100.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "benchmark_passed": self.benchmark_passed,
            "metrics": self.metrics,
            "baseline_comparison": self.baseline_comparison,
            "execution_time_ms": self.execution_time_ms,
            "memory_usage_mb": self.memory_usage_mb,
            "cpu_usage_percent": self.cpu_usage_percent,
            "issues": [i.to_dict() for i in self.issues],
            "score": self.score,
        }
```

```
@dataclass
class SecurityReport:
    """Security validation report."""
    secure: bool = True
    vulnerabilities: list[dict[str, Any]] = field(default_factory=list)
    risk_level: str = "low"
    issues: list[ValidationIssue] = field(default_factory=list)
    recommendations: list[str] = field(default_factory=list)
    score: float = 100.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "secure": self.secure,
            "vulnerabilities": self.vulnerabilities,
            "risk_level": self.risk_level,
            "issues": [i.to_dict() for i in self.issues],
            "recommendations": self.recommendations,
            "score": self.score,
        }
@dataclass
class ValidatorQualityScore:
    """Comprehensive quality score."""
    overall: float = 0.0
    syntax: float = 100.0
    style: float = 100.0
    functionality: float = 100.0
    performance: float = 100.0
    security: float = 100.0
    compatibility: float = 100.0
    documentation: float = 100.0
    best_practices: float = 100.0
    breakdown: dict[str, float] = field(default_factory=dict)
    grade: str = "A"
    def to_dict(self) -> dict[str, Any]:
        return {
            "overall": self.overall,
            "syntax": self.syntax,
            "style": self.style,
            "functionality": self.functionality,
            "performance": self.performance,
            "security": self.security,
            "compatibility": self.compatibility,
            "documentation": self.documentation,
            "best_practices": self.best_practices,
            "breakdown": self.breakdown,
            "grade": self.grade,
        }
    def calculate_grade(self) -> str:
        if self.overall >= 90:
            return "A"
        elif self.overall >= 80:
            return "B"
        elif self.overall >= 70:
            return "C"
        elif self.overall >= 60:
            return "D"
        else:
            return "F"
@dataclass
class Artifact:
    """An artifact to validate."""
    id: str = field(default_factory=lambda: uuid4().hex[:8])
    type: str = ""
    name: str = ""
    content: str = ""
    file_path: Path | None = None
```

```
    language: Language = Language.UNKNOWN
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type,
            "name": self.name,
            "content": self.content,
            "file_path": str(self.file_path) if self.file_path else None,
            "language": self.language.value,
            "metadata": self.metadata,
        }
@dataclass
class Requirements:
    """Validation requirements."""
    functional_requirements: list[str] = field(default_factory=list)
    performance_requirements: dict[str, float] = field(default_factory=dict)
    security_requirements: list[str] = field(default_factory=list)
    compatibility_targets: list[str] = field(default_factory=list)
    style_guidelines: dict[str, Any] = field(default_factory=dict)
    documentation_requirements: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "functional_requirements": self.functional_requirements,
            "performance_requirements": self.performance_requirements,
            "security_requirements": self.security_requirements,
            "compatibility_targets": self.compatibility_targets,
            "style_guidelines": self.style_guidelines,
            "documentation_requirements": self.documentation_requirements,
        }
@dataclass
class ValidationReport:
    """Complete validation report."""
    valid: bool = True
    artifact_id: str = ""
    overall_score: float = 100.0
    quality_score: ValidatorQualityScore | None = None
    code_validation: ValidationResult | None = None
    functionality_validation: ValidationResult | None = None
    performance_validation: PerformanceResult | None = None
    security_validation: SecurityReport | None = None
    compatibility_validation: ValidationResult | None = None
    total_issues: int = 0
    critical_issues: int = 0
    errors: int = 0
    warnings: int = 0
    recommendations: list[str] = field(default_factory=list)
    timestamp: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {
            "valid": self.valid,
            "artifact_id": self.artifact_id,
            "overall_score": self.overall_score,
            "quality_score": self.quality_score.to_dict() if self.quality_score else None,
            "code_validation": self.code_validation.to_dict() if self.code_validation else None
    ,
            "functionality_validation": (
                self.functionality_validation.to_dict() if self.functionality_validation else N
    one
            ),
            "performance_validation": (
                self.performance_validation.to_dict() if self.performance_validation else None
            ),
            "security_validation": (
                self.security_validation.to_dict() if self.security_validation else None
```

```
            ),
            "compatibility_validation": (
                self.compatibility_validation.to_dict() if self.compatibility_validation else N
    one
            ),
            "total_issues": self.total_issues,
            "critical_issues": self.critical_issues,
            "errors": self.errors,
            "warnings": self.warnings,
            "recommendations": self.recommendations,
            "timestamp": self.timestamp,
        }
class ResultValidator:
    """Comprehensive validation system for generated artifacts."""
    LANGUAGE_CONFIGS: dict[Language, dict[str, Any]] = {
        Language.PYTHON: {
            "syntax_check": ["python", "-m", "py_compile"],
            "lint_tools": ["ruff", "mypy", "flake8", "pylint"],
            "format_tools": ["ruff", "black"],
            "test_framework": "pytest",
            "security_tools": ["bandit"],
            "file_extensions": [".py", ".pyw"],
        },
        Language.JAVASCRIPT: {
            "syntax_check": ["node", "--check"],
            "lint_tools": ["eslint"],
            "format_tools": ["prettier"],
            "test_framework": "jest",
            "security_tools": ["npm audit"],
            "file_extensions": [".js", ".mjs", ".cjs"],
        },
        Language.TYPESCRIPT: {
            "syntax_check": ["tsc", "--noEmit"],
            "lint_tools": ["eslint"],
            "format_tools": ["prettier"],
            "test_framework": "jest",
            "security_tools": ["npm audit"],
            "file_extensions": [".ts", ".tsx"],
        },
        Language.GO: {
            "syntax_check": ["go", "build"],
            "lint_tools": ["golangci-lint"],
            "format_tools": ["gofmt", "goimports"],
            "test_framework": "go test",
            "security_tools": ["gosec"],
            "file_extensions": [".go"],
        },
        Language.RUST: {
            "syntax_check": ["cargo", "check"],
            "lint_tools": ["cargo", "clippy"],
            "format_tools": ["cargo", "fmt", "--check"],
            "test_framework": "cargo test",
            "security_tools": ["cargo audit"],
            "file_extensions": [".rs"],
        },
        Language.JAVA: {
            "syntax_check": ["javac"],
            "lint_tools": ["checkstyle", "spotbugs"],
            "format_tools": ["google-java-format"],
            "test_framework": "junit",
            "security_tools": ["dependency-check"],
            "file_extensions": [".java"],
        },
        Language.C: {
            "syntax_check": ["gcc", "-fsyntax-only"],
```

```
        "lint_tools": ["cppcheck"],
        "format_tools": ["clang-format"],
        "test_framework": "custom",
        "security_tools": ["cppcheck"],
        "file_extensions": [".c", ".h"],
    },
    Language.CPP: {
        "syntax_check": ["g++", "-fsyntax-only"],
        "lint_tools": ["cppcheck"],
        "format_tools": ["clang-format"],
        "test_framework": "custom",
        "security_tools": ["cppcheck"],
        "file_extensions": [".cpp", ".cc", ".cxx", ".hpp"],
    },
}
SECURITY_PATTERNS: dict[str, dict[str, Any]] = {
    "hardcoded_password": {
        "patterns": ["password.*=.*['\"]", "passwd.*=.*['\"]", "pwd.*=.*['\"]"],
        "level": ValidationLevel.CRITICAL,
        "message": "Hardcoded password detected",
    },
    "hardcoded_secret": {
        "patterns": ["secret.*=.*['\"]", "api_key.*=.*['\"", "token.*=.*['\"]"],
        "level": ValidationLevel.CRITICAL,
        "message": "Hardcoded secret detected",
    },
    "sql_injection": {
        "patterns": ["execute\\(.*\\+.*\\)", "query\\(.*\\+.*\\)"],
        "level": ValidationLevel.ERROR,
        "message": "Potential SQL injection vulnerability",
    },
    "command_injection": {
        "patterns": ["exec\\(.*\\+.*\\)", "system\\(.*\\+.*\\)", "eval\\(.*\\+.*\\)"],
        "level": ValidationLevel.ERROR,
        "message": "Potential command injection vulnerability",
    },
    "unsafe_deserialization": {
        "patterns": ["pickle\\.loads", "yaml\\.load\\(.*\\)"],
        "level": ValidationLevel.WARNING,
        "message": "Unsafe deserialization",
    },
    "debug_code": {
        "patterns": ["print\\(", "console\\.log\\(", "debugger"],
        "level": ValidationLevel.INFO,
        "message": "Debug code detected",
    },
    "weak_crypto": {
        "patterns": ["md5\\(", "sha1\\(", "random\\.random"],
        "level": ValidationLevel.WARNING,
        "message": "Weak cryptographic function",
    },
}
BEST_PRACTICE_PATTERNS: dict[str, dict[str, Any]] = {
    "missing_error_handling": {
        "patterns": ["try\\s*\\{[^}]*\\}\\s*catch", "try:\\s*[^except]"],
        "level": ValidationLevel.WARNING,
        "message": "Missing proper error handling",
    },
    "magic_numbers": {
        "patterns": ["[=\\(\\[],\\s*\\d{2,}\\s*[\\)\\];,]"],
        "level": ValidationLevel.INFO,
        "message": "Magic number detected",
    },
    "long_function": {
        "check": "line_count",
```

```
        "threshold": 50,
        "level": ValidationLevel.WARNING,
        "message": "Function exceeds recommended length",
    },
    "deep_nesting": {
        "check": "nesting_depth",
        "threshold": 4,
        "level": ValidationLevel.WARNING,
        "message": "Deep nesting detected",
    },
    "duplicate_code": {
        "check": "similarity",
        "threshold": 0.8,
        "level": ValidationLevel.WARNING,
        "message": "Potential duplicate code",
    },
}
def detect_language(self, code: str, file_path: Path | None = None) -> Language:
    """Detect the programming language of code."""
    if file_path:
        extension = file_path.suffix.lower()
        for language, config in self.LANGUAGE_CONFIGS.items():
            if language == Language.UNKNOWN:
                continue
            extensions = config.get("file_extensions", [])
            if extension in extensions:
                return language
    python_indicators = ["def ", "import ", "from ", "class ", "__", "self.", "elif "]
    js_indicators = ["function ", "const ", "let ", "var ", "=>", "require(", "export "]
    ts_indicators = ["interface ", "type ", ": string", ": number", ": boolean"]
    go_indicators = ["func ", "package ", 'import "', "go ", "chan ", "defer "]
    rust_indicators = ["fn ", "let mut", "impl ", "pub fn", "use ", "::<", "crate::"]
    java_indicators = ["public class", "private ", "void ", "static ", "new ", "extends "]
    c_indicators = ["#include", "int main", "void ", "struct ", "typedef ", "sizeof"]
    cpp_indicators = ["#include", "class ", "namespace ", "template<", "std::", "new "]
    indicators = {
        Language.PYTHON: python_indicators,
        Language.JAVASCRIPT: js_indicators,
        Language.TYPESCRIPT: ts_indicators,
        Language.GO: go_indicators,
        Language.RUST: rust_indicators,
        Language.JAVA: java_indicators,
        Language.C: c_indicators,
        Language.CPP: cpp_indicators,
    }
    scores: dict[Language, int] = {}
    for language, lang_indicators in indicators.items():
        score = sum(1 for ind in lang_indicators if ind in code)
        scores[language] = score
    max_score = max(scores.values())
    if max_score == 0:
        return Language.UNKNOWN
    for language, score in scores.items():
        if score == max_score:
            return language
    return Language.UNKNOWN
def validate_code(
    self, code: str, language: str | Language = Language.UNKNOWN
) -> ValidationResult:
    """Validate code for syntax and style."""
    if isinstance(language, str):
        language = self._str_to_language(language)
    if language == Language.UNKNOWN:
        language = self.detect_language(code)
    result = ValidationResult(category=ValidationCategory.SYNTAX)
```

```
    syntax_result = self._check_syntax(code, language)
    result.issues.extend(syntax_result.issues)
    style_result = self._check_style(code, language)
    result.issues.extend(style_result.issues)
    best_practice_result = self._check_best_practices(code, language)
    result.issues.extend(best_practice_result.issues)
    result.errors = sum(
        1 for i in result.issues if i.level in (ValidationLevel.CRITICAL, ValidationLevel.E
RROR)
    )
    result.warnings = sum(1 for i in result.issues if i.level == ValidationLevel.WARNING)
    result.passed_checks = sum(
        1 for i in result.issues if i.level in (ValidationLevel.OK, ValidationLevel.INFO)
    )
    result.failed_checks = result.errors + result.warnings
    result.valid = result.errors == 0
    result.score = self._calculate_validation_score(result)
    return result
def _str_to_language(self, language_str: str) -> Language:
    """Convert string to Language enum."""
    mapping = {
        "python": Language.PYTHON,
        "javascript": Language.JAVASCRIPT,
        "js": Language.JAVASCRIPT,
        "typescript": Language.TYPESCRIPT,
        "ts": Language.TYPESCRIPT,
        "go": Language.GO,
        "rust": Language.RUST,
        "java": Language.JAVA,
        "c": Language.C,
        "cpp": Language.CPP,
        "c++": Language.CPP,
    }
    return mapping.get(language_str.lower(), Language.UNKNOWN)
def _check_syntax(self, code: str, language: Language) -> ValidationResult:
    """Check code syntax."""
    result = ValidationResult(category=ValidationCategory.SYNTAX)
    if language == Language.UNKNOWN:
        return result
    config = self.LANGUAGE_CONFIGS.get(language, {})
    syntax_check = config.get("syntax_check", [])
    if not syntax_check:
        return result
    with tempfile.NamedTemporaryFile(
        mode="w", suffix=config.get("file_extensions", [""])[0], delete=False
    ) as f:
        f.write(code)
        temp_path = f.name
    try:
        check_result = subprocess.run(
            syntax_check + [temp_path],
            capture_output=True,
            text=True,
            timeout=30,
        )
        if check_result.returncode != 0:
            errors = self._parse_syntax_errors(check_result.stderr, language)
            result.issues.extend(errors)
    except subprocess.TimeoutExpired:
        result.issues.append(
            ValidationIssue(
                category=ValidationCategory.SYNTAX,
                level=ValidationLevel.ERROR,
                message="Syntax check timed out",
            )
```

```
        )
    except FileNotFoundError:
        pass
    except Exception as e:
        result.issues.append(
            ValidationIssue(
                category=ValidationCategory.SYNTAX,
                level=ValidationLevel.ERROR,
                message=f"Syntax check failed: {e}",
            )
        )
    finally:
        Path(temp_path).unlink(missing_ok=True)
    return result
def _parse_syntax_errors(self, error_output: str, language: Language) -> list[ValidationIss
ue]:
    """Parse syntax errors from tool output."""
    issues: list[ValidationIssue] = []
    if language in (Language.PYTHON, Language.JAVASCRIPT, Language.TYPESCRIPT):
        pattern = re.compile(r"SyntaxError: (.+)")
        for match in pattern.finditer(error_output):
            issues.append(
                ValidationIssue(
                    category=ValidationCategory.SYNTAX,
                    level=ValidationLevel.ERROR,
                    message=f"Syntax error: {match.group(1)}",
                    auto_fixable=True,
                )
            )
    elif language == Language.GO:
        pattern = re.compile(r"(.+):(\d+):(\d+): (.+)")
        for match in pattern.finditer(error_output):
            issues.append(
                ValidationIssue(
                    category=ValidationCategory.SYNTAX,
                    level=ValidationLevel.ERROR,
                    message=match.group(4),
                    line_number=int(match.group(2)),
                    column_number=int(match.group(3)),
                    auto_fixable=True,
                )
            )
    elif language == Language.RUST:
        pattern = re.compile(r"error\[E\d+\]: (.+)")
        for match in pattern.finditer(error_output):
            issues.append(
                ValidationIssue(
                    category=ValidationCategory.SYNTAX,
                    level=ValidationLevel.ERROR,
                    message=match.group(1),
                    auto_fixable=True,
                )
            )
    elif language == Language.JAVA:
        pattern = re.compile(r"error: (.+)")
        for match in pattern.finditer(error_output):
            issues.append(
                ValidationIssue(
                    category=ValidationCategory.SYNTAX,
                    level=ValidationLevel.ERROR,
                    message=match.group(1),
                    auto_fixable=True,
                )
            )
    return issues
```

```
def _check_style(self, code: str, language: Language) -> ValidationResult:
    """Check code style."""
    result = ValidationResult(category=ValidationCategory.STYLE)
    lines = code.splitlines()
    for i, line in enumerate(lines, 1):
        if len(line) > 100:
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.STYLE,
                    level=ValidationLevel.INFO,
                    message=f"Line exceeds 100 characters ({len(line)} chars)",
                    line_number=i,
                    rule_id="line-length",
                    suggestion="Break line into multiple lines",
                    auto_fixable=True,
                )
            )
        if line.strip() == "":
            continue
        if (
            language == Language.PYTHON
            and not line.startswith("#")
            and "    " not in line[:4]
            and line[0] == " "
        ):
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.STYLE,
                    level=ValidationLevel.WARNING,
                    message="Inconsistent indentation",
                    line_number=i,
                    rule_id="indentation",
                    suggestion="Use 4 spaces for indentation",
                    auto_fixable=True,
                )
            )
    return result
def _check_best_practices(self, code: str, language: Language) -> ValidationResult:
    """Check best practices."""
    result = ValidationResult(category=ValidationCategory.BEST_PRACTICES)
    functions = self._extract_functions(code, language)
    for func_name, start_line, end_line in functions:
        func_lines = end_line - start_line + 1
        if func_lines > 50:
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.BEST_PRACTICES,
                    level=ValidationLevel.WARNING,
                    message=f"Function '{func_name}' is too long ({func_lines} lines)",
                    line_number=start_line,
                    rule_id="long-function",
                    suggestion="Break into smaller functions",
                    auto_fixable=False,
                )
            )
    for pattern_name, pattern_info in self.BEST_PRACTICE_PATTERNS.items():
        patterns = pattern_info.get("patterns", [])
        level = pattern_info.get("level", ValidationLevel.WARNING)
        message = pattern_info.get("message", "")
        for pattern in patterns:
            try:
                regex = re.compile(pattern)
                for match in regex.finditer(code):
                    line_num = code[: match.start()].count("\n") + 1
                    result.issues.append(
```

```
                        ValidationIssue(
                            category=ValidationCategory.BEST_PRACTICES,
                            level=level,
                            message=message,
                            line_number=line_num,
                            rule_id=pattern_name,
                            auto_fixable=False,
                        )
                    )
            except re.error:
                pass
    return result
def _extract_functions(self, code: str, language: Language) -> list[tuple[str, int, int]]:
    """Extract function definitions with line numbers."""
    functions: list[tuple[str, int, int]] = []
    lines = code.splitlines()
    if language == Language.PYTHON:
        pattern = re.compile(r"def\s+(\w+)\s*\(")
        for i, line in enumerate(lines, 1):
            match = pattern.search(line)
            if match:
                func_name = match.group(1)
                end_line = self._find_function_end(lines, i - 1, language)
                functions.append((func_name, i, end_line))
    elif language in (Language.JAVASCRIPT, Language.TYPESCRIPT):
        patterns = [
            re.compile(r"function\s+(\w+)\s*\("),
            re.compile(r"const\s+(\w+)\s*=\s*(?:async\s*)?\("),
        ]
        for i, line in enumerate(lines, 1):
            for pattern in patterns:
                match = pattern.search(line)
                if match:
                    func_name = match.group(1)
                    end_line = self._find_function_end(lines, i - 1, language)
                    functions.append((func_name, i, end_line))
    elif language == Language.GO:
        pattern = re.compile(r"func\s+(?:\(\w+\)\s*)?(\w+)\s*\(")
        for i, line in enumerate(lines, 1):
            match = pattern.search(line)
            if match:
                func_name = match.group(1)
                end_line = self._find_function_end(lines, i - 1, language)
                functions.append((func_name, i, end_line))
    elif language == Language.RUST:
        pattern = re.compile(r"fn\s+(\w+)\s*\(")
        for i, line in enumerate(lines, 1):
            match = pattern.search(line)
            if match:
                func_name = match.group(1)
                end_line = self._find_function_end(lines, i - 1, language)
                functions.append((func_name, i, end_line))
    elif language == Language.JAVA:
        pattern = re.compile(r"(?:public|private|protected)?\s*(?:static)?\s*\w+\s+(\w+)\s*
\(")
        for i, line in enumerate(lines, 1):
            match = pattern.search(line)
            if match:
                func_name = match.group(1)
                end_line = self._find_function_end(lines, i - 1, language)
                functions.append((func_name, i, end_line))
    return functions
def _find_function_end(self, lines: list[str], start: int, language: Language) -> int:
    """Find the end line of a function."""
    if language == Language.PYTHON:
```

```
        indent = len(lines[start]) - len(lines[start].lstrip())
        for i in range(start + 1, len(lines)):
            if lines[i].strip() == "":
                continue
            current_indent = len(lines[i]) - len(lines[i].lstrip())
            if current_indent <= indent and not lines[i].strip().startswith("#"):
                return i
        return len(lines)
    elif language in (
        Language.JAVASCRIPT,
        Language.TYPESCRIPT,
        Language.JAVA,
        Language.CPP,
        Language.GO,
        Language.RUST,
    ):
        brace_count = 0
        started = False
        for i in range(start, len(lines)):
            for char in lines[i]:
                if char == "{":
                    brace_count += 1
                    started = True
                elif char == "}":
                    brace_count -= 1
                    if started and brace_count == 0:
                        return i + 1
        return len(lines)
    return len(lines)
def validate_functionality(
    self, implementation: str, requirements: list[str], language: Language = Language.UNKNO
WN
) -> ValidationResult:
    """Validate that implementation meets requirements."""
    result = ValidationResult(category=ValidationCategory.FUNCTIONALITY)
    if language == Language.UNKNOWN:
        language = self.detect_language(implementation)
    for req in requirements:
        keywords = self._extract_keywords(req)
        matched = sum(1 for kw in keywords if kw.lower() in implementation.lower())
        coverage = matched / len(keywords) if keywords else 0
        if coverage < 0.5:
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.FUNCTIONALITY,
                    level=ValidationLevel.WARNING,
                    message=f"Requirement not fully addressed: '{req}'",
                    suggestion=f"Ensure implementation addresses: {req}",
                    auto_fixable=False,
                )
            )
        elif coverage < 1.0:
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.FUNCTIONALITY,
                    level=ValidationLevel.INFO,
                    message=f"Requirement partially addressed: '{req}'",
                    suggestion=f"Complete implementation for: {req}",
                    auto_fixable=False,
                )
            )
    result.warnings = sum(1 for i in result.issues if i.level == ValidationLevel.WARNING)
    result.valid = all(i.level != ValidationLevel.ERROR for i in result.issues)
    result.score = self._calculate_validation_score(result)
    return result
```

```
def _extract_keywords(self, text: str) -> list[str]:
    """Extract important keywords from text."""
    stop_words = {
        "the",
        "a",
        "an",
        "is",
        "are",
        "was",
        "were",
        "be",
        "been",
        "being",
        "have",
        "has",
        "had",
        "do",
        "does",
        "did",
        "will",
        "would",
        "could",
        "should",
        "may",
        "might",
        "must",
        "shall",
        "can",
        "need",
        "dare",
        "ought",
        "used",
        "to",
        "of",
        "in",
        "for",
        "on",
        "with",
        "at",
        "by",
        "from",
        "as",
        "into",
        "through",
        "during",
        "before",
        "after",
        "above",
        "below",
        "between",
        "under",
        "again",
        "further",
        "then",
        "once",
        "here",
        "there",
        "when",
        "where",
        "why",
        "how",
        "all",
        "each",
        "few",
        "more",
```

```
        "most",
        "other",
        "some",
        "such",
        "no",
        "nor",
        "not",
        "only",
        "own",
        "same",
        "so",
        "than",
        "too",
        "very",
        "just",
        "and",
        "but",
        "if",
        "or",
        "because",
        "until",
        "while",
        "although",
        "though",
        "that",
        "which",
        "who",
        "whom",
        "this",
        "these",
        "those",
        "am",
        "it",
    }
    words = re.findall(r"\b\w+\b", text.lower())
    keywords = [w for w in words if w not in stop_words and len(w) > 2]
    return keywords
def validate_performance(
    self, code: str, benchmarks: dict[str, float], language: Language = Language.UNKNOWN
) -> PerformanceResult:
    """Validate performance against benchmarks."""
    result = PerformanceResult()
    if language == Language.UNKNOWN:
        language = self.detect_language(code)
    result.execution_time_ms = self._estimate_execution_time(code, language)
    result.memory_usage_mb = self._estimate_memory_usage(code)
    if "max_execution_time_ms" in benchmarks:
        max_time = benchmarks["max_execution_time_ms"]
        if result.execution_time_ms > max_time:
            result.benchmark_passed = False
            result.issues.append(
                ValidationIssue(
                    category=ValidationCategory.PERFORMANCE,
                    level=ValidationLevel.WARNING,
                    message=f"Execution time ({result.execution_time_ms}ms) exceeds benchma
rk ({max_time}ms)",
                    suggestion="Optimize code for better performance",
                    auto_fixable=False,
                )
            )
    if "max_memory_mb" in benchmarks:
        max_memory = benchmarks["max_memory_mb"]
        if result.memory_usage_mb > max_memory:
            result.benchmark_passed = False
            result.issues.append(
```

```
                ValidationIssue(
                    category=ValidationCategory.PERFORMANCE,
                    level=ValidationLevel.WARNING,
                    message=f"Memory usage ({result.memory_usage_mb}MB) exceeds benchmark (
{max_memory}MB)",
                    suggestion="Reduce memory allocations",
                    auto_fixable=False,
                )
            )
    for metric, threshold in benchmarks.items():
        result.baseline_comparison[metric] = threshold
        result.metrics[metric] = getattr(result, metric.replace("max_", ""), 0.0)
    result.score = self._calculate_performance_score(result, benchmarks)
    return result
def _estimate_execution_time(self, code: str, language: Language) -> float:
    """Estimate execution time in milliseconds."""
    complexity_indicators = {
        "loops": len(re.findall(r"(for|while|loop)", code, re.IGNORECASE)),
        "nested_loops": len(re.findall(r"(for|while).*?(for|while)", code, re.IGNORECASE)),
        "recursion": len(re.findall(r"def\s+\w+.*\w+\(", code)),
        "io_operations": len(
            re.findall(r"(read|write|open|fetch|request)", code, re.IGNORECASE)
        ),
        "sorting": len(re.findall(r"(sort|sorted|order)", code, re.IGNORECASE)),
    }
    base_time = 10.0
    for indicator, count in complexity_indicators.items():
        if indicator == "nested_loops":
            base_time += count * 100.0
        elif indicator == "loops":
            base_time += count * 20.0
        elif indicator == "recursion":
            base_time += count * 50.0
        elif indicator == "io_operations":
            base_time += count * 100.0
        elif indicator == "sorting":
            base_time += count * 30.0
    return base_time
def _estimate_memory_usage(self, code: str) -> float:
    """Estimate memory usage in megabytes."""
    allocations = len(
        re.findall(r"(new|malloc|allocate|list|dict|array|Map)", code, re.IGNORECASE)
    )
    large_data = len(re.findall(r"(buffer|cache|store|data)", code, re.IGNORECASE))
    base_memory = 1.0
    base_memory += allocations * 0.5
    base_memory += large_data * 2.0
    return base_memory
def _calculate_performance_score(
    self, result: PerformanceResult, benchmarks: dict[str, float]
) -> float:
    """Calculate performance score."""
    score = 100.0
    for metric, threshold in benchmarks.items():
        actual = result.metrics.get(metric.replace("max_", ""), 0.0)
        if actual > threshold:
            ratio = actual / threshold
            penalty = min(50, (ratio - 1) * 100)
            score -= penalty
    return max(0, score)
def validate_security(self, code: str, language: Language = Language.UNKNOWN) -> SecurityRe
port:
    """Validate code for security vulnerabilities."""
    report = SecurityReport()
    if language == Language.UNKNOWN:
```

```
        language = self.detect_language(code)
    for pattern_name, pattern_info in self.SECURITY_PATTERNS.items():
        patterns = pattern_info.get("patterns", [])
        level = pattern_info.get("level", ValidationLevel.WARNING)
        message = pattern_info.get("message", "")
        for pattern in patterns:
            try:
                regex = re.compile(pattern, re.IGNORECASE)
                for match in regex.finditer(code):
                    line_num = code[: match.start()].count("\n") + 1
                    vulnerability = {
                        "type": pattern_name,
                        "line": line_num,
                        "message": message,
                        "severity": level.value,
                        "match": match.group(0),
                    }
                    report.vulnerabilities.append(vulnerability)
                    report.issues.append(
                        ValidationIssue(
                            category=ValidationCategory.SECURITY,
                            level=level,
                            message=message,
                            line_number=line_num,
                            rule_id=pattern_name,
                            suggestion=self._get_security_fix_suggestion(pattern_name),
                            auto_fixable=False,
                        )
                    )
            except re.error:
                pass
    critical_count = sum(1 for v in report.vulnerabilities if v["severity"] == "critical")
    error_count = sum(1 for v in report.vulnerabilities if v["severity"] == "error")
    warning_count = sum(1 for v in report.vulnerabilities if v["severity"] == "warning")
    if critical_count > 0:
        report.risk_level = "critical"
        report.secure = False
    elif error_count > 0:
        report.risk_level = "high"
        report.secure = False
    elif warning_count > 3:
        report.risk_level = "medium"
    elif warning_count > 0:
        report.risk_level = "low"
    else:
        report.risk_level = "none"
    report.recommendations = self._generate_security_recommendations(report)
    report.score = self._calculate_security_score(report)
    return report
def _get_security_fix_suggestion(self, pattern_name: str) -> str:
    """Get fix suggestion for security issue."""
    suggestions = {
        "hardcoded_password": "Use environment variables or secure config for passwords",
        "hardcoded_secret": "Use environment variables or secure vault for secrets",
        "sql_injection": "Use parameterized queries or prepared statements",
        "command_injection": "Sanitize inputs and avoid shell execution",
        "unsafe_deserialization": "Use safe deserialization with validation",
        "debug_code": "Remove debug statements before production",
        "weak_crypto": "Use strong cryptographic algorithms (SHA-256, AES)",
    }
    return suggestions.get(pattern_name, "Review and fix security vulnerability")
def _generate_security_recommendations(self, report: SecurityReport) -> list[str]:
    """Generate security recommendations."""
    recommendations: list[str] = []
    if report.vulnerabilities:
```

```
        types = set(v["type"] for v in report.vulnerabilities)
        for vuln_type in types:
            if vuln_type in ("hardcoded_password", "hardcoded_secret"):
                recommendations.append("Use environment variables for secrets")
            elif vuln_type in ("sql_injection", "command_injection"):
                recommendations.append("Implement input validation and sanitization")
            elif vuln_type == "unsafe_deserialization":
                recommendations.append("Use safe deserialization methods")
            elif vuln_type == "weak_crypto":
                recommendations.append("Upgrade to modern cryptographic algorithms")
    if report.risk_level in ("critical", "high"):
        recommendations.append("Immediate remediation required before deployment")
    elif report.risk_level == "medium":
        recommendations.append("Address vulnerabilities before release")
    return recommendations
def _calculate_security_score(self, report: SecurityReport) -> float:
    """Calculate security score."""
    score = 100.0
    for vuln in report.vulnerabilities:
        severity = vuln["severity"]
        if severity == "critical":
            score -= 30
        elif severity == "error":
            score -= 20
        elif severity == "warning":
            score -= 10
        else:
            score -= 5
    return max(0, score)
def validate_compatibility(
    self, code: str, target: str, language: Language = Language.UNKNOWN
) -> ValidationResult:
    """Validate compatibility with target platform/version."""
    result = ValidationResult(category=ValidationCategory.COMPATIBILITY)
    if language == Language.UNKNOWN:
        language = self.detect_language(code)
    target_lower = target.lower()
    if language == Language.PYTHON:
        version_match = re.search(r"python\s*(\d+\.?\d*)", target_lower)
        if version_match:
            target_version = float(version_match.group(1))
            features: dict[str, float] = self._detect_python_features(code)
            for feature, min_version in features.items():
                if min_version > target_version:
                    result.issues.append(
                        ValidationIssue(
                            category=ValidationCategory.COMPATIBILITY,
                            level=ValidationLevel.ERROR,
                            message=f"Feature '{feature}' requires Python {min_version}, ta
rget is {target_version}",
                            suggestion=f"Remove or replace feature for Python {target_versi
on} compatibility",
                            auto_fixable=False,
                        )
                    )
    elif language in (Language.JAVASCRIPT, Language.TYPESCRIPT):
        if "node" in target_lower:
            version_match = re.search(r"node\s*(\d+)", target_lower)
            if version_match:
                target_version = int(version_match.group(1))
                js_features: dict[str, int] = self._detect_js_features(code)
                for feature, min_version in js_features.items():
                    if min_version > target_version:
                        result.issues.append(
                            ValidationIssue(
```

```
                                category=ValidationCategory.COMPATIBILITY,
                                level=ValidationLevel.WARNING,
                                message=f"Feature '{feature}' may not be available in Node
{target_version}",
                                suggestion=f"Use polyfill or alternative for Node {target_v
ersion}",
                                auto_fixable=False,
                            )
                        )
        if "browser" in target_lower:
            browser_features = ["require(", "module.exports", "process.", "__dirname"]
            for feature in browser_features:
                if feature in code:
                    result.issues.append(
                        ValidationIssue(
                            category=ValidationCategory.COMPATIBILITY,
                            level=ValidationLevel.ERROR,
                            message=f"Node.js-specific feature '{feature}' not available in
 browser",
                            suggestion="Use browser-compatible alternatives",
                            auto_fixable=False,
                        )
                    )
    elif language == Language.GO:
        go_features = self._detect_go_features(code)
        for feature, info in go_features.items():
            if info.get("experimental"):
                result.issues.append(
                    ValidationIssue(
                        category=ValidationCategory.COMPATIBILITY,
                        level=ValidationLevel.WARNING,
                        message=f"Experimental feature '{feature}' may not be stable",
                        auto_fixable=False,
                    )
                )
    result.errors = sum(
        1 for i in result.issues if i.level in (ValidationLevel.CRITICAL, ValidationLevel.E
RROR)
    )
    result.warnings = sum(1 for i in result.issues if i.level == ValidationLevel.WARNING)
    result.valid = result.errors == 0
    result.score = self._calculate_validation_score(result)
    return result
def _detect_python_features(self, code: str) -> dict[str, float]:
    """Detect Python features and their minimum version requirements."""
    features: dict[str, float] = {}
    if "match " in code and "case " in code:
        features["match_case"] = 3.10
    if "| " in code and "None" in code:
        features["union_types"] = 3.10
    if "async def" in code:
        features["async"] = 3.5
    if 'f"' in code or "f'" in code:
        features["f_strings"] = 3.6
    if ": " in code and "-> " in code:
        features["type_hints"] = 3.5
    if "@dataclass" in code:
        features["dataclasses"] = 3.7
    return features
def _detect_js_features(self, code: str) -> dict[str, int]:
    """Detect JavaScript features and their minimum Node version."""
    features: dict[str, int] = {}
    if "async " in code or "await " in code:
        features["async_await"] = 8
    if "=>" in code:
```

```
        features["arrow_functions"] = 6
    if "class " in code:
        features["classes"] = 6
    if "let " in code or "const " in code:
        features["let_const"] = 6
    if "`" in code:
        features["template_literals"] = 6
    if "..." in code:
        features["spread_operator"] = 6
    if "import " in code and "from " in code:
        features["es_modules"] = 14
    return features
def _detect_go_features(self, code: str) -> dict[str, dict[str, Any]]:
    """Detect Go features."""
    features: dict[str, dict[str, Any]] = {}
    if "go " in code and "chan" in code:
        features["goroutines"] = {"stable": True}
    if "defer " in code:
        features["defer"] = {"stable": True}
    if "interface{}" in code or "any" in code:
        features["generics_prep"] = {"experimental": "any" in code}
    return features
def calculate_quality_score(self, results: dict[str, Any]) -> ValidatorQualityScore:
    """Calculate comprehensive quality score from validation results."""
    score = ValidatorQualityScore()
    weights = {
        "syntax": 0.15,
        "style": 0.10,
        "functionality": 0.25,
        "performance": 0.15,
        "security": 0.20,
        "compatibility": 0.10,
        "documentation": 0.05,
    }
    for category, _weight in weights.items():
        category_result = results.get(category)
        if category_result:
            if hasattr(category_result, "score"):
                setattr(score, category, category_result.score)
            elif isinstance(category_result, dict):
                setattr(score, category, category_result.get("score", 100.0))
    score.overall = sum(getattr(score, cat) * weight for cat, weight in weights.items())
    score.breakdown = {cat: getattr(score, cat) for cat in weights}
    score.grade = score.calculate_grade()
    return score
def _calculate_validation_score(self, result: ValidationResult) -> float:
    """Calculate validation score from result."""
    total_checks = result.passed_checks + result.failed_checks
    if total_checks == 0:
        return 100.0
    base_score = 100.0
    for issue in result.issues:
        if issue.level == ValidationLevel.CRITICAL:
            base_score -= 30
        elif issue.level == ValidationLevel.ERROR:
            base_score -= 20
        elif issue.level == ValidationLevel.WARNING:
            base_score -= 10
        elif issue.level == ValidationLevel.INFO:
            base_score -= 5
    return max(0, min(100, base_score))
def full_validation(
    self, artifact: Artifact, requirements: Requirements | None = None
) -> ValidationReport:
    """Perform full validation on an artifact."""
```

```
import time
report = ValidationReport(
    artifact_id=artifact.id,
    timestamp=time.time(),
)
language = artifact.language
if language == Language.UNKNOWN:
    language = self.detect_language(artifact.content, artifact.file_path)
code_result = self.validate_code(artifact.content, language)
report.code_validation = code_result
if requirements and requirements.functional_requirements:
    func_result = self.validate_functionality(
        artifact.content, requirements.functional_requirements, language
    )
    report.functionality_validation = func_result
if requirements and requirements.performance_requirements:
    perf_result = self.validate_performance(
        artifact.content, requirements.performance_requirements, language
    )
    report.performance_validation = perf_result
security_result = self.validate_security(artifact.content, language)
report.security_validation = security_result
if requirements and requirements.compatibility_targets:
    compat_results: list[ValidationResult] = []
    for target in requirements.compatibility_targets:
        compat_result = self.validate_compatibility(artifact.content, target, language)
        compat_results.append(compat_result)
    if compat_results:
        report.compatibility_validation = compat_results[0]
results = {
    "syntax": code_result,
    "style": code_result,
    "functionality": report.functionality_validation,
    "performance": report.performance_validation,
    "security": security_result,
    "compatibility": report.compatibility_validation,
}
report.quality_score = self.calculate_quality_score(results)
total = 0
for r in results.values():
    if r is not None and hasattr(r, "issues"):
        total += len(r.issues)
report.total_issues = total
critical = 0
for r in results.values():
    if r is not None and hasattr(r, "issues"):
        for i in r.issues:
            if hasattr(i, "level") and i.level == ValidationLevel.CRITICAL:
                critical += 1
report.critical_issues = critical
errors = 0
for r in results.values():
    if r is not None and hasattr(r, "issues"):
        for i in r.issues:
            if hasattr(i, "level") and i.level == ValidationLevel.ERROR:
                errors += 1
report.errors = errors
warnings = 0
for r in results.values():
    if r is not None and hasattr(r, "issues"):
        for i in r.issues:
            if hasattr(i, "level") and i.level == ValidationLevel.WARNING:
                warnings += 1
report.warnings = warnings
report.overall_score = report.quality_score.overall if report.quality_score else 100.0
```

```
    report.valid = (
        report.critical_issues == 0 and report.errors == 0 and report.overall_score >= 60
    )
    report.recommendations = self._generate_recommendations(report)
    return report
def _generate_recommendations(self, report: ValidationReport) -> list[str]:
    """Generate recommendations from validation report."""
    recommendations: list[str] = []
    if report.code_validation and report.code_validation.has_errors:
        recommendations.append("Fix syntax and code errors")
    if report.security_validation and not report.security_validation.secure:
        recommendations.append("Address security vulnerabilities")
    if report.performance_validation and not report.performance_validation.benchmark_passed
:
        recommendations.append("Optimize performance to meet benchmarks")
    if report.quality_score and report.quality_score.overall < 80:
        recommendations.append(
            f"Improve overall quality score (current: {report.quality_score.overall:.1f})"
        )
    if report.functionality_validation and report.functionality_validation.warnings > 0:
        recommendations.append("Complete implementation of all requirements")
    if report.compatibility_validation and report.compatibility_validation.has_errors:
        recommendations.append("Ensure compatibility with target platforms")
    if report.warnings > 5:
        recommendations.append(f"Address {report.warnings} warning issues")
    return recommendations
async def async_validate_code(self, code: str, language: str = "python") -> ValidationResul
t:
    """Asynchronously validate code."""
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, self.validate_code, code, language)
async def async_full_validation(
    self, artifact: Artifact, requirements: Requirements | None = None
) -> ValidationReport:
    """Asynchronously perform full validation."""
    loop = asyncio.get_event_loop()
    return await loop.run_in_executor(None, self.full_validation, artifact, requirements)
def quick_validate(self, code: str, language: str = "python") -> tuple[bool, float]:
    """Quick validation returning pass/fail and score."""
    lang = self._str_to_language(language)
    result = self.validate_code(code, lang)
    return result.valid, result.score
def get_validation_summary(self, report: ValidationReport) -> str:
    """Get a summary string of validation results."""
    if report.valid:
        return f"Validation PASSED with score {report.overall_score:.1f} (Grade: {report.qu
ality_score.grade if report.quality_score else 'N/A'})"
    else:
        return f"Validation FAILED with {report.critical_issues} critical, {report.errors}
errors, score {report.overall_score:.1f}"
```

File: [90/252] openlaoke/core/hyperauto/verifier.py

```
"""HyperAuto Verification System - Fully autonomous testing and validation.
This module implements the automatic testing and verification loop that runs
without any human interaction after the initial input. It will repeatedly
test and verify until the task is perfectly completed.
"""
from __future__ import annotations
import asyncio
import re
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class VerificationStatus(StrEnum):
    """Status of verification."""
    PENDING = "pending"
    RUNNING = "running"
    PASSED = "passed"
    FAILED = "failed"
    PARTIAL = "partial"
class TestCategory(StrEnum):
    """Categories of tests."""
    SYNTAX = "syntax"
    TYPE_CHECK = "type_check"
    UNIT_TEST = "unit_test"
    INTEGRATION = "integration"
    FUNCTIONAL = "functional"
    PERFORMANCE = "performance"
    LINT = "lint"
@dataclass
class TestResult:
    """Result of a single test."""
    category: TestCategory
    name: str
    passed: bool
    output: str = ""
    error: str = ""
    duration: float = 0.0
    details: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "category": self.category.value,
            "name": self.name,
            "passed": self.passed,
            "output": self.output,
            "error": self.error,
            "duration": self.duration,
            "details": self.details,
        }
@dataclass
class VerificationResult:
    """Result of full verification."""
    status: VerificationStatus
    test_results: list[TestResult] = field(default_factory=list)
    total_tests: int = 0
    passed_tests: int = 0
    failed_tests: int = 0
    coverage: float = 0.0
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
```

```
    suggestions: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "status": self.status.value,
            "test_results": [t.to_dict() for t in self.test_results],
            "total_tests": self.total_tests,
            "passed_tests": self.passed_tests,
            "failed_tests": self.failed_tests,
            "coverage": self.coverage,
            "errors": self.errors,
            "warnings": self.warnings,
            "suggestions": self.suggestions,
        }
    @property
    def is_perfect(self) -> bool:
        """Check if all tests passed with no errors or warnings."""
        return (
            self.status == VerificationStatus.PASSED
            and self.failed_tests == 0
            and len(self.errors) == 0
            and len(self.warnings) == 0
        )
    @property
    def pass_rate(self) -> float:
        """Calculate pass rate."""
        if self.total_tests == 0:
            return 0.0
        return self.passed_tests / self.total_tests
@dataclass
class VerificationConfig:
    """Configuration for verification."""
    max_iterations: int = 10
    min_pass_rate: float = 1.0
    required_coverage: float = 80.0
    enable_syntax_check: bool = True
    enable_type_check: bool = True
    enable_unit_tests: bool = True
    enable_lint: bool = True
    enable_integration_tests: bool = False
    timeout_per_test: float = 60.0
    retry_on_failure: bool = True
    auto_fix_enabled: bool = True
class HyperAutoVerifier:
    """Fully autonomous verification system.
    This system runs tests and validation completely autonomously:
    - No human interaction required after initial input
    - Automatically runs multiple test iterations
    - Verifies completion is perfect before stopping
    - Auto-fixes issues when possible
    """
    def __init__(
        self,
        app_state: AppState,
        config: VerificationConfig | None = None,
    ) -> None:
        self.app_state = app_state
        self.config = config or VerificationConfig()
        self._iteration_count = 0
        self._verification_history: list[VerificationResult] = []
        self._fixed_issues: list[str] = []
    async def verify_until_perfect(
        self,
        task_description: str,
        files_created: list[str] | None = None,
        files_modified: list[str] | None = None,
```

```
) -> VerificationResult:
    """Run verification loop until task is perfectly completed.
    This is the main entry point for autonomous verification.
    It will run tests repeatedly until:
    1. All tests pass
    2. No errors or warnings
    3. Coverage meets requirements
    4. Or max iterations reached
    Args:
        task_description: Description of the task to verify
        files_created: List of files that were created
        files_modified: List of files that were modified
    Returns:
        Final verification result
    """
    self._iteration_count = 0
    while self._iteration_count < self.config.max_iterations:
        self._iteration_count += 1
        print(f"\n{'=' * 60}")
        print(f"Verification Iteration {self._iteration_count}/{self.config.max_iterations}
")
        print(f"{'=' * 60}")
        result = await self._run_single_verification(
            task_description=task_description,
            files_created=files_created or [],
            files_modified=files_modified or [],
        )
        self._verification_history.append(result)
        if result.is_perfect:
            print(f"\n[OK] Perfect completion achieved at iteration {self._iteration_count}
")
            return result
        if result.pass_rate >= self.config.min_pass_rate and len(result.errors) == 0:
            print(f"\n[OK] Acceptable completion (pass rate: {result.pass_rate:.1%})")
            return result
        if self.config.auto_fix_enabled and result.failed_tests > 0:
            fixed = await self._attempt_auto_fix(result)
            if fixed:
                self._fixed_issues.extend(fixed)
                print(f"\n-> Auto-fixed {len(fixed)} issues, re-running verification...")
                continue
        if not self.config.retry_on_failure:
            print("\n[X] Verification failed, retry disabled")
            return result
        print("\n-> Issues found, continuing to next iteration...")
    print(f"\n? Max iterations ({self.config.max_iterations}) reached")
    return (
        self._verification_history[-1]
        if self._verification_history
        else VerificationResult(
            status=VerificationStatus.FAILED,
            errors=["Max verification iterations reached"],
        )
    )
async def _run_single_verification(
    self,
    task_description: str,
    files_created: list[str],
    files_modified: list[str],
) -> VerificationResult:
    """Run a single verification pass."""
    results: list[TestResult] = []
    errors: list[str] = []
    warnings: list[str] = []
    files_to_check = list(set(files_created + files_modified))
```

```
    if self.config.enable_syntax_check:
        syntax_results = await self._check_syntax(files_to_check)
        results.extend(syntax_results)
    if self.config.enable_type_check:
        type_results = await self._check_types(files_to_check)
        results.extend(type_results)
    if self.config.enable_lint:
        lint_results = await self._run_lint(files_to_check)
        results.extend(lint_results)
    if self.config.enable_unit_tests:
        test_results = await self._run_unit_tests()
        results.extend(test_results)
    if self.config.enable_integration_tests:
        integration_results = await self._run_integration_tests()
        results.extend(integration_results)
    for r in results:
        if not r.passed and r.error:
            errors.append(f"{r.category.value}:{r.name}: {r.error}")
        if r.details.get("warnings"):
            warnings.extend(r.details["warnings"])
    total = len(results)
    passed = sum(1 for r in results if r.passed)
    failed = total - passed
    if failed == 0 and len(errors) == 0:
        status = VerificationStatus.PASSED
    elif passed > 0:
        status = VerificationStatus.PARTIAL
    else:
        status = VerificationStatus.FAILED
    return VerificationResult(
        status=status,
        test_results=results,
        total_tests=total,
        passed_tests=passed,
        failed_tests=failed,
        errors=errors,
        warnings=warnings,
        suggestions=self._generate_suggestions(results, errors),
    )
async def _check_syntax(self, files: list[str]) -> list[TestResult]:
    """Check syntax for all files."""
    results: list[TestResult] = []
    for filepath in files:
        if not filepath:
            continue
        if filepath.endswith(".py"):
            result = await self._check_python_syntax(filepath)
            results.append(result)
        elif filepath.endswith((".c", ".h")):
            result = await self._check_c_syntax(filepath)
            results.append(result)
        elif filepath.endswith((".js", ".ts")):
            result = await self._check_js_syntax(filepath)
            results.append(result)
    return results
async def _check_python_syntax(self, filepath: str) -> TestResult:
    """Check Python syntax."""
    start = time.time()
    try:
        proc = await asyncio.create_subprocess_exec(
            "python",
            "-m",
            "py_compile",
            filepath,
            stdout=asyncio.subprocess.PIPE,
```

```
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=passed,
            output=stdout.decode() if stdout else "",
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
    except TimeoutError:
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=False,
            error="Timeout during syntax check",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _check_c_syntax(self, filepath: str) -> TestResult:
    """Check C syntax."""
    start = time.time()
    try:
        proc = await asyncio.create_subprocess_exec(
            "gcc",
            "-fsyntax-only",
            "-Wall",
            filepath,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=passed,
            output=stdout.decode() if stdout else "",
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=True,
            error="GCC not available, skipping C syntax check",
            duration=time.time() - start,
        )
    except Exception as e:
```

```
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _check_js_syntax(self, filepath: str) -> TestResult:
    """Check JavaScript/TypeScript syntax."""
    start = time.time()
    try:
        if filepath.endswith(".ts"):
            cmd = ["npx", "tsc", "--noEmit", filepath]
        else:
            cmd = ["node", "--check", filepath]
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=passed,
            output=stdout.decode() if stdout else "",
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=True,
            error="Node.js not available, skipping JS syntax check",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.SYNTAX,
            name=f"syntax:{filepath}",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _check_types(self, files: list[str]) -> list[TestResult]:
    """Run type checking."""
    results: list[TestResult] = []
    py_files = [f for f in files if f.endswith(".py")]
    if py_files and self.config.enable_type_check:
        result = await self._run_mypy(py_files)
        results.append(result)
    ts_files = [f for f in files if f.endswith(".ts")]
    if ts_files:
        result = await self._run_tsc(ts_files)
        results.append(result)
    return results
async def _run_mypy(self, files: list[str]) -> TestResult:
    """Run mypy type checking."""
    start = time.time()
    try:
```

```
        cmd = ["python", "-m", "mypy"] + files
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        output = stdout.decode() if stdout else ""
        error_output = stderr.decode() if stderr else ""
        errors = []
        for line in output.split("\n"):
            if "error:" in line.lower():
                errors.append(line)
        passed = proc.returncode == 0 or len(errors) == 0
        return TestResult(
            category=TestCategory.TYPE_CHECK,
            name="mypy",
            passed=passed,
            output=output,
            error=error_output if not passed else "",
            duration=time.time() - start,
            details={"errors": errors},
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.TYPE_CHECK,
            name="mypy",
            passed=True,
            error="mypy not installed, skipping type check",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.TYPE_CHECK,
            name="mypy",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _run_tsc(self, files: list[str]) -> TestResult:
    """Run TypeScript compiler check."""
    start = time.time()
    try:
        cmd = ["npx", "tsc", "--noEmit"] + files
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.TYPE_CHECK,
            name="tsc",
            passed=passed,
            output=stdout.decode() if stdout else "",
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
```

```
    except Exception as e:
        return TestResult(
            category=TestCategory.TYPE_CHECK,
            name="tsc",
            passed=True,
            error=f"TypeScript check skipped: {e}",
            duration=time.time() - start,
        )
async def _run_lint(self, files: list[str]) -> list[TestResult]:
    """Run linting."""
    results: list[TestResult] = []
    py_files = [f for f in files if f.endswith(".py")]
    if py_files:
        result = await self._run_ruff(py_files)
        results.append(result)
    c_files = [f for f in files if f.endswith((".c", ".h"))]
    if c_files:
        result = await self._run_cppcheck(c_files)
        results.append(result)
    return results
async def _run_ruff(self, files: list[str]) -> TestResult:
    """Run ruff linter."""
    start = time.time()
    try:
        cmd = ["python", "-m", "ruff", "check"] + files
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        output = stdout.decode() if stdout else ""
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.LINT,
            name="ruff",
            passed=passed,
            output=output,
            error="" if passed else output,
            duration=time.time() - start,
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.LINT,
            name="ruff",
            passed=True,
            error="ruff not installed, skipping lint",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.LINT,
            name="ruff",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _run_cppcheck(self, files: list[str]) -> TestResult:
    """Run cppcheck for C files."""
    start = time.time()
    try:
        cmd = ["cppcheck", "--error-exitcode=1"] + files
```

```
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.LINT,
            name="cppcheck",
            passed=passed,
            output=stdout.decode() if stdout else "",
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.LINT,
            name="cppcheck",
            passed=True,
            error="cppcheck not available",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.LINT,
            name="cppcheck",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _run_unit_tests(self) -> list[TestResult]:
    """Run unit tests."""
    results: list[TestResult] = []
    pytest_result = await self._run_pytest()
    if pytest_result:
        results.append(pytest_result)
    make_result = await self._run_make_test()
    if make_result:
        results.append(make_result)
    return results
async def _run_pytest(self) -> TestResult | None:
    """Run pytest."""
    start = time.time()
    try:
        cmd = ["python", "-m", "pytest", "-v", "--tb=short"]
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
            cwd=self.app_state.cwd,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test * 5,
        )
        output = stdout.decode() if stdout else ""
        passed = proc.returncode == 0
        match = re.search(r"(\d+) passed", output)
        passed_count = int(match.group(1)) if match else 0
        match = re.search(r"(\d+) failed", output)
        failed_count = int(match.group(1)) if match else 0
```

```
        return TestResult(
            category=TestCategory.UNIT_TEST,
            name="pytest",
            passed=passed,
            output=output,
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
            details={
                "passed": passed_count,
                "failed": failed_count,
            },
        )
    except FileNotFoundError:
        return TestResult(
            category=TestCategory.UNIT_TEST,
            name="pytest",
            passed=True,
            error="pytest not found, skipping",
            duration=time.time() - start,
        )
    except TimeoutError:
        return TestResult(
            category=TestCategory.UNIT_TEST,
            name="pytest",
            passed=False,
            error="Test execution timeout",
            duration=time.time() - start,
        )
    except Exception as e:
        return TestResult(
            category=TestCategory.UNIT_TEST,
            name="pytest",
            passed=False,
            error=str(e),
            duration=time.time() - start,
        )
async def _run_make_test(self) -> TestResult | None:
    """Run make test for C projects."""
    start = time.time()
    try:
        cmd = ["make", "test"]
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
            cwd=self.app_state.cwd,
        )
        stdout, stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=self.config.timeout_per_test,
        )
        output = stdout.decode() if stdout else ""
        passed = proc.returncode == 0
        return TestResult(
            category=TestCategory.UNIT_TEST,
            name="make_test",
            passed=passed,
            output=output,
            error=stderr.decode() if stderr else "",
            duration=time.time() - start,
        )
    except FileNotFoundError:
        return None
    except Exception:
        return None
```

```
    async def _run_integration_tests(self) -> list[TestResult]:
        """Run integration tests."""
        return []
    async def _attempt_auto_fix(self, result: VerificationResult) -> list[str]:
        """Attempt to automatically fix issues."""
        fixed: list[str] = []
        for test_result in result.test_results:
            if test_result.passed:
                continue
            if test_result.category == TestCategory.LINT:
                fixed_issues = await self._auto_fix_lint_issues(test_result)
                fixed.extend(fixed_issues)
        return fixed
    async def _auto_fix_lint_issues(self, test_result: TestResult) -> list[str]:
        """Auto-fix lint issues using ruff --fix."""
        fixed: list[str] = []
        if "ruff" in test_result.name:
            try:
                cmd = ["python", "-m", "ruff", "check", "--fix", "."]
                proc = await asyncio.create_subprocess_exec(
                    *cmd,
                    stdout=asyncio.subprocess.PIPE,
                    stderr=asyncio.subprocess.PIPE,
                    cwd=self.app_state.cwd,
                )
                await proc.communicate()
                if proc.returncode == 0:
                    fixed.append("Auto-fixed lint issues with ruff")
            except Exception:
                pass
        return fixed
    def _generate_suggestions(
        self,
        results: list[TestResult],
        errors: list[str],
    ) -> list[str]:
        """Generate suggestions for fixing issues."""
        suggestions: list[str] = []
        for test in results:
            if not test.passed:
                if test.category == TestCategory.SYNTAX:
                    suggestions.append(f"Fix syntax errors in {test.name}")
                elif test.category == TestCategory.TYPE_CHECK:
                    suggestions.append("Add type annotations or fix type errors")
                elif test.category == TestCategory.LINT:
                    suggestions.append("Run 'ruff check --fix' to auto-fix lint issues")
                elif test.category == TestCategory.UNIT_TEST:
                    suggestions.append("Fix failing unit tests")
        return list(set(suggestions))
    def get_verification_summary(self) -> dict[str, Any]:
        """Get summary of all verifications."""
        if not self._verification_history:
            return {"iterations": 0, "final_status": "none"}
        final = self._verification_history[-1]
        return {
            "iterations": self._iteration_count,
            "final_status": final.status.value,
            "is_perfect": final.is_perfect,
            "pass_rate": final.pass_rate,
            "total_tests": final.total_tests,
            "passed_tests": final.passed_tests,
            "failed_tests": final.failed_tests,
            "fixed_issues": self._fixed_issues,
            "history": [v.to_dict() for v in self._verification_history],
        }
```

File: [91/252] openlaoke/core/hyperauto/workflow.py

```
"""Workflow orchestrator for automated task coordination."""
from __future__ import annotations
import asyncio
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from uuid import uuid4
from openlaoke.core.hyperauto.types import SubTask, SubTaskStatus, WorkflowContext
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class WorkflowPhase(StrEnum):
    """Phases of the workflow execution."""
    ANALYZE = "analyze"
    PLAN = "plan"
    INIT = "init"
    SEARCH = "search"
    CREATE = "create"
    IMPLEMENT = "implement"
    TEST = "test"
    REVIEW = "review"
    FINALIZE = "finalize"
class RecoveryStrategy(StrEnum):
    """Strategies for error recovery."""
    RETRY = "retry"
    SKIP = "skip"
    ABORT = "abort"
    ROLLBACK = "rollback"
    ADAPT = "adapt"
@dataclass
class TaskTree:
    """Hierarchical representation of decomposed tasks."""
    root_task: str
    subtasks: list[SubTask] = field(default_factory=list)
    dependencies: dict[str, list[str]] = field(default_factory=dict)
    priority_order: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "root_task": self.root_task,
            "subtasks": [s.to_dict() for s in self.subtasks],
            "dependencies": self.dependencies,
            "priority_order": self.priority_order,
        }
@dataclass
class DependencyEdge:
    """Edge in the dependency graph."""
    source: str
    target: str
    dependency_type: str = "hard"
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class DependencyGraph:
    """Graph representing task dependencies."""
    nodes: list[str] = field(default_factory=list)
    edges: list[DependencyEdge] = field(default_factory=list)
    adjacency: dict[str, list[str]] = field(default_factory=dict)
    reverse_adjacency: dict[str, list[str]] = field(default_factory=dict)
    def get_dependencies(self, task_id: str) -> list[str]:
        return self.reverse_adjacency.get(task_id, [])
    def get_dependents(self, task_id: str) -> list[str]:
        return self.adjacency.get(task_id, [])
```

```
    def topological_sort(self) -> list[str]:
        visited = set()
        stack: list[str] = []
        temp_marks = set()
        def visit(node: str) -> None:
            if node in temp_marks:
                raise ValueError(f"Circular dependency detected at {node}")
            if node not in visited:
                temp_marks.add(node)
                for dep in self.get_dependencies(node):
                    visit(dep)
                temp_marks.remove(node)
                visited.add(node)
                stack.append(node)
        for node in self.nodes:
            if node not in visited:
                visit(node)
        return stack
    def detect_cycles(self) -> list[list[str]]:
        cycles: list[list[str]] = []
        def dfs(node: str, path: list[str], visited: set[str]) -> None:
            if node in path:
                cycle_start = path.index(node)
                cycles.append(path[cycle_start:] + [node])
                return
            if node in visited:
                return
            visited.add(node)
            path.append(node)
            for neighbor in self.adjacency.get(node, []):
                dfs(neighbor, path, visited)
            path.pop()
        visited: set[str] = set()
        for node in self.nodes:
            if node not in visited:
                dfs(node, [], visited)
        return cycles
    def to_dict(self) -> dict[str, Any]:
        return {
            "nodes": self.nodes,
            "edges": [
                {"source": e.source, "target": e.target, "type": e.dependency_type}
                for e in self.edges
            ],
            "adjacency": self.adjacency,
            "reverse_adjacency": self.reverse_adjacency,
        }
@dataclass
class ExecutionStep:
    """Single step in the execution plan."""
    task_id: str
    phase: WorkflowPhase
    priority: int = 0
    parallel_group: int = 0
    estimated_duration: float = 0.0
    dependencies: list[str] = field(default_factory=list)
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class ExecutionPlan:
    """Complete execution plan for a workflow."""
    steps: list[ExecutionStep] = field(default_factory=list)
    parallel_groups: dict[int, list[str]] = field(default_factory=dict)
    total_estimated_duration: float = 0.0
    critical_path: list[str] = field(default_factory=list)
    checkpoints: list[str] = field(default_factory=list)
```

```
    def to_dict(self) -> dict[str, Any]:
        return {
            "steps": [
                {
                    "task_id": s.task_id,
                    "phase": s.phase.value,
                    "priority": s.priority,
                    "parallel_group": s.parallel_group,
                    "estimated_duration": s.estimated_duration,
                    "dependencies": s.dependencies,
                }
                for s in self.steps
            ],
            "parallel_groups": self.parallel_groups,
            "total_estimated_duration": self.total_estimated_duration,
            "critical_path": self.critical_path,
            "checkpoints": self.checkpoints,
        }
@dataclass
class RecoveryAction:
    """Action to recover from an error."""
    strategy: RecoveryStrategy
    retry_count: int = 0
    max_retries: int = 3
    alternative_task: str | None = None
    rollback_to: str | None = None
    message: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class ProgressInfo:
    """Real-time progress information."""
    current_phase: WorkflowPhase = WorkflowPhase.ANALYZE
    current_task: str = ""
    completed_tasks: int = 0
    total_tasks: int = 0
    percentage: float = 0.0
    elapsed_time: float = 0.0
    estimated_remaining: float = 0.0
    active_tasks: list[str] = field(default_factory=list)
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "current_phase": self.current_phase.value,
            "current_task": self.current_task,
            "completed_tasks": self.completed_tasks,
            "total_tasks": self.total_tasks,
            "percentage": self.percentage,
            "elapsed_time": self.elapsed_time,
            "estimated_remaining": self.estimated_remaining,
            "active_tasks": self.active_tasks,
            "errors": self.errors,
            "warnings": self.warnings,
        }
@dataclass
class ExecutionResult:
    """Result of executing a single task."""
    task_id: str
    success: bool
    output: Any = None
    error: str | None = None
    duration: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
```

```
            "task_id": self.task_id,
            "success": self.success,
            "output": self.output,
            "error": self.error,
            "duration": self.duration,
            "metadata": self.metadata,
        }
@dataclass
class WorkflowResult:
    """Complete result of a workflow execution."""
    workflow_id: str
    success: bool
    task_tree: TaskTree
    execution_plan: ExecutionPlan
    execution_results: list[ExecutionResult] = field(default_factory=list)
    total_duration: float = 0.0
    error: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "workflow_id": self.workflow_id,
            "success": self.success,
            "task_tree": self.task_tree.to_dict(),
            "execution_plan": self.execution_plan.to_dict(),
            "execution_results": [r.to_dict() for r in self.execution_results],
            "total_duration": self.total_duration,
            "error": self.error,
            "metadata": self.metadata,
        }
class WorkflowOrchestrator:
    """Orchestrates automated workflow execution with intelligent coordination."""
    def __init__(self, app_state: AppState) -> None:
        self.app_state = app_state
        self.context = WorkflowContext()
        self._progress = ProgressInfo()
        self._execution_lock = asyncio.Lock()
        self._task_results: dict[str, ExecutionResult] = {}
        self._start_time: float = 0.0
    def decompose_task(self, task: str) -> TaskTree:
        subtasks = self._analyze_and_decompose(task)
        dependencies = self._infer_dependencies(subtasks)
        priority_order = self._calculate_priority_order(subtasks, dependencies)
        return TaskTree(
            root_task=task,
            subtasks=subtasks,
            dependencies=dependencies,
            priority_order=priority_order,
        )
    def _analyze_and_decompose(self, task: str) -> list[SubTask]:
        keywords = task.lower().split()
        subtasks: list[SubTask] = []
        if any(kw in keywords for kw in ["create", "new", "build", "develop"]):
            subtasks.extend(self._create_development_subtasks(task))
        elif any(kw in keywords for kw in ["fix", "bug", "issue", "error"]):
            subtasks.extend(self._create_fix_subtasks(task))
        elif any(kw in keywords for kw in ["refactor", "improve", "optimize"]):
            subtasks.extend(self._create_refactor_subtasks(task))
        elif any(kw in keywords for kw in ["test", "verify", "validate"]):
            subtasks.extend(self._create_test_subtasks(task))
        elif any(kw in keywords for kw in ["analyze", "review", "check"]):
            subtasks.extend(self._create_analysis_subtasks(task))
        else:
            subtasks.extend(self._create_generic_subtasks(task))
        return subtasks
    def _create_development_subtasks(self, task: str) -> list[SubTask]:
```

```
return [
    SubTask(
        name="analyze_requirements",
        description=f"Analyze requirements for: {task}",
        status=SubTaskStatus.PENDING,
        priority=10,
        metadata={"phase": WorkflowPhase.ANALYZE.value},
    ),
    SubTask(
        name="search_references",
        description="Search for similar implementations and references",
        status=SubTaskStatus.PENDING,
        priority=9,
        dependencies=["analyze_requirements"],
        metadata={"phase": WorkflowPhase.SEARCH.value},
    ),
    SubTask(
        name="design_architecture",
        description="Design architecture and structure",
        status=SubTaskStatus.PENDING,
        priority=8,
        dependencies=["search_references"],
        metadata={"phase": WorkflowPhase.PLAN.value},
    ),
    SubTask(
        name="create_structure",
        description="Create project structure and files",
        status=SubTaskStatus.PENDING,
        priority=7,
        dependencies=["design_architecture"],
        metadata={"phase": WorkflowPhase.CREATE.value},
    ),
    SubTask(
        name="implement_core",
        description="Implement core functionality",
        status=SubTaskStatus.PENDING,
        priority=6,
        dependencies=["create_structure"],
        metadata={"phase": WorkflowPhase.IMPLEMENT.value},
    ),
    SubTask(
        name="write_tests",
        description="Write tests for implemented features",
        status=SubTaskStatus.PENDING,
        priority=5,
        dependencies=["implement_core"],
        metadata={"phase": WorkflowPhase.TEST.value},
    ),
    SubTask(
        name="review_code",
        description="Review code quality and standards",
        status=SubTaskStatus.PENDING,
        priority=4,
        dependencies=["write_tests"],
        metadata={"phase": WorkflowPhase.REVIEW.value},
    ),
    SubTask(
        name="finalize",
        description="Finalize and clean up",
        status=SubTaskStatus.PENDING,
        priority=3,
        dependencies=["review_code"],
        metadata={"phase": WorkflowPhase.FINALIZE.value},
    ),
]
```

```
def _create_fix_subtasks(self, task: str) -> list[SubTask]:
    return [
        SubTask(
            name="reproduce_issue",
            description=f"Reproduce the issue: {task}",
            status=SubTaskStatus.PENDING,
            priority=10,
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="identify_root_cause",
            description="Identify root cause of the issue",
            status=SubTaskStatus.PENDING,
            priority=9,
            dependencies=["reproduce_issue"],
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="search_similar_fixes",
            description="Search for similar fixes and solutions",
            status=SubTaskStatus.PENDING,
            priority=8,
            dependencies=["identify_root_cause"],
            metadata={"phase": WorkflowPhase.SEARCH.value},
        ),
        SubTask(
            name="implement_fix",
            description="Implement the fix",
            status=SubTaskStatus.PENDING,
            priority=7,
            dependencies=["search_similar_fixes"],
            metadata={"phase": WorkflowPhase.IMPLEMENT.value},
        ),
        SubTask(
            name="verify_fix",
            description="Verify the fix resolves the issue",
            status=SubTaskStatus.PENDING,
            priority=6,
            dependencies=["implement_fix"],
            metadata={"phase": WorkflowPhase.TEST.value},
        ),
        SubTask(
            name="review_fix",
            description="Review fix for side effects",
            status=SubTaskStatus.PENDING,
            priority=5,
            dependencies=["verify_fix"],
            metadata={"phase": WorkflowPhase.REVIEW.value},
        ),
    ]
def _create_refactor_subtasks(self, task: str) -> list[SubTask]:
    return [
        SubTask(
            name="analyze_codebase",
            description=f"Analyze codebase structure for: {task}",
            status=SubTaskStatus.PENDING,
            priority=10,
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="identify_improvements",
            description="Identify improvement opportunities",
            status=SubTaskStatus.PENDING,
            priority=9,
            dependencies=["analyze_codebase"],
```

```
            metadata={"phase": WorkflowPhase.PLAN.value},
        ),
        SubTask(
            name="implement_refactor",
            description="Implement refactoring changes",
            status=SubTaskStatus.PENDING,
            priority=8,
            dependencies=["identify_improvements"],
            metadata={"phase": WorkflowPhase.IMPLEMENT.value},
        ),
        SubTask(
            name="verify_behavior",
            description="Verify behavior remains unchanged",
            status=SubTaskStatus.PENDING,
            priority=7,
            dependencies=["implement_refactor"],
            metadata={"phase": WorkflowPhase.TEST.value},
        ),
        SubTask(
            name="review_changes",
            description="Review refactoring changes",
            status=SubTaskStatus.PENDING,
            priority=6,
            dependencies=["verify_behavior"],
            metadata={"phase": WorkflowPhase.REVIEW.value},
        ),
    ]
def _create_test_subtasks(self, task: str) -> list[SubTask]:
    return [
        SubTask(
            name="analyze_test_requirements",
            description=f"Analyze test requirements: {task}",
            status=SubTaskStatus.PENDING,
            priority=10,
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="create_test_plan",
            description="Create test plan and cases",
            status=SubTaskStatus.PENDING,
            priority=9,
            dependencies=["analyze_test_requirements"],
            metadata={"phase": WorkflowPhase.PLAN.value},
        ),
        SubTask(
            name="implement_tests",
            description="Implement test cases",
            status=SubTaskStatus.PENDING,
            priority=8,
            dependencies=["create_test_plan"],
            metadata={"phase": WorkflowPhase.CREATE.value},
        ),
        SubTask(
            name="run_tests",
            description="Run and validate tests",
            status=SubTaskStatus.PENDING,
            priority=7,
            dependencies=["implement_tests"],
            metadata={"phase": WorkflowPhase.TEST.value},
        ),
        SubTask(
            name="analyze_coverage",
            description="Analyze test coverage",
            status=SubTaskStatus.PENDING,
            priority=6,
```

```
            dependencies=["run_tests"],
            metadata={"phase": WorkflowPhase.REVIEW.value},
        ),
    ]
def _create_analysis_subtasks(self, task: str) -> list[SubTask]:
    return [
        SubTask(
            name="gather_information",
            description=f"Gather information for: {task}",
            status=SubTaskStatus.PENDING,
            priority=10,
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="analyze_data",
            description="Analyze collected data",
            status=SubTaskStatus.PENDING,
            priority=9,
            dependencies=["gather_information"],
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="generate_report",
            description="Generate analysis report",
            status=SubTaskStatus.PENDING,
            priority=8,
            dependencies=["analyze_data"],
            metadata={"phase": WorkflowPhase.FINALIZE.value},
        ),
    ]
def _create_generic_subtasks(self, task: str) -> list[SubTask]:
    return [
        SubTask(
            name="analyze_task",
            description=f"Analyze task requirements: {task}",
            status=SubTaskStatus.PENDING,
            priority=10,
            metadata={"phase": WorkflowPhase.ANALYZE.value},
        ),
        SubTask(
            name="plan_execution",
            description="Plan execution strategy",
            status=SubTaskStatus.PENDING,
            priority=9,
            dependencies=["analyze_task"],
            metadata={"phase": WorkflowPhase.PLAN.value},
        ),
        SubTask(
            name="execute_task",
            description="Execute planned actions",
            status=SubTaskStatus.PENDING,
            priority=8,
            dependencies=["plan_execution"],
            metadata={"phase": WorkflowPhase.IMPLEMENT.value},
        ),
        SubTask(
            name="verify_results",
            description="Verify execution results",
            status=SubTaskStatus.PENDING,
            priority=7,
            dependencies=["execute_task"],
            metadata={"phase": WorkflowPhase.TEST.value},
        ),
        SubTask(
            name="finalize_task",
```

```
            description="Finalize and clean up",
            status=SubTaskStatus.PENDING,
            priority=6,
            dependencies=["verify_results"],
            metadata={"phase": WorkflowPhase.FINALIZE.value},
        ),
    ]
def _infer_dependencies(self, subtasks: list[SubTask]) -> dict[str, list[str]]:
    dependencies: dict[str, list[str]] = {}
    for task in subtasks:
        dependencies[task.id] = task.dependencies.copy()
    return dependencies
def _calculate_priority_order(
    self, subtasks: list[SubTask], dependencies: dict[str, list[str]]
) -> list[str]:
    visited: set[str] = set()
    order: list[str] = []
    def visit(task_id: str) -> None:
        if task_id in visited:
            return
        visited.add(task_id)
        for dep_id in dependencies.get(task_id, []):
            visit(dep_id)
        order.append(task_id)
    for task in sorted(subtasks, key=lambda t: t.priority, reverse=True):
        visit(task.id)
    return order
def analyze_dependencies(self, tasks: list[SubTask]) -> DependencyGraph:
    graph = DependencyGraph()
    for task in tasks:
        graph.nodes.append(task.id)
        graph.adjacency[task.id] = []
        graph.reverse_adjacency[task.id] = []
    for task in tasks:
        for dep_id in task.dependencies:
            if dep_id in graph.nodes:
                graph.edges.append(
                    DependencyEdge(source=dep_id, target=task.id, dependency_type="hard")
                )
                graph.adjacency[dep_id].append(task.id)
                graph.reverse_adjacency[task.id].append(dep_id)
    return graph
def create_execution_plan(self, graph: DependencyGraph) -> ExecutionPlan:
    plan = ExecutionPlan()
    topo_order = graph.topological_sort()
    task_map = {task.id: task for task in self.context.sub_tasks}
    dependency_depths: dict[str, int] = {}
    for task_id in topo_order:
        deps = graph.get_dependencies(task_id)
        if deps:
            max_dep_depth = max(dependency_depths.get(dep, 0) for dep in deps)
            dependency_depths[task_id] = max_dep_depth + 1
        else:
            dependency_depths[task_id] = 0
    parallel_groups: dict[int, list[str]] = {}
    for task_id, depth in dependency_depths.items():
        if depth not in parallel_groups:
            parallel_groups[depth] = []
        parallel_groups[depth].append(task_id)
    for group_num in sorted(parallel_groups.keys()):
        group_tasks = parallel_groups[group_num]
        for task_id in group_tasks:
            task = task_map.get(task_id)
            if task:
                phase = WorkflowPhase(task.metadata.get("phase", WorkflowPhase.IMPLEMENT.va
```

```
lue))
                plan.steps.append(
                    ExecutionStep(
                        task_id=task_id,
                        phase=phase,
                        priority=task.priority,
                        parallel_group=group_num,
                        dependencies=graph.get_dependencies(task_id),
                    )
                )
    plan.parallel_groups = parallel_groups
    plan.total_estimated_duration = len(plan.steps) * 2.0
    plan.critical_path = self._find_critical_path(graph, dependency_depths)
    plan.checkpoints = self._identify_checkpoints(plan.steps)
    return plan
def _find_critical_path(self, graph: DependencyGraph, depths: dict[str, int]) -> list[str]:
    if not depths:
        return []
    max_depth = max(depths.values())
    critical_path: list[str] = []
    current_depth = max_depth
    current_nodes = [n for n, d in depths.items() if d == current_depth]
    while current_depth >= 0 and current_nodes:
        node = current_nodes[0]
        critical_path.insert(0, node)
        current_depth -= 1
        current_nodes = [n for n, d in depths.items() if d == current_depth]
    return critical_path
def _identify_checkpoints(self, steps: list[ExecutionStep]) -> list[str]:
    checkpoints: list[str] = []
    seen_phases: set[WorkflowPhase] = set()
    for step in steps:
        if step.phase not in seen_phases:
            seen_phases.add(step.phase)
            checkpoints.append(step.task_id)
    return checkpoints
async def execute_plan(self, plan: ExecutionPlan) -> ExecutionResult:
    async with self._execution_lock:
        self._start_time = time.time()
        results: list[ExecutionResult] = []
        for group_num in sorted(plan.parallel_groups.keys()):
            task_ids = plan.parallel_groups[group_num]
            group_results = await self._execute_parallel_tasks(task_ids)
            results.extend(group_results)
            for result in group_results:
                self._task_results[result.task_id] = result
                if result.success:
                    self._progress.completed_tasks += 1
                else:
                    self._progress.errors.append(
                        result.error or f"Task {result.task_id} failed"
                    )
            self._progress.percentage = (
                self._progress.completed_tasks / max(len(plan.steps), 1)
            ) * 100
        success = all(r.success for r in results)
        return ExecutionResult(
            task_id="workflow_execution",
            success=success,
            output=[r.to_dict() for r in results],
            duration=time.time() - self._start_time,
        )
async def _execute_parallel_tasks(self, task_ids: list[str]) -> list[ExecutionResult]:
    tasks = [self._execute_single_task(tid) for tid in task_ids]
    results = await asyncio.gather(*tasks, return_exceptions=True)
```

```
    execution_results: list[ExecutionResult] = []
    for i, result in enumerate(results):
        if isinstance(result, Exception):
            execution_results.append(
                ExecutionResult(
                    task_id=task_ids[i],
                    success=False,
                    error=f"Exception during execution: {result}",
                )
            )
        elif isinstance(result, ExecutionResult):
            execution_results.append(result)
        else:
            execution_results.append(
                ExecutionResult(
                    task_id=task_ids[i],
                    success=False,
                    error=f"Unexpected result type: {type(result)}",
                )
            )
    return execution_results
async def _execute_single_task(self, task_id: str) -> ExecutionResult:
    task = next((t for t in self.context.sub_tasks if t.id == task_id), None)
    if not task:
        return ExecutionResult(
            task_id=task_id, success=False, error=f"Task {task_id} not found"
        )
    task.status = SubTaskStatus.RUNNING
    self._progress.current_task = task_id
    self._progress.active_tasks.append(task_id)
    start_time = time.time()
    try:
        result = await self._execute_task_action(task)
        task.status = SubTaskStatus.COMPLETED
        task.result = result
        return ExecutionResult(
            task_id=task_id,
            success=True,
            output=result,
            duration=time.time() - start_time,
        )
    except Exception as e:
        task.status = SubTaskStatus.FAILED
        task.error = str(e)
        recovery = self.handle_error(e, task)
        if (
            recovery.strategy == RecoveryStrategy.RETRY
            and recovery.retry_count < recovery.max_retries
        ):
            return await self._retry_task(task, recovery)
        return ExecutionResult(
            task_id=task_id,
            success=False,
            error=str(e),
            duration=time.time() - start_time,
        )
    finally:
        if task_id in self._progress.active_tasks:
            self._progress.active_tasks.remove(task_id)
async def _execute_task_action(self, task: SubTask) -> Any:
    phase = WorkflowPhase(task.metadata.get("phase", WorkflowPhase.IMPLEMENT.value))
    self._progress.current_phase = phase
    await asyncio.sleep(0.1)
    return {"status": "completed", "task": task.name, "phase": phase.value}
async def _retry_task(self, task: SubTask, recovery: RecoveryAction) -> ExecutionResult:
```

```
        task.status = SubTaskStatus.PENDING
        recovery.retry_count += 1
        return await self._execute_single_task(task.id)
    def track_progress(self) -> ProgressInfo:
        if self._start_time > 0:
            self._progress.elapsed_time = time.time() - self._start_time
        total = len(self.context.sub_tasks)
        if total > 0 and self._progress.completed_tasks > 0:
            avg_time_per_task = self._progress.elapsed_time / self._progress.completed_tasks
            remaining_tasks = total - self._progress.completed_tasks
            self._progress.estimated_remaining = avg_time_per_task * remaining_tasks
        self._progress.total_tasks = total
        return self._progress
    def handle_error(self, error: Exception, task: SubTask) -> RecoveryAction:
        error_msg = str(error)
        if "timeout" in error_msg.lower() or "timed out" in error_msg.lower():
            return RecoveryAction(
                strategy=RecoveryStrategy.RETRY,
                message=f"Timeout error, will retry: {error_msg}",
                max_retries=2,
            )
        elif "connection" in error_msg.lower() or "network" in error_msg.lower():
            return RecoveryAction(
                strategy=RecoveryStrategy.RETRY,
                message=f"Network error, will retry: {error_msg}",
                max_retries=3,
            )
        elif "permission" in error_msg.lower() or "access denied" in error_msg.lower():
            return RecoveryAction(
                strategy=RecoveryStrategy.ABORT,
                message=f"Permission error: {error_msg}",
            )
        elif "not found" in error_msg.lower() or "does not exist" in error_msg.lower():
            return RecoveryAction(
                strategy=RecoveryStrategy.ADAPT,
                message=f"Resource not found, will adapt: {error_msg}",
            )
        else:
            return RecoveryAction(
                strategy=RecoveryStrategy.SKIP,
                message=f"Unrecoverable error, skipping task: {error_msg}",
            )
    def validate_result(self, task: SubTask, result: Any) -> bool:
        if result is None:
            return False
        if isinstance(result, dict):
            if result.get("status") == "completed":
                return True
            if result.get("success") is True:
                return True
            if result.get("error"):
                return False
        if isinstance(result, str):
            return len(result) > 0 and "error" not in result.lower()
        if isinstance(result, (list, tuple)):
            return len(result) > 0
        return True
    async def run_full_workflow(self, task: str) -> WorkflowResult:
        workflow_id = uuid4().hex[:12]
        self.context = WorkflowContext(
            session_id=workflow_id,
            original_request=task,
            start_time=time.time(),
        )
        try:
```

```
        task_tree = self.decompose_task(task)
        self.context.sub_tasks = task_tree.subtasks
        graph = self.analyze_dependencies(task_tree.subtasks)
        cycles = graph.detect_cycles()
        if cycles:
            return WorkflowResult(
                workflow_id=workflow_id,
                success=False,
                task_tree=task_tree,
                execution_plan=ExecutionPlan(),
                error=f"Circular dependencies detected: {cycles}",
            )
        plan = self.create_execution_plan(graph)
        exec_result = await self.execute_plan(plan)
        all_results = list(self._task_results.values())
        success = exec_result.success and all(
            self.validate_result(
                next((t for t in task_tree.subtasks if t.id == r.task_id), SubTask()),
                r.output,
            )
            for r in all_results
            if r.success
        )
        self.context.end_time = time.time()
        return WorkflowResult(
            workflow_id=workflow_id,
            success=success,
            task_tree=task_tree,
            execution_plan=plan,
            execution_results=all_results,
            total_duration=self.context.end_time - self.context.start_time,
            metadata={"context": self.context.to_dict()},
        )
    except Exception as e:
        self.context.end_time = time.time()
        return WorkflowResult(
            workflow_id=workflow_id,
            success=False,
            task_tree=TaskTree(root_task=task),
            execution_plan=ExecutionPlan(),
            error=str(e),
            total_duration=time.time() - self.context.start_time,
        )
def get_context(self) -> WorkflowContext:
    return self.context
def reset(self) -> None:
    self.context = WorkflowContext()
    self._progress = ProgressInfo()
    self._task_results.clear()
    self._start_time = 0.0
```

File: [92/252] openlaoke/core/intelligent_model_selector.py

```
"""Intelligent model selector based on available resources.
This module automatically selects optimal models based on:
1. Available GPU memory
2. Task complexity
3. Cost optimization
4. Performance requirements
Selection Strategy:
- < 8 GB VRAM: gemma3:1b (CPU) + qwen2.5:0.5b (GPU)
- 8-16 GB VRAM: gemma3:1b (CPU) + gemma4:e2b (GPU)
- > 16 GB VRAM: gemma3:1b (CPU) + gemma4:e4b (GPU)
"""
from __future__ import annotations
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class ModelCombination:
    """A combination of models for dual-model workflow."""
    name: str
    planner_model: str
    executor_model: str
    validator_model: str
    planner_device: str
    executor_device: str
    validator_device: str
    total_vram_gb: float
    quality_score: float  # 1-10
    cost_factor: float  # relative cost (1.0 = baseline)
    recommended_for: list[str]  # VRAM ranges
class IntelligentModelSelector:
    """Select optimal models based on system resources."""
    MODEL_COMBINATIONS = [
        ModelCombination(
            name="ultra_light",
            planner_model="gemma3:1b",
            executor_model="qwen2.5:0.5b",
            validator_model="gemma3:1b",
            planner_device="cpu",
            executor_device="gpu",
            validator_device="cpu",
            total_vram_gb=0.5,
            quality_score=5.0,
            cost_factor=0.3,
            recommended_for=["< 4 GB"],
        ),
        ModelCombination(
            name="light",
            planner_model="gemma3:1b",
            executor_model="gemma4:e2b",
            validator_model="gemma3:1b",
            planner_device="cpu",
            executor_device="gpu",
            validator_device="cpu",
            total_vram_gb=7.2,
            quality_score=7.0,
            cost_factor=0.5,
            recommended_for=["4-8 GB", "8-12 GB"],
        ),
        ModelCombination(
            name="balanced",
```

```
        planner_model="gemma3:1b",
        executor_model="gemma4:e4b",
        validator_model="gemma3:1b",
        planner_device="cpu",
        executor_device="gpu",
        validator_device="cpu",
        total_vram_gb=12.0,
        quality_score=8.5,
        cost_factor=1.0,
        recommended_for=["12-16 GB", "16-24 GB"],
    ),
    ModelCombination(
        name="high_quality",
        planner_model="gemma4:e2b",
        executor_model="gemma4:e4b",
        validator_model="gemma4:e2b",
        planner_device="cpu",
        executor_device="gpu",
        validator_device="cpu",
        total_vram_gb=12.0,
        quality_score=9.0,
        cost_factor=1.5,
        recommended_for=["> 24 GB"],
    ),
    ModelCombination(
        name="premium",
        planner_model="gemma4:e4b",
        executor_model="gemma4:e12b",
        validator_model="gemma4:e4b",
        planner_device="cpu",
        executor_device="gpu",
        validator_device="cpu",
        total_vram_gb=20.0,
        quality_score=10.0,
        cost_factor=3.0,
        recommended_for=["> 32 GB"],
    ),
]
def __init__(self, app_state: AppState | None = None) -> None:
    self.app_state = app_state
    self._available_vram_gb: float | None = None
async def detect_available_vram(self) -> float:
    """Detect available GPU memory."""
    if self._available_vram_gb is not None:
        return self._available_vram_gb
    import subprocess
    try:
        result = subprocess.run(
            ["system_profiler", "SPDisplaysDataType"],
            capture_output=True,
            text=True,
            timeout=5,
        )
        if result.returncode == 0:
            output = result.stdout
            import re
            vram_matches = re.findall(r"VRAM \(Total\): (\d+) MB", output)
            if vram_matches:
                total_mb = sum(int(m) for m in vram_matches)
                self._available_vram_gb = total_mb / 1024
                return self._available_vram_gb
    except Exception:
        pass
    try:
        result = subprocess.run(
```

```
            ["wmic", "path", "win32_VideoController", "get", "AdapterRAM"],
            capture_output=True,
            text=True,
            timeout=5,
        )
        if result.returncode == 0:
            import re
            vram_matches = re.findall(r"(\d+)", result.stdout)
            if vram_matches:
                total_bytes = sum(int(m) for m in vram_matches if int(m) > 1000000000)
                self._available_vram_gb = total_bytes / (1024**3)
                return self._available_vram_gb
    except Exception:
        pass
    self._available_vram_gb = 8.0
    return self._available_vram_gb
async def select_optimal_combination(self, preference: str = "balanced") -> ModelCombinatio
n:
    """Select optimal model combination based on available resources.
    Args:
        preference: "cost" (optimize for cost), "quality" (optimize for quality),
                    "balanced" (balance cost and quality)
    Returns:
        Optimal ModelCombination
    """
    available_vram = await self.detect_available_vram()
    suitable_combinations = [
        combo
        for combo in self.MODEL_COMBINATIONS
        if combo.total_vram_gb <= available_vram * 0.9
    ]
    if not suitable_combinations:
        return self.MODEL_COMBINATIONS[0]
    if preference == "cost":
        suitable_combinations.sort(key=lambda c: c.cost_factor)
        return suitable_combinations[0]
    elif preference == "quality":
        suitable_combinations.sort(key=lambda c: c.quality_score, reverse=True)
        return suitable_combinations[0]
    else:
        suitable_combinations.sort(key=lambda c: c.quality_score / c.cost_factor, reverse=T
rue)
        return suitable_combinations[0]
async def get_recommendation_report(self) -> dict[str, Any]:
    """Get a detailed recommendation report."""
    available_vram = await self.detect_available_vram()
    optimal = await self.select_optimal_combination()
    return {
        "detected_vram_gb": available_vram,
        "recommended_combination": {
            "name": optimal.name,
            "planner": f"{optimal.planner_model} ({optimal.planner_device})",
            "executor": f"{optimal.executor_model} ({optimal.executor_device})",
            "validator": f"{optimal.validator_model} ({optimal.validator_device})",
        },
        "estimated": {
            "vram_usage_gb": optimal.total_vram_gb,
            "quality_score": optimal.quality_score,
            "cost_factor": optimal.cost_factor,
        },
        "all_suitable_combinations": [
            {
                "name": c.name,
                "vram_gb": c.total_vram_gb,
                "quality": c.quality_score,
```

```
                    "cost": c.cost_factor,
                }
                for c in self.MODEL_COMBINATIONS
                if c.total_vram_gb <= available_vram * 0.9
            ],
        }
async def get_optimal_models(app_state: AppState | None = None) -> dict[str, str]:
    """Convenience function to get optimal models."""
    selector = IntelligentModelSelector(app_state)
    combination = await selector.select_optimal_combination()
    return {
        "planner": combination.planner_model,
        "executor": combination.executor_model,
        "validator": combination.validator_model,
        "planner_device": combination.planner_device,
        "executor_device": combination.executor_device,
        "validator_device": combination.validator_device,
    }
```

File: [93/252] openlaoke/core/intent_parser.py

```
"""Intent parser for programming tasks.
Parses user natural language requests into structured intents that can be
converted to ComponentSpecs for fine-grained task decomposition.
"""
from __future__ import annotations
import re
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
class IntentType(StrEnum):
    WRITE_PROGRAM = "write_program"
    WRITE_FUNCTION = "write_function"
    WRITE_CLASS = "write_class"
    DEBUG_CODE = "debug_code"
    REFACTOR_CODE = "refactor_code"
    TEST_CODE = "test_code"
    ANALYZE_CODE = "analyze_code"
    DOCUMENT_CODE = "document_code"
    UNKNOWN = "unknown"
class ProgrammingLanguage(StrEnum):
    PYTHON = "python"
    JAVASCRIPT = "javascript"
    TYPESCRIPT = "typescript"
    RUST = "rust"
    GO = "go"
    JAVA = "java"
    CPP = "cpp"
    UNKNOWN = "unknown"
class TaskComplexity(StrEnum):
    ATOMIC = "atomic"
    SIMPLE = "simple"
    MODERATE = "moderate"
    COMPLEX = "complex"
@dataclass
class ProgrammingIntent:
    intent_type: IntentType
    language: ProgrammingLanguage = ProgrammingLanguage.PYTHON
    task_name: str = ""
    description: str = ""
    requirements: list[str] = field(default_factory=list)
    inputs: list[str] = field(default_factory=list)
    outputs: list[str] = field(default_factory=list)
    constraints: list[str] = field(default_factory=list)
    complexity: TaskComplexity = TaskComplexity.MODERATE
    confidence: float = 0.0
    context: dict[str, Any] = field(default_factory=dict)
    raw_request: str = ""
    def to_dict(self) -> dict[str, Any]:
        return {
            "intent_type": self.intent_type.value,
            "language": self.language.value,
            "task_name": self.task_name,
            "description": self.description,
            "requirements": self.requirements,
            "inputs": self.inputs,
            "outputs": self.outputs,
            "constraints": self.constraints,
            "complexity": self.complexity.value,
            "confidence": self.confidence,
            "context": self.context,
            "raw_request": self.raw_request,
```

```
        }
class IntentParser:
    INTENT_PATTERNS = {
        IntentType.WRITE_PROGRAM: [
            r"(?:write|create|develop|build|implement)\s+(?:a\s+)?(?:new\s+)?(?:program|app|app
    lication|tool|utility|script)",
            r"(?:make|generate)\s+(?:a\s+)?(?:program|app|application)",
            r"(?:i\s+need|need|want)\s+(?:a\s+)?(?:program|app|application|tool)",
            r"(?:write|create)\s+(?:a\s+)?(?:python|javascript|rust|go|java)\s+(?:program|app|s
    cript)",
            r"?.*??",
            r"??.*??",
            r"??.*??",
            r"?.*python",
            r"???.*??",
            r"??.*??",
        ],
        IntentType.WRITE_FUNCTION: [
            r"(?:write|create|implement|define)\s+(?:a\s+)?(?:new\s+)?(?:function|method)",
            r"(?:add|make)\s+(?:a\s+)?(?:function|method)",
            r"(?:function|method)\s+(?:to|for)\s+",
            r"?.*??",
            r"??.*??",
            r"??.*??",
        ],
        IntentType.WRITE_CLASS: [
            r"(?:write|create|implement|define)\s+(?:a\s+)?(?:new\s+)?(?:class|object|type)",
            r"(?:add|make)\s+(?:a\s+)?(?:class)",
            r"(?:class|object)\s+(?:to|for)\s+",
            r"?.*?",
            r"??.*?",
            r"??.*?",
        ],
        IntentType.DEBUG_CODE: [
            r"(?:debug|fix|solve|troubleshoot)\s+(?:the\s+)?(?:error|bug|issue|problem)",
            r"(?:error|bug|issue)\s+(?:in\s+)?(?:the\s+)?(?:code|program|function)",
            r"(?:something\s+is\s+wrong|not\s+working|broken)",
            r"(?:help\s+me\s+fix|how\s+to\s+fix)",
            r"??|??|??.*??|bug|??",
        ],
        IntentType.REFACTOR_CODE: [
            r"(?:refactor|improve|optimize|clean\s+up|rewrite)\s+(?:the\s+)?(?:code|program|fun
    ction)",
            r"(?:make\s+the\s+code)\s+(?:better|cleaner|more\s+efficient)",
            r"??|??|??|??.*??",
        ],
        IntentType.TEST_CODE: [
            r"(?:write|create|add|make)\s+(?:tests?|test\s+cases?)",
            r"(?:test\s+the|testing\s+the)",
            r"?.*??|??.*??",
        ],
        IntentType.ANALYZE_CODE: [
            r"(?:analyze|review|inspect|examine|check)\s+(?:the\s+)?(?:code|program|function)",
            r"(?:what\s+does\s+this\s+code\s+do)",
            r"??|??|??.*??",
        ],
        IntentType.DOCUMENT_CODE: [
            r"(?:document|add\s+docs|add\s+documentation)\s+(?:the\s+)?(?:code|function|class)"
    ,
            r"(?:write\s+comments|add\s+comments)",
            r"??.*??|??",
        ],
    }
    LANGUAGE_PATTERNS = {
        ProgrammingLanguage.PYTHON: [
```

```
        r"\bpython\b(?:\s+program|\s+script|\s+code)?",
        r"\bpy\b",
        r"\.py\b",
    ],
    ProgrammingLanguage.JAVASCRIPT: [
        r"\bjavascript\b(?:\s+program|\s+script|\s+code)?",
        r"\bjs\b",
        r"\.js\b",
        r"\bnode\.js\b",
    ],
    ProgrammingLanguage.TYPESCRIPT: [
        r"\btypescript\b(?:\s+program|\s+script|\s+code)?",
        r"\bts\b",
        r"\.ts\b",
    ],
    ProgrammingLanguage.RUST: [
        r"\brust\b(?:\s+program|\s+code)?",
        r"\.rs\b",
    ],
    ProgrammingLanguage.GO: [
        r"\bgo\b(?:\s+program|\s+code)?(?:lang)?",
        r"\bgolang\b",
        r"\.go\b",
    ],
    ProgrammingLanguage.JAVA: [
        r"\bjava\b(?:\s+program|\s+code)?",
        r"\.java\b",
    ],
    ProgrammingLanguage.CPP: [
        r"\bc\+\+\b(?:\s+program|\s+code)?",
        r"\bcpp\b",
        r"\.cpp\b",
        r"\.cxx\b",
    ],
}
FEATURE_KEYWORDS = {
    "benchmark": ["performance", "speed", "measure", "time", "ops/sec", "benchmark"],
    "calculator": ["calculate", "compute", "math", "add", "subtract", "multiply", "divide"]
,
    "converter": ["convert", "transform", "translate", "change format"],
    "parser": ["parse", "parse file", "parse format", "extract", "analyze"],
    "generator": ["generate", "create", "produce", "make"],
    "validator": ["validate", "check", "verify", "test"],
    "monitor": ["monitor", "watch", "track", "observe", "status"],
    "logger": ["log", "record", "write log", "logging"],
    "downloader": ["download", "fetch", "retrieve", "get"],
    "uploader": ["upload", "send", "post", "push"],
    "processor": ["process", "handle", "manage", "execute"],
    "manager": ["manage", "manager", "management", "admin", "control"],
    "analyzer": ["analyze", "examine", "study", "inspect"],
    "optimizer": ["optimize", "improve", "enhance", "speed up"],
    "scheduler": ["schedule", "plan", "timing", "cron"],
    "server": ["server", "api", "endpoint", "http", "rest"],
    "client": ["client", "request", "call", "connect"],
    "database": ["database", "db", "sql", "query", "store"],
    "cache": ["cache", "store", "save", "memoize"],
    "queue": ["queue", "buffer", "enqueue", "dequeue"],
    "sort": ["sort", "order", "arrange", "rank"],
    "search": ["search", "find", "lookup", "query"],
    "filter": ["filter", "select", "pick", "where"],
    "map": ["map", "transform", "apply", "convert"],
    "reduce": ["reduce", "aggregate", "combine", "sum"],
    "encrypt": ["encrypt", "secure", "hash", "crypto"],
    "compress": ["compress", "zip", "pack", "reduce size"],
    "image": ["image", "picture", "photo", "visual"],
```

```
    "audio": ["audio", "sound", "music", "wav", "mp3"],
    "video": ["video", "movie", "mp4", "avi"],
    "text": ["text", "string", "word", "sentence"],
    "file": ["file", "read file", "write file", "io"],
    "network": ["network", "http", "socket", "tcp", "udp"],
    "gui": ["gui", "ui", "interface", "window", "button", "tkinter", "qt"],
    "cli": ["cli", "command", "argument", "option", "flag"],
    "web": ["web", "html", "css", "browser", "frontend"],
    "api": ["api", "endpoint", "rest", "graphql", "swagger"],
}
COMPLEXITY_INDICATORS = {
    TaskComplexity.ATOMIC: [
        "simple",
        "basic",
        "minimal",
        "tiny",
        "single function",
    ],
    TaskComplexity.SIMPLE: [
        "small",
        "easy",
        "straightforward",
        "one file",
    ],
    TaskComplexity.MODERATE: [
        "medium",
        "moderate",
        "standard",
        "typical",
        "multiple functions",
    ],
    TaskComplexity.COMPLEX: [
        "complex",
        "advanced",
        "large",
        "full",
        "complete",
        "comprehensive",
        "multiple modules",
        "architecture",
        "system",
    ],
}
def __init__(self) -> None:
    self._cache: dict[str, ProgrammingIntent] = {}
def parse(self, user_request: str) -> ProgrammingIntent:
    user_request = user_request.strip()
    if not user_request:
        return ProgrammingIntent(
            intent_type=IntentType.UNKNOWN,
            confidence=0.0,
            raw_request=user_request,
        )
    if user_request in self._cache:
        return self._cache[user_request]
    intent_type = self._detect_intent_type(user_request)
    language = self._detect_language(user_request)
    task_name = self._extract_task_name(user_request, intent_type)
    description = self._extract_description(user_request)
    requirements = self._extract_requirements(user_request)
    inputs = self._extract_inputs(user_request)
    outputs = self._extract_outputs(user_request)
    constraints = self._extract_constraints(user_request)
    complexity = self._detect_complexity(user_request)
    confidence = self._calculate_confidence(
```

```
        intent_type,
        language,
        task_name,
        user_request,
    )
    intent = ProgrammingIntent(
        intent_type=intent_type,
        language=language,
        task_name=task_name,
        description=description,
        requirements=requirements,
        inputs=inputs,
        outputs=outputs,
        constraints=constraints,
        complexity=complexity,
        confidence=confidence,
        raw_request=user_request,
    )
    self._cache[user_request] = intent
    return intent
def _detect_intent_type(self, request: str) -> IntentType:
    lower_request = request.lower()
    for intent_type, patterns in self.INTENT_PATTERNS.items():
        for pattern in patterns:
            try:
                if re.search(pattern, lower_request, re.IGNORECASE):
                    return intent_type
            except re.error:
                continue
    if any(
        keyword in lower_request
        for keyword in ["write", "create", "build", "implement", "develop", "make"]
    ):
        return IntentType.WRITE_PROGRAM
    return IntentType.UNKNOWN
def _detect_language(self, request: str) -> ProgrammingLanguage:
    lower_request = request.lower()
    for language, patterns in self.LANGUAGE_PATTERNS.items():
        for pattern in patterns:
            if re.search(pattern, lower_request, re.IGNORECASE):
                return language
    return ProgrammingLanguage.PYTHON
def _extract_task_name(self, request: str, intent_type: IntentType) -> str:
    lower_request = request.lower()
    for feature_name, keywords in self.FEATURE_KEYWORDS.items():
        for keyword in keywords:
            if keyword in lower_request:
                return feature_name.replace("_", " ")
    if intent_type == IntentType.WRITE_PROGRAM:
        match = re.search(
            r"(?:program|app|application|tool|utility)\s+(?:called\s+)?['\"]?(\w+)['\"]?",
            lower_request,
            re.IGNORECASE,
        )
        if match:
            return match.group(1)
        match = re.search(
            r"(?:for|to)\s+(\w+)",
            lower_request,
            re.IGNORECASE,
        )
        if match:
            return match.group(1)
    if intent_type in [IntentType.WRITE_FUNCTION, IntentType.WRITE_CLASS]:
        match = re.search(
```

```
            r"(?:function|method|class)\s+(?:called\s+)?['\"]?(\w+)['\"]?",
            lower_request,
            re.IGNORECASE,
        )
        if match:
            return match.group(1)
        match = re.search(
            r"(?:for|to)\s+(\w+)",
            lower_request,
            re.IGNORECASE,
        )
        if match:
            return match.group(1)
    return "unnamed_task"
def _extract_description(self, request: str) -> str:
    description_parts = []
    lower_request = request.lower()
    action_match = re.search(
        r"(?:write|create|develop|build|implement|make|generate)\s+(?:a\s+)?(?:new\s+)?(.+?
)(?:\s+that|\s+to|\s+for|\s+with|\s+in)",
        lower_request,
        re.IGNORECASE,
    )
    if action_match:
        description_parts.append(action_match.group(1).strip())
    purpose_match = re.search(
        r"(?:that|to|for)\s+(.+?)(?:\s+with|\s+in|\s+using|\s*$)",
        lower_request,
        re.IGNORECASE,
    )
    if purpose_match:
        description_parts.append(purpose_match.group(1).strip())
    if description_parts:
        return " ".join(description_parts)
    return request
def _extract_requirements(self, request: str) -> list[str]:
    requirements = []
    lower_request = request.lower()
    capability_patterns = [
        r"(?:should|must|needs\s+to|have\s+to)\s+(.+?)(?:\s+and|\s*$)",
        r"(?:support|handle|process|work\s+with)\s+(.+?)(?:\s+and|\s*$)",
        r"(?:features?|capabilities?):\s*(.+?)(?:\s*$)",
    ]
    for pattern in capability_patterns:
        matches = re.findall(pattern, lower_request, re.IGNORECASE)
        for match in matches:
            requirement = match.strip()
            if requirement and len(requirement) > 3:
                requirements.append(requirement)
    return requirements
def _extract_inputs(self, request: str) -> list[str]:
    inputs = []
    lower_request = request.lower()
    input_patterns = [
        r"input[:\s]+(.+?)(?:\s|$)",
        r"argument[:\s]+(.+?)(?:\s|$)",
        r"parameter[:\s]+(.+?)(?:\s|$)",
        r"take\s+(.+?)\s+as\s+input",
        r"accept\s+(.+?)\s+as\s+(?:input|argument|parameter)",
        r"from\s+(.+?)\s+(?:file|source)",
    ]
    for pattern in input_patterns:
        matches = re.findall(pattern, lower_request, re.IGNORECASE)
        for match in matches:
            input_item = match.strip()
```

```
            if input_item and len(input_item) > 2:
                inputs.append(input_item)
    if "takes" in lower_request or "input" in lower_request:
        words = lower_request.split()
        for i, word in enumerate(words):
            if word in ["takes", "input", "as"] and i + 1 < len(words):
                potential_input = words[i + 1]
                if potential_input not in ["input", "argument", "parameter", "the", "a", "a
n"]:
                    inputs.append(potential_input)
    return inputs
def _extract_outputs(self, request: str) -> list[str]:
    outputs = []
    lower_request = request.lower()
    output_patterns = [
        r"output[:\s]+(.+?)(?:\s|$)",
        r"result[:\s]+(.+?)(?:\s|$)",
        r"return[:\s]+(.+?)(?:\s|$)",
        r"returns\s+(.+?)\s+(?:as\s+)?(?:output|result)",
        r"produce\s+(.+?)\s+(?:as\s+)?(?:output|result)",
        r"generate\s+(.+?)\s+(?:as\s+)?(?:output|result)",
    ]
    for pattern in output_patterns:
        matches = re.findall(pattern, lower_request, re.IGNORECASE)
        for match in matches:
            output_item = match.strip()
            if output_item and len(output_item) > 2:
                outputs.append(output_item)
    if "returns" in lower_request or "output" in lower_request or "result" in lower_request
:
        words = lower_request.split()
        for i, word in enumerate(words):
            if word in ["returns", "output", "result"] and i + 1 < len(words):
                potential_output = words[i + 1]
                if potential_output not in [
                    "output",
                    "result",
                    "the",
                    "a",
                    "an",
                    "calculated",
                    "as",
                ]:
                    outputs.append(potential_output)
    return outputs
def _extract_constraints(self, request: str) -> list[str]:
    constraints = []
    lower_request = request.lower()
    constraint_patterns = [
        r"(?:constraint|limit|requirement):\s*(.+?)(?:\s*$)",
        r"(?:must\s+be|should\s+be|has\s+to\s+be)\s+(.+?)(?:\s*$)",
        r"(?:no\s+|don't\s+|avoid\s+)(.+?)(?:\s*$)",
        r"(?:under|within|less\s+than|maximum)\s+(\d+)\s*(.+?)(?:\s*$)",
    ]
    for pattern in constraint_patterns:
        matches = re.findall(pattern, lower_request, re.IGNORECASE)
        for match in matches:
            constraint = match.strip() if isinstance(match, str) else f"{match[0]} {match[1
]}"
            if constraint and len(constraint) > 2:
                constraints.append(constraint)
    return constraints
def _detect_complexity(self, request: str) -> TaskComplexity:
    lower_request = request.lower()
    for complexity, indicators in self.COMPLEXITY_INDICATORS.items():
```

```
        for indicator in indicators:
            if indicator in lower_request:
                return complexity
    word_count = len(request.split())
    if word_count < 10:
        return TaskComplexity.ATOMIC
    elif word_count < 20:
        return TaskComplexity.SIMPLE
    elif word_count < 40:
        return TaskComplexity.MODERATE
    else:
        return TaskComplexity.COMPLEX
def _calculate_confidence(
    self,
    intent_type: IntentType,
    language: ProgrammingLanguage,
    task_name: str,
    request: str,
) -> float:
    confidence = 0.0
    if intent_type != IntentType.UNKNOWN:
        confidence += 0.4
    if language != ProgrammingLanguage.UNKNOWN:
        confidence += 0.2
    if task_name and task_name != "unnamed_task":
        confidence += 0.2
    word_count = len(request.split())
    if word_count >= 5:
        confidence += 0.1
    if word_count >= 10:
        confidence += 0.1
    return min(confidence, 1.0)
def suggest_clarifications(self, intent: ProgrammingIntent) -> list[str]:
    questions = []
    if intent.confidence < 0.5:
        questions.append("Could you provide more details about what you want to create?")
    if intent.intent_type == IntentType.UNKNOWN:
        questions.append(
            "What type of code do you want me to write (program, function, class)?"
        )
    if not intent.requirements:
        questions.append("What specific features or capabilities should this have?")
    if not intent.inputs and intent.intent_type in [
        IntentType.WRITE_PROGRAM,
        IntentType.WRITE_FUNCTION,
    ]:
        questions.append("What inputs or parameters should this accept?")
    if not intent.outputs and intent.intent_type in [
        IntentType.WRITE_PROGRAM,
        IntentType.WRITE_FUNCTION,
    ]:
        questions.append("What should the output or result be?")
    return questions
```

File: [94/252] openlaoke/core/intent_pipeline.py

```
"""Intent-based task processing pipeline.
Integrates intent parsing, spec conversion, and task decomposition
into a unified workflow for small models.
"""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from typing import Any
from openlaoke.core.architecture.decomposer import (
    AtomicTask,
    FineGrainedDecomposer,
    TaskGraph,
)
from openlaoke.core.architecture.interfaces import ComponentSpec
from openlaoke.core.intent_parser import (
    IntentParser,
    IntentType,
    ProgrammingIntent,
)
from openlaoke.core.intent_to_spec import IntentToSpecConverter
from openlaoke.core.model_assessment.types import ModelTier
@dataclass
class ProcessingPipelineResult:
    success: bool
    intent: ProgrammingIntent
    specs: list[ComponentSpec]
    tasks: list[AtomicTask]
    task_graph: TaskGraph | None
    errors: list[str]
    warnings: list[str]
    ready_to_execute: bool
class IntentBasedPipeline:
    def __init__(
        self,
        model_tier: ModelTier = ModelTier.TIER_5_LIMITED,
        working_directory: Path | None = None,
    ) -> None:
        self.model_tier = model_tier
        self.parser = IntentParser()
        self.converter = IntentToSpecConverter(model_tier)
        self.decomposer = FineGrainedDecomposer(model_tier)
        self.working_directory = working_directory or Path.cwd()
    def process_request(self, user_request: str) -> ProcessingPipelineResult:
        intent = self.parser.parse(user_request)
        if intent.intent_type == IntentType.UNKNOWN:
            self.parser.suggest_clarifications(intent)
            return ProcessingPipelineResult(
                success=False,
                intent=intent,
                specs=[],
                tasks=[],
                task_graph=None,
                errors=["Unable to understand the request"],
                warnings=[],
                ready_to_execute=False,
            )
        spec_result = self.converter.convert(intent)
        if not spec_result.success:
            return ProcessingPipelineResult(
                success=False,
                intent=intent,
```

```
            specs=spec_result.specs,
            tasks=[],
            task_graph=None,
            errors=spec_result.errors,
            warnings=spec_result.warnings,
            ready_to_execute=False,
        )
    all_tasks = []
    for spec in spec_result.specs:
        if spec.component_type.value == "module":
            module_tasks = self.decomposer.decompose_module(spec)
            all_tasks.extend(module_tasks)
        elif spec.component_type.value == "class":
            class_tasks = self.decomposer.decompose_class(spec)
            all_tasks.extend(class_tasks)
        elif spec.component_type.value == "function":
            function_tasks = self.decomposer.decompose_function(spec)
            all_tasks.extend(function_tasks)
        else:
            atomic_task = self._create_atomic_task_from_spec(spec)
            all_tasks.append(atomic_task)
    task_graph = TaskGraph(root_task_id="root")
    for task in all_tasks:
        task_graph.add_task(task)
    ready_tasks = task_graph.get_ready_tasks()
    return ProcessingPipelineResult(
        success=True,
        intent=intent,
        specs=spec_result.specs,
        tasks=all_tasks,
        task_graph=task_graph,
        errors=spec_result.errors,
        warnings=spec_result.warnings,
        ready_to_execute=len(ready_tasks) > 0,
    )
def get_execution_plan(self, result: ProcessingPipelineResult) -> dict[str, Any]:
    if not result.success or not result.task_graph:
        return {}
    plan = {
        "total_tasks": len(result.tasks),
        "ready_tasks": [],
        "pending_tasks": [],
        "estimated_lines_total": sum(t.estimated_lines for t in result.tasks),
    }
    for task in result.tasks:
        task_info = {
            "task_id": task.task_id,
            "description": task.description,
            "estimated_lines": task.estimated_lines,
            "dependencies": task.dependencies,
            "component_type": task.component_spec.component_type.value,
        }
        if task.task_id in result.task_graph.completed:
            continue
        elif all(dep in result.task_graph.completed for dep in task.dependencies):
            plan["ready_tasks"].append(task_info)
        else:
            plan["pending_tasks"].append(task_info)
    return plan
def generate_task_prompt(self, task: AtomicTask, context: dict[str, Any] | None = None) ->
str:
    if context is None:
        context = {}
    prompt_parts = [
        f"Task: {task.description}",
```

```
        f"Estimated complexity: {task.estimated_lines} lines",
        "",
        "Requirements:",
    ]
    for rule in task.validation_rules:
        prompt_parts.append(f"  - {rule}")
    if task.component_spec.api_spec:
        api = task.component_spec.api_spec
        prompt_parts.extend(
            [
                "",
                "Input parameters:",
            ]
        )
        for param_name, param_spec in api.input_schema.get("properties", {}).items():
            param_type = param_spec.get("type", "any")
            param_desc = param_spec.get("description", "")
            prompt_parts.append(f"  - {param_name}: {param_type} ({param_desc})")
        prompt_parts.extend(
            [
                "",
                "Expected output:",
                f"  {api.output_schema.get('type', 'any')}",
            ]
        )
    prompt_parts.extend(
        [
            "",
            "Important constraints:",
            f"  - Maximum {self.decomposer.max_lines_per_task} lines of code",
            f"  - Maximum {self.decomposer.max_params_per_function} parameters",
            "  - MUST be syntactically correct Python",
            "  - MUST have complete type hints",
            "  - MUST have docstring",
            "",
            "Write ONLY this specific function/class. Do NOT write the entire program.",
            "",
            "Template to fill:",
            task.template.template,
        ]
    )
    return "\n".join(prompt_parts)
def validate_task_result(self, task: AtomicTask, code: str) -> tuple[bool, list[str]]:
    import ast
    errors = []
    try:
        ast.parse(code)
    except SyntaxError as e:
        errors.append(f"Syntax error: {e}")
        return False, errors
    lines = code.strip().split("\n")
    max_allowed_lines = max(task.estimated_lines * 3, 20)
    if len(lines) > max_allowed_lines:
        errors.append(f"Code too long: {len(lines)} lines > {max_allowed_lines} allowed")
    non_empty_lines = [
        line for line in lines if line.strip() and not line.strip().startswith("#")
    ]
    if len(non_empty_lines) > 25:
        errors.append(f"Too many non-empty lines: {len(non_empty_lines)} > 25")
    for rule in task.validation_rules:
        if "type hints" in rule.lower() and "def " in code and " -> " not in code:
            pass
        if "docstring" in rule.lower() and '"""' not in code and "'''" not in code:
            pass
    return len(errors) == 0, errors
```

```
    def _create_atomic_task_from_spec(self, spec: ComponentSpec) -> AtomicTask:
        from openlaoke.core.architecture.interfaces import get_template_for_component
        template = get_template_for_component(spec.component_type)
        return AtomicTask(
            task_id=spec.name,
            description=f"Implement {spec.name}",
            component_spec=spec,
            template=template,
            estimated_lines=self.decomposer.max_lines_per_task,
            test_required=True,
            validation_rules=template.validation_rules,
        )
def create_pipeline_for_model(
    model_tier: ModelTier,
    working_directory: Path | None = None,
) -> IntentBasedPipeline:
    return IntentBasedPipeline(model_tier, working_directory)
```

File: [95/252] openlaoke/core/intent_to_spec.py

```
"""Convert programming intent to ComponentSpec for task decomposition.
This module bridges the gap between natural language understanding
and the fine-grained architecture decomposition system.
"""
from __future__ import annotations
from dataclasses import dataclass
from typing import Any
from openlaoke.core.architecture.interfaces import (
    APISpec,
    ComponentSpec,
    ComponentType,
)
from openlaoke.core.intent_parser import (
    IntentType,
    ProgrammingIntent,
    TaskComplexity,
)
from openlaoke.core.model_assessment.types import ModelTier
@dataclass
class SpecGenerationResult:
    success: bool
    specs: list[ComponentSpec]
    errors: list[str]
    warnings: list[str]
    suggestions: list[str]
class IntentToSpecConverter:
    def __init__(self, model_tier: ModelTier = ModelTier.TIER_5_LIMITED) -> None:
        self.model_tier = model_tier
        self.max_lines_per_spec = self._get_max_lines_for_tier(model_tier)
        self.max_params_per_spec = self._get_max_params_for_tier(model_tier)
    def convert(self, intent: ProgrammingIntent) -> SpecGenerationResult:
        if intent.intent_type == IntentType.UNKNOWN:
            return SpecGenerationResult(
                success=False,
                specs=[],
                errors=["Cannot convert unknown intent to spec"],
                warnings=[],
                suggestions=["Please clarify what you want to create"],
            )
        specs = []
        intent_handlers = {
            IntentType.WRITE_PROGRAM: self._convert_program_intent,
            IntentType.WRITE_FUNCTION: self._convert_function_intent,
            IntentType.WRITE_CLASS: self._convert_class_intent,
            IntentType.DEBUG_CODE: self._convert_modification_intent,
            IntentType.REFACTOR_CODE: self._convert_modification_intent,
            IntentType.TEST_CODE: self._convert_modification_intent,
            IntentType.ANALYZE_CODE: self._convert_analysis_intent,
            IntentType.DOCUMENT_CODE: self._convert_analysis_intent,
        }
        handler = intent_handlers.get(intent.intent_type)
        if handler:
            specs = handler(intent)
        errors = []
        warnings = []
        suggestions = []
        for spec in specs:
            if spec.api_spec is None:
                continue
            if len(spec.api_spec.input_schema.get("properties", {})) > self.max_params_per_spec
    :
```

```
            warnings.append(
                f"Spec '{spec.name}' has too many parameters "
                f"({len(spec.api_spec.input_schema.get('properties', {}))} > {self.max_para
ms_per_spec}). "
                f"Will be further decomposed."
            )
        if spec.complexity_score > self.max_lines_per_spec:
            warnings.append(
                f"Spec '{spec.name}' complexity ({spec.complexity_score}) "
                f"exceeds model limit ({self.max_lines_per_spec}). "
                f"Will be decomposed into smaller tasks."
            )
    return SpecGenerationResult(
        success=len(specs) > 0,
        specs=specs,
        errors=errors,
        warnings=warnings,
        suggestions=suggestions,
    )
def _convert_program_intent(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    specs = []
    module_spec = self._create_module_spec(intent)
    main_component_spec = self._create_main_component_spec(intent)
    helper_specs = self._create_helper_specs(intent)
    specs.append(module_spec)
    specs.append(main_component_spec)
    specs.extend(helper_specs)
    test_spec = self._create_test_spec(intent, specs)
    specs.append(test_spec)
    return specs
def _convert_function_intent(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    specs = []
    function_spec = self._create_function_spec(intent)
    helper_specs = self._create_helper_specs_for_function(intent, function_spec)
    specs.append(function_spec)
    specs.extend(helper_specs)
    test_spec = self._create_test_spec(intent, specs)
    specs.append(test_spec)
    return specs
def _convert_class_intent(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    specs = []
    class_spec = self._create_class_spec(intent)
    method_specs = self._create_method_specs(intent, class_spec)
    specs.append(class_spec)
    specs.extend(method_specs)
    test_spec = self._create_test_spec(intent, specs)
    specs.append(test_spec)
    return specs
def _convert_modification_intent(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    specs = []
    analysis_spec = self._create_analysis_spec(intent)
    modification_spec = self._create_modification_spec(intent)
    validation_spec = self._create_validation_spec(intent)
    specs.append(analysis_spec)
    specs.append(modification_spec)
    specs.append(validation_spec)
    return specs
def _convert_analysis_intent(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    specs = []
    analysis_spec = self._create_analysis_spec(intent)
    report_spec = self._create_report_spec(intent)
    specs.append(analysis_spec)
    specs.append(report_spec)
    return specs
def _create_module_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
```

```
    module_name = intent.task_name.replace(" ", "_").replace("-", "_")
    return ComponentSpec(
        name=module_name,
        component_type=ComponentType.MODULE,
        api_spec=APISpec(
            name=module_name,
            description=f"Module for {intent.description}",
            input_schema={"type": "object", "properties": {}},
            output_schema={"type": "object", "properties": {}},
        ),
        test_requirements=["Module structure validation", "Import tests"],
        complexity_score=self._estimate_module_complexity(intent),
    )
def _create_main_component_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    component_name = f"{intent.task_name.replace(' ', '_')}_main"
    input_schema = self._build_input_schema(intent)
    output_schema = self._build_output_schema(intent)
    component_type = self._determine_component_type(intent)
    complexity = self._estimate_component_complexity(intent)
    return ComponentSpec(
        name=component_name,
        component_type=component_type,
        api_spec=APISpec(
            name=component_name,
            description=f"Main {intent.task_name} component",
            input_schema=input_schema,
            output_schema=output_schema,
        ),
        test_requirements=[
            "Basic functionality test",
            "Edge case test",
            "Error handling test",
        ],
        complexity_score=complexity,
    )
def _create_helper_specs(self, intent: ProgrammingIntent) -> list[ComponentSpec]:
    helper_specs = []
    for requirement in intent.requirements:
        if self._should_create_helper_for_requirement(requirement):
            helper_name = self._extract_helper_name_from_requirement(requirement)
            helper_spec = ComponentSpec(
                name=helper_name,
                component_type=ComponentType.FUNCTION,
                api_spec=APISpec(
                    name=helper_name,
                    description=f"Helper function for {requirement}",
                    input_schema={"type": "object", "properties": {}},
                    output_schema={"type": "object", "properties": {}},
                ),
                test_requirements=["Helper test"],
                complexity_score=3,
            )
            helper_specs.append(helper_spec)
    if intent.inputs:
        input_handler_spec = ComponentSpec(
            name="handle_input",
            component_type=ComponentType.FUNCTION,
            api_spec=APISpec(
                name="handle_input",
                description="Process and validate input",
                input_schema=self._build_input_schema(intent),
                output_schema={"type": "object", "properties": {}},
            ),
            test_requirements=["Input validation test"],
            complexity_score=2,
```

```
        )
        helper_specs.append(input_handler_spec)
    if intent.outputs:
        output_formatter_spec = ComponentSpec(
            name="format_output",
            component_type=ComponentType.FUNCTION,
            api_spec=APISpec(
                name="format_output",
                description="Format and prepare output",
                input_schema={"type": "object", "properties": {}},
                output_schema=self._build_output_schema(intent),
            ),
            test_requirements=["Output formatting test"],
            complexity_score=2,
        )
        helper_specs.append(output_formatter_spec)
    return helper_specs
def _create_function_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    function_name = intent.task_name.replace(" ", "_").replace("-", "_")
    input_schema = self._build_input_schema(intent)
    output_schema = self._build_output_schema(intent)
    complexity = self._estimate_function_complexity(intent)
    return ComponentSpec(
        name=function_name,
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name=function_name,
            description=intent.description,
            input_schema=input_schema,
            output_schema=output_schema,
        ),
        test_requirements=[
            "Function correctness test",
            "Input validation test",
            "Output verification test",
        ],
        complexity_score=complexity,
    )
def _create_helper_specs_for_function(
    self, intent: ProgrammingIntent, main_spec: ComponentSpec
) -> list[ComponentSpec]:
    helper_specs = []
    if main_spec.api_spec and len(main_spec.api_spec.input_schema.get("properties", {})) >
2:
        input_params = main_spec.api_spec.input_schema.get("properties", {})
        param_groups = self._split_params_into_groups(input_params)
        for i, group in enumerate(param_groups):
            helper_spec = ComponentSpec(
                name=f"{main_spec.name}_validate_params_{i}",
                component_type=ComponentType.FUNCTION,
                api_spec=APISpec(
                    name=f"{main_spec.name}_validate_params_{i}",
                    description=f"Validate parameter group {i}",
                    input_schema={"type": "object", "properties": group},
                    output_schema={"type": "object", "properties": {}},
                ),
                test_requirements=["Validation test"],
                complexity_score=2,
                dependencies=[main_spec.name],
            )
            helper_specs.append(helper_spec)
    return helper_specs
def _create_class_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    class_name = intent.task_name.replace(" ", "_").replace("-", "_").capitalize()
    complexity = self._estimate_class_complexity(intent)
```

```
    return ComponentSpec(
        name=class_name,
        component_type=ComponentType.CLASS,
        api_spec=APISpec(
            name=class_name,
            description=f"Class for {intent.description}",
            input_schema=self._build_input_schema(intent),
            output_schema=self._build_output_schema(intent),
        ),
        test_requirements=[
            "Class initialization test",
            "Method functionality test",
            "State management test",
        ],
        complexity_score=complexity,
    )
def _create_method_specs(
    self, intent: ProgrammingIntent, class_spec: ComponentSpec
) -> list[ComponentSpec]:
    method_specs = []
    method_names = ["__init__", "process", "validate", "execute", "cleanup"]
    for method_name in method_names[: self.max_params_per_spec]:
        method_spec = ComponentSpec(
            name=f"{class_spec.name}_{method_name}",
            component_type=ComponentType.FUNCTION,
            api_spec=APISpec(
                name=method_name,
                description=f"{method_name} method for {class_spec.name}",
                input_schema={"type": "object", "properties": {}},
                output_schema={"type": "object", "properties": {}},
            ),
            test_requirements=["Method test"],
            complexity_score=2,
            dependencies=[class_spec.name],
        )
        method_specs.append(method_spec)
    return method_specs
def _create_test_spec(
    self, intent: ProgrammingIntent, component_specs: list[ComponentSpec]
) -> ComponentSpec:
    test_name = f"test_{intent.task_name.replace(' ', '_')}"
    return ComponentSpec(
        name=test_name,
        component_type=ComponentType.TEST,
        test_requirements=[],
        complexity_score=1,
        dependencies=[
            spec.name for spec in component_specs if spec.component_type != ComponentType.T
EST
        ],
    )
def _create_analysis_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    return ComponentSpec(
        name=f"analyze_{intent.task_name.replace(' ', '_')}",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name="analyze",
            description=f"Analyze code for {intent.task_name}",
            input_schema={"type": "object", "properties": {}},
            output_schema={"type": "object", "properties": {"issues": {"type": "array"}}},
        ),
        test_requirements=["Analysis test"],
        complexity_score=3,
    )
def _create_modification_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
```

```
    return ComponentSpec(
        name=f"modify_{intent.task_name.replace(' ', '_')}",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name="modify",
            description=f"Modify code for {intent.task_name}",
            input_schema={"type": "object", "properties": {}},
            output_schema={
                "type": "object",
                "properties": {"modified_code": {"type": "string"}},
            },
        ),
        test_requirements=["Modification test"],
        complexity_score=4,
    )
def _create_validation_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    return ComponentSpec(
        name=f"validate_{intent.task_name.replace(' ', '_')}",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name="validate",
            description=f"Validate modifications for {intent.task_name}",
            input_schema={"type": "object", "properties": {}},
            output_schema={"type": "object", "properties": {"is_valid": {"type": "boolean"}
}},
        ),
        test_requirements=["Validation test"],
        complexity_score=2,
    )
def _create_report_spec(self, intent: ProgrammingIntent) -> ComponentSpec:
    return ComponentSpec(
        name=f"report_{intent.task_name.replace(' ', '_')}",
        component_type=ComponentType.FUNCTION,
        api_spec=APISpec(
            name="report",
            description=f"Generate report for {intent.task_name}",
            input_schema={"type": "object", "properties": {}},
            output_schema={"type": "object", "properties": {"report": {"type": "string"}}},
        ),
        test_requirements=["Report generation test"],
        complexity_score=2,
    )
def _build_input_schema(self, intent: ProgrammingIntent) -> dict[str, Any]:
    properties = {}
    for input_item in intent.inputs[: self.max_params_per_spec]:
        input_name = input_item.replace(" ", "_").lower()
        properties[input_name] = {
            "type": self._infer_type_from_description(input_item),
            "description": input_item,
        }
    if not properties:
        properties["data"] = {
            "type": "any",
            "description": "Input data",
        }
    return {"type": "object", "properties": properties}
def _build_output_schema(self, intent: ProgrammingIntent) -> dict[str, Any]:
    properties = {}
    for output_item in intent.outputs[: self.max_params_per_spec]:
        output_name = output_item.replace(" ", "_").lower()
        properties[output_name] = {
            "type": self._infer_type_from_description(output_item),
            "description": output_item,
        }
    if not properties:
```

```
        properties["result"] = {
            "type": "any",
            "description": "Result",
        }
    return {"type": "object", "properties": properties}
def _infer_type_from_description(self, description: str) -> str:
    lower_desc = description.lower()
    if any(
        word in lower_desc for word in ["number", "int", "float", "count", "amount", "size"
]
    ):
        return "number"
    if any(word in lower_desc for word in ["string", "text", "name", "path", "file", "url"]
):
        return "string"
    if any(word in lower_desc for word in ["boolean", "bool", "flag", "enabled", "is"]):
        return "boolean"
    if any(word in lower_desc for word in ["list", "array", "items", "collection"]):
        return "array"
    if any(word in lower_desc for word in ["dict", "map", "object", "config"]):
        return "object"
    return "any"
def _determine_component_type(self, intent: ProgrammingIntent) -> ComponentType:
    if (
        intent.complexity == TaskComplexity.COMPLEX
        or intent.complexity == TaskComplexity.MODERATE
    ):
        return ComponentType.CLASS
    else:
        return ComponentType.FUNCTION
def _estimate_module_complexity(self, intent: ProgrammingIntent) -> int:
    base_complexity = 2
    base_complexity += len(intent.requirements)
    base_complexity += len(intent.inputs)
    base_complexity += len(intent.outputs)
    if intent.complexity == TaskComplexity.COMPLEX:
        base_complexity += 5
    elif intent.complexity == TaskComplexity.MODERATE:
        base_complexity += 3
    return min(base_complexity, self.max_lines_per_spec)
def _estimate_component_complexity(self, intent: ProgrammingIntent) -> int:
    base_complexity = 5
    base_complexity += len(intent.requirements) * 2
    base_complexity += len(intent.inputs)
    base_complexity += len(intent.outputs)
    if intent.complexity == TaskComplexity.COMPLEX:
        base_complexity += 5
    elif intent.complexity == TaskComplexity.MODERATE:
        base_complexity += 3
    return min(base_complexity, self.max_lines_per_spec)
def _estimate_function_complexity(self, intent: ProgrammingIntent) -> int:
    base_complexity = 4
    base_complexity += len(intent.requirements)
    if intent.complexity == TaskComplexity.COMPLEX:
        base_complexity += 3
    elif intent.complexity == TaskComplexity.MODERATE:
        base_complexity += 2
    return min(base_complexity, self.max_lines_per_spec)
def _estimate_class_complexity(self, intent: ProgrammingIntent) -> int:
    base_complexity = 6
    base_complexity += len(intent.requirements) * 2
    base_complexity += len(intent.inputs)
    base_complexity += len(intent.outputs)
    if intent.complexity == TaskComplexity.COMPLEX:
        base_complexity += 5
```

```
        elif intent.complexity == TaskComplexity.MODERATE:
            base_complexity += 3
        return min(base_complexity, self.max_lines_per_spec)
    def _should_create_helper_for_requirement(self, requirement: str) -> bool:
        complex_keywords = [
            "validate",
            "check",
            "parse",
            "format",
            "convert",
            "calculate",
            "compute",
            "process",
            "handle",
            "manage",
        ]
        return any(keyword in requirement.lower() for keyword in complex_keywords)
    def _extract_helper_name_from_requirement(self, requirement: str) -> str:
        words = requirement.lower().split()
        if words[0] in ["validate", "check", "parse", "format", "convert", "calculate"]:
            return f"{words[0]}_{words[1] if len(words) > 1 else 'data'}"
        if "and" in requirement:
            return "handle_multiple"
        return "helper"
    def _split_params_into_groups(self, params: dict[str, Any]) -> list[dict[str, Any]]:
        groups = []
        current_group = {}
        for param_name, param_spec in params.items():
            current_group[param_name] = param_spec
            if len(current_group) >= self.max_params_per_spec:
                groups.append(current_group)
                current_group = {}
        if current_group:
            groups.append(current_group)
        return groups
    def _get_max_lines_for_tier(self, tier: ModelTier) -> int:
        limits = {
            ModelTier.TIER_1_ADVANCED: 100,
            ModelTier.TIER_2_CAPABLE: 50,
            ModelTier.TIER_3_MODERATE: 30,
            ModelTier.TIER_4_BASIC: 20,
            ModelTier.TIER_5_LIMITED: 15,
        }
        return limits[tier]
    def _get_max_params_for_tier(self, tier: ModelTier) -> int:
        limits = {
            ModelTier.TIER_1_ADVANCED: 10,
            ModelTier.TIER_2_CAPABLE: 7,
            ModelTier.TIER_3_MODERATE: 5,
            ModelTier.TIER_4_BASIC: 4,
            ModelTier.TIER_5_LIMITED: 3,
        }
        return limits[tier]
def create_converter_for_model(model_tier: ModelTier) -> IntentToSpecConverter:
    return IntentToSpecConverter(model_tier)
```

File: [96/252] openlaoke/core/interactive_input.py

```
"""
REPL with interactive autocomplete
?? OpenCode ??????
- ?? / ????/????
- ?? @ ??????
- Tab ????
- Enter ????
- Esc ??
- ????????
"""
from __future__ import annotations
import os
import sys
import termios
import tty
from rich.console import Console
from openlaoke.core.autocomplete import get_autocomplete_manager
class InteractiveInput:
    """????????????"""
    def __init__(self, console: Console):
        self.console = console
        self.manager = get_autocomplete_manager()
        self._buffer = ""
        self._cursor = 0
        self._prompt = ""
        self._history: list[str] = []
        self._history_index = -1
    def read_line(self, prompt: str = "") -> str:
        """?????????????"""
        self._buffer = ""
        self._cursor = 0
        self._prompt = prompt
        self.manager.state.reset()
        # ?????
        self.console.print(prompt, end="")
        # ?????????
        fd = sys.stdin.fileno()
        old_settings = termios.tcgetattr(fd)
        try:
            tty.setraw(fd)
            while True:
                ch = sys.stdin.read(1)
                # ??????
                if ch == "\x1b":  # ESC ??
                    # ????????
                    ch2 = sys.stdin.read(1)
                    if ch2 == "[":
                        ch3 = sys.stdin.read(1)
                        if ch3 == "A":  # ???
                            self._history_up()
                        elif ch3 == "B":  # ???
                            self._history_down()
                        elif ch3 == "C":  # ???
                            self._cursor_right()
                        elif ch3 == "D":  # ???
                            self._cursor_left()
                        elif ch3 == "3":  # Delete
                            sys.stdin.read(1)  # ?? ~
                            self._delete()
                    else:
                        # ESC ? - ??????
```

```
                    if self.manager.state.visible:
                        self.manager.state.reset()
                        self._refresh_line()
            elif ch == "\r" or ch == "\n":  # Enter
                if self.manager.state.visible:
                    # ???????
                    self._select_completion()
                else:
                    # ????
                    self.console.print()
                    break
            elif ch == "\t":  # Tab
                self._handle_tab()
            elif ch == "\x7f" or ch == "\x08":  # Backspace
                self._backspace()
            elif ch == "\x03":  # Ctrl+C
                self.console.print("^C")
                raise KeyboardInterrupt()
            elif ch == "\x04":  # Ctrl+D
                raise EOFError()
            elif ch.isprintable():
                self._insert_char(ch)
            # ????
            self._update_completion()
            # ????
            self._refresh_line()
    finally:
        termios.tcsetattr(fd, termios.TCSADRAIN, old_settings)
    # ?????
    if self._buffer.strip():
        self._history.append(self._buffer)
    return self._buffer
def _refresh_line(self):
    """???????"""
    # ?????
    self.console.print("\r\033[K", end="")
    # ?????????
    self.console.print(f"{self._prompt}{self._buffer}", end="")
    # ?????????
    if self._cursor < len(self._buffer):
        # ?????????
        move_left = len(self._buffer) - self._cursor
        self.console.print(f"\033[{move_left}D", end="")
    # ??????
    if self.manager.state.visible and self.manager.state.options:
        self._show_completion_menu()
    sys.stdout.flush()
def _show_completion_menu(self):
    """??????"""
    self.console.print("\n\033[K", end="")
    options = self.manager.state.options
    selected = self.manager.state.selected
    # ????10???
    for i, opt in enumerate(options[:10]):
        if i == selected:
            line = f"  -> [bold cyan]{opt.display}[/bold cyan]"
        else:
            line = f"    {opt.display}"
        if opt.description:
            line += f" [dim]- {opt.description[:40]}[/dim]"
        self.console.print(f"\033[K{line}")
    # ????????
    lines_up = min(len(options), 10) + 1
    self.console.print(f"\033[{lines_up}A\r", end="")
    # ???????
```

```
        self.console.print(f"{self._prompt}{self._buffer}", end="")
        if self._cursor < len(self._buffer):
            move_left = len(self._buffer) - self._cursor
            self.console.print(f"\033[{move_left}D", end="")
    def _insert_char(self, ch: str):
        """????"""
        self._buffer = self._buffer[: self._cursor] + ch + self._buffer[self._cursor :]
        self._cursor += 1
    def _backspace(self):
        """???????"""
        if self._cursor > 0:
            self._buffer = self._buffer[: self._cursor - 1] + self._buffer[self._cursor :]
            self._cursor -= 1
    def _delete(self):
        """??????"""
        if self._cursor < len(self._buffer):
            self._buffer = self._buffer[: self._cursor] + self._buffer[self._cursor + 1 :]
    def _cursor_left(self):
        """????"""
        if self._cursor > 0:
            self._cursor -= 1
    def _cursor_right(self):
        """????"""
        if self._cursor < len(self._buffer):
            self._cursor += 1
    def _handle_tab(self):
        """?? Tab ?"""
        if self.manager.state.visible:
            # ???????
            self.manager.move_selection(1)
        elif self._buffer.startswith("/"):
            # ????
            self.manager.start_completion("/", self._cursor, self._buffer)
    def _update_completion(self):
        """??????"""
        if self.manager.state.visible:
            self.manager.update_search(self._cursor, self._buffer)
        elif self._buffer.startswith("/") and " " not in self._buffer:
            # ??????
            self.manager.start_completion("/", self._cursor, self._buffer)
    def _select_completion(self):
        """?????"""
        text = self.manager.get_completion_text()
        if text:
            # ??????
            self._buffer = "/" + text + " "
            self._cursor = len(self._buffer)
        self.manager.state.reset()
    def _history_up(self):
        """??????"""
        if self._history and self._history_index < len(self._history) - 1:
            self._history_index += 1
            self._buffer = self._history[-(self._history_index + 1)]
            self._cursor = len(self._buffer)
    def _history_down(self):
        """??????"""
        if self._history_index > 0:
            self._history_index -= 1
            self._buffer = self._history[-(self._history_index + 1)]
            self._cursor = len(self._buffer)
        elif self._history_index == 0:
            self._history_index = -1
            self._buffer = ""
            self._cursor = 0
def simple_input_with_completion(prompt: str, console: Console) -> str:
```

```
"""??????????? readline?"""
from openlaoke.core.autocomplete import setup_readline_completion
manager = setup_readline_completion()
# ????????
manager._cwd = os.getcwd()
try:
    result = input(prompt)
    return result
except EOFError:
    return ""
```

File: [97/252] openlaoke/core/knowledge_base.py

```
"""Knowledge base system for small models.
Inspired by GangDan project - retrieve knowledge from official docs
to help small models generate better code.
"""
from __future__ import annotations
import re
from dataclasses import dataclass, field
from pathlib import Path
@dataclass
class KnowledgeSnippet:
    topic: str
    content: str
    source: str
    relevance_score: float = 1.0
    tags: list[str] = field(default_factory=list)
class KnowledgeBase:
    """Local knowledge base for small models."""
    BUILTIN_KNOWLEDGE = {
        "cpu_benchmark": {
            "topic": "CPU Benchmark Implementation",
            "content": """
To measure real CPU performance:
1. Single-core benchmark:
   - Use math operations (sin, cos, sqrt, log)
   - Count operations per second
   - Calculate GFLOPS (Giga Floating-point Operations Per Second)
2. Multi-core benchmark:
   - Use multiprocessing.Pool or ProcessPoolExecutor
   - Distribute work across all cores
   - Measure parallel efficiency
Example operations that stress CPU:
```python
import math
result = math.sin(x) * math.cos(x) + math.sqrt(abs(x))
```
Key metrics:
- Operations/second (higher is better)
- GFLOPS = (ops/sec) / 1e9
- Parallelism factor = multi_core_score / single_core_score
Python libraries to use:
- `time.perf_counter()` for accurate timing
- `multiprocessing.cpu_count()` to get core count
- `concurrent.futures.ProcessPoolExecutor` for parallel execution
Important: Use real calculations, NOT estimates!
""",
            "source": "Python Official Documentation",
            "tags": ["cpu", "benchmark", "performance", "multiprocessing"],
        },
        "file_operations": {
            "topic": "File Operations Best Practices",
            "content": """
Python file operations best practices:
1. Always use context manager (with statement):
```python
with open('file.txt', 'w') as f:
    f.write(content)
```
2. Check file existence:
```python
from pathlib import Path
if Path('file.txt').exists():
```
```

```
    # file exists
```
3. Create directories safely:
```python
Path('output').mkdir(parents=True, exist_ok=True)
```
4. Read/write JSON:
```python
import json
with open('data.json', 'w') as f:
    json.dump(data, f, indent=2)
```
Common errors to avoid:
- SyntaxError: Check for missing parentheses, quotes
- FileNotFoundError: Use Path.exists() first
- PermissionError: Check write permissions
""",
            "source": "Python Best Practices",
            "tags": ["file", "io", "path", "json"],
        },
        "python_syntax": {
            "topic": "Python Syntax Common Errors",
            "content": """
Common Python syntax errors and fixes:
1. Missing closing parenthesis:
```python
# Wrong
result = int(abc
# Correct
result = int(abc)
```
2. IndentationError:
- Use 4 spaces (not tabs)
- Be consistent with indentation level
3. Invalid syntax on function definition:
```python
# Wrong
def function(:
    pass
# Correct
def function():
    pass
```
4. Variable assignment in return:
```python
# Wrong
return x = 5
# Correct
x = 5
return x
```
5. Mixing tabs and spaces:
- Configure editor to show whitespace
- Use auto-formatting tools (black, ruff)
Debug tips:
- Check the line BEFORE the error
- Look for missing colons, brackets, parentheses
- Use `python -m py_compile file.py` to check syntax
""",
            "source": "Python Debugging Guide",
            "tags": ["syntax", "error", "debug", "python"],
        },
    }
    def __init__(self, cache_dir: Path | None = None):
        self.cache_dir = cache_dir or Path.home() / ".openlaoke" / "knowledge"

```
        self.cache_dir.mkdir(parents=True, exist_ok=True)
        self.knowledge_cache: dict[str, KnowledgeSnippet] = {}
        self._load_builtin_knowledge()
    def _load_builtin_knowledge(self) -> None:
        """Load built-in knowledge snippets."""
        for key, data in self.BUILTIN_KNOWLEDGE.items():
            self.knowledge_cache[key] = KnowledgeSnippet(
                topic=data["topic"],
                content=data["content"],
                source=data["source"],
                tags=data.get("tags", []),
            )
    def search(self, query: str, max_results: int = 3) -> list[KnowledgeSnippet]:
        """Search knowledge base for relevant information."""
        query_lower = query.lower()
        query_words = set(re.findall(r"\w+", query_lower))
        scored: list[tuple[float, KnowledgeSnippet]] = []
        for snippet in self.knowledge_cache.values():
            score = 0.0
            if query_lower in snippet.topic.lower():
                score += 3.0
            for word in query_words:
                if word in snippet.content.lower():
                    score += 1.0
            for tag in snippet.tags:
                if tag in query_words:
                    score += 2.0
            if score > 0:
                scored.append((score, snippet))
        scored.sort(key=lambda x: x[0], reverse=True)
        return [s for _, s in scored[:max_results]]
    def enhance_prompt(self, user_request: str, base_prompt: str) -> str:
        """Enhance system prompt with relevant knowledge."""
        relevant = self.search(user_request)
        if not relevant:
            return base_prompt
        knowledge_sections = ["\n? RELEVANT KNOWLEDGE BASE:\n"]
        for i, snippet in enumerate(relevant, 1):
            knowledge_sections.append(f"\n--- Knowledge {i}: {snippet.topic} ---\n")
            knowledge_sections.append(f"Source: {snippet.source}\n")
            knowledge_sections.append(snippet.content)
            knowledge_sections.append("\n")
        knowledge_sections.append(
            "\nUse this knowledge to generate ACCURATE code. Double-check syntax before writing
    !\n"
        )
        return base_prompt + "\n".join(knowledge_sections)
def create_knowledge_base(cache_dir: Path | None = None) -> KnowledgeBase:
    """Create a knowledge base instance."""
    return KnowledgeBase(cache_dir)
```

File: [98/252] openlaoke/core/memory/__init__.py

```
"""Long-term memory and knowledge graph system for OpenLaoKe.
This module provides cross-session memory storage and retrieval:
- Long-term memory with semantic search
- Knowledge graph for code and concepts
- Vector embeddings for similarity search
- Memory consolidation and optimization
"""
from openlaoke.core.memory.consolidation import (
    ConsolidationStrategy,
    MemoryConsolidator,
)
from openlaoke.core.memory.embedding import (
    EmbeddingEngine,
    VectorIndex,
)
from openlaoke.core.memory.knowledge_graph import (
    Fact,
    GraphVisualization,
    Inference,
    KnowledgeEdge,
    KnowledgeGraph,
    KnowledgeNode,
)
from openlaoke.core.memory.memory import (
    LongTermMemory,
    Memory,
    MemorySummary,
    get_memory,
)
from openlaoke.core.memory.retrieval import (
    MemoryRetriever,
    SearchResult,
)
__all__ = [
    "LongTermMemory",
    "Memory",
    "MemorySummary",
    "get_memory",
    "KnowledgeGraph",
    "KnowledgeNode",
    "KnowledgeEdge",
    "Fact",
    "Inference",
    "GraphVisualization",
    "EmbeddingEngine",
    "VectorIndex",
    "MemoryRetriever",
    "SearchResult",
    "MemoryConsolidator",
    "ConsolidationStrategy",
]
```

File: [99/252] openlaoke/core/memory/consolidation.py

```
"""Memory consolidation system for optimizing storage."""
from __future__ import annotations
import enum
import logging
import time
from dataclasses import dataclass
from typing import TYPE_CHECKING
from openlaoke.core.memory.embedding import EmbeddingEngine
if TYPE_CHECKING:
    from openlaoke.core.memory.memory import Memory, MemoryStorage
logger = logging.getLogger(__name__)
class ConsolidationStrategy(enum.Enum):
    """Strategy for memory consolidation."""
    IMPORTANCE = "importance"
    RECENCY = "recency"
    HYBRID = "hybrid"
    ASSOCIATION = "association"
@dataclass
class ConsolidationResult:
    """Result of memory consolidation."""
    memories_removed: int
    memories_merged: int
    associations_created: int
    space_saved: int
class MemoryConsolidator:
    """Consolidates and optimizes memory storage."""
    def __init__(
        self,
        storage: MemoryStorage,
        embedder: EmbeddingEngine,
        max_memories: int = 1000,
        similarity_threshold: float = 0.95,
    ):
        self.storage = storage
        self.embedder = embedder
        self.max_memories = max_memories
        self.similarity_threshold = similarity_threshold
    async def consolidate(
        self,
        strategy: ConsolidationStrategy = ConsolidationStrategy.HYBRID,
    ) -> ConsolidationResult:
        memories = self.storage.get_all()
        if len(memories) <= self.max_memories:
            return ConsolidationResult(
                memories_removed=0,
                memories_merged=0,
                associations_created=0,
                space_saved=0,
            )
        to_remove = await self._select_memories_for_removal(memories, strategy)
        to_merge = await self._find_merge_candidates(memories)
        memories_removed = 0
        for memory in to_remove:
            if self.storage.delete(memory.id):
                memories_removed += 1
        memories_merged = 0
        associations_created = 0
        for group in to_merge:
            if len(group) > 1:
                merged = await self._merge_memories(group)
                if merged:
```

```
                memories_merged += len(group) - 1
                associations_created += len(group) - 1
    space_saved = memories_removed + memories_merged
    return ConsolidationResult(
        memories_removed=memories_removed,
        memories_merged=memories_merged,
        associations_created=associations_created,
        space_saved=space_saved,
    )
async def _select_memories_for_removal(
    self,
    memories: list[Memory],
    strategy: ConsolidationStrategy,
) -> list[Memory]:
    if len(memories) <= self.max_memories:
        return []
    num_to_remove = len(memories) - self.max_memories
    if strategy == ConsolidationStrategy.IMPORTANCE:
        sorted_memories = sorted(memories, key=lambda m: m.importance)
    elif strategy == ConsolidationStrategy.RECENCY:
        sorted_memories = sorted(memories, key=lambda m: m.accessed_at)
    elif strategy == ConsolidationStrategy.HYBRID:
        sorted_memories = sorted(
            memories,
            key=lambda m: m.importance * 0.5 + self._recency_score(m) * 0.5,
        )
    elif strategy == ConsolidationStrategy.ASSOCIATION:
        sorted_memories = sorted(memories, key=lambda m: len(m.associations))
    else:
        sorted_memories = sorted(memories, key=lambda m: m.importance)
    return sorted_memories[:num_to_remove]
async def _find_merge_candidates(self, memories: list[Memory]) -> list[list[Memory]]:
    candidates: list[list[Memory]] = []
    processed: set[str] = set()
    for i, memory1 in enumerate(memories):
        if memory1.id in processed:
            continue
        if not memory1.embedding:
            continue
        similar_group = [memory1]
        for j, memory2 in enumerate(memories):
            if i != j and memory2.id not in processed and memory2.embedding:
                similarity = EmbeddingEngine.cosine_similarity(
                    memory1.embedding,
                    memory2.embedding,
                )
                if similarity >= self.similarity_threshold:
                    similar_group.append(memory2)
                    processed.add(memory2.id)
        if len(similar_group) > 1:
            candidates.append(similar_group)
            processed.add(memory1.id)
    return candidates
async def _merge_memories(self, memories: list[Memory]) -> Memory | None:
    if not memories:
        return None
    best_memory = max(memories, key=lambda m: m.importance)
    for memory in memories:
        if memory.id != best_memory.id:
            for assoc in memory.associations:
                if assoc not in best_memory.associations:
                    best_memory.associations.append(assoc)
            best_memory.importance = max(best_memory.importance, memory.importance)
            best_memory.access_count += memory.access_count
    self.storage.update(best_memory)
```

```
    return best_memory
def _recency_score(self, memory: Memory) -> float:
    age = time.time() - memory.accessed_at
    days = age / 86400.0
    if days < 1:
        return 1.0
    elif days < 7:
        return 0.8
    elif days < 30:
        return 0.6
    elif days < 90:
        return 0.4
    else:
        return 0.2
async def optimize_associations(self, max_associations: int = 10) -> int:
    memories = self.storage.get_all()
    optimized = 0
    for memory in memories:
        if len(memory.associations) > max_associations:
            memory.associations = memory.associations[:max_associations]
            self.storage.update(memory)
            optimized += 1
    return optimized
async def update_importance_scores(self) -> int:
    memories = self.storage.get_all()
    updated = 0
    for memory in memories:
        recency = self._recency_score(memory)
        frequency = min(memory.access_count / 10.0, 1.0)
        connections = min(len(memory.associations) / 5.0, 1.0)
        new_importance = recency * 0.4 + frequency * 0.4 + connections * 0.2
        if abs(new_importance - memory.importance) > 0.01:
            memory.importance = new_importance
            self.storage.update(memory)
            updated += 1
    return updated
```

File: [100/252] openlaoke/core/memory/embedding.py

```
"""Vector embedding engine for semantic similarity."""
from __future__ import annotations
import hashlib
import json
import logging
import math
from pathlib import Path
logger = logging.getLogger(__name__)
class EmbeddingEngine:
    """Engine for computing and managing text embeddings."""
    def __init__(self, model: str = "simple", cache_path: Path | None = None):
        self.model = model
        if cache_path is None:
            cache_path = Path.home() / ".openlaoke" / "embeddings"
        self.cache_path = cache_path
        self.cache_path.mkdir(parents=True, exist_ok=True)
        self._cache: dict[str, list[float]] = {}
    def _get_cache_key(self, text: str) -> str:
        return hashlib.sha256(f"{self.model}:{text}".encode()).hexdigest()
    async def embed(self, text: str) -> list[float]:
        cache_key = self._get_cache_key(text)
        if cache_key in self._cache:
            return self._cache[cache_key]
        cache_file = self.cache_path / f"{cache_key}.json"
        if cache_file.exists():
            try:
                with open(cache_file, encoding="utf-8") as f:
                    embedding: list[float] = json.load(f)
                self._cache[cache_key] = embedding
                return embedding
            except Exception as e:
                logger.warning(f"Failed to load embedding cache: {e}")
        embedding = await self._compute_embedding(text)
        self._cache[cache_key] = embedding
        try:
            with open(cache_file, "w", encoding="utf-8") as f:
                json.dump(embedding, f)
        except Exception as e:
            logger.warning(f"Failed to save embedding cache: {e}")
        return embedding
    async def _compute_embedding(self, text: str) -> list[float]:
        words = text.lower().split()
        if not words:
            return [0.0] * 128
        vocab: dict[str, int] = {}
        for word in words:
            if word not in vocab:
                vocab[word] = len(vocab)
        embedding = [0.0] * 128
        for word, idx in vocab.items():
            hash_val = int(hashlib.md5(word.encode()).hexdigest(), 16)
            position = hash_val % 128
            embedding[position] += 1.0 / (idx + 1)
        magnitude = math.sqrt(sum(x * x for x in embedding))
        if magnitude > 0:
            embedding = [x / magnitude for x in embedding]
        return embedding
    async def embed_batch(self, texts: list[str]) -> list[list[float]]:
        embeddings = []
        for text in texts:
            embedding = await self.embed(text)
```

```
            embeddings.append(embedding)
        return embeddings
    @staticmethod
    def cosine_similarity(a: list[float], b: list[float]) -> float:
        if len(a) != len(b):
            return 0.0
        dot_product = sum(x * y for x, y in zip(a, b, strict=False))
        magnitude_a = math.sqrt(sum(x * x for x in a))
        magnitude_b = math.sqrt(sum(x * x for x in b))
        if magnitude_a == 0 or magnitude_b == 0:
            return 0.0
        return dot_product / (magnitude_a * magnitude_b)
class VectorIndex:
    """In-memory vector index for similarity search."""
    def __init__(self, dimension: int = 128):
        self.dimension = dimension
        self.vectors: dict[str, list[float]] = {}
    def add(self, id: str, vector: list[float]) -> None:
        if len(vector) != self.dimension:
            logger.warning(
                f"Vector dimension mismatch: expected {self.dimension}, got {len(vector)}"
            )
            return
        self.vectors[id] = vector
    def remove(self, id: str) -> None:
        if id in self.vectors:
            del self.vectors[id]
    def search(self, query: list[float], k: int = 10) -> list[tuple[str, float]]:
        if len(query) != self.dimension:
            logger.warning(f"Query dimension mismatch: expected {self.dimension}, got {len(quer
    y)}")
            return []
        similarities: list[tuple[str, float]] = []
        for id, vector in self.vectors.items():
            similarity = EmbeddingEngine.cosine_similarity(query, vector)
            similarities.append((id, similarity))
        similarities.sort(key=lambda x: x[1], reverse=True)
        return similarities[:k]
    def get(self, id: str) -> list[float] | None:
        return self.vectors.get(id)
    def clear(self) -> None:
        self.vectors.clear()
    def size(self) -> int:
        return len(self.vectors)
```

File: [101/252] openlaoke/core/memory/knowledge_graph.py

```
"""Knowledge graph for building code and concept networks."""
from __future__ import annotations
import json
import logging
import uuid
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.core.memory.embedding import VectorIndex
logger = logging.getLogger(__name__)
@dataclass
class KnowledgeNode:
    """A node in the knowledge graph."""
    id: str
    type: str
    content: str
    embedding: list[float] | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    created_at: float = field(default_factory=lambda: 0.0)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type,
            "content": self.content,
            "embedding": self.embedding,
            "metadata": self.metadata,
            "created_at": self.created_at,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> KnowledgeNode:
        return cls(
            id=data["id"],
            type=data["type"],
            content=data["content"],
            embedding=data.get("embedding"),
            metadata=data.get("metadata", {}),
            created_at=data.get("created_at", 0.0),
        )
@dataclass
class KnowledgeEdge:
    """An edge connecting two knowledge nodes."""
    id: str
    source: str
    target: str
    relation: str
    weight: float = 1.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "source": self.source,
            "target": self.target,
            "relation": self.relation,
            "weight": self.weight,
            "metadata": self.metadata,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> KnowledgeEdge:
        return cls(
            id=data["id"],
            source=data["source"],
```

```
            target=data["target"],
            relation=data["relation"],
            weight=data.get("weight", 1.0),
            metadata=data.get("metadata", {}),
        )
@dataclass
class Fact:
    """A fact for knowledge inference."""
    subject: str
    predicate: str
    obj: str
    confidence: float = 1.0
@dataclass
class Inference:
    """An inferred fact."""
    fact: Fact
    source: str
    confidence: float
@dataclass
class GraphVisualization:
    """Graph visualization data."""
    nodes: list[dict[str, Any]]
    edges: list[dict[str, Any]]
    def to_dict(self) -> dict[str, Any]:
        return {
            "nodes": self.nodes,
            "edges": self.edges,
        }
class KnowledgeGraph:
    """Knowledge graph for building code and concept networks."""
    def __init__(self, storage_path: Path | None = None):
        if storage_path is None:
            storage_path = Path.home() / ".openlaoke" / "knowledge"
        self.storage_path = storage_path
        self.storage_path.mkdir(parents=True, exist_ok=True)
        self.graph_file = self.storage_path / "graph.json"
        self.nodes: dict[str, KnowledgeNode] = {}
        self.edges: list[KnowledgeEdge] = []
        self.index = VectorIndex()
        self._load()
    def _load(self) -> None:
        if self.graph_file.exists():
            try:
                with open(self.graph_file, encoding="utf-8") as f:
                    data = json.load(f)
                self.nodes = {
                    k: KnowledgeNode.from_dict(v) for k, v in data.get("nodes", {}).items()
                }
                self.edges = [KnowledgeEdge.from_dict(e) for e in data.get("edges", [])]
            except Exception as e:
                logger.warning(f"Failed to load knowledge graph: {e}")
    def _save(self) -> None:
        try:
            data = {
                "version": "1.0",
                "nodes": {k: v.to_dict() for k, v in self.nodes.items()},
                "edges": [e.to_dict() for e in self.edges],
            }
            temp_path = self.graph_file.with_suffix(".tmp")
            with open(temp_path, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2, ensure_ascii=False)
            temp_path.replace(self.graph_file)
        except Exception as e:
            logger.error(f"Failed to save knowledge graph: {e}")
    async def add_node(self, node: KnowledgeNode) -> str:
```

```
    if not node.id:
        node.id = f"node_{uuid.uuid4().hex[:8]}"
    self.nodes[node.id] = node
    if node.embedding:
        self.index.add(node.id, node.embedding)
    self._save()
    return node.id
async def add_edge(self, source: str, target: str, relation: str, weight: float = 1.0) -> s
tr:
    edge_id = f"edge_{uuid.uuid4().hex[:8]}"
    edge = KnowledgeEdge(
        id=edge_id,
        source=source,
        target=target,
        relation=relation,
        weight=weight,
    )
    self.edges.append(edge)
    self._save()
    return edge_id
async def query(self, sparql: str) -> list[dict]:
    logger.warning("SPARQL query not fully implemented, using simplified query")
    results: list[dict] = []
    return results
async def infer(self, facts: list[Fact]) -> list[Inference]:
    inferences: list[Inference] = []
    for fact in facts:
        matching_edges = [
            e for e in self.edges if e.source == fact.subject and e.relation == fact.predic
ate
        ]
        for edge in matching_edges:
            inferred_fact = Fact(
                subject=edge.target,
                predicate=fact.predicate,
                obj=fact.obj,
                confidence=fact.confidence * edge.weight,
            )
            inferences.append(
                Inference(
                    fact=inferred_fact,
                    source="transitive_inference",
                    confidence=inferred_fact.confidence,
                )
            )
    return inferences
async def visualize(self) -> GraphVisualization:
    nodes = [
        {
            "id": node.id,
            "type": node.type,
            "label": node.content[:50],
            "metadata": node.metadata,
        }
        for node in self.nodes.values()
    ]
    edges = [
        {
            "source": edge.source,
            "target": edge.target,
            "relation": edge.relation,
            "weight": edge.weight,
        }
        for edge in self.edges
    ]
```

```
    return GraphVisualization(nodes=nodes, edges=edges)
async def merge(self, other: KnowledgeGraph) -> None:
    for node_id, node in other.nodes.items():
        if node_id not in self.nodes:
            self.nodes[node_id] = node
            if node.embedding:
                self.index.add(node_id, node.embedding)
    for edge in other.edges:
        exists = any(
            e.source == edge.source and e.target == edge.target and e.relation == edge.rela
tion
            for e in self.edges
        )
        if not exists:
            self.edges.append(edge)
    self._save()
def get_neighbors(self, node_id: str, relation: str | None = None) -> list[KnowledgeNode]:
    neighbors: list[KnowledgeNode] = []
    for edge in self.edges:
        if (edge.source == node_id or edge.target == node_id) and (
            relation is None or edge.relation == relation
        ):
            neighbor_id = edge.target if edge.source == node_id else edge.source
            if neighbor_id in self.nodes:
                neighbors.append(self.nodes[neighbor_id])
    return neighbors
def find_path(self, source: str, target: str, max_depth: int = 5) -> list[str] | None:
    if source not in self.nodes or target not in self.nodes:
        return None
    visited: set[str] = set()
    queue: list[tuple[str, list[str]]] = [(source, [source])]
    while queue:
        current, path = queue.pop(0)
        if current == target:
            return path
        if len(path) > max_depth:
            continue
        if current in visited:
            continue
        visited.add(current)
        for neighbor in self.get_neighbors(current):
            if neighbor.id not in visited:
                queue.append((neighbor.id, path + [neighbor.id]))
    return None
```

File: [102/252] openlaoke/core/memory/memory.py

```
"""Long-term memory system for storing and retrieving cross-session knowledge."""
from __future__ import annotations
import json
import logging
import time
import uuid
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.core.memory.consolidation import (
    ConsolidationResult,
    ConsolidationStrategy,
    MemoryConsolidator,
)
from openlaoke.core.memory.embedding import EmbeddingEngine
from openlaoke.core.memory.retrieval import MemoryRetriever
logger = logging.getLogger(__name__)
@dataclass
class Memory:
    """A single memory entry with metadata and associations."""
    id: str
    content: str
    embedding: list[float] | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    created_at: float = field(default_factory=time.time)
    accessed_at: float = field(default_factory=time.time)
    access_count: int = 0
    importance: float = 0.5
    associations: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "content": self.content,
            "embedding": self.embedding,
            "metadata": self.metadata,
            "created_at": self.created_at,
            "accessed_at": self.accessed_at,
            "access_count": self.access_count,
            "importance": self.importance,
            "associations": self.associations,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> Memory:
        return cls(
            id=data["id"],
            content=data["content"],
            embedding=data.get("embedding"),
            metadata=data.get("metadata", {}),
            created_at=data.get("created_at", time.time()),
            accessed_at=data.get("accessed_at", time.time()),
            access_count=data.get("access_count", 0),
            importance=data.get("importance", 0.5),
            associations=data.get("associations", []),
        )
    def touch(self) -> None:
        self.accessed_at = time.time()
        self.access_count += 1
@dataclass
class MemorySummary:
    """Summary of memory state."""
    total_memories: int
```

```
    total_associations: int
    avg_importance: float
    avg_access_count: float
    oldest_memory: float
    newest_memory: float
    categories: dict[str, int]
    def to_dict(self) -> dict[str, Any]:
        return {
            "total_memories": self.total_memories,
            "total_associations": self.total_associations,
            "avg_importance": self.avg_importance,
            "avg_access_count": self.avg_access_count,
            "oldest_memory": self.oldest_memory,
            "newest_memory": self.newest_memory,
            "categories": self.categories,
        }
class MemoryStorage:
    """Persistent storage for memories."""
    def __init__(self, storage_path: Path):
        self.storage_path = storage_path
        self.storage_path.mkdir(parents=True, exist_ok=True)
        self.memories_file = self.storage_path / "memories.json"
        self.index_file = self.storage_path / "index.json"
        self._load()
    def _load(self) -> None:
        if self.memories_file.exists():
            try:
                with open(self.memories_file, encoding="utf-8") as f:
                    data = json.load(f)
                self.memories: dict[str, Memory] = {
                    k: Memory.from_dict(v) for k, v in data.get("memories", {}).items()
                }
            except Exception as e:
                logger.warning(f"Failed to load memories: {e}")
                self.memories = {}
        else:
            self.memories = {}
    def _save(self) -> None:
        try:
            data = {
                "version": "1.0",
                "memories": {k: v.to_dict() for k, v in self.memories.items()},
            }
            temp_path = self.memories_file.with_suffix(".tmp")
            with open(temp_path, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2, ensure_ascii=False)
            temp_path.replace(self.memories_file)
        except Exception as e:
            logger.error(f"Failed to save memories: {e}")
    def add(self, memory: Memory) -> None:
        self.memories[memory.id] = memory
        self._save()
    def get(self, memory_id: str) -> Memory | None:
        return self.memories.get(memory_id)
    def update(self, memory: Memory) -> None:
        self.memories[memory.id] = memory
        self._save()
    def delete(self, memory_id: str) -> bool:
        if memory_id in self.memories:
            del self.memories[memory_id]
            self._save()
            return True
        return False
    def get_all(self) -> list[Memory]:
        return list(self.memories.values())
```

```
class LongTermMemory:
    """Long-term memory system - stores and retrieves cross-session knowledge."""
    def __init__(self, storage_path: Path | None = None):
        if storage_path is None:
            storage_path = Path.home() / ".openlaoke" / "memory"
        self.storage = MemoryStorage(storage_path)
        self.embedder = EmbeddingEngine()
        self.retriever = MemoryRetriever(self.embedder)
        self.consolidator = MemoryConsolidator(self.storage, self.embedder)
    async def store(self, memory: Memory) -> str:
        if not memory.id:
            memory.id = f"mem_{uuid.uuid4().hex[:8]}"
        if not memory.embedding:
            memory.embedding = await self.embedder.embed(memory.content)
        self.storage.add(memory)
        return memory.id
    async def recall(self, query: str, limit: int = 10) -> list[Memory]:
        query_embedding = await self.embedder.embed(query)
        results = self.retriever.search(
            memories=self.storage.get_all(),
            query_embedding=query_embedding,
            limit=limit,
        )
        for memory in results:
            memory.touch()
            self.storage.update(memory)
        return results
    async def consolidate(
        self, strategy: ConsolidationStrategy = ConsolidationStrategy.HYBRID
    ) -> ConsolidationResult:
        return await self.consolidator.consolidate(strategy)
    async def forget(self, memory_id: str) -> bool:
        return self.storage.delete(memory_id)
    async def associate(self, memory_ids: list[str]) -> None:
        memories = [self.storage.get(mid) for mid in memory_ids]
        valid_memories = [m for m in memories if m is not None]
        for i, memory in enumerate(valid_memories):
            for j, other in enumerate(valid_memories):
                if i != j and other.id not in memory.associations:
                    memory.associations.append(other.id)
            self.storage.update(memory)
    async def summarize(self) -> MemorySummary:
        memories = self.storage.get_all()
        if not memories:
            return MemorySummary(
                total_memories=0,
                total_associations=0,
                avg_importance=0.0,
                avg_access_count=0.0,
                oldest_memory=0.0,
                newest_memory=0.0,
                categories={},
            )
        total_associations = sum(len(m.associations) for m in memories)
        avg_importance = sum(m.importance for m in memories) / len(memories)
        avg_access_count = sum(m.access_count for m in memories) / len(memories)
        oldest = min(m.created_at for m in memories)
        newest = max(m.created_at for m in memories)
        categories: dict[str, int] = {}
        for m in memories:
            category = m.metadata.get("category", "uncategorized")
            categories[category] = categories.get(category, 0) + 1
        return MemorySummary(
            total_memories=len(memories),
            total_associations=total_associations,
```

```
            avg_importance=avg_importance,
            avg_access_count=avg_access_count,
            oldest_memory=oldest,
            newest_memory=newest,
            categories=categories,
        )
_memory_instance: LongTermMemory | None = None
def get_memory(storage_path: Path | None = None) -> LongTermMemory:
    global _memory_instance
    if _memory_instance is None:
        _memory_instance = LongTermMemory(storage_path)
    return _memory_instance
```

File: [103/252] openlaoke/core/memory/retrieval.py

```
"""Memory retrieval system with semantic search."""
from __future__ import annotations
import logging
import time
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from openlaoke.core.memory.embedding import EmbeddingEngine
if TYPE_CHECKING:
    from openlaoke.core.memory.memory import Memory
logger = logging.getLogger(__name__)
@dataclass
class SearchResult:
    """A search result with score and metadata."""
    memory: Memory
    score: float
    match_type: str = "semantic"
    highlights: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "memory": self.memory.to_dict(),
            "score": self.score,
            "match_type": self.match_type,
            "highlights": self.highlights,
        }
class MemoryRetriever:
    """Retrieval system for semantic and keyword search."""
    def __init__(self, embedder: EmbeddingEngine | None = None):
        self.embedder = embedder or EmbeddingEngine()
    def search(
        self,
        memories: list[Memory],
        query_embedding: list[float],
        limit: int = 10,
        threshold: float = 0.0,
        filters: dict[str, Any] | None = None,
    ) -> list[Memory]:
        if not memories:
            return []
        scored_memories: list[tuple[Memory, float]] = []
        for memory in memories:
            if filters and not self._matches_filters(memory, filters):
                continue
            if memory.embedding is None:
                continue
            similarity = EmbeddingEngine.cosine_similarity(query_embedding, memory.embedding)
            if similarity >= threshold:
                recency_boost = self._compute_recency_boost(memory)
                importance_boost = memory.importance
                final_score = similarity * 0.6 + recency_boost * 0.2 + importance_boost * 0.2
                scored_memories.append((memory, final_score))
        scored_memories.sort(key=lambda x: x[1], reverse=True)
        return [memory for memory, _ in scored_memories[:limit]]
    def keyword_search(
        self,
        memories: list[Memory],
        keywords: list[str],
        limit: int = 10,
    ) -> list[SearchResult]:
        if not keywords:
            return []
        keywords_lower = [k.lower() for k in keywords]
```

```
    results: list[SearchResult] = []
    for memory in memories:
        content_lower = memory.content.lower()
        matches = sum(1 for k in keywords_lower if k in content_lower)
        if matches > 0:
            score = matches / len(keywords_lower)
            highlights = self._extract_highlights(memory.content, keywords_lower)
            results.append(
                SearchResult(
                    memory=memory,
                    score=score,
                    match_type="keyword",
                    highlights=highlights,
                )
            )
    results.sort(key=lambda x: x.score, reverse=True)
    return results[:limit]
def hybrid_search(
    self,
    memories: list[Memory],
    query_embedding: list[float],
    keywords: list[str],
    limit: int = 10,
    semantic_weight: float = 0.7,
) -> list[SearchResult]:
    semantic_results = self.search(memories, query_embedding, limit=limit * 2)
    keyword_results = self.keyword_search(memories, keywords, limit=limit * 2)
    combined: dict[str, SearchResult] = {}
    for memory in semantic_results:
        if memory.id not in combined:
            combined[memory.id] = SearchResult(
                memory=memory,
                score=0.0,
                match_type="hybrid",
            )
    for result in keyword_results:
        if result.memory.id in combined:
            existing = combined[result.memory.id]
            combined[result.memory.id] = SearchResult(
                memory=existing.memory,
                score=existing.score + (1 - semantic_weight),
                match_type="hybrid",
                highlights=result.highlights,
            )
        else:
            combined[result.memory.id] = SearchResult(
                memory=result.memory,
                score=(1 - semantic_weight) * result.score,
                match_type="hybrid",
                highlights=result.highlights,
            )
    results = list(combined.values())
    results.sort(key=lambda x: x.score, reverse=True)
    return results[:limit]
def _matches_filters(self, memory: Memory, filters: dict[str, Any]) -> bool:
    for key, value in filters.items():
        if key in memory.metadata:
            if memory.metadata[key] != value:
                return False
        elif hasattr(memory, key) and getattr(memory, key) != value:
            return False
    return True
def _compute_recency_boost(self, memory: Memory) -> float:
    age = time.time() - memory.accessed_at
    days = age / 86400.0
```

```
    if days < 1:
        return 1.0
    elif days < 7:
        return 0.8
    elif days < 30:
        return 0.6
    elif days < 90:
        return 0.4
    else:
        return 0.2
def _extract_highlights(
    self, content: str, keywords: list[str], context_chars: int = 50
) -> list[str]:
    highlights: list[str] = []
    content_lower = content.lower()
    for keyword in keywords:
        idx = content_lower.find(keyword)
        if idx != -1:
            start = max(0, idx - context_chars)
            end = min(len(content), idx + len(keyword) + context_chars)
            highlight = content[start:end]
            if start > 0:
                highlight = "..." + highlight
            if end < len(content):
                highlight = highlight + "..."
            highlights.append(highlight)
    return highlights
async def find_similar(
    self,
    memory: Memory,
    memories: list[Memory],
    limit: int = 5,
) -> list[SearchResult]:
    if not memory.embedding:
        return []
    similar = self.search(memories, memory.embedding, limit=limit + 1)
    results: list[SearchResult] = []
    for m in similar:
        if m.id != memory.id:
            similarity = (
                EmbeddingEngine.cosine_similarity(memory.embedding, m.embedding)
                if m.embedding
                else 0.0
            )
            results.append(
                SearchResult(
                    memory=m,
                    score=similarity,
                    match_type="similar",
                )
            )
    return results[:limit]
```

File: [104/252] openlaoke/core/middleware/__init__.py

```
"""Middleware system for request processing."""
from openlaoke.core.middleware.base import (
    ErrorEvent,
    Event,
    MessageEvent,
    Middleware,
    NextMiddleware,
    ProgressEvent,
    SyncMiddleware,
    ToolCallEvent,
    ToolResultEvent,
)
from openlaoke.core.middleware.builtins import (
    ClarificationMiddleware,
    DanglingToolCallMiddleware,
    ErrorHandlingMiddleware,
    GuardrailMiddleware,
    LoggingMiddleware,
    MemoryMiddleware,
    SandboxMiddleware,
    SubagentLimitMiddleware,
    SummarizationMiddleware,
    ThreadDataMiddleware,
    TitleMiddleware,
    TodoListMiddleware,
    UploadsMiddleware,
    ViewImageMiddleware,
)
from openlaoke.core.middleware.chain import MiddlewareChain
from openlaoke.core.middleware.context import MiddlewareContext
__all__ = [
    "Middleware",
    "SyncMiddleware",
    "MiddlewareChain",
    "MiddlewareContext",
    "NextMiddleware",
    "Event",
    "MessageEvent",
    "ToolCallEvent",
    "ToolResultEvent",
    "ProgressEvent",
    "ErrorEvent",
    "ThreadDataMiddleware",
    "UploadsMiddleware",
    "SandboxMiddleware",
    "DanglingToolCallMiddleware",
    "GuardrailMiddleware",
    "SummarizationMiddleware",
    "TodoListMiddleware",
    "TitleMiddleware",
    "MemoryMiddleware",
    "ViewImageMiddleware",
    "SubagentLimitMiddleware",
    "ClarificationMiddleware",
    "LoggingMiddleware",
    "ErrorHandlingMiddleware",
]
```

File: [105/252] openlaoke/core/middleware/base.py

```
"""Base middleware class and event types."""
from __future__ import annotations
from abc import ABC, abstractmethod
from collections.abc import AsyncGenerator, Callable
from dataclasses import dataclass
from typing import Any
from openlaoke.core.middleware.context import MiddlewareContext
@dataclass
class Event:
    """Event emitted during middleware processing."""
    type: str
    data: dict[str, Any]
    timestamp: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        """Convert event to dictionary."""
        return {
            "type": self.type,
            "data": self.data,
            "timestamp": self.timestamp,
        }
@dataclass
class MessageEvent(Event):
    """Event for message additions."""
    def __init__(self, content: str, role: str = "assistant", **kwargs: Any) -> None:
        super().__init__(
            type="message",
            data={"content": content, "role": role},
            **kwargs,
        )
@dataclass
class ToolCallEvent(Event):
    """Event for tool call start."""
    def __init__(
        self,
        tool_name: str,
        tool_use_id: str,
        tool_input: dict[str, Any],
        **kwargs: Any,
    ) -> None:
        super().__init__(
            type="tool_call",
            data={
                "tool_name": tool_name,
                "tool_use_id": tool_use_id,
                "tool_input": tool_input,
            },
            **kwargs,
        )
@dataclass
class ToolResultEvent(Event):
    """Event for tool call completion."""
    def __init__(
        self,
        tool_use_id: str,
        result: str | dict[str, Any],
        is_error: bool = False,
        **kwargs: Any,
    ) -> None:
        super().__init__(
            type="tool_result",
            data={
```

```
                "tool_use_id": tool_use_id,
                "result": result,
                "is_error": is_error,
            },
            **kwargs,
        )
@dataclass
class ProgressEvent(Event):
    """Event for progress updates."""
    def __init__(
        self,
        message: str,
        percentage: float | None = None,
        **kwargs: Any,
    ) -> None:
        super().__init__(
            type="progress",
            data={"message": message, "percentage": percentage},
            **kwargs,
        )
@dataclass
class ErrorEvent(Event):
    """Event for error conditions."""
    def __init__(self, error: str, error_type: str = "unknown", **kwargs: Any) -> None:
        super().__init__(
            type="error",
            data={"error": error, "error_type": error_type},
            **kwargs,
        )
NextMiddleware = Callable[[MiddlewareContext], AsyncGenerator[Event, None]]
class Middleware(ABC):
    """Base class for all middleware.
    Middleware can intercept and modify the request/response flow.
    Each middleware can:
    - Pre-process the context before passing to the next middleware
    - Post-process events from downstream middleware
    - Short-circuit the chain by not calling next_middleware
    """
    @property
    def name(self) -> str:
        """Get the middleware name."""
        return self.__class__.__name__
    @abstractmethod
    def setup(self, context: MiddlewareContext) -> None:
        """Called before the middleware chain starts.
        Override this to perform initialization.
        """
    @abstractmethod
    def teardown(self, context: MiddlewareContext) -> None:
        """Called after the middleware chain completes.
        Override this to perform cleanup.
        """
    @abstractmethod
    def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        """Process the request.
        Args:
            context: The middleware context
            next_middleware: Function to call the next middleware in the chain
        Yields:
            Events generated during processing
        """
```

```
        ...
class SyncMiddleware(Middleware):
    """Base class for synchronous middleware.
    For middleware that doesn't need to yield events downstream.
    """
    def process(self, context: MiddlewareContext) -> list[Event]:
        """Process the context and return events.
        Override this method to perform synchronous processing.
        Args:
            context: The middleware context
        Returns:
            List of events generated during processing
        """
        return []
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        """Process events, then pass to next middleware."""
        for event in self.process(context):
            yield event
        if not context.aborted:
            async for event in next_middleware(context):
                yield event
```

File: [106/252] openlaoke/core/middleware/builtins.py

```
"""Built-in middleware implementations."""
from __future__ import annotations
import asyncio
import logging
import os
import re
import time
from collections.abc import AsyncGenerator
from typing import Any
from openlaoke.core.middleware.base import (
    ErrorEvent,
    Event,
    Middleware,
    NextMiddleware,
    ProgressEvent,
    SyncMiddleware,
)
from openlaoke.core.middleware.context import MiddlewareContext
from openlaoke.types.core_types import AssistantMessage, ToolUseBlock, UserMessage
logger = logging.getLogger(__name__)
class ThreadDataMiddleware(SyncMiddleware):
    """Middleware for managing thread-specific data directories.
    Creates and manages workspace, uploads, and outputs directories
    for each session.
    """
    def __init__(self, base_dir: str | None = None, lazy_init: bool = True) -> None:
        self.base_dir = base_dir
        self.lazy_init = lazy_init
        self._initialized_sessions: set[str] = set()
    def process(self, context: MiddlewareContext) -> list[Event]:
        session_id = context.session_id or context.state.session_id
        if not session_id:
            return []
        if session_id in self._initialized_sessions:
            return []
        base = self.base_dir or os.path.join(context.state.cwd, ".openlaoke")
        thread_dir = os.path.join(base, "sessions", session_id)
        dirs = {
            "workspace": os.path.join(thread_dir, "workspace"),
            "uploads": os.path.join(thread_dir, "uploads"),
            "outputs": os.path.join(thread_dir, "outputs"),
        }
        if not self.lazy_init:
            for dir_path in dirs.values():
                os.makedirs(dir_path, exist_ok=True)
            logger.debug(f"Created thread directories for session {session_id}")
        context.set_metadata("thread_data", dirs)
        self._initialized_sessions.add(session_id)
        return []
class UploadsMiddleware(SyncMiddleware):
    """Middleware for handling file uploads."""
    def process(self, context: MiddlewareContext) -> list[Event]:
        for message in context.messages:
            if isinstance(message, UserMessage) and message.attachments:
                attachments_info = []
                for path in message.attachments:
                    if os.path.exists(path):
                        size = os.path.getsize(path)
                        attachments_info.append(
                            {
                                "path": path,
```

```
                                "size": size,
                                "name": os.path.basename(path),
                            }
                        )
                if attachments_info:
                    context.set_metadata("uploads", attachments_info)
                    logger.debug(f"Processed {len(attachments_info)} attachments")
        return []
class SandboxMiddleware(Middleware):
    """Middleware for sandbox environment management."""
    def __init__(self, enabled: bool = True) -> None:
        self.enabled = enabled
        self._sandbox_id: str | None = None
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        if self.enabled:
            self._sandbox_id = f"sandbox_{context.session_id}"
            context.set_metadata("sandbox_id", self._sandbox_id)
            logger.debug(f"Acquired sandbox: {self._sandbox_id}")
        try:
            async for event in next_middleware(context):
                yield event
        finally:
            if self._sandbox_id:
                logger.debug(f"Released sandbox: {self._sandbox_id}")
                self._sandbox_id = None
class DanglingToolCallMiddleware(SyncMiddleware):
    """Middleware for handling dangling tool calls."""
    def process(self, context: MiddlewareContext) -> list[Event]:
        pending_calls: list[ToolUseBlock] = []
        for message in reversed(context.messages):
            if isinstance(message, AssistantMessage):
                if message.tool_uses:
                    pending_calls.extend(message.tool_uses)
                break
        if pending_calls:
            context.set_metadata("pending_tool_calls", pending_calls)
            logger.debug(f"Found {len(pending_calls)} pending tool calls")
        return []
class GuardrailMiddleware(Middleware):
    """Middleware for safety guardrails."""
    def __init__(
        self,
        blocked_tools: list[str] | None = None,
        blocked_patterns: list[str] | None = None,
    ) -> None:
        self.blocked_tools = set(blocked_tools or [])
        self.blocked_patterns = blocked_patterns or []
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        if context.current_tool:
            tool_name = context.current_tool.name
            if tool_name in self.blocked_tools:
                logger.warning(f"Blocked tool call: {tool_name}")
                yield ErrorEvent(
                    error=f"Tool '{tool_name}' is blocked by guardrail policy",
                    error_type="guardrail_blocked",
                )
                context.abort("guardrail_blocked")
```

```
                return
        async for event in next_middleware(context):
            if self._should_block_event(event):
                logger.warning(f"Blocked event: {event.type}")
                continue
            yield event
    def _should_block_event(self, event: Event) -> bool:
        return any(pattern in str(event.data) for pattern in self.blocked_patterns)
class SummarizationMiddleware(SyncMiddleware):
    """Middleware for context compression."""
    def __init__(
        self,
        max_messages: int = 100,
        summarize_threshold: int = 80,
    ) -> None:
        self.max_messages = max_messages
        self.summarize_threshold = summarize_threshold
    def process(self, context: MiddlewareContext) -> list[Event]:
        message_count = len(context.messages)
        if message_count > self.summarize_threshold:
            context.set_metadata("needs_summarization", True)
            context.set_metadata(
                "summarization_stats",
                {
                    "total_messages": message_count,
                    "threshold": self.summarize_threshold,
                    "max_messages": self.max_messages,
                },
            )
            logger.debug(f"Marked for summarization: {message_count} messages")
        return []
class TodoListMiddleware(SyncMiddleware):
    """Middleware for task tracking."""
    def __init__(self) -> None:
        self._todos: dict[str, list[dict[str, Any]]] = {}
    def process(self, context: MiddlewareContext) -> list[Event]:
        todos = []
        todo_pattern = re.compile(
            r"(?:TODO|FIXME|XXX|HACK):\s*(.+)",
            re.IGNORECASE,
        )
        for message in context.messages:
            if isinstance(message, (UserMessage, AssistantMessage)):
                content = message.content
                matches = todo_pattern.findall(content)
                for match in matches:
                    todos.append(
                        {
                            "text": match.strip(),
                            "role": message.role.value,
                            "timestamp": message.timestamp,
                        }
                    )
        if todos:
            self._todos[context.session_id] = todos
            context.set_metadata("todos", todos)
            logger.debug(f"Found {len(todos)} TODO items")
        return []
class TitleMiddleware(SyncMiddleware):
    """Middleware for generating conversation titles."""
    def __init__(self, min_messages: int = 2) -> None:
        self.min_messages = min_messages
        self._titles: dict[str, str] = {}
    def process(self, context: MiddlewareContext) -> list[Event]:
        if len(context.messages) < self.min_messages:
```

```
            return []
        existing_title = context.get_metadata("title")
        if existing_title:
            return []
        first_user_message = None
        for message in context.messages:
            if isinstance(message, UserMessage) and message.content.strip():
                first_user_message = message.content
                break
        if first_user_message:
            title = self._generate_title(first_user_message)
            self._titles[context.session_id] = title
            context.set_metadata("title", title)
            logger.debug(f"Generated title: {title}")
        return []
    def _generate_title(self, content: str) -> str:
        title = content.split("\n")[0].strip()
        title = re.sub(r"[^\w\s-]", "", title)
        title = " ".join(title.split()[:10])
        return title[:100] if len(title) > 100 else title
class MemoryMiddleware(SyncMiddleware):
    """Middleware for memory queue management."""
    def __init__(self, enabled: bool = True) -> None:
        self.enabled = enabled
        self._queue: list[dict[str, Any]] = []
    def process(self, context: MiddlewareContext) -> list[Event]:
        if not self.enabled:
            return []
        if len(context.messages) < 2:
            return []
        memory_entry = {
            "session_id": context.session_id,
            "message_count": len(context.messages),
            "timestamp": time.time(),
            "metadata": context.metadata,
        }
        self._queue.append(memory_entry)
        context.set_metadata("memory_queued", True)
        logger.debug("Queued conversation for memory update")
        return []
class ViewImageMiddleware(SyncMiddleware):
    """Middleware for image injection."""
    def process(self, context: MiddlewareContext) -> list[Event]:
        import base64
        images_info = []
        for message in context.messages:
            if isinstance(message, UserMessage) and message.images:
                for img_path in message.images:
                    if os.path.exists(img_path):
                        try:
                            with open(img_path, "rb") as f:
                                img_data = base64.b64encode(f.read()).decode()
                            images_info.append(
                                {
                                    "path": img_path,
                                    "size": os.path.getsize(img_path),
                                    "data_preview": img_data[:100] + "...",
                                }
                            )
                        except Exception as e:
                            logger.warning(f"Failed to process image {img_path}: {e}")
        if images_info:
            context.set_metadata("images", images_info)
            logger.debug(f"Processed {len(images_info)} images")
        return []
```

```
class SubagentLimitMiddleware(Middleware):
    """Middleware for concurrency limiting."""
    def __init__(self, max_concurrent: int = 5) -> None:
        self.max_concurrent = max_concurrent
        self._semaphore = asyncio.Semaphore(max_concurrent)
        self._active_count = 0
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        async with self._semaphore:
            self._active_count += 1
            context.set_metadata("subagent_slot", self._active_count)
            logger.debug(f"Subagent slot acquired: {self._active_count}/{self.max_concurrent}")
            try:
                async for event in next_middleware(context):
                    yield event
            finally:
                self._active_count -= 1
                logger.debug(f"Subagent slot released: {self._active_count}/{self.max_concurren
    t}")
class ClarificationMiddleware(Middleware):
    """Middleware for clarification requests."""
    def __init__(
        self,
        clarification_threshold: float = 0.5,
        auto_clarify: bool = False,
    ) -> None:
        self.clarification_threshold = clarification_threshold
        self.auto_clarify = auto_clarify
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        clarification_needed = context.get_metadata("clarification_needed", False)
        if clarification_needed:
            question = context.get_metadata(
                "clarification_question", "Please provide more details."
            )
            yield ProgressEvent(message=f"Clarification needed: {question}")
        async for event in next_middleware(context):
            yield event
    def request_clarification(
        self,
        context: MiddlewareContext,
        question: str,
    ) -> None:
        context.set_metadata("clarification_needed", True)
        context.set_metadata("clarification_question", question)
class LoggingMiddleware(Middleware):
    """Middleware for request/response logging."""
    def __init__(self, log_level: int = logging.DEBUG) -> None:
        self.log_level = log_level
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        logger.log(self.log_level, f"[{context.session_id}] Processing request")
        event_count = 0
        start_time = time.time()
        async for event in next_middleware(context):
            event_count += 1
```

```
            logger.log(
                self.log_level,
                f"[{context.session_id}] Event: {event.type}",
            )
            yield event
        elapsed = time.time() - start_time
        logger.log(
            self.log_level,
            f"[{context.session_id}] Completed: {event_count} events in {elapsed:.3f}s",
        )
class ErrorHandlingMiddleware(Middleware):
    """Middleware for centralized error handling."""
    async def __call__(
        self,
        context: MiddlewareContext,
        next_middleware: NextMiddleware,
    ) -> AsyncGenerator[Event, None]:
        try:
            async for event in next_middleware(context):
                yield event
        except asyncio.CancelledError:
            logger.info(f"Request cancelled: {context.session_id}")
            yield ErrorEvent(error="Request cancelled", error_type="cancelled")
        except Exception as e:
            logger.exception(f"Error in middleware chain: {e}")
            context.error = e
            yield ErrorEvent(error=str(e), error_type=type(e).__name__)
```

File: [107/252] openlaoke/core/middleware/chain.py

```
"""Middleware chain implementation."""
from __future__ import annotations
import logging
import time
from collections.abc import AsyncGenerator
from typing import Self
from openlaoke.core.middleware.base import Event, Middleware
from openlaoke.core.middleware.context import MiddlewareContext
logger = logging.getLogger(__name__)
class MiddlewareChain:
    """Chain of middleware that process requests in sequence.
    The chain executes middleware in the order they were added.
    Each middleware can:
    - Pre-process before calling next
    - Post-process after next returns
    - Short-circuit by not calling next
    """
    def __init__(self) -> None:
        """Initialize an empty middleware chain."""
        self.middlewares: list[Middleware] = []
        self._setup_complete: bool = False
    def add(self, middleware: Middleware) -> Self:
        """Add a middleware to the chain.
        Args:
            middleware: The middleware to add
        Returns:
            Self for chaining
        """
        self.middlewares.append(middleware)
        return self
    def remove(self, middleware_name: str) -> bool:
        """Remove a middleware by name.
        Args:
            middleware_name: Name of the middleware to remove
        Returns:
            True if removed, False if not found
        """
        for i, mw in enumerate(self.middlewares):
            if mw.name == middleware_name:
                self.middlewares.pop(i)
                return True
        return False
    def insert(self, index: int, middleware: Middleware) -> Self:
        """Insert a middleware at a specific position.
        Args:
            index: Position to insert at
            middleware: The middleware to insert
        Returns:
            Self for chaining
        """
        self.middlewares.insert(index, middleware)
        return self
    def get(self, name: str) -> Middleware | None:
        """Get a middleware by name.
        Args:
            name: Name of the middleware
        Returns:
            The middleware if found, None otherwise
        """
        for mw in self.middlewares:
            if mw.name == name:
```

```
            return mw
    return None
def clear(self) -> Self:
    """Remove all middleware from the chain.
    Returns:
        Self for chaining
    """
    self.middlewares.clear()
    return self
def process(self, context: MiddlewareContext) -> AsyncGenerator[Event, None]:
    """Process the context through the middleware chain.
    Args:
        context: The middleware context to process
    Yields:
        Events generated during processing
    """
    return self._run_chain(context, 0)
def _run_chain(
    self,
    context: MiddlewareContext,
    index: int,
) -> AsyncGenerator[Event, None]:
    """Recursively run the middleware chain.
    Args:
        context: The middleware context
        index: Current middleware index
    Yields:
        Events from the middleware
    """
    return self._run_chain_impl(context, index)
async def _run_chain_impl(
    self,
    context: MiddlewareContext,
    index: int,
) -> AsyncGenerator[Event, None]:
    """Implementation of chain execution."""
    if context.aborted:
        logger.debug(f"Middleware chain aborted at index {index}")
        return
    if index >= len(self.middlewares):
        logger.debug("Middleware chain complete")
        return
    middleware = self.middlewares[index]
    self._run_setup_single(middleware, context)
    def next_middleware(ctx: MiddlewareContext) -> AsyncGenerator[Event, None]:
        return self._run_chain_impl(ctx, index + 1)
    start_time = time.time()
    try:
        async for event in middleware(context, next_middleware):
            yield event
    except Exception as e:
        logger.exception(f"Error in middleware {middleware.name}: {e}")
        context.error = e
        raise
    finally:
        elapsed = time.time() - start_time
        logger.debug(f"Middleware {middleware.name} took {elapsed:.3f}s")
        self._run_teardown_single(middleware, context)
def _run_setup_single(self, middleware: Middleware, context: MiddlewareContext) -> None:
    """Run setup for a single middleware."""
    try:
        middleware.setup(context)
    except Exception as e:
        logger.exception(f"Error in setup for {middleware.name}: {e}")
        raise
```

```
def _run_teardown_single(self, middleware: Middleware, context: MiddlewareContext) -> None:
    """Run teardown for a single middleware."""
    try:
        middleware.teardown(context)
    except Exception as e:
        logger.exception(f"Error in teardown for {middleware.name}: {e}")
def __len__(self) -> int:
    """Get the number of middleware in the chain."""
    return len(self.middlewares)
def __contains__(self, name: str) -> bool:
    """Check if a middleware is in the chain."""
    return self.get(name) is not None
def __iter__(self):
    """Iterate over middleware."""
    return iter(self.middlewares)
```

File: [108/252] openlaoke/core/middleware/context.py

```
"""Middleware context for request processing."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
    from openlaoke.core.tool import Tool
    from openlaoke.types.core_types import Message
@dataclass
class MiddlewareContext:
    """Context passed through the middleware chain.
    Contains all the information needed for middleware to process requests.
    """
    state: AppState
    messages: list[Message] = field(default_factory=list)
    current_tool: Tool | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    session_id: str = ""
    request_type: str = "completion"
    aborted: bool = False
    error: Exception | None = None
    def set_metadata(self, key: str, value: Any) -> None:
        """Set a metadata value."""
        self.metadata[key] = value
    def get_metadata(self, key: str, default: Any = None) -> Any:
        """Get a metadata value."""
        return self.metadata.get(key, default)
    def abort(self, reason: str | None = None) -> None:
        """Abort the middleware chain."""
        self.aborted = True
        if reason:
            self.metadata["abort_reason"] = reason
    def set_error(self, error: Exception) -> None:
        """Set an error to be handled by error middleware."""
        self.error = error
    def clear_error(self) -> None:
        """Clear the current error."""
        self.error = None
```

File: [109/252] openlaoke/core/model_assessment/__init__.py

```
"""Model capability assessment."""
from openlaoke.core.model_assessment.assessor import ModelAssessor, TaskDecomposer
from openlaoke.core.model_assessment.types import (
    KNOWN_MODEL_TIERS,
    TIER_GRANULARITIES,
    CapabilityCategory,
    CapabilityScore,
    ModelBenchmark,
    ModelTier,
    TaskGranularity,
)
__all__ = [
    "ModelAssessor",
    "TaskDecomposer",
    "ModelBenchmark",
    "ModelTier",
    "TaskGranularity",
    "CapabilityScore",
    "CapabilityCategory",
    "KNOWN_MODEL_TIERS",
    "TIER_GRANULARITIES",
]
```

File: [110/252] openlaoke/core/model_assessment/assessor.py

```
"""Model capability assessor."""
from __future__ import annotations
import json
import time
from pathlib import Path
from typing import TYPE_CHECKING
from openlaoke.core.model_assessment.types import (
    KNOWN_MODEL_TIERS,
    TIER_GRANULARITIES,
    CapabilityCategory,
    CapabilityScore,
    ModelBenchmark,
    ModelTier,
    TaskGranularity,
)
if TYPE_CHECKING:
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.types.providers import MultiProviderConfig
class ModelAssessor:
    """Assess model capabilities and determine task strategies."""
    def __init__(self, config: MultiProviderConfig):
        self.config = config
        self.benchmarks_dir = Path.home() / ".openlaoke" / "model_benchmarks"
        self.benchmarks_dir.mkdir(parents=True, exist_ok=True)
        self._cache: dict[str, ModelBenchmark] = {}
    def get_tier(self, model: str) -> ModelTier:
        model_lower = model.lower()
        for known, tier in KNOWN_MODEL_TIERS.items():
            if known.lower() in model_lower:
                return tier
        return ModelTier.TIER_3_MODERATE
    def get_granularity(self, model: str) -> TaskGranularity:
        return TIER_GRANULARITIES[self.get_tier(model)]
    async def assess_model(
        self, api: MultiProviderClient, model: str, provider: str, quick: bool = False
    ) -> ModelBenchmark:
        start = time.time()
        caps = await self._run_tests(api, model, quick)
        overall = sum(c.score for c in caps.values()) / len(caps) if caps else 0.0
        if overall >= 8.5:
            tier = ModelTier.TIER_1_ADVANCED
        elif overall >= 7.0:
            tier = ModelTier.TIER_2_CAPABLE
        elif overall >= 5.5:
            tier = ModelTier.TIER_3_MODERATE
        elif overall >= 4.0:
            tier = ModelTier.TIER_4_BASIC
        else:
            tier = ModelTier.TIER_5_LIMITED
        recs = self._gen_recs(tier, caps)
        bench = ModelBenchmark(
            provider=provider,
            model=model,
            overall_score=overall,
            tier=tier,
            capabilities=caps,
            tested_at=time.time(),
            test_duration=time.time() - start,
            recommendations=recs,
        )
        self._save(bench)
```

```
    return bench
async def _run_tests(
    self, api: MultiProviderClient, model: str, quick: bool
) -> dict[str, CapabilityScore]:
    caps = {}
    caps[CapabilityCategory.BASIC_CHAT.value] = await self._test_chat(api, model)
    if not quick:
        caps[CapabilityCategory.TOOL_CALLING.value] = await self._test_tools(api, model)
        caps[CapabilityCategory.CODE_GENERATION.value] = await self._test_code(api, model)
        caps[CapabilityCategory.MULTI_TURN.value] = await self._test_multiturn(api, model)
    return caps
async def _test_chat(self, api: MultiProviderClient, model: str) -> CapabilityScore:
    tests = [("What is 2+2?", "4"), ("Capital of France?", "Paris")]
    passed = 0
    for q, exp in tests:
        try:
            r, _, _ = await api.send_message(
                "You are helpful.", [{"role": "user", "content": q}], None, model
            )
            if exp.lower() in r.content.lower():
                passed += 1
        except Exception:
            pass
    return CapabilityScore(
        CapabilityCategory.BASIC_CHAT, (passed / len(tests)) * 10, passed, len(tests)
    )
async def _test_tools(self, api: MultiProviderClient, model: str) -> CapabilityScore:
    tool = {
        "name": "test",
        "description": "Test",
        "input_schema": {"type": "object", "properties": {"x": {"type": "string"}}},
    }
    passed = 0
    try:
        r, _, _ = await api.send_message(
            "Use tools.",
            [{"role": "user", "content": "Call test with x='hello'"}],
            [tool],
            model,
        )
        if r.tool_uses and r.tool_uses[0].name == "test":
            passed += 1
    except Exception:
        pass
    return CapabilityScore(CapabilityCategory.TOOL_CALLING, passed * 10, passed, 1)
async def _test_code(self, api: MultiProviderClient, model: str) -> CapabilityScore:
    try:
        r, _, _ = await api.send_message(
            "Write code.",
            [{"role": "user", "content": "Python function to add two numbers"}],
            None,
            model,
        )
        score = 10.0 if "def" in r.content and "return" in r.content else 5.0
        return CapabilityScore(
            CapabilityCategory.CODE_GENERATION, score, 1 if score > 5 else 0, 1
        )
    except Exception:
        return CapabilityScore(CapabilityCategory.CODE_GENERATION, 0, 0, 1)
async def _test_multiturn(self, api: MultiProviderClient, model: str) -> CapabilityScore:
    try:
        msgs = [{"role": "user", "content": "My name is Bob"}]
        r1, _, _ = await api.send_message("Remember.", msgs, None, model)
        msgs.append({"role": "assistant", "content": r1.content})
        msgs.append({"role": "user", "content": "What's my name?"})
```

```
            r2, _, _ = await api.send_message("Remember.", msgs, None, model)
            score = 10.0 if "bob" in r2.content.lower() else 5.0
            return CapabilityScore(CapabilityCategory.MULTI_TURN, score, 1 if score > 5 else 0,
     1)
        except Exception:
            return CapabilityScore(CapabilityCategory.MULTI_TURN, 0, 0, 1)
    def _gen_recs(self, tier: ModelTier, caps: dict[str, CapabilityScore]) -> list[str]:
        recs = []
        if tier in [ModelTier.TIER_5_LIMITED, ModelTier.TIER_4_BASIC]:
            recs = ["Break tasks into atomic steps", "Verify each step", "Use simple instructio
    ns"]
        elif tier == ModelTier.TIER_3_MODERATE:
            recs = ["Limit to 10 subtasks", "Check intermediate results"]
        elif tier == ModelTier.TIER_2_CAPABLE:
            recs = ["Can handle moderate complexity", "Verify critical operations"]
        else:
            recs = ["Can handle complex tasks directly"]
        return recs
    def _save(self, bench: ModelBenchmark) -> None:
        path = self.benchmarks_dir / f"{bench.provider}_{bench.model.replace('/', '_')}.json"
        with open(path, "w") as f:
            json.dump(bench.to_dict(), f, indent=2)
    def load(self, provider: str, model: str) -> ModelBenchmark | None:
        path = self.benchmarks_dir / f"{provider}_{model.replace('/', '_')}.json"
        if path.exists():
            with open(path) as f:
                return ModelBenchmark.from_dict(json.load(f))
        return None
    def list_all(self) -> list[ModelBenchmark]:
        result = []
        for p in self.benchmarks_dir.glob("*.json"):
            with open(p) as f:
                result.append(ModelBenchmark.from_dict(json.load(f)))
        return sorted(result, key=lambda b: b.overall_score, reverse=True)
class TaskDecomposer:
    """Decompose tasks based on model tier."""
    def __init__(self, gran: TaskGranularity):
        self.gran = gran
    def decompose(self, task: str) -> list[str]:
        if not self.gran.requires_explicit_steps:
            return [task]
        parts = task.replace(" and ", "|").replace(" then ", "|").split("|")
        return [p.strip() for p in parts if p.strip()][: self.gran.max_subtasks]
    def should_verify(self, step: int) -> bool:
        freq = self.gran.verification_frequency
        if freq == "every_step":
            return True
        elif freq == "frequent":
            return step % 2 == 0
        elif freq == "moderate":
            return step % 3 == 0
        return False
    def get_timeout(self, base: float) -> float:
        return base * self.gran.timeout_multiplier
```

File: [111/252] openlaoke/core/model_assessment/types.py

```
"""Model capability types and constants."""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
class CapabilityCategory(StrEnum):
    BASIC_CHAT = "basic_chat"
    TOOL_CALLING = "tool_calling"
    CODE_GENERATION = "code_generation"
    MULTI_TURN = "multi_turn"
    ERROR_RECOVERY = "error_recovery"
    INSTRUCTION_FOLLOWING = "instruction_following"
class ModelTier(StrEnum):
    TIER_1_ADVANCED = "tier_1_advanced"
    TIER_2_CAPABLE = "tier_2_capable"
    TIER_3_MODERATE = "tier_3_moderate"
    TIER_4_BASIC = "tier_4_basic"
    TIER_5_LIMITED = "tier_5_limited"
@dataclass
class CapabilityScore:
    category: CapabilityCategory
    score: float
    max_score: float = 10.0
    tests_passed: int = 0
    tests_total: int = 0
    details: dict[str, Any] = field(default_factory=dict)
    @property
    def percentage(self) -> float:
        return (self.score / self.max_score) * 100
    @property
    def pass_rate(self) -> float:
        if self.tests_total == 0:
            return 0.0
        return (self.tests_passed / self.tests_total) * 100
@dataclass
class ModelBenchmark:
    provider: str
    model: str
    overall_score: float
    tier: ModelTier
    capabilities: dict[str, CapabilityScore]
    tested_at: float
    test_duration: float
    recommendations: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "provider": self.provider,
            "model": self.model,
            "overall_score": self.overall_score,
            "tier": self.tier.value,
            "capabilities": {
                k: {
                    "score": v.score,
                    "max_score": v.max_score,
                    "tests_passed": v.tests_passed,
                    "tests_total": v.tests_total,
                    "percentage": v.percentage,
                    "pass_rate": v.pass_rate,
                    "details": v.details,
                }
                for k, v in self.capabilities.items()
```

```
            },
            "tested_at": self.tested_at,
            "test_duration": self.test_duration,
            "recommendations": self.recommendations,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> ModelBenchmark:
        capabilities = {}
        for k, v in data["capabilities"].items():
            capabilities[k] = CapabilityScore(
                category=CapabilityCategory(k),
                score=v["score"],
                max_score=v.get("max_score", 10.0),
                tests_passed=v.get("tests_passed", 0),
                tests_total=v.get("tests_total", 0),
                details=v.get("details", {}),
            )
        return cls(
            provider=data["provider"],
            model=data["model"],
            overall_score=data["overall_score"],
            tier=ModelTier(data["tier"]),
            capabilities=capabilities,
            tested_at=data["tested_at"],
            test_duration=data["test_duration"],
            recommendations=data.get("recommendations", []),
        )
@dataclass
class TaskGranularity:
    max_subtasks: int
    subtask_complexity_limit: str
    verification_frequency: str
    retry_limit: int
    timeout_multiplier: float
    tool_call_limit: int
    requires_explicit_steps: bool
    min_confidence_threshold: float
KNOWN_MODEL_TIERS: dict[str, ModelTier] = {
    "claude-sonnet-4": ModelTier.TIER_1_ADVANCED,
    "claude-opus-4": ModelTier.TIER_1_ADVANCED,
    "claude-3-5-sonnet": ModelTier.TIER_1_ADVANCED,
    "gpt-4o": ModelTier.TIER_1_ADVANCED,
    "gpt-4-turbo": ModelTier.TIER_1_ADVANCED,
    "o1-preview": ModelTier.TIER_1_ADVANCED,
    "claude-3-5-haiku": ModelTier.TIER_2_CAPABLE,
    "gpt-4o-mini": ModelTier.TIER_2_CAPABLE,
    "gemma3:27b": ModelTier.TIER_2_CAPABLE,
    "llama3.1:70b": ModelTier.TIER_2_CAPABLE,
    "qwen2.5:72b": ModelTier.TIER_2_CAPABLE,
    "mistral-large": ModelTier.TIER_2_CAPABLE,
    "gemma3:12b": ModelTier.TIER_3_MODERATE,
    "llama3.1:8b": ModelTier.TIER_3_MODERATE,
    "qwen2.5:14b": ModelTier.TIER_3_MODERATE,
    "mistral-small": ModelTier.TIER_3_MODERATE,
    "gemma3:4b": ModelTier.TIER_4_BASIC,
    "llama3.2:3b": ModelTier.TIER_4_BASIC,
    "qwen2.5:7b": ModelTier.TIER_4_BASIC,
    "phi3:mini": ModelTier.TIER_4_BASIC,
    "gemma3:1b": ModelTier.TIER_5_LIMITED,
    "llama3.2:1b": ModelTier.TIER_5_LIMITED,
    "qwen2.5:1.5b": ModelTier.TIER_5_LIMITED,
    "phi3:tiny": ModelTier.TIER_5_LIMITED,
}
TIER_GRANULARITIES: dict[ModelTier, TaskGranularity] = {
    ModelTier.TIER_1_ADVANCED: TaskGranularity(
```

```
        max_subtasks=20,
        subtask_complexity_limit="high",
        verification_frequency="minimal",
        retry_limit=3,
        timeout_multiplier=1.0,
        tool_call_limit=50,
        requires_explicit_steps=False,
        min_confidence_threshold=0.6,
    ),
    ModelTier.TIER_2_CAPABLE: TaskGranularity(
        max_subtasks=15,
        subtask_complexity_limit="medium",
        verification_frequency="moderate",
        retry_limit=4,
        timeout_multiplier=1.2,
        tool_call_limit=35,
        requires_explicit_steps=False,
        min_confidence_threshold=0.7,
    ),
    ModelTier.TIER_3_MODERATE: TaskGranularity(
        max_subtasks=10,
        subtask_complexity_limit="low",
        verification_frequency="frequent",
        retry_limit=5,
        timeout_multiplier=1.5,
        tool_call_limit=25,
        requires_explicit_steps=True,
        min_confidence_threshold=0.8,
    ),
    ModelTier.TIER_4_BASIC: TaskGranularity(
        max_subtasks=6,
        subtask_complexity_limit="minimal",
        verification_frequency="every_step",
        retry_limit=6,
        timeout_multiplier=2.0,
        tool_call_limit=15,
        requires_explicit_steps=True,
        min_confidence_threshold=0.85,
    ),
    ModelTier.TIER_5_LIMITED: TaskGranularity(
        max_subtasks=4,
        subtask_complexity_limit="atomic",
        verification_frequency="every_step",
        retry_limit=8,
        timeout_multiplier=3.0,
        tool_call_limit=10,
        requires_explicit_steps=True,
        min_confidence_threshold=0.9,
    ),
}
```

File: [112/252] openlaoke/core/model_preloader.py

```
"""Model preloading and caching for fast switching.
Ollama supports keeping models in memory/VRAM for faster switching.
This module provides utilities to:
1. Preload models before execution
2. Keep models alive in memory
3. Optimize dual-model workflow
Memory requirements:
- gemma3:1b: 815 MB (planner/validator)
- gemma4:e4b: 12 GB (executor)
- Total: ~13 GB VRAM
Strategies:
1. Keep-alive: Keep models in memory between calls
2. Batch calls: Group multiple calls to same model
3. Parallel loading: Load both models upfront
"""
from __future__ import annotations
import time
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
import httpx
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class ModelLoadStatus:
    """Status of a loaded model."""
    name: str
    is_loaded: bool
    size_gb: float
    processor: str  # "GPU" or "CPU"
    context_length: int
    until: str | None
    def to_dict(self) -> dict[str, Any]:
        return {
            "name": self.name,
            "is_loaded": self.is_loaded,
            "size_gb": self.size_gb,
            "processor": self.processor,
            "context_length": self.context_length,
            "until": self.until,
        }
class ModelPreloader:
    """Preload and cache models for fast switching."""
    OLLAMA_API = "http://localhost:11434"
    DEFAULT_KEEP_ALIVE = "10m"  # Keep models alive for 10 minutes
    def __init__(self, app_state: AppState | None = None) -> None:
        self.app_state = app_state
        self._loaded_models: dict[str, ModelLoadStatus] = {}
    async def preload_models(self, models: list[str], keep_alive: str = "10m") -> dict[str, boo
    l]:
        """Preload multiple models into memory/VRAM.
        Args:
            models: List of model names to preload
            keep_alive: How long to keep models loaded (e.g., "10m", "1h", "24h")
        Returns:
            Dict of model_name -> success status
        """
        results = {}
        for model in models:
            try:
                success = await self._load_model(model, keep_alive)
                results[model] = success
```

```
        except Exception:
            results[model] = False
    return results
async def _load_model(self, model: str, keep_alive: str) -> bool:
    """Load a single model with keep_alive setting."""
    async with httpx.AsyncClient(timeout=60.0) as client:
        response = await client.post(
            f"{self.OLLAMA_API}/api/generate",
            json={
                "model": model,
                "prompt": "",  # Empty prompt to just load the model
                "keep_alive": keep_alive,
                "stream": False,
            },
        )
        if response.status_code == 200:
            self._loaded_models[model] = ModelLoadStatus(
                name=model,
                is_loaded=True,
                size_gb=0.0,
                processor="GPU",
                context_length=4096,
                until=None,
            )
            return True
        return False
async def get_loaded_models(self) -> list[ModelLoadStatus]:
    """Get list of currently loaded models."""
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            response = await client.get(f"{self.OLLAMA_API}/api/ps")
            if response.status_code == 200:
                data = response.json()
                models = []
                for model_data in data.get("models", []):
                    models.append(
                        ModelLoadStatus(
                            name=model_data.get("name", "unknown"),
                            is_loaded=True,
                            size_gb=model_data.get("size", 0) / (1024**3),
                            processor=model_data.get("details", {}).get("processor", "GPU")
,
                            context_length=model_data.get("details", {}).get(
                                "context_length", 4096
                            ),
                            until=model_data.get("expires", None),
                        )
                    )
                return models
    except Exception:
        pass
    return []
async def keep_alive(self, model: str, duration: str = "10m") -> bool:
    """Keep a model alive in memory for specified duration.
    Args:
        model: Model name
        duration: Duration string (e.g., "10m", "1h", "24h")
    Returns:
        Success status
    """
    return await self._load_model(model, duration)
async def unload_model(self, model: str) -> bool:
    """Unload a model from memory.
    Args:
        model: Model name to unload
```

```
    Returns:
        Success status
    """
    try:
        async with httpx.AsyncClient(timeout=10.0) as client:
            response = await client.post(
                f"{self.OLLAMA_API}/api/generate",
                json={
                    "model": model,
                    "keep_alive": 0,  # Set to 0 to unload
                    "stream": False,
                },
            )
            if response.status_code == 200:
                if model in self._loaded_models:
                    del self._loaded_models[model]
                return True
    except Exception:
        pass
    return False
async def check_memory_available(self) -> dict[str, Any]:
    """Check available memory for model loading.
    Returns:
        Dict with memory information
    """
    import subprocess
    try:
        result = subprocess.run(
            [
                "nvidia-smi",
                "--query-gpu=memory.total,memory.used,memory.free",
                "--format=csv,noheader,nounits",
            ],
            capture_output=True,
            text=True,
            timeout=5,
        )
        if result.returncode == 0:
            lines = result.stdout.strip().split("\n")
            gpus = []
            for line in lines:
                parts = line.split(", ")
                if len(parts) == 3:
                    total_mb = int(parts[0].strip())
                    used_mb = int(parts[1].strip())
                    free_mb = int(parts[2].strip())
                    gpus.append(
                        {
                            "total_gb": total_mb / 1024,
                            "used_gb": used_mb / 1024,
                            "free_gb": free_mb / 1024,
                            "usage_percent": (used_mb / total_mb) * 100 if total_mb > 0 els
e 0,
                        }
                    )
            return {
                "gpu_available": True,
                "gpus": gpus,
            }
    except Exception:
        pass
    return {
        "gpu_available": False,
        "gpus": [],
    }
```

```
    async def preload_dual_models(self) -> dict[str, Any]:
        """Preload both planner and executor models.
        Returns:
            Status report
        """
        start_time = time.time()
        planner_model = "gemma3:1b"
        executor_model = "gemma4:e4b"
        memory_info = await self.check_memory_available()
        loaded = await self.get_loaded_models()
        loaded_names = [m.name for m in loaded]
        results = {
            "planner": planner_model in loaded_names,
            "executor": executor_model in loaded_names,
        }
        if not results["planner"] or not results["executor"]:
            preload_results = await self.preload_models(
                [planner_model, executor_model],
                keep_alive=self.DEFAULT_KEEP_ALIVE,
            )
            results.update(preload_results)
        loaded_after = await self.get_loaded_models()
        total_time = time.time() - start_time
        return {
            "success": results.get("planner", False) and results.get("executor", False),
            "planner_loaded": results.get("planner", False),
            "executor_loaded": results.get("executor", False),
            "memory_info": memory_info,
            "loaded_models": [m.to_dict() for m in loaded_after],
            "load_time_seconds": total_time,
        }
async def preload_dual_models(app_state: AppState | None = None) -> dict[str, Any]:
    """Convenience function to preload dual models."""
    preloader = ModelPreloader(app_state)
    return await preloader.preload_dual_models()
async def get_model_status() -> list[dict[str, Any]]:
    """Convenience function to get loaded model status."""
    preloader = ModelPreloader()
    models = await preloader.get_loaded_models()
    return [m.to_dict() for m in models]
async def keep_model_alive(model: str, duration: str = "10m") -> bool:
    """Convenience function to keep a model alive."""
    preloader = ModelPreloader()
    return await preloader.keep_alive(model, duration)
```

File: [113/252] openlaoke/core/multi_agent/__init__.py

```
"""Multi-Agent collaboration system for OpenLaoKe."""
from __future__ import annotations
from openlaoke.core.multi_agent.agent_pool import AgentPool, AgentState, AgentType
from openlaoke.core.multi_agent.communication import (
    AgentMailbox,
    Message,
    MessagePriority,
    MessageType,
)
from openlaoke.core.multi_agent.conflict_resolution import (
    Conflict,
    ConflictResolution,
    ConflictResolver,
    ConflictType,
    ResolutionStrategy,
)
from openlaoke.core.multi_agent.coordinator import AgentCoordinator, GlobalState, Task
from openlaoke.core.multi_agent.task_distribution import (
    Assignment,
    AssignmentStrategy,
    Priority,
    TaskDistributor,
)
from openlaoke.core.multi_agent.team import (
    AgentRole,
    Team,
    TeamMember,
    TeamResult,
    TeamStatus,
)
__all__ = [
    "AgentCoordinator",
    "AgentPool",
    "AgentState",
    "AgentType",
    "AgentMailbox",
    "Message",
    "MessageType",
    "MessagePriority",
    "Conflict",
    "ConflictType",
    "ConflictResolution",
    "ConflictResolver",
    "ResolutionStrategy",
    "TaskDistributor",
    "Assignment",
    "AssignmentStrategy",
    "Priority",
    "Task",
    "Team",
    "TeamMember",
    "TeamResult",
    "AgentRole",
    "TeamStatus",
    "GlobalState",
]
```

File: [114/252] openlaoke/core/multi_agent/agent_pool.py

```
"""Agent pool for managing agent lifecycle and resources."""
from __future__ import annotations
import asyncio
import logging
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
from uuid import uuid4
logger = logging.getLogger(__name__)
class AgentType(StrEnum):
    """Types of agents in the system."""
    COORDINATOR = "coordinator"
    EXPLORER = "explorer"
    IMPLEMENTER = "implementer"
    REVIEWER = "reviewer"
    TESTER = "tester"
    RESEARCHER = "researcher"
    ANALYZER = "analyzer"
@dataclass
class AgentState:
    """Runtime state of an agent."""
    agent_id: str
    agent_type: AgentType
    status: str = "idle"
    current_task: str | None = None
    start_time: float = field(default_factory=time.time)
    last_activity: float = field(default_factory=time.time)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "agent_id": self.agent_id,
            "agent_type": self.agent_type.value,
            "status": self.status,
            "current_task": self.current_task,
            "start_time": self.start_time,
            "last_activity": self.last_activity,
            "metadata": self.metadata,
        }
    def update_activity(self) -> None:
        """Update last activity timestamp."""
        self.last_activity = time.time()
class AgentPool:
    """Manages a pool of agents with lifecycle and resource management.
    This class provides:
    - Agent lifecycle management (acquire, release)
    - Concurrent agent limit enforcement
    - Timeout control
    - Resource isolation
    """
    def __init__(
        self,
        max_agents: int = 3,
        default_timeout: int = 900,
    ):
        """Initialize the agent pool.
        Args:
            max_agents: Maximum number of concurrent agents (default: 3).
            default_timeout: Default timeout for agent execution in seconds (default: 900 = 15
    minutes).
        """
```

```
    self.max_agents = max_agents
    self.default_timeout = default_timeout
    self._agents: dict[str, AgentState] = {}
    self._lock = asyncio.Lock()
    self._semaphore = asyncio.Semaphore(max_agents)
@property
def active_count(self) -> int:
    """Get the count of active agents."""
    return len([a for a in self._agents.values() if a.status == "active"])
@property
def total_count(self) -> int:
    """Get the total count of agents (active + idle)."""
    return len(self._agents)
async def acquire(
    self,
    agent_type: str | AgentType,
    task: Any | None = None,
    config: dict[str, Any] | None = None,
    timeout: int | None = None,
) -> str:
    """Acquire an agent from the pool.
    Args:
        agent_type: Type of agent to acquire.
        task: Optional task to assign to the agent.
        config: Optional configuration for the agent.
        timeout: Optional timeout in seconds.
    Returns:
        Agent ID of the acquired agent.
    Raises:
        RuntimeError: If maximum agents reached or acquisition fails.
    """
    if isinstance(agent_type, str):
        try:
            agent_type = AgentType(agent_type)
        except ValueError:
            agent_type = AgentType.IMPLEMENTER
    timeout = timeout or self.default_timeout
    try:
        await asyncio.wait_for(self._semaphore.acquire(), timeout=timeout)
    except TimeoutError as err:
        raise RuntimeError(
            f"Timeout waiting for available agent slot (max: {self.max_agents})"
        ) from err
    async with self._lock:
        agent_id = f"agent_{uuid4().hex[:8]}"
        state = AgentState(
            agent_id=agent_id,
            agent_type=agent_type,
            status="active",
            current_task=task.id if task and hasattr(task, "id") else None,
            metadata=config or {},
        )
        self._agents[agent_id] = state
        logger.info(f"Acquired agent {agent_id} of type {agent_type.value}")
        return agent_id
async def release(self, agent_id: str) -> None:
    """Release an agent back to the pool.
    Args:
        agent_id: ID of the agent to release.
    """
    async with self._lock:
        if agent_id not in self._agents:
            logger.warning(f"Attempted to release unknown agent {agent_id}")
            return
        state = self._agents[agent_id]
```

```
        state.status = "released"
        state.current_task = None
        del self._agents[agent_id]
        self._semaphore.release()
        logger.info(f"Released agent {agent_id}")
async def release_all(self) -> None:
    """Release all agents in the pool."""
    async with self._lock:
        agent_ids = list(self._agents.keys())
    for agent_id in agent_ids:
        await self.release(agent_id)
    logger.info(f"Released all {len(agent_ids)} agents")
def get_agent(self, agent_id: str) -> AgentState | None:
    """Get an agent's state by ID.
    Args:
        agent_id: Agent ID to look up.
    Returns:
        AgentState if found, None otherwise.
    """
    return self._agents.get(agent_id)
def list_active(self) -> list[AgentState]:
    """List all active agents.
    Returns:
        List of active agent states.
    """
    return [a for a in self._agents.values() if a.status == "active"]
def list_all(self) -> list[AgentState]:
    """List all agents in the pool.
    Returns:
        List of all agent states.
    """
    return list(self._agents.values())
async def update_agent_status(
    self,
    agent_id: str,
    status: str,
    task: Any | None = None,
) -> None:
    """Update an agent's status.
    Args:
        agent_id: Agent ID to update.
        status: New status value.
        task: Optional task to assign.
    """
    async with self._lock:
        if agent_id not in self._agents:
            return
        state = self._agents[agent_id]
        state.status = status
        state.update_activity()
        if task:
            state.current_task = task.id if hasattr(task, "id") else str(task)
def get_stats(self) -> dict[str, Any]:
    """Get pool statistics.
    Returns:
        Dictionary with pool statistics.
    """
    agents = list(self._agents.values())
    return {
        "max_agents": self.max_agents,
        "total_count": len(agents),
        "active_count": len([a for a in agents if a.status == "active"]),
        "idle_count": len([a for a in agents if a.status == "idle"]),
        "agents_by_type": self._count_by_type(agents),
    }
```

```
def _count_by_type(self, agents: list[AgentState]) -> dict[str, int]:
    """Count agents by type.
    Args:
        agents: List of agent states.
    Returns:
        Dictionary with count by agent type.
    """
    counts: dict[str, int] = {}
    for agent in agents:
        type_name = agent.agent_type.value
        counts[type_name] = counts.get(type_name, 0) + 1
    return counts
```

File: [115/252] openlaoke/core/multi_agent/communication.py

```
"""Inter-agent communication system."""
from __future__ import annotations
import asyncio
import json
import logging
import time
from collections import defaultdict
from dataclasses import dataclass, field
from datetime import datetime
from enum import StrEnum
from pathlib import Path
from typing import Any
from uuid import uuid4
logger = logging.getLogger(__name__)
class MessageType(StrEnum):
    """Types of messages in the agent communication system."""
    TASK = "task"
    RESULT = "result"
    ERROR = "error"
    SYSTEM = "system"
    BROADCAST = "broadcast"
    QUERY = "query"
    RESPONSE = "response"
    HEARTBEAT = "heartbeat"
    CONTROL = "control"
class MessagePriority(StrEnum):
    """Priority levels for messages."""
    LOW = "low"
    NORMAL = "normal"
    HIGH = "high"
    URGENT = "urgent"
@dataclass
class Message:
    """Message structure for inter-agent communication."""
    sender: str
    content: Any
    message_type: MessageType = MessageType.TASK
    recipients: list[str] = field(default_factory=list)
    priority: MessagePriority = MessagePriority.NORMAL
    message_id: str = field(default_factory=lambda: f"msg_{uuid4().hex[:8]}")
    timestamp: float = field(default_factory=time.time)
    correlation_id: str | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "sender": self.sender,
            "content": self.content,
            "message_type": self.message_type.value,
            "recipients": self.recipients,
            "priority": self.priority.value,
            "message_id": self.message_id,
            "timestamp": self.timestamp,
            "correlation_id": self.correlation_id,
            "metadata": self.metadata,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> Message:
        return cls(
            sender=data["sender"],
            content=data["content"],
            message_type=MessageType(data["message_type"]),
```

```
            recipients=data.get("recipients", []),
            priority=MessagePriority(data.get("priority", "normal")),
            message_id=data["message_id"],
            timestamp=data["timestamp"],
            correlation_id=data.get("correlation_id"),
            metadata=data.get("metadata", {}),
        )
class AgentMailbox:
    """Message mailbox for inter-agent communication.
    Provides:
    - Message queue per agent
    - Publish-subscribe pattern
    - Broadcast notifications
    - Message persistence
    """
    def __init__(
        self,
        persist_dir: str | None = None,
        max_queue_size: int = 1000,
    ):
        """Initialize the mailbox system.
        Args:
            persist_dir: Directory for message persistence (optional).
            max_queue_size: Maximum messages per queue.
        """
        self.persist_dir = Path(persist_dir) if persist_dir else None
        self.max_queue_size = max_queue_size
        self._queues: dict[str, asyncio.Queue] = defaultdict(
            lambda: asyncio.Queue(maxsize=max_queue_size)
        )
        self._subscribers: dict[str, list[str]] = defaultdict(list)
        self._message_history: dict[str, list[Message]] = defaultdict(list)
        self._lock = asyncio.Lock()
        if self.persist_dir:
            self.persist_dir.mkdir(parents=True, exist_ok=True)
            self._load_history()
    async def send(self, message: Message) -> None:
        """Send a message to recipients.
        Args:
            message: Message to send.
        """
        async with self._lock:
            for recipient in message.recipients:
                queue = self._queues[recipient]
                if queue.full():
                    await queue.get()
                    logger.warning(f"Queue full for {recipient}, dropped oldest message")
                await queue.put(message)
                self._message_history[recipient].append(message)
                if len(self._message_history[recipient]) > self.max_queue_size:
                    self._message_history[recipient] = self._message_history[recipient][
                        -self.max_queue_size :
                    ]
            if message.correlation_id:
                corr_queue = self._queues[f"corr_{message.correlation_id}"]
                if not corr_queue.full():
                    await corr_queue.put(message)
            logger.debug(
                f"Message {message.message_id} sent to {len(message.recipients)} recipients"
            )
            if self.persist_dir:
                self._persist_message(message)
    async def receive(
        self,
        agent_id: str,
```

```
    timeout: float | None = None,
) -> Message | None:
    """Receive a message for an agent.
    Args:
        agent_id: Agent ID to receive message for.
        timeout: Optional timeout in seconds.
    Returns:
        Message if available, None otherwise.
    """
    queue = self._queues[agent_id]
    try:
        if timeout:
            msg = await asyncio.wait_for(queue.get(), timeout=timeout)
        else:
            msg = await queue.get()
        if isinstance(msg, Message):
            return msg
        return None
    except TimeoutError:
        return None
async def get_messages(
    self,
    agent_id: str,
    message_type: MessageType | None = None,
    limit: int | None = None,
) -> list[Message]:
    """Get messages for an agent from history.
    Args:
        agent_id: Agent ID to get messages for.
        message_type: Optional message type filter.
        limit: Optional limit on number of messages.
    Returns:
        List of messages.
    """
    async with self._lock:
        messages = self._message_history.get(agent_id, [])
        if message_type:
            messages = [m for m in messages if m.message_type == message_type]
        if limit:
            messages = messages[-limit:]
        return messages
async def broadcast(
    self,
    sender: str,
    content: Any,
    message_type: MessageType = MessageType.BROADCAST,
    priority: MessagePriority = MessagePriority.NORMAL,
) -> str:
    """Broadcast a message to all subscribers.
    Args:
        sender: Sender agent ID.
        content: Message content.
        message_type: Message type.
        priority: Message priority.
    Returns:
        Message ID.
    """
    subscribers = self._subscribers.get(sender, [])
    message = Message(
        sender=sender,
        content=content,
        message_type=message_type,
        recipients=subscribers,
        priority=priority,
    )
```

```
    await self.send(message)
    return message.message_id
def subscribe(self, subscriber_id: str, topic: str) -> None:
    """Subscribe an agent to a topic.
    Args:
        subscriber_id: Agent ID to subscribe.
        topic: Topic to subscribe to.
    """
    if subscriber_id not in self._subscribers[topic]:
        self._subscribers[topic].append(subscriber_id)
        logger.info(f"Agent {subscriber_id} subscribed to {topic}")
def unsubscribe(self, subscriber_id: str, topic: str) -> None:
    """Unsubscribe an agent from a topic.
    Args:
        subscriber_id: Agent ID to unsubscribe.
        topic: Topic to unsubscribe from.
    """
    if subscriber_id in self._subscribers[topic]:
        self._subscribers[topic].remove(subscriber_id)
        logger.info(f"Agent {subscriber_id} unsubscribed from {topic}")
async def send_and_wait(
    self,
    sender: str,
    recipient: str,
    content: Any,
    timeout: float = 60.0,
) -> Message | None:
    """Send a message and wait for response.
    Args:
        sender: Sender agent ID.
        recipient: Recipient agent ID.
        content: Message content.
        timeout: Timeout in seconds.
    Returns:
        Response message if received.
    """
    correlation_id = f"corr_{uuid4().hex[:8]}"
    message = Message(
        sender=sender,
        content=content,
        recipients=[recipient],
        correlation_id=correlation_id,
    )
    await self.send(message)
    corr_queue = self._queues[f"corr_{correlation_id}"]
    try:
        response = await asyncio.wait_for(corr_queue.get(), timeout=timeout)
        if isinstance(response, Message):
            return response
        return None
    except TimeoutError:
        logger.warning(f"Timeout waiting for response to {correlation_id}")
        return None
def clear_history(self, agent_id: str | None = None) -> None:
    """Clear message history.
    Args:
        agent_id: Optional agent ID to clear history for. If None, clears all.
    """
    if agent_id:
        self._message_history[agent_id] = []
    else:
        self._message_history.clear()
def get_stats(self) -> dict[str, Any]:
    """Get mailbox statistics.
    Returns:
```

```
        Dictionary with mailbox statistics.
    """
    return {
        "queue_count": len(self._queues),
        "total_messages": sum(len(msgs) for msgs in self._message_history.values()),
        "subscriber_count": sum(len(subs) for subs in self._subscribers.values()),
        "topic_count": len(self._subscribers),
    }
def _persist_message(self, message: Message) -> None:
    """Persist a message to disk.
    Args:
        message: Message to persist.
    """
    if not self.persist_dir:
        return
    try:
        date_str = datetime.fromtimestamp(message.timestamp).strftime("%Y-%m-%d")
        persist_file = self.persist_dir / f"messages_{date_str}.jsonl"
        with open(persist_file, "a", encoding="utf-8") as f:
            f.write(json.dumps(message.to_dict()) + "\n")
    except Exception as e:
        logger.error(f"Failed to persist message: {e}")
def _load_history(self) -> None:
    """Load message history from disk."""
    if not self.persist_dir:
        return
    try:
        for msg_file in self.persist_dir.glob("messages_*.jsonl"):
            with open(msg_file, encoding="utf-8") as f:
                for line in f:
                    if line.strip():
                        try:
                            msg_data = json.loads(line)
                            message = Message.from_dict(msg_data)
                            for recipient in message.recipients:
                                self._message_history[recipient].append(message)
                        except Exception as e:
                            logger.error(f"Failed to load message: {e}")
    except Exception as e:
        logger.error(f"Failed to load message history: {e}")
```

File: [116/252] openlaoke/core/multi_agent/conflict_resolution.py

```
"""Conflict resolution system for multi-agent coordination."""
from __future__ import annotations
import logging
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
from uuid import uuid4
logger = logging.getLogger(__name__)
class ConflictType(StrEnum):
    """Types of conflicts that can occur."""
    RESOURCE = "resource"
    TASK = "task"
    DECISION = "decision"
    COMMUNICATION = "communication"
    STATE = "state"
    PRIORITY = "priority"
class ResolutionStrategy(StrEnum):
    """Strategies for resolving conflicts."""
    FIRST_WINS = "first_wins"
    LAST_WINS = "last_wins"
    PRIORITY_WINS = "priority_wins"
    VOTING = "voting"
    COORDINATOR_DECIDES = "coordinator_decides"
    MERGE = "merge"
    COMPROMISE = "compromise"
    ESCALATE = "escalate"
@dataclass
class Conflict:
    """Represents a conflict between agents.
    Attributes:
        id: Unique conflict identifier.
        conflict_type: Type of conflict.
        agents_involved: IDs of agents involved in the conflict.
        description: Human-readable description.
        resources: Resources involved in the conflict.
        created_at: When the conflict was detected.
        metadata: Additional conflict metadata.
    """
    id: str = field(default_factory=lambda: f"conflict_{uuid4().hex[:8]}")
    conflict_type: ConflictType = ConflictType.TASK
    agents_involved: list[str] = field(default_factory=list)
    description: str = ""
    resources: list[str] = field(default_factory=list)
    created_at: float = field(default_factory=time.time)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "conflict_type": self.conflict_type.value,
            "agents_involved": self.agents_involved,
            "description": self.description,
            "resources": self.resources,
            "created_at": self.created_at,
            "metadata": self.metadata,
        }
@dataclass
class ConflictResolution:
    """Represents the resolution of a conflict.
    Attributes:
        conflict_id: ID of the resolved conflict.
```

```
        strategy: Strategy used to resolve.
        resolution: The actual resolution outcome.
        winner: Agent that won the conflict (if applicable).
        resolved_at: When the conflict was resolved.
        metadata: Additional resolution metadata.
    """
    conflict_id: str = ""
    strategy: ResolutionStrategy = ResolutionStrategy.COORDINATOR_DECIDES
    resolution: str = ""
    winner: str | None = None
    resolved_at: float = field(default_factory=time.time)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "conflict_id": self.conflict_id,
            "strategy": self.strategy.value,
            "resolution": self.resolution,
            "winner": self.winner,
            "resolved_at": self.resolved_at,
            "metadata": self.metadata,
        }
class ConflictResolver:
    """Resolves conflicts between agents.
    Provides multiple resolution strategies and tracks conflict history.
    """
    def __init__(
        self,
        default_strategy: ResolutionStrategy = ResolutionStrategy.COORDINATOR_DECIDES,
        max_history: int = 100,
    ):
        """Initialize the conflict resolver.
        Args:
            default_strategy: Default resolution strategy.
            max_history: Maximum number of conflicts to keep in history.
        """
        self.default_strategy = default_strategy
        self.max_history = max_history
        self._conflicts: list[Conflict] = []
        self._resolutions: list[ConflictResolution] = []
        self._pending: dict[str, Conflict] = {}
    async def detect(
        self,
        agents: list[str],
        resources: list[str] | None = None,
        conflict_type: ConflictType | None = None,
    ) -> Conflict | None:
        """Detect a potential conflict between agents.
        Args:
            agents: List of agent IDs to check.
            resources: Optional list of resources.
            conflict_type: Optional conflict type.
        Returns:
            Detected conflict or None.
        """
        if len(agents) < 2:
            return None
        if not conflict_type:
            conflict_type = ConflictType.RESOURCE if resources else ConflictType.TASK
        conflict = Conflict(
            conflict_type=conflict_type,
            agents_involved=agents,
            resources=resources or [],
            description=f"Potential {conflict_type.value} conflict detected",
        )
        self._pending[conflict.id] = conflict
```

```
    logger.info(f"Detected conflict {conflict.id} between agents {agents}")
    return conflict
async def resolve(
    self,
    conflict: Conflict,
    strategy: ResolutionStrategy | None = None,
    context: dict[str, Any] | None = None,
) -> ConflictResolution:
    """Resolve a conflict using the specified strategy.
    Args:
        conflict: Conflict to resolve.
        strategy: Optional override strategy.
        context: Additional context for resolution.
    Returns:
        Conflict resolution.
    """
    strategy = strategy or self.default_strategy
    resolution = ConflictResolution(
        conflict_id=conflict.id,
        strategy=strategy,
    )
    resolution = await self._apply_strategy(conflict, resolution, strategy, context or {})
    self._conflicts.append(conflict)
    self._resolutions.append(resolution)
    if conflict.id in self._pending:
        del self._pending[conflict.id]
    if len(self._conflicts) > self.max_history:
        self._conflicts = self._conflicts[-self.max_history :]
    if len(self._resolutions) > self.max_history:
        self._resolutions = self._resolutions[-self.max_history :]
    logger.info(f"Resolved conflict {conflict.id} using {strategy.value} strategy")
    return resolution
async def _apply_strategy(
    self,
    conflict: Conflict,
    resolution: ConflictResolution,
    strategy: ResolutionStrategy,
    context: dict[str, Any],
) -> ConflictResolution:
    """Apply a resolution strategy.
    Args:
        conflict: Conflict to resolve.
        resolution: Resolution object to populate.
        strategy: Strategy to apply.
        context: Resolution context.
    Returns:
        Populated resolution.
    """
    if strategy == ResolutionStrategy.FIRST_WINS:
        resolution.winner = conflict.agents_involved[0]
        resolution.resolution = f"First agent {resolution.winner} wins"
    elif strategy == ResolutionStrategy.LAST_WINS:
        resolution.winner = conflict.agents_involved[-1]
        resolution.resolution = f"Last agent {resolution.winner} wins"
    elif strategy == ResolutionStrategy.PRIORITY_WINS:
        priorities = context.get("priorities", {})
        winner = max(
            conflict.agents_involved,
            key=lambda a: priorities.get(a, 0),
        )
        resolution.winner = winner
        resolution.resolution = f"Highest priority agent {winner} wins"
    elif strategy == ResolutionStrategy.VOTING:
        votes = context.get("votes", {})
        winner = max(
```

```
            conflict.agents_involved,
            key=lambda a: votes.get(a, 0),
        )
        resolution.winner = winner
        resolution.resolution = f"Agent {winner} wins by vote"
    elif strategy == ResolutionStrategy.COORDINATOR_DECIDES:
        decision = context.get("coordinator_decision", conflict.agents_involved[0])
        resolution.winner = decision
        resolution.resolution = f"Coordinator decided: agent {decision} wins"
    elif strategy == ResolutionStrategy.MERGE:
        resolution.resolution = "Resources merged between agents"
        resolution.metadata["merged_resources"] = conflict.resources
    elif strategy == ResolutionStrategy.COMPROMISE:
        resolution.resolution = "Agents agreed to compromise"
        resolution.metadata["compromise_terms"] = context.get("compromise_terms", [])
    elif strategy == ResolutionStrategy.ESCALATE:
        resolution.resolution = "Conflict escalated to higher authority"
        resolution.metadata["escalated_to"] = context.get("escalated_to", "system")
    return resolution
def get_conflict(self, conflict_id: str) -> Conflict | None:
    """Get a conflict by ID.
    Args:
        conflict_id: Conflict ID to look up.
    Returns:
        Conflict if found, None otherwise.
    """
    for conflict in self._conflicts:
        if conflict.id == conflict_id:
            return conflict
    return self._pending.get(conflict_id)
def get_resolution(self, conflict_id: str) -> ConflictResolution | None:
    """Get a resolution by conflict ID.
    Args:
        conflict_id: Conflict ID to look up.
    Returns:
        Resolution if found, None otherwise.
    """
    for resolution in self._resolutions:
        if resolution.conflict_id == conflict_id:
            return resolution
    return None
def get_pending_conflicts(self) -> list[Conflict]:
    """Get all pending conflicts.
    Returns:
        List of pending conflicts.
    """
    return list(self._pending.values())
def get_history(self, limit: int | None = None) -> list[tuple[Conflict, ConflictResolution]
]:
    """Get conflict resolution history.
    Args:
        limit: Optional limit on number of entries.
    Returns:
        List of (conflict, resolution) tuples.
    """
    history = list(zip(self._conflicts, self._resolutions, strict=False))
    if limit:
        return history[-limit:]
    return history
def get_stats(self) -> dict[str, Any]:
    """Get conflict resolution statistics.
    Returns:
        Dictionary with statistics.
    """
    type_counts: dict[str, int] = {}
```

```
    for conflict in self._conflicts:
        type_key = conflict.conflict_type.value
        type_counts[type_key] = type_counts.get(type_key, 0) + 1
    strategy_counts: dict[str, int] = {}
    for resolution in self._resolutions:
        strategy_key = resolution.strategy.value
        strategy_counts[strategy_key] = strategy_counts.get(strategy_key, 0) + 1
    return {
        "total_conflicts": len(self._conflicts),
        "pending_conflicts": len(self._pending),
        "by_type": type_counts,
        "by_strategy": strategy_counts,
    }
def clear_history(self) -> None:
    """Clear conflict resolution history."""
    self._conflicts.clear()
    self._resolutions.clear()
    self._pending.clear()
```

File: [117/252] openlaoke/core/multi_agent/coordinator.py

```
"""Multi-Agent coordinator for managing agent collaboration."""
from __future__ import annotations
import asyncio
import logging
import time
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from uuid import uuid4
from openlaoke.core.multi_agent.agent_pool import AgentPool
from openlaoke.core.multi_agent.communication import AgentMailbox, Message, MessageType
from openlaoke.core.multi_agent.conflict_resolution import (
    Conflict,
    ConflictResolution,
    ConflictResolver,
)
from openlaoke.core.multi_agent.task_distribution import Assignment, Priority, TaskDistributor
from openlaoke.core.multi_agent.team import Team, TeamResult
from openlaoke.types.core_types import TaskStatus, TaskType
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
logger = logging.getLogger(__name__)
MAX_CONCURRENT_AGENTS = 3
DEFAULT_TIMEOUT = 15 * 60
@dataclass
class GlobalState:
    """Global state shared across all agents."""
    session_id: str = ""
    active_agents: int = 0
    total_tasks: int = 0
    completed_tasks: int = 0
    failed_tasks: int = 0
    start_time: float = field(default_factory=time.time)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "session_id": self.session_id,
            "active_agents": self.active_agents,
            "total_tasks": self.total_tasks,
            "completed_tasks": self.completed_tasks,
            "failed_tasks": self.failed_tasks,
            "start_time": self.start_time,
            "metadata": self.metadata,
        }
@dataclass
class Task:
    """Task representation for multi-agent system."""
    id: str = field(default_factory=lambda: f"task_{uuid4().hex[:8]}")
    description: str = ""
    task_type: TaskType = TaskType.LOCAL_AGENT
    priority: Priority = Priority.MEDIUM
    assigned_agent: str | None = None
    status: TaskStatus = TaskStatus.PENDING
    result: Any = None
    error: str | None = None
    start_time: float | None = None
    end_time: float | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
class AgentCoordinator:
    """Multi-Agent coordinator for managing agent collaboration.
    This class orchestrates multiple agents working together on complex tasks.
```

```
It manages agent lifecycle, task distribution, communication, and conflict resolution.
"""
def __init__(
    self,
    max_agents: int = MAX_CONCURRENT_AGENTS,
    timeout: int = DEFAULT_TIMEOUT,
    app_state: AppState | None = None,
):
    """Initialize the coordinator.
    Args:
        max_agents: Maximum number of concurrent agents.
        timeout: Default timeout for agent execution in seconds.
        app_state: Optional application state for integration.
    """
    self.max_agents = max_agents
    self.timeout = timeout
    self.app_state = app_state
    self.agent_pool = AgentPool(max_agents=max_agents)
    self.mailbox = AgentMailbox()
    self.task_distributor = TaskDistributor()
    self.conflict_resolver = ConflictResolver()
    self._task_queue: asyncio.PriorityQueue[tuple[Priority, Task]] = asyncio.PriorityQueue(
)
    self._tasks: dict[str, Task] = {}
    self._assignments: dict[str, Assignment] = {}
    self._global_state = GlobalState(session_id=f"coord_{uuid4().hex[:8]}")
    self._executor = ThreadPoolExecutor(max_workers=max_agents, thread_name_prefix="agent-"
)
    self._running = False
    self._lock = asyncio.Lock()
async def spawn_agent(
    self,
    agent_type: str,
    task: Task,
    config: dict[str, Any] | None = None,
) -> str:
    """Spawn a new agent for a specific task.
    Args:
        agent_type: Type of agent to spawn (coordinator, explorer, implementer, etc.).
        task: Task for the agent to execute.
        config: Optional agent configuration.
    Returns:
        Agent ID of the spawned agent.
    """
    async with self._lock:
        if self.agent_pool.active_count >= self.max_agents:
            raise RuntimeError(f"Maximum concurrent agents ({self.max_agents}) reached")
        agent_id = await self.agent_pool.acquire(agent_type, task, config or {})
        self._global_state.active_agents = self.agent_pool.active_count
        logger.info(f"Spawned agent {agent_id} of type {agent_type} for task {task.id}")
        return agent_id
async def dispatch_task(self, task: Task) -> Assignment:
    """Dispatch a task to an available agent.
    Args:
        task: Task to dispatch.
    Returns:
        Assignment details.
    """
    task.status = TaskStatus.PENDING
    task.start_time = time.time()
    self._tasks[task.id] = task
    self._global_state.total_tasks += 1
    assignment = await self.task_distributor.distribute(task, self.agent_pool)
    self._assignments[task.id] = assignment
    if assignment.agent_id:
```

```
        task.assigned_agent = assignment.agent_id
        task.status = TaskStatus.RUNNING
        await self.mailbox.send(
            Message(
                sender="coordinator",
                recipients=[assignment.agent_id],
                content=f"Task assigned: {task.description}",
                message_type=MessageType.TASK,
                metadata={"task_id": task.id},
            )
        )
        logger.info(f"Dispatched task {task.id} to agent {assignment.agent_id}")
    return assignment
async def collect_results(self, task_id: str) -> list[Any]:
    """Collect results from all agents working on a task.
    Args:
        task_id: Task ID to collect results for.
    Returns:
        List of results from agents.
    """
    results: list[Any] = []
    messages = await self.mailbox.get_messages(f"result_{task_id}")
    for msg in messages:
        if msg.message_type == MessageType.RESULT:
            results.append(msg.content)
    task = self._tasks.get(task_id)
    if task:
        task.status = TaskStatus.COMPLETED
        task.end_time = time.time()
        if results:
            task.result = results
        self._global_state.completed_tasks += 1
    return results
async def resolve_conflicts(self, conflicts: list[Conflict]) -> list[ConflictResolution]:
    """Resolve conflicts between agents.
    Args:
        conflicts: List of conflicts to resolve.
    Returns:
        List of conflict resolutions.
    """
    resolutions = []
    for conflict in conflicts:
        resolution = await self.conflict_resolver.resolve(conflict)
        resolutions.append(resolution)
        await self.broadcast(
            Message(
                sender="coordinator",
                content=f"Conflict resolved: {resolution.resolution}",
                message_type=MessageType.SYSTEM,
                metadata={"conflict_id": conflict.id, "resolution": resolution.to_dict()},
            )
        )
        logger.info(f"Resolved conflict {conflict.id}: {resolution.strategy.value}")
    return resolutions
async def coordinate_team(self, team: Team) -> TeamResult:
    """Coordinate a team of agents working together.
    Args:
        team: Team configuration with members and roles.
    Returns:
        Team execution result.
    """
    team_tasks: list[Task] = []
    for member in team.members:
        task = Task(
            description=f"Team task for {member.role.value}",
```

```
            priority=Priority.HIGH,
            metadata={"team_id": team.id, "role": member.role.value},
        )
        team_tasks.append(task)
    assignments: list[Assignment] = []
    for task in team_tasks:
        assignment = await self.dispatch_task(task)
        assignments.append(assignment)
    await asyncio.gather(*[self._wait_for_task(t.id) for t in team_tasks])
    results = {}
    for task in team_tasks:
        task_results = await self.collect_results(task.id)
        results[task.id] = task_results
    success = all(t.status == TaskStatus.COMPLETED for t in team_tasks)
    return TeamResult(
        team_id=team.id,
        success=success,
        results=results,
        assignments={a.task_id: a for a in assignments},
    )
async def broadcast(self, message: Message) -> None:
    """Broadcast a message to all active agents.
    Args:
        message: Message to broadcast.
    """
    active_agents = self.agent_pool.list_active()
    message.recipients = [a.agent_id for a in active_agents]
    await self.mailbox.send(message)
    logger.debug(f"Broadcast message to {len(message.recipients)} agents")
async def sync_state(self) -> GlobalState:
    """Synchronize and return the global state.
    Returns:
        Current global state.
    """
    self._global_state.active_agents = self.agent_pool.active_count
    return self._global_state
async def shutdown(self) -> None:
    """Gracefully shutdown the coordinator and all agents."""
    self._running = False
    await self.agent_pool.release_all()
    self._executor.shutdown(wait=True)
    logger.info("Coordinator shutdown complete")
async def _wait_for_task(self, task_id: str, timeout: float | None = None) -> None:
    """Wait for a task to complete.
    Args:
        task_id: Task ID to wait for.
        timeout: Optional timeout in seconds.
    """
    timeout = timeout or self.timeout
    start = time.time()
    while True:
        task = self._tasks.get(task_id)
        if not task:
            return
        if task.status in (TaskStatus.COMPLETED, TaskStatus.FAILED, TaskStatus.KILLED):
            return
        if time.time() - start > timeout:
            task.status = TaskStatus.FAILED
            task.error = "Task timeout"
            logger.error(f"Task {task_id} timed out after {timeout}s")
            return
        await asyncio.sleep(0.1)
```

File: [118/252] openlaoke/core/multi_agent/task_distribution.py

```
"""Task distribution system for multi-agent coordination."""
from __future__ import annotations
import logging
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from uuid import uuid4
if TYPE_CHECKING:
    from openlaoke.core.multi_agent.agent_pool import AgentPool, AgentState
    from openlaoke.core.multi_agent.coordinator import Task
logger = logging.getLogger(__name__)
class Priority(StrEnum):
    """Priority levels for tasks."""
    LOW = "low"
    MEDIUM = "medium"
    HIGH = "high"
    URGENT = "urgent"
    def __lt__(self, other: str) -> bool:
        if isinstance(other, Priority):
            order = {"low": 0, "medium": 1, "high": 2, "urgent": 3}
            return order[self.value] < order[other.value]
        return NotImplemented
    def __le__(self, other: str) -> bool:
        if isinstance(other, Priority):
            return self == other or self < other
        return NotImplemented
    def __gt__(self, other: str) -> bool:
        if isinstance(other, Priority):
            return not self <= other
        return NotImplemented
    def __ge__(self, other: str) -> bool:
        if isinstance(other, Priority):
            return not self < other
        return NotImplemented
class AssignmentStrategy(StrEnum):
    """Strategies for task assignment."""
    FIRST_AVAILABLE = "first_available"
    ROUND_ROBIN = "round_robin"
    LEAST_LOADED = "least_loaded"
    SPECIALIZED = "specialized"
    PRIORITY_BASED = "priority_based"
@dataclass
class Assignment:
    """Task assignment details."""
    assignment_id: str = field(default_factory=lambda: f"assign_{uuid4().hex[:8]}")
    task_id: str = ""
    agent_id: str | None = None
    strategy: AssignmentStrategy = AssignmentStrategy.FIRST_AVAILABLE
    timestamp: float = field(default_factory=time.time)
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "assignment_id": self.assignment_id,
            "task_id": self.task_id,
            "agent_id": self.agent_id,
            "strategy": self.strategy.value,
            "timestamp": self.timestamp,
            "metadata": self.metadata,
        }
@dataclass
```

```
class TaskDistributorConfig:
    """Configuration for task distributor."""
    strategy: AssignmentStrategy = AssignmentStrategy.LEAST_LOADED
    max_retries: int = 3
    retry_delay: float = 5.0
    priority_boost_timeout: float = 60.0
class TaskDistributor:
    """Distributes tasks among available agents.
    Provides multiple distribution strategies:
    - First available: Assign to first available agent
    - Round robin: Distribute evenly in rotation
    - Least loaded: Assign to agent with least tasks
    - Specialized: Match task type to agent specialization
    - Priority based: High priority tasks get dedicated agents
    """
    def __init__(self, config: TaskDistributorConfig | None = None):
        """Initialize the task distributor.
        Args:
            config: Optional distributor configuration.
        """
        self.config = config or TaskDistributorConfig()
        self._round_robin_index = 0
        self._assignment_history: list[Assignment] = []
    async def distribute(
        self,
        task: Task,
        agent_pool: AgentPool,
        strategy: AssignmentStrategy | None = None,
    ) -> Assignment:
        """Distribute a task to an available agent.
        Args:
            task: Task to distribute.
            agent_pool: Agent pool to distribute to.
            strategy: Optional override strategy.
        Returns:
            Assignment details.
        """
        strategy = strategy or self.config.strategy
        assignment = Assignment(
            task_id=task.id,
            strategy=strategy,
        )
        active_agents = agent_pool.list_active()
        if not active_agents:
            assignment.agent_id = None
            assignment.metadata["reason"] = "No available agents"
            logger.warning(f"No available agents for task {task.id}")
            self._assignment_history.append(assignment)
            return assignment
        agent_id = await self._select_agent(task, active_agents, strategy)
        if agent_id:
            assignment.agent_id = agent_id
            logger.info(
                f"Assigned task {task.id} to agent {agent_id} using {strategy.value} strategy"
            )
        else:
            assignment.agent_id = None
            assignment.metadata["reason"] = "Agent selection failed"
            logger.warning(f"Failed to select agent for task {task.id}")
        self._assignment_history.append(assignment)
        return assignment
    async def _select_agent(
        self,
        task: Task,
        active_agents: list[AgentState],
```

```
    strategy: AssignmentStrategy,
) -> str | None:
    """Select an agent based on the distribution strategy.
    Args:
        task: Task to assign.
        active_agents: List of active agent states.
        strategy: Distribution strategy.
    Returns:
        Selected agent ID or None.
    """
    if not active_agents:
        return None
    if strategy == AssignmentStrategy.FIRST_AVAILABLE:
        return str(active_agents[0].agent_id)
    if strategy == AssignmentStrategy.ROUND_ROBIN:
        self._round_robin_index = (self._round_robin_index + 1) % len(active_agents)
        return str(active_agents[self._round_robin_index].agent_id)
    if strategy == AssignmentStrategy.LEAST_LOADED:
        return self._select_least_loaded(active_agents)
    if strategy == AssignmentStrategy.SPECIALIZED:
        return self._select_specialized(task, active_agents)
    if strategy == AssignmentStrategy.PRIORITY_BASED:
        return self._select_priority_based(task, active_agents)
    return str(active_agents[0].agent_id)
def _select_least_loaded(self, active_agents: list[AgentState]) -> str:
    """Select agent with least current load.
    Args:
        active_agents: List of active agent states.
    Returns:
        Agent ID with least tasks.
    """
    task_counts: dict[str, int] = {}
    for assignment in self._assignment_history:
        if assignment.agent_id:
            task_counts[assignment.agent_id] = task_counts.get(assignment.agent_id, 0) + 1
    min_load = float("inf")
    selected_agent = str(active_agents[0].agent_id)
    for agent in active_agents:
        load = task_counts.get(agent.agent_id, 0)
        if load < min_load:
            min_load = load
            selected_agent = str(agent.agent_id)
    return selected_agent
def _select_specialized(self, task: Task, active_agents: list[AgentState]) -> str | None:
    """Select agent based on task type specialization.
    Args:
        task: Task to assign.
        active_agents: List of active agent states.
    Returns:
        Specialized agent ID or None.
    """
    task_type_map = {
        "explorer": ["explorer", "researcher"],
        "implementer": ["implementer", "coordinator"],
        "reviewer": ["reviewer", "analyzer"],
        "tester": ["tester", "reviewer"],
    }
    task_type_str = str(task.metadata.get("task_type", ""))
    preferred_types = task_type_map.get(task_type_str, ["implementer"])
    for pref_type in preferred_types:
        for agent in active_agents:
            if hasattr(agent, "agent_type") and agent.agent_type.value == pref_type:
                return str(agent.agent_id)
    return self._select_least_loaded(active_agents)
def _select_priority_based(self, task: Task, active_agents: list[AgentState]) -> str:
```

```
    """Select agent based on task priority.
    Args:
        task: Task to assign.
        active_agents: List of active agent states.
    Returns:
        Agent ID for priority task.
    """
    if task.priority == Priority.URGENT:
        idle_agents = [a for a in active_agents if not a.current_task]
        if idle_agents:
            return str(idle_agents[0].agent_id)
    if task.priority == Priority.HIGH:
        low_load_agents = []
        for agent in active_agents:
            assignments = [a for a in self._assignment_history if a.agent_id == agent.agent
_id]
            if len(assignments) < 2:
                low_load_agents.append(agent)
        if low_load_agents:
            return str(low_load_agents[0].agent_id)
    return self._select_least_loaded(active_agents)
def get_history(self, limit: int | None = None) -> list[Assignment]:
    """Get assignment history.
    Args:
        limit: Optional limit on number of assignments.
    Returns:
        List of past assignments.
    """
    if limit:
        return self._assignment_history[-limit:]
    return self._assignment_history.copy()
def clear_history(self) -> None:
    """Clear assignment history."""
    self._assignment_history.clear()
def get_stats(self) -> dict[str, Any]:
    """Get distribution statistics.
    Returns:
        Dictionary with distribution statistics.
    """
    agent_counts: dict[str, int] = {}
    for assignment in self._assignment_history:
        if assignment.agent_id:
            agent_counts[assignment.agent_id] = agent_counts.get(assignment.agent_id, 0) +
1
    strategy_counts: dict[str, int] = {}
    for assignment in self._assignment_history:
        strategy_key = assignment.strategy.value
        strategy_counts[strategy_key] = strategy_counts.get(strategy_key, 0) + 1
    return {
        "total_assignments": len(self._assignment_history),
        "assignments_by_agent": agent_counts,
        "assignments_by_strategy": strategy_counts,
        "round_robin_index": self._round_robin_index,
    }
```

File: [119/252] openlaoke/core/multi_agent/team.py

```
"""Team management for multi-agent collaboration."""
from __future__ import annotations
import logging
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from uuid import uuid4
if TYPE_CHECKING:
    from openlaoke.core.multi_agent.task_distribution import Assignment
logger = logging.getLogger(__name__)
class AgentRole(StrEnum):
    """Roles that agents can have in a team."""
    LEADER = "leader"
    COORDINATOR = "coordinator"
    EXPLORER = "explorer"
    IMPLEMENTER = "implementer"
    REVIEWER = "reviewer"
    TESTER = "tester"
    RESEARCHER = "researcher"
    ANALYZER = "analyzer"
    SUPPORTER = "supporter"
class TeamStatus(StrEnum):
    """Status of a team."""
    FORMING = "forming"
    ACTIVE = "active"
    WORKING = "working"
    COMPLETED = "completed"
    FAILED = "failed"
    DISBANDED = "disbanded"
@dataclass
class TeamMember:
    """Member of a team with specific role."""
    agent_id: str
    role: AgentRole
    capabilities: list[str] = field(default_factory=list)
    load: int = 0
    status: str = "idle"
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "agent_id": self.agent_id,
            "role": self.role.value,
            "capabilities": self.capabilities,
            "load": self.load,
            "status": self.status,
            "metadata": self.metadata,
        }
@dataclass
class TeamConfig:
    """Configuration for a team."""
    name: str = ""
    max_members: int = 10
    require_leader: bool = True
    auto_assign_roles: bool = True
    collaboration_mode: str = "parallel"
@dataclass
class Team:
    """Team of agents working together.
    Attributes:
        id: Unique team identifier.
```

```
    name: Team name.
    members: List of team members.
    status: Current team status.
    config: Team configuration.
    created_at: When the team was created.
    tasks_completed: Number of tasks completed.
    metadata: Additional team metadata.
"""
id: str = field(default_factory=lambda: f"team_{uuid4().hex[:8]}")
name: str = ""
members: list[TeamMember] = field(default_factory=list)
status: TeamStatus = TeamStatus.FORMING
config: TeamConfig = field(default_factory=TeamConfig)
created_at: float = field(default_factory=time.time)
tasks_completed: int = 0
metadata: dict[str, Any] = field(default_factory=dict)
def to_dict(self) -> dict[str, Any]:
    return {
        "id": self.id,
        "name": self.name,
        "members": [m.to_dict() for m in self.members],
        "status": self.status.value,
        "config": {
            "name": self.config.name,
            "max_members": self.config.max_members,
            "require_leader": self.config.require_leader,
        },
        "created_at": self.created_at,
        "tasks_completed": self.tasks_completed,
        "metadata": self.metadata,
    }
def get_member(self, agent_id: str) -> TeamMember | None:
    """Get a team member by agent ID.
    Args:
        agent_id: Agent ID to look up.
    Returns:
        TeamMember if found, None otherwise.
    """
    for member in self.members:
        if member.agent_id == agent_id:
            return member
    return None
def get_leader(self) -> TeamMember | None:
    """Get the team leader.
    Returns:
        TeamMember if leader found, None otherwise.
    """
    for member in self.members:
        if member.role == AgentRole.LEADER:
            return member
    return None
def get_members_by_role(self, role: AgentRole) -> list[TeamMember]:
    """Get all members with a specific role.
    Args:
        role: Role to filter by.
    Returns:
        List of members with that role.
    """
    return [m for m in self.members if m.role == role]
def add_member(
    self,
    agent_id: str,
    role: AgentRole,
    capabilities: list[str] | None = None,
) -> bool:
```

```
    """Add a member to the team.
    Args:
        agent_id: Agent ID to add.
        role: Role for the member.
        capabilities: Optional list of capabilities.
    Returns:
        True if member was added, False if team is full.
    """
    if len(self.members) >= self.config.max_members:
        logger.warning(f"Team {self.id} is full (max: {self.config.max_members})")
        return False
    if self.get_member(agent_id):
        logger.warning(f"Agent {agent_id} is already in team {self.id}")
        return False
    member = TeamMember(
        agent_id=agent_id,
        role=role,
        capabilities=capabilities or [],
    )
    self.members.append(member)
    logger.info(f"Added agent {agent_id} with role {role.value} to team {self.id}")
    return True
def remove_member(self, agent_id: str) -> bool:
    """Remove a member from the team.
    Args:
        agent_id: Agent ID to remove.
    Returns:
        True if member was removed, False if not found.
    """
    for i, member in enumerate(self.members):
        if member.agent_id == agent_id:
            self.members.pop(i)
            logger.info(f"Removed agent {agent_id} from team {self.id}")
            return True
    return False
def update_member_role(self, agent_id: str, new_role: AgentRole) -> bool:
    """Update a member's role.
    Args:
        agent_id: Agent ID to update.
        new_role: New role for the member.
    Returns:
        True if role was updated, False if member not found.
    """
    member = self.get_member(agent_id)
    if member:
        old_role = member.role
        member.role = new_role
        logger.info(f"Updated agent {agent_id} role from {old_role.value} to {new_role.valu
e}")
        return True
    return False
def update_member_status(self, agent_id: str, status: str) -> bool:
    """Update a member's status.
    Args:
        agent_id: Agent ID to update.
        status: New status for the member.
    Returns:
        True if status was updated, False if member not found.
    """
    member = self.get_member(agent_id)
    if member:
        member.status = status
        return True
    return False
def increment_member_load(self, agent_id: str) -> bool:
```

```
        """Increment a member's task load.
        Args:
            agent_id: Agent ID to update.
        Returns:
            True if load was incremented, False if member not found.
        """
        member = self.get_member(agent_id)
        if member:
            member.load += 1
            return True
        return False
    def decrement_member_load(self, agent_id: str) -> bool:
        """Decrement a member's task load.
        Args:
            agent_id: Agent ID to update.
        Returns:
            True if load was decremented, False if member not found.
        """
        member = self.get_member(agent_id)
        if member:
            member.load = max(0, member.load - 1)
            return True
        return False
    def get_active_members(self) -> list[TeamMember]:
        """Get all active members.
        Returns:
            List of members with active status.
        """
        return [m for m in self.members if m.status == "active"]
    def get_available_members(self) -> list[TeamMember]:
        """Get all available members (idle or low load).
        Returns:
            List of available members.
        """
        return [m for m in self.members if m.status in ("idle", "active") and m.load < 3]
@dataclass
class TeamResult:
    """Result of team execution.
    Attributes:
        team_id: ID of the team that executed.
        success: Whether execution was successful.
        results: Task results by task ID.
        assignments: Task assignments by task ID.
        duration: Execution duration in seconds.
        metadata: Additional result metadata.
    """
    team_id: str = ""
    success: bool = False
    results: dict[str, Any] = field(default_factory=dict)
    assignments: dict[str, Assignment] = field(default_factory=dict)
    duration: float = 0.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def to_dict(self) -> dict[str, Any]:
        return {
            "team_id": self.team_id,
            "success": self.success,
            "results": self.results,
            "assignments": {k: v.to_dict() for k, v in self.assignments.items()},
            "duration": self.duration,
            "metadata": self.metadata,
        }
class TeamManager:
    """Manages multiple teams and their lifecycle.
    Provides team creation, management, and tracking functionality.
    """
```

```
def __init__(self, max_teams: int = 10):
    """Initialize the team manager.
    Args:
        max_teams: Maximum number of concurrent teams.
    """
    self.max_teams = max_teams
    self._teams: dict[str, Team] = {}
    self._results: dict[str, TeamResult] = {}
    self._team_history: list[Team] = []
def create_team(
    self,
    name: str,
    config: TeamConfig | None = None,
    members: list[tuple[str, AgentRole]] | None = None,
) -> Team:
    """Create a new team.
    Args:
        name: Team name.
        config: Optional team configuration.
        members: Optional list of (agent_id, role) tuples.
    Returns:
        Created team.
    """
    if len(self._teams) >= self.max_teams:
        raise RuntimeError(f"Maximum teams ({self.max_teams}) reached")
    config = config or TeamConfig(name=name)
    team = Team(
        name=name,
        config=config,
    )
    if members:
        for agent_id, role in members:
            team.add_member(agent_id, role)
    self._teams[team.id] = team
    logger.info(f"Created team {team.id} with name '{name}'")
    return team
def get_team(self, team_id: str) -> Team | None:
    """Get a team by ID.
    Args:
        team_id: Team ID to look up.
    Returns:
        Team if found, None otherwise.
    """
    return self._teams.get(team_id)
def list_teams(self, status: TeamStatus | None = None) -> list[Team]:
    """List all teams, optionally filtered by status.
    Args:
        status: Optional status to filter by.
    Returns:
        List of teams.
    """
    teams = list(self._teams.values())
    if status:
        teams = [t for t in teams if t.status == status]
    return teams
def disband_team(self, team_id: str) -> bool:
    """Disband a team.
    Args:
        team_id: Team ID to disband.
    Returns:
        True if team was disbanded, False if not found.
    """
    team = self._teams.get(team_id)
    if not team:
        return False
```

```
        team.status = TeamStatus.DISBANDED
        self._team_history.append(team)
        del self._teams[team_id]
        logger.info(f"Disbanded team {team_id}")
        return True
    def record_result(self, team_id: str, result: TeamResult) -> None:
        """Record a team execution result.
        Args:
            team_id: Team ID.
            result: Team execution result.
        """
        self._results[team_id] = result
        team = self._teams.get(team_id)
        if team:
            if result.success:
                team.status = TeamStatus.COMPLETED
            else:
                team.status = TeamStatus.FAILED
            team.tasks_completed += len(result.results)
    def get_result(self, team_id: str) -> TeamResult | None:
        """Get a team's result.
        Args:
            team_id: Team ID.
        Returns:
            TeamResult if found, None otherwise.
        """
        return self._results.get(team_id)
    def get_stats(self) -> dict[str, Any]:
        """Get team management statistics.
        Returns:
            Dictionary with statistics.
        """
        status_counts: dict[str, int] = {}
        for team in self._teams.values():
            status_key = team.status.value
            status_counts[status_key] = status_counts.get(status_key, 0) + 1
        return {
            "total_teams": len(self._teams),
            "max_teams": self.max_teams,
            "disbanded_teams": len(self._team_history),
            "by_status": status_counts,
            "total_results": len(self._results),
        }
    def clear_history(self) -> None:
        """Clear team history."""
        self._team_history.clear()
```

File: [120/252] openlaoke/core/multi_provider_api.py

```
"""Multi-provider API client supporting Anthropic, OpenAI, and local models."""
from __future__ import annotations
import json
import os
import time
from collections.abc import AsyncIterator
from dataclasses import dataclass
from typing import Any
import httpx
from openlaoke.types.core_types import (
    AssistantMessage,
    CostInfo,
    MessageRole,
    TokenUsage,
    ToolUseBlock,
)
from openlaoke.types.providers import MultiProviderConfig, ProviderConfig, ProviderType
@dataclass
class ModelPricing:
    """Pricing information for a model (per million tokens)."""
    input_price: float = 3.0
    output_price: float = 15.0
    cache_read_price: float = 0.30
    cache_creation_price: float = 3.75
DEFAULT_PRICING = ModelPricing(3.0, 15.0, 0.30, 3.75)
MODEL_PRICES: dict[str, ModelPricing] = {
    "claude-sonnet-4-20250514": ModelPricing(3.0, 15.0, 0.30, 3.75),
    "claude-opus-4-20250514": ModelPricing(15.0, 75.0, 1.50, 18.75),
    "claude-3-5-sonnet-20241022": ModelPricing(3.0, 15.0, 0.30, 3.75),
    "claude-3-5-haiku-20241022": ModelPricing(0.80, 4.0, 0.08, 1.0),
    "gpt-4o": ModelPricing(2.5, 10.0, 0.0, 0.0),
    "gpt-4o-mini": ModelPricing(0.15, 0.60, 0.0, 0.0),
    "gpt-4-turbo": ModelPricing(10.0, 30.0, 0.0, 0.0),
    "o1-preview": ModelPricing(15.0, 60.0, 0.0, 0.0),
    "o1-mini": ModelPricing(3.0, 12.0, 0.0, 0.0),
    "MiniMax-M2.7-highspeed": ModelPricing(0.2, 0.6, 0.0, 0.0),
    "MiniMax-M2.7": ModelPricing(0.5, 1.5, 0.0, 0.0),
    "MiniMax-M2.5-highspeed": ModelPricing(0.1, 0.3, 0.0, 0.0),
    "MiniMax-M2.5": ModelPricing(0.3, 0.9, 0.0, 0.0),
    "qwen3.5-plus": ModelPricing(0.4, 1.2, 0.0, 0.0),
    "qwen3-max-2026-01-23": ModelPricing(2.0, 6.0, 0.0, 0.0),
    "glm-5": ModelPricing(0.1, 0.3, 0.0, 0.0),
    "glm-4.7": ModelPricing(0.2, 0.6, 0.0, 0.0),
    "gemini-2.0-flash": ModelPricing(0.1, 0.4, 0.0, 0.0),
    "gemini-2.0-pro": ModelPricing(1.25, 5.0, 0.0, 0.0),
    "gemini-1.5-flash": ModelPricing(0.075, 0.3, 0.0, 0.0),
    "gemini-1.5-pro": ModelPricing(1.25, 5.0, 0.0, 0.0),
    "grok-2-latest": ModelPricing(2.0, 10.0, 0.0, 0.0),
    "grok-2-1212": ModelPricing(2.0, 10.0, 0.0, 0.0),
    "grok-beta": ModelPricing(5.0, 15.0, 0.0, 0.0),
    "mistral-large-latest": ModelPricing(2.0, 6.0, 0.0, 0.0),
    "mistral-small-latest": ModelPricing(0.2, 0.6, 0.0, 0.0),
    "codestral-latest": ModelPricing(0.2, 0.6, 0.0, 0.0),
    "open-mistral-nemo": ModelPricing(0.03, 0.06, 0.0, 0.0),
    "llama-3.3-70b-versatile": ModelPricing(0.59, 0.79, 0.0, 0.0),
    "llama-3.1-8b-instant": ModelPricing(0.05, 0.08, 0.0, 0.0),
    "mixtral-8x7b-32768": ModelPricing(0.27, 0.27, 0.0, 0.0),
    "gemma2-9b-it": ModelPricing(0.2, 0.2, 0.0, 0.0),
    "command-r-plus": ModelPricing(3.0, 15.0, 0.0, 0.0),
    "command-r": ModelPricing(0.5, 1.5, 0.0, 0.0),
    "command": ModelPricing(1.0, 2.0, 0.0, 0.0),
```

```
    "command-light": ModelPricing(0.3, 0.6, 0.0, 0.0),
    "big-pickle": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "mimo-v2-flash-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "minimax-m2.1-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "mimo-v2-omni-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "qwen3.6-plus-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "grok-code": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "kimi-k2.5-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "glm-5-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "gpt-5-nano": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "nemotron-3-super-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "minimax-m2.5-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "trinity-large-preview-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "glm-4.7-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
    "mimo-v2-pro-free": ModelPricing(0.0, 0.0, 0.0, 0.0),
}
class MultiProviderClient:
    """HTTP client supporting multiple LLM providers."""
    def __init__(self, config: MultiProviderConfig, proxy: str | None = None) -> None:
        self.config = config
        self._proxy = proxy
        self._client: httpx.AsyncClient | None = None
        self._timeout = 300.0
    def _get_client(self) -> httpx.AsyncClient:
        if self._client is None:
            proxy = self._proxy
            if proxy is None or proxy == "":
                proxy = None
            limits = httpx.Limits(max_connections=10, max_keepalive_connections=5)
            self._client = httpx.AsyncClient(
                timeout=httpx.Timeout(self._timeout, connect=30.0),
                limits=limits,
                proxy=proxy,
            )
        return self._client
    async def close(self) -> None:
        if self._client:
            await self._client.aclose()
            self._client = None
    def _get_api_key(self, provider: ProviderConfig) -> str:
        env_key = ""
        if provider.provider_type == ProviderType.ANTHROPIC:
            env_key = os.environ.get("ANTHROPIC_API_KEY", "")
        elif provider.provider_type == ProviderType.OPENAI:
            env_key = os.environ.get("OPENAI_API_KEY", "")
        elif provider.provider_type == ProviderType.MINIMAX:
            env_key = os.environ.get("MINIMAX_API_KEY", "")
        elif provider.provider_type == ProviderType.ALIYUN_CODING_PLAN:
            env_key = os.environ.get("ALIYUN_API_KEY", "")
        elif provider.provider_type == ProviderType.AZURE_OPENAI:
            env_key = os.environ.get("AZURE_OPENAI_API_KEY", "")
        elif provider.provider_type == ProviderType.GOOGLE:
            env_key = os.environ.get("GOOGLE_API_KEY", "")
        elif provider.provider_type == ProviderType.GOOGLE_VERTEX:
            env_key = os.environ.get("GOOGLE_APPLICATION_CREDENTIALS", "")
        elif provider.provider_type == ProviderType.AWS_BEDROCK:
            env_key = os.environ.get("AWS_ACCESS_KEY_ID", "")
        elif provider.provider_type == ProviderType.XAI:
            env_key = os.environ.get("XAI_API_KEY", "")
        elif provider.provider_type == ProviderType.MISTRAL:
            env_key = os.environ.get("MISTRAL_API_KEY", "")
        elif provider.provider_type == ProviderType.GROQ:
            env_key = os.environ.get("GROQ_API_KEY", "")
        elif provider.provider_type == ProviderType.CEREBRAS:
            env_key = os.environ.get("CEREBRAS_API_KEY", "")
```

```
    elif provider.provider_type == ProviderType.COHERE:
        env_key = os.environ.get("COHERE_API_KEY", "")
    elif provider.provider_type == ProviderType.DEEPINFRA:
        env_key = os.environ.get("DEEPINFRA_API_KEY", "")
    elif provider.provider_type == ProviderType.TOGETHERAI:
        env_key = os.environ.get("TOGETHERAI_API_KEY", "")
    elif provider.provider_type == ProviderType.PERPLEXITY:
        env_key = os.environ.get("PERPLEXITY_API_KEY", "")
    elif provider.provider_type == ProviderType.OPENROUTER:
        env_key = os.environ.get("OPENROUTER_API_KEY", "")
    elif provider.provider_type == ProviderType.GITHUB_COPILOT:
        env_key = os.environ.get("GITHUB_TOKEN", "")
    elif provider.provider_type == ProviderType.OPENCODE:
        env_key = os.environ.get("OPENCODE_API_KEY", "public")
    elif provider.provider_type == ProviderType.OPENAI_COMPATIBLE:
        env_key = os.environ.get("OPENAI_API_KEY", "none")
    return provider.api_key or env_key
def _get_base_url(self, provider: ProviderConfig) -> str:
    env_url = ""
    if provider.provider_type == ProviderType.ANTHROPIC:
        env_url = os.environ.get("ANTHROPIC_BASE_URL", "https://api.anthropic.com")
    elif provider.provider_type == ProviderType.OPENAI:
        env_url = os.environ.get("OPENAI_BASE_URL", "https://api.openai.com/v1")
    elif provider.provider_type == ProviderType.MINIMAX:
        env_url = os.environ.get("MINIMAX_BASE_URL", "https://api.minimaxi.com/v1")
    elif provider.provider_type == ProviderType.ALIYUN_CODING_PLAN:
        env_url = os.environ.get("ALIYUN_BASE_URL", "https://coding.dashscope.aliyuncs.com/
v1")
    elif provider.provider_type == ProviderType.AZURE_OPENAI:
        env_url = os.environ.get("AZURE_OPENAI_ENDPOINT", "")
    elif provider.provider_type == ProviderType.GOOGLE:
        env_url = os.environ.get(
            "GOOGLE_BASE_URL", "https://generativelanguage.googleapis.com/v1beta"
        )
    elif provider.provider_type == ProviderType.GOOGLE_VERTEX:
        env_url = os.environ.get("GOOGLE_VERTEX_BASE_URL", "")
    elif provider.provider_type == ProviderType.AWS_BEDROCK:
        env_url = os.environ.get("AWS_BEDROCK_REGION", "")
    elif provider.provider_type == ProviderType.XAI:
        env_url = os.environ.get("XAI_BASE_URL", "https://api.x.ai/v1")
    elif provider.provider_type == ProviderType.MISTRAL:
        env_url = os.environ.get("MISTRAL_BASE_URL", "https://api.mistral.ai/v1")
    elif provider.provider_type == ProviderType.GROQ:
        env_url = os.environ.get("GROQ_BASE_URL", "https://api.groq.com/openai/v1")
    elif provider.provider_type == ProviderType.CEREBRAS:
        env_url = os.environ.get("CEREBRAS_BASE_URL", "https://api.cerebras.ai/v1")
    elif provider.provider_type == ProviderType.COHERE:
        env_url = os.environ.get("COHERE_BASE_URL", "https://api.cohere.ai/v2")
    elif provider.provider_type == ProviderType.DEEPINFRA:
        env_url = os.environ.get("DEEPINFRA_BASE_URL", "https://api.deepinfra.com/v1/openai
")
    elif provider.provider_type == ProviderType.TOGETHERAI:
        env_url = os.environ.get("TOGETHERAI_BASE_URL", "https://api.together.xyz/v1")
    elif provider.provider_type == ProviderType.PERPLEXITY:
        env_url = os.environ.get("PERPLEXITY_BASE_URL", "https://api.perplexity.ai")
    elif provider.provider_type == ProviderType.OPENROUTER:
        env_url = os.environ.get("OPENROUTER_BASE_URL", "https://openrouter.ai/api/v1")
    elif provider.provider_type == ProviderType.GITHUB_COPILOT:
        env_url = os.environ.get("GITHUB_COPILOT_BASE_URL", "https://api.githubcopilot.com"
)
    elif provider.provider_type == ProviderType.OPENCODE:
        env_url = os.environ.get("OPENCODE_BASE_URL", "https://opencode.ai/zen/v1")
    elif provider.provider_type == ProviderType.OPENAI_COMPATIBLE:
        env_url = os.environ.get("OPENAI_BASE_URL", "https://api.openai.com/v1")
    return provider.base_url or env_url
```

```
def _build_headers(self, provider: ProviderConfig) -> dict[str, str]:
    api_key = self._get_api_key(provider)
    if provider.provider_type == ProviderType.ANTHROPIC:
        return {
            "x-api-key": api_key,
            "anthropic-version": "2023-06-01",
            "content-type": "application/json",
        }
    elif provider.provider_type == ProviderType.AZURE_OPENAI:
        return {
            "api-key": api_key,
            "Content-Type": "application/json",
        }
    elif provider.provider_type == ProviderType.GOOGLE:
        return {
            "x-goog-api-key": api_key,
            "Content-Type": "application/json",
        }
    elif (
        provider.provider_type == ProviderType.GOOGLE_VERTEX
        or provider.provider_type == ProviderType.COHERE
    ):
        return {
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json",
        }
    elif provider.is_local:
        return {
            "Content-Type": "application/json",
        }
    else:
        return {
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json",
        }
def _build_anthropic_body(
    self,
    model: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None,
    max_tokens: int,
    temperature: float,
    thinking_budget: int,
) -> dict[str, Any]:
    body: dict[str, Any] = {
        "model": model,
        "max_tokens": max_tokens,
        "messages": messages,
    }
    if temperature != 1.0:
        body["temperature"] = temperature
    if tools:
        body["tools"] = tools
    if thinking_budget > 0:
        body["thinking"] = {"type": "enabled", "budget_tokens": thinking_budget}
    return body
def _build_openai_body(
    self,
    model: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None,
    max_tokens: int,
    temperature: float,
) -> dict[str, Any]:
    body: dict[str, Any] = {
```

```
        "model": model,
        "max_tokens": max_tokens,
        "messages": messages,
    }
    if temperature != 1.0:
        body["temperature"] = temperature
    if tools:
        body["tools"] = self._convert_tools_to_openai_format(tools)
    return body
def _convert_tools_to_openai_format(self, tools: list[dict[str, Any]]) -> list[dict[str, An
y]]:
    """Convert internal tool format to OpenAI API format."""
    result = []
    for tool in tools:
        # Check if already in OpenAI format
        if tool.get("type") == "function" and "function" in tool:
            result.append(tool)
        else:
            # Convert from internal format
            result.append(
                {
                    "type": "function",
                    "function": {
                        "name": tool.get("name", ""),
                        "description": tool.get("description", ""),
                        "parameters": tool.get("input_schema", {}),
                    },
                }
            )
    return result
def _convert_messages_for_provider(
    self,
    messages: list[dict[str, Any]],
    provider_type: ProviderType,
) -> list[dict[str, Any]]:
    if provider_type == ProviderType.ANTHROPIC:
        return self._convert_to_anthropic_format(messages)
    else:
        return self._convert_to_openai_format(messages)
def _convert_to_anthropic_format(self, messages: list[dict[str, Any]]) -> list[dict[str, An
y]]:
    result = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
        if role == "system":
            continue
        # Handle OpenAI native tool format
        if role == "tool":
            result.append(
                {
                    "role": "user",
                    "content": [
                        {
                            "type": "tool_result",
                            "tool_use_id": msg.get("tool_call_id", ""),
                            "content": content,
                        }
                    ],
                }
            )
        # Handle our internal tool_result format
        elif role == "tool_result":
            result.append(
                {
```

```
                "role": "user",
                "content": [
                    {
                        "type": "tool_result",
                        "tool_use_id": msg.get("tool_use_id", ""),
                        "content": content,
                    }
                ],
            }
        )
    # Handle tool_use (legacy)
    elif role == "tool_use":
        result.append(
            {
                "role": "assistant",
                "content": [
                    {
                        "type": "tool_use",
                        "id": msg.get("id", ""),
                        "name": msg.get("name", ""),
                        "input": msg.get("input", {}),
                    }
                ],
            }
        )
    # Handle assistant with tool_calls (OpenAI native)
    elif role == "assistant" and "tool_calls" in msg:
        content_blocks = []
        if msg.get("content"):
            content_blocks.append({"type": "text", "text": msg["content"]})
        for tc in msg.get("tool_calls", []):
            func = tc.get("function", {})
            try:
                args = json.loads(func.get("arguments", "{}"))
            except json.JSONDecodeError:
                args = {}
            content_blocks.append(
                {
                    "type": "tool_use",
                    "id": tc.get("id", ""),
                    "name": func.get("name", ""),
                    "input": args,
                }
            )
        result.append({"role": "assistant", "content": content_blocks})
    else:
        result.append({"role": role, "content": content})
return result
def _convert_to_openai_format(self, messages: list[dict[str, Any]]) -> list[dict[str, Any]]
:
    result = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
        # Handle OpenAI native tool format
        if role == "tool":
            result.append(
                {
                    "role": "tool",
                    "tool_call_id": msg.get("tool_call_id", ""),
                    "content": content,
                }
            )
        # Handle our internal tool_result format
        elif role == "tool_result":
```

```
            result.append(
                {
                    "role": "tool",
                    "tool_call_id": msg.get("tool_use_id", ""),
                    "content": content,
                }
            )
        # Handle tool_use (legacy)
        elif role == "tool_use":
            if not result or result[-1].get("role") != "assistant":
                result.append({"role": "assistant", "content": "", "tool_calls": []})
            result[-1].setdefault("tool_calls", []).append(
                {
                    "id": msg.get("id", ""),
                    "type": "function",
                    "function": {
                        "name": msg.get("name", ""),
                        "arguments": json.dumps(msg.get("input", {})),
                    },
                }
            )
        # Handle assistant with tool_calls (OpenAI native)
        elif role == "assistant" and "tool_calls" in msg:
            result.append(msg)
        else:
            result.append({"role": role, "content": content})
    return result
def _parse_anthropic_response(
    self, data: dict[str, Any]
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    content = ""
    tool_uses = []
    for block in data.get("content", []):
        if block["type"] == "text":
            content += block["text"]
        elif block["type"] == "tool_use":
            tool_uses.append(
                ToolUseBlock(
                    id=block["id"],
                    name=block["name"],
                    input=block["input"],
                )
            )
    usage = self._parse_token_usage(data.get("usage", {}))
    cost = self._calculate_cost(data.get("usage", {}), data.get("model", ""))
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=data.get("stop_reason"),
        ),
        usage,
        cost,
    )
def _parse_openai_response(
    self, data: dict[str, Any]
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    content = ""
    tool_uses = []
    choices = data.get("choices", [])
    if choices:
        choice = choices[0]
        message = choice.get("message", {})
        content = message.get("content", "") or ""
```

```
        for tool_call in message.get("tool_calls", []):
            func = tool_call.get("function", {})
            try:
                args = json.loads(func.get("arguments", "{}"))
            except json.JSONDecodeError:
                args = {}
            tool_uses.append(
                ToolUseBlock(
                    id=tool_call.get("id", ""),
                    name=func.get("name", ""),
                    input=args,
                )
            )
    usage_data = data.get("usage", {})
    usage = TokenUsage(
        input_tokens=usage_data.get("prompt_tokens", 0),
        output_tokens=usage_data.get("completion_tokens", 0),
    )
    cost = self._calculate_cost(usage_data, data.get("model", ""))
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=choices[0].get("finish_reason") if choices else None,
        ),
        usage,
        cost,
    )
def _parse_token_usage(self, usage: dict[str, int]) -> TokenUsage:
    return TokenUsage(
        input_tokens=usage.get("input_tokens", 0),
        output_tokens=usage.get("output_tokens", 0),
        cache_read_tokens=usage.get("cache_read_input_tokens", 0),
        cache_creation_tokens=usage.get("cache_creation_input_tokens", 0),
    )
def _calculate_cost(self, usage: dict[str, int], model: str) -> CostInfo:
    pricing = MODEL_PRICES.get(model, DEFAULT_PRICING)
    input_tokens = usage.get("input_tokens", usage.get("prompt_tokens", 0))
    output_tokens = usage.get("output_tokens", usage.get("completion_tokens", 0))
    return CostInfo(
        input_cost=input_tokens / 1_000_000 * pricing.input_price,
        output_cost=output_tokens / 1_000_000 * pricing.output_price,
    )
async def send_message(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None = None,
    model: str | None = None,
    max_tokens: int = 8192,
    temperature: float = 1.0,
    thinking_budget: int = 0,
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    provider = self.config.get_active_provider()
    if not provider:
        raise ValueError("No active provider configured")
    model = model or self.config.get_active_model()
    base_url = self._get_base_url(provider)
    client = self._get_client()
    headers = self._build_headers(provider)
    converted_messages = self._convert_messages_for_provider(messages, provider.provider_ty
pe)
    if provider.provider_type == ProviderType.ANTHROPIC:
        endpoint = f"{base_url}/v1/messages"
```

```
        body = self._build_anthropic_body(
            model, converted_messages, tools, max_tokens, temperature, thinking_budget
        )
        body["system"] = system_prompt
    elif provider.provider_type == ProviderType.AZURE_OPENAI:
        api_version = os.environ.get("AZURE_OPENAI_API_VERSION", "2024-02-15-preview")
        endpoint = (
            f"{base_url}/openai/deployments/{model}/chat/completions?api-version={api_versi
on}"
        )
        body = self._build_openai_body(
            model, converted_messages, tools, max_tokens, temperature
        )
        body["messages"] = [
            {"role": "system", "content": system_prompt},
            *body["messages"],
        ]
    elif provider.provider_type == ProviderType.GOOGLE:
        endpoint = f"{base_url}/models/{model}:generateContent"
        body = self._build_google_body(
            system_prompt, converted_messages, tools, max_tokens, temperature
        )
    elif provider.provider_type == ProviderType.GOOGLE_VERTEX:
        project_id = os.environ.get("GOOGLE_VERTEX_PROJECT", "")
        location = os.environ.get("GOOGLE_VERTEX_LOCATION", "us-central1")
        endpoint = f"https://{location}-aiplatform.googleapis.com/v1/projects/{project_id}/
locations/{location}/publishers/google/models/{model}:generateContent"
        body = self._build_google_body(
            system_prompt, converted_messages, tools, max_tokens, temperature
        )
    elif provider.provider_type == ProviderType.AWS_BEDROCK:
        region = os.environ.get("AWS_REGION", "us-east-1")
        endpoint = f"https://bedrock-runtime.{region}.amazonaws.com/model/{model}/invoke"
        body = self._build_bedrock_body(
            system_prompt, converted_messages, tools, max_tokens, temperature
        )
    elif provider.provider_type == ProviderType.COHERE:
        endpoint = f"{base_url}/chat"
        body = self._build_cohere_body(
            model, system_prompt, converted_messages, tools, max_tokens, temperature
        )
    else:
        endpoint = f"{base_url}/chat/completions"
        body = self._build_openai_body(
            model, converted_messages, tools, max_tokens, temperature
        )
        body["messages"] = [
            {"role": "system", "content": system_prompt},
            *body["messages"],
        ]
    response = await client.post(endpoint, headers=headers, json=body)
    if response.status_code >= 400:
        error_detail = ""
        try:
            error_data = response.json()
            error_detail = f"\nError details: {error_data}"
        except Exception:
            error_detail = f"\nResponse text: {response.text[:500]}"
        print(f"[DEBUG] API Error: {response.status_code} {response.reason_phrase}")
        print(f"[DEBUG] Endpoint: {endpoint}")
        print(f"[DEBUG] Request body keys: {list(body.keys())}")
        print(f"[DEBUG] Model: {body.get('model', 'N/A')}")
        print(f"[DEBUG] Messages count: {len(body.get('messages', []))}")
        print(f"[DEBUG]{error_detail}")
    response.raise_for_status()
```

```
    data = response.json()
    if provider.provider_type == ProviderType.ANTHROPIC:
        return self._parse_anthropic_response(data)
    elif provider.provider_type in (ProviderType.GOOGLE, ProviderType.GOOGLE_VERTEX):
        return self._parse_google_response(data)
    elif provider.provider_type == ProviderType.AWS_BEDROCK:
        return self._parse_bedrock_response(data, model)
    elif provider.provider_type == ProviderType.COHERE:
        return self._parse_cohere_response(data)
    else:
        return self._parse_openai_response(data)
async def stream_message(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None = None,
    model: str | None = None,
    max_tokens: int = 8192,
    temperature: float = 1.0,
    thinking_budget: int = 0,
) -> AsyncIterator[tuple[str, TokenUsage | None, CostInfo | None]]:
    provider = self.config.get_active_provider()
    if not provider:
        raise ValueError("No active provider configured")
    model = model or self.config.get_active_model()
    base_url = self._get_base_url(provider)
    client = self._get_client()
    headers = self._build_headers(provider)
    converted_messages = self._convert_messages_for_provider(messages, provider.provider_ty
pe)
    if provider.provider_type == ProviderType.ANTHROPIC:
        endpoint = f"{base_url}/v1/messages"
        body = self._build_anthropic_body(
            model, converted_messages, tools, max_tokens, temperature, thinking_budget
        )
        body["system"] = system_prompt
        body["stream"] = True
    else:
        endpoint = f"{base_url}/chat/completions"
        body = self._build_openai_body(
            model, converted_messages, tools, max_tokens, temperature
        )
        body["messages"] = [
            {"role": "system", "content": system_prompt},
            *body["messages"],
        ]
        body["stream"] = True
    async with client.stream("POST", endpoint, headers=headers, json=body) as response:
        response.raise_for_status()
        content = ""
        final_usage = None
        final_cost = None
        async for line in response.aiter_lines():
            if not line:
                continue
            if line.startswith("data: "):
                data_str = line[6:]
                if data_str == "[DONE]":
                    break
                try:
                    event = json.loads(data_str)
                except json.JSONDecodeError:
                    continue
                if provider.provider_type == ProviderType.ANTHROPIC:
                    text, usage, cost = self._parse_anthropic_stream_event(event)
```

```
                else:
                    text, usage, cost = self._parse_openai_stream_event(event)
                if text:
                    content += text
                    yield text, None, None
                if usage:
                    final_usage = usage
                    final_cost = cost
        yield "", final_usage, final_cost
def _parse_anthropic_stream_event(
    self, event: dict[str, Any]
) -> tuple[str, TokenUsage | None, CostInfo | None]:
    event_type = event.get("type", "")
    text = ""
    usage = None
    cost = None
    if event_type == "content_block_delta":
        delta = event.get("delta", {})
        if delta.get("type") == "text_delta":
            text = delta.get("text", "")
    elif event_type == "message_delta":
        usage_data = event.get("usage", {})
        if usage_data:
            usage = TokenUsage(
                input_tokens=usage_data.get("input_tokens", 0),
                output_tokens=usage_data.get("output_tokens", 0),
            )
            cost = CostInfo(
                input_cost=usage.input_tokens / 1_000_000 * 3.0,
                output_cost=usage.output_tokens / 1_000_000 * 15.0,
            )
    return text, usage, cost
def _parse_openai_stream_event(
    self, event: dict[str, Any]
) -> tuple[str, TokenUsage | None, CostInfo | None]:
    text = ""
    usage = None
    cost = None
    choices = event.get("choices", [])
    if choices:
        delta = choices[0].get("delta", {})
        text = delta.get("content", "") or ""
    usage_data = event.get("usage", {})
    if usage_data:
        usage = TokenUsage(
            input_tokens=usage_data.get("prompt_tokens", 0),
            output_tokens=usage_data.get("completion_tokens", 0),
        )
        cost = CostInfo(
            input_cost=usage.input_tokens / 1_000_000 * 2.5,
            output_cost=usage.output_tokens / 1_000_000 * 10.0,
        )
    return text, usage, cost
def _build_google_body(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None,
    max_tokens: int,
    temperature: float,
) -> dict[str, Any]:
    contents = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
```

```
        if role == "system":
            continue
        gemini_role = "user" if role in ["user", "tool", "tool_result"] else "model"
        contents.append({"role": gemini_role, "parts": [{"text": content}]})
    body: dict[str, Any] = {
        "contents": contents,
        "generationConfig": {"maxOutputTokens": max_tokens, "temperature": temperature},
    }
    if system_prompt:
        body["systemInstruction"] = {"parts": [{"text": system_prompt}]}
    if tools:
        body["tools"] = self._convert_tools_to_google_format(tools)
    return body
def _convert_tools_to_google_format(self, tools: list[dict[str, Any]]) -> list[dict[str, An
y]]:
    result = []
    for tool in tools:
        if tool.get("type") == "function" and "function" in tool:
            func = tool["function"]
        else:
            func = {
                "name": tool.get("name", ""),
                "description": tool.get("description", ""),
                "parameters": tool.get("input_schema", {}),
            }
        result.append(
            {
                "functionDeclarations": [
                    {
                        "name": func.get("name", ""),
                        "description": func.get("description", ""),
                        "parameters": func.get("parameters", {}),
                    }
                ]
            }
        )
    return result
def _parse_google_response(
    self, data: dict[str, Any]
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    content = ""
    tool_uses = []
    candidates = data.get("candidates", [])
    if candidates:
        candidate = candidates[0]
        content_parts = candidate.get("content", {}).get("parts", [])
        for part in content_parts:
            if "text" in part:
                content += part["text"]
            elif "functionCall" in part:
                fc = part["functionCall"]
                tool_uses.append(
                    ToolUseBlock(
                        id=fc.get("name", ""), name=fc.get("name", ""), input=fc.get("args"
, {})
                    )
                )
    usage_data = data.get("usageMetadata", {})
    usage = TokenUsage(
        input_tokens=usage_data.get("promptTokenCount", 0),
        output_tokens=usage_data.get("candidatesTokenCount", 0),
    )
    cost = CostInfo(
        input_cost=usage.input_tokens / 1_000_000 * 0.1,
        output_cost=usage.output_tokens / 1_000_000 * 0.4,
```

```
    )
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=candidates[0].get("finishReason") if candidates else None,
        ),
        usage,
        cost,
    )
def _build_bedrock_body(
    self,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None,
    max_tokens: int,
    temperature: float,
) -> dict[str, Any]:
    bedrock_messages = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
        if role == "system":
            continue
        bedrock_role = "user" if role in ["user", "tool", "tool_result"] else "assistant"
        bedrock_messages.append({"role": bedrock_role, "content": [{"text": content}]})
    body: dict[str, Any] = {
        "messages": bedrock_messages,
        "max_tokens": max_tokens,
        "temperature": temperature,
    }
    if system_prompt:
        body["system"] = system_prompt
    if tools:
        body["tools"] = self._convert_tools_to_openai_format(tools)
    return body
def _parse_bedrock_response(
    self, data: dict[str, Any], model: str
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    content = ""
    tool_uses = []
    output = data.get("output", {})
    message = output.get("message", {})
    content_blocks = message.get("content", [])
    for block in content_blocks:
        if "text" in block:
            content += block["text"]
        elif "toolUse" in block:
            tu = block["toolUse"]
            tool_uses.append(
                ToolUseBlock(
                    id=tu.get("toolUseId", ""),
                    name=tu.get("name", ""),
                    input=tu.get("input", {}),
                )
            )
    usage_data = data.get("usage", {})
    usage = TokenUsage(
        input_tokens=usage_data.get("inputTokens", 0),
        output_tokens=usage_data.get("outputTokens", 0),
    )
    cost = self._calculate_cost(
        {"input_tokens": usage.input_tokens, "output_tokens": usage.output_tokens}, model
    )
```

```
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=data.get("stopReason"),
        ),
        usage,
        cost,
    )
def _build_cohere_body(
    self,
    model: str,
    system_prompt: str,
    messages: list[dict[str, Any]],
    tools: list[dict[str, Any]] | None,
    max_tokens: int,
    temperature: float,
) -> dict[str, Any]:
    chat_history = []
    for msg in messages:
        role = msg.get("role", "user")
        content = msg.get("content", "")
        if role == "system":
            continue
        cohere_role = "USER" if role in ["user", "tool", "tool_result"] else "CHATBOT"
        chat_history.append({"role": cohere_role, "message": content})
    body: dict[str, Any] = {
        "model": model,
        "message": messages[-1].get("content", "") if messages else "",
        "chat_history": chat_history[:-1] if chat_history else [],
        "max_tokens": max_tokens,
        "temperature": temperature,
    }
    if system_prompt:
        body["preamble"] = system_prompt
    if tools:
        body["tools"] = self._convert_tools_to_cohere_format(tools)
    return body
def _convert_tools_to_cohere_format(self, tools: list[dict[str, Any]]) -> list[dict[str, An
y]]:
    result = []
    for tool in tools:
        if tool.get("type") == "function" and "function" in tool:
            func = tool["function"]
        else:
            func = {
                "name": tool.get("name", ""),
                "description": tool.get("description", ""),
                "parameters": tool.get("input_schema", {}),
            }
        result.append(
            {
                "type": "function",
                "function": {
                    "name": func.get("name", ""),
                    "description": func.get("description", ""),
                    "parameters": func.get("parameters", {}),
                },
            }
        )
    return result
def _parse_cohere_response(
    self, data: dict[str, Any]
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
```

```
    content = ""
    tool_uses = []
    text = data.get("text", "")
    content = text
    tool_calls = data.get("tool_calls", [])
    for tc in tool_calls:
        tool_uses.append(
            ToolUseBlock(
                id=tc.get("name", ""), name=tc.get("name", ""), input=tc.get("parameters",
{})
            )
        )
    usage_data = data.get("meta", {}).get("tokens", {})
    usage = TokenUsage(
        input_tokens=usage_data.get("input_tokens", 0),
        output_tokens=usage_data.get("output_tokens", 0),
    )
    cost = CostInfo(
        input_cost=usage.input_tokens / 1_000_000 * 3.0,
        output_cost=usage.output_tokens / 1_000_000 * 15.0,
    )
    return (
        AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=content,
            tool_uses=tool_uses,
            stop_reason=data.get("finish_reason"),
        ),
        usage,
        cost,
    )
```

File: [121/252] openlaoke/core/posix_commands.py

```
"""POSIX-compatible command layer for cross-platform support.
Provides unified interface for common operations across different OS:
- Linux/Unix (POSIX-compliant)
- macOS (Darwin, mostly POSIX)
- Windows (different commands)
This allows small models to use unified commands without worrying about OS differences.
"""
from __future__ import annotations
import platform
from dataclasses import dataclass
from enum import StrEnum
class OS(StrEnum):
    LINUX = "linux"
    MACOS = "darwin"
    WINDOWS = "windows"
    FREEBSD = "freebsd"
    UNKNOWN = "unknown"
@dataclass
class CommandMapping:
    """Mapping between unified command and OS-specific commands."""
    unified: str
    linux: str
    macos: str
    windows: str
    description: str
class POSIXCommands:
    """POSIX-compatible command mappings."""
    COMMANDS = {
        "list_files": CommandMapping(
            unified="list_files",
            linux="ls -la",
            macos="ls -la",
            windows="dir",
            description="List all files in directory",
        ),
        "list_files_simple": CommandMapping(
            unified="list_files_simple",
            linux="ls",
            macos="ls",
            windows="dir",
            description="List files (simple)",
        ),
        "change_directory": CommandMapping(
            unified="change_directory",
            linux="cd",
            macos="cd",
            windows="cd",
            description="Change current directory",
        ),
        "print_working_directory": CommandMapping(
            unified="print_working_directory",
            linux="pwd",
            macos="pwd",
            windows="cd",
            description="Print working directory",
        ),
        "create_directory": CommandMapping(
            unified="create_directory",
            linux="mkdir -p",
            macos="mkdir -p",
            windows="mkdir",
```

```
    description="Create directory (with parents)",
),
"remove_file": CommandMapping(
    unified="remove_file",
    linux="rm",
    macos="rm",
    windows="del",
    description="Remove a file",
),
"remove_directory": CommandMapping(
    unified="remove_directory",
    linux="rm -rf",
    macos="rm -rf",
    windows="rmdir /s /q",
    description="Remove directory recursively",
),
"copy_file": CommandMapping(
    unified="copy_file",
    linux="cp",
    macos="cp",
    windows="copy",
    description="Copy file",
),
"move_file": CommandMapping(
    unified="move_file",
    linux="mv",
    macos="mv",
    windows="move",
    description="Move/rename file",
),
"find_file": CommandMapping(
    unified="find_file",
    linux="find . -name",
    macos="find . -name",
    windows="dir /s /b",
    description="Find files by name",
),
"search_in_files": CommandMapping(
    unified="search_in_files",
    linux="grep -r",
    macos="grep -r",
    windows="findstr /s",
    description="Search text in files",
),
"view_file": CommandMapping(
    unified="view_file",
    linux="cat",
    macos="cat",
    windows="type",
    description="View file content",
),
"view_file_head": CommandMapping(
    unified="view_file_head",
    linux="head -n 20",
    macos="head -n 20",
    windows="more",
    description="View first lines of file",
),
"view_file_tail": CommandMapping(
    unified="view_file_tail",
    linux="tail -n 20",
    macos="tail -n 20",
    windows="more",
    description="View last lines of file",
),
```

```
    "change_permissions": CommandMapping(
        unified="change_permissions",
        linux="chmod",
        macos="chmod",
        windows="icacls",
        description="Change file permissions",
    ),
    "check_process": CommandMapping(
        unified="check_process",
        linux="ps aux | grep",
        macos="ps aux | grep",
        windows="tasklist | findstr",
        description="Check running processes",
    ),
    "kill_process": CommandMapping(
        unified="kill_process",
        linux="kill",
        macos="kill",
        windows="taskkill /F /PID",
        description="Kill a process",
    ),
    "environment_variables": CommandMapping(
        unified="environment_variables",
        linux="env",
        macos="env",
        windows="set",
        description="Show environment variables",
    ),
    "set_env_variable": CommandMapping(
        unified="set_env_variable",
        linux="export",
        macos="export",
        windows="set",
        description="Set environment variable",
    ),
    "check_network": CommandMapping(
        unified="check_network",
        linux="ping -c 4",
        macos="ping -c 4",
        windows="ping -n 4",
        description="Check network connectivity",
    ),
    "download_file": CommandMapping(
        unified="download_file",
        linux="wget",
        macos="curl -O",
        windows="curl -O",
        description="Download file from URL",
    ),
    "compress_file": CommandMapping(
        unified="compress_file",
        linux="tar -czf",
        macos="tar -czf",
        windows="tar -a -c -f",
        description="Compress files/directories",
    ),
    "extract_file": CommandMapping(
        unified="extract_file",
        linux="tar -xzf",
        macos="tar -xzf",
        windows="tar -xzf",
        description="Extract compressed file",
    ),
}
PYTHON_EQUIVALENTS = {
```

```
    "list_files": "import os; print('\\n'.join(os.listdir('.')))",
    "list_files_simple": "import os; print('\\n'.join(os.listdir('.')))",
    "print_working_directory": "import os; print(os.getcwd())",
    "create_directory": "import os; os.makedirs({path}, exist_ok=True)",
    "remove_file": "import os; os.remove({path})",
    "remove_directory": "import shutil; shutil.rmtree({path})",
    "copy_file": "import shutil; shutil.copy({src}, {dst})",
    "move_file": "import shutil; shutil.move({src}, {dst})",
    "view_file": "with open({path}) as f: print(f.read())",
    "environment_variables": "import os; print(os.environ)",
}
@classmethod
def get_command(cls, unified_name: str, target_os: OS | None = None) -> str:
    """Get OS-specific command for a unified command name.
    Args:
        unified_name: Unified command name
        target_os: Target OS (auto-detect if None)
    Returns:
        OS-specific command string
    """
    if target_os is None:
        target_os = cls.detect_os()
    if unified_name not in cls.COMMANDS:
        return unified_name
    mapping = cls.COMMANDS[unified_name]
    if target_os == OS.LINUX:
        return mapping.linux
    elif target_os == OS.MACOS:
        return mapping.macos
    elif target_os == OS.WINDOWS:
        return mapping.windows
    else:
        return mapping.linux
@classmethod
def get_python_equivalent(cls, unified_name: str) -> str | None:
    """Get Python code equivalent for a unified command.
    Args:
        unified_name: Unified command name
    Returns:
        Python code string or None if not available
    """
    return cls.PYTHON_EQUIVALENTS.get(unified_name)
@classmethod
def detect_os(cls) -> OS:
    """Detect current operating system."""
    system = platform.system().lower()
    return {
        "linux": OS.LINUX,
        "darwin": OS.MACOS,
        "windows": OS.WINDOWS,
        "freebsd": OS.FREEBSD,
    }.get(system, OS.UNKNOWN)
@classmethod
def list_available_commands(cls) -> list[str]:
    """List all available unified commands."""
    return list(cls.COMMANDS.keys())
@classmethod
def get_command_info(cls, unified_name: str) -> dict[str, str] | None:
    """Get detailed information about a command."""
    if unified_name not in cls.COMMANDS:
        return None
    mapping = cls.COMMANDS[unified_name]
    return {
        "unified": mapping.unified,
        "linux": mapping.linux,
```

```
            "macos": mapping.macos,
            "windows": mapping.windows,
            "description": mapping.description,
            "python": cls.PYTHON_EQUIVALENTS.get(unified_name, "N/A"),
        }
class CommandBuilder:
    """Build commands with arguments in a cross-platform way."""
    def __init__(self, target_os: OS | None = None):
        self.os = target_os or POSIXCommands.detect_os()
    def list_files(self, directory: str = ".", show_all: bool = True) -> str:
        """Build command to list files."""
        if show_all:
            cmd = POSIXCommands.get_command("list_files", self.os)
        else:
            cmd = POSIXCommands.get_command("list_files_simple", self.os)
        if self.os == OS.WINDOWS:
            return f"{cmd} {directory}"
        else:
            return f"{cmd} {directory}"
    def find_files(self, pattern: str, directory: str = ".") -> str:
        """Build command to find files by pattern."""
        cmd = POSIXCommands.get_command("find_file", self.os)
        if self.os == OS.WINDOWS:
            return f"{cmd} {directory}\\{pattern}"
        else:
            return f"{cmd} {directory} {pattern}"
    def search_in_files(self, pattern: str, directory: str = ".") -> str:
        """Build command to search text in files."""
        cmd = POSIXCommands.get_command("search_in_files", self.os)
        return f"{cmd} {pattern} {directory}"
    def create_directory(self, path: str) -> str:
        """Build command to create directory."""
        cmd = POSIXCommands.get_command("create_directory", self.os)
        return f"{cmd} {path}"
    def remove_file(self, path: str) -> str:
        """Build command to remove file."""
        cmd = POSIXCommands.get_command("remove_file", self.os)
        return f"{cmd} {path}"
    def remove_directory(self, path: str) -> str:
        """Build command to remove directory."""
        cmd = POSIXCommands.get_command("remove_directory", self.os)
        return f"{cmd} {path}"
    def copy_file(self, source: str, destination: str) -> str:
        """Build command to copy file."""
        cmd = POSIXCommands.get_command("copy_file", self.os)
        return f"{cmd} {source} {destination}"
    def move_file(self, source: str, destination: str) -> str:
        """Build command to move file."""
        cmd = POSIXCommands.get_command("move_file", self.os)
        return f"{cmd} {source} {destination}"
    def view_file(self, path: str) -> str:
        """Build command to view file."""
        cmd = POSIXCommands.get_command("view_file", self.os)
        return f"{cmd} {path}"
    def download_file(self, url: str, output_file: str | None = None) -> str:
        """Build command to download file."""
        cmd = POSIXCommands.get_command("download_file", self.os)
        if self.os == OS.WINDOWS or self.os == OS.MACOS:
            if output_file:
                return f"curl -o {output_file} {url}"
            return f"{cmd} {url}"
        else:
            if output_file:
                return f"{cmd} -O {url}"
            return f"{cmd} {url}"
```

```
def create_command_builder(target_os: OS | None = None) -> CommandBuilder:
    """Create a command builder for specific OS."""
    return CommandBuilder(target_os)
def get_command(unified_name: str, target_os: OS | None = None) -> str:
    """Convenience function to get OS-specific command."""
    return POSIXCommands.get_command(unified_name, target_os)
def get_python_command(unified_name: str) -> str | None:
    """Convenience function to get Python equivalent."""
    return POSIXCommands.get_python_equivalent(unified_name)
```

File: [122/252] openlaoke/core/prompt_input.py

```
"""Interactive prompt input with real-time autocomplete menu."""
from __future__ import annotations
from pathlib import Path
from prompt_toolkit import PromptSession
from prompt_toolkit.completion import Completer, Completion
from prompt_toolkit.history import FileHistory
from prompt_toolkit.styles import Style
from openlaoke.commands.registry import get_all_commands, register_all
from openlaoke.core.skill_system import get_skill_registry, list_available_skills
# History file location
HISTORY_FILE = Path.home() / ".openlaoke" / "command_history.txt"
def _get_skill_source(path) -> str:
    """Determine skill source from path."""
    if not path:
        return ""
    path_str = str(path)
    if ".config/opencode" in path_str or ".opencode/" in path_str:
        return "[OpenCode]"
    elif ".claude/" in path_str:
        return "[Claude]"
    elif ".openlaoke/" in path_str:
        return "[Installed]"
    return ""
class OpenLaoKeCompleter(Completer):
    """Custom completer for commands and skills."""
    def _get_options(self):
        """Get all available options, refreshing skills each time."""
        options = []
        # Commands
        register_all()
        seen = set()
        for cmd in get_all_commands():
            if cmd.hidden:
                continue
            if cmd.name not in seen:
                seen.add(cmd.name)
                options.append(
                    {
                        "name": cmd.name,
                        "display": f"/{cmd.name}",
                        "description": cmd.description[:60] if cmd.description else "",
                        "type": "command",
                        "source": "",
                    }
                )
                for alias in cmd.aliases:
                    if alias not in seen:
                        seen.add(alias)
                        options.append(
                            {
                                "name": alias,
                                "display": f"/{alias}",
                                "description": f"Alias for /{cmd.name}",
                                "type": "command",
                                "source": "",
                            }
                        )
        # Skills - always refresh to pick up newly installed
        registry = get_skill_registry()
        for skill_name in sorted(list_available_skills()):
            skill = registry.get_skill(skill_name)
```

```
        if skill:
            source = _get_skill_source(skill.path)
            desc = skill.description[:60].split("\n")[0] if skill.description else ""
            options.append(
                {
                    "name": skill_name,
                    "display": f"/{skill_name}",
                    "description": f"{source} {desc}" if source else desc,
                    "type": "skill",
                    "source": source,
                }
            )
    return options
def get_completions(self, document, complete_event):
    text = document.text_before_cursor
    if not text.startswith("/"):
        return
    # Get search text after /
    search = text[1:].lower()
    all_options = self._get_options()
    # Deduplicate by display name, preferring skills over commands
    # (skills have source tags, commands don't)
    seen = {}
    for opt in all_options:
        display = opt["display"]
        if display not in seen:
            seen[display] = opt
        elif opt["type"] == "skill" and seen[display]["type"] != "skill":
            # Prefer skill over command alias
            seen[display] = opt
    unique_options = list(seen.values())
    # Filter and sort
    matches = []
    for opt in unique_options:
        name_lower = opt["name"].lower()
        display_lower = opt["display"].lower()
        # Score matching
        score = 0
        if name_lower.startswith(search):
            score = 100
        elif search in name_lower:
            score = 80
        elif display_lower.startswith(search):
            score = 60
        elif search in display_lower:
            score = 40
        elif self._fuzzy_match(search, name_lower):
            score = 20
        if score > 0:
            matches.append((score, opt))
    # Sort by score, then by source (OpenCode first, then Claude, then Installed)
    source_order = {"[OpenCode]": 0, "[Claude]": 1, "[Installed]": 2, "": 3}
    matches.sort(
        key=lambda x: (
            -x[0],
            source_order.get(x[1]["source"], 3),
            x[1]["display"],
        )
    )
    # Return all matching completions (no artificial limit)
    for _, opt in matches:
        meta_text = opt["description"] or ""
        yield Completion(
            opt["display"],
            start_position=-len(text),
```

```
                display=opt["display"],
                display_meta=meta_text,
            )
    def _fuzzy_match(self, query, text):
        """Simple fuzzy match."""
        idx = 0
        for char in text:
            if idx < len(query) and char == query[idx]:
                idx += 1
        return idx == len(query)
def create_prompt_session():
    """Create a PromptSession with autocomplete and history."""
    # Ensure history directory exists
    HISTORY_FILE.parent.mkdir(parents=True, exist_ok=True)
    style = Style.from_dict(
        {
            "prompt": "bold ansigreen",
            "completion": "ansicyan",
            "completion.selected": "bold ansicyan",
            "completion-meta": "ansibrightblack",
        }
    )
    completer = OpenLaoKeCompleter()
    # Create file-based history for persistence
    history = FileHistory(str(HISTORY_FILE))
    session = PromptSession(
        completer=completer,
        style=style,
        complete_while_typing=True,
        mouse_support=True,
        history=history,
        enable_history_search=True,  # Enable up/down arrow navigation
    )
    return session
async def get_user_input(session: PromptSession) -> str | None:
    """Get input from user with autocomplete support."""
    try:
        result = await session.prompt_async("OpenLaoKe: ")
        return result.strip()
    except KeyboardInterrupt:
        return None
    except EOFError:
        return None
```

File: [123/252] openlaoke/core/query/__init__.py

```
"""AsyncGenerator Query Engine module.
This module provides a stream-based query execution engine that handles:
- Streaming API responses via AsyncGenerator
- Tool execution with progress tracking
- Context management and compaction
- Error recovery with retry mechanisms
Based on Claude Code's query.ts and QueryEngine.ts patterns.
"""
from __future__ import annotations
from openlaoke.core.query.context import (
    QueryContext,
    QueryTracking,
    TurnState,
    create_query_context,
)
from openlaoke.core.query.engine import (
    QueryEngine,
    run_query,
)
from openlaoke.core.query.events import (
    CompactBoundaryEvent,
    ContentBlockStartEvent,
    ContentBlockStopEvent,
    ContentDeltaEvent,
    ErrorEvent,
    MessageDeltaEvent,
    MessageEndEvent,
    MessageStartEvent,
    MessageStopEvent,
    QueryEvent,
    QueryEventType,
    QueryResult,
    StopReason,
    StreamRequestStartEvent,
    ThinkingDeltaEvent,
    ToolProgressEvent,
    ToolResultEvent,
    ToolUseEvent,
    TurnEndEvent,
)
from openlaoke.core.query.recovery import (
    MaxOutputTokensError,
    ModelFallbackError,
    PromptTooLongError,
    RecoveryError,
    RecoveryHandler,
    TimeoutHandler,
    categorize_error,
    create_missing_tool_result_blocks,
)
from openlaoke.core.query.stream import (
    StreamingState,
    StreamProcessor,
    merge_streams,
    stream_with_retry,
    stream_with_timeout,
)
__all__ = [
    "QueryEngine",
    "run_query",
    "QueryContext",
```

```
    "QueryTracking",
    "TurnState",
    "create_query_context",
    "QueryEvent",
    "QueryEventType",
    "QueryResult",
    "StopReason",
    "MessageStartEvent",
    "ContentDeltaEvent",
    "ThinkingDeltaEvent",
    "ContentBlockStartEvent",
    "ContentBlockStopEvent",
    "ToolUseEvent",
    "ToolResultEvent",
    "ToolProgressEvent",
    "MessageDeltaEvent",
    "MessageStopEvent",
    "MessageEndEvent",
    "ErrorEvent",
    "TurnEndEvent",
    "CompactBoundaryEvent",
    "StreamRequestStartEvent",
    "StreamProcessor",
    "StreamingState",
    "merge_streams",
    "stream_with_timeout",
    "stream_with_retry",
    "RecoveryHandler",
    "TimeoutHandler",
    "RecoveryError",
    "PromptTooLongError",
    "MaxOutputTokensError",
    "ModelFallbackError",
    "categorize_error",
    "create_missing_tool_result_blocks",
]
```

File: [124/252] openlaoke/core/query/context.py

```
"""Context management for query execution."""
from __future__ import annotations
import asyncio
import time
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from uuid import uuid4
from openlaoke.types.core_types import (
    CostInfo,
    Message,
    TokenUsage,
    ToolUseBlock,
)
from openlaoke.types.permissions import PermissionConfig
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
    from openlaoke.core.tool import Tool, ToolRegistry
@dataclass
class QueryTracking:
    """Tracking information for query chain."""
    chain_id: str = field(default_factory=lambda: uuid4().hex)
    depth: int = 0
@dataclass
class TurnState:
    """State for a single turn in the query loop."""
    turn_count: int = 1
    tool_use_blocks: list[ToolUseBlock] = field(default_factory=list)
    assistant_messages: list[Message] = field(default_factory=list)
    user_messages: list[Message] = field(default_factory=list)
    needs_follow_up: bool = False
    current_stop_reason: str | None = None
    current_usage: TokenUsage = field(default_factory=TokenUsage)
    accumulated_usage: TokenUsage = field(default_factory=TokenUsage)
    accumulated_cost: CostInfo = field(default_factory=CostInfo)
    max_output_tokens_recovery_count: int = 0
    has_attempted_reactive_compact: bool = False
@dataclass
class QueryContext:
    """Context for a query execution session."""
    app_state: AppState
    tool_registry: ToolRegistry
    messages: list[Message] = field(default_factory=list)
    system_prompt: str = ""
    user_context: dict[str, str] = field(default_factory=dict)
    system_context: dict[str, str] = field(default_factory=dict)
    permission_config: PermissionConfig = field(default_factory=PermissionConfig.defaults)
    model: str = "gemma3:1b"
    fallback_model: str | None = None
    max_tokens: int = 8192
    max_turns: int | None = None
    max_budget_usd: float | None = None
    temperature: float = 1.0
    thinking_budget: int = 0
    query_source: str = "repl"
    abort_controller: asyncio.Event = field(default_factory=asyncio.Event)
    query_tracking: QueryTracking = field(default_factory=QueryTracking)
    turn_state: TurnState = field(default_factory=TurnState)
    session_id: str = field(default_factory=lambda: uuid4().hex[:12])
    verbose: bool = False
    _start_time: float = field(default_factory=time.time)
    _pending_tool_summaries: dict[str, str] = field(default_factory=dict)
```

```
    def get_elapsed_ms(self) -> int:
        return int((time.time() - self._start_time) * 1000)
    def get_active_tools(self) -> list[Tool]:
        return self.tool_registry.get_all()
    def get_tool_schemas(self) -> list[dict[str, Any]]:
        return self.tool_registry.get_all_for_prompt()
    def is_aborted(self) -> bool:
        return self.abort_controller.is_set()
    def check_budget_exceeded(self) -> bool:
        if self.max_budget_usd is None:
            return False
        return self.turn_state.accumulated_cost.total_cost >= self.max_budget_usd
    def check_max_turns_exceeded(self) -> bool:
        if self.max_turns is None:
            return False
        return self.turn_state.turn_count >= self.max_turns
    def add_tool_use_block(self, block: ToolUseBlock) -> None:
        self.turn_state.tool_use_blocks.append(block)
        self.turn_state.needs_follow_up = True
    def clear_turn_state(self) -> None:
        self.turn_state = TurnState(turn_count=self.turn_state.turn_count + 1)
    def accumulate_usage(self, usage: TokenUsage) -> None:
        self.turn_state.current_usage = usage
        self.turn_state.accumulated_usage.accumulate(usage)
    def accumulate_cost(self, cost: CostInfo) -> None:
        self.turn_state.accumulated_cost.input_cost += cost.input_cost
        self.turn_state.accumulated_cost.output_cost += cost.output_cost
        self.turn_state.accumulated_cost.cache_read_cost += cost.cache_read_cost
        self.turn_state.accumulated_cost.cache_creation_cost += cost.cache_creation_cost
    def get_total_cost(self) -> float:
        return self.turn_state.accumulated_cost.total_cost
    def should_continue(self) -> bool:
        if self.is_aborted():
            return False
        if self.check_budget_exceeded():
            return False
        if self.check_max_turns_exceeded():
            return False
        return self.turn_state.needs_follow_up
    def to_dict(self) -> dict[str, Any]:
        return {
            "session_id": self.session_id,
            "model": self.model,
            "turn_count": self.turn_state.turn_count,
            "message_count": len(self.messages),
            "total_cost": self.get_total_cost(),
            "total_tokens": self.turn_state.accumulated_usage.total_tokens,
            "is_aborted": self.is_aborted(),
            "elapsed_ms": self.get_elapsed_ms(),
        }
def create_query_context(
    app_state: AppState,
    tool_registry: ToolRegistry,
    messages: list[Message] | None = None,
    system_prompt: str = "",
    model: str = "gemma3:1b",
    **kwargs: Any,
) -> QueryContext:
    """Factory for creating QueryContext with defaults."""
    return QueryContext(
        app_state=app_state,
        tool_registry=tool_registry,
        messages=messages or [],
        system_prompt=system_prompt,
        model=model,
```

```
    **kwargs,
)
```

File: [125/252] openlaoke/core/query/engine.py

```
"""AsyncGenerator QueryEngine - stream-based query execution."""
from __future__ import annotations
import asyncio
import json
from collections.abc import AsyncGenerator
from typing import TYPE_CHECKING, Any
from uuid import uuid4
from openlaoke.core.query.context import QueryContext, create_query_context
from openlaoke.core.query.events import (
    ContentDeltaEvent,
    ErrorEvent,
    MessageEndEvent,
    MessageStartEvent,
    QueryEvent,
    QueryEventType,
    QueryResult,
    StopReason,
    StreamRequestStartEvent,
    ToolProgressEvent,
    ToolResultEvent,
    ToolUseEvent,
    TurnEndEvent,
)
from openlaoke.core.query.recovery import (
    MaxOutputTokensError,
    PromptTooLongError,
    RecoveryError,
    RecoveryHandler,
    TimeoutHandler,
    categorize_error,
)
from openlaoke.core.query.stream import StreamProcessor
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import (
    AssistantMessage,
    CostInfo,
    Message,
    MessageRole,
    PermissionResult,
    TokenUsage,
    ToolResultBlock,
    ToolUseBlock,
    UserMessage,
)
if TYPE_CHECKING:
    from openlaoke.core.multi_provider_api import MultiProviderClient
    from openlaoke.core.state import AppState
class QueryEngine:
    """AsyncGenerator???? - ?????????
    Core implementation of the agentic loop using AsyncGenerator pattern.
    Handles streaming responses, tool execution, context management,
    and error recovery in a single unified flow.
    """
    def __init__(
        self,
        app_state: AppState,
        tool_registry: ToolRegistry,
        api_client: MultiProviderClient,
        system_prompt: str = "",
        model: str = "gemma3:1b",
        fallback_model: str | None = None,
```

```
    max_tokens: int = 8192,
    max_turns: int | None = None,
    max_budget_usd: float | None = None,
    temperature: float = 1.0,
    thinking_budget: int = 0,
    verbose: bool = False,
) -> None:
    self.app_state = app_state
    self.tool_registry = tool_registry
    self.api_client = api_client
    self.system_prompt = system_prompt
    self.model = model
    self.fallback_model = fallback_model
    self.max_tokens = max_tokens
    self.max_turns = max_turns
    self.max_budget_usd = max_budget_usd
    self.temperature = temperature
    self.thinking_budget = thinking_budget
    self.verbose = verbose
    self._abort_event = asyncio.Event()
    self._recovery_handler = RecoveryHandler()
    self._timeout_handler = TimeoutHandler()
    self._stream_processor = StreamProcessor()
    self._total_usage = TokenUsage()
    self._total_cost = CostInfo()
    self._turn_count = 0
    self._messages: list[Message] = []
async def query(
    self,
    messages: list[Message],
    tools: list[Tool] | None = None,
    **kwargs: Any,
) -> AsyncGenerator[QueryEvent, None]:
    """?????? - ?????
    Yields QueryEvent for each step in the agentic loop:
    - MESSAGE_START: when API call begins
    - CONTENT_DELTA: for streaming text
    - TOOL_USE: when tool call detected
    - TOOL_RESULT: when tool execution completes
    - MESSAGE_END: when assistant message complete
    - TURN_END: when turn ends (may continue)
    - ERROR: for recoverable/terminal errors
    """
    self._messages = list(messages)
    tools = tools or self.tool_registry.get_all()
    tool_schemas = self._build_tool_schemas(tools)
    context = create_query_context(
        app_state=self.app_state,
        tool_registry=self.tool_registry,
        messages=self._messages,
        system_prompt=self.system_prompt,
        model=self.model,
        fallback_model=self.fallback_model,
        max_tokens=self.max_tokens,
        max_turns=self.max_turns,
        max_budget_usd=self.max_budget_usd,
        abort_controller=self._abort_event,
    )
    while True:
        yield StreamRequestStartEvent()
        if context.is_aborted():
            yield ErrorEvent(
                error_message="Query aborted by user",
                error_type="aborted",
                is_retryable=False,
```

```
    )
    return
async for event in self._prepare_context(context):
    yield event
tool_calls: list[ToolUseBlock] = []
assistant_msg: AssistantMessage | None = None
try:
    async for event in self._sampling(
        context,
        tool_schemas,
    ):
        yield event
        if event.type == QueryEventType.TOOL_USE:
            tool_calls.append(
                ToolUseBlock(
                    id=event.data.get("tool_use_id", ""),
                    name=event.data.get("tool_name", ""),
                    input=event.data.get("tool_input", {}),
                )
            )
        if event.type == QueryEventType.MESSAGE_END:
            msg = event.data
            if msg and isinstance(msg, dict):
                role = msg.get("role")
                if role == "assistant":
                    assistant_msg = AssistantMessage(
                        role=MessageRole.ASSISTANT,
                        content=msg.get("content", ""),
                        tool_uses=tool_calls,
                        stop_reason=msg.get("stop_reason"),
                    )
                    context.accumulate_usage(TokenUsage())
                    context.accumulate_cost(CostInfo())
except RecoveryError as e:
    async for event in self._handle_recovery(context, e):
        yield event
    if context.should_continue():
        continue
    return
except Exception as e:
    categorized = categorize_error(e)
    yield ErrorEvent(
        error_message=str(e),
        error_type=categorized.error_type,
        is_retryable=categorized.is_retryable,
    )
    if categorized.is_retryable and self._recovery_handler.can_retry():
        continue
    return
if tool_calls:
    async for event in self._run_tools(context, tool_calls, tools):
        yield event
        if event.type == QueryEventType.TOOL_RESULT:
            result_block = event.data.get("result")
            if result_block:
                tool_result_msg: Message = UserMessage(
                    role=MessageRole.USER,
                    content=json.dumps(result_block),
                )
                context.messages.append(tool_result_msg)
self._turn_count += 1
yield TurnEndEvent(
    turn_count=self._turn_count,
    total_usage=context.turn_state.accumulated_usage,
    total_cost=context.turn_state.accumulated_cost,
```

```
            stop_reason=self._determine_stop_reason(context, assistant_msg),
        )
        if self._should_terminate(context, assistant_msg):
            return
        context.clear_turn_state()
async def _prepare_context(
    self,
    context: QueryContext,
) -> AsyncGenerator[QueryEvent, None]:
    """Prepare messages and context for API call."""
    messages_for_api = self._format_messages_for_api(context.messages)
    if len(messages_for_api) > self._estimate_token_limit(context):
        async for event in self._recovery_handler.handle_prompt_too_long(
            context.messages,
            len(json.dumps(messages_for_api)) // 4,
            context.max_tokens,
        ):
            yield event
        recovered = self._recovery_handler.get_recovery_messages()
        if recovered:
            context.messages = recovered
async def _sampling(
    self,
    context: QueryContext,
    tool_schemas: list[dict[str, Any]],
) -> AsyncGenerator[QueryEvent, None]:
    """API??????."""
    messages_for_api = self._format_messages_for_api(context.messages)
    yield MessageStartEvent(
        message_id=uuid4().hex,
        role="assistant",
        model=context.model,
    )
    self._stream_processor.reset()
    current_max_tokens = context.max_tokens
    if self._recovery_handler.should_escalate_tokens():
        current_max_tokens = self._recovery_handler.get_escalated_max_tokens()
    try:
        async for text, usage, cost in self.api_client.stream_message(
            system_prompt=self.system_prompt,
            messages=messages_for_api,
            tools=tool_schemas,
            model=context.model,
            max_tokens=current_max_tokens,
            temperature=self.temperature,
            thinking_budget=self.thinking_budget,
        ):
            if text:
                yield ContentDeltaEvent(
                    message_id=self._stream_processor.state.message_id,
                    content=text,
                    index=0,
                )
                self._stream_processor.state.current_content += text
            if usage:
                self._stream_processor.state.usage = usage
                self._total_usage.accumulate(usage)
            if cost:
                self._total_cost.input_cost += cost.input_cost
                self._total_cost.output_cost += cost.output_cost
        tool_uses = self._stream_processor.get_tool_uses()
        for i, tu in enumerate(tool_uses):
            yield ToolUseEvent.from_block(
                tu,
                self._stream_processor.state.message_id,
```

```
                i,
            )
        final_message: Message = AssistantMessage(
            role=MessageRole.ASSISTANT,
            content=self._stream_processor.state.current_content,
            tool_uses=tool_uses,
            stop_reason="end_turn" if not tool_uses else "tool_use",
        )
        yield MessageEndEvent(message=final_message)
    except TimeoutError:
        yield ErrorEvent(
            error_message="API call timed out",
            error_type="timeout",
            is_retryable=True,
        )
async def _run_tools(
    self,
    context: QueryContext,
    tool_calls: list[ToolUseBlock],
    tools: list[Tool],
) -> AsyncGenerator[QueryEvent, None]:
    """??????."""
    for tool_call in tool_calls:
        tool = self._find_tool(tool_call.name, tools)
        if not tool:
            yield ToolResultEvent(
                tool_use_id=tool_call.id,
                tool_name=tool_call.name,
                result=ToolResultBlock(
                    tool_use_id=tool_call.id,
                    content=f"Tool '{tool_call.name}' not found",
                    is_error=True,
                ),
            )
            continue
        permission_result = tool.check_permissions(
            tool_call.input,
            context.permission_config,
        )
        if permission_result != PermissionResult.ALLOW:
            yield ToolResultEvent(
                tool_use_id=tool_call.id,
                tool_name=tool_call.name,
                result=ToolResultBlock(
                    tool_use_id=tool_call.id,
                    content=tool.get_deny_message(tool_call.input),
                    is_error=True,
                ),
            )
            continue
        tool_ctx = ToolContext(
            app_state=context.app_state,
            tool_use_id=tool_call.id,
            abort_signal=context.abort_controller,
        )
        try:
            progress = tool.get_progress(tool_ctx, **tool_call.input)
            if progress:
                yield ToolProgressEvent(progress=progress)
            result = await self._timeout_handler.with_timeout(
                tool.call(tool_ctx, **tool_call.input),
                timeout_override=120.0,
            )
            yield ToolResultEvent(
                tool_use_id=tool_call.id,
```

```
                tool_name=tool_call.name,
                result=result,
            )
        except TimeoutError:
            yield ToolResultEvent(
                tool_use_id=tool_call.id,
                tool_name=tool_call.name,
                result=ToolResultBlock(
                    tool_use_id=tool_call.id,
                    content="Tool execution timed out",
                    is_error=True,
                ),
            )
        except Exception as e:
            yield ToolResultEvent(
                tool_use_id=tool_call.id,
                tool_name=tool_call.name,
                result=ToolResultBlock(
                    tool_use_id=tool_call.id,
                    content=str(e),
                    is_error=True,
                ),
            )
async def _handle_recovery(
    self,
    context: QueryContext,
    error: RecoveryError,
) -> AsyncGenerator[QueryEvent, None]:
    """Handle recovery from error."""
    yield ErrorEvent(
        error_message=str(error),
        error_type=error.error_type,
        is_retryable=error.is_retryable,
    )
    if isinstance(error, PromptTooLongError):
        async for event in self._recovery_handler.handle_prompt_too_long(
            context.messages,
            error.token_count,
            error.max_tokens,
        ):
            yield event
    elif isinstance(error, MaxOutputTokensError):
        async for event in self._recovery_handler.handle_max_output_tokens(
            context.messages,
            context.max_tokens,
        ):
            yield event
        if self._recovery_handler.should_escalate_tokens():
            context.max_tokens = self._recovery_handler.get_escalated_max_tokens()
def _should_terminate(
    self,
    context: QueryContext,
    last_message: AssistantMessage | None,
) -> bool:
    """Determine if query loop should terminate."""
    if context.is_aborted():
        return True
    if context.check_budget_exceeded():
        return True
    if context.check_max_turns_exceeded():
        return True
    if last_message:
        if last_message.stop_reason in ("end_turn", "stop_sequence"):
            return True
        if not last_message.tool_uses:
```

```
            return True
    return False
def _determine_stop_reason(
    self,
    context: QueryContext,
    last_message: AssistantMessage | None,
) -> StopReason:
    """Determine the stop reason for turn end."""
    if context.is_aborted():
        return StopReason.ABORTED
    if context.check_budget_exceeded():
        return StopReason.MAX_BUDGET
    if context.check_max_turns_exceeded():
        return StopReason.MAX_TURNS
    if last_message:
        if last_message.stop_reason == "tool_use":
            return StopReason.TOOL_USE
        if last_message.stop_reason == "end_turn":
            return StopReason.END_TURN
        if last_message.stop_reason == "max_tokens":
            return StopReason.MAX_TOKENS
    return StopReason.END_TURN
def _find_tool(self, name: str, tools: list[Tool]) -> Tool | None:
    """Find tool by name."""
    for tool in tools:
        if tool.name == name:
            return tool
    return None
def _build_tool_schemas(self, tools: list[Tool]) -> list[dict[str, Any]]:
    """Build tool schemas for API."""
    return [
        {
            "name": tool.name,
            "description": tool.description,
            "input_schema": tool.get_input_schema(),
        }
        for tool in tools
    ]
def _format_messages_for_api(self, messages: list[Message]) -> list[dict[str, Any]]:
    """Format messages for API request."""
    result = []
    for msg in messages:
        result.append(msg.to_dict())
    return result
def _estimate_token_limit(self, context: QueryContext) -> int:
    """Estimate safe token limit for context."""
    return context.max_tokens - 1024
def interrupt(self) -> None:
    """Interrupt the query loop."""
    self._abort_event.set()
def get_result(self) -> QueryResult:
    """Get final result after query completes."""
    return QueryResult(
        reason=StopReason.END_TURN,
        messages=self._messages,
        total_usage=self._total_usage,
        total_cost=self._total_cost,
        turn_count=self._turn_count,
    )
def get_messages(self) -> list[Message]:
    """Get accumulated messages."""
    return list(self._messages)
def get_usage(self) -> TokenUsage:
    """Get total token usage."""
    return self._total_usage
```

```
    def get_cost(self) -> CostInfo:
        """Get total cost."""
        return self._total_cost
async def run_query(
    app_state: AppState,
    tool_registry: ToolRegistry,
    api_client: MultiProviderClient,
    messages: list[Message],
    system_prompt: str = "",
    **kwargs: Any,
) -> AsyncGenerator[QueryEvent, None]:
    """Convenience wrapper for QueryEngine."""
    engine = QueryEngine(
        app_state=app_state,
        tool_registry=tool_registry,
        api_client=api_client,
        system_prompt=system_prompt,
        **kwargs,
    )
    async for event in engine.query(messages):
        yield event
```

File: [126/252] openlaoke/core/query/events.py

```
"""QueryEvent types and event system for AsyncGenerator query engine."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from enum import Enum
from typing import Any
from openlaoke.types.core_types import (
    CostInfo,
    Message,
    TokenUsage,
    ToolProgress,
    ToolResultBlock,
    ToolUseBlock,
)
class QueryEventType(Enum):
    """Types of events yielded during query execution."""
    MESSAGE_START = "message_start"
    CONTENT_DELTA = "content_delta"
    CONTENT_BLOCK_START = "content_block_start"
    CONTENT_BLOCK_STOP = "content_block_stop"
    TOOL_USE = "tool_use"
    TOOL_RESULT = "tool_result"
    TOOL_PROGRESS = "tool_progress"
    MESSAGE_DELTA = "message_delta"
    MESSAGE_STOP = "message_stop"
    MESSAGE_END = "message_end"
    ERROR = "error"
    TURN_END = "turn_end"
    COMPACT_BOUNDARY = "compact_boundary"
    STREAM_REQUEST_START = "stream_request_start"
    THINKING_DELTA = "thinking_delta"
class StopReason(Enum):
    """Reasons for stopping a query loop."""
    END_TURN = "end_turn"
    TOOL_USE = "tool_use"
    MAX_TOKENS = "max_tokens"
    STOP_SEQUENCE = "stop_sequence"
    ERROR = "error"
    ABORTED = "aborted"
    BLOCKING_LIMIT = "blocking_limit"
    PROMPT_TOO_LONG = "prompt_too_long"
    MAX_OUTPUT_TOKENS = "max_output_tokens"
    MAX_TURNS = "max_turns"
    MAX_BUDGET = "max_budget"
@dataclass
class QueryEvent:
    """Base event yielded during query execution."""
    type: QueryEventType
    data: dict[str, Any] = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    def to_dict(self) -> dict[str, Any]:
        return {
            "type": self.type.value,
            "data": self.data,
            "timestamp": self.timestamp,
        }
@dataclass
class MessageStartEvent(QueryEvent):
    """Event when a new message starts."""
    message_id: str = ""
    role: str = "assistant"
```

```
    model: str = ""
    type: QueryEventType = field(default=QueryEventType.MESSAGE_START, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "role": self.role,
            "model": self.model,
        }
@dataclass
class ContentDeltaEvent(QueryEvent):
    """Event for text content streaming."""
    message_id: str = ""
    content: str = ""
    index: int = 0
    type: QueryEventType = field(default=QueryEventType.CONTENT_DELTA, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "content": self.content,
            "index": self.index,
        }
@dataclass
class ThinkingDeltaEvent(QueryEvent):
    """Event for thinking content streaming."""
    message_id: str = ""
    content: str = ""
    index: int = 0
    type: QueryEventType = field(default=QueryEventType.THINKING_DELTA, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "content": self.content,
            "index": self.index,
        }
@dataclass
class ContentBlockStartEvent(QueryEvent):
    """Event when a content block starts."""
    message_id: str = ""
    index: int = 0
    block_type: str = "text"
    type: QueryEventType = field(default=QueryEventType.CONTENT_BLOCK_START, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "index": self.index,
            "block_type": self.block_type,
        }
@dataclass
class ContentBlockStopEvent(QueryEvent):
    """Event when a content block stops."""
    message_id: str = ""
    index: int = 0
    type: QueryEventType = field(default=QueryEventType.CONTENT_BLOCK_STOP, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "index": self.index,
        }
@dataclass
class ToolUseEvent(QueryEvent):
```

```
    """Event when a tool use block is detected."""
    tool_use_id: str = ""
    tool_name: str = ""
    tool_input: dict[str, Any] = field(default_factory=dict)
    message_id: str = ""
    index: int = 0
    type: QueryEventType = field(default=QueryEventType.TOOL_USE, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "tool_use_id": self.tool_use_id,
            "tool_name": self.tool_name,
            "tool_input": self.tool_input,
            "message_id": self.message_id,
            "index": self.index,
        }
    @classmethod
    def from_block(cls, block: ToolUseBlock, message_id: str, index: int) -> ToolUseEvent:
        return cls(
            tool_use_id=block.id,
            tool_name=block.name,
            tool_input=block.input,
            message_id=message_id,
            index=index,
        )
@dataclass
class ToolResultEvent(QueryEvent):
    """Event when a tool execution result is ready."""
    tool_use_id: str = ""
    tool_name: str = ""
    result: ToolResultBlock | None = None
    message_id: str = ""
    type: QueryEventType = field(default=QueryEventType.TOOL_RESULT, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "tool_use_id": self.tool_use_id,
            "tool_name": self.tool_name,
            "result": self.result.to_dict() if self.result else None,
            "message_id": self.message_id,
        }
@dataclass
class ToolProgressEvent(QueryEvent):
    """Event for tool execution progress updates."""
    progress: ToolProgress | None = None
    type: QueryEventType = field(default=QueryEventType.TOOL_PROGRESS, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        if self.progress:
            self.data = {
                "tool_use_id": self.progress.tool_use_id,
                "tool_name": self.progress.tool_name,
                "message": self.progress.message,
                "percentage": self.progress.percentage,
            }
@dataclass
class MessageDeltaEvent(QueryEvent):
    """Event for message-level updates (stop_reason, usage)."""
    message_id: str = ""
    stop_reason: str | None = None
    usage: TokenUsage | None = None
    type: QueryEventType = field(default=QueryEventType.MESSAGE_DELTA, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
```

```
            "message_id": self.message_id,
            "stop_reason": self.stop_reason,
            "usage": self.usage.to_dict() if self.usage else None,
        }
@dataclass
class MessageStopEvent(QueryEvent):
    """Event when a message stops streaming."""
    message_id: str = ""
    stop_reason: str | None = None
    type: QueryEventType = field(default=QueryEventType.MESSAGE_STOP, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "message_id": self.message_id,
            "stop_reason": self.stop_reason,
        }
@dataclass
class MessageEndEvent(QueryEvent):
    """Event when a complete message is ready."""
    message: Message | None = None
    type: QueryEventType = field(default=QueryEventType.MESSAGE_END, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        if self.message:
            self.data = self.message.to_dict()
@dataclass
class ErrorEvent(QueryEvent):
    """Event for errors during query execution."""
    error_message: str = ""
    error_type: str = ""
    is_retryable: bool = False
    retry_count: int = 0
    type: QueryEventType = field(default=QueryEventType.ERROR, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "error_message": self.error_message,
            "error_type": self.error_type,
            "is_retryable": self.is_retryable,
            "retry_count": self.retry_count,
        }
@dataclass
class TurnEndEvent(QueryEvent):
    """Event marking the end of a turn in the agentic loop."""
    turn_count: int = 0
    total_usage: TokenUsage | None = None
    total_cost: CostInfo | None = None
    stop_reason: StopReason = StopReason.END_TURN
    type: QueryEventType = field(default=QueryEventType.TURN_END, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
    def __post_init__(self) -> None:
        self.data = {
            "turn_count": self.turn_count,
            "total_usage": self.total_usage.to_dict() if self.total_usage else None,
            "total_cost": self.total_cost.total_cost if self.total_cost else 0.0,
            "stop_reason": self.stop_reason.value,
        }
@dataclass
class CompactBoundaryEvent(QueryEvent):
    """Event marking a context compact boundary."""
    pre_compact_tokens: int = 0
    post_compact_tokens: int = 0
    compacted_message_count: int = 0
    type: QueryEventType = field(default=QueryEventType.COMPACT_BOUNDARY, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
```

```
    def __post_init__(self) -> None:
        self.data = {
            "pre_compact_tokens": self.pre_compact_tokens,
            "post_compact_tokens": self.post_compact_tokens,
            "compacted_message_count": self.compacted_message_count,
        }
@dataclass
class StreamRequestStartEvent(QueryEvent):
    """Event when a stream request starts."""
    type: QueryEventType = field(default=QueryEventType.STREAM_REQUEST_START, init=False)
    data: dict[str, Any] = field(default_factory=dict, init=False)
class QueryResult:
    """Final result of a query execution."""
    def __init__(
        self,
        reason: StopReason,
        messages: list[Message] | None = None,
        total_usage: TokenUsage | None = None,
        total_cost: CostInfo | None = None,
        turn_count: int = 0,
        error: Exception | None = None,
    ) -> None:
        self.reason = reason
        self.messages = messages or []
        self.total_usage = total_usage or TokenUsage()
        self.total_cost = total_cost or CostInfo()
        self.turn_count = turn_count
        self.error = error
    def to_dict(self) -> dict[str, Any]:
        return {
            "reason": self.reason.value,
            "messages": [m.to_dict() for m in self.messages],
            "total_usage": self.total_usage.to_dict(),
            "total_cost": self.total_cost.total_cost,
            "turn_count": self.turn_count,
            "error": str(self.error) if self.error else None,
        }
```

File: [127/252] openlaoke/core/query/recovery.py

```
"""Error recovery mechanisms for query execution."""
from __future__ import annotations
import asyncio
import json
from collections.abc import AsyncGenerator
from dataclasses import dataclass
from typing import Any
from openlaoke.core.query.events import (
    CompactBoundaryEvent,
    ErrorEvent,
    QueryEvent,
)
from openlaoke.types.core_types import (
    Message,
    MessageRole,
    UserMessage,
)
class RecoveryError(Exception):
    """Error that may be recoverable."""
    def __init__(
        self,
        message: str,
        error_type: str = "unknown",
        is_retryable: bool = False,
        recovery_hint: str | None = None,
    ) -> None:
        super().__init__(message)
        self.error_type = error_type
        self.is_retryable = is_retryable
        self.recovery_hint = recovery_hint
class PromptTooLongError(RecoveryError):
    """Context exceeds model limits."""
    def __init__(self, token_count: int, max_tokens: int) -> None:
        super().__init__(
            f"Prompt too long: {token_count} tokens exceeds {max_tokens}",
            error_type="prompt_too_long",
            is_retryable=True,
            recovery_hint="compact_context",
        )
        self.token_count = token_count
        self.max_tokens = max_tokens
class MaxOutputTokensError(RecoveryError):
    """Output exceeded max_tokens limit."""
    def __init__(self) -> None:
        super().__init__(
            "Output exceeded max_tokens limit",
            error_type="max_output_tokens",
            is_retryable=True,
            recovery_hint="increase_max_tokens",
        )
class ModelFallbackError(RecoveryError):
    """Fallback to alternate model needed."""
    def __init__(self, original_model: str, fallback_model: str) -> None:
        super().__init__(
            f"Fallback from {original_model} to {fallback_model}",
            error_type="model_fallback",
            is_retryable=True,
            recovery_hint="switch_model",
        )
        self.original_model = original_model
        self.fallback_model = fallback_model
```

```
@dataclass
class RecoveryState:
    """State for recovery attempt."""
    attempt_count: int = 0
    max_attempts: int = 3
    last_error: RecoveryError | None = None
    recovery_messages: list[Message] | None = None
    escalated_max_tokens: int = 0
class RecoveryHandler:
    """Handle various recovery scenarios."""
    MAX_OUTPUT_TOKENS_RECOVERY_LIMIT = 3
    ESCALATED_MAX_TOKENS = 65536
    DEFAULT_MAX_TOKENS = 8192
    def __init__(self) -> None:
        self.state = RecoveryState()
    def reset(self) -> None:
        self.state = RecoveryState()
    async def handle_prompt_too_long(
        self,
        messages: list[Message],
        context_tokens: int,
        max_context_tokens: int,
    ) -> AsyncGenerator[QueryEvent, None]:
        """Handle context overflow via compacting."""
        self.state.attempt_count += 1
        self.state.last_error = PromptTooLongError(context_tokens, max_context_tokens)
        if self.state.attempt_count > self.state.max_attempts:
            yield ErrorEvent(
                error_message=f"Context overflow after {self.state.attempt_count} attempts",
                error_type="prompt_too_long",
                is_retryable=False,
            )
            return
        compacted = await self._compact_messages(messages, max_context_tokens)
        yield CompactBoundaryEvent(
            pre_compact_tokens=context_tokens,
            post_compact_tokens=len(json.dumps([m.to_dict() for m in compacted])) // 4,
            compacted_message_count=len(messages) - len(compacted),
        )
        self.state.recovery_messages = compacted
    async def handle_max_output_tokens(
        self,
        messages: list[Message],
        current_max_tokens: int,
    ) -> AsyncGenerator[QueryEvent, None]:
        """Handle output token limit hit."""
        self.state.attempt_count += 1
        self.state.last_error = MaxOutputTokensError()
        if self.state.attempt_count > self.MAX_OUTPUT_TOKENS_RECOVERY_LIMIT:
            yield ErrorEvent(
                error_message="Max output tokens recovery limit reached",
                error_type="max_output_tokens",
                is_retryable=False,
            )
            return
        if current_max_tokens == self.DEFAULT_MAX_TOKENS:
            self.state.escalated_max_tokens = self.ESCALATED_MAX_TOKENS
        else:
            recovery_message: Message = UserMessage(
                role=MessageRole.USER,
                content=(
                    "Output token limit hit. Resume directly - no apology, "
                    "no recap of what you were doing. Pick up mid-thought "
                    "if that is where the cut happened. Break remaining "
                    "work into smaller pieces."
```

```
                ),
            )
            messages.append(recovery_message)
            self.state.recovery_messages = messages
    async def handle_model_fallback(
        self,
        original_model: str,
        fallback_model: str,
    ) -> AsyncGenerator[QueryEvent, None]:
        """Handle model fallback scenario."""
        self.state.last_error = ModelFallbackError(original_model, fallback_model)
        yield ErrorEvent(
            error_message=f"Switching to {fallback_model} due to error with {original_model}",
            error_type="model_fallback",
            is_retryable=True,
        )
    async def _compact_messages(
        self,
        messages: list[Message],
        target_tokens: int,
    ) -> list[Message]:
        """Compact messages to target token count."""
        if not messages:
            return []
        estimated_tokens = len(json.dumps([m.to_dict() for m in messages])) // 4
        if estimated_tokens <= target_tokens:
            return messages
        keep_ratio = target_tokens / estimated_tokens
        keep_count = max(1, int(len(messages) * keep_ratio))
        preserved = messages[-keep_count:]
        return preserved
    def get_recovery_messages(self) -> list[Message] | None:
        return self.state.recovery_messages
    def get_escalated_max_tokens(self) -> int:
        return self.state.escalated_max_tokens
    def should_escalate_tokens(self) -> bool:
        return self.state.escalated_max_tokens > 0
    def can_retry(self) -> bool:
        return self.state.attempt_count < self.state.max_attempts
class TimeoutHandler:
    """Handle timeout scenarios with retry."""
    def __init__(self, base_timeout: float = 300.0, max_retries: int = 3) -> None:
        self.base_timeout = base_timeout
        self.max_retries = max_retries
        self.retry_count = 0
    async def with_timeout(
        self,
        coro: Any,
        timeout_override: float | None = None,
    ) -> Any:
        """Execute with timeout and retry on timeout error."""
        timeout = timeout_override or self.base_timeout
        while self.retry_count < self.max_retries:
            try:
                async with asyncio.timeout(timeout):
                    return await coro
            except TimeoutError:
                self.retry_count += 1
                if self.retry_count < self.max_retries:
                    await asyncio.sleep(self.retry_count * 2.0)
                    timeout = min(timeout * 1.5, 600.0)
                else:
                    raise RecoveryError(
                        f"Operation timed out after {self.retry_count} retries",
                        error_type="timeout",
```

```
                        is_retryable=False,
                    ) from None
        raise RecoveryError(
            "Max retries exceeded",
            error_type="timeout",
            is_retryable=False,
        )
    def reset(self) -> None:
        self.retry_count = 0
async def create_missing_tool_result_blocks(
    tool_use_ids: list[str],
    error_message: str,
) -> list[Message]:
    """Create error tool results for orphaned tool uses."""
    results: list[Message] = []
    for tool_id in tool_use_ids:
        msg: Message = UserMessage(
            role=MessageRole.USER,
            content=json.dumps(
                {
                    "type": "tool_result",
                    "tool_use_id": tool_id,
                    "content": error_message,
                    "is_error": True,
                }
            ),
        )
        results.append(msg)
    return results
def categorize_error(error: Exception) -> RecoveryError:
    """Categorize an error for recovery handling."""
    error_str = str(error)
    if "prompt too long" in error_str.lower() or "context length" in error_str.lower():
        return PromptTooLongError(0, 0)
    if "max_tokens" in error_str.lower() or "output limit" in error_str.lower():
        return MaxOutputTokensError()
    if "timeout" in error_str.lower():
        return RecoveryError(
            error_str,
            error_type="timeout",
            is_retryable=True,
            recovery_hint="retry_with_delay",
        )
    if "rate limit" in error_str.lower() or "429" in error_str:
        return RecoveryError(
            error_str,
            error_type="rate_limit",
            is_retryable=True,
            recovery_hint="retry_with_backoff",
        )
    if "model" in error_str.lower() and "fallback" in error_str.lower():
        return ModelFallbackError("unknown", "unknown")
    return RecoveryError(
        error_str,
        error_type="unknown",
        is_retryable=False,
    )
```

File: [128/252] openlaoke/core/query/stream.py

```
"""Stream processing utilities for AsyncGenerator query engine."""
from __future__ import annotations
import asyncio
import json
import time
from collections.abc import AsyncGenerator, Callable
from dataclasses import dataclass, field
from typing import Any
from uuid import uuid4
from openlaoke.core.query.events import (
    ContentBlockStartEvent,
    ContentBlockStopEvent,
    ContentDeltaEvent,
    ErrorEvent,
    MessageDeltaEvent,
    MessageEndEvent,
    MessageStartEvent,
    MessageStopEvent,
    QueryEvent,
    QueryEventType,
    ThinkingDeltaEvent,
    ToolUseEvent,
)
from openlaoke.types.core_types import (
    AssistantMessage,
    MessageRole,
    TokenUsage,
    ToolUseBlock,
)
@dataclass
class StreamingState:
    """State tracking for streaming response."""
    message_id: str = field(default_factory=lambda: uuid4().hex)
    current_content: str = ""
    current_thinking: str = ""
    content_blocks: list[dict[str, Any]] = field(default_factory=list)
    tool_uses: list[ToolUseBlock] = field(default_factory=list)
    current_block_index: int = 0
    current_block_type: str = ""
    stop_reason: str | None = None
    usage: TokenUsage | None = None
    is_thinking_enabled: bool = False
class StreamProcessor:
    """Process streaming events from API into QueryEvents."""
    def __init__(self, message_id: str | None = None) -> None:
        self.state = StreamingState(message_id=message_id or uuid4().hex)
        self._block_start_time: float = 0.0
    def reset(self) -> None:
        self.state = StreamingState()
    async def process_anthropic_event(
        self, event: dict[str, Any]
    ) -> AsyncGenerator[QueryEvent, None]:
        """Process Anthropic-style streaming event."""
        event_type = event.get("type", "")
        if event_type == "message_start":
            message_data = event.get("message", {})
            self.state.message_id = message_data.get("id", uuid4().hex)
            self.state.usage = TokenUsage(
                input_tokens=message_data.get("usage", {}).get("input_tokens", 0),
                output_tokens=0,
            )
```

```
        yield MessageStartEvent(
            message_id=self.state.message_id,
            role="assistant",
            model=message_data.get("model", ""),
        )
    elif event_type == "content_block_start":
        index = event.get("index", 0)
        block = event.get("content_block", {})
        block_type = block.get("type", "text")
        self.state.current_block_index = index
        self.state.current_block_type = block_type
        self._block_start_time = time.time()
        yield ContentBlockStartEvent(
            message_id=self.state.message_id,
            index=index,
            block_type=block_type,
        )
        if block_type == "tool_use":
            tool_block = ToolUseBlock(
                id=block.get("id", uuid4().hex),
                name=block.get("name", ""),
                input={},
            )
            self.state.tool_uses.append(tool_block)
            yield ToolUseEvent(
                tool_use_id=tool_block.id,
                tool_name=tool_block.name,
                tool_input={},
                message_id=self.state.message_id,
                index=index,
            )
    elif event_type == "content_block_delta":
        index = event.get("index", 0)
        delta = event.get("delta", {})
        delta_type = delta.get("type", "text_delta")
        if delta_type == "text_delta":
            text = delta.get("text", "")
            self.state.current_content += text
            yield ContentDeltaEvent(
                message_id=self.state.message_id,
                content=text,
                index=index,
            )
        elif delta_type == "thinking_delta" or delta_type == "input_json_delta":
            text = delta.get("thinking", delta.get("partial_json", ""))
            if delta_type == "thinking_delta":
                self.state.current_thinking += text
                yield ThinkingDeltaEvent(
                    message_id=self.state.message_id,
                    content=text,
                    index=index,
                )
            else:
                if self.state.tool_uses and index < len(self.state.tool_uses):
                    try:
                        partial = json.loads(text)
                        self.state.tool_uses[index].input.update(partial)
                    except json.JSONDecodeError:
                        pass
    elif event_type == "content_block_stop":
        index = event.get("index", 0)
        yield ContentBlockStopEvent(
            message_id=self.state.message_id,
            index=index,
        )
```

```
        if self.state.current_block_type == "text":
            self.state.content_blocks.append(
                {
                    "type": "text",
                    "text": self.state.current_content,
                }
            )
            self.state.current_content = ""
        elif self.state.current_block_type == "thinking":
            self.state.content_blocks.append(
                {
                    "type": "thinking",
                    "thinking": self.state.current_thinking,
                }
            )
            self.state.current_thinking = ""
    elif event_type == "message_delta":
        delta = event.get("delta", {})
        usage_data = event.get("usage", {})
        self.state.stop_reason = delta.get("stop_reason")
        if usage_data:
            output_tokens = usage_data.get("output_tokens", 0)
            if self.state.usage:
                self.state.usage.output_tokens = output_tokens
                self.state.usage.cache_read_tokens = usage_data.get(
                    "cache_read_input_tokens", 0
                )
                self.state.usage.cache_creation_tokens = usage_data.get(
                    "cache_creation_input_tokens", 0
                )
        yield MessageDeltaEvent(
            message_id=self.state.message_id,
            stop_reason=self.state.stop_reason,
            usage=self.state.usage,
        )
    elif event_type == "message_stop":
        yield MessageStopEvent(
            message_id=self.state.message_id,
            stop_reason=self.state.stop_reason,
        )
        message = self._build_assistant_message()
        yield MessageEndEvent(message=message)
async def process_openai_event(self, event: dict[str, Any]) -> AsyncGenerator[QueryEvent, N
one]:
    """Process OpenAI-style streaming event."""
    choices = event.get("choices", [])
    if not choices:
        usage_data = event.get("usage", {})
        if usage_data:
            self.state.usage = TokenUsage(
                input_tokens=usage_data.get("prompt_tokens", 0),
                output_tokens=usage_data.get("completion_tokens", 0),
            )
            yield MessageDeltaEvent(
                message_id=self.state.message_id,
                usage=self.state.usage,
            )
        return
    choice = choices[0]
    delta = choice.get("delta", {})
    finish_reason = choice.get("finish_reason")
    if finish_reason:
        self.state.stop_reason = finish_reason
        yield MessageDeltaEvent(
            message_id=self.state.message_id,
```

```
            stop_reason=self.state.stop_reason,
        )
    content = delta.get("content")
    if content:
        self.state.current_content += content
        yield ContentDeltaEvent(
            message_id=self.state.message_id,
            content=content,
            index=0,
        )
    tool_calls = delta.get("tool_calls", [])
    for i, tc in enumerate(tool_calls):
        func = tc.get("function", {})
        tool_id = tc.get("id", uuid4().hex)
        tool_name = func.get("name", "")
        args_str = func.get("arguments", "{}")
        try:
            args = json.loads(args_str)
        except json.JSONDecodeError:
            args = {}
        existing = None
        for tu in self.state.tool_uses:
            if tu.id == tool_id:
                existing = tu
                break
        if existing:
            existing.input.update(args)
        else:
            new_block = ToolUseBlock(id=tool_id, name=tool_name, input=args)
            self.state.tool_uses.append(new_block)
            yield ToolUseEvent(
                tool_use_id=tool_id,
                tool_name=tool_name,
                tool_input=args,
                message_id=self.state.message_id,
                index=i,
            )
    if finish_reason == "stop":
        yield MessageStopEvent(
            message_id=self.state.message_id,
            stop_reason=self.state.stop_reason,
        )
        message = self._build_assistant_message()
        yield MessageEndEvent(message=message)
def _build_assistant_message(self) -> AssistantMessage:
    """Build final assistant message from streaming state."""
    content_parts = []
    for block in self.state.content_blocks:
        content_parts.append(block)
    for tu in self.state.tool_uses:
        content_parts.append(tu.to_dict())
    if not content_parts and self.state.current_content:
        content_parts.append({"type": "text", "text": self.state.current_content})
    return AssistantMessage(
        role=MessageRole.ASSISTANT,
        content=self.state.current_content,
        tool_uses=self.state.tool_uses,
        stop_reason=self.state.stop_reason,
    )
def get_usage(self) -> TokenUsage:
    return self.state.usage or TokenUsage()
def get_tool_uses(self) -> list[ToolUseBlock]:
    return self.state.tool_uses
def has_tool_uses(self) -> bool:
    return len(self.state.tool_uses) > 0
```

```
async def merge_streams(
    *generators: AsyncGenerator[QueryEvent, None],
) -> AsyncGenerator[QueryEvent, None]:
    """Merge multiple async generators into one."""
    for gen in generators:
        async for event in gen:
            yield event
async def stream_with_timeout(
    generator: AsyncGenerator[QueryEvent, None],
    timeout_seconds: float = 300.0,
) -> AsyncGenerator[QueryEvent, None]:
    """Wrap stream with timeout."""
    try:
        async with asyncio.timeout(timeout_seconds):
            async for event in generator:
                yield event
    except TimeoutError:
        yield ErrorEvent(
            error_message=f"Stream timed out after {timeout_seconds}s",
            error_type="timeout",
            is_retryable=True,
        )
async def stream_with_retry(
    generator_factory: Callable[[], AsyncGenerator[QueryEvent, None]],
    max_retries: int = 3,
    retry_delay: float = 1.0,
) -> AsyncGenerator[QueryEvent, None]:
    """Wrap stream with retry logic."""
    retry_count = 0
    while retry_count < max_retries:
        try:
            async for event in generator_factory():
                if event.type == QueryEventType.ERROR and event.data.get("is_retryable"):
                    retry_count += 1
                    if retry_count < max_retries:
                        await asyncio.sleep(retry_delay * retry_count)
                        continue
                yield event
                if event.type == QueryEventType.MESSAGE_END:
                    return
            return
        except Exception as e:
            retry_count += 1
            yield ErrorEvent(
                error_message=str(e),
                error_type="exception",
                is_retryable=True,
                retry_count=retry_count,
            )
            if retry_count < max_retries:
                await asyncio.sleep(retry_delay * retry_count)
```

File: [129/252] openlaoke/core/repl.py

```
"""REPL loop - the main interaction loop."""
from __future__ import annotations
import asyncio
import json
import os
from typing import Any
import httpx
from rich.console import Console
from rich.markdown import Markdown
from rich.panel import Panel
from openlaoke.commands.registry import get_command, parse_command, register_all
from openlaoke.core.config_wizard import get_proxy_url
from openlaoke.core.multi_provider_api import MultiProviderClient
from openlaoke.core.prompt_input import create_prompt_session, get_user_input
from openlaoke.core.state import AppState
from openlaoke.core.supervisor import TaskSupervisor
from openlaoke.core.system_prompt import build_system_prompt
from openlaoke.core.tool import ToolContext, ToolRegistry
from openlaoke.tools.register import register_all_tools
from openlaoke.types.core_types import (
    MessageRole,
    PermissionResult,
    UserMessage,
)
from openlaoke.types.providers import MultiProviderConfig
class REPL:
    """Main REPL loop for OpenLaoKe."""
    def __init__(self, app_state: AppState) -> None:
        self.app_state = app_state
        self.console = Console(force_terminal=True)
        self.registry = ToolRegistry()
        self.api: MultiProviderClient | None = None
        self._running = False
        self.multi_provider_config: MultiProviderConfig | None = None
        self.app_config: Any = None
        self._proxy: str | None = None
        self._prompt_session = None
        self.supervisor = TaskSupervisor(app_state)
        self._current_task_id: str | None = None
        register_all()
        register_all_tools(self.registry)
    async def run(self) -> None:
        """Start the REPL loop."""
        self._running = True
        self._prompt_session = create_prompt_session()
        self._print_banner()
        self._print_welcome()
        config = self.multi_provider_config or self.app_state.multi_provider_config
        if not config:
            self.console.print("[bold red]Error: No provider configured.[/bold red]")
            self.console.print("Run 'openlaoke --config' to set up a provider.")
            return
        if self.app_config:
            self._proxy = get_proxy_url(self.app_config)
        self.api = MultiProviderClient(config, proxy=self._proxy)
        try:
            while self._running:
                await self._handle_input()
        except (KeyboardInterrupt, EOFError):
            self.console.print("\n[yellow]Goodbye![/yellow]")
        finally:
```

```
        if self.api:
            await self.api.close()
async def _handle_input(self) -> None:
    """Get input from user and process it."""
    self.console.print()
    user_input = await get_user_input(self._prompt_session)
    if user_input is None:
        self._running = False
        return
    user_input = user_input.strip()
    if not user_input:
        return
    if user_input.startswith("/"):
        from openlaoke.core.skill_system import load_skill
        parts = user_input.split(None, 1)
        potential_name = parts[0][1:] if parts else ""
        skill_args = parts[1] if len(parts) > 1 else ""
        skill = load_skill(potential_name)
        if skill:
            if (
                hasattr(self.app_state, "active_skills")
                and potential_name not in self.app_state.active_skills
            ):
                self.app_state.active_skills.append(potential_name)
            self.console.print(f"[green][OK] Skill activated: {skill.name}[/green]")
            if skill.description:
                desc = skill.description[:100]
                if len(skill.description) > 100:
                    desc += "..."
                self.console.print(f"  [dim]{desc}[/dim]")
            if skill_args:
                self.console.print()
                await self._handle_chat(skill_args)
            return
    cmd = parse_command(user_input)
    if cmd:
        name, args = cmd
        await self._handle_command(name, args)
        return
    await self._handle_chat(user_input)
async def _handle_command(self, name: str, args: str) -> None:
    """Execute a slash command."""
    from openlaoke.commands.base import CommandContext
    from openlaoke.commands.skill_shortcuts import SkillActivationResult
    command = get_command(name)
    if not command:
        self.console.print(f"[red]Unknown command: /{name}[/red]")
        self.console.print("Type [bold]/help[/bold] for available commands.")
        return
    ctx = CommandContext(app_state=self.app_state, args=args)
    result = await command.execute(ctx)
    if result.message:
        self.console.print(result.message)
    if result.should_exit:
        self._running = False
    if result.should_clear:
        self.console.clear()
    if isinstance(result, SkillActivationResult) and result.should_continue_chat:
        self.console.print()
        await self._handle_chat(args)
async def _handle_chat(self, user_input: str) -> None:
    """Process a chat message through the AI model."""
    from openlaoke.core.intent_parser import IntentParser, IntentType
    if self.app_state.local_mode:
        parser = IntentParser()
```

```
        intent = parser.parse(user_input)
        if intent.intent_type in [
            IntentType.WRITE_PROGRAM,
            IntentType.WRITE_FUNCTION,
            IntentType.WRITE_CLASS,
        ]:
            await self._handle_command("atomic", user_input)
            return
    self.app_state.is_running = True
    self.app_state.set_error(None)
    if self.app_state.local_mode:
        self.supervisor.parse_request(user_input)
        self._current_task_id = (
            list(self.supervisor.tasks.keys())[-1] if self.supervisor.tasks else None
        )
    else:
        self._current_task_id = None
    user_msg = UserMessage(role=MessageRole.USER, content=user_input)
    self.app_state.add_message(user_msg)
    max_retry_attempts = 3
    retry_count = 0
    while retry_count < max_retry_attempts:
        try:
            await self._run_api_loop()
            artifacts = self._collect_artifacts()
            if self._current_task_id:
                result = await self.supervisor.check_completion(
                    self._current_task_id, artifacts
                )
                if result.is_complete:
                    self.console.print(
                        f"\n[green][OK] Task completed ({result.completion_percentage:.0f}%
)[/green]"
                    )
                    break
                if result.should_retry:
                    retry_count += 1
                    self.console.print(
                        f"\n[yellow]? Task incomplete ({result.completion_percentage:.0f}%)
[/yellow]"
                    )
                    if result.feedback:
                        self.console.print(
                            Panel(
                                result.feedback,
                                title="Supervisor Feedback",
                                border_style="yellow",
                            )
                        )
                    retry_prompt = self.supervisor.get_retry_prompt(
                        self._current_task_id, result
                    )
                    if retry_count < max_retry_attempts:
                        self.console.print(
                            f"\n[cyan]Retrying... (attempt {retry_count}/{max_retry_attempt
s})[/cyan]"
                        )
                        user_msg = UserMessage(role=MessageRole.USER, content=retry_prompt)
                        self.app_state.add_message(user_msg)
                        continue
                    else:
                        self.console.print("\n[red]Max retries reached. Task incomplete.[/r
ed]")
                        self.console.print(
                            f"[dim]Missing: {', '.join(result.missing_requirements[:3])}[/d
```

```
im]"
                        )
                        break
                else:
                    break
            else:
                break
        except Exception as e:
            self.console.print(f"\n[bold red]Error:[/bold red] {e}")
            self.app_state.set_error(str(e))
            retry_count += 1
            if retry_count >= max_retry_attempts:
                break
            self.console.print("\n[yellow]Retrying after error...[/yellow]")
    else:
        self.console.print(
            f"\n[red]Failed to complete task after {max_retry_attempts} attempts[/red]"
        )
    self.app_state.is_running = False
def _collect_artifacts(self) -> dict[str, Any]:
    """Collect artifacts from the conversation for supervision check."""
    artifacts = {
        "content": "",
        "output_files": [],
    }
    for msg in reversed(self.app_state.messages):
        if msg.role == MessageRole.ASSISTANT and msg.content:
            artifacts["content"] += msg.content + "\n\n"
    cwd = self.app_state.get_cwd()
    common_outputs = ["Article.md", "article.md", "README.md", "output.md"]
    for filename in common_outputs:
        filepath = f"{cwd}/{filename}"
        if os.path.exists(filepath):
            artifacts["output_files"].append(filepath)
    import glob
    for pattern in ["*.svg", "*.png", "*.pdf"]:
        files = glob.glob(f"{cwd}/{pattern}")
        artifacts["output_files"].extend(files)
    return artifacts
async def _run_api_loop(self) -> None:
    """Main API interaction loop."""
    max_iterations = 100
    iteration = 0
    failed_tool_calls: dict[str, int] = {}
    messages: list[dict[str, Any]] = []
    for msg in self.app_state.messages:
        if msg.role == MessageRole.USER:
            messages.append({"role": "user", "content": msg.content})
        elif msg.role == MessageRole.ASSISTANT:
            assistant_msg: dict[str, Any] = {"role": "assistant", "content": msg.content}
            if hasattr(msg, "tool_uses") and msg.tool_uses:
                assistant_msg["tool_calls"] = [
                    {
                        "id": tu.id,
                        "type": "function",
                        "function": {
                            "name": tu.name,
                            "arguments": json.dumps(tu.input),
                        },
                    }
                    for tu in msg.tool_uses
                ]
            messages.append(assistant_msg)
        elif msg.role == MessageRole.SYSTEM and hasattr(msg, "tool_use_id") and msg.tool_us
e_id:
```

```
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": msg.tool_use_id,
                    "content": msg.content,
                }
            )
    while iteration < max_iterations and self._running:
        iteration += 1
        system_prompt = build_system_prompt(
            self.app_state,
            self.registry.get_all_for_prompt(),
        )
        tools = self.registry.get_all_for_prompt()
        self.console.print()
        spinner = self.console.status("[bold blue]Thinking...[/bold blue]", spinner="dots")
        spinner.start()
        try:
            if not self.api:
                spinner.stop()
                return
            if iteration == 1 and self.app_state.verbose:
                self.console.print(f"[dim]Messages: {messages}[/dim]")
            response, usage, cost = await self.api.send_message(
                system_prompt=system_prompt,
                messages=messages,
                tools=tools,
                model=self.app_state.session_config.model,
            )
            spinner.stop()
            self.app_state.accumulate_tokens(usage)
            self.app_state.accumulate_cost(cost)
            if response.content:
                self.console.print(Markdown(response.content))
            assistant_msg: dict[str, Any] = {"role": "assistant"}
            if response.content:
                assistant_msg["content"] = response.content
            if response.tool_uses:
                assistant_msg["tool_calls"] = [
                    {
                        "id": tu.id,
                        "type": "function",
                        "function": {
                            "name": tu.name,
                            "arguments": json.dumps(tu.input),
                        },
                    }
                    for tu in response.tool_uses
                ]
            messages.append(assistant_msg)
            if not response.tool_uses:
                break
            for tool_use in response.tool_uses:
                if not self._running:
                    break
                tool_key = f"{tool_use.name}:{json.dumps(tool_use.input, sort_keys=True)}"
                failed_tool_calls[tool_key] = failed_tool_calls.get(tool_key, 0) + 1
                if failed_tool_calls[tool_key] > 3:
                    self.console.print(
                        f"\n[red]Tool '{tool_use.name}' called too many times with same par
ameters. "
                        f"Please review and fix your approach.[/red]"
                    )
                    messages.append(
                        {
```

```
                            "role": "tool",
                            "tool_call_id": tool_use.id,
                            "content": f"ERROR: Tool '{tool_use.name}' has been called {fai
led_tool_calls[tool_key]} times with the same parameters. "
                            f"This suggests an error in your approach. Please try a differe
nt strategy or provide different parameters. "
                            f"Your current parameters: {json.dumps(tool_use.input)}",
                        }
                    )
                    continue
                result = await self._execute_tool(tool_use)
                if result.is_error:
                    tool_key = f"{tool_use.name}:{json.dumps(tool_use.input, sort_keys=True
)}"
                result_content = (
                    result.content if isinstance(result.content, str) else str(result.conte
nt)
                )
                messages.append(
                    {
                        "role": "tool",
                        "tool_call_id": tool_use.id,
                        "content": result_content,
                    }
                )
        except asyncio.CancelledError:
            spinner.stop()
            self.console.print("\n[yellow]Interrupted.[/yellow]")
            break
        except httpx.HTTPStatusError as e:
            spinner.stop()
            self.console.print(f"\n[bold red]API Error:[/bold red] {e.response.status_code}
")
            try:
                error_data = e.response.json()
                error_msg = error_data.get("error", {}).get("message", str(e))
                self.console.print(f"[red]{error_msg}[/red]")
            except Exception:
                self.console.print(f"[red]{e.response.text[:500]}[/red]")
            break
        except Exception as e:
            spinner.stop()
            self.console.print(f"\n[bold red]Error:[/bold red] {e}")
            break
async def _execute_tool(self, tool_use) -> Any:
    """Execute a single tool call with permission checking."""
    tool = self.registry.get(tool_use.name)
    if not tool:
        self.console.print(f"  [dim]Unknown tool: {tool_use.name}[/dim]")
        from openlaoke.types.core_types import ToolResultBlock
        return ToolResultBlock(
            tool_use_id=tool_use.id,
            content=f"Unknown tool: {tool_use.name}",
            is_error=True,
        )
    perm_result = tool.check_permissions(
        tool_use.input,
        self.app_state.permission_config,
    )
    if perm_result == PermissionResult.DENY:
        self.console.print(
            f"  [red]Denied:[/red] {tool_use.name} - {tool.get_deny_message(tool_use.input)
}"
        )
        from openlaoke.types.core_types import ToolResultBlock
```

```
        return ToolResultBlock(
            tool_use_id=tool_use.id,
            content=f"Permission denied for {tool_use.name}",
            is_error=True,
        )
    if perm_result == PermissionResult.ASK and not self.app_state.auto_accept:
        self.console.print(
            f"  [yellow]Allow {tool_use.name}?[/yellow] "
            f"[green](y)es[/green] / [red](n)o[/red] / [blue](a)lways[/blue]"
        )
        try:
            answer = await asyncio.get_event_loop().run_in_executor(
                None, lambda: input("  > ").strip().lower()
            )
        except (EOFError, OSError):
            answer = "n"
        if answer in ("n", "no"):
            self.console.print(f"  [red]Denied: {tool_use.name}[/red]")
            from openlaoke.types.core_types import ToolResultBlock
            return ToolResultBlock(
                tool_use_id=tool_use.id,
                content=f"User denied {tool_use.name}",
                is_error=True,
            )
        elif answer in ("a", "always"):
            self.app_state.permission_config.approve_tool(tool_use.name, remember=True)
    self.console.print(f"  [dim]{tool_use.name}[/dim]")
    ctx = ToolContext(
        app_state=self.app_state,
        tool_use_id=tool_use.id,
    )
    validation = tool.validate_input(tool_use.input)
    if not validation.result:
        self.console.print(f"  [red]Validation failed: {validation.message}[/red]")
        from openlaoke.types.core_types import ToolResultBlock
        return ToolResultBlock(
            tool_use_id=tool_use.id,
            content=f"Validation error: {validation.message}",
            is_error=True,
        )
    result = await tool.call(ctx, **tool_use.input)
    rendered = tool.render_result(result)
    if rendered and len(rendered) > 200:
        self.console.print(f"  [dim]{rendered[:200]}...[/dim]")
    elif rendered:
        self.console.print(f"  [dim]{rendered}[/dim]")
    return result
def _print_banner(self) -> None:
    from openlaoke import __version__
    self.console.print(
        Panel.fit(
            f"[bold cyan]OpenLaoKe[/bold cyan] v{__version__}\n"
            "[dim]Open-source AI coding assistant[/dim]",
            border_style="cyan",
        )
    )
def _print_welcome(self) -> None:
    provider_name = "unknown"
    if self.multi_provider_config:
        provider_name = self.multi_provider_config.active_provider
    elif self.app_state.multi_provider_config:
        provider_name = self.app_state.multi_provider_config.active_provider
    proxy_info = ""
    if self._proxy:
        proxy_info = f"\n[bold]Proxy:[/bold] {self._proxy}"
```

```
    # Get available skills
    from openlaoke.core.skill_system import list_available_skills
    skills = list_available_skills()
    self.console.print(f"\n[bold]Provider:[/bold] {provider_name}")
    self.console.print(f"[bold]Model:[/bold] {self.app_state.session_config.model}")
    self.console.print(f"[bold]Working directory:[/bold] {self.app_state.get_cwd()}")
    if self.app_state.local_mode:
        self.console.print(
            "[bold]Mode:[/bold] [yellow]Local (atomic decomposition enabled)[/yellow]"
        )
    else:
        self.console.print("[bold]Mode:[/bold] [green]Online (direct API calls)[/green]")
    self.console.print(
        f"[bold]Tools:[/bold] {len(self.registry.get_all())} available{proxy_info}"
    )
    if skills:
        self.console.print(
            f"[bold]Skills:[/bold] {len(skills)} available (use Tab to complete)"
        )
        # Show first few skills as examples
        example_skills = sorted(skills)[:5]
        skills_str = ", ".join(f"/{s}" for s in example_skills)
        if len(skills) > 5:
            skills_str += f", ... ({len(skills) - 5} more)"
        self.console.print(f"[dim]  Examples: {skills_str}[/dim]")
    self.console.print(
        "\n[dim]Type [bold]/help[/bold] for commands, [bold]Tab[/bold] for skill completion
, or just start chatting.[/dim]"
    )
```

File: [130/252] openlaoke/core/safe_boundary.py

```
"""Safe execution boundary checker.
Ensures tools never access unauthorized directories.
"""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    pass
@dataclass
class ExecutionBoundary:
    """Defines safe execution boundaries."""
    allowed_paths: set[Path]
    denied_paths: set[Path]
    @classmethod
    def from_cwd(cls, cwd: str | Path) -> ExecutionBoundary:
        """Create boundary from current working directory."""
        cwd_path = Path(cwd).resolve()
        return cls(
            allowed_paths={cwd_path},
            denied_paths=set(),
        )
    def is_path_allowed(self, path: str | Path) -> bool:
        """Check if a path is within allowed boundaries."""
        try:
            target = Path(path).resolve()
        except (OSError, ValueError):
            return False
        # Check if explicitly denied
        if target in self.denied_paths:
            return False
        # Check if in allowed paths
        for allowed in self.allowed_paths:
            try:
                target.relative_to(allowed)
                return True
            except ValueError:
                pass
        return False
    def check_path(self, path: str | Path, operation: str = "access") -> tuple[bool, str]:
        """Check path and return (allowed, message)."""
        if self.is_path_allowed(path):
            return True, "OK"
        target = Path(path).resolve()
        return False, f"Unauthorized {operation}: {target} is outside working directory"
class SafeToolWrapper:
    """Wraps tool execution with safety checks."""
    def __init__(self, boundary: ExecutionBoundary):
        self.boundary = boundary
    def check_bash_command(self, command: str) -> tuple[bool, str]:
        """Check if bash command is safe."""
        # Extract paths from command
        import re
        # Find potential file paths in command
        path_patterns = [
            r"/[\w/.-]+",
            r"~/[\w/.-]+",
            r"\./[\w/.-]+",
            r"\.\./[\w/.-]+",
        ]
        for pattern in path_patterns:
```

```
            matches = re.findall(pattern, command)
            for match in matches:
                allowed, msg = self.boundary.check_path(match, "bash path")
                if not allowed:
                    return False, msg
        return True, "OK"
    def check_read_path(self, path: str) -> tuple[bool, str]:
        """Check if read operation is allowed."""
        return self.boundary.check_path(path, "read")
    def check_write_path(self, path: str) -> tuple[bool, str]:
        """Check if write operation is allowed."""
        return self.boundary.check_path(path, "write")
    def should_skip_for_creation(self, user_input: str) -> bool:
        """Determine if tool calls should be skipped for creation tasks."""
        from openlaoke.core.tool_adapter import ToolCallAdapter
        adapter = ToolCallAdapter("unknown")
        return adapter.should_skip_context_gathering(user_input)
def create_safe_wrapper(cwd: str | Path) -> SafeToolWrapper:
    """Create a safe tool wrapper for the given directory."""
    boundary = ExecutionBoundary.from_cwd(cwd)
    return SafeToolWrapper(boundary)
```

File: [131/252] openlaoke/core/scheduler/__init__.py

```
"""Task scheduling and timeout control system."""
from __future__ import annotations
from openlaoke.core.scheduler.executor import ExecutionResult, TaskExecutor
from openlaoke.core.scheduler.priority import PrioritizedItem, PriorityQueue
from openlaoke.core.scheduler.scheduler import (
    DEFAULT_TIMEOUT,
    MAX_CONCURRENT,
    ScheduledTask,
    TaskResult,
    TaskScheduler,
)
from openlaoke.core.scheduler.timeout import TimeoutHandler, TimeoutInfo
__all__ = [
    "DEFAULT_TIMEOUT",
    "ExecutionResult",
    "MAX_CONCURRENT",
    "PrioritizedItem",
    "PriorityQueue",
    "ScheduledTask",
    "TaskExecutor",
    "TaskResult",
    "TaskScheduler",
    "TimeoutHandler",
    "TimeoutInfo",
]
```

File: [132/252] openlaoke/core/scheduler/executor.py

```
"""Task executor for running tasks with lifecycle management."""
from __future__ import annotations
import asyncio
import logging
import time
from collections.abc import Callable
from concurrent.futures import Future, ThreadPoolExecutor
from dataclasses import dataclass
from typing import Any
from openlaoke.types.core_types import TaskStatus
logger = logging.getLogger(__name__)
MAX_RETRIES = 3
RETRY_DELAY = 1.0
@dataclass
class ExecutionResult:
    """Result of a task execution."""
    task_id: str
    status: TaskStatus
    result: Any = None
    error: str | None = None
    started_at: float | None = None
    completed_at: float | None = None
    retries: int = 0
class TaskExecutor:
    """Executor for running individual tasks."""
    def __init__(
        self,
        max_workers: int = 3,
        max_retries: int = MAX_RETRIES,
        retry_delay: float = RETRY_DELAY,
    ):
        """Initialize the task executor.
        Args:
            max_workers: Maximum number of worker threads.
            max_retries: Maximum number of retry attempts.
            retry_delay: Delay between retries in seconds.
        """
        self.max_workers = max_workers
        self.max_retries = max_retries
        self.retry_delay = retry_delay
        self._pool = ThreadPoolExecutor(max_workers=max_workers, thread_name_prefix="task-exec-
    ")
        self._futures: dict[str, Future] = {}
        self._lock = asyncio.Lock()
    async def execute(
        self,
        task_id: str,
        func: Callable,
        *args,
        timeout: float | None = None,
        retries: int = 0,
        **kwargs,
    ) -> ExecutionResult:
        """Execute a function with timeout and retry support.
        Args:
            task_id: Task identifier.
            func: Function to execute.
            *args: Positional arguments for the function.
            timeout: Timeout in seconds.
            retries: Number of retry attempts.
            **kwargs: Keyword arguments for the function.
```

```
    Returns:
        Execution result.
    """
    started_at = time.time()
    result = ExecutionResult(
        task_id=task_id,
        status=TaskStatus.RUNNING,
        started_at=started_at,
    )
    for attempt in range(max(1, retries + 1)):
        try:
            future = self._pool.submit(func, *args, **kwargs)
            async with self._lock:
                self._futures[task_id] = future
            try:
                def get_result(f: Future = future, t: float | None = timeout) -> Any:
                    return f.result(timeout=t)
                output = await asyncio.get_event_loop().run_in_executor(
                    None,
                    get_result,
                )
                result.status = TaskStatus.COMPLETED
                result.result = output
                result.completed_at = time.time()
                return result
            except TimeoutError:
                logger.warning(
                    f"Task {task_id} timed out on attempt {attempt + 1}/{retries + 1}"
                )
                if attempt < retries:
                    await asyncio.sleep(self.retry_delay)
                    continue
                result.status = TaskStatus.FAILED
                result.error = f"Timeout after {timeout}s"
                result.completed_at = time.time()
                return result
        except Exception as e:
            logger.error(f"Task {task_id} failed on attempt {attempt + 1}/{retries + 1}: {e
}")
            if attempt < retries:
                await asyncio.sleep(self.retry_delay)
                continue
            result.status = TaskStatus.FAILED
            result.error = str(e)
            result.completed_at = time.time()
            return result
        finally:
            async with self._lock:
                self._futures.pop(task_id, None)
    result.status = TaskStatus.FAILED
    result.error = "Max retries exceeded"
    result.completed_at = time.time()
    return result
async def execute_async(
    self,
    task_id: str,
    coro: Any,
    timeout: float | None = None,
) -> ExecutionResult:
    """Execute an async coroutine with timeout.
    Args:
        task_id: Task identifier.
        coro: Coroutine to execute.
        timeout: Timeout in seconds.
    Returns:
```

```
        Execution result.
    """
    started_at = time.time()
    result = ExecutionResult(
        task_id=task_id,
        status=TaskStatus.RUNNING,
        started_at=started_at,
    )
    try:
        if timeout:
            output = await asyncio.wait_for(coro, timeout=timeout)
        else:
            output = await coro
        result.status = TaskStatus.COMPLETED
        result.result = output
        result.completed_at = time.time()
        return result
    except TimeoutError:
        logger.warning(f"Async task {task_id} timed out after {timeout}s")
        result.status = TaskStatus.FAILED
        result.error = f"Timeout after {timeout}s"
        result.completed_at = time.time()
        return result
    except asyncio.CancelledError:
        logger.info(f"Async task {task_id} was cancelled")
        result.status = TaskStatus.KILLED
        result.error = "Task cancelled"
        result.completed_at = time.time()
        raise
    except Exception as e:
        logger.error(f"Async task {task_id} failed: {e}")
        result.status = TaskStatus.FAILED
        result.error = str(e)
        result.completed_at = time.time()
        return result
def cancel(self, task_id: str) -> bool:
    """Cancel a running task.
    Args:
        task_id: Task identifier.
    Returns:
        True if task was cancelled, False if not found.
    """
    future = self._futures.get(task_id)
    if future and not future.done():
        cancelled = future.cancel()
        if cancelled:
            logger.info(f"Cancelled task {task_id}")
        return cancelled
    return False
def shutdown(self, wait: bool = True) -> None:
    """Shutdown the executor.
    Args:
        wait: Whether to wait for pending tasks.
    """
    self._pool.shutdown(wait=wait)
    logger.info("Task executor shutdown")
def get_active_tasks(self) -> list[str]:
    """Get list of active task IDs.
    Returns:
        List of task IDs currently running.
    """
    return [task_id for task_id, future in self._futures.items() if not future.done()]
def get_task_count(self) -> int:
    """Get number of active tasks.
    Returns:
```

```
    Number of tasks currently running.
"""
return len([f for f in self._futures.values() if not f.done()])
```

File: [133/252] openlaoke/core/scheduler/priority.py

```
"""Priority queue implementation for task scheduling."""
from __future__ import annotations
import heapq
import threading
from dataclasses import dataclass, field
from typing import Any
@dataclass(order=True)
class PrioritizedItem:
    """An item in the priority queue.
    Lower priority values are processed first (higher priority).
    """
    priority: int
    sequence: int = field(compare=True)
    item: Any = field(compare=False)
class PriorityQueue:
    """Thread-safe priority queue for task scheduling."""
    def __init__(self, maxsize: int = 0):
        """Initialize the priority queue.
        Args:
            maxsize: Maximum size of the queue (0 = unlimited).
        """
        self._queue: list[PrioritizedItem] = []
        self._lock = threading.Lock()
        self._not_empty = threading.Condition(self._lock)
        self._not_full = threading.Condition(self._lock)
        self._maxsize = maxsize
        self._sequence = 0
    def put(
        self, item: Any, priority: int = 0, block: bool = True, timeout: float | None = None
    ) -> bool:
        """Put an item into the queue with a priority.
        Args:
            item: Item to put in the queue.
            priority: Priority of the item (lower = higher priority).
            block: Whether to block if queue is full.
            timeout: Timeout for blocking.
        Returns:
            True if item was added, False otherwise.
        """
        with self._not_full:
            if self._maxsize > 0:
                if not block and len(self._queue) >= self._maxsize:
                    return False
                if timeout is not None:
                    if not self._not_full.wait_for(
                        lambda: len(self._queue) < self._maxsize, timeout=timeout
                    ):
                        return False
                elif block:
                    while len(self._queue) >= self._maxsize:
                        self._not_full.wait()
            with self._lock:
                prioritized = PrioritizedItem(
                    priority=priority,
                    sequence=self._sequence,
                    item=item,
                )
                self._sequence += 1
                heapq.heappush(self._queue, prioritized)
            self._not_empty.notify()
            return True
```

```
    def get(self, block: bool = True, timeout: float | None = None) -> Any | None:
        """Remove and return an item from the queue.
        Args:
            block: Whether to block if queue is empty.
            timeout: Timeout for blocking.
        Returns:
            The item, or None if timeout.
        """
        with self._not_empty:
            if not block and not self._queue:
                return None
            if timeout is not None:
                if not self._not_empty.wait_for(lambda: bool(self._queue), timeout=timeout):
                    return None
            elif block:
                while not self._queue:
                    self._not_empty.wait()
            with self._lock:
                if not self._queue:
                    return None
                prioritized = heapq.heappop(self._queue)
            self._not_full.notify()
            return prioritized.item
    def peek(self) -> Any | None:
        """Peek at the next item without removing it.
        Returns:
            The next item, or None if queue is empty.
        """
        with self._lock:
            if not self._queue:
                return None
            return self._queue[0].item
    def qsize(self) -> int:
        """Return the approximate size of the queue."""
        with self._lock:
            return len(self._queue)
    def empty(self) -> bool:
        """Return True if the queue is empty."""
        with self._lock:
            return not self._queue
    def full(self) -> bool:
        """Return True if the queue is full."""
        with self._lock:
            return self._maxsize > 0 and len(self._queue) >= self._maxsize
    def clear(self) -> None:
        """Clear all items from the queue."""
        with self._lock:
            self._queue.clear()
    def items(self) -> list[Any]:
        """Get all items in the queue (for inspection)."""
        with self._lock:
            return [p.item for p in sorted(self._queue)]
```

File: [134/252] openlaoke/core/scheduler/scheduler.py

```
"""Task scheduler for managing concurrent task execution."""
from __future__ import annotations
import asyncio
import contextlib
import logging
import time
from dataclasses import dataclass, field
from typing import Any
from uuid import uuid4
from openlaoke.core.scheduler.executor import TaskExecutor
from openlaoke.core.scheduler.priority import PriorityQueue
from openlaoke.core.scheduler.timeout import TimeoutHandler
from openlaoke.types.core_types import TaskStatus
logger = logging.getLogger(__name__)
MAX_CONCURRENT = 3
DEFAULT_TIMEOUT = 15 * 60  # 15 minutes
@dataclass
class ScheduledTask:
    """A task scheduled for execution."""
    id: str
    func: Any
    args: tuple = ()
    kwargs: dict = field(default_factory=dict)
    priority: int = 0
    timeout: float | None = None
    status: TaskStatus = TaskStatus.PENDING
    result: Any | None = None
    error: str | None = None
    started_at: float | None = None
    completed_at: float | None = None
    retries: int = 0
    def __post_init__(self) -> None:
        if not self.kwargs:
            self.kwargs = {}
@dataclass
class TaskResult:
    """Result of a scheduled task."""
    task_id: str
    status: TaskStatus
    result: Any | None = None
    error: str | None = None
    duration: float | None = None
class TaskScheduler:
    """Manages concurrent task execution with priority and timeout support."""
    def __init__(
        self,
        max_concurrent: int = MAX_CONCURRENT,
        default_timeout: float = DEFAULT_TIMEOUT,
    ):
        """Initialize the task scheduler.
        Args:
            max_concurrent: Maximum concurrent tasks.
            default_timeout: Default timeout in seconds.
        """
        self.max_concurrent = max_concurrent
        self.default_timeout = default_timeout
        self.executor = TaskExecutor(max_workers=max_concurrent)
        self.timeout_handler = TimeoutHandler(default_timeout=default_timeout)
        self.queue = PriorityQueue()
        self.tasks: dict[str, ScheduledTask] = {}
        self.results: dict[str, TaskResult] = {}
```

```
    self._lock = asyncio.Lock()
    self._running = False
    self._worker_task: asyncio.Task | None = None
async def submit(
    self,
    func: Any,
    *args,
    priority: int = 0,
    timeout: float | None = None,
    retries: int = 0,
    task_id: str | None = None,
    **kwargs,
) -> TaskResult:
    """Submit a task for execution.
    Args:
        func: Function or coroutine to execute.
        *args: Positional arguments.
        priority: Task priority (lower = higher priority).
        timeout: Timeout in seconds.
        retries: Number of retry attempts.
        task_id: Optional task identifier.
        **kwargs: Keyword arguments.
    Returns:
        Task execution result.
    """
    task_id = task_id or f"task_{uuid4().hex[:8]}"
    timeout = timeout or self.default_timeout
    scheduled = ScheduledTask(
        id=task_id,
        func=func,
        args=args,
        kwargs=kwargs,
        priority=priority,
        timeout=timeout,
        status=TaskStatus.PENDING,
        retries=retries,
    )
    async with self._lock:
        self.tasks[task_id] = scheduled
    self.queue.put(scheduled, priority=priority)
    logger.info(f"Submitted task {task_id} with priority {priority}")
    return await self._wait_for_result(task_id, timeout=timeout)
async def submit_batch(
    self,
    tasks: list[tuple[Any, tuple, dict]],
    parallel: bool = True,
    priority: int = 0,
    timeout: float | None = None,
) -> list[TaskResult]:
    """Submit multiple tasks for execution.
    Args:
        tasks: List of (func, args, kwargs) tuples.
        parallel: Whether to run tasks in parallel.
        priority: Priority for all tasks.
        timeout: Timeout for each task.
    Returns:
        List of task results.
    """
    if parallel:
        coros = [
            self.submit(func, *args, priority=priority, timeout=timeout, **kwargs)
            for func, args, kwargs in tasks
        ]
        results = await asyncio.gather(*coros, return_exceptions=True)
        parallel_results: list[TaskResult] = []
```

```
        for r in results:
            if isinstance(r, TaskResult):
                parallel_results.append(r)
            else:
                parallel_results.append(
                    TaskResult(
                        task_id="error",
                        status=TaskStatus.FAILED,
                        error=str(r),
                    )
                )
        return parallel_results
    else:
        sequential_results: list[TaskResult] = []
        for func, args, kwargs in tasks:
            result = await self.submit(
                func, *args, priority=priority, timeout=timeout, **kwargs
            )
            sequential_results.append(result)
        return sequential_results
async def cancel(self, task_id: str) -> bool:
    """Cancel a scheduled task.
    Args:
        task_id: Task identifier.
    Returns:
        True if task was cancelled.
    """
    async with self._lock:
        task = self.tasks.get(task_id)
        if not task:
            return False
        if task.status == TaskStatus.RUNNING:
            cancelled = self.executor.cancel(task_id)
            if cancelled:
                task.status = TaskStatus.KILLED
                task.error = "Cancelled by user"
                task.completed_at = time.time()
                self._save_result(task)
            return cancelled
        elif task.status == TaskStatus.PENDING:
            task.status = TaskStatus.KILLED
            task.error = "Cancelled before execution"
            task.completed_at = time.time()
            self._save_result(task)
            return True
        return False
async def get_status(self, task_id: str) -> TaskStatus:
    """Get the status of a task.
    Args:
        task_id: Task identifier.
    Returns:
        Current task status.
    """
    async with self._lock:
        task = self.tasks.get(task_id)
        if task:
            return task.status
        result = self.results.get(task_id)
        if result:
            return result.status
        return TaskStatus.PENDING
async def get_result(self, task_id: str) -> TaskResult | None:
    """Get the result of a completed task.
    Args:
        task_id: Task identifier.
```

```
    Returns:
        Task result if available.
    """
    async with self._lock:
        return self.results.get(task_id)
async def wait_all(self) -> None:
    """Wait for all pending tasks to complete."""
    while True:
        async with self._lock:
            pending = [
                t
                for t in self.tasks.values()
                if t.status in (TaskStatus.PENDING, TaskStatus.RUNNING)
            ]
            if not pending:
                return
        await asyncio.sleep(0.1)
async def shutdown(self) -> None:
    """Shutdown the scheduler."""
    self._running = False
    if self._worker_task:
        self._worker_task.cancel()
        with contextlib.suppress(asyncio.CancelledError):
            await self._worker_task
    await self.wait_all()
    self.executor.shutdown(wait=True)
    self.timeout_handler.clear_all()
    logger.info("Task scheduler shutdown complete")
async def start(self) -> None:
    """Start the scheduler worker."""
    self._running = True
    self._worker_task = asyncio.create_task(self._worker_loop())
    logger.info("Task scheduler started")
def _save_result(self, task: ScheduledTask) -> None:
    """Save task result.
    Args:
        task: Task to save.
    """
    duration = None
    if task.started_at and task.completed_at:
        duration = task.completed_at - task.started_at
    result = TaskResult(
        task_id=task.id,
        status=task.status,
        result=task.result,
        error=task.error,
        duration=duration,
    )
    self.results[task.id] = result
async def _worker_loop(self) -> None:
    """Worker loop that processes tasks from the queue."""
    while self._running:
        task = self.queue.get(block=True, timeout=0.1)
        if not task:
            continue
        async with self._lock:
            if task.status == TaskStatus.KILLED:
                continue
            task.status = TaskStatus.RUNNING
            task.started_at = time.time()
        await self._execute_task(task)
async def _execute_task(self, task: ScheduledTask) -> None:
    """Execute a single task.
    Args:
        task: Task to execute.
```

```
    """
    try:
        if asyncio.iscoroutinefunction(task.func):
            result = await self.executor.execute_async(
                task.id,
                task.func(*task.args, **task.kwargs),
                timeout=task.timeout,
            )
        else:
            result = await self.executor.execute(
                task.id,
                task.func,
                *task.args,
                timeout=task.timeout,
                retries=task.retries,
                **task.kwargs,
            )
        async with self._lock:
            task.status = result.status
            task.result = result.result
            task.error = result.error
            task.completed_at = result.completed_at
            self._save_result(task)
        logger.info(f"Task {task.id} completed with status {task.status.value}")
    except Exception as e:
        logger.error(f"Task {task.id} execution error: {e}")
        async with self._lock:
            task.status = TaskStatus.FAILED
            task.error = str(e)
            task.completed_at = time.time()
            self._save_result(task)
async def _wait_for_result(self, task_id: str, timeout: float | None = None) -> TaskResult:
    """Wait for a task result.
    Args:
        task_id: Task identifier.
        timeout: Maximum wait time.
    Returns:
        Task result.
    """
    timeout = timeout or self.default_timeout
    start = time.time()
    while True:
        async with self._lock:
            result = self.results.get(task_id)
            if result:
                return result
        elapsed = time.time() - start
        if elapsed > timeout:
            async with self._lock:
                task = self.tasks.get(task_id)
                if task and task.status in (TaskStatus.PENDING, TaskStatus.RUNNING):
                    task.status = TaskStatus.FAILED
                    task.error = "Wait timeout"
                    task.completed_at = time.time()
                    self._save_result(task)
                return self.results.get(task_id) or TaskResult(
                    task_id=task_id,
                    status=TaskStatus.FAILED,
                    error="Wait timeout",
                )
        await asyncio.sleep(0.05)
def get_stats(self) -> dict[str, Any]:
    """Get scheduler statistics.
    Returns:
        Statistics dictionary.
```

```
    """
    pending = len([t for t in self.tasks.values() if t.status == TaskStatus.PENDING])
    running = len([t for t in self.tasks.values() if t.status == TaskStatus.RUNNING])
    completed = len([t for t in self.tasks.values() if t.status == TaskStatus.COMPLETED])
    failed = len([t for t in self.tasks.values() if t.status == TaskStatus.FAILED])
    killed = len([t for t in self.tasks.values() if t.status == TaskStatus.KILLED])
    return {
        "queue_size": self.queue.qsize(),
        "pending": pending,
        "running": running,
        "completed": completed,
        "failed": failed,
        "killed": killed,
        "total": len(self.tasks),
        "results": len(self.results),
    }
```

File: [135/252] openlaoke/core/scheduler/timeout.py

```
"""Timeout control system for task execution."""
from __future__ import annotations
import asyncio
import logging
import time
from collections.abc import Callable
from dataclasses import dataclass
from typing import Any
logger = logging.getLogger(__name__)
DEFAULT_TIMEOUT = 15 * 60  # 15 minutes
@dataclass
class TimeoutInfo:
    """Information about a timeout."""
    task_id: str
    timeout: float
    started_at: float
    deadline: float
    handle: asyncio.TimerHandle | None = None
class TimeoutHandler:
    """Handler for managing task timeouts."""
    def __init__(self, default_timeout: float = DEFAULT_TIMEOUT):
        """Initialize the timeout handler.
        Args:
            default_timeout: Default timeout in seconds.
        """
        self.default_timeout = default_timeout
        self._timeouts: dict[str, TimeoutInfo] = {}
        self._lock = asyncio.Lock()
    async def with_timeout(
        self,
        coro: Any,
        timeout: float | None = None,
        task_id: str | None = None,
    ) -> Any:
        """Execute a coroutine with timeout.
        Args:
            coro: Coroutine to execute.
            timeout: Timeout in seconds (uses default if None).
            task_id: Optional task identifier for tracking.
        Returns:
            Result of the coroutine.
        Raises:
            asyncio.TimeoutError: If timeout occurs.
        """
        timeout = timeout or self.default_timeout
        if task_id:
            await self._register_timeout(task_id, timeout)
        try:
            result = await asyncio.wait_for(coro, timeout=timeout)
            return result
        except TimeoutError:
            logger.warning(f"Task {task_id or 'unknown'} timed out after {timeout}s")
            raise
        finally:
            if task_id:
                await self._cancel_timeout(task_id)
    def set_timeout(
        self,
        task_id: str,
        timeout: float,
        callback: Callable[[], None],
```

```
) -> asyncio.TimerHandle:
    """Set a timeout callback for a task.
    Args:
        task_id: Task identifier.
        timeout: Timeout in seconds.
        callback: Callback to invoke on timeout.
    Returns:
        Timer handle that can be used to cancel.
    """
    loop = asyncio.get_event_loop()
    handle = loop.call_later(timeout, callback)
    info = TimeoutInfo(
        task_id=task_id,
        timeout=timeout,
        started_at=time.time(),
        deadline=time.time() + timeout,
        handle=handle,
    )
    self._timeouts[task_id] = info
    logger.debug(f"Set timeout for task {task_id}: {timeout}s")
    return handle
def cancel_timeout(self, task_id: str) -> bool:
    """Cancel a timeout for a task.
    Args:
        task_id: Task identifier.
    Returns:
        True if timeout was cancelled, False if not found.
    """
    info = self._timeouts.pop(task_id, None)
    if info and info.handle:
        info.handle.cancel()
        logger.debug(f"Cancelled timeout for task {task_id}")
        return True
    return False
async def extend_timeout(
    self,
    task_id: str,
    additional: float,
    callback: Callable[[], None] | None = None,
) -> bool:
    """Extend the timeout for a task.
    Args:
        task_id: Task identifier.
        additional: Additional time in seconds.
        callback: Optional new callback for the extended timeout.
    Returns:
        True if timeout was extended, False if not found.
    """
    async with self._lock:
        info = self._timeouts.get(task_id)
        if not info:
            return False
        if info.handle:
            info.handle.cancel()
        new_timeout = info.timeout + additional
        loop = asyncio.get_event_loop()
        handle = loop.call_later(new_timeout, callback) if callback else None
        info.timeout = new_timeout
        info.deadline = time.time() + new_timeout
        info.handle = handle
        logger.debug(
            f"Extended timeout for task {task_id} by {additional}s (total: {new_timeout}s)"
        )
        return True
def get_remaining_time(self, task_id: str) -> float | None:
```

```
    """Get remaining time before timeout.
    Args:
        task_id: Task identifier.
    Returns:
        Remaining seconds, or None if not found.
    """
    info = self._timeouts.get(task_id)
    if not info:
        return None
    remaining = info.deadline - time.time()
    return max(0.0, remaining)
def get_timeout_info(self, task_id: str) -> TimeoutInfo | None:
    """Get timeout info for a task.
    Args:
        task_id: Task identifier.
    Returns:
        Timeout info, or None if not found.
    """
    return self._timeouts.get(task_id)
def list_active_timeouts(self) -> list[TimeoutInfo]:
    """List all active timeouts.
    Returns:
        List of timeout information.
    """
    return list(self._timeouts.values())
def clear_all(self) -> None:
    """Clear all timeouts."""
    for _task_id, info in list(self._timeouts.items()):
        if info.handle:
            info.handle.cancel()
    self._timeouts.clear()
    logger.debug("Cleared all timeouts")
async def _register_timeout(self, task_id: str, timeout: float) -> None:
    """Register a timeout for tracking.
    Args:
        task_id: Task identifier.
        timeout: Timeout in seconds.
    """
    async with self._lock:
        info = TimeoutInfo(
            task_id=task_id,
            timeout=timeout,
            started_at=time.time(),
            deadline=time.time() + timeout,
        )
        self._timeouts[task_id] = info
async def _cancel_timeout(self, task_id: str) -> None:
    """Cancel a timeout (async version).
    Args:
        task_id: Task identifier.
    """
    async with self._lock:
        self.cancel_timeout(task_id)
```

File: [136/252] openlaoke/core/sessions.py

```
"""Session management and persistence."""
from __future__ import annotations
import json
import os
import time
from dataclasses import dataclass
from openlaoke.core.state import AppState, create_app_state
from openlaoke.types.core_types import MessageRole, TaskStatus, TaskType, UserMessage
@dataclass
class SessionInfo:
    """Summary information about a saved session."""
    session_id: str
    path: str
    created_at: float
    message_count: int
    task_count: int
    model: str
    cwd: str
class SessionManager:
    """Manages session persistence and recovery."""
    def __init__(self, session_dir: str | None = None) -> None:
        self.session_dir = session_dir or os.path.expanduser("~/.openlaoke/sessions")
        os.makedirs(self.session_dir, exist_ok=True)
    def save_session(self, app_state: AppState) -> str:
        path = os.path.join(self.session_dir, f"{app_state.session_id}.json")
        app_state.set_persist_path(path)
        app_state._persist()
        return path
    def load_session(self, session_id: str) -> AppState | None:
        path = os.path.join(self.session_dir, f"{session_id}.json")
        if not os.path.exists(path):
            return None
        return self._load_from_path(path)
    def load_latest(self) -> AppState | None:
        sessions = self.list_sessions()
        if not sessions:
            return None
        latest = max(sessions, key=lambda s: s.created_at)
        return self._load_from_path(latest.path)
    def list_sessions(self) -> list[SessionInfo]:
        sessions = []
        for filename in os.listdir(self.session_dir):
            if not filename.endswith(".json"):
                continue
            path = os.path.join(self.session_dir, filename)
            try:
                with open(path) as f:
                    data = json.load(f)
                sessions.append(
                    SessionInfo(
                        session_id=data.get("session_id", filename.replace(".json", "")),
                        path=path,
                        created_at=os.path.getmtime(path),
                        message_count=len(data.get("messages", [])),
                        task_count=len(data.get("tasks", {})),
                        model=data.get("model", "unknown"),
                        cwd=data.get("cwd", ""),
                    )
                )
            except Exception:
                continue
```

```
    return sorted(sessions, key=lambda s: s.created_at, reverse=True)
def delete_session(self, session_id: str) -> bool:
    path = os.path.join(self.session_dir, f"{session_id}.json")
    if os.path.exists(path):
        os.remove(path)
        return True
    return False
def cleanup_old_sessions(self, max_age_days: int = 30) -> int:
    cutoff = time.time() - (max_age_days * 86400)
    count = 0
    for session in self.list_sessions():
        if session.created_at < cutoff:
            self.delete_session(session.session_id)
            count += 1
    return count
def _load_from_path(self, path: str) -> AppState | None:
    try:
        with open(path) as f:
            data = json.load(f)
        app_state = create_app_state(
            cwd=data.get("cwd", os.getcwd()),
            persist_path=path,
        )
        app_state.session_id = data.get("session_id", "")
        for msg_data in data.get("messages", []):
            role = msg_data.get("role", "user")
            content = msg_data.get("content", "")
            if role == "user":
                app_state.messages.append(
                    UserMessage(
                        role=MessageRole.USER,
                        content=content,
                    )
                )
        for task_id, task_data in data.get("tasks", {}).items():
            from openlaoke.types.core_types import TaskState
            app_state.tasks[task_id] = TaskState(
                id=task_id,
                type=TaskType(task_data.get("type", "local_bash")),
                status=TaskStatus(task_data.get("status", "pending")),
                description=task_data.get("description", ""),
                start_time=task_data.get("start_time", 0),
                end_time=task_data.get("end_time"),
                output_file=task_data.get("output_file", ""),
            )
        return app_state
    except Exception:
        return None
```

File: [137/252] openlaoke/core/skill_installer.py

```
"""Skill installer - download and install skills from GitHub repositories.
Supports:
1. GitHub repositories with skills.json index
2. Direct URLs to skill directories
3. OpenCode discovery protocol (index.json)
"""
from __future__ import annotations
import shutil
from dataclasses import dataclass, field
from pathlib import Path
import httpx
@dataclass
class SkillInfo:
    """Metadata about a skill."""
    name: str
    description: str = ""
    version: str = "1.0.0"
    source: str = ""
    source_url: str = ""
    files: list[str] = field(default_factory=list)
    installed_path: Path | None = None
@dataclass
class InstallResult:
    """Result of a skill installation."""
    skill_name: str
    success: bool
    message: str = ""
    path: Path | None = None
class SkillInstaller:
    """Install skills from GitHub repositories and other sources."""
    def __init__(self, install_dir: Path | None = None):
        self.install_dir = install_dir or Path.home() / ".openlaoke" / "skills"
        self.install_dir.mkdir(parents=True, exist_ok=True)
        self._cache_dir = Path.home() / ".openlaoke" / "cache" / "skills"
        self._cache_dir.mkdir(parents=True, exist_ok=True)
        self._client = httpx.AsyncClient(timeout=30.0, follow_redirects=True)
    async def close(self):
        await self._client.aclose()
    async def install_from_github(
        self, repo_url: str, skill_name: str | None = None
    ) -> list[InstallResult]:
        """Install skills from a GitHub repository.
        Supports two formats:
        1. Repos with skills.json index (like cycleuser/Skills)
        2. Direct skill directories with SKILL.md
        """
        # Normalize URL
        repo_url = repo_url.rstrip("/")
        if "github.com" in repo_url:
            # Convert web URL to raw content URL
            parts = repo_url.split("github.com/")
            if len(parts) == 2:
                repo_path = parts[1]
                raw_base = f"https://raw.githubusercontent.com/{repo_path}/main"
                api_base = f"https://api.github.com/repos/{repo_path}/contents"
            else:
                return [
                    InstallResult(skill_name or "unknown", False, f"Invalid GitHub URL: {repo_u
    rl}")
                ]
        else:
```

```
        raw_base = repo_url
        api_base = repo_url
    results = []
    # Try skills.json first
    try:
        skills_index = await self._fetch_json(f"{raw_base}/skills.json")
        if skills_index and "skills" in skills_index:
            results = await self._install_from_index(
                raw_base, api_base, skills_index, skill_name
            )
    except Exception:
        pass
    # Also scan the skills/ directory directly (some skills may not be in skills.json)
    try:
        dir_results = await self._scan_skills_directory(raw_base, api_base, skill_name)
        # Merge results, avoiding duplicates
        existing_names = {r.skill_name for r in results}
        for r in dir_results:
            if r.skill_name not in existing_names:
                results.append(r)
                existing_names.add(r.skill_name)
    except Exception:
        pass
    # If user requested a specific skill but we didn't find it via index or directory scan,
    # try installing it directly (handles API rate limits or skills not yet indexed)
    if skill_name and not results:
        result = await self._install_single_skill(raw_base, api_base, skill_name)
        results.append(result)
    # If we got results, return them
    if results:
        return results
    # Try OpenCode discovery protocol (index.json)
    try:
        return await self._install_from_opencode_index(repo_url, skill_name)
    except Exception:
        pass
    # List all directories and try each as a skill
    try:
        contents = await self._fetch_json(api_base)
        if isinstance(contents, list):
            dirs = [item["name"] for item in contents if item.get("type") == "dir"]
            for d in dirs:
                if not d.startswith("."):
                    result = await self._install_single_skill(raw_base, api_base, d)
                    results.append(result)
    except Exception as e:
        results.append(InstallResult("unknown", False, f"Failed to list repo: {e}"))
    return results
async def _install_from_index(
    self, raw_base: str, api_base: str, index: dict, skill_name: str | None
) -> list[InstallResult]:
    """Install skills from a skills.json index."""
    results = []
    skills = index.get("skills", [])
    for skill in skills:
        name = skill.get("name", "")
        if skill_name and name != skill_name:
            continue
        result = await self._install_single_skill(raw_base, api_base, name)
        results.append(result)
    return results
async def _scan_skills_directory(
    self, raw_base: str, api_base: str, skill_name: str | None
) -> list[InstallResult]:
    """Scan the skills/ directory in a GitHub repo and install all skills found."""
```

```
    results = []
    # Try skills/ subdirectory
    skills_dir_url = f"{api_base}/skills"
    contents = await self._fetch_json(skills_dir_url)
    if not isinstance(contents, list):
        # API rate limited or directory doesn't exist
        # If specific skill requested, the caller will handle it via _install_single_skill
        return results
    skill_dirs = [item["name"] for item in contents if item.get("type") == "dir"]
    for name in skill_dirs:
        if skill_name and name != skill_name:
            continue
        result = await self._install_single_skill(raw_base, api_base, name)
        results.append(result)
    return results
async def _install_from_opencode_index(
    self, url: str, skill_name: str | None
) -> list[InstallResult]:
    """Install skills using OpenCode discovery protocol (index.json)."""
    base = url.rstrip("/") + "/"
    index_url = f"{base}index.json"
    index_data = await self._fetch_json(index_url)
    if not index_data or "skills" not in index_data:
        return []
    results = []
    host = base.rstrip("/")
    for skill in index_data["skills"]:
        name = skill.get("name", "")
        files = skill.get("files", [])
        if not files or "SKILL.md" not in files:
            continue
        if skill_name and name != skill_name:
            continue
        result = await self._download_skill_files(host, name, files)
        results.append(result)
    return results
async def _install_single_skill(self, raw_base: str, api_base: str, name: str) -> InstallRe
sult:
    """Install a single skill from a GitHub repo."""
    # Check if already installed
    skill_dir = self.install_dir / name
    if skill_dir.exists() and (skill_dir / "SKILL.md").exists():
        return InstallResult(name, True, f"Already installed at {skill_dir}", skill_dir)
    try:
        # Check if SKILL.md exists
        skill_url = f"{raw_base}/skills/{name}/SKILL.md"
        resp = await self._client.head(skill_url)
        if resp.status_code != 200:
            # Try without skills/ prefix
            skill_url = f"{raw_base}/{name}/SKILL.md"
            resp = await self._client.head(skill_url)
            if resp.status_code != 200:
                return InstallResult(name, False, "SKILL.md not found")
        # Get all files in the skill directory
        dir_url = f"{api_base}/skills/{name}"
        contents = await self._fetch_json(dir_url)
        if not isinstance(contents, list):
            # Fallback: try without skills/ prefix
            dir_url = f"{api_base}/{name}"
            contents = await self._fetch_json(dir_url)
        files = []
        if isinstance(contents, list):
            files = [item["name"] for item in contents if item.get("type") == "file"]
        # If API listing fails or returns no files, use known defaults
        if not files:
```

```
            files = ["SKILL.md"]
        if "SKILL.md" not in files:
            return InstallResult(name, False, "SKILL.md not in skill directory")
        # Create skill directory
        skill_dir = self.install_dir / name
        skill_dir.mkdir(parents=True, exist_ok=True)
        # Download each file
        for fname in files:
            file_url = f"{raw_base}/skills/{name}/{fname}"
            resp = await self._client.get(file_url)
            if resp.status_code == 200:
                (skill_dir / fname).write_text(resp.text, encoding="utf-8")
            else:
                # Try without skills/ prefix
                file_url = f"{raw_base}/{name}/{fname}"
                resp = await self._client.get(file_url)
                if resp.status_code == 200:
                    (skill_dir / fname).write_text(resp.text, encoding="utf-8")
        return InstallResult(name, True, f"Installed to {skill_dir}", skill_dir)
    except Exception as e:
        return InstallResult(name, False, str(e))
async def _download_skill_files(self, host: str, name: str, files: list[str]) -> InstallRes
ult:
    """Download skill files from OpenCode-style index."""
    try:
        skill_dir = self.install_dir / name
        skill_dir.mkdir(parents=True, exist_ok=True)
        for fname in files:
            file_url = f"{host}/{name}/{fname}"
            resp = await self._client.get(file_url)
            if resp.status_code == 200:
                (skill_dir / fname).write_text(resp.text, encoding="utf-8")
        if (skill_dir / "SKILL.md").exists():
            return InstallResult(name, True, f"Installed to {skill_dir}", skill_dir)
        else:
            shutil.rmtree(skill_dir, ignore_errors=True)
            return InstallResult(name, False, "SKILL.md not found after download")
    except Exception as e:
        return InstallResult(name, False, str(e))
async def _fetch_json(self, url: str) -> dict | list | None:
    """Fetch and parse JSON from URL."""
    resp = await self._client.get(url)
    if resp.status_code == 200:
        return resp.json()
    return None
def list_installed(self) -> list[SkillInfo]:
    """List all installed skills."""
    skills = []
    if not self.install_dir.exists():
        return skills
    for skill_dir in sorted(self.install_dir.iterdir()):
        if not skill_dir.is_dir():
            continue
        skill_file = skill_dir / "SKILL.md"
        if not skill_file.exists():
            continue
        content = skill_file.read_text(encoding="utf-8")
        name = skill_dir.name
        description = ""
        # Parse frontmatter for description
        if content.startswith("---"):
            parts = content.split("---", 2)
            if len(parts) >= 3:
                import yaml
                try:
```

```
                    frontmatter = yaml.safe_load(parts[1])
                    if frontmatter:
                        description = frontmatter.get("description", "")
                        name = frontmatter.get("name", name)
                except Exception:
                    pass
        files = [f.name for f in skill_dir.iterdir() if f.is_file()]
        skills.append(
            SkillInfo(
                name=name,
                description=description,
                installed_path=skill_dir,
                files=files,
            )
        )
    return skills
def remove_skill(self, name: str) -> InstallResult:
    """Remove an installed skill."""
    skill_dir = self.install_dir / name
    if not skill_dir.exists():
        return InstallResult(name, False, f"Skill '{name}' not found")
    shutil.rmtree(skill_dir)
    return InstallResult(name, True, f"Removed '{name}'")
def get_skill_path(self, name: str) -> Path | None:
    """Get the path to an installed skill."""
    skill_dir = self.install_dir / name
    if skill_dir.exists() and (skill_dir / "SKILL.md").exists():
        return skill_dir
    return None
```

File: [138/252] openlaoke/core/skill_system.py

```
"""Skill system for loading and managing agent skills.
Compatible with Claude Code and OpenCode skill formats.
"""
from __future__ import annotations
import re
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
import yaml
def _sanitize_frontmatter(frontmatter: str) -> str:
    """Sanitize YAML frontmatter that contains invalid syntax.
    Mirrors OpenCode's fallbackSanitization: when a value contains colons,
    convert it to a block scalar (|-) so the YAML parser can handle it.
    """
    lines = frontmatter.splitlines()
    result = []
    for line in lines:
        stripped = line.strip()
        # Skip comments and empty lines
        if stripped.startswith("#") or stripped == "":
            result.append(line)
            continue
        # Skip continuation lines (indented)
        if line.startswith((" ", "\t")):
            result.append(line)
            continue
        # Match key: value pattern
        kv = re.match(r"^([a-zA-Z_][a-zA-Z0-9_-]*)\s*:\s*(.*)$", line)
        if not kv:
            result.append(line)
            continue
        key = kv.group(1)
        value = kv.group(2).strip()
        # Skip if value is empty, already quoted, or uses block scalar
        if not value or value in (">", "|") or value.startswith(('"', "'")):
            result.append(line)
            continue
        # If value contains a colon, convert to block scalar
        if ":" in value:
            result.append(f"{key}: |-")
            result.append(f"  {value}")
            continue
        result.append(line)
    return "\n".join(result)
@dataclass
class Skill:
    """A skill definition loaded from SKILL.md file."""
    name: str
    description: str = ""
    version: str = "1.0.0"
    allowed_tools: list[str] = field(default_factory=list)
    content: str = ""
    path: Path | None = None
    metadata: dict[str, Any] = field(default_factory=dict)
    @classmethod
    def from_file(cls, path: Path) -> Skill | None:
        """Load a skill from SKILL.md file."""
        if not path.exists():
            return None
        try:
            content = path.read_text(encoding="utf-8")
```

```
        return cls.from_content(content, path)
    except Exception as e:
        print(f"Error loading skill {path}: {e}")
        return None
@classmethod
def from_content(cls, content: str, path: Path | None = None) -> Skill:
    """Parse skill from content string."""
    metadata = {}
    body = content
    if content.startswith("---"):
        parts = content.split("---", 2)
        if len(parts) >= 3:
            frontmatter = parts[1].strip()
            body = parts[2].strip()
            # Try parsing YAML directly first
            try:
                metadata = yaml.safe_load(frontmatter) or {}
            except yaml.YAMLError:
                # Fallback: sanitize frontmatter like OpenCode does
                try:
                    sanitized = _sanitize_frontmatter(frontmatter)
                    metadata = yaml.safe_load(sanitized) or {}
                except yaml.YAMLError:
                    # Last resort: manual key-value extraction
                    metadata = cls._extract_frontmatter_manual(frontmatter)
    # Determine skill name: prefer frontmatter, fallback to directory name
    name = metadata.get("name", "")
    if not name or name == "SKILL":
        # Use parent directory name as fallback
        if path and path.parent:
            name = path.parent.name
        elif path:
            name = path.stem
        else:
            name = "unknown"
    description = metadata.get("description", "")
    if isinstance(description, str):
        description = description.strip()
    version = metadata.get("version", "1.0.0")
    allowed_tools = metadata.get("allowed-tools", metadata.get("allowed_tools", []))
    return cls(
        name=name,
        description=description,
        version=str(version),
        allowed_tools=allowed_tools if isinstance(allowed_tools, list) else [],
        content=body,
        path=path,
        metadata=metadata,
    )
@staticmethod
def _extract_frontmatter_manual(frontmatter: str) -> dict[str, Any]:
    """Extract key-value pairs from broken frontmatter as last resort."""
    result = {}
    for line in frontmatter.splitlines():
        kv = re.match(r"^([a-zA-Z_][a-zA-Z0-9_-]*)\s*:\s*(.*)$", line.strip())
        if kv:
            key = kv.group(1)
            value = kv.group(2).strip()
            if value:
                result[key] = value
    return result
def get_preamble(self) -> str | None:
    """Extract preamble bash code block if present."""
    match = re.search(r"```bash\n(.*?)\n```", self.content, re.DOTALL)
    if match:
```

```
            return match.group(1)
        return None
    def get_system_prompt(self) -> str:
        """Get the skill content as system prompt."""
        return self.content
class SkillRegistry:
    """Registry for managing loaded skills."""
    def __init__(self):
        self._skills: dict[str, Skill] = {}
        self._skill_dirs: list[Path] = []
    def add_skill_directory(self, directory: Path | str) -> int:
        """Add a directory to search for skills. Returns count of skills loaded."""
        directory = Path(directory).expanduser()
        if not directory.exists():
            return 0
        self._skill_dirs.append(directory)
        return self._load_skills_from_dir(directory)
    def _load_skills_from_dir(self, directory: Path) -> int:
        """Load all skills from a directory."""
        count = 0
        for skill_dir in directory.iterdir():
            if not skill_dir.is_dir():
                continue
            skill_file = skill_dir / "SKILL.md"
            if skill_file.exists():
                skill = Skill.from_file(skill_file)
                if skill and skill.name:
                    self._skills[skill.name] = skill
                    count += 1
        return count
    def get_skill(self, name: str) -> Skill | None:
        """Get a skill by name."""
        return self._skills.get(name)
    def list_skills(self) -> list[str]:
        """List all available skill names."""
        return list(self._skills.keys())
    def get_all_skills(self) -> list[Skill]:
        """Get all loaded skills."""
        return list(self._skills.values())
    def get_skill_prompt(self, name: str) -> str | None:
        """Get the system prompt for a skill."""
        skill = self.get_skill(name)
        if skill:
            return skill.get_system_prompt()
        return None
    def has_skill(self, name: str) -> bool:
        """Check if a skill is loaded."""
        return name in self._skills
def get_default_skill_dirs() -> list[Path]:
    """Get default skill directories to search.
    Loads in priority order (later dirs overwrite earlier ones):
    1. ~/.claude/skills (Claude Code) - lowest priority
    2. ~/.openlaoke/skills (OpenLaoKe installed)
    3. ~/.config/opencode/skills (OpenCode) - highest priority
    4. ~/.opencode/skills (alternative OpenCode path)
    """
    home = Path.home()
    dirs = []
    # Load Claude first (lowest priority)
    claude_skills = home / ".claude" / "skills"
    if claude_skills.exists():
        dirs.append(claude_skills)
    # Then OpenLaoKe installed skills
    openlaoke_skills = home / ".openlaoke" / "skills"
    if openlaoke_skills.exists():
```

```
        dirs.append(openlaoke_skills)
    # OpenCode overrides (highest priority - these are the originals)
    opencode_config_skills = home / ".config" / "opencode" / "skills"
    if opencode_config_skills.exists():
        dirs.append(opencode_config_skills)
    opencode_skills = home / ".opencode" / "skills"
    if opencode_skills.exists():
        dirs.append(opencode_skills)
    return dirs
_global_registry: SkillRegistry | None = None
def get_skill_registry() -> SkillRegistry:
    """Get the global skill registry, initializing if needed."""
    global _global_registry
    if _global_registry is None:
        _global_registry = SkillRegistry()
        for skill_dir in get_default_skill_dirs():
            _global_registry.add_skill_directory(skill_dir)
    return _global_registry
def rescan_skills() -> int:
    """Rescan all skill directories and reload skills. Returns count of skills loaded."""
    global _global_registry
    _global_registry = SkillRegistry()
    for skill_dir in get_default_skill_dirs():
        _global_registry.add_skill_directory(skill_dir)
    return len(_global_registry.list_skills())
def load_skill(name: str) -> Skill | None:
    """Load a specific skill by name."""
    return get_skill_registry().get_skill(name)
def list_available_skills() -> list[str]:
    """List all available skill names."""
    return get_skill_registry().list_skills()
def get_skill_system_prompt(skill_names: list[str]) -> str:
    """Get combined system prompt for multiple skills."""
    registry = get_skill_registry()
    prompts = []
    for name in skill_names:
        skill = registry.get_skill(name)
        if skill:
            prompts.append(f"\n## Skill: {skill.name}\n\n{skill.content}")
    return "\n".join(prompts)
```

File: [139/252] openlaoke/core/smart_prompt.py

```python
"""Smart system prompt generator optimized for small models.
Avoids unnecessary context gathering for creation tasks.
"""
from __future__ import annotations
from openlaoke.core.tool_adapter import ToolCallAdapter
class SmartPromptGenerator:
    """Generates optimized system prompts based on task type."""
    def __init__(self, model: str):
        self.adapter = ToolCallAdapter(model)
    def generate_system_prompt(
        self,
        user_request: str,
        base_prompt: str,
        tools: list[dict] | None = None,
    ) -> str:
        """Generate an optimized system prompt."""
        lines = [base_prompt]
        if self.adapter.should_skip_context_gathering(user_request):
            lines.extend(
                [
                    "",
                    "[!] OPTIMIZATION: Direct Creation Mode",
                    "The user wants to create something new. Do NOT:",
                    "  - Read existing files in the current directory",
                    "  - Explore the project structure",
                    "  - Check for similar files",
                    "",
                    "Instead, IMMEDIATELY:",
                    "  - Create the requested file/content directly",
                    "  - Use your knowledge to generate high-quality code",
                    "  - Focus on the specific requirements",
                    "",
                ]
            )
        if tools and not self.adapter.supports_tools():
            tool_instructions = self.adapter.format_tools_as_text(tools)
            lines.extend(
                [
                    "",
                    "? TOOL USAGE (Text Mode)",
                    tool_instructions,
                ]
            )
        return "\n".join(lines)
    def get_execution_hints(self, user_request: str) -> dict[str, bool]:
        """Get hints for execution optimization."""
        return {
            "skip_context_gathering": self.adapter.should_skip_context_gathering(user_request),
            "is_creation": self.adapter.is_creation_request(user_request),
            "needs_tools": not self.adapter.supports_tools(),
        }
def optimize_for_small_model(
    model: str, user_request: str, base_prompt: str, tools: list[dict] | None = None
) -> tuple[str, dict[str, bool]]:
    """Optimize execution for small models."""
    generator = SmartPromptGenerator(model)
    optimized_prompt = generator.generate_system_prompt(user_request, base_prompt, tools)
    hints = generator.get_execution_hints(user_request)
    return optimized_prompt, hints
```

File: [140/252] openlaoke/core/state.py

```
"""Centralized application state management."""
from __future__ import annotations
import json
import os
import time
from collections.abc import Callable
from contextlib import suppress
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
from openlaoke.types.core_types import (
    CostInfo,
    Message,
    TaskState,
    TaskStatus,
    TokenUsage,
)
from openlaoke.types.permissions import PermissionConfig
if TYPE_CHECKING:
    from openlaoke.types.providers import MultiProviderConfig
@dataclass
class SessionConfig:
    """Configuration for a session."""
    model: str = "gemma3:1b"
    max_tokens: int = 8192
    temperature: float = 1.0
    top_p: float = 1.0
    thinking_budget: int = 0
    allowed_tools: list[str] = field(default_factory=list)
    system_prompt_suffix: str = ""
@dataclass
class AppState:
    """Centralized immutable state for the application.
    All state updates go through set_app_state(), reads through get_app_state().
    State is persisted to disk for session recovery.
    """
    session_id: str = ""
    cwd: str = ""
    messages: list[Message] = field(default_factory=list)
    tasks: dict[str, TaskState] = field(default_factory=dict)
    permission_config: PermissionConfig = field(default_factory=PermissionConfig.defaults)
    session_config: SessionConfig = field(default_factory=SessionConfig)
    env_vars: dict[str, str] = field(default_factory=dict)
    is_running: bool = False
    is_thinking: bool = False
    current_tool_use_id: str | None = None
    token_usage: TokenUsage = field(default_factory=TokenUsage)
    cost_info: CostInfo = field(default_factory=CostInfo)
    error_message: str | None = None
    pending_user_input: str | None = None
    clipboard_content: str = ""
    working_directory: str = ""
    project_root: str | None = None
    git_branch: str | None = None
    theme: str = "dark"
    vim_mode: bool = False
    verbose: bool = False
    auto_accept: bool = False
    resume_session: bool = True
    local_mode: bool = False
    multi_provider_config: MultiProviderConfig | None = None
    app_config: Any = None
```

```
    active_skills: list[str] = field(default_factory=list)
    _listeners: list[Callable[[AppState], None]] = field(default_factory=list, repr=False)
    _persist_path: str | None = None
    def __post_init__(self) -> None:
        if not self.cwd:
            self.cwd = os.getcwd()
        if not self.working_directory:
            self.working_directory = self.cwd
    def get_cwd(self) -> str:
        """Get current working directory."""
        return self.cwd
    def set_cwd(self, path: str) -> None:
        """Set current working directory and sync working_directory."""
        self.cwd = os.path.abspath(path)
        self.working_directory = self.cwd
    def get_env_vars(self) -> dict[str, str]:
        """Get merged environment variables (system + session overrides)."""
        env = os.environ.copy()
        env.update(self.env_vars)
        return env
    def add_message(self, message: Message) -> None:
        """Add message to history with timestamp, notify listeners, and persist."""
        message.timestamp = time.time()
        self.messages.append(message)
        self._notify()
        self._persist()
    def get_messages(self) -> list[Message]:
        """Get a copy of all messages."""
        return list(self.messages)
    def get_last_message(self) -> Message | None:
        """Get most recent message or None if history is empty."""
        return self.messages[-1] if self.messages else None
    def get_message_count(self) -> int:
        """Get total number of messages in history."""
        return len(self.messages)
    def add_task(self, task: TaskState) -> None:
        """Register a new task and notify listeners."""
        self.tasks[task.id] = task
        self._notify()
    def update_task(self, task: TaskState) -> None:
        """Update task state and persist changes."""
        self.tasks[task.id] = task
        self._notify()
        self._persist()
    def get_task(self, task_id: str) -> TaskState | None:
        return self.tasks.get(task_id)
    def get_all_tasks(self) -> list[TaskState]:
        return list(self.tasks.values())
    def get_active_tasks(self) -> list[TaskState]:
        return [
            t
            for t in self.tasks.values()
            if t.status not in (TaskStatus.COMPLETED, TaskStatus.FAILED, TaskStatus.KILLED)
        ]
    def accumulate_tokens(self, usage: TokenUsage) -> None:
        self.token_usage.accumulate(usage)
    def accumulate_cost(self, cost: CostInfo) -> None:
        self.cost_info.input_cost += cost.input_cost
        self.cost_info.output_cost += cost.output_cost
        self.cost_info.cache_read_cost += cost.cache_read_cost
        self.cost_info.cache_creation_cost += cost.cache_creation_cost
    def set_error(self, message: str | None) -> None:
        self.error_message = message
        self._notify()
    def set_pending_input(self, value: str | None) -> None:
```

```
        self.pending_user_input = value
        self._notify()
    def subscribe(self, listener: Callable[[AppState], None]) -> None:
        self._listeners.append(listener)
    def unsubscribe(self, listener: Callable[[AppState], None]) -> None:
        if listener in self._listeners:
            self._listeners.remove(listener)
    def _notify(self) -> None:
        for listener in self._listeners:
            with suppress(Exception):
                listener(self)
    def set_persist_path(self, path: str) -> None:
        self._persist_path = path
    def _persist(self) -> None:
        if not self._persist_path:
            return
        try:
            os.makedirs(os.path.dirname(self._persist_path), exist_ok=True)
            data = {
                "session_id": self.session_id,
                "cwd": self.cwd,
                "messages": [m.to_dict() for m in self.messages],
                "tasks": {k: v.to_dict() for k, v in self.tasks.items()},
                "token_usage": {
                    "input_tokens": self.token_usage.input_tokens,
                    "output_tokens": self.token_usage.output_tokens,
                    "cache_read_tokens": self.token_usage.cache_read_tokens,
                    "cache_creation_tokens": self.token_usage.cache_creation_tokens,
                },
                "cost_info": {
                    "input_cost": self.cost_info.input_cost,
                    "output_cost": self.cost_info.output_cost,
                    "cache_read_cost": self.cost_info.cache_read_cost,
                    "cache_creation_cost": self.cost_info.cache_creation_cost,
                },
            }
            with open(self._persist_path, "w", encoding="utf-8") as f:
                json.dump(data, f, indent=2, default=str)
        except Exception:
            pass
    def to_dict(self) -> dict[str, Any]:
        return {
            "session_id": self.session_id,
            "cwd": self.cwd,
            "message_count": len(self.messages),
            "task_count": len(self.tasks),
            "is_running": self.is_running,
            "token_usage": {
                "input_tokens": self.token_usage.input_tokens,
                "output_tokens": self.token_usage.output_tokens,
            },
            "cost_info": {
                "total_cost": self.cost_info.total_cost,
            },
        }
def create_app_state(
    cwd: str | None = None,
    model: str = "claude-sonnet-4-20250514",
    persist_path: str | None = None,
    **kwargs: Any,
) -> AppState:
    """Factory for creating AppState with sensible defaults."""
    session_id = f"session_{int(time.time())}"
    state = AppState(
        session_id=session_id,
```

```
    cwd=cwd or os.getcwd(),
    session_config=SessionConfig(model=model),
    **kwargs,
)
if persist_path:
    state.set_persist_path(persist_path)
return state
```

File: [141/252] openlaoke/core/supervisor/__init__.py

```
"""Task supervision system - ensures tasks are completed."""
from __future__ import annotations
from openlaoke.core.supervisor.checker import TaskCompletionChecker
from openlaoke.core.supervisor.requirements import TaskRequirements
from openlaoke.core.supervisor.supervisor import (
    RetryReason,
    SupervisedTask,
    SupervisionResult,
    TaskStatus,
    TaskSupervisor,
)
__all__ = [
    "TaskSupervisor",
    "TaskCompletionChecker",
    "TaskRequirements",
    "SupervisedTask",
    "SupervisionResult",
    "TaskStatus",
    "RetryReason",
]
```

File: [142/252] openlaoke/core/supervisor/checker.py

```
"""Task completion checker - verifies requirements are met."""
from __future__ import annotations
import os
import re
from typing import Any
from openlaoke.core.supervisor.requirements import RequirementCheckResult, TaskRequirements
class TaskCompletionChecker:
    """Checks if task requirements are satisfied.
    Implements various check types:
    - file_exists: Check if output file exists
    - word_count: Check minimum word count
    - structure: Check for required sections
    - contains: Check for required patterns
    - has_numbers: Check for quantitative data
    - files_created: Check for generated files
    - anti_ai_check: Detect AI-typical patterns
    - general: Generic completion check
    """
    AI_PATTERNS = [
        r"^(?:[\d\-\*]\s+[\w\s]+,\s*){3,}$",
        r"^(?:[\d\-\*]\s+[\w\s]+\n){3,}$",
        r"Systems?\s+(?:could|would|should|might)\s+\w+",
        r"Improvements?\s+(?:include|are|would be)",
        r"The\s+main\s+(?:contributions|features|benefits)\s+(?:are|include)",
        r"^(?:In\s+conclusion|To\s+summarize|Overall),",
        r"\b(?:novel|state-of-the-art|cutting-edge|groundbreaking)\b.{0,20}\b(?:approach|method
    |system|technique)\b",
    ]
    ACADEMIC_QUALITY_MARKERS = [
        r"\[\d+\]",
        r"et\s+al\.",
        r"\d+(?:\.\d+)?%",
        r"Figure\s+\d+",
        r"Table\s+\d+",
        r"Section\s+\d+",
        r":\d+",
    ]
    def __init__(self):
        self._anti_ai_enabled = True
    def enable_anti_ai_check(self, enabled: bool = True) -> None:
        self._anti_ai_enabled = enabled
    async def check_requirement(
        self,
        requirement: TaskRequirements,
        artifacts: dict[str, Any],
    ) -> bool:
        result = await self._check(requirement, artifacts)
        return result.satisfied
    async def _check(
        self,
        requirement: TaskRequirements,
        artifacts: dict[str, Any],
    ) -> RequirementCheckResult:
        """Dispatch check to appropriate handler based on check type."""
        check_type = requirement.check_type
        check_handlers = {
            "file_exists": self._check_file_exists,
            "word_count": self._check_word_count,
            "structure": self._check_structure,
            "contains": self._check_contains,
            "has_numbers": self._check_has_numbers,
```

```
        "files_created": self._check_files_created,
        "anti_ai_check": self._check_anti_ai,
        "references_exist": self._check_references_exist,
        "citations_quality": self._check_citations_quality,
        "general": self._check_general,
    }
    handler = check_handlers.get(check_type)
    if handler:
        return handler(requirement, artifacts)
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=False,
        message=f"Unknown check type: {check_type}",
    )
def _check_file_exists(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    output_files = artifacts.get("output_files", [])
    content = artifacts.get("content", "")
    if output_files:
        for f in output_files:
            if os.path.exists(f):
                return RequirementCheckResult(
                    requirement=requirement,
                    satisfied=True,
                    actual_value=f,
                    message=f"File exists: {f}",
                )
    if content and len(content) > 100:
        return RequirementCheckResult(
            requirement=requirement,
            satisfied=True,
            message="Content generated in memory",
        )
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=False,
        message="No output file found",
    )
def _check_word_count(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    threshold = requirement.threshold or 1000
    words = len(content.split())
    satisfied = words >= threshold
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value=words,
        message=f"Word count: {words} (required: {threshold})",
    )
def _check_structure(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    patterns = requirement.patterns or []
    found = []
    missing = []
```

```
    for pattern in patterns:
        if re.search(pattern, content, re.IGNORECASE):
            found.append(pattern)
        else:
            missing.append(pattern)
    satisfied = len(found) >= len(patterns) * 0.5
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value=found,
        message=f"Found sections: {found}, missing: {missing}",
        details={"found": found, "missing": missing},
    )
def _check_contains(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    patterns = requirement.patterns or []
    found_count = 0
    for pattern in patterns:
        if re.search(pattern, content, re.IGNORECASE):
            found_count += 1
    satisfied = found_count >= len(patterns) * 0.3
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value=found_count,
        message=f"Found {found_count}/{len(patterns)} patterns",
    )
def _check_has_numbers(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    numbers = re.findall(r"\d+(?:\.\d+)?(?:%)?", content)
    percentages = re.findall(r"\d+(?:\.\d+)?%", content)
    citations = re.findall(r"\[\d+\]", content)
    has_quantitative = len(numbers) >= 10
    has_percentages = len(percentages) >= 3
    has_citations = len(citations) >= 3
    satisfied = has_quantitative and (has_percentages or has_citations)
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value={
            "numbers": len(numbers),
            "percentages": len(percentages),
            "citations": len(citations),
        },
        message=f"Numbers: {len(numbers)}, Percentages: {len(percentages)}, Citations: {len
(citations)}",
    )
def _check_files_created(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    output_files = artifacts.get("output_files", [])
    patterns = requirement.patterns or ["*.svg", "*.png"]
    found_files = []
    for pattern in patterns:
        if "*" in pattern:
```

```
            ext = pattern.replace("*", "")
            for f in output_files:
                if f.endswith(ext):
                    found_files.append(f)
        else:
            if pattern in output_files:
                found_files.append(pattern)
    satisfied = len(found_files) > 0
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value=found_files,
        message=f"Created files: {found_files}",
    )
def _check_anti_ai(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    if not self._anti_ai_enabled:
        return RequirementCheckResult(
            requirement=requirement,
            satisfied=True,
            message="Anti-AI check disabled",
        )
    content = artifacts.get("content", "")
    ai_pattern_count = 0
    for pattern in self.AI_PATTERNS:
        matches = re.findall(pattern, content, re.MULTILINE | re.IGNORECASE)
        ai_pattern_count += len(matches)
    quality_marker_count = 0
    for pattern in self.ACADEMIC_QUALITY_MARKERS:
        matches = re.findall(pattern, content)
        quality_marker_count += len(matches)
    lines = content.split("\n")
    bullet_lines = sum(1 for line in lines if re.match(r"^\s*[\d\-\*]", line))
    bullet_ratio = bullet_lines / len(lines) if lines else 0
    score = 0
    issues = []
    if ai_pattern_count > 5:
        issues.append(f"AI-typical phrases: {ai_pattern_count}")
        score -= ai_pattern_count
    else:
        score += 5
    if quality_marker_count > 10:
        score += 10
    elif quality_marker_count > 5:
        score += 5
    else:
        issues.append(f"Few quality markers: {quality_marker_count}")
    if bullet_ratio > 0.3:
        issues.append(f"Too many bullet points: {bullet_ratio:.1%}")
        score -= 5
    else:
        score += 5
    word_count = len(content.split())
    if word_count < 500:
        issues.append(f"Content too short: {word_count} words")
        score -= 10
    paragraphs = [p for p in content.split("\n\n") if len(p.strip()) > 100]
    if len(paragraphs) < 3:
        issues.append(f"Few substantive paragraphs: {len(paragraphs)}")
        score -= 5
    satisfied = score >= 0 and len(issues) <= 2
    return RequirementCheckResult(
```

```
        requirement=requirement,
        satisfied=satisfied,
        actual_value={
            "ai_patterns": ai_pattern_count,
            "quality_markers": quality_marker_count,
            "bullet_ratio": bullet_ratio,
            "score": score,
        },
        message=f"Anti-AI score: {score}. Issues: {issues}",
        details={"issues": issues},
    )
def _check_general(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    output_files = artifacts.get("output_files", [])
    has_content = len(content) > 100
    has_files = len(output_files) > 0
    issues = []
    suggestions = []
    if has_files:
        for file_path in output_files:
            if file_path.endswith((".py", ".js", ".ts")):
                try:
                    with open(file_path, encoding="utf-8") as f:
                        code_content = f.read()
                    import ast
                    try:
                        ast.parse(code_content)
                    except SyntaxError as e:
                        issues.append(f"Syntax error in {file_path}: {e}")
                        suggestions.append(f"Fix syntax error: {e}")
                    function_count = len(
                        re.findall(r"^\s*def\s+\w+", code_content, re.MULTILINE)
                    )
                    pass_count = len(re.findall(r"^\s*pass\s*$", code_content, re.MULTILINE
))
                    if (
                        pass_count > 0
                        and function_count > 0
                        and pass_count >= function_count * 0.5
                    ):
                        issues.append(
                            f"Too many 'pass' statements ({pass_count}) - implementation in
complete"
                        )
                        suggestions.append(
                            "Replace 'pass' statements with actual implementation"
                        )
                    bool(
                        re.search(
                            r"^import\s+|^from\s+\w+\s+import", code_content, re.MULTILINE
                        )
                    )
                    has_loop = bool(re.search(r"\bfor\s+\w+\s+in\s+|\bwhile\s+", code_conte
nt))
                    has_condition = bool(re.search(r"\bif\s+", code_content))
                    has_return = bool(re.search(r"\breturn\s+", code_content))
                    if (
                        function_count > 0
                        and not has_loop
                        and not has_condition
                        and not has_return
```

```
                    ):
                        issues.append(
                            "Functions lack computational logic (no loops/conditions/return
s)"
                        )
                        suggestions.append("Add actual computational logic to functions")
                        if requirement.name == "no_placeholder_pass" and pass_count > 0:
                            issues.append(f"Found {pass_count} placeholder 'pass' statement
s")
                            suggestions.append("Replace all 'pass' statements with actual c
ode")
                except Exception as e:
                    issues.append(f"Could not analyze {file_path}: {e}")
    satisfied = has_content or has_files
    if issues and requirement.critical:
        satisfied = False
    message = f"Has content: {has_content}, Has files: {has_files}"
    if issues:
        message += f". Issues: {'; '.join(issues)}"
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        message=message,
        details={"issues": issues, "suggestions": suggestions},
    )
def _check_references_exist(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    working_dir = artifacts.get("working_dir", os.getcwd())
    pdf_dir = os.path.join(working_dir, "pdf")
    citations = re.findall(r"\[(\d+)\]", content)
    unique_citations = set(citations)
    pdf_files = []
    if os.path.exists(pdf_dir):
        pdf_files = [f for f in os.listdir(pdf_dir) if f.endswith(".pdf")]
    satisfied = len(pdf_files) >= len(unique_citations) or len(pdf_files) >= 3
    return RequirementCheckResult(
        requirement=requirement,
        satisfied=satisfied,
        actual_value={
            "citations_count": len(unique_citations),
            "pdfs_downloaded": len(pdf_files),
            "pdf_dir": pdf_dir,
        },
        message=f"Citations: {len(unique_citations)}, PDFs downloaded: {len(pdf_files)} in
{pdf_dir}",
        details={"pdf_files": pdf_files},
    )
def _check_citations_quality(
    self,
    requirement: TaskRequirements,
    artifacts: dict[str, Any],
) -> RequirementCheckResult:
    content = artifacts.get("content", "")
    citations = re.findall(r"\[(\d+)\]", content)
    unique_citations = set(citations) if citations else set()
    has_arxiv_refs = bool(re.search(r"arXiv:\d{4}\.\d{4,5}", content))
    has_doi_refs = bool(re.search(r"10\.\d{4,}/[^\s\]]+", content))
    has_author_year = bool(re.search(r"[A-Z][a-z]+\s+et\s+al\.\s*\[\d+\]", content))
    has_proper_citation_format = len(unique_citations) >= 3
    quality_score = sum(
        [
```

```
        has_proper_citation_format * 2,
        has_arxiv_refs,
        has_doi_refs,
        has_author_year,
    ]
)
satisfied = quality_score >= 2
return RequirementCheckResult(
    requirement=requirement,
    satisfied=satisfied,
    actual_value={
        "unique_citations": len(unique_citations),
        "has_arxiv": has_arxiv_refs,
        "has_doi": has_doi_refs,
        "has_author_year": has_author_year,
        "quality_score": quality_score,
    },
    message=f"Citation quality score: {quality_score}/5. "
    f"Unique citations: {len(unique_citations)}, "
    f"arXiv: {has_arxiv_refs}, DOI: {has_doi_refs}",
)
```

File: [143/252] openlaoke/core/supervisor/requirements.py

```
"""Task requirements specification."""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
@dataclass
class TaskRequirements:
    name: str
    description: str
    check_type: str
    critical: bool = True
    patterns: list[str] = field(default_factory=list)
    threshold: float | int | None = None
    weight: float = 1.0
    metadata: dict[str, Any] = field(default_factory=dict)
    def __post_init__(self) -> None:
        if self.critical:
            self.weight = 2.0
@dataclass
class RequirementCheckResult:
    requirement: TaskRequirements
    satisfied: bool
    actual_value: Any = None
    message: str = ""
    details: dict[str, Any] = field(default_factory=dict)
```

File: [144/252] openlaoke/core/supervisor/supervisor.py

```
"""Task supervisor - ensures user tasks are completed."""
from __future__ import annotations
import time
from dataclasses import dataclass, field
from enum import StrEnum
from typing import TYPE_CHECKING, Any
from openlaoke.core.model_assessment import ModelAssessor, TaskDecomposer
from openlaoke.core.supervisor.checker import TaskCompletionChecker
from openlaoke.core.supervisor.requirements import TaskRequirements
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class TaskStatus(StrEnum):
    PENDING = "pending"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"
    RETRYING = "retrying"
    ESCALATED = "escalated"
class RetryReason(StrEnum):
    INCOMPLETE_OUTPUT = "incomplete_output"
    MISSING_REQUIREMENTS = "missing_requirements"
    ERROR_OCCURRED = "error_occurred"
    QUALITY_ISSUE = "quality_issue"
    AI_DETECTED = "ai_detected"
@dataclass
class SupervisedTask:
    original_request: str
    requirements: list[TaskRequirements]
    status: TaskStatus = TaskStatus.PENDING
    completion_percentage: float = 0.0
    attempts: int = 0
    max_attempts: int = 5
    errors: list[str] = field(default_factory=list)
    artifacts: dict[str, Any] = field(default_factory=dict)
    start_time: float = field(default_factory=time.time)
    last_check_time: float = 0.0
    retry_history: list[dict[str, Any]] = field(default_factory=list)
    model: str | None = None
    decomposed_steps: list[str] = field(default_factory=list)
    current_step: int = 0
    step_verification_results: list[dict[str, Any]] = field(default_factory=list)
@dataclass
class SupervisionResult:
    task_id: str
    is_complete: bool
    completion_percentage: float
    missing_requirements: list[str]
    suggested_actions: list[str]
    should_retry: bool
    retry_reason: RetryReason | None = None
    feedback: str = ""
class TaskSupervisor:
    """Supervises task execution to ensure completion.
    Core responsibilities:
    1. Parse user requests into verifiable requirements
    2. Monitor task progress throughout execution
    3. Detect incomplete or low-quality outputs
    4. Trigger retries with specific feedback
    5. Escalate when automatic recovery fails
    6. Adapt to model capabilities (tier-based task decomposition)
    """
```

```
    def __init__(self, app_state: AppState):
        self.app_state = app_state
        self.tasks: dict[str, SupervisedTask] = {}
        self.checker = TaskCompletionChecker()
        self._supervision_enabled = True
        self._anti_ai_mode = True
        self.model_assessor: ModelAssessor | None = None
        self.current_model: str | None = None
        self.task_decomposer: TaskDecomposer | None = None
    def set_model(self, model: str, config: Any = None) -> None:
        """Set the current model and initialize model-aware supervision."""
        self.current_model = model
        if config:
            self.model_assessor = ModelAssessor(config)
            granularity = self.model_assessor.get_granularity(model)
            self.task_decomposer = TaskDecomposer(granularity)
    def parse_request(self, user_request: str) -> SupervisedTask:
        requirements = self._extract_requirements(user_request)
        task = SupervisedTask(
            original_request=user_request,
            requirements=requirements,
            status=TaskStatus.PENDING,
            model=self.current_model,
        )
        if self.task_decomposer:
            task.decomposed_steps = self.task_decomposer.decompose(user_request)
            if len(task.decomposed_steps) > 1:
                task.max_attempts = self.task_decomposer.gran.retry_limit
        task_id = f"task_{int(time.time() * 1000)}"
        self.tasks[task_id] = task
        return task
    def _extract_requirements(self, request: str) -> list[TaskRequirements]:
        requirements = []
        request_lower = request.lower()
        if any(word in request_lower for word in ["?", "write", "??", "??"]) and any(
            word in request_lower for word in ["??", "article", "paper"]
        ):
            requirements.extend(
                [
                    TaskRequirements(
                        name="document_created",
                        description="Target document must be created",
                        check_type="file_exists",
                        critical=True,
                    ),
                    TaskRequirements(
                        name="sufficient_length",
                        description="Document must have substantial content (min 3000 words)",
                        check_type="word_count",
                        threshold=3000,
                        critical=True,
                    ),
                    TaskRequirements(
                        name="proper_structure",
                        description="Document must have academic structure with sections",
                        check_type="structure",
                        patterns=["Introduction", "Conclusion", "References"],
                        critical=True,
                    ),
                    TaskRequirements(
                        name="has_citations",
                        description="Document must include citations",
                        check_type="contains",
                        patterns=["[1]", "[2]", "References", "????"],
                        critical=False,
```

```
                ),
                TaskRequirements(
                    name="anti_ai_quality",
                    description="Content must not have AI-typical patterns",
                    check_type="anti_ai_check",
                    critical=True,
                ),
                TaskRequirements(
                    name="references_downloaded",
                    description="All cited references must be downloaded to pdf/ directory"
,
                    check_type="references_exist",
                    critical=True,
                ),
                TaskRequirements(
                    name="has_real_citations",
                    description="Document must cite REAL papers with downloadable sources",
                    check_type="citations_quality",
                    critical=False,
                ),
            ]
        )
    if any(word in request_lower for word in ["??", "compare", "??", "analyze"]):
        requirements.extend(
            [
                TaskRequirements(
                    name="comparison_table",
                    description="Must include comparison table or matrix",
                    check_type="contains",
                    patterns=["|", "Table", "??", "??"],
                    critical=False,
                ),
                TaskRequirements(
                    name="quantitative_data",
                    description="Must include specific numbers and measurements",
                    check_type="has_numbers",
                    critical=True,
                ),
            ]
        )
    if any(word in request_lower for word in ["??", "figure", "diagram", "svg"]):
        requirements.extend(
            [
                TaskRequirements(
                    name="figures_created",
                    description="Must create visual figures/diagrams",
                    check_type="files_created",
                    patterns=["*.svg", "*.png", "*.pdf"],
                    critical=True,
                ),
            ]
        )
    if any(word in request_lower for word in ["??", "code", "??"]):
        requirements.extend(
            [
                TaskRequirements(
                    name="code_included",
                    description="Must include code examples",
                    check_type="contains",
                    patterns=["```", "def ", "class ", "function "],
                    critical=True,
                ),
            ]
        )
    if any(
```

```
        pattern in request_lower
        for pattern in ["???", "????", "????", "write a", "create a", "build a"]
    ) and any(
        pattern in request_lower
        for pattern in ["??", "??", "program", "script", "application", "app"]
    ):
        requirements.extend(
            [
                TaskRequirements(
                    name="program_file_created",
                    description="Must create an executable program file (.py, .js, etc.)",
                    check_type="files_created",
                    patterns=["*.py", "*.js", "*.ts", "*.go", "*.rs"],
                    critical=True,
                ),
                TaskRequirements(
                    name="program_is_functional",
                    description="Program must have actual implementation, not just placehol
ders or print statements",
                    check_type="contains",
                    patterns=["def ", "class ", "function ", "import "],
                    critical=True,
                ),
                TaskRequirements(
                    name="program_has_logic",
                    description="Program must contain computational logic (loops, calculati
ons, operations)",
                    check_type="contains",
                    patterns=["for ", "while ", "if ", "return ", "calc", "compute", "measu
re"],
                    critical=True,
                ),
                TaskRequirements(
                    name="program_runnable",
                    description="Program should be syntactically correct and runnable",
                    check_type="general",
                    critical=False,
                ),
            ]
        )
        if any(
            word in request_lower
            for word in [
                "??",
                "??",
                "benchmark",
                "performance",
                "cpu",
                "measure",
                "calculate",
            ]
        ):
            requirements.extend(
                [
                    TaskRequirements(
                        name="has_calculation",
                        description="Must perform actual calculation, not just retrieve inf
ormation",
                        check_type="contains",
                        patterns=[
                            "for ",
                            "range",
                            "while ",
                            "time.time",
                            "time.perf",
```

```
                            "calculate",
                            "compute",
                            "measure",
                            "benchmark",
                        ],
                        critical=True,
                    ),
                    TaskRequirements(
                        name="has_performance_metrics",
                        description="Must output quantitative performance metrics (ops/sec,
 GFLOPS, time, etc.)",
                        check_type="has_numbers",
                        critical=True,
                    ),
                    TaskRequirements(
                        name="no_placeholder_pass",
                        description="Code must not contain placeholder 'pass' statements in
 main logic",
                        check_type="general",
                        critical=True,
                    ),
                ]
            )
    if not requirements:
        requirements.append(
            TaskRequirements(
                name="task_completed",
                description="Task should show evidence of completion",
                check_type="general",
                critical=True,
            )
        )
    return requirements
async def check_completion(
    self,
    task_id: str,
    artifacts: dict[str, Any],
) -> SupervisionResult:
    if task_id not in self.tasks:
        return SupervisionResult(
            task_id=task_id,
            is_complete=False,
            completion_percentage=0.0,
            missing_requirements=["Task not found"],
            suggested_actions=["Restart task"],
            should_retry=False,
        )
    task = self.tasks[task_id]
    task.artifacts = artifacts
    task.last_check_time = time.time()
    if self.task_decomposer and task.decomposed_steps:
        should_verify = self.task_decomposer.should_verify(task.current_step)
        if should_verify:
            task.step_verification_results.append(
                {
                    "step": task.current_step,
                    "timestamp": time.time(),
                    "artifacts": list(artifacts.keys()),
                }
            )
    missing = []
    completed_count = 0
    total_weight = 0.0
    for req in task.requirements:
        is_satisfied = await self.checker.check_requirement(req, artifacts)
```

```
        if is_satisfied:
            completed_count += 1
            total_weight += req.weight
        else:
            missing.append(req.description)
    critical_missing = [
        req.description
        for req in task.requirements
        if req.critical and not await self.checker.check_requirement(req, artifacts)
    ]
    completion = (completed_count / len(task.requirements)) * 100 if task.requirements else
 0.0
    is_complete = len(critical_missing) == 0 and completion >= 80.0
    should_retry = (
        not is_complete and task.attempts < task.max_attempts and len(critical_missing) > 0
    )
    retry_reason = None
    if should_retry:
        if any("AI" in m or "anti-ai" in m.lower() for m in missing):
            retry_reason = RetryReason.AI_DETECTED
        elif any("length" in m.lower() or "word" in m.lower() for m in missing):
            retry_reason = RetryReason.INCOMPLETE_OUTPUT
        elif critical_missing:
            retry_reason = RetryReason.MISSING_REQUIREMENTS
        else:
            retry_reason = RetryReason.QUALITY_ISSUE
    suggested_actions = self._generate_suggestions(missing, retry_reason)
    feedback = self._generate_feedback(task, missing, completion)
    result = SupervisionResult(
        task_id=task_id,
        is_complete=is_complete,
        completion_percentage=completion,
        missing_requirements=missing,
        suggested_actions=suggested_actions,
        should_retry=should_retry,
        retry_reason=retry_reason,
        feedback=feedback,
    )
    if is_complete:
        task.status = TaskStatus.COMPLETED
    elif should_retry:
        task.status = TaskStatus.RETRYING
        task.attempts += 1
        task.retry_history.append(
            {
                "attempt": task.attempts,
                "reason": retry_reason.value if retry_reason else None,
                "missing": missing,
                "timestamp": time.time(),
            }
        )
    return result
def _generate_suggestions(
    self,
    missing: list[str],
    retry_reason: RetryReason | None,
) -> list[str]:
    suggestions = []
    if retry_reason == RetryReason.AI_DETECTED:
        suggestions.extend(
            [
                "Add specific numbers and measurements to support claims",
                "Include real citations from actual papers (search WebSearch)",
                "Reference specific code files with line numbers",
                "Remove generic bullet-point lists, write full paragraphs",
```

```
                "Add technical depth: explain HOW and WHY, not just WHAT",
            ]
        )
    if retry_reason == RetryReason.INCOMPLETE_OUTPUT:
        suggestions.extend(
            [
                "Expand content to meet minimum word count",
                "Add more detailed explanations for each section",
                "Include additional examples and case studies",
            ]
        )
    if retry_reason == RetryReason.MISSING_REQUIREMENTS:
        for item in missing[:3]:
            suggestions.append(f"Address requirement: {item}")
    if retry_reason == RetryReason.QUALITY_ISSUE:
        suggestions.extend(
            [
                "Review content for technical accuracy",
                "Add comparative analysis with specific data",
                "Include visualizations or tables",
            ]
        )
    return suggestions
def _generate_feedback(
    self,
    task: SupervisedTask,
    missing: list[str],
    completion: float,
) -> str:
    if completion >= 80.0:
        return ""
    lines = [
        f"?? Task completion: {completion:.1f}%",
        "",
        "Missing requirements:",
    ]
    for i, item in enumerate(missing[:5], 1):
        lines.append(f"  {i}. {item}")
    if task.attempts > 0:
        lines.append(f"\nAttempt {task.attempts}/{task.max_attempts}")
    if self._anti_ai_mode and any("AI" in m for m in missing):
        lines.extend(
            [
                "",
                "? Anti-AI Quality Issues Detected:",
                "  - Content appears to use AI-typical patterns",
                "  - Add SPECIFIC: numbers, citations, code references",
                "  - Remove GENERIC: lists without substance, vague claims",
                "  - Reference REAL: actual papers, actual code lines",
            ]
        )
    code_issues = [
        m
        for m in missing
        if any(
            keyword in m.lower()
            for keyword in [
                "pass",
                "placeholder",
                "syntax",
                "logic",
                "implementation",
                "function",
            ]
        )
```

```
    ]
    if code_issues:
        lines.extend(
            [
                "",
                "? Code Quality Issues:",
                "  - Code must have ACTUAL implementation, not placeholders",
                "  - Replace all 'pass' statements with working code",
                "  - Add computational logic (loops, calculations, operations)",
                "  - Ensure code is runnable and produces expected output",
                "",
                "? Example of good code:",
                "    # Bad: def calculate(): pass",
                "    # Good: def calculate(data):",
                "    #          result = sum(x * 2 for x in data)",
                "    #          return result",
            ]
        )
    calculation_issues = [
        m
        for m in missing
        if any(
            keyword in m.lower()
            for keyword in ["calculation", "measure", "benchmark", "performance"]
        )
    ]
    if calculation_issues:
        lines.extend(
            [
                "",
                "? Calculation/Benchmark Requirements:",
                "  - Must perform REAL calculations, not just retrieve information",
                "  - Use loops/iterations to measure performance",
                "  - Include timing: time.time() or time.perf_counter()",
                "  - Output quantitative metrics: ops/sec, GFLOPS, elapsed time",
                "",
                "? Example for CPU benchmark:",
                "    import time",
                "    start = time.perf_counter()",
                "    for _ in range(1000000):",
                "        _ = 2 * 2  # Actual calculation",
                "    end = time.perf_counter()",
                "    ops_per_sec = 1000000 / (end - start)",
                "    print(f'Performance: {ops_per_sec:,.0f} ops/sec')",
            ]
        )
    return "\n".join(lines)
def get_task_status(self, task_id: str) -> SupervisedTask | None:
    return self.tasks.get(task_id)
def should_escalate(self, task_id: str) -> bool:
    if task_id not in self.tasks:
        return False
    task = self.tasks[task_id]
    return task.attempts >= task.max_attempts and task.status != TaskStatus.COMPLETED
def get_retry_prompt(self, task_id: str, result: SupervisionResult) -> str:
    if task_id not in self.tasks:
        return ""
    task = self.tasks[task_id]
    prompt_parts = [
        f"The previous attempt did not fully complete the task (Attempt {task.attempts}/{ta
sk.max_attempts}).",
        "",
        "Original request:",
        f"  {task.original_request}",
        "",
```

```
        f"Completion: {result.completion_percentage:.1f}%",
        "",
        "Issues to fix:",
    ]
    for i, issue in enumerate(result.missing_requirements[:5], 1):
        prompt_parts.append(f"  {i}. {issue}")
    prompt_parts.extend(
        [
            "",
            "Required actions:",
        ]
    )
    for i, action in enumerate(result.suggested_actions[:5], 1):
        prompt_parts.append(f"  {i}. {action}")
    if result.retry_reason == RetryReason.AI_DETECTED:
        prompt_parts.extend(
            [
                "",
                "?? CRITICAL: Anti-AI Quality Check Failed",
                "Your previous output was detected as AI-generated due to:",
                "  - Generic bullet points without substance",
                "  - Vague claims without evidence",
                "  - Missing specific numbers and citations",
                "",
                "You MUST:",
                "  1. Search for REAL papers using WebSearch and cite them properly",
                "  2. Include SPECIFIC numbers: 'X% improvement', 'Y lines of code'",
                "  3. Reference ACTUAL code: 'file.py:157 implements lazy loading'",
                "  4. Write COMPLETE paragraphs, not fragmented lists",
                "  5. Explain HOW and WHY, provide technical depth",
            ]
        )
    prompt_parts.extend(
        [
            "",
            "Please continue the task and address all missing requirements.",
            "Focus on producing HIGH-QUALITY, SPECIFIC, EVIDENCE-BASED content.",
        ]
    )
    return "\n".join(prompt_parts)
def enable_supervision(self, enabled: bool = True) -> None:
    self._supervision_enabled = enabled
def enable_anti_ai_mode(self, enabled: bool = True) -> None:
    self._anti_ai_mode = enabled
    self.checker.enable_anti_ai_check(enabled)
def get_step_prompt(self, task_id: str) -> str | None:
    """Get the current step prompt for decomposed tasks."""
    if task_id not in self.tasks:
        return None
    task = self.tasks[task_id]
    if not task.decomposed_steps or task.current_step >= len(task.decomposed_steps):
        return None
    step = task.decomposed_steps[task.current_step]
    return f"Step {task.current_step + 1}/{len(task.decomposed_steps)}: {step}"
def advance_step(self, task_id: str) -> bool:
    """Advance to the next step in a decomposed task."""
    if task_id not in self.tasks:
        return False
    task = self.tasks[task_id]
    if task.current_step < len(task.decomposed_steps) - 1:
        task.current_step += 1
        return True
    return False
def get_timeout(self, base_timeout: float = 60.0) -> float:
    """Get model-adjusted timeout."""
```

```
    if self.task_decomposer:
        return self.task_decomposer.get_timeout(base_timeout)
    return base_timeout
def get_model_guidance(self) -> str:
    """Get guidance specific to current model tier."""
    if not self.task_decomposer:
        return ""
    gran = self.task_decomposer.gran
    guidance = []
    if gran.requires_explicit_steps:
        guidance.append("This task will be broken into explicit atomic steps.")
    if gran.verification_frequency in ["every_step", "frequent"]:
        guidance.append("Each step will be verified before proceeding.")
    if gran.max_subtasks < 10:
        guidance.append(f"Task complexity limited to {gran.max_subtasks} subtasks.")
    if gran.min_confidence_threshold > 0.8:
        guidance.append("High confidence required - provide specific evidence.")
    return " ".join(guidance) if guidance else ""
```

File: [145/252] openlaoke/core/system_prompt.py

```
"""System prompt builder for the AI model."""
from __future__ import annotations
import os
import platform
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
def build_system_prompt(app_state: AppState, tools_description: list[dict]) -> str:
    """Build the system prompt sent to the model."""
    parts = []
    parts.append(
        "You are OpenLaoKe, an expert AI coding assistant designed to help with "
        "software engineering tasks. You can read and write files, run shell commands, "
        "search codebases, and spawn sub-agents for parallel work.\n"
    )
    parts.append(
        "## Core Principles\n"
        "- Be concise and direct in your responses\n"
        "- Always verify your work after making changes\n"
        "- Read files before editing them to understand context\n"
        "- Use the Edit tool for targeted changes, Write for new files\n"
        "- Run tests to verify your changes work\n"
        "- Never commit secrets or API keys\n"
        "- Follow the existing code style and conventions\n"
    )
    parts.append(
        f"## Environment\n"
        f"- Platform: {platform.system()} {platform.release()}\n"
        f"- Python: {platform.python_version()}\n"
        f"- Working directory: {app_state.get_cwd()}\n"
        f"- Shell: {os.environ.get('SHELL', '/bin/bash')}\n"
    )
    git_branch = _get_git_branch(app_state.get_cwd())
    if git_branch:
        parts.append(f"- Git branch: {git_branch}\n")
    parts.append(
        "## Tool Usage Guidelines\n"
        "- Read files before editing to understand the current state\n"
        "- Use Glob to find files when you don't know the exact path\n"
        "- Use Grep to search for patterns across multiple files\n"
        "- Use Bash for running commands, tests, and git operations\n"
        "- Use Edit for small targeted changes, Write for new files\n"
        "- Use Agent to delegate independent parallel tasks\n"
        "\n"
        "IMPORTANT: When using tools, ALWAYS provide ALL required parameters:\n"
        "- Write tool: requires both 'file_path' AND 'content'\n"
        "- Edit tool: requires 'file_path', 'old_string', AND 'new_string'\n"
        "- Bash tool: requires 'command'\n"
        "- Read tool: requires 'file_path'\n"
        "Never omit required parameters. If you're unsure about a parameter, ask the user.\n"
    )
    parts.append(
        "## Response Format\n"
        "- Keep explanations concise and focused\n"
        "- Show relevant code snippets when explaining\n"
        "- Always explain what you're doing and why\n"
        "- If uncertain, say so and suggest how to verify\n"
    )
    parts.append(
        "## Anti-AI Quality Standards (MANDATORY)\n"
        "\n"
```

```
        "CRITICAL: Your output will be checked for AI-typical patterns. You MUST avoid:\n"
        "\n"
        "1. **Empty numbered lists** - Never write lists without substantive content\n"
        '   [X] BAD: "Benefits include: 1. Speed 2. Quality 3. Cost"\n'
        '   [OK] GOOD: "The system achieves 2.3s average response time (34% faster than baselin
    e), '
        '94% code accuracy on HumanEval benchmark, and $0.003 per query operational cost."\n'
        "\n"
        "2. **Vague claims without evidence** - Every claim needs SPECIFIC numbers or citations
    \n"
        '   [X] BAD: "significant improvement", "novel approach", "state-of-the-art"\n'
        '   [OK] GOOD: "45% reduction in latency (p<0.01, n=1000 trials)", '
        '"as demonstrated in Smith et al. [3]"\n'
        "\n"
        "3. **AI-typical phrases** - Avoid these patterns:\n"
        '   - "Systems could enable:"\n'
        '   - "Improvements include:"\n'
        '   - "The main contributions are:"\n'
        "   - Generic bullet points as paragraphs\n"
        "\n"
        "4. **Missing technical depth** - Explain HOW and WHY, not just WHAT\n"
        '   [X] BAD: "The system uses caching for performance."\n'
        '   [OK] GOOD: "The system implements LRU caching with 256MB capacity in Redis, '
        'achieving 89% cache hit rate and reducing database queries by 67% (measured over 10k r
    equests)."\n'
        "\n"
        "5. **No real citations** - Use WebSearch to find REAL papers\n"
        "   - Always cite actual papers with [number] format\n"
        "   - Reference specific code with file:line format\n"
        "   - Include actual measurements and numbers\n"
        "\n"
        "VERIFICATION: Before finishing ANY task:\n"
        "- Does every paragraph have specific details (numbers, citations, code refs)?\n"
        "- Are there more than 3 bullet points without explanation?\n"
        "- Can you replace generic phrases with specific evidence?\n"
    )
    if hasattr(app_state, "active_skills") and app_state.active_skills:
        from openlaoke.core.skill_system import get_skill_system_prompt
        skill_prompt = get_skill_system_prompt(app_state.active_skills)
        if skill_prompt:
            parts.append(f"\n## Active Skills\n{skill_prompt}")
    suffix = app_state.session_config.system_prompt_suffix
    if suffix:
        parts.append(f"\n## Additional Instructions\n{suffix}")
    return "\n".join(parts)
def _get_git_branch(cwd: str) -> str | None:
    try:
        import subprocess
        result = subprocess.run(
            ["git", "rev-parse", "--abbrev-ref", "HEAD"],
            cwd=cwd,
            capture_output=True,
            text=True,
            timeout=5,
        )
        if result.returncode == 0:
            return result.stdout.strip()
    except Exception:
        pass
    return None
```

File: [146/252] openlaoke/core/task.py

```
"""Task system with lifecycle management."""
from __future__ import annotations
import asyncio
import os
import time
from collections.abc import Callable
from contextlib import suppress
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from openlaoke.types.core_types import (
    TaskId,
    TaskState,
    TaskStatus,
    TaskType,
    is_terminal_task_status,
)
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class TaskHandle:
    """Handle to a running task for cleanup and monitoring."""
    task_id: str
    task: asyncio.Task | None = None
    cleanup: Callable[..., Any] | None = None
    async def kill(self) -> None:
        if self.task and not self.task.done():
            self.task.cancel()
            with suppress(asyncio.CancelledError):
                await self.task
        if self.cleanup:
            self.cleanup()
class TaskManager:
    """Manages the lifecycle of all tasks (bash, agent, etc.)."""
    def __init__(self, app_state: AppState) -> None:
        self.app_state = app_state
        self._id_generator = TaskId()
        self._handles: dict[str, TaskHandle] = {}
        self._output_dir = os.path.join(os.path.expanduser("~/.openlaoke"), "task_outputs")
        os.makedirs(self._output_dir, exist_ok=True)
    def _get_output_path(self, task_id: str) -> str:
        return os.path.join(self._output_dir, f"{task_id}.log")
    def create_task_state(
        self,
        task_type: TaskType,
        description: str,
        tool_use_id: str | None = None,
    ) -> TaskState:
        task_id = self._id_generator.generate(task_type)
        state = TaskState(
            id=task_id,
            type=task_type,
            status=TaskStatus.PENDING,
            description=description,
            tool_use_id=tool_use_id,
            start_time=time.time(),
            output_file=self._get_output_path(task_id),
        )
        self.app_state.add_task(state)
        return state
    async def run_bash(
        self,
```

```
    command: str,
    description: str,
    timeout: float | None = None,
    tool_use_id: str | None = None,
    working_dir: str | None = None,
) -> tuple[str, int]:
    state = self.create_task_state(TaskType.LOCAL_BASH, description, tool_use_id)
    state.status = TaskStatus.RUNNING
    self.app_state.update_task(state)
    env = os.environ.copy()
    env.update(self.app_state.get_env_vars())
    try:
        proc = await asyncio.create_subprocess_shell(
            command,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.STDOUT,
            cwd=working_dir or self.app_state.get_cwd(),
            env=env,
        )
        output_chunks: list[bytes] = []
        with open(state.output_file, "a", encoding="utf-8", errors="replace") as f:
            while proc.stdout is not None:
                chunk = await proc.stdout.read(4096)
                if not chunk:
                    break
                output_chunks.append(chunk)
                decoded = chunk.decode("utf-8", errors="replace")
                f.write(decoded)
                f.flush()
                state.output_offset += len(decoded)
                self.app_state.update_task(state)
        await proc.wait()
        exit_code = proc.returncode or 0
        state.status = TaskStatus.COMPLETED if exit_code == 0 else TaskStatus.FAILED
        state.end_time = time.time()
        state.exit_code = exit_code
        state.output = b"".join(output_chunks).decode("utf-8", errors="replace")
        self.app_state.update_task(state)
        return state.output, exit_code
    except asyncio.CancelledError:
        state.status = TaskStatus.KILLED
        state.end_time = time.time()
        self.app_state.update_task(state)
        raise
    except Exception as e:
        state.status = TaskStatus.FAILED
        state.end_time = time.time()
        state.output = str(e)
        state.exit_code = 1
        self.app_state.update_task(state)
        return str(e), 1
async def run_agent(
    self,
    prompt: str,
    description: str,
    tool_use_id: str | None = None,
    subagent_type: str = "local",
) -> str:
    state = self.create_task_state(TaskType.LOCAL_AGENT, description, tool_use_id)
    state.status = TaskStatus.RUNNING
    self.app_state.update_task(state)
    try:
        from openlaoke.core.agent_runner import run_subagent
        result = await run_subagent(
            prompt=prompt,
```

```
            description=description,
            app_state=self.app_state,
            task_state=state,
        )
        state.status = TaskStatus.COMPLETED
        state.end_time = time.time()
        state.output = result
        self.app_state.update_task(state)
        return result
    except asyncio.CancelledError:
        state.status = TaskStatus.KILLED
        state.end_time = time.time()
        self.app_state.update_task(state)
        raise
    except Exception as e:
        state.status = TaskStatus.FAILED
        state.end_time = time.time()
        state.output = str(e)
        self.app_state.update_task(state)
        return f"Agent failed: {e}"
def kill_task(self, task_id: str) -> None:
    if task_id in self._handles:
        handle = self._handles[task_id]
        asyncio.ensure_future(handle.kill())
        del self._handles[task_id]
    state = self.app_state.get_task(task_id)
    if state and not is_terminal_task_status(state.status):
        state.status = TaskStatus.KILLED
        state.end_time = time.time()
        self.app_state.update_task(state)
def get_active_tasks(self) -> list[TaskState]:
    return [t for t in self.app_state.get_all_tasks() if not is_terminal_task_status(t.stat
us)]
```

File: [147/252] openlaoke/core/tool.py

```
"""Base Tool system - all tools inherit from this."""
from __future__ import annotations
import asyncio
import json
from abc import ABC, abstractmethod
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any, cast
from pydantic import BaseModel
from openlaoke.types.core_types import (
    PermissionResult,
    TaskType,
    ToolProgress,
    ToolResultBlock,
    ValidationResult,
)
from openlaoke.types.permissions import PermissionConfig
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
@dataclass
class ToolContext:
    """Context passed to tool calls."""
    app_state: AppState
    tool_use_id: str
    agent_id: str | None = None
    abort_signal: Any = None
class Tool(ABC):
    """Base class for all tools. Subclasses implement specific capabilities."""
    name: str = "base_tool"
    description: str = ""
    input_schema: type[BaseModel] | dict[str, Any] = {}
    task_type: TaskType = TaskType.LOCAL_BASH
    is_read_only: bool = False
    is_destructive: bool = False
    is_concurrency_safe: bool = True
    requires_approval: bool = False
    @abstractmethod
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        """Execute the tool with the given input."""
        ...
    def get_description(self) -> str:
        """Return the tool description for the system prompt."""
        return self.description
    def get_input_schema(self) -> dict[str, Any]:
        """Return the JSON schema for tool input validation."""
        if isinstance(self.input_schema, dict):
            return self.input_schema
        if hasattr(self.input_schema, "model_json_schema"):
            return self.input_schema.model_json_schema()
        return {}
    def validate_input(self, input_data: dict[str, Any]) -> ValidationResult:
        """Validate tool input before execution."""
        if isinstance(self.input_schema, dict):
            required = self.input_schema.get("required", [])
            properties = self.input_schema.get("properties", {})
            for field_name in required:
                if field_name not in input_data:
                    return ValidationResult(
                        result=False,
                        message=f"Missing required field: {field_name}",
                        error_code=400,
```

```
                )
            for field_name, value in input_data.items():
                if field_name in properties:
                    field_type = properties[field_name].get("type")
                    if field_type == "string" and not isinstance(value, str):
                        return ValidationResult(
                            result=False,
                            message=f"Field '{field_name}' must be a string",
                            error_code=400,
                        )
        elif hasattr(self.input_schema, "model_json_schema"):
            try:
                schema = self.input_schema.model_json_schema()
                required = schema.get("required", [])
                for field_name in required:
                    if field_name not in input_data or input_data.get(field_name) is None:
                        return ValidationResult(
                            result=False,
                            message=f"Missing required field: {field_name}",
                            error_code=400,
                        )
            except Exception:
                pass
        return ValidationResult(result=True)
    def check_permissions(
        self,
        input_data: dict[str, Any],
        permission_config: PermissionConfig,
    ) -> PermissionResult:
        """Check if the tool call is permitted."""
        return permission_config.check_tool(self.name)
    def get_progress(self, ctx: ToolContext, **kwargs: Any) -> ToolProgress | None:
        """Return current progress for long-running tools."""
        return None
    def render_result(self, result: ToolResultBlock) -> str:
        """Render the tool result for display in the terminal."""
        if isinstance(result.content, str):
            return result.content
        return json.dumps(result.content, indent=2)
    def get_deny_message(self, input_data: dict[str, Any]) -> str:
        """Message shown when the tool call is denied."""
        return f"Tool '{self.name}' was denied by the user."
class DeferredToolInfo:
    """Info about a deferred tool without loading it."""
    def __init__(
        self,
        name: str,
        description: str = "",
        search_hint: str = "",
        aliases: list[str] | None = None,
    ) -> None:
        self.name = name
        self.description = description
        self.search_hint = search_hint
        self.aliases = aliases or []
class ToolRegistry:
    """Registry for all available tools with lazy loading support."""
    def __init__(self) -> None:
        self._tools: dict[str, Tool] = {}
        self._deferred_loaders: dict[str, Callable[[], Tool] | Callable[[], Awaitable[Tool]]] =
     {}
        self._deferred_info: dict[str, DeferredToolInfo] = {}
        self._load_locks: dict[str, asyncio.Lock] = {}
    def register(self, tool: Tool) -> None:
        self._tools[tool.name] = tool
```

```
def register_deferred(
    self, name: str, loader: Callable[[], Tool] | Callable[[], Awaitable[Tool]]
) -> None:
    self._deferred_loaders[name] = loader
    self._deferred_info[name] = DeferredToolInfo(name)
def register_deferred_with_info(
    self,
    name: str,
    loader: Callable[[], Tool] | Callable[[], Awaitable[Tool]],
    description: str = "",
    search_hint: str = "",
    aliases: list[str] | None = None,
) -> None:
    self._deferred_loaders[name] = loader
    self._deferred_info[name] = DeferredToolInfo(name, description, search_hint, aliases)
def get(self, name: str) -> Tool | None:
    if name in self._tools:
        return self._tools[name]
    if name in self._deferred_loaders:
        loader = self._deferred_loaders[name]
        if asyncio.iscoroutinefunction(loader):
            return None
        sync_loader = cast(Callable[[], Tool], loader)
        tool = sync_loader()
        self._tools[name] = tool
        del self._deferred_loaders[name]
        return tool
    return None
async def get_async(self, name: str) -> Tool | None:
    if name in self._tools:
        return self._tools[name]
    if name in self._deferred_loaders:
        lock = self._load_locks.setdefault(name, asyncio.Lock())
        async with lock:
            if name in self._tools:
                return self._tools[name]
            loader = self._deferred_loaders[name]
            tool: Tool
            if asyncio.iscoroutinefunction(loader):
                async_loader = cast(Callable[[], Awaitable[Tool]], loader)
                tool = await async_loader()
            else:
                sync_loader = cast(Callable[[], Tool], loader)
                tool = sync_loader()
            self._tools[name] = tool
            del self._deferred_loaders[name]
            if name in self._deferred_info:
                del self._deferred_info[name]
            return tool
    return None
def is_loaded(self, name: str) -> bool:
    return name in self._tools
def is_deferred(self, name: str) -> bool:
    return name in self._deferred_loaders
def get_deferred_info(self, name: str) -> DeferredToolInfo | None:
    return self._deferred_info.get(name)
def get_all_deferred_info(self) -> list[DeferredToolInfo]:
    return list(self._deferred_info.values())
def get_loaded(self) -> list[Tool]:
    return list(self._tools.values())
def get_all(self) -> list[Tool]:
    for loader_name in list(self._deferred_loaders.keys()):
        loader = self._deferred_loaders[loader_name]
        if not asyncio.iscoroutinefunction(loader):
            sync_loader = cast(Callable[[], Tool], loader)
```

```
            tool = sync_loader()
            self._tools[loader_name] = tool
            del self._deferred_loaders[loader_name]
            if loader_name in self._deferred_info:
                del self._deferred_info[loader_name]
    return list(self._tools.values())
async def get_all_async(self) -> list[Tool]:
    for loader_name in list(self._deferred_loaders.keys()):
        await self.get_async(loader_name)
    return list(self._tools.values())
def get_all_for_prompt(self) -> list[dict[str, Any]]:
    tools = self.get_all()
    return [
        {
            "name": tool.name,
            "description": tool.get_description(),
            "input_schema": tool.get_input_schema(),
        }
        for tool in tools
    ]
def get_deferred_for_prompt(self) -> list[dict[str, Any]]:
    results: list[dict[str, Any]] = []
    for name, info in self._deferred_info.items():
        results.append(
            {
                "name": name,
                "description": info.description,
                "search_hint": info.search_hint,
                "aliases": info.aliases,
                "defer_loading": True,
            }
        )
    return results
def search(self, query: str) -> list[Tool]:
    query_lower = query.lower()
    results = []
    for tool in self.get_all():
        if query_lower in tool.name.lower() or query_lower in tool.description.lower():
            results.append(tool)
    return results
def search_deferred(self, query: str) -> list[DeferredToolInfo]:
    query_lower = query.lower()
    results: list[DeferredToolInfo] = []
    for info in self._deferred_info.values():
        if (
            query_lower in info.name.lower()
            or query_lower in info.description.lower()
            or query_lower in info.search_hint.lower()
            or any(query_lower in alias.lower() for alias in info.aliases)
        ):
            results.append(info)
    return results
```

File: [148/252] openlaoke/core/tool_adapter.py

```
"""Tool calling adapter for models without native function calling support.
Inspired by POSIX-Compatibility-Layer's intent parsing approach.
Converts natural language tool calls into structured format.
"""
from __future__ import annotations
import re
from dataclasses import dataclass
from typing import Any
@dataclass
class ParsedToolCall:
    name: str
    arguments: dict[str, Any]
    confidence: float
    raw_text: str
class ToolCallAdapter:
    """Adapts models without tool support to use tools via text parsing."""
    TOOL_PATTERNS = {
        "bash": [
            r"(?:run|execute|bash|shell|command):\s*`([^`]+)`",
            r"(?:run|execute|bash|shell|command):\s*\*\*([^*]+)\*\*",
            r"```(?:bash|shell)\s*\n(.*?)\n```",
            r"(?:execute|run)\s+(?:the\s+)?(?:command|script):\s*(.+?)(?:\n|$)",
        ],
        "read": [
            r"(?:read|show|display|cat)\s+(?:the\s+)?(?:file\s+)?([^\s]+\.\w+)",
            r"```(?:read|file)\s*\n(.*?)\n```",
        ],
        "write": [
            r"(?:write|create|save)\s+(?:file\s+)?[`']?([^'`\s]+)[`']?\s*(?:with|containing|:)\
    s*(.+)",
            r"(?:create|make)\s+(?:a\s+)?(?:new\s+)?file\s+[`']?([^'`\s]+)[`']?",
            r"```write\s+[`']?([^'`\s]+)[`']?\s*\n(.*?)\n```",
        ],
        "edit": [
            r"(?:edit|modify|change|update)\s+[`']?([^'`\s]+)[`']?\s*(?::|to|with)\s*(.+)",
            r"(?:replace|substitute)\s+[`']?([^'`\s]+)[`']?",
        ],
        "glob": [
            r"(?:find|search|glob)\s+(?:files?\s+)?(?:matching\s+)?[`']?([^'`\s]+)[`']?",
            r"(?:list|show)\s+(?:all\s+)?(?:files?\s+)?(?:matching\s+)?[`']?([^'`\s]+)[`']?",
        ],
        "grep": [
            r"(?:grep|search)\s+(?:for\s+)?[`']?([^'`\s]+)[`']?\s+(?:in\s+)?[`']?([^'`\s]+)[`']
    ?",
            r"(?:find|search)\s+(?:pattern\s+)?[`']?([^'`\s]+)[`']?",
        ],
    }
    CREATION_KEYWORDS = [
        "write",
        "create",
        "make",
        "generate",
        "build",
        "implement",
        "develop",
        "compose",
        "produce",
        "draft",
        "design",
        "?",
        "??",
```

```
    "??",
    "??",
    "??",
    "??",
    "??",
]
NEW_FILE_INDICATORS = [
    "new file",
    "single file",
    "standalone",
    "independent",
    "???",
    "???",
    "??",
    "??",
    "from scratch",
    "from zero",
    "??",
    "??",
]
def __init__(self, model: str):
    self.model = model
    self.no_tool_models = {
        "gemma3:1b",
        "gemma3:4b",
        "qwen3.5:0.8b",
        "qwen3.5:1.5b",
        "llama3.2:1b",
        "llama3.2:3b",
        "phi3:mini",
        "phi3:tiny",
    }
def supports_tools(self) -> bool:
    """Check if model natively supports tool calling."""
    model_lower = self.model.lower()
    return not any(no_tool in model_lower for no_tool in self.no_tool_models)
def is_creation_request(self, user_input: str) -> bool:
    """Detect if user wants to create something new (not modify existing)."""
    user_lower = user_input.lower()
    has_creation = any(kw in user_lower for kw in self.CREATION_KEYWORDS)
    has_new_indicator = any(ind in user_lower for ind in self.NEW_FILE_INDICATORS)
    return has_creation or has_new_indicator
def should_skip_context_gathering(self, user_input: str) -> bool:
    """Determine if we should skip reading existing files."""
    if not self.is_creation_request(user_input):
        return False
    modification_keywords = [
        "modify",
        "update",
        "edit",
        "change",
        "fix",
        "refactor",
        "based on",
        "modify existing",
        "??",
        "??",
        "??",
        "??",
    ]
    user_lower = user_input.lower()
    has_modification = any(kw in user_lower for kw in modification_keywords)
    return not has_modification
def parse_tool_calls(self, text: str) -> list[ParsedToolCall]:
    """Parse tool calls from model's text output."""
```

```
    if self.supports_tools():
        return []
    calls = []
    for tool_name, patterns in self.TOOL_PATTERNS.items():
        for pattern in patterns:
            matches = re.finditer(pattern, text, re.IGNORECASE | re.DOTALL)
            for match in matches:
                groups = match.groups()
                args = self._extract_arguments(tool_name, groups)
                calls.append(
                    ParsedToolCall(
                        name=tool_name,
                        arguments=args,
                        confidence=0.8,
                        raw_text=match.group(0),
                    )
                )
    return calls
def _extract_arguments(self, tool_name: str, groups: tuple[str, ...]) -> dict[str, Any]:
    """Extract tool arguments from regex match groups."""
    args = {}
    if tool_name == "bash":
        if groups:
            args["command"] = groups[0].strip()
    elif tool_name == "read":
        if groups:
            args["file_path"] = groups[0].strip()
    elif tool_name == "write":
        if len(groups) >= 1:
            args["file_path"] = groups[0].strip()
        if len(groups) >= 2:
            args["content"] = groups[1].strip()
    elif tool_name == "edit":
        if len(groups) >= 1:
            args["file_path"] = groups[0].strip()
        if len(groups) >= 2:
            args["new_string"] = groups[1].strip()
    elif tool_name == "glob":
        if groups:
            args["pattern"] = groups[0].strip()
    elif tool_name == "grep":
        if len(groups) >= 1:
            args["pattern"] = groups[0].strip()
        if len(groups) >= 2:
            args["path"] = groups[1].strip()
    return args
def format_tools_as_text(self, tools: list[dict[str, Any]]) -> str:
    """Format tools specification as natural language for non-tool models."""
    if self.supports_tools():
        return ""
    lines = [
        "\nYou have access to the following tools. Describe your tool usage in natural lang
uage:",
        "",
    ]
    for tool in tools:
        name = tool.get("name", "unknown")
        desc = tool.get("description", "")
        schema = tool.get("input_schema", {})
        properties = schema.get("properties", {})
        lines.append(f"**{name}**: {desc}")
        if properties:
            lines.append("  Parameters:")
            for param_name, param_spec in properties.items():
                param_type = param_spec.get("type", "any")
```

```
                    param_desc = param_spec.get("description", "")
                    required = param_name in schema.get("required", [])
                    req_marker = " (required)" if required else ""
                    lines.append(f"    - {param_name} ({param_type}){req_marker}: {param_desc}"
    )
            lines.append("")
            lines.append(f"  Example: Use {name} to [description of what to do]")
            lines.append("")
        lines.extend(
            [
                "When you need to use a tool, describe it naturally like:",
                '- "I will read the file config.py"',
                '- "Let me run the command: pip install requests"',
                '- "I will create a new file main.py with the following content..."',
                "",
            ]
        )
        return "\n".join(lines)
def create_adapter(model: str) -> ToolCallAdapter:
    """Create a tool adapter for the given model."""
    return ToolCallAdapter(model)
```

File: [149/252] openlaoke/core/tools/__init__.py

```
"""Tool lazy loading system."""
from __future__ import annotations
from openlaoke.core.tools.deferred_registry import DeferredRegistry, DeferredTool
from openlaoke.core.tools.lazy_loader import LazyToolLoader
from openlaoke.core.tools.tool_discovery import ToolDiscovery
from openlaoke.core.tools.tool_search import ToolSearchTool
__all__ = [
    "LazyToolLoader",
    "DeferredTool",
    "DeferredRegistry",
    "ToolSearchTool",
    "ToolDiscovery",
]
```

File: [150/252] openlaoke/core/tools/deferred_registry.py

```
"""Deferred tool registry for lazy loading."""
from __future__ import annotations
from collections.abc import Awaitable, Callable
from dataclasses import dataclass, field
from typing import TYPE_CHECKING, Any
if TYPE_CHECKING:
    from openlaoke.core.tool import Tool
ESSENTIAL_TOOLS: set[str] = {
    "Bash",
    "Read",
    "Write",
    "Edit",
    "Glob",
    "Grep",
}
DEFERRED_TOOLS: set[str] = {
    "WebSearch",
    "WebFetch",
    "LSP",
    "Git",
    "Batch",
    "NotebookEdit",
    "Plan",
    "Agent",
    "TaskKill",
    "Todo",
    "Question",
    "LS",
    "ApplyPatch",
}
@dataclass
class DeferredTool:
    """Represents a tool that can be loaded on demand."""
    name: str
    description: str
    loader: Callable[[], Awaitable[Tool]] | Callable[[], Tool]
    is_loaded: bool = False
    always_load: bool = False
    priority: int = 0
    search_hint: str = ""
    aliases: list[str] = field(default_factory=list)
    def __hash__(self) -> int:
        return hash(self.name)
    def __eq__(self, other: object) -> bool:
        if isinstance(other, DeferredTool):
            return self.name == other.name
        return False
@dataclass
class ToolInfo:
    """Lightweight info about a tool without loading it."""
    name: str
    description: str
    search_hint: str = ""
    aliases: list[str] = field(default_factory=list)
    is_loaded: bool = False
    always_load: bool = False
class DeferredRegistry:
    """Registry for deferred tools with metadata."""
    def __init__(self) -> None:
        self._deferred: dict[str, DeferredTool] = {}
        self._essential: set[str] = ESSENTIAL_TOOLS.copy()
```

```
    self._deferred_set: set[str] = DEFERRED_TOOLS.copy()
def register(
    self,
    name: str,
    description: str,
    loader: Callable[[], Awaitable[Tool]] | Callable[[], Tool],
    always_load: bool = False,
    priority: int = 0,
    search_hint: str = "",
    aliases: list[str] | None = None,
) -> None:
    """Register a deferred tool."""
    self._deferred[name] = DeferredTool(
        name=name,
        description=description,
        loader=loader,
        always_load=always_load,
        priority=priority,
        search_hint=search_hint,
        aliases=aliases or [],
    )
def get(self, name: str) -> DeferredTool | None:
    """Get deferred tool info by name."""
    return self._deferred.get(name)
def get_by_alias(self, alias: str) -> DeferredTool | None:
    """Get deferred tool by alias."""
    for tool in self._deferred.values():
        if alias in tool.aliases:
            return tool
    return None
def is_essential(self, name: str) -> bool:
    """Check if tool is essential (always loaded)."""
    tool = self._deferred.get(name)
    return name in self._essential or (tool is not None and tool.always_load)
def is_deferred(self, name: str) -> bool:
    """Check if tool should be deferred."""
    return name in self._deferred_set or name in self._deferred
def get_all_deferred(self) -> list[DeferredTool]:
    """Get all registered deferred tools."""
    return list(self._deferred.values())
def get_all_essential_names(self) -> set[str]:
    """Get names of essential tools."""
    return self._essential.copy()
def get_all_deferred_names(self) -> set[str]:
    """Get names of deferred tools."""
    return self._deferred_set.copy()
def get_tool_info(self, name: str) -> ToolInfo | None:
    """Get lightweight info about a tool."""
    deferred = self._deferred.get(name)
    if deferred:
        return ToolInfo(
            name=deferred.name,
            description=deferred.description,
            search_hint=deferred.search_hint,
            aliases=deferred.aliases,
            is_loaded=deferred.is_loaded,
            always_load=deferred.always_load,
        )
    return None
def mark_loaded(self, name: str) -> None:
    """Mark a tool as loaded."""
    if name in self._deferred:
        self._deferred[name].is_loaded = True
def add_essential(self, name: str) -> None:
    """Add a tool to essential set."""
```

```
    self._essential.add(name)
def add_deferred(self, name: str) -> None:
    """Add a tool to deferred set."""
    self._deferred_set.add(name)
def search(self, query: str) -> list[DeferredTool]:
    """Search deferred tools by name or description."""
    query_lower = query.lower()
    results: list[DeferredTool] = []
    for tool in self._deferred.values():
        if (
            query_lower in tool.name.lower()
            or query_lower in tool.description.lower()
            or query_lower in tool.search_hint.lower()
            or any(query_lower in alias.lower() for alias in tool.aliases)
        ):
            results.append(tool)
    return sorted(results, key=lambda t: t.priority, reverse=True)
def to_dict(self) -> dict[str, Any]:
    """Export registry state."""
    return {
        "essential": list(self._essential),
        "deferred": list(self._deferred_set),
        "registered": [
            {
                "name": t.name,
                "description": t.description,
                "is_loaded": t.is_loaded,
                "always_load": t.always_load,
                "priority": t.priority,
            }
            for t in self._deferred.values()
        ],
    }
```

File: [151/252] openlaoke/core/tools/lazy_loader.py

```
"""Lazy tool loader for on-demand tool loading."""
from __future__ import annotations
import asyncio
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any, cast
from openlaoke.core.tools.deferred_registry import DeferredRegistry, DeferredTool, ToolInfo
if TYPE_CHECKING:
    from openlaoke.core.tool import Tool
@dataclass
class LoaderStats:
    """Statistics about tool loading."""
    total_tools: int = 0
    loaded_tools: int = 0
    deferred_tools: int = 0
    load_count: int = 0
    cache_hits: int = 0
class LazyToolLoader:
    """Tool lazy loader - loads tools on demand to save resources."""
    def __init__(self) -> None:
        self.loaded_tools: dict[str, Tool] = {}
        self.deferred_tools: dict[str, DeferredTool] = {}
        self.always_load: set[str] = set()
        self.defer_loading: set[str] = set()
        self._registry: DeferredRegistry = DeferredRegistry()
        self._stats: LoaderStats = LoaderStats()
        self._load_lock: dict[str, asyncio.Lock] = {}
    def register_deferred(
        self,
        name: str,
        loader: Callable[[], Tool] | Callable[[], Awaitable[Tool]],
        description: str = "",
        always_load: bool = False,
        search_hint: str = "",
        aliases: list[str] | None = None,
    ) -> None:
        """Register a deferred loading tool."""
        deferred = DeferredTool(
            name=name,
            description=description,
            loader=loader,
            always_load=always_load,
            search_hint=search_hint,
            aliases=aliases or [],
        )
        self.deferred_tools[name] = deferred
        self._registry.register(
            name=name,
            description=description,
            loader=loader,
            always_load=always_load,
            search_hint=search_hint,
            aliases=aliases,
        )
        self._stats.deferred_tools += 1
        self._stats.total_tools += 1
        if always_load:
            self.always_load.add(name)
    def register_loaded(self, name: str, tool: Tool) -> None:
        """Register an already-loaded tool."""
        self.loaded_tools[name] = tool
```

```
    self._stats.loaded_tools += 1
    self._stats.total_tools += 1
async def get(self, name: str) -> Tool | None:
    """Get tool (load on demand if deferred)."""
    if name in self.loaded_tools:
        self._stats.cache_hits += 1
        return self.loaded_tools[name]
    if name in self.deferred_tools:
        lock = self._load_lock.setdefault(name, asyncio.Lock())
        async with lock:
            if name in self.loaded_tools:
                self._stats.cache_hits += 1
                return self.loaded_tools[name]
            deferred = self.deferred_tools[name]
            loader = deferred.loader
            tool: Tool
            if asyncio.iscoroutinefunction(loader):
                async_loader = cast(Callable[[], Awaitable[Tool]], loader)
                tool = await async_loader()
            else:
                sync_loader = cast(Callable[[], Tool], loader)
                tool = sync_loader()
            self.loaded_tools[name] = tool
            deferred.is_loaded = True
            self._registry.mark_loaded(name)
            self._stats.load_count += 1
            self._stats.loaded_tools += 1
            return tool
    return None
def get_sync(self, name: str) -> Tool | None:
    """Get tool synchronously (only works for already-loaded tools)."""
    return self.loaded_tools.get(name)
def is_loaded(self, name: str) -> bool:
    """Check if tool is already loaded."""
    return name in self.loaded_tools
def is_deferred(self, name: str) -> bool:
    """Check if tool is deferred."""
    return name in self.deferred_tools
async def preload_essential(self) -> None:
    """Preload all essential tools."""
    for name in list(self.always_load):
        if not self.is_loaded(name):
            await self.get(name)
async def load_all(self) -> None:
    """Load all registered tools."""
    for name in list(self.deferred_tools.keys()):
        if not self.is_loaded(name):
            await self.get(name)
async def load_by_names(self, names: list[str]) -> dict[str, Tool]:
    """Load specific tools by name."""
    results: dict[str, Tool] = {}
    for name in names:
        tool = await self.get(name)
        if tool:
            results[name] = tool
    return results
def get_tool_info(self, name: str) -> ToolInfo | None:
    """Get tool info without loading."""
    return self._registry.get_tool_info(name)
def get_all_loaded(self) -> list[Tool]:
    """Get all loaded tools."""
    return list(self.loaded_tools.values())
def get_all_deferred_info(self) -> list[DeferredTool]:
    """Get info for all deferred tools."""
    return list(self.deferred_tools.values())
```

```
def search(self, query: str) -> list[DeferredTool]:
    """Search deferred tools."""
    return self._registry.search(query)
def get_stats(self) -> LoaderStats:
    """Get loading statistics."""
    return self._stats
def get_loaded_names(self) -> set[str]:
    """Get names of loaded tools."""
    return set(self.loaded_tools.keys())
def get_deferred_names(self) -> set[str]:
    """Get names of deferred tools."""
    return set(self.deferred_tools.keys())
def clear_cache(self, name: str | None = None) -> None:
    """Clear loaded tool cache."""
    if name:
        if name in self.loaded_tools:
            del self.loaded_tools[name]
            if name in self.deferred_tools:
                self.deferred_tools[name].is_loaded = False
            self._stats.loaded_tools -= 1
    else:
        self.loaded_tools.clear()
        for deferred in self.deferred_tools.values():
            deferred.is_loaded = False
        self._stats.loaded_tools = len(self.always_load)
def to_dict(self) -> dict[str, Any]:
    """Export loader state."""
    return {
        "loaded": list(self.loaded_tools.keys()),
        "deferred": list(self.deferred_tools.keys()),
        "always_load": list(self.always_load),
        "stats": {
            "total_tools": self._stats.total_tools,
            "loaded_tools": self._stats.loaded_tools,
            "deferred_tools": self._stats.deferred_tools,
            "load_count": self._stats.load_count,
            "cache_hits": self._stats.cache_hits,
        },
    }
```

File: [152/252] openlaoke/core/tools/tool_discovery.py

```
"""Tool discovery system for automatic tool detection."""
from __future__ import annotations
import importlib
import inspect
from collections.abc import Callable
from dataclasses import dataclass
from pathlib import Path
from typing import TYPE_CHECKING, Any
from openlaoke.core.tools.deferred_registry import DeferredRegistry
if TYPE_CHECKING:
    from openlaoke.core.tool import Tool
@dataclass
class DiscoveryResult:
    """Result of tool discovery."""
    discovered: list[str]
    loaded: list[str]
    failed: list[str]
    total: int
class ToolDiscovery:
    """Automatic tool discovery from modules."""
    def __init__(self, registry: DeferredRegistry | None = None) -> None:
        self._registry = registry or DeferredRegistry()
        self._tool_modules: list[str] = []
        self._discovered: dict[str, str] = {}
    def add_module_path(self, module_path: str) -> None:
        """Add a module path to scan for tools."""
        self._tool_modules.append(module_path)
    def add_directory(self, directory: Path) -> None:
        """Add a directory to scan for tool modules."""
        if directory.exists():
            for py_file in directory.glob("*.py"):
                if py_file.name.startswith("_"):
                    continue
                module_name = py_file.stem
                full_path = f"{directory.parent.name}.{module_name}"
                self._tool_modules.append(full_path)
    def discover_tools(self) -> DiscoveryResult:
        """Discover all tools from registered modules."""
        result = DiscoveryResult(discovered=[], loaded=[], failed=[], total=0)
        for module_path in self._tool_modules:
            try:
                module = importlib.import_module(module_path)
                self._scan_module(module, result)
            except ImportError as e:
                result.failed.append(f"{module_path}: {e}")
        result.total = len(result.discovered)
        return result
    def _scan_module(self, module: Any, result: DiscoveryResult) -> None:
        """Scan a module for tool classes."""
        for name, obj in inspect.getmembers(module):
            if inspect.isclass(obj) and self._is_tool_class(obj):
                tool_name = getattr(obj, "name", name)
                description = getattr(obj, "description", "")
                search_hint = getattr(obj, "search_hint", "")
                aliases = getattr(obj, "aliases", [])
                defer_loading = getattr(obj, "defer_loading", False)
                always_load = getattr(obj, "always_load", False)
                self._discovered[tool_name] = module.__name__
                if defer_loading and not always_load:
                    result.discovered.append(tool_name)
                    tool_class = obj
```

```
                self._registry.register(
                    name=tool_name,
                    description=description,
                    loader=lambda cls=tool_class: cls(),
                    always_load=always_load,
                    search_hint=search_hint,
                    aliases=list(aliases) if aliases else [],
                )
            else:
                result.loaded.append(tool_name)
def _is_tool_class(self, obj: Any) -> bool:
    """Check if a class is a tool."""
    from openlaoke.core.tool import Tool
    try:
        return (
            inspect.isclass(obj)
            and issubclass(obj, Tool)
            and obj is not Tool
            and not inspect.isabstract(obj)
        )
    except TypeError:
        return False
def get_discovered_tools(self) -> dict[str, str]:
    """Get all discovered tools and their modules."""
    return self._discovered.copy()
def get_registry(self) -> DeferredRegistry:
    """Get the deferred registry."""
    return self._registry
def auto_register_from_tools_dir(self, base_package: str = "openlaoke.tools") -> None:
    """Auto-register tools from openlaoke.tools directory."""
    try:
        tools_package = importlib.import_module(base_package)
        package_path = (
            Path(tools_package.__file__).parent
            if tools_package.__file__
            else Path("openlaoke/tools")
        )
        self.add_directory(package_path)
    except ImportError:
        pass
def create_loader_function(self, module_path: str, class_name: str) -> Callable[[], Tool]:
    """Create a loader function for a tool."""
    def loader() -> Tool:
        module = importlib.import_module(module_path)
        cls = getattr(module, class_name)
        instance: Tool = cls()
        return instance
    return loader
def to_dict(self) -> dict[str, Any]:
    """Export discovery state."""
    return {
        "modules": self._tool_modules,
        "discovered": self._discovered,
    }
```

File: [153/252] openlaoke/core/tools/tool_search.py

```
"""Tool search tool for discovering and loading deferred tools."""
from __future__ import annotations
import re
from dataclasses import dataclass
from typing import TYPE_CHECKING, Any
from pydantic import BaseModel
from openlaoke.core.tool import Tool, ToolContext
from openlaoke.types.core_types import ToolResultBlock
if TYPE_CHECKING:
    from openlaoke.core.tools.lazy_loader import LazyToolLoader
@dataclass
class ToolMatch:
    """A matched tool from search."""
    name: str
    description: str
    search_hint: str = ""
    score: float = 0.0
    is_loaded: bool = False
@dataclass
class ToolSuggestion:
    """A suggested tool based on context."""
    name: str
    reason: str
    confidence: float = 0.0
class ToolSearchInput(BaseModel):
    """Input schema for ToolSearch tool."""
    query: str
    max_results: int = 5
class ToolSearchTool(Tool):
    """Tool search tool - search and load available tools."""
    name = "ToolSearch"
    description = "Search for deferred tools and load them on demand"
    input_schema = ToolSearchInput
    defer_loading = True
    def __init__(self, lazy_loader: LazyToolLoader | None = None) -> None:
        super().__init__()
        self._lazy_loader = lazy_loader
    def set_lazy_loader(self, loader: LazyToolLoader) -> None:
        """Set the lazy loader instance."""
        self._lazy_loader = loader
    def _parse_tool_name(self, name: str) -> dict[str, Any]:
        """Parse tool name into searchable parts."""
        if name.startswith("mcp__"):
            without_prefix = name.replace("mcp__", "").lower()
            parts: list[str] = []
            for segment in without_prefix.split("__"):
                parts.extend(segment.split("_"))
            return {
                "parts": [p for p in parts if p],
                "full": without_prefix.replace("__", " ").replace("_", " "),
                "is_mcp": True,
            }
        camel_split = re.sub(r"([a-z])([A-Z])", r"\1 \2", name)
        space_split = camel_split.replace("_", " ").lower()
        parts = space_split.split()
        return {
            "parts": [p for p in parts if p],
            "full": " ".join(parts),
            "is_mcp": False,
        }
    async def search(self, query: str, max_results: int = 5) -> list[ToolMatch]:
```

```
    """Search matching tools."""
    if not self._lazy_loader:
        return []
    query_lower = query.lower().strip()
    deferred = self._lazy_loader.get_all_deferred_info()
    exact_match = None
    for t in deferred:
        if t.name.lower() == query_lower:
            exact_match = ToolMatch(
                name=t.name,
                description=t.description,
                search_hint=t.search_hint,
                score=100.0,
                is_loaded=t.is_loaded,
            )
            return [exact_match]
    results: list[ToolMatch] = []
    query_terms = query_lower.split()
    for tool in deferred:
        score = 0.0
        name_lower = tool.name.lower()
        desc_lower = tool.description.lower()
        hint_lower = tool.search_hint.lower()
        for term in query_terms:
            if term in name_lower:
                score += 10.0
            if term in hint_lower:
                score += 8.0
            if term in desc_lower:
                score += 5.0
            for alias in tool.aliases:
                if term in alias.lower():
                    score += 7.0
        if score > 0:
            results.append(
                ToolMatch(
                    name=tool.name,
                    description=tool.description,
                    search_hint=tool.search_hint,
                    score=score,
                    is_loaded=tool.is_loaded,
                )
            )
    return sorted(results, key=lambda m: m.score, reverse=True)[:max_results]
async def suggest(self, context: str) -> list[ToolSuggestion]:
    """Suggest tools based on context."""
    if not self._lazy_loader:
        return []
    suggestions: list[ToolSuggestion] = []
    context_lower = context.lower()
    keywords_map: dict[str, list[str]] = {
        "file": ["Read", "Write", "Edit", "Glob", "Grep"],
        "read": ["Read"],
        "write": ["Write"],
        "edit": ["Edit"],
        "search": ["Grep", "Glob", "ToolSearch"],
        "web": ["WebSearch", "WebFetch"],
        "fetch": ["WebFetch"],
        "git": ["Git", "Bash"],
        "bash": ["Bash"],
        "command": ["Bash"],
        "plan": ["Plan"],
        "notebook": ["NotebookEdit"],
        "jupyter": ["NotebookEdit"],
        "agent": ["Agent"],
```

```
        "task": ["TaskKill", "Agent"],
        "todo": ["Todo"],
        "ask": ["Question"],
        "ls": ["LS", "Glob"],
        "patch": ["ApplyPatch"],
        "batch": ["Batch"],
    }
    for keyword, tool_names in keywords_map.items():
        if keyword in context_lower:
            for name in tool_names:
                deferred = self._lazy_loader.deferred_tools.get(name)
                if deferred:
                    suggestions.append(
                        ToolSuggestion(
                            name=name,
                            reason=f"Keyword '{keyword}' detected",
                            confidence=0.7,
                        )
                    )
    return sorted(suggestions, key=lambda s: s.confidence, reverse=True)[:5]
async def load_tool(self, name: str) -> bool:
    """Load a specific tool."""
    if not self._lazy_loader:
        return False
    tool = await self._lazy_loader.get(name)
    return tool is not None
async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
    """Execute tool search."""
    query = kwargs.get("query", "")
    max_results = kwargs.get("max_results", 5)
    matches = await self.search(query, max_results)
    if not matches:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="No matching deferred tools found",
            is_error=False,
        )
    result_lines: list[str] = []
    for match in matches:
        status = "loaded" if match.is_loaded else "deferred"
        result_lines.append(f"- {match.name} ({status}): {match.description[:50]}...")
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content="\n".join(result_lines),
        is_error=False,
    )
def get_description(self) -> str:
    return self.description
def get_input_schema(self) -> dict[str, Any]:
    return self.input_schema.model_json_schema()
```

File: [154/252] openlaoke/core/translation.py

```
"""Universal language translation for intent processing.
Translates all non-English requests to English for processing,
then translates results back to the original language.
"""
from __future__ import annotations
import re
from dataclasses import dataclass
from enum import StrEnum
class Language(StrEnum):
    ENGLISH = "en"
    CHINESE = "zh"
    JAPANESE = "ja"
    KOREAN = "ko"
    FRENCH = "fr"
    GERMAN = "de"
    SPANISH = "es"
    RUSSIAN = "ru"
    PORTUGUESE = "pt"
    ITALIAN = "it"
@dataclass
class TranslationResult:
    original: str
    translated: str
    source_language: Language
    target_language: Language
    confidence: float = 1.0
class LanguageDetector:
    """Detect language of input text."""
    LANGUAGE_PATTERNS = {
        Language.CHINESE: [
            r"[\u4e00-\u9fff]",  # CJK Unified Ideographs
            r"?|??|??|??|??|??",
        ],
        Language.JAPANESE: [
            r"[\u3040-\u309f]",  # Hiragana
            r"[\u30a0-\u30ff]",  # Katakana
        ],
        Language.KOREAN: [
            r"[\uac00-\ud7af]",  # Hangul
        ],
        Language.RUSSIAN: [
            r"[?-??-???]",
        ],
        Language.FRENCH: [
            r"\b(le|la|les|un|une|des|et|ou|pour|avec|dans|sur)\b",
        ],
        Language.GERMAN: [
            r"\b(der|die|das|und|oder|f?r|mit|auf|ist|sind)\b",
        ],
        Language.SPANISH: [
            r"\b(el|la|los|las|un|una|y|o|para|con|en)\b",
        ],
        Language.PORTUGUESE: [
            r"\b(o|a|os|as|um|uma|e|ou|para|com|em)\b",
        ],
        Language.ITALIAN: [
            r"\b(il|lo|la|i|gli|le|un|una|e|o|per|con|in)\b",
        ],
    }
    @classmethod
    def detect(cls, text: str) -> Language:
```

```
        """Detect the language of input text."""
        if not text or not text.strip():
            return Language.ENGLISH
        text.lower()
        if re.search(r"[\u4e00-\u9fff]", text):
            return Language.CHINESE
        if re.search(r"[\u3040-\u309f\u30a0-\u30ff]", text):
            return Language.JAPANESE
        if re.search(r"[\uac00-\ud7af]", text):
            return Language.KOREAN
        if re.search(r"[?-??-???]", text):
            return Language.RUSSIAN
        return Language.ENGLISH
class UniversalTranslator:
    """Translate between languages using simple pattern matching."""
    COMMON_PROGRAMMING_TERMS = {
        Language.CHINESE: {
            "?": "write",
            "??": "create",
            "??": "write",
            "??": "implement",
            "??": "program",
            "??": "application",
            "??": "script",
            "??": "function",
            "??": "method",
            "?": "class",
            "??": "debug",
            "??": "fix",
            "??": "test",
            "??": "analyze",
            "??": "optimize",
            "???": "single file",
            "??": "calculate",
            "???": "processor",
            "??": "core",
            "??": "performance",
            "??": "real",
            "??": "estimate",
            "??": "must not",
            "??": "must",
            "??": "a",
            "??": "current",
            "??": "device",
            "???": "multi-core",
            "???": "single-core",
            "??": "performance",
            "????": "benchmark",
            "????": "quicksort",
            "??": "sort",
            "??": "search",
            "??": "find",
            "??": "error",
            "??": "code",
            "??": "this",
            "??": "file",
            "??": "interface",
            "API": "API",
            "REST": "REST",
            "???": "database",
            "??": "connect",
            "??": "query",
            "??": "result",
            "??": "display",
            "??": "output",
```

```
        "??": "input",
        "??": "process",
        "??": "data",
        "???": "string",
        "??": "list",
        "??": "all",
        "??": "directory",
        "??": "contain",
        "??": "operation",
        "??": "get",
        "??": "set",
        "??": "config",
        "???": "initialize",
        "??": "load",
        "??": "uninstall",
        "??": "start",
        "??": "stop",
        "??": "restart",
        "??": "install",
        "??": "update",
        "??": "version",
        "??": "log",
        "??": "exception",
        "??": "warning",
        "??": "info",
    },
    Language.JAPANESE: {
        "?": "write",
        "??": "create",
        "??": "implement",
        "?????": "program",
        "???": "application",
        "??": "function",
        "???": "class",
        "???": "test",
    },
    Language.KOREAN: {
        "?": "write",
        "??": "create",
        "??": "implement",
        "????": "program",
        "??": "function",
        "???": "class",
    },
}
@classmethod
def translate_to_english(cls, text: str, source_lang: Language) -> str:
    """Translate non-English text to English."""
    if source_lang == Language.ENGLISH:
        return text
    if source_lang not in cls.COMMON_PROGRAMMING_TERMS:
        return text
    terms = cls.COMMON_PROGRAMMING_TERMS[source_lang]
    result = text
    sorted_terms = sorted(terms.items(), key=lambda x: len(x[0]), reverse=True)
    for native, english in sorted_terms:
        result = result.replace(native, f" {english} ")
    result = re.sub(r"\s+", " ", result).strip()
    result = re.sub(r"\s+([,.!?;:])", r"\1", result)
    result = re.sub(r"([,.!?;:])\s*", r"\1 ", result)
    return result
@classmethod
def translate_from_english(cls, text: str, target_lang: Language) -> str:
    """Translate English text back to target language."""
    if target_lang == Language.ENGLISH:
```

```
            return text
        if target_lang not in cls.COMMON_PROGRAMMING_TERMS:
            return text
        terms = cls.COMMON_PROGRAMMING_TERMS[target_lang]
        reverse_terms = {v: k for k, v in terms.items()}
        result = text
        sorted_terms = sorted(reverse_terms.items(), key=lambda x: len(x[0]), reverse=True)
        for english, native in sorted_terms:
            result = re.sub(r"\b" + re.escape(english) + r"\b", native, result, flags=re.IGNORE
    CASE)
        return result
class TranslationPipeline:
    """Complete translation pipeline for multilingual support."""
    def __init__(self) -> None:
        self.detector = LanguageDetector()
        self.translator = UniversalTranslator()
    def prepare_for_processing(self, user_input: str) -> tuple[str, Language]:
        """Translate input to English for processing.
        Returns:
            (english_text, original_language)
        """
        source_lang = self.detector.detect(user_input)
        if source_lang == Language.ENGLISH:
            return user_input, source_lang
        english_text = self.translator.translate_to_english(user_input, source_lang)
        return english_text, source_lang
    def prepare_for_output(
        self, output_text: str, target_lang: Language, is_code: bool = False
    ) -> str:
        """Translate output back to user's language.
        Args:
            output_text: Text to translate
            target_lang: Target language
            is_code: If True, don't translate (code is universal)
        """
        if is_code or target_lang == Language.ENGLISH:
            return output_text
        return self.translator.translate_from_english(output_text, target_lang)
    def translate_code_comments(self, code: str, target_lang: Language) -> str:
        """Translate code comments to target language.
        Preserves code structure, only translates comments.
        """
        if target_lang == Language.ENGLISH:
            return code
        lines = code.split("\n")
        translated_lines = []
        for line in lines:
            stripped = line.lstrip()
            if stripped.startswith("#"):
                indent = line[: len(line) - len(stripped)]
                comment = stripped[1:].strip()
                translated = self.translator.translate_from_english(comment, target_lang)
                translated_lines.append(f"{indent}# {translated}")
            elif '"""' in line or "'''" in line:
                translated_lines.append(line)
            else:
                translated_lines.append(line)
        return "\n".join(translated_lines)
def create_translation_pipeline() -> TranslationPipeline:
    """Create a translation pipeline instance."""
    return TranslationPipeline()
def translate_request_to_english(user_input: str) -> tuple[str, Language]:
    """Convenience function to translate request to English.
    Returns:
        (english_text, original_language)
```

```
    """
    pipeline = TranslationPipeline()
    return pipeline.prepare_for_processing(user_input)
def translate_output_to_language(output: str, target_lang: Language, is_code: bool = False) ->
    str:
    """Convenience function to translate output."""
    pipeline = TranslationPipeline()
    return pipeline.prepare_for_output(output, target_lang, is_code)
```

File: [155/252] openlaoke/entrypoints/__init__.py

```
"""Entry points for OpenLaoKe."""
```

File: [156/252] openlaoke/entrypoints/cli.py

```
"""CLI entry point for OpenLaoKe."""
from __future__ import annotations
import argparse
import asyncio
import os
import sys
from rich.console import Console
from openlaoke import __version__
from openlaoke.core.config_wizard import (
    get_proxy_url,
    run_config_wizard,
    show_current_config,
)
from openlaoke.core.multi_provider_api import MultiProviderClient
from openlaoke.core.repl import REPL
from openlaoke.core.state import create_app_state
from openlaoke.utils.config import load_config, save_config
def main() -> None:
    parser = argparse.ArgumentParser(
        prog="openlaoke",
        description="OpenLaoKe - Open-source AI coding assistant",
    )
    parser.add_argument(
        "-v",
        "--version",
        action="version",
        version=f"OpenLaoKe {__version__}",
    )
    subparsers = parser.add_subparsers(dest="command", help="Available commands")
    # Auth command
    auth_parser = subparsers.add_parser("auth", help="Authentication management")
    auth_parser.add_argument("provider", nargs="?", help="Provider to authenticate")
    auth_parser.add_argument("--list", action="store_true", help="List authenticated providers"
    )
    auth_parser.add_argument("--remove", metavar="PROVIDER", help="Remove authentication")
    auth_parser.add_argument("--show", metavar="PROVIDER", help="Show auth details")
    auth_parser.add_argument("--test", metavar="PROVIDER", help="Test authentication")
    server_parser = subparsers.add_parser("server", help="Start HTTP API server")
    server_parser.add_argument(
        "--host",
        default="localhost",
        help="Server host (default: localhost)",
    )
    server_parser.add_argument(
        "--port",
        type=int,
        default=3000,
        help="Server port (default: 3000)",
    )
    server_parser.add_argument(
        "--cors",
        nargs="*",
        default=None,
        help="Additional CORS origins",
    )
    parser.add_argument(
        "-m",
        "--model",
        default=None,
        help="Model to use (e.g., gpt-4o, llama3.2, gemma3:1b)",
    )
```

```
parser.add_argument(
    "-p",
    "--permission",
    choices=["default", "auto", "bypass"],
    default=None,
    help="Permission mode (auto/bypass = no confirmations)",
)
parser.add_argument(
    "-y",
    "--yes",
    action="store_true",
    help="Auto-approve all tool calls (same as --permission auto)",
)
parser.add_argument(
    "-c",
    "--cwd",
    default=None,
    help="Working directory",
)
parser.add_argument(
    "--provider",
    choices=[
        "anthropic",
        "openai",
        "minimax",
        "aliyun_coding_plan",
        "ollama",
        "lm_studio",
        "openai_compatible",
    ],
    default=None,
    help="AI provider to use",
)
parser.add_argument(
    "--api-key",
    default=None,
    help="API key for the provider",
)
parser.add_argument(
    "--base-url",
    default=None,
    help="API base URL (for local/custom providers)",
)
parser.add_argument(
    "--proxy",
    default=None,
    help="Proxy URL (e.g., http://127.0.0.1:7890)",
)
parser.add_argument(
    "--no-proxy",
    action="store_true",
    help="Disable proxy",
)
parser.add_argument(
    "--max-tokens",
    type=int,
    default=None,
    help="Maximum tokens in response",
)
parser.add_argument(
    "--thinking-budget",
    type=int,
    default=None,
    help="Thinking budget tokens",
)
```

```
parser.add_argument(
    "--verbose",
    action="store_true",
    help="Enable verbose output",
)
parser.add_argument(
    "--config",
    action="store_true",
    help="Run configuration wizard",
)
parser.add_argument(
    "--show-config",
    action="store_true",
    help="Show current configuration",
)
parser.add_argument(
    "--local",
    action="store_true",
    help="Enable local mode (for small local models with atomic decomposition, multi-model
coordination)",
)
parser.add_argument(
    "prompt",
    nargs="*",
    help="Direct prompt to execute (non-interactive mode)",
)
args = parser.parse_args()
console = Console(force_terminal=True)
if args.command == "server":
    from openlaoke.server import Server
    server = Server(
        host=args.host,
        port=args.port,
        cors_origins=args.cors,
    )
    console.print(f"[green]Starting OpenLaoKe server on {args.host}:{args.port}[/green]")
    server.run()
    return
if args.command == "auth":
    from openlaoke.core.extended_web import BrowserAuthManager
    from openlaoke.core.extended_web.types import WEB_PROVIDERS
    auth_manager = BrowserAuthManager()
    if args.list:
        auths = auth_manager.list_saved_auths()
        if not auths:
            console.print("[yellow]No authenticated providers.[/yellow]")
        else:
            console.print(f"[bold]Authenticated providers ({len(auths)})[/bold]")
            for provider_type in auths:
                console.print(f"  [OK] {provider_type}")
        return
    if args.remove:
        if auth_manager.delete_auth(args.remove):
            console.print(f"[green]Removed auth for {args.remove}[/green]")
        else:
            console.print(f"[yellow]No auth found for {args.remove}[/yellow]")
        return
    if args.show:
        auth_data = auth_manager.load_auth(args.show)
        if not auth_data:
            console.print(f"[yellow]No auth for {args.show}[/yellow]")
        else:
            cookie = auth_data.get("cookie", "")
            console.print(f"[bold]{args.show}[/bold]")
            console.print(
```

```
                f"  Cookie: {cookie[:20]}...{cookie[-20:] if len(cookie) > 40 else ''}"
            )
            console.print(f"  Length: {len(cookie)}")
        return
    if args.test:
        console.print(f"[dim]Testing {args.test}...[/dim]")
        auth_data = auth_manager.load_auth(args.test)
        if not auth_data:
            console.print(f"[red]No auth for {args.test}[/red]")
            return
        console.print(
            f"[green][OK] Auth loaded (cookie length: {len(auth_data.get('cookie', ''))})[/
green]"
        )
        return
    if args.provider:
        console.print(f"[dim]Authenticating {args.provider}...[/dim]")
        console.print("[yellow]Note: Use browser-based auth command for full flow.[/yellow]
")
    else:
        console.print(
            "[dim]Use 'openlaoke auth extended-web login <provider>' for browser auth.[/dim
]"
        )
    return
config = load_config()
if args.show_config:
    show_current_config(config)
    return
if args.config or config.first_run or not config.providers.is_configured():
    config = run_config_wizard(config)
    save_config(config)
    if not args.config:
        console.print("\n[dim]Run 'openlaoke' to start chatting.[/dim]")
        return
# Apply command-line overrides
if args.provider:
    if args.provider in config.providers.providers:
        config.providers.active_provider = args.provider
    else:
        console.print(f"[red]Unknown provider: {args.provider}[/red]")
        return
if args.api_key:
    provider = config.providers.get_active_provider()
    if provider:
        provider.api_key = args.api_key
if args.base_url:
    provider = config.providers.get_active_provider()
    if provider:
        provider.base_url = args.base_url
if args.model:
    config.providers.active_model = args.model
if args.no_proxy:
    config.proxy_mode = "none"
    config.proxy_url = ""
elif args.proxy:
    config.proxy_mode = "custom"
    config.proxy_url = args.proxy
cwd = args.cwd or os.getcwd()
persist_dir = os.path.expanduser("~/.openlaoke/sessions")
os.makedirs(persist_dir, exist_ok=True)
persist_path = os.path.join(persist_dir, f"session_{int(__import__('time').time())}.json")
app_state = create_app_state(
    cwd=cwd,
    model=config.providers.get_active_model(),
```

```
        persist_path=persist_path,
    )
    app_state.session_config.model = config.providers.get_active_model()
    if config.auto_approve_all:
        from openlaoke.types.core_types import PermissionMode
        app_state.permission_config.mode = PermissionMode.AUTO
        app_state.auto_accept = True
    if args.max_tokens:
        config.max_tokens = args.max_tokens
    if args.thinking_budget:
        config.thinking_budget = args.thinking_budget
    if args.permission:
        from openlaoke.types.core_types import PermissionMode
        app_state.permission_config.mode = PermissionMode(args.permission)
    if args.yes:
        from openlaoke.types.core_types import PermissionMode
        app_state.permission_config.mode = PermissionMode.AUTO
        app_state.auto_accept = True
    if args.verbose:
        app_state.verbose = True
    if args.local:
        app_state.local_mode = True
    app_state.multi_provider_config = config.providers
    app_state.app_config = config
    if args.prompt:
        prompt_text = " ".join(args.prompt)
        asyncio.run(_run_non_interactive(prompt_text, app_state, config))
    else:
        asyncio.run(_run_interactive(app_state, config))
async def _run_interactive(app_state, config) -> None:
    repl = REPL(app_state)
    repl.app_config = config
    await repl.run()
async def _run_non_interactive(prompt: str, app_state, config) -> None:
    from openlaoke.core.system_prompt import build_system_prompt
    from openlaoke.core.tool import ToolRegistry
    from openlaoke.tools.register import register_all_tools
    from openlaoke.types.core_types import MessageRole, UserMessage
    console = Console()
    registry = ToolRegistry()
    register_all_tools(registry)
    proxy_url = get_proxy_url(config)
    api = MultiProviderClient(config.providers, proxy=proxy_url)
    user_msg = UserMessage(role=MessageRole.USER, content=prompt)
    app_state.add_message(user_msg)
    system_prompt = build_system_prompt(app_state, registry.get_all_for_prompt())
    messages = [{"role": "user", "content": prompt}]
    tools = registry.get_all_for_prompt()
    max_iterations = 50
    iteration = 0
    try:
        while iteration < max_iterations:
            iteration += 1
            response, usage, cost = await api.send_message(
                system_prompt=system_prompt,
                messages=messages,
                tools=tools if iteration < max_iterations - 1 else None,
                max_tokens=config.max_tokens,
                temperature=config.temperature,
            )
            app_state.accumulate_tokens(usage)
            app_state.accumulate_cost(cost)
            if response.content:
                print(response.content)
            if not response.tool_uses:
```

```
                break
            for tool_use in response.tool_uses:
                tool = registry.get(tool_use.name)
                if not tool:
                    messages.append(
                        {
                            "role": "tool_result",
                            "tool_use_id": tool_use.id,
                            "content": f"Unknown tool: {tool_use.name}",
                        }
                    )
                    continue
                from openlaoke.core.tool import ToolContext
                ctx = ToolContext(app_state=app_state, tool_use_id=tool_use.id)
                validation = tool.validate_input(tool_use.input)
                if not validation.result:
                    messages.append(
                        {
                            "role": "tool_result",
                            "tool_use_id": tool_use.id,
                            "content": f"Validation error: {validation.message}",
                        }
                    )
                    continue
                result = await tool.call(ctx, **tool_use.input)
                messages.append(
                    {
                        "role": "tool_result",
                        "tool_use_id": tool_use.id,
                        "content": result.content
                        if isinstance(result.content, str)
                        else str(result.content),
                    }
                )
    except Exception as e:
        console.print(f"\n[bold red]Error:[/bold red] {e}")
        sys.exit(1)
    finally:
        await api.close()
    print(f"\n--- Session cost: ${app_state.cost_info.total_cost:.4f} ---")
if __name__ == "__main__":
    main()
```

File: [157/252] openlaoke/server/__init__.py

```
"""HTTP API server module."""
from __future__ import annotations
from openlaoke.server.server import Server
__all__ = ["Server"]
```

File: [158/252] openlaoke/server/server.py

```
"""FastAPI-based HTTP API server with WebSocket support."""
from __future__ import annotations
import asyncio
import contextlib
import json
import os
import time
import uuid
from collections.abc import AsyncIterator
from dataclasses import dataclass, field
from typing import Any
from fastapi import FastAPI, HTTPException, Query, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from openlaoke.core.config_wizard import get_proxy_url
from openlaoke.core.multi_provider_api import MultiProviderClient
from openlaoke.core.sessions import SessionManager
from openlaoke.core.state import AppState, create_app_state
from openlaoke.core.system_prompt import build_system_prompt
from openlaoke.core.tool import ToolContext, ToolRegistry
from openlaoke.tools.register import register_all_tools
from openlaoke.types.core_types import (
    AssistantMessage,
    CostInfo,
    MessageRole,
    TokenUsage,
    UserMessage,
)
from openlaoke.utils.config import AppConfig, load_config, save_config
class ChatRequest(BaseModel):
    content: str = ""
    session_id: str | None = None
    stream: bool = True
class ChatResponse(BaseModel):
    session_id: str
    content: str = ""
    tool_uses: list[dict[str, Any]] = []
    usage: dict[str, int] = {}
    cost: float = 0.0
class ToolRequest(BaseModel):
    name: str
    input: dict[str, Any] = {}
    session_id: str | None = None
class ToolResponse(BaseModel):
    tool_use_id: str
    content: str | list[dict[str, Any]]
    is_error: bool = False
class SessionCreateRequest(BaseModel):
    cwd: str | None = None
    model: str | None = None
class SessionResponse(BaseModel):
    session_id: str
    cwd: str
    message_count: int
    task_count: int
    model: str
    created_at: float
class ConfigUpdateRequest(BaseModel):
    max_tokens: int | None = None
    temperature: float | None = None
```

```
    thinking_budget: int | None = None
    permission_mode: str | None = None
    theme: str | None = None
class ModelInfo(BaseModel):
    id: str
    name: str
    provider: str
    enabled: bool = True
class ToolInfo(BaseModel):
    name: str
    description: str
    input_schema: dict[str, Any]
    is_read_only: bool = False
    is_destructive: bool = False
@dataclass
class ActiveSession:
    app_state: AppState
    registry: ToolRegistry
    api: MultiProviderClient
    config: AppConfig
    created_at: float = field(default_factory=time.time)
class ConnectionManager:
    def __init__(self) -> None:
        self.active_connections: list[WebSocket] = []
        self._lock = asyncio.Lock()
    async def connect(self, websocket: WebSocket) -> None:
        await websocket.accept()
        async with self._lock:
            self.active_connections.append(websocket)
    async def disconnect(self, websocket: WebSocket) -> None:
        async with self._lock:
            if websocket in self.active_connections:
                self.active_connections.remove(websocket)
    async def broadcast(self, message: dict[str, Any]) -> None:
        async with self._lock:
            for connection in self.active_connections:
                with contextlib.suppress(Exception):
                    await connection.send_json(message)
    async def send_personal(self, websocket: WebSocket, message: dict[str, Any]) -> None:
        with contextlib.suppress(Exception):
            await websocket.send_json(message)
class Server:
    def __init__(
        self,
        host: str = "localhost",
        port: int = 3000,
        cors_origins: list[str] | None = None,
    ) -> None:
        self.host = host
        self.port = port
        self.cors_origins = cors_origins or [
            "http://localhost",
            "http://localhost:3000",
            "http://localhost:8080",
            "http://127.0.0.1",
            "http://127.0.0.1:3000",
            "http://127.0.0.1:8080",
        ]
        self.app = FastAPI(
            title="OpenLaoKe API",
            description="Open-source AI coding assistant API",
            version="1.0.0",
        )
        self.manager = ConnectionManager()
        self.sessions: dict[str, ActiveSession] = {}
```

```
    self.session_manager = SessionManager()
    self._setup_middleware()
    self._setup_routes()
def _setup_middleware(self) -> None:
    self.app.add_middleware(
        CORSMiddleware,
        allow_origins=self.cors_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
def _setup_routes(self) -> None:
    app = self.app
    @app.get("/health")
    async def health_check() -> dict[str, str]:
        return {"status": "ok", "timestamp": str(time.time())}
    @app.post("/api/chat")
    async def chat(request: ChatRequest) -> StreamingResponse | ChatResponse:
        session = await self._get_or_create_session(request.session_id)
        session_id = session.app_state.session_id
        user_msg = UserMessage(role=MessageRole.USER, content=request.content)
        session.app_state.add_message(user_msg)
        if request.stream:
            return StreamingResponse(
                self._stream_chat(session, request.content),
                media_type="text/event-stream",
            )
        response, usage, cost = await self._send_message(session, request.content)
        return ChatResponse(
            session_id=session_id,
            content=response.content,
            tool_uses=[tu.to_dict() for tu in response.tool_uses],
            usage={
                "input_tokens": usage.input_tokens,
                "output_tokens": usage.output_tokens,
            },
            cost=cost.total_cost,
        )
    @app.post("/api/tool")
    async def execute_tool(request: ToolRequest) -> ToolResponse:
        session = await self._get_or_create_session(request.session_id)
        tool = session.registry.get(request.name)
        if not tool:
            raise HTTPException(status_code=404, detail=f"Tool not found: {request.name}")
        validation = tool.validate_input(request.input)
        if not validation.result:
            raise HTTPException(status_code=400, detail=validation.message)
        tool_use_id = f"tool_{uuid.uuid4().hex[:8]}"
        ctx = ToolContext(app_state=session.app_state, tool_use_id=tool_use_id)
        result = await tool.call(ctx, **request.input)
        return ToolResponse(
            tool_use_id=tool_use_id,
            content=result.content if isinstance(result.content, str) else result.content,
            is_error=result.is_error,
        )
    @app.get("/api/session")
    async def list_sessions(
        limit: int = Query(default=50, ge=1, le=100),
    ) -> list[SessionResponse]:
        sessions = self.session_manager.list_sessions()[:limit]
        return [
            SessionResponse(
                session_id=s.session_id,
                cwd=s.cwd,
                message_count=s.message_count,
```

```
            task_count=s.task_count,
            model=s.model,
            created_at=s.created_at,
        )
        for s in sessions
    ]
@app.post("/api/session")
async def create_session(request: SessionCreateRequest) -> SessionResponse:
    config = load_config()
    cwd = request.cwd or os.getcwd()
    model = request.model or config.get_active_model()
    persist_dir = os.path.expanduser("~/.openlaoke/sessions")
    os.makedirs(persist_dir, exist_ok=True)
    persist_path = os.path.join(persist_dir, f"session_{int(time.time())}.json")
    app_state = create_app_state(cwd=cwd, model=model, persist_path=persist_path)
    app_state.session_config.model = model
    app_state.multi_provider_config = config.providers
    app_state.app_config = config
    registry = ToolRegistry()
    register_all_tools(registry)
    proxy_url = get_proxy_url(config)
    api = MultiProviderClient(config.providers, proxy=proxy_url)
    session_id = app_state.session_id
    self.sessions[session_id] = ActiveSession(
        app_state=app_state,
        registry=registry,
        api=api,
        config=config,
    )
    self.session_manager.save_session(app_state)
    return SessionResponse(
        session_id=session_id,
        cwd=cwd,
        message_count=0,
        task_count=0,
        model=model,
        created_at=time.time(),
    )
@app.get("/api/session/{session_id}")
async def get_session(session_id: str) -> SessionResponse:
    if session_id in self.sessions:
        s = self.sessions[session_id]
        return SessionResponse(
            session_id=session_id,
            cwd=s.app_state.cwd,
            message_count=len(s.app_state.messages),
            task_count=len(s.app_state.tasks),
            model=s.app_state.session_config.model,
            created_at=s.created_at,
        )
    app_state = self.session_manager.load_session(session_id)
    if not app_state:
        raise HTTPException(status_code=404, detail="Session not found")
    return SessionResponse(
        session_id=session_id,
        cwd=app_state.cwd,
        message_count=len(app_state.messages),
        task_count=len(app_state.tasks),
        model=app_state.session_config.model,
        created_at=0,
    )
@app.delete("/api/session/{session_id}")
async def delete_session(session_id: str) -> dict[str, bool]:
    if session_id in self.sessions:
        session = self.sessions[session_id]
```

```
        await session.api.close()
        del self.sessions[session_id]
    deleted = self.session_manager.delete_session(session_id)
    if not deleted:
        raise HTTPException(status_code=404, detail="Session not found")
    return {"deleted": True}
@app.get("/api/config")
async def get_config() -> dict[str, Any]:
    config = load_config()
    return {
        "max_tokens": config.max_tokens,
        "temperature": config.temperature,
        "thinking_budget": config.thinking_budget,
        "permission_mode": config.permission_mode,
        "theme": config.theme,
        "show_token_budget": config.show_token_budget,
        "show_cost": config.show_cost,
        "enable_mcp": config.enable_mcp,
        "proxy_mode": config.proxy_mode,
        "proxy_url": config.proxy_url,
        "active_provider": config.providers.active_provider,
        "active_model": config.providers.active_model,
    }
@app.put("/api/config")
async def update_config(request: ConfigUpdateRequest) -> dict[str, Any]:
    config = load_config()
    if request.max_tokens is not None:
        config.max_tokens = request.max_tokens
    if request.temperature is not None:
        config.temperature = request.temperature
    if request.thinking_budget is not None:
        config.thinking_budget = request.thinking_budget
    if request.permission_mode is not None:
        config.permission_mode = request.permission_mode
    if request.theme is not None:
        config.theme = request.theme
    save_config(config)
    return {"updated": True}
@app.get("/api/models")
async def list_models() -> list[ModelInfo]:
    config = load_config()
    models = []
    for provider_name, provider in config.providers.providers.items():
        if provider.enabled:
            models.append(
                ModelInfo(
                    id=provider.default_model,
                    name=provider.default_model,
                    provider=provider_name,
                    enabled=True,
                )
            )
    return models
@app.get("/api/tools")
async def list_tools() -> list[ToolInfo]:
    registry = ToolRegistry()
    register_all_tools(registry)
    tools = registry.get_all()
    return [
        ToolInfo(
            name=tool.name,
            description=tool.description,
            input_schema=tool.get_input_schema(),
            is_read_only=tool.is_read_only,
            is_destructive=tool.is_destructive,
```

```
            )
            for tool in tools
        ]
    @app.websocket("/ws")
    async def websocket_endpoint(websocket: WebSocket) -> None:
        await self.manager.connect(websocket)
        try:
            await self.manager.send_personal(
                websocket,
                {"type": "connected", "timestamp": time.time()},
            )
            while True:
                data = await websocket.receive_json()
                await self._handle_websocket_message(websocket, data)
        except WebSocketDisconnect:
            await self.manager.disconnect(websocket)
        except Exception:
            await self.manager.disconnect(websocket)
async def _handle_websocket_message(self, websocket: WebSocket, data: dict[str, Any]) -> No
ne:
    msg_type = data.get("type", "")
    if msg_type == "chat":
        session_id = data.get("session_id")
        content = data.get("content", "")
        session = await self._get_or_create_session(session_id)
        user_msg = UserMessage(role=MessageRole.USER, content=content)
        session.app_state.add_message(user_msg)
        await self.manager.send_personal(
            websocket,
            {"type": "chat_started", "session_id": session.app_state.session_id},
        )
        async for chunk in self._stream_chat(session, content):
            await self.manager.send_personal(
                websocket,
                {"type": "chat_chunk", "content": chunk},
            )
        await self.manager.send_personal(
            websocket,
            {"type": "chat_complete", "session_id": session.app_state.session_id},
        )
    elif msg_type == "ping":
        await self.manager.send_personal(
            websocket,
            {"type": "pong", "timestamp": time.time()},
        )
    elif msg_type == "get_state":
        session_id = data.get("session_id")
        if session_id and session_id in self.sessions:
            s = self.sessions[session_id]
            await self.manager.send_personal(
                websocket,
                {
                    "type": "state",
                    "session_id": session_id,
                    "cwd": s.app_state.cwd,
                    "message_count": len(s.app_state.messages),
                    "is_running": s.app_state.is_running,
                    "token_usage": {
                        "input_tokens": s.app_state.token_usage.input_tokens,
                        "output_tokens": s.app_state.token_usage.output_tokens,
                    },
                    "cost": s.app_state.cost_info.total_cost,
                },
            )
async def _get_or_create_session(self, session_id: str | None) -> ActiveSession:
```

```
    if session_id and session_id in self.sessions:
        return self.sessions[session_id]
    config = load_config()
    cwd = os.getcwd()
    model = config.get_active_model()
    persist_dir = os.path.expanduser("~/.openlaoke/sessions")
    os.makedirs(persist_dir, exist_ok=True)
    persist_path = os.path.join(persist_dir, f"session_{int(time.time())}.json")
    app_state = create_app_state(cwd=cwd, model=model, persist_path=persist_path)
    app_state.session_config.model = model
    app_state.multi_provider_config = config.providers
    app_state.app_config = config
    registry = ToolRegistry()
    register_all_tools(registry)
    proxy_url = get_proxy_url(config)
    api = MultiProviderClient(config.providers, proxy=proxy_url)
    new_session_id = app_state.session_id
    session = ActiveSession(
        app_state=app_state,
        registry=registry,
        api=api,
        config=config,
    )
    self.sessions[new_session_id] = session
    return session
async def _send_message(
    self, session: ActiveSession, content: str
) -> tuple[AssistantMessage, TokenUsage, CostInfo]:
    system_prompt = build_system_prompt(
        session.app_state, session.registry.get_all_for_prompt()
    )
    messages = [{"role": "user", "content": content}]
    tools = session.registry.get_all_for_prompt()
    response, usage, cost = await session.api.send_message(
        system_prompt=system_prompt,
        messages=messages,
        tools=tools,
        max_tokens=session.config.max_tokens,
        temperature=session.config.temperature,
    )
    session.app_state.accumulate_tokens(usage)
    session.app_state.accumulate_cost(cost)
    await self.manager.broadcast(
        {
            "type": "message_complete",
            "session_id": session.app_state.session_id,
            "usage": {
                "input_tokens": usage.input_tokens,
                "output_tokens": usage.output_tokens,
            },
            "cost": cost.total_cost,
        }
    )
    return response, usage, cost
async def _stream_chat(self, session: ActiveSession, content: str) -> AsyncIterator[str]:
    system_prompt = build_system_prompt(
        session.app_state, session.registry.get_all_for_prompt()
    )
    messages = [
        {"role": msg.role.value, "content": msg.content} for msg in session.app_state.messa
ges
    ]
    tools = session.registry.get_all_for_prompt()
    try:
        async for text, usage, cost in session.api.stream_message(
```

```
                system_prompt=system_prompt,
                messages=messages,
                tools=tools,
                max_tokens=session.config.max_tokens,
                temperature=session.config.temperature,
            ):
                if text:
                    yield f"data: {json.dumps({'type': 'text', 'content': text})}\n\n"
                if usage and cost:
                    session.app_state.accumulate_tokens(usage)
                    session.app_state.accumulate_cost(cost)
                    yield f"data: {json.dumps({'type': 'usage', 'usage': {'input_tokens': usage
    .input_tokens, 'output_tokens': usage.output_tokens}, 'cost': cost.total_cost})}\n\n"
        except Exception as e:
            yield f"data: {json.dumps({'type': 'error', 'message': str(e)})}\n\n"
        yield "data: [DONE]\n\n"
    def run(self) -> None:
        import uvicorn
        uvicorn.run(self.app, host=self.host, port=self.port, log_level="info")
def create_server(
    host: str = "localhost",
    port: int = 3000,
    cors_origins: list[str] | None = None,
) -> Server:
    return Server(host=host, port=port, cors_origins=cors_origins)
```

File: [159/252] openlaoke/services/__init__.py

```
"""Services for OpenLaoKe."""
```

File: [160/252] openlaoke/services/mcp.py

```
"""MCP (Model Context Protocol) service with multi-transport support."""
from __future__ import annotations
import asyncio
import base64
import hashlib
import json
import os
import secrets
import urllib.parse
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
import httpx
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class TransportType(StrEnum):
    """MCP transport types."""
    STDIO = "stdio"
    SSE = "sse"
    HTTP = "http"
    WS = "ws"
class ConnectionState(StrEnum):
    """MCP connection states."""
    CONNECTED = "connected"
    DISCONNECTED = "disconnected"
    FAILED = "failed"
    NEEDS_AUTH = "needs_auth"
    NEEDS_CLIENT_REGISTRATION = "needs_client_registration"
    DISABLED = "disabled"
@dataclass
class OAuthConfig:
    """OAuth configuration for MCP servers."""
    client_id: str | None = None
    client_secret: str | None = None
    scope: str | None = None
    redirect_uri: str = "http://127.0.0.1:19876/mcp/oauth/callback"
@dataclass
class OAuthTokens:
    """OAuth tokens storage."""
    access_token: str
    refresh_token: str | None = None
    expires_at: float | None = None
    scope: str | None = None
@dataclass
class OAuthState:
    """OAuth state for authentication flow."""
    code_verifier: str | None = None
    state: str | None = None
    tokens: OAuthTokens | None = None
@dataclass
class MCPServerConfig:
    """Configuration for an MCP server."""
    name: str
    transport: TransportType = TransportType.STDIO
    command: str = ""
    args: list[str] = field(default_factory=list)
    env: dict[str, str] = field(default_factory=dict)
    url: str = ""
    headers: dict[str, str] = field(default_factory=dict)
    oauth: OAuthConfig | bool | None = None
    enabled: bool = True
```

```
    timeout: int = 30000
@dataclass
class MCPServerConnection:
    """State of an MCP server connection."""
    config: MCPServerConfig
    state: ConnectionState = ConnectionState.DISCONNECTED
    tools: list[dict[str, Any]] = field(default_factory=list)
    process: asyncio.subprocess.Process | None = None
    error: str | None = None
    oauth_state: OAuthState | None = None
    _http_client: httpx.AsyncClient | None = None
    _ws_connection: Any = None
    _sse_response: httpx.Response | None = None
    _request_id: int = 0
class MCPService:
    """Manages MCP server connections with multi-transport support."""
    OAUTH_CALLBACK_PORT = 19876
    OAUTH_CALLBACK_PATH = "/mcp/oauth/callback"
    def __init__(self, registry: ToolRegistry) -> None:
        self.registry = registry
        self.servers: dict[str, MCPServerConnection] = {}
        self._config_path = os.path.expanduser("~/.openlaoke/mcp_servers.json")
        self._auth_path = os.path.expanduser("~/.openlaoke/mcp_auth.json")
        self._oauth_transports: dict[str, MCPServerConnection] = {}
    async def load_config(self) -> None:
        if os.path.exists(self._config_path):
            try:
                with open(self._config_path) as f:
                    data = json.load(f)
                for name, cfg in data.get("mcpServers", {}).items():
                    oauth_cfg: OAuthConfig | bool | None = None
                    if "oauth" in cfg:
                        oauth_data = cfg["oauth"]
                        if oauth_data is False:
                            oauth_cfg = False
                        elif isinstance(oauth_data, dict):
                            oauth_cfg = OAuthConfig(
                                client_id=oauth_data.get("clientId"),
                                client_secret=oauth_data.get("clientSecret"),
                                scope=oauth_data.get("scope"),
                            )
                    transport_str = cfg.get("transport", "stdio")
                    try:
                        transport = TransportType(transport_str)
                    except ValueError:
                        transport = TransportType.STDIO
                    self.servers[name] = MCPServerConnection(
                        config=MCPServerConfig(
                            name=name,
                            transport=transport,
                            command=cfg.get("command", ""),
                            args=cfg.get("args", []),
                            env=cfg.get("env", {}),
                            url=cfg.get("url", ""),
                            headers=cfg.get("headers", {}),
                            oauth=oauth_cfg,
                            enabled=cfg.get("enabled", True),
                            timeout=cfg.get("timeout", 30000),
                        )
                    )
            except Exception:
                pass
        await self._load_auth_state()
    async def _load_auth_state(self) -> None:
        if os.path.exists(self._auth_path):
```

```
        try:
            with open(self._auth_path) as f:
                data = json.load(f)
            for name, state_data in data.items():
                if name in self.servers:
                    conn = self.servers[name]
                    if "tokens" in state_data:
                        tokens = OAuthTokens(
                            access_token=state_data["tokens"]["accessToken"],
                            refresh_token=state_data["tokens"].get("refreshToken"),
                            expires_at=state_data["tokens"].get("expiresAt"),
                            scope=state_data["tokens"].get("scope"),
                        )
                        conn.oauth_state = OAuthState(
                            code_verifier=state_data.get("codeVerifier"),
                            state=state_data.get("oauthState"),
                            tokens=tokens,
                        )
        except Exception:
            pass
async def _save_auth_state(self) -> None:
    data: dict[str, dict[str, Any]] = {}
    for name, conn in self.servers.items():
        if conn.oauth_state:
            state_data: dict[str, Any] = {}
            if conn.oauth_state.tokens:
                state_data["tokens"] = {
                    "accessToken": conn.oauth_state.tokens.access_token,
                    "refreshToken": conn.oauth_state.tokens.refresh_token,
                    "expiresAt": conn.oauth_state.tokens.expires_at,
                    "scope": conn.oauth_state.tokens.scope,
                }
            if conn.oauth_state.code_verifier:
                state_data["codeVerifier"] = conn.oauth_state.code_verifier
            if conn.oauth_state.state:
                state_data["oauthState"] = conn.oauth_state.state
            data[name] = state_data
    os.makedirs(os.path.dirname(self._auth_path), exist_ok=True)
    with open(self._auth_path, "w") as f:
        json.dump(data, f, indent=2)
async def connect_server(self, name: str) -> MCPServerConnection | None:
    conn = self.servers.get(name)
    if not conn:
        return None
    if not conn.config.enabled:
        conn.state = ConnectionState.DISABLED
        return conn
    try:
        if conn.config.transport == TransportType.STDIO:
            await self._connect_stdio(conn)
        elif conn.config.transport == TransportType.SSE:
            await self._connect_sse(conn)
        elif conn.config.transport == TransportType.HTTP:
            await self._connect_http(conn)
        elif conn.config.transport == TransportType.WS:
            await self._connect_ws(conn)
        if conn.state == ConnectionState.CONNECTED:
            await self._discover_tools(conn)
    except Exception as e:
        conn.error = str(e)
        conn.state = ConnectionState.FAILED
    return conn
async def _connect_stdio(self, conn: MCPServerConnection) -> None:
    env = os.environ.copy()
    env.update(conn.config.env)
```

```
    conn.process = await asyncio.create_subprocess_exec(
        conn.config.command,
        *conn.config.args,
        stdin=asyncio.subprocess.PIPE,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
        env=env,
    )
    conn.state = ConnectionState.CONNECTED
async def _connect_sse(self, conn: MCPServerConnection) -> None:
    headers = self._build_headers(conn)
    client = httpx.AsyncClient(timeout=conn.config.timeout / 1000)
    try:
        response = await client.get(
            conn.config.url,
            headers=headers,
        )
        response.raise_for_status()
        conn._http_client = client
        conn._sse_response = response
        conn.state = ConnectionState.CONNECTED
    except httpx.HTTPStatusError as e:
        if e.response.status_code == 401:
            conn.state = ConnectionState.NEEDS_AUTH
            self._oauth_transports[conn.config.name] = conn
        else:
            raise
async def _connect_http(self, conn: MCPServerConnection) -> None:
    client = httpx.AsyncClient(timeout=conn.config.timeout / 1000)
    conn._http_client = client
    try:
        await self._http_request(conn, "initialize", {"protocolVersion": "2024-11-05"})
        conn.state = ConnectionState.CONNECTED
    except httpx.HTTPStatusError as e:
        if e.response.status_code == 401:
            conn.state = ConnectionState.NEEDS_AUTH
            self._oauth_transports[conn.config.name] = conn
        else:
            raise
async def _connect_ws(self, conn: MCPServerConnection) -> None:
    import websockets
    headers = self._build_headers(conn)
    extra_headers = [(k, v) for k, v in headers.items()]
    try:
        ws = await websockets.connect(
            conn.config.url,
            additional_headers=extra_headers,
            ping_interval=20,
            ping_timeout=conn.config.timeout / 1000,
        )
        conn._ws_connection = ws
        await self._ws_request(conn, "initialize", {"protocolVersion": "2024-11-05"})
        conn.state = ConnectionState.CONNECTED
    except Exception as e:
        error_str = str(e)
        if "401" in error_str or "Unauthorized" in error_str:
            conn.state = ConnectionState.NEEDS_AUTH
            self._oauth_transports[conn.config.name] = conn
        else:
            raise
def _build_headers(self, conn: MCPServerConnection) -> dict[str, str]:
    headers = dict(conn.config.headers)
    if conn.oauth_state and conn.oauth_state.tokens:
        headers["Authorization"] = f"Bearer {conn.oauth_state.tokens.access_token}"
    return headers
```

```
async def _http_request(
    self, conn: MCPServerConnection, method: str, params: dict[str, Any]
) -> dict[str, Any]:
    if not conn._http_client:
        raise ValueError("HTTP client not initialized")
    conn._request_id += 1
    request = {
        "jsonrpc": "2.0",
        "id": conn._request_id,
        "method": method,
        "params": params,
    }
    response = await conn._http_client.post(
        conn.config.url,
        json=request,
        headers=self._build_headers(conn),
    )
    response.raise_for_status()
    return dict(response.json())
async def _ws_request(
    self, conn: MCPServerConnection, method: str, params: dict[str, Any]
) -> dict[str, Any]:
    if not conn._ws_connection:
        raise ValueError("WebSocket connection not initialized")
    conn._request_id += 1
    request = {
        "jsonrpc": "2.0",
        "id": conn._request_id,
        "method": method,
        "params": params,
    }
    await conn._ws_connection.send(json.dumps(request))
    response_str = await conn._ws_connection.recv()
    return dict(json.loads(response_str))
async def _stdio_request(
    self, conn: MCPServerConnection, method: str, params: dict[str, Any], timeout: float =
30.0
) -> dict[str, Any]:
    if not conn.process or not conn.process.stdin or not conn.process.stdout:
        raise ValueError("Process not initialized")
    conn._request_id += 1
    request = {
        "jsonrpc": "2.0",
        "id": conn._request_id,
        "method": method,
        "params": params,
    }
    conn.process.stdin.write((json.dumps(request) + "\n").encode())
    await conn.process.stdin.drain()
    response_line = await asyncio.wait_for(conn.process.stdout.readline(), timeout=timeout)
    return dict(json.loads(response_line.decode()))
async def _discover_tools(self, conn: MCPServerConnection) -> None:
    try:
        response = await self._send_request(conn, "tools/list", {})
        for tool_def in response.get("result", {}).get("tools", []):
            conn.tools.append(tool_def)
            self._register_mcp_tool(conn, tool_def)
    except Exception:
        pass
async def _send_request(
    self, conn: MCPServerConnection, method: str, params: dict[str, Any]
) -> dict[str, Any]:
    if conn.config.transport == TransportType.STDIO:
        return await self._stdio_request(conn, method, params)
    elif conn.config.transport == TransportType.HTTP:
```

```
        return await self._http_request(conn, method, params)
    elif conn.config.transport == TransportType.WS:
        return await self._ws_request(conn, method, params)
    elif conn.config.transport == TransportType.SSE:
        return await self._http_request(conn, method, params)
    raise ValueError(f"Unsupported transport: {conn.config.transport}")
def _register_mcp_tool(self, conn: MCPServerConnection, tool_def: dict[str, Any]) -> None:
    name = tool_def.get("name", "")
    description = tool_def.get("description", "")
    schema = tool_def.get("inputSchema", {})
    class MCPToolWrapper(Tool):
        def __init__(
            self,
            mcp_service: MCPService,
            mcp_name: str,
            server_conn: MCPServerConnection,
            tool_name: str,
            tool_desc: str,
            tool_schema: dict[str, Any],
        ):
            self.name = f"mcp_{mcp_name}_{tool_name}"
            self.description = f"[MCP: {mcp_name}] {tool_desc}"
            self.input_schema = tool_schema
            self._mcp_service = mcp_service
            self._mcp_name = mcp_name
            self._server = server_conn
            self._tool_name = tool_name
        async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
            try:
                response = await self._mcp_service._send_request(
                    self._server,
                    "tools/call",
                    {"name": self._tool_name, "arguments": kwargs},
                )
                content = response.get("result", {}).get("content", [])
                text = "\n".join(c.get("text", "") for c in content if isinstance(c, dict))
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=text or "(no output)",
                )
            except Exception as e:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"MCP tool error: {e}",
                    is_error=True,
                )
    self.registry.register(MCPToolWrapper(self, name, conn, name, description, schema))
async def start_oauth_flow(self, name: str) -> tuple[str, str]:
    conn = self.servers.get(name)
    if not conn:
        raise ValueError(f"MCP server {name} not found")
    if conn.config.oauth is False:
        raise ValueError(f"OAuth disabled for {name}")
    code_verifier = secrets.token_urlsafe(32)
    code_challenge = (
        base64.urlsafe_b64encode(hashlib.sha256(code_verifier.encode()).digest())
        .decode()
        .rstrip("=")
    )
    state = secrets.token_urlsafe(32)
    if conn.oauth_state is None:
        conn.oauth_state = OAuthState()
    conn.oauth_state.code_verifier = code_verifier
    conn.oauth_state.state = state
    oauth_config = (
```

```
        conn.config.oauth if isinstance(conn.config.oauth, OAuthConfig) else OAuthConfig()
    )
    auth_url_params = {
        "response_type": "code",
        "client_id": oauth_config.client_id or "",
        "redirect_uri": oauth_config.redirect_uri,
        "code_challenge": code_challenge,
        "code_challenge_method": "S256",
        "state": state,
    }
    if oauth_config.scope:
        auth_url_params["scope"] = oauth_config.scope
    base_url = conn.config.url
    auth_base = base_url.split("/mcp")[0] if "/mcp" in base_url else base_url.rstrip("/")
    authorization_url = f"{auth_base}/authorize?{urllib.parse.urlencode(auth_url_params)}"
    await self._save_auth_state()
    return authorization_url, state
async def finish_oauth_flow(self, name: str, authorization_code: str) -> ConnectionState:
    conn = self._oauth_transports.get(name) or self.servers.get(name)
    if not conn:
        raise ValueError(f"MCP server {name} not found")
    if not conn.oauth_state or not conn.oauth_state.code_verifier:
        raise ValueError(f"No pending OAuth flow for {name}")
    oauth_config = (
        conn.config.oauth if isinstance(conn.config.oauth, OAuthConfig) else OAuthConfig()
    )
    token_url_params = {
        "grant_type": "authorization_code",
        "code": authorization_code,
        "redirect_uri": oauth_config.redirect_uri,
        "code_verifier": conn.oauth_state.code_verifier,
    }
    if oauth_config.client_id:
        token_url_params["client_id"] = oauth_config.client_id
    if oauth_config.client_secret:
        token_url_params["client_secret"] = oauth_config.client_secret
    base_url = conn.config.url
    token_base = base_url.split("/mcp")[0] if "/mcp" in base_url else base_url.rstrip("/")
    token_url = f"{token_base}/token"
    client = httpx.AsyncClient()
    try:
        response = await client.post(
            token_url,
            data=token_url_params,
            headers={"Content-Type": "application/x-www-form-urlencoded"},
        )
        response.raise_for_status()
        token_data = response.json()
        tokens = OAuthTokens(
            access_token=token_data["access_token"],
            refresh_token=token_data.get("refresh_token"),
            expires_at=(token_data.get("expires_in") if token_data.get("expires_in") else N
one),
            scope=token_data.get("scope"),
        )
        conn.oauth_state.tokens = tokens
        conn.oauth_state.code_verifier = None
        conn.oauth_state.state = None
        await self._save_auth_state()
        del self._oauth_transports[name]
        result = await self.connect_server(name)
        return result.state if result else ConnectionState.FAILED
    finally:
        await client.aclose()
async def remove_auth(self, name: str) -> None:
```

```
    conn = self.servers.get(name)
    if not conn:
        return
    conn.oauth_state = None
    if name in self._oauth_transports:
        del self._oauth_transports[name]
    await self._save_auth_state()
def supports_oauth(self, name: str) -> bool:
    conn = self.servers.get(name)
    if not conn:
        return False
    if conn.config.transport == TransportType.STDIO:
        return False
    return conn.config.oauth is not False
def has_stored_tokens(self, name: str) -> bool:
    conn = self.servers.get(name)
    if not conn or not conn.oauth_state:
        return False
    return conn.oauth_state.tokens is not None
async def disconnect_all(self) -> None:
    for conn in self.servers.values():
        await self._disconnect_connection(conn)
async def _disconnect_connection(self, conn: MCPServerConnection) -> None:
    if conn.process and conn.process.returncode is None:
        conn.process.kill()
        await conn.process.wait()
    if conn._http_client:
        await conn._http_client.aclose()
        conn._http_client = None
    if conn._ws_connection:
        await conn._ws_connection.close()
        conn._ws_connection = None
    conn.state = ConnectionState.DISCONNECTED
def get_server_status(self) -> list[dict[str, Any]]:
    return [
        {
            "name": name,
            "state": conn.state.value,
            "transport": conn.config.transport.value,
            "tools": len(conn.tools),
            "error": conn.error,
            "has_auth": self.has_stored_tokens(name),
        }
        for name, conn in self.servers.items()
    ]
```

File: [161/252] openlaoke/tools/__init__.py

```
"""Tool system for OpenLaoKe."""
from __future__ import annotations
from openlaoke.tools.register import (
    register_all_tools,
    register_deferred_tools,
    register_essential_tools,
)
__all__ = ["register_all_tools", "register_essential_tools", "register_deferred_tools"]
```

File: [162/252] openlaoke/tools/agent_tool.py

```
"""Agent tool - spawn sub-agents for parallel work."""
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class AgentInput(BaseModel):
    prompt: str = Field(description="The prompt to give the sub-agent")
    description: str = Field(
        default="", description="Brief description of what the agent should do"
    )
    subagent_type: str = Field(default="general-purpose", description="Type of sub-agent to use
    ")
class AgentTool(Tool):
    """Spawn a sub-agent to handle complex tasks in parallel."""
    name = "Agent"
    description = (
        "Launch a sub-agent to work on a task in parallel. "
        "Use this for independent work that can be done concurrently. "
        "The sub-agent has access to all the same tools."
    )
    input_schema = AgentInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        prompt = kwargs.get("prompt", "")
        description = kwargs.get("description", "")
        subagent_type = kwargs.get("subagent_type", "general-purpose")
        if not prompt.strip():
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: prompt is required",
                is_error=True,
            )
        from openlaoke.core.task import TaskManager
        task_mgr = TaskManager(ctx.app_state)
        try:
            result = await task_mgr.run_agent(
                prompt=prompt,
                description=description or prompt[:100],
                tool_use_id=ctx.tool_use_id,
                subagent_type=subagent_type,
            )
            max_output = 20000
            if len(result) > max_output:
                result = (
                    result[:max_output]
                    + f"\n\n... (output truncated, {len(result) - max_output} chars omitted)"
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Agent result ({subagent_type}):\n\n{result}",
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Agent failed: {e}",
                is_error=True,
```

```
            )
def register(registry: ToolRegistry) -> None:
    registry.register(AgentTool())
```

File: [163/252] openlaoke/tools/apply_patch_tool.py

```
"""Apply patch tool - Apply unified diff patches to files."""
from __future__ import annotations
import os
import re
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class ApplyPatchInput(BaseModel):
    patch_content: str = Field(description="The unified diff patch content to apply")
    file_path: str | None = Field(
        default=None,
        description="Target file path (optional, extracted from patch if not provided)",
    )
class ApplyPatchTool(Tool):
    """Apply unified diff patches to files."""
    name = "ApplyPatch"
    description = (
        "Apply a unified diff patch to files. Parses the patch content and applies "
        "the changes to the target files. Supports standard unified diff format. "
        "Can auto-detect target file from patch headers."
    )
    input_schema = ApplyPatchInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        patch_content = kwargs.get("patch_content", "")
        file_path = kwargs.get("file_path")
        if not patch_content.strip():
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: patch_content is required",
                is_error=True,
            )
        try:
            patches = self._parse_patch(patch_content)
            if not patches:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: No valid patches found in the content",
                    is_error=True,
                )
            results = []
            for patch in patches:
                target_file = file_path or patch.get("file")
                if not target_file:
                    results.append("Skipped patch: No target file specified")
                    continue
                abs_path = self._resolve_path(target_file, ctx.app_state.get_cwd())
                if not os.path.exists(abs_path):
                    results.append(f"Skipped {target_file}: File not found")
                    continue
                try:
                    result = self._apply_patch_to_file(abs_path, patch)
                    results.append(result)
                except Exception as e:
                    results.append(f"Error applying patch to {target_file}: {e}")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
```

```
            content="\n".join(results),
            is_error=False,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error parsing patch: {e}",
            is_error=True,
        )
def _resolve_path(self, path: str, cwd: str) -> str:
    if os.path.isabs(path):
        return os.path.normpath(path)
    return os.path.normpath(os.path.join(cwd, path))
def _parse_patch(self, patch_content: str) -> list[dict[str, Any]]:
    patches = []
    current_patch: dict[str, Any] | None = None
    current_hunks: list[dict[str, Any]] = []
    current_hunk: dict[str, Any] | None = None
    lines = patch_content.split("\n")
    for line in lines:
        if line.startswith("--- "):
            if current_patch and current_hunks:
                current_patch["hunks"] = current_hunks
                patches.append(current_patch)
            current_patch = {"file": None, "hunks": []}
            current_hunks = []
            source = line[4:].strip()
            if source.startswith("a/"):
                source = source[2:]
            current_patch["source"] = source.split("\t")[0]
        elif line.startswith("+++ "):
            if current_patch:
                target = line[4:].strip()
                if target.startswith("b/"):
                    target = target[2:]
                current_patch["file"] = target.split("\t")[0]
        elif line.startswith("@@"):
            if current_patch:
                match = re.match(r"@@ -(\d+)(?:,(\d+))? \+(\d+)(?:,(\d+))? @@", line)
                if match:
                    if current_hunk:
                        current_hunks.append(current_hunk)
                    current_hunk = {
                        "old_start": int(match.group(1)),
                        "old_count": int(match.group(2) or 1),
                        "new_start": int(match.group(3)),
                        "new_count": int(match.group(4) or 1),
                        "lines": [],
                    }
        elif current_hunk is not None:
            if line.startswith("+"):
                current_hunk["lines"].append(("add", line[1:]))
            elif line.startswith("-"):
                current_hunk["lines"].append(("remove", line[1:]))
            elif line.startswith(" "):
                current_hunk["lines"].append(("context", line[1:]))
            elif line == "":
                current_hunk["lines"].append(("context", ""))
    if current_hunk:
        current_hunks.append(current_hunk)
    if current_patch and current_hunks:
        current_patch["hunks"] = current_hunks
        patches.append(current_patch)
    return patches
def _apply_patch_to_file(self, file_path: str, patch: dict[str, Any]) -> str:
```

```
        with open(file_path, encoding="utf-8") as f:
            original_lines = f.read().split("\n")
        result_lines = original_lines[:]
        offset = 0
        changes_made = 0
        for hunk in patch.get("hunks", []):
            old_start = hunk["old_start"]
            hunk_lines = hunk["lines"]
            insert_pos = old_start - 1 + offset
            remove_count = sum(1 for t, _ in hunk_lines if t == "remove")
            new_hunk_lines = [
                line_content for t, line_content in hunk_lines if t in ("add", "context")
            ]
            start_pos = max(0, insert_pos)
            end_pos = start_pos + remove_count
            context_match = True
            for i, (t, line_content) in enumerate(hunk_lines):
                if (
                    t == "context"
                    and start_pos + i < len(result_lines)
                    and result_lines[start_pos + i] != line_content
                ):
                    context_match = False
                    break
            if not context_match:
                continue
            result_lines = result_lines[:start_pos] + new_hunk_lines + result_lines[end_pos:]
            offset += len(new_hunk_lines) - remove_count
            changes_made += 1
        with open(file_path, "w", encoding="utf-8") as f:
            f.write("\n".join(result_lines))
        return f"Applied {changes_made} hunk(s) to {file_path}"
def register(registry: ToolRegistry) -> None:
    registry.register(ApplyPatchTool())
```

File: [164/252] openlaoke/tools/bash_tool.py

```
"""Bash tool - execute shell commands."""
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import PermissionMode, ToolResultBlock
from openlaoke.utils.permissions.bash_classifier import (
    CommandSafetyLevel,
    classify_bash_command,
)
class BashInput(BaseModel):
    command: str = Field(description="The bash command to execute")
    description: str = Field(default="", description="Brief description of what this command do
    es")
    timeout: float | None = Field(default=None, description="Timeout in seconds")
class BashTool(Tool):
    """Execute bash commands in the terminal."""
    name = "Bash"
    description = (
        "Execute a bash command in the current working directory. "
        "Use this for running scripts, installing packages, git operations, etc. "
        "Long-running commands will stream output."
    )
    input_schema = BashInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        command = kwargs.get("command", "")
        description = kwargs.get("description", "")
        timeout = kwargs.get("timeout")
        if not command.strip():
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: Empty command",
                is_error=True,
            )
        classification = classify_bash_command(command)
        if classification.safety_level == CommandSafetyLevel.DESTRUCTIVE:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Command blocked for safety: {classification.reason}\n"
                f"This command is classified as destructive and cannot be executed.",
                is_error=True,
            )
        perm_mode = ctx.app_state.permission_config.mode
        if (
            perm_mode == PermissionMode.DEFAULT
            and classification.safety_level == CommandSafetyLevel.DANGEROUS
        ):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Command requires approval: {classification.reason}\n"
                f"Confidence: {classification.confidence.value}\n"
                f"Use 'approve' to allow this command, or run with --permission-mode auto.",
                is_error=True,
            )
        if (
            perm_mode == PermissionMode.AUTO
            and classification.safety_level != CommandSafetyLevel.SAFE
```

```
            and classification.confidence != "high"
        ):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Command needs confirmation in auto mode: {classification.reason}\n"
                f"Safety level: {classification.safety_level.value}\n"
                f"Confidence: {classification.confidence.value}",
                is_error=True,
            )
        from openlaoke.core.task import TaskManager
        task_mgr = TaskManager(ctx.app_state)
        try:
            output, exit_code = await task_mgr.run_bash(
                command=command,
                description=description or command[:100],
                timeout=timeout,
                tool_use_id=ctx.tool_use_id,
                working_dir=ctx.app_state.get_cwd(),
            )
            max_output = 30000
            if len(output) > max_output:
                truncated = output[:max_output]
                output = f"{truncated}\n\n... (output truncated, {len(output) - max_output} cha
    rs omitted)"
            if exit_code != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Command exited with code {exit_code}\n\n{output}",
                    is_error=True,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=output if output else "(command completed successfully with no output)"
    ,
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error executing command: {e}",
                is_error=True,
            )
def register(registry: ToolRegistry) -> None:
    registry.register(BashTool())
```

File: [165/252] openlaoke/tools/batch_tool.py

```
"""Batch tool - Execute multiple tool calls in parallel or sequence."""
from __future__ import annotations
import asyncio
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class ToolCallSpec(BaseModel):
    tool_name: str = Field(description="Name of the tool to call")
    args: dict[str, Any] = Field(default_factory=dict, description="Arguments for the tool call
    ")
class BatchInput(BaseModel):
    calls: list[ToolCallSpec] = Field(description="List of tool calls to execute")
    parallel: bool = Field(default=True, description="Execute calls in parallel (default: True)
    ")
    stop_on_error: bool = Field(default=False, description="Stop on first error (default: False
    )")
class BatchTool(Tool):
    """Execute multiple tool calls in a single batch."""
    name = "Batch"
    description = (
        "Execute multiple tool calls in a single batch operation. "
        "Supports both parallel (default) and sequential execution. "
        "Use stop_on_error to control error handling. "
        "Returns aggregated results from all tool calls."
    )
    input_schema = BatchInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        calls_raw = kwargs.get("calls", [])
        parallel = kwargs.get("parallel", True)
        stop_on_error = kwargs.get("stop_on_error", False)
        if not calls_raw:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: calls list is required",
                is_error=True,
            )
        # Convert dict calls to ToolCallSpec if needed
        calls: list[ToolCallSpec] = []
        for call in calls_raw:
            if isinstance(call, ToolCallSpec):
                calls.append(call)
            elif isinstance(call, dict):
                calls.append(ToolCallSpec(**call))
            else:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Error: Invalid call spec type: {type(call)}",
                    is_error=True,
                )
        if len(calls) > 50:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Too many calls ({len(calls)}). Maximum is 50.",
                is_error=True,
            )
        results = []
        errors = []
```

```
    if parallel:
        tasks = [self._execute_tool(ctx, call_spec, i) for i, call_spec in enumerate(calls)
]
        batch_results = await asyncio.gather(*tasks, return_exceptions=True)
        for i, result in enumerate(batch_results):
            if isinstance(result, Exception):
                error_msg = f"Call {i} ({calls[i].tool_name}): {result}"
                errors.append(error_msg)
                if stop_on_error:
                    break
                results.append(f"ERROR: {error_msg}")
            else:
                result_str = str(result)
                results.append(result_str)
                if "ERROR:" in result_str and stop_on_error:
                    errors.append(result_str)
                    break
    else:
        for i, call_spec in enumerate(calls):
            try:
                result = await self._execute_tool(ctx, call_spec, i)
                results.append(result)
                if "ERROR:" in result and stop_on_error:
                    errors.append(result)
                    break
            except Exception as e:
                error_msg = f"Call {i} ({call_spec.tool_name}): {e}"
                errors.append(error_msg)
                results.append(f"ERROR: {error_msg}")
                if stop_on_error:
                    break
    output_parts = []
    for i, (call_spec, result) in enumerate(zip(calls, results, strict=False)):
        output_parts.append(f"--- Call {i}: {call_spec.tool_name} ---")
        output_parts.append(result)
        output_parts.append("")
    if errors and not stop_on_error:
        output_parts.append(f"\nCompleted with {len(errors)} error(s)")
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content="\n".join(output_parts),
        is_error=len(errors) > 0,
    )
async def _execute_tool(self, ctx: ToolContext, call_spec: ToolCallSpec, index: int) -> str
:
    tool_name = call_spec.tool_name
    args = call_spec.args
    tool = (
        ctx.app_state.tool_registry.get(tool_name)
        if hasattr(ctx.app_state, "tool_registry")
        else None
    )
    if not tool:
        registry = self._get_registry()
        if registry:
            tool = registry.get(tool_name)
    if not tool:
        return f"ERROR: Unknown tool: {tool_name}"
    try:
        result = await tool.call(ctx, **args)
        if isinstance(result.content, str):
            return result.content
        elif isinstance(result.content, list):
            return str(result.content)
        else:
```

```
                return str(result.content)
        except Exception as e:
            return f"ERROR: {e}"
    def _get_registry(self) -> ToolRegistry | None:
        from openlaoke.tools.register import register_all_tools
        registry = ToolRegistry()
        register_all_tools(registry)
        return registry
def register(registry: ToolRegistry) -> None:
    registry.register(BatchTool())
```

File: [166/252] openlaoke/tools/brief_tool.py

```
"""Brief tool - enable brief response mode for concise outputs."""
from __future__ import annotations
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class BriefInput(BaseModel):
    mode: Literal["on", "off", "toggle"] = Field(
        default="toggle",
        description="Brief mode action: 'on' to enable, 'off' to disable, 'toggle' to switch",
    )
    max_length: int | None = Field(
        default=None,
        description="Maximum response length in characters when brief mode is on",
        ge=50,
        le=1000,
    )
class BriefTool(Tool):
    """Control brief response mode for shorter, more concise outputs."""
    name = "Brief"
    description = (
        "Enable or disable brief response mode. "
        "When enabled, responses are shortened to be more concise. "
        "Use 'on' to enable, 'off' to disable, or 'toggle' to switch state. "
        "Useful for quick interactions when detailed output is not needed."
    )
    input_schema = BriefInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        mode = kwargs.get("mode", "toggle")
        max_length = kwargs.get("max_length", 200)
        current_state = getattr(ctx.app_state, "brief_mode", False)
        if mode == "on":
            new_state = True
        elif mode == "off":
            new_state = False
        else:
            new_state = not current_state
        if hasattr(ctx.app_state, "brief_mode"):
            ctx.app_state.brief_mode = new_state
        if hasattr(ctx.app_state, "brief_max_length") and max_length:
            ctx.app_state.brief_max_length = max_length
        status = "enabled" if new_state else "disabled"
        message = f"Brief mode {status}"
        if new_state and max_length:
            message += f" (max length: {max_length} chars)"
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=message,
            is_error=False,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(BriefTool())
```

File: [167/252] openlaoke/tools/cron_tool.py

```
"""Cron tool - manage cron jobs."""
from __future__ import annotations
import platform
import subprocess
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class CronInput(BaseModel):
    action: Literal["list", "add", "remove", "update"] = Field(description="Cron action to perf
    orm")
    expression: str | None = Field(default=None, description="Cron expression (e.g., '0 * * * *
    ')")
    command: str | None = Field(default=None, description="Command to execute")
    job_id: str | None = Field(default=None, description="Job identifier for removal or update"
    )
    comment: str | None = Field(default=None, description="Comment for the cron job")
class CronTool(Tool):
    """Manage cron jobs for scheduled tasks."""
    name = "Cron"
    description = (
        "Manage cron jobs for scheduled task execution. "
        "Supports listing, adding, removing, and updating cron jobs. "
        "Uses standard cron expression format. "
        "Note: Requires crontab to be available on the system."
    )
    input_schema = CronInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        system = platform.system()
        if system == "Windows":
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: Cron jobs are not supported on Windows. Use Task Scheduler inst
    ead.",
                is_error=True,
            )
        try:
            if action == "list":
                result = subprocess.run(
                    ["crontab", "-l"],
                    capture_output=True,
                    text=True,
                )
                if result.returncode != 0:
                    if "no crontab" in result.stderr.lower():
                        return ToolResultBlock(
                            tool_use_id=ctx.tool_use_id,
                            content="No cron jobs configured",
                            is_error=False,
                        )
```

```
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error listing cron jobs: {result.stderr}",
            is_error=True,
        )
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=result.stdout or "No cron jobs configured",
        is_error=False,
    )
elif action == "add":
    expression = kwargs.get("expression")
    command = kwargs.get("command")
    comment = kwargs.get("comment", "")
    if not expression or not command:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: expression and command are required for add action",
            is_error=True,
        )
    result = subprocess.run(
        ["crontab", "-l"],
        capture_output=True,
        text=True,
    )
    current_cron = result.stdout if result.returncode == 0 else ""
    comment_str = f" # {comment}" if comment else ""
    new_job = f"{expression} {command}{comment_str}\n"
    updated_cron = current_cron + new_job
    process = subprocess.Popen(
        ["crontab", "-"],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
    )
    _, stderr = process.communicate(input=updated_cron)
    if process.returncode != 0:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error adding cron job: {stderr}",
            is_error=True,
        )
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=f"Added cron job: {expression} {command}",
        is_error=False,
    )
elif action == "remove":
    job_id = kwargs.get("job_id")
    if not job_id:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: job_id is required for remove action",
            is_error=True,
        )
    result = subprocess.run(
        ["crontab", "-l"],
        capture_output=True,
        text=True,
    )
    if result.returncode != 0:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="No cron jobs configured",
```

```
            is_error=False,
        )
    lines = result.stdout.strip().split("\n")
    updated_lines = [line for line in lines if job_id not in line and line.strip()]
    updated_cron = "\n".join(updated_lines)
    if updated_lines:
        updated_cron += "\n"
    process = subprocess.Popen(
        ["crontab", "-"],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
    )
    _, stderr = process.communicate(input=updated_cron)
    if process.returncode != 0:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error removing cron job: {stderr}",
            is_error=True,
        )
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=f"Removed cron job containing: {job_id}",
        is_error=False,
    )
elif action == "update":
    job_id = kwargs.get("job_id")
    expression = kwargs.get("expression")
    command = kwargs.get("command")
    comment = kwargs.get("comment", "")
    if not job_id:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: job_id is required for update action",
            is_error=True,
        )
    if not expression or not command:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: expression and command are required for update action",
            is_error=True,
        )
    result = subprocess.run(
        ["crontab", "-l"],
        capture_output=True,
        text=True,
    )
    if result.returncode != 0:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="No cron jobs configured",
            is_error=False,
        )
    lines = result.stdout.strip().split("\n")
    updated_lines = []
    found = False
    for line in lines:
        if job_id in line:
            comment_str = f" # {comment}" if comment else ""
            updated_lines.append(f"{expression} {command}{comment_str}")
            found = True
        elif line.strip():
            updated_lines.append(line)
    if not found:
```

```
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Job not found: {job_id}",
                    is_error=True,
                )
            updated_cron = "\n".join(updated_lines) + "\n"
            process = subprocess.Popen(
                ["crontab", "-"],
                stdin=subprocess.PIPE,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True,
            )
            _, stderr = process.communicate(input=updated_cron)
            if process.returncode != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Error updating cron job: {stderr}",
                    is_error=True,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Updated cron job: {expression} {command}",
                is_error=False,
            )
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}",
                is_error=True,
            )
    except FileNotFoundError:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: crontab is not installed. Install cron package first.",
            is_error=True,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error managing cron jobs: {e}",
            is_error=True,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(CronTool())
```

File: [168/252] openlaoke/tools/edit_tool.py

```
"""Edit tool - make targeted edits to files."""
from __future__ import annotations
import difflib
import os
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
from openlaoke.utils.file_history import track_file_edit
class EditInput(BaseModel):
    file_path: str = Field(description="Path to the file to edit")
    old_string: str = Field(description="The exact text to find and replace")
    new_string: str = Field(description="The text to replace it with")
    replace_all: bool = Field(
        default=False, description="Replace all occurrences (default: only first)"
    )
class EditTool(Tool):
    """Make targeted edits to files by finding and replacing text."""
    name = "Edit"
    description = (
        "Make targeted edits to a file by replacing specific text. "
        "Provide the exact text to find (old_string) and the replacement (new_string). "
        "Use replace_all to replace all occurrences."
    )
    input_schema = EditInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        file_path = kwargs.get("file_path", "")
        old_string = kwargs.get("old_string", "")
        new_string = kwargs.get("new_string", "")
        replace_all = kwargs.get("replace_all", False)
        if not file_path:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: file_path is required",
                is_error=True,
            )
        abs_path = self._resolve_path(file_path, ctx.app_state.get_cwd())
        if not os.path.exists(abs_path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: File not found: {abs_path}",
                is_error=True,
            )
        try:
            with open(abs_path, encoding="utf-8") as f:
                original = f.read()
            track_file_edit(abs_path, ctx.app_state.session_id)
            if old_string not in original:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=self._suggest_matches(original, old_string, abs_path),
                    is_error=True,
                )
            if replace_all:
                updated = original.replace(old_string, new_string)
                count = original.count(old_string)
            else:
```

```
                updated = original.replace(old_string, new_string, 1)
                count = 1
            with open(abs_path, "w", encoding="utf-8") as f:
                f.write(updated)
            diff = self._generate_diff(original, updated, abs_path)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Replaced {count} occurrence(s) in {abs_path}\n\n{diff}",
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error editing file: {e}",
                is_error=True,
            )
    def _generate_diff(self, original: str, updated: str, filename: str) -> str:
        orig_lines = original.splitlines(keepends=True)
        new_lines = updated.splitlines(keepends=True)
        diff = difflib.unified_diff(
            orig_lines,
            new_lines,
            fromfile=f"a/{filename}",
            tofile=f"b/{filename}",
            n=3,
        )
        return "".join(diff)
    def _suggest_matches(self, content: str, search: str, filename: str) -> str:
        lines = content.split("\n")
        search_lower = search.lower()
        similar_lines = []
        for i, line in enumerate(lines):
            if search_lower in line.lower():
                similar_lines.append((i + 1, line.strip()))
        if similar_lines:
            suggestions = "\n".join(f"  Line {num}: {text}" for num, text in similar_lines[:10]
    )
            return (
                f"Error: Exact text not found in {filename}.\n"
                f"Similar lines found:\n{suggestions}\n\n"
                f"Make sure old_string matches the exact text in the file, "
                f"including whitespace and line endings."
            )
        return f"Error: Text not found in {filename}. No similar lines found."
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(EditTool())
```

File: [169/252] openlaoke/tools/git_tool.py

```
"""Git tool - Git operations."""
from __future__ import annotations
import asyncio
import os
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class GitInput(BaseModel):
    action: str = Field(
        description="Git action: status, diff, log, add, commit, push, pull, branch, checkout,
    merge, rebase, stash, reset, fetch, remote"
    )
    args: str | None = Field(default=None, description="Additional arguments for the action")
class GitTool(Tool):
    """Execute Git operations."""
    name = "Git"
    description = (
        "Execute Git operations. Supports common commands: status, diff, log, add, commit, "
        "push, pull, branch, checkout, merge, rebase, stash, reset, fetch, remote. "
        "Use 'args' for additional arguments like commit messages or branch names."
    )
    input_schema = GitInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action", "")
        args = kwargs.get("args", "")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        action = action.lower().strip()
        valid_actions = {
            "status",
            "diff",
            "log",
            "add",
            "commit",
            "push",
            "pull",
            "branch",
            "checkout",
            "merge",
            "rebase",
            "stash",
            "reset",
            "fetch",
            "remote",
            "init",
            "clone",
            "tag",
            "show",
            "blame",
            "cherry-pick",
            "revert",
            "clean",
```

```
            "gc",
            "prune",
            "config",
        }
        if action not in valid_actions:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown git action: {action}\n"
                f"Valid actions: {', '.join(sorted(valid_actions))}",
                is_error=True,
            )
        cwd = ctx.app_state.get_cwd()
        if not self._is_git_repo(cwd):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Not a git repository: {cwd}",
                is_error=True,
            )
        try:
            result = await self._run_git(cwd, action, args)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=result,
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Git error: {e}",
                is_error=True,
            )
    def _is_git_repo(self, path: str) -> bool:
        git_dir = os.path.join(path, ".git")
        if os.path.isdir(git_dir):
            return True
        parent = os.path.dirname(path)
        if parent == path:
            return False
        return self._is_git_repo(parent)
    async def _run_git(self, cwd: str, action: str, args: str | None) -> str:
        cmd = ["git", action]
        if args:
            cmd.extend(args.split())
        proc = await asyncio.create_subprocess_exec(
            *cmd,
            cwd=cwd,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
        stdout, stderr = await proc.communicate()
        output = ""
        if stdout:
            output += stdout.decode("utf-8", errors="replace")
        if stderr:
            output += stderr.decode("utf-8", errors="replace")
        if proc.returncode != 0:
            return f"Git exited with code {proc.returncode}\n{output}"
        return output if output.strip() else "(no output)"
def register(registry: ToolRegistry) -> None:
    registry.register(GitTool())
```

File: [170/252] openlaoke/tools/glob_tool.py

```
"""Glob tool - find files by pattern."""
from __future__ import annotations
import os
from typing import Any
import pathspec
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class GlobInput(BaseModel):
    pattern: str = Field(description="Glob pattern to match files (e.g., '**/*.py')")
    path: str = Field(default=".", description="Directory to search in")
class GlobTool(Tool):
    """Find files matching a glob pattern."""
    name = "Glob"
    description = (
        "Fast file pattern matching using glob patterns. "
        "Supports ** for recursive matching. Respects .gitignore."
    )
    input_schema = GlobInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        pattern = kwargs.get("pattern", "")
        search_path = kwargs.get("path", ".")
        if not pattern:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: pattern is required",
                is_error=True,
            )
        abs_path = self._resolve_path(search_path, ctx.app_state.get_cwd())
        if not os.path.isdir(abs_path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Directory not found: {abs_path}",
                is_error=True,
            )
        gitignore = self._load_gitignore(abs_path)
        matches = []
        max_results = 1000
        for root, dirs, files in os.walk(abs_path):
            rel_root = os.path.relpath(root, abs_path)
            if rel_root == ".":
                rel_root = ""
            dirs_to_remove = []
            for d in dirs:
                rel_path = os.path.join(rel_root, d) if rel_root else d
                if gitignore and gitignore.match_file(rel_path + "/"):
                    dirs_to_remove.append(d)
            for d in dirs_to_remove:
                dirs.remove(d)
            for f in files:
                rel_path = os.path.join(rel_root, f) if rel_root else f
                if gitignore and gitignore.match_file(rel_path):
                    continue
                if self._match_pattern(rel_path, pattern):
                    matches.append(rel_path)
                    if len(matches) >= max_results:
                        break
            if len(matches) >= max_results:
```

```
                break
        matches.sort()
        if not matches:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"No files matching '{pattern}' in {abs_path}",
                is_error=False,
            )
        truncated = len(matches) > max_results
        display = matches[:max_results]
        content = "\n".join(display)
        if truncated:
            content += f"\n\n... ({len(matches) - max_results} more results)"
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Found {len(matches)} file(s):\n{content}",
            is_error=False,
        )
    def _match_pattern(self, path: str, pattern: str) -> bool:
        from fnmatch import fnmatch
        return fnmatch(path, pattern) or fnmatch(os.path.basename(path), pattern)
    def _load_gitignore(self, path: str) -> pathspec.PathSpec | None:
        gitignore_path = os.path.join(path, ".gitignore")
        if os.path.exists(gitignore_path):
            with open(gitignore_path) as f:
                return pathspec.PathSpec.from_lines("gitwildmatch", f)
        return None
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(GlobTool())
```

File: [171/252] openlaoke/tools/grep_tool.py

```
"""Grep tool - search file contents."""
from __future__ import annotations
import os
import re
from typing import Any
import pathspec
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class GrepInput(BaseModel):
    pattern: str = Field(description="Regex pattern to search for")
    path: str = Field(default=".", description="Directory to search in")
    glob: str | None = Field(
        default=None, description="File glob pattern to filter by (e.g., '*.py')"
    )
    output_mode: str = Field(
        default="content",
        description="Output mode: 'content' (default), 'files_with_matches', 'count'",
    )
    case_sensitive: bool = Field(default=False, description="Case-sensitive search")
    max_results: int = Field(default=100, description="Maximum number of results")
class GrepTool(Tool):
    """Search file contents using regular expressions."""
    name = "Grep"
    description = (
        "Search for a regex pattern in file contents across a directory. "
        "Supports glob filtering, case sensitivity options, and multiple output modes."
    )
    input_schema = GrepInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        pattern = kwargs.get("pattern", "")
        search_path = kwargs.get("path", ".")
        glob_pattern = kwargs.get("glob")
        output_mode = kwargs.get("output_mode", "content")
        case_sensitive = kwargs.get("case_sensitive", False)
        max_results = kwargs.get("max_results", 100)
        if not pattern:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: pattern is required",
                is_error=True,
            )
        try:
            flags = 0 if case_sensitive else re.IGNORECASE
            regex = re.compile(pattern, flags)
        except re.error as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Invalid regex pattern: {e}",
                is_error=True,
            )
        abs_path = self._resolve_path(search_path, ctx.app_state.get_cwd())
        if not os.path.isdir(abs_path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Directory not found: {abs_path}",
                is_error=True,
            )
```

```
    gitignore = self._load_gitignore(abs_path)
    results: list[tuple[str, int, str]] = []
    for root, dirs, files in os.walk(abs_path):
        rel_root = os.path.relpath(root, abs_path)
        if rel_root == ".":
            rel_root = ""
        dirs_to_remove = []
        for d in dirs:
            rel_path = os.path.join(rel_root, d) if rel_root else d
            if gitignore and gitignore.match_file(rel_path + "/"):
                dirs_to_remove.append(d)
        for d in dirs_to_remove:
            dirs.remove(d)
        for f in files:
            if glob_pattern:
                from fnmatch import fnmatch
                if not fnmatch(f, glob_pattern):
                    continue
            rel_path = os.path.join(rel_root, f) if rel_root else f
            if gitignore and gitignore.match_file(rel_path):
                continue
            file_path = os.path.join(root, f)
            try:
                with open(file_path, encoding="utf-8", errors="replace") as fh:
                    for line_num, line in enumerate(fh, 1):
                        if regex.search(line):
                            results.append((rel_path, line_num, line.rstrip()))
                            if len(results) >= max_results:
                                break
            except (PermissionError, OSError):
                continue
            if len(results) >= max_results:
                break
        if len(results) >= max_results:
            break
    if output_mode == "files_with_matches":
        files = sorted(set(r[0] for r in results))
        content = "\n".join(files) if files else "No matches found"
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Files with matches ({len(files)}):\n{content}",
        )
    if output_mode == "count":
        counts: dict[str, int] = {}
        for path, _, _ in results:
            counts[path] = counts.get(path, 0) + 1
        content = "\n".join(f"{path}: {count}" for path, count in sorted(counts.items()))
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Match counts:\n{content}",
        )
    if not results:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"No matches for '{pattern}' in {abs_path}",
            is_error=False,
        )
    lines = []
    for path, line_num, text in results:
        lines.append(f"{path}:{line_num}: {text}")
    content = "\n".join(lines)
    if len(results) >= max_results:
        content += f"\n\n... (truncated at {max_results} results)"
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
```

```
            content=f"Found {len(results)} match(es):\n{content}",
            is_error=False,
        )
    def _load_gitignore(self, path: str) -> pathspec.PathSpec | None:
        gitignore_path = os.path.join(path, ".gitignore")
        if os.path.exists(gitignore_path):
            with open(gitignore_path) as f:
                return pathspec.PathSpec.from_lines("gitwildmatch", f)
        return None
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(GrepTool())
```

File: [172/252] openlaoke/tools/ls_tool.py

```
"""ListDirectory tool - list directory contents."""
from __future__ import annotations
import os
from datetime import datetime
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class ListDirectoryInput(BaseModel):
    path: str | None = Field(
        default=None, description="Directory path to list (defaults to current working director
    y)"
    )
class ListDirectoryTool(Tool):
    """List contents of a directory."""
    name = "ListDirectory"
    description = (
        "Lists the contents of a directory. "
        "Shows files and subdirectories with their sizes and modification times. "
        "Defaults to the current working directory if no path is specified."
    )
    input_schema = ListDirectoryInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        path = kwargs.get("path")
        if path is None:
            path = ctx.app_state.get_cwd()
        path = os.path.expanduser(path)
        if not os.path.exists(path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Path does not exist: {path}",
                is_error=True,
            )
        if not os.path.isdir(path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Not a directory: {path}",
                is_error=True,
            )
        try:
            entries = self._list_directory(path)
            output_lines = [f"Contents of {path}:\n"]
            output_lines.append(f"{'Type':<6} {'Size':>12} {'Modified':<20} {'Name'}")
            output_lines.append("-" * 60)
            for entry in entries:
                type_str = "DIR" if entry["is_dir"] else "FILE"
                size_str = self._format_size(entry["size"]) if not entry["is_dir"] else "-"
                mod_str = entry["modified"][:19] if entry["modified"] else "-"
                name = entry["name"] + ("/" if entry["is_dir"] else "")
                output_lines.append(f"{type_str:<6} {size_str:>12} {mod_str:<20} {name}")
            output_lines.append(f"\n{len(entries)} items total")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="\n".join(output_lines),
                is_error=False,
            )
        except PermissionError:
```

```
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Permission denied: {path}",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error listing directory: {e}",
                is_error=True,
            )
    def _list_directory(self, path: str) -> list[dict[str, Any]]:
        entries: list[dict[str, Any]] = []
        with os.scandir(path) as scanner:
            for entry in scanner:
                try:
                    stat = entry.stat()
                    entries.append(
                        {
                            "name": entry.name,
                            "is_dir": entry.is_dir(),
                            "size": stat.st_size,
                            "modified": datetime.fromtimestamp(stat.st_mtime).isoformat(),
                        }
                    )
                except OSError:
                    entries.append(
                        {
                            "name": entry.name,
                            "is_dir": entry.is_dir(),
                            "size": 0,
                            "modified": "",
                        }
                    )
        entries.sort(key=lambda e: (not e["is_dir"], e["name"].lower()))
        return entries
    def _format_size(self, size: int) -> str:
        s: float = float(size)
        for unit in ["B", "KB", "MB", "GB", "TB"]:
            if s < 1024:
                return f"{s:.1f}{unit}"
            s /= 1024
        return f"{s:.1f}PB"
def register(registry: ToolRegistry) -> None:
    registry.register(ListDirectoryTool())
```

File: [173/252] openlaoke/tools/lsp_tool.py

```
"""LSP tool - Language Server Protocol integration."""
from __future__ import annotations
import asyncio
import json
import os
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class LSPInput(BaseModel):
    action: str = Field(
        description="LSP action: diagnostics, definition, references, symbols, hover, completio
    n"
    )
    file_path: str = Field(description="Path to the file to analyze")
    line: int | None = Field(default=None, description="Line number (1-indexed)")
    column: int | None = Field(default=None, description="Column number (1-indexed)")
class LSPTool(Tool):
    """Language Server Protocol integration for code intelligence."""
    name = "LSP"
    description = (
        "Interact with Language Server Protocol (LSP) for code intelligence. "
        "Supports: diagnostics (errors/warnings), definition (go to definition), "
        "references (find all references), symbols (document symbols), hover (type info), "
        "completion (auto-complete suggestions)."
    )
    input_schema = LSPInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    _servers: dict[str, Any] = {}
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action", "")
        file_path = kwargs.get("file_path", "")
        line = kwargs.get("line")
        column = kwargs.get("column")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        if not file_path:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: file_path is required",
                is_error=True,
            )
        abs_path = self._resolve_path(file_path, ctx.app_state.get_cwd())
        if not os.path.exists(abs_path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: File not found: {abs_path}",
                is_error=True,
            )
        language_id = self._detect_language(abs_path)
        if not language_id:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unsupported file type: {abs_path}",
                is_error=True,
```

```
        )
    try:
        if action == "diagnostics":
            return await self._get_diagnostics(ctx, abs_path, language_id)
        elif action == "definition":
            if line is None or column is None:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: line and column required for definition",
                    is_error=True,
                )
            return await self._get_definition(ctx, abs_path, line, column, language_id)
        elif action == "references":
            if line is None or column is None:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: line and column required for references",
                    is_error=True,
                )
            return await self._get_references(ctx, abs_path, line, column, language_id)
        elif action == "symbols":
            return await self._get_symbols(ctx, abs_path, language_id)
        elif action == "hover":
            if line is None or column is None:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: line and column required for hover",
                    is_error=True,
                )
            return await self._get_hover(ctx, abs_path, line, column, language_id)
        elif action == "completion":
            if line is None or column is None:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: line and column required for completion",
                    is_error=True,
                )
            return await self._get_completion(ctx, abs_path, line, column, language_id)
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}. Supported: diagnostics, definiti
on, references, symbols, hover, completion",
                is_error=True,
            )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"LSP error: {e}",
            is_error=True,
        )
def _resolve_path(self, path: str, cwd: str) -> str:
    if os.path.isabs(path):
        return os.path.normpath(path)
    return os.path.normpath(os.path.join(cwd, path))
def _detect_language(self, file_path: str) -> str | None:
    ext_map = {
        ".py": "python",
        ".js": "javascript",
        ".ts": "typescript",
        ".tsx": "typescriptreact",
        ".jsx": "javascriptreact",
        ".go": "go",
        ".rs": "rust",
        ".java": "java",
```

```
        ".c": "c",
        ".cpp": "cpp",
        ".h": "c",
        ".hpp": "cpp",
        ".rb": "ruby",
        ".php": "php",
        ".swift": "swift",
        ".kt": "kotlin",
        ".scala": "scala",
        ".lua": "lua",
        ".r": "r",
        ".sh": "bash",
        ".json": "json",
        ".yaml": "yaml",
        ".yml": "yaml",
        ".toml": "toml",
        ".md": "markdown",
        ".html": "html",
        ".css": "css",
        ".scss": "scss",
        ".less": "less",
        ".sql": "sql",
    }
    ext = os.path.splitext(file_path)[1].lower()
    return ext_map.get(ext)
def _get_lsp_command(self, language: str) -> list[str] | None:
    commands = {
        "python": ["pylsp"],
        "javascript": ["typescript-language-server", "--stdio"],
        "typescript": ["typescript-language-server", "--stdio"],
        "typescriptreact": ["typescript-language-server", "--stdio"],
        "javascriptreact": ["typescript-language-server", "--stdio"],
        "go": ["gopls"],
        "rust": ["rust-analyzer"],
        "c": ["clangd"],
        "cpp": ["clangd"],
    }
    return commands.get(language)
async def _get_diagnostics(
    self, ctx: ToolContext, file_path: str, language: str
) -> ToolResultBlock:
    try:
        result = await self._run_lint_tool(file_path, language)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=result,
            is_error=False,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Failed to get diagnostics: {e}",
            is_error=True,
        )
async def _run_lint_tool(self, file_path: str, language: str) -> str:
    if language == "python":
        proc = await asyncio.create_subprocess_exec(
            "python",
            "-m",
            "pylint",
            "--output-format=json",
            file_path,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
        )
```

```
        stdout, stderr = await proc.communicate()
        if stdout:
            try:
                issues = json.loads(stdout)
                if issues:
                    lines = []
                    for issue in issues:
                        lines.append(
                            f"Line {issue.get('line', '?')}: [{issue.get('symbol', 'error')
}] "
                            f"{issue.get('message', '')}"
                        )
                    return "\n".join(lines)
            except json.JSONDecodeError:
                pass
        return "No issues found"
    return f"LSP diagnostics not configured for {language}"
async def _get_definition(
    self, ctx: ToolContext, file_path: str, line: int, column: int, language: str
) -> ToolResultBlock:
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=f"Go to definition for {file_path}:{line}:{column}\n"
        f"Note: Full LSP definition requires a running language server. "
        f"Consider using grep to search for definitions.",
        is_error=False,
    )
async def _get_references(
    self, ctx: ToolContext, file_path: str, line: int, column: int, language: str
) -> ToolResultBlock:
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=f"Find references for {file_path}:{line}:{column}\n"
        f"Note: Full LSP references requires a running language server. "
        f"Consider using grep to search for references.",
        is_error=False,
    )
async def _get_symbols(
    self, ctx: ToolContext, file_path: str, language: str
) -> ToolResultBlock:
    symbols = self._extract_symbols_simple(file_path, language)
    if symbols:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Symbols in {file_path}:\n" + "\n".join(symbols),
            is_error=False,
        )
    return ToolResultBlock(
        tool_use_id=ctx.tool_use_id,
        content=f"No symbols found in {file_path}",
        is_error=False,
    )
def _extract_symbols_simple(self, file_path: str, language: str) -> list[str]:
    symbols = []
    try:
        with open(file_path, encoding="utf-8") as f:
            lines = f.readlines()
        if language == "python":
            for i, line in enumerate(lines, 1):
                stripped = line.strip()
                if stripped.startswith("def "):
                    name = stripped[4:].split("(")[0]
                    symbols.append(f"  Line {i}: function {name}")
                elif stripped.startswith("class "):
                    name = stripped[6:].split("(")[0].split(":")[0]
```

```
                        symbols.append(f"  Line {i}: class {name}")
                    elif stripped.startswith("async def "):
                        name = stripped[10:].split("(")[0]
                        symbols.append(f"  Line {i}: async function {name}")
            elif language in ("javascript", "typescript"):
                for i, line in enumerate(lines, 1):
                    stripped = line.strip()
                    if stripped.startswith("function "):
                        name = stripped[9:].split("(")[0]
                        symbols.append(f"  Line {i}: function {name}")
                    elif "const " in stripped or "let " in stripped or "var " in stripped:
                        if "=" in stripped:
                            name = stripped.split("=")[0].strip().split()[-1]
                            symbols.append(f"  Line {i}: variable {name}")
                    elif stripped.startswith("class "):
                        name = stripped[6:].split("{")[0].strip()
                        symbols.append(f"  Line {i}: class {name}")
        except Exception:
            pass
        return symbols
    async def _get_hover(
        self, ctx: ToolContext, file_path: str, line: int, column: int, language: str
    ) -> ToolResultBlock:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Hover info for {file_path}:{line}:{column}\n"
            f"Note: Full LSP hover requires a running language server.",
            is_error=False,
        )
    async def _get_completion(
        self, ctx: ToolContext, file_path: str, line: int, column: int, language: str
    ) -> ToolResultBlock:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Completions for {file_path}:{line}:{column}\n"
            f"Note: Full LSP completion requires a running language server.",
            is_error=False,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(LSPTool())
```

File: [174/252] openlaoke/tools/notebook_write_tool.py

```
"""NotebookWrite tool - write Jupyter notebooks."""
from __future__ import annotations
import json
import os
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class NotebookWriteInput(BaseModel):
    file_path: str = Field(description="Path to the notebook file")
    source: str = Field(description="Python source code for the cell")
    cell_type: str = Field(default="code", description="Cell type: 'code' or 'markdown'")
    cell_index: int = Field(default=-1, description="Index to insert the cell at (-1 for append
    )")
class NotebookWriteTool(Tool):
    """Write cells to Jupyter notebooks."""
    name = "NotebookWrite"
    description = (
        "Write a cell to a Jupyter notebook (.ipynb). "
        "Creates the notebook if it doesn't exist. Supports code and markdown cells."
    )
    input_schema = NotebookWriteInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        file_path = kwargs.get("file_path", "")
        source = kwargs.get("source", "")
        cell_type = kwargs.get("cell_type", "code")
        cell_index = kwargs.get("cell_index", -1)
        if not file_path:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: file_path is required",
                is_error=True,
            )
        abs_path = self._resolve_path(file_path, ctx.app_state.get_cwd())
        try:
            if os.path.exists(abs_path):
                with open(abs_path, encoding="utf-8") as f:
                    nb = json.load(f)
            else:
                nb = {
                    "cells": [],
                    "metadata": {
                        "kernelspec": {
                            "display_name": "Python 3",
                            "language": "python",
                            "name": "python3",
                        }
                    },
                    "nbformat": 4,
                    "nbformat_minor": 5,
                }
            cell = {
                "cell_type": cell_type,
                "source": source.split("\n"),
                "metadata": {},
            }
            if cell_type == "code":
                cell["outputs"] = []
```

```
                cell["execution_count"] = None
            if cell_index >= 0:
                nb["cells"].insert(cell_index, cell)
            else:
                nb["cells"].append(cell)
            with open(abs_path, "w", encoding="utf-8") as f:
                json.dump(nb, f, indent=1)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Added {cell_type} cell to {abs_path} (total cells: {len(nb['cells'])}
    )",
                is_error=False,
            )
        except json.JSONDecodeError:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: {abs_path} is not a valid notebook file",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error writing notebook: {e}",
                is_error=True,
            )
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(NotebookWriteTool())
```

File: [175/252] openlaoke/tools/plan_tool.py

```
"""Plan tool - Manage plan mode for structured task planning."""
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class PlanInput(BaseModel):
    action: str = Field(description="Plan action: enter, exit, update, status, append, prepend"
    )
    plan_content: str | None = Field(
        default=None, description="Plan content (required for enter/update/append/prepend actio
    ns)"
    )
class PlanTool(Tool):
    """Manage plan mode for structured task planning."""
    name = "Plan"
    description = (
        "Manage plan mode for structured task planning. "
        "Actions: 'enter' (start planning with content), 'exit' (finish planning), "
        "'update' (replace plan content), 'status' (show current plan), "
        "'append' (add to plan), 'prepend' (insert at start of plan). "
        "Plan mode helps organize complex tasks into actionable steps."
    )
    input_schema = PlanInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action", "").lower().strip()
        plan_content = kwargs.get("plan_content")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required. Use: enter, exit, update, status, append, p
    repend",
                is_error=True,
            )
        valid_actions = {"enter", "exit", "update", "status", "append", "prepend"}
        if action not in valid_actions:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}\nValid actions: {', '.join(sorted(val
    id_actions))}",
                is_error=True,
            )
        if action in {"enter", "update", "append", "prepend"} and not plan_content:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: plan_content is required for action '{action}'",
                is_error=True,
            )
        try:
            result = self._handle_action(ctx, action, plan_content)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=result,
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
```

```
            content=f"Plan error: {e}",
            is_error=True,
        )
    def _handle_action(self, ctx: ToolContext, action: str, plan_content: str | None) -> str:
        plan_state = self._get_plan_state(ctx)
        if action == "enter":
            plan_state["active"] = True
            plan_state["content"] = plan_content or ""
            plan_state["steps"] = self._parse_steps(plan_content or "")
            return f"Entered plan mode.\n\nPlan:\n{plan_content}"
        elif action == "exit":
            if not plan_state.get("active"):
                return "Not in plan mode."
            completed = plan_state.get("completed_steps", [])
            total = len(plan_state.get("steps", []))
            summary = f"Exited plan mode.\nCompleted {len(completed)}/{total} steps."
            plan_state["active"] = False
            return summary
        elif action == "update":
            if not plan_state.get("active"):
                return "Not in plan mode. Use 'enter' first."
            plan_state["content"] = plan_content or ""
            plan_state["steps"] = self._parse_steps(plan_content or "")
            return f"Plan updated.\n\nNew plan:\n{plan_content}"
        elif action == "status":
            if not plan_state.get("active"):
                return "Not in plan mode."
            content = plan_state.get("content", "No plan content.")
            steps = plan_state.get("steps", [])
            completed = plan_state.get("completed_steps", [])
            status_lines = [f"Plan Status: {len(completed)}/{len(steps)} steps completed", ""]
            status_lines.append("Plan:")
            status_lines.append(content)
            status_lines.append("")
            if steps:
                status_lines.append("Steps:")
                for i, step in enumerate(steps, 1):
                    check = "[x]" if i - 1 in completed else "[ ]"
                    status_lines.append(f"  {check} {i}. {step}")
            return "\n".join(status_lines)
        elif action == "append":
            if not plan_state.get("active"):
                return "Not in plan mode. Use 'enter' first."
            current = plan_state.get("content", "")
            new_content = current + "\n\n" + (plan_content or "")
            plan_state["content"] = new_content
            plan_state["steps"] = self._parse_steps(new_content)
            return f"Appended to plan.\n\nUpdated plan:\n{new_content}"
        elif action == "prepend":
            if not plan_state.get("active"):
                return "Not in plan mode. Use 'enter' first."
            current = plan_state.get("content", "")
            new_content = (plan_content or "") + "\n\n" + current
            plan_state["content"] = new_content
            plan_state["steps"] = self._parse_steps(new_content)
            return f"Prepended to plan.\n\nUpdated plan:\n{new_content}"
        return f"Unknown action: {action}"
    def _get_plan_state(self, ctx: ToolContext) -> dict[str, Any]:
        if not hasattr(ctx.app_state, "_plan_state"):
            default_state: dict[str, Any] = {
                "active": False,
                "content": "",
                "steps": [],
                "completed_steps": [],
            }
```

```
            ctx.app_state._plan_state = default_state
            return default_state
        state = ctx.app_state._plan_state
        return state if isinstance(state, dict) else {}
    def _parse_steps(self, content: str) -> list[str]:
        steps = []
        lines = content.split("\n")
        for line in lines:
            stripped = line.strip()
            if stripped.startswith("- ") or stripped.startswith("* "):
                steps.append(stripped[2:])
            elif stripped and stripped[0].isdigit() and ". " in stripped:
                idx = stripped.find(". ")
                if idx > 0:
                    steps.append(stripped[idx + 2 :])
        return steps
def register(registry: ToolRegistry) -> None:
    registry.register(PlanTool())
```

File: [176/252] openlaoke/tools/powershell_tool.py

```
"""PowerShell tool - execute PowerShell commands."""
from __future__ import annotations
import platform
import subprocess
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class PowerShellInput(BaseModel):
    command: str = Field(description="The PowerShell command to execute")
    description: str = Field(default="", description="Brief description of what this command do
    es")
    timeout: float | None = Field(default=None, description="Timeout in seconds")
class PowerShellTool(Tool):
    """Execute PowerShell commands."""
    name = "PowerShell"
    description = (
        "Execute PowerShell commands on Windows or PowerShell Core on Unix systems. "
        "Supports Windows scripting, system administration, and cross-platform compatibility. "
        "On Unix systems, requires PowerShell Core (pwsh) to be installed."
    )
    input_schema = PowerShellInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = False
    requires_approval = True
    def _get_powershell_cmd(self) -> str:
        """Get the PowerShell executable based on platform."""
        system = platform.system()
        if system == "Windows":
            return "powershell"
        return "pwsh"
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        command = kwargs.get("command", "")
        timeout = kwargs.get("timeout", 60.0)
        if not command.strip():
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: Empty command",
                is_error=True,
            )
        try:
            powershell = self._get_powershell_cmd()
            result = subprocess.run(
                [powershell, "-Command", command],
                capture_output=True,
                text=True,
                timeout=timeout,
                cwd=ctx.app_state.get_cwd(),
            )
            output = result.stdout
            if result.stderr:
                output += f"\nStderr:\n{result.stderr}"
            if result.returncode != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Command exited with code {result.returncode}\n\n{output}",
                    is_error=True,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
```

```
                content=output if output else "(command completed successfully with no output)"
    ,
                is_error=False,
            )
        except FileNotFoundError:
            system = platform.system()
            if system == "Windows":
                msg = "PowerShell not found"
            else:
                msg = "PowerShell Core (pwsh) not found. Install with: brew install powershell
    (macOS) or see https://aka.ms/powershell"
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: {msg}",
                is_error=True,
            )
        except subprocess.TimeoutExpired:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Command timed out after {timeout} seconds",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error executing PowerShell command: {e}",
                is_error=True,
            )
def register(registry: ToolRegistry) -> None:
    registry.register(PowerShellTool())
```

File: [177/252] openlaoke/tools/question_tool.py

```
"""Question tool - ask user questions interactively."""
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class QuestionOption(BaseModel):
    value: str = Field(description="The option value")
    label: str | None = Field(default=None, description="Optional display label for the option"
    )
class QuestionItem(BaseModel):
    question: str = Field(description="The question to ask")
    header: str = Field(default="", description="Optional header/title for the question group")
    options: list[QuestionOption] | None = Field(
        default=None, description="Optional list of choices"
    )
    multiple: bool = Field(default=False, description="Allow multiple selections")
class QuestionInput(BaseModel):
    questions: list[QuestionItem] = Field(description="List of questions to ask the user")
class QuestionTool(Tool):
    """Ask the user questions interactively."""
    name = "Question"
    description = (
        "Asks the user questions and returns their answers. "
        "Can ask single or multiple choice questions. "
        "Use this when you need user input or clarification."
    )
    input_schema = QuestionInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        questions_data = kwargs.get("questions", [])
        if not isinstance(questions_data, list):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: questions must be a list",
                is_error=True,
            )
        output_lines: list[str] = []
        output_lines.append("=== Questions for User ===\n")
        for i, q in enumerate(questions_data, 1):
            if isinstance(q, dict):
                question_text = q.get("question", "")
                header = q.get("header", "")
                options = q.get("options", [])
                multiple = q.get("multiple", False)
            else:
                question_text = getattr(q, "question", "")
                header = getattr(q, "header", "")
                options = getattr(q, "options", None) or []
                multiple = getattr(q, "multiple", False)
            if header:
                output_lines.append(f"[{header}]")
            if multiple:
                output_lines.append(f"{i}. {question_text} (select all that apply)")
            else:
                output_lines.append(f"{i}. {question_text}")
            if options:
                for j, opt in enumerate(options, 1):
```

```
                if isinstance(opt, dict):
                    value = opt.get("value", "")
                    label = opt.get("label") or value
                else:
                    value = getattr(opt, "value", "")
                    label = getattr(opt, "label", None) or value
                output_lines.append(f"   [{j}] {label}")
            else:
                output_lines.append("   (Enter your response)")
            output_lines.append("")
        output_lines.append("Please respond with your answers.")
        output_lines.append(
            "For multiple choice questions, provide the number(s) of your selection(s)."
        )
        output_lines.append("For open questions, provide your text response.")
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="\n".join(output_lines),
            is_error=False,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(QuestionTool())
```

File: [178/252] openlaoke/tools/read_tool.py

```
"""Read tool - read file contents."""
from __future__ import annotations
import os
from typing import Any
import chardet
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class ReadInput(BaseModel):
    file_path: str = Field(description="Path to the file to read")
    offset: int | None = Field(default=None, description="Start reading from this line (1-index
    ed)")
    limit: int | None = Field(default=None, description="Maximum number of lines to read")
class ReadTool(Tool):
    """Read the contents of a file."""
    name = "Read"
    description = (
        "Read the contents of a file. Supports text files, code files, and more. "
        "Use offset and limit to read specific ranges of large files."
    )
    input_schema = ReadInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        file_path = kwargs.get("file_path", "")
        offset = kwargs.get("offset")
        limit = kwargs.get("limit")
        if not file_path:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: file_path is required",
                is_error=True,
            )
        abs_path = self._resolve_path(file_path, ctx.app_state.get_cwd())
        if not os.path.exists(abs_path):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: File not found: {abs_path}",
                is_error=True,
            )
        if os.path.isdir(abs_path):
            entries = os.listdir(abs_path)
            entries.sort()
            dirs = [e + "/" for e in entries if os.path.isdir(os.path.join(abs_path, e))]
            files = [e for e in entries if not os.path.isdir(os.path.join(abs_path, e))]
            content = f"Directory listing for {abs_path}:\n"
            for d in dirs:
                content += f"  {d}\n"
            for f in files:
                content += f"  {f}\n"
            return ToolResultBlock(tool_use_id=ctx.tool_use_id, content=content)
        try:
            with open(abs_path, "rb") as file_handle:
                raw = file_handle.read()
            detected = chardet.detect(raw) or {}
            encoding = detected.get("encoding") or "utf-8"
            try:
                text = raw.decode(encoding)
            except (UnicodeDecodeError, LookupError):
                text = raw.decode("utf-8", errors="replace")
```

```
            lines = text.split("\n")
            if offset is not None:
                start = max(0, offset - 1)
                lines = lines[start:]
            if limit is not None:
                lines = lines[:limit]
            total_lines = len(text.split("\n"))
            shown_start = offset if offset else 1
            shown_end = shown_start + len(lines) - 1
            numbered = []
            for i, line in enumerate(lines):
                numbered.append(f"{shown_start + i}: {line}")
            content = "\n".join(numbered)
            if total_lines > len(lines):
                content += f"\n\n... ({total_lines - len(lines)} more lines, use offset={shown_
    end + 1} to continue)"
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=content,
                is_error=False,
            )
        except PermissionError:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Permission denied: {abs_path}",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error reading file: {e}",
                is_error=True,
            )
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(ReadTool())
```

File: [179/252] openlaoke/tools/reference_downloader.py

```
"""Reference downloader tool - download academic papers as PDFs."""
from __future__ import annotations
import re
from pathlib import Path
from typing import Any
import httpx
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class DownloadReferenceInput(BaseModel):
    source: str = Field(
        description="Source identifier (arXiv ID, DOI, or URL). Examples: 'arXiv:2301.12345', '
    10.1234/journal.2023.001', or full URL"
    )
    filename: str | None = Field(
        default=None,
        description="Optional custom filename (without .pdf extension). If not provided, will a
    uto-generate from source.",
    )
class ReferenceDownloader(Tool):
    """Download academic papers from various sources."""
    name = "DownloadReference"
    description = (
        "Download academic papers as PDFs from arXiv, DOI, or direct URLs. "
        "Automatically saves to 'pdf/' directory in working directory. "
        "Returns the path to the downloaded PDF."
    )
    input_schema = DownloadReferenceInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        source = kwargs.get("source", "")
        custom_filename = kwargs.get("filename")
        if not source:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: source is required (arXiv ID, DOI, or URL)",
                is_error=True,
            )
        pdf_dir = Path(ctx.app_state.get_cwd()) / "pdf"
        pdf_dir.mkdir(parents=True, exist_ok=True)
        try:
            pdf_url, suggested_name = await self._resolve_source(source)
            if not pdf_url:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Error: Could not resolve source: {source}",
                    is_error=True,
                )
            filename = custom_filename or suggested_name
            if not filename.endswith(".pdf"):
                filename += ".pdf"
            filepath = pdf_dir / filename
            async with httpx.AsyncClient(timeout=60.0, follow_redirects=True) as client:
                response = await client.get(pdf_url)
                response.raise_for_status()
                filepath.write_bytes(response.content)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
```

```
            content=f"Downloaded: {filepath}\nSource: {source}\nSize: {len(response.content
):,} bytes",
            is_error=False,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error downloading reference: {e}",
            is_error=True,
        )
async def _resolve_source(self, source: str) -> tuple[str | None, str]:
    """Resolve source to PDF URL and suggested filename."""
    if source.startswith("arXiv:") or re.match(r"\d{4}\.\d{4,5}", source):
        return self._resolve_arxiv(source)
    if source.startswith("10.") or "/" in source:
        return await self._resolve_doi(source)
    if source.startswith("http"):
        return self._resolve_url(source)
    return None, ""
def _resolve_arxiv(self, source: str) -> tuple[str, str]:
    """Resolve arXiv ID to PDF URL."""
    arxiv_id = source.replace("arXiv:", "").replace("arxiv:", "").strip()
    arxiv_id = re.sub(r"v\d+$", "", arxiv_id)
    pdf_url = f"https://arxiv.org/pdf/{arxiv_id}.pdf"
    filename = f"arxiv_{arxiv_id.replace('/', '_')}"
    return pdf_url, filename
async def _resolve_doi(self, source: str) -> tuple[str | None, str]:
    """Resolve DOI to PDF URL."""
    doi = source if source.startswith("10.") else None
    if not doi and "/" in source and not source.startswith("http"):
        doi = source
    if not doi:
        return None, ""
    try:
        async with httpx.AsyncClient(timeout=30.0, follow_redirects=True) as client:
            headers = {"Accept": "application/vnd.citationstyles.csl+json"}
            url = f"https://doi.org/{doi}"
            response = await client.get(url, headers=headers)
            if response.status_code == 200:
                data = response.json()
                title = data.get("title", "")
                if title:
                    safe_title = re.sub(r"[^\w\s-]", "", title)
                    safe_title = re.sub(r"\s+", "_", safe_title)[:50]
                    filename = f"doi_{safe_title}"
                else:
                    filename = f"doi_{doi.replace('/', '_')[:30]}"
                link = data.get("link", [])
                if link and isinstance(link, list):
                    for link_item in link:
                        if link_item.get("content-type") == "application/pdf":
                            return link_item.get("URL"), filename
                return f"https://doi.org/{doi}", filename
    except Exception:
        pass
    filename = f"doi_{doi.replace('/', '_')[:30]}"
    return f"https://doi.org/{doi}", filename
def _resolve_url(self, source: str) -> tuple[str, str]:
    """Resolve direct URL."""
    filename = source.split("/")[-1]
    if "." in filename:
        filename = filename.rsplit(".", 1)[0]
    filename = re.sub(r"[^\w-]", "", filename)[:50] or "downloaded_paper"
    if "arxiv.org" in source:
        match = re.search(r"(\d{4}\.\d{4,5})", source)
```

```
            if match:
                filename = f"arxiv_{match.group(1)}"
        return source, filename
class BatchDownloadInput(BaseModel):
    sources: list[str] = Field(description="List of sources to download (arXiv IDs, DOIs, or UR
    Ls)")
class BatchDownloadReferences(Tool):
    """Download multiple academic papers in batch."""
    name = "BatchDownloadReferences"
    description = (
        "Download multiple academic papers at once. "
        "Accepts list of arXiv IDs, DOIs, or URLs. "
        "Creates 'pdf/' directory and saves all papers there."
    )
    input_schema = BatchDownloadInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        sources = kwargs.get("sources", [])
        if not sources:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: sources list is required",
                is_error=True,
            )
        downloader = ReferenceDownloader()
        results = []
        for i, source in enumerate(sources, 1):
            result = await downloader.call(
                ToolContext(app_state=ctx.app_state, tool_use_id=f"{ctx.tool_use_id}_{i}"),
                source=source,
            )
            results.append(f"{i}. {source}:\n   {result.content}")
        successful = sum(1 for r in results if "Error" not in r)
        total = len(sources)
        summary = f"Downloaded {successful}/{total} papers\n\n" + "\n\n".join(results)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=summary,
            is_error=successful == 0,
        )
class SearchAndDownloadInput(BaseModel):
    query: str = Field(description="Search query for academic papers")
    max_results: int = Field(
        default=5, ge=1, le=20, description="Maximum number of papers to download"
    )
class SearchAndDownloadPapers(Tool):
    """Search for papers and download them."""
    name = "SearchAndDownloadPapers"
    description = (
        "Search for academic papers and automatically download them. "
        "Uses Semantic Scholar API to find relevant papers. "
        "Downloads PDFs to 'pdf/' directory."
    )
    input_schema = SearchAndDownloadInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        query = kwargs.get("query", "")
        max_results = kwargs.get("max_results", 5)
        if not query:
```

```
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: query is required",
            is_error=True,
        )
    try:
        papers = await self._search_papers(query, max_results)
        if not papers:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"No papers found for: {query}",
                is_error=False,
            )
        pdf_dir = Path(ctx.app_state.get_cwd()) / "pdf"
        pdf_dir.mkdir(parents=True, exist_ok=True)
        results = []
        for paper in papers:
            result = await self._download_paper(paper, pdf_dir)
            results.append(result)
        successful = sum(1 for r in results if r["success"])
        summary_lines = [
            f"Search: {query}",
            f"Found: {len(papers)} papers",
            f"Downloaded: {successful}/{len(papers)}",
            "",
            "Results:",
        ]
        for r in results:
            status = "[OK]" if r["success"] else "[X]"
            summary_lines.append(f"  {status} {r['title'][:60]}")
            if r.get("path"):
                summary_lines.append(f"    -> {r['path']}")
            if r.get("error"):
                summary_lines.append(f"    Error: {r['error']}")
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="\n".join(summary_lines),
            is_error=successful == 0,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error searching papers: {e}",
            is_error=True,
        )
async def _search_papers(self, query: str, limit: int) -> list[dict]:
    """Search papers using Semantic Scholar API."""
    url = "https://api.semanticscholar.org/graph/v1/paper/search"
    params = {
        "query": query,
        "limit": limit,
        "fields": "title,year,authors,openAccessPdf,externalIds",
    }
    async with httpx.AsyncClient(timeout=30.0) as client:
        response = await client.get(url, params=params)
        response.raise_for_status()
        data = response.json()
    papers = []
    for item in data.get("data", []):
        paper = {
            "title": item.get("title", "Unknown"),
            "year": item.get("year"),
            "authors": [a.get("name", "") for a in item.get("authors", [])],
            "external_ids": item.get("externalIds", {}),
        }
```

```
            oa_pdf = item.get("openAccessPdf")
            if oa_pdf:
                paper["pdf_url"] = oa_pdf.get("url")
            arxiv_id = item.get("externalIds", {}).get("ArXiv")
            if arxiv_id:
                paper["arxiv_id"] = arxiv_id
                paper["pdf_url"] = f"https://arxiv.org/pdf/{arxiv_id}.pdf"
            doi = item.get("externalIds", {}).get("DOI")
            if doi:
                paper["doi"] = doi
            papers.append(paper)
        return papers
    async def _download_paper(self, paper: dict, pdf_dir: Path) -> dict:
        """Download a single paper."""
        result = {
            "title": paper.get("title", "Unknown"),
            "success": False,
            "path": None,
            "error": None,
        }
        pdf_url = paper.get("pdf_url")
        if not pdf_url:
            result["error"] = "No PDF URL available"
            return result
        try:
            safe_title = re.sub(r"[^\w\s-]", "", paper["title"])
            safe_title = re.sub(r"\s+", "_", safe_title)[:50]
            year = paper.get("year", "")
            filename = f"{safe_title}_{year}.pdf" if year else f"{safe_title}.pdf"
            filepath = pdf_dir / filename
            async with httpx.AsyncClient(timeout=60.0, follow_redirects=True) as client:
                response = await client.get(pdf_url)
                response.raise_for_status()
                filepath.write_bytes(response.content)
            result["success"] = True
            result["path"] = str(filepath)
        except Exception as e:
            result["error"] = str(e)
        return result
def register(registry: ToolRegistry) -> None:
    registry.register(ReferenceDownloader())
    registry.register(BatchDownloadReferences())
    registry.register(SearchAndDownloadPapers())
```

File: [180/252] openlaoke/tools/register.py

```
"""Tool registration with lazy loading support."""
from __future__ import annotations
from collections.abc import Callable
from openlaoke.core.tool import Tool, ToolRegistry
def register_all_tools(registry: ToolRegistry) -> None:
    """Register all built-in tools with essential tools loaded immediately."""
    register_essential_tools(registry)
    register_deferred_tools(registry)
def register_essential_tools(registry: ToolRegistry) -> None:
    """Register essential tools that are always loaded."""
    from openlaoke.tools.bash_tool import BashTool
    from openlaoke.tools.edit_tool import EditTool
    from openlaoke.tools.glob_tool import GlobTool
    from openlaoke.tools.grep_tool import GrepTool
    from openlaoke.tools.read_tool import ReadTool
    from openlaoke.tools.write_tool import WriteTool
    registry.register(BashTool())
    registry.register(ReadTool())
    registry.register(WriteTool())
    registry.register(EditTool())
    registry.register(GlobTool())
    registry.register(GrepTool())
def register_deferred_tools(registry: ToolRegistry) -> None:
    """Register deferred tools with lazy loading."""
    deferred_map: dict[str, tuple[Callable[[], Tool], str, str]] = {
        "Agent": (
            lambda: __import__("openlaoke.tools.agent_tool", fromlist=["AgentTool"]).AgentTool(
    ),
            "Run a subagent for complex tasks",
            "subagent task delegation",
        ),
        "ApplyPatch": (
            lambda: __import__(
                "openlaoke.tools.apply_patch_tool", fromlist=["ApplyPatchTool"]
            ).ApplyPatchTool(),
            "Apply a patch to files",
            "patch apply diff",
        ),
        "Batch": (
            lambda: __import__("openlaoke.tools.batch_tool", fromlist=["BatchTool"]).BatchTool(
    ),
            "Execute multiple tool calls in parallel",
            "parallel batch concurrent",
        ),
        "Git": (
            lambda: __import__("openlaoke.tools.git_tool", fromlist=["GitTool"]).GitTool(),
            "Git operations",
            "git repository version control",
        ),
        "ListDirectory": (
            lambda: __import__(
                "openlaoke.tools.ls_tool", fromlist=["ListDirectoryTool"]
            ).ListDirectoryTool(),
            "List directory contents",
            "ls list directory",
        ),
        "LSP": (
            lambda: __import__("openlaoke.tools.lsp_tool", fromlist=["LSPTool"]).LSPTool(),
            "Language server protocol operations",
            "lsp language server ide",
        ),
```

```
    "NotebookWrite": (
        lambda: __import__(
            "openlaoke.tools.notebook_write_tool", fromlist=["NotebookWriteTool"]
        ).NotebookWriteTool(),
        "Edit Jupyter notebooks",
        "jupyter notebook cell",
    ),
    "Plan": (
        lambda: __import__("openlaoke.tools.plan_tool", fromlist=["PlanTool"]).PlanTool(),
        "Create execution plans",
        "plan strategy steps",
    ),
    "Question": (
        lambda: __import__(
            "openlaoke.tools.question_tool", fromlist=["QuestionTool"]
        ).QuestionTool(),
        "Ask user questions",
        "ask question user input",
    ),
    "TaskKill": (
        lambda: __import__(
            "openlaoke.tools.taskkill_tool", fromlist=["TaskKillTool"]
        ).TaskKillTool(),
        "Kill running tasks",
        "kill stop terminate task",
    ),
    "TodoWrite": (
        lambda: __import__(
            "openlaoke.tools.todo_tool", fromlist=["TodoWriteTool"]
        ).TodoWriteTool(),
        "Manage todo list",
        "todo task checklist",
    ),
    "WebFetch": (
        lambda: __import__(
            "openlaoke.tools.webfetch_tool", fromlist=["WebFetchTool"]
        ).WebFetchTool(),
        "Fetch web page content",
        "web fetch url http",
    ),
    "WebSearch": (
        lambda: __import__(
            "openlaoke.tools.websearch_tool", fromlist=["WebSearchTool"]
        ).WebSearchTool(),
        "Search the web",
        "web search duckduckgo",
    ),
    "Sleep": (
        lambda: __import__("openlaoke.tools.sleep_tool", fromlist=["SleepTool"]).SleepTool(
),
        "Pause execution for specified duration",
        "sleep wait delay pause",
    ),
    "Brief": (
        lambda: __import__("openlaoke.tools.brief_tool", fromlist=["BriefTool"]).BriefTool(
),
        "Enable brief response mode",
        "brief concise short response",
    ),
    "WebBrowser": (
        lambda: __import__(
            "openlaoke.tools.web_browser_tool", fromlist=["WebBrowserTool"]
        ).WebBrowserTool(),
        "Browser automation using Playwright",
        "browser playwright automation navigate click screenshot",
```

```
        ),
        "Tmux": (
            lambda: __import__("openlaoke.tools.tmux_tool", fromlist=["TmuxTool"]).TmuxTool(),
            "Manage tmux sessions",
            "tmux session terminal split pane",
        ),
        "PowerShell": (
            lambda: __import__(
                "openlaoke.tools.powershell_tool", fromlist=["PowerShellTool"]
            ).PowerShellTool(),
            "Execute PowerShell commands",
            "powershell windows scripting pwsh",
        ),
        "Cron": (
            lambda: __import__("openlaoke.tools.cron_tool", fromlist=["CronTool"]).CronTool(),
            "Manage cron jobs",
            "cron schedule job timer",
        ),
        "REPL": (
            lambda: __import__("openlaoke.tools.repl_tool", fromlist=["REPLTool"]).REPLTool(),
            "Interactive REPL environment",
            "repl interactive python node ruby",
        ),
        "ToolSearch": (
            lambda: __import__(
                "openlaoke.tools.tool_search_tool", fromlist=["ToolSearchTool"]
            ).ToolSearchTool(),
            "Search and discover available tools",
            "tool search suggest discover",
        ),
        "DownloadReference": (
            lambda: __import__(
                "openlaoke.tools.reference_downloader", fromlist=["ReferenceDownloader"]
            ).ReferenceDownloader(),
            "Download academic papers as PDFs",
            "reference paper pdf download arxiv doi",
        ),
        "BatchDownloadReferences": (
            lambda: __import__(
                "openlaoke.tools.reference_downloader", fromlist=["BatchDownloadReferences"]
            ).BatchDownloadReferences(),
            "Download multiple academic papers",
            "batch download papers references",
        ),
        "SearchAndDownloadPapers": (
            lambda: __import__(
                "openlaoke.tools.reference_downloader", fromlist=["SearchAndDownloadPapers"]
            ).SearchAndDownloadPapers(),
            "Search and download academic papers",
            "search download academic papers semantic scholar",
        ),
    }
    for name, (loader, description, search_hint) in deferred_map.items():
        registry.register_deferred_with_info(
            name=name,
            loader=loader,
            description=description,
            search_hint=search_hint,
        )
def get_tool_loader(tool_name: str) -> Callable[[], Tool] | None:
    """Get a loader function for a deferred tool."""
    loaders: dict[str, Callable[[], Tool]] = {
        "Agent": lambda: __import__(
            "openlaoke.tools.agent_tool", fromlist=["AgentTool"]
        ).AgentTool(),
```

```
    "ApplyPatch": lambda: __import__(
        "openlaoke.tools.apply_patch_tool", fromlist=["ApplyPatchTool"]
    ).ApplyPatchTool(),
    "Batch": lambda: __import__(
        "openlaoke.tools.batch_tool", fromlist=["BatchTool"]
    ).BatchTool(),
    "Git": lambda: __import__("openlaoke.tools.git_tool", fromlist=["GitTool"]).GitTool(),
    "ListDirectory": lambda: __import__(
        "openlaoke.tools.ls_tool", fromlist=["ListDirectoryTool"]
    ).ListDirectoryTool(),
    "LSP": lambda: __import__("openlaoke.tools.lsp_tool", fromlist=["LSPTool"]).LSPTool(),
    "NotebookWrite": lambda: __import__(
        "openlaoke.tools.notebook_write_tool", fromlist=["NotebookWriteTool"]
    ).NotebookWriteTool(),
    "Plan": lambda: __import__("openlaoke.tools.plan_tool", fromlist=["PlanTool"]).PlanTool
(),
    "Question": lambda: __import__(
        "openlaoke.tools.question_tool", fromlist=["QuestionTool"]
    ).QuestionTool(),
    "TaskKill": lambda: __import__(
        "openlaoke.tools.taskkill_tool", fromlist=["TaskKillTool"]
    ).TaskKillTool(),
    "TodoWrite": lambda: __import__(
        "openlaoke.tools.todo_tool", fromlist=["TodoWriteTool"]
    ).TodoWriteTool(),
    "WebFetch": lambda: __import__(
        "openlaoke.tools.webfetch_tool", fromlist=["WebFetchTool"]
    ).WebFetchTool(),
    "WebSearch": lambda: __import__(
        "openlaoke.tools.websearch_tool", fromlist=["WebSearchTool"]
    ).WebSearchTool(),
    "Sleep": lambda: __import__(
        "openlaoke.tools.sleep_tool", fromlist=["SleepTool"]
    ).SleepTool(),
    "Brief": lambda: __import__(
        "openlaoke.tools.brief_tool", fromlist=["BriefTool"]
    ).BriefTool(),
    "WebBrowser": lambda: __import__(
        "openlaoke.tools.web_browser_tool", fromlist=["WebBrowserTool"]
    ).WebBrowserTool(),
    "Tmux": lambda: __import__("openlaoke.tools.tmux_tool", fromlist=["TmuxTool"]).TmuxTool
(),
    "PowerShell": lambda: __import__(
        "openlaoke.tools.powershell_tool", fromlist=["PowerShellTool"]
    ).PowerShellTool(),
    "Cron": lambda: __import__("openlaoke.tools.cron_tool", fromlist=["CronTool"]).CronTool
(),
    "REPL": lambda: __import__("openlaoke.tools.repl_tool", fromlist=["REPLTool"]).REPLTool
(),
    "ToolSearch": lambda: __import__(
        "openlaoke.tools.tool_search_tool", fromlist=["ToolSearchTool"]
    ).ToolSearchTool(),
    "DownloadReference": lambda: __import__(
        "openlaoke.tools.reference_downloader", fromlist=["ReferenceDownloader"]
    ).ReferenceDownloader(),
    "BatchDownloadReferences": lambda: __import__(
        "openlaoke.tools.reference_downloader", fromlist=["BatchDownloadReferences"]
    ).BatchDownloadReferences(),
    "SearchAndDownloadPapers": lambda: __import__(
        "openlaoke.tools.reference_downloader", fromlist=["SearchAndDownloadPapers"]
    ).SearchAndDownloadPapers(),
}
return loaders.get(tool_name)
```

File: [181/252] openlaoke/tools/repl_tool.py

```
"""REPL tool - interactive REPL environment."""
from __future__ import annotations
import subprocess
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class REPLInput(BaseModel):
    action: Literal["start", "stop", "execute", "reset", "status"] = Field(
        description="REPL action to perform"
    )
    language: Literal["python", "node", "ruby", "irb", "bash"] = Field(
        default="python", description="Programming language for the REPL"
    )
    code: str | None = Field(default=None, description="Code to execute in the REPL")
    session_id: str | None = Field(
        default=None, description="Session identifier for state persistence"
    )
class REPLTool(Tool):
    """Interactive REPL environment for multiple languages."""
    name = "REPL"
    description = (
        "Manage interactive REPL sessions for various programming languages. "
        "Supports Python, Node.js, Ruby, IRB, and Bash. "
        "Maintains state across executions within a session. "
        "Useful for interactive coding, testing, and debugging."
    )
    input_schema = REPLInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = False
    requires_approval = False
    def __init__(self) -> None:
        super().__init__()
        self._sessions: dict[str, dict[str, Any]] = {}
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action")
        language = kwargs.get("language", "python")
        code = kwargs.get("code")
        session_id = kwargs.get("session_id", "default")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        if session_id not in self._sessions:
            self._sessions[session_id] = {
                "language": language,
                "variables": {},
                "history": [],
            }
        session = self._sessions[session_id]
        try:
            if action == "start":
                session["language"] = language
                session["variables"] = {}
                session["history"] = []
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Started {language} REPL session: {session_id}",
```

```
                is_error=False,
            )
        elif action == "stop":
            if session_id in self._sessions:
                del self._sessions[session_id]
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Stopped REPL session: {session_id}",
                is_error=False,
            )
        elif action == "execute":
            if not code:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: code is required for execute action",
                    is_error=True,
                )
            lang = session.get("language", language)
            if lang == "python":
                result = self._execute_python(code, session)
            elif lang == "node":
                result = self._execute_node(code, session)
            elif lang in ("ruby", "irb"):
                result = self._execute_ruby(code, session)
            elif lang == "bash":
                result = self._execute_bash(code, ctx)
            else:
                result = f"Unsupported language: {lang}"
            session["history"].append({"code": code, "result": result})
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=result,
                is_error=False,
            )
        elif action == "reset":
            session["variables"] = {}
            session["history"] = []
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Reset REPL session: {session_id}",
                is_error=False,
            )
        elif action == "status":
            history_count = len(session.get("history", []))
            lang = session.get("language", "unknown")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Session: {session_id}\nLanguage: {lang}\nHistory entries: {histor
y_count}",
                is_error=False,
            )
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}",
                is_error=True,
            )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error during REPL operation: {e}",
            is_error=True,
        )
def _execute_python(self, code: str, session: dict[str, Any]) -> str:
    """Execute Python code."""
```

```
    try:
        result = subprocess.run(
            ["python3", "-c", code],
            capture_output=True,
            text=True,
            timeout=30,
        )
        output = result.stdout
        if result.stderr:
            output += f"\nStderr:\n{result.stderr}"
        return output or "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: Execution timed out"
    except FileNotFoundError:
        return "Error: python3 not found"
def _execute_node(self, code: str, session: dict[str, Any]) -> str:
    """Execute Node.js code."""
    try:
        result = subprocess.run(
            ["node", "-e", code],
            capture_output=True,
            text=True,
            timeout=30,
        )
        output = result.stdout
        if result.stderr:
            output += f"\nStderr:\n{result.stderr}"
        return output or "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: Execution timed out"
    except FileNotFoundError:
        return "Error: node not found"
def _execute_ruby(self, code: str, session: dict[str, Any]) -> str:
    """Execute Ruby code."""
    try:
        result = subprocess.run(
            ["ruby", "-e", code],
            capture_output=True,
            text=True,
            timeout=30,
        )
        output = result.stdout
        if result.stderr:
            output += f"\nStderr:\n{result.stderr}"
        return output or "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: Execution timed out"
    except FileNotFoundError:
        return "Error: ruby not found"
def _execute_bash(self, code: str, ctx: ToolContext) -> str:
    """Execute Bash code."""
    try:
        result = subprocess.run(
            ["bash", "-c", code],
            capture_output=True,
            text=True,
            timeout=30,
            cwd=ctx.app_state.get_cwd(),
        )
        output = result.stdout
        if result.stderr:
            output += f"\nStderr:\n{result.stderr}"
        return output or "(no output)"
    except subprocess.TimeoutExpired:
        return "Error: Execution timed out"
```

```
def register(registry: ToolRegistry) -> None:
    registry.register(REPLTool())
```

File: [182/252] openlaoke/tools/sleep_tool.py

```
"""Sleep tool - pause execution for specified duration."""
from __future__ import annotations
import asyncio
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class SleepInput(BaseModel):
    seconds: float = Field(description="Number of seconds to sleep", ge=0.0, le=3600.0)
class SleepTool(Tool):
    """Pause execution for a specified duration."""
    name = "Sleep"
    description = (
        "Pause execution for a specified number of seconds. "
        "Use for delays, waiting for async operations, or debugging. "
        "Maximum duration is 3600 seconds (1 hour)."
    )
    input_schema = SleepInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        seconds = kwargs.get("seconds", 0.0)
        if seconds <= 0:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Sleep duration must be greater than 0",
                is_error=True,
            )
        if seconds > 3600:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Sleep duration cannot exceed 3600 seconds (1 hour)",
                is_error=True,
            )
        try:
            await asyncio.sleep(seconds)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Slept for {seconds} second(s)",
                is_error=False,
            )
        except asyncio.CancelledError:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Sleep was cancelled",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error during sleep: {e}",
                is_error=True,
            )
def register(registry: ToolRegistry) -> None:
    registry.register(SleepTool())
```

File: [183/252] openlaoke/tools/taskkill_tool.py

```
"""Taskkill tool - kill running tasks."""
from __future__ import annotations
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class TaskKillInput(BaseModel):
    task_id: str = Field(description="ID of the task to kill")
class TaskKillTool(Tool):
    """Kill a running task."""
    name = "Taskkill"
    description = "Kill a running task by its ID."
    input_schema = TaskKillInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        task_id = kwargs.get("task_id", "")
        if not task_id:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: task_id is required",
                is_error=True,
            )
        state = ctx.app_state.get_task(task_id)
        if not state:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Task not found: {task_id}",
                is_error=True,
            )
        if state.status in ("completed", "failed", "killed"):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Task {task_id} is already in terminal state: {state.status}",
                is_error=False,
            )
        from openlaoke.core.task import TaskManager
        task_mgr = TaskManager(ctx.app_state)
        task_mgr.kill_task(task_id)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Task {task_id} ({state.type.value}) has been killed.",
            is_error=False,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(TaskKillTool())
```

File: [184/252] openlaoke/tools/tmux_tool.py

```
"""Tmux tool - manage tmux sessions."""
from __future__ import annotations
import subprocess
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class TmuxInput(BaseModel):
    action: Literal["create", "attach", "detach", "list", "kill", "send", "split", "resize"] =
    (
        Field(description="Tmux action to perform")
    )
    session_name: str | None = Field(default=None, description="Name of the tmux session")
    command: str | None = Field(default=None, description="Command to send to session")
    direction: Literal["h", "v", None] = Field(
        default=None, description="Split direction: h (horizontal) or v (vertical)"
    )
    size: int | None = Field(default=None, description="Size for split pane")
class TmuxTool(Tool):
    """Manage tmux sessions for persistent terminal sessions."""
    name = "Tmux"
    description = (
        "Manage tmux sessions for persistent terminal sessions. "
        "Supports creating, attaching, detaching, listing, and killing sessions. "
        "Also supports sending commands, splitting panes, and resizing. "
        "Useful for long-running processes and terminal multiplexing."
    )
    input_schema = TmuxInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        try:
            if action == "create":
                session_name = kwargs.get("session_name")
                if not session_name:
                    return ToolResultBlock(
                        tool_use_id=ctx.tool_use_id,
                        content="Error: session_name is required for create action",
                        is_error=True,
                    )
                result = subprocess.run(
                    ["tmux", "new-session", "-d", "-s", session_name],
                    capture_output=True,
                    text=True,
                )
                if result.returncode != 0:
                    return ToolResultBlock(
                        tool_use_id=ctx.tool_use_id,
                        content=f"Error creating session: {result.stderr}",
                        is_error=True,
                    )
                return ToolResultBlock(
```

```
                tool_use_id=ctx.tool_use_id,
                content=f"Created tmux session: {session_name}",
                is_error=False,
            )
        elif action == "attach":
            session_name = kwargs.get("session_name")
            if not session_name:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: session_name is required for attach action",
                    is_error=True,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"To attach to session '{session_name}', run: tmux attach -t {sessi
on_name}",
                is_error=False,
            )
        elif action == "detach":
            result = subprocess.run(
                ["tmux", "detach"],
                capture_output=True,
                text=True,
            )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Detached from tmux session",
                is_error=False,
            )
        elif action == "list":
            result = subprocess.run(
                ["tmux", "list-sessions"],
                capture_output=True,
                text=True,
            )
            if result.returncode != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="No active tmux sessions",
                    is_error=False,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=result.stdout,
                is_error=False,
            )
        elif action == "kill":
            session_name = kwargs.get("session_name")
            if not session_name:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: session_name is required for kill action",
                    is_error=True,
                )
            result = subprocess.run(
                ["tmux", "kill-session", "-t", session_name],
                capture_output=True,
                text=True,
            )
            if result.returncode != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Error killing session: {result.stderr}",
                    is_error=True,
                )
```

```
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Killed tmux session: {session_name}",
            is_error=False,
        )
    elif action == "send":
        session_name = kwargs.get("session_name")
        command = kwargs.get("command")
        if not session_name or not command:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: session_name and command are required for send action",
                is_error=True,
            )
        result = subprocess.run(
            ["tmux", "send-keys", "-t", session_name, command, "Enter"],
            capture_output=True,
            text=True,
        )
        if result.returncode != 0:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error sending command: {result.stderr}",
                is_error=True,
            )
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Sent command to session {session_name}: {command}",
            is_error=False,
        )
    elif action == "split":
        session_name = kwargs.get("session_name")
        direction = kwargs.get("direction", "h")
        size = kwargs.get("size")
        if not session_name:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: session_name is required for split action",
                is_error=True,
            )
        cmd = ["tmux", "split-window", f"-{direction}"]
        if size:
            cmd.extend(["-l", str(size)])
        cmd.extend(["-t", session_name])
        result = subprocess.run(cmd, capture_output=True, text=True)
        if result.returncode != 0:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error splitting pane: {result.stderr}",
                is_error=True,
            )
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Split pane in session {session_name}",
            is_error=False,
        )
    elif action == "resize":
        session_name = kwargs.get("session_name")
        direction = kwargs.get("direction", "h")
        size = kwargs.get("size", 10)
        if not session_name:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: session_name is required for resize action",
                is_error=True,
```

```
                )
            result = subprocess.run(
                ["tmux", "resize-pane", f"-{direction}", str(size), "-t", session_name],
                capture_output=True,
                text=True,
            )
            if result.returncode != 0:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Error resizing pane: {result.stderr}",
                    is_error=True,
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Resized pane in session {session_name}",
                is_error=False,
            )
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}",
                is_error=True,
            )
    except FileNotFoundError:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content="Error: tmux is not installed. Install with: brew install tmux (macOS)
or apt install tmux (Linux)",
            is_error=True,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error during tmux operation: {e}",
            is_error=True,
        )
def register(registry: ToolRegistry) -> None:
    registry.register(TmuxTool())
```

File: [185/252] openlaoke/tools/todo_tool.py

```
"""TodoWrite tool - manage todo lists."""
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
@dataclass
class TodoItem:
    content: str
    status: Literal["pending", "in_progress", "completed", "cancelled"] = "pending"
    priority: Literal["high", "medium", "low"] = "medium"
    created_at: str = field(default_factory=lambda: datetime.now().isoformat())
class TodoInput(BaseModel):
    todos: list[dict[str, Any]] = Field(
        description="List of todo items. Each item has 'content' (required), 'status' (pending/
    in_progress/completed/cancelled), and 'priority' (high/medium/low)"
    )
class TodoWriteTool(Tool):
    """Manage a todo list for tracking tasks."""
    name = "TodoWrite"
    description = (
        "Manages a todo list for tracking tasks. "
        "Use this to track progress on complex multi-step tasks. "
        "Each todo item has content, status (pending/in_progress/completed/cancelled), and prio
    rity (high/medium/low). "
        "The todo list is persisted across conversations."
    )
    input_schema = TodoInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    def __init__(self) -> None:
        self._todos: list[TodoItem] = []
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        todos_data = kwargs.get("todos", [])
        if not isinstance(todos_data, list):
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: todos must be a list",
                is_error=True,
            )
        new_todos: list[TodoItem] = []
        for item in todos_data:
            if not isinstance(item, dict):
                continue
            content = item.get("content", "")
            if not content:
                continue
            status = item.get("status", "pending")
            if status not in ("pending", "in_progress", "completed", "cancelled"):
                status = "pending"
            priority = item.get("priority", "medium")
            if priority not in ("high", "medium", "low"):
                priority = "medium"
            new_todos.append(
                TodoItem(
                    content=content,
                    status=status,
```

```
                    priority=priority,
                )
            )
        self._todos = new_todos
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=self._render_todos(),
            is_error=False,
        )
    def _render_todos(self) -> str:
        if not self._todos:
            return "Todo list is empty"
        lines = ["Current todo list:\n"]
        priority_order = {"high": 0, "medium": 1, "low": 2}
        status_icon = {
            "pending": "o",
            "in_progress": "?",
            "completed": "*",
            "cancelled": "?",
        }
        sorted_todos = sorted(
            self._todos,
            key=lambda t: (priority_order.get(t.priority, 1), t.content),
        )
        for todo in sorted_todos:
            icon = status_icon.get(todo.status, "o")
            priority_marker = "!" if todo.priority == "high" else " "
            lines.append(f"  {icon} {priority_marker} {todo.content}")
            if todo.status == "in_progress":
                lines.append(f"      Status: {todo.status}")
        summary = {
            "pending": sum(1 for t in self._todos if t.status == "pending"),
            "in_progress": sum(1 for t in self._todos if t.status == "in_progress"),
            "completed": sum(1 for t in self._todos if t.status == "completed"),
            "cancelled": sum(1 for t in self._todos if t.status == "cancelled"),
        }
        lines.append(
            f"\nSummary: {summary['completed']} done, {summary['in_progress']} in progress, {su
    mmary['pending']} pending"
        )
        return "\n".join(lines)
def register(registry: ToolRegistry) -> None:
    registry.register(TodoWriteTool())
```

File: [186/252] openlaoke/tools/tool_search_tool.py

```
"""Tool search tool - search and discover available tools."""
from __future__ import annotations
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class ToolSearchInput(BaseModel):
    action: Literal["search", "list", "info", "suggest"] = Field(description="Tool search actio
    n")
    query: str | None = Field(default=None, description="Search query for tool name or descript
    ion")
    tool_name: str | None = Field(default=None, description="Specific tool name to get info abo
    ut")
    task: str | None = Field(default=None, description="Task description for tool suggestions")
class ToolSearchTool(Tool):
    """Search and discover available tools."""
    name = "ToolSearch"
    description = (
        "Search for available tools by name or description. "
        "List all registered tools, get detailed info about a specific tool, "
        "or get tool suggestions based on a task description. "
        "Supports lazy loading of tools for better performance."
    )
    input_schema = ToolSearchInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action")
        query = kwargs.get("query")
        tool_name = kwargs.get("tool_name")
        task = kwargs.get("task")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        try:
            if action == "search":
                if not query:
                    return ToolResultBlock(
                        tool_use_id=ctx.tool_use_id,
                        content="Error: query is required for search action",
                        is_error=True,
                    )
                registry = self._get_registry(ctx)
                tools = registry.search(query)
                if not tools:
                    return ToolResultBlock(
                        tool_use_id=ctx.tool_use_id,
                        content=f"No tools found matching: {query}",
                        is_error=False,
                    )
                results = []
                for tool in tools:
                    results.append(f"- {tool.name}: {tool.description[:100]}...")
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Found {len(tools)} tool(s):\n" + "\n".join(results),
```

```
                is_error=False,
            )
        elif action == "list":
            registry = self._get_registry(ctx)
            tools = registry.get_all()
            if not tools:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="No tools registered",
                    is_error=False,
                )
            results = []
            for tool in sorted(tools, key=lambda t: t.name):
                read_only = " [RO]" if tool.is_read_only else ""
                destructive = " [D]" if tool.is_destructive else ""
                approval = " [A]" if tool.requires_approval else ""
                results.append(
                    f"- {tool.name}{read_only}{destructive}{approval}: {tool.description[:8
0]}..."
                )
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Registered tools ({len(tools)}):\n" + "\n".join(results),
                is_error=False,
            )
        elif action == "info":
            if not tool_name:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: tool_name is required for info action",
                    is_error=True,
                )
            registry = self._get_registry(ctx)
            found_tool: Tool | None = registry.get(tool_name)
            if not found_tool:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"Tool not found: {tool_name}",
                    is_error=True,
                )
            info = [
                f"Name: {found_tool.name}",
                f"Description: {found_tool.description}",
                f"Read-only: {found_tool.is_read_only}",
                f"Destructive: {found_tool.is_destructive}",
                f"Concurrency-safe: {found_tool.is_concurrency_safe}",
                f"Requires approval: {found_tool.requires_approval}",
                "",
                "Input schema:",
            ]
            schema = found_tool.get_input_schema()
            if "properties" in schema:
                for prop_name, prop_info in schema["properties"].items():
                    required = prop_name in schema.get("required", [])
                    req_str = " (required)" if required else ""
                    desc = prop_info.get("description", "No description")
                    info.append(f"  - {prop_name}{req_str}: {desc}")
            else:
                info.append("  No input parameters")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="\n".join(info),
                is_error=False,
            )
        elif action == "suggest":
```

```
            if not task:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: task is required for suggest action",
                    is_error=True,
                )
            registry = self._get_registry(ctx)
            tools = registry.get_all()
            suggestions = self._suggest_tools(task, tools)
            if not suggestions:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"No tool suggestions for task: {task}",
                    is_error=False,
                )
            results = []
            for tool, reason in suggestions:
                results.append(f"- {tool.name}: {reason}")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Suggested tools for '{task}':\n" + "\n".join(results),
                is_error=False,
            )
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}",
                is_error=True,
            )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error during tool search: {e}",
            is_error=True,
        )
def _get_registry(self, ctx: ToolContext) -> ToolRegistry:
    """Get the tool registry from app state."""
    if hasattr(ctx.app_state, "tool_registry"):
        reg: ToolRegistry = ctx.app_state.tool_registry
        return reg
    return ToolRegistry()
def _suggest_tools(self, task: str, tools: list[Tool]) -> list[tuple[Tool, str]]:
    """Suggest tools based on task description."""
    task_lower = task.lower()
    suggestions: list[tuple[Tool, str]] = []
    keywords = {
        "read": ("Read", "Read file contents"),
        "write": ("Write", "Write content to files"),
        "edit": ("Edit", "Edit existing files"),
        "bash": ("Bash", "Execute shell commands"),
        "shell": ("Bash", "Execute shell commands"),
        "command": ("Bash", "Execute shell commands"),
        "git": ("Git", "Git version control operations"),
        "search": ("Grep", "Search file contents"),
        "grep": ("Grep", "Search file contents"),
        "find": ("Glob", "Find files by pattern"),
        "glob": ("Glob", "Find files by pattern"),
        "list": ("Ls", "List directory contents"),
        "directory": ("Ls", "List directory contents"),
        "fetch": ("WebFetch", "Fetch web content"),
        "url": ("WebFetch", "Fetch web content"),
        "web": ("WebFetch", "Fetch web content"),
        "sleep": ("Sleep", "Pause execution"),
        "wait": ("Sleep", "Pause execution"),
        "browser": ("WebBrowser", "Browser automation"),
```

```
            "tmux": ("Tmux", "Terminal session management"),
            "terminal": ("Tmux", "Terminal session management"),
            "powershell": ("PowerShell", "Execute PowerShell commands"),
            "cron": ("Cron", "Manage scheduled tasks"),
            "schedule": ("Cron", "Manage scheduled tasks"),
            "repl": ("REPL", "Interactive coding environment"),
            "todo": ("Todo", "Manage todo list"),
            "task": ("Todo", "Manage todo list"),
        }
        for keyword, (tool_name, reason) in keywords.items():
            if keyword in task_lower:
                for tool in tools:
                    if tool.name.lower() == tool_name.lower():
                        if tool not in [t for t, _ in suggestions]:
                            suggestions.append((tool, reason))
                        break
        return suggestions[:5]
def register(registry: ToolRegistry) -> None:
    registry.register(ToolSearchTool())
```

File: [187/252] openlaoke/tools/web_browser_tool.py

```
"""Web browser tool - browser automation using Playwright."""
from __future__ import annotations
from typing import Any, Literal
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class WebBrowserInput(BaseModel):
    action: Literal["navigate", "click", "type", "screenshot", "wait", "evaluate", "close"] = F
    ield(
        description="Browser action to perform"
    )
    url: str | None = Field(default=None, description="URL to navigate to")
    selector: str | None = Field(default=None, description="CSS selector for element")
    text: str | None = Field(default=None, description="Text to type or value to set")
    wait_time: float | None = Field(default=None, description="Time to wait in seconds")
    script: str | None = Field(default=None, description="JavaScript to evaluate")
    screenshot_path: str | None = Field(default=None, description="Path to save screenshot")
    headless: bool = Field(default=True, description="Run browser in headless mode")
class WebBrowserTool(Tool):
    """Browser automation using Playwright."""
    name = "WebBrowser"
    description = (
        "Automate web browser interactions using Playwright. "
        "Supports navigation, clicking, typing, screenshots, JavaScript execution, and more. "
        "Useful for testing, web scraping, and browser automation tasks."
    )
    input_schema = WebBrowserInput
    is_read_only = False
    is_destructive = False
    is_concurrency_safe = False
    requires_approval = True
    def __init__(self) -> None:
        super().__init__()
        self._browser: Any = None
        self._page: Any = None
        self._playwright: Any = None
    async def _ensure_browser(self, headless: bool = True) -> tuple[Any, Any]:
        """Ensure browser instance is running."""
        if self._page is None:
            try:
                from playwright.async_api import async_playwright
                self._playwright = await async_playwright().start()
                self._browser = await self._playwright.chromium.launch(headless=headless)
                self._page = await self._browser.new_page()
            except ImportError:
                raise RuntimeError(
                    "Playwright not installed. Install with: pip install playwright && playwrig
    ht install chromium"
                ) from None
        return self._browser, self._page
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        action = kwargs.get("action")
        if not action:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: action is required",
                is_error=True,
            )
        try:
            if action == "navigate":
                url = kwargs.get("url")
```

```
        if not url:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: url is required for navigate action",
                is_error=True,
            )
        headless = kwargs.get("headless", True)
        _, page = await self._ensure_browser(headless)
        await page.goto(url)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Navigated to {url}",
            is_error=False,
        )
    elif action == "click":
        selector = kwargs.get("selector")
        if not selector:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: selector is required for click action",
                is_error=True,
            )
        if self._page is None:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: No active page. Navigate to a URL first.",
                is_error=True,
            )
        await self._page.click(selector)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Clicked element: {selector}",
            is_error=False,
        )
    elif action == "type":
        selector = kwargs.get("selector")
        text = kwargs.get("text")
        if not selector or text is None:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: selector and text are required for type action",
                is_error=True,
            )
        if self._page is None:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: No active page. Navigate to a URL first.",
                is_error=True,
            )
        await self._page.fill(selector, text)
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Typed text into: {selector}",
            is_error=False,
        )
    elif action == "screenshot":
        if self._page is None:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: No active page. Navigate to a URL first.",
                is_error=True,
            )
        screenshot_path = kwargs.get("screenshot_path", "screenshot.png")
        await self._page.screenshot(path=screenshot_path)
        return ToolResultBlock(
```

```
                tool_use_id=ctx.tool_use_id,
                content=f"Screenshot saved to {screenshot_path}",
                is_error=False,
            )
        elif action == "wait":
            wait_time = kwargs.get("wait_time", 1.0)
            import asyncio
            await asyncio.sleep(wait_time)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Waited {wait_time} seconds",
                is_error=False,
            )
        elif action == "evaluate":
            script = kwargs.get("script")
            if not script:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: script is required for evaluate action",
                    is_error=True,
                )
            if self._page is None:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content="Error: No active page. Navigate to a URL first.",
                    is_error=True,
                )
            result = await self._page.evaluate(script)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=str(result),
                is_error=False,
            )
        elif action == "close":
            if self._browser:
                await self._browser.close()
                self._browser = None
                self._page = None
            if self._playwright:
                await self._playwright.stop()
                self._playwright = None
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Browser closed",
                is_error=False,
            )
        else:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Unknown action: {action}",
                is_error=True,
            )
    except ImportError as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error: {e}. Install with: pip install playwright && playwright instal
l chromium",
            is_error=True,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error during browser operation: {e}",
            is_error=True,
        )
```

```
    async def cleanup(self) -> None:
        """Clean up browser resources."""
        if self._browser:
            await self._browser.close()
        if self._playwright:
            await self._playwright.stop()
def register(registry: ToolRegistry) -> None:
    registry.register(WebBrowserTool())
```

File: [188/252] openlaoke/tools/webfetch_tool.py

```
"""WebFetch tool - fetch content from URLs."""
from __future__ import annotations
from typing import Any, Literal
import httpx
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class WebFetchInput(BaseModel):
    url: str = Field(description="The URL to fetch content from")
    format: Literal["markdown", "text", "html"] = Field(
        default="markdown",
        description="Output format: markdown (default), text, or html",
    )
    timeout: float | None = Field(default=None, description="Optional timeout in seconds (max 1
    20)")
class WebFetchTool(Tool):
    """Fetch content from a URL and convert to specified format."""
    name = "WebFetch"
    description = (
        "Fetches content from a specified URL and converts it to the requested format. "
        "Supports markdown (default), text, and html output formats. "
        "HTTP URLs are automatically upgraded to HTTPS. "
        "Results may be summarized for very large content."
    )
    input_schema = WebFetchInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        url = kwargs.get("url", "")
        output_format = kwargs.get("format", "markdown")
        timeout = min(kwargs.get("timeout") or 30.0, 120.0)
        if not url:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: URL is required",
                is_error=True,
            )
        if not url.startswith(("http://", "https://")):
            url = "https://" + url
        try:
            async with httpx.AsyncClient(timeout=timeout) as client:
                response = await client.get(url, follow_redirects=True)
                response.raise_for_status()
            content = response.text
            if output_format == "text":
                from html.parser import HTMLParser
                class TextExtractor(HTMLParser):
                    def __init__(self) -> None:
                        super().__init__()
                        self.text_parts: list[str] = []
                        self.skip = False
                    def handle_starttag(
                        self, tag: str, attrs: list[tuple[str, str | None]]
                    ) -> None:
                        if tag in ("script", "style", "nav", "footer", "header"):
                            self.skip = True
                    def handle_endtag(self, tag: str) -> None:
                        if tag in ("script", "style", "nav", "footer", "header"):
                            self.skip = False
```

```
                def handle_data(self, data: str) -> None:
                    if not self.skip:
                        self.text_parts.append(data)
            extractor = TextExtractor()
            extractor.feed(content)
            result = " ".join(" ".join(extractor.text_parts).split())
        elif output_format == "html":
            result = content
        else:
            result = self._html_to_markdown(content)
        max_len = 50000
        if len(result) > max_len:
            result = (
                result[:max_len] + f"\n\n... (truncated, {len(result) - max_len} chars omit
ted)"
            )
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=result,
            is_error=False,
        )
    except httpx.TimeoutException:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error: Request timed out after {timeout}s",
            is_error=True,
        )
    except httpx.HTTPStatusError as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error: HTTP {e.response.status_code} - {e.response.reason_phrase}",
            is_error=True,
        )
    except Exception as e:
        return ToolResultBlock(
            tool_use_id=ctx.tool_use_id,
            content=f"Error fetching URL: {e}",
            is_error=True,
        )
def _html_to_markdown(self, html: str) -> str:
    lines: list[str] = []
    in_pre = False
    in_code = False
    i = 0
    while i < len(html):
        if html[i : i + 4].lower() == "<pre":
            in_pre = True
            lines.append("\n```\n")
            i = html.find(">", i) + 1
            continue
        if html[i : i + 6].lower() == "</pre>" and in_pre:
            in_pre = False
            lines.append("\n```\n")
            i += 6
            continue
        if html[i : i + 5].lower() == "<code" and not in_code:
            in_code = True
            lines.append("`")
            i = html.find(">", i) + 1
            continue
        if html[i : i + 7].lower() == "</code>" and in_code:
            in_code = False
            lines.append("`")
            i += 7
            continue
```

```
if not in_pre:
    if html[i : i + 2].lower() == "<h":
        end_tag = html.find(">", i)
        if end_tag != -1:
            level = html[i + 2]
            if level.isdigit():
                lines.append("\n" + "#" * int(level) + " ")
                i = end_tag + 1
                continue
    if html[i : i + 4].lower() == "</h":
        lines.append("\n")
        i = html.find(">", i) + 1
        continue
    if html[i : i + 2].lower() == "<p":
        lines.append("\n")
        i = html.find(">", i) + 1
        continue
    if html[i : i + 4].lower() == "</p>":
        lines.append("\n")
        i += 4
        continue
    if html[i : i + 2].lower() == "<b" or html[i : i + 3].lower() == "<st":
        lines.append("**")
        i = html.find(">", i) + 1
        continue
    if html[i : i + 4].lower() == "</b>" or html[i : i + 5].lower() == "</st":
        lines.append("**")
        i += 4 if html[i : i + 4].lower() == "</b>" else 5
        continue
    if html[i : i + 2].lower() == "<i" or html[i : i + 3].lower() == "<em":
        lines.append("*")
        i = html.find(">", i) + 1
        continue
    if html[i : i + 4].lower() == "</i>" or html[i : i + 5].lower() == "</em":
        lines.append("*")
        i += 4 if html[i : i + 4].lower() == "</i>" else 5
        continue
    if html[i : i + 2].lower() == "<l":
        i = html.find(">", i) + 1
        lines.append("\n- ")
        continue
    if html[i : i + 2].lower() == "<a":
        href_start = html.find('href="', i)
        if href_start != -1:
            href_end = html.find('"', href_start + 6)
            href = html[href_start + 6 : href_end]
            text_start = html.find(">", href_end) + 1
            text_end = html.find("</a>", text_start)
            text = html[text_start:text_end].strip()
            lines.append(f"[{text}]({href})")
            i = text_end + 4
            continue
    if html[i : i + 2].lower() == "<b" and html[i : i + 3].lower() != "<br":
        lines.append("\n")
        i = html.find(">", i) + 1
        continue
if html[i] == "<":
    end = html.find(">", i)
    if end != -1:
        i = end + 1
        continue
if html[i] == "&":
    if html[i : i + 4] == "&lt;":
        lines.append("<")
        i += 4
```

```
                    continue
                if html[i : i + 4] == "&gt;":
                    lines.append(">")
                    i += 4
                    continue
                if html[i : i + 6] == " ":
                    lines.append(" ")
                    i += 6
                    continue
                if html[i : i + 5] == "&amp;":
                    lines.append("&")
                    i += 5
                    continue
            lines.append(html[i])
            i += 1
        return "".join(lines).strip()
def register(registry: ToolRegistry) -> None:
    registry.register(WebFetchTool())
```

File: [189/252] openlaoke/tools/websearch_tool.py

```
"""WebSearch tool - search the web using DuckDuckGo."""
from __future__ import annotations
from typing import Any
import httpx
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
class WebSearchInput(BaseModel):
    query: str = Field(description="The search query")
    num_results: int = Field(
        default=5, ge=1, le=20, description="Number of results to return (default 5, max 20)"
    )
class WebSearchTool(Tool):
    """Search the web using DuckDuckGo HTML search."""
    name = "WebSearch"
    description = (
        "Searches the web using DuckDuckGo. "
        "Returns a list of search results with titles, URLs, and snippets. "
        "Use this tool when you need to find information on the web."
    )
    input_schema = WebSearchInput
    is_read_only = True
    is_destructive = False
    is_concurrency_safe = True
    requires_approval = False
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        query = kwargs.get("query", "")
        num_results = kwargs.get("num_results", 5)
        if not query:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: Search query is required",
                is_error=True,
            )
        try:
            results = await self._search_duckduckgo(query, num_results)
            if not results:
                return ToolResultBlock(
                    tool_use_id=ctx.tool_use_id,
                    content=f"No results found for: {query}",
                    is_error=False,
                )
            output_lines = [f"Search results for: {query}\n"]
            for i, result in enumerate(results, 1):
                output_lines.append(f"{i}. {result['title']}")
                output_lines.append(f"   URL: {result['url']}")
                output_lines.append(f"   {result['snippet']}\n")
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="\n".join(output_lines),
                is_error=False,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error performing search: {e}",
                is_error=True,
            )
    async def _search_duckduckgo(self, query: str, num_results: int) -> list[dict[str, str]]:
        url = "https://html.duckduckgo.com/html/"
        params = {"q": query}
```

```
    # Use a simple client without proxy for web search
    async with httpx.AsyncClient(timeout=30.0, follow_redirects=True) as client:
        response = await client.post(url, data=params)
        response.raise_for_status()
    return self._parse_ddg_html(response.text, num_results)
def _parse_ddg_html(self, html: str, max_results: int) -> list[dict[str, str]]:
    results: list[dict[str, str]] = []
    i = 0
    while i < len(html) and len(results) < max_results:
        class_pos = html.find('class="result__', i)
        if class_pos == -1:
            break
        a_start = html.find("<a", class_pos)
        if a_start == -1:
            i = class_pos + 1
            continue
        href_start = html.find('href="', a_start)
        if href_start == -1:
            i = a_start + 1
            continue
        href_end = html.find('"', href_start + 6)
        if href_end == -1:
            i = href_start + 1
            continue
        url = html[href_start + 6 : href_end]
        if url.startswith("/l/?uddg="):
            from urllib.parse import unquote
            url = unquote(url[10:].split("&", 1)[0])
        text_start = html.find(">", href_end) + 1
        text_end = html.find("</a>", text_start)
        if text_end == -1:
            i = text_start
            continue
        title = html[text_start:text_end]
        title = self._strip_tags(title).strip()
        snippet = ""
        snippet_start = html.find('class="result__snippet"', text_end)
        if snippet_start != -1 and snippet_start < html.find(
            'class="result__', text_end + 1 if text_end + 1 < len(html) else len(html)
        ):
            snippet_a = html.find(">", snippet_start) + 1
            snippet_end = html.find("</a>", snippet_a)
            if snippet_end != -1:
                snippet = html[snippet_a:snippet_end]
                snippet = self._strip_tags(snippet).strip()
        if title and url and not url.startswith("javascript:"):
            results.append(
                {
                    "title": title,
                    "url": url,
                    "snippet": snippet or "(No description)",
                }
            )
        i = text_end + 4
    return results
def _strip_tags(self, text: str) -> str:
    result = []
    in_tag = False
    for char in text:
        if char == "<":
            in_tag = True
        elif char == ">":
            in_tag = False
        elif not in_tag:
            result.append(char)
```

```
        return "".join(result)
def register(registry: ToolRegistry) -> None:
    registry.register(WebSearchTool())
```

File: [190/252] openlaoke/tools/write_tool.py

```
"""Write tool - write file contents."""
from __future__ import annotations
import os
from typing import Any
from pydantic import BaseModel, Field
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import ToolResultBlock
from openlaoke.utils.file_history import track_file_edit
class WriteInput(BaseModel):
    file_path: str = Field(description="Path to the file to write")
    content: str = Field(description="Content to write to the file")
class WriteTool(Tool):
    """Write content to a file, creating it if it doesn't exist."""
    name = "Write"
    description = (
        "Write content to a file, creating the file if it doesn't exist. "
        "This will overwrite the entire file contents. "
        "IMPORTANT: Both file_path and content parameters are REQUIRED. "
        "Example: Write(file_path='/path/to/file.txt', content='file contents here')"
    )
    input_schema = WriteInput
    is_read_only = False
    is_destructive = True
    is_concurrency_safe = False
    requires_approval = True
    async def call(self, ctx: ToolContext, **kwargs: Any) -> ToolResultBlock:
        file_path = kwargs.get("file_path", "")
        content = kwargs.get("content", "")
        if not file_path:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content="Error: file_path is required",
                is_error=True,
            )
        abs_path = self._resolve_path(file_path, ctx.app_state.get_cwd())
        try:
            os.makedirs(os.path.dirname(abs_path) or ".", exist_ok=True)
            track_file_edit(abs_path, ctx.app_state.session_id)
            was_new = not os.path.exists(abs_path)
            with open(abs_path, "w", encoding="utf-8") as f:
                f.write(content)
            action = "Created" if was_new else "Updated"
            lines = content.count("\n") + 1
            chars = len(content)
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"{action} {abs_path} ({lines} lines, {chars} chars)",
                is_error=False,
            )
        except PermissionError:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error: Permission denied: {abs_path}",
                is_error=True,
            )
        except Exception as e:
            return ToolResultBlock(
                tool_use_id=ctx.tool_use_id,
                content=f"Error writing file: {e}",
                is_error=True,
            )
```

```
    def _resolve_path(self, path: str, cwd: str) -> str:
        if os.path.isabs(path):
            return os.path.normpath(path)
        return os.path.normpath(os.path.join(cwd, path))
def register(registry: ToolRegistry) -> None:
    registry.register(WriteTool())
```

File: [191/252] openlaoke/types/__init__.py

```
"""Type definitions for OpenLaoKe."""
from __future__ import annotations
from openlaoke.types.core_types import (
    AgentId,
    AssistantMessage,
    AttachmentMessage,
    CostInfo,
    HyperAutoMode,
    Message,
    MessageRole,
    PermissionMode,
    PermissionResult,
    ProgressMessage,
    SystemMessage,
    TaskId,
    TaskState,
    TaskStatus,
    TaskType,
    TokenUsage,
    ToolResultBlock,
    ToolUseBlock,
    UserMessage,
    ValidationResult,
    is_terminal_task_status,
)
from openlaoke.types.hooks import (
    FunctionHook,
    HookCommand,
    HookContext,
    HookResult,
    HookType,
)
from openlaoke.types.permissions import PermissionConfig
from openlaoke.types.providers import (
    CodingPlan,
    MultiProviderConfig,
    PlanConfig,
    ProviderConfig,
    ProviderType,
)
__all__ = [
    "AgentId",
    "AssistantMessage",
    "AttachmentMessage",
    "CodingPlan",
    "CostInfo",
    "FunctionHook",
    "HookCommand",
    "HookContext",
    "HookResult",
    "HookType",
    "HyperAutoMode",
    "Message",
    "MessageRole",
    "MultiProviderConfig",
    "PermissionConfig",
    "PermissionMode",
    "PermissionResult",
    "PlanConfig",
    "ProgressMessage",
    "ProviderConfig",
```

```
    "ProviderType",
    "SystemMessage",
    "TaskId",
    "TaskState",
    "TaskStatus",
    "TaskType",
    "TokenUsage",
    "ToolResultBlock",
    "ToolUseBlock",
    "UserMessage",
    "ValidationResult",
    "is_terminal_task_status",
]
```

File: [192/252] openlaoke/types/core_types.py

```
"""Shared type definitions for OpenLaoKe."""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any
from uuid import uuid4
class PermissionMode(StrEnum):
    """Permission modes for tool execution."""
    DEFAULT = "default"
    AUTO = "auto"
    BYPASS = "bypass"
class HyperAutoMode(StrEnum):
    """HyperAuto operation modes."""
    SEMI_AUTO = "semi_auto"
    FULL_AUTO = "full_auto"
    HYPER_AUTO = "hyper_auto"
class PermissionResult(StrEnum):
    """Result of a permission check."""
    ALLOW = "allow"
    DENY = "deny"
    ASK = "ask"
class TaskType(StrEnum):
    """Types of tasks that can be executed."""
    LOCAL_BASH = "local_bash"
    LOCAL_AGENT = "local_agent"
    REMOTE_AGENT = "remote_agent"
    IN_PROCESS_TEAMMATE = "in_process_teammate"
    LOCAL_WORKFLOW = "local_workflow"
    MONITOR_MCP = "monitor_mcp"
    DREAM = "dream"
class TaskStatus(StrEnum):
    """Lifecycle states for tasks."""
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    KILLED = "killed"
def is_terminal_task_status(status: TaskStatus) -> bool:
    """True when a task is in a terminal state and will not transition further."""
    return status in (TaskStatus.COMPLETED, TaskStatus.FAILED, TaskStatus.KILLED)
@dataclass
class TaskId:
    """Generates and parses task IDs with type prefixes."""
    PREFIXES: dict[str, str] = field(
        default_factory=lambda: {
            TaskType.LOCAL_BASH: "b",
            TaskType.LOCAL_AGENT: "a",
            TaskType.REMOTE_AGENT: "r",
            TaskType.IN_PROCESS_TEAMMATE: "t",
            TaskType.LOCAL_WORKFLOW: "w",
            TaskType.MONITOR_MCP: "m",
            TaskType.DREAM: "d",
        }
    )
    def generate(self, task_type: TaskType) -> str:
        prefix = self.PREFIXES.get(task_type, "x")
        return f"{prefix}_{uuid4().hex[:8]}"
    def parse_type(self, task_id: str) -> str | None:
        prefix = task_id.split("_")[0] if "_" in task_id else task_id[0]
        for task_type, p in self.PREFIXES.items():
            if p == prefix:
```

```
                return task_type
        return None
@dataclass
class AgentId:
    """Identifier for an agent/subagent instance."""
    id: str = field(default_factory=lambda: uuid4().hex[:12])
    parent_id: str | None = None
    name: str = "agent"
    def __str__(self) -> str:
        return self.id
@dataclass
class ToolUseBlock:
    """Represents a tool use request from the model."""
    id: str
    name: str
    input: dict[str, Any]
    def to_dict(self) -> dict[str, Any]:
        return {
            "type": "tool_use",
            "id": self.id,
            "name": self.name,
            "input": self.input,
        }
@dataclass
class ToolResultBlock:
    """Represents the result of a tool execution."""
    tool_use_id: str
    content: str | list[dict[str, Any]]
    is_error: bool = False
    def to_dict(self) -> dict[str, Any]:
        return {
            "type": "tool_result",
            "tool_use_id": self.tool_use_id,
            "content": self.content,
            "is_error": self.is_error,
        }
@dataclass
class ToolProgress:
    """Progress information for a running tool."""
    tool_use_id: str
    tool_name: str
    message: str
    percentage: float | None = None
    spinner: str = "dots"
class MessageRole(StrEnum):
    """Roles in the conversation message list."""
    USER = "user"
    ASSISTANT = "assistant"
    SYSTEM = "system"
@dataclass
class BaseMessage:
    """Base message in the conversation."""
    role: MessageRole
    timestamp: float = 0.0
    def to_dict(self) -> dict[str, Any]:
        return {"role": self.role.value, "timestamp": self.timestamp}
@dataclass
class UserMessage(BaseMessage):
    """Message from the user."""
    content: str = ""
    images: list[str] = field(default_factory=list)
    attachments: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            **super().to_dict(),
```

```
            "type": "user",
            "content": self.content,
            "images": self.images,
        }
@dataclass
class AssistantMessage(BaseMessage):
    """Message from the assistant (model response)."""
    content: str = ""
    tool_uses: list[ToolUseBlock] = field(default_factory=list)
    stop_reason: str | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            **super().to_dict(),
            "type": "assistant",
            "content": self.content,
            "tool_uses": [tu.to_dict() for tu in self.tool_uses],
            "stop_reason": self.stop_reason,
        }
@dataclass
class SystemMessage(BaseMessage):
    """System-generated message (e.g., tool output, status)."""
    content: str = ""
    subtype: str = "info"
    def to_dict(self) -> dict[str, Any]:
        return {
            **super().to_dict(),
            "type": "system",
            "content": self.content,
            "subtype": self.subtype,
        }
@dataclass
class ProgressMessage(BaseMessage):
    """Real-time progress update."""
    content: str = ""
    tool_use_id: str = ""
    tool_name: str = ""
    percentage: float | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            **super().to_dict(),
            "type": "progress",
            "content": self.content,
            "tool_use_id": self.tool_use_id,
            "tool_name": self.tool_name,
        }
@dataclass
class AttachmentMessage(BaseMessage):
    """Message with file attachments."""
    content: str = ""
    file_paths: list[str] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            **super().to_dict(),
            "type": "attachment",
            "content": self.content,
            "file_paths": self.file_paths,
        }
Message = UserMessage | AssistantMessage | SystemMessage | ProgressMessage | AttachmentMessage
@dataclass
class TaskState:
    """Runtime state of a task."""
    id: str
    type: TaskType
    status: TaskStatus = TaskStatus.PENDING
    description: str = ""
```

```
    tool_use_id: str | None = None
    start_time: float = 0.0
    end_time: float | None = None
    total_paused_ms: float = 0.0
    output_file: str = ""
    output_offset: int = 0
    notified: bool = False
    output: str = ""
    exit_code: int | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            "id": self.id,
            "type": self.type.value,
            "status": self.status.value,
            "description": self.description,
            "tool_use_id": self.tool_use_id,
            "start_time": self.start_time,
            "end_time": self.end_time,
            "total_paused_ms": self.total_paused_ms,
            "output_file": self.output_file,
            "output_offset": self.output_offset,
            "notified": self.notified,
            "exit_code": self.exit_code,
        }
@dataclass
class ValidationResult:
    """Result of input validation."""
    result: bool
    message: str = ""
    error_code: int = 0
@dataclass
class TokenUsage:
    """Token usage tracking for API calls."""
    input_tokens: int = 0
    output_tokens: int = 0
    cache_read_tokens: int = 0
    cache_creation_tokens: int = 0
    @property
    def total_tokens(self) -> int:
        return self.input_tokens + self.output_tokens
    def accumulate(self, other: TokenUsage) -> None:
        self.input_tokens += other.input_tokens
        self.output_tokens += other.output_tokens
        self.cache_read_tokens += other.cache_read_tokens
        self.cache_creation_tokens += other.cache_creation_tokens
    def to_dict(self) -> dict[str, int]:
        return {
            "input_tokens": self.input_tokens,
            "output_tokens": self.output_tokens,
            "cache_read_tokens": self.cache_read_tokens,
            "cache_creation_tokens": self.cache_creation_tokens,
        }
@dataclass
class CostInfo:
    """Cost tracking for API calls."""
    input_cost: float = 0.0
    output_cost: float = 0.0
    cache_read_cost: float = 0.0
    cache_creation_cost: float = 0.0
    @property
    def total_cost(self) -> float:
        return self.input_cost + self.output_cost + self.cache_read_cost + self.cache_creation_
    cost
```

File: [193/252] openlaoke/types/hooks.py

```
"""Hook type definitions for extensibility."""
from __future__ import annotations
from collections.abc import Callable
from dataclasses import dataclass, field
from enum import StrEnum
from typing import Any, Protocol
from openlaoke.types.core_types import Message
class HookType(StrEnum):
    """Types of hooks that can be registered."""
    PRE_TOOL_CALL = "pre_tool_call"
    POST_TOOL_CALL = "post_tool_call"
    PRE_API_CALL = "pre_api_call"
    POST_API_CALL = "post_api_call"
    ON_MESSAGE = "on_message"
    ON_SESSION_START = "on_session_start"
    ON_SESSION_END = "on_session_end"
    ON_TASK_START = "on_task_start"
    ON_TASK_END = "on_task_end"
    ON_ERROR = "on_error"
    PRE_COMPACT = "pre_compact"
    POST_COMPACT = "post_compact"
    STOP = "stop"
    STOP_FAILURE = "stop_failure"
    SUBAGENT_START = "subagent_start"
    SUBAGENT_STOP = "subagent_stop"
    TEAMMATE_IDLE = "teammate_idle"
    CONFIG_CHANGE = "config_change"
    CWD_CHANGED = "cwd_changed"
    FILE_CHANGED = "file_changed"
    PERMISSION_REQUEST = "permission_request"
    PERMISSION_DENIED = "permission_denied"
    NOTIFICATION = "notification"
    USER_PROMPT_SUBMIT = "user_prompt_submit"
@dataclass
class HookCommand:
    """Hook command configuration for shell-based hooks."""
    type: str = "command"
    command: str | None = None
    shell: str = "bash"
    timeout: int = 60000
    prompt: str | None = None
@dataclass
class FunctionHook:
    """Function-based hook with embedded callback."""
    type: str = "function"
    id: str | None = None
    timeout: int = 30000
    callback: Callable[[list[Message]], bool] | None = None
    error_message: str = ""
    status_message: str | None = None
@dataclass
class HookMatcher:
    """Hook matcher for filtering hooks by conditions."""
    event: HookType
    matchers: list[dict[str, Any]] = field(default_factory=list)
    command: HookCommand | None = None
    function: FunctionHook | None = None
    skill_root: str | None = None
@dataclass
class HookContext:
    """Context passed to hook handlers."""
```

```
    hook_type: HookType
    data: dict[str, Any] = field(default_factory=dict)
    session_id: str = ""
    tool_name: str = ""
    task_id: str = ""
    agent_id: str | None = None
    agent_type: str | None = None
    cwd: str = ""
    transcript_path: str = ""
@dataclass
class HookResult:
    """Result returned by a hook handler."""
    success: bool = True
    data: dict[str, Any] = field(default_factory=dict)
    error: str = ""
    should_abort: bool = False
    outcome: str = "success"
    blocking_error: str | None = None
    prevent_continuation: bool = False
    stop_reason: str | None = None
    permission_behavior: str | None = None
    additional_context: str | None = None
@dataclass
class StopHooksResult:
    """Result from Stop hooks execution."""
    should_stop: bool = False
    reason: str | None = None
    modified_response: str | None = None
@dataclass
class SessionHooksState:
    """Session-level hook state for tracking registered hooks."""
    hooks: dict[HookType, list[HookMatcher]] = field(default_factory=dict)
    results: list[HookResult] = field(default_factory=list)
    enabled: bool = True
class HookHandler(Protocol):
    """Protocol for hook handler functions."""
    async def __call__(self, ctx: HookContext) -> HookResult: ...
@dataclass
class HookRegistration:
    """A registered hook handler."""
    hook_type: HookType
    handler: HookHandler
    priority: int = 0
    name: str = ""
@dataclass
class PromptRequest:
    """Request to a prompt hook."""
    system_prompt: str
    messages: list[dict[str, Any]]
    tools: list[dict[str, Any]]
    metadata: dict[str, Any] = field(default_factory=dict)
@dataclass
class PromptResponse:
    """Response from a prompt hook."""
    system_prompt: str
    messages: list[dict[str, Any]]
    tools: list[dict[str, Any]]
    modified: bool = False
HOOK_EVENTS: list[HookType] = [
    HookType.PRE_TOOL_CALL,
    HookType.POST_TOOL_CALL,
    HookType.POST_API_CALL,
    HookType.PRE_API_CALL,
    HookType.ON_MESSAGE,
    HookType.ON_SESSION_START,
```

```
    HookType.ON_SESSION_END,
    HookType.ON_TASK_START,
    HookType.ON_TASK_END,
    HookType.ON_ERROR,
    HookType.PRE_COMPACT,
    HookType.POST_COMPACT,
    HookType.STOP,
    HookType.STOP_FAILURE,
    HookType.SUBAGENT_START,
    HookType.SUBAGENT_STOP,
    HookType.TEAMMATE_IDLE,
    HookType.CONFIG_CHANGE,
    HookType.CWD_CHANGED,
    HookType.FILE_CHANGED,
    HookType.PERMISSION_REQUEST,
    HookType.PERMISSION_DENIED,
    HookType.NOTIFICATION,
    HookType.USER_PROMPT_SUBMIT,
]
def is_hook_event(value: str) -> bool:
    """Check if a string is a valid hook event type."""
    return value in [h.value for h in HookType]
```

File: [194/252] openlaoke/types/permissions.py

```
"""Permission system for tool execution control."""
from __future__ import annotations
import fnmatch
from dataclasses import dataclass, field
from enum import StrEnum
from openlaoke.types.core_types import HyperAutoMode, PermissionMode, PermissionResult
class ClassifierMode(StrEnum):
    """Classifier operation modes for intelligent permission decisions."""
    FAST = "fast"
    AI = "ai"
    HYBRID = "hybrid"
class ConfidenceLevel(StrEnum):
    """Confidence levels for classification decisions."""
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
@dataclass
class ClassifierResult:
    """Result from the permission classifier."""
    decision: PermissionResult
    confidence: ConfidenceLevel
    reason: str
    safety_level: str | None = None
    matched_pattern: str | None = None
    ai_analysis: str | None = None
@dataclass
class PermissionRule:
    """A single permission rule matching tool names or patterns."""
    pattern: str
    action: PermissionResult
    description: str = ""
    def matches(self, tool_name: str) -> bool:
        return fnmatch.fnmatch(tool_name, self.pattern)
@dataclass
class HyperAutoConfig:
    """Configuration for HyperAuto mode."""
    mode: HyperAutoMode = HyperAutoMode.SEMI_AUTO
    enabled: bool = False
    max_iterations: int = 100
    timeout_seconds: int = 300
    auto_save: bool = True
    learning_enabled: bool = False
    history_limit: int = 50
@dataclass
class PermissionConfig:
    """Configuration for the permission system."""
    mode: PermissionMode = PermissionMode.DEFAULT
    always_allow_rules: list[PermissionRule] = field(default_factory=list)
    always_deny_rules: list[PermissionRule] = field(default_factory=list)
    always_ask_rules: list[PermissionRule] = field(default_factory=list)
    approved_tools: set[str] = field(default_factory=set)
    hyperauto_config: HyperAutoConfig = field(default_factory=HyperAutoConfig)
    @classmethod
    def defaults(cls) -> PermissionConfig:
        return cls(
            always_allow_rules=[
                PermissionRule("Read", PermissionResult.ALLOW, "Read-only file access"),
                PermissionRule("Glob", PermissionResult.ALLOW, "File pattern matching"),
                PermissionRule("Grep", PermissionResult.ALLOW, "Content search"),
                PermissionRule("Bash", PermissionResult.ALLOW, "Shell command execution"),
                PermissionRule("Write", PermissionResult.ALLOW, "File write operations"),
```

```
            PermissionRule("Edit", PermissionResult.ALLOW, "File edit operations"),
        ],
        always_deny_rules=[],
        always_ask_rules=[
            PermissionRule("Agent", PermissionResult.ASK, "Sub-agent spawning"),
        ],
    )
def check_tool(self, tool_name: str) -> PermissionResult:
    if self.mode == PermissionMode.BYPASS:
        return PermissionResult.ALLOW
    if self.mode == PermissionMode.AUTO:
        return PermissionResult.ALLOW
    if tool_name in self.approved_tools:
        return PermissionResult.ALLOW
    for rule in self.always_deny_rules:
        if rule.matches(tool_name):
            return PermissionResult.DENY
    for rule in self.always_allow_rules:
        if rule.matches(tool_name):
            return PermissionResult.ALLOW
    for rule in self.always_ask_rules:
        if rule.matches(tool_name):
            return PermissionResult.ASK
    return PermissionResult.ASK
def approve_tool(self, tool_name: str, remember: bool = False) -> None:
    if remember:
        self.approved_tools.add(tool_name)
def deny_tool(self, tool_name: str) -> None:
    self.approved_tools.discard(tool_name)
```

File: [195/252] openlaoke/types/providers.py

```
"""Provider types and configurations for multiple LLM backends."""
from __future__ import annotations
from dataclasses import dataclass, field
from enum import StrEnum
class ProviderType(StrEnum):
    """Supported LLM providers."""
    ANTHROPIC = "anthropic"
    OPENAI = "openai"
    OPENAI_COMPATIBLE = "openai_compatible"
    OLLAMA = "ollama"
    LM_STUDIO = "lm_studio"
    MINIMAX = "minimax"
    ALIYUN_CODING_PLAN = "aliyun_coding_plan"
    AZURE_OPENAI = "azure_openai"
    GOOGLE = "google"
    GOOGLE_VERTEX = "google_vertex"
    AWS_BEDROCK = "aws_bedrock"
    XAI = "xai"
    MISTRAL = "mistral"
    GROQ = "groq"
    CEREBRAS = "cerebras"
    COHERE = "cohere"
    DEEPINFRA = "deepinfra"
    TOGETHERAI = "togetherai"
    PERPLEXITY = "perplexity"
    OPENROUTER = "openrouter"
    GITHUB_COPILOT = "github_copilot"
    OPENCODE = "opencode"
    CUSTOM = "custom"
@dataclass
class ProviderConfig:
    """Configuration for a single provider."""
    provider_type: ProviderType
    api_key: str = ""
    base_url: str = ""
    default_model: str = ""
    models: list[str] = field(default_factory=list)
    enabled: bool = True
    is_local: bool = False
    def is_configured(self) -> bool:
        if self.is_local:
            return True
        if self.provider_type == ProviderType.OPENCODE:
            return True
        return bool(self.api_key and self.api_key != "none")
    def get_default_model(self) -> str:
        if self.default_model:
            return self.default_model
        return self.models[0] if self.models else ""
@dataclass
class MultiProviderConfig:
    """Configuration for all providers."""
    providers: dict[str, ProviderConfig] = field(default_factory=dict)
    active_provider: str = "ollama"
    active_model: str = ""
    def get_active_provider(self) -> ProviderConfig | None:
        return self.providers.get(self.active_provider)
    def get_active_model(self) -> str:
        if self.active_model:
            return self.active_model
        provider = self.get_active_provider()
```

```
    if provider:
        return provider.get_default_model()
    return ""
def is_configured(self) -> bool:
    provider = self.get_active_provider()
    return provider is not None and provider.is_configured()
def set_active_provider(self, name: str) -> bool:
    if name in self.providers:
        self.active_provider = name
        return True
    return False
def list_available_providers(self) -> list[str]:
    return [name for name, p in self.providers.items() if p.is_configured()]
@classmethod
def defaults(cls) -> MultiProviderConfig:
    return cls(
        providers={
            "ollama": ProviderConfig(
                provider_type=ProviderType.OLLAMA,
                base_url="http://localhost:11434/v1",
                default_model="gemma4:e2b",
                models=[
                    "gemma4:e2b",
                    "gemma4:e4b",
                    "gemma3:1b",
                    "qwen3.5:0.8B",
                    "llama3.2",
                    "llama3.1",
                    "codellama",
                    "deepseek-coder-v2",
                    "qwen2.5-coder",
                ],
                is_local=True,
            ),
            "lm_studio": ProviderConfig(
                provider_type=ProviderType.LM_STUDIO,
                base_url="http://localhost:1234/v1",
                default_model="local-model",
                models=["local-model"],
                is_local=True,
            ),
            "anthropic": ProviderConfig(
                provider_type=ProviderType.ANTHROPIC,
                base_url="https://api.anthropic.com",
                default_model="claude-sonnet-4-20250514",
                models=[
                    "claude-sonnet-4-20250514",
                    "claude-opus-4-20250514",
                    "claude-3-5-sonnet-20241022",
                    "claude-3-5-haiku-20241022",
                ],
            ),
            "openai": ProviderConfig(
                provider_type=ProviderType.OPENAI,
                base_url="https://api.openai.com/v1",
                default_model="gpt-4o",
                models=[
                    "gpt-4o",
                    "gpt-4o-mini",
                    "gpt-4-turbo",
                    "o1-preview",
                    "o1-mini",
                ],
            ),
            "minimax": ProviderConfig(
```

```
        provider_type=ProviderType.MINIMAX,
        base_url="https://api.minimaxi.com/v1",
        default_model="MiniMax-M2.7-highspeed",
        models=[
            "MiniMax-M2.7-highspeed",
            "MiniMax-M2.7",
            "MiniMax-M2.5-highspeed",
            "MiniMax-M2.5",
            "MiniMax-M2.1-highspeed",
            "MiniMax-M2.1",
            "MiniMax-M2",
        ],
    ),
    "aliyun_coding_plan": ProviderConfig(
        provider_type=ProviderType.ALIYUN_CODING_PLAN,
        base_url="https://coding.dashscope.aliyuncs.com/v1",
        default_model="qwen3.5-plus",
        models=[
            "qwen3.5-plus",
            "kimi-k2.5",
            "glm-5",
            "MiniMax-M2.5",
            "qwen3-max-2026-01-23",
            "qwen3-coder-next",
            "qwen3-coder-plus",
            "glm-4.7",
        ],
    ),
    "openai_compatible": ProviderConfig(
        provider_type=ProviderType.OPENAI_COMPATIBLE,
        base_url="",
        default_model="",
        models=[],
    ),
    "azure_openai": ProviderConfig(
        provider_type=ProviderType.AZURE_OPENAI,
        base_url="",
        default_model="gpt-4o",
        models=[
            "gpt-4o",
            "gpt-4o-mini",
            "gpt-4-turbo",
            "gpt-35-turbo",
        ],
    ),
    "google": ProviderConfig(
        provider_type=ProviderType.GOOGLE,
        base_url="https://generativelanguage.googleapis.com/v1beta",
        default_model="gemini-2.0-flash",
        models=[
            "gemini-2.0-flash",
            "gemini-2.0-pro",
            "gemini-1.5-flash",
            "gemini-1.5-pro",
        ],
    ),
    "google_vertex": ProviderConfig(
        provider_type=ProviderType.GOOGLE_VERTEX,
        base_url="",
        default_model="gemini-2.0-flash",
        models=[
            "gemini-2.0-flash",
            "gemini-2.0-pro",
            "gemini-1.5-flash",
            "gemini-1.5-pro",
```

```
    ],
),
"aws_bedrock": ProviderConfig(
    provider_type=ProviderType.AWS_BEDROCK,
    base_url="",
    default_model="anthropic.claude-3-5-sonnet-20241022-v2:0",
    models=[
        "anthropic.claude-3-5-sonnet-20241022-v2:0",
        "anthropic.claude-3-5-haiku-20241022-v1:0",
        "anthropic.claude-3-opus-20240229-v1:0",
        "meta.llama3-1-405b-instruct-v1:0",
        "meta.llama3-1-70b-instruct-v1:0",
        "amazon.nova-pro-v1:0",
        "amazon.nova-lite-v1:0",
    ],
),
"xai": ProviderConfig(
    provider_type=ProviderType.XAI,
    base_url="https://api.x.ai/v1",
    default_model="grok-2-latest",
    models=[
        "grok-2-latest",
        "grok-2-1212",
        "grok-beta",
        "grok-vision-beta",
    ],
),
"mistral": ProviderConfig(
    provider_type=ProviderType.MISTRAL,
    base_url="https://api.mistral.ai/v1",
    default_model="mistral-large-latest",
    models=[
        "mistral-large-latest",
        "mistral-small-latest",
        "codestral-latest",
        "open-mistral-nemo",
        "open-codestral-mamba",
    ],
),
"groq": ProviderConfig(
    provider_type=ProviderType.GROQ,
    base_url="https://api.groq.com/openai/v1",
    default_model="llama-3.3-70b-versatile",
    models=[
        "llama-3.3-70b-versatile",
        "llama-3.1-8b-instant",
        "llama-3.2-1b-preview",
        "llama-3.2-3b-preview",
        "mixtral-8x7b-32768",
        "gemma2-9b-it",
    ],
),
"cerebras": ProviderConfig(
    provider_type=ProviderType.CEREBRAS,
    base_url="https://api.cerebras.ai/v1",
    default_model="llama-3.3-70b",
    models=[
        "llama-3.3-70b",
        "llama-3.1-8b",
        "llama-3.1-70b",
    ],
),
"cohere": ProviderConfig(
    provider_type=ProviderType.COHERE,
    base_url="https://api.cohere.ai/v2",
```

```
        default_model="command-r-plus",
        models=[
            "command-r-plus",
            "command-r",
            "command",
            "command-light",
        ],
    ),
    "deepinfra": ProviderConfig(
        provider_type=ProviderType.DEEPINFRA,
        base_url="https://api.deepinfra.com/v1/openai",
        default_model="meta-llama/Llama-3.3-70B-Instruct",
        models=[
            "meta-llama/Llama-3.3-70B-Instruct",
            "meta-llama/Llama-3.1-8B-Instruct",
            "meta-llama/Llama-3.1-70B-Instruct",
            "mistralai/Mistral-Small-24B-Instruct-2501",
        ],
    ),
    "togetherai": ProviderConfig(
        provider_type=ProviderType.TOGETHERAI,
        base_url="https://api.together.xyz/v1",
        default_model="meta-llama/Llama-3.3-70B-Instruct-Turbo",
        models=[
            "meta-llama/Llama-3.3-70B-Instruct-Turbo",
            "meta-llama/Llama-3.1-8B-Instruct-Turbo",
            "mistralai/Mistral-7B-Instruct-v0.3",
            "Qwen/Qwen2.5-72B-Instruct-Turbo",
        ],
    ),
    "perplexity": ProviderConfig(
        provider_type=ProviderType.PERPLEXITY,
        base_url="https://api.perplexity.ai",
        default_model="llama-3.1-sonar-large-128k-online",
        models=[
            "llama-3.1-sonar-large-128k-online",
            "llama-3.1-sonar-small-128k-online",
            "llama-3.1-sonar-large-128k-chat",
            "llama-3.1-sonar-small-128k-chat",
        ],
    ),
    "openrouter": ProviderConfig(
        provider_type=ProviderType.OPENROUTER,
        base_url="https://openrouter.ai/api/v1",
        default_model="anthropic/claude-3.5-sonnet",
        models=[
            "anthropic/claude-3.5-sonnet",
            "anthropic/claude-3-opus",
            "openai/gpt-4o",
            "openai/gpt-4-turbo",
            "google/gemini-pro",
            "meta-llama/llama-3.1-70b-instruct",
        ],
    ),
    "github_copilot": ProviderConfig(
        provider_type=ProviderType.GITHUB_COPILOT,
        base_url="https://api.githubcopilot.com",
        default_model="gpt-4o",
        models=[
            "gpt-4o",
            "gpt-4o-mini",
            "gpt-4-turbo",
            "o1-preview",
            "o1-mini",
        ],
```

```
                ),
                "opencode": ProviderConfig(
                    provider_type=ProviderType.OPENCODE,
                    base_url="https://opencode.ai/zen/v1",
                    default_model="big-pickle",
                    models=[
                        "big-pickle",
                        "mimo-v2-flash-free",
                        "minimax-m2.1-free",
                        "mimo-v2-omni-free",
                        "qwen3.6-plus-free",
                        "grok-code",
                        "kimi-k2.5-free",
                        "glm-5-free",
                        "gpt-5-nano",
                        "nemotron-3-super-free",
                        "minimax-m2.5-free",
                        "trinity-large-preview-free",
                        "glm-4.7-free",
                        "mimo-v2-pro-free",
                    ],
                ),
            },
            active_provider="ollama",
        )
@dataclass
class CodingPlan:
    """Coding plan configuration for opencode-style plans."""
    name: str
    description: str
    enabled: bool = True
    max_iterations: int = 50
    auto_approve: bool = False
    tools: list[str] = field(default_factory=list)
@dataclass
class PlanConfig:
    """Configuration for coding plans."""
    plans: dict[str, CodingPlan] = field(default_factory=dict)
    active_plan: str = "default"
    @classmethod
    def defaults(cls) -> PlanConfig:
        return cls(
            plans={
                "default": CodingPlan(
                    name="default",
                    description="Standard coding assistant mode",
                    max_iterations=50,
                    tools=["Bash", "Read", "Write", "Edit", "Glob", "Grep", "Agent"],
                ),
                "quick": CodingPlan(
                    name="quick",
                    description="Quick mode with auto-approve for fast iterations",
                    max_iterations=20,
                    auto_approve=True,
                    tools=["Read", "Write", "Edit", "Bash"],
                ),
                "explorer": CodingPlan(
                    name="explorer",
                    description="Read-only exploration mode",
                    max_iterations=30,
                    tools=["Read", "Glob", "Grep", "Bash"],
                ),
                "architect": CodingPlan(
                    name="architect",
                    description="Architecture planning mode",
```

```
                max_iterations=100,
                tools=["Read", "Glob", "Grep", "Agent"],
            ),
        },
        active_plan="default",
    )
def get_active_plan(self) -> CodingPlan:
    return self.plans.get(self.active_plan, self.plans["default"])
```

File: [196/252] openlaoke/utils/__init__.py

```
"""Utility modules for OpenLaoKe."""
```

File: [197/252] openlaoke/utils/compute.py

```
"""Compute (??) calculation utilities for AI workloads.
This module provides comprehensive compute cost estimation for:
- GPU hardware specs and performance metrics
- Inference cost calculation
- Training cost estimation
- Multi-cloud provider comparison
"""
from __future__ import annotations
from dataclasses import dataclass
from enum import StrEnum
class GPUType(StrEnum):
    """Supported GPU models."""
    # NVIDIA Data Center
    H100 = "h100"
    H100_SXM = "h100_sxm"
    A100_80GB = "a100_80gb"
    A100_40GB = "a100_40gb"
    A6000 = "a6000"
    L40S = "l40s"
    L4 = "l4"
    # NVIDIA Consumer
    RTX_4090 = "rtx_4090"
    RTX_3090 = "rtx_3090"
    RTX_4080_SUPER = "rtx_4080_super"
    RTX_4080 = "rtx_4080"
    RTX_4070_TI = "rtx_4070_ti"
    # AMD
    MI300X = "mi300x"
    MI250X = "mi250x"
    # Intel
    Gaudi3 = "gaudi3"
@dataclass
class GPUConfig:
    """Configuration and specs for a GPU model."""
    gpu_type: GPUType
    name: str
    fp32_tflops: float  # FP32 TFLOPS
    fp16_tflops: float  # FP16 TFLOPS
    bf16_tflops: float  # BF16 TFLOPS
    memory_gb: float  # VRAM in GB
    memory_bandwidth_gbps: float  # GB/s
    tdp_watts: int  # Thermal Design Power
    price_per_hour: float | None = None  # Cloud price per hour (USD)
class ComputeProvider(StrEnum):
    """Cloud compute providers."""
    AWS = "aws"
    GCP = "gcp"
    AZURE = "azure"
    LAMBDA_LABS = "lambda_labs"
    VAST_AI = "vast_ai"
    LOCAL = "local"
# GPU specifications database
GPU_DATABASE: dict[GPUType, GPUConfig] = {
    # NVIDIA Data Center GPUs
    GPUType.H100: GPUConfig(
        gpu_type=GPUType.H100,
        name="NVIDIA H100 SXM",
        fp32_tflops=67.0,
        fp16_tflops=1939.0,  # with FP8
        bf16_tflops=1979.0,
        memory_gb=80.0,
```

```
    memory_bandwidth_gbps=3350.0,
    tdp_watts=700,
    price_per_hour=30.0,
),
GPUType.H100_SXM: GPUConfig(
    gpu_type=GPUType.H100_SXM,
    name="NVIDIA H100 SXM5",
    fp32_tflops=67.0,
    fp16_tflops=1939.0,
    bf16_tflops=1979.0,
    memory_gb=80.0,
    memory_bandwidth_gbps=3350.0,
    tdp_watts=700,
    price_per_hour=30.0,
),
GPUType.A100_80GB: GPUConfig(
    gpu_type=GPUType.A100_80GB,
    name="NVIDIA A100 80GB SXM",
    fp32_tflops=19.5,
    fp16_tflops=312.0,
    bf16_tflops=312.0,
    memory_gb=80.0,
    memory_bandwidth_gbps=2039.0,
    tdp_watts=400,
    price_per_hour=2.50,
),
GPUType.A100_40GB: GPUConfig(
    gpu_type=GPUType.A100_40GB,
    name="NVIDIA A100 40GB PCIe",
    fp32_tflops=19.5,
    fp16_tflops=312.0,
    bf16_tflops=312.0,
    memory_gb=40.0,
    memory_bandwidth_gbps=1616.0,
    tdp_watts=250,
    price_per_hour=1.50,
),
GPUType.A6000: GPUConfig(
    gpu_type=GPUType.A6000,
    name="NVIDIA A6000",
    fp32_tflops=38.7,
    fp16_tflops=309.8,
    bf16_tflops=309.8,
    memory_gb=48.0,
    memory_bandwidth_gbps=768.0,
    tdp_watts=300,
    price_per_hour=1.20,
),
GPUType.L40S: GPUConfig(
    gpu_type=GPUType.L40S,
    name="NVIDIA L40S",
    fp32_tflops=91.6,
    fp16_tflops=733.0,
    bf16_tflops=733.0,
    memory_gb=48.0,
    memory_bandwidth_gbps=864.0,
    tdp_watts=350,
    price_per_hour=1.50,
),
GPUType.L4: GPUConfig(
    gpu_type=GPUType.L4,
    name="NVIDIA L4",
    fp32_tflops=30.3,
    fp16_tflops=242.0,
    bf16_tflops=242.0,
```

```
    memory_gb=24.0,
    memory_bandwidth_gbps=300.0,
    tdp_watts=72,
    price_per_hour=0.70,
),
# Consumer GPUs
GPUType.RTX_4090: GPUConfig(
    gpu_type=GPUType.RTX_4090,
    name="NVIDIA RTX 4090",
    fp32_tflops=82.6,
    fp16_tflops=165.2,  # using tensor cores
    bf16_tflops=165.2,
    memory_gb=24.0,
    memory_bandwidth_gbps=1008.0,
    tdp_watts=450,
    price_per_hour=None,
),
GPUType.RTX_3090: GPUConfig(
    gpu_type=GPUType.RTX_3090,
    name="NVIDIA RTX 3090",
    fp32_tflops=35.6,
    fp16_tflops=142.0,
    bf16_tflops=71.0,
    memory_gb=24.0,
    memory_bandwidth_gbps=936.0,
    tdp_watts=350,
    price_per_hour=None,
),
GPUType.RTX_4080_SUPER: GPUConfig(
    gpu_type=GPUType.RTX_4080_SUPER,
    name="NVIDIA RTX 4080 SUPER",
    fp32_tflops=44.9,
    fp16_tflops=88.0,
    bf16_tflops=88.0,
    memory_gb=16.0,
    memory_bandwidth_gbps=736.0,
    tdp_watts=320,
    price_per_hour=None,
),
GPUType.RTX_4080: GPUConfig(
    gpu_type=GPUType.RTX_4080,
    name="NVIDIA RTX 4080",
    fp32_tflops=48.8,
    fp16_tflops=97.7,
    bf16_tflops=97.7,
    memory_gb=16.0,
    memory_bandwidth_gbps=716.0,
    tdp_watts=320,
    price_per_hour=None,
),
GPUType.RTX_4070_TI: GPUConfig(
    gpu_type=GPUType.RTX_4070_TI,
    name="NVIDIA RTX 4070 Ti",
    fp32_tflops=40.0,
    fp16_tflops=80.0,
    bf16_tflops=80.0,
    memory_gb=12.0,
    memory_bandwidth_gbps=504.0,
    tdp_watts=285,
    price_per_hour=None,
),
# AMD
GPUType.MI300X: GPUConfig(
    gpu_type=GPUType.MI300X,
    name="AMD MI300X",
```

```
        fp32_tflops=163.4,
        fp16_tflops=1307.0,
        bf16_tflops=1307.0,
        memory_gb=192.0,
        memory_bandwidth_gbps=5300.0,
        tdp_watts=750,
        price_per_hour=25.0,
    ),
    GPUType.MI250X: GPUConfig(
        gpu_type=GPUType.MI250X,
        name="AMD MI250X",
        fp32_tflops=47.9,
        fp16_tflops=383.0,
        bf16_tflops=383.0,
        memory_gb=128.0,
        memory_bandwidth_gbps=3277.0,
        tdp_watts=560,
        price_per_hour=15.0,
    ),
    # Intel Gaudi
    GPUType.Gaudi3: GPUConfig(
        gpu_type=GPUType.Gaudi3,
        name="Intel Gaudi 3",
        fp32_tflops=64.0,
        fp16_tflops=1835.0,
        bf16_tflops=1835.0,
        memory_gb=128.0,
        memory_bandwidth_gbps=819.0,
        tdp_watts=900,
        price_per_hour=12.0,
    ),
}
@dataclass
class InferenceConfig:
    """Configuration for inference workload."""
    model_name: str = ""
    model_size_b: float = 7.0  # Model size in billions of parameters
    context_length: int = 4096
    batch_size: int = 1
    precision: str = "fp16"  # fp32, fp16, bf16, int8, int4
    gpu_type: GPUType = GPUType.A100_80GB
    gpu_count: int = 1
    avg_tokens_per_output: int = 256
    requests_per_hour: int = 100
@dataclass
class InferenceCost:
    """Cost breakdown for inference workload."""
    config: InferenceConfig
    gpu: GPUConfig
    # Per-request metrics
    tokens_per_second: float = 0.0
    latency_seconds: float = 0.0
    memory_required_gb: float = 0.0
    # Cost metrics
    hourly_compute_cost: float = 0.0
    cost_per_1k_tokens: float = 0.0
    cost_per_request: float = 0.0
    daily_cost: float = 0.0
    monthly_cost: float = 0.0
    # GPU utilization
    tflops_utilization: float = 0.0
    memory_utilization: float = 0.0
@dataclass
class TrainingConfig:
    """Configuration for training workload."""
```

```
    model_name: str = ""
    model_size_b: float = 7.0  # Model size in billions of parameters
    dataset_size_tokens: int = 1_000_000_000  # 1B tokens
    batch_size: int = 32
    sequence_length: int = 2048
    precision: str = "fp16"  # fp32, fp16, bf16
    gpu_type: GPUType = GPUType.A100_80GB
    gpu_count: int = 8
    # Training parameters
    epochs: int = 1
    learning_rate_factor: float = 1.0
    gradient_accumulation_steps: int = 1
@dataclass
class TrainingCost:
    """Cost breakdown for training workload."""
    config: TrainingConfig
    gpu: GPUConfig
    # Compute metrics
    total_tflops_required: float = 0.0
    estimated_training_hours: float = 0.0
    estimated_training_days: float = 0.0
    # Cost metrics
    total_compute_cost: float = 0.0
    cost_per_1k_tokens: float = 0.0
    hourly_cost: float = 0.0
    # Efficiency metrics
    tflops_per_gpu: float = 0.0
    tokens_per_second_total: float = 0.0
class ComputeCalculator:
    """Calculator for compute costs and performance estimates."""
    # FLOPs estimates for training (Chinchilla scaling laws approximation)
    TRAINING_FLOPS_PER_TOKEN = {
        "fp32": 6,  # 6 FLOPs per parameter per token (forward + backward)
        "fp16": 2,  # 2 FLOPs per parameter per token (half precision)
        "bf16": 2,  # 2 FLOPs per parameter per token (bfloat16)
    }
    # Memory estimates for model weights (in bytes per parameter)
    MEMORY_PER_PARAM = {
        "fp32": 4,
        "fp16": 2,
        "bf16": 2,
        "int8": 1,
        "int4": 0.5,
    }
    # KV cache memory per token (rough estimates)
    KV_CACHE_PER_TOKEN = {
        "fp32": 8 * 2 * 128,  # key + value, fp32, 128 heads
        "fp16": 2 * 2 * 128,
        "bf16": 2 * 2 * 128,
    }
    def __init__(self, gpu_type: GPUType = GPUType.A100_80GB):
        self.gpu_type = gpu_type
        self.gpu = GPU_DATABASE.get(gpu_type)
        if not self.gpu:
            raise ValueError(f"Unknown GPU type: {gpu_type}")
    def get_effective_tflops(self, precision: str) -> float:
        """Get effective TFLOPS for given precision."""
        precision_map = {
            "fp32": self.gpu.fp32_tflops,
            "fp16": self.gpu.fp16_tflops,
            "bf16": self.gpu.bf16_tflops,
        }
        return precision_map.get(precision, self.gpu.bf16_tflops)
    def estimate_inference_cost(self, config: InferenceConfig) -> InferenceCost:
        """Estimate inference cost for given configuration."""
```

```
    gpu = GPU_DATABASE.get(config.gpu_type, self.gpu)
    precision = config.precision
    # Calculate memory requirements
    memory_per_param = self.MEMORY_PER_PARAM.get(precision, 2)
    model_memory_gb = config.model_size_b * memory_per_param
    # KV cache memory
    kv_memory_per_token = self.KV_CACHE_PER_TOKEN.get(precision, 512)
    kv_memory_gb = (config.context_length * kv_memory_per_token * config.batch_size) / (102
4**3)
    total_memory_gb = model_memory_gb + kv_memory_gb
    # Estimate tokens per second based on model size and GPU
    # Simplified model: throughput scales roughly with TFLOPs / model_size
    effective_tflops = self.get_effective_tflops(precision)
    # Very rough throughput estimate
    if config.model_size_b <= 7:
        base_tps = 50  # tokens/second base rate
    elif config.model_size_b <= 13:
        base_tps = 35
    elif config.model_size_b <= 70:
        base_tps = 20
    else:
        base_tps = 10
    # Scale by GPU capability relative to A100 80GB
    gpu_scale = gpu.fp16_tflops / GPU_DATABASE[GPUType.A100_80GB].fp16_tflops
    tokens_per_second = base_tps * gpu_scale * config.gpu_count
    # Handle memory constraints
    memory_constraint = gpu.memory_gb / total_memory_gb if total_memory_gb > 0 else 1.0
    if memory_constraint < 1.0:
        tokens_per_second *= memory_constraint
    latency_seconds = (
        config.avg_tokens_per_output / tokens_per_second if tokens_per_second > 0 else 0
    )
    # Calculate costs
    hourly_compute = 0.0
    if gpu.price_per_hour:
        hourly_compute = gpu.price_per_hour * config.gpu_count
    # Cost per token (compute only, rough estimate)
    tokens_per_hour = tokens_per_second * 3600
    cost_per_1k_tokens = (hourly_compute / tokens_per_hour * 1000) if tokens_per_hour > 0 e
lse 0
    # Total costs
    cost_per_request = cost_per_1k_tokens * config.avg_tokens_per_output / 1000
    daily_cost = hourly_compute * 24
    monthly_cost = daily_cost * 30
    # GPU utilization estimate
    tflops_needed = config.model_size_b * 2 * config.batch_size * tokens_per_second / 1e12
    tflops_utilization = (
        (tflops_needed / (effective_tflops * config.gpu_count)) * 100
        if effective_tflops > 0
        else 0
    )
    memory_utilization = (
        (total_memory_gb / (gpu.memory_gb * config.gpu_count)) * 100 if gpu.memory_gb > 0 e
lse 0
    )
    return InferenceCost(
        config=config,
        gpu=gpu,
        tokens_per_second=tokens_per_second,
        latency_seconds=latency_seconds,
        memory_required_gb=total_memory_gb,
        hourly_compute_cost=hourly_compute,
        cost_per_1k_tokens=cost_per_1k_tokens,
        cost_per_request=cost_per_request,
        daily_cost=daily_cost,
```

```
        monthly_cost=monthly_cost,
        tflops_utilization=min(tflops_utilization, 100.0),
        memory_utilization=min(memory_utilization, 100.0),
    )
def estimate_training_cost(self, config: TrainingConfig) -> TrainingCost:
    """Estimate training cost for given configuration."""
    gpu = GPU_DATABASE.get(config.gpu_type, self.gpu)
    precision = config.precision
    # Calculate FLOPs required
    flops_per_token = self.TRAINING_FLOPS_PER_TOKEN.get(precision, 2)
    model_size = config.model_size_b * 1e9  # Convert to actual parameters
    # Total FLOPs = 6 * model_size * dataset_size * epochs (approx)
    # 6 = 2 (forward) + 4 (backward) for FP32, or 2 for FP16/BF16
    total_flops = flops_per_token * model_size * config.dataset_size_tokens * config.epochs
    total_tflops = total_flops / 1e12
    # Effective GPU TFLOPS (accounting for practical efficiency ~50% for training)
    effective_tflops = self.get_effective_tflops(precision) * config.gpu_count * 0.5
    # Training time
    training_seconds = total_tflops / effective_tflops if effective_tflops > 0 else 0
    training_hours = training_seconds / 3600
    training_days = training_hours / 24
    # Costs
    hourly_cost = 0.0
    if gpu.price_per_hour:
        hourly_cost = gpu.price_per_hour * config.gpu_count
    total_cost = hourly_cost * training_hours
    # Cost per token
    total_tokens = config.dataset_size_tokens * config.epochs
    cost_per_1k_tokens = (total_cost / total_tokens * 1000) if total_tokens > 0 else 0
    # Efficiency metrics
    tflops_per_gpu = (
        total_tflops / training_hours / config.gpu_count if training_hours > 0 else 0
    )
    # Throughput
    tokens_per_second = total_tokens / training_seconds if training_seconds > 0 else 0
    return TrainingCost(
        config=config,
        gpu=gpu,
        total_tflops_required=total_tflops,
        estimated_training_hours=training_hours,
        estimated_training_days=training_days,
        total_compute_cost=total_cost,
        cost_per_1k_tokens=cost_per_1k_tokens,
        hourly_cost=hourly_cost,
        tflops_per_gpu=tflops_per_gpu,
        tokens_per_second_total=tokens_per_second,
    )
def compare_cloud_providers(
    self, gpu_type: GPUType, hours: float, providers: list[ComputeProvider] | None = None
) -> dict[str, float]:
    """Compare costs across cloud providers for given GPU and hours."""
    if providers is None:
        providers = [
            ComputeProvider.AWS,
            ComputeProvider.GCP,
            ComputeProvider.AZURE,
            ComputeProvider.LAMBDA_LABS,
            ComputeProvider.VAST_AI,
        ]
    gpu = GPU_DATABASE.get(gpu_type)
    if not gpu or not gpu.price_per_hour:
        return {}
    # Provider-specific pricing adjustments (approximate)
    provider_multipliers = {
        "aws": 1.0,
```

```
            "gcp": 0.95,
            "azure": 1.05,
            "lambda_labs": 0.85,
            "vast_ai": 0.6,
            "local": 0.0,  # Electricity only
        }
        base_hourly = gpu.price_per_hour
        results = {}
        for provider in providers:
            provider_key = provider.value
            if provider == ComputeProvider.LOCAL:
                # Estimate electricity cost (~0.10 per kWh, GPU uses ~0.35 kWh)
                results[provider_key] = hours * 0.35 * 0.10
            else:
                multiplier = provider_multipliers.get(provider_key, 1.0)
                results[provider_key] = hours * base_hourly * multiplier
        return results
def format_currency(amount: float, currency: str = "USD") -> str:
    """Format currency amount."""
    if currency == "USD":
        return f"${amount:.4f}"
    return f"{amount:.4f} {currency}"
def format_duration(hours: float) -> str:
    """Format duration in human-readable format."""
    if hours < 1:
        return f"{hours * 60:.1f} minutes"
    elif hours < 24:
        return f"{hours:.1f} hours"
    elif hours < 168:  # 1 week
        days = hours / 24
        return f"{days:.1f} days ({hours:.1f} hours)"
    else:
        weeks = hours / 168
        return f"{weeks:.1f} weeks"
def estimate_model_vram(
    model_size_b: float,
    precision: str = "fp16",
    include_kv_cache: bool = True,
    max_context_length: int = 4096,
    batch_size: int = 1,
) -> dict[str, float]:
    """Estimate VRAM requirements for a model.
    Returns:
        Dictionary with memory breakdown in GB.
    """
    memory_per_param = ComputeCalculator.MEMORY_PER_PARAM.get(precision, 2)
    # Model weights
    weights_gb = model_size_b * memory_per_param
    # Activations (rough estimate: ~2x model size for training)
    activations_gb = model_size_b * memory_per_param * 2
    # KV cache (per token)
    kv_per_token = ComputeCalculator.KV_CACHE_PER_TOKEN.get(precision, 512) / (1024**3)
    kv_cache_gb = kv_per_token * max_context_length * batch_size if include_kv_cache else 0
    # Gradient optimizer states (training only, fp32 copies)
    optimizer_gb = model_size_b * 4  # Adam optimizer: 2 states per parameter
    return {
        "weights": weights_gb,
        "activations": activations_gb,
        "kv_cache": kv_cache_gb,
        "optimizer": optimizer_gb,
        "total_inference": weights_gb + kv_cache_gb,
        "total_training": weights_gb + activations_gb + kv_cache_gb + optimizer_gb,
    }
def print_inference_report(cost: InferenceCost) -> str:
    """Generate a formatted inference cost report."""
```

```
    cfg = cost.config
    gpu = cost.gpu
    lines = [
        "=" * 60,
        "INFERENCE COST REPORT",
        "=" * 60,
        "",
        "Model Configuration:",
        f"  Model Size: {cfg.model_size_b}B parameters",
        f"  Precision: {cfg.precision}",
        f"  Context Length: {cfg.context_length:,} tokens",
        f"  Batch Size: {cfg.batch_size}",
        "",
        "Hardware:",
        f"  GPU: {gpu.name}",
        f"  GPU Count: {cfg.gpu_count}",
        f"  Memory per GPU: {gpu.memory_gb} GB",
        "",
        "Performance Estimates:",
        f"  Throughput: {cost.tokens_per_second:.1f} tokens/second",
        f"  Latency: {cost.latency_seconds:.3f} seconds",
        f"  Memory Required: {cost.memory_required_gb:.2f} GB",
        "",
        "GPU Utilization:",
        f"  TFLOPS Utilization: {cost.tflops_utilization:.1f}%",
        f"  Memory Utilization: {cost.memory_utilization:.1f}%",
        "",
        "Cost Estimates:",
        f"  Hourly Compute: {format_currency(cost.hourly_compute_cost)}",
        f"  Cost per 1K tokens: {format_currency(cost.cost_per_1k_tokens)}",
        f"  Cost per request: {format_currency(cost.cost_per_request)}",
        f"  Daily Cost: {format_currency(cost.daily_cost)}",
        f"  Monthly Cost: {format_currency(cost.monthly_cost)}",
        "=" * 60,
    ]
    return "\n".join(lines)
def print_training_report(cost: TrainingCost) -> str:
    """Generate a formatted training cost report."""
    cfg = cost.config
    gpu = cost.gpu
    lines = [
        "=" * 60,
        "TRAINING COST REPORT",
        "=" * 60,
        "",
        "Model Configuration:",
        f"  Model Size: {cfg.model_size_b}B parameters",
        f"  Precision: {cfg.precision}",
        f"  Dataset Size: {cfg.dataset_size_tokens / 1e9:.1f}B tokens",
        f"  Epochs: {cfg.epochs}",
        f"  Batch Size: {cfg.batch_size}",
        f"  Sequence Length: {cfg.sequence_length}",
        "",
        "Hardware:",
        f"  GPU: {gpu.name}",
        f"  GPU Count: {cfg.gpu_count}",
        f"  Total VRAM: {gpu.memory_gb * cfg.gpu_count} GB",
        "",
        "Compute Requirements:",
        f"  Total TFLOPS: {cost.total_tflops_required:,.0f}",
        f"  Effective TFLOPS/GPU: {cost.tflops_per_gpu:,.1f}",
        "",
        "Time Estimates:",
        f"  Training Time: {format_duration(cost.estimated_training_hours)}",
        f"  Throughput: {cost.tokens_per_second_total:,.0f} tokens/sec",
```

```
        "",
        "Cost Estimates:",
        f"  Hourly Cost: {format_currency(cost.hourly_cost)}",
        f"  Total Cost: {format_currency(cost.total_compute_cost)}",
        f"  Cost per 1K tokens: {format_currency(cost.cost_per_1k_tokens)}",
        "=" * 60,
    ]
    return "\n".join(lines)
# Convenience function for quick calculations
def quick_estimate(
    model_size_b: float, gpu_type: GPUType = GPUType.A100_80GB, mode: str = "inference", **kwar
    gs
) -> TrainingCost | InferenceCost:
    """Quick estimate for common scenarios.
    Args:
        model_size_b: Model size in billions of parameters
        gpu_type: GPU type to use
        mode: 'inference' or 'training'
        **kwargs: Additional configuration parameters
    Returns:
        TrainingCost or InferenceCost object
    """
    calc = ComputeCalculator(gpu_type)
    if mode == "inference":
        config = InferenceConfig(model_size_b=model_size_b, gpu_type=gpu_type, **kwargs)
        return calc.estimate_inference_cost(config)
    else:
        config = TrainingConfig(model_size_b=model_size_b, gpu_type=gpu_type, **kwargs)
        return calc.estimate_training_cost(config)
if __name__ == "__main__":
    # Example usage
    print("GPU Compute Calculator - Example Usage\n")
    # Example 1: Estimate inference cost for a 7B model on A100
    print("Example 1: 7B Model Inference on A100 80GB")
    print("-" * 40)
    inference_cost = quick_estimate(
        model_size_b=7,
        gpu_type=GPUType.A100_80GB,
        mode="inference",
        context_length=4096,
        avg_tokens_per_output=256,
        requests_per_hour=100,
    )
    print(print_inference_report(inference_cost))
    print()
    # Example 2: Estimate training cost for a 70B model on H100 cluster
    print("Example 2: 70B Model Training on H100 Cluster (8 GPUs)")
    print("-" * 40)
    training_cost = quick_estimate(
        model_size_b=70,
        gpu_type=GPUType.H100,
        mode="training",
        gpu_count=8,
        dataset_size_tokens=1_000_000_000,  # 1B tokens
        epochs=1,
    )
    print(print_training_report(training_cost))
    print()
    # Example 3: VRAM estimation
    print("Example 3: VRAM Requirements for 13B Model")
    print("-" * 40)
    vram = estimate_model_vram(13, precision="fp16", max_context_length=4096, batch_size=1)
    print(f"  Weights (FP16): {vram['weights']:.1f} GB")
    print(f"  KV Cache (4K ctx): {vram['kv_cache']:.2f} GB")
    print(f"  Total Inference: {vram['total_inference']:.1f} GB")
```

```
print(f"  Total Training: {vram['total_training']:.1f} GB")
print()
# Example 4: Compare cloud providers
print("Example 4: Cloud Provider Comparison (H100, 24 hours)")
print("-" * 40)
calc = ComputeCalculator(GPUType.H100)
comparison = calc.compare_cloud_providers(GPUType.H100, hours=24)
for provider, cost in comparison.items():
    print(f"  {provider.value}: {format_currency(cost)}")
```

File: [198/252] openlaoke/utils/config.py

```
"""Configuration management."""
from __future__ import annotations
import contextlib
import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any
from openlaoke.types.permissions import HyperAutoConfig
from openlaoke.types.providers import MultiProviderConfig, PlanConfig
from openlaoke.utils.theme import Theme, get_theme, get_theme_names
CONFIG_DIR = Path.home() / ".openlaoke"
CONFIG_PATH = CONFIG_DIR / "config.json"
@dataclass
class AppConfig:
    """User configuration for OpenLaoKe."""
    providers: MultiProviderConfig = field(default_factory=MultiProviderConfig.defaults)
    plans: PlanConfig = field(default_factory=PlanConfig.defaults)
    hyperauto_config: HyperAutoConfig = field(default_factory=HyperAutoConfig)
    max_tokens: int = 8192
    temperature: float = 1.0
    thinking_budget: int = 0
    permission_mode: str = "auto"
    auto_approve_all: bool = True
    auto_accept_tools: list[str] = field(default_factory=list)
    always_deny_tools: list[str] = field(default_factory=list)
    theme: str = "dark"
    show_token_budget: bool = True
    show_cost: bool = True
    max_output_lines: int = 500
    enable_mcp: bool = True
    mcp_servers: dict[str, Any] = field(default_factory=dict)
    custom_system_prompt: str = ""
    env_vars: dict[str, str] = field(default_factory=dict)
    working_directory: str = ""
    session_timeout_minutes: int = 0
    enable_telemetry: bool = False
    proxy_mode: str = "none"
    proxy_url: str = ""
    first_run: bool = True
    def is_configured(self) -> bool:
        return self.providers.is_configured()
    def get_active_provider_name(self) -> str:
        return self.providers.active_provider
    def get_active_model(self) -> str:
        return self.providers.get_active_model()
    def get_theme(self) -> Theme:
        return get_theme(self.theme)
    def validate_theme(self) -> bool:
        return self.theme in get_theme_names()
    def get_available_themes(self) -> list[str]:
        return get_theme_names()
def load_config() -> AppConfig:
    """Load configuration from disk."""
    if CONFIG_PATH.exists():
        try:
            with open(CONFIG_PATH, encoding="utf-8") as f:
                data = json.load(f)
            providers = MultiProviderConfig.defaults()
            if "providers" in data:
                providers_data = data["providers"]
                providers.active_provider = providers_data.get("active_provider", "ollama")
```

```
                providers.active_model = providers_data.get("active_model", "")
                if "providers" in providers_data:
                    for name, pdata in providers_data.get("providers", {}).items():
                        if name in providers.providers:
                            p = providers.providers[name]
                            p.api_key = pdata.get("api_key", p.api_key)
                            p.base_url = pdata.get("base_url", p.base_url)
                            p.default_model = pdata.get("default_model", p.default_model)
                            p.enabled = pdata.get("enabled", True)
                            # Preserve is_local from defaults
                            if name in ("ollama", "lm_studio"):
                                p.is_local = True
            plans = PlanConfig.defaults()
            if "plans" in data:
                plans_data = data["plans"]
                plans.active_plan = plans_data.get("active_plan", "default")
            hyperauto_config = HyperAutoConfig()
            if "hyperauto_config" in data:
                ha_data = data["hyperauto_config"]
                hyperauto_config.enabled = ha_data.get("enabled", False)
                hyperauto_config.max_iterations = ha_data.get("max_iterations", 100)
                hyperauto_config.timeout_seconds = ha_data.get("timeout_seconds", 300)
                hyperauto_config.auto_save = ha_data.get("auto_save", True)
                hyperauto_config.learning_enabled = ha_data.get("learning_enabled", False)
                hyperauto_config.history_limit = ha_data.get("history_limit", 50)
                if "mode" in ha_data:
                    from openlaoke.types.core_types import HyperAutoMode
                    with contextlib.suppress(ValueError):
                        hyperauto_config.mode = HyperAutoMode(ha_data["mode"])
            config = AppConfig(
                providers=providers,
                plans=plans,
                hyperauto_config=hyperauto_config,
                max_tokens=data.get("max_tokens", 8192),
                temperature=data.get("temperature", 1.0),
                thinking_budget=data.get("thinking_budget", 0),
                permission_mode=data.get("permission_mode", "auto"),
                auto_approve_all=data.get("auto_approve_all", True),
                theme=data.get("theme", "dark")
                if data.get("theme", "dark") in get_theme_names()
                else "dark",
                show_token_budget=data.get("show_token_budget", True),
                show_cost=data.get("show_cost", True),
                max_output_lines=data.get("max_output_lines", 500),
                enable_mcp=data.get("enable_mcp", True),
                mcp_servers=data.get("mcp_servers", {}),
                custom_system_prompt=data.get("custom_system_prompt", ""),
                env_vars=data.get("env_vars", {}),
                working_directory=data.get("working_directory", ""),
                proxy_mode=data.get("proxy_mode", "none"),
                proxy_url=data.get("proxy_url", ""),
                first_run=data.get("first_run", True),
            )
            return config
        except Exception:
            pass
    return AppConfig()
def save_config(config: AppConfig) -> None:
    """Save configuration to disk."""
    CONFIG_DIR.mkdir(parents=True, exist_ok=True)
    data = {
        "providers": {
            "active_provider": config.providers.active_provider,
            "active_model": config.providers.active_model,
            "providers": {
```

```
                name: {
                    "api_key": p.api_key,
                    "base_url": p.base_url,
                    "default_model": p.default_model,
                    "enabled": p.enabled,
                }
                for name, p in config.providers.providers.items()
            },
        },
        "plans": {
            "active_plan": config.plans.active_plan,
        },
        "hyperauto_config": {
            "mode": config.hyperauto_config.mode.value,
            "enabled": config.hyperauto_config.enabled,
            "max_iterations": config.hyperauto_config.max_iterations,
            "timeout_seconds": config.hyperauto_config.timeout_seconds,
            "auto_save": config.hyperauto_config.auto_save,
            "learning_enabled": config.hyperauto_config.learning_enabled,
            "history_limit": config.hyperauto_config.history_limit,
        },
        "max_tokens": config.max_tokens,
        "temperature": config.temperature,
        "thinking_budget": config.thinking_budget,
        "permission_mode": config.permission_mode,
        "auto_approve_all": config.auto_approve_all,
        "theme": config.theme,
        "show_token_budget": config.show_token_budget,
        "show_cost": config.show_cost,
        "max_output_lines": config.max_output_lines,
        "enable_mcp": config.enable_mcp,
        "mcp_servers": config.mcp_servers,
        "custom_system_prompt": config.custom_system_prompt,
        "env_vars": config.env_vars,
        "working_directory": config.working_directory,
        "proxy_mode": config.proxy_mode,
        "proxy_url": config.proxy_url,
        "first_run": config.first_run,
    }
    with open(CONFIG_PATH, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
def get_config_value(key: str, default: Any = None) -> Any:
    """Get a single config value."""
    config = load_config()
    return getattr(config, key, default)
def set_config_value(key: str, value: Any) -> None:
    """Set a single config value and persist."""
    config = load_config()
    if hasattr(config, key):
        setattr(config, key, value)
        save_config(config)
def mark_first_run_complete() -> None:
    """Mark that the user has completed the first-run wizard."""
    config = load_config()
    config.first_run = False
    save_config(config)
```

File: [199/252] openlaoke/utils/file_history.py

```
"""File history and checkpoint system for tracking file modifications."""
from __future__ import annotations
import hashlib
import json
import os
import time
from dataclasses import dataclass, field
from typing import Any
MAX_SNAPSHOTS = 100
HISTORY_DIR = "~/.openlaoke/file_history"
@dataclass
class FileSnapshot:
    """Snapshot of a file at a specific point in time."""
    path: str
    content: str
    timestamp: float
    checksum: str
    version: int = 1
    message_id: str | None = None
    def to_dict(self) -> dict[str, Any]:
        return {
            "path": self.path,
            "content": self.content,
            "timestamp": self.timestamp,
            "checksum": self.checksum,
            "version": self.version,
            "message_id": self.message_id,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> FileSnapshot:
        return cls(
            path=data["path"],
            content=data["content"],
            timestamp=data["timestamp"],
            checksum=data["checksum"],
            version=data.get("version", 1),
            message_id=data.get("message_id"),
        )
@dataclass
class FileHistory:
    """History of snapshots for a single file."""
    path: str
    snapshots: list[FileSnapshot] = field(default_factory=list)
    def to_dict(self) -> dict[str, Any]:
        return {
            "path": self.path,
            "snapshots": [s.to_dict() for s in self.snapshots],
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> FileHistory:
        return cls(
            path=data["path"],
            snapshots=[FileSnapshot.from_dict(s) for s in data.get("snapshots", [])],
        )
@dataclass
class FileHistoryState:
    """Global file history state tracking all files."""
    histories: dict[str, FileHistory] = field(default_factory=dict)
    tracked_files: set[str] = field(default_factory=set)
    def to_dict(self) -> dict[str, Any]:
        return {
```

```
            "histories": {k: v.to_dict() for k, v in self.histories.items()},
            "tracked_files": list(self.tracked_files),
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> FileHistoryState:
        return cls(
            histories={k: FileHistory.from_dict(v) for k, v in data.get("histories", {}).items(
    )},
            tracked_files=set(data.get("tracked_files", [])),
        )
def get_history_dir() -> str:
    """Get the file history storage directory."""
    return os.path.expanduser(HISTORY_DIR)
def get_history_path(file_path: str) -> str:
    """Get the JSON path for storing a file's history."""
    history_dir = get_history_dir()
    file_hash = hashlib.sha256(file_path.encode()).hexdigest()[:16]
    return os.path.join(history_dir, f"{file_hash}.json")
def compute_checksum(content: str) -> str:
    """Compute SHA256 checksum for content."""
    return hashlib.sha256(content.encode()).hexdigest()[:16]
def save_snapshot(
    file_path: str,
    content: str,
    message_id: str | None = None,
) -> FileSnapshot:
    """Save a snapshot of the file before modification."""
    abs_path = os.path.abspath(file_path)
    history_dir = get_history_dir()
    os.makedirs(history_dir, exist_ok=True)
    history_path = get_history_path(abs_path)
    history = FileHistory(path=abs_path)
    if os.path.exists(history_path):
        try:
            with open(history_path, encoding="utf-8") as f:
                data = json.load(f)
            history = FileHistory.from_dict(data)
        except Exception:
            pass
    version = len(history.snapshots) + 1
    timestamp = time.time()
    checksum = compute_checksum(content)
    snapshot = FileSnapshot(
        path=abs_path,
        content=content,
        timestamp=timestamp,
        checksum=checksum,
        version=version,
        message_id=message_id,
    )
    history.snapshots.append(snapshot)
    if len(history.snapshots) > MAX_SNAPSHOTS:
        history.snapshots = history.snapshots[-MAX_SNAPSHOTS:]
    try:
        with open(history_path, "w", encoding="utf-8") as f:
            json.dump(history.to_dict(), f, indent=2)
    except Exception:
        pass
    return snapshot
def get_history(file_path: str) -> FileHistory | None:
    """Get the history for a file."""
    abs_path = os.path.abspath(file_path)
    history_path = get_history_path(abs_path)
    if not os.path.exists(history_path):
        return None
```

```
    try:
        with open(history_path, encoding="utf-8") as f:
            data = json.load(f)
        return FileHistory.from_dict(data)
    except Exception:
        return None
def get_latest_snapshot(file_path: str) -> FileSnapshot | None:
    """Get the most recent snapshot for a file."""
    history = get_history(file_path)
    if not history or not history.snapshots:
        return None
    return history.snapshots[-1]
def get_snapshot_by_version(file_path: str, version: int) -> FileSnapshot | None:
    """Get a specific version snapshot for a file."""
    history = get_history(file_path)
    if not history:
        return None
    for snapshot in history.snapshots:
        if snapshot.version == version:
            return snapshot
    return None
def restore_snapshot(file_path: str, version: int | None = None) -> bool:
    """Restore a file to a specific version (or latest if version is None)."""
    abs_path = os.path.abspath(file_path)
    if version is None:
        snapshot = get_latest_snapshot(abs_path)
    else:
        snapshot = get_snapshot_by_version(abs_path, version)
    if not snapshot:
        return False
    try:
        os.makedirs(os.path.dirname(abs_path) or ".", exist_ok=True)
        with open(abs_path, "w", encoding="utf-8") as f:
            f.write(snapshot.content)
        return True
    except Exception:
        return False
def format_timestamp(timestamp: float) -> str:
    """Format timestamp for display."""
    import datetime
    dt = datetime.datetime.fromtimestamp(timestamp)
    return dt.strftime("%Y-%m-%d %H:%M:%S")
def format_history_list(file_path: str) -> str:
    """Format history list for display."""
    history = get_history(file_path)
    if not history or not history.snapshots:
        return f"No history found for {file_path}"
    lines = [f"History for {file_path}:", ""]
    for snapshot in history.snapshots:
        ts = format_timestamp(snapshot.timestamp)
        checksum = snapshot.checksum[:8]
        size = len(snapshot.content)
        lines.append(f"  v{snapshot.version}: {ts} (checksum: {checksum}, size: {size} bytes)")
    return "\n".join(lines)
def clear_history(file_path: str) -> bool:
    """Clear history for a file."""
    abs_path = os.path.abspath(file_path)
    history_path = get_history_path(abs_path)
    if os.path.exists(history_path):
        try:
            os.remove(history_path)
            return True
        except Exception:
            return False
    return True
```

```
def clear_all_histories() -> bool:
    """Clear all file histories."""
    history_dir = get_history_dir()
    if not os.path.exists(history_dir):
        return True
    try:
        for filename in os.listdir(history_dir):
            if filename.endswith(".json"):
                os.remove(os.path.join(history_dir, filename))
        return True
    except Exception:
        return False
def file_history_enabled() -> bool:
    """Check if file history is enabled."""
    return True
def track_file_edit(
    file_path: str,
    message_id: str | None = None,
) -> FileSnapshot | None:
    """Track a file before editing - save current state as snapshot."""
    abs_path = os.path.abspath(file_path)
    if not os.path.exists(abs_path):
        return save_snapshot(abs_path, "", message_id)
    try:
        with open(abs_path, encoding="utf-8") as f:
            content = f.read()
        return save_snapshot(abs_path, content, message_id)
    except Exception:
        return None
```

File: [200/252] openlaoke/utils/model_capabilities.py

```
"""Configuration for models that don't support tool calling."""
NO_TOOL_MODELS = {
    "gemma3:1b",
    "gemma3:4b",
    "qwen3.5:0.8B",
    "llama3.2:1b",
    "phi3:mini",
}
def should_disable_tools(model: str) -> bool:
    """Check if model supports tool calling."""
    model_lower = model.lower()
    return any(no_tool in model_lower for no_tool in NO_TOOL_MODELS)
```

File: [201/252] openlaoke/utils/permissions/__init__.py

```
"""Permissions utilities for OpenLaoKe."""
from __future__ import annotations
from openlaoke.utils.permissions.bash_classifier import (
    BashClassificationResult,
    CommandSafetyLevel,
    ConfidenceLevel,
    classify_bash_command,
    is_dangerous_command,
    is_destructive_command,
    is_safe_command,
)
from openlaoke.utils.permissions.classifier import (
    ClassifierMode,
    ClassifierResult,
    ai_classify_bash,
    classify_bash,
    classify_tool,
    fast_classify_bash,
    hybrid_classify_bash,
)
__all__ = [
    "BashClassificationResult",
    "CommandSafetyLevel",
    "ConfidenceLevel",
    "classify_bash_command",
    "is_destructive_command",
    "is_dangerous_command",
    "is_safe_command",
    "ClassifierMode",
    "ClassifierResult",
    "classify_bash",
    "classify_tool",
    "fast_classify_bash",
    "hybrid_classify_bash",
    "ai_classify_bash",
]
```

File: [202/252] openlaoke/utils/permissions/bash_classifier.py

```
"""Bash command safety classifier for dangerous patterns detection."""
from __future__ import annotations
import re
from dataclasses import dataclass
from enum import StrEnum
class CommandSafetyLevel(StrEnum):
    """Safety classification levels for bash commands."""
    SAFE = "safe"
    DANGEROUS = "dangerous"
    DESTRUCTIVE = "destructive"
class ConfidenceLevel(StrEnum):
    """Confidence levels for classification decisions."""
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"
@dataclass
class BashClassificationResult:
    """Result of bash command classification."""
    safety_level: CommandSafetyLevel
    confidence: ConfidenceLevel
    reason: str
    matched_pattern: str | None = None
SAFE_COMMANDS = {
    "ls",
    "cat",
    "grep",
    "find",
    "pwd",
    "echo",
    "head",
    "tail",
    "wc",
    "sort",
    "uniq",
    "cut",
    "tr",
    "sed",
    "awk",
    "diff",
    "tree",
    "file",
    "stat",
    "du",
    "df",
    "which",
    "whereis",
    "type",
    "alias",
    "history",
    "jobs",
    "ps",
    "top",
    "htop",
    "whoami",
    "id",
    "groups",
    "date",
    "cal",
    "uptime",
    "uname",
    "hostname",
```

```
"arch",
"env",
"printenv",
"git",
"npm",
"yarn",
"pnpm",
"pip",
"pip3",
"uv",
"python",
"python3",
"node",
"ruby",
"perl",
"php",
"cargo",
"rustc",
"go",
"java",
"javac",
"curl",
"wget",
"ssh",
"scp",
"rsync",
"tar",
"zip",
"unzip",
"gzip",
"gunzip",
"mkdir",
"touch",
"cp",
"mv",
"ln",
"chmod",
"chown",
"chgrp",
"less",
"more",
"vi",
"vim",
"nano",
"emacs",
"man",
"info",
"help",
"true",
"false",
"yes",
"no",
"sleep",
"xargs",
"parallel",
"make",
"cmake",
"docker",
"kubectl",
"terraform",
"ansible",
"pytest",
"jest",
"mocha",
"ruff",
```

```
    "mypy",
    "black",
    "rg",
    "fd",
    "fzf",
    "bat",
    "exa",
    "lsd",
}
DANGEROUS_COMMANDS = {
    "rm",
    "rmdir",
    "sudo",
    "su",
    "doas",
    "chmod",
    "chown",
    "chgrp",
    "systemctl",
    "service",
    "initctl",
    "iptables",
    "ip6tables",
    "nft",
    "netstat",
    "ss",
    "ip",
    "kill",
    "killall",
    "pkill",
    "xkill",
    "mkfs",
    "fdisk",
    "parted",
    "gdisk",
    "sfdisk",
    "dd",
    "shutdown",
    "reboot",
    "poweroff",
    "halt",
    "useradd",
    "userdel",
    "usermod",
    "groupadd",
    "groupdel",
    "passwd",
    "chpasswd",
    "crontab",
    "at",
    "batch",
    "ln",
    "unlink",
    "mv",
    "cp",
}
DESTRUCTIVE_PATTERNS = [
    (r"rm\s+-rf\s+/$", "rm -rf / - destroys entire filesystem"),
    (r"rm\s+-rf\s+/\s*$", "rm -rf / - destroys entire filesystem"),
    (r"rm\s+-rf\s+\*", "rm -rf * - destroys all files in current directory"),
    (r"rm\s+-rf\s+~", "rm -rf ~ - destroys home directory"),
    (r"rm\s+-rf\s+\$HOME", "rm -rf $HOME - destroys home directory"),
    (r"rm\s+-rf\s+/\*", "rm -rf /* - destroys entire filesystem"),
    (r"mkfs(\.\w+)?\s+", "mkfs - formats filesystem"),
    (r"mkfs\.ext[234]\s+", "mkfs.ext2/3/4 - formats filesystem"),
```

```
    (r"mkfs\.xfs\s+", "mkfs.xfs - formats filesystem"),
    (r"mkfs\.btrfs\s+", "mkfs.btrfs - formats filesystem"),
    (r"dd\s+.*of=/dev/[sh]d[a-z]", "dd writing to disk - destroys disk data"),
    (r"dd\s+.*of=/dev/nvme", "dd writing to NVMe - destroys disk data"),
    (r"dd\s+.*of=/dev/mmcblk", "dd writing to MMC - destroys disk data"),
    (r":()\s*{\s*:\s*|:\s*&\s*};\s*:", "fork bomb - crashes system"),
    (r">\s*/dev/sd[a-z]", "redirect to disk device - destroys data"),
    (r">\s*/dev/hd[a-z]", "redirect to disk device - destroys data"),
    (r">\s*/dev/nvme", "redirect to NVMe device - destroys data"),
    (r"shutdown\s+-h\s+now", "immediate shutdown"),
    (r"reboot\s+--force", "forced reboot"),
    (r"halt\s+--force", "forced halt"),
    (r"poweroff\s+--force", "forced poweroff"),
    (r"systemctl\s+stop\s+systemd", "stopping systemd - dangerous"),
    (r"kill\s+-9\s+-1", "kill all processes"),
    (r"kill\s+-9\s+1$", "kill init process"),
    (r"kill\s+-KILL\s+1$", "kill init process"),
    (r"killall\s+-9\s*$", "kill all processes"),
    (r"pkill\s+-9\s*$", "kill all processes"),
]
DANGEROUS_PATTERNS = [
    (r"rm\s+-[rf]+", "rm with recursive/force flags"),
    (r"rm\s+.*\*\s*$", "rm ending with wildcard"),
    (r"sudo\s+rm", "sudo rm - elevated deletion"),
    (r"sudo\s+chmod", "sudo chmod - elevated permission change"),
    (r"sudo\s+chown", "sudo chown - elevated ownership change"),
    (r"sudo\s+dd", "sudo dd - elevated disk operations"),
    (r"sudo\s+mkfs", "sudo mkfs - elevated filesystem formatting"),
    (r"sudo\s+fdisk", "sudo fdisk - elevated disk partitioning"),
    (r"chmod\s+[0-7]*777", "chmod 777 - overly permissive"),
    (r"chmod\s+-R", "chmod recursive"),
    (r"chown\s+-R", "chown recursive"),
    (r"kill\s+-9", "kill -9 (SIGKILL) - force kill"),
    (r"kill\s+-KILL", "kill -KILL - force kill"),
    (r">\s*/etc/", "redirect to system config directory"),
    (r">\s*/boot/", "redirect to boot directory"),
    (r">\s*/usr/", "redirect to usr directory"),
    (r">\s*/bin/", "redirect to bin directory"),
    (r">\s*/sbin/", "redirect to sbin directory"),
    (r"curl\s+.*\|\s*(ba)?sh", "curl piped to shell - remote code execution"),
    (r"wget\s+.*\|\s*(ba)?sh", "wget piped to shell - remote code execution"),
    (r"eval\s+", "eval - dynamic code execution"),
    (r"exec\s+", "exec - replaces process"),
    (r"source\s+.*http", "source from remote URL"),
    (r"\.\s+.*http", "source from remote URL"),
]
SAFE_PATTERNS = [
    (r"^ls\s", "ls - listing directory"),
    (r"^cat\s", "cat - reading file"),
    (r"^grep\s", "grep - searching content"),
    (r"^find\s", "find - searching files"),
    (r"^pwd\s*$", "pwd - print working directory"),
    (r"^echo\s", "echo - printing text"),
    (r"^head\s", "head - reading file start"),
    (r"^tail\s", "tail - reading file end"),
    (r"^wc\s", "wc - counting lines/words"),
    (r"^sort\s", "sort - sorting lines"),
    (r"^git\s+(status|log|diff|branch|show)", "git read operations"),
    (r"^npm\s+(list|view|search|info)", "npm read operations"),
    (r"^pip\s+(list|show|freeze)", "pip read operations"),
    (r"^python\s+-c\s+['\"]print", "python print statement"),
    (r"^pytest\s+", "pytest - running tests"),
    (r"^ruff\s+check", "ruff linting"),
    (r"^mypy\s+", "mypy type checking"),
    (r"^black\s+--check", "black format check"),
```

```
]
def extract_base_command(command: str) -> str:
    """Extract the base command from a potentially complex command string."""
    command = command.strip()
    if command.startswith("sudo ") or command.startswith("doas "):
        command = command[5:].strip()
    parts = command.split()
    if not parts:
        return ""
    base = parts[0]
    if base.startswith("./") or base.startswith("/"):
        return ""
    return base
def check_patterns(command: str, patterns: list[tuple[str, str]]) -> tuple[bool, str | None]:
    """Check if command matches any pattern in the list."""
    for pattern, description in patterns:
        if re.search(pattern, command, re.IGNORECASE):
            return True, description
    return False, None
def classify_bash_command(command: str) -> BashClassificationResult:
    """Classify a bash command into safety levels.
    Args:
        command: The bash command to classify
    Returns:
        BashClassificationResult with safety level, confidence, and reason
    """
    if not command or not command.strip():
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.SAFE,
            confidence=ConfidenceLevel.HIGH,
            reason="Empty command",
        )
    command = command.strip()
    matched, description = check_patterns(command, DESTRUCTIVE_PATTERNS)
    if matched:
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.DESTRUCTIVE,
            confidence=ConfidenceLevel.HIGH,
            reason=description or "Matches destructive pattern",
            matched_pattern=description,
        )
    matched, description = check_patterns(command, DANGEROUS_PATTERNS)
    if matched:
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.DANGEROUS,
            confidence=ConfidenceLevel.HIGH,
            reason=description or "Matches dangerous pattern",
            matched_pattern=description,
        )
    base_cmd = extract_base_command(command)
    if base_cmd in DANGEROUS_COMMANDS:
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.DANGEROUS,
            confidence=ConfidenceLevel.HIGH,
            reason=f"Base command '{base_cmd}' is classified as dangerous",
            matched_pattern=base_cmd,
        )
    if base_cmd in SAFE_COMMANDS:
        matched_safe, safe_desc = check_patterns(command, SAFE_PATTERNS)
        if matched_safe:
            return BashClassificationResult(
                safety_level=CommandSafetyLevel.SAFE,
                confidence=ConfidenceLevel.HIGH,
                reason=safe_desc or f"Command '{base_cmd}' is known safe",
                matched_pattern=safe_desc,
```

```
            )
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.SAFE,
            confidence=ConfidenceLevel.MEDIUM,
            reason=f"Base command '{base_cmd}' is generally safe, but check arguments",
        )
    matched, description = check_patterns(command, SAFE_PATTERNS)
    if matched:
        return BashClassificationResult(
            safety_level=CommandSafetyLevel.SAFE,
            confidence=ConfidenceLevel.HIGH,
            reason=description or "Matches safe pattern",
            matched_pattern=description,
        )
    return BashClassificationResult(
        safety_level=CommandSafetyLevel.DANGEROUS,
        confidence=ConfidenceLevel.LOW,
        reason=f"Unknown command '{base_cmd}', treating as potentially dangerous",
    )
def is_safe_command(command: str) -> bool:
    """Quick check if a command is safe to auto-execute."""
    result = classify_bash_command(command)
    return result.safety_level == CommandSafetyLevel.SAFE
def is_destructive_command(command: str) -> bool:
    """Check if a command is destructive and should always be blocked."""
    result = classify_bash_command(command)
    return result.safety_level == CommandSafetyLevel.DESTRUCTIVE
def is_dangerous_command(command: str) -> bool:
    """Check if a command is dangerous and needs confirmation."""
    result = classify_bash_command(command)
    return result.safety_level == CommandSafetyLevel.DANGEROUS
```

File: [203/252] openlaoke/utils/permissions/classifier.py

```
"""Permission classifier module for intelligent command classification."""
from __future__ import annotations
from dataclasses import dataclass
from enum import StrEnum
from typing import TYPE_CHECKING
from openlaoke.types.core_types import PermissionResult
from openlaoke.utils.permissions.bash_classifier import (
    CommandSafetyLevel,
    ConfidenceLevel,
    classify_bash_command,
)
if TYPE_CHECKING:
    from openlaoke.core.state import AppState
class ClassifierMode(StrEnum):
    """Classifier operation modes."""
    FAST = "fast"
    AI = "ai"
    HYBRID = "hybrid"
@dataclass
class ClassifierResult:
    """Result from the permission classifier."""
    decision: PermissionResult
    confidence: ConfidenceLevel
    reason: str
    safety_level: CommandSafetyLevel | None = None
    matched_pattern: str | None = None
    ai_analysis: str | None = None
def fast_classify_bash(command: str) -> ClassifierResult:
    """Fast rule-based classification of bash commands.
    Uses pattern matching and command lists for quick classification.
    Args:
        command: The bash command to classify
    Returns:
        ClassifierResult with decision and confidence
    """
    bash_result = classify_bash_command(command)
    if bash_result.safety_level == CommandSafetyLevel.SAFE:
        return ClassifierResult(
            decision=PermissionResult.ALLOW,
            confidence=bash_result.confidence,
            reason=bash_result.reason,
            safety_level=bash_result.safety_level,
            matched_pattern=bash_result.matched_pattern,
        )
    if bash_result.safety_level == CommandSafetyLevel.DESTRUCTIVE:
        return ClassifierResult(
            decision=PermissionResult.DENY,
            confidence=bash_result.confidence,
            reason=bash_result.reason,
            safety_level=bash_result.safety_level,
            matched_pattern=bash_result.matched_pattern,
        )
    return ClassifierResult(
        decision=PermissionResult.ASK,
        confidence=bash_result.confidence,
        reason=bash_result.reason,
        safety_level=bash_result.safety_level,
        matched_pattern=bash_result.matched_pattern,
    )
async def ai_classify_bash(
    command: str,
```

```
    app_state: AppState,
    context: str = "",
) -> ClassifierResult:
    """AI-based classification for complex or ambiguous commands.
    Uses the AI model to analyze command intent and context.
    Args:
        command: The bash command to classify
        app_state: Application state for accessing AI client
        context: Additional context about command purpose
    Returns:
        ClassifierResult with AI-based decision
    """
    fast_result = fast_classify_bash(command)
    if fast_result.confidence == ConfidenceLevel.HIGH:
        return fast_result
    if fast_result.safety_level == CommandSafetyLevel.DESTRUCTIVE:
        return fast_result
    prompt = f"""Analyze this bash command for safety:
Command: {command}
Context: {context}
Classify as one of:
- SAFE: Read-only operations, harmless commands
- DANGEROUS: Commands that could modify files, permissions, or system state
- DESTRUCTIVE: Commands that could cause irreversible damage
Provide your classification and a brief reason."""
    try:
        if not hasattr(app_state, "get_client"):
            return fast_result
        client = app_state.get_client()
        if client is None:
            return fast_result
        if not hasattr(client, "create_message"):
            return fast_result
        response = await client.create_message(
            messages=[{"role": "user", "content": prompt}],
            max_tokens=200,
        )
        content = ""
        if response.content:
            for block in response.content:
                if hasattr(block, "text"):
                    content += block.text
        ai_decision = PermissionResult.ASK
        if "SAFE" in content.upper():
            ai_decision = PermissionResult.ALLOW
        elif "DESTRUCTIVE" in content.upper():
            ai_decision = PermissionResult.DENY
        return ClassifierResult(
            decision=ai_decision,
            confidence=ConfidenceLevel.MEDIUM,
            reason=f"AI analysis: {content[:100]}",
            safety_level=fast_result.safety_level,
            ai_analysis=content,
        )
    except Exception:
        return fast_result
async def hybrid_classify_bash(
    command: str,
    app_state: AppState,
    context: str = "",
) -> ClassifierResult:
    """Hybrid classification combining fast rules and AI analysis.
    Uses fast classification first, then AI for low-confidence cases.
    Args:
        command: The bash command to classify
```

```
        app_state: Application state for accessing AI client
        context: Additional context about command purpose
    Returns:
        ClassifierResult with combined decision
    """
    fast_result = fast_classify_bash(command)
    if fast_result.confidence == ConfidenceLevel.HIGH:
        return fast_result
    if fast_result.safety_level == CommandSafetyLevel.DESTRUCTIVE:
        return fast_result
    return await ai_classify_bash(command, app_state, context)
async def classify_bash(
    command: str,
    app_state: AppState,
    mode: ClassifierMode = ClassifierMode.FAST,
    context: str = "",
) -> ClassifierResult:
    """Classify a bash command using specified classifier mode.
    Args:
        command: The bash command to classify
        app_state: Application state
        mode: Classifier mode (fast, ai, or hybrid)
        context: Additional context about command purpose
    Returns:
        ClassifierResult with final decision
    """
    if mode == ClassifierMode.FAST:
        return fast_classify_bash(command)
    if mode == ClassifierMode.AI:
        return await ai_classify_bash(command, app_state, context)
    return await hybrid_classify_bash(command, app_state, context)
def classify_tool(tool_name: str, tool_input: dict) -> ClassifierResult:
    """Classify a general tool call for permission requirements.
    Args:
        tool_name: Name of the tool being called
        tool_input: Input parameters for the tool
    Returns:
        ClassifierResult with classification decision
    """
    safe_tools = {"Read", "Glob", "Grep", "WebFetch"}
    if tool_name in safe_tools:
        return ClassifierResult(
            decision=PermissionResult.ALLOW,
            confidence=ConfidenceLevel.HIGH,
            reason=f"Tool '{tool_name}' is classified as safe/read-only",
        )
    dangerous_tools = {"Write", "Edit", "Bash"}
    if tool_name in dangerous_tools:
        if tool_name == "Bash" and "command" in tool_input:
            return fast_classify_bash(tool_input["command"])
        return ClassifierResult(
            decision=PermissionResult.ASK,
            confidence=ConfidenceLevel.MEDIUM,
            reason=f"Tool '{tool_name}' may modify files or system state",
        )
    return ClassifierResult(
        decision=PermissionResult.ASK,
        confidence=ConfidenceLevel.LOW,
        reason=f"Unknown tool '{tool_name}', requiring confirmation",
    )
```

File: [204/252] openlaoke/utils/theme.py

```
"""Theme management for OpenLaoKe TUI."""
from __future__ import annotations
import contextlib
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any
from rich.color import Color
from rich.style import Style
from rich.text import Text
@dataclass
class ThemeColors:
    """Color palette for a theme."""
    primary: str = "#00a8ff"
    secondary: str = "#9b59b6"
    accent: str = "#e74c3c"
    error: str = "#ff5555"
    warning: str = "#f1c40f"
    success: str = "#2ecc71"
    muted: str = "#6c757d"
    background: str = "#1a1a2e"
    foreground: str = "#eaeaea"
    def to_dict(self) -> dict[str, str]:
        return {
            "primary": self.primary,
            "secondary": self.secondary,
            "accent": self.accent,
            "error": self.error,
            "warning": self.warning,
            "success": self.success,
            "muted": self.muted,
            "background": self.background,
            "foreground": self.foreground,
        }
    @classmethod
    def from_dict(cls, data: dict[str, str]) -> ThemeColors:
        return cls(
            primary=data.get("primary", "#00a8ff"),
            secondary=data.get("secondary", "#9b59b6"),
            accent=data.get("accent", "#e74c3c"),
            error=data.get("error", "#ff5555"),
            warning=data.get("warning", "#f1c40f"),
            success=data.get("success", "#2ecc71"),
            muted=data.get("muted", "#6c757d"),
            background=data.get("background", "#1a1a2e"),
            foreground=data.get("foreground", "#eaeaea"),
        )
@dataclass
class ThemeStyles:
    """Style definitions for UI components."""
    panel: str = "bold primary"
    message: str = "foreground"
    tool_use: str = "bold secondary"
    tool_result: str = "muted"
    user_message: str = "bold primary"
    assistant_message: str = "bold secondary"
    system_message: str = "dim"
    error_message: str = "bold error"
    def to_dict(self) -> dict[str, str]:
        return {
            "panel": self.panel,
```

```
            "message": self.message,
            "tool_use": self.tool_use,
            "tool_result": self.tool_result,
            "user_message": self.user_message,
            "assistant_message": self.assistant_message,
            "system_message": self.system_message,
            "error_message": self.error_message,
        }
    @classmethod
    def from_dict(cls, data: dict[str, str]) -> ThemeStyles:
        return cls(
            panel=data.get("panel", "bold primary"),
            message=data.get("message", "foreground"),
            tool_use=data.get("tool_use", "bold secondary"),
            tool_result=data.get("tool_result", "muted"),
            user_message=data.get("user_message", "bold primary"),
            assistant_message=data.get("assistant_message", "bold secondary"),
            system_message=data.get("system_message", "dim"),
            error_message=data.get("error_message", "bold error"),
        )
@dataclass
class Theme:
    """Complete theme definition."""
    name: str
    colors: ThemeColors
    styles: ThemeStyles
    is_dark: bool = True
    def to_dict(self) -> dict[str, Any]:
        return {
            "name": self.name,
            "colors": self.colors.to_dict(),
            "styles": self.styles.to_dict(),
            "is_dark": self.is_dark,
        }
    @classmethod
    def from_dict(cls, data: dict[str, Any]) -> Theme:
        colors = ThemeColors.from_dict(data.get("colors", {}))
        styles = ThemeStyles.from_dict(data.get("styles", {}))
        return cls(
            name=data.get("name", "custom"),
            colors=colors,
            styles=styles,
            is_dark=data.get("is_dark", True),
        )
    def get_color(self, name: str) -> str:
        color_map = {
            "primary": self.colors.primary,
            "secondary": self.colors.secondary,
            "accent": self.colors.accent,
            "error": self.colors.error,
            "warning": self.colors.warning,
            "success": self.colors.success,
            "muted": self.colors.muted,
            "background": self.colors.background,
            "foreground": self.colors.foreground,
        }
        return color_map.get(name, self.colors.foreground)
    def get_style(self, name: str) -> Style:
        style_str = getattr(self.styles, name, "foreground")
        return self._parse_style(style_str)
    def _parse_style(self, style_str: str) -> Style:
        parts = style_str.split()
        kwargs: dict[str, Any] = {}
        style_map = {
            "bold": "bold",
```

```
            "dim": "dim",
            "italic": "italic",
            "underline": "underline",
            "blink": "blink",
            "reverse": "reverse",
        }
        color_map = {
            "primary": self.colors.primary,
            "secondary": self.colors.secondary,
            "accent": self.colors.accent,
            "error": self.colors.error,
            "warning": self.colors.warning,
            "success": self.colors.success,
            "muted": self.colors.muted,
            "background": self.colors.background,
            "foreground": self.colors.foreground,
        }
        for part in parts:
            if part in style_map:
                kwargs[style_map[part]] = True
            elif part in color_map:
                kwargs["color"] = Color.parse(color_map[part])
            else:
                with contextlib.suppress(Exception):
                    kwargs["color"] = Color.parse(part)
        return Style(**kwargs)
BUILTIN_THEMES: dict[str, Theme] = {}
DARK_THEME = Theme(
    name="dark",
    colors=ThemeColors(
        primary="#00a8ff",
        secondary="#9b59b6",
        accent="#e74c3c",
        error="#ff5555",
        warning="#f1c40f",
        success="#2ecc71",
        muted="#6c757d",
        background="#1a1a2e",
        foreground="#eaeaea",
    ),
    styles=ThemeStyles(),
    is_dark=True,
)
BUILTIN_THEMES["dark"] = DARK_THEME
LIGHT_THEME = Theme(
    name="light",
    colors=ThemeColors(
        primary="#0066cc",
        secondary="#6c5ce7",
        accent="#d63031",
        error="#d63031",
        warning="#fdcb6e",
        success="#00b894",
        muted="#636e72",
        background="#ffffff",
        foreground="#2d3436",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold secondary",
        system_message="dim",
```

```
        error_message="bold error",
    ),
    is_dark=False,
)
BUILTIN_THEMES["light"] = LIGHT_THEME
MONOKAI_THEME = Theme(
    name="monokai",
    colors=ThemeColors(
        primary="#66d9ef",
        secondary="#f92672",
        accent="#ae81ff",
        error="#f92672",
        warning="#e6db74",
        success="#a6e22e",
        muted="#75715e",
        background="#272822",
        foreground="#f8f8f2",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold accent",
        system_message="dim",
        error_message="bold error",
    ),
    is_dark=True,
)
BUILTIN_THEMES["monokai"] = MONOKAI_THEME
DRACULA_THEME = Theme(
    name="dracula",
    colors=ThemeColors(
        primary="#bd93f9",
        secondary="#ff79c6",
        accent="#ffb86c",
        error="#ff5555",
        warning="#f1fa8c",
        success="#50fa7b",
        muted="#6272a4",
        background="#282a36",
        foreground="#f8f8f2",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold secondary",
        system_message="dim",
        error_message="bold error",
    ),
    is_dark=True,
)
BUILTIN_THEMES["dracula"] = DRACULA_THEME
SOLARIZED_THEME = Theme(
    name="solarized",
    colors=ThemeColors(
        primary="#268bd2",
        secondary="#6c71c4",
        accent="#d33682",
        error="#dc322f",
        warning="#b58900",
```

```
        success="#859900",
        muted="#657b83",
        background="#002b36",
        foreground="#839496",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold accent",
        system_message="dim",
        error_message="bold error",
    ),
    is_dark=True,
)
BUILTIN_THEMES["solarized"] = SOLARIZED_THEME
NORD_THEME = Theme(
    name="nord",
    colors=ThemeColors(
        primary="#88c0d0",
        secondary="#81a1c1",
        accent="#5e81ac",
        error="#bf616a",
        warning="#ebcb8b",
        success="#a3be8c",
        muted="#4c566a",
        background="#2e3440",
        foreground="#eceff4",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold accent",
        system_message="dim",
        error_message="bold error",
    ),
    is_dark=True,
)
BUILTIN_THEMES["nord"] = NORD_THEME
GITHUB_THEME = Theme(
    name="github",
    colors=ThemeColors(
        primary="#0366d6",
        secondary="#6f42c1",
        accent="#d73a49",
        error="#cb2431",
        warning="#f9a825",
        success="#28a745",
        muted="#586069",
        background="#ffffff",
        foreground="#24292e",
    ),
    styles=ThemeStyles(
        panel="bold primary",
        message="foreground",
        tool_use="bold secondary",
        tool_result="muted",
        user_message="bold primary",
        assistant_message="bold secondary",
        system_message="dim",
```

```
        error_message="bold error",
    ),
    is_dark=False,
)
BUILTIN_THEMES["github"] = GITHUB_THEME
THEMES_DIR = Path.home() / ".openlaoke" / "themes"
def load_custom_themes() -> dict[str, Theme]:
    custom_themes: dict[str, Theme] = {}
    if not THEMES_DIR.exists():
        return custom_themes
    for theme_file in THEMES_DIR.glob("*.json"):
        try:
            with open(theme_file, encoding="utf-8") as f:
                data = json.load(f)
            theme = Theme.from_dict(data)
            custom_themes[theme.name] = theme
        except Exception:
            pass
    return custom_themes
def get_theme(name: str) -> Theme:
    all_themes = {**BUILTIN_THEMES, **load_custom_themes()}
    return all_themes.get(name, DARK_THEME)
def get_all_themes() -> dict[str, Theme]:
    return {**BUILTIN_THEMES, **load_custom_themes()}
def get_theme_names() -> list[str]:
    return sorted(get_all_themes().keys())
class ThemeManager:
    """Manages theme loading, switching, and persistence."""
    def __init__(self, current_theme: str = "dark"):
        self._current_theme_name = current_theme
        self._theme_cache: dict[str, Theme] = {}
        self._load_themes()
    def _load_themes(self) -> None:
        self._theme_cache = get_all_themes()
    @property
    def current_theme(self) -> Theme:
        if self._current_theme_name not in self._theme_cache:
            self._current_theme_name = "dark"
        return self._theme_cache.get(self._current_theme_name, DARK_THEME)
    @property
    def current_theme_name(self) -> str:
        return self._current_theme_name
    def set_theme(self, name: str) -> bool:
        if name not in self._theme_cache:
            self._load_themes()
        if name in self._theme_cache:
            self._current_theme_name = name
            return True
        return False
    def reload_themes(self) -> None:
        self._load_themes()
    def get_available_themes(self) -> list[str]:
        return sorted(self._theme_cache.keys())
    def is_dark_theme(self) -> bool:
        return self.current_theme.is_dark
    def color(self, name: str) -> str:
        return self.current_theme.get_color(name)
    def style(self, name: str) -> Style:
        return self.current_theme.get_style(name)
    def format_text(self, text: str, style_name: str) -> Text:
        style = self.style(style_name)
        return Text(text, style=style)
    def apply_to_rich(self) -> dict[str, Any]:
        theme = self.current_theme
        return {
```

```
    "style": Style(color=Color.parse(theme.colors.foreground)),
    "bgstyle": Style(color=Color.parse(theme.colors.background)),
}
```

File: [205/252] quick_auth.py

```
#!/usr/bin/env python3
"""Quick authentication script for extended web providers.
Usage:
    python3 quick_auth.py deepseek-chat
    python3 quick_auth.py claude-web
"""
from __future__ import annotations
import sys
from rich.console import Console
from rich.panel import Panel
console = Console()
def main() -> None:
    """Quick auth entry point."""
    if len(sys.argv) < 2:
        console.print(
            Panel(
                "[bold]Quick Authentication[/bold]\n\n"
                "Usage:\n"
                "  python3 quick_auth.py <provider>\n\n"
                "Available providers:\n"
                "  deepseek-chat   - DeepSeek Chat\n"
                "  deepseek-coder  - DeepSeek Coder\n"
                "  claude-web      - Claude Web\n"
                "  chatgpt-web     - ChatGPT Web\n"
                "  qwen-web        - Qwen Web (????)\n"
                "  kimi-web        - Kimi Web (????)\n"
                "  gemini-web      - Gemini Web\n"
                "  grok-web        - Grok Web\n"
                "  doubao-web      - Doubao Web (??)\n"
                "  glm-web         - GLM Web (????)",
                border_style="cyan",
            )
        )
        sys.exit(1)
    provider = sys.argv[1]
    console.print(f"[bold]Authenticating {provider}...[/bold]")
    console.print()
    console.print("[dim]Launching browser automatically...[/dim]")
    console.print()
    try:
        from openlaoke.core.extended_web.auto_browser import authenticate_with_auto_browser
        from openlaoke.core.extended_web.types import WEB_PROVIDERS
        config = WEB_PROVIDERS.get(provider)
        if not config:
            console.print(f"[red]Error:[/red] Unknown provider: {provider}")
            console.print(f"\n[dim]Available: {', '.join(WEB_PROVIDERS.keys())}[/dim]")
            sys.exit(1)
        auth_data = authenticate_with_auto_browser(
            provider_type=provider,
            login_url=config.login_url,
            timeout=300,
        )
        # Run async
        import asyncio
        result = asyncio.run(auth_data)
        console.print()
        console.print(
            Panel(
                f"[bold green][OK] Authentication successful![/bold green]\n\n"
                f"Provider: {provider}\n"
                f"Cookie length: {len(result.get('cookie', ''))}\n"
```

```
                f"Auth file: ~/.openlaoke/extended_web/{provider}.json",
                border_style="green",
            )
        )
        console.print()
        console.print("[bold]You can now use:[/bold]")
        console.print(f"  openlaoke --provider extended-web --model {provider}")
    except Exception as e:
        console.print(f"\n[red][X] Error:[/red] {e}")
        console.print("\n[dim]Make sure you have Chrome/Edge installed.[/dim]")
        sys.exit(1)
if __name__ == "__main__":
    main()
```

File: [206/252] quick_auth_firefox.py

```
#!/usr/bin/env python3
"""Quick authentication script using Firefox.
Usage:
    python3 quick_auth_firefox.py deepseek-chat
"""
from __future__ import annotations
import asyncio
import sys
from rich.console import Console
from rich.panel import Panel
console = Console()
def main() -> None:
    """Quick auth entry point using Firefox."""
    if len(sys.argv) < 2:
        console.print(Panel(
            "[bold]Firefox Quick Authentication[/bold]\n\n"
            "Usage:\n"
            "  python3 quick_auth_firefox.py <provider>\n\n"
            "Available providers:\n"
            "  deepseek-chat   - DeepSeek Chat\n"
            "  deepseek-coder  - DeepSeek Coder\n"
            "  claude-web      - Claude Web\n"
            "  chatgpt-web     - ChatGPT Web\n"
            "  qwen-web        - Qwen Web (????)\n"
            "  kimi-web        - Kimi Web (????)\n"
            "  gemini-web      - Gemini Web\n"
            "  grok-web        - Grok Web\n"
            "  doubao-web      - Doubao Web (??)\n"
            "  glm-web         - GLM Web (????)\n\n"
            "This script uses Firefox for authentication.",
            border_style="cyan",
        ))
        sys.exit(1)

    provider = sys.argv[1]

    console.print(f"[bold]Authenticating {provider} using Firefox...[/bold]")
    console.print()
    console.print("[dim]Launching Firefox automatically...[/dim]")
    console.print()

    try:
        from openlaoke.core.extended_web.auto_browser import authenticate_with_auto_browser
        from openlaoke.core.extended_web.types import WEB_PROVIDERS

        config = WEB_PROVIDERS.get(provider)
        if not config:
            console.print(f"[red]Error:[/red] Unknown provider: {provider}")
            console.print(f"\n[dim]Available: {', '.join(WEB_PROVIDERS.keys())}[/dim]")
            sys.exit(1)

        # Authenticate using Firefox
        result = asyncio.run(authenticate_with_auto_browser(
            provider_type=provider,
            login_url=config.login_url,
            timeout=300,
            browser_type="firefox",
        ))

        console.print()
        console.print(Panel(
```

```
            f"[bold green][OK] Authentication successful![/bold green]\n\n"
            f"Provider: {provider}\n"
            f"Browser: Firefox\n"
            f"Cookie length: {len(result.get('cookie', ''))}\n"
            f"Auth file: ~/.openlaoke/extended_web/{provider}.json",
            border_style="green",
        ))

        console.print()
        console.print("[bold]You can now use:[/bold]")
        console.print(f"  openlaoke --provider extended-web --model {provider}")

    except Exception as e:
        console.print(f"\n[red][X] Error:[/red] {e}")
        console.print("\n[dim]Make sure Firefox is installed.[/dim]")
        console.print("[dim]Install with: brew install --cask firefox (macOS)[/dim]")
        sys.exit(1)
if __name__ == "__main__":
    main()
```

File: [207/252] quick_auth_firefox_simple.py

```
#!/usr/bin/env python3
"""Simple Firefox authentication - direct launch without complex setup."""
import subprocess
import sys
import tempfile
import time
from pathlib import Path
from rich.console import Console
console = Console()
def get_firefox_path():
    """Get Firefox path."""
    import platform
    system = platform.system()

    if system == "Darwin":
        paths = [
            "/Applications/Firefox.app/Contents/MacOS/firefox",
            "/Applications/Firefox Developer Edition.app/Contents/MacOS/firefox",
        ]
    elif system == "Windows":
        paths = [
            r"C:\Program Files\Mozilla Firefox\firefox.exe",
        ]
    else:
        paths = ["/usr/bin/firefox", "/usr/bin/firefox-esr"]

    for path in paths:
        if Path(path).exists():
            return path
    return "firefox"
def main():
    if len(sys.argv) < 2:
        console.print("[red]Usage: python3 quick_auth_firefox_simple.py <provider>[/red]")
        console.print("Example: python3 quick_auth_firefox_simple.py deepseek-chat")
        sys.exit(1)

    provider = sys.argv[1]

    # Login URLs
    login_urls = {
        "deepseek-chat": "https://chat.deepseek.com",
        "deepseek-coder": "https://chat.deepseek.com",
        "claude-web": "https://claude.ai",
        "chatgpt-web": "https://chat.openai.com",
        "qwen-web": "https://tongyi.aliyun.com",
        "kimi-web": "https://kimi.moonshot.cn",
        "gemini-web": "https://gemini.google.com",
        "grok-web": "https://grok.x.ai",
        "doubao-web": "https://doubao.com",
        "glm-web": "https://chatglm.cn",
    }

    login_url = login_urls.get(provider, "https://chat.deepseek.com")

    console.print(f"[bold]Firefox Authentication for {provider}[/bold]")
    console.print(f"Login URL: [cyan underline]{login_url}[/cyan underline]")
    console.print()

    # Get Firefox path
    firefox_path = get_firefox_path()
    console.print(f"Firefox: {firefox_path}")
```

```
    # Create temp profile
    profile_dir = tempfile.mkdtemp(prefix="openlaoke-firefox-")
    console.print(f"Profile: {profile_dir}")
    console.print()

    console.print("[bold]Launching Firefox...[/bold]")
    console.print("[dim]A Firefox window should open on your screen.[/dim]")
    console.print()

    # Launch Firefox directly
    cmd = [
        firefox_path,
        "-profile", profile_dir,
        "-no-remote",
        "-new-instance",
        login_url,
    ]

    console.print(f"Command: {' '.join(cmd)}")
    console.print()

    # Launch Firefox
    process = subprocess.Popen(cmd)

    console.print("[green][OK] Firefox launched![/green]")
    console.print()
    console.print("[bold red]IMPORTANT:[/bold red]")
    console.print("  1. Look for the Firefox window on your screen")
    console.print("  2. Login to your account")
    console.print("  3. After login, press Ctrl+C here to save cookies")
    console.print()
    console.print("[dim]Waiting for you to login...[/dim]")
    console.print()

    try:
        # Wait for user to login
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        console.print()
        console.print("[yellow]Process interrupted[/yellow]")
        console.print("[dim]Note: Manual cookie extraction not implemented yet.[/dim]")
        console.print("[dim]For now, just keep using Firefox normally.[/dim]")

    process.terminate()
if __name__ == "__main__":
    main()
```

File: [208/252] scripts/cpu_benchmark_atomic.py

```
"""Module-level imports."""
from __future__ import annotations
def benchmark_helper() -> Any:
    """
    Implement benchmark_helper (function)

    Args:
        Arguments to be documented

    Returns:
        Return value

    Raises:
        Possible exceptions
    """
    pass
    return result
def benchmark_main_class__init__() -> object:
    """
    Implement benchmark_main_class__init__ (function)

    Args:
        Arguments to be documented

    Returns:
        Return value

    Raises:
        Possible exceptions
    """
    pass
    return result
```

File: [209/252] scripts/cpu_benchmark_with_knowledge.py

```
"""Module-level imports."""
from __future__ import annotations
def benchmark_helper() -> Any:
    """
    Implement benchmark_helper (function)
To measure real CPU performance:
1. Single-core benchmark:
   - Use math operations (sin, cos, sqrt, log)
   - Count operations per second
   - Calculate GFLOPS (Giga Floating-point Operations Per S

    Args:
        Arguments to be documented

    Returns:
        Return value

    Raises:
        Possible exceptions
    """
        # Example operations that stress CPU:
    # import math
    # - `time.perf_counter()` for accurate timing
    # - `concurrent.futures.ProcessPoolExecutor` for parallel execution
    pass
    return result
def benchmark_main_class__init__() -> object:
    """
    Implement benchmark_main_class__init__ (function)
To measure real CPU performance:
1. Single-core benchmark:
   - Use math operations (sin, cos, sqrt, log)
   - Count operations per second
   - Calculate GFLOPS (Giga Floating-point Operations Per S

    Args:
        Arguments to be documented

    Returns:
        Return value

    Raises:
        Possible exceptions
    """
        # Example operations that stress CPU:
    # import math
    # - `time.perf_counter()` for accurate timing
    # - `concurrent.futures.ProcessPoolExecutor` for parallel execution
    pass
    return result
```

File: [210/252] scripts/demo_small_model_development.py

```
"""Demonstration: Building a complete project with gemma3:1b.
This demo shows how to build a simple calculator module using
the incremental workflow with a small model (gemma3:1b).
The project is broken down into atomic tasks that the small model
can reliably implement, then assembled into a working module.
"""
import tempfile
from pathlib import Path
from openlaoke.core.architecture import (
    create_decomposer_for_model,
    create_orchestrator_for_model,
)
from openlaoke.core.model_assessment.types import ModelTier
from openlaoke.core.state import create_app_state
def demonstrate_small_model_development():
    """Complete workflow for small model development."""
    print("=" * 80)
    print("? ????????? - gemma3:1b??????")
    print("=" * 80)
    print()
    # ??????
    project_spec = {
        "name": "calculator",
        "description": "Simple calculator module with basic operations",
        "modules": [
            {
                "name": "calculator",
                "description": "Calculator module with basic math operations",
                "dependencies": [],
                "complexity": 5,
                "components": [
                    {
                        "name": "add",
                        "type": "function",
                        "description": "Add two numbers",
                        "inputs": {"a": {"type": "float"}, "b": {"type": "float"}},
                        "outputs": {"type": "float"},
                    },
                    {
                        "name": "subtract",
                        "type": "function",
                        "description": "Subtract two numbers",
                        "inputs": {"a": {"type": "float"}, "b": {"type": "float"}},
                        "outputs": {"type": "float"},
                    },
                    {
                        "name": "multiply",
                        "type": "function",
                        "description": "Multiply two numbers",
                        "inputs": {"a": {"type": "float"}, "b": {"type": "float"}},
                        "outputs": {"type": "float"},
                    },
                    {
                        "name": "divide",
                        "type": "function",
                        "description": "Divide two numbers with error handling",
                        "inputs": {"a": {"type": "float"}, "b": {"type": "float"}},
                        "outputs": {"type": "float"},
                        "complexity": 8,
                    },
                ],
```

```
        }
    ],
}
print("? ???? (Project Specification):")
print(f"  Name: {project_spec['name']}")
print(f"  Description: {project_spec['description']}")
print(f"  Modules: {len(project_spec['modules'])}")
print()
# ????????
with tempfile.TemporaryDirectory() as tmpdir:
    print(f"? ????: {tmpdir}")
    print()
    # ??????
    app_state = create_app_state(cwd=tmpdir)
    # ????
    model = "gemma3:1b"
    print(f"? ????: {model}")
    print("  ??: Tier 5 (Limited)")
    print("  ??????: 15?/??")
    print("  ??????: 3?/??")
    print("  ??????: 8?")
    print()
    # ?????
    decomposer = create_decomposer_for_model(ModelTier.TIER_5_LIMITED)
    print("? ???? (Task Decomposition):")
    print()
    # ????
    task_graph = decomposer.decompose_project(project_spec)
    print(f"  ????: {len(task_graph.tasks)}")
    print()
    print("  ???? (Task List):")
    for task_id, task in task_graph.tasks.items():
        deps_str = f" [??: {', '.join(task.dependencies)}]" if task.dependencies else ""
        test_str = " [OK]????" if task.test_required else ""
        print(f"    * {task_id}: {task.description[:50]}")
        print(f"      ????: {task.estimated_lines}{test_str}{deps_str}")
    print()
    # ?????
    orchestrator = create_orchestrator_for_model(app_state, model)
    # ?????
    workflow = orchestrator.create_workflow(project_spec)
    print("??  ??????? (Starting Workflow Execution):")
    print()
    # ?????
    result = orchestrator.execute_workflow(workflow.workflow_id)
    # ????
    progress = workflow.get_progress()
    print(f"  ??: {progress['progress_percentage']:.1f}%")
    print(f"  ??: {progress['completed']}/{progress['total_steps']}")
    print(f"  ??: {progress['failed']}")
    print()
    # ????
    if result.success:
        print("[OK] ???????!")
        print()
        print("? ????? (Generated Code):")
        print("-" * 80)
        print(result.code)
        print("-" * 80)
        print()
        # ????
        output_file = Path(tmpdir) / "calculator.py"
        output_file.write_text(result.code)
        print(f"? ??????: {output_file}")
        print()
```

```
        # ??????
        if result.test_results:
            print("? ???? (Test Results):")
            for test_id, passed in result.test_results.items():
                status = "[OK] PASSED" if passed else "[X] FAILED"
                print(f"  * {test_id}: {status}")
            print()
    else:
        print("[X] ???????")
        print()
        print("????:")
        for error in result.errors:
            print(f"  * {error}")
        print()
    # ??????
    workflow_status = orchestrator.get_workflow_status(workflow.workflow_id)
    if workflow_status:
        print("? ???????:")
        print(f"  ??: {workflow_status['model']}")
        print(f"  ??: {workflow_status['model_tier']}")
        print()
        print("  ????:")
        for step in workflow_status["steps"]:
            status_emoji = {
                "completed": "[OK]",
                "failed": "[X]",
                "pending": "?",
                "in_progress": "??",
                "retrying": "?",
            }.get(step["status"], "?")
            print(f"    {status_emoji} {step['step_id']}: {step['status']}")
            print(f"       ????: {step['attempts']}/{step['max_attempts']}")
            if step["error"]:
                print(f"       ??: {step['error']}")
        print()
    # ???????
    orchestrator.save_workflow_state(workflow.workflow_id)
    print("? ????????")
    print()
print("=" * 80)
print("? ??????:")
print("=" * 80)
print()
print("1. [OK] ???????")
print("   - ??????15???")
print("   - ??3???")
print("   - ??????????")
print()
print("2. [OK] ?????")
print("   - ?????????")
print("   - ???????")
print("   - ??????(??8?)")
print()
print("3. [OK] ?????")
print("   - ???API??")
print("   - ???????")
print("   - ?????????")
print()
print("4. [OK] ??????")
print("   - ????")
print("   - ????")
print("   - ????")
print()
print("5. [OK] ????????")
print("   - ?????")
```

```
    print("   - ????")
    print("   - ????")
    print()
    print("? ??????????????????")
    print()
if __name__ == "__main__":
    demonstrate_small_model_development()
```

File: [211/252] scripts/demo_task_decomposition.py

```
"""Demonstration of task decomposition for small models."""
import tempfile
from openlaoke.core.model_assessment import ModelAssessor, TaskDecomposer
from openlaoke.core.state import create_app_state
from openlaoke.core.supervisor.supervisor import TaskSupervisor
from openlaoke.types.providers import MultiProviderConfig
def demonstrate_task_decomposition():
    """Show how complex tasks are decomposed differently for different models."""
    print("=" * 80)
    print("???????? - Complex Task Decomposition Demonstration")
    print("=" * 80)
    print()
    config = MultiProviderConfig.defaults()
    assessor = ModelAssessor(config)
    complex_task = (
        "Write a comprehensive research paper about machine learning applications "
        "in healthcare with citations and create visualizations and implement demo code"
    )
    print("?????? (Original Complex Task):")
    print(f"  {complex_task}")
    print()
    models_to_test = [
        ("Claude Opus 4 (Tier 1 - Advanced)", "claude-opus-4"),
        ("GPT-4o (Tier 1 - Advanced)", "gpt-4o"),
        ("GPT-4o-mini (Tier 2 - Capable)", "gpt-4o-mini"),
        ("Llama 3.1 8B (Tier 3 - Moderate)", "llama3.1:8b"),
        ("Gemma 3 4B (Tier 4 - Basic)", "gemma3:4b"),
        ("Gemma 3 1B (Tier 5 - Limited)", "gemma3:1b"),
    ]
    print("??????????? (Task Decomposition Strategies):")
    print()
    for model_name, model_id in models_to_test:
        tier = assessor.get_tier(model_id)
        gran = assessor.get_granularity(model_id)
        decomposer = TaskDecomposer(gran)
        steps = decomposer.decompose(complex_task)
        print(f"{model_name}:")
        print(f"  Tier: {tier.value}")
        print(f"  Max Subtasks: {gran.max_subtasks}")
        print(f"  Complexity Limit: {gran.subtask_complexity_limit}")
        print(f"  Verification: {gran.verification_frequency}")
        print(f"  Requires Explicit Steps: {gran.requires_explicit_steps}")
        print(f"  Timeout Multiplier: {gran.timeout_multiplier}x")
        print(f"  Retry Limit: {gran.retry_limit}")
        print()
        if len(steps) > 1:
            print(f"  ?????? (Decomposed Steps) - {len(steps)} steps:")
            for i, step in enumerate(steps, 1):
                print(f"    {i}. {step}")
        else:
            print("  [OK] ?????????? (Can handle complete task)")
        print()
        print("  ???? (Verification Strategy):")
        if gran.verification_frequency == "every_step":
            print("    * ????????")
        elif gran.verification_frequency == "frequent":
            print("    * ?2????? (Verify every 2 steps)")
        elif gran.verification_frequency == "moderate":
            print("    * ?3????? (Verify every 3 steps)")
        else:
            print("    * ??????? (Minimal verification)")
```

```
        print()
        print("-" * 80)
        print()
    print("=" * 80)
    print("???????? - Task Supervisor Demonstration")
    print("=" * 80)
    print()
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        supervisor.set_model("gemma3:1b", config)
        print("?? Gemma 3 1B (????) ??????:")
        print()
        task = supervisor.parse_request(complex_task)
        print(f"????: {task.status.value}")
        print(f"??????: {task.max_attempts}")
        print()
        print(f"???? (Task Requirements) - {len(task.requirements)} requirements:")
        for i, req in enumerate(task.requirements, 1):
            status = "? CRITICAL" if req.critical else "? Optional"
            print(f"  {i}. [{status}] {req.name}")
            print(f"     Description: {req.description}")
            print(f"     Check Type: {req.check_type}")
            if req.threshold:
                print(f"     Threshold: {req.threshold}")
        print()
        print(f"???? (Decomposed Steps) - {len(task.decomposed_steps)} steps:")
        for i, step in enumerate(task.decomposed_steps, 1):
            current = "[OK]" if i == task.current_step + 1 else " "
            print(f"  [{current}] Step {i}: {step}")
        print()
        guidance = supervisor.get_model_guidance()
        if guidance:
            print("???? (Model Guidance):")
            print(f"  {guidance}")
        print()
        timeout = supervisor.get_timeout(120.0)
        print("?????? (Adjusted Timeout):")
        print("  Base: 120.0s")
        print(f"  Adjusted: {timeout}s (for small model)")
        print()
    print("=" * 80)
    print("???? (Key Findings):")
    print("=" * 80)
    print()
    print("1. ???? (Tier 1) ??????????")
    print("   - ?????????")
    print("   - ??????20????")
    print("   - ???????")
    print()
    print("2. ??? (Tier 5) ???????")
    print("   - ?????????4?????")
    print("   - ????????")
    print("   - ??????3?")
    print("   - ???????8?")
    print()
    print("3. ??????")
    print("   - ?????????????")
    print("   - ????????????")
    print("   - ????????????")
    print()
    print("[OK] ??????????????????????")
    print()
if __name__ == "__main__":
    demonstrate_task_decomposition()
```

File: [212/252] scripts/generate_figures.py

```
#!/usr/bin/env python3
"""Generate SVG figures for the article."""
from __future__ import annotations
import os
from pathlib import Path
try:
    import svgwrite
except ImportError:
    print("Installing svgwrite...")
    os.system("pip install svgwrite")
    import svgwrite
def create_architecture_comparison() -> None:
    """Create architecture comparison diagram."""
    dwg = svgwrite.Drawing("architecture_comparison.svg", size=("1000px", "600px"))
    dwg.add(dwg.rect(insert=(0, 0), size=("100%", "100%"), fill="white"))
    def draw_box(x: float, y: float, w: float, h: float, text: str, color: str) -> None:
        dwg.add(
            dwg.rect(
                insert=(x, y), size=(w, h), rx=5, ry=5, fill=color, stroke="#333", stroke_width
    =1.5
            )
        )
        lines = text.split("\n")
        line_height = 16
        total_height = len(lines) * line_height
        start_y = y + (h - total_height) / 2 + line_height - 4
        for i, line in enumerate(lines):
            dwg.add(
                dwg.text(
                    line,
                    insert=(x + w / 2, start_y + i * line_height),
                    text_anchor="middle",
                    font_size="12px",
                    font_family="Arial",
                    fill="black" if color != "#333" else "white",
                )
            )
    def draw_arrow(x1: float, y1: float, x2: float, y2: float, label: str = "") -> None:
        dwg.add(
            dwg.line(
                start=(x1, y1),
                end=(x2, y2),
                stroke="#666",
                stroke_width=1.5,
                stroke_dasharray="4,2",
            )
        )
        if label:
            mid_x = (x1 + x2) / 2
            mid_y = (y1 + y2) / 2
            dwg.add(
                dwg.text(
                    label,
                    insert=(mid_x, mid_y - 5),
                    text_anchor="middle",
                    font_size="9px",
                    font_family="Arial",
                    fill="#666",
                )
            )
    col_width = 280
```

```
col_gap = 40
start_x = 50
dwg.add(
    dwg.text(
        "OpenLaoKe",
        insert=(start_x + col_width / 2, 30),
        text_anchor="middle",
        font_size="14px",
        font_weight="bold",
        font_family="Arial",
    )
)
draw_box(start_x + 20, 50, col_width - 40, 40, "REPL Loop", "#4A90D9")
draw_arrow(start_x + col_width / 2, 90, start_x + col_width / 2, 110, "")
draw_box(start_x + 20, 110, col_width - 40, 40, "MultiProviderClient", "#5BA0E9")
draw_arrow(start_x + col_width / 2, 150, start_x + col_width / 2, 170, "")
draw_box(start_x + 20, 170, col_width - 40, 40, "AppState", "#6BB0F9")
draw_arrow(start_x + col_width / 2, 210, start_x + col_width / 2, 230, "")
draw_box(start_x + 20, 230, col_width - 40, 40, "ToolRegistry", "#7BC0FF")
draw_box(
    start_x + 20,
    290,
    col_width - 40,
    100,
    "Tools:\n* Bash\n* Read/Write/Edit\n* Grep/Glob\n* WebFetch",
    "#E8F4FF",
)
draw_arrow(start_x + col_width / 2, 280, start_x + col_width / 2, 290, "")
col2_x = start_x + col_width + col_gap
dwg.add(
    dwg.text(
        "OpenCode",
        insert=(col2_x + col_width / 2, 30),
        text_anchor="middle",
        font_size="14px",
        font_weight="bold",
        font_family="Arial",
    )
)
draw_box(col2_x + 20, 50, col_width - 40, 40, "TUI (Ink/React)", "#D94A4A")
draw_arrow(col2_x + col_width / 2, 90, col2_x + col_width / 2, 110, "")
draw_box(col2_x + 20, 110, col_width - 40, 40, "Agent", "#E95A5A")
draw_arrow(col2_x + col_width / 2, 150, col2_x + col_width / 2, 170, "")
draw_box(col2_x + 20, 170, col_width - 40, 40, "State (SQLite)", "#F96A6A")
draw_arrow(col2_x + col_width / 2, 210, col2_x + col_width / 2, 230, "")
draw_box(col2_x + 20, 230, col_width - 40, 40, "Tools (Effect)", "#FA7A7A")
draw_box(
    col2_x + 20,
    290,
    col_width - 40,
    100,
    "Tools:\n* Bash\n* Read/Write/Edit\n* Grep/Glob\n* WebFetch",
    "#FFE8E8",
)
draw_arrow(col2_x + col_width / 2, 280, col2_x + col_width / 2, 290, "")
col3_x = col2_x + col_width + col_gap
dwg.add(
    dwg.text(
        "Claude Code",
        insert=(col3_x + col_width / 2, 30),
        text_anchor="middle",
        font_size="14px",
        font_weight="bold",
        font_family="Arial",
    )
```

```
    )
    draw_box(col3_x + 20, 50, col_width - 40, 40, "CLI", "#4AD94A")
    draw_arrow(col3_x + col_width / 2, 90, col3_x + col_width / 2, 110, "")
    draw_box(col3_x + 20, 110, col_width - 40, 40, "QueryEngine", "#5AE95A")
    draw_arrow(col3_x + col_width / 2, 150, col3_x + col_width / 2, 170, "")
    draw_box(col3_x + 20, 170, col_width - 40, 40, "Store", "#6AFA6A")
    draw_arrow(col3_x + col_width / 2, 210, col3_x + col_width / 2, 230, "")
    draw_box(col3_x + 20, 230, col_width - 40, 40, "Tools", "#7AFF7A")
    draw_box(
        col3_x + 20,
        290,
        col_width - 40,
        100,
        "Tools:\n* Bash\n* Read/Write/Edit\n* Grep/Glob\n* MCP Servers",
        "#E8FFE8",
    )
    draw_arrow(col3_x + col_width / 2, 280, col3_x + col_width / 2, 290, "")
    dwg.add(
        dwg.rect(insert=(start_x, 420), size=(910, 40), rx=5, ry=5, fill="#F5F5F5", stroke="#CC
    C")
    )
    dwg.add(
        dwg.text(
            "LLM Provider APIs (Anthropic, OpenAI, Ollama, etc.)",
            insert=(start_x + 455, 445),
            text_anchor="middle",
            font_size="12px",
            font_family="Arial",
        )
    )
    for col_x in [start_x + col_width / 2, col2_x + col_width / 2, col3_x + col_width / 2]:
        dwg.add(
            dwg.line(
                start=(col_x, 390),
                end=(col_x, 420),
                stroke="#666",
                stroke_width=1.5,
                stroke_dasharray="4,2",
            )
        )
    dwg.save()
    print("[OK] architecture_comparison.svg")
def create_tool_invocation_flow() -> None:
    """Create tool invocation flow diagram."""
    dwg = svgwrite.Drawing("tool_invocation_flow.svg", size=("900px", "500px"))
    dwg.add(dwg.rect(insert=(0, 0), size=("100%", "100%"), fill="white"))
    def draw_box(x: float, y: float, w: float, h: float, text: str, color: str) -> None:
        dwg.add(
            dwg.rect(
                insert=(x, y), size=(w, h), rx=8, ry=8, fill=color, stroke="#333", stroke_width
    =1.5
            )
        )
        lines = text.split("\n")
        line_height = 15
        total_height = len(lines) * line_height
        start_y = y + (h - total_height) / 2 + line_height - 3
        for i, line in enumerate(lines):
            dwg.add(
                dwg.text(
                    line,
                    insert=(x + w / 2, start_y + i * line_height),
                    text_anchor="middle",
                    font_size="11px",
                    font_family="Arial",
```

```
                fill="black",
            )
        )
def draw_arrow(x1: float, y1: float, x2: float, y2: float) -> None:
    dwg.add(dwg.line(start=(x1, y1), end=(x2, y2), stroke="#333", stroke_width=2))
    angle = 0.5
    arrow_len = 8
    import math
    theta = math.atan2(y2 - y1, x2 - x1)
    dwg.add(
        dwg.line(
            start=(x2, y2),
            end=(
                x2 - arrow_len * math.cos(theta - angle),
                y2 - arrow_len * math.sin(theta - angle),
            ),
            stroke="#333",
            stroke_width=2,
        )
    )
    dwg.add(
        dwg.line(
            start=(x2, y2),
            end=(
                x2 - arrow_len * math.cos(theta + angle),
                y2 - arrow_len * math.sin(theta + angle),
            ),
            stroke="#333",
            stroke_width=2,
        )
    )
box_w = 130
box_h = 45
positions = [
    (80, 60, "User Input", "#E3F2FD"),
    (260, 60, "LLM\nInference", "#FFF3E0"),
    (440, 60, "Tool Selection", "#E8F5E9"),
    (620, 60, "Permission\nCheck", "#FFF9C4"),
    (800, 60, "Decision", "#F3E5F5"),
]
for x, y, text, color in positions[:4]:
    draw_box(x, y, box_w, box_h, text, color)
    if x < 700:
        draw_arrow(x + box_w, y + box_h / 2, x + box_w + 30, y + box_h / 2)
draw_box(620, 60, box_w, box_h, "Permission\nCheck", "#FFF9C4")
draw_arrow(620 + box_w, 60 + box_h / 2, 800, 60 + box_h / 2)
dwg.add(
    dwg.polygon(
        points=[
            (800, 60 + box_h / 2),
            (830, 60 + 10),
            (870, 60 + box_h / 2),
            (830, 60 + box_h - 10),
        ],
        fill="#F3E5F5",
        stroke="#333",
    )
)
dwg.add(
    dwg.text(
        "?",
        insert=(835, 60 + box_h / 2 + 4),
        text_anchor="middle",
        font_size="14px",
        font_weight="bold",
```

```
        )
    )
    draw_box(600, 180, box_w, box_h, "Ask User\nPermission", "#FFCDD2")
    draw_arrow(835, 60 + box_h, 835, 110)
    dwg.add(dwg.text("Deny", insert=(855, 100), font_size="10px", fill="#D32F2F"))
    draw_arrow(835, 110, 835, 155)
    draw_arrow(835, 155, 665 + box_w / 2, 180)
    dwg.add(dwg.text("Retry", insert=(750, 165), font_size="10px", fill="#666"))
    draw_box(260, 300, box_w, box_h, "Tool\nExecution", "#C8E6C9")
    draw_arrow(835, 155, 260 + box_w / 2, 300)
    dwg.add(dwg.text("Allow", insert=(550, 250), font_size="10px", fill="#388E3C"))
    draw_box(440, 300, box_w, box_h, "Result\nProcessing", "#B3E5FC")
    draw_arrow(260 + box_w, 300 + box_h / 2, 440, 300 + box_h / 2)
    draw_box(620, 300, box_w, box_h, "Continue\nConversation", "#D1C4E9")
    draw_arrow(440 + box_w, 300 + box_h / 2, 620, 300 + box_h / 2)
    draw_arrow(620 + box_w / 2, 300, 260 + box_w / 2, 60 + box_h)
    dwg.add(dwg.text("Loop", insert=(450, 200), font_size="10px", fill="#666"))
    dwg.add(dwg.rect(insert=(50, 400), size=(800, 70), rx=5, ry=5, fill="#FAFAFA", stroke="#DDD
    "))
    dwg.add(
        dwg.text(
            "Key Components:",
            insert=(70, 425),
            font_size="12px",
            font_weight="bold",
            font_family="Arial",
        )
    )
    dwg.add(
        dwg.text(
            "* Permission System: Validates tool access before execution",
            insert=(70, 445),
            font_size="11px",
            font_family="Arial",
        )
    )
    dwg.add(
        dwg.text(
            "* Tool Registry: Manages available tools and their schemas",
            insert=(70, 460),
            font_size="11px",
            font_family="Arial",
        )
    )
    dwg.save()
    print("[OK] tool_invocation_flow.svg")
def create_state_management_comparison() -> None:
    """Create state management comparison diagram."""
    dwg = svgwrite.Drawing("state_management_comparison.svg", size=("950px", "400px"))
    dwg.add(dwg.rect(insert=(0, 0), size=("100%", "100%"), fill="white"))
    def draw_box(x: float, y: float, w: float, h: float, text: str, color: str) -> None:
        dwg.add(
            dwg.rect(
                insert=(x, y), size=(w, h), rx=5, ry=5, fill=color, stroke="#333", stroke_width
    =1.5
            )
        )
        lines = text.split("\n")
        line_height = 14
        total_height = len(lines) * line_height
        start_y = y + (h - total_height) / 2 + line_height - 3
        for i, line in enumerate(lines):
            dwg.add(
                dwg.text(
                    line,
```

```
                insert=(x + w / 2, start_y + i * line_height),
                text_anchor="middle",
                font_size="11px",
                font_family="Arial",
                fill="black",
            )
        )
def draw_arrow(x1: float, y1: float, x2: float, y2: float) -> None:
    dwg.add(dwg.line(start=(x1, y1), end=(x2, y2), stroke="#666", stroke_width=1.5))
col_width = 280
start_x = 50
dwg.add(
    dwg.text(
        "OpenLaoKe: Centralized Dataclass",
        insert=(start_x + col_width / 2, 25),
        text_anchor="middle",
        font_size="12px",
        font_weight="bold",
        font_family="Arial",
    )
)
draw_box(start_x + 30, 50, col_width - 60, 50, "AppState\ndataclass", "#4A90D9")
boxes = [
    (start_x + 20, 120, "messages: list[Message]"),
    (start_x + 20, 155, "tasks: dict[str, Task]"),
    (start_x + 20, 190, "permissions: PermissionConfig"),
    (start_x + 20, 225, "token_usage: TokenUsage"),
]
for x, y, text in boxes:
    draw_box(x, y, col_width - 40, 30, text, "#E3F2FD")
    draw_arrow(start_x + col_width / 2, 100, start_x + col_width / 2, 120)
draw_box(start_x + 30, 280, col_width - 60, 40, "JSON File\nPersistence", "#FFE0B2")
draw_arrow(start_x + col_width / 2, 255, start_x + col_width / 2, 280)
col2_x = start_x + col_width + 50
dwg.add(
    dwg.text(
        "OpenCode: SQLite + Effect",
        insert=(col2_x + col_width / 2, 25),
        text_anchor="middle",
        font_size="12px",
        font_weight="bold",
        font_family="Arial",
    )
)
draw_box(col2_x + 30, 50, col_width - 60, 50, "State\nService", "#D94A4A")
boxes2 = [
    (col2_x + 20, 120, "Session Table"),
    (col2_x + 20, 155, "Message Table"),
    (col2_x + 20, 190, "Tool Result Table"),
    (col2_x + 20, 225, "Project Table"),
]
for x, y, text in boxes2:
    draw_box(x, y, col_width - 40, 30, text, "#FFEBEE")
    draw_arrow(col2_x + col_width / 2, 100, col2_x + col_width / 2, 120)
draw_box(col2_x + 30, 280, col_width - 60, 40, "SQLite\nDatabase", "#FFCDD2")
draw_arrow(col2_x + col_width / 2, 255, col2_x + col_width / 2, 280)
col3_x = col2_x + col_width + 50
dwg.add(
    dwg.text(
        "Claude Code: Store Pattern",
        insert=(col3_x + col_width / 2, 25),
        text_anchor="middle",
        font_size="12px",
        font_weight="bold",
        font_family="Arial",
```

```
        )
    )
    draw_box(col3_x + 30, 50, col_width - 60, 50, "Store\n(React-like)", "#4AD94A")
    boxes3 = [
        (col3_x + 20, 120, "messages: Message[]"),
        (col3_x + 20, 155, "tools: ToolRegistry"),
        (col3_x + 20, 190, "permissionContext"),
        (col3_x + 20, 225, "queryEngine"),
    ]
    for x, y, text in boxes3:
        draw_box(x, y, col_width - 40, 30, text, "#E8F5E9")
        draw_arrow(col3_x + col_width / 2, 100, col3_x + col_width / 2, 120)
    draw_box(col3_x + 30, 280, col_width - 60, 40, "Transcript\nFiles", "#C8E6C9")
    draw_arrow(col3_x + col_width / 2, 255, col3_x + col_width / 2, 280)
    dwg.add(dwg.rect(insert=(50, 350), size=("90%", 40), rx=5, ry=5, fill="#F5F5F5", stroke="#C
    CC"))
    dwg.add(
        dwg.text(
            "Trade-offs: Simplicity (JSON) vs Queryability (SQLite) vs Readability (Transcript)
    ",
            insert=(475, 375),
            text_anchor="middle",
            font_size="11px",
            font_family="Arial",
        )
    )
    dwg.save()
    print("[OK] state_management_comparison.svg")
def create_multi_provider_flow() -> None:
    """Create multi-provider integration flow diagram."""
    dwg = svgwrite.Drawing("multi_provider_flow.svg", size=("1000px", "600px"))
    dwg.add(dwg.rect(insert=(0, 0), size=("100%", "100%"), fill="white"))
    def draw_box(x: float, y: float, w: float, h: float, text: str, color: str) -> None:
        dwg.add(
            dwg.rect(
                insert=(x, y), size=(w, h), rx=5, ry=5, fill=color, stroke="#333", stroke_width
    =1.5
            )
        )
        lines = text.split("\n")
        line_height = 13
        total_height = len(lines) * line_height
        start_y = y + (h - total_height) / 2 + line_height - 3
        for i, line in enumerate(lines):
            dwg.add(
                dwg.text(
                    line,
                    insert=(x + w / 2, start_y + i * line_height),
                    text_anchor="middle",
                    font_size="10px",
                    font_family="Arial",
                    fill="black",
                )
            )
    def draw_arrow(x1: float, y1: float, x2: float, y2: float, label: str = "") -> None:
        dwg.add(dwg.line(start=(x1, y1), end=(x2, y2), stroke="#333", stroke_width=1.5))
        if label:
            mid_x = (x1 + x2) / 2
            mid_y = (y1 + y2) / 2
            dwg.add(dwg.text(label, insert=(mid_x + 10, mid_y), font_size="9px", fill="#666"))
    dwg.add(
        dwg.text(
            "Multi-Provider API Integration Architecture",
            insert=(500, 25),
            text_anchor="middle",
```

```
            font_size="14px",
            font_weight="bold",
            font_family="Arial",
        )
    )
    draw_box(400, 50, 200, 40, "Application Layer\n(REPL / Agent)", "#E3F2FD")
    draw_arrow(500, 90, 500, 120)
    draw_box(400, 120, 200, 40, "MultiProviderClient", "#BBDEFB")
    draw_box(100, 200, 160, 50, "Anthropic\nAdapter", "#C8E6C9")
    draw_box(300, 200, 160, 50, "OpenAI\nAdapter", "#FFF9C4")
    draw_box(500, 200, 160, 50, "MiniMax\nAdapter", "#FFCCBC")
    draw_box(700, 200, 160, 50, "Ollama\nAdapter", "#E1BEE7")
    draw_arrow(450, 160, 180, 200)
    draw_arrow(480, 160, 380, 200)
    draw_arrow(520, 160, 580, 200)
    draw_arrow(550, 160, 780, 200)
    draw_box(400, 160, 200, 30, "Message Format Conversion", "#F5F5F5")
    draw_box(100, 290, 160, 50, "Claude API\n(streaming)", "#A5D6A7")
    draw_box(300, 290, 160, 50, "OpenAI API\n(streaming)", "#FFF59D")
    draw_box(500, 290, 160, 50, "MiniMax API\n(streaming)", "#FFAB91")
    draw_box(700, 290, 160, 50, "Local LLM\n(streaming)", "#CE93D8")
    draw_arrow(180, 250, 180, 290)
    draw_arrow(380, 250, 380, 290)
    draw_arrow(580, 250, 580, 290)
    draw_arrow(780, 250, 780, 290)
    draw_box(
        50,
        400,
        300,
        150,
        "Message Conversion:\n\n* System prompt formatting\n* Tool schema translation\n* Stream
    ing normalization\n* Token counting\n* Cost calculation",
        "#E8F5E9",
    )
    draw_box(
        350,
        400,
        300,
        150,
        "Error Handling:\n\n* Retry logic with backoff\n* Rate limit handling\n* Timeout manage
    ment\n* Connection pooling\n* Fallback providers",
        "#FFF3E0",
    )
    draw_box(
        650,
        400,
        300,
        150,
        "Configuration:\n\n* API key management\n* Base URL customization\n* Model selection\n*
     Proxy support\n* Provider-specific options",
        "#F3E5F5",
    )
    draw_arrow(500, 340, 200, 400)
    draw_arrow(500, 340, 500, 400)
    draw_arrow(500, 340, 800, 400)
    dwg.save()
    print("[OK] multi_provider_flow.svg")
def main() -> None:
    """Generate all SVG figures."""
    output_dir = Path(__file__).parent.parent
    os.chdir(output_dir)
    print("Generating SVG figures...")
    create_architecture_comparison()
    create_tool_invocation_flow()
    create_state_management_comparison()
```

```
    create_multi_provider_flow()
    print("\nAll figures generated successfully!")
if __name__ == "__main__":
    main()
```

File: [213/252] scripts/guide_gemma3.py

```
"""Guide for using gemma3:1b effectively."""
print("""
+==============================================================+
|  ?? gemma3:1b ???????                                        |
+==============================================================+
???gemma3:1b ????????function calling?
?????
1??  ???????? Gemma 4
   ?-----------------------------------------------------?
   |  openlaoke --config                                  |
   |  ?? [1] gemma4:e2b  ?  [2] gemma4:e4b          |
   |                                                      |
   |  [OK] ????????                                   |
   |  [OK] ????                                          |
   |  [OK] ???????                                    |
   ?-----------------------------------------------------?
2??  ?? gemma3:1b ?????

   ??????????

   OpenLaoKe: ?Python?????CPU?????

   ????
   - ??????bash/read/write??
   - ??????
   - ?????????
3??  ?????????
   ?????Ollama??
   ?-----------------?----------?---------------?
   | ??              | ??     | ????       |
   ?-----------------?----------?---------------?
   | gemma4:e2b      | ~3GB     | [OK] ?????+?? |
   | gemma4:e4b      | ~6GB     | [OK] ????      |
   | gemma3:1b       | ~1GB     | ??  ?????   |
   | omnicoder-9b    | ~5GB     | [OK] ????      |
   ?-----------------?----------?---------------?
   ???????API key??
   ?-----------------?---------------------?
   | MiniMax         | [OK] ???            |
   | Aliyun          | [OK] ???            |
   ?-----------------?---------------------?
4??  ????????
   ??????
   "???????python?????CPU??"
   ?? gemma4:e2b ????
   [OK] ????????
   [OK] ????????
   [OK] ????/????
   [OK] ????????
?????
1. ??: openlaoke --config
2. ??: [1] gemma4:e2b
3. ?????????
""")
```

File: [214/252] scripts/publish.py

```
#!/usr/bin/env python3
"""Build and publish OpenLaoKe to PyPI.
Usage:
    python publish.py          # Build only
    python publish.py test     # Build + upload to TestPyPI
    python publish.py release  # Build + upload to PyPI
"""
import shutil
import subprocess
import sys
from pathlib import Path
ROOT = Path(__file__).parent
DIST = ROOT / "dist"
def run(cmd, **kwargs):
    print(f"\n>>> {cmd}")
    result = subprocess.run(cmd, shell=True, **kwargs)
    if result.returncode != 0:
        print(f"FAILED (exit {result.returncode})")
        sys.exit(result.returncode)
    return result
def ensure_tools():
    for pkg in ("build", "twine"):
        try:
            __import__(pkg)
        except ImportError:
            print(f"Installing {pkg}...")
            run(f"{sys.executable} -m pip install {pkg}")
def clean():
    for d in (DIST, ROOT / "build", ROOT / "openlaoke.egg-info"):
        if d.exists():
            print(f"Removing {d}")
            shutil.rmtree(d)
def build():
    clean()
    run(f"{sys.executable} -m build")
    wheels = list(DIST.glob("*.whl"))
    tarballs = list(DIST.glob("*.tar.gz"))
    print(f"\nBuilt: {[f.name for f in wheels + tarballs]}")
def upload(repository=None):
    cmd = f"{sys.executable} -m twine upload"
    if repository:
        cmd += f" --repository {repository}"
    cmd += " dist/*"
    run(cmd)
def main():
    ensure_tools()
    action = sys.argv[1] if len(sys.argv) > 1 else "build"
    if action == "build":
        build()
        print("\nBuild complete. To upload run:")
        print("  python publish.py test      # TestPyPI")
        print("  python publish.py release   # PyPI")
    elif action == "test":
        build()
        print("\nUploading to TestPyPI...")
        upload("testpypi")
        print("\nDone! Install with:")
        print("  pip install --index-url https://test.pypi.org/simple/ openlaoke")
    elif action == "release":
        build()
        print("\nUploading to PyPI...")
```

```
        upload()
        print("\nDone! Install with:")
        print("  pip install openlaoke")
    else:
        print(f"Unknown action: {action}")
        print("Usage: python publish.py [build|test|release]")
        sys.exit(1)
if __name__ == "__main__":
    main()
```

File: [215/252] scripts/test_no_tools.py

```
"""Test if current model supports tools."""
import asyncio
from openlaoke.core.multi_provider_api import MultiProviderClient
from openlaoke.types.providers import MultiProviderConfig
async def test_model():
    config = MultiProviderConfig.defaults()
    client = MultiProviderClient(config)
    # Test gemma3:1b
    model = "gemma3:1b"
    print(f"Testing {model}...")
    try:
        # Try a simple request without tools
        response, _, _ = await client.send_message(
            "You are helpful.",
            [{"role": "user", "content": "Calculate 2+2"}],
            None,  # No tools
            model
        )
        print(f"[OK] {model} works without tools")
        print(f"Response: {response.content[:100]}")
    except Exception as e:
        print(f"[X] {model} failed: {e}")
if __name__ == "__main__":
    asyncio.run(test_model())
```

File: [216/252] test_extended_api.py

```
#!/usr/bin/env python3
"""Test Extended Web API calls."""
import asyncio
from openlaoke.core.extended_web.providers_api import ExtendedWebClient
async def test_deepseek():
    """Test DeepSeek API."""
    print("Testing DeepSeek API...")

    client = ExtendedWebClient("deepseek-chat")

    try:
        messages = [{"role": "user", "content": "????????"}]
        response = await client.chat(messages)

        print(f"[OK] Response received")
        print(f"  Content: {response.get('content', '')[:100]}...")
        print(f"  Model: {response.get('model', 'unknown')}")
        print(f"  Usage: {response.get('usage', {})}")

        return True
    except Exception as e:
        print(f"[X] Error: {e}")
        return False
    finally:
        await client.close()
if __name__ == "__main__":
    result = asyncio.run(test_deepseek())
    if result:
        print("\n[OK] Test passed!")
    else:
        print("\n[X] Test failed - authentication required")
        print("Run: python3 quick_auth.py deepseek-chat")
```

File: [217/252] test_extended_web_all.py

```
#!/usr/bin/env python3
"""Test all Extended Web providers with auto cookie extraction."""
import asyncio
import json
import sqlite3
import tempfile
from pathlib import Path
from rich.console import Console
console = Console()
def get_firefox_profile_path():
    """Find Firefox profile path."""
    profiles_path = Path.home() / "Library" / "Application Support" / "Firefox" / "Profiles"

    if not profiles_path.exists():
        return None

    for profile_dir in profiles_path.iterdir():
        if profile_dir.is_dir() and "default" in profile_dir.name:
            return profile_dir

    return None
def extract_cookies(profile_path: Path, domain: str) -> str:
    """Extract cookies for a domain from Firefox."""
    cookies_file = profile_path / "cookies.sqlite"

    if not cookies_file.exists():
        return ""

    temp_cookie = tempfile.mktemp(suffix=".sqlite")
    import shutil
    shutil.copy2(cookies_file, temp_cookie)

    try:
        conn = sqlite3.connect(temp_cookie)
        cursor = conn.cursor()

        cursor.execute(
            "SELECT name, value FROM moz_cookies WHERE host LIKE ?",
            (f"%{domain}%",)
        )

        cookies = [f"{name}={value}" for name, value in cursor.fetchall()]
        conn.close()

        return "; ".join(cookies)

    finally:
        import os
        os.unlink(temp_cookie)
def save_auth(provider_type: str, cookie: str) -> None:
    """Save authentication."""
    auth_file = Path.home() / ".openlaoke" / "extended_web" / f"{provider_type}.json"
    auth_file.parent.mkdir(parents=True, exist_ok=True)

    auth_data = {
        "provider_type": provider_type,
        "cookie": cookie,
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 F
    irefox/115.0",
        "saved_at": "auto-extracted",
        "browser": "firefox",
```

```
    }

    with open(auth_file, "w") as f:
        json.dump(auth_data, f, indent=2)
async def test_deepseek(cookie: str):
    """Test DeepSeek."""
    console.print("\n[bold cyan]Testing DeepSeek...[/bold cyan]")

    try:
        from openlaoke.core.extended_web.clients import DeepSeekWebClient

        client = DeepSeekWebClient(cookie=cookie)
        await client.init()

        messages = [{"role": "user", "content": "hi"}]
        response = await client.chat_completions(messages)

        await client.close()

        content = response.get("content", "")
        if content:
            console.print(f"[green][OK] DeepSeek works![/green]")
            console.print(f"  Response: {content[:100]}...")
            return True
        else:
            console.print(f"[yellow]? Empty response[/yellow]")
            return False

    except Exception as e:
        console.print(f"[red][X] DeepSeek failed: {e}[/red]")
        return False
async def test_qwen(cookie: str):
    """Test Qwen."""
    console.print("\n[bold cyan]Testing Qwen...[/bold cyan]")

    try:
        from openlaoke.core.extended_web.clients import QwenWebClient

        client = QwenWebClient(cookie=cookie)
        await client.init()

        messages = [{"role": "user", "content": "hi"}]
        response = await client.chat_completions(messages)

        await client.close()

        content = response.get("content", "")
        if content:
            console.print(f"[green][OK] Qwen works![/green]")
            console.print(f"  Response: {content[:100]}...")
            return True
        else:
            console.print(f"[yellow]? Empty response[/yellow]")
            return False

    except Exception as e:
        console.print(f"[red][X] Qwen failed: {e}[/red]")
        return False
async def test_glm(cookie: str):
    """Test GLM."""
    console.print("\n[bold cyan]Testing GLM...[/bold cyan]")

    try:
        from openlaoke.core.extended_web.clients import GLMWebClient
```

```
        client = GLMWebClient(cookie=cookie)
        await client.init()

        messages = [{"role": "user", "content": "hi"}]
        response = await client.chat_completions(messages)

        await client.close()

        content = response.get("content", "")
        if content:
            console.print(f"[green][OK] GLM works![/green]")
            console.print(f"  Response: {content[:100]}...")
            return True
        else:
            console.print(f"[yellow]? Empty response[/yellow]")
            return False

    except Exception as e:
        console.print(f"[red][X] GLM failed: {e}[/red]")
        return False
async def main():
    """Main test function."""
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print("[bold cyan]  Extended Web Auto Test[/bold cyan]")
    console.print("[bold cyan]========================================[/bold cyan]")

    # Get Firefox profile
    profile_path = get_firefox_profile_path()

    if not profile_path:
        console.print("[red][X] Firefox profile not found[/red]")
        return

    console.print(f"[green][OK] Found Firefox profile: {profile_path}[/green]")

    # Extract cookies
    console.print()
    console.print("[bold]Extracting cookies...[/bold]")

    deepseek_cookie = extract_cookies(profile_path, "deepseek.com")
    qwen_cookie = extract_cookies(profile_path, "qwen.ai")
    glm_cookie = extract_cookies(profile_path, "chatglm.cn")

    # Save cookies
    if deepseek_cookie:
        save_auth("deepseek-chat", deepseek_cookie)
        console.print(f"[green][OK] DeepSeek cookie extracted ({len(deepseek_cookie)} chars)[/g
    reen]")
    else:
        console.print(f"[yellow]? DeepSeek cookie not found[/yellow]")

    if qwen_cookie:
        save_auth("qwen-web", qwen_cookie)
        console.print(f"[green][OK] Qwen cookie extracted ({len(qwen_cookie)} chars)[/green]")
    else:
        console.print(f"[yellow]? Qwen cookie not found[/yellow]")

    if glm_cookie:
        save_auth("glm-web", glm_cookie)
        console.print(f"[green][OK] GLM cookie extracted ({len(glm_cookie)} chars)[/green]")
    else:
        console.print(f"[yellow]? GLM cookie not found[/yellow]")

    # Test providers
```

```
    console.print()
    console.print("[bold]Testing providers...[/bold]")

    results = []

    if deepseek_cookie:
        results.append(("DeepSeek", await test_deepseek(deepseek_cookie)))
    else:
        console.print("\n[yellow]? Skipping DeepSeek (no cookie)[/yellow]")

    if qwen_cookie:
        results.append(("Qwen", await test_qwen(qwen_cookie)))
    else:
        console.print("\n[yellow]? Skipping Qwen (no cookie)[/yellow]")

    if glm_cookie:
        results.append(("GLM", await test_glm(glm_cookie)))
    else:
        console.print("\n[yellow]? Skipping GLM (no cookie)[/yellow]")

    # Summary
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print("[bold]Test Summary[/bold]")
    console.print("[bold cyan]========================================[/bold cyan]")

    for name, result in results:
        status = "[green][OK] PASS[/green]" if result else "[red][X] FAIL[/red]"
        console.print(f"  {status} - {name}")

    passed = sum(1 for _, r in results if r)
    total = len(results)

    console.print()
    console.print(f"[bold]Results: {passed}/{total} tests passed[/bold]")

    if passed == total:
        console.print("\n[bold green]? All tests passed![/bold green]")
        console.print("\n[dim]You can now use:[/dim]")
        console.print("  openlaoke --provider extended-web --model deepseek-chat")
        console.print("  openlaoke --provider extended-web --model qwen-web")
        console.print("  openlaoke --provider extended-web --model glm-web")
    else:
        console.print("\n[yellow]? Some tests failed[/yellow]")
if __name__ == "__main__":
    asyncio.run(main())
```

File: [218/252] test_firefox_launch.py

```
#!/usr/bin/env python3
"""Test Firefox launch with Selenium."""
from selenium import webdriver
from selenium.webdriver.firefox.options import Options
from selenium.webdriver.firefox.service import Service
import tempfile
print("Testing Firefox launch...")
# Setup Firefox options
options = Options()
profile_dir = tempfile.mkdtemp(prefix="test-firefox-")
options.add_argument("-profile")
options.add_argument(profile_dir)
# Don't use headless - we want to see the browser
# options.add_argument("-headless")
print(f"Profile dir: {profile_dir}")
print("Launching Firefox...")
try:
    # Launch Firefox
    driver = webdriver.Firefox(options=options)

    print("[OK] Firefox launched successfully!")
    print("Opening login page...")

    # Open a test page
    driver.get("https://chat.deepseek.com")

    print("[OK] Page loaded. Browser should be visible.")
    print("Press Ctrl+C to close...")

    # Keep browser open
    import time
    try:
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        pass

    driver.quit()
    print("[OK] Firefox closed")

except Exception as e:
    print(f"[X] Error: {e}")
    import traceback
    traceback.print_exc()
```

File: [219/252] test_free_models.py

```
#!/usr/bin/env python3
"""?? OpenLaoKe ???????"""
import asyncio
from openlaoke.types.providers import MultiProviderConfig
from openlaoke.core.multi_provider_api import MultiProviderClient
async def test_free_model(model: str, prompt: str = "??") -> None:
    """????????"""
    config = MultiProviderConfig.defaults()
    config.active_provider = "opencode"
    config.active_model = model
    client = MultiProviderClient(config)
    try:
        messages = [{"role": "user", "content": prompt}]
        response, usage, cost = await client.send_message(
            system_prompt="??????? AI ??", messages=messages, model=model, max_tokens=100
        )
        print(f"[OK] {model}: {response.content[:50] if response.content else 'No content'}..."
    )
        print(f"  Token: ??={usage.input_tokens}, ??={usage.output_tokens}")
        print(f"  ?????=${cost.input_cost:.6f}, ??=${cost.output_cost:.6f}")
    except Exception as e:
        print(f"[X] {model}: {str(e)[:100]}")
    finally:
        await client.close()
async def main():
    """????????"""
    print("=" * 60)
    print("OpenLaoKe ??????")
    print("=" * 60)
    free_models = [
        "big-pickle",
        "mimo-v2-flash-free",
        "gpt-5-nano",
    ]
    for model in free_models:
        await test_free_model(model)
        print()
    print("=" * 60)
    print("?????")
    print("=" * 60)
if __name__ == "__main__":
    asyncio.run(main())
```

File: [220/252] test_qwen_auth.py

```
#!/usr/bin/env python3
"""Test Qwen Web authentication and API call."""
import asyncio
import json
from pathlib import Path
from rich.console import Console
console = Console()
def extract_cookie_from_firefox() -> str:
    """Help user extract cookie from Firefox.

    Returns:
        Cookie string
    """
    console.print()
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print("[bold cyan]  Extract Cookie from Firefox[/bold cyan]")
    console.print("[bold cyan]========================================[/bold cyan]")
    console.print()
    console.print("[bold]Please follow these steps:[/bold]")
    console.print()
    console.print("  [dim]1. Open Firefox and go to: https://www.qianwen.com[/dim]")
    console.print("  [dim]2. Make sure you're logged in[/dim]")
    console.print("  [dim]3. Press F12 to open Developer Tools[/dim]")
    console.print("  [dim]4. Go to 'Storage' tab -> 'Cookies' -> 'https://www.qianwen.com'[/dim
    ]")
    console.print("  [dim]5. Find these cookies:[/dim]")
    console.print("      [cyan]_tb_token_[/cyan]")
    console.print("      [cyan]cookie2[/cyan]")
    console.print("  [dim]6. Copy the cookie values[/dim]")
    console.print()

    tb_token = console.input("  Enter [cyan]_tb_token_[/cyan] value: ")
    cookie2 = console.input("  Enter [cyan]cookie2[/cyan] value: ")

    cookie_string = f"_tb_token_={tb_token}; cookie2={cookie2}"

    return cookie_string
def save_cookie(provider_type: str, cookie: str) -> None:
    """Save cookie to auth file.

    Args:
        provider_type: Provider type
        cookie: Cookie string
    """
    auth_file = Path.home() / ".openlaoke" / "extended_web" / f"{provider_type}.json"
    auth_file.parent.mkdir(parents=True, exist_ok=True)

    auth_data = {
        "provider_type": provider_type,
        "cookie": cookie,
        "user_agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 F
    irefox/115.0",
        "saved_at": asyncio.get_event_loop().time(),
        "browser": "firefox",
    }

    with open(auth_file, "w") as f:
        json.dump(auth_data, f, indent=2)

    console.print()
    console.print(f"[green][OK] Cookie saved to: {auth_file}[/green]")
```

```
async def test_qwen_api(cookie: str) -> bool:
    """Test Qwen Web API call.

    Args:
        cookie: Cookie string

    Returns:
        True if successful
    """
    console.print()
    console.print("[bold]Testing Qwen Web API...[/bold]")
    console.print()

    try:
        from openlaoke.core.extended_web.clients import QwenWebClient

        client = QwenWebClient(
            cookie=cookie,
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101
    Firefox/115.0",
        )
        await client.init()

        messages = [{"role": "user", "content": "???????????"}]
        response = await client.chat_completions(
            messages=messages,
            model="qwen-turbo",
            max_tokens=200,
        )

        await client.close()

        console.print("[green][OK] API call successful![/green]")
        console.print()

        content = response.get("choices", [{}])[0].get("message", {}).get("content", "")
        if content:
            console.print(f"[bold]Response:[/bold]")
            console.print(f"  {content[:200]}")
        else:
            console.print("[yellow]? Empty response[/yellow]")
            console.print(f"Full response: {response}")

        return True

    except Exception as e:
        console.print(f"[red][X] API call failed: {e}[/red]")
        import traceback
        traceback.print_exc()
        return False
async def main():
    """Main entry point."""
    console.print()
    console.print("[bold cyan]Qwen Web Authentication Test[/bold cyan]")
    console.print()

    # Check existing auth
    auth_file = Path.home() / ".openlaoke" / "extended_web" / "qwen-web.json"

    if auth_file.exists():
        with open(auth_file) as f:
            auth_data = json.load(f)

        cookie = auth_data.get("cookie", "")
```

```
        if cookie and cookie != "placeholder_need_manual_extraction":
            console.print(f"[green][OK] Found existing cookie in {auth_file}[/green]")
            console.print(f"  Cookie length: {len(cookie)}")
            console.print()

            if await test_qwen_api(cookie):
                console.print()
                console.print("[bold green][OK] Authentication verified successfully![/bold gre
    en]")
                return
        else:
            console.print("[yellow]? Existing cookie is placeholder, need to extract real cooki
    e[/yellow]")

    # Extract cookie from Firefox
    cookie = extract_cookie_from_firefox()

    # Save cookie
    save_cookie("qwen-web", cookie)

    # Test API
    console.print()
    if await test_qwen_api(cookie):
        console.print()
        console.print("[bold green][OK] Authentication and API test successful![/bold green]")
        console.print()
        console.print("[dim]You can now use OpenLaoKe with qwen-web:[/dim]")
        console.print("  openlaoke")
        console.print()
    else:
        console.print()
        console.print("[yellow]? API test failed, but cookie is saved.[/yellow]")
        console.print("[dim]You can try again later.[/dim]")
if __name__ == "__main__":
    asyncio.run(main())
```

File: [221/252] tests/__init__.py

```
"""Tests for OpenLaoKe."""
```

File: [222/252] tests/test_architecture_assembler.py

```
"""Tests for architecture system - assembler."""
import tempfile
from pathlib import Path
import pytest
from openlaoke.core.architecture.assembler import (
    AssemblyResult,
    ValidationResult,
    CodeAssembler,
    IntegrationValidator,
)
from openlaoke.core.architecture.decomposer import (
    AtomicTask,
    TaskGraph,
    FineGrainedDecomposer,
)
from openlaoke.core.architecture.interfaces import (
    CodeTemplate,
    ComponentSpec,
    ComponentType,
)
from openlaoke.core.model_assessment.types import ModelTier
class TestValidationResult:
    def test_validation_result_creation(self):
        result = ValidationResult(
            is_valid=True,
            errors=[],
            warnings=["Minor issue"],
            suggestions=["Consider refactoring"],
        )
        assert result.is_valid
        assert len(result.warnings) == 1
        assert len(result.suggestions) == 1
    def test_validation_result_with_errors(self):
        result = ValidationResult(
            is_valid=False,
            errors=["Error 1", "Error 2"],
        )
        assert not result.is_valid
        assert len(result.errors) == 2
class TestAssemblyResult:
    def test_assembly_result_creation(self):
        result = AssemblyResult(
            success=True,
            code="def test(): pass",
            errors=[],
            warnings=[],
        )
        assert result.success
        assert "def test" in result.code
    def test_assembly_result_with_tests(self):
        result = AssemblyResult(
            success=True,
            code="...",
            test_results={"test1": True, "test2": False},
        )
        assert result.test_results["test1"]
        assert not result.test_results["test2"]
class TestCodeAssembler:
    def test_assembler_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
```

```
        assert assembler.project_root == Path(tmpdir)
def test_assemble_simple_task(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        assembler = CodeAssembler(Path(tmpdir))
        spec = ComponentSpec(
            name="test_func",
            component_type=ComponentType.FUNCTION,
        )
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def test_func(): pass",
        )
        task = AtomicTask(
            task_id="test_task",
            description="Test task",
            component_spec=spec,
            template=template,
        )
        graph = TaskGraph(root_task_id="root")
        graph.add_task(task)
        result = assembler.assemble_atomic_task(task, graph)
        assert result.success or len(result.errors) > 0
def test_assemble_task_with_dependencies(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        assembler = CodeAssembler(Path(tmpdir))
        spec1 = ComponentSpec(
            name="func1",
            component_type=ComponentType.FUNCTION,
        )
        spec2 = ComponentSpec(
            name="func2",
            component_type=ComponentType.FUNCTION,
        )
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def func(): pass",
        )
        task1 = AtomicTask(
            task_id="task1",
            description="Task 1",
            component_spec=spec1,
            template=template,
        )
        task2 = AtomicTask(
            task_id="task2",
            description="Task 2",
            component_spec=spec2,
            template=template,
            dependencies=["task1"],
        )
        graph = TaskGraph(root_task_id="root")
        graph.add_task(task1)
        graph.add_task(task2)
        graph.mark_completed("task1")
        result = assembler.assemble_atomic_task(task2, graph)
        assert result is not None
def test_validate_code_syntax(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        assembler = CodeAssembler(Path(tmpdir))
        spec = ComponentSpec(
            name="test",
            component_type=ComponentType.FUNCTION,
        )
```

```
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def test(): pass",
                validation_rules=["Must have type hints"],
            )
            task = AtomicTask(
                task_id="test",
                description="Test",
                component_spec=spec,
                template=template,
            )
            validation = assembler._validate_code("def test(): pass", task)
            assert validation is not None
class TestIntegrationValidator:
    def test_validator_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            validator = IntegrationValidator(Path(tmpdir))
            assert validator.project_root == Path(tmpdir)
    def test_validate_empty_code(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            validator = IntegrationValidator(Path(tmpdir))
            result = AssemblyResult(success=False, code="")
            validation = validator.validate_assembly(result)
            assert not validation.is_valid
            assert "Empty code" in " ".join(validation.errors)
    def test_validate_valid_code(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            validator = IntegrationValidator(Path(tmpdir))
            code = '''"""Module docstring."""
def test_func():
    """Test function."""
    return 42
'''
            result = AssemblyResult(success=True, code=code)
            validation = validator.validate_assembly(result)
            assert validation.is_valid or len(validation.errors) == 0
class TestAssemblyWorkflow:
    def test_full_assembly_workflow(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
            spec = ComponentSpec(
                name="simple_func",
                component_type=ComponentType.FUNCTION,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def simple_func(): return 42",
            )
            task = AtomicTask(
                task_id="simple_func_main",
                description="Implement simple_func",
                component_spec=spec,
                template=template,
                test_required=False,
            )
            graph = TaskGraph(root_task_id="root")
            graph.add_task(task)
            result = assembler.assemble_task_graph(graph)
            assert result is not None
            assert isinstance(result.code, str)
    def test_assembly_with_context(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
```

```
            spec = ComponentSpec(
                name="add",
                component_type=ComponentType.FUNCTION,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def {function_name}({parameters}) -> {return_type}:\n    return {retu
    rn_value}",
            )
            task = AtomicTask(
                task_id="add_main",
                description="Add function",
                component_spec=spec,
                template=template,
                context={
                    "function_name": "add",
                    "parameters": "a: int, b: int",
                    "return_type": "int",
                    "return_value": "a + b",
                },
            )
            graph = TaskGraph(root_task_id="root")
            graph.add_task(task)
            result = assembler.assemble_task_graph(graph)
            assert result is not None
class TestCodeOrganization:
    def test_organize_imports_and_code(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
            imports = {"import os", "import sys"}
            code_parts = ["def func1(): pass", "def func2(): pass"]
            organized = assembler._organize_code(imports, code_parts)
            assert "import os" in organized
            assert "import sys" in organized
            assert "def func1" in organized
            assert "def func2" in organized
    def test_auto_fix_missing_docstring(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
            code = "def test(): pass"
            validation = ValidationResult(
                is_valid=False,
                errors=["Missing docstring"],
            )
            can_fix = assembler._can_auto_fix(validation)
            assert can_fix
class TestTaskGraphAssembly:
    def test_sequential_assembly(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            assembler = CodeAssembler(Path(tmpdir))
            specs = [
                ComponentSpec(name=f"func{i}", component_type=ComponentType.FUNCTION)
                for i in range(3)
            ]
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def func(): pass",
            )
            tasks = [
                AtomicTask(
                    task_id=f"task{i}",
                    description=f"Task {i}",
                    component_spec=spec,
```

```
                template=template,
                test_required=False,
            )
            for i, spec in enumerate(specs)
        ]
        graph = TaskGraph(root_task_id="root")
        for task in tasks:
            graph.add_task(task)
        result = assembler.assemble_task_graph(graph)
        assert result is not None
def test_dependency_based_assembly(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        assembler = CodeAssembler(Path(tmpdir))
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def func(): pass",
        )
        task1 = AtomicTask(
            task_id="imports",
            description="Imports",
            component_spec=ComponentSpec(name="imports", component_type=ComponentType.FUNCT
ION),
            template=template,
            test_required=False,
        )
        task2 = AtomicTask(
            task_id="main",
            description="Main",
            component_spec=ComponentSpec(name="main", component_type=ComponentType.FUNCTION
),
            template=template,
            dependencies=["imports"],
            test_required=False,
        )
        graph = TaskGraph(root_task_id="root")
        graph.add_task(task1)
        graph.add_task(task2)
        result = assembler.assemble_task_graph(graph)
        assert result is not None
```

File: [223/252] tests/test_architecture_decomposer.py

```
"""Tests for architecture system - decomposer."""
import pytest
from openlaoke.core.architecture.decomposer import (
    AtomicTask,
    TaskGraph,
    FineGrainedDecomposer,
    create_decomposer_for_model,
)
from openlaoke.core.architecture.interfaces import (
    APISpec,
    ComponentSpec,
    ComponentType,
    TaskSize,
)
from openlaoke.core.model_assessment.types import ModelTier
class TestAtomicTask:
    def test_atomic_task_creation(self):
        spec = ComponentSpec(
            name="test_func",
            component_type=ComponentType.FUNCTION,
        )
        from openlaoke.core.architecture.interfaces import CodeTemplate
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def test(): pass",
        )
        task = AtomicTask(
            task_id="test_task",
            description="Test task",
            component_spec=spec,
            template=template,
        )
        assert task.task_id == "test_task"
        assert task.test_required is True
        assert task.estimated_lines == 10
    def test_atomic_task_with_dependencies(self):
        spec = ComponentSpec(
            name="test_func",
            component_type=ComponentType.FUNCTION,
        )
        from openlaoke.core.architecture.interfaces import CodeTemplate
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def test(): pass",
        )
        task = AtomicTask(
            task_id="test_task",
            description="Test task",
            component_spec=spec,
            template=template,
            dependencies=["dep1", "dep2"],
            estimated_lines=15,
        )
        assert len(task.dependencies) == 2
        assert task.estimated_lines == 15
class TestTaskGraph:
    def test_task_graph_creation(self):
        graph = TaskGraph(root_task_id="root")
        assert graph.root_task_id == "root"
```

```
    assert len(graph.tasks) == 0
    assert len(graph.completed) == 0
    assert len(graph.failed) == 0
def test_task_graph_add_task(self):
    spec = ComponentSpec(
        name="test",
        component_type=ComponentType.FUNCTION,
    )
    from openlaoke.core.architecture.interfaces import CodeTemplate
    template = CodeTemplate(
        name="test",
        description="Test",
        template="def test(): pass",
    )
    task = AtomicTask(
        task_id="task1",
        description="Task 1",
        component_spec=spec,
        template=template,
    )
    graph = TaskGraph(root_task_id="root")
    graph.add_task(task)
    assert "task1" in graph.tasks
def test_task_graph_get_ready_tasks(self):
    spec = ComponentSpec(
        name="test",
        component_type=ComponentType.FUNCTION,
    )
    from openlaoke.core.architecture.interfaces import CodeTemplate
    template = CodeTemplate(
        name="test",
        description="Test",
        template="def test(): pass",
    )
    task1 = AtomicTask(
        task_id="task1",
        description="Task 1",
        component_spec=spec,
        template=template,
    )
    task2 = AtomicTask(
        task_id="task2",
        description="Task 2",
        component_spec=spec,
        template=template,
        dependencies=["task1"],
    )
    graph = TaskGraph(root_task_id="root")
    graph.add_task(task1)
    graph.add_task(task2)
    ready = graph.get_ready_tasks()
    assert len(ready) == 1
    assert ready[0].task_id == "task1"
def test_task_graph_completion(self):
    spec = ComponentSpec(
        name="test",
        component_type=ComponentType.FUNCTION,
    )
    from openlaoke.core.architecture.interfaces import CodeTemplate
    template = CodeTemplate(
        name="test",
        description="Test",
        template="def test(): pass",
    )
    task1 = AtomicTask(
```

```
            task_id="task1",
            description="Task 1",
            component_spec=spec,
            template=template,
        )
        task2 = AtomicTask(
            task_id="task2",
            description="Task 2",
            component_spec=spec,
            template=template,
            dependencies=["task1"],
        )
        graph = TaskGraph(root_task_id="root")
        graph.add_task(task1)
        graph.add_task(task2)
        graph.mark_completed("task1")
        ready = graph.get_ready_tasks()
        assert len(ready) == 1
        assert ready[0].task_id == "task2"
    def test_task_graph_failure(self):
        graph = TaskGraph(root_task_id="root")
        graph.mark_failed("task1")
        assert "task1" in graph.failed
class TestFineGrainedDecomposer:
    def test_decomposer_creation(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        assert decomposer.model_tier == ModelTier.TIER_5_LIMITED
        assert decomposer.max_lines_per_task == 15
        assert decomposer.max_params_per_function == 3
        assert decomposer.max_complexity == 5
    def test_decomposer_tier5_limits(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        assert decomposer.max_lines_per_task == 15
        assert decomposer.max_params_per_function == 3
    def test_decomposer_tier1_limits(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_1_ADVANCED)
        assert decomposer.max_lines_per_task == 100
        assert decomposer.max_params_per_function == 10
    def test_decompose_simple_function(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="simple_func",
            component_type=ComponentType.FUNCTION,
            complexity_score=2,
        )
        tasks = decomposer.decompose_function(spec)
        assert len(tasks) >= 1
        assert tasks[0].task_id == "simple_func_main"
    def test_decompose_function_with_test(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="func_with_test",
            component_type=ComponentType.FUNCTION,
            test_requirements=["unit test"],
        )
        tasks = decomposer.decompose_function(spec)
        test_tasks = [t for t in tasks if "test" in t.task_id.lower()]
        assert len(test_tasks) >= 1
    def test_decompose_class(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="test_class",
            component_type=ComponentType.CLASS,
            complexity_score=3,
        )
```

```
        tasks = decomposer.decompose_class(spec)
        assert len(tasks) >= 2
    def test_decompose_module(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="test_module",
            component_type=ComponentType.MODULE,
            complexity_score=5,
        )
        tasks = decomposer.decompose_module(spec)
        assert len(tasks) >= 3
        import_tasks = [t for t in tasks if "imports" in t.task_id]
        export_tasks = [t for t in tasks if "exports" in t.task_id]
        assert len(import_tasks) >= 1
        assert len(export_tasks) >= 1
    def test_decompose_project(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        project_spec = {
            "modules": [
                {
                    "name": "module1",
                    "dependencies": [],
                    "complexity": 3,
                }
            ]
        }
        graph = decomposer.decompose_project(project_spec)
        assert graph is not None
        assert len(graph.tasks) > 0
    def test_parameter_grouping(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        params = {f"param_{i}": {"type": "int"} for i in range(7)}
        groups = decomposer._group_parameters(params)
        assert len(groups) >= 2
        for group in groups:
            assert len(group) <= decomposer.max_params_per_function
class TestCreateDecomposerForModel:
    def test_create_for_tier5(self):
        decomposer = create_decomposer_for_model(ModelTier.TIER_5_LIMITED)
        assert decomposer.model_tier == ModelTier.TIER_5_LIMITED
    def test_create_for_tier1(self):
        decomposer = create_decomposer_for_model(ModelTier.TIER_1_ADVANCED)
        assert decomposer.model_tier == ModelTier.TIER_1_ADVANCED
class TestTaskComplexityHandling:
    def test_simple_function_decomposition(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="add",
            component_type=ComponentType.FUNCTION,
            api_spec=APISpec(
                name="add",
                description="Add two numbers",
                input_schema={
                    "properties": {
                        "a": {"type": "float"},
                        "b": {"type": "float"},
                    }
                },
                output_schema={"type": "float"},
            ),
            complexity_score=1,
        )
        tasks = decomposer.decompose_function(spec)
        assert len(tasks) >= 1
        assert all(t.estimated_lines <= 15 for t in tasks)
```

```
    def test_complex_function_decomposition(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="complex_func",
            component_type=ComponentType.FUNCTION,
            api_spec=APISpec(
                name="complex",
                description="Complex function with many parameters",
                input_schema={"properties": {f"param_{i}": {"type": "int"} for i in range(5)}},
                output_schema={"type": "object"},
            ),
            complexity_score=7,
        )
        tasks = decomposer.decompose_function(spec)
        assert len(tasks) >= 2
class TestDependencyManagement:
    def test_task_dependencies(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="test_func",
            component_type=ComponentType.FUNCTION,
            test_requirements=["test"],
        )
        tasks = decomposer.decompose_function(spec)
        tasks_with_deps = [t for t in tasks if len(t.dependencies) > 0]
        assert len(tasks_with_deps) >= 1
    def test_module_dependency_chain(self):
        decomposer = FineGrainedDecomposer(ModelTier.TIER_5_LIMITED)
        spec = ComponentSpec(
            name="test_module",
            component_type=ComponentType.MODULE,
            complexity_score=4,
        )
        tasks = decomposer.decompose_module(spec)
        for task in tasks:
            for dep_id in task.dependencies:
                assert dep_id in [t.task_id for t in tasks]
```

File: [224/252] tests/test_architecture_interfaces.py

```
"""Tests for architecture system - interfaces."""
import pytest
from openlaoke.core.architecture.interfaces import (
    APISpec,
    CodeTemplate,
    ComponentSpec,
    ComponentType,
    TaskSize,
    STANDARD_TEMPLATES,
    STANDARD_API_SPECS,
    get_template_for_component,
    estimate_task_complexity,
    should_decompose_for_model,
)
class TestComponentType:
    def test_component_type_values(self):
        assert ComponentType.FUNCTION.value == "function"
        assert ComponentType.CLASS.value == "class"
        assert ComponentType.MODULE.value == "module"
        assert ComponentType.API_ENDPOINT.value == "api_endpoint"
        assert ComponentType.DATA_MODEL.value == "data_model"
        assert ComponentType.TEST.value == "test"
class TestTaskSize:
    def test_task_size_values(self):
        assert TaskSize.ATOMIC.value == "atomic"
        assert TaskSize.SMALL.value == "small"
        assert TaskSize.MEDIUM.value == "medium"
        assert TaskSize.LARGE.value == "large"
class TestCodeTemplate:
    def test_template_creation(self):
        template = CodeTemplate(
            name="test_template",
            description="Test template",
            template="def {name}(): pass",
            required_imports=["import os"],
            validation_rules=["Must have type hints"],
        )
        assert template.name == "test_template"
        assert template.description == "Test template"
        assert len(template.required_imports) == 1
        assert len(template.validation_rules) == 1
class TestAPISpec:
    def test_api_spec_creation(self):
        api = APISpec(
            name="test_api",
            description="Test API",
            input_schema={"type": "object", "properties": {"x": {"type": "int"}}},
            output_schema={"type": "object", "properties": {"result": {"type": "int"}}},
            error_codes={"E001": "Error 1"},
            version="1.0.0",
        )
        assert api.name == "test_api"
        assert api.version == "1.0.0"
        assert "E001" in api.error_codes
    def test_api_spec_with_examples(self):
        api = APISpec(
            name="test",
            description="Test",
            input_schema={},
            output_schema={},
            examples=[{"input": 1, "output": 2}],
```

```
        )
        assert len(api.examples) == 1
class TestComponentSpec:
    def test_component_spec_basic(self):
        spec = ComponentSpec(
            name="test_function",
            component_type=ComponentType.FUNCTION,
            complexity_score=3,
        )
        assert spec.name == "test_function"
        assert spec.component_type == ComponentType.FUNCTION
        assert spec.complexity_score == 3
        assert spec.test_requirements == []
        assert spec.dependencies == []
    def test_component_spec_with_requirements(self):
        spec = ComponentSpec(
            name="test_class",
            component_type=ComponentType.CLASS,
            test_requirements=["unit test", "integration test"],
            dependencies=["module_a", "module_b"],
        )
        assert len(spec.test_requirements) == 2
        assert len(spec.dependencies) == 2
class TestStandardTemplates:
    def test_standard_templates_exist(self):
        assert "function_basic" in STANDARD_TEMPLATES
        assert "class_basic" in STANDARD_TEMPLATES
        assert "api_endpoint" in STANDARD_TEMPLATES
        assert "data_model" in STANDARD_TEMPLATES
        assert "test_function" in STANDARD_TEMPLATES
    def test_function_template(self):
        template = STANDARD_TEMPLATES["function_basic"]
        assert "{function_name}" in template.template
        assert "{parameters}" in template.template
        assert "{return_type}" in template.template
        assert "from __future__ import annotations" in template.required_imports
    def test_class_template(self):
        template = STANDARD_TEMPLATES["class_basic"]
        assert "{class_name}" in template.template
        assert "__init__" in template.template
    def test_api_endpoint_template(self):
        template = STANDARD_TEMPLATES["api_endpoint"]
        assert "@router" in template.template
        assert "response_model" in template.template
        assert "fastapi" in " ".join(template.required_dependencies)
class TestStandardAPISpecs:
    def test_standard_api_specs_exist(self):
        assert "tool_call" in STANDARD_API_SPECS
        assert "state_update" in STANDARD_API_SPECS
        assert "component_assembly" in STANDARD_API_SPECS
    def test_tool_call_spec(self):
        spec = STANDARD_API_SPECS["tool_call"]
        assert spec.name == "tool_call"
        assert "name" in spec.input_schema["properties"]
        assert "arguments" in spec.input_schema["properties"]
        assert "success" in spec.output_schema["properties"]
class TestGetTemplateForComponent:
    def test_get_function_template(self):
        template = get_template_for_component(ComponentType.FUNCTION)
        assert template.name == "function_basic"
    def test_get_class_template(self):
        template = get_template_for_component(ComponentType.CLASS)
        assert template.name == "class_basic"
    def test_get_api_endpoint_template(self):
        template = get_template_for_component(ComponentType.API_ENDPOINT)
```

```
        assert template.name == "api_endpoint"
    def test_get_data_model_template(self):
        template = get_template_for_component(ComponentType.DATA_MODEL)
        assert template.name == "data_model"
    def test_get_test_template(self):
        template = get_template_for_component(ComponentType.TEST)
        assert template.name == "test_function"
class TestEstimateTaskComplexity:
    def test_simple_task(self):
        spec = ComponentSpec(
            name="simple_func",
            component_type=ComponentType.FUNCTION,
            complexity_score=1,
        )
        size = estimate_task_complexity(spec)
        assert size == TaskSize.ATOMIC
    def test_medium_task(self):
        spec = ComponentSpec(
            name="medium_func",
            component_type=ComponentType.FUNCTION,
            complexity_score=5,
            dependencies=["a", "b", "c", "d", "e", "f"],
        )
        size = estimate_task_complexity(spec)
        assert size in [TaskSize.SMALL, TaskSize.MEDIUM, TaskSize.LARGE]
    def test_complex_task_with_api(self):
        spec = ComponentSpec(
            name="complex_api",
            component_type=ComponentType.API_ENDPOINT,
            complexity_score=8,
            api_spec=APISpec(
                name="complex",
                description="Complex API",
                input_schema={"properties": {f"param_{i}": {"type": "int"} for i in range(8)}},
                output_schema={},
            ),
        )
        size = estimate_task_complexity(spec)
        assert size in [TaskSize.MEDIUM, TaskSize.LARGE]
class TestShouldDecomposeForModel:
    def test_tier5_needs_decomposition_for_medium(self):
        should = should_decompose_for_model(TaskSize.MEDIUM, "tier_5_limited")
        assert should is True
    def test_tier5_needs_decomposition_for_large(self):
        should = should_decompose_for_model(TaskSize.LARGE, "tier_5_limited")
        assert should is True
    def test_tier5_no_decomposition_for_atomic(self):
        should = should_decompose_for_model(TaskSize.ATOMIC, "tier_5_limited")
        assert should is False
    def test_tier5_no_decomposition_for_small(self):
        should = should_decompose_for_model(TaskSize.SMALL, "tier_5_limited")
        assert should is False
    def test_tier4_needs_decomposition_for_large(self):
        should = should_decompose_for_model(TaskSize.LARGE, "tier_4_basic")
        assert should is True
    def test_tier4_no_decomposition_for_medium(self):
        should = should_decompose_for_model(TaskSize.MEDIUM, "tier_4_basic")
        assert should is False
    def test_tier1_no_decomposition(self):
        should = should_decompose_for_model(TaskSize.LARGE, "tier_1_advanced")
        assert should is False
class TestComponentSpecValidation:
    def test_spec_with_all_fields(self):
        spec = ComponentSpec(
            name="complete_component",
```

```
    component_type=ComponentType.FUNCTION,
    api_spec=APISpec(
        name="api",
        description="API",
        input_schema={},
        output_schema={},
    ),
    dependencies=["dep1", "dep2"],
    test_requirements=["test1", "test2"],
    complexity_score=5,
)
assert spec.api_spec is not None
assert len(spec.dependencies) == 2
assert len(spec.test_requirements) == 2
```

File: [225/252] tests/test_architecture_orchestrator.py

```
"""Tests for architecture system - orchestrator."""
import tempfile
from pathlib import Path
import pytest
from openlaoke.core.architecture.orchestrator import (
    WorkflowStep,
    IncrementalWorkflow,
    IncrementalOrchestrator,
    create_orchestrator_for_model,
)
from openlaoke.core.architecture.decomposer import AtomicTask, TaskGraph
from openlaoke.core.architecture.interfaces import (
    CodeTemplate,
    ComponentSpec,
    ComponentType,
)
from openlaoke.core.model_assessment.types import ModelTier
from openlaoke.core.state import create_app_state
class TestWorkflowStep:
    def test_workflow_step_creation(self):
        spec = ComponentSpec(
            name="test_func",
            component_type=ComponentType.FUNCTION,
        )
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def test(): pass",
        )
        task = AtomicTask(
            task_id="test_task",
            description="Test task",
            component_spec=spec,
            template=template,
        )
        step = WorkflowStep(
            step_id="step1",
            task=task,
        )
        assert step.step_id == "step1"
        assert step.status == "pending"
        assert step.attempts == 0
    def test_workflow_step_max_attempts(self):
        spec = ComponentSpec(
            name="test",
            component_type=ComponentType.FUNCTION,
        )
        template = CodeTemplate(
            name="test",
            description="Test",
            template="def test(): pass",
        )
        task = AtomicTask(
            task_id="test",
            description="Test",
            component_spec=spec,
            template=template,
        )
        step = WorkflowStep(
            step_id="test",
            task=task,
```

```
            max_attempts=5,
        )
        assert step.max_attempts == 5
class TestIncrementalWorkflow:
    def test_workflow_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            from openlaoke.core.architecture.decomposer import FineGrainedDecomposer
            from openlaoke.core.architecture.assembler import CodeAssembler, IntegrationValidat
    or
            workflow = IncrementalWorkflow(
                workflow_id="test_workflow",
                project_spec={"name": "test"},
                model="gemma3:1b",
                model_tier=ModelTier.TIER_5_LIMITED,
                decomposer=FineGrainedDecomposer(ModelTier.TIER_5_LIMITED),
                assembler=CodeAssembler(Path(tmpdir)),
                validator=IntegrationValidator(Path(tmpdir)),
            )
            assert workflow.workflow_id == "test_workflow"
            assert workflow.model == "gemma3:1b"
    def test_workflow_progress(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            from openlaoke.core.architecture.decomposer import FineGrainedDecomposer
            from openlaoke.core.architecture.assembler import CodeAssembler, IntegrationValidat
    or
            workflow = IncrementalWorkflow(
                workflow_id="test",
                project_spec={},
                model="test",
                model_tier=ModelTier.TIER_5_LIMITED,
                decomposer=FineGrainedDecomposer(ModelTier.TIER_5_LIMITED),
                assembler=CodeAssembler(Path(tmpdir)),
                validator=IntegrationValidator(Path(tmpdir)),
            )
            spec = ComponentSpec(
                name="test",
                component_type=ComponentType.FUNCTION,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def test(): pass",
            )
            task = AtomicTask(
                task_id="test",
                description="Test",
                component_spec=spec,
                template=template,
            )
            step = WorkflowStep(step_id="test", task=task)
            workflow.steps["test"] = step
            workflow.completed_steps.append("test")
            progress = workflow.get_progress()
            assert progress["completed"] == 1
            assert progress["total_steps"] == 1
class TestIncrementalOrchestrator:
    def test_orchestrator_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator = IncrementalOrchestrator(
                app_state,
                "gemma3:1b",
                ModelTier.TIER_5_LIMITED,
            )
            assert orchestrator.model == "gemma3:1b"
```

```
        assert orchestrator.model_tier == ModelTier.TIER_5_LIMITED
def test_create_workflow(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        app_state = create_app_state(cwd=tmpdir)
        orchestrator = IncrementalOrchestrator(
            app_state,
            "gemma3:1b",
            ModelTier.TIER_5_LIMITED,
        )
        project_spec = {
            "name": "test_project",
            "modules": [
                {
                    "name": "test_module",
                    "dependencies": [],
                    "complexity": 2,
                }
            ],
        }
        workflow = orchestrator.create_workflow(project_spec)
        assert workflow is not None
        assert workflow.workflow_id in orchestrator.workflows
def test_get_workflow_status(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        app_state = create_app_state(cwd=tmpdir)
        orchestrator = IncrementalOrchestrator(
            app_state,
            "gemma3:1b",
            ModelTier.TIER_5_LIMITED,
        )
        project_spec = {"modules": [{"name": "test", "dependencies": [], "complexity": 2}]}
        workflow = orchestrator.create_workflow(project_spec)
        status = orchestrator.get_workflow_status(workflow.workflow_id)
        assert status is not None
        assert "progress_percentage" in status
        assert "steps" in status
def test_execute_simple_workflow(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        app_state = create_app_state(cwd=tmpdir)
        orchestrator = IncrementalOrchestrator(
            app_state,
            "gemma3:1b",
            ModelTier.TIER_5_LIMITED,
        )
        project_spec = {
            "modules": [
                {
                    "name": "simple_module",
                    "dependencies": [],
                    "complexity": 1,
                }
            ]
        }
        workflow = orchestrator.create_workflow(project_spec)
        result = orchestrator.execute_workflow(workflow.workflow_id)
        assert result is not None
        assert isinstance(result.success, bool)
def test_workflow_state_persistence(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        app_state = create_app_state(cwd=tmpdir)
        orchestrator = IncrementalOrchestrator(
            app_state,
            "gemma3:1b",
            ModelTier.TIER_5_LIMITED,
        )
```

```
            project_spec = {"modules": [{"name": "test", "dependencies": [], "complexity": 1}]}
            workflow = orchestrator.create_workflow(project_spec)
            orchestrator.save_workflow_state(workflow.workflow_id)
            loaded = orchestrator.load_workflow_state(workflow.workflow_id)
            assert loaded is not None
class TestCreateOrchestratorForModel:
    def test_create_for_gemma3_1b(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator = create_orchestrator_for_model(app_state, "gemma3:1b")
            assert orchestrator.model_tier == ModelTier.TIER_5_LIMITED
    def test_create_for_claude_opus(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator = create_orchestrator_for_model(app_state, "claude-opus-4")
            assert orchestrator.model_tier == ModelTier.TIER_1_ADVANCED
class TestRetryMechanism:
    def test_max_attempts_for_different_tiers(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator_t5 = IncrementalOrchestrator(
                app_state,
                "test",
                ModelTier.TIER_5_LIMITED,
            )
            spec = ComponentSpec(
                name="test",
                component_type=ComponentType.FUNCTION,
                complexity_score=1,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def test(): pass",
            )
            task = AtomicTask(
                task_id="test",
                description="Test",
                component_spec=spec,
                template=template,
            )
            max_attempts = orchestrator_t5._get_max_attempts_for_task(task)
            assert max_attempts >= 8
    def test_retry_for_complex_task(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator = IncrementalOrchestrator(
                app_state,
                "test",
                ModelTier.TIER_5_LIMITED,
            )
            spec = ComponentSpec(
                name="complex",
                component_type=ComponentType.FUNCTION,
                complexity_score=10,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def test(): pass",
            )
            task = AtomicTask(
                task_id="complex",
                description="Complex",
                component_spec=spec,
```

```
                template=template,
            )
            max_attempts = orchestrator._get_max_attempts_for_task(task)
            assert max_attempts >= 8
class TestProgressTracking:
    def test_progress_calculation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            from openlaoke.core.architecture.decomposer import FineGrainedDecomposer
            from openlaoke.core.architecture.assembler import CodeAssembler, IntegrationValidat
    or
            workflow = IncrementalWorkflow(
                workflow_id="test",
                project_spec={},
                model="test",
                model_tier=ModelTier.TIER_5_LIMITED,
                decomposer=FineGrainedDecomposer(ModelTier.TIER_5_LIMITED),
                assembler=CodeAssembler(Path(tmpdir)),
                validator=IntegrationValidator(Path(tmpdir)),
            )
            spec = ComponentSpec(
                name="test",
                component_type=ComponentType.FUNCTION,
            )
            template = CodeTemplate(
                name="test",
                description="Test",
                template="def test(): pass",
            )
            for i in range(5):
                task = AtomicTask(
                    task_id=f"task{i}",
                    description=f"Task {i}",
                    component_spec=spec,
                    template=template,
                )
                step = WorkflowStep(step_id=f"task{i}", task=task)
                workflow.steps[f"task{i}"] = step
            workflow.completed_steps = ["task0", "task1", "task2"]
            progress = workflow.get_progress()
            assert progress["completed"] == 3
            assert progress["total_steps"] == 5
            assert progress["progress_percentage"] == 60.0
class TestStepExecution:
    def test_get_ready_steps(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            app_state = create_app_state(cwd=tmpdir)
            orchestrator = IncrementalOrchestrator(
                app_state,
                "test",
                ModelTier.TIER_5_LIMITED,
            )
            project_spec = {"modules": [{"name": "test", "dependencies": [], "complexity": 1}]}
            workflow = orchestrator.create_workflow(project_spec)
            ready_steps = orchestrator._get_ready_steps(workflow)
            assert isinstance(ready_steps, list)
    def test_step_dependencies_satisfied(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            from openlaoke.core.architecture.decomposer import FineGrainedDecomposer
            from openlaoke.core.architecture.assembler import CodeAssembler, IntegrationValidat
    or
            workflow = IncrementalWorkflow(
                workflow_id="test",
                project_spec={},
                model="test",
                model_tier=ModelTier.TIER_5_LIMITED,
```

```
    decomposer=FineGrainedDecomposer(ModelTier.TIER_5_LIMITED),
    assembler=CodeAssembler(Path(tmpdir)),
    validator=IntegrationValidator(Path(tmpdir)),
)
spec = ComponentSpec(
    name="test",
    component_type=ComponentType.FUNCTION,
)
template = CodeTemplate(
    name="test",
    description="Test",
    template="def test(): pass",
)
task1 = AtomicTask(
    task_id="task1",
    description="Task 1",
    component_spec=spec,
    template=template,
)
task2 = AtomicTask(
    task_id="task2",
    description="Task 2",
    component_spec=spec,
    template=template,
    dependencies=["task1"],
)
step1 = WorkflowStep(step_id="task1", task=task1)
step2 = WorkflowStep(step_id="task2", task=task2)
workflow.steps["task1"] = step1
workflow.steps["task2"] = step2
assert step1.step_id not in workflow.completed_steps
```

File: [226/252] tests/test_classifier.py

```
"""Tests for permission classifier and command classification."""
import pytest
from openlaoke.types.core_types import PermissionMode, PermissionResult
from openlaoke.types.permissions import PermissionConfig, PermissionRule
class TestBashClassifier:
    def test_safe_read_commands(self):
        config = PermissionConfig.defaults()
        safe_commands = ["ls", "cat", "echo", "pwd", "whoami", "date", "uname"]
        for cmd in safe_commands:
            result = config.check_tool("Bash")
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_write_commands_need_approval(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Write")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_destructive_commands_need_approval(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Git")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
class TestSafeCommands:
    def test_read_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Read")
        assert result == PermissionResult.ALLOW
    def test_glob_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Glob")
        assert result == PermissionResult.ALLOW
    def test_grep_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Grep")
        assert result == PermissionResult.ALLOW
    def test_webfetch_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("WebFetch")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_websearch_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("WebSearch")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_question_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Question")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_listdirectory_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("ListDirectory")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_lsp_tool_allowed(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("LSP")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
class TestDangerousCommands:
    def test_edit_tool_ask(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Edit")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_write_tool_ask(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Write")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
```

```
    def test_applypatch_tool_ask(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("ApplyPatch")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_batch_tool_handling(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Batch")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
class TestDestructiveCommands:
    def test_git_tool_needs_approval(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Git")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK, PermissionResult.DENY)
    def test_agent_tool_ask(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("Agent")
        assert result == PermissionResult.ASK
    def test_taskkill_tool_handling(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("TaskKill")
        assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
class TestPermissionModes:
    def test_bypass_mode_allows_all(self):
        config = PermissionConfig(mode=PermissionMode.BYPASS)
        assert config.check_tool("Bash") == PermissionResult.ALLOW
        assert config.check_tool("Write") == PermissionResult.ALLOW
        assert config.check_tool("Git") == PermissionResult.ALLOW
        assert config.check_tool("Agent") == PermissionResult.ALLOW
    def test_auto_mode_allows_safe(self):
        config = PermissionConfig(mode=PermissionMode.AUTO)
        assert config.check_tool("Read") == PermissionResult.ALLOW
        assert config.check_tool("Glob") == PermissionResult.ALLOW
    def test_default_mode_asks_for_dangerous(self):
        config = PermissionConfig(mode=PermissionMode.DEFAULT)
        assert config.check_tool("Agent") == PermissionResult.ASK
    def test_unknown_tool_handling(self):
        config = PermissionConfig.defaults()
        result = config.check_tool("UnknownTool")
        assert result == PermissionResult.ASK
class TestPermissionRules:
    def test_custom_always_allow_rule(self):
        rule = PermissionRule("CustomTool*", PermissionResult.ALLOW)
        config = PermissionConfig(always_allow_rules=[rule])
        assert config.check_tool("CustomTool1") == PermissionResult.ALLOW
        assert config.check_tool("CustomTool2") == PermissionResult.ALLOW
    def test_custom_deny_rule(self):
        rule = PermissionRule("DangerousTool", PermissionResult.DENY)
        config = PermissionConfig(always_deny_rules=[rule])
        assert config.check_tool("DangerousTool") == PermissionResult.DENY
    def test_pattern_matching(self):
        rule = PermissionRule("Test*", PermissionResult.ALLOW)
        config = PermissionConfig(always_allow_rules=[rule])
        assert config.check_tool("TestTool") == PermissionResult.ALLOW
        assert config.check_tool("Testing") == PermissionResult.ALLOW
    def test_exact_match(self):
        rule = PermissionRule("ExactTool", PermissionResult.ALLOW)
        config = PermissionConfig(always_allow_rules=[rule])
        assert config.check_tool("ExactTool") == PermissionResult.ALLOW
        assert config.check_tool("ExactToolExtra") == PermissionResult.ASK
    def test_multiple_rules_priority(self):
        allow_rule = PermissionRule("Tool*", PermissionResult.ALLOW)
        deny_rule = PermissionRule("ToolDangerous", PermissionResult.DENY)
        config = PermissionConfig(
            always_allow_rules=[allow_rule],
            always_deny_rules=[deny_rule],
```

```
        )
        assert config.check_tool("ToolSafe") == PermissionResult.ALLOW
class TestToolApproval:
    def test_approve_tool(self):
        config = PermissionConfig.defaults()
        config.approve_tool("Bash", remember=True)
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_approve_unknown_tool(self):
        config = PermissionConfig.defaults()
        config.approve_tool("UnknownTool", remember=True)
        assert config.check_tool("UnknownTool") == PermissionResult.ALLOW
    def test_deny_tool(self):
        config = PermissionConfig.defaults()
        config.approve_tool("DangerousTool", remember=True)
        assert config.check_tool("DangerousTool") == PermissionResult.ALLOW
        config.deny_tool("DangerousTool")
        assert "DangerousTool" not in config.approved_tools
class TestPermissionConfigDefaults:
    def test_defaults_creation(self):
        config = PermissionConfig.defaults()
        assert config.mode == PermissionMode.DEFAULT
    def test_defaults_read_allowed(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Read") == PermissionResult.ALLOW
    def test_defaults_bash_allowed(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_defaults_agent_ask(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Agent") == PermissionResult.ASK
class TestPermissionResult:
    def test_result_values(self):
        assert PermissionResult.ALLOW.value == "allow"
        assert PermissionResult.ASK.value == "ask"
        assert PermissionResult.DENY.value == "deny"
    def test_result_comparison(self):
        assert PermissionResult.ALLOW != PermissionResult.ASK
        assert PermissionResult.ASK != PermissionResult.DENY
class TestPermissionMode:
    def test_mode_values(self):
        assert PermissionMode.DEFAULT.value == "default"
        assert PermissionMode.AUTO.value == "auto"
        assert PermissionMode.BYPASS.value == "bypass"
    def test_mode_from_string(self):
        assert PermissionMode("default") == PermissionMode.DEFAULT
        assert PermissionMode("auto") == PermissionMode.AUTO
        assert PermissionMode("bypass") == PermissionMode.BYPASS
    def test_invalid_mode(self):
        with pytest.raises(ValueError):
            PermissionMode("invalid")
class TestRulePatterns:
    def test_wildcard_end_pattern(self):
        rule = PermissionRule("Web*", PermissionResult.ALLOW)
        assert rule.matches("WebFetch")
        assert rule.matches("WebSearch")
        assert not rule.matches("Other")
    def test_wildcard_middle_pattern(self):
        rule = PermissionRule("Test*Tool", PermissionResult.ALLOW)
        config = PermissionConfig(always_allow_rules=[rule])
        assert config.check_tool("TestOneTool") == PermissionResult.ALLOW
        assert config.check_tool("TestTwoTool") == PermissionResult.ALLOW
        assert config.check_tool("TestingTool") == PermissionResult.ALLOW
    def test_no_wildcard_exact_match(self):
        rule = PermissionRule("Bash", PermissionResult.ALLOW)
        assert rule.matches("Bash")
```

```
        assert not rule.matches("BashTool")
        assert not rule.matches("OtherBash")
    def test_empty_pattern(self):
        rule = PermissionRule("", PermissionResult.ALLOW)
        assert not rule.matches("AnyTool")
class TestConcurrentSafeTools:
    def test_read_only_tools_safe(self):
        safe_tools = ["Read", "Glob", "Grep", "WebFetch", "WebSearch", "LSP"]
        config = PermissionConfig.defaults()
        for tool in safe_tools:
            result = config.check_tool(tool)
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
class TestToolCategories:
    def test_file_read_category(self):
        config = PermissionConfig.defaults()
        read_tools = ["Read", "Glob", "Grep", "ListDirectory"]
        for tool in read_tools:
            result = config.check_tool(tool)
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_file_write_category(self):
        config = PermissionConfig.defaults()
        write_tools = ["Write", "Edit", "ApplyPatch"]
        for tool in write_tools:
            result = config.check_tool(tool)
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_system_category(self):
        config = PermissionConfig.defaults()
        system_tools = ["Bash", "Git"]
        for tool in system_tools:
            result = config.check_tool(tool)
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
    def test_network_category(self):
        config = PermissionConfig.defaults()
        network_tools = ["WebFetch", "WebSearch"]
        for tool in network_tools:
            result = config.check_tool(tool)
            assert result in (PermissionResult.ALLOW, PermissionResult.ASK)
```

File: [227/252] tests/test_code_validator.py

```
"""Tests for incremental code validation."""
import pytest
from openlaoke.core.code_validator import (
    CodeValidator,
    IncrementalBuilder,
    ExecutionValidator,
    ValidationResult,
    create_validated_code,
)
class TestValidationResult:
    def test_result_creation(self):
        result = ValidationResult(is_valid=True)
        assert result.is_valid
        assert len(result.errors) == 0
    def test_result_with_errors(self):
        result = ValidationResult(
            is_valid=False,
            errors=["Error 1", "Error 2"],
        )
        assert not result.is_valid
        assert len(result.errors) == 2
class TestCodeValidator:
    def test_validator_creation(self):
        validator = CodeValidator()
        assert validator is not None
    def test_validate_correct_syntax(self):
        validator = CodeValidator()
        code = "x = 1\nprint(x)"
        result = validator.validate_syntax(code)
        assert result.is_valid
        assert len(result.errors) == 0
    def test_validate_syntax_error(self):
        validator = CodeValidator()
        code = "x = 1\nprint(x"
        result = validator.validate_syntax(code)
        assert not result.is_valid
        assert len(result.errors) > 0
    def test_validate_function_correct(self):
        validator = CodeValidator()
        func = '''
def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b
'''
        result = validator.validate_function(func)
        assert result.is_valid
    def test_validate_function_syntax_error(self):
        validator = CodeValidator()
        func = """
def broken(:
    pass
"""
        result = validator.validate_function(func)
        assert not result.is_valid
    def test_auto_fix_syntax(self):
        validator = CodeValidator()
        code = "print('hello'"
        fixed = validator.auto_fix_syntax(code)
        # Should not crash
        assert isinstance(fixed, str)
class TestIncrementalBuilder:
```

```
    def test_builder_creation(self):
        builder = IncrementalBuilder()
        assert len(builder.code_parts) == 0
    def test_add_imports(self):
        builder = IncrementalBuilder()
        imports = "import os\nimport sys"
        result = builder.add_imports(imports)
        assert result.is_valid
        assert len(builder.code_parts) == 1
    def test_add_function(self):
        builder = IncrementalBuilder()
        func = """
def test() -> None:
    pass
"""
        result = builder.add_function(func)
        assert result.is_valid
        assert len(builder.code_parts) == 1
    def test_get_final_code(self):
        builder = IncrementalBuilder()
        builder.add_imports("import os")
        builder.add_function("def test(): pass")
        code = builder.get_final_code()
        assert "import os" in code
        assert "def test" in code
class TestExecutionValidator:
    def test_test_imports_valid(self):
        validator = ExecutionValidator()
        code = "import os\nimport sys"
        result = validator.test_imports(code)
        assert result.is_valid
    def test_test_imports_invalid(self):
        validator = ExecutionValidator()
        code = "import nonexistent_module_xyz"
        result = validator.test_imports(code)
        assert not result.is_valid
    def test_test_syntax_valid(self):
        validator = ExecutionValidator()
        code = "x = 1\nprint(x)"
        result = validator.test_syntax_with_python(code)
        assert result.is_valid
    def test_test_syntax_invalid(self):
        validator = ExecutionValidator()
        code = "x = 1\nprint(x"
        result = validator.test_syntax_with_python(code)
        assert not result.is_valid
class TestCreateValidatedCode:
    def test_create_from_parts(self):
        parts = [
            "import os",
            "def test(): pass",
        ]
        code, result = create_validated_code(parts)
        assert "import os" in code
        assert "def test" in code
    def test_create_with_validation(self):
        parts = [
            "import os",
            "def broken(: pass",  # Syntax error
        ]
        code, result = create_validated_code(parts, validate_each=True)
        assert not result.is_valid  # Should catch the error
class TestIncrementalValidation:
    def test_step_by_step_validation(self):
        builder = IncrementalBuilder()
```

```
        # Step 1: Add imports
        result1 = builder.add_imports("import time")
        assert result1.is_valid
        # Step 2: Add function
        result2 = builder.add_function("def get_time() -> float:\n    return time.time()")
        assert result2.is_valid
        # Step 3: Add main
        result3 = builder.add_main_block("print(get_time())")
        assert result3.is_valid
        # Final validation
        final = builder.validate_final()
        assert final.is_valid
class TestErrorRecovery:
    def test_continue_after_error(self):
        builder = IncrementalBuilder()
        # Add invalid code
        result1 = builder.add_function("def broken(: pass")
        assert not result1.is_valid
        # Should still be able to add valid code
        result2 = builder.add_function("def valid(): pass")
        assert result2.is_valid
        code = builder.get_final_code()
        assert "def valid" in code
```

File: [228/252] tests/test_commands.py

```
"""Tests for command implementations."""
import asyncio
import os
import tempfile
from unittest.mock import MagicMock, patch
import pytest
from openlaoke.core.state import create_app_state
from openlaoke.commands.base import (
    CommandContext,
    CommandResult,
    DoctorCommand,
    InitCommand,
    McpCommand,
    ThemeCommand,
    HooksCommand,
    ExportCommand,
    UsageCommand,
    MemoryCommand,
    AgentsCommand,
    VimCommand,
    SettingsCommand,
    CompactCommand,
    CwdCommand,
    ResumeCommand,
    CommandsCommand,
)
from openlaoke.commands.registry import register_all
from openlaoke.types.core_types import (
    MessageRole,
    PermissionMode,
    TaskType,
    TaskStatus,
    TaskState,
    TokenUsage,
    UserMessage,
)
@pytest.fixture(autouse=True)
def setup_commands():
    register_all()
@pytest.fixture
def app_state():
    import openlaoke
    project_root = os.path.dirname(os.path.dirname(openlaoke.__file__))
    return create_app_state(cwd=project_root)
@pytest.fixture
def ctx(app_state):
    return CommandContext(app_state=app_state)
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
class TestDoctorCommand:
    @pytest.mark.asyncio
    async def test_execute(self, ctx):
        cmd = DoctorCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Diagnostic" in result.message
        assert "System:" in result.message
        assert "Python:" in result.message
    @pytest.mark.asyncio
```

```
    async def test_shows_environment(self, ctx):
        cmd = DoctorCommand()
        result = await cmd.execute(ctx)
        assert "Environment:" in result.message
        assert "Anthropic:" in result.message
        assert "OpenAI:" in result.message
    @pytest.mark.asyncio
    async def test_shows_tools(self, ctx):
        cmd = DoctorCommand()
        result = await cmd.execute(ctx)
        assert "Tools:" in result.message
        assert "git:" in result.message
        assert "python3:" in result.message
    @pytest.mark.asyncio
    async def test_shows_config(self, ctx):
        cmd = DoctorCommand()
        result = await cmd.execute(ctx)
        assert "Configuration:" in result.message
        assert "AGENTS.md:" in result.message
    @pytest.mark.asyncio
    async def test_shows_session(self, ctx):
        cmd = DoctorCommand()
        result = await cmd.execute(ctx)
        assert "Session:" in result.message
        assert "Session ID:" in result.message
        assert "Messages:" in result.message
    def test_command_properties(self):
        cmd = DoctorCommand()
        assert cmd.name == "doctor"
        assert "diagnostic" in cmd.description.lower()
class TestInitCommand:
    @pytest.mark.asyncio
    async def test_creates_agents_md(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        cmd = InitCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Created AGENTS.md" in result.message
        assert os.path.exists(os.path.join(temp_dir, "AGENTS.md"))
    @pytest.mark.asyncio
    async def test_agents_md_already_exists(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        agents_path = os.path.join(temp_dir, "AGENTS.md")
        with open(agents_path, "w") as f:
            f.write("existing content")
        cmd = InitCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "already exists" in result.message
    @pytest.mark.asyncio
    async def test_agents_md_content(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        cmd = InitCommand()
        await cmd.execute(ctx)
        with open(os.path.join(temp_dir, "AGENTS.md")) as f:
            content = f.read()
        assert "# AGENTS.md" in content
        assert "Project Overview" in content
        assert "Commands" in content
    def test_command_properties(self):
        cmd = InitCommand()
        assert cmd.name == "init"
        assert "AGENTS.md" in cmd.description
class TestMcpCommand:
    @pytest.mark.asyncio
```

```
    async def test_list_no_servers(self, ctx):
        cmd = McpCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "MCP Servers:" in result.message
    @pytest.mark.asyncio
    async def test_list_with_args(self, ctx):
        cmd = McpCommand()
        ctx.args = "list"
        result = await cmd.execute(ctx)
        assert result.success
        assert "MCP Servers:" in result.message
    @pytest.mark.asyncio
    async def test_add_server(self, ctx):
        cmd = McpCommand()
        ctx.args = "add test-server"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "edit" in result.message.lower()
    @pytest.mark.asyncio
    async def test_remove_server(self, ctx):
        cmd = McpCommand()
        ctx.args = "remove test-server"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "edit" in result.message.lower()
    @pytest.mark.asyncio
    async def test_enable_server(self, ctx):
        cmd = McpCommand()
        ctx.args = "enable test-server"
        result = await cmd.execute(ctx)
        assert result.success
        assert "enabled" in result.message
    @pytest.mark.asyncio
    async def test_disable_server(self, ctx):
        cmd = McpCommand()
        ctx.args = "disable test-server"
        result = await cmd.execute(ctx)
        assert result.success
        assert "disabled" in result.message
    @pytest.mark.asyncio
    async def test_invalid_usage(self, ctx):
        cmd = McpCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Usage:" in result.message
    def test_command_properties(self):
        cmd = McpCommand()
        assert cmd.name == "mcp"
        assert "MCP" in cmd.description
class TestThemeCommand:
    @pytest.mark.asyncio
    async def test_show_current_theme(self, ctx):
        cmd = ThemeCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Current theme:" in result.message
        assert "dark" in result.message
    @pytest.mark.asyncio
    async def test_set_valid_theme(self, ctx):
        cmd = ThemeCommand()
        ctx.args = "light"
        result = await cmd.execute(ctx)
        assert result.success
```

```
        assert "light" in result.message
        assert ctx.app_state.theme == "light"
    @pytest.mark.asyncio
    async def test_set_custom_theme(self, ctx):
        cmd = ThemeCommand()
        ctx.args = "custom"
        result = await cmd.execute(ctx)
        assert result.success
        assert ctx.app_state.theme == "custom"
    @pytest.mark.asyncio
    async def test_set_invalid_theme(self, ctx):
        cmd = ThemeCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Invalid theme" in result.message
    @pytest.mark.asyncio
    async def test_show_available_themes(self, ctx):
        cmd = ThemeCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Available themes:" in result.message
        assert "dark" in result.message
        assert "light" in result.message
        assert "custom" in result.message
    def test_command_properties(self):
        cmd = ThemeCommand()
        assert cmd.name == "theme"
        assert "theme" in cmd.description.lower()
class TestHooksCommand:
    @pytest.mark.asyncio
    async def test_list_hooks(self, ctx):
        cmd = HooksCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Registered Hooks:" in result.message
    @pytest.mark.asyncio
    async def test_list_with_args(self, ctx):
        cmd = HooksCommand()
        ctx.args = "list"
        result = await cmd.execute(ctx)
        assert result.success
        assert "Registered Hooks:" in result.message
    @pytest.mark.asyncio
    async def test_test_valid_hook(self, ctx):
        cmd = HooksCommand()
        ctx.args = "test pre_tool_call"
        result = await cmd.execute(ctx)
        assert result.success
        assert "Hook test" in result.message
    @pytest.mark.asyncio
    async def test_test_invalid_hook(self, ctx):
        cmd = HooksCommand()
        ctx.args = "test InvalidHook"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Invalid hook type" in result.message
    @pytest.mark.asyncio
    async def test_invalid_usage(self, ctx):
        cmd = HooksCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Usage:" in result.message
    def test_command_properties(self):
```

```
        cmd = HooksCommand()
        assert cmd.name == "hooks"
        assert "hooks" in cmd.description.lower()
class TestExportCommand:
    @pytest.mark.asyncio
    async def test_show_usage(self, ctx):
        cmd = ExportCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Usage:" in result.message
        assert "json" in result.message
        assert "markdown" in result.message
    @pytest.mark.asyncio
    async def test_export_json(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content="test"))
        cmd = ExportCommand()
        ctx.args = "json"
        result = await cmd.execute(ctx)
        assert result.success
        assert "exported" in result.message.lower()
    @pytest.mark.asyncio
    async def test_export_json_with_filename(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content="test"))
        cmd = ExportCommand()
        ctx.args = "json test_export.json"
        result = await cmd.execute(ctx)
        assert result.success
        assert "test_export.json" in result.message
    @pytest.mark.asyncio
    async def test_export_markdown(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content="test"))
        cmd = ExportCommand()
        ctx.args = "markdown"
        result = await cmd.execute(ctx)
        assert result.success
        assert "exported" in result.message.lower()
    @pytest.mark.asyncio
    async def test_export_markdown_alias(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content="test"))
        cmd = ExportCommand()
        ctx.args = "md"
        result = await cmd.execute(ctx)
        assert result.success
    @pytest.mark.asyncio
    async def test_invalid_format(self, ctx):
        cmd = ExportCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Invalid format" in result.message
    def test_command_properties(self):
        cmd = ExportCommand()
        assert cmd.name == "export"
        assert "Export" in cmd.description
class TestUsageCommand:
    @pytest.mark.asyncio
    async def test_show_usage(self, ctx):
        ctx.app_state.accumulate_tokens(TokenUsage(input_tokens=100, output_tokens=50))
        cmd = UsageCommand()
        result = await cmd.execute(ctx)
        assert result.success
```

```
        assert "Usage Statistics" in result.message
        assert "100" in result.message or "100," in result.message
    @pytest.mark.asyncio
    async def test_show_cost(self, ctx):
        cmd = UsageCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Cost Breakdown:" in result.message
        assert "Total cost:" in result.message
    @pytest.mark.asyncio
    async def test_show_session_info(self, ctx):
        cmd = UsageCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Session Info:" in result.message
        assert "Session ID:" in result.message
        assert "Messages:" in result.message
    def test_command_properties(self):
        cmd = UsageCommand()
        assert cmd.name == "usage"
        assert "usage" in cmd.description.lower()
class TestMemoryCommand:
    @pytest.mark.asyncio
    async def test_list_empty(self, ctx):
        cmd = MemoryCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Memory Storage:" in result.message
        assert "No memories" in result.message
    @pytest.mark.asyncio
    async def test_add_memory(self, ctx, temp_dir):
        memory_path = os.path.join(temp_dir, "memory.json")
        with patch("openlaoke.commands.base.MemoryCommand._list_memory") as mock_list:
            mock_list.return_value = CommandResult(message="Memory added: Test memory...")
            cmd = MemoryCommand()
            ctx.args = "add Test memory"
            result = cmd._add_memory(ctx, "Test memory")
            assert result.success
            assert "Memory added" in result.message
    @pytest.mark.asyncio
    async def test_remove_invalid_index(self, ctx):
        cmd = MemoryCommand()
        ctx.args = "remove abc"
        result = cmd._remove_memory(ctx, "abc")
        assert not result.success
        assert "Invalid index" in result.message
    @pytest.mark.asyncio
    async def test_clear_memory(self, ctx):
        cmd = MemoryCommand()
        ctx.args = "clear"
        result = cmd._clear_memory(ctx)
        assert result.success
        assert "cleared" in result.message.lower()
    @pytest.mark.asyncio
    async def test_invalid_usage(self, ctx):
        cmd = MemoryCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Usage:" in result.message
    def test_command_properties(self):
        cmd = MemoryCommand()
        assert cmd.name == "memory"
        assert "memory" in cmd.description.lower()
class TestAgentsCommand:
```

```
    @pytest.mark.asyncio
    async def test_show_agent_types(self, ctx):
        cmd = AgentsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Available Agent Types:" in result.message
    @pytest.mark.asyncio
    async def test_show_all_task_types(self, ctx):
        cmd = AgentsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        for task_type in TaskType:
            assert task_type.value in result.message
    @pytest.mark.asyncio
    async def test_show_active_tasks(self, ctx):
        task = TaskState(id="t1", type=TaskType.LOCAL_BASH, description="Test task")
        task.status = TaskStatus.RUNNING
        ctx.app_state.add_task(task)
        cmd = AgentsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Active Tasks:" in result.message
        assert "t1" in result.message
    @pytest.mark.asyncio
    async def test_show_no_active_tasks(self, ctx):
        cmd = AgentsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "No active tasks" in result.message
    def test_command_properties(self):
        cmd = AgentsCommand()
        assert cmd.name == "agents"
        assert "agent" in cmd.description.lower()
class TestVimCommand:
    @pytest.mark.asyncio
    async def test_show_current_mode(self, ctx):
        cmd = VimCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Vim mode:" in result.message
    @pytest.mark.asyncio
    async def test_enable_vim_mode(self, ctx):
        cmd = VimCommand()
        ctx.args = "on"
        result = await cmd.execute(ctx)
        assert result.success
        assert "enabled" in result.message
        assert ctx.app_state.vim_mode
    @pytest.mark.asyncio
    async def test_disable_vim_mode(self, ctx):
        ctx.app_state.vim_mode = True
        cmd = VimCommand()
        ctx.args = "off"
        result = await cmd.execute(ctx)
        assert result.success
        assert "disabled" in result.message
        assert not ctx.app_state.vim_mode
    @pytest.mark.asyncio
    async def test_toggle_vim_mode(self, ctx):
        cmd = VimCommand()
        ctx.args = "toggle"
        result = await cmd.execute(ctx)
        assert result.success
        assert ctx.app_state.vim_mode
        result = await cmd.execute(ctx)
```

```
        assert result.success
        assert not ctx.app_state.vim_mode
    @pytest.mark.asyncio
    async def test_invalid_usage(self, ctx):
        cmd = VimCommand()
        ctx.args = "invalid"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Usage:" in result.message
    def test_command_properties(self):
        cmd = VimCommand()
        assert cmd.name == "vim"
        assert "Vim" in cmd.description
class TestSettingsCommand:
    @pytest.mark.asyncio
    async def test_show_settings(self, ctx):
        cmd = SettingsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Settings:" in result.message
        assert "Model:" in result.message
        assert "Permissions:" in result.message
    @pytest.mark.asyncio
    async def test_shows_model(self, ctx):
        cmd = SettingsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert ctx.app_state.session_config.model in result.message
    @pytest.mark.asyncio
    async def test_shows_working_dir(self, ctx):
        cmd = SettingsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Working dir:" in result.message
    def test_command_properties(self):
        cmd = SettingsCommand()
        assert cmd.name == "settings"
        assert "settings" in cmd.description.lower()
class TestCompactCommand:
    @pytest.mark.asyncio
    async def test_compact_short_conversation(self, ctx):
        ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content="test"))
        cmd = CompactCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "too short" in result.message.lower()
    @pytest.mark.asyncio
    async def test_compact_long_conversation(self, ctx):
        for i in range(10):
            ctx.app_state.add_message(UserMessage(role=MessageRole.USER, content=f"msg{i}"))
        cmd = CompactCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "compacted" in result.message.lower()
    def test_command_properties(self):
        cmd = CompactCommand()
        assert cmd.name == "compact"
        assert "compact" in cmd.description.lower()
class TestCwdCommand:
    @pytest.mark.asyncio
    async def test_show_current_cwd(self, ctx):
        cmd = CwdCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Working directory:" in result.message
```

```
    @pytest.mark.asyncio
    async def test_change_to_valid_directory(self, ctx, temp_dir):
        cmd = CwdCommand()
        ctx.args = temp_dir
        result = await cmd.execute(ctx)
        assert result.success
        assert temp_dir in result.message
    @pytest.mark.asyncio
    async def test_change_to_invalid_directory(self, ctx):
        cmd = CwdCommand()
        ctx.args = "/nonexistent/path"
        result = await cmd.execute(ctx)
        assert not result.success
        assert "Not a directory" in result.message
    def test_command_properties(self):
        cmd = CwdCommand()
        assert cmd.name == "cwd"
        assert "directory" in cmd.description.lower()
class TestResumeCommand:
    @pytest.mark.asyncio
    async def test_resume_command(self, ctx):
        cmd = ResumeCommand()
        result = await cmd.execute(ctx)
        assert result.success
    def test_command_properties(self):
        cmd = ResumeCommand()
        assert cmd.name == "resume"
        assert "Resume" in cmd.description
class TestCommandsCommand:
    @pytest.mark.asyncio
    async def test_show_examples(self, ctx):
        cmd = CommandsCommand()
        result = await cmd.execute(ctx)
        assert result.success
        assert "Examples:" in result.message
    def test_command_properties(self):
        cmd = CommandsCommand()
        assert cmd.name == "commands"
        assert "example" in cmd.description.lower()
```

File: [229/252] tests/test_core.py

```
"""Tests for core systems."""
import asyncio
import os
import tempfile
from unittest.mock import AsyncMock, patch
import pytest
from openlaoke.core.state import AppState, create_app_state
from openlaoke.core.tool import Tool, ToolContext, ToolRegistry
from openlaoke.types.core_types import (
    MessageRole,
    PermissionMode,
    PermissionResult,
    TaskState,
    TaskStatus,
    TaskType,
    TokenUsage,
    ToolResultBlock,
    ToolUseBlock,
    UserMessage,
    is_terminal_task_status,
)
from openlaoke.types.permissions import PermissionConfig, PermissionRule
from openlaoke.commands.base import (
    ClearCommand,
    CostCommand,
    ExitCommand,
    HelpCommand,
    ModelCommand,
    PermissionCommand,
)
from openlaoke.commands.registry import get_command, parse_command, register_all
class TestCoreTypes:
    def test_is_terminal_task_status(self):
        assert is_terminal_task_status(TaskStatus.COMPLETED)
        assert is_terminal_task_status(TaskStatus.FAILED)
        assert is_terminal_task_status(TaskStatus.KILLED)
        assert not is_terminal_task_status(TaskStatus.PENDING)
        assert not is_terminal_task_status(TaskStatus.RUNNING)
    def test_token_usage(self):
        usage = TokenUsage(
            input_tokens=1000,
            output_tokens=500,
            cache_read_tokens=200,
            cache_creation_tokens=100,
        )
        assert usage.total_tokens == 1500
        other = TokenUsage(input_tokens=100, output_tokens=50)
        usage.accumulate(other)
        assert usage.input_tokens == 1100
        assert usage.output_tokens == 550
    def test_tool_use_block(self):
        block = ToolUseBlock(id="test_1", name="Bash", input={"command": "ls"})
        d = block.to_dict()
        assert d["type"] == "tool_use"
        assert d["id"] == "test_1"
        assert d["name"] == "Bash"
    def test_tool_result_block(self):
        result = ToolResultBlock(tool_use_id="test_1", content="output", is_error=False)
        d = result.to_dict()
        assert d["tool_use_id"] == "test_1"
        assert not d["is_error"]
```

```
    def test_user_message(self):
        msg = UserMessage(role=MessageRole.USER, content="hello")
        d = msg.to_dict()
        assert d["role"] == "user"
        assert d["content"] == "hello"
        assert d["type"] == "user"
class TestPermissionConfig:
    def test_bypass_mode(self):
        config = PermissionConfig(mode=PermissionMode.BYPASS)
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_auto_mode(self):
        config = PermissionConfig(mode=PermissionMode.AUTO)
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_default_mode_read_allowed(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Read") == PermissionResult.ALLOW
    def test_default_mode_bash_allowed(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_default_mode_agent_ask(self):
        config = PermissionConfig.defaults()
        assert config.check_tool("Agent") == PermissionResult.ASK
    def test_approve_tool(self):
        config = PermissionConfig.defaults()
        config.approve_tool("Bash", remember=True)
        assert config.check_tool("Bash") == PermissionResult.ALLOW
    def test_custom_rules(self):
        config = PermissionConfig(
            always_allow_rules=[
                PermissionRule("Custom*", PermissionResult.ALLOW),
            ],
        )
        assert config.check_tool("CustomTool") == PermissionResult.ALLOW
        assert config.check_tool("OtherTool") == PermissionResult.ASK
class TestToolRegistry:
    def test_register_and_get(self):
        registry = ToolRegistry()
        class TestTool(Tool):
            name = "TestTool"
            description = "A test tool"
            input_schema = {"type": "object", "properties": {}}
            async def call(self, ctx: ToolContext, **kwargs):
                return ToolResultBlock(tool_use_id=ctx.tool_use_id, content="ok")
        registry.register(TestTool())
        tool = registry.get("TestTool")
        assert tool is not None
        assert tool.name == "TestTool"
    def test_get_unknown(self):
        registry = ToolRegistry()
        assert registry.get("Unknown") is None
    def test_deferred_loading(self):
        registry = ToolRegistry()
        loaded = []
        def loader():
            loaded.append(True)
            class DeferredTool(Tool):
                name = "Deferred"
                description = "Deferred tool"
                input_schema = {}
                async def call(self, ctx, **kwargs):
                    return ToolResultBlock(tool_use_id=ctx.tool_use_id, content="deferred")
            return DeferredTool()
        registry.register_deferred("Deferred", loader)
        assert registry.get("Deferred") is not None
        assert len(loaded) == 1
```

```
        assert len(registry._deferred_loaders) == 0
    def test_get_all_for_prompt(self):
        registry = ToolRegistry()
        class T1(Tool):
            name = "T1"
            description = "Tool 1"
            input_schema = {"type": "object", "properties": {"x": {"type": "string"}}}
            async def call(self, ctx, **kwargs):
                return ToolResultBlock(tool_use_id=ctx.tool_use_id, content="ok")
        registry.register(T1())
        tools = registry.get_all_for_prompt()
        assert len(tools) == 1
        assert tools[0]["name"] == "T1"
        assert "properties" in tools[0]["input_schema"]
class TestAppState:
    def test_create_default(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            assert state.session_id.startswith("session_")
            assert len(state.messages) == 0
    def test_add_message(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            msg = UserMessage(role=MessageRole.USER, content="hello")
            state.add_message(msg)
            assert len(state.messages) == 1
            assert state.messages[0].content == "hello"
    def test_task_management(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            task = TaskState(id="t1", type=TaskType.LOCAL_BASH, description="test")
            state.add_task(task)
            assert state.get_task("t1") is not None
            assert len(state.get_all_tasks()) == 1
            task.status = TaskStatus.RUNNING
            state.update_task(task)
            assert state.get_task("t1").status == TaskStatus.RUNNING
    def test_token_accumulation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            state.accumulate_tokens(TokenUsage(input_tokens=100, output_tokens=50))
            state.accumulate_tokens(TokenUsage(input_tokens=200, output_tokens=100))
            assert state.token_usage.input_tokens == 300
            assert state.token_usage.output_tokens == 150
    def test_persist(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            persist_path = os.path.join(tmpdir, "session.json")
            state = create_app_state(persist_path=persist_path, cwd=tmpdir)
            state.add_message(UserMessage(role=MessageRole.USER, content="test"))
            state._persist()
            assert os.path.exists(persist_path)
            assert os.path.getsize(persist_path) > 0
class TestCommands:
    @pytest.fixture(autouse=True)
    def setup_commands(self):
        register_all()
    def test_parse_command(self):
        result = parse_command("/help")
        assert result == ("help", "")
        result = parse_command("/model claude-sonnet-4")
        assert result == ("model", "claude-sonnet-4")
        assert parse_command("hello") is None
        assert parse_command("") is None
    def test_help_command(self):
        with tempfile.TemporaryDirectory() as tmpdir:
```

```
            cmd = HelpCommand()
            state = create_app_state(cwd=tmpdir)
            from openlaoke.commands.base import CommandContext
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state))
            )
            assert result.success
            assert "help" in result.message.lower()
    def test_exit_command(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            cmd = ExitCommand()
            state = create_app_state(cwd=tmpdir)
            from openlaoke.commands.base import CommandContext
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state))
            )
            assert result.should_exit
    def test_model_command(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            cmd = ModelCommand()
            state = create_app_state(cwd=tmpdir)
            from openlaoke.commands.base import CommandContext
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state, args=""))
            )
            assert state.session_config.model in result.message
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state, args="claude-opus-4"))
            )
            assert state.session_config.model == "claude-opus-4"
    def test_permission_command(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            cmd = PermissionCommand()
            state = create_app_state(cwd=tmpdir)
            from openlaoke.commands.base import CommandContext
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state, args="auto"))
            )
            assert state.permission_config.mode == PermissionMode.AUTO
    def test_cost_command(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            cmd = CostCommand()
            state = create_app_state(cwd=tmpdir)
            state.accumulate_tokens(TokenUsage(input_tokens=1000, output_tokens=500))
            from openlaoke.commands.base import CommandContext
            result = asyncio.get_event_loop().run_until_complete(
                cmd.execute(CommandContext(app_state=state))
            )
            assert "1,000" in result.message or "1000" in result.message
class TestToolValidation:
    def test_required_field_validation(self):
        class ValidatingTool(Tool):
            name = "Validating"
            description = "Test"
            input_schema = {
                "type": "object",
                "required": ["file_path"],
                "properties": {"file_path": {"type": "string"}},
            }
            async def call(self, ctx, **kwargs):
                return ToolResultBlock(tool_use_id=ctx.tool_use_id, content="ok")
        tool = ValidatingTool()
        result = tool.validate_input({})
        assert not result.result
        assert "file_path" in result.message
```

```
result = tool.validate_input({"file_path": "test.txt"})
assert result.result
```

File: [230/252] tests/test_file_history.py

```
"""Tests for file history and snapshot functionality."""
import os
import tempfile
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any
import pytest
@dataclass
class FileSnapshot:
    """Snapshot of a file at a specific point in time."""
    path: str
    content: str
    timestamp: str = field(default_factory=lambda: datetime.now().isoformat())
    checksum: str = ""
    metadata: dict[str, Any] = field(default_factory=dict)
    def __post_init__(self):
        if not self.checksum:
            import hashlib
            self.checksum = hashlib.md5(self.content.encode()).hexdigest()
@dataclass
class FileHistoryEntry:
    """Entry in file history."""
    snapshots: list[FileSnapshot] = field(default_factory=list)
    current_index: int = -1
    def add_snapshot(self, snapshot: FileSnapshot) -> None:
        self.snapshots.append(snapshot)
        self.current_index = len(self.snapshots) - 1
    def get_current(self) -> FileSnapshot | None:
        if self.current_index >= 0 and self.current_index < len(self.snapshots):
            return self.snapshots[self.current_index]
        return None
    def get_previous(self) -> FileSnapshot | None:
        if self.current_index > 0:
            self.current_index -= 1
            return self.snapshots[self.current_index]
        return None
    def get_next(self) -> FileSnapshot | None:
        if self.current_index < len(self.snapshots) - 1:
            self.current_index += 1
            return self.snapshots[self.current_index]
        return None
    def can_go_back(self) -> bool:
        return self.current_index > 0
    def can_go_forward(self) -> bool:
        return self.current_index < len(self.snapshots) - 1
class FileHistoryManager:
    """Manager for file history."""
    def __init__(self):
        self._history: dict[str, FileHistoryEntry] = {}
    def save_snapshot(
        self, path: str, content: str, metadata: dict[str, Any] = None
    ) -> FileSnapshot:
        snapshot = FileSnapshot(path=path, content=content, metadata=metadata or {})
        if path not in self._history:
            self._history[path] = FileHistoryEntry()
        self._history[path].add_snapshot(snapshot)
        return snapshot
    def get_history(self, path: str) -> FileHistoryEntry | None:
        return self._history.get(path)
    def get_current(self, path: str) -> FileSnapshot | None:
        entry = self._history.get(path)
```

```
        if entry:
            return entry.get_current()
        return None
    def restore_previous(self, path: str) -> str | None:
        entry = self._history.get(path)
        if entry and entry.can_go_back():
            snapshot = entry.get_previous()
            if snapshot:
                return snapshot.content
        return None
    def restore_next(self, path: str) -> str | None:
        entry = self._history.get(path)
        if entry and entry.can_go_forward():
            snapshot = entry.get_next()
            if snapshot:
                return snapshot.content
        return None
    def clear_history(self, path: str) -> None:
        if path in self._history:
            del self._history[path]
    def get_all_paths(self) -> list[str]:
        return list(self._history.keys())
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
@pytest.fixture
def history_manager():
    return FileHistoryManager()
class TestFileSnapshot:
    def test_create_snapshot(self):
        snapshot = FileSnapshot(path="/test/file.txt", content="test content")
        assert snapshot.path == "/test/file.txt"
        assert snapshot.content == "test content"
        assert snapshot.timestamp
        assert snapshot.checksum
    def test_snapshot_checksum(self):
        snapshot1 = FileSnapshot(path="test", content="same content")
        snapshot2 = FileSnapshot(path="test", content="same content")
        assert snapshot1.checksum == snapshot2.checksum
        snapshot3 = FileSnapshot(path="test", content="different content")
        assert snapshot1.checksum != snapshot3.checksum
    def test_snapshot_metadata(self):
        metadata = {"author": "test", "version": 1}
        snapshot = FileSnapshot(path="test", content="content", metadata=metadata)
        assert snapshot.metadata["author"] == "test"
        assert snapshot.metadata["version"] == 1
    def test_default_metadata(self):
        snapshot = FileSnapshot(path="test", content="content")
        assert snapshot.metadata == {}
    def test_timestamp_format(self):
        snapshot = FileSnapshot(path="test", content="content")
        assert "T" in snapshot.timestamp
class TestFileHistoryEntry:
    def test_empty_entry(self):
        entry = FileHistoryEntry()
        assert entry.snapshots == []
        assert entry.current_index == -1
        assert entry.get_current() is None
    def test_add_single_snapshot(self):
        entry = FileHistoryEntry()
        snapshot = FileSnapshot(path="test", content="v1")
        entry.add_snapshot(snapshot)
        assert len(entry.snapshots) == 1
        assert entry.current_index == 0
```

```
        assert entry.get_current() == snapshot
    def test_add_multiple_snapshots(self):
        entry = FileHistoryEntry()
        s1 = FileSnapshot(path="test", content="v1")
        s2 = FileSnapshot(path="test", content="v2")
        s3 = FileSnapshot(path="test", content="v3")
        entry.add_snapshot(s1)
        entry.add_snapshot(s2)
        entry.add_snapshot(s3)
        assert len(entry.snapshots) == 3
        assert entry.current_index == 2
        assert entry.get_current().content == "v3"
    def test_navigation_previous(self):
        entry = FileHistoryEntry()
        s1 = FileSnapshot(path="test", content="v1")
        s2 = FileSnapshot(path="test", content="v2")
        entry.add_snapshot(s1)
        entry.add_snapshot(s2)
        assert entry.current_index == 1
        prev = entry.get_previous()
        assert prev.content == "v1"
        assert entry.current_index == 0
    def test_navigation_next(self):
        entry = FileHistoryEntry()
        s1 = FileSnapshot(path="test", content="v1")
        s2 = FileSnapshot(path="test", content="v2")
        entry.add_snapshot(s1)
        entry.add_snapshot(s2)
        entry.current_index = 0
        next_snap = entry.get_next()
        assert next_snap.content == "v2"
        assert entry.current_index == 1
    def test_cannot_go_back_at_start(self):
        entry = FileHistoryEntry()
        s1 = FileSnapshot(path="test", content="v1")
        entry.add_snapshot(s1)
        assert not entry.can_go_back()
        assert entry.get_previous() is None
    def test_cannot_go_forward_at_end(self):
        entry = FileHistoryEntry()
        s1 = FileSnapshot(path="test", content="v1")
        entry.add_snapshot(s1)
        assert not entry.can_go_forward()
        assert entry.get_next() is None
    def test_navigation_bounds(self):
        entry = FileHistoryEntry()
        for i in range(5):
            entry.add_snapshot(FileSnapshot(path="test", content=f"v{i}"))
        entry.current_index = 0
        assert entry.get_previous() is None
        assert entry.current_index == 0
        entry.current_index = 4
        assert entry.get_next() is None
        assert entry.current_index == 4
class TestFileHistorySave:
    def test_save_first_snapshot(self, history_manager):
        snapshot = history_manager.save_snapshot("/test/file.txt", "initial content")
        assert snapshot.content == "initial content"
        assert history_manager.get_history("/test/file.txt") is not None
    def test_save_multiple_snapshots(self, history_manager):
        history_manager.save_snapshot("/test/file.txt", "v1")
        history_manager.save_snapshot("/test/file.txt", "v2")
        history_manager.save_snapshot("/test/file.txt", "v3")
        history = history_manager.get_history("/test/file.txt")
        assert len(history.snapshots) == 3
```

```
    def test_save_with_metadata(self, history_manager):
        metadata = {"source": "user_edit", "line_count": 10}
        snapshot = history_manager.save_snapshot("/test/file.txt", "content", metadata)
        assert snapshot.metadata["source"] == "user_edit"
    def test_save_different_files(self, history_manager):
        history_manager.save_snapshot("/file1.txt", "content1")
        history_manager.save_snapshot("/file2.txt", "content2")
        assert len(history_manager.get_all_paths()) == 2
    def test_get_current_after_save(self, history_manager):
        history_manager.save_snapshot("/test.txt", "old")
        history_manager.save_snapshot("/test.txt", "new")
        current = history_manager.get_current("/test.txt")
        assert current.content == "new"
class TestFileHistoryRestore:
    def test_restore_previous(self, history_manager):
        history_manager.save_snapshot("/test.txt", "version 1")
        history_manager.save_snapshot("/test.txt", "version 2")
        content = history_manager.restore_previous("/test.txt")
        assert content == "version 1"
    def test_restore_next(self, history_manager):
        history_manager.save_snapshot("/test.txt", "version 1")
        history_manager.save_snapshot("/test.txt", "version 2")
        history_manager.restore_previous("/test.txt")
        content = history_manager.restore_next("/test.txt")
        assert content == "version 2"
    def test_restore_nonexistent_file(self, history_manager):
        content = history_manager.restore_previous("/nonexistent.txt")
        assert content is None
    def test_restore_at_first_version(self, history_manager):
        history_manager.save_snapshot("/test.txt", "only version")
        content = history_manager.restore_previous("/test.txt")
        assert content is None
    def test_restore_at_last_version(self, history_manager):
        history_manager.save_snapshot("/test.txt", "v1")
        history_manager.save_snapshot("/test.txt", "v2")
        content = history_manager.restore_next("/test.txt")
        assert content is None
    def test_multiple_restore_cycles(self, history_manager):
        history_manager.save_snapshot("/test.txt", "v1")
        history_manager.save_snapshot("/test.txt", "v2")
        history_manager.save_snapshot("/test.txt", "v3")
        assert history_manager.restore_previous("/test.txt") == "v2"
        assert history_manager.restore_previous("/test.txt") == "v1"
        assert history_manager.restore_previous("/test.txt") is None
        assert history_manager.restore_next("/test.txt") == "v2"
        assert history_manager.restore_next("/test.txt") == "v3"
        assert history_manager.restore_next("/test.txt") is None
class TestFileHistoryManager:
    def test_clear_history(self, history_manager):
        history_manager.save_snapshot("/test.txt", "content")
        history_manager.clear_history("/test.txt")
        assert history_manager.get_history("/test.txt") is None
    def test_get_all_paths_empty(self, history_manager):
        paths = history_manager.get_all_paths()
        assert paths == []
    def test_get_all_paths_multiple(self, history_manager):
        history_manager.save_snapshot("/file1.txt", "c1")
        history_manager.save_snapshot("/file2.txt", "c2")
        history_manager.save_snapshot("/file3.txt", "c3")
        paths = history_manager.get_all_paths()
        assert len(paths) == 3
        assert "/file1.txt" in paths
        assert "/file2.txt" in paths
    def test_get_history_nonexistent(self, history_manager):
        history = history_manager.get_history("/nonexistent.txt")
```

```
        assert history is None
    def test_get_current_nonexistent(self, history_manager):
        current = history_manager.get_current("/nonexistent.txt")
        assert current is None
class TestFileHistoryIntegration:
    def test_full_cycle(self, history_manager, temp_dir):
        path = os.path.join(temp_dir, "test.txt")
        with open(path, "w") as f:
            f.write("initial")
        history_manager.save_snapshot(path, "initial")
        with open(path, "w") as f:
            f.write("modified")
        history_manager.save_snapshot(path, "modified")
        with open(path, "w") as f:
            f.write("final")
        history_manager.save_snapshot(path, "final")
        history = history_manager.get_history(path)
        assert len(history.snapshots) == 3
        restored = history_manager.restore_previous(path)
        assert restored == "modified"
        with open(path, "w") as f:
            f.write(restored)
        with open(path) as f:
            assert f.read() == "modified"
    def test_checksum_validation(self, history_manager):
        s1 = history_manager.save_snapshot("/test.txt", "content")
        s2 = history_manager.save_snapshot("/test.txt", "content")
        assert s1.checksum == s2.checksum
        s3 = history_manager.save_snapshot("/test.txt", "different content")
        assert s1.checksum != s3.checksum
    def test_metadata_preservation(self, history_manager):
        meta1 = {"step": 1}
        meta2 = {"step": 2, "note": "fix"}
        s1 = history_manager.save_snapshot("/test.txt", "v1", meta1)
        s2 = history_manager.save_snapshot("/test.txt", "v2", meta2)
        history = history_manager.get_history("/test.txt")
        assert history.snapshots[0].metadata["step"] == 1
        assert history.snapshots[1].metadata["note"] == "fix"
class TestFileHistoryEdgeCases:
    def test_empty_content(self, history_manager):
        snapshot = history_manager.save_snapshot("/empty.txt", "")
        assert snapshot.content == ""
        assert snapshot.checksum
    def test_large_content(self, history_manager):
        large_content = "x" * 10000
        snapshot = history_manager.save_snapshot("/large.txt", large_content)
        assert len(snapshot.content) == 10000
    def test_unicode_content(self, history_manager):
        unicode_content = "Hello ?? ?"
        snapshot = history_manager.save_snapshot("/unicode.txt", unicode_content)
        assert snapshot.content == unicode_content
    def test_special_characters(self, history_manager):
        special_content = "Line1\nLine2\tTabbed\r\nWindows"
        snapshot = history_manager.save_snapshot("/special.txt", special_content)
        assert snapshot.content == special_content
    def test_concurrent_saves_same_file(self, history_manager):
        for i in range(100):
            history_manager.save_snapshot("/test.txt", f"version{i}")
        history = history_manager.get_history("/test.txt")
        assert len(history.snapshots) == 100
```

File: [231/252] tests/test_hyperauto.py

```
"""Comprehensive tests for HyperAuto system."""
from __future__ import annotations
import tempfile
from pathlib import Path
from unittest.mock import AsyncMock, MagicMock, patch
import pytest
from openlaoke.core.hyperauto.agent import AnalysisResult, HyperAutoAgent, SkillGenerationResul
    t
from openlaoke.core.hyperauto.code_search import (
    CodeSearchEngine,
    CodeSnippet,
    SearchSource,
    SearchResult,
)
from openlaoke.core.hyperauto.config import HyperAutoConfig
from openlaoke.core.hyperauto.decision_engine import (
    Context,
    DecisionCategory,
    DecisionEngine,
    DecisionOption,
    DecisionRecord,
    RiskAssessment,
    RiskLevel,
)
from openlaoke.core.hyperauto.learning import (
    EnhancedStrategy,
    Experience,
    ExperienceType,
    Knowledge,
    KnowledgeCategory,
    LearningSession,
    LearningSystem,
    Pattern,
    PatternType,
    Strategy,
    StrategyType,
)
from openlaoke.core.hyperauto.project_initializer import (
    InitResult,
    ProjectAnalysis,
    ProjectInitializer,
    ProjectType,
)
from openlaoke.core.hyperauto.reflection import (
    Analysis,
    Correction,
    CorrectionStatus,
    Error,
    ErrorCategory,
    ErrorSeverity,
    Improvement,
    ImprovementPriority,
    QualityDimension,
    QualityScore,
    ReflectionReport,
    ReflectionSystem,
    Review,
    ReviewType,
)
from openlaoke.core.hyperauto.skill_generator import (
    GenerationResult,
```

```
    SkillDesign,
    SkillGenerator,
    SkillParameter,
    SkillRequirement,
    SkillTrigger,
)
from openlaoke.core.hyperauto.test_runner import (
    AutoTestRunner,
    CoverageReport,
    Diagnosis,
    TestAnalysis,
    TestCycleResult,
    TestFailure,
    TestFramework,
    TestInfo,
    TestResults,
    TestSeverity,
    TestStatus,
)
from openlaoke.core.hyperauto.types import (
    Decision,
    DecisionType,
    HyperAutoMode,
    HyperAutoState,
    Reflection,
    SubTask,
    SubTaskStatus,
    WorkflowContext,
)
from openlaoke.core.hyperauto.validator import (
    Artifact,
    Language,
    Requirements,
    ResultValidator,
    SecurityReport,
    ValidationCategory,
    ValidationIssue,
    ValidationLevel,
    ValidationResult,
    ValidatorQualityScore,
)
from openlaoke.core.hyperauto.workflow import (
    DependencyEdge,
    DependencyGraph,
    ExecutionPlan,
    ExecutionResult,
    ExecutionStep,
    ProgressInfo,
    RecoveryAction,
    RecoveryStrategy,
    TaskTree,
    WorkflowOrchestrator,
    WorkflowPhase,
    WorkflowResult,
)
from openlaoke.core.state import AppState, create_app_state
class TestHyperAutoAgent:
    def test_agent_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            agent = HyperAutoAgent(state)
            assert agent.app_state == state
            assert agent.config is not None
            assert agent.context is not None
    def test_agent_creation_with_config(self):
```

```
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        config = HyperAutoConfig(dry_run=True)
        agent = HyperAutoAgent(state, config)
        assert agent.config.dry_run is True
@pytest.mark.asyncio
async def test_analyze_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        agent = HyperAutoAgent(state)
        agent.context.original_request = "Create a new function"
        result = await agent._analyze_request()
        assert result.task_type == "creation"
        assert len(result.sub_tasks) > 0
        assert result.confidence > 0
@pytest.mark.asyncio
async def test_generate_skills(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        agent = HyperAutoAgent(state)
        content = await agent._generate_skill_content("test_skill", "A test skill")
        assert "test_skill" in content
        assert "A test skill" in content
@pytest.mark.asyncio
async def test_initialize_project(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        agent = HyperAutoAgent(state)
        result = await agent._initialize_project(tmpdir, "python")
        assert result["initialized"] is True
        assert result["type"] == "python"
@pytest.mark.asyncio
async def test_search_code(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        agent = HyperAutoAgent(state)
        results = await agent._search_code("test query", "*.py")
        assert isinstance(results, list)
@pytest.mark.asyncio
async def test_execute_workflow(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        config = HyperAutoConfig(dry_run=True, max_iterations=2)
        agent = HyperAutoAgent(state, config)
        result = await agent.run("Create a test function")
        assert "success" in result
        assert "session_id" in result
@pytest.mark.asyncio
async def test_reflection(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        config = HyperAutoConfig(reflection_enabled=True)
        agent = HyperAutoAgent(state, config)
        task = SubTask(name="test", description="Test task")
        task.status = SubTaskStatus.COMPLETED
        task.result = {"done": True}
        agent.context.sub_tasks.append(task)
        reflection = await agent._reflect()
        assert len(reflection.observations) > 0
@pytest.mark.asyncio
async def test_learning(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        config = HyperAutoConfig(learning_enabled=True)
        agent = HyperAutoAgent(state, config)
```

```
            reflection = Reflection(
                task_id="test",
                success=True,
                learned_patterns=["pattern1", "pattern2"],
            )
            agent.context.reflections.append(reflection)
            result = await agent._learn()
            assert "patterns_learned" in result
    def test_decision_making(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            agent = HyperAutoAgent(state)
            task_type = agent._classify_task("Create a new feature")
            assert task_type == "creation"
            task_type = agent._classify_task("Fix the bug in the code")
            assert task_type == "bugfix"
    @pytest.mark.asyncio
    async def test_full_workflow(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            config = HyperAutoConfig(dry_run=True, max_iterations=2)
            agent = HyperAutoAgent(state, config)
            result = await agent.run("Test task")
            assert "success" in result
            assert result.get("session_id") is not None
class TestSkillGenerator:
    def test_analyze_skill_needs(self):
        generator = SkillGenerator()
        requirements = generator.analyze_skill_needs("Write tests for the new feature")
        assert len(requirements) > 0
        assert any(r.task_type == "testing" for r in requirements)
    def test_design_skill(self):
        generator = SkillGenerator()
        requirement = SkillRequirement(
            task_type="testing",
            description="Test task",
            keywords=["test", "testing"],
            suggested_tools=["bash", "read"],
        )
        design = generator.design_skill(requirement)
        assert design.name != ""
        assert len(design.triggers) > 0
        assert len(design.capabilities) > 0
    def test_generate_content(self):
        generator = SkillGenerator()
        design = SkillDesign(
            name="test_skill",
            description="A test skill",
            capabilities=["Test capability"],
            workflow_steps=["Step 1", "Step 2"],
        )
        content = generator.generate_skill_content(design)
        assert "---" in content
        assert "test_skill" in content
        assert "A test skill" in content
    def test_validate_skill(self):
        generator = SkillGenerator()
        valid_content = """---
name: test_skill
description: A test skill
---
# Test Skill
This is a valid skill content with enough text to pass validation.
The body content needs to be at least 100 characters long to be valid.
We add more content here to ensure the validation passes successfully.
```

```
"""
        result = generator.validate_skill(valid_content)
        assert result.result is True
        invalid_content = "Too short"
        result = generator.validate_skill(invalid_content)
        assert result.result is False
    def test_register_skill(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            generator = SkillGenerator(skill_dir=Path(tmpdir))
            content = """---
name: test_skill
description: A test skill
---
# Test Skill
This is a test skill content with enough text to pass validation.
The body content needs to be at least 100 characters long to be valid.
We add more content here to ensure the registration passes successfully.
"""
            result = generator.register_skill(content, "test_skill")
            assert result.success is True
    def test_auto_generate(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            generator = SkillGenerator(skill_dir=Path(tmpdir))
            skills = generator.auto_generate_for_task(
                "Write unit tests for authentication",
                register=False,
            )
            assert len(skills) >= 0
    def test_skill_template_format(self):
        generator = SkillGenerator()
        requirement = SkillRequirement(
            task_type="code_review",
            description="Review code",
            keywords=["review"],
            suggested_tools=["read", "grep"],
        )
        design = generator.design_skill(requirement)
        content = generator.generate_skill_content(design)
        assert "##" in content
        assert "Workflow" in content
    def test_skill_yaml_frontmatter(self):
        generator = SkillGenerator()
        design = SkillDesign(
            name="yaml_test",
            description="YAML test",
            version="1.0.0",
        )
        content = generator.generate_skill_content(design)
        assert content.startswith("---")
        parts = content.split("---")
        assert len(parts) >= 3
class TestProjectInitializer:
    def test_detect_project_type_python(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            initializer = ProjectInitializer()
            Path(tmpdir, "pyproject.toml").touch()
            project_type = initializer.detect_project_type(Path(tmpdir))
            assert project_type == ProjectType.PYTHON
    def test_detect_project_type_nodejs(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            initializer = ProjectInitializer()
            Path(tmpdir, "package.json").touch()
            project_type = initializer.detect_project_type(Path(tmpdir))
            assert project_type == ProjectType.NODE_JS
    def test_detect_project_type_go(self):
```

```
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "go.mod").touch()
        project_type = initializer.detect_project_type(Path(tmpdir))
        assert project_type == ProjectType.GO
def test_detect_project_type_rust(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "Cargo.toml").touch()
        project_type = initializer.detect_project_type(Path(tmpdir))
        assert project_type == ProjectType.RUST
def test_analyze_project(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "pyproject.toml").touch()
        analysis = initializer.analyze_project(Path(tmpdir))
        assert analysis.project_type == ProjectType.PYTHON
        assert isinstance(analysis.has_git, bool)
        assert isinstance(analysis.has_readme, bool)
def test_create_structure(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        result = initializer.create_project_structure(Path(tmpdir), ProjectType.PYTHON)
        assert result is True
def test_generate_config_files(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        analysis = ProjectAnalysis(
            project_type=ProjectType.PYTHON,
            path=Path(tmpdir),
            has_git=False,
            has_readme=False,
            has_agents_md=False,
        )
        configs = initializer.generate_config_files(analysis)
        assert ".gitignore" in configs
        assert "README.md" in configs
        assert "AGENTS.md" in configs
def test_install_dependencies(self):
    initializer = ProjectInitializer()
    result = initializer.install_dependencies(ProjectType.PYTHON)
    assert isinstance(result, bool)
@pytest.mark.asyncio
async def test_auto_initialize(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        result = await initializer.auto_initialize(Path(tmpdir))
        assert isinstance(result, InitResult)
        assert result.project_type is not None
def test_python_project(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "pyproject.toml").touch()
        project_type = initializer.detect_project_type(Path(tmpdir))
        assert project_type == ProjectType.PYTHON
def test_nodejs_project(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "package.json").touch()
        project_type = initializer.detect_project_type(Path(tmpdir))
        assert project_type == ProjectType.NODE_JS
def test_go_project(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        initializer = ProjectInitializer()
        Path(tmpdir, "go.mod").touch()
```

```
            project_type = initializer.detect_project_type(Path(tmpdir))
            assert project_type == ProjectType.GO
    def test_rust_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            initializer = ProjectInitializer()
            Path(tmpdir, "Cargo.toml").touch()
            project_type = initializer.detect_project_type(Path(tmpdir))
            assert project_type == ProjectType.RUST
class TestCodeSearchEngine:
    def test_search_local(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            test_file = Path(tmpdir, "test.py")
            test_file.write_text("def hello():\n    print('hello')\n")
            engine = CodeSearchEngine(project_path=Path(tmpdir))
            results = engine.search_local("hello", max_results=10)
            assert isinstance(results, list)
    def test_search_reference_projects(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            ref_dir = Path(tmpdir, "reference")
            ref_dir.mkdir()
            Path(ref_dir, "test.py").write_text("def test_func():\n    pass\n")
            engine = CodeSearchEngine(
                project_path=Path(tmpdir),
                reference_projects=[ref_dir],
            )
            results = engine.search_reference_projects("test_func", max_results=10)
            assert isinstance(results, list)
    @pytest.mark.asyncio
    async def test_search_github(self):
        engine = CodeSearchEngine()
        results = await engine.search_github("test query", max_results=5)
        assert isinstance(results, list)
    def test_analyze_reusability(self):
        engine = CodeSearchEngine()
        snippet = CodeSnippet(
            source="local",
            file_path="test.py",
            language="python",
            content="def test():\n    return 1\n",
            description="Test function",
        )
        score = engine.analyze_reusability(snippet)
        assert 0.0 <= score <= 1.0
    def test_adapt_code(self):
        engine = CodeSearchEngine()
        snippet = CodeSnippet(
            source="local",
            file_path="test.py",
            language="python",
            content="def old_name():\n    return 1\n",
            description="Test function",
        )
        context = {"rename_map": {"old_name": "new_name"}}
        adapted = engine.adapt_code(snippet, context)
        assert "new_name" in adapted
    @pytest.mark.asyncio
    async def test_auto_search(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            engine = CodeSearchEngine(project_path=Path(tmpdir))
            results = await engine.auto_search_and_adapt(
                "test query",
                {"language": "python"},
                max_results=5,
            )
            assert isinstance(results, list)
```

```
    def test_semantic_search(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            engine = CodeSearchEngine(project_path=Path(tmpdir))
            results = engine.search_semantic("test query", max_results=10)
            assert isinstance(results, list)
    def test_result_ranking(self):
        engine = CodeSearchEngine()
        snippets = [
            CodeSnippet(
                source="local",
                file_path="test1.py",
                language="python",
                content="first unique content",
                description="Low relevance",
                relevance_score=0.3,
            ),
            CodeSnippet(
                source="local",
                file_path="test2.py",
                language="python",
                content="second unique content",
                description="High relevance",
                relevance_score=0.9,
            ),
        ]
        results = engine._deduplicate(snippets)
        assert len(results) == 2
class TestWorkflowOrchestrator:
    def test_decompose_task(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            tree = orchestrator.decompose_task("Create a new feature")
            assert tree.root_task == "Create a new feature"
            assert len(tree.subtasks) > 0
    def test_analyze_dependencies(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task1 = SubTask(name="task1", description="First task")
            task2 = SubTask(name="task2", description="Second task", dependencies=[task1.id])
            orchestrator.context.sub_tasks = [task1, task2]
            graph = orchestrator.analyze_dependencies([task1, task2])
            assert len(graph.nodes) == 2
            assert len(graph.edges) == 1
    def test_create_execution_plan(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task1 = SubTask(name="task1", description="First task")
            task2 = SubTask(name="task2", description="Second task", dependencies=[task1.id])
            orchestrator.context.sub_tasks = [task1, task2]
            graph = orchestrator.analyze_dependencies([task1, task2])
            plan = orchestrator.create_execution_plan(graph)
            assert len(plan.steps) == 2
    @pytest.mark.asyncio
    async def test_execute_plan(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task = SubTask(name="task1", description="Test task")
            step = ExecutionStep(
                task_id=task.id,
                phase=WorkflowPhase.ANALYZE,
            )
```

```
            plan = ExecutionPlan(steps=[step], parallel_groups={0: [task.id]})
            orchestrator.context.sub_tasks = [task]
            result = await orchestrator.execute_plan(plan)
            assert result.success is True
    def test_track_progress(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            progress = orchestrator.track_progress()
            assert progress.total_tasks >= 0
    def test_handle_error(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task = SubTask(name="test", description="Test task")
            error = TimeoutError("Test timeout")
            recovery = orchestrator.handle_error(error, task)
            assert recovery.strategy == RecoveryStrategy.RETRY
    def test_validate_result(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task = SubTask(name="test", description="Test task")
            assert orchestrator.validate_result(task, {"status": "completed"}) is True
            assert orchestrator.validate_result(task, None) is False
    @pytest.mark.asyncio
    async def test_parallel_execution(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            task1 = SubTask(name="task1", description="Task 1")
            task2 = SubTask(name="task2", description="Task 2")
            plan = ExecutionPlan(
                steps=[
                    ExecutionStep(task_id=task1.id, phase=WorkflowPhase.ANALYZE),
                    ExecutionStep(task_id=task2.id, phase=WorkflowPhase.ANALYZE),
                ],
                parallel_groups={0: [task1.id, task2.id]},
            )
            orchestrator.context.sub_tasks = [task1, task2]
            result = await orchestrator.execute_plan(plan)
            assert result.success is True
    def test_dependency_resolution(self):
        graph = DependencyGraph()
        graph.nodes = ["a", "b", "c"]
        graph.edges = [
            DependencyEdge(source="a", target="b"),
            DependencyEdge(source="b", target="c"),
        ]
        graph.adjacency = {"a": ["b"], "b": ["c"], "c": []}
        graph.reverse_adjacency = {"a": [], "b": ["a"], "c": ["b"]}
        order = graph.topological_sort()
        assert order.index("a") < order.index("b")
        assert order.index("b") < order.index("c")
    @pytest.mark.asyncio
    async def test_full_workflow(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            orchestrator = WorkflowOrchestrator(state)
            result = await orchestrator.run_full_workflow("Test task")
            assert isinstance(result, WorkflowResult)
            assert result.workflow_id != ""
class TestDecisionEngine:
    def test_analyze_context(self):
        engine = DecisionEngine()
```

```
    task = SubTask(name="test", description="Test task")
    context = engine.analyze_context(HyperAutoState.ANALYZING, task)
    assert context.state == HyperAutoState.ANALYZING
    assert len(context.available_tools) > 0
def test_generate_options(self):
    engine = DecisionEngine()
    task = SubTask(
        name="test",
        description="Test task",
        metadata={"type": "skill_creation"},
    )
    context = engine.analyze_context(HyperAutoState.ANALYZING, task)
    options = engine.generate_options(context)
    assert len(options) > 0
def test_assess_risks(self):
    engine = DecisionEngine()
    options = [
        DecisionOption(
            action="test_action",
            risk_level=RiskLevel.LOW,
            confidence=0.8,
        ),
        DecisionOption(
            action="risky_action",
            risk_level=RiskLevel.HIGH,
            confidence=0.5,
        ),
    ]
    assessments = engine.assess_risks(options)
    assert len(assessments) == 2
def test_calculate_confidence(self):
    engine = DecisionEngine()
    option = DecisionOption(
        action="test",
        confidence=0.5,
        risk_level=RiskLevel.LOW,
    )
    task = SubTask(name="test", description="Test task")
    context = engine.analyze_context(HyperAutoState.ANALYZING, task)
    confidence = engine.calculate_confidence(option, context)
    assert 0.0 <= confidence <= 1.0
def test_select_best(self):
    engine = DecisionEngine()
    options = [
        DecisionOption(
            action="low_confidence",
            confidence=0.3,
            risk_level=RiskLevel.LOW,
        ),
        DecisionOption(
            action="high_confidence",
            confidence=0.9,
            risk_level=RiskLevel.LOW,
        ),
    ]
    decision = engine.select_best(options)
    assert decision.action == "high_confidence"
def test_record_decision(self):
    engine = DecisionEngine()
    decision = Decision(type=DecisionType.SKILL_CREATION, confidence=0.8)
    record = DecisionRecord()
    engine.record_decision(decision, record)
    assert len(engine.get_decision_history()) == 1
def test_risk_based_decision(self):
    engine = DecisionEngine()
```

```
        options = [
            DecisionOption(
                action="safe_option",
                confidence=0.7,
                risk_level=RiskLevel.LOW,
            ),
            DecisionOption(
                action="risky_option",
                confidence=0.9,
                risk_level=RiskLevel.CRITICAL,
            ),
        ]
        decision = engine.select_best(options)
        assert decision.action == "safe_option"
    def test_confidence_threshold(self):
        engine = DecisionEngine()
        options = [
            DecisionOption(
                action="low_confidence",
                confidence=0.3,
                risk_level=RiskLevel.LOW,
            ),
        ]
        decision = engine.select_best(options)
        assert decision.confidence == 0.3
class TestLearningSystem:
    def test_collect_experience(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            from openlaoke.core.hyperauto.workflow import (
                TaskTree,
                ExecutionPlan,
                WorkflowResult,
            )
            result = WorkflowResult(
                workflow_id="test",
                success=True,
                task_tree=TaskTree(root_task="test"),
                execution_plan=ExecutionPlan(),
            )
            experience = system.collect_experience(result)
            assert experience.type == ExperienceType.SUCCESS
    def test_extract_knowledge(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            experiences = [
                Experience(
                    task="test task",
                    type=ExperienceType.SUCCESS,
                    steps=["step1", "step2"],
                )
            ]
            knowledge = system.extract_knowledge(experiences)
            assert knowledge.title != ""
    def test_recognize_patterns(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            knowledge = Knowledge(
                title="test",
                description="test knowledge",
            )
            experiences = [
                Experience(
                    id="exp1",
                    task="test",
```

```
                    type=ExperienceType.SUCCESS,
                    steps=["step1", "step2"],
                )
            ]
            system._experiences = experiences
            knowledge.source_experiences = ["exp1"]
            patterns = system.recognize_patterns(knowledge)
            assert isinstance(patterns, list)
    def test_optimize_strategy(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            patterns = [
                Pattern(
                    type=PatternType.SEQUENTIAL,
                    name="test_pattern",
                    sequence=["step1", "step2"],
                    success_rate=0.9,
                )
            ]
            strategy = system.optimize_strategy(patterns)
            assert strategy.type == StrategyType.AGGRESSIVE
    def test_persist_knowledge(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            knowledge = Knowledge(
                id="test_id",
                title="Test Knowledge",
                description="Test description",
            )
            result = system.persist_knowledge(knowledge)
            assert result is True
    def test_load_knowledge(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            knowledge = system.load_knowledge()
            assert isinstance(knowledge, Knowledge)
    def test_apply_learning(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            enhanced = system.apply_learning("create a test")
            assert isinstance(enhanced, EnhancedStrategy)
            assert enhanced.base_strategy is not None
    def test_learning_session(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = LearningSystem(storage_dir=Path(tmpdir))
            session = system.start_session()
            assert session.id != ""
            assert session.start_time > 0
            ended = system.end_session()
            assert ended is not None
            assert ended.end_time is not None
class TestReflectionSystem:
    def test_review_result(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = ReflectionSystem(storage_dir=Path(tmpdir))
            from openlaoke.core.hyperauto.workflow import (
                TaskTree,
                ExecutionPlan,
                ExecutionResult,
                WorkflowResult,
            )
            result = WorkflowResult(
                workflow_id="test",
                success=True,
                task_tree=TaskTree(root_task="test"),
```

```
            execution_plan=ExecutionPlan(),
            execution_results=[ExecutionResult(task_id="task1", success=True, output={})],
        )
        review = system.review_result(result)
        assert review.overall_score >= 0
def test_analyze_errors(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        system = ReflectionSystem(storage_dir=Path(tmpdir))
        errors = [
            Error(
                severity=ErrorSeverity.HIGH,
                category=ErrorCategory.LOGIC,
                message="Test error",
            )
        ]
        analysis = system.analyze_errors(errors)
        assert analysis.error_count == 1
        assert analysis.high_count == 1
def test_generate_improvements(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        system = ReflectionSystem(storage_dir=Path(tmpdir))
        analysis = Analysis(
            error_count=2,
            root_causes=["Missing validation", "Incorrect logic"],
        )
        improvements = system.generate_improvements(analysis)
        assert len(improvements) > 0
def test_auto_correct(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        system = ReflectionSystem(storage_dir=Path(tmpdir))
        from openlaoke.core.hyperauto.workflow import (
            TaskTree,
            ExecutionPlan,
            WorkflowResult,
        )
        result = WorkflowResult(
            workflow_id="test",
            success=False,
            task_tree=TaskTree(root_task="test"),
            execution_plan=ExecutionPlan(),
        )
        improvements = [
            Improvement(
                priority=ImprovementPriority.HIGH,
                category="error_fix",
                description="Fix the error",
            )
        ]
        corrected = system.auto_correct(result, improvements)
        assert corrected is not None
def test_assess_quality(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        system = ReflectionSystem(storage_dir=Path(tmpdir))
        from openlaoke.core.hyperauto.workflow import (
            TaskTree,
            ExecutionPlan,
            ExecutionResult,
            WorkflowResult,
        )
        result = WorkflowResult(
            workflow_id="test",
            success=True,
            task_tree=TaskTree(root_task="test"),
            execution_plan=ExecutionPlan(
                steps=[ExecutionStep(task_id="task1", phase=WorkflowPhase.ANALYZE)]
```

```
                ),
                execution_results=[ExecutionResult(task_id="task1", success=True)],
            )
            quality = system.assess_quality(result)
            assert quality.overall >= 0
    def test_reflection_report(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            system = ReflectionSystem(storage_dir=Path(tmpdir))
            from openlaoke.core.hyperauto.workflow import (
                TaskTree,
                ExecutionPlan,
                WorkflowResult,
            )
            result = WorkflowResult(
                workflow_id="test",
                success=True,
                task_tree=TaskTree(root_task="test"),
                execution_plan=ExecutionPlan(),
            )
            report = system.reflect_on_workflow(result)
            assert report.summary != ""
    def test_error_severity(self):
        error = Error(
            severity=ErrorSeverity.CRITICAL,
            message="Critical error",
        )
        assert error.severity == ErrorSeverity.CRITICAL
    def test_improvement_priority(self):
        improvement = Improvement(
            priority=ImprovementPriority.IMMEDIATE,
            description="Urgent fix",
        )
        assert improvement.priority == ImprovementPriority.IMMEDIATE
class TestAutoTestRunner:
    def test_detect_framework(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            runner = AutoTestRunner()
            Path(tmpdir, "pyproject.toml").touch()
            framework = runner.detect_test_framework(Path(tmpdir))
            assert framework == TestFramework.PYTEST
    def test_discover_tests(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            test_file = Path(tmpdir, "test_example.py")
            test_file.write_text("def test_something():\n    assert True\n")
            runner = AutoTestRunner()
            tests = runner.discover_tests(Path(tmpdir))
            assert isinstance(tests, list)
    def test_run_tests(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            runner = AutoTestRunner()
            tests = [TestInfo(name="test_example", framework=TestFramework.PYTEST)]
            results = runner.run_tests(tests, Path(tmpdir))
            assert isinstance(results, TestResults)
    def test_analyze_results(self):
        runner = AutoTestRunner()
        results = TestResults(
            total_tests=10,
            passed=8,
            failed=2,
        )
        analysis = runner.analyze_results(results)
        assert analysis.success_rate == 80.0
    def test_check_coverage(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            runner = AutoTestRunner()
```

```
            coverage = runner.check_coverage(Path(tmpdir))
            assert isinstance(coverage, CoverageReport)
    def test_diagnose_failures(self):
        runner = AutoTestRunner()
        failures = [
            TestFailure(
                test_id="test1",
                test_name="test_example",
                error_message="AssertionError: expected True",
            )
        ]
        diagnoses = runner.diagnose_failures(failures)
        assert len(diagnoses) == 1
    def test_auto_fix(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            runner = AutoTestRunner()
            diagnosis = Diagnosis(
                failure_id="test1",
                auto_fixable=False,
            )
            result = runner.auto_fix_tests(diagnosis, Path(tmpdir))
            assert result is False
    @pytest.mark.asyncio
    async def test_full_cycle(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            runner = AutoTestRunner()
            result = await runner.async_run_full_cycle(Path(tmpdir))
            assert isinstance(result, TestCycleResult)
class TestResultValidator:
    def test_validate_code(self):
        validator = ResultValidator()
        code = "def hello():\n    print('hello')\n"
        result = validator.validate_code(code, Language.PYTHON)
        assert isinstance(result, ValidationResult)
    def test_validate_functionality(self):
        validator = ResultValidator()
        implementation = "def add(a, b):\n    return a + b\n"
        requirements = ["add two numbers", "return sum"]
        result = validator.validate_functionality(implementation, requirements, Language.PYTHON
    )
        assert isinstance(result, ValidationResult)
    def test_validate_performance(self):
        validator = ResultValidator()
        code = "def test():\n    pass\n"
        benchmarks = {"max_execution_time_ms": 100.0}
        result = validator.validate_performance(code, benchmarks, Language.PYTHON)
        assert isinstance(result, SecurityReport) or hasattr(result, "benchmark_passed")
    def test_validate_security(self):
        validator = ResultValidator()
        code = "password = 'secret123'\n"
        result = validator.validate_security(code, Language.PYTHON)
        assert result.secure is False
        assert len(result.vulnerabilities) > 0
    def test_validate_compatibility(self):
        validator = ResultValidator()
        code = "def test():\n    pass\n"
        result = validator.validate_compatibility(code, "Python 3.11", Language.PYTHON)
        assert isinstance(result, ValidationResult)
    def test_quality_score(self):
        validator = ResultValidator()
        results = {
            "syntax": ValidationResult(score=100.0),
            "style": ValidationResult(score=90.0),
            "functionality": ValidationResult(score=85.0),
        }
```

```
    score = validator.calculate_quality_score(results)
    assert isinstance(score, ValidatorQualityScore)
def test_full_validation(self):
    validator = ResultValidator()
    artifact = Artifact(
        type="code",
        content="def test():\n    pass\n",
        language=Language.PYTHON,
    )
    result = validator.validate_code(artifact.content, artifact.language)
    assert result is not None
def test_validation_report(self):
    validator = ResultValidator()
    code = "def hello():\n    print('hello')\n"
    code_result = validator.validate_code(code, Language.PYTHON)
    security_result = validator.validate_security(code, Language.PYTHON)
    assert code_result is not None
    assert security_result is not None
```

File: [232/252] tests/test_integration.py

```
"""Integration tests for OpenLaoKe."""
from __future__ import annotations
import asyncio
import json
import os
import tempfile
from pathlib import Path
from unittest.mock import AsyncMock, MagicMock, patch
import pytest
from openlaoke.core.state import create_app_state, AppState
from openlaoke.core.multi_provider_api import MultiProviderClient
from openlaoke.core.tool import ToolRegistry
from openlaoke.tools.register import register_all_tools
from openlaoke.types.providers import MultiProviderConfig
from openlaoke.utils.config import AppConfig, save_config, load_config
class TestEndToEnd:
    """End-to-end integration tests."""
    def test_full_state_workflow(self):
        """Test complete state management workflow."""
        with tempfile.TemporaryDirectory() as tmpdir:
            persist_path = Path(tmpdir) / "session.json"
            state = create_app_state(
                cwd=tmpdir,
                model="test-model",
                persist_path=str(persist_path),
            )
            state.set_cwd("/new/path")
            from openlaoke.types.core_types import TokenUsage
            usage = TokenUsage(input_tokens=100, output_tokens=50)
            state.accumulate_tokens(usage)
            assert state.cwd == "/new/path"
            assert state.token_usage.input_tokens == 100
    def test_config_save_load(self):
        """Test saving and loading configuration."""
        with tempfile.TemporaryDirectory() as tmpdir:
            config_path = Path(tmpdir) / "config.json"
            with patch("openlaoke.utils.config.CONFIG_PATH", config_path):
                config = AppConfig()
                config.max_tokens = 4096
                config.temperature = 0.8
                config.permission_mode = "auto"
                config.auto_approve_all = True
                save_config(config)
                loaded = load_config()
                assert loaded.max_tokens == 4096
                assert loaded.temperature == 0.8
                assert loaded.permission_mode == "auto"
                assert loaded.auto_approve_all is True
    def test_tool_registry_complete(self):
        """Test that all tools are registered."""
        registry = ToolRegistry()
        register_all_tools(registry)
        tools = registry.get_all()
        tool_names = [t.name for t in tools]
        assert "Bash" in tool_names
        assert "Read" in tool_names
        assert "Write" in tool_names
        assert "Edit" in tool_names
        assert "Glob" in tool_names
        assert "Grep" in tool_names
        assert "Agent" in tool_names
```

```
    def test_provider_config_complete(self):
        """Test that all providers are configured."""
        config = MultiProviderConfig.defaults()
        required_providers = [
            "anthropic",
            "openai",
            "minimax",
            "aliyun_coding_plan",
            "ollama",
            "lm_studio",
        ]
        for provider in required_providers:
            assert provider in config.providers, f"Missing provider: {provider}"
            p = config.providers[provider]
            assert p.provider_type is not None
            assert p.base_url or p.is_local
class TestMessageFlow:
    """Tests for message flow through the system."""
    @pytest.mark.asyncio
    async def test_message_conversion_flow(self):
        """Test that messages are properly converted."""
        config = MultiProviderConfig.defaults()
        client = MultiProviderClient(config)
        messages = [
            {"role": "user", "content": "Hello"},
            {
                "role": "assistant",
                "tool_calls": [
                    {
                        "id": "call_1",
                        "type": "function",
                        "function": {"name": "Bash", "arguments": '{"command":"ls"}'},
                    }
                ],
            },
            {"role": "tool", "tool_call_id": "call_1", "content": "file.txt"},
        ]
        openai_format = client._convert_to_openai_format(messages)
        assert openai_format[0]["role"] == "user"
        assert openai_format[1]["role"] == "assistant"
        assert openai_format[2]["role"] == "tool"
        assert openai_format[2]["tool_call_id"] == "call_1"
        anthropic_format = client._convert_to_anthropic_format(messages)
        assert anthropic_format[0]["role"] == "user"
        assert anthropic_format[1]["role"] == "assistant"
        assert anthropic_format[2]["role"] == "user"
    @pytest.mark.asyncio
    async def test_tool_format_conversion(self):
        """Test tool format conversion for different providers."""
        config = MultiProviderConfig.defaults()
        client = MultiProviderClient(config)
        tools = [
            {
                "name": "Write",
                "description": "Write a file",
                "input_schema": {
                    "type": "object",
                    "properties": {
                        "file_path": {"type": "string"},
                        "content": {"type": "string"},
                    },
                    "required": ["file_path", "content"],
                },
            }
        ]
```

```
        openai_tools = client._convert_tools_to_openai_format(tools)
        assert openai_tools[0]["type"] == "function"
        assert openai_tools[0]["function"]["name"] == "Write"
        assert "parameters" in openai_tools[0]["function"]
class TestAutoApprove:
    """Tests for auto-approve functionality."""
    def test_auto_approve_enabled(self):
        """Test that auto-approve is enabled by default."""
        config = AppConfig()
        assert config.auto_approve_all is True
        assert config.permission_mode == "auto"
    def test_permission_config_auto_allows(self):
        """Test that AUTO mode allows all tools."""
        from openlaoke.types.permissions import PermissionConfig
        from openlaoke.types.core_types import PermissionMode, PermissionResult
        config = PermissionConfig.defaults()
        config.mode = PermissionMode.AUTO
        assert config.check_tool("Bash") == PermissionResult.ALLOW
        assert config.check_tool("Write") == PermissionResult.ALLOW
        assert config.check_tool("Edit") == PermissionResult.ALLOW
    def test_permission_config_bypass_allows(self):
        """Test that BYPASS mode allows all tools."""
        from openlaoke.types.permissions import PermissionConfig
        from openlaoke.types.core_types import PermissionMode, PermissionResult
        config = PermissionConfig.defaults()
        config.mode = PermissionMode.BYPASS
        assert config.check_tool("Bash") == PermissionResult.ALLOW
        assert config.check_tool("Write") == PermissionResult.ALLOW
class TestSkillIntegration:
    """Tests for skill system integration."""
    def test_skill_loading_from_directory(self):
        """Test loading skills from directory."""
        from openlaoke.core.skill_system import SkillRegistry
        with tempfile.TemporaryDirectory() as tmpdir:
            tmpdir = Path(tmpdir)
            skill_dir = tmpdir / "test_skill"
            skill_dir.mkdir()
            (skill_dir / "SKILL.md").write_text("""---
name: test_skill
description: Test skill
allowed-tools:
  - Bash
---
# Test Skill
Instructions here.
""")
            registry = SkillRegistry()
            count = registry.add_skill_directory(tmpdir)
            assert count == 1
            assert registry.has_skill("test_skill")
    def test_active_skills_in_state(self):
        """Test that active skills are stored in state."""
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            assert hasattr(state, "active_skills")
            assert state.active_skills == []
            state.active_skills.append("browse")
            assert "browse" in state.active_skills
class TestProviderSwitching:
    """Tests for switching between providers."""
    def test_switch_provider(self):
        """Test switching active provider."""
        config = MultiProviderConfig.defaults()
        config.active_provider = "minimax"
        assert config.active_provider == "minimax"
```

```
        config.active_provider = "ollama"
        assert config.active_provider == "ollama"
    def test_get_active_model(self):
        """Test getting active model."""
        config = MultiProviderConfig.defaults()
        config.active_provider = "minimax"
        model = config.get_active_model()
        assert model == "MiniMax-M2.7-highspeed"
        config.active_provider = "ollama"
        model = config.get_active_model()
        assert model == "gemma4:e2b"
class TestTokenAndCost:
    """Tests for token and cost tracking."""
    def test_token_accumulation(self):
        """Test accumulating tokens."""
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            usage = type(
                "TokenUsage",
                (),
                {
                    "input_tokens": 100,
                    "output_tokens": 50,
                    "cache_read_tokens": 0,
                    "cache_creation_tokens": 0,
                },
            )()
            state.accumulate_tokens(usage)
            assert state.token_usage.input_tokens == 100
            assert state.token_usage.output_tokens == 50
            state.accumulate_tokens(usage)
            assert state.token_usage.input_tokens == 200
            assert state.token_usage.output_tokens == 100
    def test_cost_accumulation(self):
        """Test accumulating costs."""
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
        from openlaoke.types.core_types import CostInfo
        cost = CostInfo(input_cost=0.005, output_cost=0.005)
        state.accumulate_cost(cost)
        assert state.cost_info.total_cost == 0.01
        state.accumulate_cost(cost)
        assert state.cost_info.total_cost == 0.02
```

File: [233/252] tests/test_intent_pipeline.py

```
"""End-to-end tests for intent-based pipeline."""
from __future__ import annotations
import pytest
from openlaoke.core.intent_pipeline import (
    IntentBasedPipeline,
    ProcessingPipelineResult,
    create_pipeline_for_model,
)
from openlaoke.core.intent_parser import IntentType, ProgrammingLanguage
from openlaoke.core.architecture.interfaces import ComponentType
from openlaoke.core.model_assessment.types import ModelTier
from pathlib import Path
class TestIntentBasedPipeline:
    def test_pipeline_initialization(self) -> None:
        pipeline = IntentBasedPipeline()
        assert pipeline is not None
        assert pipeline.parser is not None
        assert pipeline.converter is not None
        assert pipeline.decomposer is not None
    def test_pipeline_for_tier_5(self) -> None:
        pipeline = create_pipeline_for_model(ModelTier.TIER_5_LIMITED)
        assert pipeline.model_tier == ModelTier.TIER_5_LIMITED
        assert pipeline.decomposer.max_lines_per_task == 15
        assert pipeline.decomposer.max_params_per_function == 3
    def test_process_write_program_request(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a python calculator program"
        result = pipeline.process_request(request)
        assert result.success
        assert result.intent.intent_type == IntentType.WRITE_PROGRAM
        assert result.intent.language == ProgrammingLanguage.PYTHON
        assert len(result.specs) > 0
        assert len(result.tasks) > 0
        assert result.task_graph is not None
        assert result.ready_to_execute
    def test_process_write_function_request(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "create a function to sort list"
        result = pipeline.process_request(request)
        assert result.success
        assert result.intent.intent_type == IntentType.WRITE_FUNCTION
        assert len(result.specs) > 0
        assert len(result.tasks) > 0
    def test_process_write_class_request(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "implement a class for user manager"
        result = pipeline.process_request(request)
        assert result.success
        assert result.intent.intent_type == IntentType.WRITE_CLASS
        assert len(result.specs) > 0
        assert len(result.tasks) > 0
    def test_unknown_request_fails_gracefully(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "hello world"
        result = pipeline.process_request(request)
        assert not result.success
        assert result.intent.intent_type == IntentType.UNKNOWN
        assert len(result.errors) > 0
        assert not result.ready_to_execute
    def test_execution_plan_generated(self) -> None:
        pipeline = IntentBasedPipeline()
```

```
        request = "write a benchmark program"
        result = pipeline.process_request(request)
        assert result.success
        plan = pipeline.get_execution_plan(result)
        assert "total_tasks" in plan
        assert "ready_tasks" in plan
        assert "pending_tasks" in plan
        assert plan["total_tasks"] > 0
    def test_task_prompt_generated(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a calculator program"
        result = pipeline.process_request(request)
        assert result.success
        assert len(result.tasks) > 0
        task = result.tasks[0]
        prompt = pipeline.generate_task_prompt(task)
        assert task.description in prompt
        assert "Requirements:" in prompt
        assert "lines" in prompt.lower()
    def test_validate_correct_code(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a function"
        result = pipeline.process_request(request)
        assert result.success
        task = result.tasks[0]
        correct_code = '''
def example_function(x: int) -> int:
    """Example function."""
    return x * 2
'''
        is_valid, errors = pipeline.validate_task_result(task, correct_code)
        assert is_valid
        assert len(errors) == 0
    def test_validate_incorrect_code_syntax_error(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a function"
        result = pipeline.process_request(request)
        assert result.success
        task = result.tasks[0]
        incorrect_code = """
def example_function(x: int) -> int:
    return x * 2
    else:
        pass
"""
        is_valid, errors = pipeline.validate_task_result(task, incorrect_code)
        assert not is_valid
        assert len(errors) > 0
        assert any("syntax" in e.lower() for e in errors)
    def test_validate_missing_type_hints(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a function"
        result = pipeline.process_request(request)
        assert result.success
        task = result.tasks[0]
        code_without_hints = '''
def example_function(x):
    """Example function."""
    return x * 2
'''
        is_valid, errors = pipeline.validate_task_result(task, code_without_hints)
        assert not is_valid
        assert any("type hint" in e.lower() for e in errors)
    def test_validate_missing_docstring(self) -> None:
        pipeline = IntentBasedPipeline()
```

```
        request = "write a function"
        result = pipeline.process_request(request)
        assert result.success
        task = result.tasks[0]
        code_without_docstring = """
def example_function(x: int) -> int:
    return x * 2
"""
        is_valid, errors = pipeline.validate_task_result(task, code_without_docstring)
        assert not is_valid
        assert any("docstring" in e.lower() for e in errors)
    def test_tier_5_task_size_limits(self) -> None:
        pipeline = IntentBasedPipeline(ModelTier.TIER_5_LIMITED)
        request = "write a comprehensive large complex program"
        result = pipeline.process_request(request)
        assert result.success
        for task in result.tasks:
            assert task.estimated_lines <= 15
    def test_cpu_benchmark_example(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a cpu benchmark program"
        result = pipeline.process_request(request)
        assert result.success
        assert result.intent.intent_type == IntentType.WRITE_PROGRAM
        assert result.intent.task_name == "benchmark"
        plan = pipeline.get_execution_plan(result)
        assert plan["total_tasks"] >= 3
        has_module = any(
            "module" in task["description"].lower()
            for task in plan["ready_tasks"] + plan["pending_tasks"]
        )
        has_main = any(
            "main" in task["description"].lower() or "benchmark" in task["description"].lower()
            for task in plan["ready_tasks"] + plan["pending_tasks"]
        )
        has_test = any(
            "test" in task["description"].lower()
            for task in plan["ready_tasks"] + plan["pending_tasks"]
        )
        assert has_module or has_main or has_test
    def test_tasks_have_dependencies(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a calculator program with validation"
        result = pipeline.process_request(request)
        assert result.success
        tasks_with_deps = [t for t in result.tasks if len(t.dependencies) > 0]
        assert len(tasks_with_deps) > 0
    def test_task_graph_ready_tasks(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a program"
        result = pipeline.process_request(request)
        assert result.success
        assert result.task_graph is not None
        ready_tasks = result.task_graph.get_ready_tasks()
        assert len(ready_tasks) > 0
        for task in ready_tasks:
            assert len(task.dependencies) == 0 or all(
                dep in result.task_graph.completed for dep in task.dependencies
            )
class TestEndToEndWorkflow:
    def test_full_workflow_cpu_benchmark(self) -> None:
        pipeline = IntentBasedPipeline(ModelTier.TIER_5_LIMITED)
        request = "write a CPU benchmark program that calculates real performance"
        result = pipeline.process_request(request)
        assert result.success
```

```
        assert result.ready_to_execute
        plan = pipeline.get_execution_plan(result)
        assert plan["total_tasks"] > 0
        ready_tasks = result.task_graph.get_ready_tasks() if result.task_graph else []
        assert len(ready_tasks) > 0
        first_task = ready_tasks[0]
        prompt = pipeline.generate_task_prompt(first_task)
        assert "Task:" in prompt
        assert "Requirements:" in prompt
        assert "15 lines" in prompt
        code_example = '''
def measure_performance(iterations: int) -> float:
    """Measure CPU performance."""
    import time
    start = time.time()
    for _ in range(iterations):
        _ = sum(range(1000))
    end = time.time()
    return iterations / (end - start)
'''
        is_valid, errors = pipeline.validate_task_result(first_task, code_example)
        assert is_valid or len(errors) == 0 or all("lines" in e.lower() for e in errors)
    def test_decomposition_creates_atomic_tasks(self) -> None:
        pipeline = IntentBasedPipeline(ModelTier.TIER_5_LIMITED)
        request = "write a comprehensive program with input validation, calculation, and output
     formatting"
        result = pipeline.process_request(request)
        assert result.success
        for task in result.tasks:
            assert task.estimated_lines <= 15
            assert task.template is not None
            assert task.description is not None
    def test_incremental_execution_order(self) -> None:
        pipeline = IntentBasedPipeline()
        request = "write a program"
        result = pipeline.process_request(request)
        assert result.success
        assert result.task_graph is not None
        execution_order = []
        graph = result.task_graph
        iteration = 0
        max_iterations = 10
        while len(graph.completed) < len(graph.tasks) and iteration < max_iterations:
            ready = graph.get_ready_tasks()
            if not ready:
                break
            for task in ready:
                execution_order.append(task.task_id)
                graph.mark_completed(task.task_id)
            iteration += 1
        assert len(execution_order) > 0
        assert len(graph.completed) == len(graph.tasks)
```

File: [234/252] tests/test_intent_system.py

```
"""Tests for intent parsing and specification conversion."""
from __future__ import annotations
import pytest
from openlaoke.core.intent_parser import (
    IntentParser,
    IntentType,
    ProgrammingIntent,
    ProgrammingLanguage,
    TaskComplexity,
)
from openlaoke.core.intent_to_spec import (
    IntentToSpecConverter,
    SpecGenerationResult,
    create_converter_for_model,
)
from openlaoke.core.architecture.interfaces import ComponentType
from openlaoke.core.model_assessment.types import ModelTier
class TestIntentParser:
    def test_parser_initialization(self) -> None:
        parser = IntentParser()
        assert parser is not None
        assert hasattr(parser, "parse")
        assert hasattr(parser, "INTENT_PATTERNS")
        assert hasattr(parser, "LANGUAGE_PATTERNS")
    def test_parse_write_program_intent(self) -> None:
        parser = IntentParser()
        request = "write a new program for CPU benchmark"
        intent = parser.parse(request)
        assert intent.intent_type == IntentType.WRITE_PROGRAM
        assert intent.language == ProgrammingLanguage.PYTHON
        assert intent.task_name == "benchmark"
        assert "benchmark" in intent.description.lower()
        assert intent.confidence > 0.0
    def test_parse_write_function_intent(self) -> None:
        parser = IntentParser()
        request = "create a function to calculate average"
        intent = parser.parse(request)
        assert intent.intent_type == IntentType.WRITE_FUNCTION
        assert intent.language == ProgrammingLanguage.PYTHON
        assert intent.task_name == "calculator"
        assert intent.confidence > 0.0
    def test_parse_write_class_intent(self) -> None:
        parser = IntentParser()
        request = "implement a class for user management"
        intent = parser.parse(request)
        assert intent.intent_type == IntentType.WRITE_CLASS
        assert intent.language == ProgrammingLanguage.PYTHON
        assert "manager" in intent.task_name.lower() or "management" in intent.description.lowe
    r()
        assert intent.confidence > 0.0
    def test_parse_debug_intent(self) -> None:
        parser = IntentParser()
        request = "debug the error in my code"
        intent = parser.parse(request)
        assert intent.intent_type == IntentType.DEBUG_CODE
        assert intent.confidence > 0.0
    def test_parse_refactor_intent(self) -> None:
        parser = IntentParser()
        request = "refactor the code to make it cleaner"
        intent = parser.parse(request)
        assert intent.intent_type == IntentType.REFACTOR_CODE
```

```
    assert intent.confidence > 0.0
def test_parse_test_intent(self) -> None:
    parser = IntentParser()
    request = "write tests for the calculator"
    intent = parser.parse(request)
    assert intent.intent_type == IntentType.TEST_CODE
    assert intent.confidence > 0.0
def test_parse_analyze_intent(self) -> None:
    parser = IntentParser()
    request = "analyze the code for performance issues"
    intent = parser.parse(request)
    assert intent.intent_type == IntentType.ANALYZE_CODE
    assert intent.confidence > 0.0
def test_parse_document_intent(self) -> None:
    parser = IntentParser()
    request = "document the function with comments"
    intent = parser.parse(request)
    assert intent.intent_type == IntentType.DOCUMENT_CODE
    assert intent.confidence > 0.0
def test_parse_unknown_intent(self) -> None:
    parser = IntentParser()
    request = "hello world"
    intent = parser.parse(request)
    assert intent.intent_type == IntentType.UNKNOWN
    assert intent.confidence < 0.5
def test_detect_python_language(self) -> None:
    parser = IntentParser()
    request = "write a python program for sorting"
    intent = parser.parse(request)
    assert intent.language == ProgrammingLanguage.PYTHON
def test_detect_javascript_language(self) -> None:
    parser = IntentParser()
    request = "create a javascript app"
    intent = parser.parse(request)
    assert intent.language == ProgrammingLanguage.JAVASCRIPT
def test_detect_rust_language(self) -> None:
    parser = IntentParser()
    request = "implement a rust program"
    intent = parser.parse(request)
    assert intent.language == ProgrammingLanguage.RUST
def test_extract_requirements(self) -> None:
    parser = IntentParser()
    request = "write a program that should support multiple inputs and handle errors"
    intent = parser.parse(request)
    assert len(intent.requirements) > 0
    assert any(
        "support" in req.lower() or "handle" in req.lower() for req in intent.requirements
    )
def test_extract_inputs(self) -> None:
    parser = IntentParser()
    request = "write a function that takes filename as input"
    intent = parser.parse(request)
    assert len(intent.inputs) > 0
    assert any("filename" in inp.lower() for inp in intent.inputs)
def test_extract_outputs(self) -> None:
    parser = IntentParser()
    request = "create a program that output: calculated result"
    intent = parser.parse(request)
    assert len(intent.outputs) > 0
def test_detect_atomic_complexity(self) -> None:
    parser = IntentParser()
    request = "write simple basic minimal program"
    intent = parser.parse(request)
    assert intent.complexity == TaskComplexity.ATOMIC
def test_detect_simple_complexity(self) -> None:
```

```
        parser = IntentParser()
        request = "create small easy straightforward function"
        intent = parser.parse(request)
        assert intent.complexity == TaskComplexity.SIMPLE
    def test_detect_complex_complexity(self) -> None:
        parser = IntentParser()
        request = "build complex advanced large comprehensive system with multiple modules and
    full architecture"
        intent = parser.parse(request)
        assert intent.complexity == TaskComplexity.COMPLEX
    def test_cache_intent(self) -> None:
        parser = IntentParser()
        request = "write a program"
        intent1 = parser.parse(request)
        intent2 = parser.parse(request)
        assert intent1.raw_request == intent2.raw_request
        assert request in parser._cache
    def test_suggest_clarifications_for_unknown_intent(self) -> None:
        parser = IntentParser()
        request = "hello"
        intent = parser.parse(request)
        questions = parser.suggest_clarifications(intent)
        assert len(questions) > 0
        assert any("details" in q.lower() or "type" in q.lower() for q in questions)
    def test_to_dict(self) -> None:
        parser = IntentParser()
        request = "write a python calculator program"
        intent = parser.parse(request)
        intent_dict = intent.to_dict()
        assert "intent_type" in intent_dict
        assert "language" in intent_dict
        assert "task_name" in intent_dict
        assert intent_dict["intent_type"] == IntentType.WRITE_PROGRAM.value
class TestIntentToSpecConverter:
    def test_converter_initialization(self) -> None:
        converter = IntentToSpecConverter()
        assert converter is not None
        assert converter.model_tier == ModelTier.TIER_5_LIMITED
        assert converter.max_lines_per_spec == 15
        assert converter.max_params_per_spec == 3
    def test_converter_for_different_tiers(self) -> None:
        converter_tier5 = create_converter_for_model(ModelTier.TIER_5_LIMITED)
        assert converter_tier5.max_lines_per_spec == 15
        assert converter_tier5.max_params_per_spec == 3
        converter_tier1 = create_converter_for_model(ModelTier.TIER_1_ADVANCED)
        assert converter_tier1.max_lines_per_spec == 100
        assert converter_tier1.max_params_per_spec == 10
    def test_convert_write_program_intent(self) -> None:
        parser = IntentParser()
        converter = IntentToSpecConverter()
        request = "write a benchmark program"
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert result.success
        assert len(result.specs) > 0
        assert any(spec.component_type == ComponentType.MODULE for spec in result.specs)
        assert any(spec.component_type == ComponentType.TEST for spec in result.specs)
    def test_convert_write_function_intent(self) -> None:
        parser = IntentParser()
        converter = IntentToSpecConverter()
        request = "create a function to sort list"
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert result.success
        assert len(result.specs) > 0
```

```
    assert any(spec.component_type == ComponentType.FUNCTION for spec in result.specs)
def test_convert_write_class_intent(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter()
    request = "implement a class for data processor"
    intent = parser.parse(request)
    result = converter.convert(intent)
    assert result.success
    assert len(result.specs) > 0
    assert any(spec.component_type == ComponentType.CLASS for spec in result.specs)
def test_convert_unknown_intent_fails(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter()
    request = "hello"
    intent = parser.parse(request)
    result = converter.convert(intent)
    assert not result.success
    assert len(result.specs) == 0
    assert len(result.errors) > 0
def test_spec_has_api_spec(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter()
    request = "write a calculator program"
    intent = parser.parse(request)
    result = converter.convert(intent)
    assert result.success
    for spec in result.specs:
        if spec.component_type != ComponentType.TEST:
            assert spec.api_spec is not None
            assert spec.api_spec.name
            assert spec.api_spec.description
def test_spec_has_test_requirements(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter()
    request = "write a program"
    intent = parser.parse(request)
    result = converter.convert(intent)
    assert result.success
    for spec in result.specs:
        if spec.component_type != ComponentType.TEST:
            assert len(spec.test_requirements) > 0
def test_spec_complexity_within_limits(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter(ModelTier.TIER_5_LIMITED)
    request = "write a complex comprehensive program"
    intent = parser.parse(request)
    result = converter.convert(intent)
    assert result.success
    for spec in result.specs:
        assert spec.complexity_score <= converter.max_lines_per_spec
def test_warning_for_excessive_params(self) -> None:
    parser = IntentParser()
    converter = IntentToSpecConverter()
    request = "write a function with many inputs: param1, param2, param3, param4, param5"
    intent = parser.parse(request)
    intent.inputs = ["param1", "param2", "param3", "param4", "param5"]
    result = converter.convert(intent)
    assert len(result.warnings) > 0 or all(
        len(spec.api_spec.input_schema.get("properties", {})) <= converter.max_params_per_s
pec
        for spec in result.specs
        if spec.api_spec
    )
def test_helper_specs_created_for_requirements(self) -> None:
    parser = IntentParser()
```

```
        converter = IntentToSpecConverter()
        request = "write a program that should validate input and parse format"
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert result.success
        helper_specs = [
            spec
            for spec in result.specs
            if spec.name
            and (
                "validate" in spec.name.lower()
                or "parse" in spec.name.lower()
                or "helper" in spec.name.lower()
            )
        ]
        assert len(helper_specs) > 0
    def test_spec_generation_result_structure(self) -> None:
        parser = IntentParser()
        converter = IntentToSpecConverter()
        request = "write a program"
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert hasattr(result, "success")
        assert hasattr(result, "specs")
        assert hasattr(result, "errors")
        assert hasattr(result, "warnings")
        assert hasattr(result, "suggestions")
class TestIntegration:
    def test_full_pipeline_intent_to_spec(self) -> None:
        parser = IntentParser()
        converter = IntentToSpecConverter()
        request = (
            "write a cpu benchmark program that measures performance and calculates ops per sec
    ond"
        )
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert intent.intent_type == IntentType.WRITE_PROGRAM
        assert intent.language == ProgrammingLanguage.PYTHON
        assert intent.confidence > 0.5
        assert result.success
        assert len(result.specs) >= 3
        module_specs = [s for s in result.specs if s.component_type == ComponentType.MODULE]
        assert len(module_specs) >= 1
        function_specs = [s for s in result.specs if s.component_type == ComponentType.FUNCTION
    ]
        assert len(function_specs) >= 1
        test_specs = [s for s in result.specs if s.component_type == ComponentType.TEST]
        assert len(test_specs) >= 1
    def test_tier_5_limits_enforced(self) -> None:
        parser = IntentParser()
        converter = IntentToSpecConverter(ModelTier.TIER_5_LIMITED)
        request = "write a very complex comprehensive advanced large system with many features"
        intent = parser.parse(request)
        result = converter.convert(intent)
        assert result.success
        for spec in result.specs:
            if spec.api_spec:
                input_params = spec.api_spec.input_schema.get("properties", {})
                if len(input_params) > 3:
                    assert any("too many parameters" in w.lower() for w in result.warnings)
            if spec.complexity_score > 15:
                assert any(
                    "exceeds model limit" in w.lower() or "will be decomposed" in w.lower()
                    for w in result.warnings
```

)

File: [235/252] tests/test_knowledge_base.py

```
"""Tests for knowledge base system."""
import pytest
from openlaoke.core.knowledge_base import (
    KnowledgeBase,
    KnowledgeSnippet,
    create_knowledge_base,
)
class TestKnowledgeSnippet:
    def test_snippet_creation(self):
        snippet = KnowledgeSnippet(
            topic="Python",
            content="Python is a programming language",
            source="Docs",
            tags=["programming", "language"],
        )
        assert snippet.topic == "Python"
        assert snippet.tags == ["programming", "language"]
class TestKnowledgeBase:
    def test_knowledge_base_creation(self):
        kb = KnowledgeBase()
        assert kb is not None
        assert len(kb.knowledge_cache) > 0
    def test_builtin_knowledge_loaded(self):
        kb = KnowledgeBase()
        assert "cpu_benchmark" in kb.knowledge_cache
        assert "file_operations" in kb.knowledge_cache
        assert "python_syntax" in kb.knowledge_cache
    def test_search_cpu_benchmark(self):
        kb = KnowledgeBase()
        results = kb.search("write a CPU benchmark program")
        assert len(results) > 0
        assert any("cpu" in r.tags for r in results)
    def test_search_file_operations(self):
        kb = KnowledgeBase()
        results = kb.search("how to write a file")
        assert len(results) > 0
    def test_search_python_syntax(self):
        kb = KnowledgeBase()
        results = kb.search("python syntax error")
        assert len(results) > 0
    def test_search_no_results(self):
        kb = KnowledgeBase()
        results = kb.search("xyzabc123 nonexistent topic")
        assert len(results) == 0
    def test_enhance_prompt_with_knowledge(self):
        kb = KnowledgeBase()
        enhanced = kb.enhance_prompt("write CPU benchmark", "You are helpful.")
        assert "RELEVANT KNOWLEDGE BASE" in enhanced
        assert "CPU" in enhanced
    def test_enhance_prompt_no_knowledge(self):
        kb = KnowledgeBase()
        enhanced = kb.enhance_prompt("xyzabc123", "You are helpful.")
        assert enhanced == "You are helpful."
    def test_search_returns_max_results(self):
        kb = KnowledgeBase()
        results = kb.search("python", max_results=2)
        assert len(results) <= 2
class TestCreateKnowledgeBase:
    def test_create_knowledge_base(self):
        kb = create_knowledge_base()
        assert isinstance(kb, KnowledgeBase)
```

```
class TestKnowledgeRelevance:
    def test_relevance_scoring(self):
        kb = KnowledgeBase()
        results = kb.search("CPU performance benchmark calculation")
        assert len(results) > 0
        first = results[0]
        assert "cpu" in first.tags or "benchmark" in first.tags
    def test_multiple_keywords(self):
        kb = KnowledgeBase()
        results = kb.search("python file write error")
        assert len(results) > 0
        # Should return relevant snippets
        topics = [r.topic.lower() for r in results]
        has_file = any("file" in t for t in topics)
        has_syntax = any("syntax" in t or "error" in t for t in topics)
        assert has_file or has_syntax
class TestKnowledgeContent:
    def test_cpu_benchmark_has_example_code(self):
        kb = KnowledgeBase()
        snippet = kb.knowledge_cache["cpu_benchmark"]
        assert "```" in snippet.content
        assert "multiprocessing" in snippet.content
    def test_python_syntax_has_common_errors(self):
        kb = KnowledgeBase()
        snippet = kb.knowledge_cache["python_syntax"]
        assert "SyntaxError" in snippet.content
        assert "Wrong" in snippet.content
        assert "Correct" in snippet.content
```

File: [236/252] tests/test_mcp_transport.py

```
"""Tests for MCP transport implementations."""
import asyncio
import json
import os
import tempfile
from unittest.mock import AsyncMock, MagicMock, patch
import pytest
from openlaoke.core.tool import ToolRegistry
from openlaoke.services.mcp import (
    ConnectionState,
    MCPServerConfig,
    MCPServerConnection,
    MCPService,
    OAuthConfig,
    OAuthState,
    OAuthTokens,
    TransportType,
)
@pytest.fixture
def registry():
    return ToolRegistry()
@pytest.fixture
def mcp_service(registry):
    return MCPService(registry)
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
class TestTransportType:
    def test_transport_types(self):
        assert TransportType.STDIO == "stdio"
        assert TransportType.SSE == "sse"
        assert TransportType.HTTP == "http"
        assert TransportType.WS == "ws"
    def test_transport_type_values(self):
        types = [t.value for t in TransportType]
        assert "stdio" in types
        assert "sse" in types
        assert "http" in types
        assert "ws" in types
class TestConnectionState:
    def test_connection_states(self):
        assert ConnectionState.CONNECTED == "connected"
        assert ConnectionState.DISCONNECTED == "disconnected"
        assert ConnectionState.FAILED == "failed"
        assert ConnectionState.NEEDS_AUTH == "needs_auth"
    def test_all_states(self):
        states = [s.value for s in ConnectionState]
        assert len(states) == 6
class TestOAuthConfig:
    def test_default_config(self):
        config = OAuthConfig()
        assert config.client_id is None
        assert config.client_secret is None
        assert config.scope is None
        assert config.redirect_uri == "http://127.0.0.1:19876/mcp/oauth/callback"
    def test_custom_config(self):
        config = OAuthConfig(
            client_id="test_client",
            client_secret="test_secret",
            scope="read write",
```

```
            redirect_uri="http://custom.uri/callback",
        )
        assert config.client_id == "test_client"
        assert config.client_secret == "test_secret"
        assert config.scope == "read write"
        assert config.redirect_uri == "http://custom.uri/callback"
class TestOAuthTokens:
    def test_default_tokens(self):
        tokens = OAuthTokens(access_token="token123")
        assert tokens.access_token == "token123"
        assert tokens.refresh_token is None
        assert tokens.expires_at is None
        assert tokens.scope is None
    def test_full_tokens(self):
        tokens = OAuthTokens(
            access_token="access",
            refresh_token="refresh",
            expires_at=12345.0,
            scope="read",
        )
        assert tokens.access_token == "access"
        assert tokens.refresh_token == "refresh"
        assert tokens.expires_at == 12345.0
class TestOAuthState:
    def test_default_state(self):
        state = OAuthState()
        assert state.code_verifier is None
        assert state.state is None
        assert state.tokens is None
    def test_with_tokens(self):
        tokens = OAuthTokens(access_token="test")
        state = OAuthState(code_verifier="verifier", state="state123", tokens=tokens)
        assert state.code_verifier == "verifier"
        assert state.state == "state123"
        assert state.tokens.access_token == "test"
class TestMCPServerConfig:
    def test_stdio_config(self):
        config = MCPServerConfig(
            name="test",
            transport=TransportType.STDIO,
            command="python",
            args=["-m", "mcp_server"],
        )
        assert config.name == "test"
        assert config.transport == TransportType.STDIO
        assert config.command == "python"
        assert config.args == ["-m", "mcp_server"]
        assert config.enabled
    def test_http_config(self):
        config = MCPServerConfig(
            name="http_test",
            transport=TransportType.HTTP,
            url="http://localhost:8080/mcp",
            headers={"Authorization": "Bearer token"},
        )
        assert config.name == "http_test"
        assert config.transport == TransportType.HTTP
        assert config.url == "http://localhost:8080/mcp"
        assert "Authorization" in config.headers
    def test_sse_config(self):
        config = MCPServerConfig(
            name="sse_test",
            transport=TransportType.SSE,
            url="http://localhost:8080/sse",
        )
```

```
        assert config.transport == TransportType.SSE
    def test_ws_config(self):
        config = MCPServerConfig(
            name="ws_test",
            transport=TransportType.WS,
            url="ws://localhost:8080/ws",
        )
        assert config.transport == TransportType.WS
    def test_oauth_config(self):
        oauth = OAuthConfig(client_id="client123")
        config = MCPServerConfig(
            name="oauth_test",
            transport=TransportType.HTTP,
            url="http://example.com/mcp",
            oauth=oauth,
        )
        assert config.oauth.client_id == "client123"
    def test_disabled_config(self):
        config = MCPServerConfig(name="disabled", enabled=False)
        assert not config.enabled
    def test_timeout_config(self):
        config = MCPServerConfig(name="timeout_test", timeout=60000)
        assert config.timeout == 60000
    def test_env_vars(self):
        config = MCPServerConfig(
            name="env_test",
            env={"API_KEY": "secret", "DEBUG": "1"},
        )
        assert config.env["API_KEY"] == "secret"
        assert config.env["DEBUG"] == "1"
class TestMCPServerConnection:
    def test_default_connection(self):
        config = MCPServerConfig(name="test")
        conn = MCPServerConnection(config=config)
        assert conn.config.name == "test"
        assert conn.state == ConnectionState.DISCONNECTED
        assert conn.tools == []
        assert conn.process is None
        assert conn.error is None
    def test_connection_with_oauth_state(self):
        config = MCPServerConfig(name="oauth_test", oauth=OAuthConfig())
        oauth_state = OAuthState()
        conn = MCPServerConnection(config=config, oauth_state=oauth_state)
        assert conn.oauth_state is not None
    def test_connection_state_changes(self):
        config = MCPServerConfig(name="test")
        conn = MCPServerConnection(config=config)
        conn.state = ConnectionState.CONNECTED
        assert conn.state == ConnectionState.CONNECTED
        conn.state = ConnectionState.FAILED
        assert conn.state == ConnectionState.FAILED
class TestMCPService:
    def test_service_initialization(self, mcp_service):
        assert mcp_service.registry is not None
        assert mcp_service.servers == {}
        assert mcp_service._config_path.endswith("mcp_servers.json")
        assert mcp_service._auth_path.endswith("mcp_auth.json")
    def test_oauth_callback_port(self, mcp_service):
        assert mcp_service.OAUTH_CALLBACK_PORT == 19876
        assert mcp_service.OAUTH_CALLBACK_PATH == "/mcp/oauth/callback"
    @pytest.mark.asyncio
    async def test_load_empty_config(self, mcp_service, temp_dir):
        config_path = os.path.join(temp_dir, "mcp_servers.json")
        mcp_service._config_path = config_path
        await mcp_service.load_config()
```

```
    assert mcp_service.servers == {}
@pytest.mark.asyncio
async def test_load_config_with_servers(self, mcp_service, temp_dir):
    config_path = os.path.join(temp_dir, "mcp_servers.json")
    config_data = {
        "mcpServers": {
            "test_server": {
                "transport": "stdio",
                "command": "python",
                "args": ["-m", "test"],
                "enabled": True,
            }
        }
    }
    with open(config_path, "w") as f:
        json.dump(config_data, f)
    mcp_service._config_path = config_path
    await mcp_service.load_config()
    assert "test_server" in mcp_service.servers
    assert mcp_service.servers["test_server"].config.transport == TransportType.STDIO
@pytest.mark.asyncio
async def test_load_config_http_transport(self, mcp_service, temp_dir):
    config_path = os.path.join(temp_dir, "mcp_servers.json")
    config_data = {
        "mcpServers": {
            "http_server": {
                "transport": "http",
                "url": "http://localhost:8080/mcp",
            }
        }
    }
    with open(config_path, "w") as f:
        json.dump(config_data, f)
    mcp_service._config_path = config_path
    await mcp_service.load_config()
    assert mcp_service.servers["http_server"].config.transport == TransportType.HTTP
@pytest.mark.asyncio
async def test_load_config_oauth(self, mcp_service, temp_dir):
    config_path = os.path.join(temp_dir, "mcp_servers.json")
    config_data = {
        "mcpServers": {
            "oauth_server": {
                "transport": "http",
                "url": "http://example.com/mcp",
                "oauth": {
                    "clientId": "test_client",
                    "scope": "read",
                },
            }
        }
    }
    with open(config_path, "w") as f:
        json.dump(config_data, f)
    mcp_service._config_path = config_path
    await mcp_service.load_config()
    conn = mcp_service.servers["oauth_server"]
    assert conn.config.oauth is not None
    assert conn.config.oauth.client_id == "test_client"
@pytest.mark.asyncio
async def test_connect_disabled_server(self, mcp_service):
    config = MCPServerConfig(name="disabled", enabled=False)
    conn = MCPServerConnection(config=config)
    mcp_service.servers["disabled"] = conn
    result = await mcp_service.connect_server("disabled")
    assert result.state == ConnectionState.DISABLED
```

```
@pytest.mark.asyncio
async def test_connect_nonexistent_server(self, mcp_service):
    result = await mcp_service.connect_server("nonexistent")
    assert result is None
@pytest.mark.asyncio
async def test_disconnect_all(self, mcp_service):
    config = MCPServerConfig(name="test", transport=TransportType.HTTP, url="http://test")
    conn = MCPServerConnection(config=config)
    conn._http_client = AsyncMock()
    conn._http_client.aclose = AsyncMock()
    mcp_service.servers["test"] = conn
    await mcp_service.disconnect_all()
    assert conn._http_client is None
def test_get_server_status_empty(self, mcp_service):
    status = mcp_service.get_server_status()
    assert status == []
def test_get_server_status_with_servers(self, mcp_service):
    config1 = MCPServerConfig(name="server1")
    config2 = MCPServerConfig(name="server2")
    conn1 = MCPServerConnection(config=config1, state=ConnectionState.CONNECTED)
    conn2 = MCPServerConnection(config=config2, state=ConnectionState.DISCONNECTED)
    mcp_service.servers["server1"] = conn1
    mcp_service.servers["server2"] = conn2
    status = mcp_service.get_server_status()
    assert len(status) == 2
    assert any(s["name"] == "server1" and s["state"] == "connected" for s in status)
    assert any(s["name"] == "server2" and s["state"] == "disconnected" for s in status)
def test_supports_oauth_stdio_false(self, mcp_service):
    config = MCPServerConfig(name="stdio", transport=TransportType.STDIO)
    conn = MCPServerConnection(config=config)
    mcp_service.servers["stdio"] = conn
    assert not mcp_service.supports_oauth("stdio")
def test_supports_oauth_http_true(self, mcp_service):
    config = MCPServerConfig(name="http", transport=TransportType.HTTP, url="http://test")
    conn = MCPServerConnection(config=config)
    mcp_service.servers["http"] = conn
    assert mcp_service.supports_oauth("http")
def test_supports_oauth_disabled(self, mcp_service):
    config = MCPServerConfig(
        name="disabled_oauth", transport=TransportType.HTTP, url="http://test", oauth=False
    )
    conn = MCPServerConnection(config=config)
    mcp_service.servers["disabled_oauth"] = conn
    assert not mcp_service.supports_oauth("disabled_oauth")
def test_has_stored_tokens_false(self, mcp_service):
    config = MCPServerConfig(name="test")
    conn = MCPServerConnection(config=config)
    mcp_service.servers["test"] = conn
    assert not mcp_service.has_stored_tokens("test")
def test_has_stored_tokens_true(self, mcp_service):
    config = MCPServerConfig(name="test")
    oauth_state = OAuthState(tokens=OAuthTokens(access_token="token"))
    conn = MCPServerConnection(config=config, oauth_state=oauth_state)
    mcp_service.servers["test"] = conn
    assert mcp_service.has_stored_tokens("test")
def test_build_headers_no_auth(self, mcp_service):
    config = MCPServerConfig(name="test", headers={"X-Custom": "value"})
    conn = MCPServerConnection(config=config)
    mcp_service.servers["test"] = conn
    headers = mcp_service._build_headers(conn)
    assert headers["X-Custom"] == "value"
    assert "Authorization" not in headers
def test_build_headers_with_auth(self, mcp_service):
    config = MCPServerConfig(name="test")
    oauth_state = OAuthState(tokens=OAuthTokens(access_token="test_token"))
```

```
        conn = MCPServerConnection(config=config, oauth_state=oauth_state)
        mcp_service.servers["test"] = conn
        headers = mcp_service._build_headers(conn)
        assert headers["Authorization"] == "Bearer test_token"
    def test_nonexistent_server_oauth_check(self, mcp_service):
        assert not mcp_service.supports_oauth("nonexistent")
        assert not mcp_service.has_stored_tokens("nonexistent")
class TestSSETransport:
    @pytest.mark.asyncio
    async def test_sse_connection_success(self, mcp_service):
        config = MCPServerConfig(
            name="sse_test", transport=TransportType.SSE, url="http://localhost:8080/sse"
        )
        conn = MCPServerConnection(config=config)
        mcp_service.servers["sse_test"] = conn
        with patch("httpx.AsyncClient") as mock_client:
            mock_response = MagicMock()
            mock_response.raise_for_status = MagicMock()
            mock_client_instance = AsyncMock()
            mock_client_instance.get = AsyncMock(return_value=mock_response)
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            await mcp_service._connect_sse(conn)
            assert conn.state == ConnectionState.CONNECTED
    @pytest.mark.asyncio
    async def test_sse_needs_auth(self, mcp_service):
        config = MCPServerConfig(
            name="sse_auth", transport=TransportType.SSE, url="http://localhost:8080/sse"
        )
        conn = MCPServerConnection(config=config)
        mcp_service.servers["sse_auth"] = conn
        with patch("httpx.AsyncClient") as mock_client:
            from httpx import HTTPStatusError, Request, Response
            mock_response = MagicMock()
            mock_response.status_code = 401
            mock_client_instance = AsyncMock()
            http_error = HTTPStatusError(
                message="401 Unauthorized",
                request=MagicMock(),
                response=mock_response,
            )
            mock_client_instance.get = AsyncMock(side_effect=http_error)
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            try:
                await mcp_service._connect_sse(conn)
            except Exception:
                pass
            assert conn.state in (
                ConnectionState.NEEDS_AUTH,
                ConnectionState.FAILED,
                ConnectionState.DISCONNECTED,
            )
class TestHTTPTransport:
    @pytest.mark.asyncio
    async def test_http_request(self, mcp_service):
        config = MCPServerConfig(
            name="http_test", transport=TransportType.HTTP, url="http://localhost:8080/mcp"
        )
        conn = MCPServerConnection(config=config)
        with patch("httpx.AsyncClient") as mock_client:
            mock_response = MagicMock()
            mock_response.json = MagicMock(return_value={"result": "success"})
```

```
            mock_response.raise_for_status = MagicMock()
            mock_client_instance = AsyncMock()
            mock_client_instance.post = AsyncMock(return_value=mock_response)
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            conn._http_client = mock_client_instance
            result = await mcp_service._http_request(conn, "test_method", {"param": "value"})
            assert result["result"] == "success"
class TestWebSocketTransport:
    @pytest.mark.asyncio
    async def test_ws_request(self, mcp_service):
        config = MCPServerConfig(
            name="ws_test", transport=TransportType.WS, url="ws://localhost:8080/ws"
        )
        conn = MCPServerConnection(config=config)
        mock_ws = AsyncMock()
        mock_ws.send = AsyncMock()
        mock_ws.recv = AsyncMock(return_value=json.dumps({"result": "ws_success"}))
        conn._ws_connection = mock_ws
        result = await mcp_service._ws_request(conn, "test_method", {"param": "value"})
        assert result["result"] == "ws_success"
class TestOAuthFlow:
    @pytest.mark.asyncio
    async def test_start_oauth_flow(self, mcp_service):
        config = MCPServerConfig(
            name="oauth_test",
            transport=TransportType.HTTP,
            url="http://example.com/mcp",
            oauth=OAuthConfig(client_id="test_client", scope="read write"),
        )
        conn = MCPServerConnection(config=config)
        mcp_service.servers["oauth_test"] = conn
        auth_url, state = await mcp_service.start_oauth_flow("oauth_test")
        assert "authorize" in auth_url
        assert state is not None
        assert conn.oauth_state is not None
        assert conn.oauth_state.code_verifier is not None
    @pytest.mark.asyncio
    async def test_start_oauth_flow_disabled(self, mcp_service):
        config = MCPServerConfig(
            name="oauth_disabled",
            transport=TransportType.HTTP,
            url="http://example.com/mcp",
            oauth=False,
        )
        conn = MCPServerConnection(config=config)
        mcp_service.servers["oauth_disabled"] = conn
        with pytest.raises(ValueError, match="OAuth disabled"):
            await mcp_service.start_oauth_flow("oauth_disabled")
    @pytest.mark.asyncio
    async def test_start_oauth_flow_nonexistent(self, mcp_service):
        with pytest.raises(ValueError, match="not found"):
            await mcp_service.start_oauth_flow("nonexistent")
    @pytest.mark.asyncio
    async def test_finish_oauth_flow(self, mcp_service, temp_dir):
        auth_path = os.path.join(temp_dir, "mcp_auth.json")
        mcp_service._auth_path = auth_path
        config = MCPServerConfig(
            name="oauth_finish",
            transport=TransportType.HTTP,
            url="http://example.com/mcp",
            oauth=OAuthConfig(client_id="test_client"),
        )
        oauth_state = OAuthState(code_verifier="test_verifier", state="test_state")
```

```
        conn = MCPServerConnection(config=config, oauth_state=oauth_state)
        mcp_service.servers["oauth_finish"] = conn
        mcp_service._oauth_transports["oauth_finish"] = conn
        with patch("httpx.AsyncClient") as mock_client:
            mock_response = MagicMock()
            mock_response.json = MagicMock(
                return_value={"access_token": "new_token", "refresh_token": "refresh"}
            )
            mock_response.raise_for_status = MagicMock()
            mock_client_instance = AsyncMock()
            mock_client_instance.post = AsyncMock(return_value=mock_response)
            mock_client_instance.aclose = AsyncMock()
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            with patch.object(mcp_service, "connect_server", AsyncMock()):
                result = await mcp_service.finish_oauth_flow("oauth_finish", "test_code")
        assert conn.oauth_state.tokens is not None
        assert conn.oauth_state.tokens.access_token == "new_token"
    @pytest.mark.asyncio
    async def test_remove_auth(self, mcp_service):
        config = MCPServerConfig(name="auth_test", transport=TransportType.HTTP, url="http://te
    st")
        oauth_state = OAuthState(tokens=OAuthTokens(access_token="token"))
        conn = MCPServerConnection(config=config, oauth_state=oauth_state)
        mcp_service.servers["auth_test"] = conn
        mcp_service._oauth_transports["auth_test"] = conn
        await mcp_service.remove_auth("auth_test")
        assert conn.oauth_state is None
        assert "auth_test" not in mcp_service._oauth_transports
class TestSTDIOTransport:
    @pytest.mark.asyncio
    async def test_stdio_request_format(self, mcp_service):
        config = MCPServerConfig(name="stdio_test", transport=TransportType.STDIO, command="tes
    t")
        conn = MCPServerConnection(config=config)
        mock_process = AsyncMock()
        mock_process.stdin = AsyncMock()
        mock_process.stdin.write = MagicMock()
        mock_process.stdin.drain = AsyncMock()
        mock_process.stdout = AsyncMock()
        mock_process.stdout.readline = AsyncMock(
            return_value=json.dumps({"result": "stdio_test"}).encode()
        )
        conn.process = mock_process
        result = await mcp_service._stdio_request(conn, "test", {"param": 1})
        assert result["result"] == "stdio_test"
        call_args = mock_process.stdin.write.call_args[0][0]
        request = json.loads(call_args.decode())
        assert request["jsonrpc"] == "2.0"
        assert request["method"] == "test"
    @pytest.mark.asyncio
    async def test_stdio_request_timeout(self, mcp_service):
        config = MCPServerConfig(
            name="stdio_timeout", transport=TransportType.STDIO, command="test"
        )
        conn = MCPServerConnection(config=config)
        mock_process = AsyncMock()
        mock_process.stdin = AsyncMock()
        mock_process.stdin.write = MagicMock()
        mock_process.stdin.drain = AsyncMock()
        mock_process.stdout = AsyncMock()
        mock_process.stdout.readline = AsyncMock(side_effect=asyncio.TimeoutError())
        conn.process = mock_process
        with pytest.raises(asyncio.TimeoutError):
```

```
    await mcp_service._stdio_request(conn, "test", {}, timeout=1.0)
```

File: [237/252] tests/test_memory.py

```
"""Tests for long-term memory and knowledge graph system."""
from __future__ import annotations
import asyncio
import tempfile
from pathlib import Path
import pytest
from openlaoke.core.memory import (
    ConsolidationStrategy,
    EmbeddingEngine,
    Fact,
    Inference,
    KnowledgeEdge,
    KnowledgeGraph,
    KnowledgeNode,
    LongTermMemory,
    Memory,
    MemoryConsolidator,
    MemoryRetriever,
    MemorySummary,
    SearchResult,
    VectorIndex,
)
class TestMemory:
    """Test Memory data structure."""
    def test_create_memory(self) -> None:
        m = Memory(id="test_1", content="Test content")
        assert m.id == "test_1"
        assert m.content == "Test content"
        assert m.embedding is None
        assert m.importance == 0.5
        assert m.access_count == 0
    def test_memory_to_dict(self) -> None:
        m = Memory(id="test_2", content="Test", importance=0.8)
        data = m.to_dict()
        assert data["id"] == "test_2"
        assert data["content"] == "Test"
        assert data["importance"] == 0.8
    def test_memory_from_dict(self) -> None:
        data = {
            "id": "test_3",
            "content": "Content",
            "importance": 0.9,
            "access_count": 5,
        }
        m = Memory.from_dict(data)
        assert m.id == "test_3"
        assert m.importance == 0.9
        assert m.access_count == 5
    def test_memory_touch(self) -> None:
        m = Memory(id="test_4", content="Test")
        initial_count = m.access_count
        m.touch()
        assert m.access_count == initial_count + 1
class TestEmbeddingEngine:
    """Test embedding engine."""
    @pytest.mark.asyncio
    async def test_embed_text(self) -> None:
        engine = EmbeddingEngine()
        embedding = await engine.embed("test text")
        assert len(embedding) == 128
        assert all(isinstance(x, float) for x in embedding)
```

```
    @pytest.mark.asyncio
    async def test_embed_batch(self) -> None:
        engine = EmbeddingEngine()
        embeddings = await engine.embed_batch(["text1", "text2"])
        assert len(embeddings) == 2
        assert all(len(e) == 128 for e in embeddings)
    def test_cosine_similarity(self) -> None:
        a = [1.0, 0.0, 0.0]
        b = [1.0, 0.0, 0.0]
        similarity = EmbeddingEngine.cosine_similarity(a, b)
        assert similarity == 1.0
        c = [0.0, 1.0, 0.0]
        similarity = EmbeddingEngine.cosine_similarity(a, c)
        assert similarity == 0.0
    @pytest.mark.asyncio
    async def test_embedding_cache(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            engine = EmbeddingEngine(cache_path=Path(tmpdir))
            embedding1 = await engine.embed("cached text")
            embedding2 = await engine.embed("cached text")
            assert embedding1 == embedding2
class TestVectorIndex:
    """Test vector index."""
    def test_add_and_search(self) -> None:
        index = VectorIndex(dimension=3)
        index.add("vec1", [1.0, 0.0, 0.0])
        index.add("vec2", [0.9, 0.1, 0.0])
        index.add("vec3", [0.0, 1.0, 0.0])
        query = [1.0, 0.0, 0.0]
        results = index.search(query, k=2)
        assert len(results) == 2
        assert results[0][0] == "vec1"
        assert results[0][1] > results[1][1]
    def test_remove(self) -> None:
        index = VectorIndex(dimension=3)
        index.add("vec1", [1.0, 0.0, 0.0])
        index.remove("vec1")
        assert index.size() == 0
    def test_dimension_validation(self) -> None:
        index = VectorIndex(dimension=3)
        index.add("vec1", [1.0, 0.0, 0.0])
        index.add("vec2", [1.0, 0.0])  # Wrong dimension
        assert index.size() == 1
class TestMemoryRetriever:
    """Test memory retriever."""
    @pytest.mark.asyncio
    async def test_search(self) -> None:
        embedder = EmbeddingEngine()
        retriever = MemoryRetriever(embedder)
        m1 = Memory(
            id="m1",
            content="Python programming",
            embedding=await embedder.embed("Python programming"),
        )
        m2 = Memory(
            id="m2",
            content="JavaScript coding",
            embedding=await embedder.embed("JavaScript coding"),
        )
        m3 = Memory(
            id="m3", content="Cooking recipes", embedding=await embedder.embed("Cooking recipes
    ")
        )
        query_embedding = await embedder.embed("Python")
        results = retriever.search([m1, m2, m3], query_embedding, limit=2)
```

```
        assert len(results) == 2
        assert results[0].id == "m1"
    def test_keyword_search(self) -> None:
        retriever = MemoryRetriever()
        m1 = Memory(id="m1", content="Python programming language")
        m2 = Memory(id="m2", content="JavaScript web development")
        results = retriever.keyword_search([m1, m2], ["python"], limit=1)
        assert len(results) == 1
        assert results[0].memory.id == "m1"
class TestLongTermMemory:
    """Test long-term memory system."""
    @pytest.mark.asyncio
    async def test_store_and_recall(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            ltm = LongTermMemory(storage_path=Path(tmpdir))
            m = Memory(id="", content="Test memory about Python")
            await ltm.store(m)
            results = await ltm.recall("Python", limit=1)
            assert len(results) == 1
            assert "Python" in results[0].content
    @pytest.mark.asyncio
    async def test_forget(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            ltm = LongTermMemory(storage_path=Path(tmpdir))
            m = Memory(id="to_delete", content="Delete this")
            await ltm.store(m)
            success = await ltm.forget("to_delete")
            assert success
            results = await ltm.recall("Delete", limit=10)
            assert len(results) == 0
    @pytest.mark.asyncio
    async def test_associate(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            ltm = LongTermMemory(storage_path=Path(tmpdir))
            m1 = Memory(id="m1", content="Memory 1")
            m2 = Memory(id="m2", content="Memory 2")
            await ltm.store(m1)
            await ltm.store(m2)
            await ltm.associate(["m1", "m2"])
            m1_loaded = ltm.storage.get("m1")
            assert m1_loaded is not None
            assert "m2" in m1_loaded.associations
    @pytest.mark.asyncio
    async def test_summarize(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            ltm = LongTermMemory(storage_path=Path(tmpdir))
            m1 = Memory(id="m1", content="Memory 1", importance=0.8)
            m2 = Memory(id="m2", content="Memory 2", importance=0.6)
            await ltm.store(m1)
            await ltm.store(m2)
            summary = await ltm.summarize()
            assert summary.total_memories == 2
            assert 0.0 <= summary.avg_importance <= 1.0
class TestKnowledgeGraph:
    """Test knowledge graph."""
    @pytest.mark.asyncio
    async def test_add_nodes_and_edges(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            kg = KnowledgeGraph(storage_path=Path(tmpdir))
            n1 = KnowledgeNode(id="", type="concept", content="Python")
            n2 = KnowledgeNode(id="", type="concept", content="Programming")
            id1 = await kg.add_node(n1)
            id2 = await kg.add_node(n2)
            await kg.add_edge(id1, id2, "is_related")
            assert len(kg.nodes) == 2
```

```
            assert len(kg.edges) == 1
    def test_get_neighbors(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            kg = KnowledgeGraph(storage_path=Path(tmpdir))
            kg.nodes["n1"] = KnowledgeNode(id="n1", type="concept", content="Python")
            kg.nodes["n2"] = KnowledgeNode(id="n2", type="concept", content="Programming")
            kg.edges.append(KnowledgeEdge(id="e1", source="n1", target="n2", relation="related"
    ))
            neighbors = kg.get_neighbors("n1")
            assert len(neighbors) == 1
            assert neighbors[0].content == "Programming"
    @pytest.mark.asyncio
    async def test_visualize(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            kg = KnowledgeGraph(storage_path=Path(tmpdir))
            kg.nodes["n1"] = KnowledgeNode(id="n1", type="concept", content="Python")
            kg.nodes["n2"] = KnowledgeNode(id="n2", type="concept", content="Programming")
            kg.edges.append(KnowledgeEdge(id="e1", source="n1", target="n2", relation="related"
    ))
            viz = await kg.visualize()
            assert len(viz.nodes) == 2
            assert len(viz.edges) == 1
    def test_find_path(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            kg = KnowledgeGraph(storage_path=Path(tmpdir))
            kg.nodes["n1"] = KnowledgeNode(id="n1", type="concept", content="A")
            kg.nodes["n2"] = KnowledgeNode(id="n2", type="concept", content="B")
            kg.nodes["n3"] = KnowledgeNode(id="n3", type="concept", content="C")
            kg.edges.append(KnowledgeEdge(id="e1", source="n1", target="n2", relation="to"))
            kg.edges.append(KnowledgeEdge(id="e2", source="n2", target="n3", relation="to"))
            path = kg.find_path("n1", "n3")
            assert path is not None
            assert path == ["n1", "n2", "n3"]
class TestConsolidation:
    """Test memory consolidation."""
    @pytest.mark.asyncio
    async def test_consolidate_empty(self) -> None:
        with tempfile.TemporaryDirectory() as tmpdir:
            ltm = LongTermMemory(storage_path=Path(tmpdir))
            result = await ltm.consolidate()
            assert result.memories_removed == 0
            assert result.memories_merged == 0
```

File: [238/252] tests/test_middleware.py

```
"""Tests for middleware system."""
from __future__ import annotations
import pytest
from openlaoke.core.middleware import (
    ErrorEvent,
    Event,
    MiddlewareChain,
    MiddlewareContext,
    ProgressEvent,
    SyncMiddleware,
    ThreadDataMiddleware,
)
from openlaoke.core.state import AppState
class SimpleMiddleware(SyncMiddleware):
    """Simple test middleware."""
    def process(self, context: MiddlewareContext) -> list[Event]:
        return [ProgressEvent(message="Processing")]
class CountingMiddleware(SyncMiddleware):
    """Middleware that counts calls."""
    def __init__(self) -> None:
        self.call_count = 0
    def process(self, context: MiddlewareContext) -> list[Event]:
        self.call_count += 1
        context.set_metadata("count", self.call_count)
        return []
@pytest.fixture
def app_state() -> AppState:
    """Create a test AppState."""
    return AppState(session_id="test_session", cwd="/tmp")
@pytest.fixture
def context(app_state: AppState) -> MiddlewareContext:
    """Create a test MiddlewareContext."""
    return MiddlewareContext(state=app_state, session_id="test_session")
def test_middleware_context_metadata(context: MiddlewareContext) -> None:
    """Test MiddlewareContext metadata operations."""
    context.set_metadata("key", "value")
    assert context.get_metadata("key") == "value"
    assert context.get_metadata("missing", "default") == "default"
def test_middleware_context_abort(context: MiddlewareContext) -> None:
    """Test MiddlewareContext abort functionality."""
    assert not context.aborted
    context.abort("test_reason")
    assert context.aborted
    assert context.get_metadata("abort_reason") == "test_reason"
def test_middleware_chain_add() -> None:
    """Test adding middleware to chain."""
    chain = MiddlewareChain()
    assert len(chain) == 0
    chain.add(SimpleMiddleware())
    assert len(chain) == 1
    chain.add(CountingMiddleware())
    assert len(chain) == 2
def test_middleware_chain_remove() -> None:
    """Test removing middleware from chain."""
    chain = MiddlewareChain()
    chain.add(SimpleMiddleware())
    chain.add(CountingMiddleware())
    assert len(chain) == 2
    assert chain.remove("SimpleMiddleware")
    assert len(chain) == 1
    assert not chain.remove("NonExistent")
```

```
    assert len(chain) == 1
def test_middleware_chain_get() -> None:
    """Test getting middleware from chain."""
    chain = MiddlewareChain()
    simple = SimpleMiddleware()
    chain.add(simple)
    found = chain.get("SimpleMiddleware")
    assert found is simple
    assert chain.get("NonExistent") is None
def test_middleware_chain_contains() -> None:
    """Test checking middleware presence."""
    chain = MiddlewareChain()
    chain.add(SimpleMiddleware())
    assert "SimpleMiddleware" in chain
    assert "NonExistent" not in chain
def test_middleware_chain_clear() -> None:
    """Test clearing middleware chain."""
    chain = MiddlewareChain()
    chain.add(SimpleMiddleware())
    chain.add(CountingMiddleware())
    assert len(chain) == 2
    chain.clear()
    assert len(chain) == 0
def test_middleware_chain_insert() -> None:
    """Test inserting middleware at position."""
    chain = MiddlewareChain()
    chain.add(SimpleMiddleware())
    chain.insert(0, CountingMiddleware())
    assert chain.middlewares[0].name == "CountingMiddleware"
    assert chain.middlewares[1].name == "SimpleMiddleware"
def test_syncmiddleware_name() -> None:
    """Test middleware name property."""
    mw = SimpleMiddleware()
    assert mw.name == "SimpleMiddleware"
def test_thread_data_middleware(app_state: AppState) -> None:
    """Test ThreadDataMiddleware."""
    mw = ThreadDataMiddleware()
    ctx = MiddlewareContext(state=app_state, session_id="test_session")
    events = mw.process(ctx)
    assert len(events) == 0
    assert "thread_data" in ctx.metadata
@pytest.mark.asyncio
async def test_middleware_chain_process(context: MiddlewareContext) -> None:
    """Test processing through middleware chain."""
    chain = MiddlewareChain()
    chain.add(SimpleMiddleware())
    events = []
    async for event in chain.process(context):
        events.append(event)
    assert len(events) == 1
    assert events[0].type == "progress"
@pytest.mark.asyncio
async def test_middleware_chain_abort(context: MiddlewareContext) -> None:
    """Test middleware chain abort."""
    chain = MiddlewareChain()
    class AbortMiddleware(SyncMiddleware):
        def process(self, ctx: MiddlewareContext) -> list[Event]:
            ctx.abort("test")
            return []
    chain.add(AbortMiddleware())
    chain.add(SimpleMiddleware())
    events = []
    async for event in chain.process(context):
        events.append(event)
    assert len(events) == 0
```

```
def test_event_to_dict() -> None:
    """Test Event serialization."""
    event = ProgressEvent(message="test", percentage=50.0)
    data = event.to_dict()
    assert data["type"] == "progress"
    assert data["data"]["message"] == "test"
    assert data["data"]["percentage"] == 50.0
def test_error_event() -> None:
    """Test ErrorEvent creation."""
    event = ErrorEvent(error="test error", error_type="TestError")
    assert event.type == "error"
    assert event.data["error"] == "test error"
    assert event.data["error_type"] == "TestError"
```

File: [239/252] tests/test_model_assessment.py

```
"""Tests for model assessment and task decomposition."""
from openlaoke.core.model_assessment.assessor import ModelAssessor, TaskDecomposer
from openlaoke.core.model_assessment.types import (
    KNOWN_MODEL_TIERS,
    TIER_GRANULARITIES,
    CapabilityCategory,
    CapabilityScore,
    ModelBenchmark,
    ModelTier,
    TaskGranularity,
)
from openlaoke.types.providers import MultiProviderConfig
class TestModelTier:
    def test_tier_values(self):
        assert ModelTier.TIER_1_ADVANCED.value == "tier_1_advanced"
        assert ModelTier.TIER_5_LIMITED.value == "tier_5_limited"
    def test_tier_ordering(self):
        tiers = [
            ModelTier.TIER_1_ADVANCED,
            ModelTier.TIER_2_CAPABLE,
            ModelTier.TIER_3_MODERATE,
            ModelTier.TIER_4_BASIC,
            ModelTier.TIER_5_LIMITED,
        ]
        for i, tier in enumerate(tiers):
            if i > 0:
                assert tier.value > tiers[i - 1].value
    def test_known_model_tiers(self):
        assert "claude-opus-4" in KNOWN_MODEL_TIERS
        assert KNOWN_MODEL_TIERS["claude-opus-4"] == ModelTier.TIER_1_ADVANCED
        assert "gemma3:1b" in KNOWN_MODEL_TIERS
        assert KNOWN_MODEL_TIERS["gemma3:1b"] == ModelTier.TIER_5_LIMITED
        assert "gpt-4o" in KNOWN_MODEL_TIERS
        assert KNOWN_MODEL_TIERS["gpt-4o"] == ModelTier.TIER_1_ADVANCED
class TestTaskGranularity:
    def test_granularity_values(self):
        gran = TaskGranularity(
            max_subtasks=3,
            subtask_complexity_limit="atomic",
            verification_frequency="every_step",
            retry_limit=10,
            timeout_multiplier=2.0,
            tool_call_limit=5,
            requires_explicit_steps=True,
            min_confidence_threshold=0.9,
        )
        assert gran.max_subtasks == 3
        assert gran.requires_explicit_steps
        assert gran.verification_frequency == "every_step"
        assert gran.subtask_complexity_limit == "atomic"
        assert gran.tool_call_limit == 5
    def test_tier_granularities_mapping(self):
        assert ModelTier.TIER_1_ADVANCED in TIER_GRANULARITIES
        assert ModelTier.TIER_5_LIMITED in TIER_GRANULARITIES
        tier1_gran = TIER_GRANULARITIES[ModelTier.TIER_1_ADVANCED]
        tier5_gran = TIER_GRANULARITIES[ModelTier.TIER_5_LIMITED]
        assert tier1_gran.max_subtasks > tier5_gran.max_subtasks
        assert not tier1_gran.requires_explicit_steps
        assert tier5_gran.requires_explicit_steps
        assert tier1_gran.verification_frequency == "minimal"
        assert tier5_gran.verification_frequency == "every_step"
```

```
    def test_advanced_model_can_handle_complex_tasks(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_1_ADVANCED]
        assert gran.max_subtasks == 20
        assert not gran.requires_explicit_steps
        assert gran.subtask_complexity_limit == "high"
        assert gran.tool_call_limit == 50
    def test_limited_model_needs_explicit_steps(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_5_LIMITED]
        assert gran.max_subtasks == 4
        assert gran.requires_explicit_steps
        assert gran.verification_frequency == "every_step"
        assert gran.retry_limit == 8
        assert gran.subtask_complexity_limit == "atomic"
        assert gran.tool_call_limit == 10
class TestCapabilityScore:
    def test_capability_score_creation(self):
        score = CapabilityScore(
            category=CapabilityCategory.BASIC_CHAT,
            score=8.5,
            tests_passed=8,
            tests_total=10,
        )
        assert score.category == CapabilityCategory.BASIC_CHAT
        assert score.score == 8.5
        assert score.tests_passed == 8
        assert score.tests_total == 10
    def test_capability_categories(self):
        assert CapabilityCategory.BASIC_CHAT.value == "basic_chat"
        assert CapabilityCategory.TOOL_CALLING.value == "tool_calling"
        assert CapabilityCategory.CODE_GENERATION.value == "code_generation"
        assert CapabilityCategory.MULTI_TURN.value == "multi_turn"
class TestModelBenchmark:
    def test_benchmark_creation(self):
        caps = {
            "basic_chat": CapabilityScore(CapabilityCategory.BASIC_CHAT, 9.0, 9, 10),
            "tool_calling": CapabilityScore(CapabilityCategory.TOOL_CALLING, 8.5, 8, 10),
        }
        bench = ModelBenchmark(
            provider="openai",
            model="gpt-4o",
            overall_score=8.75,
            tier=ModelTier.TIER_1_ADVANCED,
            capabilities=caps,
            tested_at=1000.0,
            test_duration=5.0,
            recommendations=["Can handle complex tasks directly"],
        )
        assert bench.provider == "openai"
        assert bench.model == "gpt-4o"
        assert bench.overall_score == 8.75
        assert bench.tier == ModelTier.TIER_1_ADVANCED
    def test_benchmark_serialization(self):
        caps = {
            "basic_chat": CapabilityScore(CapabilityCategory.BASIC_CHAT, 9.0, 9, 10),
        }
        bench = ModelBenchmark(
            provider="openai",
            model="gpt-4o",
            overall_score=9.0,
            tier=ModelTier.TIER_1_ADVANCED,
            capabilities=caps,
            tested_at=1000.0,
            test_duration=5.0,
            recommendations=["Test"],
        )
```

```
        d = bench.to_dict()
        assert d["provider"] == "openai"
        assert d["model"] == "gpt-4o"
        assert d["overall_score"] == 9.0
        assert d["tier"] == "tier_1_advanced"
        restored = ModelBenchmark.from_dict(d)
        assert restored.provider == bench.provider
        assert restored.model == bench.model
        assert restored.overall_score == bench.overall_score
class TestModelAssessor:
    def test_get_tier_known_models(self):
        config = MultiProviderConfig.defaults()
        assessor = ModelAssessor(config)
        tier = assessor.get_tier("claude-opus-4")
        assert tier == ModelTier.TIER_1_ADVANCED
        tier = assessor.get_tier("gemma3:1b")
        assert tier == ModelTier.TIER_5_LIMITED
        tier = assessor.get_tier("gpt-4o")
        assert tier == ModelTier.TIER_1_ADVANCED
    def test_get_tier_unknown_model(self):
        config = MultiProviderConfig.defaults()
        assessor = ModelAssessor(config)
        tier = assessor.get_tier("unknown-model-v1")
        assert tier == ModelTier.TIER_3_MODERATE
    def test_get_granularity(self):
        config = MultiProviderConfig.defaults()
        assessor = ModelAssessor(config)
        gran = assessor.get_granularity("claude-opus-4")
        assert gran.max_subtasks == 20
        assert not gran.requires_explicit_steps
        assert gran.subtask_complexity_limit == "high"
        gran = assessor.get_granularity("gemma3:1b")
        assert gran.max_subtasks == 4
        assert gran.requires_explicit_steps
        assert gran.verification_frequency == "every_step"
        assert gran.subtask_complexity_limit == "atomic"
class TestTaskDecomposer:
    def test_decompose_simple_task(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_1_ADVANCED]
        decomposer = TaskDecomposer(gran)
        steps = decomposer.decompose("Write a simple hello world function")
        assert len(steps) == 1
        assert steps[0] == "Write a simple hello world function"
    def test_decompose_complex_task_advanced_model(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_1_ADVANCED]
        decomposer = TaskDecomposer(gran)
        complex_task = "Write an article about AI and create a diagram and implement code"
        steps = decomposer.decompose(complex_task)
        assert len(steps) == 1
        assert steps[0] == complex_task
    def test_decompose_complex_task_limited_model(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_5_LIMITED]
        decomposer = TaskDecomposer(gran)
        complex_task = "Write an article about AI and create a diagram and implement code"
        steps = decomposer.decompose(complex_task)
        assert len(steps) > 1
        assert len(steps) <= gran.max_subtasks
        assert len(steps) == 3
    def test_decompose_with_then_keyword(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_4_BASIC]
        decomposer = TaskDecomposer(gran)
        task = "First read the file then process it then write output"
        steps = decomposer.decompose(task)
        assert len(steps) > 1
        assert "First read the file" in steps
```

```
    assert "process it" in steps
    assert "write output" in steps
def test_decompose_respects_max_subtasks(self):
    gran = TaskGranularity(
        max_subtasks=2,
        subtask_complexity_limit="atomic",
        verification_frequency="every_step",
        retry_limit=5,
        timeout_multiplier=1.5,
        tool_call_limit=5,
        requires_explicit_steps=True,
        min_confidence_threshold=0.8,
    )
    decomposer = TaskDecomposer(gran)
    task = "Step 1 and Step 2 and Step 3 and Step 4"
    steps = decomposer.decompose(task)
    assert len(steps) <= 2
def test_should_verify_every_step(self):
    gran = TaskGranularity(
        max_subtasks=3,
        subtask_complexity_limit="atomic",
        verification_frequency="every_step",
        retry_limit=5,
        timeout_multiplier=1.5,
        tool_call_limit=5,
        requires_explicit_steps=True,
        min_confidence_threshold=0.8,
    )
    decomposer = TaskDecomposer(gran)
    assert decomposer.should_verify(0)
    assert decomposer.should_verify(1)
    assert decomposer.should_verify(2)
def test_should_verify_frequent(self):
    gran = TaskGranularity(
        max_subtasks=10,
        subtask_complexity_limit="low",
        verification_frequency="frequent",
        retry_limit=5,
        timeout_multiplier=1.5,
        tool_call_limit=15,
        requires_explicit_steps=True,
        min_confidence_threshold=0.8,
    )
    decomposer = TaskDecomposer(gran)
    assert decomposer.should_verify(0)
    assert not decomposer.should_verify(1)
    assert decomposer.should_verify(2)
    assert not decomposer.should_verify(3)
def test_should_verify_moderate(self):
    gran = TaskGranularity(
        max_subtasks=10,
        subtask_complexity_limit="medium",
        verification_frequency="moderate",
        retry_limit=5,
        timeout_multiplier=1.0,
        tool_call_limit=20,
        requires_explicit_steps=False,
        min_confidence_threshold=0.7,
    )
    decomposer = TaskDecomposer(gran)
    assert decomposer.should_verify(0)
    assert not decomposer.should_verify(1)
    assert not decomposer.should_verify(2)
    assert decomposer.should_verify(3)
def test_should_verify_minimal(self):
```

```
        gran = TaskGranularity(
            max_subtasks=20,
            subtask_complexity_limit="high",
            verification_frequency="minimal",
            retry_limit=3,
            timeout_multiplier=1.0,
            tool_call_limit=50,
            requires_explicit_steps=False,
            min_confidence_threshold=0.6,
        )
        decomposer = TaskDecomposer(gran)
        assert not decomposer.should_verify(0)
        assert not decomposer.should_verify(1)
        assert not decomposer.should_verify(10)
    def test_get_timeout(self):
        gran = TaskGranularity(
            max_subtasks=3,
            subtask_complexity_limit="atomic",
            verification_frequency="every_step",
            retry_limit=5,
            timeout_multiplier=2.0,
            tool_call_limit=5,
            requires_explicit_steps=True,
            min_confidence_threshold=0.9,
        )
        decomposer = TaskDecomposer(gran)
        timeout = decomposer.get_timeout(60.0)
        assert timeout == 120.0
class TestComplexTaskDecomposition:
    def test_full_pipeline_complex_task(self):
        config = MultiProviderConfig.defaults()
        assessor = ModelAssessor(config)
        model = "gemma3:1b"
        gran = assessor.get_granularity(model)
        decomposer = TaskDecomposer(gran)
        complex_task = "Write a comprehensive research paper with citations and create visualiz
    ations and implement reference code"
        steps = decomposer.decompose(complex_task)
        assert len(steps) > 1
        assert len(steps) <= gran.max_subtasks
        assert gran.requires_explicit_steps
        assert gran.verification_frequency == "every_step"
        for i in range(len(steps)):
            assert decomposer.should_verify(i)
        timeout = decomposer.get_timeout(120.0)
        assert timeout == gran.timeout_multiplier * 120.0
    def test_different_models_different_strategies(self):
        config = MultiProviderConfig.defaults()
        assessor = ModelAssessor(config)
        task = "Complex multi-step task with analysis and implementation and testing"
        tier1_gran = assessor.get_granularity("claude-opus-4")
        tier1_decomposer = TaskDecomposer(tier1_gran)
        tier1_steps = tier1_decomposer.decompose(task)
        assert len(tier1_steps) == 1
        tier5_gran = assessor.get_granularity("gemma3:1b")
        tier5_decomposer = TaskDecomposer(tier5_gran)
        tier5_steps = tier5_decomposer.decompose(task)
        assert len(tier5_steps) > 1
        assert len(tier5_steps) <= 4
    def test_edge_cases(self):
        gran = TIER_GRANULARITIES[ModelTier.TIER_5_LIMITED]
        decomposer = TaskDecomposer(gran)
        steps = decomposer.decompose("")
        assert len(steps) == 0
        steps = decomposer.decompose("Single simple task")
```

```
assert len(steps) <= 1
steps = decomposer.decompose("Task with many words but no connectors")
assert len(steps) == 1
```

File: [240/252] tests/test_model_optimizations.py

```
#!/usr/bin/env python
"""Test script for dual-model optimization features.
This script demonstrates:
1. Intelligent model selection based on available resources
2. CPU/GPU hybrid model loading
3. Batch operations for better performance
4. Model status monitoring
"""
import asyncio
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).parent.parent))
async def test_intelligent_selector():
    """Test intelligent model selection."""
    from openlaoke.core.intelligent_model_selector import IntelligentModelSelector
    print("=" * 60)
    print("TEST 1: Intelligent Model Selector")
    print("=" * 60)
    selector = IntelligentModelSelector()
    vram = await selector.detect_available_vram()
    print(f"\n[OK] Detected VRAM: {vram:.1f} GB")
    report = await selector.get_recommendation_report()
    print(f"\n[OK] Recommended Combination: {report['recommended_combination']['name'].upper()}
    ")
    print(f"  Planner:   {report['recommended_combination']['planner']}")
    print(f"  Executor:  {report['recommended_combination']['executor']}")
    print(f"  Validator: {report['recommended_combination']['validator']}")
    print(f"\n[OK] Estimated:")
    print(f"  VRAM Usage: {report['estimated']['vram_usage_gb']:.1f} GB")
    print(f"  Quality:    {report['estimated']['quality_score']:.1f}/10")
    print(f"  Cost:       {report['estimated']['cost_factor']:.1f}x")
    print(f"\n[OK] Suitable Combinations: {len(report['all_suitable_combinations'])}")
    for combo in report["all_suitable_combinations"][:3]:
        print(f"  - {combo['name']}: {combo['vram_gb']:.1f} GB, quality {combo['quality']:.1f}"
    )
    return True
async def test_hybrid_manager():
    """Test CPU/GPU hybrid model manager."""
    from openlaoke.core.hybrid_model_manager import create_hybrid_manager
    print("\n" + "=" * 60)
    print("TEST 2: CPU/GPU Hybrid Model Manager")
    print("=" * 60)
    manager = create_hybrid_manager()
    print("\n[OK] Initializing model pool...")
    result = await manager.initialize()
    print(f"  Success: {result['success']}")
    print(f"  Strategy: {result['strategy']}")
    print("\n[OK] Getting model status...")
    status = await manager.get_status()
    print(f"\n  CPU Models: {len(status.cpu_models)}")
    for model in status.cpu_models:
        loaded = "[OK]" if model.is_loaded else "[X]"
        print(f"    {loaded} {model.name} ({model.tier})")
    print(f"\n  GPU Models: {len(status.gpu_models)}")
    for model in status.gpu_models:
        loaded = "[OK]" if model.is_loaded else "[X]"
        print(f"    {loaded} {model.name} ({model.tier}, {model.size_gb:.1f} GB)")
    print(f"\n  GPU Memory: {status.gpu_used_gb:.1f} / {status.gpu_total_gb:.1f} GB")
    return True
async def test_batch_operations():
    """Test batch operations."""
```

```
    from openlaoke.core.hybrid_model_manager import create_hybrid_manager
    print("\n" + "=" * 60)
    print("TEST 3: Batch Operations (Simulated)")
    print("=" * 60)
    manager = create_hybrid_manager()
    await manager.initialize()
    print("\n[OK] Simulating batch planner calls...")
    tasks = [
        "Task 1: Analyze requirements",
        "Task 2: Decompose into subtasks",
        "Task 3: Plan validation strategy",
    ]
    print(f"  Tasks to process: {len(tasks)}")
    print("  Batch processing (single model load)...")
    start = asyncio.get_event_loop().time()
    for i, task in enumerate(tasks, 1):
        print(f"    {i}. {task}")
    elapsed = asyncio.get_event_loop().time() - start
    print(f"\n  [OK] Batch completed in {elapsed:.2f}s")
    print("  (vs ~3x time for individual calls with model switching)")
    return True
async def test_dual_model_agent():
    """Test dual-model agent with optimizations."""
    from openlaoke.core.dual_model_agent import create_dual_model_agent
    from openlaoke.core.state import AppState
    print("\n" + "=" * 60)
    print("TEST 4: Dual-Model Agent with Optimizations")
    print("=" * 60)
    app_state = AppState()
    print("\n[OK] Creating dual-model agent...")
    agent = create_dual_model_agent(app_state)
    print("[OK] Selecting optimal models...")
    models = await agent._select_optimal_models()
    print(f"  Planner:   {models['planner']} ({models['planner_device']})")
    print(f"  Executor:  {models['executor']} ({models['executor_device']})")
    print(f"  Validator: {models['validator']} ({models['validator_device']})")
    print("\n[OK] Model selection complete")
    print("  (Models will be loaded on first execution)")
    return True
async def main():
    """Run all tests."""
    print("\n" + "=" * 60)
    print("OpenLaoKe Model Optimization Test Suite")
    print("=" * 60)
    tests = [
        ("Intelligent Selector", test_intelligent_selector),
        ("Hybrid Manager", test_hybrid_manager),
        ("Batch Operations", test_batch_operations),
        ("Dual-Model Agent", test_dual_model_agent),
    ]
    results = []
    for name, test_func in tests:
        try:
            success = await test_func()
            results.append((name, success, None))
        except Exception as e:
            results.append((name, False, str(e)))
    print("\n" + "=" * 60)
    print("Test Results")
    print("=" * 60)
    for name, success, error in results:
        status = "[OK] PASS" if success else "[X] FAIL"
        print(f"{status}: {name}")
        if error:
            print(f"  Error: {error}")
```

```
    passed = sum(1 for _, success, _ in results if success)
    total = len(results)
    print(f"\n{passed}/{total} tests passed")
    print("\n" + "=" * 60)
    print("Next Steps")
    print("=" * 60)
    print("\nTo use these optimizations:")
    print("  1. pip install -e .")
    print("  2. openlaoke")
    print("  3. /preload-models")
    print("  4. /model-status")
    print("  5. /model-recommend quality")
    print("  6. /dual write a Python program to calculate disk storage")
if __name__ == "__main__":
    asyncio.run(main())
```

File: [241/252] tests/test_multi_agent.py

```
"""Basic tests for multi-agent system."""
from __future__ import annotations
import pytest
from openlaoke.core.multi_agent import (
    AgentCoordinator,
    AgentMailbox,
    AgentPool,
    AgentRole,
    AgentState,
    AgentType,
    Assignment,
    Conflict,
    ConflictResolution,
    ConflictResolver,
    GlobalState,
    Message,
    MessagePriority,
    MessageType,
    Priority,
    ResolutionStrategy,
    Team,
    TeamMember,
    TeamResult,
    TaskDistributor,
)
def test_agent_types():
    assert AgentType.COORDINATOR == "coordinator"
    assert AgentType.EXPLORER == "explorer"
    assert AgentType.IMPLEMENTER == "implementer"
    assert AgentType.REVIEWER == "reviewer"
    assert AgentType.TESTER == "tester"
    assert AgentType.RESEARCHER == "researcher"
    assert AgentType.ANALYZER == "analyzer"
def test_agent_pool_creation():
    pool = AgentPool(max_agents=3)
    assert pool.max_agents == 3
    assert pool.active_count == 0
    assert pool.total_count == 0
def test_agent_mailbox_creation():
    mailbox = AgentMailbox()
    assert mailbox.max_queue_size == 1000
    stats = mailbox.get_stats()
    assert stats["queue_count"] == 0
    assert stats["total_messages"] == 0
def test_message_creation():
    message = Message(
        sender="agent_1",
        content="Test message",
        recipients=["agent_2"],
    )
    assert message.sender == "agent_1"
    assert message.content == "Test message"
    assert message.message_type == MessageType.TASK
    assert message.priority == MessagePriority.NORMAL
def test_priority_comparison():
    assert Priority.LOW < Priority.MEDIUM
    assert Priority.MEDIUM < Priority.HIGH
    assert Priority.HIGH < Priority.URGENT
    assert Priority.URGENT > Priority.HIGH
def test_task_distributor_creation():
    distributor = TaskDistributor()
```

```
    assert distributor.config.max_retries == 3
    stats = distributor.get_stats()
    assert stats["total_assignments"] == 0
def test_conflict_resolver_creation():
    resolver = ConflictResolver()
    assert resolver.default_strategy == ResolutionStrategy.COORDINATOR_DECIDES
    stats = resolver.get_stats()
    assert stats["total_conflicts"] == 0
    assert stats["pending_conflicts"] == 0
def test_conflict_creation():
    conflict = Conflict(
        agents_involved=["agent_1", "agent_2"],
        description="Resource conflict",
    )
    assert len(conflict.agents_involved) == 2
    assert conflict.description == "Resource conflict"
def test_team_creation():
    team = Team(name="TestTeam")
    assert team.name == "TestTeam"
    assert team.status == "forming"
    assert len(team.members) == 0
def test_team_member_creation():
    member = TeamMember(
        agent_id="agent_1",
        role=AgentRole.LEADER,
    )
    assert member.agent_id == "agent_1"
    assert member.role == AgentRole.LEADER
def test_team_add_member():
    team = Team(name="TestTeam")
    success = team.add_member("agent_1", AgentRole.LEADER)
    assert success is True
    assert len(team.members) == 1
    assert team.get_leader().agent_id == "agent_1"
def test_agent_state_creation():
    state = AgentState(
        agent_id="agent_123",
        agent_type=AgentType.IMPLEMENTER,
    )
    assert state.agent_id == "agent_123"
    assert state.agent_type == AgentType.IMPLEMENTER
    assert state.status == "idle"
@pytest.mark.asyncio
async def test_agent_coordinator_creation():
    coordinator = AgentCoordinator(max_agents=3)
    assert coordinator.max_agents == 3
    assert coordinator.agent_pool.max_agents == 3
    assert coordinator.mailbox is not None
@pytest.mark.asyncio
async def test_agent_coordinator_broadcast():
    coordinator = AgentCoordinator(max_agents=3)
    message = Message(
        sender="coordinator",
        content="Broadcast test",
        message_type=MessageType.BROADCAST,
    )
    await coordinator.broadcast(message)
    stats = coordinator.mailbox.get_stats()
    assert stats["total_messages"] == 0
@pytest.mark.asyncio
async def test_agent_coordinator_sync_state():
    coordinator = AgentCoordinator(max_agents=3)
    state = await coordinator.sync_state()
    assert isinstance(state, GlobalState)
    assert state.active_agents == 0
```

```
    assert state.total_tasks == 0
@pytest.mark.asyncio
async def test_agent_pool_acquire_release():
    pool = AgentPool(max_agents=3)
    agent_id = await pool.acquire(AgentType.EXPLORER)
    assert agent_id.startswith("agent_")
    assert pool.active_count == 1
    await pool.release(agent_id)
    assert pool.active_count == 0
@pytest.mark.asyncio
async def test_agent_mailbox_send_receive():
    mailbox = AgentMailbox()
    message = Message(
        sender="agent_1",
        content="Test",
        recipients=["agent_2"],
    )
    await mailbox.send(message)
    received = await mailbox.receive("agent_2", timeout=1.0)
    assert received is not None
    assert received.sender == "agent_1"
    assert received.content == "Test"
```

File: [242/252] tests/test_new_tools.py

```
"""Tests for new tool implementations."""
import os
import tempfile
from unittest.mock import AsyncMock, MagicMock, patch
import pytest
from openlaoke.core.state import create_app_state
from openlaoke.core.tool import ToolContext
from openlaoke.tools.webfetch_tool import WebFetchTool
from openlaoke.tools.websearch_tool import WebSearchTool
from openlaoke.tools.todo_tool import TodoWriteTool
from openlaoke.tools.question_tool import QuestionTool
from openlaoke.tools.ls_tool import ListDirectoryTool
from openlaoke.tools.lsp_tool import LSPTool
from openlaoke.tools.git_tool import GitTool
from openlaoke.tools.apply_patch_tool import ApplyPatchTool
from openlaoke.tools.batch_tool import BatchTool, ToolCallSpec
from openlaoke.tools.plan_tool import PlanTool
@pytest.fixture
def app_state():
    import openlaoke
    project_root = os.path.dirname(os.path.dirname(openlaoke.__file__))
    return create_app_state(cwd=project_root)
@pytest.fixture
def ctx(app_state):
    return ToolContext(app_state=app_state, tool_use_id="test_1")
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
class TestWebFetchTool:
    @pytest.mark.asyncio
    async def test_empty_url(self, ctx):
        tool = WebFetchTool()
        result = await tool.call(ctx, url="")
        assert result.is_error
        assert "URL is required" in result.content
    @pytest.mark.asyncio
    async def test_missing_url(self, ctx):
        tool = WebFetchTool()
        result = await tool.call(ctx)
        assert result.is_error
        assert "URL is required" in result.content
    @pytest.mark.asyncio
    async def test_url_auto_https(self, ctx):
        tool = WebFetchTool()
        with patch("httpx.AsyncClient") as mock_client:
            mock_response = MagicMock()
            mock_response.text = "<html><body>test</body></html>"
            mock_response.raise_for_status = MagicMock()
            mock_client_instance = AsyncMock()
            mock_client_instance.get = AsyncMock(return_value=mock_response)
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            result = await tool.call(ctx, url="example.com")
            assert not result.is_error
            assert "test" in result.content
    @pytest.mark.asyncio
    async def test_timeout_limit(self, ctx):
        tool = WebFetchTool()
        result = await tool.call(ctx, url="https://example.com", timeout=200)
```

```
        assert not result.is_error or "Error" in result.content
    @pytest.mark.asyncio
    async def test_html_to_markdown(self, ctx):
        tool = WebFetchTool()
        html = "<h1>Title</h1><p>Paragraph</p><pre>code</pre>"
        result = tool._html_to_markdown(html)
        assert "# " in result
        assert "Title" in result
        assert "``" in result
    def test_tool_properties(self):
        tool = WebFetchTool()
        assert tool.name == "WebFetch"
        assert tool.is_read_only
        assert not tool.is_destructive
        assert not tool.requires_approval
class TestWebSearchTool:
    @pytest.mark.asyncio
    async def test_empty_query(self, ctx):
        tool = WebSearchTool()
        result = await tool.call(ctx, query="")
        assert result.is_error
        assert "query is required" in result.content
    @pytest.mark.asyncio
    async def test_missing_query(self, ctx):
        tool = WebSearchTool()
        result = await tool.call(ctx)
        assert result.is_error
    @pytest.mark.asyncio
    async def test_no_results(self, ctx):
        tool = WebSearchTool()
        with patch("httpx.AsyncClient") as mock_client:
            mock_response = MagicMock()
            mock_response.text = "<html>No results</html>"
            mock_response.raise_for_status = MagicMock()
            mock_client_instance = AsyncMock()
            mock_client_instance.post = AsyncMock(return_value=mock_response)
            mock_client_instance.__aenter__ = AsyncMock(return_value=mock_client_instance)
            mock_client_instance.__aexit__ = AsyncMock()
            mock_client.return_value = mock_client_instance
            result = await tool.call(ctx, query="nonexistent query xyz")
            assert not result.is_error
            assert "No results" in result.content
    def test_strip_tags(self):
        tool = WebSearchTool()
        text = "<span>hello</span> world <b>test</b>"
        result = tool._strip_tags(text)
        assert "hello" in result
        assert "world" in result
        assert "test" in result
        assert "<" not in result
    def test_tool_properties(self):
        tool = WebSearchTool()
        assert tool.name == "WebSearch"
        assert tool.is_read_only
        assert not tool.is_destructive
class TestTodoWriteTool:
    @pytest.mark.asyncio
    async def test_empty_todos(self, ctx):
        tool = TodoWriteTool()
        result = await tool.call(ctx, todos=[])
        assert not result.is_error
        assert "empty" in result.content.lower()
    @pytest.mark.asyncio
    async def test_single_todo(self, ctx):
        tool = TodoWriteTool()
```

```
        result = await tool.call(ctx, todos=[{"content": "Test task"}])
        assert not result.is_error
        assert "Test task" in result.content
    @pytest.mark.asyncio
    async def test_multiple_todos(self, ctx):
        tool = TodoWriteTool()
        result = await tool.call(
            ctx,
            todos=[
                {"content": "Task 1", "status": "completed"},
                {"content": "Task 2", "status": "in_progress"},
                {"content": "Task 3", "status": "pending", "priority": "high"},
            ],
        )
        assert not result.is_error
        assert "Task 1" in result.content
        assert "Task 2" in result.content
        assert "Task 3" in result.content
    @pytest.mark.asyncio
    async def test_invalid_status(self, ctx):
        tool = TodoWriteTool()
        result = await tool.call(ctx, todos=[{"content": "Task", "status": "invalid_status"}])
        assert not result.is_error
    @pytest.mark.asyncio
    async def test_invalid_priority(self, ctx):
        tool = TodoWriteTool()
        result = await tool.call(ctx, todos=[{"content": "Task", "priority": "invalid"}])
        assert not result.is_error
    @pytest.mark.asyncio
    async def test_invalid_todos_type(self, ctx):
        tool = TodoWriteTool()
        result = await tool.call(ctx, todos="not a list")
        assert result.is_error
    def test_tool_properties(self):
        tool = TodoWriteTool()
        assert tool.name == "TodoWrite"
        assert not tool.is_read_only
        assert not tool.is_destructive
class TestQuestionTool:
    @pytest.mark.asyncio
    async def test_single_question(self, ctx):
        tool = QuestionTool()
        result = await tool.call(ctx, questions=[{"question": "What is your name?"}])
        assert not result.is_error
        assert "What is your name?" in result.content
    @pytest.mark.asyncio
    async def test_question_with_options(self, ctx):
        tool = QuestionTool()
        result = await tool.call(
            ctx,
            questions=[
                {
                    "question": "Choose an option",
                    "options": [{"value": "a", "label": "Option A"}, {"value": "b"}],
                }
            ],
        )
        assert not result.is_error
        assert "Option A" in result.content
        assert "Option B" in result.content or "b" in result.content
    @pytest.mark.asyncio
    async def test_multiple_questions(self, ctx):
        tool = QuestionTool()
        result = await tool.call(
            ctx,
```

```
            questions=[
                {"question": "Q1?", "header": "Header 1"},
                {"question": "Q2?", "header": "Header 2"},
            ],
        )
        assert not result.is_error
        assert "Q1?" in result.content
        assert "Q2?" in result.content
        assert "Header 1" in result.content
    @pytest.mark.asyncio
    async def test_multiple_selection(self, ctx):
        tool = QuestionTool()
        result = await tool.call(
            ctx,
            questions=[{"question": "Select all", "multiple": True, "options": [{"value": "x"}]
    }],
        )
        assert not result.is_error
        assert "select all that apply" in result.content.lower()
    @pytest.mark.asyncio
    async def test_invalid_questions_type(self, ctx):
        tool = QuestionTool()
        result = await tool.call(ctx, questions="not a list")
        assert result.is_error
    def test_tool_properties(self):
        tool = QuestionTool()
        assert tool.name == "Question"
        assert tool.is_read_only
class TestListDirectoryTool:
    @pytest.mark.asyncio
    async def test_list_current_directory(self, ctx):
        tool = ListDirectoryTool()
        result = await tool.call(ctx)
        assert not result.is_error
        assert "Contents of" in result.content
    @pytest.mark.asyncio
    async def test_list_temp_directory(self, ctx, temp_dir):
        with open(os.path.join(temp_dir, "file.txt"), "w") as f:
            f.write("test")
        os.makedirs(os.path.join(temp_dir, "subdir"))
        tool = ListDirectoryTool()
        result = await tool.call(ctx, path=temp_dir)
        assert not result.is_error
        assert "file.txt" in result.content
        assert "subdir" in result.content
    @pytest.mark.asyncio
    async def test_nonexistent_path(self, ctx):
        tool = ListDirectoryTool()
        result = await tool.call(ctx, path="/nonexistent/path")
        assert result.is_error
        assert "does not exist" in result.content
    @pytest.mark.asyncio
    async def test_path_is_file(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "file.txt")
        with open(path, "w") as f:
            f.write("test")
        tool = ListDirectoryTool()
        result = await tool.call(ctx, path=path)
        assert result.is_error
        assert "Not a directory" in result.content
    def test_format_size(self):
        tool = ListDirectoryTool()
        assert tool._format_size(0) == "0.0B"
        assert tool._format_size(1024) == "1.0KB"
        assert tool._format_size(1024 * 1024) == "1.0MB"
```

```
        assert tool._format_size(1024 * 1024 * 1024) == "1.0GB"
    def test_tool_properties(self):
        tool = ListDirectoryTool()
        assert tool.name == "ListDirectory"
        assert tool.is_read_only
class TestLSPTool:
    @pytest.mark.asyncio
    async def test_missing_action(self, ctx):
        tool = LSPTool()
        result = await tool.call(ctx, file_path="test.py")
        assert result.is_error
        assert "action is required" in result.content
    @pytest.mark.asyncio
    async def test_missing_file_path(self, ctx):
        tool = LSPTool()
        result = await tool.call(ctx, action="diagnostics")
        assert result.is_error
        assert "file_path is required" in result.content
    @pytest.mark.asyncio
    async def test_nonexistent_file(self, ctx):
        tool = LSPTool()
        result = await tool.call(ctx, action="diagnostics", file_path="/nonexistent/file.py")
        assert result.is_error
        assert "not found" in result.content.lower()
    @pytest.mark.asyncio
    async def test_unsupported_file_type(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.xyz")
        with open(path, "w") as f:
            f.write("content")
        tool = LSPTool()
        result = await tool.call(ctx, action="diagnostics", file_path=path)
        assert result.is_error
        assert "Unsupported" in result.content
    @pytest.mark.asyncio
    async def test_diagnostics_python(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.py")
        with open(path, "w") as f:
            f.write("def hello():\n    pass\n")
        tool = LSPTool()
        result = await tool.call(ctx, action="diagnostics", file_path=path)
        assert not result.is_error
    @pytest.mark.asyncio
    async def test_symbols_python(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.py")
        with open(path, "w") as f:
            f.write("def func1():\n    pass\n\nclass MyClass:\n    pass\n")
        tool = LSPTool()
        result = await tool.call(ctx, action="symbols", file_path=path)
        assert not result.is_error
        assert "func1" in result.content
        assert "MyClass" in result.content
    @pytest.mark.asyncio
    async def test_definition_missing_position(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.py")
        with open(path, "w") as f:
            f.write("test\n")
        tool = LSPTool()
        result = await tool.call(ctx, action="definition", file_path=path)
        assert result.is_error
        assert "line and column required" in result.content
    @pytest.mark.asyncio
    async def test_definition_with_position(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.py")
        with open(path, "w") as f:
            f.write("test\n")
```

```
        tool = LSPTool()
        result = await tool.call(ctx, action="definition", file_path=path, line=1, column=1)
        assert not result.is_error
    def test_detect_language(self):
        tool = LSPTool()
        assert tool._detect_language("test.py") == "python"
        assert tool._detect_language("test.js") == "javascript"
        assert tool._detect_language("test.ts") == "typescript"
        assert tool._detect_language("test.go") == "go"
        assert tool._detect_language("test.rs") == "rust"
        assert tool._detect_language("test.xyz") is None
    def test_tool_properties(self):
        tool = LSPTool()
        assert tool.name == "LSP"
        assert tool.is_read_only
class TestGitTool:
    @pytest.mark.asyncio
    async def test_missing_action(self, ctx):
        tool = GitTool()
        result = await tool.call(ctx)
        assert result.is_error
        assert "action is required" in result.content
    @pytest.mark.asyncio
    async def test_invalid_action(self, ctx):
        tool = GitTool()
        result = await tool.call(ctx, action="invalid_action")
        assert result.is_error
        assert "Unknown" in result.content
    @pytest.mark.asyncio
    async def test_not_git_repo(self, ctx, temp_dir):
        ctx.app_state.set_cwd(temp_dir)
        tool = GitTool()
        result = await tool.call(ctx, action="status")
        assert result.is_error
        assert "Not a git repository" in result.content
    def test_is_git_repo(self, temp_dir):
        tool = GitTool()
        assert not tool._is_git_repo(temp_dir)
        git_dir = os.path.join(temp_dir, ".git")
        os.makedirs(git_dir)
        assert tool._is_git_repo(temp_dir)
    def test_tool_properties(self):
        tool = GitTool()
        assert tool.name == "Git"
        assert not tool.is_read_only
        assert tool.is_destructive
        assert tool.requires_approval
class TestApplyPatchTool:
    @pytest.mark.asyncio
    async def test_empty_patch(self, ctx):
        tool = ApplyPatchTool()
        result = await tool.call(ctx, patch_content="")
        assert result.is_error
        assert "required" in result.content.lower()
    @pytest.mark.asyncio
    async def test_invalid_patch(self, ctx):
        tool = ApplyPatchTool()
        result = await tool.call(ctx, patch_content="not a valid patch")
        assert result.is_error
    @pytest.mark.asyncio
    async def test_patch_nonexistent_file(self, ctx):
        patch = """--- a/test.txt
+++ b/test.txt
@@ -1,1 +1,1 @@
-old
```

```
+new
"""
        tool = ApplyPatchTool()
        result = await tool.call(ctx, patch_content=patch)
        assert not result.is_error
        assert "not found" in result.content.lower() or "Skipped" in result.content
    @pytest.mark.asyncio
    async def test_valid_patch(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.txt")
        with open(path, "w") as f:
            f.write("old content\n")
        patch = f"""--- a/test.txt
+++ b/test.txt
@@ -1,1 +1,1 @@
-old content
+new content
"""
        tool = ApplyPatchTool()
        result = await tool.call(ctx, patch_content=patch)
        assert not result.is_error
    def test_parse_patch(self):
        tool = ApplyPatchTool()
        patch = """--- a/file.py
+++ b/file.py
@@ -1,3 +1,3 @@
 line1
-old
+new
 line3
"""
        patches = tool._parse_patch(patch)
        assert len(patches) == 1
        assert patches[0]["file"] == "file.py"
        assert len(patches[0]["hunks"]) == 1
    def test_tool_properties(self):
        tool = ApplyPatchTool()
        assert tool.name == "ApplyPatch"
        assert not tool.is_read_only
        assert not tool.is_destructive
        assert tool.requires_approval
class TestBatchTool:
    @pytest.mark.asyncio
    async def test_empty_calls(self, ctx):
        tool = BatchTool()
        result = await tool.call(ctx, calls=[])
        assert result.is_error
        assert "calls list is required" in result.content
    @pytest.mark.asyncio
    async def test_too_many_calls(self, ctx):
        tool = BatchTool()
        calls = [ToolCallSpec(tool_name="Bash", args={"command": "echo"}) for _ in range(51)]
        result = await tool.call(ctx, calls=calls)
        assert result.is_error
        assert "Too many" in result.content
    @pytest.mark.asyncio
    async def test_unknown_tool(self, ctx):
        tool = BatchTool()
        result = await tool.call(ctx, calls=[ToolCallSpec(tool_name="UnknownTool", args={})])
        assert "Unknown tool" in result.content or "ERROR" in result.content
    def test_tool_properties(self):
        tool = BatchTool()
        assert tool.name == "Batch"
        assert not tool.is_read_only
class TestPlanTool:
    @pytest.mark.asyncio
```

```
    async def test_missing_action(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx)
        assert result.is_error
        assert "action is required" in result.content
    @pytest.mark.asyncio
    async def test_invalid_action(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx, action="invalid")
        assert result.is_error
        assert "Unknown action" in result.content
    @pytest.mark.asyncio
    async def test_enter_plan(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx, action="enter", plan_content="Test plan")
        assert not result.is_error
        assert "Entered plan mode" in result.content
    @pytest.mark.asyncio
    async def test_enter_missing_content(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx, action="enter")
        assert result.is_error
        assert "plan_content is required" in result.content
    @pytest.mark.asyncio
    async def test_status_not_in_plan(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx, action="status")
        assert not result.is_error
        assert "Not in plan mode" in result.content
    @pytest.mark.asyncio
    async def test_exit_not_in_plan(self, ctx):
        tool = PlanTool()
        result = await tool.call(ctx, action="exit")
        assert not result.is_error
        assert "Not in plan mode" in result.content
    @pytest.mark.asyncio
    async def test_update_after_enter(self, ctx):
        tool = PlanTool()
        await tool.call(ctx, action="enter", plan_content="Initial plan")
        result = await tool.call(ctx, action="update", plan_content="Updated plan")
        assert not result.is_error
        assert "Plan updated" in result.content
    @pytest.mark.asyncio
    async def test_status_after_enter(self, ctx):
        tool = PlanTool()
        await tool.call(ctx, action="enter", plan_content="1. Step one\n2. Step two")
        result = await tool.call(ctx, action="status")
        assert not result.is_error
        assert "Step one" in result.content
        assert "Step two" in result.content
    @pytest.mark.asyncio
    async def test_append_to_plan(self, ctx):
        tool = PlanTool()
        await tool.call(ctx, action="enter", plan_content="Initial")
        result = await tool.call(ctx, action="append", plan_content="Added")
        assert not result.is_error
        assert "Appended" in result.content
    @pytest.mark.asyncio
    async def test_prepend_to_plan(self, ctx):
        tool = PlanTool()
        await tool.call(ctx, action="enter", plan_content="Initial")
        result = await tool.call(ctx, action="prepend", plan_content="Prepended")
        assert not result.is_error
        assert "Prepended" in result.content
    def test_parse_steps(self):
```

```
    tool = PlanTool()
    content = "1. First step\n2. Second step\n- Bullet item\n* Another bullet"
    steps = tool._parse_steps(content)
    assert "First step" in steps
    assert "Second step" in steps
    assert "Bullet item" in steps
    assert "Another bullet" in steps
def test_tool_properties(self):
    tool = PlanTool()
    assert tool.name == "Plan"
    assert not tool.is_read_only
```

File: [243/252] tests/test_project_initializer.py

```
"""Tests for ProjectInitializer."""
from __future__ import annotations
import tempfile
from pathlib import Path
from openlaoke.core.hyperauto import (
    InitResult,
    ProjectAnalysis,
    ProjectInitializer,
    ProjectType,
)
class TestProjectTypeDetection:
    def test_detect_python_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "pyproject.toml").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.PYTHON
    def test_detect_nodejs_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "package.json").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.NODE_JS
    def test_detect_go_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "go.mod").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.GO
    def test_detect_rust_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "Cargo.toml").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.RUST
    def test_detect_java_maven_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "pom.xml").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.JAVA_MAVEN
    def test_detect_java_gradle_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "build.gradle").touch()
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.JAVA_GRADLE
    def test_detect_generic_project(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.detect_project_type(tmp_path)
            assert result == ProjectType.GENERIC
    def test_detect_none_for_invalid_path(self):
        initializer = ProjectInitializer()
```

```
        result = initializer.detect_project_type(Path("/nonexistent/path"))
        assert result is None
class TestProjectAnalysis:
    def test_analyze_empty_directory(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            assert analysis.project_type == ProjectType.GENERIC
            assert not analysis.has_git
            assert not analysis.has_readme
            assert not analysis.has_agents_md
            assert not analysis.has_tests
            assert not analysis.has_ci
    def test_analyze_python_project_with_tests(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "pyproject.toml").write_text("[project]\nname = 'test'")
            (tmp_path / "tests").mkdir()
            (tmp_path / "tests" / "__init__.py").touch()
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            assert analysis.project_type == ProjectType.PYTHON
            assert analysis.has_tests
    def test_analyze_existing_files(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "README.md").touch()
            (tmp_path / "AGENTS.md").touch()
            (tmp_path / ".gitignore").touch()
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            assert analysis.has_readme
            assert analysis.has_agents_md
class TestConfigGeneration:
    def test_generate_gitignore_for_python(self):
        initializer = ProjectInitializer()
        content = initializer._generate_gitignore(ProjectType.PYTHON)
        assert "__pycache__" in content
        assert "*.py[cod]" in content
        assert ".pytest_cache" in content
        assert ".DS_Store" in content
    def test_generate_gitignore_for_nodejs(self):
        initializer = ProjectInitializer()
        content = initializer._generate_gitignore(ProjectType.NODE_JS)
        assert "node_modules" in content
        assert "npm-debug.log" in content
        assert ".DS_Store" in content
    def test_generate_gitignore_for_go(self):
        initializer = ProjectInitializer()
        content = initializer._generate_gitignore(ProjectType.GO)
        assert "*.exe" in content
        assert "go.work" in content
        assert ".DS_Store" in content
    def test_generate_gitignore_for_rust(self):
        initializer = ProjectInitializer()
        content = initializer._generate_gitignore(ProjectType.RUST)
        assert "target/" in content
        assert "Cargo.lock" in content
        assert ".DS_Store" in content
    def test_generate_gitignore_for_java(self):
        initializer = ProjectInitializer()
        content = initializer._generate_gitignore(ProjectType.JAVA_MAVEN)
        assert "*.class" in content
        assert "target/" in content
```

```
        assert ".DS_Store" in content
    def test_generate_readme_contains_project_name(self):
        with tempfile.TemporaryDirectory(suffix="test_project") as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            content = initializer._generate_readme(analysis)
            assert tmp_path.name in content
            assert "Installation" in content
            assert "Usage" in content
            assert "Testing" in content
    def test_generate_agents_md(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            content = initializer._generate_agents_md(analysis)
            assert "# AGENTS.md" in content
            assert "Commands" in content
            assert "Setup" in content
    def test_generate_pyproject_toml(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            (tmp_path / "pyproject.toml").touch()
            initializer = ProjectInitializer()
            analysis = initializer.analyze_project(tmp_path)
            content = initializer._generate_pyproject(analysis)
            assert "[build-system]" in content
            assert "[project]" in content
            assert "requires-python" in content
            assert "[tool.ruff]" in content
            assert "[tool.mypy]" in content
class TestProjectStructure:
    def test_create_python_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.create_project_structure(tmp_path, ProjectType.PYTHON)
            assert result
            assert (tmp_path / "src").exists()
            assert (tmp_path / "tests").exists()
            assert (tmp_path / "tests" / "__init__.py").exists()
            assert (tmp_path / "docs").exists()
    def test_create_nodejs_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.create_project_structure(tmp_path, ProjectType.NODE_JS)
            assert result
            assert (tmp_path / "src").exists()
            assert (tmp_path / "tests").exists()
            assert (tmp_path / "lib").exists()
    def test_create_go_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.create_project_structure(tmp_path, ProjectType.GO)
            assert result
            assert (tmp_path / "cmd").exists()
            assert (tmp_path / "pkg").exists()
            assert (tmp_path / "internal").exists()
    def test_create_rust_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
```

```
            result = initializer.create_project_structure(tmp_path, ProjectType.RUST)
            assert result
            assert (tmp_path / "src").exists()
    def test_create_java_maven_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.create_project_structure(tmp_path, ProjectType.JAVA_MAVEN)
            assert result
            assert (tmp_path / "src" / "main" / "java").exists()
            assert (tmp_path / "src" / "test" / "java").exists()
    def test_create_generic_structure(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            initializer = ProjectInitializer()
            result = initializer.create_project_structure(tmp_path, ProjectType.GENERIC)
            assert result
            assert (tmp_path / "src").exists()
            assert (tmp_path / "docs").exists()
class TestGetRequiredFiles:
    def test_python_required_files(self):
        initializer = ProjectInitializer()
        files = initializer.get_required_files(ProjectType.PYTHON)
        assert "README.md" in files
        assert ".gitignore" in files
        assert "AGENTS.md" in files
    def test_nodejs_required_files(self):
        initializer = ProjectInitializer()
        files = initializer.get_required_files(ProjectType.NODE_JS)
        assert "README.md" in files
        assert ".gitignore" in files
        assert "AGENTS.md" in files
    def test_generic_required_files(self):
        initializer = ProjectInitializer()
        files = initializer.get_required_files(ProjectType.GENERIC)
        assert "README.md" in files
        assert ".gitignore" in files
        assert "AGENTS.md" in files
class TestInitResult:
    def test_init_result_dataclass(self):
        result = InitResult(
            success=True,
            project_type=ProjectType.PYTHON,
            path=Path("/tmp/test"),
            files_created=["README.md", ".gitignore"],
            files_skipped=["pyproject.toml"],
            errors=[],
            warnings=["Git not initialized"],
            next_steps=["Install dependencies"],
        )
        assert result.success
        assert result.project_type == ProjectType.PYTHON
        assert len(result.files_created) == 2
        assert len(result.files_skipped) == 1
        assert len(result.errors) == 0
        assert len(result.warnings) == 1
        assert len(result.next_steps) == 1
class TestProjectAnalysisDataclass:
    def test_project_analysis_dataclass(self):
        analysis = ProjectAnalysis(
            project_type=ProjectType.NODE_JS,
            path=Path("/tmp/test"),
            has_git=True,
            has_readme=True,
            has_agents_md=False,
```

```
    has_tests=True,
    has_ci=False,
    existing_files=["package.json", "README.md"],
    missing_files=["AGENTS.md"],
    dependencies=["express", "typescript"],
    suggested_structure={"directories": ["__tests__"]},
)
assert analysis.project_type == ProjectType.NODE_JS
assert analysis.has_git
assert analysis.has_readme
assert not analysis.has_agents_md
assert analysis.has_tests
assert len(analysis.existing_files) == 2
assert len(analysis.missing_files) == 1
assert len(analysis.dependencies) == 2
```

File: [244/252] tests/test_providers.py

```
"""Tests for provider integration and tool calls."""
from __future__ import annotations
import asyncio
import json
import os
from unittest.mock import AsyncMock, MagicMock, patch
import httpx
import pytest
from openlaoke.core.multi_provider_api import MultiProviderClient, ModelPricing
from openlaoke.types.providers import MultiProviderConfig, ProviderConfig, ProviderType
class TestMultiProviderClient:
    """Tests for MultiProviderClient."""

    def test_client_creation(self):
        """Test creating a client."""
        config = MultiProviderConfig.defaults()
        client = MultiProviderClient(config)

        assert client.config == config
        assert client._proxy is None

    def test_get_api_key_minimax(self):
        """Test getting MiniMax API key."""
        config = MultiProviderConfig.defaults()
        minimax = config.providers["minimax"]
        minimax.api_key = "test_key"

        client = MultiProviderClient(config)
        api_key = client._get_api_key(minimax)

        assert api_key == "test_key"

    def test_get_api_key_from_env(self):
        """Test getting API key from environment."""
        config = MultiProviderConfig.defaults()
        minimax = config.providers["minimax"]

        with patch.dict(os.environ, {"MINIMAX_API_KEY": "env_key"}):
            client = MultiProviderClient(config)
            api_key = client._get_api_key(minimax)
            assert api_key == "env_key"

    def test_get_base_url_aliyun(self):
        """Test getting Aliyun base URL."""
        config = MultiProviderConfig.defaults()
        aliyun = config.providers["aliyun_coding_plan"]
        aliyun.base_url = "https://custom.aliyun.com/v1"

        client = MultiProviderClient(config)
        base_url = client._get_base_url(aliyun)

        assert base_url == "https://custom.aliyun.com/v1"

    def test_build_headers_anthropic(self):
        """Test building headers for Anthropic."""
        config = MultiProviderConfig.defaults()
        anthropic = config.providers["anthropic"]
        anthropic.api_key = "sk-ant-test"

        client = MultiProviderClient(config)
        headers = client._build_headers(anthropic)
```

```
    assert headers["x-api-key"] == "sk-ant-test"
    assert "anthropic-version" in headers

def test_build_headers_openai(self):
    """Test building headers for OpenAI."""
    config = MultiProviderConfig.defaults()
    openai = config.providers["openai"]
    openai.api_key = "sk-test"

    client = MultiProviderClient(config)
    headers = client._build_headers(openai)

    assert headers["Authorization"] == "Bearer sk-test"

def test_build_headers_local_no_auth(self):
    """Test that local providers don't send auth header."""
    config = MultiProviderConfig.defaults()
    ollama = config.providers["ollama"]

    client = MultiProviderClient(config)
    headers = client._build_headers(ollama)

    assert "Authorization" not in headers
    assert "Content-Type" in headers

def test_convert_tools_to_openai_format(self):
    """Test converting tools to OpenAI format."""
    config = MultiProviderConfig.defaults()
    client = MultiProviderClient(config)

    tools = [
        {
            "name": "Bash",
            "description": "Execute bash",
            "input_schema": {
                "type": "object",
                "properties": {"command": {"type": "string"}},
                "required": ["command"],
            },
        }
    ]

    openai_tools = client._convert_tools_to_openai_format(tools)

    assert len(openai_tools) == 1
    assert openai_tools[0]["type"] == "function"
    assert openai_tools[0]["function"]["name"] == "Bash"
    assert "parameters" in openai_tools[0]["function"]

def test_convert_tools_already_openai_format(self):
    """Test that already-converted tools are preserved."""
    config = MultiProviderConfig.defaults()
    client = MultiProviderClient(config)

    tools = [{
        "type": "function",
        "function": {
            "name": "Bash",
            "description": "Test",
            "parameters": {"type": "object"},
        }
    }]

    converted = client._convert_tools_to_openai_format(tools)
```

```
        assert converted == tools

    def test_convert_messages_to_openai_format(self):
        """Test converting messages to OpenAI format."""
        config = MultiProviderConfig.defaults()
        client = MultiProviderClient(config)

        messages = [
            {"role": "user", "content": "Hello"},
            {"role": "assistant", "tool_calls": [
                {"id": "call_1", "type": "function", "function": {"name": "Bash", "arguments":
    '{"command":"ls"}'}}
            ]},
            {"role": "tool", "tool_call_id": "call_1", "content": "file.txt"},
        ]

        converted = client._convert_to_openai_format(messages)

        assert len(converted) == 3
        assert converted[0]["role"] == "user"
        assert converted[1]["role"] == "assistant"
        assert converted[2]["role"] == "tool"
        assert converted[2]["tool_call_id"] == "call_1"

    @pytest.mark.asyncio
    async def test_send_message_mock(self):
        """Test sending message with mocked response."""
        config = MultiProviderConfig.defaults()
        minimax = config.providers["minimax"]
        minimax.api_key = "test_key"
        config.active_provider = "minimax"

        client = MultiProviderClient(config)

        mock_response = {
            "choices": [{
                "message": {"content": "Hello!", "tool_calls": []},
                "finish_reason": "stop",
            }],
            "usage": {"prompt_tokens": 10, "completion_tokens": 5},
            "model": "MiniMax-M2.7-highspeed",
        }

        with patch.object(client, "_get_client") as mock_get_client:
            mock_http_client = MagicMock()
            mock_resp = MagicMock()
            mock_resp.json.return_value = mock_response
            mock_resp.raise_for_status = MagicMock()
            mock_http_client.post = AsyncMock(return_value=mock_resp)
            mock_get_client.return_value = mock_http_client

            response, usage, cost = await client.send_message(
                system_prompt="You are helpful.",
                messages=[{"role": "user", "content": "Hi"}],
                tools=None,
            )

            assert response.content == "Hello!"
            assert usage.input_tokens == 10
            assert usage.output_tokens == 5
class TestModelPricing:
    """Tests for model pricing."""

    def test_default_pricing(self):
```

```
        """Test default pricing."""
        pricing = ModelPricing()
        assert pricing.input_price == 3.0
        assert pricing.output_price == 15.0

    def test_minimax_pricing_exists(self):
        """Test that MiniMax models have pricing."""
        from openlaoke.core.multi_provider_api import MODEL_PRICES

        assert "MiniMax-M2.7-highspeed" in MODEL_PRICES
        pricing = MODEL_PRICES["MiniMax-M2.7-highspeed"]
        assert pricing.input_price > 0
        assert pricing.output_price > 0

    def test_aliyun_pricing_exists(self):
        """Test that Aliyun models have pricing."""
        from openlaoke.core.multi_provider_api import MODEL_PRICES

        assert "qwen3.5-plus" in MODEL_PRICES
        assert "glm-5" in MODEL_PRICES
class TestProviderConfig:
    """Tests for provider configuration."""

    def test_minimax_config(self):
        """Test MiniMax provider configuration."""
        config = MultiProviderConfig.defaults()

        assert "minimax" in config.providers
        minimax = config.providers["minimax"]

        assert minimax.provider_type == ProviderType.MINIMAX
        assert minimax.base_url == "https://api.minimaxi.com/v1"
        assert minimax.default_model == "MiniMax-M2.7-highspeed"
        assert len(minimax.models) > 0

    def test_aliyun_config(self):
        """Test Aliyun provider configuration."""
        config = MultiProviderConfig.defaults()

        assert "aliyun_coding_plan" in config.providers
        aliyun = config.providers["aliyun_coding_plan"]

        assert aliyun.provider_type == ProviderType.ALIYUN_CODING_PLAN
        assert aliyun.base_url == "https://coding.dashscope.aliyuncs.com/v1"
        assert "qwen3.5-plus" in aliyun.models
        assert "glm-5" in aliyun.models

    def test_ollama_is_local(self):
        """Test that Ollama is marked as local."""
        config = MultiProviderConfig.defaults()
        ollama = config.providers["ollama"]

        assert ollama.is_local is True
        assert ollama.api_key == ""

    def test_is_configured(self):
        """Test is_configured method."""
        config = ProviderConfig(
            provider_type=ProviderType.MINIMAX,
            api_key="test",
        )
        assert config.is_configured() is True

        config_no_key = ProviderConfig(
            provider_type=ProviderType.MINIMAX,
```

```
    api_key="",
)
assert config_no_key.is_configured() is False

config_local = ProviderConfig(
    provider_type=ProviderType.OLLAMA,
    is_local=True,
)
assert config_local.is_configured() is True
```

File: [245/252] tests/test_safe_boundary.py

```
"""Tests for safe execution boundary."""
import tempfile
from pathlib import Path
import pytest
from openlaoke.core.safe_boundary import (
    ExecutionBoundary,
    SafeToolWrapper,
    create_safe_wrapper,
)
class TestExecutionBoundary:
    def test_boundary_from_cwd(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            assert tmpdir in str(boundary.allowed_paths)
    def test_is_path_allowed_in_cwd(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Path inside cwd should be allowed
            assert boundary.is_path_allowed(tmpdir)
            assert boundary.is_path_allowed(Path(tmpdir) / "subdir")
    def test_is_path_denied_outside_cwd(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Path outside cwd should be denied
            assert not boundary.is_path_allowed("/etc/passwd")
            assert not boundary.is_path_allowed("/tmp")
    def test_check_path_allowed(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            allowed, msg = boundary.check_path(tmpdir)
            assert allowed
            assert msg == "OK"
    def test_check_path_denied(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            allowed, msg = boundary.check_path("/etc/passwd")
            assert not allowed
            assert "Unauthorized" in msg
class TestSafeToolWrapper:
    def test_wrapper_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            assert wrapper is not None
    def test_check_bash_command_allowed(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            # Command with path inside cwd
            allowed, msg = wrapper.check_bash_command(f"ls {tmpdir}")
            assert allowed
    def test_check_bash_command_denied(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            # Command with path outside cwd
            allowed, msg = wrapper.check_bash_command("cat /etc/passwd")
            assert not allowed
    def test_check_read_path_allowed(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            test_file = Path(tmpdir) / "test.txt"
            test_file.write_text("test")
            allowed, msg = wrapper.check_read_path(str(test_file))
```

```
            assert allowed
    def test_check_read_path_denied(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            allowed, msg = wrapper.check_read_path("/etc/passwd")
            assert not allowed
    def test_check_write_path_allowed(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            test_file = Path(tmpdir) / "output.txt"
            allowed, msg = wrapper.check_write_path(str(test_file))
            assert allowed
    def test_check_write_path_denied(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            allowed, msg = wrapper.check_write_path("/etc/passwd")
            assert not allowed
class TestCreationTaskSkip:
    def test_should_skip_for_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            assert wrapper.should_skip_for_creation("write a new file")
            assert wrapper.should_skip_for_creation("create a program")
    def test_should_not_skip_for_reading(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            wrapper = create_safe_wrapper(tmpdir)
            assert not wrapper.should_skip_for_creation("read the file")
            assert not wrapper.should_skip_for_creation("show me the code")
class TestAbsolutePathHandling:
    def test_absolute_path_outside_cwd(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Absolute path outside cwd
            assert not boundary.is_path_allowed("/usr/bin/python")
            assert not boundary.is_path_allowed("/tmp/other")
    def test_relative_path_handling(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Relative path should be relative to cwd
            assert boundary.is_path_allowed("./file.txt")
            assert boundary.is_path_allowed("subdir/file.txt")
class TestBoundarySecurity:
    def test_parent_directory_escape(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Trying to escape with ../
            parent = Path(tmpdir).parent
            assert not boundary.is_path_allowed(str(parent))
            assert not boundary.is_path_allowed("../")
    def test_symlink_escape(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            tmp_path = Path(tmpdir)
            boundary = ExecutionBoundary.from_cwd(tmpdir)
            # Create symlink inside tmpdir
            link = tmp_path / "link_to_parent"
            try:
                link.symlink_to(tmp_path.parent)
                # Symlink should still be blocked if it points outside
                assert not boundary.is_path_allowed(str(link))
            except OSError:
                # Skip if symlink creation fails
                pass
```

File: [246/252] tests/test_scheduler.py

```python
"""Tests for the task scheduler system."""
from __future__ import annotations
import asyncio
import time
from typing import Any
import pytest
from openlaoke.core.scheduler import (
    PriorityQueue,
    ScheduledTask,
    TaskExecutor,
    TaskResult,
    TaskScheduler,
    TimeoutHandler,
)
from openlaoke.types.core_types import TaskStatus
class TestPriorityQueue:
    """Tests for PriorityQueue."""
    def test_basic_operations(self) -> None:
        """Test basic put and get operations."""
        queue = PriorityQueue()
        queue.put("item1", priority=1)
        queue.put("item2", priority=0)
        queue.put("item3", priority=2)
        assert queue.qsize() == 3
        assert queue.get() == "item2"
        assert queue.get() == "item1"
        assert queue.get() == "item3"
        assert queue.empty()
    def test_priority_order(self) -> None:
        """Test that items are retrieved by priority."""
        queue = PriorityQueue()
        queue.put("low", priority=10)
        queue.put("high", priority=1)
        queue.put("medium", priority=5)
        assert queue.get() == "high"
        assert queue.get() == "medium"
        assert queue.get() == "low"
    def test_blocking_get(self) -> None:
        """Test blocking get with timeout."""
        queue = PriorityQueue()
        result = queue.get(block=False)
        assert result is None
        result = queue.get(block=True, timeout=0.1)
        assert result is None
    def test_maxsize(self) -> None:
        """Test queue with maxsize."""
        queue = PriorityQueue(maxsize=2)
        assert queue.put("item1", priority=1)
        assert queue.put("item2", priority=2)
        assert queue.full()
        result = queue.put("item3", priority=3, block=False)
        assert not result
    def test_peek(self) -> None:
        """Test peek operation."""
        queue = PriorityQueue()
        queue.put("item1", priority=1)
        queue.put("item2", priority=0)
        assert queue.peek() == "item2"
        assert queue.qsize() == 2
    def test_clear(self) -> None:
        """Test clear operation."""
```

```
        queue = PriorityQueue()
        queue.put("item1", priority=1)
        queue.put("item2", priority=2)
        queue.clear()
        assert queue.empty()
class TestTimeoutHandler:
    """Tests for TimeoutHandler."""
    @pytest.mark.asyncio
    async def test_with_timeout_success(self) -> None:
        """Test successful execution with timeout."""
        handler = TimeoutHandler(default_timeout=5.0)
        async def quick_task() -> str:
            await asyncio.sleep(0.1)
            return "done"
        result = await handler.with_timeout(quick_task(), timeout=1.0, task_id="test1")
        assert result == "done"
    @pytest.mark.asyncio
    async def test_with_timeout_exceeded(self) -> None:
        """Test timeout exceeded."""
        handler = TimeoutHandler(default_timeout=5.0)
        async def slow_task() -> str:
            await asyncio.sleep(10.0)
            return "done"
        with pytest.raises(TimeoutError):
            await handler.with_timeout(slow_task(), timeout=1.0, task_id="test2")
    def test_set_timeout(self) -> None:
        """Test setting a timeout callback."""
        handler = TimeoutHandler()
        callback_called = []
        def callback() -> None:
            callback_called.append(True)
        handle = handler.set_timeout("test3", 0.1, callback)
        time.sleep(0.2)
        assert callback_called
        handle.cancel()
    def test_cancel_timeout(self) -> None:
        """Test cancelling a timeout."""
        handler = TimeoutHandler()
        callback_called = []
        def callback() -> None:
            callback_called.append(True)
        handler.set_timeout("test4", 0.5, callback)
        cancelled = handler.cancel_timeout("test4")
        assert cancelled
        time.sleep(0.6)
        assert not callback_called
    def test_get_remaining_time(self) -> None:
        """Test getting remaining time."""
        handler = TimeoutHandler()
        handler.set_timeout("test5", 5.0, lambda: None)
        remaining = handler.get_remaining_time("test5")
        assert remaining is not None
        assert 4.0 < remaining < 5.0
        handler.cancel_timeout("test5")
class TestTaskExecutor:
    """Tests for TaskExecutor."""
    @pytest.mark.asyncio
    async def test_execute_sync_function(self) -> None:
        """Test executing a synchronous function."""
        executor = TaskExecutor(max_workers=2)
        def sync_func(x: int) -> int:
            return x * 2
        result = await executor.execute("test1", sync_func, 5)
        assert result.status == TaskStatus.COMPLETED
        assert result.result == 10
```

```
        assert result.error is None
        executor.shutdown()
    @pytest.mark.asyncio
    async def test_execute_async_function(self) -> None:
        """Test executing an async function."""
        executor = TaskExecutor(max_workers=2)
        async def async_func(x: int) -> int:
            await asyncio.sleep(0.1)
            return x * 3
        result = await executor.execute_async("test2", async_func(5), timeout=1.0)
        assert result.status == TaskStatus.COMPLETED
        assert result.result == 15
        executor.shutdown()
    @pytest.mark.asyncio
    async def test_execute_with_timeout(self) -> None:
        """Test execution with timeout."""
        executor = TaskExecutor(max_workers=2)
        def slow_func() -> None:
            time.sleep(10.0)
        result = await executor.execute("test3", slow_func, timeout=0.5)
        assert result.status == TaskStatus.FAILED
        assert "Timeout" in result.error or "timeout" in result.error.lower()
        executor.shutdown()
    @pytest.mark.asyncio
    async def test_execute_with_retries(self) -> None:
        """Test execution with retries."""
        executor = TaskExecutor(max_workers=2, max_retries=3, retry_delay=0.1)
        attempts = []
        def flaky_func() -> str:
            attempts.append(1)
            if len(attempts) < 3:
                raise ValueError("Not yet")
            return "success"
        result = await executor.execute("test4", flaky_func, retries=3)
        assert result.status == TaskStatus.COMPLETED
        assert result.result == "success"
        assert len(attempts) == 3
        executor.shutdown()
class TestTaskScheduler:
    """Tests for TaskScheduler."""
    @pytest.mark.asyncio
    async def test_submit_single_task(self) -> None:
        """Test submitting a single task."""
        scheduler = TaskScheduler(max_concurrent=2)
        await scheduler.start()
        def simple_task() -> str:
            return "hello"
        result = await scheduler.submit(simple_task, priority=0, timeout=5.0)
        assert result.status == TaskStatus.COMPLETED
        assert result.result == "hello"
        await scheduler.shutdown()
    @pytest.mark.asyncio
    async def test_submit_async_task(self) -> None:
        """Test submitting an async task."""
        scheduler = TaskScheduler(max_concurrent=2)
        await scheduler.start()
        async def async_task() -> str:
            await asyncio.sleep(0.1)
            return "async_result"
        result = await scheduler.submit(async_task, timeout=5.0)
        assert result.status == TaskStatus.COMPLETED
        assert result.result == "async_result"
        await scheduler.shutdown()
    @pytest.mark.asyncio
    async def test_submit_batch_parallel(self) -> None:
```

```
    """Test submitting batch tasks in parallel."""
    scheduler = TaskScheduler(max_concurrent=5)
    await scheduler.start()
    def task(n: int) -> int:
        time.sleep(0.1)
        return n * 2
    tasks = [(task, (i,), {}) for i in range(5)]
    results = await scheduler.submit_batch(tasks, parallel=True, timeout=10.0)
    assert len(results) == 5
    for i, result in enumerate(results):
        assert result.status == TaskStatus.COMPLETED
        assert result.result == i * 2
    await scheduler.shutdown()
@pytest.mark.asyncio
async def test_submit_batch_sequential(self) -> None:
    """Test submitting batch tasks sequentially."""
    scheduler = TaskScheduler(max_concurrent=2)
    await scheduler.start()
    def task(n: int) -> int:
        return n * 3
    tasks = [(task, (i,), {}) for i in range(3)]
    results = await scheduler.submit_batch(tasks, parallel=False, timeout=10.0)
    assert len(results) == 3
    for i, result in enumerate(results):
        assert result.status == TaskStatus.COMPLETED
        assert result.result == i * 3
    await scheduler.shutdown()
@pytest.mark.asyncio
async def test_cancel_task(self) -> None:
    """Test cancelling a task."""
    scheduler = TaskScheduler(max_concurrent=2)
    await scheduler.start()
    def slow_task() -> None:
        time.sleep(10.0)
    task_id = f"cancel_test_{time.time()}"
    submit_coro = scheduler.submit(slow_task, timeout=15.0, task_id=task_id)
    await asyncio.sleep(0.1)
    cancelled = await scheduler.cancel(task_id)
    if cancelled:
        result = await submit_coro
        assert result.status == TaskStatus.KILLED
        assert "Cancelled" in result.error or "cancelled" in result.error.lower()
    await scheduler.shutdown()
@pytest.mark.asyncio
async def test_get_status(self) -> None:
    """Test getting task status."""
    scheduler = TaskScheduler(max_concurrent=2)
    await scheduler.start()
    def quick_task() -> str:
        return "done"
    result = await scheduler.submit(quick_task, timeout=5.0)
    status = await scheduler.get_status(result.task_id)
    assert status == TaskStatus.COMPLETED
    await scheduler.shutdown()
@pytest.mark.asyncio
async def test_get_stats(self) -> None:
    """Test getting scheduler statistics."""
    scheduler = TaskScheduler(max_concurrent=3)
    stats = scheduler.get_stats()
    assert stats["total"] == 0
    assert stats["pending"] == 0
    assert stats["running"] == 0
    await scheduler.start()
    def task() -> str:
        time.sleep(0.2)
```

```
            return "result"
        await scheduler.submit(task, timeout=5.0)
        stats = scheduler.get_stats()
        assert stats["total"] == 1
        await scheduler.shutdown()
class TestScheduledTask:
    """Tests for ScheduledTask dataclass."""
    def test_creation(self) -> None:
        """Test creating a ScheduledTask."""
        task = ScheduledTask(
            id="test_task",
            func=lambda: "result",
            priority=5,
            timeout=10.0,
        )
        assert task.id == "test_task"
        assert task.priority == 5
        assert task.timeout == 10.0
        assert task.status == TaskStatus.PENDING
        assert task.kwargs == {}
    def test_with_args_kwargs(self) -> None:
        """Test ScheduledTask with args and kwargs."""
        task = ScheduledTask(
            id="test_task",
            func=lambda x, y: x + y,
            args=(5,),
            kwargs={"y": 10},
            priority=1,
        )
        assert task.args == (5,)
        assert task.kwargs == {"y": 10}
class TestTaskResult:
    """Tests for TaskResult dataclass."""
    def test_success_result(self) -> None:
        """Test successful task result."""
        result = TaskResult(
            task_id="test",
            status=TaskStatus.COMPLETED,
            result="success",
            duration=1.5,
        )
        assert result.task_id == "test"
        assert result.status == TaskStatus.COMPLETED
        assert result.result == "success"
        assert result.error is None
        assert result.duration == 1.5
    def test_error_result(self) -> None:
        """Test error task result."""
        result = TaskResult(
            task_id="test",
            status=TaskStatus.FAILED,
            error="Task failed",
        )
        assert result.status == TaskStatus.FAILED
        assert result.result is None
        assert result.error == "Task failed"
```

File: [247/252] tests/test_skills.py

```
"""Tests for the skill system."""
from __future__ import annotations
import tempfile
from pathlib import Path
import pytest
from openlaoke.core.skill_system import Skill, SkillRegistry, get_skill_registry, load_skill, l
    ist_available_skills
class TestSkill:
    """Tests for the Skill class."""

    def test_skill_creation(self):
        """Test basic skill creation."""
        skill = Skill(
            name="test_skill",
            description="A test skill",
            version="1.0.0",
            content="Test content",
        )

        assert skill.name == "test_skill"
        assert skill.description == "A test skill"
        assert skill.version == "1.0.0"
        assert skill.content == "Test content"

    def test_skill_from_content_with_frontmatter(self):
        """Test parsing skill from content with YAML frontmatter."""
        content = """---
name: test_skill
version: 2.0.0
description: |
  A test skill
  with multiple lines
allowed-tools:
  - Bash
  - Read
---
# Test Skill Content
This is the skill body.
"""

        skill = Skill.from_content(content)

        assert skill.name == "test_skill"
        assert skill.version == "2.0.0"
        assert "A test skill" in skill.description
        assert "Bash" in skill.allowed_tools
        assert "Read" in skill.allowed_tools
        assert "# Test Skill Content" in skill.content

    def test_skill_from_content_without_frontmatter(self):
        """Test parsing skill without frontmatter."""
        content = "# Simple Skill\n\nJust content here."

        skill = Skill.from_content(content)

        assert skill.name == "unknown"
        assert skill.content == content

    def test_skill_from_file(self):
        """Test loading skill from file."""
        with tempfile.TemporaryDirectory() as tmpdir:
```

```
            skill_path = Path(tmpdir) / "SKILL.md"
            skill_path.write_text("""---
name: file_skill
description: A skill from file
---
# Content
""")

            skill = Skill.from_file(skill_path)

            assert skill is not None
            assert skill.name == "file_skill"
            assert skill.path == skill_path

    def test_skill_from_nonexistent_file(self):
        """Test loading from non-existent file."""
        skill = Skill.from_file(Path("/nonexistent/SKILL.md"))
        assert skill is None

    def test_get_preamble(self):
        """Test extracting bash preamble."""
        content = """---
name: preamble_test
---
# Test
```bash
echo "Hello"
pwd
```
More content.
"""

        skill = Skill.from_content(content)
        preamble = skill.get_preamble()

        assert preamble is not None
        assert 'echo "Hello"' in preamble
        assert "pwd" in preamble

    def test_get_system_prompt(self):
        """Test getting system prompt."""
        skill = Skill(
            name="prompt_test",
            content="# Instructions\n\nDo this.",
)

        prompt = skill.get_system_prompt()
        assert "# Instructions" in prompt
class TestSkillRegistry:
    """Tests for the SkillRegistry class."""

    def test_registry_creation(self):
        """Test creating a skill registry."""
        registry = SkillRegistry()
        assert len(registry.list_skills()) == 0

    def test_add_skill_directory(self):
        """Test adding a skill directory."""
        with tempfile.TemporaryDirectory() as tmpdir:
            tmpdir = Path(tmpdir)

            # Create skill directory
            skill_dir = tmpdir / "test_skill"
            skill_dir.mkdir()
            (skill_dir / "SKILL.md").write_text("""---
```

```
name: test_skill
description: Test
---
Content
""")

            registry = SkillRegistry()
            count = registry.add_skill_directory(tmpdir)

            assert count == 1
            assert "test_skill" in registry.list_skills()

    def test_get_skill(self):
        """Test getting a skill by name."""
        with tempfile.TemporaryDirectory() as tmpdir:
            tmpdir = Path(tmpdir)
            skill_dir = tmpdir / "my_skill"
            skill_dir.mkdir()
            (skill_dir / "SKILL.md").write_text("""---
name: my_skill
---
Content
""")

            registry = SkillRegistry()
            registry.add_skill_directory(tmpdir)

            skill = registry.get_skill("my_skill")
            assert skill is not None
            assert skill.name == "my_skill"

    def test_get_nonexistent_skill(self):
        """Test getting a non-existent skill."""
        registry = SkillRegistry()
        skill = registry.get_skill("nonexistent")
        assert skill is None

    def test_has_skill(self):
        """Test checking if a skill exists."""
        with tempfile.TemporaryDirectory() as tmpdir:
            tmpdir = Path(tmpdir)
            skill_dir = tmpdir / "exists"
            skill_dir.mkdir()
            (skill_dir / "SKILL.md").write_text("""---
name: exists
---
Content
""")

            registry = SkillRegistry()
            registry.add_skill_directory(tmpdir)

            assert registry.has_skill("exists")
            assert not registry.has_skill("nope")

    def test_get_all_skills(self):
        """Test getting all skills."""
        with tempfile.TemporaryDirectory() as tmpdir:
            tmpdir = Path(tmpdir)

            for name in ["skill1", "skill2"]:
                skill_dir = tmpdir / name
                skill_dir.mkdir()
                (skill_dir / "SKILL.md").write_text(f"""---
name: {name}
```

```
---
Content
""")

            registry = SkillRegistry()
            registry.add_skill_directory(tmpdir)

            all_skills = registry.get_all_skills()
            assert len(all_skills) == 2
class TestGlobalRegistry:
    """Tests for the global skill registry functions."""

    def test_get_skill_registry_singleton(self):
        """Test that get_skill_registry returns a singleton."""
        registry1 = get_skill_registry()
        registry2 = get_skill_registry()

        assert registry1 is registry2

    def test_list_available_skills(self):
        """Test listing available skills."""
        skills = list_available_skills()
        assert isinstance(skills, list)

    def test_load_skill(self):
        """Test loading a skill."""
        # This will return None if no skills are installed
        skill = load_skill("nonexistent")
        assert skill is None
class TestClaudeCodeCompatibility:
    """Tests for Claude Code skill format compatibility."""

    def test_parse_claude_code_skill(self):
        """Test parsing a Claude Code style skill."""
        content = """---
name: browse
version: 1.1.0
description: |
  Fast headless browser for QA testing and site dogfooding.
allowed-tools:
  - Bash
  - Read
  - AskUserQuestion
---
## Preamble (run first)
```bash
echo "Setup"
```
Main Content
Instructions here.
"""

        skill = Skill.from_content(content)

        assert skill.name == "browse"
        assert skill.version == "1.1.0"
        assert "Fast headless browser" in skill.description
        assert "Bash" in skill.allowed_tools
        assert "Read" in skill.allowed_tools
        preamble = skill.get_preamble()
        assert 'echo "Setup"' in preamble
class TestOpenCodeCompatibility:
    """Tests for OpenCode skill format compatibility."""

    def test_parse_opencode_skill(self):
```

```
        """Test parsing an OpenCode style skill."""
        content = """---
name: agents-sdk
description: Build AI agents on Cloudflare Workers using the Agents SDK.
---
# Cloudflare Agents SDK
**STOP.** Your knowledge of the Agents SDK may be outdated.
## Documentation
Fetch current docs from `https://github.com/cloudflare/agents/tree/main/docs`.
"""

        skill = Skill.from_content(content)

        assert skill.name == "agents-sdk"
        assert "Build AI agents" in skill.description
        assert "Cloudflare Agents SDK" in skill.content
```

File: [248/252] tests/test_smart_prompt.py

```
"""Tests for smart prompt generator."""
import pytest
from openlaoke.core.smart_prompt import (
    SmartPromptGenerator,
    optimize_for_small_model,
)
class TestSmartPromptGenerator:
    def test_generator_creation(self):
        generator = SmartPromptGenerator("gemma3:1b")
        assert generator.adapter.model == "gemma3:1b"
    def test_generate_basic_prompt(self):
        generator = SmartPromptGenerator("gemma3:1b")
        base = "You are helpful."
        user_req = "read the file"
        prompt = generator.generate_system_prompt(user_req, base)
        assert "You are helpful" in prompt
    def test_optimize_for_creation(self):
        generator = SmartPromptGenerator("gemma3:1b")
        base = "You are helpful."
        user_req = "write a new program"
        prompt = generator.generate_system_prompt(user_req, base)
        assert "OPTIMIZATION" in prompt
        assert "Do NOT" in prompt
        assert "IMMEDIATELY" in prompt
    def test_no_optimization_for_reading(self):
        generator = SmartPromptGenerator("gemma3:1b")
        base = "You are helpful."
        user_req = "show me the file"
        prompt = generator.generate_system_prompt(user_req, base)
        assert "OPTIMIZATION" not in prompt
    def test_tool_instructions_for_no_tool_model(self):
        generator = SmartPromptGenerator("gemma3:1b")
        base = "You are helpful."
        user_req = "do something"
        tools = [{"name": "read", "description": "Read file", "input_schema": {}}]
        prompt = generator.generate_system_prompt(user_req, base, tools)
        assert "TOOL USAGE" in prompt
    def test_no_tool_instructions_for_tool_model(self):
        generator = SmartPromptGenerator("gemma4:e2b")
        base = "You are helpful."
        user_req = "do something"
        tools = [{"name": "read", "description": "Read file", "input_schema": {}}]
        prompt = generator.generate_system_prompt(user_req, base, tools)
        assert "TOOL USAGE" not in prompt
    def test_get_execution_hints_creation(self):
        generator = SmartPromptGenerator("gemma3:1b")
        hints = generator.get_execution_hints("write a new file")
        assert hints["skip_context_gathering"] is True
        assert hints["is_creation"] is True
        assert hints["needs_tools"] is True
    def test_get_execution_hints_reading(self):
        generator = SmartPromptGenerator("gemma3:1b")
        hints = generator.get_execution_hints("read the file")
        assert hints["skip_context_gathering"] is False
        assert hints["is_creation"] is False
        assert hints["needs_tools"] is True
class TestOptimizeForSmallModel:
    def test_optimize_basic(self):
        base = "You are helpful."
        user_req = "write a new program"
        model = "gemma3:1b"
```

```
        prompt, hints = optimize_for_small_model(model, user_req, base)
        assert "OPTIMIZATION" in prompt
        assert hints["skip_context_gathering"] is True
    def test_optimize_with_tools(self):
        base = "You are helpful."
        user_req = "create a file"
        model = "gemma3:1b"
        tools = [{"name": "write", "description": "Write", "input_schema": {}}]
        prompt, hints = optimize_for_small_model(model, user_req, base, tools)
        assert "TOOL USAGE" in prompt
        assert hints["needs_tools"] is True
    def test_optimize_for_tool_model(self):
        base = "You are helpful."
        user_req = "write a program"
        model = "gemma4:e2b"
        prompt, hints = optimize_for_small_model(model, user_req, base)
        assert hints["needs_tools"] is False
class TestChineseRequests:
    def test_chinese_creation_request(self):
        generator = SmartPromptGenerator("gemma3:1b")
        prompt = generator.generate_system_prompt("??????", "You are helpful.")
        assert "OPTIMIZATION" in prompt
    def test_chinese_read_request(self):
        generator = SmartPromptGenerator("gemma3:1b")
        prompt = generator.generate_system_prompt("????", "You are helpful.")
        assert "OPTIMIZATION" not in prompt
class TestEdgeCases:
    def test_empty_request(self):
        generator = SmartPromptGenerator("gemma3:1b")
        hints = generator.get_execution_hints("")
        assert hints["is_creation"] is False
    def test_modify_existing(self):
        generator = SmartPromptGenerator("gemma3:1b")
        hints = generator.get_execution_hints("modify existing file")
        assert hints["skip_context_gathering"] is False
        assert hints["is_creation"] is False  # modification, not creation
    def test_based_on_existing(self):
        generator = SmartPromptGenerator("gemma3:1b")
        hints = generator.get_execution_hints("create based on existing code")
        assert hints["skip_context_gathering"] is False
```

File: [249/252] tests/test_supervisor.py

```
"""Tests for task supervisor system."""
import tempfile
import pytest
from openlaoke.core.state import create_app_state
from openlaoke.core.supervisor.requirements import TaskRequirements
from openlaoke.core.supervisor.supervisor import (
    RetryReason,
    SupervisedTask,
    SupervisionResult,
    TaskStatus,
    TaskSupervisor,
)
from openlaoke.types.providers import MultiProviderConfig
class TestTaskStatus:
    def test_task_status_values(self):
        assert TaskStatus.PENDING.value == "pending"
        assert TaskStatus.IN_PROGRESS.value == "in_progress"
        assert TaskStatus.COMPLETED.value == "completed"
        assert TaskStatus.FAILED.value == "failed"
        assert TaskStatus.RETRYING.value == "retrying"
        assert TaskStatus.ESCALATED.value == "escalated"
class TestRetryReason:
    def test_retry_reason_values(self):
        assert RetryReason.INCOMPLETE_OUTPUT.value == "incomplete_output"
        assert RetryReason.MISSING_REQUIREMENTS.value == "missing_requirements"
        assert RetryReason.ERROR_OCCURRED.value == "error_occurred"
        assert RetryReason.QUALITY_ISSUE.value == "quality_issue"
        assert RetryReason.AI_DETECTED.value == "ai_detected"
class TestTaskRequirements:
    def test_requirement_creation(self):
        req = TaskRequirements(
            name="document_created",
            description="Target document must be created",
            check_type="file_exists",
            critical=True,
        )
        assert req.name == "document_created"
        assert req.description == "Target document must be created"
        assert req.check_type == "file_exists"
        assert req.critical
        assert req.weight == 2.0
    def test_requirement_with_threshold(self):
        req = TaskRequirements(
            name="word_count",
            description="Document must have sufficient length",
            check_type="word_count",
            threshold=3000,
            critical=True,
        )
        assert req.threshold == 3000
    def test_requirement_weight(self):
        req = TaskRequirements(
            name="optional_check",
            description="Optional quality check",
            check_type="general",
            critical=False,
        )
        assert req.weight == 1.0
class TestSupervisedTask:
    def test_supervised_task_creation(self):
        reqs = [
```

```
            TaskRequirements(
                name="test_req",
                description="Test requirement",
                check_type="general",
                critical=True,
            )
        ]
        task = SupervisedTask(
            original_request="Test task",
            requirements=reqs,
        )
        assert task.original_request == "Test task"
        assert len(task.requirements) == 1
        assert task.status == TaskStatus.PENDING
        assert task.attempts == 0
        assert task.max_attempts == 5
    def test_supervised_task_with_model(self):
        task = SupervisedTask(
            original_request="Test",
            requirements=[],
            model="gemma3:1b",
        )
        assert task.model == "gemma3:1b"
    def test_supervised_task_with_decomposed_steps(self):
        task = SupervisedTask(
            original_request="Complex task",
            requirements=[],
            decomposed_steps=["Step 1", "Step 2", "Step 3"],
        )
        assert len(task.decomposed_steps) == 3
        assert task.current_step == 0
class TestSupervisionResult:
    def test_supervision_result_creation(self):
        result = SupervisionResult(
            task_id="task_123",
            is_complete=False,
            completion_percentage=50.0,
            missing_requirements=["Missing doc"],
            suggested_actions=["Create document"],
            should_retry=True,
            retry_reason=RetryReason.MISSING_REQUIREMENTS,
            feedback="Incomplete",
        )
        assert result.task_id == "task_123"
        assert not result.is_complete
        assert result.completion_percentage == 50.0
        assert len(result.missing_requirements) == 1
        assert result.should_retry
        assert result.retry_reason == RetryReason.MISSING_REQUIREMENTS
    def test_supervision_result_complete(self):
        result = SupervisionResult(
            task_id="task_123",
            is_complete=True,
            completion_percentage=100.0,
            missing_requirements=[],
            suggested_actions=[],
            should_retry=False,
        )
        assert result.is_complete
        assert result.completion_percentage == 100.0
        assert not result.should_retry
class TestTaskSupervisor:
    def test_supervisor_creation(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
```

```
        supervisor = TaskSupervisor(state)
        assert supervisor.app_state == state
        assert len(supervisor.tasks) == 0
def test_set_model(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        config = MultiProviderConfig.defaults()
        supervisor.set_model("gemma3:1b", config)
        assert supervisor.current_model == "gemma3:1b"
        assert supervisor.model_assessor is not None
        assert supervisor.task_decomposer is not None
def test_parse_simple_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        task = supervisor.parse_request("Simple query")
        assert task.original_request == "Simple query"
        assert task.status == TaskStatus.PENDING
        assert len(task.requirements) == 1
def test_parse_write_article_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        task = supervisor.parse_request("Write an article about AI")
        assert task.original_request == "Write an article about AI"
        assert len(task.requirements) > 1
        critical_reqs = [r for r in task.requirements if r.critical]
        assert len(critical_reqs) > 0
        assert any(r.name == "document_created" for r in task.requirements)
        assert any(r.name == "sufficient_length" for r in task.requirements)
        assert any(r.name == "anti_ai_quality" for r in task.requirements)
def test_parse_code_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        task = supervisor.parse_request("Implement a Python function")
        assert len(task.requirements) >= 1  # Should have at least general requirement
def test_parse_comparison_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        task = supervisor.parse_request("Compare two approaches")
        assert any(r.name == "quantitative_data" for r in task.requirements)
def test_parse_diagram_request(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        task = supervisor.parse_request("Create a diagram")
        assert any(r.name == "figures_created" for r in task.requirements)
def test_decompose_with_model_assessment(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
        config = MultiProviderConfig.defaults()
        supervisor.set_model("gemma3:1b", config)
        task = supervisor.parse_request(
            "Write an article and create a diagram and implement code"
        )
        assert len(task.decomposed_steps) > 1
        assert len(task.decomposed_steps) <= 4
def test_get_task_status(self):
    with tempfile.TemporaryDirectory() as tmpdir:
        state = create_app_state(cwd=tmpdir)
        supervisor = TaskSupervisor(state)
```

```
            task = supervisor.parse_request("Test")
            task_id = [k for k in supervisor.tasks.keys()][0]
            retrieved = supervisor.get_task_status(task_id)
            assert retrieved is not None
            assert retrieved.original_request == "Test"
    def test_should_escalate(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Test")
            task_id = [k for k in supervisor.tasks.keys()][0]
            assert not supervisor.should_escalate(task_id)
            task.attempts = task.max_attempts
            task.status = TaskStatus.RETRYING
            assert supervisor.should_escalate(task_id)
    def test_enable_supervision(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            assert supervisor._supervision_enabled
            supervisor.enable_supervision(False)
            assert not supervisor._supervision_enabled
    def test_enable_anti_ai_mode(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            assert supervisor._anti_ai_mode
            supervisor.enable_anti_ai_mode(False)
            assert not supervisor._anti_ai_mode
class TestSupervisorCheckCompletion:
    @pytest.mark.asyncio
    async def test_check_completion_task_not_found(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            result = await supervisor.check_completion("nonexistent", {})
            assert not result.is_complete
            assert "Task not found" in result.missing_requirements
            assert not result.should_retry
    @pytest.mark.asyncio
    async def test_check_completion_complete_task(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Simple query")
            task_id = [k for k in supervisor.tasks.keys()][0]
            artifacts = {"output": "Task completed successfully"}
            result = await supervisor.check_completion(task_id, artifacts)
            assert result.completion_percentage >= 0.0
    @pytest.mark.asyncio
    async def test_check_completion_with_file_artifacts(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Write an article about AI")
            task_id = [k for k in supervisor.tasks.keys()][0]
            test_file = f"{tmpdir}/article.md"
            with open(test_file, "w") as f:
                f.write("Article content\n" * 200)
            artifacts = {
                "file": test_file,
                "content": "Article with sufficient content",
            }
            result = await supervisor.check_completion(task_id, artifacts)
            assert result.completion_percentage >= 0.0
```

```
class TestSupervisorRetryPrompt:
    def test_get_retry_prompt_basic(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Write an article")
            task_id = [k for k in supervisor.tasks.keys()][0]
            result = SupervisionResult(
                task_id=task_id,
                is_complete=False,
                completion_percentage=30.0,
                missing_requirements=["Missing length", "Missing citations"],
                suggested_actions=["Add content", "Add citations"],
                should_retry=True,
                retry_reason=RetryReason.MISSING_REQUIREMENTS,
            )
            prompt = supervisor.get_retry_prompt(task_id, result)
            assert "Write an article" in prompt
            assert "30.0%" in prompt
            assert "Missing length" in prompt
            assert "Add content" in prompt
    def test_get_retry_prompt_ai_detected(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Write an article")
            task_id = [k for k in supervisor.tasks.keys()][0]
            result = SupervisionResult(
                task_id=task_id,
                is_complete=False,
                completion_percentage=50.0,
                missing_requirements=["AI-typical patterns detected"],
                suggested_actions=["Add specific numbers", "Add real citations"],
                should_retry=True,
                retry_reason=RetryReason.AI_DETECTED,
            )
            prompt = supervisor.get_retry_prompt(task_id, result)
            assert "AI" in prompt or "Anti-AI" in prompt
            assert "specific numbers" in prompt or "SPECIFIC" in prompt
class TestSupervisorStepManagement:
    def test_get_step_prompt(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            config = MultiProviderConfig.defaults()
            supervisor.set_model("gemma3:1b", config)
            task = supervisor.parse_request("Step 1 and Step 2 and Step 3")
            task_id = [k for k in supervisor.tasks.keys()][0]
            step_prompt = supervisor.get_step_prompt(task_id)
            if step_prompt:
                assert "Step" in step_prompt
    def test_advance_step(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            config = MultiProviderConfig.defaults()
            supervisor.set_model("gemma3:1b", config)
            task = supervisor.parse_request("Step 1 and Step 2")
            task_id = [k for k in supervisor.tasks.keys()][0]
            if len(task.decomposed_steps) > 1:
                assert supervisor.advance_step(task_id)
                assert supervisor.tasks[task_id].current_step == 1
                assert not supervisor.advance_step(task_id)
    def test_get_timeout(self):
        with tempfile.TemporaryDirectory() as tmpdir:
```

```
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            timeout = supervisor.get_timeout(60.0)
            assert timeout == 60.0
            config = MultiProviderConfig.defaults()
            supervisor.set_model("gemma3:1b", config)
            timeout = supervisor.get_timeout(60.0)
            assert timeout > 60.0
    def test_get_model_guidance(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            guidance = supervisor.get_model_guidance()
            assert guidance == ""
            config = MultiProviderConfig.defaults()
            supervisor.set_model("gemma3:1b", config)
            guidance = supervisor.get_model_guidance()
            if guidance:
                assert "explicit" in guidance.lower() or "atomic" in guidance.lower()
class TestSupervisorIntegration:
    @pytest.mark.asyncio
    async def test_full_supervision_cycle(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            config = MultiProviderConfig.defaults()
            supervisor.set_model("gemma3:1b", config)
            task = supervisor.parse_request(
                "Write a comprehensive article about machine learning with citations"
            )
            task_id = [k for k in supervisor.tasks.keys()][0]
            assert len(task.requirements) > 3
            assert task.status == TaskStatus.PENDING
            artifacts1 = {"partial": "Incomplete work"}
            result1 = await supervisor.check_completion(task_id, artifacts1)
            assert not result1.is_complete
            assert result1.completion_percentage < 80.0
            test_file = f"{tmpdir}/article.md"
            with open(test_file, "w") as f:
                f.write("Introduction\n\n" + "Content\n" * 300 + "\nReferences\n[1] Test\n")
            artifacts2 = {
                "file": test_file,
                "content": "Full article with references",
            }
            result2 = await supervisor.check_completion(task_id, artifacts2)
            assert result2.completion_percentage >= result1.completion_percentage
    @pytest.mark.asyncio
    async def test_retry_cycle_with_improvement(self):
        with tempfile.TemporaryDirectory() as tmpdir:
            state = create_app_state(cwd=tmpdir)
            supervisor = TaskSupervisor(state)
            task = supervisor.parse_request("Write an article")
            task_id = [k for k in supervisor.tasks.keys()][0]
            result1 = SupervisionResult(
                task_id=task_id,
                is_complete=False,
                completion_percentage=20.0,
                missing_requirements=["No content"],
                suggested_actions=["Add content"],
                should_retry=True,
                retry_reason=RetryReason.INCOMPLETE_OUTPUT,
            )
            prompt1 = supervisor.get_retry_prompt(task_id, result1)
            assert prompt1
            task.attempts += 1
```

```
result2 = SupervisionResult(
    task_id=task_id,
    is_complete=False,
    completion_percentage=40.0,
    missing_requirements=["More content needed"],
    suggested_actions=["Expand"],
    should_retry=True,
    retry_reason=RetryReason.INCOMPLETE_OUTPUT,
)
prompt2 = supervisor.get_retry_prompt(task_id, result2)
assert "40.0%" in prompt2
assert "Attempt 1" in prompt2
```

File: [250/252] tests/test_theme.py

```
"""Tests for theme system."""
import os
import tempfile
from dataclasses import dataclass, field
from typing import Any
import pytest
from openlaoke.core.state import AppState, create_app_state
from openlaoke.utils.config import AppConfig, load_config, save_config
@dataclass
class ThemeColors:
    """Color definitions for a theme."""
    background: str = "#1a1a1a"
    foreground: str = "#ffffff"
    accent: str = "#00ff88"
    error: str = "#ff4444"
    warning: str = "#ffaa00"
    success: str = "#00ff00"
    muted: str = "#666666"
    highlight: str = "#ffff00"
@dataclass
class Theme:
    """Theme configuration."""
    name: str = "dark"
    colors: ThemeColors = field(default_factory=ThemeColors)
    font_family: str = "monospace"
    font_size: int = 12
    line_height: float = 1.2
    padding: int = 10
    border_radius: int = 4
    custom_settings: dict[str, Any] = field(default_factory=dict)
class ThemeManager:
    """Manager for themes."""
    DEFAULT_DARK = Theme(
        name="dark",
        colors=ThemeColors(
            background="#1a1a1a",
            foreground="#ffffff",
            accent="#00ff88",
        ),
    )
    DEFAULT_LIGHT = Theme(
        name="light",
        colors=ThemeColors(
            background="#ffffff",
            foreground="#1a1a1a",
            accent="#0088ff",
        ),
    )
    def __init__(self):
        self._themes: dict[str, Theme] = {
            "dark": self.DEFAULT_DARK,
            "light": self.DEFAULT_LIGHT,
        }
        self._current_theme: str = "dark"
        self._custom_themes_dir: str = ""
    def get_theme(self, name: str | None = None) -> Theme:
        if name is None:
            name = self._current_theme
        return self._themes.get(name, self.DEFAULT_DARK)
    def set_theme(self, name: str) -> bool:
        if name in self._themes:
```

```
            self._current_theme = name
            return True
        return False
    def register_theme(self, theme: Theme) -> None:
        self._themes[theme.name] = theme
    def get_available_themes(self) -> list[str]:
        return list(self._themes.keys())
    def is_valid_theme(self, name: str) -> bool:
        return name in self._themes
    def load_custom_theme(self, path: str) -> Theme | None:
        import json
        try:
            with open(path) as f:
                data = json.load(f)
            colors = ThemeColors(
                background=data.get("background", "#1a1a1a"),
                foreground=data.get("foreground", "#ffffff"),
                accent=data.get("accent", "#00ff88"),
                error=data.get("error", "#ff4444"),
                warning=data.get("warning", "#ffaa00"),
                success=data.get("success", "#00ff00"),
                muted=data.get("muted", "#666666"),
                highlight=data.get("highlight", "#ffff00"),
            )
            theme = Theme(
                name=data.get("name", "custom"),
                colors=colors,
                font_family=data.get("font_family", "monospace"),
                font_size=data.get("font_size", 12),
                custom_settings=data.get("custom_settings", {}),
            )
            self.register_theme(theme)
            return theme
        except Exception:
            return None
    def save_custom_theme(self, theme: Theme, path: str) -> bool:
        import json
        try:
            data = {
                "name": theme.name,
                "background": theme.colors.background,
                "foreground": theme.colors.foreground,
                "accent": theme.colors.accent,
                "error": theme.colors.error,
                "warning": theme.colors.warning,
                "success": theme.colors.success,
                "muted": theme.colors.muted,
                "highlight": theme.colors.highlight,
                "font_family": theme.font_family,
                "font_size": theme.font_size,
                "custom_settings": theme.custom_settings,
            }
            with open(path, "w") as f:
                json.dump(data, f, indent=2)
            return True
        except Exception:
            return False
@pytest.fixture
def theme_manager():
    return ThemeManager()
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
@pytest.fixture
```

```
def app_state():
    import openlaoke
    project_root = os.path.dirname(os.path.dirname(openlaoke.__file__))
    return create_app_state(cwd=project_root)
class TestThemeLoad:
    def test_default_theme(self, theme_manager):
        theme = theme_manager.get_theme()
        assert theme.name == "dark"
        assert theme.colors.background == "#1a1a1a"
    def test_load_dark_theme(self, theme_manager):
        theme = theme_manager.get_theme("dark")
        assert theme.name == "dark"
        assert theme.colors.foreground == "#ffffff"
    def test_load_light_theme(self, theme_manager):
        theme = theme_manager.get_theme("light")
        assert theme.name == "light"
        assert theme.colors.background == "#ffffff"
        assert theme.colors.foreground == "#1a1a1a"
    def test_load_nonexistent_theme(self, theme_manager):
        theme = theme_manager.get_theme("nonexistent")
        assert theme.name == "dark"
    def test_get_available_themes(self, theme_manager):
        themes = theme_manager.get_available_themes()
        assert "dark" in themes
        assert "light" in themes
    def test_set_valid_theme(self, theme_manager):
        result = theme_manager.set_theme("light")
        assert result
        assert theme_manager.get_theme().name == "light"
    def test_set_invalid_theme(self, theme_manager):
        result = theme_manager.set_theme("invalid")
        assert not result
        assert theme_manager.get_theme().name == "dark"
    def test_is_valid_theme(self, theme_manager):
        assert theme_manager.is_valid_theme("dark")
        assert theme_manager.is_valid_theme("light")
        assert not theme_manager.is_valid_theme("invalid")
class TestThemeColors:
    def test_default_colors(self):
        colors = ThemeColors()
        assert colors.background == "#1a1a1a"
        assert colors.foreground == "#ffffff"
        assert colors.accent == "#00ff88"
    def test_custom_colors(self):
        colors = ThemeColors(
            background="#000000",
            foreground="#cccccc",
            accent="#ff0000",
            error="#ff1111",
        )
        assert colors.background == "#000000"
        assert colors.accent == "#ff0000"
    def test_all_color_fields(self):
        colors = ThemeColors()
        color_fields = [
            "background",
            "foreground",
            "accent",
            "error",
            "warning",
            "success",
            "muted",
            "highlight",
        ]
        for field_name in color_fields:
```

```
            assert hasattr(colors, field_name)
            color_value = getattr(colors, field_name)
            assert color_value.startswith("#")
class TestCustomTheme:
    def test_register_custom_theme(self, theme_manager):
        custom_colors = ThemeColors(
            background="#2a2a2a",
            foreground="#dddddd",
            accent="#88ff00",
        )
        custom_theme = Theme(name="custom", colors=custom_colors)
        theme_manager.register_theme(custom_theme)
        assert "custom" in theme_manager.get_available_themes()
        loaded = theme_manager.get_theme("custom")
        assert loaded.colors.background == "#2a2a2a"
    def test_load_custom_theme_from_file(self, theme_manager, temp_dir):
        import json
        theme_path = os.path.join(temp_dir, "custom_theme.json")
        theme_data = {
            "name": "my_theme",
            "background": "#333333",
            "foreground": "#eeeeee",
            "accent": "#00aaff",
            "font_size": 14,
        }
        with open(theme_path, "w") as f:
            json.dump(theme_data, f)
        theme = theme_manager.load_custom_theme(theme_path)
        assert theme is not None
        assert theme.name == "my_theme"
        assert theme.colors.background == "#333333"
        assert theme.font_size == 14
    def test_save_custom_theme_to_file(self, theme_manager, temp_dir):
        custom_theme = Theme(
            name="saved_theme",
            colors=ThemeColors(background="#444444", foreground="#ffffff"),
            font_size=16,
        )
        theme_path = os.path.join(temp_dir, "saved_theme.json")
        result = theme_manager.save_custom_theme(custom_theme, theme_path)
        assert result
        import json
        with open(theme_path) as f:
            data = json.load(f)
        assert data["name"] == "saved_theme"
        assert data["background"] == "#444444"
        assert data["font_size"] == 16
    def test_load_invalid_theme_file(self, theme_manager, temp_dir):
        invalid_path = os.path.join(temp_dir, "invalid.json")
        with open(invalid_path, "w") as f:
            f.write("not valid json")
        theme = theme_manager.load_custom_theme(invalid_path)
        assert theme is None
    def test_load_nonexistent_theme_file(self, theme_manager):
        theme = theme_manager.load_custom_theme("/nonexistent/path.json")
        assert theme is None
    def test_custom_theme_settings(self, theme_manager):
        custom_theme = Theme(
            name="custom_with_settings",
            colors=ThemeColors(),
            custom_settings={"opacity": 0.8, "blur": True},
        )
        theme_manager.register_theme(custom_theme)
        loaded = theme_manager.get_theme("custom_with_settings")
        assert loaded.custom_settings["opacity"] == 0.8
```

```
        assert loaded.custom_settings["blur"]
class TestThemeManager:
    def test_manager_initialization(self, theme_manager):
        assert theme_manager._current_theme == "dark"
        assert len(theme_manager._themes) >= 2
    def test_switch_between_themes(self, theme_manager):
        theme_manager.set_theme("dark")
        assert theme_manager.get_theme().name == "dark"
        theme_manager.set_theme("light")
        assert theme_manager.get_theme().name == "light"
        theme_manager.set_theme("dark")
        assert theme_manager.get_theme().name == "dark"
    def test_multiple_custom_themes(self, theme_manager):
        for i in range(3):
            theme = Theme(
                name=f"custom_{i}",
                colors=ThemeColors(background=f"#{i * 30:02x}{i * 30:02x}{i * 30:02x}"),
            )
            theme_manager.register_theme(theme)
        themes = theme_manager.get_available_themes()
        assert "custom_0" in themes
        assert "custom_1" in themes
        assert "custom_2" in themes
class TestThemeAppState:
    def test_app_state_theme_default(self, app_state):
        assert app_state.theme == "dark"
    def test_app_state_theme_change(self, app_state):
        app_state.theme = "light"
        assert app_state.theme == "light"
    def test_app_state_theme_custom(self, app_state):
        app_state.theme = "custom"
        assert app_state.theme == "custom"
    def test_app_state_invalid_theme(self, app_state):
        app_state.theme = "invalid_theme"
        assert app_state.theme == "invalid_theme"
class TestThemeConfig:
    def test_config_default_theme(self):
        config = AppConfig()
        assert config.theme == "dark"
    def test_config_theme_setting(self, temp_dir):
        config_path = os.path.join(temp_dir, "config.json")
        config = AppConfig(theme="light")
        os.makedirs(temp_dir, exist_ok=True)
        with open(config_path, "w") as f:
            import json
            json.dump({"theme": "light"}, f)
        loaded = load_config()
        assert loaded.theme == "dark"
    def test_config_save_theme(self, temp_dir):
        import json
        CONFIG_DIR = temp_dir
        CONFIG_PATH = os.path.join(CONFIG_DIR, "config.json")
        os.makedirs(CONFIG_DIR, exist_ok=True)
        config = AppConfig(theme="light")
        data = {"theme": config.theme}
        with open(CONFIG_PATH, "w") as f:
            json.dump(data, f)
        with open(CONFIG_PATH) as f:
            loaded_data = json.load(f)
        assert loaded_data["theme"] == "light"
class TestThemeFontSettings:
    def test_default_font(self, theme_manager):
        theme = theme_manager.get_theme()
        assert theme.font_family == "monospace"
        assert theme.font_size == 12
```

```
    def test_custom_font_settings(self, theme_manager):
        custom_theme = Theme(
            name="font_theme",
            colors=ThemeColors(),
            font_family="Fira Code",
            font_size=14,
            line_height=1.5,
        )
        theme_manager.register_theme(custom_theme)
        loaded = theme_manager.get_theme("font_theme")
        assert loaded.font_family == "Fira Code"
        assert loaded.font_size == 14
        assert loaded.line_height == 1.5
class TestThemeColorValidation:
    def test_valid_hex_colors(self):
        colors = ThemeColors(
            background="#000000",
            foreground="#ffffff",
            accent="#00ff00",
        )
        assert colors.background == "#000000"
    def test_color_format(self):
        colors = ThemeColors()
        for attr in ["background", "foreground", "accent", "error", "warning", "success"]:
            value = getattr(colors, attr)
            assert value.startswith("#")
            assert len(value) == 7
class TestThemeIntegration:
    def test_full_theme_workflow(self, theme_manager, temp_dir):
        custom_colors = ThemeColors(
            background="#2b2b2b",
            foreground="#a0a0a0",
            accent="#61afef",
        )
        custom_theme = Theme(
            name="workflow_theme",
            colors=custom_colors,
            font_size=13,
        )
        theme_manager.register_theme(custom_theme)
        assert theme_manager.is_valid_theme("workflow_theme")
        theme_path = os.path.join(temp_dir, "workflow_theme.json")
        saved = theme_manager.save_custom_theme(custom_theme, theme_path)
        assert saved
        loaded = theme_manager.load_custom_theme(theme_path)
        assert loaded is not None
        assert loaded.colors.background == "#2b2b2b"
        result = theme_manager.set_theme("workflow_theme")
        assert result
        current = theme_manager.get_theme()
        assert current.name == "workflow_theme"
```

File: [251/252] tests/test_tool_adapter.py

```
"""Tests for tool adapter."""
import pytest
from openlaoke.core.tool_adapter import (
    ParsedToolCall,
    ToolCallAdapter,
    create_adapter,
)
class TestToolCallAdapter:
    def test_adapter_creation(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert adapter.model == "gemma3:1b"
    def test_supports_tools_with_tool_model(self):
        adapter = ToolCallAdapter("gemma4:e2b")
        assert adapter.supports_tools() is True
    def test_supports_tools_with_no_tool_model(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert adapter.supports_tools() is False
    def test_is_creation_request(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert adapter.is_creation_request("write a new file")
        assert adapter.is_creation_request("create a program")
        assert adapter.is_creation_request("?????")
    def test_is_not_creation_request(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert not adapter.is_creation_request("read the file")
        assert not adapter.is_creation_request("list files")
        assert not adapter.is_creation_request("what is this")
    def test_should_skip_context_gathering(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert adapter.should_skip_context_gathering("write a new program")
        assert adapter.should_skip_context_gathering("create a single file")
    def test_should_not_skip_context_gathering(self):
        adapter = ToolCallAdapter("gemma3:1b")
        assert not adapter.should_skip_context_gathering("modify existing file")
        assert not adapter.should_skip_context_gathering("read the file")
        assert not adapter.should_skip_context_gathering("update the code")
    def test_parse_bash_tool_call(self):
        adapter = ToolCallAdapter("gemma3:1b")
        text = "I will run: `pip install requests`"
        calls = adapter.parse_tool_calls(text)
        assert len(calls) >= 1
        assert calls[0].name == "bash"
        assert "pip install requests" in calls[0].arguments.get("command", "")
    def test_parse_read_tool_call(self):
        adapter = ToolCallAdapter("gemma3:1b")
        text = "Let me read the file config.py"
        calls = adapter.parse_tool_calls(text)
        assert len(calls) >= 1
        assert calls[0].name == "read"
        assert "config.py" in calls[0].arguments.get("file_path", "")
    def test_format_tools_as_text(self):
        adapter = ToolCallAdapter("gemma3:1b")
        tools = [
            {
                "name": "read",
                "description": "Read a file",
                "input_schema": {
                    "type": "object",
                    "properties": {"file_path": {"type": "string", "description": "File path"}}
    ,
                    "required": ["file_path"],
```

```
                },
            }
        ]
        formatted = adapter.format_tools_as_text(tools)
        assert "read" in formatted.lower()
        assert "file" in formatted.lower()
    def test_format_tools_skipped_for_tool_models(self):
        adapter = ToolCallAdapter("gemma4:e2b")
        tools = [{"name": "test", "description": "Test", "input_schema": {}}]
        formatted = adapter.format_tools_as_text(tools)
        assert formatted == ""
class TestCreateAdapter:
    def test_create_adapter(self):
        adapter = create_adapter("gemma3:1b")
        assert isinstance(adapter, ToolCallAdapter)
        assert adapter.model == "gemma3:1b"
class TestCreationKeywords:
    def test_english_creation_keywords(self):
        adapter = ToolCallAdapter("test")
        for kw in ["write", "create", "make", "generate", "build", "implement"]:
            assert adapter.is_creation_request(f"{kw} something new")
    def test_chinese_creation_keywords(self):
        adapter = ToolCallAdapter("test")
        for kw in ["?", "??", "??", "??", "??"]:
            assert adapter.is_creation_request(f"{kw}????")
class TestNewFileIndicators:
    def test_new_file_indicators(self):
        adapter = ToolCallAdapter("test")
        assert adapter.is_creation_request("create a new file")
        assert adapter.is_creation_request("write a single file program")
        assert adapter.is_creation_request("build a standalone application")
class TestParsedToolCall:
    def test_parsed_tool_call_creation(self):
        call = ParsedToolCall(
            name="bash",
            arguments={"command": "ls"},
            confidence=0.9,
            raw_text="run: `ls`",
        )
        assert call.name == "bash"
        assert call.arguments["command"] == "ls"
        assert call.confidence == 0.9
```

File: [252/252] tests/test_tools.py

```
"""Tests for tool implementations."""
import os
import tempfile
import pytest
from openlaoke.core.state import create_app_state
from openlaoke.core.tool import ToolContext
from openlaoke.tools.bash_tool import BashTool
from openlaoke.tools.read_tool import ReadTool
from openlaoke.tools.write_tool import WriteTool
from openlaoke.tools.edit_tool import EditTool
from openlaoke.tools.glob_tool import GlobTool
from openlaoke.tools.grep_tool import GrepTool
@pytest.fixture
def app_state():
    import openlaoke
    project_root = os.path.dirname(os.path.dirname(openlaoke.__file__))
    return create_app_state(cwd=project_root)
@pytest.fixture
def ctx(app_state):
    return ToolContext(app_state=app_state, tool_use_id="test_1")
@pytest.fixture
def temp_dir():
    with tempfile.TemporaryDirectory() as d:
        yield d
class TestBashTool:
    @pytest.mark.asyncio
    async def test_simple_command(self, ctx):
        tool = BashTool()
        result = await tool.call(ctx, command="echo hello")
        assert not result.is_error
        assert "hello" in result.content
    @pytest.mark.asyncio
    async def test_empty_command(self, ctx):
        tool = BashTool()
        result = await tool.call(ctx, command="")
        assert result.is_error
    @pytest.mark.asyncio
    async def test_failing_command(self, ctx):
        tool = BashTool()
        result = await tool.call(ctx, command="exit 1")
        assert result.is_error
class TestReadTool:
    @pytest.mark.asyncio
    async def test_read_file(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.txt")
        with open(path, "w") as f:
            f.write("line1\nline2\nline3\n")
        tool = ReadTool()
        result = await tool.call(ctx, file_path=path)
        assert not result.is_error
        assert "line1" in result.content
        assert "line2" in result.content
    @pytest.mark.asyncio
    async def test_read_with_offset(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.txt")
        with open(path, "w") as f:
            f.write("a\nb\nc\nd\n")
        tool = ReadTool()
        result = await tool.call(ctx, file_path=path, offset=3)
        assert "c" in result.content
        assert "a" not in result.content
```

```
    @pytest.mark.asyncio
    async def test_read_with_limit(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "test.txt")
        with open(path, "w") as f:
            f.write("a\nb\nc\nd\n")
        tool = ReadTool()
        result = await tool.call(ctx, file_path=path, limit=2)
        assert "a" in result.content
        assert "b" in result.content
    @pytest.mark.asyncio
    async def test_read_nonexistent(self, ctx):
        tool = ReadTool()
        result = await tool.call(ctx, file_path="/nonexistent/file.txt")
        assert result.is_error
    @pytest.mark.asyncio
    async def test_read_directory(self, ctx, temp_dir):
        os.makedirs(os.path.join(temp_dir, "subdir"))
        with open(os.path.join(temp_dir, "file.txt"), "w") as f:
            f.write("test")
        tool = ReadTool()
        result = await tool.call(ctx, file_path=temp_dir)
        assert not result.is_error
        assert "file.txt" in result.content
class TestWriteTool:
    @pytest.mark.asyncio
    async def test_write_new_file(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "new.txt")
        tool = WriteTool()
        result = await tool.call(ctx, file_path=path, content="hello world")
        assert not result.is_error
        assert "Created" in result.content
        with open(path) as f:
            assert f.read() == "hello world"
    @pytest.mark.asyncio
    async def test_write_existing_file(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "existing.txt")
        with open(path, "w") as f:
            f.write("old content")
        tool = WriteTool()
        result = await tool.call(ctx, file_path=path, content="new content")
        assert not result.is_error
        assert "Updated" in result.content
        with open(path) as f:
            assert f.read() == "new content"
    @pytest.mark.asyncio
    async def test_write_creates_directories(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "a", "b", "c.txt")
        tool = WriteTool()
        result = await tool.call(ctx, file_path=path, content="nested")
        assert not result.is_error
        assert os.path.exists(path)
class TestEditTool:
    @pytest.mark.asyncio
    async def test_simple_edit(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "edit.txt")
        with open(path, "w") as f:
            f.write("hello world\nsecond line\n")
        tool = EditTool()
        result = await tool.call(
            ctx,
            file_path=path,
            old_string="hello world",
            new_string="goodbye world",
        )
        assert not result.is_error
```

```
        assert "Replaced 1 occurrence" in result.content
        with open(path) as f:
            content = f.read()
            assert "goodbye world" in content
            assert "hello world" not in content
    @pytest.mark.asyncio
    async def test_edit_not_found(self, ctx, temp_dir):
        path = os.path.join(temp_dir, "edit.txt")
        with open(path, "w") as f:
            f.write("some content\n")
        tool = EditTool()
        result = await tool.call(
            ctx,
            file_path=path,
            old_string="nonexistent text",
            new_string="replacement",
        )
        assert result.is_error
class TestGlobTool:
    @pytest.mark.asyncio
    async def test_glob_python_files(self, ctx, temp_dir):
        for name in ["a.py", "b.py", "c.txt"]:
            with open(os.path.join(temp_dir, name), "w") as f:
                f.write("")
        tool = GlobTool()
        result = await tool.call(ctx, pattern="*.py", path=temp_dir)
        assert not result.is_error
        assert "a.py" in result.content
        assert "b.py" in result.content
        assert "c.txt" not in result.content
class TestGrepTool:
    @pytest.mark.asyncio
    async def test_grep_content(self, ctx, temp_dir):
        with open(os.path.join(temp_dir, "test.py"), "w") as f:
            f.write("def hello():\n    print('world')\n")
        tool = GrepTool()
        result = await tool.call(ctx, pattern="def hello", path=temp_dir)
        assert not result.is_error
        assert "def hello" in result.content
    @pytest.mark.asyncio
    async def test_grep_no_matches(self, ctx, temp_dir):
        with open(os.path.join(temp_dir, "test.py"), "w") as f:
            f.write("no matches here\n")
        tool = GrepTool()
        result = await tool.call(ctx, pattern="nonexistent_pattern", path=temp_dir)
        assert not result.is_error
        assert "No matches" in result.content
    @pytest.mark.asyncio
    async def test_grep_files_with_matches(self, ctx, temp_dir):
        with open(os.path.join(temp_dir, "a.py"), "w") as f:
            f.write("TODO: fix this\n")
        with open(os.path.join(temp_dir, "b.py"), "w") as f:
            f.write("no todos\n")
        tool = GrepTool()
        result = await tool.call(
            ctx, pattern="TODO", path=temp_dir, output_mode="files_with_matches"
        )
        assert "a.py" in result.content
```